

Сборник заданий для семинарских занятий
по курсу
«Объектно-ориентированное программирование на Python»

Содержание

1	Общие сведения	4
2	Задания	4
2.1	Семинар «Правила формирования класса для программирования в IDE PyCharm. Оработка навыков создания простых классов и объектов класса» (2 очных часа)	5
2.2	Семинар «Конструкторы, наследование и полиморфизм. 1 часть» (2 часа) . .	20
2.2.1	Задача 1	20
2.2.2	Задача 2	45
2.2.3	Задача 3	65
2.2.4	Задача 4	92
2.3	Семинар «Структуры данных в ООП-реализации» (2 часа)	119
2.3.1	Задача 1 (дерево)	119
2.3.2	Задача 2 (стек)	160
2.3.3	Задача 3 (двусвязный список)	192
2.3.4	Задача 4 (очередь)	225
2.4	Семинар «Структуры данных (закрепление) и __new__» (2 часа)	253
2.4.1	Задача 1 (Singleton)	253
2.4.2	Задача 2 (ограничение количества экземпляров)	277
2.4.3	Задача 3 (именование)	303
2.4.4	Задача 4	330
2.4.5	Задача 5	365
2.5	Семинар «Композиция» (2 часа)	390
2.5.1	Задача 1	390
2.5.2	Задача 2	414
2.5.3	Задача 3	431
2.6	Семинар «Ограничения доступа и Unit-тестирование» (2 часа)	442
2.6.1	Принципы unit-тестирования в Python	442
2.6.2	Как анализировать код для тестирования всех случаев	442
2.6.3	Пример простого класса с unit-тестами	443
2.6.4	Задача 1	444
2.6.5	Задача 2	509
2.7	Семинар «Наследование #2» (2 часа)	512
2.7.1	Задача 1.	512
2.7.2	Задача 2.	525
2.7.3	Задача 3.	550
2.8	Семинар «Полиморфизм» (2 часа)	569
2.8.1	Полиморфизм с наследованием (частичная модификация метода родительского класса)	569
2.8.2	Полиморфизм с наследованием (полное переопределение методов родительского класса)	582
2.9	Семинар «Множественное наследование» (2 часа)	606
2.9.1	Задача 1	606
2.9.2	Задача 2	639
2.9.3	Задача 3	639
2.9.4	Задача 4	659
2.9.5	Задача 5	659

2.10	Семинар «Классы-миксины и множественное наследование» (2 часа)	660
2.11	Семинар «Классы-миксины и множественное наследование (продолжение)» (2 часа)	711
2.12	Семинар «Перегрузка операций в классах» (2 часа)	742

1 Общие сведения

Сборник содержит задания для семинарских занятий по курсу «Объектно-ориентированное программирование на Python» (32 часа).

Прототипы заданий разработаны Игнатовым Юрием Юрьевичем (кандидатом социальных наук), а далее были размножены Глускером Александром Игоревичем с целью охвата аудитории в 35 человек. В ходе размножения были проведены небольшие корректировки заданий, а также добавлено задание для проведения Unit-тестирования.

Возможна сдача другого кода (например, выполненного в ходе проектной деятельности), если они полностью покрывают материал семинара.

Оценивание работ осуществляется на основе следующих принципов:

- строгий учёт сроков сдачи (при просрочке учитываются официальные документы от врача и отправление на мероприятия в интересах ВУЗа, а также пропуск по причине назначения пересдач строго на время пары), просрочка понижает оценку на балл (но не ниже 3),
- каждая работа защищается (задаются вопросы и проводится собеседование по коду и общим принципам), если защита не успешна несколько раз, то оценка может быть понижена,
- если в работе обнаружены недостатки, то студенту предоставляется возможность их исправить без понижения оценки,
- оценка за практическую часть это средняя оценка всех работ с учетом того, что недосданные работы оцениваются пропорционально ниже оценки, которая была бы получена при условии сдачи,
- на пересдаче осуществляется сдача работ, оценка на пересдаче за конкретную работу не может быть выше 3 за исключением случая, когда имеются оправдательные документы согласно первому принципу.

2 Задания

2.1 Семинар «Правила формирования класса для программирования в IDE PyCharm. Отработка навыков создания простых классов и объектов класса» (2 очных часа)

В ходе работы создайте 5 классов с соответствующими методами, описанными в индивидуальном задании. Предполагается, что пользователь класса не имеет права обращаться к свойствам напрямую (соблюдая принцип инкапсуляции), а должен использовать методы. Важно: в задании не всегда указаны все необходимые методы и свойства, при необходимости вам надо самостоятельно их добавить. Продемонстрируйте работоспособность всех методов (из задания) посредством создания запускаемых файлов, где осуществляется вызов методов для разных ситуаций (без ручного ввода, но с выводом результатов в консоль). Каждый класс должен сохраняться в отдельном исходном файле. Необходимо соблюдать все стандартные требования к качеству кода (отступы, именования переменных, классов, методов, проверка корректности входных данных). Для каждого класса создайте отдельный запускаемый файл для проверки всех его методов (допускается использование других классов в этих тестах).

Все предлагаемые классы в заданиях упрощенные; для использования в production-окружении они требуют серьезной доработки. Суть задания — в отработке базовых навыков, а не в идеальном моделировании предложенных ситуаций.

Для сдачи работы будьте готовы пояснить или аналогично заданию модифицировать любую часть кода, а также ответить на вопросы:

1. Кратко опишите парадигму объектно-ориентированного программирования (ООП).
2. Что такое класс в парадигме ООП?
3. Что такое объект (экземпляр) в парадигме ООП?
4. Что обозначает свойство инкапсуляции в парадигме ООП?
5. Синтаксис классов в Python (в рамках выполненной работы), создание и работа с объектами в Python.

При выполнении задания предполагается самое простое базовое описание классов, соответствующее следующему примеру (вы можете использовать то, что вы ЗНАЕТЕ дополнительно, но это остается на ваше усмотрение):

Если вы нашли в задачнике ошибки, опечатки и другие недостатки, то вы можете сделать pull-request.

```
class Worker:
    def set_last_name(self, last_name):
        self.last_name = last_name

    def print_last_name(self):
        print (f"Фамилия: {self.last_name}")

    def get_last_name(self):
        return last_name

worker = Worker()
worker.set_last_name(self, "Иванов")
worker.print_last_name()
print(worker.get_last_name())
```

Срок сдачи работы (начала сдачи): следующее занятие после его выдачи. В последующие сроки оценка будет снижаться (при отсутствии оправдывающих документов).

1. **Описание ситуации:** Рассмотрим работу грузовой железнодорожной станции. На станции есть несколько путей, по которым поезда могут прибывать и отправляться. Каждый путь имеет свой номер и может вместить несколько поездов. Поезда формируются из вагонов, каждый из которых может перевозить разные грузы. Работники станции отвечают за диспетчерское управление маневровыми локомотивами, осмотр вагонов, выполнение погрузочно-разгрузочных работ, прием груза к перевозке, ремонт путей, обеспечение безопасности и т.п. Они используют радиостанции для связи друг с другом и для отслеживания положения поездов и передвижения вагонов.

Создаваемые классы: 'Путь', 'Поезд', 'Вагон', 'Станция', 'РаботникСтанции'.

Для классов реализовать следующие простые методы (ниже приведен не исчерпывающий список методов; для демонстрации работы классов вам потребуются дополнительные методы, позволяющие отследить состояние объектов), используя для хранения данных списки (['']) Python:

- (a) **Путь:** добавить поезд на путь, убрать поезд с пути, получить список поездов на конкретном пути.
 - (b) **Поезд:** прицепить вагон к поезду, отцепить вагон от поезда, получить (распечатать) список вагонов в поезде, вывести информацию о грузе в поезде.
 - (c) **Вагон:** добавить номер поезда, в который включался конкретный вагон, удалить номер поезда из истории, отобразить историю поездов для конкретного вагона.
 - (d) **РаботникСтанции:** класс, представляющий отдельного работника на станции, имеющий идентификатор, информацию о персональной радиостанции, список закрепленных за ним поездов для осмотра, ФИО, должность.
 - (e) **Станция:** добавить станционный путь, добавить поезд на станцию, нанять работника станции, вывести информацию о всех путях, поездах, работниках, удалить путь, удалить поезд, уволить работника.
2. **Описание ситуации:** Рассмотрим работу крупного логистического терминала для обработки грузовых автомобилей. На терминале есть несколько доков (рамп), куда фуры прибывают для проведения погрузочно-разгрузочных работ. Каждый док имеет свой номер и может одновременно обслуживать одну машину. Грузовики перевозят паллеты, каждая из которых содержит определенный товар. Сотрудники терминала отвечают за прием грузовиков, управление погрузочной техникой, проверку сопроводительных документов, приемку и отгрузку товара, а также техническое обслуживание доков. Они используют портативные радиостанции для координации действий и отслеживания статуса обработки автомобилей.

Создаваемые классы: 'Док', 'Грузовик', 'Паллета', 'Терминал', 'Сотрудник'.

Для классов реализовать следующие простые методы, используя для хранения данных списки (['']) Python:

- (a) **Док:** занять док конкретным грузовиком, освободить док, получить информацию о грузовике, который сейчас находится на доке.
- (b) **Грузовик:** добавить паллету в грузовик, выгрузить паллету из грузовика, получить (распечатать) список паллет в грузовике, вывести информацию о товарах в грузовике.
- (c) **Паллета:** добавить номер грузовика, в который загружалась конкретная паллета, удалить номер грузовика из истории, отобразить историю перевозок (номера грузовиков) для конкретной паллеты.

- (d) **Сотрудник:** класс, представляющий отдельного сотрудника терминала, имеющий идентификатор, номер радиации, список доков, за которые он отвечает, ФИО, должность.
- (e) **Терминал:** добавить новый док на терминале, зарегистрировать прибытие грузовика, нанять нового сотрудника, вывести список всех доков, грузовиков на территории, сотрудников, удалить док, удалить грузовик, уволить сотрудника.

3. **Описание ситуации:** Рассмотрим работу аэропорта. В аэропорту есть несколько взлетно-посадочных полос (ВПП), которые принимают и отправляют рейсы. Каждая ВПП имеет свой номер, длину и статус доступности. Самолеты перевозят пассажиров и их ручную кладь, размещенную в салоне. Авиадиспетчеры управляют движением самолетов, назначают полосы для взлета и посадки, следят за воздушной обстановкой и координируют действия с помощью радиосвязи.

Создаваемые классы: 'ВПП', 'Самолет', 'Пассажир', 'Аэропорт', 'Авиадиспетчер'.

Для классов реализовать следующие простые методы, используя для хранения данных списки ('[]') Python:

- (a) **ВПП:** занять полосу для взлета/посадки, освободить полосу, получить список рейсов, использовавших полосу.
- (b) **Самолет:** добавить пассажира на борт (включая вес его ручной клади), высадить пассажира, получить (распечатать) список пассажиров на борту, рассчитать общий вес ручной клади.
- (c) **Пассажир:** добавить рейс в историю перелетов пассажира, удалить рейс из истории (ошибка бронирования), отобразить всю историю перелетов.
- (d) **Авиадиспетчер:** класс, представляющий диспетчера, имеющий идентификатор, рабочую частоту, график работы (список интервалов времени в сутках), ФИО.
- (e) **Аэропорт:** добавить новую ВПП, зарегистрировать прибытие самолета, нанять диспетчера, вывести список всех ВПП, самолетов в аэропорту, диспетчеров, удалить ВПП (на ремонт), списать самолет, уволить диспетчера.

4. **Описание ситуации:** Рассмотрим работу речного порта. В порту есть несколько причалов для швартовки грузовых барж и буксиров. Каждый причал имеет уникальный номер и максимальную глубину, определяющую осадку судов, которые могут к нему подойти. Баржи перевозят контейнеры с различными грузами. Их характеризуют вес судна, максимальная грузоподъемность и осадка (как без груза, так и с максимальным грузом). Портовые рабочие отвечают за швартовку судов, управление портовыми кранами для погрузки/разгрузки контейнеров, оформление документов и поддержание порядка на территории.

Создаваемые классы: 'Причал', 'Баржа', 'Контейнер', 'Порт', 'ПортовыйРабочий'.

Для классов реализовать следующие простые методы, используя для хранения данных списки ('[]') Python:

- (a) **Причал:** пришвартовать баржу к причалу, отшвартовать баржу, получить список барж, находящихся у причала.
- (b) **Баржа:** загрузить контейнер на баржу (с указанием веса контейнера), разгрузить контейнер с баржи, получить (распечатать) список контейнеров на барже, рассчитать текущую осадку судна (предполагается линейная зависимость осадки от суммарного веса груза и баржи).

- (с) **Контейнер:** добавить номер баржи, на которую погрузили контейнер, удалить номер баржи, отобразить историю перемещений контейнера между баржами.
- (d) **ПортовыйРабочий:** класс, представляющий рабочего, имеющий идентификатор, допуск к работе с краном, список закрепленных причалов, ФИО, должность.
- (е) **Порт:** ввести новый причал в эксплуатацию, принять баржу в акваторию порта, принять на работу рабочего, вывести список причалов, барж в акватории, рабочих, списать причал, отправить баржу, уволить рабочего.

5. **Описание ситуации:** Рассмотрим работу автобусного парка. В парке есть несколько маршрутов, которые обслуживаются автобусами. Каждый маршрут имеет номер и список остановок. Автобусы имеют государственный номер, количество мест и текущий пробег. Водители закреплены за автобусами и маршрутами. Диспетчеры автопарка составляют расписание, следят за выходами автобусов на линию, учетом пробега и техническим состоянием.

Создаваемые классы: 'Маршрут', 'Автобус', 'Остановка', 'Автопарк', 'Водитель'.

Для классов реализовать следующие простые методы, используя для хранения данных списки (['']) Python:

- (a) **Маршрут:** добавить остановку в маршрут, удалить остановку из маршрута, получить список всех остановок на маршруте.
- (b) **Автобус:** назначить автобус на маршрут, снять с маршрута, увеличить пробег на заданное значение, получить текущий пробег.
- (с) **Остановка:** добавить маршрут, проходящий через остановку, удалить маршрут, отобразить список всех маршрутов, проходящих через данную остановку.
- (d) **Водитель:** класс, представляющий водителя, имеющий идентификатор, права категории, закрепленный автобус, ФИО, график работы.
- (е) **Автопарк:** добавить новый маршрут, приобрести новый автобус, принять на работу водителя, вывести список маршрутов, автобусов (с указанием их состояния), водителей, списать автобус, уволить водителя.

6. **Описание ситуации:** Рассмотрим работу метрополитена. В метро есть линии, состоящие из станций и тоннелей между ними. Составы из вагонов перемещаются по линиям. Каждая станция имеет название и может быть точкой пересадки на другие линии. Машинисты управляют поездами. Дежурные по станции следят за порядком на платформах и работой оборудования. Управление метрополитеном координирует движение составов.

Создаваемые классы: 'ЛинияМетро', 'ПоездМетро', 'Станция', 'УправлениеМетрополитеном', 'Машинист'.

Для классов реализовать следующие простые методы, используя для хранения данных списки (['']) Python:

- (a) **ЛинияМетро:** добавить станцию на линию, получить список станций на линии, получить список поездов на линии.
- (b) **ПоездМетро:** добавить вагон в состав, отцепить вагон, назначить машиниста на поезд.
- (с) **Станция:** добавить линию, проходящую через станцию (для моделирования пересадочных узлов), получить список линий на станции.

- (d) **Машинист:** класс, представляющий машиниста, имеющий идентификатор, допуск к управлению, закрепленный поезд, ФИО, стаж.
- (e) **Управление Метрополитеном:** открыть новую линию, ввести новый поезд в эксплуатацию, принять на работу машиниста, вывести список линий, поездов (в депо и на линиях), машинистов, закрыть линию на техобслуживание, списать поезд, вывести полную схему метро (в текстовом виде).

7. **Описание ситуации:** Рассмотрим работу службы доставки пиццы. В службе есть несколько филиалов. Каждый филиал обслуживает определенный район и имеет курьеров. Заказы формируются из позиций меню. Курьеры используют скутеры для доставки. Менеджеры филиалов принимают заказы, назначают курьеров и следят за выполнением заказов.

Создаваемые классы: 'Филиал', 'Заказ', 'Курьер', 'Скутер', 'Менеджер'.

Для классов реализовать следующие простые методы, используя для хранения данных списки ([]) Python:

- (a) **Филиал:** добавить курьера в филиал, уволить курьера, получить список активных заказов филиала.
- (b) **Заказ:** добавить позицию в заказ (название + цена), удалить позицию, рассчитать стоимость заказа, изменить статус заказа (принят, готовится, в пути, доставлен).
- (c) **Курьер:** назначить заказ курьеру, завершить доставку заказа, получить список доставленных заказов за смену, закрепить скутер за курьером.
- (d) **Менеджер:** класс, представляющий менеджера, имеющий идентификатор, закрепленный филиал, ФИО, смену.
- (e) **Скутер:** отправить на зарядку, вернуть в строй, увеличить пробег, получить текущий пробег.

Описание ситуации: Рассмотрим работу трамвайного депо. В депо есть несколько маршрутов, обслуживаемых трамвайными вагонами. Каждый трамвайный вагон имеет бортовой номер, вместимость и текущий пробег. Маршруты состоят из остановок и имеют определенный график движения. Водители трамваев закреплены за конкретными вагонами и маршрутами. Диспетчеры управляют выпуском трамваев на линию и ведут учет технического состояния.

Создаваемые классы: Маршрут, Трамвай, Остановка, Депо, Водитель.

Для классов реализовать следующие простые методы, используя для хранения данных списки ([]) Python:

- (a) **Маршрут:** добавить остановку в маршрут, удалить остановку из маршрута, получить список всех остановок на маршруте.
- (b) **Трамвай:** назначить трамвай на маршрут, снять с маршрута, увеличить пробег на заданное значение, получить текущий пробег.
- (c) **Остановка:** добавить маршрут, проходящий через остановку, удалить маршрут, отобразить список всех маршрутов, проходящих через данную остановку.
- (d) **Водитель:** класс, представляющий водителя, имеющий идентификатор, права категории, закрепленный трамвай, ФИО, график работы.

- (е) **Депо:** добавить новый маршрут, принять новый трамвай в депо, принять на работу водителя, выполнить вывод списка маршрутов, трамваев (с указанием их состояния), водителей, списать трамвай, уволить водителя.
8. **Описание ситуации:** Рассмотрим работу морского порта для приёма пассажирских паромов. В порту есть несколько причалов, каждый из которых обслуживает один паром за раз. Паромы перевозят пассажиров и автомобили. Пассажиры покупают билеты, автомобили записываются в список грузовой палубы. Сотрудники порта координируют погрузку, проверку билетов и безопасность. **Создаваемые классы:** Причал, Паром, Пассажир, Автомобиль, СотрудникПорта.
- (а) **Причал:** пришвартовать паром, освободить причал, получить информацию о пароме у причала.
- (б) **Паром:** добавить пассажира, добавить автомобиль, посадить пассажира, выгрузить автомобиль.
- (с) **Пассажир:** добавить рейс в историю поездок, удалить рейс из истории, вывести историю поездок.
- (d) **Автомобиль:** зарегистрировать номер парома, удалить номер парома, вывести историю перевозок.
- (е) **СотрудникПорта:** идентификатор, должность, ФИО, список закреплённых причалов.
9. **Описание ситуации:** Рассмотрим работу пригородной электрички. В системе есть станции, между которыми курсируют электрички. У каждой электрички есть номер, список вагонов и машинист. Пассажиры покупают билеты и занимают места в вагонах. Диспетчеры контролируют движение электричек. **Создаваемые классы:** Станция, Электричка, Вагон, Пассажир, Диспетчер.
- (а) **Станция:** принять электричку, отправить электричку, вывести список электричек на станции.
- (б) **Электричка:** добавить вагон, отцепить вагон, получить список вагонов.
- (с) **Вагон:** посадить пассажира, посадить пассажира, вывести список пассажиров.
- (d) **Пассажир:** добавить поездку в историю, удалить поездку, показать историю поездок.
- (е) **Диспетчер:** идентификатор, ФИО, рабочая смена, список контролируемых электричек.
10. **Описание ситуации:** Рассмотрим работу таксопарка. В таксопарке есть автомобили, водители и диспетчеры. Автомобиль закрепляется за водителем. Диспетчеры принимают заказы и назначают их водителям. Пассажиры совершают поездки. **Создаваемые классы:** Таксопарк, Автомобиль, Водитель, Заказ, Диспетчер.
- (а) **Таксопарк:** добавить автомобиль, принять водителя, вывести список машин и водителей, уволить водителя.
- (б) **Автомобиль:** назначить водителя, снять водителя, увеличить пробег, получить пробег.
- (с) **Водитель:** назначить заказ, завершить заказ, вывести список выполненных заказов.

- (d) **Заказ:** назначить пассажира, завершить поездку, вывести информацию о заказе.
 - (e) **Диспетчер:** идентификатор, ФИО, список назначенных заказов.
11. **Описание ситуации:** Рассмотрим работу грузового аэропорта. Самолёты перевозят контейнеры. В аэропорту есть ангары для хранения самолётов и площадки для погрузки. Работники аэропорта координируют загрузку и выгрузку контейнеров. **Создаваемые классы:** Самолёт, Контейнер, Ангар, РаботникАэропорта, Аэропорт.
- (a) **Самолёт:** загрузить контейнер, выгрузить контейнер, вывести список контейнеров.
 - (b) **Контейнер:** добавить номер самолёта, удалить номер самолёта, вывести историю перевозок.
 - (c) **Ангар:** принять самолёт, вывести список самолётов, освободить ангар.
 - (d) **РаботникАэропорта:** идентификатор, ФИО, должность, список самолётов в обслуживании.
 - (e) **Аэропорт:** принять самолёт, убрать самолёт, принять раотника, уволить работника, вывести список самолётов и работников.
12. **Описание ситуации:** Рассмотрим работу велопроката. В прокате есть велосипеды, станции для их хранения, клиенты и сотрудники. Клиенты арендуют велосипеды и возвращают их на станцию. **Создаваемые классы:** Велосипед, СтанцияПроката, Клиент, Сотрудник, Прокат.
- (a) **Велосипед:** выдать в аренду, вернуть на станцию, получить пробог.
 - (b) **СтанцияПроката:** добавить велосипед, убрать велосипед, вывести список велосипедов.
 - (c) **Клиент:** арендовать велосипед, вернуть велосипед, вывести историю аренд.
 - (d) **Сотрудник:** идентификатор, ФИО, должность, список закреплённых станций.
 - (e) **Прокат:** добавить станцию, демонтировать станцию, вывести список станций и велосипедов, уволить сотрудника, нанять сотрудника, вывести список сотрудников.
13. **Описание ситуации:** Рассмотрим работу речных теплоходов. У каждого теплохода есть рейсы и список пассажиров. Пассажиры покупают билеты. Работники пристани обслуживают теплоходы. **Создаваемые классы:** Теплоход, Рейс, Пассажир, Пристань, РаботникПристани.
- (a) **Теплоход:** добавить рейс, убрать рейс, вывести список рейсов.
 - (b) **Рейс:** добавить пассажира, удалить пассажира, вывести список пассажиров.
 - (c) **Пассажир:** добавить рейс в историю, удалить рейс, вывести историю.
 - (d) **Пристань:** принять теплоход, отправить теплоход, вывести список теплоходов.
 - (e) **РаботникПристани:** идентификатор, ФИО, должность, закреплённые рейсы.
14. **Описание ситуации:** Рассмотрим работу каршеринга. В системе есть автомобили, клиенты и диспетчеры. Автомобили бронируются клиентами и возвращаются после поездки. Диспетчеры контролируют состояние машин. **Создаваемые классы:** Автомобиль, Клиент, Диспетчер, Заказ, Каршеринг.

- (a) **Автомобиль:** выдать клиенту, вернуть, увеличить пробег, вывести пробег.
 - (b) **Клиент:** арендовать автомобиль, завершить аренду, вывести историю аренд.
 - (c) **Диспетчер:** идентификатор, ФИО, список автомобилей под контролем.
 - (d) **Заказ:** назначить автомобиль, завершить поездку, вывести данные заказа.
 - (e) **Каршеринг:** добавить автомобиль, списать автомобиль, добавить клиента, удалить клиента, добавить диспетчера, удалить диспетчера, вывести список клиентов, диспетчеров и машин.
15. **Описание ситуации:** Рассмотрим работу железнодорожного музея. В музее есть экспонаты (локомотивы и вагоны), экскурсии и экскурсоводы. Посетители записываются на экскурсии. **Создаваемые классы:** Экспонат, Экскурсия, Экскурсовод, Посетитель, Музей.
- (a) **Экспонат:** добавить к экскурсии, убрать, вывести список экскурсий.
 - (b) **Экскурсия:** записать посетителя, удалить, вывести список посетителей.
 - (c) **Экскурсовод:** идентификатор, ФИО, список экскурсий.
 - (d) **Посетитель:** записаться на экскурсию, отменить запись, вывести историю.
 - (e) **Музей:** добавить экспонат, списать экспонат, добавить экскурсовода, уволить экскурсовода, провести экскурсию, вывести список всех экскурсий и экскурсоводов.
16. **Описание ситуации:** Рассмотрим работу автозаправочной станции. На станции есть топливо, колонки и операторы. Автомобили приезжают заправляться. **Создаваемые классы:** Колонка, Автомобиль, Оператор, Топливо, АЗС.
- (a) **Колонка:** заправить автомобиль, освободить колонку, вывести статус.
 - (b) **Автомобиль:** получить заправку, вывести историю заправок.
 - (c) **Оператор:** идентификатор, ФИО, список закреплённых колонок.
 - (d) **Топливо:** уменьшить количество, увеличить количество, вывести остаток.
 - (e) **АЗС:** добавить колонку, нанять оператора, уволить оператора, демонтировать колонку, вывести список машин, операторов и колонок.
17. **Описание ситуации:** Рассмотрим работу сортировочного центра курьерской службы. В центре есть зоны обработки посылок, конвейерные линии и сотрудники. Каждая посылка имеет трек-номер и проходит через несколько этапов обработки. Сотрудники сканируют посылки, сортируют их по направлениям и загружают в транспортировочные контейнеры. Менеджеры контролируют процесс сортировки и работу оборудования.
- Создаваемые классы:** 'ЗонаОбработки', 'Посылка', 'Конвейер', 'СотрудникЦентра', 'СортировочныйЦентр'.
- Для классов реализовать следующие простые методы, используя для хранения данных списки (['']) Python:
- (a) **ЗонаОбработки:** добавить посылку в зону, удалить посылку из зоны, получить список посылок в зоне.
 - (b) **Посылка:** добавить статус обработки (принята, сортируется, отправлена), удалить ошибочный статус, отобразить историю статусов обработки.

- (с) **Конвейер:** запустить конвейерную ленту, остановить конвейер, добавить посылку на конвейер, снять посылку с конвейера.
- (d) **СотрудникЦентра:** класс, представляющий сотрудника, имеющий идентификатор, смену, список закрепленных зон обработки, ФИО, должность.
- (е) **СортировочныйЦентр:** добавить новую зону обработки, ввести в эксплуатацию конвейер, нанять сотрудника, вывести список всех зон, конвейеров, сотрудников, удалить зону, вывести из эксплуатации конвейер, уволить сотрудника.

18. **Описание ситуации:** Рассмотрим работу диспетчерской службы городского пассажирского транспорта. Диспетчеры отслеживают движение автобусов, троллейбусов и трамваев на маршрутах, регулируют интервалы движения, фиксируют отклонения от графика. Транспортные средства оснащены GPS-трекерами для передачи местоположения.

Создаваемые классы: 'Маршрут', 'ТранспортноеСредство', 'Диспетчер', 'Остановка', 'ДиспетчерскаяСлужба'.

Для классов реализовать следующие простые методы, используя для хранения данных списки (['']) Python:

- (a) **Маршрут:** добавить транспортное средство на маршрут, снять с маршрута, получить список транспорта на маршруте.
- (b) **ТранспортноеСредство:** обновить местоположение (координаты), получить текущее местоположение, добавить информацию о задержке/опережении графика.
- (с) **Диспетчер:** класс, представляющий диспетчера, имеющий идентификатор, смену, список контролируемых маршрутов, ФИО.
- (d) **Остановка:** добавить маршрут, проходящий через остановку, удалить маршрут, получить список маршрутов на остановке.
- (е) **ДиспетчерскаяСлужба:** добавить новый маршрут, зарегистрировать транспортное средство, нанять диспетчера, вывести информацию о всех маршрутах, транспорте, диспетчерах, удалить маршрут, списать транспорт, уволить диспетчера.

19. **Описание ситуации:** Рассмотрим работу центра технического обслуживания городского транспорта. В центре есть ремонтные зоны для разных видов транспорта, запасы запчастей и бригады механиков. Транспортные средства проходят плановое ТО и внеплановый ремонт.

Создаваемые классы: 'РемонтнаяЗона', 'ТранспортноеСредство', 'Запчасть', 'Механик', 'ЦентрТехОбслуживания'.

Для классов реализовать следующие простые методы, используя для хранения данных списки (['']) Python:

- (a) **РемонтнаяЗона:** поставить транспорт на ремонт, завершить ремонт, получить список транспорта в ремонте.
- (b) **ТранспортноеСредство:** добавить запись о ремонте (дата, вид работ), удалить ошибочную запись, отобразить историю ремонтов.
- (с) **Запчасть:** уменьшить количество на складе, увеличить количество, получить текущий остаток.

- (d) **Механик:** класс, представляющий механика, имеющий идентификатор, квалификацию, список закрепленных ремонтных зон, ФИО.
- (e) **ЦентрТехОбслуживания:** добавить ремонтную зону, закупить запчасти, нанять механика, вывести информацию о зонах, запчастях, механиках, удалить зону, уволить механика.

20. **Описание ситуации:** Рассмотрим работу логистического центра междугородных автобусных перевозок. Автобусы совершают рейсы между городами по определенным маршрутам, перевозят пассажиров и их багаж. Диспетчеры формируют расписание, продают билеты и контролируют отправку автобусов.

Создаваемые классы: 'Автобус', 'Маршрут', 'Пассажир', 'Диспетчер', 'ЛогистическийЦентр'.

Для классов реализовать следующие простые методы, используя для хранения данных списки (['']) Python:

- (a) **Автобус:** назначить на маршрут, снять с маршрута, добавить пассажира, высадить пассажира, получить список пассажиров.
- (b) **Маршрут:** добавить город в маршрут, удалить город, получить список всех городов на маршруте.
- (c) **Пассажир:** купить билет (добавить маршрут в историю), сдать билет (удалить маршрут), показать историю поездок.
- (d) **Диспетчер:** класс, представляющий диспетчера, имеющий идентификатор, список закрепленных маршрутов, ФИО, график работы.
- (e) **ЛогистическийЦентр:** добавить автобус в парк, добавить маршрут, нанять диспетчера, вывести список автобусов, маршрутов и диспетчеров, списать автобус, уволить диспетчера.

21. **Описание ситуации:** Рассмотрим работу центра управления интеллектуальной транспортной системой города. Система включает в себя управление светофорами, камеры видеонаблюдения, датчики транспортного потока. Операторы следят за дорожной ситуацией и оперативно реагируют на инциденты.

Создаваемые классы: 'Перекресток', 'Светофор', 'КамераНаблюдения', 'ОператорИТС', 'ЦентрУправления'.

Для классов реализовать следующие простые методы, используя для хранения данных списки (['']) Python:

- (a) **Перекресток:** добавить светофор к перекрестку, удалить светофор, получить список светофоров на перекрестке.
- (b) **Светофор:** изменить режим работы (красный/желтый/зеленый), получить текущий режим, добавить информацию о неисправности, вывести список неисправностей.
- (c) **КамераНаблюдения:** включить запись, выключить запись, получить статус работы, зафиксировать нарушение ПДД, вывести список нарушений.
- (d) **ОператорИТС:** класс, представляющий оператора, имеющий идентификатор, смену, список контролируемых перекрестков, ФИО.

- (е) **ЦентрУправления:** добавить новый перекресток в систему, установить светофор, установить камеру, нанять оператора, вывести информацию о перекрестках, светофорах, камерах, операторах, удалить перекресток, уволить оператора, снять камеру, снять светофор.

22. **Описание ситуации:** Рассмотрим работу службы эвакуации аварийных транспортных средств. Эвакуаторы дежурят на специальных парковках и выезжают по вызову на места ДТП или поломок. Диспетчеры принимают вызовы и направляют ближайший свободный эвакуатор.

Создаваемые классы: 'Эвакуатор', 'Вызов', 'ПарковкаЭвакуаторов', 'ДиспетчерЭвакуации', 'СлужбаЭвакуации'.

Для классов реализовать следующие простые методы, используя для хранения данных списки (['']) Python:

- (а) **Эвакуатор:** принять вызов, завершить вызов, получить текущий статус (свободен/занят), обновить местоположение.
- (б) **Вызов:** зафиксировать время принятия, время выполнения, получить статус выполнения.
- (с) **ПарковкаЭвакуаторов:** принять эвакуатор на парковку, выпустить эвакуатор с парковки, получить список эвакуаторов на парковке.
- (d) **ДиспетчерЭвакуации:** класс, представляющий диспетчера, имеющий идентификатор, смену, список обработанных вызовов, ФИО.
- (е) **СлужбаЭвакуации:** добавить эвакуатор в парк, списать эвакуатор, нанять диспетчера, вывести информацию о эвакуаторах, вызовах, диспетчерах, уволить диспетчера.

23. **Описание ситуации:** Рассмотрим работу центра контроля коммерческих грузоперевозок. Система отслеживает движение грузовых автомобилей, контролирует соблюдение маршрутов, норм труда водителей и расход топлива. Менеджеры по логистике планируют маршруты и анализируют отчеты.

Создаваемые классы: 'ГрузовойАвтомобиль', 'МаршрутПеревозки', 'Водитель', 'Рейс', 'МенеджерЛогистики'.

Для классов реализовать следующие простые методы, используя для хранения данных списки (['']) Python:

- (а) **ГрузовойАвтомобиль:** начать рейс, завершить рейс, получить текущий статус, зафиксировать расход топлива.
- (б) **МаршрутПеревозки:** добавить точку маршрута (город, склад), удалить точку, получить полный маршрут.
- (с) **Водитель:** класс, представляющий водителя, имеющий идентификатор, права, график работы, ФИО, стаж.
- (d) **Рейс:** закрепить автомобиль за рейсом, закрепить водителя за рейсом, открепить автомобиль, снять водителя, получить информацию о рейсе.
- (е) **МенеджерЛогистики:** класс, представляющий менеджера, имеющий идентификатор, список контролируемых маршрутов, ФИО.

24. **Описание ситуации:** Рассмотрим работу службы парковки аэропорта. На территории аэропорта есть несколько парковочных зон для разных типов транспорта (краткосрочная, долгосрочная, VIP). Операторы контролируют занятость мест, прием оплаты и работу шлагбаумов.

Создаваемые классы: 'ПарковочнаяЗона', 'ПарковочноеМесто', 'Автомобиль', 'ОператорПарковки', 'СлужбаПарковки'.

Для классов реализовать следующие простые методы, используя для хранения данных списки (['']) Python:

- (a) **ПарковочнаяЗона:** добавить парковочное место, удалить место, получить список мест в зоне, получить список всех автомобилей. Так же парковочной зоне соответствует стоимость часа стоянки.
- (b) **ПарковочноеМесто:** занять место автомобилем, освободить место, получить текущий статус (свободно/занято).
- (c) **Автомобиль:** зафиксировать время въезда, время выезда + рассчитать стоимость парковки (с учетом стоимости часа), получить историю.
- (d) **ОператорПарковки:** класс, представляющий оператора, имеющий идентификатор, смену, список контролируемых зон, ФИО.
- (e) **СлужбаПарковки:** добавить новую парковочную зону, нанять оператора, вывести информацию о зонах, местах, операторах, удалить зону, уволить оператора.

25. **Описание ситуации:** Рассмотрим работу центра управления речным судоходством. Диспетчеры следят за движением судов по фарватеру, распределяют шлюзы, контролируют соблюдение графика движения и обеспечивают безопасность судоходства.

Создаваемые классы: 'Судно', 'Шлюз', 'Фарватер', 'ДиспетчерСудоходства', 'ЦентрУправления'.

Для классов реализовать следующие простые методы, используя для хранения данных списки (['']) Python:

- (a) **Судно:** начать движение по фарватеру, завершить движение, получить текущее местоположение, зафиксировать прохождение шлюза.
- (b) **Шлюз:** принять судно для шлюзования, завершить шлюзование, получить текущий статус (свободен/занят).
- (c) **Фарватер:** добавить участок фарватера, удалить участок, получить список судов на фарватере.
- (d) **ДиспетчерСудоходства:** класс, представляющий диспетчера, имеющий идентификатор, смену, список контролируемых шлюзов, ФИО.
- (e) **ЦентрУправления:** добавить шлюз в систему, зарегистрировать судно, нанять диспетчера, вывести информацию о шлюзах, фарватерах, судах, диспетчерах, удалить шлюз, уволить диспетчера.

26. **Описание ситуации:** Рассмотрим работу службы технического контроля метрополитена. Инспекторы проверяют состояние путей, тоннелей, подвижного состава и оборудования станций. Дефекты фиксируются в системе для оперативного устранения ремонтными бригадами.

Создаваемые классы: 'УчастокПути', 'ПодвижнойСостав', 'Инспектор', 'Дефект', 'СлужбаКонтроля'.

Для классов реализовать следующие простые методы, использующие для хранения данных списки ([]) Python:

- (a) **УчастокПути:** добавить информацию о дефекте, получить список неустраненных дефектов на участке.
- (b) **ПодвижнойСостав:** добавить запись о техническом осмотре, удалить ошибочную запись, отобразить историю осмотров.
- (c) **Инспектор:** класс, представляющий инспектора, имеющий идентификатор, квалификацию, список закрепленных участков, ФИО.
- (d) **Дефект:** зафиксировать время обнаружения, время устранения, получить статус устранения.
- (e) **СлужбаКонтроля:** добавить участок пути в систему, зарегистрировать подвижной состав, нанять инспектора, вывести информацию об участках, составе, инспекторах, дефектах, удалить участок, уволить инспектора, снять с эксплуатации подвижной состав.

27. **Описание ситуации:** Рассмотрим работу центра управления умными светофорами на перекрестках. Умные светофоры адаптивно меняют режим работы в зависимости от транспортного потока, приоритизируя общественный транспорт и спецтранспорт. Система анализирует данные с датчиков и камер, оптимизируя пропускную способность перекрестков.

Создаваемые классы: УмныйСветофор, Перекресток, ДатчикТранспортногоПотока, ИнженерАТС, ЦентрУправленияСветофорами.

Для классов реализовать следующие простые методы, используя для хранения данных списки ([]) Python:

- (a) **УмныйСветофор:** изменить длительность фаз (красный/зеленый), перейти в аварийный режим, получить текущий режим работы.
- (b) **Перекресток:** добавить светофор к перекрестку, удалить светофор, получить список всех светофоров перекрестка.
- (c) **ДатчикТранспортногоПотока:** установить текущие данные о интенсивности движения, получить текущие показания, получить историю показаний.
- (d) **ИнженерАТС:** класс, представляющий инженера автоматизированной транспортной системы, имеющий идентификатор, квалификацию, список закрепленных перекрестков, ФИО.
- (e) **ЦентрУправленияСветофорами:** добавить новый перекресток в систему, установить умный светофор, нанять инженера, вывести информацию о перекрестках, светофорах, инженерах, удалить перекресток, уволить инженера, снять умный светофор.

28. **Описание ситуации:** Рассмотрим работу монорельсовой транспортной системы. Монорельс движется по эстакаде, состоящей из станций и перегонов. Составы имеют фиксированное количество вагонов. Операторы управляют движением составов, следят за соблюдением графика и безопасностью пассажиров.

Создаваемые классы: СтанцияМонорельса, СоставМонорельса, ВагонМонорельса, ОператорСистемы, УправлениеМонорельсом.

Для классов реализовать следующие простые методы, используя для хранения данных списки ([]) Python:

- (a) **СтанцияМонорельса:** принять состав, отправить состав, получить список составов на станции.
- (b) **СоставМонорельса:** добавить вагон в состав (при техническом обслуживании), удалить вагон, получить список вагонов.
- (c) **ВагонМонорельса:** зафиксировать текущий пробег, провести техническое обслуживание, получить историю обслуживаний.
- (d) **ОператорСистемы:** класс, представляющий оператора, имеющий идентификатор, смену, список закрепленных станций, ФИО.
- (e) **УправлениеМонорельсом:** добавить новую станцию, ввести состав в эксплуатацию, нанять оператора, вывести информацию о станциях, составах, операторах, закрыть станцию на ремонт, списать состав, уволить оператора.

29. **Описание ситуации:** Рассмотрим работу канатной дороги. Канатная дорога состоит из линий с опорами и кабинок, перемещающихся между станциями. Кабинки имеют ограниченную вместимость. Техники обслуживают механизмы и следят за безопасностью.

Создаваемые классы: ЛинияКанатнойДороги, Кабинка, СтанцияКанатнойДороги, Техник, УправлениеКанатнойДорогой.

Для классов реализовать следующие простые методы, используя для хранения данных списки ([]) Python:

- (a) **ЛинияКанатнойДороги:** добавить кабинку на линию, снять кабинку, получить список кабинок на линии.
- (b) **Кабинка:** запустить в движение, остановить для посадки/высадки, получить текущий статус (движется/стоит).
- (c) **СтанцияКанатнойДороги:** принять кабинку, отправить кабинку, получить список кабинок на станции.
- (d) **Техник:** класс, представляющий техника, имеющий идентификатор, квалификацию, список закрепленных линий, ФИО.
- (e) **УправлениеКанатнойДорогой:** добавить новую линию, ввести кабинку в эксплуатацию, нанять техника, вывести информацию о линиях, кабинках, техниках, закрыть линию на обслуживание, списать кабинку, уволить техника.

30. **Описание ситуации:** Рассмотрим работу службы доставки с использованием дронов. Дроны осуществляют доставку небольших грузов между пунктами выдачи. Каждый дрон имеет грузоподъемность и дальность полета. Операторы управляют полетами дронов и обслуживают пункты выдачи.

Создаваемые классы: ПунктВыдачи, Дрон, Груз, ОператорДронов, СлужбаДоставки.

Для классов реализовать следующие простые методы, используя для хранения данных списки ([]) Python:

- (a) **ПунктВыдачи:** принять дрон с грузом, отправить дрон, получить список дронов в пункте.
- (b) **Дрон:** загрузить груз, выгрузить груз, начать полет, завершить полет, получить текущий статус (в полете/на земле).

- (c) **Груз:** зарегистрировать отправку, зарегистрировать доставку, получить историю перемещений.
- (d) **ОператорДронов:** класс, представляющий оператора, имеющий идентификатор, смену, список закрепленных пунктов выдачи, ФИО.
- (e) **СлужбаДоставки:** добавить новый пункт выдачи, ввести дрон в эксплуатацию, нанять оператора, вывести информацию о пунктах, дронах, операторах, закрыть пункт, списать дрон, уволить оператора.

2.2 Семинар «Конструкторы, наследование и полиморфизм. 1 часть» (2 часа)

В ходе работы решите 4 задачи. Предполагается, что пользователь класса не имеет права обращаться к свойствам напрямую (соблюдая принцип инкапсуляции), а должен использовать методы.

Продемонстрируйте работоспособность всех методов (из задания) посредством создания запускаемых файлов, где осуществляется вызов методов для разных ситуаций (без ручного ввода, но с выводом результатов в консоль).

Каждый класс должен сохраняться в отдельном исходном файле. Необходимо соблюдать все стандартные требования к качеству кода (отступы, именования переменных, классов, методов, проверка корректности входных данных). Для каждого класса создайте отдельный запускаемый файл для проверки всех его методов (допускается использование других классов в этих тестах).

Все предлагаемые классы в заданиях упрощенные; для использования в production-окружении они требуют серьезной доработки. Суть задания — в отработке базовых навыков, а не в идеальном моделировании предложенных ситуаций.

Для сдачи работы будьте готовы пояснить или аналогично заданию модифицировать любую часть кода, а также ответить на вопросы:

1. Что обозначает свойство наследования в парадигме ООП?
2. Что обозначает свойство полиморфизма в парадигме ООП?
3. Опишите реализацию наследования в Python
4. Как создать конструктор в Python
5. Как реализовать абстрактный класс в Python (и что это значит)
6. Как реализовать абстрактные методы в Python (и что это значит)

Если вы нашли в задачнике ошибки, опечатки и другие недостатки, то вы можете сделать pull-request.

Срок сдачи работы (начала сдачи): через одно занятие после его выдачи. В последующие сроки оценка будет снижаться (при отсутствии оправдывающих документов).

2.2.1 Задача 1

1. Напишите программу, которая создаёт класс `Circle` с методами для вычисления площади и длины окружности (периметра). Программа должна запрашивать у пользователя радиус и выводить вычисленные площадь и длину окружности.

Инструкции:

- (a) Создайте класс `Circle` с методом `__init__`, который принимает радиус окружности в качестве аргумента и сохраняет его в атрибуте `self.__radius`.
- (b) Создайте метод `calculate_circle_area`, без аргументов, который вычисляет площадь круга по формуле:

$$\pi \cdot \text{__radius}^2$$

- (c) Создайте метод `calculate_circle_perimeter` без аргументов, который вычисляет длину окружности по формуле:

$$2 \cdot \pi \cdot \text{__radius}$$

- (d) Напишите цикл, который повторяется 10 раз. В каждой итерации программа должна:
- запрашивать у пользователя радиус окружности,
 - создавать экземпляр класса `Circle` с этим радиусом,
 - вычислять площадь и длину окружности с помощью соответствующих методов,
 - выводить результаты на экран.

Пример использования:

```
radius = 3
circle = Circle(radius)
area = circle.calculate_circle_area()
perimeter = circle.calculate_circle_perimeter()
print(f"Площадь окружности равна: {area}")
print(f"Периметр окружности равен: {perimeter}")
```

Вывод:

```
Площадь окружности равна: 28.274333882308138
Периметр окружности равен: 18.84955592153876
```

2. Напишите программу, которая создаёт класс `Square` с методами для вычисления площади и периметра. Программа должна запрашивать у пользователя длину стороны и выводить вычисленные площадь и периметр.

Инструкции:

- (a) Создайте класс `Square` с методом `__init__`, который принимает длину стороны квадрата в качестве аргумента и сохраняет её в атрибуте `self.__side`.
- (b) Создайте метод `calculate_area`, без аргументов, который вычисляет площадь квадрата по формуле:

$$\text{__side}^2$$

- (c) Создайте метод `calculate_perimeter` без аргументов, который вычисляет периметр квадрата по формуле:

$$4 \cdot \text{__side}$$

- (d) Напишите цикл, который повторяется 10 раз. В каждой итерации программа должна:
- запрашивать у пользователя длину стороны квадрата,
 - создавать экземпляр класса `Square` с этой длиной,
 - вычислять площадь и периметр с помощью соответствующих методов,
 - выводить результаты на экран.

Пример использования:

```
side = 5
square = Square(side)
area = square.calculate_area()
perimeter = square.calculate_perimeter()
print(f"Площадь квадрата равна: {area}")
print(f"Периметр квадрата равен: {perimeter}")
```

Вывод:

Площадь квадрата равна: 25
Периметр квадрата равен: 20

3. Напишите программу, которая создаёт класс **Rectangle** с методами для вычисления площади и периметра. Программа должна запрашивать у пользователя ширину прямоугольника (при соотношении сторон 1:2) и выводить вычисленные площадь и периметр.

Инструкции:

- (a) Создайте класс **Rectangle** с методом `__init__`, который принимает ширину прямоугольника в качестве аргумента и сохраняет её в атрибуте `self.__width`. Высота прямоугольника должна быть в два раза больше ширины.
- (b) Создайте метод `calculate_area`, без аргументов, который вычисляет площадь прямоугольника по формуле:

$$\text{__width} \cdot (2 \cdot \text{__width})$$

- (c) Создайте метод `calculate_perimeter` без аргументов, который вычисляет периметр прямоугольника по формуле:

$$2 \cdot (\text{__width} + 2 \cdot \text{__width})$$

- (d) Напишите цикл, который повторяется 10 раз. В каждой итерации программа должна:
 - i. запрашивать у пользователя ширину прямоугольника,
 - ii. создавать экземпляр класса **Rectangle** с этой шириной,
 - iii. вычислять площадь и периметр с помощью соответствующих методов,
 - iv. выводить результаты на экран.

Пример использования:

```
width = 3
rectangle = Rectangle(width)
area = rectangle.calculate_area()
perimeter = rectangle.calculate_perimeter()
print(f"Площадь прямоугольника равна: {area}")
print(f"Периметр прямоугольника равен: {perimeter}")
```

Вывод:

Площадь прямоугольника равна: 18
Периметр прямоугольника равен: 18

4. Напишите программу, которая создаёт класс **Triangle** с методами для вычисления площади и периметра. Программа должна запрашивать у пользователя длину стороны равностороннего треугольника и выводить вычисленные площадь и периметр.

Инструкции:

- (a) Создайте класс **Triangle** с методом `__init__`, который принимает длину стороны треугольника в качестве аргумента и сохраняет её в атрибуте `self.__side`.
(b) Создайте метод `calculate_area`, без аргументов, который вычисляет площадь равностороннего треугольника по формуле:

$$\frac{\sqrt{3}}{4} \cdot \text{__side}^2$$

- (c) Создайте метод `calculate_perimeter` без аргументов, который вычисляет периметр треугольника по формуле:

$$3 \cdot \text{__side}$$

- (d) Напишите цикл, который повторяется 10 раз. В каждой итерации программа должна:
- запрашивать у пользователя длину стороны треугольника,
 - создавать экземпляр класса **Triangle** с этой длиной,
 - вычислять площадь и периметр с помощью соответствующих методов,
 - выводить результаты на экран.

Пример использования:

```
side = 4
triangle = Triangle(side)
area = triangle.calculate_area()
perimeter = triangle.calculate_perimeter()
print(f"Площадь треугольника равна: {area}")
print(f"Периметр треугольника равен: {perimeter}")
```

Вывод:

Площадь треугольника равна: 6.928203230275509
Периметр треугольника равен: 12

5. Напишите программу, которая создаёт класс **Sphere** с методами для вычисления площади поверхности и объёма. Программа должна запрашивать у пользователя радиус сферы и выводить вычисленные площадь поверхности и объём.

Инструкции:

- (a) Создайте класс `Sphere` с методом `__init__`, который принимает радиус сферы в качестве аргумента и сохраняет его в атрибуте `self.__radius`.
- (b) Создайте метод `calculate_surface_area`, без аргументов, который вычисляет площадь поверхности сферы по формуле:

$$4 \cdot \pi \cdot \text{__radius}^2$$

- (c) Создайте метод `calculate_volume` без аргументов, который вычисляет объём сферы по формуле:

$$\frac{4}{3} \cdot \pi \cdot \text{__radius}^3$$

- (d) Напишите цикл, который повторяется 10 раз. В каждой итерации программа должна:
 - i. запрашивать у пользователя радиус сферы,
 - ii. создавать экземпляр класса `Sphere` с этим радиусом,
 - iii. вычислять площадь поверхности и объём с помощью соответствующих методов,
 - iv. выводить результаты на экран.

Пример использования:

```
radius = 2
sphere = Sphere(radius)
surface_area = sphere.calculate_surface_area()
volume = sphere.calculate_volume()
print(f"Площадь поверхности сферы равна: {surface_area}")
print(f"Объём сферы равен: {volume}")
```

Вывод:

```
Площадь поверхности сферы равна: 50.26548245743669
Объём сферы равен: 33.510321638291124
```

- 6. Напишите программу, которая создаёт класс `Cylinder` с методами для вычисления объёма и площади боковой поверхности. Программа должна запрашивать у пользователя радиус основания и выводить вычисленные объём и площадь боковой поверхности (высота цилиндра фиксирована и равна 5).

Инструкции:

- (a) Создайте класс `Cylinder` с методом `__init__`, который принимает радиус основания цилиндра в качестве аргумента и сохраняет его в атрибуте `self.__radius`. Высота цилиндра фиксирована и равна 5.
- (b) Создайте метод `calculate_volume`, без аргументов, который вычисляет объём цилиндра по формуле:

$$\pi \cdot \text{__radius}^2 \cdot 5$$

- (c) Создайте метод `calculate_lateral_area` без аргументов, который вычисляет площадь боковой поверхности цилиндра по формуле:

$$2 \cdot \pi \cdot \text{_radius} \cdot 5$$

- (d) Напишите цикл, который повторяется 10 раз. В каждой итерации программа должна:
- запрашивать у пользователя радиус основания цилиндра,
 - создавать экземпляр класса `Cylinder` с этим радиусом,
 - вычислять объём и площадь боковой поверхности с помощью соответствующих методов,
 - выводить результаты на экран.

Пример использования:

```
radius = 3
cylinder = Cylinder(radius)
volume = cylinder.calculate_volume()
lateral_area = cylinder.calculate_lateral_area()
print(f"Объём цилиндра равен: {volume}")
print(f"Площадь боковой поверхности равна: {lateral_area}")
```

Вывод:

```
Объём цилиндра равен: 141.3716694115407
Площадь боковой поверхности равна: 94.24777960769379
```

7. Напишите программу, которая создаёт класс `Cone` с методами для вычисления объёма и площади боковой поверхности. Программа должна запрашивать у пользователя радиус основания и выводить вычисленные объём и площадь боковой поверхности (высота конуса фиксирована и равна 10).

Инструкции:

- (a) Создайте класс `Cone` с методом `__init__`, который принимает радиус основания конуса в качестве аргумента и сохраняет его в атрибуте `self.__radius`. Высота конуса фиксирована и равна 10.
- (b) Создайте метод `calculate_volume`, без аргументов, который вычисляет объём конуса по формуле:

$$\frac{1}{3} \cdot \pi \cdot \text{_radius}^2 \cdot 10$$

- (c) Создайте метод `calculate_lateral_area` без аргументов, который вычисляет площадь боковой поверхности конуса по формуле:

$$\pi \cdot \text{_radius} \cdot \sqrt{\text{_radius}^2 + 10^2}$$

- (d) Напишите цикл, который повторяется 10 раз. В каждой итерации программа должна:

- i. запрашивать у пользователя радиус основания конуса,
- ii. создавать экземпляр класса `Cone` с этим радиусом,
- iii. вычислять объём и площадь боковой поверхности с помощью соответствующих методов,
- iv. выводить результаты на экран.

Пример использования:

```
radius = 3
cone = Cone(radius)
volume = cone.calculate_volume()
lateral_area = cone.calculate_lateral_area()
print(f"Объём конуса равен: {volume}")
print(f"Площадь боковой поверхности равна: {lateral_area}")
```

Вывод:

Объём конуса равен: 94.24777960769379
Площадь боковой поверхности равна: 94.86832980505137

8. Напишите программу, которая создаёт класс `Cube` с методами для вычисления объёма и площади полной поверхности. Программа должна запрашивать у пользователя длину ребра куба и выводить вычисленные объём и площадь.

Инструкции:

- (a) Создайте класс `Cube` с методом `__init__`, который принимает длину ребра куба в качестве аргумента и сохраняет её в атрибуте `self.__side`.
- (b) Создайте метод `calculate_volume`, без аргументов, который вычисляет объём куба по формуле:

$$\text{__side}^3$$

- (c) Создайте метод `calculate_surface_area` без аргументов, который вычисляет площадь полной поверхности куба по формуле:

$$6 \cdot \text{__side}^2$$

- (d) Напишите цикл, который повторяется 10 раз. В каждой итерации программа должна:
 - i. запрашивать у пользователя длину ребра куба,
 - ii. создавать экземпляр класса `Cube` с этой длиной,
 - iii. вычислять объём и площадь полной поверхности с помощью соответствующих методов,
 - iv. выводить результаты на экран.

Пример использования:

```
side = 4
cube = Cube(side)
volume = cube.calculate_volume()
surface_area = cube.calculate_surface_area()
print(f"Объём куба равен: {volume}")
print(f"Площадь полной поверхности равна: {surface_area}")
```

Вывод:

Объём куба равен: 64
Площадь полной поверхности равна: 96

9. Напишите программу, которая создаёт класс **Parallelogram** с методами для вычисления площади и периметра. Программа должна запрашивать у пользователя длину основания параллелограмма и выводить вычисленные площадь и периметр (высота параллелограмма фиксирована и равна 8, а боковая сторона равна 6).

Инструкции:

- (a) Создайте класс **Parallelogram** с методом **__init__**, который принимает длину основания параллелограмма в качестве аргумента и сохраняет её в атрибуте **self.__base**. Высота параллелограмма фиксирована и равна 8, а боковая сторона равна 6.
- (b) Создайте метод **calculate_area**, без аргументов, который вычисляет площадь параллелограмма по формуле:

$$\text{__base} \cdot 8$$

- (c) Создайте метод **calculate_perimeter** без аргументов, который вычисляет периметр параллелограмма по формуле:

$$2 \cdot (\text{__base} + 6)$$

- (d) Напишите цикл, который повторяется 10 раз. В каждой итерации программа должна:
 - i. запрашивать у пользователя длину основания параллелограмма,
 - ii. создавать экземпляр класса **Parallelogram** с этой длиной,
 - iii. вычислять площадь и периметр с помощью соответствующих методов,
 - iv. выводить результаты на экран.

Пример использования:

```
base = 5
parallelogram = Parallelogram(base)
area = parallelogram.calculate_area()
perimeter = parallelogram.calculate_perimeter()
print(f"Площадь параллелограмма равна: {area}")
print(f"Периметр параллелограмма равен: {perimeter}")
```

Вывод:

Площадь параллелограмма равна: 40

Периметр параллелограмма равен: 22

10. Напишите программу, которая создаёт класс `Ellipse` с методами для вычисления площади и приближённого значения периметра. Программа должна запрашивать у пользователя длину большой полуоси и выводить вычисленные площадь и периметр (длина малой полуоси фиксирована и равна 3).

Инструкции:

- (a) Создайте класс `Ellipse` с методом `__init__`, который принимает длину большой полуоси эллипса в качестве аргумента и сохраняет её в атрибуте `self.__major_axis`. Длина малой полуоси фиксирована и равна 3.
- (b) Создайте метод `calculate_area`, без аргументов, который вычисляет площадь эллипса по формуле:

$$\pi \cdot \text{__major_axis} \cdot 3$$

- (c) Создайте метод `calculate_perimeter` без аргументов, который вычисляет приближённое значение периметра эллипса по формуле Рамануджана:

$$\pi \cdot \left(3(\text{__major_axis} + 3) - \sqrt{(3\text{__major_axis} + 3)(\text{__major_axis} + 9)} \right)$$

- (d) Напишите цикл, который повторяется 10 раз. В каждой итерации программа должна:
- запрашивать у пользователя длину большой полуоси эллипса,
 - создавать экземпляр класса `Ellipse` с этой длиной,
 - вычислять площадь и периметр с помощью соответствующих методов,
 - выводить результаты на экран.

Пример использования:

```
major_axis = 5
ellipse = Ellipse(major_axis)
area = ellipse.calculate_area()
perimeter = ellipse.calculate_perimeter()
print(f"Площадь эллипса равна: {area}")
print(f"Периметр эллипса равен: {perimeter}")
```

Вывод:

Площадь эллипса равна: 47.12388980384689

Периметр эллипса равен: 25.74488980384689

11. Напишите программу, которая создаёт класс `BankAccount` с методами для вычисления начисленных процентов и суммы налога на доход. Программа должна запрашивать у пользователя начальный баланс счёта и выводить вычисленные проценты и налог (процентная ставка фиксирована и равна 5%, налоговая ставка на доход фиксирована и равна 13%).

Инструкции:

- (a) Создайте класс `BankAccount` с методом `__init__`, который принимает начальный баланс счёта в качестве аргумента и сохраняет его в атрибуте `self.__balance`.
- (b) Создайте метод `calculate_interest`, без аргументов, который вычисляет начисленные проценты по формуле:

$$\text{__balance} \cdot 0.05$$

- (c) Создайте метод `calculate_tax` без аргументов, который вычисляет сумму налога на полученный доход (проценты) по формуле:

$$(\text{__balance} \cdot 0.05) \cdot 0.13$$

- (d) Напишите цикл, который повторяется 10 раз. В каждой итерации программа должна:
 - i. запрашивать у пользователя начальный баланс счёта,
 - ii. создавать экземпляр класса `BankAccount` с этим балансом,
 - iii. вычислять начисленные проценты и сумму налога с помощью соответствующих методов,
 - iv. выводить результаты на экран.

Пример использования:

```
balance = 1000
account = BankAccount(balance)
interest = account.calculate_interest()
tax = account.calculate_tax()
print(f"Начисленные проценты: {interest}")
print(f"Сумма налога на доход: {tax}")
```

Вывод:

```
Начисленные проценты: 50.0
Сумма налога на доход: 6.5
```

12. Напишите программу, которая создаёт класс `TemperatureConverter` с методами для преобразования температуры из градусов Цельсия в Фаренгейты и Кельвины. Программа должна запрашивать у пользователя температуру в Цельсиях и выводить преобразованные значения.

Инструкции:

- (a) Создайте класс `TemperatureConverter` с методом `__init__`, который принимает температуру в градусах Цельсия в качестве аргумента и сохраняет её в атрибуте `self.__celsius`.

- (b) Создайте метод `to_fahrenheit`, без аргументов, который преобразует температуру в Фаренгейты по формуле:

$$(__celsius \times \frac{9}{5}) + 32$$

- (c) Создайте метод `to_kelvin` без аргументов, который преобразует температуру в Кельвины по формуле:

$$__celsius + 273.15$$

- (d) Напишите цикл, который повторяется 10 раз. В каждой итерации программа должна:
- запрашивать у пользователя температуру в градусах Цельсия,
 - создавать экземпляр класса `TemperatureConverter` с этим значением,
 - вычислять температуру в Фаренгейтах и Кельвинах с помощью соответствующих методов,
 - выводить результаты на экран.

Пример использования:

```
celsius = 25
converter = TemperatureConverter(celsius)
fahrenheit = converter.to_fahrenheit()
kelvin = converter.to_kelvin()
print(f"Температура в Фаренгейтах: {fahrenheit}")
print(f"Температура в Кельвинах: {kelvin}")
```

Вывод:

```
Температура в Фаренгейтах: 77.0
Температура в Кельвинах: 298.15
```

13. Напишите программу, которая создаёт класс `DistanceConverter` с методами для преобразования расстояния из метров в километры и мили. Программа должна запрашивать у пользователя расстояние в метрах и выводить преобразованные значения.

Инструкции:

- (a) Создайте класс `DistanceConverter` с методом `__init__`, который принимает расстояние в метрах в качестве аргумента и сохраняет его в атрибуте `self.__meters`.
- (b) Создайте метод `to_kilometers`, без аргументов, который преобразует расстояние в километры по формуле:

$$__meters \div 1000$$

- (c) Создайте метод `to_miles` без аргументов, который преобразует расстояние в мили по формуле:

$$__meters \div 1609.344$$

- (d) Напишите цикл, который повторяется 10 раз. В каждой итерации программа должна:

- i. запрашивать у пользователя расстояние в метрах,
- ii. создавать экземпляр класса `DistanceConverter` с этим значением,
- iii. вычислять расстояние в километрах и милях с помощью соответствующих методов,
- iv. выводить результаты на экран.

Пример использования:

```
meters = 1609.344
converter = DistanceConverter(meters)
kilometers = converter.to_kilometers()
miles = converter.to_miles()
print(f"Расстояние в километрах: {kilometers}")
print(f"Расстояние в милях: {miles}")
```

Вывод:

```
Расстояние в километрах: 1.609344
Расстояние в милях: 1.0
```

14. Напишите программу, которая создаёт класс `WeightConverter` с методами для преобразования массы из килограммов в граммы и фунты. Программа должна запрашивать у пользователя массу в килограммах и выводить преобразованные значения.

Инструкции:

- (a) Создайте класс `WeightConverter` с методом `__init__`, который принимает массу в килограммах в качестве аргумента и сохраняет её в атрибуте `self.__kg`.
- (b) Создайте метод `to_grams`, без аргументов, который преобразует массу в граммы по формуле:

$$\text{__kg} \times 1000$$

- (c) Создайте метод `to_pounds` без аргументов, который преобразует массу в фунты по формуле:

$$\text{__kg} \times 2.20462$$

- (d) Напишите цикл, который повторяется 10 раз. В каждой итерации программа должна:
 - i. запрашивать у пользователя массу в килограммах,
 - ii. создавать экземпляр класса `WeightConverter` с этим значением,
 - iii. вычислять массу в граммах и фунтах с помощью соответствующих методов,
 - iv. выводить результаты на экран.

Пример использования:

```
kg = 2.5
converter = WeightConverter(kg)
grams = converter.to_grams()
pounds = converter.to_pounds()
print(f"Масса в граммах: {grams}")
print(f"Масса в фунтах: {pounds}")
```

Вывод:

```
Масса в граммах: 2500.0
Масса в фунтах: 5.51155
```

15. Напишите программу, которая создаёт класс `TimeConverter` с методами для преобразования времени из секунд в минуты и часы. Программа должна запрашивать у пользователя время в секундах и выводить преобразованные значения.

Инструкции:

- (a) Создайте класс `TimeConverter` с методом `__init__`, который принимает время в секундах в качестве аргумента и сохраняет его в атрибуте `self.__seconds`.
- (b) Создайте метод `to_minutes`, без аргументов, который преобразует время в минуты по формуле:

$$\text{__seconds} \div 60$$

- (c) Создайте метод `to_hours` без аргументов, который преобразует время в часы по формуле:

$$\text{__seconds} \div 3600$$

- (d) Напишите цикл, который повторяется 10 раз. В каждой итерации программа должна:
 - i. запрашивать у пользователя время в секундах,
 - ii. создавать экземпляр класса `TimeConverter` с этим значением,
 - iii. вычислять время в минутах и часах с помощью соответствующих методов,
 - iv. выводить результаты на экран.

Пример использования:

```
seconds = 7200
converter = TimeConverter(seconds)
minutes = converter.to_minutes()
hours = converter.to_hours()
print(f"Время в минутах: {minutes}")
print(f"Время в часах: {hours}")
```

Вывод:

Время в минутах: 120.0

Время в часах: 2.0

16. Напишите программу, которая создаёт класс **SpeedConverter** с методами для преобразования скорости из километров в час в метры в секунду и мили в час. Программа должна запрашивать у пользователя скорость в км/ч и выводить преобразованные значения.

Инструкции:

- (a) Создайте класс **SpeedConverter** с методом `__init__`, который принимает скорость в км/ч в качестве аргумента и сохраняет её в атрибуте `self.__kmh`.
- (b) Создайте метод `to_ms`, без аргументов, который преобразует скорость в м/с по формуле:

$$\text{__kmh} \times \frac{1000}{3600}$$

- (c) Создайте метод `to_mph` без аргументов, который преобразует скорость в мили/ч по формуле:

$$\text{__kmh} \div 1.60934$$

- (d) Напишите цикл, который повторяется 10 раз. В каждой итерации программа должна:
- запрашивать у пользователя скорость в км/ч,
 - создавать экземпляр класса **SpeedConverter** с этим значением,
 - вычислять скорость в м/с и милях/ч с помощью соответствующих методов,
 - выводить результаты на экран.

Пример использования:

```
kmh = 100
converter = SpeedConverter(kmh)
ms = converter.to_ms()
mph = converter.to_mph()
print(f"Скорость в м/с: {ms}")
print(f"Скорость в милях/ч: {mph}")
```

Вывод:

Скорость в м/с: 27.77777777777778

Скорость в милях/ч: 62.13727366498068

17. Напишите программу, которая создаёт класс **AreaConverter** с методами для преобразования площади из квадратных метров в гектары и акры. Программа должна запрашивать у пользователя площадь в м² и выводить преобразованные значения.

Инструкции:

- (a) Создайте класс `AreaConverter` с методом `__init__`, который принимает площадь в м² в качестве аргумента и сохраняет её в атрибуте `self.__sq_meters`.
- (b) Создайте метод `to_hectares`, без аргументов, который преобразует площадь в гектары по формуле:

$$\text{__sq_meters} \div 10000$$

- (c) Создайте метод `to_acres` без аргументов, который преобразует площадь в акры по формуле:

$$\text{__sq_meters} \div 4046.86$$

- (d) Напишите цикл, который повторяется 10 раз. В каждой итерации программа должна:
 - i. запрашивать у пользователя площадь в м²,
 - ii. создавать экземпляр класса `AreaConverter` с этим значением,
 - iii. вычислять площадь в гектарах и акрах с помощью соответствующих методов,
 - iv. выводить результаты на экран.

Пример использования:

```
sq_meters = 10000
converter = AreaConverter(sq_meters)
hectares = converter.to_hectares()
acres = converter.to_acres()
print(f"Площадь в гектарах: {hectares}")
print(f"Площадь в акрах: {acres}")
```

Вывод:

```
Площадь в гектары: 1.0
Площадь в акрах: 2.4710514233241505
```

18. Напишите программу, которая создаёт класс `VolumeConverter` с методами для преобразования объёма из литров в галлоны и кубические метры. Программа должна запрашивать у пользователя объём в литрах и выводить преобразованные значения.

Инструкции:

- (a) Создайте класс `VolumeConverter` с методом `__init__`, который принимает объём в литрах в качестве аргумента и сохраняет его в атрибуте `self.__liters`.
- (b) Создайте метод `to_gallons`, без аргументов, который преобразует объём в галлоны по формуле:

$$\text{__liters} \div 3.78541$$

- (c) Создайте метод `to_cubic_meters` без аргументов, который преобразует объём в кубические метры по формуле:

$$\text{__liters} \div 1000$$

- (d) Напишите цикл, который повторяется 10 раз. В каждой итерации программа должна:
- запрашивать у пользователя объём в литрах,
 - создавать экземпляр класса **VolumeConverter** с этим значением,
 - вычислять объём в галлонах и кубических метрах с помощью соответствующих методов,
 - выводить результаты на экран.

Пример использования:

```
liters = 10
converter = VolumeConverter(liters)
gallons = converter.to_gallons()
cubic_meters = converter.to_cubic_meters()
print(f"Объём в галлонах: {gallons}")
print(f"Объём в кубических метрах: {cubic_meters}")
```

Вывод:

```
Объём в галлонах: 2.641720523581484
Объём в кубических метрах: 0.01
```

19. Напишите программу, которая создаёт класс **EnergyConverter** с методами для преобразования энергии из джоулей в калории и киловатт-часы. Программа должна запрашивать у пользователя энергию в джоулях и выводить преобразованные значения.

Инструкции:

- (a) Создайте класс **EnergyConverter** с методом `__init__`, который принимает энергию в джоулях в качестве аргумента и сохраняет её в атрибуте `self.__joules`.
- (b) Создайте метод `to_calories`, без аргументов, который преобразует энергию в калории по формуле:

$$\text{__joules} \div 4.184$$

- (c) Создайте метод `to_kwh` без аргументов, который преобразует энергию в киловатт-часы по формуле:

$$\text{__joules} \div 3.6 \times 10^6$$

- (d) Напишите цикл, который повторяется 10 раз. В каждой итерации программа должна:
- запрашивать у пользователя энергию в джоулях,
 - создавать экземпляр класса **EnergyConverter** с этим значением,
 - вычислять энергию в калориях и киловатт-часах с помощью соответствующих методов,
 - выводить результаты на экран.

Пример использования:

```
joules = 10000
converter = EnergyConverter(joules)
calories = converter.to_calories()
kwh = converter.to_kwh()
print(f"Энергия в калориях: {calories}")
print(f"Энергия в киловатт-часах: {kwh}")
```

Вывод:

```
Энергия в калориях: 2390.057361376673
Энергия в киловатт-часах: 0.0027777777777777778
```

20. Напишите программу, которая создаёт класс **PowerConverter** с методами для преобразования мощности из ватт в лошадиные силы и киловатты. Программа должна запрашивать у пользователя мощность в ваттах и выводить преобразованные значения.

Инструкции:

- (a) Создайте класс **PowerConverter** с методом `__init__`, который принимает мощность в ваттах в качестве аргумента и сохраняет её в атрибуте `self.__watts`.
- (b) Создайте метод `to_horsepower`, без аргументов, который преобразует мощность в лошадиные силы по формуле:

$$\text{__watts} \div 745.7$$

- (c) Создайте метод `to_kilowatts` без аргументов, который преобразует мощность в киловатты по формуле:

$$\text{__watts} \div 1000$$

- (d) Напишите цикл, который повторяется 10 раз. В каждой итерации программа должна:
 - i. запрашивать у пользователя мощность в ваттах,
 - ii. создавать экземпляр класса **PowerConverter** с этим значением,
 - iii. вычислять мощность в л.с. и киловаттах с помощью соответствующих методов,
 - iv. выводить результаты на экран.

Пример использования:

```
watts = 1000
converter = PowerConverter(watts)
horsepower = converter.to_horsepower()
kilowatts = converter.to_kilowatts()
print(f"Мощность в л.с.: {horsepower}")
print(f"Мощность в киловаттах: {kilowatts}")
```

Вывод:

Мощность в л.с.: 1.3410220903956017

Мощность в киловаттах: 1.0

21. Напишите программу, которая создаёт класс `PressureConverter` с методами для преобразования давления из паскалей в атмосферы и бары. Программа должна запрашивать у пользователя давление в паскалях и выводить преобразованные значения.

Инструкции:

- (a) Создайте класс `PressureConverter` с методом `__init__`, который принимает давление в паскалях в качестве аргумента и сохраняет его в атрибуте `self.__pascals`.
- (b) Создайте метод `to_atm`, без аргументов, который преобразует давление в атмосферы по формуле:

$$\text{__pascals} \div 101325$$

- (c) Создайте метод `to_bar` без аргументов, который преобразует давление в бары по формуле:

$$\text{__pascals} \div 100000$$

- (d) Напишите цикл, который повторяется 10 раз. В каждой итерации программа должна:
- запрашивать у пользователя давление в паскалях,
 - создавать экземпляр класса `PressureConverter` с этим значением,
 - вычислять давление в атмосферах и барах с помощью соответствующих методов,
 - выводить результаты на экран.

Пример использования:

```
pascals = 101325
converter = PressureConverter(pascals)
atm = converter.to_atm()
bar = converter.to_bar()
print(f"Давление в атмосферах: {atm}")
print(f"Давление в барах: {bar}")
```

Вывод:

Давление в атмосферах: 1.0

Давление в барах: 1.01325

22. Напишите программу, которая создаёт класс `ForceConverter` с методами для преобразования силы из ньютонов в дины и фунты-силы. Программа должна запрашивать у пользователя силу в ньютонах и выводить преобразованные значения.

Инструкции:

- (a) Создайте класс `ForceConverter` с методом `__init__`, который принимает силу в ньютонах в качестве аргумента и сохраняет её в атрибуте `self.__newtons`.
- (b) Создайте метод `to_dyne`, без аргументов, который преобразует силу в дины по формуле:

$$__\text{newtons} \times 100000$$

- (c) Создайте метод `to_pound_force` без аргументов, который преобразует силу в фунты-силы по формуле:

$$__\text{newtons} \div 4.44822$$

- (d) Напишите цикл, который повторяется 10 раз. В каждой итерации программа должна:
 - i. запрашивать у пользователя силу в ньютонах,
 - ii. создавать экземпляр класса `ForceConverter` с этим значением,
 - iii. вычислять силу в динах и фунтах-силы с помощью соответствующих методов,
 - iv. выводить результаты на экран.

Пример использования:

```
newtons = 10
converter = ForceConverter(newtons)
dyne = converter.to_dyne()
pound_force = converter.to_pound_force()
print(f"Сила в динах: {dyne}")
print(f"Сила в фунтах-силы: {pound_force}")
```

Вывод:

```
Сила в динах: 1000000.0
Сила в фунтах-силы: 2.248089430997145
```

23. Задание: Конвертер силы

Напишите программу, которая создаёт класс `ForceConverter` с методами для преобразования силы из ньютонов в дины и фунты-силы. Программа должна запрашивать у пользователя силу в ньютонах и выводить преобразованные значения.

Инструкции:

- (a) Создайте класс `ForceConverter` с методом `__init__`, который принимает силу в ньютонах в качестве аргумента и сохраняет её в атрибуте `self.__newtons`.
- (b) Создайте метод `to_dyne`, без аргументов, который преобразует силу в дины по формуле:

$$__\text{newtons} \times 100000$$

- (c) Создайте метод `to_pound_force` без аргументов, который преобразует силу в фунты-силы по формуле:

$$__newtons \div 4.44822$$

- (d) Напишите цикл, который повторяется 10 раз. В каждой итерации программа должна:
- запрашивать у пользователя силу в ньютонах,
 - создавать экземпляр класса `ForceConverter` с этим значением,
 - вычислять силу в динах и фунтах-силы с помощью соответствующих методов,
 - выводить результаты на экран.

Пример использования:

```
newtons = 10
converter = ForceConverter(newtons)
dyne = converter.to_dyne()
pound_force = converter.to_pound_force()
print(f"Сила в динах: {dyne}")
print(f"Сила в фунтах-силы: {pound_force}")
```

Вывод:

```
Сила в динах: 1000000.0
Сила в фунтах-силы: 2.248089430997145
```

24. Напишите программу, которая создаёт класс `ResistanceConverter` с методами для преобразования электрического сопротивления из омов в килоомы и мегаомы. Программа должна запрашивать у пользователя сопротивление в омах и выводить преобразованные значения.

Инструкции:

- (a) Создайте класс `ResistanceConverter` с методом `__init__`, который принимает сопротивление в омах в качестве аргумента и сохраняет его в атрибуте `self.__ohms`.
- (b) Создайте метод `to_kiloohms`, без аргументов, который преобразует сопротивление в килоомы по формуле:

$$__ohms \div 1000$$

- (c) Создайте метод `to_megaohms` без аргументов, который преобразует сопротивление в мегаомы по формуле:

$$__ohms \div 1000000$$

- (d) Напишите цикл, который повторяется 10 раз. В каждой итерации программа должна:
- запрашивать у пользователя сопротивление в омах,
 - создавать экземпляр класса `ResistanceConverter` с этим значением,
 - вычислять сопротивление в килоомах и мегаомах с помощью соответствующих методов,
 - выводить результаты на экран.

Пример использования:

```
ohms = 10000
converter = ResistanceConverter(ohms)
kiloohms = converter.to_kiloohms()
megaohms = converter.to_megaohms()
print(f"Сопротивление в килоомах: {kiloohms}")
print(f"Сопротивление в мегаомах: {megaohms}")
```

Вывод:

```
Сопротивление в килоомах: 10.0
Сопротивление в мегаомах: 0.01
```

25. Дополнительные задания

26. Напишите программу, которая создаёт класс `Pentagon` с методами для вычисления площади и периметра правильного пятиугольника. Программа должна запрашивать у пользователя длину стороны и выводить вычисленные площадь и периметр.

Инструкции:

- (a) Создайте класс `Pentagon` с методом `__init__`, который принимает длину стороны пятиугольника в качестве аргумента и сохраняет её в атрибуте `self.__side`.
- (b) Создайте метод `calculate_area`, без аргументов, который вычисляет площадь правильного пятиугольника по формуле:

$$\frac{1}{4} \sqrt{5(5 + 2\sqrt{5})} \cdot \text{__side}^2$$

- (c) Создайте метод `calculate_perimeter` без аргументов, который вычисляет периметр пятиугольника по формуле:

$$5 \cdot \text{__side}$$

- (d) Напишите цикл, который повторяется 10 раз. В каждой итерации программа должна:
 - i. запрашивать у пользователя длину стороны пятиугольника,
 - ii. создавать экземпляр класса `Pentagon` с этой длиной,
 - iii. вычислять площадь и периметр с помощью соответствующих методов,
 - iv. выводить результаты на экран.

Пример использования:

```
side = 5
pentagon = Pentagon(side)
area = pentagon.calculate_area()
perimeter = pentagon.calculate_perimeter()
print(f"Площадь пятиугольника: {area}")
print(f"Периметр пятиугольника: {perimeter}")
```


Вывод:

Площадь пятиугольника: 43.01193501472417

Периметр пятиугольника: 25

27. Напишите программу, которая создаёт класс `Hexagon` с методами для вычисления площади и периметра правильного шестиугольника. Программа должна запрашивать у пользователя длину стороны и выводить вычисленные площадь и периметр.

Инструкции:

- (a) Создайте класс `Hexagon` с методом `__init__`, который принимает длину стороны шестиугольника в качестве аргумента и сохраняет её в атрибуте `self.__side`.
- (b) Создайте метод `calculate_area`, без аргументов, который вычисляет площадь правильного шестиугольника по формуле:

$$\frac{3\sqrt{3}}{2} \cdot \text{__side}^2$$

- (c) Создайте метод `calculate_perimeter` без аргументов, который вычисляет периметр шестиугольника по формуле:

$$6 \cdot \text{__side}$$

- (d) Напишите цикл, который повторяется 10 раз. В каждой итерации программа должна:
 - i. запрашивать у пользователя длину стороны шестиугольника,
 - ii. создавать экземпляр класса `Hexagon` с этой длиной,
 - iii. вычислять площадь и периметр с помощью соответствующих методов,
 - iv. выводить результаты на экран.

Пример использования:

```
side = 4
hexagon = Hexagon(side)
area = hexagon.calculate_area()
perimeter = hexagon.calculate_perimeter()
print(f"Площадь шестиугольника: {area}")
print(f"Периметр шестиугольника: {perimeter}")
```

Вывод:

Площадь шестиугольника: 41.569219381653056

Периметр шестиугольника: 24

28. Напишите программу, которая создаёт класс `AngleConverter` с методами для преобразования углов из градусов в радианы и градусы. Программа должна запрашивать у пользователя угол в градусах и выводить преобразованные значения.

Инструкции:

- (a) Создайте класс `AngleConverter` с методом `__init__`, который принимает угол в градусах в качестве аргумента и сохраняет его в атрибуте `self.__degrees`.
- (b) Создайте метод `to_radians`, без аргументов, который преобразует угол в радианы по формуле:

$$\text{__degrees} \times \frac{\pi}{180}$$

- (c) Создайте метод `to_gradians` без аргументов, который преобразует угол в грады по формуле:

$$\text{__degrees} \times \frac{10}{9}$$

- (d) Напишите цикл, который повторяется 10 раз. В каждой итерации программа должна:
 - i. запрашивать у пользователя угол в градусах,
 - ii. создавать экземпляр класса `AngleConverter` с этим значением,
 - iii. вычислять угол в радианах и градах с помощью соответствующих методов,
 - iv. выводить результаты на экран.

Пример использования:

```
degrees = 90
converter = AngleConverter(degrees)
radians = converter.to_radians()
gradians = converter.to_gradians()
print(f"Угол в радианах: {radians}")
print(f"Угол в градах: {gradians}")
```

Вывод:

```
Угол в радианах: 1.5707963267948966
Угол в градах: 100.0
```

29. Напишите программу, которая создаёт класс `Tetrahedron` с методами для вычисления объёма и площади поверхности правильного тетраэдра. Программа должна запрашивать у пользователя длину ребра и выводить вычисленные объём и площадь поверхности.

Инструкции:

- (a) Создайте класс `Tetrahedron` с методом `__init__`, который принимает длину ребра тетраэдра в качестве аргумента и сохраняет её в атрибуте `self.__edge`.
- (b) Создайте метод `calculate_volume`, без аргументов, который вычисляет объём тетраэдра по формуле:

$$\frac{\text{__edge}^3}{6\sqrt{2}}$$

- (c) Создайте метод `calculate_surface_area` без аргументов, который вычисляет площадь поверхности тетраэдра по формуле:

$$\sqrt{3} \cdot \text{__edge}^2$$

- (d) Напишите цикл, который повторяется 10 раз. В каждой итерации программа должна:
- запрашивать у пользователя длину ребра тетраэдра,
 - создавать экземпляр класса `Tetrahedron` с этой длиной,
 - вычислять объём и площадь поверхности с помощью соответствующих методов,
 - выводить результаты на экран.

Пример использования:

```
edge = 3
tetrahedron = Tetrahedron(edge)
volume = tetrahedron.calculate_volume()
surface_area = tetrahedron.calculate_surface_area()
print(f"Объём тетраэдра: {volume}")
print(f"Площадь поверхности: {surface_area}")
```

Вывод:

```
Объём тетраэдра: 3.181980515339464
Площадь поверхности: 15.588457268119896
```

30. Напишите программу, которая создаёт класс `CubicMeterConverter` с методами для преобразования объёма из кубических метров в литры и кубические футы. Программа должна запрашивать у пользователя объём в кубометрах и выводить преобразованные значения.

Инструкции:

- (a) Создайте класс `CubicMeterConverter` с методом `__init__`, который принимает объём в кубических метрах в качестве аргумента и сохраняет его в атрибуте `self.__cubic_meters`.
- (b) Создайте метод `to_liters`, без аргументов, который преобразует объём в литры по формуле:

$$\text{__cubic_meters} \times 1000$$

- (c) Создайте метод `__cubic_feet` без аргументов, который преобразует объём в кубические футы по формуле:

$$\text{__cubic_meters} \times 35.3147$$

- (d) Напишите цикл, который повторяется 10 раз. В каждой итерации программа должна:

- i. запрашивать у пользователя объём в кубических метрах,
- ii. создавать экземпляр класса `CubicMeterConverter` с этим значением,
- iii. вычислять объём в литрах и кубических футах с помощью соответствующих методов,
- iv. выводить результаты на экран.

Пример использования:

```
cubic_meters = 2
converter = CubicMeterConverter(cubic_meters)
liters = converter.to_liters()
cubic_feet = converter.to_cubic_feet()
print(f"Объём в литрах: {liters}")
print(f"Объём в кубических футах: {cubic_feet}")
```

Вывод:

```
Объём в литрах: 2000.0
Объём в кубических футах: 70.6294
```

31. Напишите программу, которая создаёт класс `RightTriangle` с методами для вычисления гипотенузы и площади прямоугольного треугольника. Программа должна запрашивать у пользователя длину одного катета (второй катет фиксирован и равен 4) и выводить вычисленные гипотенузу и площадь.

Инструкции:

- (a) Создайте класс `RightTriangle` с методом `__init__`, который принимает длину первого катета в качестве аргумента и сохраняет его в атрибуте `self.__cathetus`. Второй катет фиксирован и равен 4.
- (b) Создайте метод `calculate_hypotenuse`, без аргументов, который вычисляет гипотенузу по формуле:

$$\sqrt{\text{__cathetus}^2 + 4^2}$$

- (c) Создайте метод `calculate_area` без аргументов, который вычисляет площадь треугольника по формуле:

$$\frac{\text{__cathetus} \times 4}{2}$$

- (d) Напишите цикл, который повторяется 10 раз. В каждой итерации программа должна:
 - i. запрашивать у пользователя длину катета,
 - ii. создавать экземпляр класса `RightTriangle` с этой длиной,
 - iii. вычислять гипотенузу и площадь с помощью соответствующих методов,
 - iv. выводить результаты на экран.

Пример использования:

```
cathetus = 3
triangle = RightTriangle(cathetus)
hypotenuse = triangle.calculate_hypotenuse()
area = triangle.calculate_area()
print(f"Гипотенуза: {hypotenuse}")
print(f"Площадь: {area}")
```

Вывод:

```
Гипотенуза: 5.0
Площадь: 6.0
```

2.2.2 Задача 2

1. Написать программу, которая создаёт класс `LeapYearChecker` для определения високосного года. В классе должен быть статический метод `is_leap_year` и возвращать `True`, если год високосный, и `False` в противном случае. Программа также должна использовать цикл для проверки каждого года от 2000 до 2099 и вывода результата на экран.

Инструкции:

- (a) Создайте класс `LeapYearChecker`.
- (b) Создайте **статический** метод `is_leap_year`, который принимает год в качестве аргумента и проверяет, является ли год високосным. Если год делится на 4 без остатка и не делится на 100 без остатка, или делится на 400 без остатка, то возвращает `True`. В противном случае возвращает `False`.
- (c) Используйте цикл для проверки каждого года от 2000 до 2099 (включительно), вызывая статический метод `is_leap_year` и вывода результат на экран.

Пример использования:

```
v = LeapYearChecker.is_leap_year(1999)
```

Вывод (первые и последние строки):

```
2000 True
2001 False
...
2098 False
2099 False
```

2. Написать программу, которая создаёт класс `PrimeChecker` для определения простого числа. В классе должен быть статический метод `is_prime` и возвращать `True`, если число простое, и `False` в противном случае. Программа также должна использовать цикл для проверки каждого числа от 1 до 100 и вывода результата на экран.

Инструкции:

- (a) Создайте класс `PrimeChecker`.
- (b) Создайте **статический** метод `is_prime`, который принимает число в качестве аргумента и проверяет, является ли число простым. Простое число делится только на 1 и на само себя.
- (c) Используйте цикл для проверки каждого числа от 1 до 100 (включительно), вызывая статический метод `is_prime` и выводя результат на экран.

Пример использования:

```
v = PrimeChecker.is_prime(17)
```

Вывод (первые и последние строки):

```
1 False
2 True
3 True
...
98 False
99 False
100 False
```

- 3. Написать программу, которая создаёт класс `EvenChecker` для определения чётности числа. В классе должен быть статический метод `is_even` и возвращать `True`, если число чётное, и `False` в противном случае. Программа также должна использовать цикл для проверки каждого числа от 1 до 50 и вывода результата на экран.

Инструкции:

- (a) Создайте класс `EvenChecker`.
- (b) Создайте **статический** метод `is_even`, который принимает число в качестве аргумента и проверяет, является ли число чётным.
- (c) Используйте цикл для проверки каждого числа от 1 до 50 (включительно), вызывая статический метод `is_even` и выводя результат на экран.

Пример использования:

```
v = EvenChecker.is_even(25)
```

Вывод (первые и последние строки):

```
1 False
2 True
3 False
...
48 True
49 False
50 True
```

4. Написать программу, которая создаёт класс **SquareChecker** для определения квадратного числа. В классе должен быть статический метод **is_square** и возвращать **True**, если число является квадратом целого числа, и **False** в противном случае. Программа также должна использовать цикл для проверки каждого числа от 1 до 100 и вывода результата на экран.

Инструкции:

- (a) Создайте класс **SquareChecker**.
- (b) Создайте **статический** метод **is_square**, который принимает число в качестве аргумента и проверяет, является ли число квадратом целого числа.
- (c) Используйте цикл для проверки каждого числа от 1 до 100 (включительно), вызывая статический метод **is_square** и выводя результат на экран.

Пример использования:

```
v = SquareChecker.is_square(36)
```

Вывод (первые и последние строки):

```
1 True
2 False
3 False
...
99 False
100 True
```

5. Написать программу, которая создаёт класс **FactorialCalculator** для вычисления факториала числа. В классе должен быть статический метод **factorial** и возвращать факториал числа. Программа также должна использовать цикл для вычисления факториала каждого числа от 1 до 10 и вывода результата на экран.

Инструкции:

- (a) Создайте класс **FactorialCalculator**.
- (b) Создайте **статический** метод **factorial**, который принимает число в качестве аргумента и возвращает его факториал.
- (c) Используйте цикл для вычисления факториала каждого числа от 1 до 10 (включительно), вызывая статический метод **factorial** и выводя результат на экран.

Пример использования:

```
v = FactorialCalculator.factorial(5)
```

Вывод (первые и последние строки):

```

1 1
2 2
3 6
...
9 362880
10 3628800

```

6. Написать программу, которая создаёт класс `PalindromeChecker` для определения палиндрома числа. В классе должен быть статический метод `is_palindrome` и возвращать `True`, если число является палиндромом, и `False` в противном случае. Программа также должна использовать цикл для проверки каждого числа от 100 до 200 и вывода результата на экран.

Инструкции:

- (a) Создайте класс `PalindromeChecker`.
- (b) Создайте **статический** метод `is_palindrome`, который принимает число в качестве аргумента и проверяет, является ли число палиндромом (читается одинаково слева направо и справа налево).
- (c) Используйте цикл для проверки каждого числа от 100 до 200 (включительно), вызывая статический метод `is_palindrome` и выводя результат на экран.

Пример использования:

```
v = PalindromeChecker.is_palindrome(121)
```

Вывод (первые и последние строки):

```

100 False
101 True
102 False
...
199 False
200 False

```

7. Написать программу, которая создаёт класс `ArmstrongChecker` для определения числа Армстронга. В классе должен быть статический метод `is_armstrong` и возвращать `True`, если число является числом Армстронга, и `False` в противном случае. Программа также должна использовать цикл для проверки каждого числа от 100 до 500 и вывода результата на экран.

Инструкции:

- (a) Создайте класс `ArmstrongChecker`.
- (b) Создайте **статический** метод `is_armstrong`, который принимает число в качестве аргумента и проверяет, является ли число числом Армстронга (сумма цифр в степени, равной количеству цифр, равна самому числу).
- (c) Используйте цикл для проверки каждого числа от 100 до 500 (включительно), вызывая статический метод `is_armstrong` и выводя результат на экран.

Пример использования:

```
v = ArmstrongChecker.is_armstrong(153)
```

Вывод (первые и последние строки):

```
100 False
101 False
102 False
...
499 False
500 False
```

8. Написать программу, которая создаёт класс `PerfectNumberChecker` для определения совершенного числа. В классе должен быть статический метод `is_perfect` и возвращать `True`, если число является совершенным, и `False` в противном случае. Программа также должна использовать цикл для проверки каждого числа от 1 до 1000 и вывода результата на экран.

Инструкции:

- (a) Создайте класс `PerfectNumberChecker`.
- (b) Создайте **статический** метод `is_perfect`, который принимает число в качестве аргумента и проверяет, является ли число совершенным (сумма делителей равна числу).
- (c) Используйте цикл для проверки каждого числа от 1 до 1000 (включительно), вызывая статический метод `is_perfect` и выводя результат на экран.

Пример использования:

```
v = PerfectNumberChecker.is_perfect(28)
```

Вывод (первые и последние строки):

```
1 False
2 False
3 False
...
998 False
999 False
1000 False
```

9. Написать программу, которая создаёт класс `FibonacciChecker` для проверки числа Фибоначчи. В классе должен быть статический метод `is_fibonacci` и возвращать `True`, если число является числом Фибоначчи, и `False` в противном случае. Программа также должна использовать цикл для проверки каждого числа от 1 до 100 и вывода результата на экран.

Инструкции:

- (a) Создайте класс `FibonacciChecker`.
- (b) Создайте **статический** метод `is_fibonacci`, который принимает число в качестве аргумента и проверяет, является ли число числом Фибоначчи.
- (c) Используйте цикл для проверки каждого числа от 1 до 100 (включительно), вызывая статический метод `is_fibonacci` и выводя результат на экран.

Пример использования:

```
v = FibonacciChecker.is_fibonacci(21)
```

Вывод (первые и последние строки):

```
1 True
2 True
3 True
...
98 False
99 False
100 False
```

10. Написать программу, которая создаёт класс `PowerOfTwoChecker` для проверки степени двойки. В классе должен быть статический метод `is_power_of_two` и возвращать `True`, если число является степенью двойки, и `False` в противном случае. Программа также должна использовать цикл для проверки каждого числа от 1 до 128 и вывода результата на экран.

Инструкции:

- (a) Создайте класс `PowerOfTwoChecker`.
- (b) Создайте **статический** метод `is_power_of_two`, который принимает число в качестве аргумента и проверяет, является ли число степенью двойки.
- (c) Используйте цикл для проверки каждого числа от 1 до 128 (включительно), вызывая статический метод `is_power_of_two` и выводя результат на экран.

Пример использования:

```
v = PowerOfTwoChecker.is_power_of_two(64)
```

Вывод (первые и последние строки):

```
1 True
2 True
3 False
...
127 False
128 True
```

11. Написать программу, которая создаёт класс `SumOfDigitsCalculator` для вычисления суммы цифр числа. В классе должен быть статический метод `sum_of_digits` и возвращать сумму цифр. Программа также должна использовать цикл для вычисления суммы цифр каждого числа от 1 до 50 и вывода результата на экран.

Инструкции:

- (a) Создайте класс `SumOfDigitsCalculator`.
- (b) Создайте **статический** метод `sum_of_digits`, который принимает число в качестве аргумента и возвращает сумму его цифр.
- (c) Используйте цикл для вычисления суммы цифр каждого числа от 1 до 50 (включительно), вызывая статический метод `sum_of_digits` и выводя результат на экран.

Пример использования:

```
v = SumOfDigitsCalculator.sum_of_digits(123)
```

Вывод (первые и последние строки):

```
1 1
2 2
3 3
...
49 13
50 5
```

12. Написать программу, которая создаёт класс `PrimeSumCalculator` для вычисления суммы простых чисел в диапазоне. В классе должен быть статический метод `sum_of_primes` и возвращать сумму простых чисел в заданном диапазоне. Программа также должна использовать цикл для вычисления суммы простых чисел от 1 до 100 и вывода результата на экран.

Инструкции:

- (a) Создайте класс `PrimeSumCalculator`.
- (b) Создайте **статический** метод `sum_of_primes`, который принимает два аргумента (начало и конец диапазона) и возвращает сумму простых чисел в этом диапазоне.
- (c) Используйте метод для вычисления суммы простых чисел от 1 до 100 и выведите результат.

Пример использования:

```
v = PrimeSumCalculator.sum_of_primes(1, 10)
```

Вывод:

Сумма простых чисел от 1 до 100: 1060

13. Написать программу, которая создаёт класс `DigitCountCalculator` для подсчёта количества цифр в числе. В классе должен быть статический метод `digit_count` и возвращать количество цифр. Программа также должна использовать цикл для подсчёта цифр каждого числа от 1 до 100 и вывода результата на экран.

Инструкции:

- (a) Создайте класс `DigitCountCalculator`.
- (b) Создайте **статический** метод `digit_count`, который принимает число в качестве аргумента и возвращает количество его цифр.
- (c) Используйте цикл для подсчёта цифр каждого числа от 1 до 100 (включительно), вызывая статический метод `digit_count` и выводя результат на экран.

Пример использования:

```
v = DigitCountCalculator.digit_count(12345)
```

Вывод (первые и последние строки):

```
1 1
2 1
3 1
...
99 2
100 3
```

14. Написать программу, которая создаёт класс `BinaryConverter` для преобразования числа в двоичное представление. В классе должен быть статический метод `to_binary` и возвращать строку с двоичным представлением числа. Программа также должна использовать цикл для преобразования каждого числа от 1 до 16 и вывода результата на экран.

Инструкции:

- (a) Создайте класс `BinaryConverter`.
- (b) Создайте **статический** метод `to_binary`, который принимает число в качестве аргумента и возвращает его двоичное представление в виде строки.
- (c) Используйте цикл для преобразования каждого числа от 1 до 16 (включительно), вызывая статический метод `to_binary` и выводя результат на экран.

Пример использования:

```
v = BinaryConverter.to_binary(10)
```

Вывод (первые и последние строки):

```

1 1
2 10
3 11
...
15 1111
16 10000

```

15. Написать программу, которая создаёт класс `HexConverter` для преобразования числа в шестнадцатеричное представление. В классе должен быть статический метод `to_hex` и возвращать строку с шестнадцатеричным представлением числа. Программа также должна использовать цикл для преобразования каждого числа от 1 до 20 и вывода результата на экран.

Инструкции:

- (a) Создайте класс `HexConverter`.
- (b) Создайте **статический** метод `to_hex`, который принимает число в качестве аргумента и возвращает его шестнадцатеричное представление в виде строки.
- (c) Используйте цикл для преобразования каждого числа от 1 до 20 (включительно), вызывая статический метод `to_hex` и выводя результат на экран.

Пример использования:

```
v = HexConverter.to_hex(255)
```

Вывод (первые и последние строки):

```

1 1
2 2
3 3
...
19 13
20 14

```

16. Написать программу, которая создаёт класс `DivisorChecker` для проверки делителей числа. В классе должен быть статический метод `get_divisors` и возвращать список делителей числа. Программа также должна использовать цикл для вывода делителей каждого числа от 1 до 20 и вывода результата на экран.

Инструкции:

- (a) Создайте класс `DivisorChecker`.
- (b) Создайте **статический** метод `get_divisors`, который принимает число в качестве аргумента и возвращает список его делителей.
- (c) Используйте цикл для вывода делителей каждого числа от 1 до 20 (включительно), вызывая статический метод `get_divisors` и выводя результат на экран.

Пример использования:

```
v = DivisorChecker.get_divisors(12)
```

Вывод (первые и последние строки):

```
1 [1]
2 [1, 2]
3 [1, 3]
...
19 [1, 19]
20 [1, 2, 4, 5, 10, 20]
```

17. Написать программу, которая создаёт класс `Multiplier` для создания таблицы умножения. В классе должен быть статический метод `multiply_table` и выводить таблицу умножения для заданного числа. Программа также должна использовать цикл для вывода таблицы умножения для чисел от 1 до 10 и вывода результата на экран.

Инструкции:

- (a) Создайте класс `Multiplier`.
- (b) Создайте **статический** метод `multiply_table`, который принимает число в качестве аргумента и выводит таблицу умножения для этого числа от 1 до 10.
- (c) Используйте цикл для вывода таблицы умножения для чисел от 1 до 10 (включительно), вызывая статический метод `multiply_table` и выводя результат на экран.

Пример использования:

```
Multiplier.multiply_table(5)
```

Вывод (для числа 5):

```
5 * 1 = 5
5 * 2 = 10
...
5 * 10 = 50
```

18. Написать программу, которая создаёт класс `GCDCalculator` для вычисления НОД двух чисел. В классе должен быть статический метод `gcd` и возвращать наибольший общий делитель. Программа также должна использовать цикл для вычисления НОД чисел (1,1), (2,4), (3,9), ..., (10,100) и вывода результата на экран.

Инструкции:

- (a) Создайте класс `GCDCalculator`.
- (b) Создайте **статический** метод `gcd`, который принимает два числа в качестве аргументов и возвращает их наибольший общий делитель.
- (c) Используйте цикл для вычисления НОД пар чисел (1,1), (2,4), (3,9), ..., (10,100), вызывая статический метод `gcd` и выводя результат на экран.

Пример использования:

```
v = GCDCalculator.gcd(48, 18)
```

Вывод:

```
НОД(1, 1) = 1
НОД(2, 4) = 2
НОД(3, 9) = 3
...
НОД(10, 100) = 10
```

19. Написать программу, которая создаёт класс `LCMCalculator` для вычисления НОК двух чисел. В классе должен быть статический метод `lcm` и возвращать наименьшее общее кратное. Программа также должна использовать цикл для вычисления НОК чисел (1,1), (2,3), (3,5), ..., (10,11) и вывода результата на экран.

Инструкции:

- (a) Создайте класс `LCMCalculator`.
- (b) Создайте **статический** метод `lcm`, который принимает два числа в качестве аргументов и возвращает их наименьшее общее кратное.
- (c) Используйте цикл для вычисления НОК пар чисел (1,1), (2,3), (3,5), ..., (10,11), вызывая статический метод `lcm` и выводя результат на экран.

Пример использования:

```
v = LCMCalculator.lcm(4, 6)
```

Вывод:

```
НОК(1, 1) = 1
НОК(2, 3) = 6
НОК(3, 5) = 15
...
НОК(10, 11) = 110
```

20. Написать программу, которая создаёт класс `DigitReverse` для разворота цифр числа. В классе должен быть статический метод `reverse_digits` и возвращать число с обратным порядком цифр. Программа также должна использовать цикл для разворота каждого числа от 10 до 20 и вывода результата на экран.

Инструкции:

- (a) Создайте класс `DigitReverse`.
- (b) Создайте **статический** метод `reverse_digits`, который принимает число в качестве аргумента и возвращает число с обратным порядком цифр.
- (c) Используйте цикл для разворота каждого числа от 10 до 20 (включительно), вызывая статический метод `reverse_digits` и выводя результат на экран.

Пример использования:

```
v = DigitReverse.reverse_digits(123)
```

Вывод:

```
10 1
11 11
12 21
13 31
...
19 91
20 2
```

21. Написать программу, которая создаёт класс `NumberTypeChecker` для определения типа числа (положительное/отрицательное/ноль). В классе должен быть статический метод `check_number_type` и возвращать строку с типом числа. Программа также должна использовать цикл для проверки чисел `[-5, -4, -3, -2, -1, 0, 1, 2, 3, 4, 5]` и вывода результата на экран.

Инструкции:

- (a) Создайте класс `NumberTypeChecker`.
- (b) Создайте **статический** метод `check_number_type`, который принимает число в качестве аргумента и возвращает строку "positive" "negative" или "zero".
- (c) Используйте цикл для проверки чисел `[-5, -4, -3, -2, -1, 0, 1, 2, 3, 4, 5]`, вызывая статический метод `check_number_type` и выводя результат на экран.

Пример использования:

```
v = NumberTypeChecker.check_number_type(-7)
```

Вывод:

```
-5 negative
-4 negative
-3 negative
-2 negative
-1 negative
0 zero
1 positive
2 positive
3 positive
4 positive
5 positive
```

22. Написать программу, которая создаёт класс `FactorialChecker` для проверки факториала числа. В классе должен быть статический метод `is_factorial` и возвращать `True`, если число является факториалом какого-либо числа, и `False` в противном случае. Программа также должна использовать цикл для проверки каждого числа от 1 до 120 и вывода результата на экран.

Инструкции:

- (a) Создайте класс `FactorialChecker`.
- (b) Создайте **статический** метод `is_factorial`, который принимает число в качестве аргумента и проверяет, является ли число факториалом какого-либо числа.
- (c) Используйте цикл для проверки каждого числа от 1 до 120 (включительно), вызывая статический метод `is_factorial` и выводя результат на экран.

Пример использования:

```
v = FactorialChecker.is_factorial(24)
```

Вывод (первые и последние строки):

```
1 True
2 True
3 False
...
119 False
120 True
```

23. Написать программу, которая создаёт класс `PowerChecker` для проверки степени числа. В классе должен быть статический метод `is_power` и возвращать `True`, если число является степенью заданного основания, и `False` в противном случае. Программа также должна использовать цикл для проверки каждого числа от 1 до 100 относительно основания 3 и вывода результата на экран.

Инструкции:

- (a) Создайте класс `PowerChecker`.
- (b) Создайте **статический** метод `is_power`, который принимает число и основание в качестве аргументов и проверяет, является ли число степенью основания.
- (c) Используйте цикл для проверки каждого числа от 1 до 100 (включительно) относительно основания 3, вызывая статический метод `is_power` и выводя результат на экран.

Пример использования:

```
v = PowerChecker.is_power(81, 3)
```

Вывод (первые и последние строки):

```
1 True
2 False
3 True
...
99 False
100 False
```

24. Написать программу, которая создаёт класс `DigitProductCalculator` для вычисления произведения цифр числа. В классе должен быть статический метод `digit_product` и возвращать произведение цифр. Программа также должна использовать цикл для вычисления произведения цифр каждого числа от 1 до 50 и вывода результата на экран.

Инструкции:

- (a) Создайте класс `DigitProductCalculator`.
- (b) Создайте **статический** метод `digit_product`, который принимает число в качестве аргумента и возвращает произведение его цифр.
- (c) Используйте цикл для вычисления произведения цифр каждого числа от 1 до 50 (включительно), вызывая статический метод `digit_product` и выводя результат на экран.

Пример использования:

```
v = DigitProductCalculator.digit_product(123)
```

Вывод (первые и последние строки):

```
1 1
2 2
3 3
...
49 36
50 0
```

25. Написать программу, которая создаёт класс `NumberLengthChecker` для проверки длины числа. В классе должен быть статический метод `get_length` и возвращать количество цифр в числе. Программа также должна использовать цикл для проверки длины каждого числа от 1 до 1000 с шагом 100 и вывода результата на экран.

Инструкции:

- (a) Создайте класс `NumberLengthChecker`.
- (b) Создайте **статический** метод `get_length`, который принимает число в качестве аргумента и возвращает количество его цифр.
- (c) Используйте цикл для проверки длины чисел 1, 100, 200, 300, 400, 500, 600, 700, 800, 900, 1000, вызывая статический метод `get_length` и выводя результат на экран.

Пример использования:

```
v = NumberLengthChecker.get_length(12345)
```

Вывод:

```
1 1
100 3
200 3
300 3
400 3
500 3
600 3
700 3
800 3
900 3
1000 4
```

26. Написать программу, которая создаёт класс `NumberSquareSumCalculator` для вычисления суммы квадратов чисел. В классе должен быть статический метод `square_sum` и возвращать сумму квадратов чисел в диапазоне. Программа также должна использовать метод для вычисления суммы квадратов чисел от 1 до 10 и вывода результата на экран.

Инструкции:

- (a) Создайте класс `NumberSquareSumCalculator`.
- (b) Создайте **статический** метод `square_sum`, который принимает два аргумента (начало и конец диапазона) и возвращает сумму квадратов чисел в этом диапазоне.
- (c) Используйте метод для вычисления суммы квадратов чисел от 1 до 10 и выведите результат.

Пример использования:

```
v = NumberSquareSumCalculator.square_sum(1, 3)
```

Вывод:

Сумма квадратов чисел от 1 до 10: 385

27. Написать программу, которая создаёт класс `NumberCubeSumCalculator` для вычисления суммы кубов чисел. В классе должен быть статический метод `cube_sum` и возвращать сумму кубов чисел в диапазоне. Программа также должна использовать метод для вычисления суммы кубов чисел от 1 до 10 и вывода результата на экран.

Инструкции:

- (a) Создайте класс `NumberCubeSumCalculator`.
- (b) Создайте **статический** метод `cube_sum`, который принимает два аргумента (начало и конец диапазона) и возвращает сумму кубов чисел в этом диапазоне.
- (c) Используйте метод для вычисления суммы кубов чисел от 1 до 10 и выведите результат.

Пример использования:

```
v = NumberCubeSumCalculator.cube_sum(1, 3)
```

Вывод:

Сумма кубов чисел от 1 до 10: 3025

28. Написать программу, которая создаёт класс `NumberRangeChecker` для проверки числа на принадлежность диапазону. В классе должен быть статический метод `in_range` и возвращать `True`, если число находится в заданном диапазоне, и `False` в противном случае. Программа также должна использовать цикл для проверки чисел от -5 до 5 на принадлежность диапазону `[0, 10]` и вывода результата на экран.

Инструкции:

- (a) Создайте класс `NumberRangeChecker`.
- (b) Создайте **статический** метод `in_range`, который принимает число, начало и конец диапазона и проверяет, находится ли число в этом диапазоне.
- (c) Используйте цикл для проверки чисел от -5 до 5 (включительно) на принадлежность диапазону `[0, 10]`, вызывая статический метод `in_range` и выводя результат на экран.

Пример использования:

```
v = NumberRangeChecker.in_range(5, 0, 10)
```

Вывод:

```
-5 False
-4 False
-3 False
-2 False
-1 False
0 True
1 True
2 True
3 True
4 True
5 True
```

29. Написать программу, которая создаёт класс `NumberSignChecker` для проверки знака числа. В классе должен быть статический метод `get_sign` и возвращать строку с знаком числа (+, - или 0). Программа также должна использовать цикл для проверки чисел `[-5, -4, -3, -2, -1, 0, 1, 2, 3, 4, 5]` и вывода результата на экран.

Инструкции:

- (a) Создайте класс `NumberSignChecker`.
- (b) Создайте **статический** метод `get_sign`, который принимает число в качестве аргумента и возвращает строку с его знаком (+, - или 0).
- (c) Используйте цикл для проверки чисел [-5, -4, -3, -2, -1, 0, 1, 2, 3, 4, 5], вызывая статический метод `get_sign` и выводя результат на экран.

Пример использования:

```
v = NumberSignChecker.get_sign(-7)
```

Вывод:

```
-5 -
-4 -
-3 -
-2 -
-1 -
0 0
1 +
2 +
3 +
4 +
5 +
```

30. Написать программу, которая создаёт класс `NumberPalindromeChecker` для проверки палиндрома числа. В классе должен быть статический метод `is_palindrome` и возвращать `True`, если число является палиндромом, и `False` в противном случае. Программа также должна использовать цикл для проверки каждого числа от 100 до 150 и вывода результата на экран.

Инструкции:

- (a) Создайте класс `NumberPalindromeChecker`.
- (b) Создайте **статический** метод `is_palindrome`, который принимает число в качестве аргумента и проверяет, является ли число палиндромом.
- (c) Используйте цикл для проверки каждого числа от 100 до 150 (включительно), вызывая статический метод `is_palindrome` и выводя результат на экран.

Пример использования:

```
v = NumberPalindromeChecker.is_palindrome(121)
```

Вывод (первые и последние строки):

```
100 False
101 True
102 False
...
149 False
150 False
```

31. Написать программу, которая создаёт класс `NumberAscendingChecker` для проверки, что цифры числа идут в порядке возрастания. В классе должен быть статический метод `is_ascending` и возвращать `True`, если цифры числа идут в порядке возрастания, и `False` в противном случае. Программа также должна использовать цикл для проверки каждого числа от 10 до 100 и вывода результата на экран.

Инструкции:

- (a) Создайте класс `NumberAscendingChecker`.
- (b) Создайте **статический** метод `is_ascending`, который принимает число в качестве аргумента и проверяет, идут ли его цифры в порядке возрастания.
- (c) Используйте цикл для проверки каждого числа от 10 до 100 (включительно), вызывая статический метод `is_ascending` и выводя результат на экран.

Пример использования:

```
v = NumberAscendingChecker.is_ascending(123)
```

Вывод (первые и последние строки):

```
10 False
11 False
12 True
13 True
...
98 False
99 False
100 False
```

32. Написать программу, которая создаёт класс `NumberDescendingChecker` для проверки, что цифры числа идут в порядке убывания. В классе должен быть статический метод `is_descending` и возвращать `True`, если цифры числа идут в порядке убывания, и `False` в противном случае. Программа также должна использовать цикл для проверки каждого числа от 10 до 100 и вывода результата на экран.

Инструкции:

- (a) Создайте класс `NumberDescendingChecker`.
- (b) Создайте **статический** метод `is_descending`, который принимает число в качестве аргумента и проверяет, идут ли его цифры в порядке убывания.
- (c) Используйте цикл для проверки каждого числа от 10 до 100 (включительно), вызывая статический метод `is_descending` и выводя результат на экран.

Пример использования:

```
v = NumberDescendingChecker.is_descending(321)
```

Вывод (первые и последние строки):

```
10 False
11 False
12 False
13 False
...
98 True
99 True
100 False
```

33. Написать программу, которая создаёт класс `NumberPrimeDigitChecker` для проверки, что все цифры числа простые. В классе должен быть статический метод `all_digits_prime` и возвращать `True`, если все цифры числа простые, и `False` в противном случае. Программа также должна использовать цикл для проверки каждого числа от 10 до 100 и вывода результата на экран.

Инструкции:

- (a) Создайте класс `NumberPrimeDigitChecker`.
- (b) Создайте **статический** метод `all_digits_prime`, который принимает число в качестве аргумента и проверяет, являются ли все его цифры простыми числами.
- (c) Используйте цикл для проверки каждого числа от 10 до 100 (включительно), вызывая статический метод `all_digits_prime` и выводя результат на экран.

Пример использования:

```
v = NumberPrimeDigitChecker.all_digits_prime(23)
```

Вывод (первые и последние строки):

```
10 False
11 False
12 False
13 False
...
98 False
99 False
100 False
```

34. Написать программу, которая создаёт класс `NumberEvenDigitChecker` для проверки, что все цифры числа чётные. В классе должен быть статический метод `all_digits_even` и возвращать `True`, если все цифры числа чётные, и `False` в противном случае. Программа также должна использовать цикл для проверки каждого числа от 10 до 100 и вывода результата на экран.

Инструкции:

- (a) Создайте класс `NumberEvenDigitChecker`.
- (b) Создайте **статический** метод `all_digits_even`, который принимает число в качестве аргумента и проверяет, являются ли все его цифры чётными.
- (c) Используйте цикл для проверки каждого числа от 10 до 100 (включительно), вызывая статический метод `all_digits_even` и выводя результат на экран.

Пример использования:

```
v = NumberEvenDigitChecker.all_digits_even(24)
```

Вывод (первые и последние строки):

```
10 False
11 False
12 False
13 False
...
98 False
99 False
100 False
```

35. Написать программу, которая создаёт класс `NumberOddDigitChecker` для проверки, что все цифры числа нечётные. В классе должен быть статический метод `all_digits_odd` и возвращать `True`, если все цифры числа нечётные, и `False` в противном случае. Программа также должна использовать цикл для проверки каждого числа от 10 до 100 и вывода результата на экран.

Инструкции:

- (a) Создайте класс `NumberOddDigitChecker`.
- (b) Создайте **статический** метод `all_digits_odd`, который принимает число в качестве аргумента и проверяет, являются ли все его цифры нечётными.
- (c) Используйте цикл для проверки каждого числа от 10 до 100 (включительно), вызывая статический метод `all_digits_odd` и выводя результат на экран.

Пример использования:

```
v = NumberOddDigitChecker.all_digits_odd(135)
```

Вывод (первые и последние строки):

```
10 False
11 True
12 False
13 True
```



```
...
98 False
99 True
100 False
```

2.2.3 Задача 3

1. Написать программу на Python, которая создает класс `Person` для представления сотрудника персонала. Класс должен содержать закрытые атрибуты `__name`, `__country`, `__date_of_birth` и метод `calculate_age`. Доступ к атрибутам только через методы-геттеры. Создать экземпляры и вывести информацию о каждом человеке.

Инструкции:

- (a) Создайте класс `Person` с методом `__init__`, который принимает имя, страну и дату рождения.
- (b) Создайте методы-геттеры: `get_name()`, `get_country()`, `get_date_of_birth()`.
- (c) Создайте метод `calculate_age()` для вычисления возраста.
- (d) Создайте несколько экземпляров класса `Person`.
- (e) Выведите данные каждого человека через методы класса.

Пример использования:

Листинг 1: Пример кода

```
from datetime import date

person1 = Person("Иванов Иван Иванович", "Россия", date(1946, 8, 15))
person2 = Person("Петров Сергей Александрович", "Белоруссия", date(1982, 10,
    22))

print("Персона 1:")
print("Имя: ", person1.get_name())
print("Страна: ", person1.get_country())
print("Дата рождения: ", person1.get_date_of_birth())
print("Возраст: ", person1.calculate_age())

print("Персона 2:")
print("Имя: ", person2.get_name())
print("Страна: ", person2.get_country())
print("Дата рождения: ", person2.get_date_of_birth())
print("Возраст: ", person2.calculate_age())
```

Вывод:

Листинг 2: Ожидаемый вывод

```
Персона 1:
Имя:  Иванов Иван Иванович
Страна:  Россия
Дата рождения:  1946-08-15
Возраст:  77
```

Персона 2:
Имя: Петров Сергей Александрович
Страна: Белоруссия
Дата рождения: 1982-10-22
Возраст: 41

2. Создайте класс `Student` с закрытыми атрибутами `__full_name`, `__enrollment_date`, `__major`. Реализуйте методы-геттеры и метод `study_duration()` для вычисления количества лет с момента зачисления.

Инструкции:

- (a) Создайте класс `Student` с методом `__init__`.
- (b) Методы-геттеры: `get_full_name()`, `get_enrollment_date()`, `get_major()`.
- (c) Метод `study_duration()` вычисляет количество лет с зачисления.
- (d) Создайте несколько экземпляров класса.
- (e) Выведите данные каждого студента.

Пример использования:

Листинг 3: Пример кода

```
from datetime import date

student1 = Student("Сидоров Алексей", date(2018, 9, 1), "Математика")
student2 = Student("Иванова Мария", date(2021, 9, 1), "Физика")

print("Студент 1:")
print("Имя: ", student1.get_full_name())
print("Направление: ", student1.get_major())
print("Дата зачисления: ", student1.get_enrollment_date())
print("Стаж учёбы: ", student1.study_duration())

print("Студент 2:")
print("Имя: ", student2.get_full_name())
print("Направление: ", student2.get_major())
print("Дата зачисления: ", student2.get_enrollment_date())
print("Стаж учёбы: ", student2.study_duration())
```

Вывод:

Листинг 4: Ожидаемый вывод

```
Студент 1:
Имя: Сидоров Алексей
Направление: Математика
Дата зачисления: 2018-09-01
Стаж учёбы: 5
Студент 2:
Имя: Иванова Мария
Направление: Физика
Дата зачисления: 2021-09-01
Стаж учёбы: 2
```

3. Создайте класс `Employee` с закрытыми атрибутами `__name`, `__position`, `__hire_date`. Реализуйте методы-геттеры и метод `work_experience()` для вычисления количества лет работы.

Инструкции:

- (a) Создайте класс `Employee` с методом `__init__`.
- (b) Методы-геттеры: `get_name()`, `get_position()`, `get_hire_date()`.
- (c) Метод `work_experience()` вычисляет стаж в годах.
- (d) Создайте несколько экземпляров класса.
- (e) Выведите данные каждого сотрудника.

Пример использования:

Листинг 5: Пример кода

```
from datetime import date

emp1 = Employee("Кузнецов Дмитрий", "Инженер", date(2010, 5, 10))
emp2 = Employee("Смирнова Ольга", "Менеджер", date(2015, 8, 1))

print("Сотрудник 1:")
print("Имя: ", emp1.get_name())
print("Должность: ", emp1.get_position())
print("Дата приёма: ", emp1.get_hire_date())
print("Стаж: ", emp1.work_experience())

print("Сотрудник 2:")
print("Имя: ", emp2.get_name())
print("Должность: ", emp2.get_position())
print("Дата приёма: ", emp2.get_hire_date())
print("Стаж: ", emp2.work_experience())
```

Вывод:

Листинг 6: Ожидаемый вывод

```
Сотрудник 1:
Имя: Кузнецов Дмитрий
Должность: Инженер
Дата приёма: 2010-05-10
Стаж: 17
Сотрудник 2:
Имя: Смирнова Ольга
Должность: Менеджер
Дата приёма: 2015-08-01
Стаж: 8
```

4. Создайте класс `Book` с закрытыми атрибутами `__title`, `__author`, `__publish_date`. Реализуйте геттеры и метод `book_age()` для вычисления возраста книги.

Инструкции:

- (a) Создайте класс `Book`.
- (b) Методы-геттеры: `get_title()`, `get_author()`, `get_publish_date()`.
- (c) Метод `book_age()` вычисляет возраст книги.
- (d) Создайте экземпляры класса.
- (e) Выведите данные каждой книги.

Пример использования:

Листинг 7: Пример кода

```
from datetime import date

book1 = Book("Программирование на Python", "Иванов И.И.", date(2015, 3, 10))
book2 = Book("Алгебра", "Петров П.П.", date(2000, 9, 1))

print("Книга 1:")
print("Название: ", book1.get_title())
print("Автор: ", book1.get_author())
print("Дата публикации: ", book1.get_publish_date())
print("Возраст книги: ", book1.book_age())

print("Книга 2:")
print("Название: ", book2.get_title())
print("Автор: ", book2.get_author())
print("Дата публикации: ", book2.get_publish_date())
print("Возраст книги: ", book2.book_age())
```

Вывод:

Листинг 8: Ожидаемый вывод

```
Книга 1:
Название: Программирование на Python
Автор: Иванов И.И.
Дата публикации: 2015-03-10
Возраст книги: 8
Книга 2:
Название: Алгебра
Автор: Петров П.П.
Дата публикации: 2000-09-01
Возраст книги: 23
```

5. Создайте класс `Car` с закрытыми атрибутами `__model`, `__manufacturer`, `__production_date`. Геттеры и метод `car_age()` для вычисления возраста автомобиля.

Инструкции:

- (a) Создайте класс `Car`.
- (b) Методы-геттеры: `get_model()`, `get_manufacturer()`, `get_production_date()`.
- (c) Метод `car_age()` вычисляет возраст автомобиля.
- (d) Создайте экземпляры класса.
- (e) Выведите данные каждого автомобиля.

Пример использования:

Листинг 9: Пример кода

```
from datetime import date

car1 = Car("Camry", "Toyota", date(2012, 6, 15))
car2 = Car("Focus", "Ford", date(2018, 4, 20))

print("Автомобиль 1:")
print("Модель: ", car1.get_model())
print("Производитель: ", car1.get_manufacturer())
print("Дата выпуска: ", car1.get_production_date())
print("Возраст авто: ", car1.car_age())

print("Автомобиль 2:")
print("Модель: ", car2.get_model())
print("Производитель: ", car2.get_manufacturer())
print("Дата выпуска: ", car2.get_production_date())
print("Возраст авто: ", car2.car_age())
```

Вывод:

Листинг 10: Ожидаемый вывод

```
Автомобиль 1:
Модель: Camry
Производитель: Toyota
Дата выпуска: 2012-06-15
Возраст авто: 11
Автомобиль 2:
Модель: Focus
Производитель: Ford
Дата выпуска: 2018-04-20
Возраст авто: 5
```

6. Создайте класс `Pet` с закрытыми атрибутами `__name`, `__species`, `__birth_date`. Реализуйте методы-геттеры и метод `pet_age()` для вычисления возраста питомца. Создайте несколько экземпляров и выведите их данные.

Инструкции:

- (a) Создайте класс `Pet` с методом `__init__`.
- (b) Методы-геттеры: `get_name()`, `get_species()`, `get_birth_date()`.
- (c) Метод `pet_age()` вычисляет возраст питомца в годах.
- (d) Создайте несколько экземпляров класса.
- (e) Выведите данные каждого питомца через методы класса.

Пример использования:

Листинг 11: Пример кода

```
from datetime import date

pet1 = Pet("Барсик", "Кошка", date(2018, 5, 12))
pet2 = Pet("Рекс", "Собака", date(2015, 8, 1))

print("Питомец 1:")
print("Имя: ", pet1.get_name())
print("Вид: ", pet1.get_species())
print("Дата рождения: ", pet1.get_birth_date())
print("Возраст: ", pet1.pet_age())

print("Питомец 2:")
print("Имя: ", pet2.get_name())
print("Вид: ", pet2.get_species())
print("Дата рождения: ", pet2.get_birth_date())
print("Возраст: ", pet2.pet_age())
```

Вывод:

Листинг 12: Ожидаемый вывод

```
Питомец 1:
Имя: Барсик
Вид: Кошка
Дата рождения: 2018-05-12
Возраст: 7
Питомец 2:
Имя: Рекс
Вид: Собака
Дата рождения: 2015-08-01
Возраст: 10
```

7. Создайте класс `Membership` с закрытыми атрибутами `__member_name`, `__membership_type`, `__join_date`. Реализуйте методы-геттеры и метод `membership_duration()` для вычисления длительности членства в годах.

Инструкции:

- (a) Создайте класс `Membership`.
- (b) Методы-геттеры: `get_member_name()`, `get_membership_type()`, `get_join_date()`.
- (c) Метод `membership_duration()` вычисляет длительность членства в годах.
- (d) Создайте несколько экземпляров.
- (e) Выведите данные каждого участника.

Пример использования:

Листинг 13: Пример кода

```
from datetime import date

member1 = Membership("Иванов Иван", "Золотой", date(2018, 3, 15))
member2 = Membership("Петров Петр", "Серебряный", date(2020, 6, 1))
```

```

print("Член 1:")
print("Имя: ", member1.get_member_name())
print("Тип членства: ", member1.get_membership_type())
print("Дата вступления: ", member1.get_join_date())
print("Длительность членства: ", member1.membership_duration())

print("Член 2:")
print("Имя: ", member2.get_member_name())
print("Тип членства: ", member2.get_membership_type())
print("Дата вступления: ", member2.get_join_date())
print("Длительность членства: ", member2.membership_duration())

```

Вывод:

Листинг 14: Ожидаемый вывод

```

Член 1:
Имя:  Иванов Иван
Тип членства:  Золотой
Дата вступления:  2018-03-15
Длительность членства:  5
Член 2:
Имя:  Петров Петр
Тип членства:  Серебряный
Дата вступления:  2020-06-01
Длительность членства:  3

```

8. Создайте класс `Event` с закрытыми атрибутами `__event_name`, `__location`, `__event_date`. Реализуйте методы-геттеры и метод `days_until_event()` для вычисления количества дней до события.

Инструкции:

- (a) Создайте класс `Event`.
- (b) Методы-геттеры: `get_event_name()`, `get_location()`, `get_event_date()`.
- (c) Метод `days_until_event()` вычисляет дни до события.
- (d) Создайте несколько экземпляров.
- (e) Выведите данные каждого события.

Пример использования:

Листинг 15: Пример кода

```

from datetime import date

event1 = Event("Концерт", "Стадион", date(2025, 12, 1))
event2 = Event("Выставка", "Музей", date(2025, 11, 20))

print("Событие 1:")
print("Название: ", event1.get_event_name())
print("Место: ", event1.get_location())
print("Дата: ", event1.get_event_date())

```

```

print("Дней до события: ", event1.days_until_event())

print("Событие 2:")
print("Название: ", event2.get_event_name())
print("Место: ", event2.get_location())
print("Дата: ", event2.get_event_date())
print("Дней до события: ", event2.days_until_event())

```

Вывод:

Листинг 16: Ожидаемый вывод

```

Событие 1:
Название: Концерт
Место: Стадион
Дата: 2025-12-01
Дней до события: 112
Событие 2:
Название: Выставка
Место: Музей
Дата: 2025-11-20
Дней до события: 101

```

9. Создайте класс `Course` с закрытыми атрибутами `__course_name`, `__start_date`, `__duration_weeks`. Реализуйте методы-геттеры и метод `weeks_elapsed()` для вычисления прошедших недель с начала курса.

Инструкции:

- (a) Создайте класс `Course`.
- (b) Методы-геттеры: `get_course_name()`, `get_start_date()`, `get_duration_weeks()`.
- (c) Метод `weeks_elapsed()` вычисляет количество прошедших недель.
- (d) Создайте несколько экземпляров.
- (e) Выведите данные каждого курса.

Пример использования:

Листинг 17: Пример кода

```

from datetime import date

course1 = Course("Python", date(2025, 1, 1), 12)
course2 = Course("Алгебра", date(2025, 2, 1), 10)

print("Курс 1:")
print("Название: ", course1.get_course_name())
print("Дата начала: ", course1.get_start_date())
print("Продолжительность (недель): ", course1.get_duration_weeks())
print("Прошло недель: ", course1.weeks_elapsed())

print("Курс 2:")
print("Название: ", course2.get_course_name())
print("Дата начала: ", course2.get_start_date())
print("Продолжительность (недель): ", course2.get_duration_weeks())
print("Прошло недель: ", course2.weeks_elapsed())

```


Вывод:

Листинг 18: Ожидаемый вывод

```
Курс 1:
Название: Python
Дата начала: 2025-01-01
Продолжительность (недель): 12
Прошло недель: 36
Курс 2:
Название: Алгебра
Дата начала: 2025-02-01
Продолжительность (недель): 10
Прошло недель: 31
```

10. Создайте класс `Subscription` с закрытыми атрибутами `__user`, `__plan`, `__start_date`. Реализуйте методы-геттеры и метод `subscription_age()` для вычисления возраста подписки в годах.

Инструкции:

- (a) Создайте класс `Subscription`.
- (b) Методы-геттеры: `get_user()`, `get_plan()`, `get_start_date()`.
- (c) Метод `subscription_age()` вычисляет возраст подписки.
- (d) Создайте несколько экземпляров.
- (e) Выведите данные каждой подписки.

Пример использования:

Листинг 19: Пример кода

```
from datetime import date

sub1 = Subscription("Иванов И.", "Premium", date(2021, 3, 1))
sub2 = Subscription("Петров П.", "Basic", date(2022, 7, 15))

print("Подписка 1:")
print("Пользователь: ", sub1.get_user())
print("План: ", sub1.get_plan())
print("Дата начала: ", sub1.get_start_date())
print("Возраст подписки: ", sub1.subscription_age())

print("Подписка 2:")
print("Пользователь: ", sub2.get_user())
print("План: ", sub2.get_plan())
print("Дата начала: ", sub2.get_start_date())
print("Возраст подписки: ", sub2.subscription_age())
```

Вывод:

Листинг 20: Ожидаемый вывод

```
Подписка 1:  
Пользователь:  Иванов И.  
План:    Premium  
Дата начала:  2021-03-01  
Возраст подписки:  4  
Подписка 2:  
Пользователь:  Петров П.  
План:    Basic  
Дата начала:  2022-07-15  
Возраст подписки:  3
```

11. Создайте класс `Flight` с закрытыми атрибутами `__flight_number`, `__departure_date`, `__destination`. Реализуйте методы-геттеры и метод `days_until_departure()` для вычисления количества дней до вылета.

Инструкции:

- (a) Создайте класс `Flight`.
- (b) Методы-геттеры: `get_flight_number()`, `get_departure_date()`, `get_destination()`.
- (c) Метод `days_until_departure()` вычисляет количество дней до вылета.
- (d) Создайте несколько экземпляров.
- (e) Выведите данные каждого рейса.

Пример использования:

Листинг 21: Пример кода

```
from datetime import date  
  
flight1 = Flight("SU123", date(2025, 10, 15), "Москва")  
flight2 = Flight("AF456", date(2025, 11, 1), "Париж")  
  
print("Рейс 1:")  
print("Номер: ", flight1.get_flight_number())  
print("Дата вылета: ", flight1.get_departure_date())  
print("Пункт назначения: ", flight1.get_destination())  
print("Дней до вылета: ", flight1.days_until_departure())  
  
print("Рейс 2:")  
print("Номер: ", flight2.get_flight_number())  
print("Дата вылета: ", flight2.get_departure_date())  
print("Пункт назначения: ", flight2.get_destination())  
print("Дней до вылета: ", flight2.days_until_departure())
```

Вывод:

Листинг 22: Ожидаемый вывод

```
Рейс 1:  
Номер:  SU123  
Дата вылета:  2025-10-15  
Пункт назначения:  Москва
```

Дней до вылета: 54
Рейс 2:
Номер: AF456
Дата вылета: 2025-11-01
Пункт назначения: Париж
Дней до вылета: 71

12. Создайте класс `Project` с закрытыми атрибутами `__project_name`, `__start_date`, `__deadline`. Реализуйте методы-геттеры и метод `days_remaining()` для вычисления количества дней до завершения проекта.

Инструкции:

- (a) Создайте класс `Project`.
- (b) Методы-геттеры: `get_project_name()`, `get_start_date()`, `get_deadline()`.
- (c) Метод `days_remaining()` вычисляет дни до дедлайна.
- (d) Создайте несколько экземпляров.
- (e) Выведите данные каждого проекта.

Пример использования:

Листинг 23: Пример кода

```
from datetime import date

project1 = Project("Разработка сайта", date(2025, 9, 1), date(2025, 12, 1))
project2 = Project("Анализ данных", date(2025, 10, 1), date(2025, 11, 30))

print("Проект 1:")
print("Название: ", project1.get_project_name())
print("Дата начала: ", project1.get_start_date())
print("Дедлайн: ", project1.get_deadline())
print("Дней до завершения: ", project1.days_remaining())

print("Проект 2:")
print("Название: ", project2.get_project_name())
print("Дата начала: ", project2.get_start_date())
print("Дедлайн: ", project2.get_deadline())
print("Дней до завершения: ", project2.days_remaining())
```

Вывод:

Листинг 24: Ожидаемый вывод

```
Проект 1:
Название:  Разработка сайта
Дата начала:  2025-09-01
Дедлайн:  2025-12-01
Дней до завершения:  101
Проект 2:
Название:  Анализ данных
Дата начала:  2025-10-01
Дедлайн:  2025-11-30
Дней до завершения:  91
```

13. Создайте класс `Doctor` с закрытыми атрибутами `__full_name`, `__specialty`, `__birth_date`. Реализуйте методы-геттеры и метод `calculate_age()` для вычисления возраста врача.

Инструкции:

- (a) Создайте класс `Doctor`.
- (b) Методы-геттеры: `get_full_name()`, `get_specialty()`, `get_birth_date()`.
- (c) Метод `calculate_age()` вычисляет возраст.
- (d) Создайте несколько экземпляров.
- (e) Выведите данные каждого врача.

Пример использования:

Листинг 25: Пример кода

```
from datetime import date

doc1 = Doctor("Иванов И.И.", "Терапевт", date(1980, 5, 12))
doc2 = Doctor("Петров П.П.", "Хирург", date(1975, 8, 1))

print("Врач 1:")
print("Имя: ", doc1.get_full_name())
print("Специальность: ", doc1.get_specialty())
print("Дата рождения: ", doc1.get_birth_date())
print("Возраст: ", doc1.calculate_age())

print("Врач 2:")
print("Имя: ", doc2.get_full_name())
print("Специальность: ", doc2.get_specialty())
print("Дата рождения: ", doc2.get_birth_date())
print("Возраст: ", doc2.calculate_age())
```

Вывод:

Листинг 26: Ожидаемый вывод

```
Врач 1:
Имя:  Иванов И.И.
Специальность:  Терапевт
Дата рождения:  1980-05-12
Возраст:  45
Врач 2:
Имя:  Петров П.П.
Специальность:  Хирург
Дата рождения:  1975-08-01
Возраст:  50
```

14. Создайте класс `Patient` с закрытыми атрибутами `__full_name`, `__admission_date`, `__diagnosis`. Реализуйте методы-геттеры и метод `hospital_stay()` для вычисления количества дней пребывания в больнице.

Инструкции:

- (a) Создайте класс `Patient`.
- (b) Методы-геттеры: `get_full_name()`, `get_admission_date()`, `get_diagnosis()`.
- (c) Метод `hospital_stay()` вычисляет дни пребывания.
- (d) Создайте несколько экземпляров.
- (e) Выведите данные каждого пациента.

Пример использования:

Листинг 27: Пример кода

```
from datetime import date

patient1 = Patient("Сидоров С.С.", date(2025, 9, 1), "ОРВИ")
patient2 = Patient("Кузнецов К.К.", date(2025, 8, 28), "Грипп")

print("Пациент 1:")
print("Имя: ", patient1.get_full_name())
print("Дата госпитализации: ", patient1.get_admission_date())
print("Диагноз: ", patient1.get_diagnosis())
print("Дней в больнице: ", patient1.hospital_stay())

print("Пациент 2:")
print("Имя: ", patient2.get_full_name())
print("Дата госпитализации: ", patient2.get_admission_date())
print("Диагноз: ", patient2.get_diagnosis())
print("Дней в больнице: ", patient2.hospital_stay())
```

Вывод:

Листинг 28: Ожидаемый вывод

```
Пациент 1:
Имя: Сидоров С.С.
Дата госпитализации: 2025-09-01
Диагноз: ОРВИ
Дней в больнице: 15
Пациент 2:
Имя: Кузнецов К.К.
Дата госпитализации: 2025-08-28
Диагноз: Грипп
Дней в больнице: 19
```

15. Создайте класс `Concert` с закрытыми атрибутами `__artist`, `__venue`, `__concert_date`. Реализуйте методы-геттеры и метод `days_until_concert()`.

Инструкции:

- (a) Создайте класс `Concert`.
- (b) Методы-геттеры: `get_artist()`, `get_venue()`, `get_concert_date()`.
- (c) Метод `days_until_concert()` вычисляет дни до концерта.
- (d) Создайте несколько экземпляров.
- (e) Выведите данные каждого концерта.

Пример использования:

Листинг 29: Пример кода

```
from datetime import date

concert1 = Concert("Imagine Dragons", "Лужники", date(2025, 10, 10))
concert2 = Concert("Coldplay", "O2 Arena", date(2025, 11, 5))

print("Концерт 1:")
print("Исполнитель: ", concert1.get_artist())
print("Место: ", concert1.get_venue())
print("Дата: ", concert1.get_concert_date())
print("Дней до концерта: ", concert1.days_until_concert())

print("Концерт 2:")
print("Исполнитель: ", concert2.get_artist())
print("Место: ", concert2.get_venue())
print("Дата: ", concert2.get_concert_date())
print("Дней до концерта: ", concert2.days_until_concert())
```

Вывод:

Листинг 30: Ожидаемый вывод

```
Концерт 1:
Исполнитель:  Imagine Dragons
Место:  Лужники
Дата:  2025-10-10
Дней до концерта:  49
Концерт 2:
Исполнитель:  Coldplay
Место:  O2 Arena
Дата:  2025-11-05
Дней до концерта:  75
```

16. Создайте класс `Holiday` с закрытыми атрибутами `__name`, `__country`, `__holiday_date`. Реализуйте методы-геттеры и метод `days_until_holiday()`.

Инструкции:

- (a) Создайте класс `Holiday`.
- (b) Методы-геттеры: `get_name()`, `get_country()`, `get_holiday_date()`.
- (c) Метод `days_until_holiday()` вычисляет дни до праздника.
- (d) Создайте несколько экземпляров.
- (e) Выведите данные каждого праздника.

Пример использования:

Листинг 31: Пример кода

```
from datetime import date
```

```

holiday1 = Holiday("Новый Год", "Россия", date(2026, 1, 1))
holiday2 = Holiday("Рождество", "Германия", date(2025, 12, 25))

print("Праздник 1:")
print("Название: ", holiday1.get_name())
print("Страна: ", holiday1.get_country())
print("Дата: ", holiday1.get_holiday_date())
print("Дней до праздника: ", holiday1.days_until_holiday())

print("Праздник 2:")
print("Название: ", holiday2.get_name())
print("Страна: ", holiday2.get_country())
print("Дата: ", holiday2.get_holiday_date())
print("Дней до праздника: ", holiday2.days_until_holiday())

```

Вывод:

Листинг 32: Ожидаемый вывод

```

Праздник 1:
Название:  Новый Год
Страна:   Россия
Дата:     2026-01-01
Дней до праздника:  83
Праздник 2:
Название:  Рождество
Страна:   Германия
Дата:     2025-12-25
Дней до праздника:  67

```

17. Создайте класс `Employee` с закрытыми атрибутами `__full_name`, `__position`, `__hire_date`. Реализуйте методы-геттеры и метод `years_worked()` для вычисления стажа работы в годах.

Инструкции:

- (a) Создайте класс `Employee`.
- (b) Методы-геттеры: `get_full_name()`, `get_position()`, `get_hire_date()`.
- (c) Метод `years_worked()` вычисляет стаж в годах.
- (d) Создайте несколько экземпляров.
- (e) Выведите данные каждого сотрудника.

Пример использования:

Листинг 33: Пример кода

```

from datetime import date

emp1 = Employee("Иванов И.И.", "Менеджер", date(2015, 4, 1))
emp2 = Employee("Петров П.П.", "Разработчик", date(2018, 7, 15))

print("Сотрудник 1:")
print("Имя: ", emp1.get_full_name())

```

```

print("Должность: ", emp1.get_position())
print("Дата приема: ", emp1.get_hire_date())
print("Стаж: ", emp1.years_worked())

print("Сотрудник 2:")
print("Имя: ", emp2.get_full_name())
print("Должность: ", emp2.get_position())
print("Дата приема: ", emp2.get_hire_date())
print("Стаж: ", emp2.years_worked())

```

Вывод:

Листинг 34: Ожидаемый вывод

```

Сотрудник 1:
Имя:  Иванов И.И.
Должность:  Менеджер
Дата приема:  2015-04-01
Стаж:  10
Сотрудник 2:
Имя:  Петров П.П.
Должность:  Разработчик
Дата приема:  2018-07-15
Стаж:  7

```

18. Создайте класс `LibraryBook` с закрытыми атрибутами `__title`, `__author`, `__publication_date`. Реализуйте методы-геттеры и метод `book_age()` для вычисления возраста книги.

Инструкции:

- (a) Создайте класс `LibraryBook`.
- (b) Методы-геттеры: `get_title()`, `get_author()`, `get_publication_date()`.
- (c) Метод `book_age()` вычисляет возраст книги в годах.
- (d) Создайте несколько экземпляров.
- (e) Выведите данные каждой книги.

Пример использования:

Листинг 35: Пример кода

```

from datetime import date

book1 = LibraryBook("Война и мир", "Толстой", date(1869, 1, 1))
book2 = LibraryBook("Мастер и Маргарита", "Булгаков", date(1967, 5, 1))

print("Книга 1:")
print("Название: ", book1.get_title())
print("Автор: ", book1.get_author())
print("Дата публикации: ", book1.get_publication_date())
print("Возраст книги: ", book1.book_age())

print("Книга 2:")
print("Название: ", book2.get_title())

```



```

print("Автор: ", book2.get_author())
print("Дата публикации: ", book2.get_publication_date())
print("Возраст книги: ", book2.book_age())

```

Вывод:

Листинг 36: Ожидаемый вывод

```

Книга 1:
Название:  Война и мир
Автор:    Толстой
Дата публикации:  1869-01-01
Возраст книги:  156
Книга 2:
Название:  Мастер и Маргарита
Автор:    Булгаков
Дата публикации:  1967-05-01
Возраст книги:  59

```

19. Создайте класс `Vehicle` с закрытыми атрибутами `__brand`, `__model`, `__manufacture_date`. Реализуйте методы-геттеры и метод `vehicle_age()`.

Инструкции:

- (a) Создайте класс `Vehicle`.
- (b) Методы-геттеры: `get_brand()`, `get_model()`, `get_manufacture_date()`.
- (c) Метод `vehicle_age()` вычисляет возраст транспортного средства.
- (d) Создайте несколько экземпляров.
- (e) Выведите данные каждого транспортного средства.

Пример использования:

Листинг 37: Пример кода

```

from datetime import date

vehicle1 = Vehicle("Toyota", "Camry", date(2015, 5, 1))
vehicle2 = Vehicle("BMW", "X5", date(2018, 3, 10))

print("Транспорт 1:")
print("Марка: ", vehicle1.get_brand())
print("Модель: ", vehicle1.get_model())
print("Дата производства: ", vehicle1.get_manufacture_date())
print("Возраст: ", vehicle1.vehicle_age())

print("Транспорт 2:")
print("Марка: ", vehicle2.get_brand())
print("Модель: ", vehicle2.get_model())
print("Дата производства: ", vehicle2.get_manufacture_date())
print("Возраст: ", vehicle2.vehicle_age())

```

Вывод:

Листинг 38: Ожидаемый вывод

```
Транспорт 1:
Марка: Toyota
Модель: Camry
Дата производства: 2015-05-01
Возраст: 10
Транспорт 2:
Марка: BMW
Модель: X5
Дата производства: 2018-03-10
Возраст: 7
```

20. Создайте класс `Student` с закрытыми атрибутами `__full_name`, `__enrollment_date`, `__major`. Реализуйте методы-геттеры и метод `study_years()`.

Инструкции:

- (a) Создайте класс `Student`.
- (b) Методы-геттеры: `get_full_name()`, `get_enrollment_date()`, `get_major()`.
- (c) Метод `study_years()` вычисляет количество лет учебы.
- (d) Создайте несколько экземпляров.
- (e) Выведите данные каждого студента.

Пример использования:

Листинг 39: Пример кода

```
from datetime import date

student1 = Student("Иванов И.И.", date(2020, 9, 1), "Математика")
student2 = Student("Петров П.П.", date(2021, 9, 1), "Физика")

print("Студент 1:")
print("Имя: ", student1.get_full_name())
print("Дата зачисления: ", student1.get_enrollment_date())
print("Специальность: ", student1.get_major())
print("Лет учебы: ", student1.study_years())

print("Студент 2:")
print("Имя: ", student2.get_full_name())
print("Дата зачисления: ", student2.get_enrollment_date())
print("Специальность: ", student2.get_major())
print("Лет учебы: ", student2.study_years())
```

Вывод:

Листинг 40: Ожидаемый вывод

```
Студент 1:
Имя: Иванов И.И.
```

```
Дата зачисления: 2020-09-01
Специальность: Математика
Лет учебы: 5
Студент 2:
Имя: Петров П.П.
Дата зачисления: 2021-09-01
Специальность: Физика
Лет учебы: 4
```

21. Создайте класс `Ticket` с закрытыми атрибутами `__ticket_number`, `__issue_date`, `__valid_until`. Реализуйте методы-геттеры и метод `days_valid()`.

Инструкции:

- (a) Создайте класс `Ticket`.
- (b) Методы-геттеры: `get_ticket_number()`, `get_issue_date()`, `get_valid_until()`.
- (c) Метод `days_valid()` вычисляет дни до окончания действия билета.
- (d) Создайте несколько экземпляров.
- (e) Выведите данные каждого билета.

Пример использования:

Листинг 41: Пример кода

```
from datetime import date

ticket1 = Ticket("A123", date(2025, 9, 1), date(2025, 12, 1))
ticket2 = Ticket("B456", date(2025, 10, 1), date(2026, 1, 1))

print("Билет 1:")
print("Номер: ", ticket1.get_ticket_number())
print("Дата выдачи: ", ticket1.get_issue_date())
print("Действителен до: ", ticket1.get_valid_until())
print("Дней до окончания: ", ticket1.days_valid())

print("Билет 2:")
print("Номер: ", ticket2.get_ticket_number())
print("Дата выдачи: ", ticket2.get_issue_date())
print("Действителен до: ", ticket2.get_valid_until())
print("Дней до окончания: ", ticket2.days_valid())
```

Вывод:

Листинг 42: Ожидаемый вывод

```
Билет 1:
Номер: A123
Дата выдачи: 2025-09-01
Действителен до: 2025-12-01
Дней до окончания: 91
Билет 2:
Номер: B456
Дата выдачи: 2025-10-01
Действителен до: 2026-01-01
Дней до окончания: 92
```

22. Создайте класс `Appointment` с закрытыми атрибутами `__client`, `__service`, `__appointment_date`. Реализуйте методы-геттеры и метод `days_until_appointment()`.

Инструкции:

- (a) Создайте класс `Appointment`.
- (b) Методы-геттеры: `get_client()`, `get_service()`, `get_appointment_date()`.
- (c) Метод `days_until_appointment()` вычисляет дни до приёма.
- (d) Создайте несколько экземпляров.
- (e) Выведите данные каждого приёма.

Пример использования:

Листинг 43: Пример кода

```
from datetime import date

app1 = Appointment("Иванов И.", "Массаж", date(2025, 10, 5))
app2 = Appointment("Петров П.", "Стрижка", date(2025, 10, 15))

print("Приём 1:")
print("Клиент: ", app1.get_client())
print("Услуга: ", app1.get_service())
print("Дата: ", app1.get_appointment_date())
print("Дней до приёма: ", app1.days_until_appointment())

print("Приём 2:")
print("Клиент: ", app2.get_client())
print("Услуга: ", app2.get_service())
print("Дата: ", app2.get_appointment_date())
print("Дней до приёма: ", app2.days_until_appointment())
```

Вывод:

Листинг 44: Ожидаемый вывод

```
Приём 1:
Клиент:  Иванов И.
Услуга:  Массаж
Дата:    2025-10-05
Дней до приёма:  44
Приём 2:
Клиент:  Петров П.
Услуга:  Стрижка
Дата:    2025-10-15
Дней до приёма:  54
```

23. Создайте класс `Subscription` с закрытыми атрибутами `__subscriber`, `__start_date`, `__end_date`. Реализуйте методы-геттеры и метод `days_remaining()`.

Инструкции:

- (a) Создайте класс `Subscription`.
- (b) Методы-геттеры: `get_subscriber()`, `get_start_date()`, `get_end_date()`.
- (c) Метод `days_remaining()` вычисляет дни до окончания подписки.
- (d) Создайте несколько экземпляров.
- (e) Выведите данные каждой подписки.

Пример использования:

Листинг 45: Пример кода

```
from datetime import date

sub1 = Subscription("Иванов И.", date(2025, 1, 1), date(2025, 12, 31))
sub2 = Subscription("Петров П.", date(2025, 6, 1), date(2026, 5, 31))

print("Подписка 1:")
print("Абонент: ", sub1.get_subscriber())
print("Дата начала: ", sub1.get_start_date())
print("Дата окончания: ", sub1.get_end_date())
print("Дней до окончания: ", sub1.days_remaining())

print("Подписка 2:")
print("Абонент: ", sub2.get_subscriber())
print("Дата начала: ", sub2.get_start_date())
print("Дата окончания: ", sub2.get_end_date())
print("Дней до окончания: ", sub2.days_remaining())
```

Вывод:

Листинг 46: Ожидаемый вывод

```
Подписка 1:
Абонент:  Иванов И.
Дата начала:  2025-01-01
Дата окончания:  2025-12-31
Дней до окончания:  113
Подписка 2:
Абонент:  Петров П.
Дата начала:  2025-06-01
Дата окончания:  2026-05-31
Дней до окончания:  245
```

24. Создайте класс `MembershipCard` с закрытыми атрибутами `__owner`, `__issue_date`, `__expiry_date`. Реализуйте методы-геттеры и метод `days_until_expiry()`.

Инструкции:

- (a) Создайте класс `MembershipCard`.
- (b) Методы-геттеры: `get_owner()`, `get_issue_date()`, `get_expiry_date()`.
- (c) Метод `days_until_expiry()` вычисляет дни до истечения действия карты.
- (d) Создайте несколько экземпляров.
- (e) Выведите данные каждой карты.

Пример использования:

Листинг 47: Пример кода

```
from datetime import date

card1 = MembershipCard("Иванов И.", date(2025, 1, 1), date(2026, 1, 1))
card2 = MembershipCard("Петров П.", date(2025, 5, 1), date(2026, 5, 1))

print("Карта 1:")
print("Владелец: ", card1.get_owner())
print("Дата выдачи: ", card1.get_issue_date())
print("Срок действия: ", card1.get_expiry_date())
print("Дней до окончания: ", card1.days_until_expiry())

print("Карта 2:")
print("Владелец: ", card2.get_owner())
print("Дата выдачи: ", card2.get_issue_date())
print("Срок действия: ", card2.get_expiry_date())
print("Дней до окончания: ", card2.days_until_expiry())
```

Вывод:

Листинг 48: Ожидаемый вывод

```
Карта 1:
Владелец:  Иванов И.
Дата выдачи:  2025-01-01
Срок действия:  2026-01-01
Дней до окончания:  113
Карта 2:
Владелец:  Петров П.
Дата выдачи:  2025-05-01
Срок действия:  2026-05-01
Дней до окончания:  204
```

25. Создайте класс `Event` с закрытыми атрибутами `__title`, `__location`, `__event_date`. Реализуйте методы-геттеры и метод `days_until_event()`.

Инструкции:

- (a) Создайте класс `Event`.
- (b) Методы-геттеры: `get_title()`, `get_location()`, `get_event_date()`.
- (c) Метод `days_until_event()` вычисляет дни до события.
- (d) Создайте несколько экземпляров.
- (e) Выведите данные каждого события.

Пример использования:

Листинг 49: Пример кода

```
from datetime import date
```

```

event1 = Event("Фестиваль науки", "Москва", date(2025, 10, 20))
event2 = Event("Конференция IT", "Санкт-Петербург", date(2025, 11, 10))

print("Событие 1:")
print("Название: ", event1.get_title())
print("Место: ", event1.get_location())
print("Дата: ", event1.get_event_date())
print("Дней до события: ", event1.days_until_event())

print("Событие 2:")
print("Название: ", event2.get_title())
print("Место: ", event2.get_location())
print("Дата: ", event2.get_event_date())
print("Дней до события: ", event2.days_until_event())

```

Вывод:

Листинг 50: Ожидаемый вывод

```

Событие 1:
Название: Фестиваль науки
Место: Москва
Дата: 2025-10-20
Дней до события: 59
Событие 2:
Название: Конференция IT
Место: Санкт-Петербург
Дата: 2025-11-10
Дней до события: 80

```

26. Создайте класс `CarRental` с закрытыми атрибутами `__client`, `__rental_date`, `__return_date`. Реализуйте методы-геттеры и метод `rental_duration()`.

Инструкции:

- (a) Создайте класс `CarRental`.
- (b) Методы-геттеры: `get_client()`, `get_rental_date()`, `get_return_date()`.
- (c) Метод `rental_duration()` вычисляет длительность аренды в днях.
- (d) Создайте несколько экземпляров.
- (e) Выведите данные каждой аренды.

Пример использования:

Листинг 51: Пример кода

```

from datetime import date

rental1 = CarRental("Иванов И.", date(2025, 10, 1), date(2025, 10, 10))
rental2 = CarRental("Петров П.", date(2025, 11, 1), date(2025, 11, 5))

print("Аренда 1:")
print("Клиент: ", rental1.get_client())
print("Дата аренды: ", rental1.get_rental_date())

```

```

print("Дата возврата: ", rental1.get_return_date())
print("Длительность аренды: ", rental1.rental_duration())

print("Аренда 2:")
print("Клиент: ", rental2.get_client())
print("Дата аренды: ", rental2.get_rental_date())
print("Дата возврата: ", rental2.get_return_date())
print("Длительность аренды: ", rental2.rental_duration())

```

Вывод:

Листинг 52: Ожидаемый вывод

```

Аренда 1:
Клиент:  Иванов И.
Дата аренды:  2025-10-01
Дата возврата:  2025-10-10
Длительность аренды:  9
Аренда 2:
Клиент:  Петров П.
Дата аренды:  2025-11-01
Дата возврата:  2025-11-05
Длительность аренды:  4

```

27. Создайте класс `Visa` с закрытыми атрибутами `__holder`, `__issue_date`, `__expiry_date`. Реализуйте методы-геттеры и метод `days_until_expiry()`.

Инструкции:

- Создайте класс `Visa`.
- Методы-геттеры: `get_holder()`, `get_issue_date()`, `get_expiry_date()`.
- Метод `days_until_expiry()` вычисляет дни до окончания визы.
- Создайте несколько экземпляров.
- Выведите данные каждой визы.

Пример использования:

Листинг 53: Пример кода

```

from datetime import date

visa1 = Visa("Иванов И.", date(2025, 1, 1), date(2026, 1, 1))
visa2 = Visa("Петров П.", date(2025, 6, 1), date(2026, 6, 1))

print("Виза 1:")
print("Держатель: ", visa1.get_holder())
print("Дата выдачи: ", visa1.get_issue_date())
print("Дата окончания: ", visa1.get_expiry_date())
print("Дней до окончания: ", visa1.days_until_expiry())

print("Виза 2:")
print("Держатель: ", visa2.get_holder())
print("Дата выдачи: ", visa2.get_issue_date())
print("Дата окончания: ", visa2.get_expiry_date())
print("Дней до окончания: ", visa2.days_until_expiry())

```


Вывод:

Листинг 54: Ожидаемый вывод

```
Виза 1:
Держатель:  Иванов И.
Дата выдачи:  2025-01-01
Дата окончания:  2026-01-01
Дней до окончания:  113
Виза 2:
Держатель:  Петров П.
Дата выдачи:  2025-06-01
Дата окончания:  2026-06-01
Дней до окончания:  204
```

28. Создайте класс `Reservation` с закрытыми атрибутами `__guest`, `__checkin_date`, `__checkout_date`. Реализуйте методы-геттеры и метод `stay_duration()`.

Инструкции:

- (a) Создайте класс `Reservation`.
- (b) Методы-геттеры: `get_guest()`, `get_checkin_date()`, `get_checkout_date()`.
- (c) Метод `stay_duration()` вычисляет продолжительность пребывания в днях.
- (d) Создайте несколько экземпляров.
- (e) Выведите данные каждой брони.

Пример использования:

Листинг 55: Пример кода

```
from datetime import date

res1 = Reservation("Иванов И.", date(2025, 10, 1), date(2025, 10, 7))
res2 = Reservation("Петров П.", date(2025, 11, 5), date(2025, 11, 12))

print("Бронь 1:")
print("Гость: ", res1.get_guest())
print("Дата заезда: ", res1.get_checkin_date())
print("Дата выезда: ", res1.get_checkout_date())
print("Продолжительность пребывания: ", res1.stay_duration())

print("Бронь 2:")
print("Гость: ", res2.get_guest())
print("Дата заезда: ", res2.get_checkin_date())
print("Дата выезда: ", res2.get_checkout_date())
print("Продолжительность пребывания: ", res2.stay_duration())
```

Вывод:

Листинг 56: Ожидаемый вывод

```
Бронь 1:
Гость:  Иванов И.
```

```
Дата заезда: 2025-10-01
Дата выезда: 2025-10-07
Продолжительность пребывания: 6
Бронь 2:
Гость: Петров П.
Дата заезда: 2025-11-05
Дата выезда: 2025-11-12
Продолжительность пребывания: 7
```

29. Создайте класс `Conference` с закрытыми атрибутами `__name`, `__city`, `__start_date`. Реализуйте методы-геттеры и метод `days_until_start()`.

Инструкции:

- (a) Создайте класс `Conference`.
- (b) Методы-геттеры: `get_name()`, `get_city()`, `get_start_date()`.
- (c) Метод `days_until_start()` вычисляет дни до начала конференции.
- (d) Создайте несколько экземпляров.
- (e) Выведите данные каждой конференции.

Пример использования:

Листинг 57: Пример кода

```
from datetime import date

conf1 = Conference("PythonConf", "Москва", date(2025, 10, 20))
conf2 = Conference("DataScience Summit", "Санкт-Петербург", date(2025, 11,
    15))

print("Конференция 1:")
print("Название: ", conf1.get_name())
print("Город: ", conf1.get_city())
print("Дата начала: ", conf1.get_start_date())
print("Дней до начала: ", conf1.days_until_start())

print("Конференция 2:")
print("Название: ", conf2.get_name())
print("Город: ", conf2.get_city())
print("Дата начала: ", conf2.get_start_date())
print("Дней до начала: ", conf2.days_until_start())
```

Вывод:

Листинг 58: Ожидаемый вывод

```
Конференция 1:
Название: PythonConf
Город: Москва
Дата начала: 2025-10-20
Дней до начала: 59
Конференция 2:
Название: DataScience Summit
```

Город: Санкт-Петербург
Дата начала: 2025-11-15
Дней до начала: 85

30. Создайте класс `Medication` с закрытыми атрибутами `__name`, `__manufacturer`, `__expiry_date`. Реализуйте методы-геттеры и метод `days_until_expiry()`.

Инструкции:

- (a) Создайте класс `Medication`.
- (b) Методы-геттеры: `get_name()`, `get_manufacturer()`, `get_expiry_date()`.
- (c) Метод `days_until_expiry()` вычисляет дни до окончания срока годности.
- (d) Создайте несколько экземпляров.
- (e) Выведите данные каждого лекарства.

Пример использования:

Листинг 59: Пример кода

```
from datetime import date

med1 = Medication("Парацетамол", "Фармком", date(2026, 1, 1))
med2 = Medication("Ибупрофен", "БиоФарм", date(2025, 12, 1))

print("Лекарство 1:")
print("Название: ", med1.get_name())
print("Производитель: ", med1.get_manufacturer())
print("Срок годности: ", med1.get_expiry_date())
print("Дней до окончания: ", med1.days_until_expiry())

print("Лекарство 2:")
print("Название: ", med2.get_name())
print("Производитель: ", med2.get_manufacturer())
print("Срок годности: ", med2.get_expiry_date())
print("Дней до окончания: ", med2.days_until_expiry())
```

Вывод:

Листинг 60: Ожидаемый вывод

```
Лекарство 1:
Название: Парацетамол
Производитель: Фармком
Срок годности: 2026-01-01
Дней до окончания: 113
Лекарство 2:
Название: Ибупрофен
Производитель: БиоФарм
Срок годности: 2025-12-01
Дней до окончания: 92
```

31. Создайте класс `Project` с закрытыми атрибутами `__title`, `__start_date`, `__deadline`. Реализуйте методы-геттеры и метод `days_until_deadline()`.

Инструкции:

- (a) Создайте класс `Project`.
- (b) Методы-геттеры: `get_title()`, `get_start_date()`, `get_deadline()`.
- (c) Метод `days_until_deadline()` вычисляет дни до дедлайна.
- (d) Создайте несколько экземпляров.
- (e) Выведите данные каждого проекта.

Пример использования:

Листинг 61: Пример кода

```
from datetime import date

proj1 = Project("Разработка сайта", date(2025, 9, 1), date(2025, 12, 1))
proj2 = Project("Мобильное приложение", date(2025, 10, 1), date(2026, 1, 15))

print("Проект 1:")
print("Название: ", proj1.get_title())
print("Дата начала: ", proj1.get_start_date())
print("Дедлайн: ", proj1.get_deadline())
print("Дней до дедлайна: ", proj1.days_until_deadline())

print("Проект 2:")
print("Название: ", proj2.get_title())
print("Дата начала: ", proj2.get_start_date())
print("Дедлайн: ", proj2.get_deadline())
print("Дней до дедлайна: ", proj2.days_until_deadline())
```

Вывод:

Листинг 62: Ожидаемый вывод

```
Проект 1:
Название:  Разработка сайта
Дата начала:  2025-09-01
Дедлайн:  2025-12-01
Дней до дедлайна:  91
Проект 2:
Название:  Мобильное приложение
Дата начала:  2025-10-01
Дедлайн:  2026-01-15
Дней до дедлайна:  106
```

2.2.4 Задача 4

1. Написать программу на Python, которая создает абстрактный класс `Shape` для представления геометрической фигуры. Класс должен содержать абстрактные методы `calculate_area` и `calculate_perimeter`, которые вычисляют площадь и периметр фигуры соответственно. Программа также должна создавать дочерние классы `Circle`, `Rectangle` и `Triangle`, которые наследуют от класса `Shape` и реализуют специфические для каждого класса методы вычисления площади и периметра.

Инструкции:

- (a) Создайте абстрактный класс `Shape` (с использованием модуля `abc`) с абстрактными методами `calculate_area()` и `calculate_perimeter()`.
- (b) Создайте класс `Circle` с конструктором `__init__(self, radius)`, который принимает радиус окружности в качестве аргумента и сохраняет его в приватном атрибуте `_radius`. Добавьте `@property`-getter `radius` для получения значения радиуса. Реализуйте методы `calculate_area()` и `calculate_perimeter()` для вычисления площади и периметра окружности.
- (c) Создайте класс `Rectangle` с конструктором `__init__(self, length, width)`, который принимает длину и ширину прямоугольника в качестве аргументов и сохраняет их в приватных атрибутах `_length` и `_width`. Добавьте `@property`-getter `length` и `width` для получения значений атрибутов. Реализуйте методы `calculate_area()` и `calculate_perimeter()` для вычисления площади и периметра прямоугольника.
- (d) Создайте класс `Triangle` с конструктором `__init__(self, base, height, side1, side2, side3)`, который принимает основание, высоту и три стороны треугольника в качестве аргументов и сохраняет их в приватных атрибутах `_base`, `_height`, `_side1`, `_side2` и `_side3`. Добавьте `@property`-getter `base`, `height`, `side1`, `side2`, `side3`. Реализуйте методы `calculate_area()` и `calculate_perimeter()` для вычисления площади и периметра треугольника.
- (e) Создайте экземпляр каждого класса и вызовите методы `calculate_area()` и `calculate_perimeter()` для вычисления площади и периметра фигуры. Выведите результаты на экран, используя геттеры для доступа к атрибутам.

Пример использования:

```
# Вычисление параметров окружности.
r = 7
circle = Circle(r)
print("Радиус окружности:", circle.radius)
print("Площадь окружности:", circle.calculate_area())
print("Периметр окружности:", circle.calculate_perimeter())
```

Примечание: В этом примере используется библиотека `math` для вычисления числа π и квадратного корня.

Вывод:

```
Радиус окружности: 7
Площадь окружности: 153.93804002589985
Периметр окружности: 43.982297150257104
```

Далее вывод для прямоугольника и треугольника.

2. Написать программу на Python, которая создает абстрактный класс `ElectricalComponent` (с использованием модуля `abc`) для представления электрических элементов. Класс должен содержать абстрактные методы `calculate_power()` и `calculate_energy()`. Программа также должна создавать дочерние классы `Resistor`, `Capacitor` и `Inductor`, которые наследуют от класса `ElectricalComponent` и реализуют специфические для каждого класса методы вычисления мощности и энергии.

Подсказка по формулам:

- Resistor: $P = U^2/R$, $E = P \cdot t$
- Capacitor: $P = V \cdot I$, $E = 0.5 \cdot C \cdot V^2$
- Inductor: $P = L \cdot I^2$, $E = 0.5 \cdot L \cdot I^2$

Инструкции:

- Создайте абстрактный класс `ElectricalComponent` с методами `calculate_power()` и `calculate_energy()`, используя модуль `abc`.
- Создайте класс `Resistor` с конструктором `__init__(self, voltage, resistance, time)`, который сохраняет приватные атрибуты `__voltage`, `__resistance`, `__time`. Добавьте `@property`-геттеры для всех атрибутов. Реализуйте методы вычисления мощности и энергии.
- Создайте класс `Capacitor` с конструктором `__init__(self, voltage, current, capacitance)`, приватными атрибутами `__voltage`, `__current`, `__capacitance` и геттерами. Реализуйте методы.
- Создайте класс `Inductor` с конструктором `__init__(self, inductance, current)`, приватными атрибутами `__inductance`, `__current` и геттерами. Реализуйте методы.
- Создайте экземпляр каждого класса и вызовите методы `calculate_power()` и `calculate_energy()`, используя геттеры для доступа к атрибутам. Выведите результаты на экран.

Пример использования:

```
r = Resistor(10, 5, 10)
print("Сопротивление резистора:", r.resistance)
print("Мощность резистора:", r.calculate_power())
print("Энергия резистора:", r.calculate_energy())
```

Вывод:

```
Сопротивление резистора: 5
Мощность резистора: 20
Энергия резистора: 200
```

Далее вывод для конденсатора и катушки индуктивности.

- Написать программу на Python, которая создает абстрактный класс `MotionObject` (с использованием модуля `abc`) для представления движущихся тел. Класс должен содержать абстрактные методы `calculate_kinetic_energy()` и `calculate_momentum()`. Программа также должна создавать дочерние классы `LinearBody`, `RotatingBody` и `FallingBody`, которые наследуют от класса `MotionObject` и реализуют специфические для каждого класса методы вычисления кинетической энергии и импульса.

Подсказка по формулам:

- `LinearBody`: $KE = 0.5 \cdot m \cdot v^2$, $p = m \cdot v$
- `RotatingBody`: $KE = 0.5 \cdot I \cdot \omega^2$, $p = I \cdot \omega$
- `FallingBody`: $KE = m \cdot g \cdot h$, $p = m \cdot v$

Инструкции:

- (a) Создайте абстрактный класс `MotionObject` с методами `calculate_kinetic_energy()` и `calculate_momentum()`, используя модуль `abc`.
- (b) Создайте класс `LinearBody` с конструктором `__init__(self, mass, velocity)`, приватными атрибутами `__mass`, `__velocity` и геттерами. Реализуйте методы.
- (c) Создайте класс `RotatingBody` с конструктором `__init__(self, moment_of_inertia, angular_velocity)`, приватными атрибутами `__moment_of_inertia`, `__angular_velocity` и геттерами. Реализуйте методы.
- (d) Создайте класс `FallingBody` с конструктором `__init__(self, mass, height, velocity)`, приватными атрибутами `__mass`, `__height`, `__velocity` и геттерами. Реализуйте методы.
- (e) Создайте экземпляр каждого класса и вызовите методы `calculate_kinetic_energy()` и `calculate_momentum()`, используя геттеры для доступа к атрибутам. Выведите результаты на экран.

Пример использования:

```
body = LinearBody(2, 3)
print("Масса тела:", body.mass)
print("Кинетическая энергия:", body.calculate_kinetic_energy())
print("Импульс:", body.calculate_momentum())
```

Вывод:

Масса тела: 2
Кинетическая энергия: 6
Импульс: 6

Далее вывод для вращающегося тела и падающего тела.

4. Написать программу на Python, которая создает абстрактный класс `Investment` (с использованием модуля `abc`) для финансовых вложений. Класс должен содержать абстрактные методы `calculate_simple_interest()` и `calculate_total_value()`. Программа также должна создавать дочерние классы `ShortTerm`, `LongTerm` и `CompoundInvestment`, которые наследуют от класса `Investment` и реализуют специфические методы вычисления процентов и итоговой суммы.

Подсказка по формулам:

- `ShortTerm`: $SI = P \cdot R \cdot T / 100$, $Total = P + SI$
- `LongTerm`: $SI = P \cdot R \cdot T / 100 + 50$, $Total = P + SI$
- `CompoundInvestment`: $Total = P \cdot (1 + R/100)^T$, $SI = Total - P$

Инструкции:

- (a) Создайте абстрактный класс `Investment` с методами `calculate_simple_interest()` и `calculate_total_value()`, используя модуль `abc`.
- (b) Создайте класс `ShortTerm` с конструктором `__init__(self, principal, rate, time)`, приватными атрибутами `__principal`, `__rate`, `__time` и геттерами. Реализуйте методы.

- (c) Создайте класс `LongTerm` с конструктором `__init__(self, principal, rate, time)`, приватными атрибутами `__principal`, `__rate`, `__time` и геттерами. Реализуйте методы.
- (d) Создайте класс `CompoundInvestment` с конструктором `__init__(self, principal, rate, time)`, приватными атрибутами `__principal`, `__rate`, `__time` и геттерами. Реализуйте методы.
- (e) Создайте экземпляр каждого класса и вызовите методы `calculate_simple_interest()` и `calculate_total_value()`, используя геттеры для доступа к атрибутам. Выведите результаты на экран.

Пример использования:

```
inv = ShortTerm(1000, 5, 2)
print("Начальная сумма:", inv.principal)
print("Простой процент:", inv.calculate_simple_interest())
print("Итоговая сумма:", inv.calculate_total_value())
```

Вывод:

```
Начальная сумма: 1000
Простой процент: 100
Итоговая сумма: 1100
```

Далее вывод для долгосрочного и сложного вложения.

5. Написать программу на Python, которая создает абстрактный класс `Solid` (с использованием модуля `abc`) для твердого тела. Класс должен содержать абстрактные методы `calculate_volume()` и `calculate_surface_area()`. Программа также должна создавать дочерние классы `Cube`, `RectangularPrism` и `Cylinder`, которые наследуют от класса `Solid` и реализуют специфические методы вычисления объема и площади поверхности.

Подсказка по формулам:

- `Cube`: $V = a^3$, $S = 6 \cdot a^2$
- `RectangularPrism`: $V = l \cdot w \cdot h$, $S = 2(lw + lh + wh)$
- `Cylinder`: $V = \pi r^2 h$, $S = 2\pi r(r + h)$

Инструкции:

- (a) Создайте абстрактный класс `Solid` с методами `calculate_volume()` и `calculate_surface_area()`, используя модуль `abc`.
- (b) Создайте класс `Cube` с конструктором `__init__(self, side)`, приватным атрибутом `__side` и геттером. Реализуйте методы.
- (c) Создайте класс `RectangularPrism` с конструктором `__init__(self, length, width, height)`, приватными атрибутами `__length`, `__width`, `__height` и геттерами. Реализуйте методы.
- (d) Создайте класс `Cylinder` с конструктором `__init__(self, radius, height)`, приватными атрибутами `__radius`, `__height` и геттерами. Реализуйте методы.

- (e) Создайте экземпляр каждого класса и вызовите методы `calculate_volume()` и `calculate_surface_area()`, используя геттеры. Выведите результаты на экран.

Пример использования:

```
cube = Cube(3)
print("Сторона куба:", cube.side)
print("Объем куба:", cube.calculate_volume())
print("Площадь поверхности куба:", cube.calculate_surface_area())
```

Вывод:

```
Сторона куба: 3
Объем куба: 27
Площадь поверхности куба: 54
```

Далее вывод для прямоугольного параллелепипеда и цилиндра.

6. Написать программу на Python, которая создает абстрактный класс `ChemicalSubstance` (с использованием модуля `abc`) для химических веществ. Класс должен содержать абстрактные методы `calculate_molar_mass()` и `calculate_density()`. Программа также должна создавать дочерние классы `Element`, `Compound` и `Mixture`, которые наследуют от класса `ChemicalSubstance` и реализуют специфические методы вычисления молярной массы и плотности.

Подсказка по формулам:

- `Element`: $M = atomic_mass$, $\rho = mass/volume$
- `Compound`: $M = \sum(fraction \cdot atomic_mass)$, $\rho = mass/volume$
- `Mixture`: $M = \sum(fraction \cdot molar_mass)$, $\rho = \sum(fraction \cdot density)$

Инструкции:

- (a) Создайте абстрактный класс `ChemicalSubstance` с методами `calculate_molar_mass()` и `calculate_density()`, используя модуль `abc`.
- (b) Создайте класс `Element` с конструктором `__init__(self, atomic_mass, mass, volume)`, приватными атрибутами и геттерами. Реализуйте методы.
- (c) Создайте класс `Compound` с конструктором `__init__(self, fractions, atomic_masses, mass, volume)`, приватными атрибутами и геттерами. Реализуйте методы.
- (d) Создайте класс `Mixture` с конструктором `__init__(self, fractions, molar_masses, densities)`, приватными атрибутами и геттерами. Реализуйте методы.
- (e) Создайте экземпляр каждого класса и вызовите методы, используя геттеры, и выведите результаты.

Пример использования:

```
el = Element(12, 24, 2)
print("Атомная масса элемента:", el.atomic_mass)
print("Молярная масса:", el.calculate_molar_mass())
print("Плотность:", el.calculate_density())
```

Вывод:

Атомная масса элемента: 12
Молярная масса: 12
Плотность: 12

Далее вывод для соединения и смеси.

7. Написать программу на Python, которая создает абстрактный класс `BankAccount` (с использованием модуля `abc`) для банковских счетов. Класс должен содержать абстрактные методы `calculate_interest()` и `calculate_balance()`. Программа также должна создавать дочерние классы `Savings`, `Checking` и `FixedDeposit`, которые наследуют от класса `BankAccount` и реализуют специфические методы вычисления процентов и баланса.

Подсказка по формулам:

- `Savings`: $Interest = balance \cdot rate \cdot time / 100$, $Balance = balance + Interest$
- `Checking`: $Interest = balance \cdot rate \cdot time / 100 - fee$, $Balance = balance + Interest$
- `FixedDeposit`: $Balance = principal \cdot (1 + rate/100)^{time}$, $Interest = Balance - principal$

Инструкции:

- Создайте абстрактный класс `BankAccount` с методами `calculate_interest()` и `calculate_balance()`, используя модуль `abc`.
- Создайте класс `Savings` с конструктором `__init__(self, balance, rate, time)`, приватными атрибутами и геттерами. Реализуйте методы.
- Создайте класс `Checking` с конструктором `__init__(self, balance, rate, time, fee)`, приватными атрибутами и геттерами. Реализуйте методы.
- Создайте класс `FixedDeposit` с конструктором `__init__(self, principal, rate, time)`, приватными атрибутами и геттерами. Реализуйте методы.
- Создайте экземпляр каждого класса и вызовите методы, используя геттеры, и выведите результаты.

Пример использования:

```
s = Savings(1000, 5, 2)
print("Баланс на сберегательном счете:", s.balance)
print("Проценты:", s.calculate_interest())
print("Итоговый баланс:", s.calculate_balance())
```

Вывод:

Баланс на сберегательном счете: 1000
Проценты: 100
Итоговый баланс: 1100

Далее вывод для расчетного счета и срочного депозита.

8. Написать программу на Python, которая создает абстрактный класс `Shape3D` (с использованием модуля `abc`) для трехмерных фигур. Класс должен содержать абстрактные методы `calculate_volume()` и `calculate_surface_area()`. Программа также должна создавать дочерние классы `Sphere`, `Cone` и `Pyramid`, которые наследуют от класса `Shape3D` и реализуют специфические методы вычисления объема и площади поверхности.

Подсказка по формулам:

- Sphere: $V = 4/3 \cdot \pi r^3$, $S = 4 \cdot \pi r^2$
- Cone: $V = 1/3 \cdot \pi r^2 h$, $S = \pi r(r + \sqrt{r^2 + h^2})$
- Pyramid: $V = 1/3 \cdot base_area \cdot height$, $S = base_area + lateral_area$

Инструкции:

- Создайте абстрактный класс `Shape3D` с методами `calculate_volume()` и `calculate_surface_area()`, используя модуль `abc`.
- Создайте класс `Sphere` с конструктором `__init__(self, radius)`, приватным атрибутом и геттером. Реализуйте методы.
- Создайте класс `Cone` с конструктором `__init__(self, radius, height)`, приватными атрибутами и геттерами. Реализуйте методы.
- Создайте класс `Pyramid` с конструктором `__init__(self, base_area, lateral_area, height)`, приватными атрибутами и геттерами. Реализуйте методы.
- Создайте экземпляр каждого класса и вызовите методы, используя геттеры, и выведите результаты.

Пример использования:

```
s = Sphere(3)
print("Радиус сферы:", s.radius)
print("Объем сферы:", s.calculate_volume())
print("Площадь поверхности сферы:", s.calculate_surface_area())
```

Вывод:

```
Радиус сферы: 3
Объем сферы: 113.097
Площадь поверхности сферы: 113.097
```

Далее вывод для конуса и пирамиды.

9. Написать программу на Python, которая создает абстрактный класс `Vehicle` (с использованием модуля `abc`) для транспортных средств. Класс должен содержать абстрактные методы `calculate_fuel_consumption()` и `calculate_range()`. Программа также должна создавать дочерние классы `Car`, `Truck` и `Motorcycle`, которые наследуют от класса `Vehicle` и реализуют специфические методы вычисления расхода топлива и запаса хода.

Подсказка по формулам:

- Car: $fuel = distance/efficiency$, $range = tank_capacity \cdot efficiency$

- Truck: $fuel = (distance/efficiency) \cdot load_factor$, $range = tank_capacity \cdot efficiency/load_factor$
- Motorcycle: $fuel = distance/efficiency \cdot 0.8$, $range = tank_capacity \cdot efficiency \cdot 1.2$

Инструкции:

- Создайте абстрактный класс `Vehicle` с методами `calculate_fuel_consumption()` и `calculate_range()`, используя модуль `abc`.
- Создайте класс `Car` с конструктором `__init__(self, efficiency, distance, tank_capacity)`, приватными атрибутами и геттерами. Реализуйте методы.
- Создайте класс `Truck` с конструктором `__init__(self, efficiency, distance, tank_capacity, load_factor)`, приватными атрибутами и геттерами. Реализуйте методы.
- Создайте класс `Motorcycle` с конструктором `__init__(self, efficiency, distance, tank_capacity)`, приватными атрибутами и геттерами. Реализуйте методы.
- Создайте экземпляр каждого класса и вызовите методы, используя геттеры, и выведите результаты.

Пример использования:

```
car = Car(15, 150, 50)
print("Эффективность автомобиля:", car.efficiency)
print("Расход топлива:", car.calculate_fuel_consumption())
print("Запас хода:", car.calculate_range())
```

Вывод:

```
Эффективность автомобиля: 15
Расход топлива: 10
Запас хода: 750
```

Далее вывод для грузовика и мотоцикла.

10. Написать программу на Python, которая создает абстрактный класс `Plant` (с использованием модуля `abc`) для растений. Класс должен содержать абстрактные методы `calculate_growth()` и `calculate_water_needs()`. Программа также должна создавать дочерние классы `Tree`, `Flower` и `Shrub`, которые наследуют от класса `Plant` и реализуют специфические методы вычисления роста и потребности в воде.

Подсказка по формулам:

- Tree: $growth = height_rate \cdot time$, $water = area \cdot water_rate$
- Flower: $growth = height_rate \cdot time \cdot 0.5$, $water = area \cdot water_rate \cdot 0.3$
- Shrub: $growth = height_rate \cdot time \cdot 0.8$, $water = area \cdot water_rate \cdot 0.6$

Инструкции:

- Создайте абстрактный класс `Plant` с методами `calculate_growth()` и `calculate_water_needs()`, используя модуль `abc`.

- (b) Создайте класс `Tree` с конструктором `__init__(self, height_rate, time, area, water_rate)`, приватными атрибутами и геттерами. Реализуйте методы.
- (c) Создайте класс `Flower` с конструктором `__init__(self, height_rate, time, area, water_rate)`, приватными атрибутами и геттерами. Реализуйте методы.
- (d) Создайте класс `Shrub` с конструктором `__init__(self, height_rate, time, area, water_rate)`, приватными атрибутами и геттерами. Реализуйте методы.
- (e) Создайте экземпляр каждого класса и вызовите методы, используя геттеры, и выведите результаты.

Пример использования:

```
tree = Tree(2, 5, 10, 3)
print("Скорость роста дерева:", tree.height_rate)
print("Рост:", tree.calculate_growth())
print("Потребность в воде:", tree.calculate_water_needs())
```

Вывод:

```
Скорость роста дерева: 2
Рост: 10
Потребность в воде: 30
```

Далее вывод для цветка и кустарника.

11. Написать программу на Python, которая создает абстрактный класс `Sensor` (с использованием модуля `abc`) для измерительных датчиков. Класс должен содержать абстрактные методы `calculate_signal()` и `calculate_accuracy()`. Программа также должна создавать дочерние классы `TemperatureSensor`, `PressureSensor` и `LightSensor`, которые наследуют от класса `Sensor` и реализуют специфические методы вычисления сигнала и точности.

Подсказка по формулам:

- `TemperatureSensor`: $signal = voltage \cdot sensitivity$, $accuracy = tolerance$
- `PressureSensor`: $signal = pressure \cdot sensitivity$, $accuracy = tolerance \cdot 1.1$
- `LightSensor`: $signal = intensity \cdot sensitivity$, $accuracy = tolerance \cdot 0.9$

Инструкции:

- (a) Создайте абстрактный класс `Sensor` с методами `calculate_signal()` и `calculate_accuracy()`, используя модуль `abc`.
- (b) Создайте класс `TemperatureSensor` с конструктором `__init__(self, voltage, sensitivity, tolerance)`, приватными атрибутами и геттерами. Реализуйте методы.
- (c) Создайте класс `PressureSensor` с конструктором `__init__(self, pressure, sensitivity, tolerance)`, приватными атрибутами и геттерами. Реализуйте методы.
- (d) Создайте класс `LightSensor` с конструктором `__init__(self, intensity, sensitivity, tolerance)`, приватными атрибутами и геттерами. Реализуйте методы.

- (e) Создайте экземпляр каждого класса и вызовите методы, используя геттеры, и выведите результаты.

Пример использования:

```
temp_sensor = TemperatureSensor(5, 2, 0.1)
print("Напряжение:", temp_sensor.voltage)
print("Сигнал:", temp_sensor.calculate_signal())
print("Точность:", temp_sensor.calculate_accuracy())
```

Вывод:

Напряжение: 5
Сигнал: 10
Точность: 0.1

Далее вывод для датчиков давления и света.

12. Написать программу на Python, которая создает абстрактный класс `CookingIngredient` (с использованием модуля `abc`) для ингредиентов. Класс должен содержать абстрактные методы `calculate_calories()` и `calculate_mass()`. Программа также должна создавать дочерние классы `Vegetable`, `Meat` и `Grain`, которые наследуют от класса `CookingIngredient` и реализуют специфические методы вычисления калорий и массы.

Подсказка по формулам:

- `Vegetable`: $calories = weight \cdot cal_per_100g / 100$, $mass = weight$
- `Meat`: $calories = weight \cdot cal_per_100g / 100 \cdot 1.2$, $mass = weight$
- `Grain`: $calories = weight \cdot cal_per_100g / 100 \cdot 1.1$, $mass = weight$

Инструкции:

- Создайте абстрактный класс `CookingIngredient` с методами `calculate_calories()` и `calculate_mass()`, используя модуль `abc`.
- Создайте класс `Vegetable` с конструктором `__init__(self, weight, cal_per_100g)`, приватными атрибутами и геттерами. Реализуйте методы.
- Создайте класс `Meat` с конструктором `__init__(self, weight, cal_per_100g)`, приватными атрибутами и геттерами. Реализуйте методы.
- Создайте класс `Grain` с конструктором `__init__(self, weight, cal_per_100g)`, приватными атрибутами и геттерами. Реализуйте методы.
- Создайте экземпляр каждого класса и вызовите методы, используя геттеры, и выведите результаты.

Пример использования:

```
veg = Vegetable(200, 30)
print("Вес овоща:", veg.weight)
print("Калории:", veg.calculate_calories())
print("Масса:", veg.calculate_mass())
```

Вывод:

Вес овоща: 200
Калории: 60
Масса: 200

Далее вывод для мяса и зерна.

13. Написать программу на Python, которая создает абстрактный класс `ElectronicDevice` (с использованием модуля `abc`) для электронных устройств. Класс должен содержать абстрактные методы `calculate_power()` и `calculate_efficiency()`. Программа также должна создавать дочерние классы `Laptop`, `Smartphone` и `Tablet`, которые наследуют от класса `ElectronicDevice` и реализуют специфические методы вычисления мощности и эффективности.

Подсказка по формулам:

- Laptop: $power = voltage \cdot current$, $efficiency = useful_power / power$
- Smartphone: $power = voltage \cdot current \cdot 0.8$, $efficiency = useful_power / power$
- Tablet: $power = voltage \cdot current \cdot 0.9$, $efficiency = useful_power / power$

Инструкции:

- Создайте абстрактный класс `ElectronicDevice` с методами `calculate_power()` и `calculate_efficiency()`, используя модуль `abc`.
- Создайте класс `Laptop` с конструктором `__init__(self, voltage, current, useful_power)`, приватными атрибутами и геттерами. Реализуйте методы.
- Создайте класс `Smartphone` с конструктором `__init__(self, voltage, current, useful_power)`, приватными атрибутами и геттерами. Реализуйте методы.
- Создайте класс `Tablet` с конструктором `__init__(self, voltage, current, useful_power)`, приватными атрибутами и геттерами. Реализуйте методы.
- Создайте экземпляр каждого класса и вызовите методы, используя геттеры, и выведите результаты.

Пример использования:

```
laptop = Laptop(19, 3, 50)
print("Напряжение ноутбука:", laptop.voltage)
print("Мощность:", laptop.calculate_power())
print("Эффективность:", laptop.calculate_efficiency())
```

Вывод:

Напряжение ноутбука: 19
Мощность: 57
Эффективность: 0.877

Далее вывод для смартфона и планшета.

14. Написать программу на Python, которая создает абстрактный класс `MusicalInstrument` (с использованием модуля `abc`) для музыкальных инструментов. Класс должен содержать абстрактные методы `calculate_sound_level()` и `calculate_frequency()`. Программа также должна создавать дочерние классы `Piano`, `Guitar` и `Flute`, которые наследуют от класса `MusicalInstrument` и реализуют специфические методы вычисления уровня звука и частоты.

Подсказка по формулам:

- Piano: $sound_level = keys \cdot intensity$, $frequency = 440 \cdot 2^{(note-49)/12}$
- Guitar: $sound_level = strings \cdot intensity \cdot 0.8$, $frequency = 440 \cdot 2^{(note-49)/12}$
- Flute: $sound_level = holes \cdot intensity \cdot 0.9$, $frequency = 440 \cdot 2^{(note-49)/12}$

Инструкции:

- Создайте абстрактный класс `MusicalInstrument` с методами `calculate_sound_level()` и `calculate_frequency()`, используя модуль `abc`.
- Создайте класс `Piano` с конструктором `__init__(self, keys, intensity, note)`, приватными атрибутами и геттерами. Реализуйте методы.
- Создайте класс `Guitar` с конструктором `__init__(self, strings, intensity, note)`, приватными атрибутами и геттерами. Реализуйте методы.
- Создайте класс `Flute` с конструктором `__init__(self, holes, intensity, note)`, приватными атрибутами и геттерами. Реализуйте методы.
- Создайте экземпляр каждого класса и вызовите методы, используя геттеры, и выведите результаты.

Пример использования:

```
piano = Piano(88, 5, 49)
print("Клавиши:", piano.keys)
print("Уровень звука:", piano.calculate_sound_level())
print("Частота:", piano.calculate_frequency())
```

Вывод:

```
Клавиши: 88
Уровень звука: 440
Частота: 440
```

Далее вывод для гитары и флейты.

15. Написать программу на Python, которая создает абстрактный класс `Workout` (с использованием модуля `abc`) для физических упражнений. Класс должен содержать абстрактные методы `calculate_calories_burned()` и `calculate_duration()`. Программа также должна создавать дочерние классы `Cardio`, `Strength` и `Flexibility`, которые наследуют от класса `Workout` и реализуют специфические методы вычисления сожженных калорий и длительности тренировки.

Подсказка по формулам:

- Cardio: $calories = weight \cdot time \cdot 0.1$, $duration = time$

- **Strength:** $calories = weight \cdot time \cdot 0.08$, $duration = time$
- **Flexibility:** $calories = weight \cdot time \cdot 0.05$, $duration = time$

Инструкции:

- Создайте абстрактный класс `Workout` с методами `calculate_calories_burned()` и `calculate_duration()`, используя модуль `abc`.
- Создайте класс `Cardio` с конструктором `__init__(self, weight, time)`, приватными атрибутами и геттерами. Реализуйте методы.
- Создайте класс `Strength` с конструктором `__init__(self, weight, time)`, приватными атрибутами и геттерами. Реализуйте методы.
- Создайте класс `Flexibility` с конструктором `__init__(self, weight, time)`, приватными атрибутами и геттерами. Реализуйте методы.
- Создайте экземпляр каждого класса и вызовите методы, используя геттеры, и выведите результаты.

Пример использования:

```
cardio = Cardio(70, 30)
print("Вес:", cardio.weight)
print("Сожженные калории:", cardio.calculate_calories_burned())
print("Длительность:", cardio.calculate_duration())
```

Вывод:

```
Вес: 70
Сожженные калории: 210
Длительность: 30
```

Далее вывод для силовой и растяжки.

- Написать программу на Python, которая создает абстрактный класс `ComputerComponent` (с использованием модуля `abc`) для компонентов компьютера. Класс должен содержать абстрактные методы `calculate_power_consumption()` и `calculate_cost()`. Программа также должна создавать дочерние классы `CPU`, `GPU` и `RAM`, которые наследуют от класса `ComputerComponent` и реализуют специфические методы вычисления энергопотребления и стоимости.

Подсказка по формулам:

- CPU: $power = cores \cdot frequency \cdot 10$, $cost = cores \cdot 50$
- GPU: $power = cores \cdot frequency \cdot 12$, $cost = cores \cdot 80$
- RAM: $power = size \cdot 3$, $cost = size \cdot 20$

Инструкции:

- Создайте абстрактный класс `ComputerComponent` с методами `calculate_power_consumption()` и `calculate_cost()`, используя модуль `abc`.
- Создайте класс `CPU` с конструктором `__init__(self, cores, frequency)`, приватными атрибутами и геттерами. Реализуйте методы.

- (c) Создайте класс `GPU` с конструктором `__init__(self, cores, frequency)`, приватными атрибутами и геттерами. Реализуйте методы.
- (d) Создайте класс `RAM` с конструктором `__init__(self, size)`, приватным атрибутом и геттером. Реализуйте методы.
- (e) Создайте экземпляр каждого класса и вызовите методы, используя геттеры, и выведите результаты.

Пример использования:

```
cpu = CPU(4, 3.5)
print("Ядра CPU:", cpu.cores)
print("Энергопотребление:", cpu.calculate_power_consumption())
print("Стоимость:", cpu.calculate_cost())
```

Вывод:

```
Ядра CPU: 4
Энергопотребление: 140
Стоимость: 200
```

Далее вывод для `GPU` и `RAM`.

17. Написать программу на Python, которая создает абстрактный класс `Building` (с использованием модуля `abc`) для зданий. Класс должен содержать абстрактные методы `calculate_volume()` и `calculate_floor_area()`. Программа также должна создавать дочерние классы `House`, `Office` и `Warehouse`, которые наследуют от класса `Building` и реализуют специфические методы вычисления объема и площади.

Подсказка по формулам:

- `House`: $volume = length \cdot width \cdot height$, $floor_area = length \cdot width$
- `Office`: $volume = length \cdot width \cdot height \cdot 1.2$, $floor_area = length \cdot width \cdot 1.1$
- `Warehouse`: $volume = length \cdot width \cdot height \cdot 1.5$, $floor_area = length \cdot width \cdot 1.3$

Инструкции:

- (a) Создайте абстрактный класс `Building` с методами `calculate_volume()` и `calculate_floor_area()`, используя модуль `abc`.
- (b) Создайте класс `House` с конструктором `__init__(self, length, width, height)`, приватными атрибутами и геттерами. Реализуйте методы.
- (c) Создайте класс `Office` с конструктором `__init__(self, length, width, height)`, приватными атрибутами и геттерами. Реализуйте методы.
- (d) Создайте класс `Warehouse` с конструктором `__init__(self, length, width, height)`, приватными атрибутами и геттерами. Реализуйте методы.
- (e) Создайте экземпляр каждого класса и вызовите методы, используя геттеры, и выведите результаты.

Пример использования:

```

house = House(10, 8, 3)
print("Длина дома:", house.length)
print("Объем:", house.calculate_volume())
print("Площадь пола:", house.calculate_floor_area())

```

Вывод:

```

Длина дома: 10
Объем: 240
Площадь пола: 80

```

Далее вывод для офиса и склада.

18. Написать программу на Python, которая создает абстрактный класс `Vehicle` (с использованием модуля `abc`) для транспортных средств. Класс должен содержать абстрактные методы `calculate_max_speed()` и `calculate_range()`. Программа также должна создавать дочерние классы `Car`, `Motorcycle` и `Bicycle`, которые наследуют от класса `Vehicle` и реализуют специфические методы вычисления максимальной скорости и дальности.

Подсказка по формулам:

- `Car`: $max_speed = engine_power \cdot 2$, $range = fuel_capacity \cdot 10$
- `Motorcycle`: $max_speed = engine_power \cdot 2.5$, $range = fuel_capacity \cdot 8$
- `Bicycle`: $max_speed = pedaling_power \cdot 3$, $range = stamina \cdot 5$

Инструкции:

- (a) Создайте абстрактный класс `Vehicle` с методами `calculate_max_speed()` и `calculate_range()`, используя модуль `abc`.
- (b) Создайте класс `Car` с конструктором `__init__(self, engine_power, fuel_capacity)`, приватными атрибутами и геттерами. Реализуйте методы.
- (c) Создайте класс `Motorcycle` с конструктором `__init__(self, engine_power, fuel_capacity)`, приватными атрибутами и геттерами. Реализуйте методы.
- (d) Создайте класс `Bicycle` с конструктором `__init__(self, pedaling_power, stamina)`, приватными атрибутами и геттерами. Реализуйте методы.
- (e) Создайте экземпляр каждого класса и вызовите методы, используя геттеры, и выведите результаты.

Пример использования:

```

car = Car(150, 50)
print("Мощность двигателя автомобиля:", car.engine_power)
print("Максимальная скорость:", car.calculate_max_speed())
print("Дальность:", car.calculate_range())

```

Вывод:

```

Мощность двигателя автомобиля: 150
Максимальная скорость: 300
Дальность: 500

```

Далее вывод для мотоцикла и велосипеда.

19. Написать программу на Python, которая создает абстрактный класс `BankAccount` (с использованием модуля `abc`) для банковских счетов. Класс должен содержать абстрактные методы `calculate_interest()` и `calculate_fees()`. Программа также должна создавать дочерние классы `SavingsAccount`, `CheckingAccount` и `InvestmentAccount`, которые наследуют от класса `BankAccount` и реализуют специфические методы вычисления процентов и комиссий.

Подсказка по формулам:

- `SavingsAccount`: $interest = balance \cdot 0.03$, $fees = 5$
- `CheckingAccount`: $interest = balance \cdot 0.01$, $fees = 2$
- `InvestmentAccount`: $interest = balance \cdot 0.05$, $fees = 10$

Инструкции:

- Создайте абстрактный класс `BankAccount` с методами `calculate_interest()` и `calculate_fees()`, используя модуль `abc`.
- Создайте класс `SavingsAccount` с конструктором `__init__(self, balance)`, приватными атрибутами и геттерами. Реализуйте методы.
- Создайте класс `CheckingAccount` с конструктором `__init__(self, balance)`, приватными атрибутами и геттерами. Реализуйте методы.
- Создайте класс `InvestmentAccount` с конструктором `__init__(self, balance)`, приватными атрибутами и геттерами. Реализуйте методы.
- Создайте экземпляр каждого класса и вызовите методы, используя геттеры, и выведите результаты.

Пример использования:

```
savings = SavingsAccount(1000)
print("Баланс сберегательного счета:", savings.balance)
print("Проценты:", savings.calculate_interest())
print("Комиссии:", savings.calculate_fees())
```

Вывод:

```
Баланс сберегательного счета: 1000
Проценты: 30.0
Комиссии: 5
```

Далее вывод для расчетного и инвестиционного счета.

20. Написать программу на Python, которая создает абстрактный класс `Appliance` (с использованием модуля `abc`) для бытовой техники. Класс должен содержать абстрактные методы `calculate_energy_usage()` и `calculate_operating_cost()`. Программа также должна создавать дочерние классы `Refrigerator`, `WashingMachine` и `Microwave`, которые наследуют от класса `Appliance` и реализуют специфические методы вычисления энергопотребления и стоимости эксплуатации.

Подсказка по формулам:

- Refrigerator: $energy = power \cdot hours$, $cost = energy \cdot 0.12$
- WashingMachine: $energy = power \cdot hours \cdot 1.1$, $cost = energy \cdot 0.12$
- Microwave: $energy = power \cdot hours \cdot 0.8$, $cost = energy \cdot 0.12$

Инструкции:

- Создайте абстрактный класс `Appliance` с методами `calculate_energy_usage()` и `calculate_operating_cost()`, используя модуль `abc`.
- Создайте класс `Refrigerator` с конструктором `__init__(self, power, hours)`, приватными атрибутами и геттерами. Реализуйте методы.
- Создайте класс `WashingMachine` с конструктором `__init__(self, power, hours)`, приватными атрибутами и геттерами. Реализуйте методы.
- Создайте класс `Microwave` с конструктором `__init__(self, power, hours)`, приватными атрибутами и геттерами. Реализуйте методы.
- Создайте экземпляр каждого класса и вызовите методы, используя геттеры, и выведите результаты.

Пример использования:

```
fridge = Refrigerator(150, 24)
print("Мощность холодильника:", fridge.power)
print("Энергопотребление:", fridge.calculate_energy_usage())
print("Стоимость эксплуатации:", fridge.calculate_operating_cost())
```

Вывод:

```
Мощность холодильника: 150
Энергопотребление: 3600
Стоимость эксплуатации: 432.0
```

Далее вывод для стиральной машины и микроволновки.

21. Написать программу на Python, которая создает абстрактный класс `Planet` (с использованием модуля `abc`) для планет. Класс должен содержать абстрактные методы `calculate_surface_area()` и `calculate_gravity()`. Программа также должна создавать дочерние классы `Earth`, `Mars` и `Jupiter`, которые наследуют от класса `Planet` и реализуют специфические методы вычисления площади поверхности и силы гравитации.

Подсказка по формулам:

- Earth: $surface_area = 4 \cdot \pi \cdot radius^2$, $gravity = G \cdot mass / radius^2$
- Mars: $surface_area = 4 \cdot \pi \cdot radius^2 \cdot 0.95$, $gravity = G \cdot mass / radius^2 \cdot 0.38$
- Jupiter: $surface_area = 4 \cdot \pi \cdot radius^2 \cdot 11.2$, $gravity = G \cdot mass / radius^2 \cdot 2.5$

Инструкции:

- Создайте абстрактный класс `Planet` с методами `calculate_surface_area()` и `calculate_gravity()`, используя модуль `abc`.

- (b) Создайте класс `Earth` с конструктором `__init__(self, radius, mass)`, приватными атрибутами и геттерами. Реализуйте методы.
- (c) Создайте класс `Mars` с конструктором `__init__(self, radius, mass)`, приватными атрибутами и геттерами. Реализуйте методы.
- (d) Создайте класс `Jupiter` с конструктором `__init__(self, radius, mass)`, приватными атрибутами и геттерами. Реализуйте методы.
- (e) Создайте экземпляр каждого класса и вызовите методы, используя геттеры, и выведите результаты.

Пример использования:

```
earth = Earth(6371, 5.97e24)
print("Радиус Земли:", earth.radius)
print("Площадь поверхности:", earth.calculate_surface_area())
print("Сила гравитации:", earth.calculate_gravity())
```

Вывод:

```
Радиус Земли: 6371
Площадь поверхности: 510064471
Сила гравитации: 9.8
```

Далее вывод для Марса и Юпитера.

22. Написать программу на Python, которая создает абстрактный класс `FoodItem` (с использованием модуля `abc`) для пищевых продуктов. Класс должен содержать абстрактные методы `calculate_calories()` и `calculate_price()`. Программа также должна создавать дочерние классы `Fruit`, `Vegetable` и `Meat`, которые наследуют от класса `FoodItem` и реализуют специфические методы вычисления калорийности и стоимости.

Подсказка по формулам:

- `Fruit`: $calories = weight \cdot 0.52$, $price = weight \cdot 3$
- `Vegetable`: $calories = weight \cdot 0.3$, $price = weight \cdot 2$
- `Meat`: $calories = weight \cdot 2.5$, $price = weight \cdot 10$

Инструкции:

- (a) Создайте абстрактный класс `FoodItem` с методами `calculate_calories()` и `calculate_price()`, используя модуль `abc`.
- (b) Создайте класс `Fruit` с конструктором `__init__(self, weight)`, приватным атрибутом и геттером. Реализуйте методы.
- (c) Создайте класс `Vegetable` с конструктором `__init__(self, weight)`, приватным атрибутом и геттером. Реализуйте методы.
- (d) Создайте класс `Meat` с конструктором `__init__(self, weight)`, приватным атрибутом и геттером. Реализуйте методы.
- (e) Создайте экземпляр каждого класса и вызовите методы, используя геттеры, и выведите результаты.

Пример использования:

```
apple = Fruit(150)
print("Вес фрукта:", apple.weight)
print("Калории:", apple.calculate_calories())
print("Стоимость:", apple.calculate_price())
```

Вывод:

```
Вес фрукта: 150
Калории: 78.0
Стоимость: 450
```

Далее вывод для овощей и мяса.

23. Написать программу на Python, которая создает абстрактный класс `Tool` (с использованием модуля `abc`) для инструментов. Класс должен содержать абстрактные методы `calculate_efficiency()` и `calculate_durability()`. Программа также должна создавать дочерние классы `Hammer`, `Screwdriver` и `Wrench`, которые наследуют от класса `Tool` и реализуют специфические методы вычисления эффективности и прочности.

Подсказка по формулам:

- Hammer: $efficiency = weight \cdot swing_speed$, $durability = material_hardness \cdot 10$
- Screwdriver: $efficiency = length \cdot torque$, $durability = material_hardness \cdot 8$
- Wrench: $efficiency = size \cdot torque$, $durability = material_hardness \cdot 12$

Инструкции:

- Создайте абстрактный класс `Tool` с методами `calculate_efficiency()` и `calculate_durability()`, используя модуль `abc`.
- Создайте класс `Hammer` с конструктором `__init__(self, weight, swing_speed, material_hardness)`, приватными атрибутами и геттерами. Реализуйте методы.
- Создайте класс `Screwdriver` с конструктором `__init__(self, length, torque, material_hardness)`, приватными атрибутами и геттерами. Реализуйте методы.
- Создайте класс `Wrench` с конструктором `__init__(self, size, torque, material_hardness)`, приватными атрибутами и геттерами. Реализуйте методы.
- Создайте экземпляр каждого класса и вызовите методы, используя геттеры, и выведите результаты.

Пример использования:

```
hammer = Hammer(2, 5, 7)
print("Вес молотка:", hammer.weight)
print("Эффективность:", hammer.calculate_efficiency())
print("Прочность:", hammer.calculate_durability())
```

Вывод:

Вес молотка: 2
Эффективность: 10
Прочность: 70

Далее вывод для отвертки и ключа.

24. Написать программу на Python, которая создает абстрактный класс `Book` (с использованием модуля `abc`) для книг. Класс должен содержать абстрактные методы `calculate_reading_time()` и `calculate_cost()`. Программа также должна создавать дочерние классы `Fiction`, `NonFiction` и `Comics`, которые наследуют от класса `Book` и реализуют специфические методы вычисления времени чтения и стоимости.

Подсказка по формулам:

- Fiction: $reading_time = pages \cdot 2$, $cost = pages \cdot 1.5$
- NonFiction: $reading_time = pages \cdot 2.5$, $cost = pages \cdot 2$
- Comics: $reading_time = pages \cdot 1$, $cost = pages \cdot 1$

Инструкции:

- Создайте абстрактный класс `Book` с методами `calculate_reading_time()` и `calculate_cost()`, используя модуль `abc`.
- Создайте класс `Fiction` с конструктором `__init__(self, pages)`, приватным атрибутом и геттером. Реализуйте методы.
- Создайте класс `NonFiction` с конструктором `__init__(self, pages)`, приватным атрибутом и геттером. Реализуйте методы.
- Создайте класс `Comics` с конструктором `__init__(self, pages)`, приватным атрибутом и геттером. Реализуйте методы.
- Создайте экземпляр каждого класса и вызовите методы, используя геттеры, и выведите результаты.

Пример использования:

```
novel = Fiction(300)
print("Количество страниц:", novel.pages)
print("Время чтения:", novel.calculate_reading_time())
print("Стоимость:", novel.calculate_cost())
```

Вывод:

Количество страниц: 300
Время чтения: 600
Стоимость: 450.0

Далее вывод для научной литературы и комиксов.

25. Написать программу на Python, которая создает абстрактный класс `ElectronicDevice` (с использованием модуля `abc`) для электронных устройств. Класс должен содержать абстрактные методы `calculate_power_consumption()` и `calculate_battery_life()`. Программа также должна создавать дочерние классы `Smartphone`, `Laptop` и `Tablet`, которые наследуют от класса `ElectronicDevice` и реализуют специфические методы вычисления потребляемой мощности и времени работы от батареи.

Подсказка по формулам:

- Smartphone: $power = voltage \cdot current \cdot hours$, $battery_life = battery_capacity / current$
- Laptop: $power = voltage \cdot current \cdot hours \cdot 1.5$, $battery_life = battery_capacity / (current \cdot 1.5)$
- Tablet: $power = voltage \cdot current \cdot hours \cdot 1.2$, $battery_life = battery_capacity / (current \cdot 1.2)$

Инструкции:

- Создайте абстрактный класс `ElectronicDevice` с методами `calculate_power_consumption()` и `calculate_battery_life()`, используя модуль `abc`.
- Создайте класс `Smartphone` с конструктором `__init__(self, voltage, current, hours, battery_capacity)`, приватными атрибутами и геттерами. Реализуйте методы.
- Создайте класс `Laptop` с конструктором `__init__(self, voltage, current, hours, battery_capacity)`, приватными атрибутами и геттерами. Реализуйте методы.
- Создайте класс `Tablet` с конструктором `__init__(self, voltage, current, hours, battery_capacity)`, приватными атрибутами и геттерами. Реализуйте методы.
- Создайте экземпляр каждого класса и вызовите методы, используя геттеры, и выведите результаты.

Пример использования:

```
phone = Smartphone(5, 1, 10, 5000)
print("Напряжение смартфона:", phone.voltage)
print("Потребляемая мощность:", phone.calculate_power_consumption())
print("Время работы от батареи:", phone.calculate_battery_life())
```

Вывод:

```
Напряжение смартфона: 5
Потребляемая мощность: 50
Время работы от батареи: 5000.0
```

Далее вывод для ноутбука и планшета.

- Написать программу на Python, которая создает абстрактный класс `MusicalInstrument` (с использованием модуля `abc`) для музыкальных инструментов. Класс должен содержать абстрактные методы `calculate_sound_volume()` и `calculate_weight()`. Программа также должна создавать дочерние классы `Piano`, `Guitar` и `Drum`, которые наследуют от класса `MusicalInstrument` и реализуют специфические методы вычисления громкости и веса.

Подсказка по формулам:

- Piano: $volume = keys \cdot 2$, $weight = base_weight \cdot 3$
- Guitar: $volume = strings \cdot 3$, $weight = base_weight \cdot 1.5$
- Drum: $volume = diameter \cdot 4$, $weight = base_weight \cdot 2$

Инструкции:

- (a) Создайте абстрактный класс `MusicalInstrument` с методами `calculate_sound_volume()` и `calculate_weight()`, используя модуль `abc`.
- (b) Создайте класс `Piano` с конструктором `__init__(self, keys, base_weight)`, приватными атрибутами и геттерами. Реализуйте методы.
- (c) Создайте класс `Guitar` с конструктором `__init__(self, strings, base_weight)`, приватными атрибутами и геттерами. Реализуйте методы.
- (d) Создайте класс `Drum` с конструктором `__init__(self, diameter, base_weight)`, приватными атрибутами и геттерами. Реализуйте методы.
- (e) Создайте экземпляр каждого класса и вызовите методы, используя геттеры, и выведите результаты.

Пример использования:

```
piano = Piano(88, 200)
print("Количество клавиш:", piano.keys)
print("Громкость:", piano.calculate_sound_volume())
print("Вес:", piano.calculate_weight())
```

Вывод:

```
Количество клавиш: 88
Громкость: 176
Вес: 600
```

Далее вывод для гитары и барабана.

27. Написать программу на Python, которая создает абстрактный класс `VehiclePart` (с использованием модуля `abc`) для частей транспортного средства. Класс должен содержать абстрактные методы `calculate_durability()` и `calculate_maintenance_cost()`. Программа также должна создавать дочерние классы `Engine`, `Wheel` и `Brake`, которые наследуют от класса `VehiclePart` и реализуют специфические методы вычисления долговечности и стоимости обслуживания.

Подсказка по формулам:

- `Engine`: $durability = hours_run \cdot 1.2$, $maintenance = base_cost \cdot 5$
- `Wheel`: $durability = rotation_count \cdot 0.8$, $maintenance = base_cost \cdot 2$
- `Brake`: $durability = pressure_applied \cdot 0.5$, $maintenance = base_cost \cdot 3$

Инструкции:

- (a) Создайте абстрактный класс `VehiclePart` с методами `calculate_durability()` и `calculate_maintenance_cost()`, используя модуль `abc`.
- (b) Создайте класс `Engine` с конструктором `__init__(self, hours_run, base_cost)`, приватными атрибутами и геттерами. Реализуйте методы.
- (c) Создайте класс `Wheel` с конструктором `__init__(self, rotation_count, base_cost)`, приватными атрибутами и геттерами. Реализуйте методы.
- (d) Создайте класс `Brake` с конструктором `__init__(self, pressure_applied, base_cost)`, приватными атрибутами и геттерами. Реализуйте методы.

- (е) Создайте экземпляр каждого класса и вызовите методы, используя геттеры, и выведите результаты.

Пример использования:

```
engine = Engine(1000, 200)
print("Наработка двигателя:", engine.hours_run)
print("Долговечность:", engine.calculate_durability())
print("Стоимость обслуживания:", engine.calculate_maintenance_cost())
```

Вывод:

Наработка двигателя: 1000
Долговечность: 1200.0
Стоимость обслуживания: 1000

Далее вывод для колес и тормозов.

28. Написать программу на Python, которая создает абстрактный класс `Appliance` (с использованием модуля `abc`) для бытовых приборов. Класс должен содержать абстрактные методы `calculate_energy_consumption()` и `calculate_cost()`. Программа также должна создавать дочерние классы `Refrigerator`, `WashingMachine` и `Microwave`, которые наследуют от класса `Appliance` и реализуют специфические методы вычисления потребляемой энергии и стоимости эксплуатации.

Подсказка по формулам:

- Refrigerator: $energy = power \cdot hours \cdot 30$, $cost = energy \cdot rate$
- WashingMachine: $energy = power \cdot hours \cdot 1.5$, $cost = energy \cdot rate$
- Microwave: $energy = power \cdot hours \cdot 0.8$, $cost = energy \cdot rate$

Инструкции:

- Создайте абстрактный класс `Appliance` с методами `calculate_energy_consumption()` и `calculate_cost()`, используя модуль `abc`.
- Создайте класс `Refrigerator` с конструктором `__init__(self, power, hours, rate)`, приватными атрибутами и геттерами. Реализуйте методы.
- Создайте класс `WashingMachine` с конструктором `__init__(self, power, hours, rate)`, приватными атрибутами и геттерами. Реализуйте методы.
- Создайте класс `Microwave` с конструктором `__init__(self, power, hours, rate)`, приватными атрибутами и геттерами. Реализуйте методы.
- Создайте экземпляр каждого класса и вызовите методы, используя геттеры, и выведите результаты.

Пример использования:

```
fridge = Refrigerator(150, 24, 0.1)
print("Мощность холодильника:", fridge.power)
print("Энергопотребление:", fridge.calculate_energy_consumption())
print("Стоимость эксплуатации:", fridge.calculate_cost())
```

Вывод:

Мощность холодильника: 150
Энергопотребление: 108000
Стоимость эксплуатации: 10800.0

Далее вывод для стиральной машины и микроволновки.

29. Написать программу на Python, которая создает абстрактный класс `SportActivity` (с использованием модуля `abc`) для спортивных занятий. Класс должен содержать абстрактные методы `calculate_calories_burned()` и `calculate_duration()`. Программа также должна создавать дочерние классы `Running`, `Swimming` и `Cycling`, которые наследуют от класса `SportActivity` и реализуют специфические методы вычисления сожженных калорий и продолжительности.

Подсказка по формулам:

- `Running`: $calories = weight \cdot distance \cdot 1.036$, $duration = distance / speed$
- `Swimming`: $calories = weight \cdot distance \cdot 1.5$, $duration = distance / speed$
- `Cycling`: $calories = weight \cdot distance \cdot 0.8$, $duration = distance / speed$

Инструкции:

- Создайте абстрактный класс `SportActivity` с методами `calculate_calories_burned()` и `calculate_duration()`, используя модуль `abc`.
- Создайте класс `Running` с конструктором `__init__(self, weight, distance, speed)`, приватными атрибутами и геттерами. Реализуйте методы.
- Создайте класс `Swimming` с конструктором `__init__(self, weight, distance, speed)`, приватными атрибутами и геттерами. Реализуйте методы.
- Создайте класс `Cycling` с конструктором `__init__(self, weight, distance, speed)`, приватными атрибутами и геттерами. Реализуйте методы.
- Создайте экземпляр каждого класса и вызовите методы, используя геттеры, и выведите результаты.

Пример использования:

```
run = Running(70, 5, 10)
print("Вес бегуна:", run.weight)
print("Сожженные калории:", run.calculate_calories_burned())
print("Продолжительность:", run.calculate_duration())
```

Вывод:

Вес бегуна: 70
Сожженные калории: 362.6
Продолжительность: 0.5

Далее вывод для плавания и езды на велосипеде.

30. Написать программу на Python, которая создает абстрактный класс `BuildingMaterial` (с использованием модуля `abc`) для строительных материалов. Класс должен содержать абстрактные методы `calculate_strength()` и `calculate_cost()`. Программа также должна создавать дочерние классы `Concrete`, `Wood`, `Steel`, которые наследуют от класса `BuildingMaterial` и реализуют специфические методы вычисления прочности и стоимости.

Подсказка по формулам:

- Concrete: $strength = density \cdot compressive_factor$, $cost = volume \cdot price_per_m3$
- Wood: $strength = density \cdot elastic_factor$, $cost = volume \cdot price_per_m3$
- Steel: $strength = density \cdot tensile_factor$, $cost = volume \cdot price_per_m3$

Инструкции:

- Создайте абстрактный класс `BuildingMaterial` с методами `calculate_strength()` и `calculate_cost()`, используя модуль `abc`.
- Создайте класс `Concrete` с конструктором `__init__(self, density, compressive_factor, volume, price_per_m3)`, приватными атрибутами и геттерами. Реализуйте методы.
- Создайте класс `Wood` с конструктором `__init__(self, density, elastic_factor, volume, price_per_m3)`, приватными атрибутами и геттерами. Реализуйте методы.
- Создайте класс `Steel` с конструктором `__init__(self, density, tensile_factor, volume, price_per_m3)`, приватными атрибутами и геттерами. Реализуйте методы.
- Создайте экземпляр каждого класса и вызовите методы, используя геттеры, и выведите результаты.

Пример использования:

```
concrete = Concrete(2400, 30, 2, 100)
print("Плотность бетона:", concrete.density)
print("Прочность:", concrete.calculate_strength())
print("Стоимость:", concrete.calculate_cost())
```

Вывод:

```
Плотность бетона: 2400
Прочность: 72000
Стоимость: 200
```

Далее вывод для древесины и стали.

31. Написать программу на Python, которая создает абстрактный класс `TransportVehicle` (с использованием модуля `abc`) для транспортных средств. Класс должен содержать абстрактные методы `calculate_range()` и `calculate_fuel_cost()`. Программа также должна создавать дочерние классы `Car`, `Motorcycle` и `ElectricScooter`, которые наследуют от класса `TransportVehicle` и реализуют специфические методы вычисления дальности хода и стоимости топлива/энергии.

Подсказка по формулам:

- **Car:** $range = tank_capacity / consumption \cdot 100$, $fuel_cost = tank_capacity \cdot fuel_price$
- **Motorcycle:** $range = tank_capacity / consumption \cdot 120$, $fuel_cost = tank_capacity \cdot fuel_price$
- **ElectricScooter:** $range = battery_capacity / consumption \cdot 100$, $fuel_cost = battery_capacity \cdot electricity_rate$

Инструкции:

- Создайте абстрактный класс `TransportVehicle` с методами `calculate_range()` и `calculate_fuel_cost()`, используя модуль `abc`.
- Создайте класс `Car` с конструктором `__init__(self, tank_capacity, consumption, fuel_price)`, приватными атрибутами и геттерами. Реализуйте методы.
- Создайте класс `Motorcycle` с конструктором `__init__(self, tank_capacity, consumption, fuel_price)`, приватными атрибутами и геттерами. Реализуйте методы.
- Создайте класс `ElectricScooter` с конструктором `__init__(self, battery_capacity, consumption, electricity_rate)`, приватными атрибутами и геттерами. Реализуйте методы.
- Создайте экземпляр каждого класса и вызовите методы, используя геттеры, и выведите результаты.

Пример использования:

```
car = Car(50, 8, 1.5)
print("Ёмкость бака автомобиля:", car.tank_capacity)
print("Дальность хода:", car.calculate_range())
print("Стоимость топлива:", car.calculate_fuel_cost())
```

Вывод:

```
Ёмкость бака автомобиля: 50
Дальность хода: 625.0
Стоимость топлива: 75.0
```

Далее вывод для `Motorcycle` и `ElectricScooter`.

2.3 Семинар «Структуры данных в ООП-реализации» (2 часа)

В ходе работы решите 4 задачи. Предполагается, что пользователь класса не имеет права обращаться к свойствам напрямую (соблюдая принцип инкапсуляции), а должен использовать методы.

Продемонстрируйте работоспособность всех методов (из задания) посредством создания запускаемых файлов, где осуществляется вызов методов для разных ситуаций (без ручного ввода, но с выводом результатов в консоль).

Каждый класс должен сохраняться в отдельном исходном файле. Необходимо соблюдать все стандартные требования к качеству кода (отступы, именования переменных, классов, методов, проверка корректности входных данных).

Задания этого семинара предназначены для освоения не только ООП, но и структур данных, поэтому требуется структуры формировать вручную без использования библиотечных вариантов.

Для сдачи работы будьте готовы пояснить или аналогично заданию модифицировать любую часть кода, а также ответить на вопросы:

1. Что обозначает свойство наследования в парадигме ООП?
2. Что обозначает свойство полиморфизма в парадигме ООП?
3. Опишите реализацию наследования в Python
4. Как создать конструктор в Python
5. Как реализовать абстрактный класс в Python (и что это значит)
6. Как реализовать абстрактные методы в Python (и что это значит)
7. Опишите бинарное дерево
8. Как вставить элемент в бинарное дерево
9. Как найти элемент в бинарном дереве
10. Опишите, что такое стек
11. Опишите, что такое очередь
12. Опишите двусвязный список
13. Сравните стек, очередь и двусвязный список

Если вы нашли в задачнике ошибки, опечатки и другие недостатки, то вы можете сделать pull-request.

Срок сдачи работы (начала сдачи): через одно занятие после его выдачи. В последующие сроки оценка будет снижаться (при отсутствии оправдывающих документов).

2.3.1 Задача 1 (дерево)

1. Написать программу на Python, которая реализует бинарное дерево поиска с инкапсуляцией внутренней структуры. Программа должна создавать экземпляры класса `TreeNode`, которые представляют узлы дерева, и класса `SearchTree`, который представляет дерево поиска. Класс `SearchTree` должен содержать методы для добавления,

поиска и удаления элементов из дерева, при этом все вспомогательные методы должны быть приватными. Программа также должна создавать дерево поиска, вставлять в него случайные числа и выполнять поиск элементов в дереве.

Инструкции:

- (a) Создайте класс `TreeNode` с методом `__init__`, который принимает значение в качестве аргумента и сохраняет его в атрибуте `self.data`. Атрибуты `left` и `right` должны быть инициализированы как `None`.
- (b) Создайте класс `SearchTree` с методом `__init__`, который инициализирует корневой узел дерева как `None`.
- (c) Создайте публичный метод `add` в классе `SearchTree`, который добавляет значение в дерево. Если корневой узел отсутствует, создайте новый узел с добавляемым значением. В противном случае, вызовите приватный метод `_add_helper`, передав ему корень и значение.
- (d) Создайте приватный метод `_add_helper` в классе `SearchTree`, который рекурсивно добавляет значение в дерево. Если значение меньше или равно значению текущего узла, добавьте его в левое поддерево. Если значение строго больше значения текущего узла, добавьте его в правое поддерево.
- (e) Создайте публичный метод `locate` в классе `SearchTree`, который ищет значение в дереве. Если дерево пустое, верните `None`. В противном случае, вызовите приватный метод `_locate_helper`, передав ему корень и искомое значение.
- (f) Создайте приватный метод `_locate_helper` в классе `SearchTree`, который рекурсивно ищет значение в дереве. Если текущий узел равен `None` или значение текущего узла равно искомому значению, верните текущий узел. В противном случае, рекурсивно вызывайте метод `_locate_helper` для поиска значения в левом поддереве (если искомое значение меньше или равно значению текущего узла) или в правом поддереве (если искомое значение больше).
- (g) Создайте экземпляр класса `SearchTree` и вставьте в него 15 случайных чисел от 1 до 30.
- (h) Выполните поиск элементов в дереве и выведите результаты на экран.

Пример использования:

```
tree = SearchTree()
for i in range(15):
    tree.add(random.randint(1, 30))

print("Поиск элементов:")
print(tree.locate(7))    # Обнаружено, возвращен узел (7)
print(tree.locate(25))   # Не обнаружено, возвращено None
print(tree.locate(15))   # Обнаружено, возвращен узел (15)
```

2. Написать программу на Python, которая реализует бинарное дерево поиска с соблюдением принципов инкапсуляции. Программа должна создавать экземпляры класса `Vertex`, которые представляют узлы дерева, и класса `BinaryTree`, который представляет дерево поиска. Класс `BinaryTree` должен содержать методы для вставки, поиска и удаления элементов, при этом все рекурсивные вспомогательные методы должны быть скрыты от внешнего доступа. Программа также должна создавать дерево поиска, вставлять в него случайные числа и выполнять поиск элементов в дереве.

Инструкции:

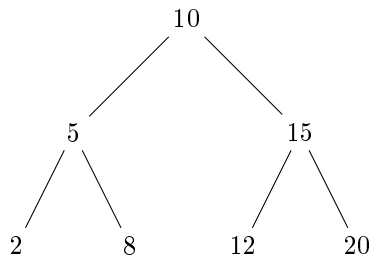


Рис. 1: Пример бинарного дерева поиска

- Создайте класс `Vertex` с методом `__init__`, который принимает значение `value` и сохраняет его в атрибуте `self.key`. Атрибуты `self.left_child` и `self.right_child` должны быть инициализированы как `None`.
- Создайте класс `BinaryTree` с методом `__init__`, который инициализирует атрибут `self.top` как `None`.
- Создайте публичный метод `put` в классе `BinaryTree`, который вставляет значение в дерево. Если `self.top` отсутствует, создайте новый узел с вставляемым значением. В противном случае, вызовите приватный метод `_put_recursively`, передав ему `self.top` и значение.
- Создайте приватный метод `_put_recursively` в классе `BinaryTree`, который рекурсивно вставляет значение в дерево. Если значение строго меньше значения текущего узла, вставьте его в левое поддерево. Если значение больше или равно значению текущего узла, вставьте его в правое поддерево.
- Создайте публичный метод `find` в классе `BinaryTree`, который ищет значение в дереве. Если дерево пустое, верните `None`. В противном случае, вызовите приватный метод `_find_recursively`, передав ему `self.top` и искомое значение.
- Создайте приватный метод `_find_recursively` в классе `BinaryTree`, который рекурсивно ищет значение в дереве. Если текущий узел равен `None` или значение текущего узла равно искомому значению, верните текущий узел. В противном случае, рекурсивно вызывайте метод `_find_recursively` для поиска значения в левом поддереве (если искомое значение меньше текущего) или в правом поддереве (если искомое значение больше или равно текущему).
- Создайте экземпляр класса `BinaryTree` и вставьте в него 18 случайных чисел от 5 до 35.
- Выполните поиск элементов в дереве и выведите результаты на экран.

Пример использования:

```

bt = BinaryTree()
for i in range(18):
    bt.put(random.randint(5, 35))

print("Поиск элементов:")
print(bt.find(10)) # Обнаружено, возвращен узел (10)
print(bt.find(40)) # Не обнаружено, возвращено None
print(bt.find(22)) # Обнаружено, возвращен узел (22)

```

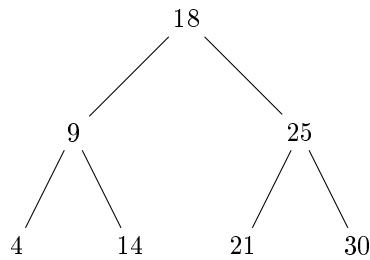


Рис. 2: Пример бинарного дерева поиска

3. Написать программу на Python, которая реализует бинарное дерево поиска с инкапсуляцией внутренней логики. Программа должна создавать экземпляры класса `BNode`, которые представляют узлы дерева, и класса `BSTree`, который представляет дерево поиска. Класс `BSTree` должен содержать методы для вставки, поиска и удаления элементов, при этом все рекурсивные функции должны быть приватными. Программа также должна создавать дерево поиска, вставлять в него случайные числа и выполнять поиск элементов в дереве.

Инструкции:

- (a) Создайте класс `BNode` с методом `__init__`, который принимает параметр `item` и сохраняет его в атрибуте `self.element`. Атрибуты `self.left_branch` и `self.right_branch` должны быть инициализированы как `None`.
- (b) Создайте класс `BSTree` с методом `__init__`, который инициализирует атрибут `self.root_node` как `None`.
- (c) Создайте публичный метод `insert_value` в классе `BSTree`, который вставляет значение в дерево. Если `self.root_node` отсутствует, создайте новый узел с вставляемым значением. В противном случае, вызовите приватный метод `_recursive_insert`, передав ему `self.root_node` и значение.
- (d) Создайте приватный метод `_recursive_insert` в классе `BSTree`, который рекурсивно вставляет значение в дерево. Если значение меньше или равно значению текущего узла, вставьте его в левое поддерево. Если значение строго больше значения текущего узла, вставьте его в правое поддерево.
- (e) Создайте публичный метод `retrieve` в классе `BSTree`, который ищет значение в дереве. Если дерево пустое, верните `None`. В противном случае, вызовите приватный метод `_recursive_retrieve`, передав ему `self.root_node` и искомое значение.
- (f) Создайте приватный метод `_recursive_retrieve` в классе `BSTree`, который рекурсивно ищет значение в дереве. Если текущий узел равен `None` или значение текущего узла равно искомому значению, верните текущий узел. В противном случае, рекурсивно вызывайте метод `_recursive_retrieve` для поиска значения в левом поддереве (если искомое значение меньше или равно текущему) или в правом поддереве (если искомое значение больше).
- (g) Создайте экземпляр класса `BSTree` и вставьте в него 20 случайных чисел от 1 до 40.
- (h) Выполните поиск элементов в дереве и выведите результаты на экран.

Пример использования:

```

bst = BSTree()
for i in range(20):
    bst.insert_value(random.randint(1, 40))

print("Поиск элементов:")
print(bst.retrieve(12)) # Обнаружено, возвращен узел (12)
print(bst.retrieve(50)) # Не обнаружено, возвращено None
print(bst.retrieve(33)) # Обнаружено, возвращен узел (33)

```

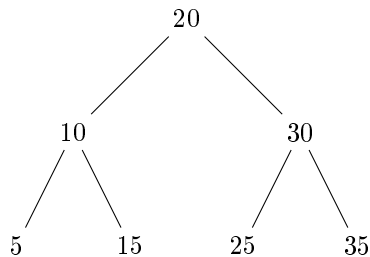


Рис. 3: Пример бинарного дерева поиска

4. Написать программу на Python, которая реализует бинарное дерево поиска с инкапсуляцией. Программа должна создавать экземпляры класса `ElementNode`, которые представляют узлы дерева, и класса `OrderedTree`, который представляет дерево поиска. Класс `OrderedTree` должен содержать методы для вставки, поиска и удаления элементов, при этом все вспомогательные методы должны быть приватными. Программа также должна создавать дерево поиска, вставляя в него случайные числа и выполнять поиск элементов в дереве.

Инструкции:

- (a) Создайте класс `ElementNode` с методом `__init__`, который принимает параметр `content` и сохраняет его в атрибуте `self.payload`. Атрибуты `self.left` и `self.right` должны быть инициализированы как `None`.
- (b) Создайте класс `OrderedTree` с методом `__init__`, который инициализирует атрибут `self.head` как `None`.
- (c) Создайте публичный метод `store` в классе `OrderedTree`, который вставляет значение в дерево. Если `self.head` отсутствует, создайте новый узел с вставляемым значением. В противном случае, вызовите приватный метод `_store_recursive`, передав ему `self.head` и значение.
- (d) Создайте приватный метод `_store_recursive` в классе `OrderedTree`, который рекурсивно вставляет значение в дерево. Если значение строго меньше значения текущего узла, вставьте его в левое поддерево. Если значение больше или равно значению текущего узла, вставьте его в правое поддерево.
- (e) Создайте публичный метод `query` в классе `OrderedTree`, который ищет значение в дереве. Если дерево пустое, верните `None`. В противном случае, вызовите приватный метод `_query_recursive`, передав ему `self.head` и искомое значение.
- (f) Создайте приватный метод `_query_recursive` в классе `OrderedTree`, который рекурсивно ищет значение в дереве. Если текущий узел равен `None` или значение текущего узла равно искомому значению, верните текущий узел. В противном

случае, рекурсивно вызывайте метод `_query_recursive` для поиска значения в левом поддереве (если искомое значение меньше текущего) или в правом поддереве (если искомое значение больше или равно текущему).

- (g) Создайте экземпляр класса `OrderedTree` и вставьте в него 17 случайных чисел от 3 до 33.
- (h) Выполните поиск элементов в дереве и выведите результаты на экран.

Пример использования:

```
ot = OrderedTree()
for i in range(17):
    ot.store(random.randint(3, 33))

print("Поиск элементов:")
print(ot.query(8))    # Обнаружено, возвращен узел (8)
print(ot.query(45))   # Не обнаружено, возвращено None
print(ot.query(27))   # Обнаружено, возвращен узел (27)
```

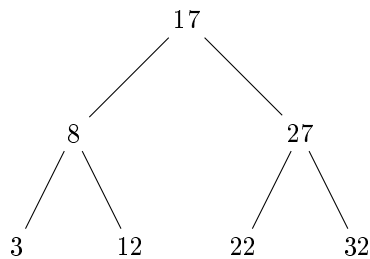


Рис. 4: Пример бинарного дерева поиска

5. Написать программу на Python, которая реализует бинарное дерево поиска с инкапсуляцией внутренней структуры. Программа должна создавать экземпляры класса `TreeNode`, которые представляют узлы дерева, и класса `SortedTree`, который представляет дерево поиска. Класс `SortedTree` должен содержать методы для вставки, поиска и удаления элементов, при этом все рекурсивные методы должны быть скрыты. Программа также должна создавать дерево поиска, вставлять в него случайные числа и выполнять поиск элементов в дереве.

Инструкции:

- (a) Создайте класс `TreeNode` с методом `__init__`, который принимает параметр `val` и сохраняет его в атрибуте `self.entry`. Атрибуты `self.left` и `self.right` должны быть инициализированы как `None`.
- (b) Создайте класс `SortedTree` с методом `__init__`, который инициализирует атрибут `self.first_node` как `None`.
- (c) Создайте публичный метод `enqueue` в классе `SortedTree`, который вставляет значение в дерево. Если `self.first_node` отсутствует, создайте новый узел с вставляемым значением. В противном случае, вызовите приватный метод `_enqueue_helper`, передав ему `self.first_node` и значение.
- (d) Создайте приватный метод `_enqueue_helper` в классе `SortedTree`, который рекурсивно вставляет значение в дерево. Если значение меньше или равно значению

текущего узла, вставьте его в левое поддерево. Если значение строго больше значения текущего узла, вставьте его в правое поддерево.

- (e) Создайте публичный метод `lookup` в классе `SortedTree`, который ищет значение в дереве. Если дерево пустое, верните `None`. В противном случае, вызовите приватный метод `_lookup_helper`, передав ему `self.first_node` и искомое значение.
- (f) Создайте приватный метод `_lookup_helper` в классе `SortedTree`, который рекурсивно ищет значение в дереве. Если текущий узел равен `None` или значение текущего узла равно искомому значению, верните текущий узел. В противном случае, рекурсивно вызывайте метод `_lookup_helper` для поиска значения в левом поддереве (если искомое значение меньше или равно текущему) или в правом поддереве (если искомое значение больше).
- (g) Создайте экземпляр класса `SortedTree` и вставьте в него 16 случайных чисел от 2 до 28.
- (h) Выполните поиск элементов в дереве и выведите результаты на экран.

Пример использования:

```
st = SortedTree()
for i in range(16):
    st.enqueue(random.randint(2, 28))

print("Поиск элементов:")
print(st.lookup(6))    # Обнаружено, возвращен узел (6)
print(st.lookup(35))   # Не обнаружено, возвращено None
print(st.lookup(19))   # Обнаружено, возвращен узел (19)
```

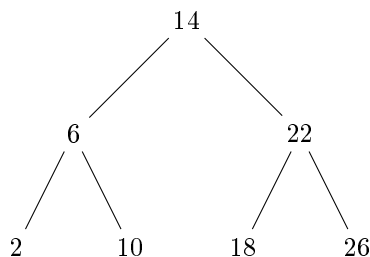


Рис. 5: Пример бинарного дерева поиска

- 6. Написать программу на Python, которая реализует бинарное дерево поиска с инкапсуляцией. Программа должна создавать экземпляры класса `BinNode`, которые представляют узлы дерева, и класса `LookupTree`, который представляет дерево поиска. Класс `LookupTree` должен содержать методы для вставки, поиска и удаления элементов, при этом все вспомогательные методы должны быть приватными. Программа также должна создавать дерево поиска, вставлять в него случайные числа и выполнять поиск элементов в дереве.

Инструкции:

- (a) Создайте класс `BinNode` с методом `__init__`, который принимает параметр `num` и сохраняет его в атрибуте `self.number`. Атрибуты `self.left` и `self.right` должны быть инициализированы как `None`.

- (b) Создайте класс `LookupTree` с методом `__init__`, который инициализирует атрибут `self.initial_node` как `None`.
- (c) Создайте публичный метод `add_entry` в классе `LookupTree`, который вставляет значение в дерево. Если `self.initial_node` отсутствует, создайте новый узел с вставляемым значением. В противном случае, вызовите приватный метод `_add_entry_rec`, передав ему `self.initial_node` и значение.
- (d) Создайте приватный метод `_add_entry_rec` в классе `LookupTree`, который рекурсивно вставляет значение в дерево. Если значение строго меньше значения текущего узла, вставьте его в левое поддерево. Если значение больше или равно значению текущего узла, вставьте его в правое поддерево.
- (e) Создайте публичный метод `fetch` в классе `LookupTree`, который ищет значение в дереве. Если дерево пустое, верните `None`. В противном случае, вызовите приватный метод `_fetch_rec`, передав ему `self.initial_node` и искомое значение.
- (f) Создайте приватный метод `_fetch_rec` в классе `LookupTree`, который рекурсивно ищет значение в дереве. Если текущий узел равен `None` или значение текущего узла равно искомому значению, верните текущий узел. В противном случае, рекурсивно вызывайте метод `_fetch_rec` для поиска значения в левом поддереве (если искомое значение меньше текущего) или в правом поддереве (если искомое значение больше или равно текущему).
- (g) Создайте экземпляр класса `LookupTree` и вставьте в него 19 случайных чисел от 4 до 34.
- (h) Выполните поиск элементов в дереве и выведите результаты на экран.

Пример использования:

```
lt = LookupTree()
for i in range(19):
    lt.add_entry(random.randint(4, 34))

print("Поиск элементов:")
print(lt.fetch(9))      # Обнаружено, возвращен узел (9)
print(lt.fetch(40))     # Не обнаружено, возвращено None
print(lt.fetch(24))     # Обнаружено, возвращен узел (24)
```

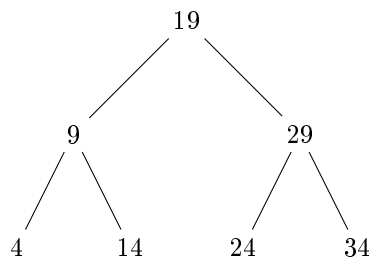


Рис. 6: Пример бинарного дерева поиска

7. Написать программу на Python, которая реализует бинарное дерево поиска с инкапсуляцией. Программа должна создавать экземпляры класса `NodeItem`, которые представляют узлы дерева, и класса `BinaryTreeSearch`, который представляет дерево поиска. Класс `BinaryTreeSearch` должен содержать методы для вставки, поиска и удаления

элементов, при этом все рекурсивные методы должны быть приватными. Программа также должна создавать дерево поиска, вставлять в него случайные числа и выполнять поиск элементов в дереве.

Инструкции:

- (a) Создайте класс `NodeItem` с методом `__init__`, который принимает параметр `item_value` и сохраняет его в атрибуте `self.val`. Атрибуты `self.left` и `self.right` должны быть инициализированы как `None`.
- (b) Создайте класс `BinaryTreeSearch` с методом `__init__`, который инициализирует атрибут `self.start_node` как `None`.
- (c) Создайте публичный метод `insert_item` в классе `BinaryTreeSearch`, который вставляет значение в дерево. Если `self.start_node` отсутствует, создайте новый узел с вставляемым значением. В противном случае, вызовите приватный метод `_insert_item_helper`, передав ему `self.start_node` и значение.
- (d) Создайте приватный метод `_insert_item_helper` в классе `BinaryTreeSearch`, который рекурсивно вставляет значение в дерево. Если значение меньше или равно значению текущего узла, вставьте его в левое поддерево. Если значение строго больше значения текущего узла, вставьте его в правое поддерево.
- (e) Создайте публичный метод `find_item` в классе `BinaryTreeSearch`, который ищет значение в дереве. Если дерево пустое, верните `None`. В противном случае, вызовите приватный метод `_find_item_helper`, передав ему `self.start_node` и искомое значение.
- (f) Создайте приватный метод `_find_item_helper` в классе `BinaryTreeSearch`, который рекурсивно ищет значение в дереве. Если текущий узел равен `None` или значение текущего узла равно искомому значению, верните текущий узел. В противном случае, рекурсивно вызывайте метод `_find_item_helper` для поиска значения в левом поддереве (если искомое значение меньше или равно текущему) или в правом поддереве (если искомое значение больше).
- (g) Создайте экземпляр класса `BinaryTreeSearch` и вставьте в него 21 случайное число от 1 до 38.
- (h) Выполните поиск элементов в дереве и выведите результаты на экран.

Пример использования:

```
bts = BinaryTreeSearch()
for i in range(21):
    bts.insert_item(random.randint(1, 38))

print("Поиск элементов:")
print(bts.find_item(11)) # Обнаружено, возвращен узел (11)
print(bts.find_item(50)) # Не обнаружено, возвращено None
print(bts.find_item(29)) # Обнаружено, возвращен узел (29)
```

8. Написать программу на Python, которая реализует бинарное дерево поиска с инкапсуляцией. Программа должна создавать экземпляры класса `TreeVertex`, которые представляют узлы дерева, и класса `SearchBinTree`, который представляет дерево поиска. Класс `SearchBinTree` должен содержать методы для вставки, поиска и удаления элементов, при этом все вспомогательные методы должны быть приватными. Программа также должна создавать дерево поиска, вставлять в него случайные числа и выполнять поиск элементов в дереве.

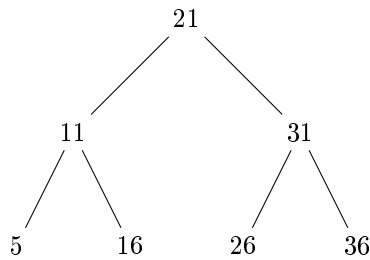


Рис. 7: Пример бинарного дерева поиска

Инструкции:

- Создайте класс `TreeVertex` с методом `__init__`, который принимает параметр `vertex_data` и сохраняет его в атрибуте `self.info`. Атрибуты `self.left` и `self.right` должны быть инициализированы как `None`.
- Создайте класс `SearchBinTree` с методом `__init__`, который инициализирует атрибут `self.root_vertex` как `None`.
- Создайте публичный метод `insert_data` в классе `SearchBinTree`, который вставляет значение в дерево. Если `self.root_vertex` отсутствует, создайте новый узел с вставляемым значением. В противном случае, вызовите приватный метод `_insert_data_rec`, передав ему `self.root_vertex` и значение.
- Создайте приватный метод `_insert_data_rec` в классе `SearchBinTree`, который рекурсивно вставляет значение в дерево. Если значение строго меньше значения текущего узла, вставьте его в левое поддерево. Если значение больше или равно значению текущего узла, вставьте его в правое поддерево.
- Создайте публичный метод `search_data` в классе `SearchBinTree`, который ищет значение в дереве. Если дерево пустое, верните `None`. В противном случае, вызовите приватный метод `_search_data_rec`, передав ему `self.root_vertex` и искомое значение.
- Создайте приватный метод `_search_data_rec` в классе `SearchBinTree`, который рекурсивно ищет значение в дереве. Если текущий узел равен `None` или значение текущего узла равно искомому значению, верните текущий узел. В противном случае, рекурсивно вызывайте метод `_search_data_rec` для поиска значения в левом поддереве (если искомое значение меньше текущего) или в правом поддереве (если искомое значение больше или равно текущему).
- Создайте экземпляр класса `SearchBinTree` и вставьте в него 14 случайных чисел от 6 до 36.
- Выполните поиск элементов в дереве и выведите результаты на экран.

Пример использования:

```

sbt = SearchBinTree()
for i in range(14):
    sbt.insert_data(random.randint(6, 36))

print("Поиск элементов:")
print(sbt.search_data(13)) # Обнаружено, возвращен узел (13)
print(sbt.search_data(42)) # Не обнаружено, возвращено None
print(sbt.search_data(28)) # Обнаружено, возвращен узел (28)

```

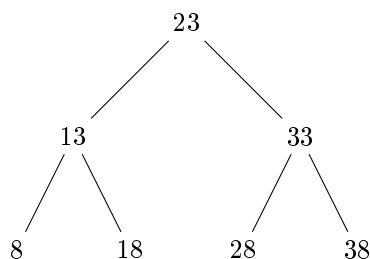



Рис. 8: Пример бинарного дерева поиска

9. Написать программу на Python, которая реализует бинарное дерево поиска с инкапсуляцией. Программа должна создавать экземпляры класса `BranchNode`, которые представляют узлы дерева, и класса `BinaryTreeLookup`, который представляет дерево поиска. Класс `BinaryTreeLookup` должен содержать методы для вставки, поиска и удаления элементов, при этом все рекурсивные методы должны быть приватными. Программа также должна создавать дерево поиска, вставлять в него случайные числа и выполнять поиск элементов в дереве.

Инструкции:

- Создайте класс `BranchNode` с методом `__init__`, который принимает параметр `node_val` и сохраняет его в атрибуте `self.data_point`. Атрибуты `self.left_link` и `self.right_link` должны быть инициализированы как `None`.
- Создайте класс `BinaryTreeLookup` с методом `__init__`, который инициализирует атрибут `self.base_node` как `None`.
- Создайте публичный метод `add_point` в классе `BinaryTreeLookup`, который вставляет значение в дерево. Если `self.base_node` отсутствует, создайте новый узел с вставляемым значением. В противном случае, вызовите приватный метод `_add_point_recursive`, передав ему `self.base_node` и значение.
- Создайте приватный метод `_add_point_recursive` в классе `BinaryTreeLookup`, который рекурсивно вставляет значение в дерево. Если значение меньше или равно значению текущего узла, вставьте его в левое поддерево. Если значение строго больше значения текущего узла, вставьте его в правое поддерево.
- Создайте публичный метод `locate_point` в классе `BinaryTreeLookup`, который ищет значение в дереве. Если дерево пустое, верните `None`. В противном случае, вызовите приватный метод `_locate_point_recursive`, передав ему `self.base_node` и искомое значение.
- Создайте приватный метод `_locate_point_recursive` в классе `BinaryTreeLookup`, который рекурсивно ищет значение в дереве. Если текущий узел равен `None` или значение текущего узла равно искомому значению, верните текущий узел. В противном случае, рекурсивно вызывайте метод `_locate_point_recursive` для поиска значения в левом поддереве (если искомое значение меньше или равно текущему) или в правом поддереве (если искомое значение больше).
- Создайте экземпляр класса `BinaryTreeLookup` и вставьте в него 13 случайных чисел от 7 до 37.
- Выполните поиск элементов в дереве и выведите результаты на экран.

Пример использования:

```
btl = BinaryTreeLookup()
for i in range(13):
    btl.add_point(random.randint(7, 37))

print("Поиск элементов:")
print(btl.locate_point(14)) # Обнаружено, возвращен узел (14)
print(btl.locate_point(45)) # Не обнаружено, возвращено None
print(btl.locate_point(27)) # Обнаружено, возвращен узел (27)
```

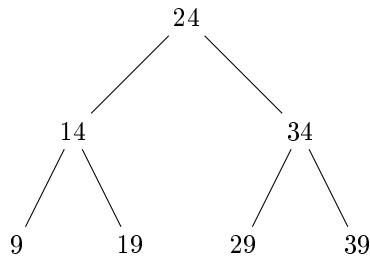


Рис. 9: Пример бинарного дерева поиска

10. Написать программу на Python, которая реализует бинарное дерево поиска с инкапсуляцией. Программа должна создавать экземпляры класса `TreeNodeStruct`, которые представляют узлы дерева, и класса `BinSearchStructure`, который представляет дерево поиска. Класс `BinSearchStructure` должен содержать методы для вставки, поиска и удаления элементов, при этом все вспомогательные методы должны быть приватными. Программа также должна создавать дерево поиска, вставлять в него случайные числа и выполнять поиск элементов в дереве.

Инструкции:

- Создайте класс `TreeNodeStruct` с методом `__init__`, который принимает параметр `struct_value` и сохраняет его в атрибуте `self.node_value`. Атрибуты `self.left_sub` и `self.right_sub` должны быть инициализированы как `None`.
- Создайте класс `BinSearchStructure` с методом `__init__`, который инициализирует атрибут `self.top_element` как `None`.
- Создайте публичный метод `insert_struct` в классе `BinSearchStructure`, который вставляет значение в дерево. Если `self.top_element` отсутствует, создайте новый узел с вставляемым значением. В противном случае, вызовите приватный метод `_insert_struct_helper`, передав ему `self.top_element` и значение.
- Создайте приватный метод `_insert_struct_helper` в классе `BinSearchStructure`, который рекурсивно вставляет значение в дерево. Если значение строго меньше значения текущего узла, вставьте его в левое поддерево. Если значение больше или равно значению текущего узла, вставьте его в правое поддерево.
- Создайте публичный метод `find_struct` в классе `BinSearchStructure`, который ищет значение в дереве. Если дерево пустое, верните `None`. В противном случае, вызовите приватный метод `_find_struct_helper`, передав ему `self.top_element` и искомое значение.

- (f) Создайте приватный метод `_find_struct_helper` в классе `BinSearchStructure`, который рекурсивно ищет значение в дереве. Если текущий узел равен `None` или значение текущего узла равно искомому значению, верните текущий узел. В противном случае, рекурсивно вызывайте метод `_find_struct_helper` для поиска значения в левом поддереве (если искомое значение меньше текущего) или в правом поддереве (если искомое значение больше или равно текущему).
- (g) Создайте экземпляр класса `BinSearchStructure` и вставьте в него 22 случайных числа от 2 до 42.
- (h) Выполните поиск элементов в дереве и выведите результаты на экран.

Пример использования:

```
bss = BinSearchStructure()
for i in range(22):
    bss.insert_struct(random.randint(2, 42))

print("Поиск элементов:")
print(bss.find_struct(15)) # Обнаружено, возвращен узел (15)
print(bss.find_struct(55)) # Не обнаружено, возвращено None
print(bss.find_struct(35)) # Обнаружено, возвращен узел (35)
```

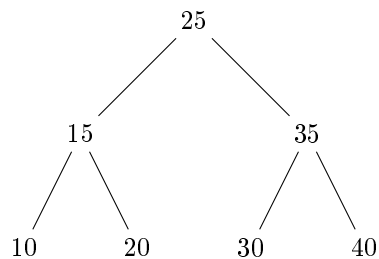


Рис. 10: Пример бинарного дерева поиска

11. Написать программу на Python, которая реализует бинарное дерево поиска с инкапсуляцией. Программа должна создавать экземпляры класса `NodeElement`, которые представляют узлы дерева, и класса `TreeIndex`, который представляет дерево поиска. Класс `TreeIndex` должен содержать методы для вставки, поиска и удаления элементов, при этом все рекурсивные методы должны быть приватными. Программа также должна создавать дерево поиска, вставлять в него случайные числа и выполнять поиск элементов в дереве.

Инструкции:

- (a) Создайте класс `NodeElement` с методом `__init__`, который принимает параметр `elem_value` и сохраняет его в атрибуте `self.index_key`. Атрибуты `self.left` и `self.right` должны быть инициализированы как `None`.
- (b) Создайте класс `TreeIndex` с методом `__init__`, который инициализирует атрибут `self.root_elem` как `None`.
- (c) Создайте публичный метод `add_key` в классе `TreeIndex`, который вставляет значение в дерево. Если `self.root_elem` отсутствует, создайте новый узел с вставляемым значением. В противном случае, вызовите приватный метод `_add_key_rec`, передав ему `self.root_elem` и значение.

- (d) Создайте приватный метод `_add_key_rec` в классе `TreeIndex`, который рекурсивно вставляет значение в дерево. Если значение меньше или равно значению текущего узла, вставьте его в левое поддерево. Если значение строго больше значения текущего узла, вставьте его в правое поддерево.
- (e) Создайте публичный метод `get_key` в классе `TreeIndex`, который ищет значение в дереве. Если дерево пустое, верните `None`. В противном случае, вызовите приватный метод `_get_key_rec`, передав ему `self.root_elem` и искомое значение.
- (f) Создайте приватный метод `_get_key_rec` в классе `TreeIndex`, который рекурсивно ищет значение в дереве. Если текущий узел равен `None` или значение текущего узла равно искомому значению, верните текущий узел. В противном случае, рекурсивно вызывайте метод `_get_key_rec` для поиска значения в левом поддереве (если искомое значение меньше или равно текущему) или в правом поддереве (если искомое значение больше).
- (g) Создайте экземпляр класса `TreeIndex` и вставьте в него 23 случайных числа от 3 до 43.
- (h) Выполните поиск элементов в дереве и выведите результаты на экран.

Пример использования:

```
ti = TreeIndex()
for i in range(23):
    ti.add_key(random.randint(3, 43))

print("Поиск элементов:")
print(ti.get_key(16)) # Обнаружено, возвращен узел (16)
print(ti.get_key(56)) # Не обнаружено, возвращено None
print(ti.get_key(36)) # Обнаружено, возвращен узел (36)
```

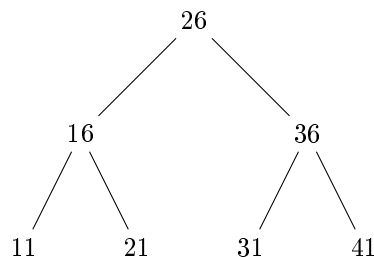


Рис. 11: Пример бинарного дерева поиска

12. Написать программу на Python, которая реализует бинарное дерево поиска с инкапсуляцией. Программа должна создавать экземпляры класса `BinElement`, которые представляют узлы дерева, и класса `IndexTree`, который представляет дерево поиска. Класс `IndexTree` должен содержать методы для вставки, поиска и удаления элементов, при этом все вспомогательные методы должны быть приватными. Программа также должна создавать дерево поиска, вставлять в него случайные числа и выполнять поиск элементов в дереве.

Инструкции:

- (a) Создайте класс `BinElement` с методом `__init__`, который принимает параметр `bin_val` и сохраняет его в атрибуте `self.key_value`. Атрибуты `self.left_node` и `self.right_node` должны быть инициализированы как `None`.

- (b) Создайте класс `IndexTree` с методом `__init__`, который инициализирует атрибут `self.first_element` как `None`.
- (c) Создайте публичный метод `insert_key` в классе `IndexTree`, который вставляет значение в дерево. Если `self.first_element` отсутствует, создайте новый узел с вставляемым значением. В противном случае, вызовите приватный метод `_insert_key_helper`, передав ему `self.first_element` и значение.
- (d) Создайте приватный метод `_insert_key_helper` в классе `IndexTree`, который рекурсивно вставляет значение в дерево. Если значение строго меньше значения текущего узла, вставьте его в левое поддерево. Если значение больше или равно значению текущего узла, вставьте его в правое поддерево.
- (e) Создайте публичный метод `search_key` в классе `IndexTree`, который ищет значение в дереве. Если дерево пустое, верните `None`. В противном случае, вызовите приватный метод `_search_key_helper`, передав ему `self.first_element` и искомое значение.
- (f) Создайте приватный метод `_search_key_helper` в классе `IndexTree`, который рекурсивно ищет значение в дереве. Если текущий узел равен `None` или значение текущего узла равно искомому значению, верните текущий узел. В противном случае, рекурсивно вызывайте метод `_search_key_helper` для поиска значения в левом поддереве (если искомое значение меньше текущего) или в правом поддереве (если искомое значение больше или равно текущему).
- (g) Создайте экземпляр класса `IndexTree` и вставьте в него 24 случайных числа от 4 до 44.
- (h) Выполните поиск элементов в дереве и выведите результаты на экран.

Пример использования:

```
it = IndexTree()
for i in range(24):
    it.insert_key(random.randint(4, 44))

print("Поиск элементов:")
print(it.search_key(17)) # Обнаружено, возвращен узел (17)
print(it.search_key(57)) # Не обнаружено, возвращено None
print(it.search_key(37)) # Обнаружено, возвращен узел (37)
```

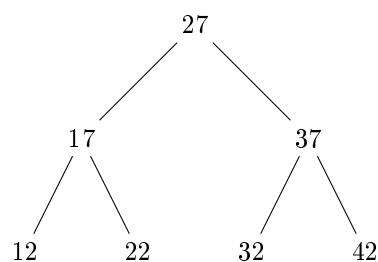


Рис. 12: Пример бинарного дерева поиска

13. Написать программу на Python, которая реализует бинарное дерево поиска с инкапсуляцией. Программа должна создавать экземпляры класса `SearchNode`, которые представляют узлы дерева, и класса `BinaryTreeIndex`, который представляет дерево поиска. Класс `BinaryTreeIndex` должен содержать методы для вставки, поиска и удаления

элементов, при этом все рекурсивные методы должны быть приватными. Программа также должна создавать дерево поиска, вставлять в него случайные числа и выполнять поиск элементов в дереве.

Инструкции:

- (a) Создайте класс `SearchNode` с методом `__init__`, который принимает параметр `search_val` и сохраняет его в атрибуте `self.node_key`. Атрибуты `self.left_child` и `self.right_child` должны быть инициализированы как `None`.
- (b) Создайте класс `BinaryTreeIndex` с методом `__init__`, который инициализирует атрибут `self.initial_element` как `None`.
- (c) Создайте публичный метод `add_node` в классе `BinaryTreeIndex`, который вставляет значение в дерево. Если `self.initial_element` отсутствует, создайте новый узел с вставляемым значением. В противном случае, вызовите приватный метод `_add_node_recursive`, передав ему `self.initial_element` и значение.
- (d) Создайте приватный метод `_add_node_recursive` в классе `BinaryTreeIndex`, который рекурсивно вставляет значение в дерево. Если значение меньше или равно значению текущего узла, вставьте его в левое поддерево. Если значение строго больше значения текущего узла, вставьте его в правое поддерево.
- (e) Создайте публичный метод `find_node` в классе `BinaryTreeIndex`, который ищет значение в дереве. Если дерево пустое, верните `None`. В противном случае, вызовите приватный метод `_find_node_recursive`, передав ему `self.initial_element` и искомое значение.
- (f) Создайте приватный метод `_find_node_recursive` в классе `BinaryTreeIndex`, который рекурсивно ищет значение в дереве. Если текущий узел равен `None` или значение текущего узла равно искомому значению, верните текущий узел. В противном случае, рекурсивно вызывайте метод `_find_node_recursive` для поиска значения в левом поддереве (если искомое значение меньше или равно текущему) или в правом поддереве (если искомое значение больше).
- (g) Создайте экземпляр класса `BinaryTreeIndex` и вставьте в него 25 случайных чисел от 5 до 45.
- (h) Выполните поиск элементов в дереве и выведите результаты на экран.

Пример использования:

```
bti = BinaryTreeIndex()
for i in range(25):
    bti.add_node(random.randint(5, 45))

print("Поиск элементов:")
print(bti.find_node(18)) # Обнаружено, возвращен узел (18)
print(bti.find_node(58)) # Не обнаружено, возвращено None
print(bti.find_node(38)) # Обнаружено, возвращен узел (38)
```

14. Написать программу на Python, которая реализует бинарное дерево поиска с инкапсуляцией. Программа должна создавать экземпляры класса `IndexNode`, которые представляют узлы дерева, и класса `SearchStructure`, который представляет дерево поиска. Класс `SearchStructure` должен содержать методы для вставки, поиска и удаления элементов, при этом все вспомогательные методы должны быть приватными. Программа также должна создавать дерево поиска, вставлять в него случайные числа и выполнять поиск элементов в дереве.

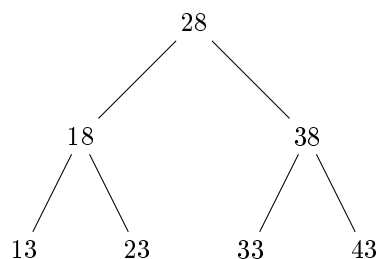


Рис. 13: Пример бинарного дерева поиска

Инструкции:

- Создайте класс `IndexNode` с методом `__init__`, который принимает параметр `idx_value` и сохраняет его в атрибуте `self.element_key`. Атрибуты `self.left_elem` и `self.right_elem` должны быть инициализированы как `None`.
- Создайте класс `SearchStructure` с методом `__init__`, который инициализирует атрибут `self.start_element` как `None`.
- Создайте публичный метод `insert_elem` в классе `SearchStructure`, который вставляет значение в дерево. Если `self.start_element` отсутствует, создайте новый узел с вставляемым значением. В противном случае, вызовите приватный метод `_insert_elem_rec`, передав ему `self.start_element` и значение.
- Создайте приватный метод `_insert_elem_rec` в классе `SearchStructure`, который рекурсивно вставляет значение в дерево. Если значение строго меньше значения текущего узла, вставьте его в левое поддерево. Если значение больше или равно значению текущего узла, вставьте его в правое поддерево.
- Создайте публичный метод `locate_elem` в классе `SearchStructure`, который ищет значение в дереве. Если дерево пустое, верните `None`. В противном случае, вызовите приватный метод `_locate_elem_rec`, передав ему `self.start_element` и искомое значение.
- Создайте приватный метод `_locate_elem_rec` в классе `SearchStructure`, который рекурсивно ищет значение в дереве. Если текущий узел равен `None` или значение текущего узла равно искомому значению, верните текущий узел. В противном случае, рекурсивно вызывайте метод `_locate_elem_rec` для поиска значения в левом поддереве (если искомое значение меньше текущего) или в правом поддереве (если искомое значение больше или равно текущему).
- Создайте экземпляр класса `SearchStructure` и вставьте в него 26 случайных чисел от 6 до 46.
- Выполните поиск элементов в дереве и выведите результаты на экран.

Пример использования:

```

ss = SearchStructure()
for i in range(26):
    ss.insert_elem(random.randint(6, 46))

print("Поиск элементов:")
print(ss.locate_elem(19)) # Обнаружено, возвращен узел (19)
print(ss.locate_elem(59)) # Не обнаружено, возвращено None
print(ss.locate_elem(39)) # Обнаружено, возвращен узел (39)
  
```

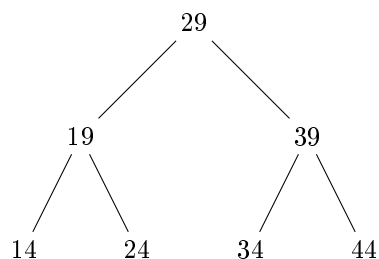


Рис. 14: Пример бинарного дерева поиска

15. Написать программу на Python, которая реализует бинарное дерево поиска с инкапсуляцией. Программа должна создавать экземпляры класса `KeyValueNode`, которые представляют узлы дерева, и класса `BinaryTreeMap`, который представляет дерево поиска. Класс `BinaryTreeMap` должен содержать методы для вставки, поиска и удаления элементов, при этом все рекурсивные методы должны быть приватными. Программа также должна создавать дерево поиска, вставлять в него случайные числа и выполнять поиск элементов в дереве.

Инструкции:

- Создайте класс `KeyValueNode` с методом `__init__`, который принимает параметр `key_val` и сохраняет его в атрибуте `self.map_key`. Атрибуты `self.left_branch` и `self.right_branch` должны быть инициализированы как `None`.
- Создайте класс `BinaryTreeMap` с методом `__init__`, который инициализирует атрибут `self.root_key` как `None`.
- Создайте публичный метод `put_key` в классе `BinaryTreeMap`, который вставляет значение в дерево. Если `self.root_key` отсутствует, создайте новый узел с вставляемым значением. В противном случае, вызовите приватный метод `_put_key_helper`, передав ему `self.root_key` и значение.
- Создайте приватный метод `_put_key_helper` в классе `BinaryTreeMap`, который рекурсивно вставляет значение в дерево. Если значение меньше или равно значению текущего узла, вставьте его в левое поддерево. Если значение строго больше значения текущего узла, вставьте его в правое поддерево.
- Создайте публичный метод `get_key` в классе `BinaryTreeMap`, который ищет значение в дереве. Если дерево пустое, верните `None`. В противном случае, вызовите приватный метод `_get_key_helper`, передав ему `self.root_key` и искомое значение.
- Создайте приватный метод `_get_key_helper` в классе `BinaryTreeMap`, который рекурсивно ищет значение в дереве. Если текущий узел равен `None` или значение текущего узла равно искомому значению, верните текущий узел. В противном случае, рекурсивно вызывайте метод `_get_key_helper` для поиска значения в левом поддереве (если искомое значение меньше или равно текущему) или в правом поддереве (если искомое значение больше).
- Создайте экземпляр класса `BinaryTreeMap` и вставьте в него 27 случайных чисел от 7 до 47.
- Выполните поиск элементов в дереве и выведите результаты на экран.

Пример использования:


```

btm = BinaryTreeMap()
for i in range(27):
    btm.put_key(random.randint(7, 47))

print("Поиск элементов:")
print(btm.get_key(20))    # Обнаружено, возвращен узел (20)
print(btm.get_key(60))    # Не обнаружено, возвращено None
print(btm.get_key(40))    # Обнаружено, возвращен узел (40)

```

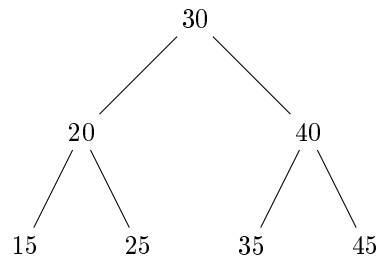


Рис. 15: Пример бинарного дерева поиска

16. Написать программу на Python, которая реализует бинарное дерево поиска с инкапсуляцией. Программа должна создавать экземпляры класса `MapNode`, которые представляют узлы дерева, и класса `KeyTree`, который представляет дерево поиска. Класс `KeyTree` должен содержать методы для вставки, поиска и удаления элементов, при этом все вспомогательные методы должны быть приватными. Программа также должна создавать дерево поиска, вставлять в него случайные числа и выполнять поиск элементов в дереве.

Инструкции:

- Создайте класс `MapNode` с методом `__init__`, который принимает параметр `map_value` и сохраняет его в атрибуте `self.tree_key`. Атрибуты `self.left_part` и `self.right_part` должны быть инициализированы как `None`.
- Создайте класс `KeyTree` с методом `__init__`, который инициализирует атрибут `self.base_key` как `None`.
- Создайте публичный метод `insert_map` в классе `KeyTree`, который вставляет значение в дерево. Если `self.base_key` отсутствует, создайте новый узел с вставляемым значением. В противном случае, вызовите приватный метод `_insert_map_rec`, передав ему `self.base_key` и значение.
- Создайте приватный метод `_insert_map_rec` в классе `KeyTree`, который рекурсивно вставляет значение в дерево. Если значение строго меньше значения текущего узла, вставьте его в левое поддерево. Если значение больше или равно значению текущего узла, вставьте его в правое поддерево.
- Создайте публичный метод `search_map` в классе `KeyTree`, который ищет значение в дереве. Если дерево пустое, верните `None`. В противном случае, вызовите приватный метод `_search_map_rec`, передав ему `self.base_key` и искомое значение.
- Создайте приватный метод `_search_map_rec` в классе `KeyTree`, который рекурсивно ищет значение в дереве. Если текущий узел равен `None` или значение текущего узла равно искомому значению, верните текущий узел. В противном случае,

рекурсивно вызывайте метод `_search_map_гес` для поиска значения в левом поддереве (если искомое значение меньше текущего) или в правом поддереве (если искомое значение больше или равно текущему).

- (g) Создайте экземпляр класса `KeyTree` и вставьте в него 28 случайных чисел от 8 до 48.
- (h) Выполните поиск элементов в дереве и выведите результаты на экран.

Пример использования:

```
kt = KeyTree()
for i in range(28):
    kt.insert_map(random.randint(8, 48))

print("Поиск элементов:")
print(kt.search_map(21)) # Обнаружено, возвращен узел (21)
print(kt.search_map(61)) # Не обнаружено, возвращено None
print(kt.search_map(41)) # Обнаружено, возвращен узел (41)
```

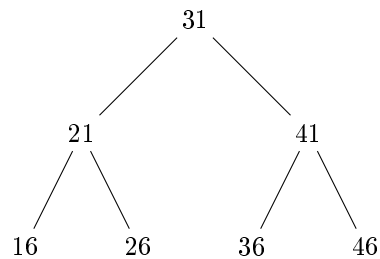


Рис. 16: Пример бинарного дерева поиска

17. Написать программу на Python, которая реализует бинарное дерево поиска с инкапсуляцией. Программа должна создавать экземпляры класса `TreeKeyNode`, которые представляют узлы дерева, и класса `ValueTree`, который представляет дерево поиска. Класс `ValueTree` должен содержать методы для вставки, поиска и удаления элементов, при этом все рекурсивные методы должны быть приватными. Программа также должна создавать дерево поиска, вставлять в него случайные числа и выполнять поиск элементов в дереве.

Инструкции:

- (a) Создайте класс `TreeKeyNode` с методом `__init__`, который принимает параметр `tree_key_val` и сохраняет его в атрибуте `self.value_key`. Атрибуты `self.left` и `self.right` должны быть инициализированы как `None`.
- (b) Создайте класс `ValueTree` с методом `__init__`, который инициализирует атрибут `self.first_key` как `None`.
- (c) Создайте публичный метод `add_value` в классе `ValueTree`, который вставляет значение в дерево. Если `self.first_key` отсутствует, создайте новый узел с вставляемым значением. В противном случае, вызовите приватный метод `_add_value_helper`, передав ему `self.first_key` и значение.
- (d) Создайте приватный метод `_add_value_helper` в классе `ValueTree`, который рекурсивно вставляет значение в дерево. Если значение меньше или равно значению

текущего узла, вставьте его в левое поддерево. Если значение строго больше значения текущего узла, вставьте его в правое поддерево.

- (e) Создайте публичный метод `retrieve_value` в классе `ValueTree`, который ищет значение в дереве. Если дерево пустое, верните `None`. В противном случае, вызовите приватный метод `_retrieve_value_helper`, передав ему `self.first_key` и искомое значение.
- (f) Создайте приватный метод `_retrieve_value_helper` в классе `ValueTree`, который рекурсивно ищет значение в дереве. Если текущий узел равен `None` или значение текущего узла равно искомому значению, верните текущий узел. В противном случае, рекурсивно вызывайте метод `_retrieve_value_helper` для поиска значения в левом поддереве (если искомое значение меньше или равно текущему) или в правом поддереве (если искомое значение больше).
- (g) Создайте экземпляр класса `ValueTree` и вставьте в него 29 случайных чисел от 9 до 49.
- (h) Выполните поиск элементов в дереве и выведите результаты на экран.

Пример использования:

```
vt = ValueTree()
for i in range(29):
    vt.add_value(random.randint(9, 49))

print("Поиск элементов:")
print(vt.retrieve_value(22)) # Обнаружено, возвращен узел (22)
print(vt.retrieve_value(62)) # Не обнаружено, возвращено None
print(vt.retrieve_value(42)) # Обнаружено, возвращен узел (42)
```

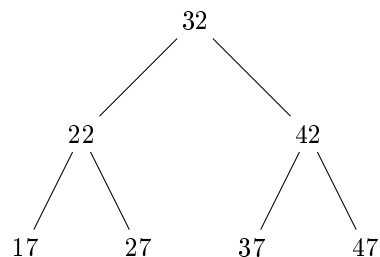


Рис. 17: Пример бинарного дерева поиска

18. Написать программу на Python, которая реализует бинарное дерево поиска с инкапсуляцией. Программа должна создавать экземпляры класса `ValueNode`, которые представляют узлы дерева, и класса `KeyedTree`, который представляет дерево поиска. Класс `KeyedTree` должен содержать методы для вставки, поиска и удаления элементов, при этом все вспомогательные методы должны быть приватными. Программа также должна создавать дерево поиска, вставлять в него случайные числа и выполнять поиск элементов в дереве.

Инструкции:

- (a) Создайте класс `ValueNode` с методом `__init__`, который принимает параметр `node_value` и сохраняет его в атрибуте `self.keyed_value`. Атрибуты `self.left_side` и `self.right_side` должны быть инициализированы как `None`.

- (b) Создайте класс `KeyedTree` с методом `__init__`, который инициализирует атрибут `self.start_key` как `None`.
- (c) Создайте публичный метод `store_value` в классе `KeyedTree`, который вставляет значение в дерево. Если `self.start_key` отсутствует, создайте новый узел с вставляемым значением. В противном случае, вызовите приватный метод `_store_value_rec`, передав ему `self.start_key` и значение.
- (d) Создайте приватный метод `_store_value_rec` в классе `KeyedTree`, который рекурсивно вставляет значение в дерево. Если значение строго меньше значения текущего узла, вставьте его в левое поддерево. Если значение больше или равно значению текущего узла, вставьте его в правое поддерево.
- (e) Создайте публичный метод `fetch_value` в классе `KeyedTree`, который ищет значение в дереве. Если дерево пустое, верните `None`. В противном случае, вызовите приватный метод `_fetch_value_rec`, передав ему `self.start_key` и искомое значение.
- (f) Создайте приватный метод `_fetch_value_rec` в классе `KeyedTree`, который рекурсивно ищет значение в дереве. Если текущий узел равен `None` или значение текущего узла равно искомому значению, верните текущий узел. В противном случае, рекурсивно вызывайте метод `_fetch_value_rec` для поиска значения в левом поддереве (если искомое значение меньше текущего) или в правом поддереве (если искомое значение больше или равно текущему).
- (g) Создайте экземпляр класса `KeyedTree` и вставьте в него 30 случайных чисел от 10 до 50.
- (h) Выполните поиск элементов в дереве и выведите результаты на экран.

Пример использования:

```
kt = KeyedTree()
for i in range(30):
    kt.store_value(random.randint(10, 50))

print("Поиск элементов:")
print(kt.fetch_value(23)) # Обнаружено, возвращен узел (23)
print(kt.fetch_value(63)) # Не обнаружено, возвращено None
print(kt.fetch_value(43)) # Обнаружено, возвращен узел (43)
```

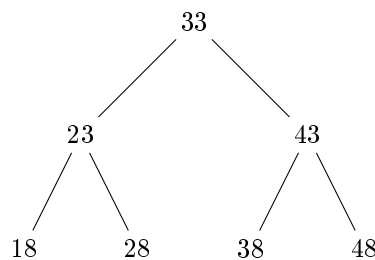


Рис. 18: Пример бинарного дерева поиска

19. Написать программу на Python, которая реализует бинарное дерево поиска с инкапсуляцией. Программа должна создавать экземпляры класса `KeyedNode`, которые представляют узлы дерева, и класса `ValuedTree`, который представляет дерево поиска.

Класс `ValuedTree` должен содержать методы для вставки, поиска и удаления элементов, при этом все рекурсивные методы должны быть приватными. Программа также должна создавать дерево поиска, вставлять в него случайные числа и выполнять поиск элементов в дереве.

Инструкции:

- (a) Создайте класс `KeyedNode` с методом `__init__`, который принимает параметр `keyed_val` и сохраняет его в атрибуте `self.node_content`. Атрибуты `self.left_path` и `self.right_path` должны быть инициализированы как `None`.
- (b) Создайте класс `ValuedTree` с методом `__init__`, который инициализирует атрибут `self.root_content` как `None`.
- (c) Создайте публичный метод `insert_content` в классе `ValuedTree`, который вставляет значение в дерево. Если `self.root_content` отсутствует, создайте новый узел с вставляемым значением. В противном случае, вызовите приватный метод `_insert_content_helper`, передав ему `self.root_content` и значение.
- (d) Создайте приватный метод `_insert_content_helper` в классе `ValuedTree`, который рекурсивно вставляет значение в дерево. Если значение меньше или равно значению текущего узла, вставьте его в левое поддерево. Если значение строго больше значения текущего узла, вставьте его в правое поддерево.
- (e) Создайте публичный метод `search_content` в классе `ValuedTree`, который ищет значение в дереве. Если дерево пустое, верните `None`. В противном случае, вызовите приватный метод `_search_content_helper`, передав ему `self.root_content` и искомое значение.
- (f) Создайте приватный метод `_search_content_helper` в классе `ValuedTree`, который рекурсивно ищет значение в дереве. Если текущий узел равен `None` или значение текущего узла равно искомому значению, верните текущий узел. В противном случае, рекурсивно вызывайте метод `_search_content_helper` для поиска значения в левом поддереве (если искомое значение меньше или равно текущему) или в правом поддереве (если искомое значение больше).
- (g) Создайте экземпляр класса `ValuedTree` и вставьте в него 31 случайное число от 11 до 51.
- (h) Выполните поиск элементов в дереве и выведите результаты на экран.

Пример использования:

```
vt = ValuedTree()
for i in range(31):
    vt.insert_content(random.randint(11, 51))

print("Поиск элементов:")
print(vt.search_content(24)) # Обнаружено, возвращен узел (24)
print(vt.search_content(64)) # Не обнаружено, возвращено None
print(vt.search_content(44)) # Обнаружено, возвращен узел (44)
```

20. Написать программу на Python, которая реализует бинарное дерево поиска с инкапсуляцией. Программа должна создавать экземпляры класса `ContentNode`, которые представляют узлы дерева, и класса `KeyTreeStructure`, который представляет дерево поиска. Класс `KeyTreeStructure` должен содержать методы для вставки, поиска и

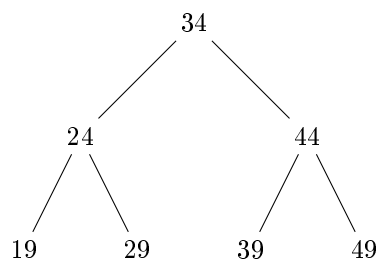


Рис. 19: Пример бинарного дерева поиска

удаления элементов, при этом все вспомогательные методы должны быть приватными. Программа также должна создавать дерево поиска, вставлять в него случайные числа и выполнять поиск элементов в дереве.

Инструкции:

- Создайте класс `ContentNode` с методом `__init__`, который принимает параметр `content_val` и сохраняет его в атрибуте `self.node_data`. Атрибуты `self.left_item` и `self.right_item` должны быть инициализированы как `None`.
- Создайте класс `KeyTreeStructure` с методом `__init__`, который инициализирует атрибут `self.top_data` как `None`.
- Создайте публичный метод `add_data` в классе `KeyTreeStructure`, который вставляет значение в дерево. Если `self.top_data` отсутствует, создайте новый узел с вставляемым значением. В противном случае, вызовите приватный метод `_add_data_rec`, передав ему `self.top_data` и значение.
- Создайте приватный метод `_add_data_rec` в классе `KeyTreeStructure`, который рекурсивно вставляет значение в дерево. Если значение строго меньше значения текущего узла, вставьте его в левое поддерево. Если значение больше или равно значению текущего узла, вставьте его в правое поддерево.
- Создайте публичный метод `find_data` в классе `KeyTreeStructure`, который ищет значение в дереве. Если дерево пустое, верните `None`. В противном случае, вызовите приватный метод `_find_data_rec`, передав ему `self.top_data` и искомое значение.
- Создайте приватный метод `_find_data_rec` в классе `KeyTreeStructure`, который рекурсивно ищет значение в дереве. Если текущий узел равен `None` или значение текущего узла равно искомому значению, верните текущий узел. В противном случае, рекурсивно вызывайте метод `_find_data_rec` для поиска значения в левом поддереве (если искомое значение меньше текущего) или в правом поддереве (если искомое значение больше или равно текущему).
- Создайте экземпляр класса `KeyTreeStructure` и вставьте в него 32 случайных числа от 12 до 52.
- Выполните поиск элементов в дереве и выведите результаты на экран.

Пример использования:

```

kts = KeyTreeStructure()
for i in range(32):
    kts.add_data(random.randint(12, 52))

```

```

print("Поиск элементов:")
print(fts.find_data(25)) # Обнаружено, возвращен узел (25)
print(fts.find_data(65)) # Не обнаружено, возвращено None
print(fts.find_data(45)) # Обнаружено, возвращен узел (45)

```

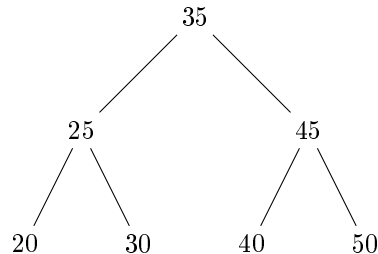


Рис. 20: Пример бинарного дерева поиска

21. Написать программу на Python, которая реализует бинарное дерево поиска с инкапсуляцией. Программа должна создавать экземпляры класса `TreeNode`, которые представляют узлы дерева, и класса `ContentTree`, который представляет дерево поиска. Класс `ContentTree` должен содержать методы для вставки, поиска и удаления элементов, при этом все рекурсивные методы должны быть приватными. Программа также должна создавать дерево поиска, вставлять в него случайные числа и выполнять поиск элементов в дереве.

Инструкции:

- Создайте класс `TreeNode` с методом `__init__`, который принимает параметр `data_value` и сохраняет его в атрибуте `self.data_value`. Атрибуты `self.left_child` и `self.right_child` должны быть инициализированы как `None`.
- Создайте класс `ContentTree` с методом `__init__`, который инициализирует атрибут `self.root` как `None`.
- Создайте публичный метод `insert` в классе `ContentTree`, который вставляет значение в дерево. Если `self.root` отсутствует, создайте новый узел с вставляемым значением. В противном случае, вызовите приватный метод `_insert_helper`, передав ему `self.root` и значение.
- Создайте приватный метод `_insert_helper` в классе `ContentTree`, который рекурсивно вставляет значение в дерево. Если значение меньше или равно значению текущего узла, вставьте его в левое поддерево. Если значение строго больше значения текущего узла, вставьте его в правое поддерево.
- Создайте публичный метод `search` в классе `ContentTree`, который ищет значение в дереве. Если дерево пустое, верните `None`. В противном случае, вызовите приватный метод `_search_helper`, передав ему `self.root` и искомое значение.
- Создайте приватный метод `_search_helper` в классе `ContentTree`, который рекурсивно ищет значение в дереве. Если текущий узел равен `None` или значение текущего узла равно искомому значению, верните текущий узел. В противном случае, рекурсивно вызывайте метод `_search_helper` для поиска значения.

в левом поддереве (если искомое значение меньше или равно текущему) или в правом поддереве (если искомое значение больше).

(g) Создайте экземпляр класса `ContentTree` и вставьте в него 33 случайных числа от 13 до 53.

(h) Выполните поиск элементов в дереве и выведите результаты на экран.

Пример использования:

```
ct = ContentTree()
for i in range(33):
    ct.insert_entry(random.randint(13, 53))

print("Поиск элементов:")
print(ct.search_entry(26)) # Обнаружено, возвращен узел (26)
print(ct.search_entry(66)) # Не обнаружено, возвращено None
print(ct.search_entry(46)) # Обнаружено, возвращен узел (46)
```

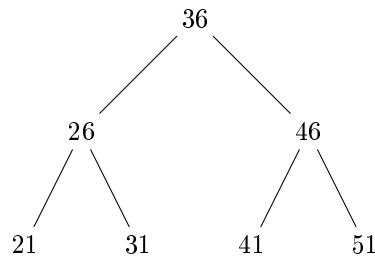


Рис. 21: Пример бинарного дерева поиска

22. Написать программу на Python, которая реализует бинарное дерево поиска с инкапсуляцией. Программа должна создавать экземпляры класса `EntryNode`, которые представляют узлы дерева, и класса `DataStructureTree`, который представляет дерево поиска. Класс `DataStructureTree` должен содержать методы для вставки, поиска и удаления элементов, при этом все вспомогательные методы должны быть приватными. Программа также должна создавать дерево поиска, вставлять в него случайные числа и выполнять поиск элементов в дереве.

Инструкции:

- Создайте класс `EntryNode` с методом `__init__`, который принимает параметр `entry_val` и сохраняет его в атрибуте `self.content_item`. Атрибуты `self.left_data` и `self.right_data` должны быть инициализированы как `None`.
- Создайте класс `DataStructureTree` с методом `__init__`, который инициализирует атрибут `self.first_item` как `None`.
- Создайте публичный метод `add_item` в классе `DataStructureTree`, который вставляет значение в дерево. Если `self.first_item` отсутствует, создайте новый узел с вставляемым значением. В противном случае, вызовите приватный метод `_add_item_rec`, передав ему `self.first_item` и значение.
- Создайте приватный метод `_add_item_rec` в классе `DataStructureTree`, который рекурсивно вставляет значение в дерево. Если значение строго меньше значения текущего узла, вставьте его в левое поддерево. Если значение больше или равно значению текущего узла, вставьте его в правое поддерево.

- (e) Создайте публичный метод `locate_item` в классе `DataStructureTree`, который ищет значение в дереве. Если дерево пустое, верните `None`. В противном случае, вызовите приватный метод `_locate_item_rec`, передав ему `self.first_item` и искомое значение.
- (f) Создайте приватный метод `_locate_item_rec` в классе `DataStructureTree`, который рекурсивно ищет значение в дереве. Если текущий узел равен `None` или значение текущего узла равно искомому значению, верните текущий узел. В противном случае, рекурсивно вызывайте метод `_locate_item_rec` для поиска значения в левом поддереве (если искомое значение меньше текущего) или в правом поддереве (если искомое значение больше или равно текущему).
- (g) Создайте экземпляр класса `DataStructureTree` и вставьте в него 34 случайных числа от 14 до 54.
- (h) Выполните поиск элементов в дереве и выведите результаты на экран.

Пример использования:

```
dst = DataStructureTree()
for i in range(34):
    dst.add_item(random.randint(14, 54))

print("Поиск элементов:")
print(dst.locate_item(27)) # Обнаружено, возвращен узел (27)
print(dst.locate_item(67)) # Не обнаружено, возвращено None
print(dst.locate_item(47)) # Обнаружено, возвращен узел (47)
```

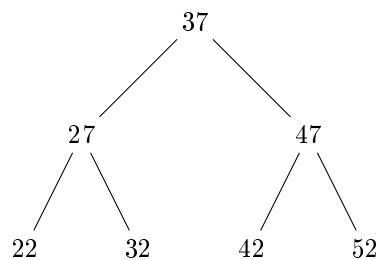


Рис. 22: Пример бинарного дерева поиска

23. Написать программу на Python, которая реализует бинарное дерево поиска с инкапсуляцией. Программа должна создавать экземпляры класса `ItemNode`, которые представляют узлы дерева, и класса `EntryTree`, который представляет дерево поиска. Класс `EntryTree` должен содержать методы для вставки, поиска и удаления элементов, при этом все рекурсивные методы должны быть приватными. Программа также должна создавать дерево поиска, вставлять в него случайные числа и выполнять поиск элементов в дереве.

Инструкции:

- (a) Создайте класс `ItemNode` с методом `__init__`, который принимает параметр `item_value` и сохраняет его в атрибуте `self.data_entry`. Атрибуты `self.left_position` и `self.right_position` должны быть инициализированы как `None`.
- (b) Создайте класс `EntryTree` с методом `__init__`, который инициализирует атрибут `self.root_entry` как `None`.

- (c) Создайте публичный метод `insert_position` в классе `EntryTree`, который вставляет значение в дерево. Если `self.root_entry` отсутствует, создайте новый узел с вставляемым значением. В противном случае, вызовите приватный метод `_insert_position_helper`, передав ему `self.root_entry` и значение.
- (d) Создайте приватный метод `_insert_position_helper` в классе `EntryTree`, который рекурсивно вставляет значение в дерево. Если значение меньше или равно значению текущего узла, вставьте его в левое поддерево. Если значение строго больше значения текущего узла, вставьте его в правое поддерево.
- (e) Создайте публичный метод `find_position` в классе `EntryTree`, который ищет значение в дереве. Если дерево пустое, верните `None`. В противном случае, вызовите приватный метод `_find_position_helper`, передав ему `self.root_entry` и искомое значение.
- (f) Создайте приватный метод `_find_position_helper` в классе `EntryTree`, который рекурсивно ищет значение в дереве. Если текущий узел равен `None` или значение текущего узла равно искомому значению, верните текущий узел. В противном случае, рекурсивно вызывайте метод `_find_position_helper` для поиска значения в левом поддереве (если искомое значение меньше или равно текущему) или в правом поддереве (если искомое значение больше).
- (g) Создайте экземпляр класса `EntryTree` и вставьте в него 35 случайных чисел от 15 до 55.
- (h) Выполните поиск элементов в дереве и выведите результаты на экран.

Пример использования:

```
et = EntryTree()
for i in range(35):
    et.insert_position(random.randint(15, 55))

print("Поиск элементов:")
print(et.find_position(28))    # Обнаружено, возвращен узел (28)
print(et.find_position(68))    # Не обнаружено, возвращено None
print(et.find_position(48))    # Обнаружено, возвращен узел (48)
```

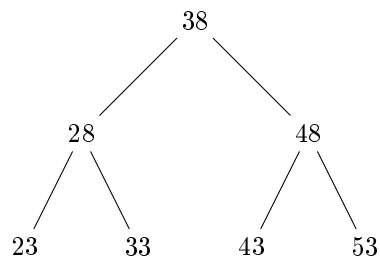


Рис. 23: Пример бинарного дерева поиска

24. Написать программу на Python, которая реализует бинарное дерево поиска с инкапсуляцией. Программа должна создавать экземпляры класса `PositionNode`, которые представляют узлы дерева, и класса `ItemStructure`, который представляет дерево поиска. Класс `ItemStructure` должен содержать методы для вставки, поиска и удаления

элементов, при этом все вспомогательные методы должны быть приватными. Программа также должна создавать дерево поиска, вставлять в него случайные числа и выполнять поиск элементов в дереве.

Инструкции:

- (a) Создайте класс `PositionNode` с методом `__init__`, который принимает параметр `position_val` и сохраняет его в атрибуте `self.entry_data`. Атрибуты `self.left_slot` и `self.right_slot` должны быть инициализированы как `None`.
- (b) Создайте класс `ItemStructure` с методом `__init__`, который инициализирует атрибут `self.top_entry` как `None`.
- (c) Создайте публичный метод `add_slot` в классе `ItemStructure`, который вставляет значение в дерево. Если `self.top_entry` отсутствует, создайте новый узел с вставляемым значением. В противном случае, вызовите приватный метод `_add_slot_rec`, передав ему `self.top_entry` и значение.
- (d) Создайте приватный метод `_add_slot_rec` в классе `ItemStructure`, который рекурсивно вставляет значение в дерево. Если значение строго меньше значения текущего узла, вставьте его в левое поддерево. Если значение больше или равно значению текущего узла, вставьте его в правое поддерево.
- (e) Создайте публичный метод `search_slot` в классе `ItemStructure`, который ищет значение в дереве. Если дерево пустое, верните `None`. В противном случае, вызовите приватный метод `_search_slot_rec`, передав ему `self.top_entry` и искомое значение.
- (f) Создайте приватный метод `_search_slot_rec` в классе `ItemStructure`, который рекурсивно ищет значение в дереве. Если текущий узел равен `None` или значение текущего узла равно искомому значению, верните текущий узел. В противном случае, рекурсивно вызывайте метод `_search_slot_rec` для поиска значения в левом поддереве (если искомое значение меньше текущего) или в правом поддереве (если искомое значение больше или равно текущему).
- (g) Создайте экземпляр класса `ItemStructure` и вставьте в него 36 случайных чисел от 16 до 56.
- (h) Выполните поиск элементов в дереве и выведите результаты на экран.

Пример использования:

```
is_ = ItemStructure()
for i in range(36):
    is_.add_slot(random.randint(16, 56))

print("Поиск элементов:")
print(is_.search_slot(29)) # Обнаружено, возвращен узел (29)
print(is_.search_slot(69)) # Не обнаружено, возвращено None
print(is_.search_slot(49)) # Обнаружено, возвращен узел (49)
```

25. Написать программу на Python, которая реализует бинарное дерево поиска с инкапсуляцией. Программа должна создавать экземпляры класса `SlotNode`, которые представляют узлы дерева, и класса `PositionTree`, который представляет дерево поиска. Класс `PositionTree` должен содержать методы для вставки, поиска и удаления элементов, при этом все рекурсивные методы должны быть приватными. Программа также должна создавать дерево поиска, вставлять в него случайные числа и выполнять поиск элементов в дереве.

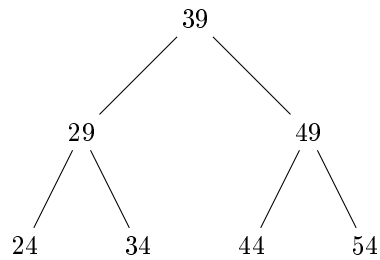


Рис. 24: Пример бинарного дерева поиска

Инструкции:

- Создайте класс `SlotNode` с методом `__init__`, который принимает параметр `slot_value` и сохраняет его в атрибуте `self.item_position`. Атрибуты `self.left_place` и `self.right_place` должны быть инициализированы как `None`.
- Создайте класс `PositionTree` с методом `__init__`, который инициализирует атрибут `self.first_position` как `None`.
- Создайте публичный метод `insert_place` в классе `PositionTree`, который вставляет значение в дерево. Если `self.first_position` отсутствует, создайте новый узел с вставляемым значением. В противном случае, вызовите приватный метод `_insert_place_helper`, передав ему `self.first_position` и значение.
- Создайте приватный метод `_insert_place_helper` в классе `PositionTree`, который рекурсивно вставляет значение в дерево. Если значение меньше или равно значению текущего узла, вставьте его в левое поддерево. Если значение строго больше значения текущего узла, вставьте его в правое поддерево.
- Создайте публичный метод `locate_place` в классе `PositionTree`, который ищет значение в дереве. Если дерево пустое, верните `None`. В противном случае, вызовите приватный метод `_locate_place_helper`, передав ему `self.first_position` и искомое значение.
- Создайте приватный метод `_locate_place_helper` в классе `PositionTree`, который рекурсивно ищет значение в дереве. Если текущий узел равен `None` или значение текущего узла равно искомому значению, верните текущий узел. В противном случае, рекурсивно вызывайте метод `_locate_place_helper` для поиска значения в левом поддереве (если искомое значение меньше или равно текущему) или в правом поддереве (если искомое значение больше).
- Создайте экземпляр класса `PositionTree` и вставьте в него 37 случайных чисел от 17 до 57.
- Выполните поиск элементов в дереве и выведите результаты на экран.

Пример использования:

```

pt = PositionTree()
for i in range(37):
    pt.insert_place(random.randint(17, 57))

print("Поиск элементов:")
print(pt.locate_place(30)) # Обнаружено, возвращен узел (30)
print(pt.locate_place(70)) # Не обнаружено, возвращено None
print(pt.locate_place(50)) # Обнаружено, возвращен узел (50)
  
```

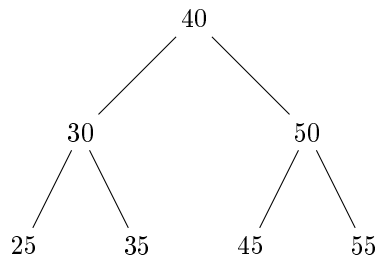


Рис. 25: Пример бинарного дерева поиска

26. Написать программу на Python, которая реализует бинарное дерево поиска с инкапсуляцией. Программа должна создавать экземпляры класса `PlaceNode`, которые представляют узлы дерева, и класса `SlotTree`, который представляет дерево поиска. Класс `SlotTree` должен содержать методы для вставки, поиска и удаления элементов, при этом все вспомогательные методы должны быть приватными. Программа также должна создавать дерево поиска, вставлять в него случайные числа и выполнять поиск элементов в дереве.

Инструкции:

- Создайте класс `PlaceNode` с методом `__init__`, который принимает параметр `place_val` и сохраняет его в атрибуте `self.position_item`. Атрибуты `self.left_spot` и `self.right_spot` должны быть инициализированы как `None`.
- Создайте класс `SlotTree` с методом `__init__`, который инициализирует атрибут `self.root_position` как `None`.
- Создайте публичный метод `add_spot` в классе `SlotTree`, который вставляет значение в дерево. Если `self.root_position` отсутствует, создайте новый узел с вставляемым значением. В противном случае, вызовите приватный метод `_add_spot_rec`, передав ему `self.root_position` и значение.
- Создайте приватный метод `_add_spot_rec` в классе `SlotTree`, который рекурсивно вставляет значение в дерево. Если значение строго меньше значения текущего узла, вставьте его в левое поддерево. Если значение больше или равно значению текущего узла, вставьте его в правое поддерево.
- Создайте публичный метод `find_spot` в классе `SlotTree`, который ищет значение в дереве. Если дерево пустое, верните `None`. В противном случае, вызовите приватный метод `_find_spot_rec`, передав ему `self.root_position` и искомое значение.
- Создайте приватный метод `_find_spot_rec` в классе `SlotTree`, который рекурсивно ищет значение в дереве. Если текущий узел равен `None` или значение текущего узла равно искомому значению, верните текущий узел. В противном случае, рекурсивно вызывайте метод `_find_spot_rec` для поиска значения в левом поддерево (если искомое значение меньше текущего) или в правом поддерево (если искомое значение больше или равно текущему).
- Создайте экземпляр класса `SlotTree` и вставьте в него 38 случайных чисел от 18 до 58.
- Выполните поиск элементов в дереве и выведите результаты на экран.

Пример использования:

```

st = SlotTree()
for i in range(38):
    st.add_spot(random.randint(18, 58))

print("Поиск элементов:")
print(st.find_spot(31)) # Обнаружено, возвращен узел (31)
print(st.find_spot(71)) # Не обнаружено, возвращено None
print(st.find_spot(51)) # Обнаружено, возвращен узел (51)

```

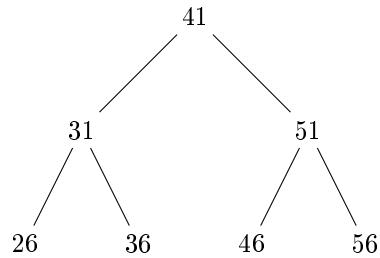


Рис. 26: Пример бинарного дерева поиска

27. Написать программу на Python, которая реализует бинарное дерево поиска с инкапсуляцией. Программа должна создавать экземпляры класса `SpotNode`, которые представляют узлы дерева, и класса `PlaceIndex`, который представляет дерево поиска. Класс `PlaceIndex` должен содержать методы для вставки, поиска и удаления элементов, при этом все рекурсивные методы должны быть приватными. Программа также должна создавать дерево поиска, вставлять в него случайные числа и выполнять поиск элементов в дереве.

Инструкции:

- Создайте класс `SpotNode` с методом `__init__`, который принимает параметр `spot_value` и сохраняет его в атрибуте `self.index_position`. Атрибуты `self.left_location` и `self.right_location` должны быть инициализированы как `None`.
- Создайте класс `PlaceIndex` с методом `__init__`, который инициализирует атрибут `self.start_position` как `None`.
- Создайте публичный метод `insert_location` в классе `PlaceIndex`, который вставляет значение в дерево. Если `self.start_position` отсутствует, создайте новый узел с вставляемым значением. В противном случае, вызовите приватный метод `_insert_location_helper`, передав ему `self.start_position` и значение.
- Создайте приватный метод `_insert_location_helper` в классе `PlaceIndex`, который рекурсивно вставляет значение в дерево. Если значение меньше или равно значению текущего узла, вставьте его в левое поддерево. Если значение строго больше значения текущего узла, вставьте его в правое поддерево.
- Создайте публичный метод `search_location` в классе `PlaceIndex`, который ищет значение в дереве. Если дерево пустое, верните `None`. В противном случае, вызовите приватный метод `_search_location_helper`, передав ему `self.start_position` и искомое значение.
- Создайте приватный метод `_search_location_helper` в классе `PlaceIndex`, который рекурсивно ищет значение в дереве. Если текущий узел равен `None` или значение

текущего узла равно искомому значению, верните текущий узел. В противном случае, рекурсивно вызывайте метод `_search_location_helper` для поиска значения в левом поддереве (если искомое значение меньше или равно текущему) или в правом поддереве (если искомое значение больше).

- (g) Создайте экземпляр класса `PlaceIndex` и вставьте в него 39 случайных чисел от 19 до 59.
- (h) Выполните поиск элементов в дереве и выведите результаты на экран.

Пример использования:

```
pi = PlaceIndex()
for i in range(39):
    pi.insert_location(random.randint(19, 59))

print("Поиск элементов:")
print(pi.search_location(32)) # Обнаружено, возвращен узел (32)
print(pi.search_location(72)) # Не обнаружено, возвращено None
print(pi.search_location(52)) # Обнаружено, возвращен узел (52)
```

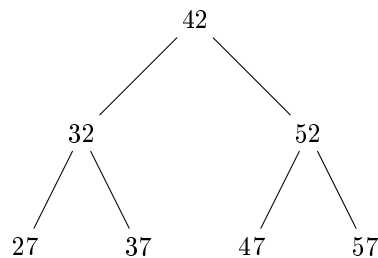


Рис. 27: Пример бинарного дерева поиска

28. Написать программу на Python, которая реализует бинарное дерево поиска с инкапсуляцией. Программа должна создавать экземпляры класса `LocationNode`, которые представляют узлы дерева, и класса `SpotTree`, который представляет дерево поиска. Класс `SpotTree` должен содержать методы для вставки, поиска и удаления элементов, при этом все вспомогательные методы должны быть приватными. Программа также должна создавать дерево поиска, вставлять в него случайные числа и выполнять поиск элементов в дереве.

Инструкции:

- (a) Создайте класс `LocationNode` с методом `__init__`, который принимает параметр `location_val` и сохраняет его в атрибуте `self.tree_spot`. Атрибуты `self.left_site` и `self.right_site` должны быть инициализированы как `None`.
- (b) Создайте класс `SpotTree` с методом `__init__`, который инициализирует атрибут `self.base_spot` как `None`.
- (c) Создайте публичный метод `add_site` в классе `SpotTree`, который вставляет значение в дерево. Если `self.base_spot` отсутствует, создайте новый узел с вставляемым значением. В противном случае, вызовите приватный метод `_add_site_rec`, передав ему `self.base_spot` и значение.

- (d) Создайте приватный метод `_add_site_rec` в классе `SpotTree`, который рекурсивно вставляет значение в дерево. Если значение строго меньше значения текущего узла, вставьте его в левое поддерево. Если значение больше или равно значению текущего узла, вставьте его в правое поддерево.
- (e) Создайте публичный метод `locate_site` в классе `SpotTree`, который ищет значение в дереве. Если дерево пустое, верните `None`. В противном случае, вызовите приватный метод `_locate_site_rec`, передав ему `self.base_spot` и искомое значение.
- (f) Создайте приватный метод `_locate_site_rec` в классе `SpotTree`, который рекурсивно ищет значение в дереве. Если текущий узел равен `None` или значение текущего узла равно искомому значению, верните текущий узел. В противном случае, рекурсивно вызывайте метод `_locate_site_rec` для поиска значения в левом поддерево (если искомое значение меньше текущего) или в правом поддерево (если искомое значение больше или равно текущему).
- (g) Создайте экземпляр класса `SpotTree` и вставьте в него 40 случайных чисел от 20 до 60.
- (h) Выполните поиск элементов в дереве и выведите результаты на экран.

Пример использования:

```
spot_tree = SpotTree()
for i in range(40):
    spot_tree.add_site(random.randint(20, 60))

print("Поиск элементов:")
print(spot_tree.locate_site(33)) # Обнаружено, возвращен узел (33)
print(spot_tree.locate_site(73)) # Не обнаружено, возвращено None
print(spot_tree.locate_site(53)) # Обнаружено, возвращен узел (53)
```

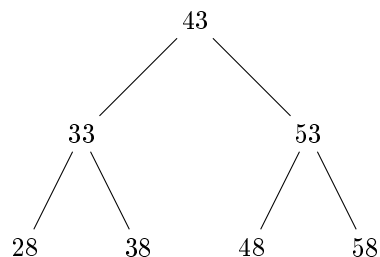


Рис. 28: Пример бинарного дерева поиска

29. Написать программу на Python, которая реализует бинарное дерево поиска с инкапсуляцией. Программа должна создавать экземпляры класса `SiteNode`, которые представляют узлы дерева, и класса `LocationIndex`, который представляет дерево поиска. Класс `LocationIndex` должен содержать методы для вставки, поиска и удаления элементов, при этом все рекурсивные методы должны быть приватными. Программа также должна создавать дерево поиска, вставлять в него случайные числа и выполнять поиск элементов в дереве.

Инструкции:

- (a) Создайте класс `SiteNode` с методом `__init__`, который принимает параметр `site_value` и сохраняет его в атрибуте `self.index_location`. Атрибуты `self.left_zone` и `self.right_zone` должны быть инициализированы как `None`.

- (b) Создайте класс `LocationIndex` с методом `__init__`, который инициализирует атрибут `self.root_location` как `None`.
- (c) Создайте публичный метод `insert_zone` в классе `LocationIndex`, который вставляет значение в дерево. Если `self.root_location` отсутствует, создайте новый узел с вставляемым значением. В противном случае, вызовите приватный метод `_insert_zone_helper`, передав ему `self.root_location` и значение.
- (d) Создайте приватный метод `_insert_zone_helper` в классе `LocationIndex`, который рекурсивно вставляет значение в дерево. Если значение меньше или равно значению текущего узла, вставьте его в левое поддерево. Если значение строго больше значения текущего узла, вставьте его в правое поддерево.
- (e) Создайте публичный метод `find_zone` в классе `LocationIndex`, который ищет значение в дереве. Если дерево пустое, верните `None`. В противном случае, вызовите приватный метод `_find_zone_helper`, передав ему `self.root_location` и искомое значение.
- (f) Создайте приватный метод `_find_zone_helper` в классе `LocationIndex`, который рекурсивно ищет значение в дереве. Если текущий узел равен `None` или значение текущего узла равно искомому значению, верните текущий узел. В противном случае, рекурсивно вызывайте метод `_find_zone_helper` для поиска значения в левом поддереве (если искомое значение меньше или равно текущему) или в правом поддереве (если искомое значение больше).
- (g) Создайте экземпляр класса `LocationIndex` и вставьте в него 41 случайное число от 21 до 61.
- (h) Выполните поиск элементов в дереве и выведите результаты на экран.

Пример использования:

```
li = LocationIndex()
for i in range(41):
    li.insert_zone(random.randint(21, 61))

print("Поиск элементов:")
print(li.find_zone(34)) # Обнаружено, возвращен узел (34)
print(li.find_zone(74)) # Не обнаружено, возвращено None
print(li.find_zone(54)) # Обнаружено, возвращен узел (54)
```

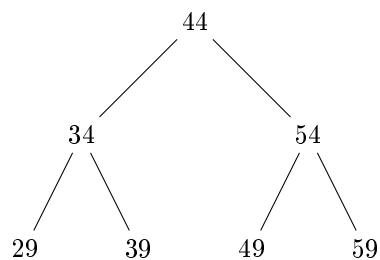


Рис. 29: Пример бинарного дерева поиска

30. Написать программу на Python, которая реализует бинарное дерево поиска с инкапсуляцией. Программа должна создавать экземпляры класса `ZoneNode`, которые представляют узлы дерева, и класса `SiteStructure`, который представляет дерево поиска.

Класс `SiteStructure` должен содержать методы для вставки, поиска и удаления элементов, при этом все вспомогательные методы должны быть приватными. Программа также должна создавать дерево поиска, вставлять в него случайные числа и выполнять поиск элементов в дереве.

Инструкции:

- (a) Создайте класс `ZoneNode` с методом `__init__`, который принимает параметр `zone_val` и сохраняет его в атрибуте `self.structure_site`. Атрибуты `self.left_region` и `self.right_region` должны быть инициализированы как `None`.
- (b) Создайте класс `SiteStructure` с методом `__init__`, который инициализирует атрибут `self.top_site` как `None`.
- (c) Создайте публичный метод `add_region` в классе `SiteStructure`, который вставляет значение в дерево. Если `self.top_site` отсутствует, создайте новый узел с вставляемым значением. В противном случае, вызовите приватный метод `_add_region_rec`, передав ему `self.top_site` и значение.
- (d) Создайте приватный метод `_add_region_rec` в классе `SiteStructure`, который рекурсивно вставляет значение в дерево. Если значение строго меньше значения текущего узла, вставьте его в левое поддерево. Если значение больше или равно значению текущего узла, вставьте его в правое поддерево.
- (e) Создайте публичный метод `search_region` в классе `SiteStructure`, который ищет значение в дереве. Если дерево пустое, верните `None`. В противном случае, вызовите приватный метод `_search_region_rec`, передав ему `self.top_site` и искомое значение.
- (f) Создайте приватный метод `_search_region_rec` в классе `SiteStructure`, который рекурсивно ищет значение в дереве. Если текущий узел равен `None` или значение текущего узла равно искомому значению, верните текущий узел. В противном случае, рекурсивно вызывайте метод `_search_region_rec` для поиска значения в левом поддереве (если искомое значение меньше текущего) или в правом поддереве (если искомое значение больше или равно текущему).
- (g) Создайте экземпляр класса `SiteStructure` и вставьте в него 42 случайных числа от 22 до 62.
- (h) Выполните поиск элементов в дереве и выведите результаты на экран.

Пример использования:

```
ss = SiteStructure()
for i in range(42):
    ss.add_region(random.randint(22, 62))

print("Поиск элементов:")
print(ss.search_region(35)) # Обнаружено, возвращен узел (35)
print(ss.search_region(75)) # Не обнаружено, возвращено None
print(ss.search_region(55)) # Обнаружено, возвращен узел (55)
```

31. Написать программу на Python, которая реализует бинарное дерево поиска с инкапсуляцией. Программа должна создавать экземпляры класса `RegionNode`, которые представляют узлы дерева, и класса `ZoneTree`, который представляет дерево поиска. Класс `ZoneTree` должен содержать методы для вставки, поиска и удаления элементов, при

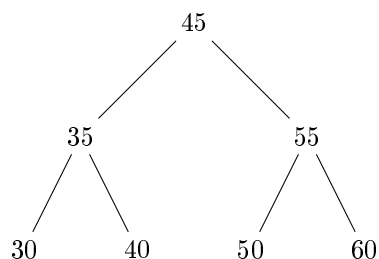


Рис. 30: Пример бинарного дерева поиска

этом все рекурсивные методы должны быть приватными. Программа также должна создавать дерево поиска, вставлять в него случайные числа и выполнять поиск элементов в дереве.

Инструкции:

- Создайте класс `RegionNode` с методом `__init__`, который принимает параметр `region_value` и сохраняет его в атрибуте `self.tree_zone`. Атрибуты `self.left_area` и `self.right_area` должны быть инициализированы как `None`.
- Создайте класс `ZoneTree` с методом `__init__`, который инициализирует атрибут `self.first_zone` как `None`.
- Создайте публичный метод `insert_area` в классе `ZoneTree`, который вставляет значение в дерево. Если `self.first_zone` отсутствует, создайте новый узел с вставляемым значением. В противном случае, вызовите приватный метод `_insert_area_helper`, передав ему `self.first_zone` и значение.
- Создайте приватный метод `_insert_area_helper` в классе `ZoneTree`, который рекурсивно вставляет значение в дерево. Если значение меньше или равно значению текущего узла, вставьте его в левое поддерево. Если значение строго больше значения текущего узла, вставьте его в правое поддерево.
- Создайте публичный метод `locate_area` в классе `ZoneTree`, который ищет значение в дереве. Если дерево пустое, верните `None`. В противном случае, вызовите приватный метод `_locate_area_rec`, передав ему `self.first_zone` и искомое значение.
- Создайте приватный метод `_locate_area_rec` в классе `ZoneTree`, который рекурсивно ищет значение в дереве. Если текущий узел равен `None` или значение текущего узла равно искомому значению, верните текущий узел. В противном случае, рекурсивно вызывайте метод `_locate_area_rec` для поиска значения в левом поддереве (если искомое значение меньше или равно текущему) или в правом поддереве (если искомое значение больше).
- Создайте экземпляр класса `ZoneTree` и вставьте в него 43 случайных числа от 23 до 63.
- Выполните поиск элементов в дереве и выведите результаты на экран.

Пример использования:

```

zt = ZoneTree()
for i in range(43):
    zt.insert_area(random.randint(23, 63))
  
```

```

print("Поиск элементов:")
print(zt.locate_area(36)) # Обнаружено, возвращен узел (36)
print(zt.locate_area(76)) # Не обнаружено, возвращено None
print(zt.locate_area(56)) # Обнаружено, возвращен узел (56)

```

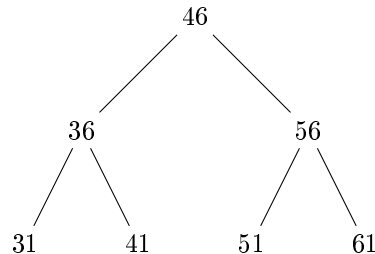


Рис. 31: Пример бинарного дерева поиска

32. Написать программу на Python, которая реализует бинарное дерево поиска с инкапсуляцией. Программа должна создавать экземпляры класса `AreaNode`, которые представляют узлы дерева, и класса `RegionIndex`, который представляет дерево поиска. Класс `RegionIndex` должен содержать методы для вставки, поиска и удаления элементов, при этом все вспомогательные методы должны быть приватными. Программа также должна создавать дерево поиска, вставлять в него случайные числа и выполнять поиск элементов в дереве.

Инструкции:

- Создайте класс `AreaNode` с методом `__init__`, который принимает параметр `area_val` и сохраняет его в атрибуте `self.index_region`. Атрибуты `self.left_district` и `self.right_district` должны быть инициализированы как `None`.
- Создайте класс `RegionIndex` с методом `__init__`, который инициализирует атрибут `self.root_region` как `None`.
- Создайте публичный метод `add_district` в классе `RegionIndex`, который вставляет значение в дерево. Если `self.root_region` отсутствует, создайте новый узел с вставляемым значением. В противном случае, вызовите приватный метод `_add_district_rec`, передав ему `self.root_region` и значение.
- Создайте приватный метод `_add_district_rec` в классе `RegionIndex`, который рекурсивно вставляет значение в дерево. Если значение строго меньше значения текущего узла, вставьте его в левое поддерево. Если значение больше или равно значению текущего узла, вставьте его в правое поддерево.
- Создайте публичный метод `find_district` в классе `RegionIndex`, который ищет значение в дереве. Если дерево пустое, верните `None`. В противном случае, вызовите приватный метод `_find_district_helper`, передав ему `self.root_region` и искомое значение.
- Создайте приватный метод `_find_district_helper` в классе `RegionIndex`, который рекурсивно ищет значение в дереве. Если текущий узел равен `None` или значение текущего узла равно искомому значению, верните текущий узел. В противном случае, рекурсивно вызывайте метод `_find_district_helper` для поиска значения.

в левом поддереве (если искомое значение меньше текущего) или в правом поддереве (если искомое значение больше или равно текущему).

(g) Создайте экземпляр класса `RegionIndex` и вставьте в него 44 случайных числа от 24 до 64.

(h) Выполните поиск элементов в дереве и выведите результаты на экран.

Пример использования:

```
ri = RegionIndex()
for i in range(44):
    ri.add_district(random.randint(24, 64))

print("Поиск элементов:")
print(ri.find_district(37)) # Обнаружено, возвращен узел (37)
print(ri.find_district(77)) # Не обнаружено, возвращено None
print(ri.find_district(57)) # Обнаружено, возвращен узел (57)
```

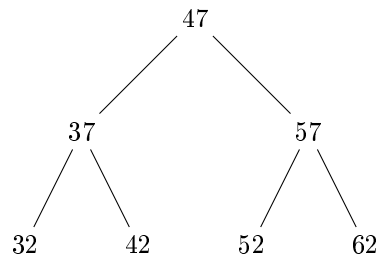


Рис. 32: Пример бинарного дерева поиска

33. Написать программу на Python, которая реализует бинарное дерево поиска с инкапсуляцией. Программа должна создавать экземпляры класса `DistrictNode`, которые представляют узлы дерева, и класса `AreaTree`, который представляет дерево поиска. Класс `AreaTree` должен содержать методы для вставки, поиска и удаления элементов, при этом все рекурсивные методы должны быть приватными. Программа также должна создавать дерево поиска, вставлять в него случайные числа и выполнять поиск элементов в дереве.

Инструкции:

- (a) Создайте класс `DistrictNode` с методом `__init__`, который принимает параметр `district_value` и сохраняет его в атрибуте `self.tree_area`. Атрибуты `self.left_sector` и `self.right_sector` должны быть инициализированы как `None`.
- (b) Создайте класс `AreaTree` с методом `__init__`, который инициализирует атрибут `self.start_area` как `None`.
- (c) Создайте публичный метод `insert_sector` в классе `AreaTree`, который вставляет значение в дерево. Если `self.start_area` отсутствует, создайте новый узел с вставляемым значением. В противном случае, вызовите приватный метод `_insert_sector_helper`, передав ему `self.start_area` и значение.
- (d) Создайте приватный метод `_insert_sector_helper` в классе `AreaTree`, который рекурсивно вставляет значение в дерево. Если значение меньше или равно значению текущего узла, вставьте его в левое поддерево. Если значение строго больше значения текущего узла, вставьте его в правое поддерево.

- (e) Создайте публичный метод `search_sector` в классе `AreaTree`, который ищет значение в дереве. Если дерево пустое, верните `None`. В противном случае, вызовите приватный метод `_search_sector_rec`, передав ему `self.start_area` и искомое значение.
- (f) Создайте приватный метод `_search_sector_rec` в классе `AreaTree`, который рекурсивно ищет значение в дереве. Если текущий узел равен `None` или значение текущего узла равно искомому значению, верните текущий узел. В противном случае, рекурсивно вызывайте метод `_search_sector_rec` для поиска значения в левом поддереве (если искомое значение меньше или равно текущему) или в правом поддереве (если искомое значение больше).
- (g) Создайте экземпляр класса `AreaTree` и вставьте в него 45 случайных чисел от 25 до 65.
- (h) Выполните поиск элементов в дереве и выведите результаты на экран.

Пример использования:

```
at = AreaTree()
for i in range(45):
    at.insert_sector(random.randint(25, 65))

print("Поиск элементов:")
print(at.search_sector(38))  # Обнаружено, возвращен узел (38)
print(at.search_sector(78))  # Не обнаружено, возвращено None
print(at.search_sector(58))  # Обнаружено, возвращен узел (58)
```

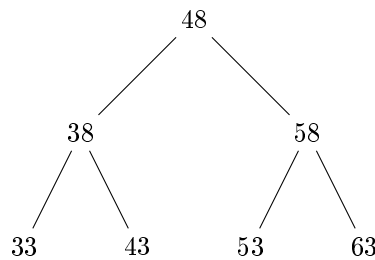


Рис. 33: Пример бинарного дерева поиска

34. Написать программу на Python, которая реализует бинарное дерево поиска с инкапсуляцией. Программа должна создавать экземпляры класса `SectorNode`, которые представляют узлы дерева, и класса `DistrictStructure`, который представляет дерево поиска. Класс `DistrictStructure` должен содержать методы для вставки, поиска и удаления элементов, при этом все вспомогательные методы должны быть приватными. Программа также должна создавать дерево поиска, вставлять в него случайные числа и выполнять поиск элементов в дереве.

Инструкции:

- (a) Создайте класс `SectorNode` с методом `__init__`, который принимает параметр `sector_val` и сохраняет его в атрибуте `self.structure_district`. Атрибуты `self.left_block` и `self.right_block` должны быть инициализированы как `None`.
- (b) Создайте класс `DistrictStructure` с методом `__init__`, который инициализирует атрибут `self.top_district` как `None`.

- (c) Создайте публичный метод `add_block` в классе `DistrictStructure`, который вставляет значение в дерево. Если `self.top_district` отсутствует, создайте новый узел с вставляемым значением. В противном случае, вызовите приватный метод `_add_block_rec`, передав ему `self.top_district` и значение.
- (d) Создайте приватный метод `_add_block_rec` в классе `DistrictStructure`, который рекурсивно вставляет значение в дерево. Если значение строго меньше значения текущего узла, вставьте его в левое поддерево. Если значение больше или равно значению текущего узла, вставьте его в правое поддерево.
- (e) Создайте публичный метод `locate_block` в классе `DistrictStructure`, который ищет значение в дереве. Если дерево пустое, верните `None`. В противном случае, вызовите приватный метод `_locate_block_helper`, передав ему `self.top_district` и искомое значение.
- (f) Создайте приватный метод `_locate_block_helper` в классе `DistrictStructure`, который рекурсивно ищет значение в дереве. Если текущий узел равен `None` или значение текущего узла равно искомому значению, верните текущий узел. В противном случае, рекурсивно вызывайте метод `_locate_block_helper` для поиска значения в левом поддереве (если искомое значение меньше текущего) или в правом поддереве (если искомое значение больше или равно текущему).
- (g) Создайте экземпляр класса `DistrictStructure` и вставьте в него 46 случайных чисел от 26 до 66.
- (h) Выполните поиск элементов в дереве и выведите результаты на экран.

Пример использования:

```
ds = DistrictStructure()
for i in range(46):
    ds.add_block(random.randint(26, 66))

print("Поиск элементов:")
print(ds.locate_block(39)) # Обнаружено, возвращен узел (39)
print(ds.locate_block(79)) # Не обнаружено, возвращено None
print(ds.locate_block(59)) # Обнаружено, возвращен узел (59)
```

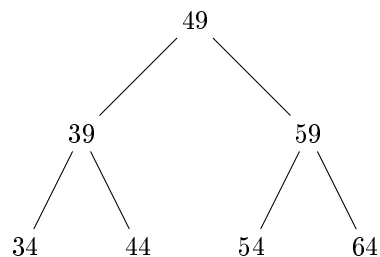


Рис. 34: Пример бинарного дерева поиска

35. Написать программу на Python, которая реализует бинарное дерево поиска с инкапсуляцией. Программа должна создавать экземпляры класса `BlockNode`, которые представляют узлы дерева, и класса `SectorIndex`, который представляет дерево поиска. Класс `SectorIndex` должен содержать методы для вставки, поиска и удаления элементов, при этом все рекурсивные методы должны быть приватными. Программа также

должна создавать дерево поиска, вставлять в него случайные числа и выполнять поиск элементов в дереве.

Инструкции:

- (a) Создайте класс `BlockNode` с методом `__init__`, который принимает параметр `block_value` и сохраняет его в атрибуте `self.index_sector`. Атрибуты `self.left_unit` и `self.right_unit` должны быть инициализированы как `None`.
- (b) Создайте класс `SectorIndex` с методом `__init__`, который инициализирует атрибут `self.root_sector` как `None`.
- (c) Создайте публичный метод `insert_unit` в классе `SectorIndex`, который вставляет значение в дерево. Если `self.root_sector` отсутствует, создайте новый узел с вставляемым значением. В противном случае, вызовите приватный метод `_insert_unit_helper`, передав ему `self.root_sector` и значение.
- (d) Создайте приватный метод `_insert_unit_helper` в классе `SectorIndex`, который рекурсивно вставляет значение в дерево. Если значение меньше или равно значению текущего узла, вставьте его в левое поддерево. Если значение строго больше значения текущего узла, вставьте его в правое поддерево.
- (e) Создайте публичный метод `find_unit` в классе `SectorIndex`, который ищет значение в дереве. Если дерево пустое, верните `None`. В противном случае, вызовите приватный метод `_find_unit_rec`, передав ему `self.root_sector` и искомое значение.
- (f) Создайте приватный метод `_find_unit_rec` в классе `SectorIndex`, который рекурсивно ищет значение в дереве. Если текущий узел равен `None` или значение текущего узла равно искомому значению, верните текущий узел. В противном случае, рекурсивно вызывайте метод `_find_unit_rec` для поиска значения в левом поддереве (если искомое значение меньше или равно текущему) или в правом поддереве (если искомое значение больше).
- (g) Создайте экземпляр класса `SectorIndex` и вставьте в него 47 случайных чисел от 27 до 67.
- (h) Выполните поиск элементов в дереве и выведите результаты на экран.

Пример использования:

```
si = SectorIndex()
for i in range(47):
    si.insert_unit(random.randint(27, 67))

print("Поиск элементов:")
print(si.find_unit(40)) # Обнаружено, возвращен узел (40)
print(si.find_unit(80)) # Не обнаружено, возвращено None
print(si.find_unit(60)) # Обнаружено, возвращен узел (60)
```

2.3.2 Задача 2 (стек)

1. Написать программу на Python, которая создает класс `Stack` для представления стека с инкапсуляцией внутреннего состояния. Класс должен содержать методы `push`, `pop`, `is_empty`, `size` и `peek`, которые реализуют операции вталкивания, выталкивания, проверки пустоты, получения размера и просмотра вершины стека соответственно. Программа также должна создавать экземпляр класса `Stack`, вталкивать в него элементы, выталкивать элементы и выводить информацию о стеке на экран.

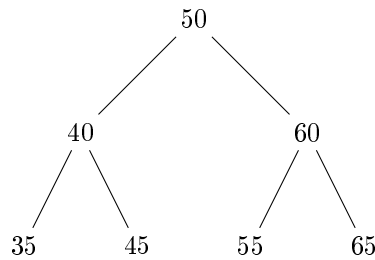


Рис. 35: Пример бинарного дерева поиска

Инструкции:

- Создайте класс `Stack` с методом `__init__`, который принимает необязательный аргумент `initial_element`. Если он передан, стек инициализируется с этим элементом (в виде списка из одного элемента), иначе — пустым списком.
- Создайте метод `push`, который принимает элемент в качестве аргумента и вталкивает его в стек только в том случае, если он не равен текущему верхнему элементу (если стек не пуст). Если стек пуст, элемент добавляется без проверки.
- Создайте метод `pop`, который выталкивает верхний элемент из стека и возвращает его. Если стек пуст, метод должен вернуть `None` и вывести сообщение "Стек пуст — извлечение невозможно" в стандартный поток ошибок (`sys.stderr`).
- Создайте метод `is_empty`, который возвращает `True`, если стек пуст, и `False` в противном случае.
- Создайте метод `size`, который возвращает текущее количество элементов в стеке.
- Создайте метод `peek`, который возвращает верхний элемент стека, если стек не пуст. Если стек пуст, возвращает `None` и выводит сообщение "Стек пуст — просмотр невозможен" в `sys.stderr`.
- Создайте экземпляр класса `Stack`, передав в конструктор начальный элемент 10.
- Последовательно вызовите метод `push` с аргументами: 10, 20, 20, 30, 40 (обратите внимание, что повторяющийся элемент 20 не должен быть добавлен дважды подряд).
- Выведите размер стека и верхний элемент.
- Вызовите метод `pop` дважды, каждый раз выводя вытолкнутый элемент.
- После каждого `pop` выводите текущий размер стека и результат вызова `peek`.

Пример использования:

```

import sys

stack = Stack(10)
stack.push(10)    # не добавится, т.к. равен верхнему
stack.push(20)    # добавится
stack.push(20)    # не добавится, т.к. равен верхнему
stack.push(30)
stack.push(40)

print("Размер стека:", stack.size())
print("Верхний элемент:", stack.peek())
  
```

```

popped = stack.pop()
print("Вытолкнут:", popped)
print("Размер после pop:", stack.size())
print("Верхний элемент:", stack.peek())

popped = stack.pop()
print("Вытолкнут:", popped)
print("Размер после pop:", stack.size())
print("Верхний элемент:", stack.peek())

```

2. Написать программу на Python, которая создает класс Stack для представления стека с инкапсуляцией. Класс должен содержать методы push, pop, is_empty, size и peek, которые реализуют операции вталкивания, выталкивания, проверки пустоты, получения размера и просмотра вершины стека соответственно. Программа также должна создавать экземпляр класса Stack, вталкивать в него элементы, выталкивать элементы и выводить информацию о стеке на экран.

Инструкции:

- (a) Создайте класс Stack с методом `__init__`, который инициализирует пустой стек. Дополнительно принимает необязательный параметр `max_size`, ограничивающий максимальное количество элементов в стеке (по умолчанию — None, то есть без ограничений).
- (b) Создайте метод `push`, который принимает два аргумента: `element` и `force=False`. Элемент добавляется в стек, только если не превышает `max_size`. Если `force=True`, то элемент добавляется даже при превышении лимита (с заменой самого нижнего элемента, если стек полон).
- (c) Создайте метод `pop`, который выталкивает верхний элемент из стека и возвращает его. Если стек пуст, возвращает строку "Стек пуст".
- (d) Создайте метод `is_empty`, который возвращает True, если стек пуст, и False в противном случае.
- (e) Создайте метод `size`, который возвращает текущее количество элементов в стеке.
- (f) Создайте метод `peek`, который возвращает верхний элемент стека, если стек не пуст. Если стек пуст, возвращает строку "Нет элементов для просмотра".
- (g) Создайте экземпляр класса Stack с `max_size=3`.
- (h) Последовательно вызовите `push` с элементами 5, 15, 25 (все добавятся).
- (i) Попытайтесь добавить 35 без `force` — не должно добавиться.
- (j) Добавьте 35 с `force=True` — должен заменить нижний элемент (5), стек станет [15, 25, 35].
- (k) Выведите размер стека и верхний элемент.
- (l) Вызовите `pop` и выведите результат.
- (m) Повторите вывод размера и верхнего элемента.

Пример использования:

```

stack = Stack(max_size=3)
stack.push(5)
stack.push(15)
stack.push(25)

```

```

stack.push(35)           # не добавится
stack.push(35, force=True) # добавится с заменой нижнего

print("Размер стека:", stack.size())
print("Верхний элемент:", stack.peek())

popped = stack.pop()
print("Вытолкнут:", popped)
print("Размер после pop:", stack.size())
print("Верхний элемент:", stack.peek())

```

3. Написать программу на Python, которая создает класс Stack для представления стека с инкапсуляцией. Класс должен содержать методы push, pop, is_empty, size и peek, которые реализуют операции вталкивания, выталкивания, проверки пустоты, получения размера и просмотра вершины стека соответственно. Программа также должна создавать экземпляр класса Stack, вталкивать в него элементы, выталкивать элементы и выводить информацию о стеке на экран.

Инструкции:

- (a) Создайте класс Stack с методом `__init__`, который инициализирует пустой стек. Может принимать список элементов в качестве аргумента `items`, который будет использован для первоначального заполнения стека (в порядке, как в списке: первый элемент — внизу стека).
- (b) Создайте метод `push`, который принимает один элемент и добавляет его в стек. Если добавляемый элемент отрицательный, он не добавляется, а в `sys.stderr` выводится предупреждение "Отрицательные значения не допускаются".
- (c) Создайте метод `pop`, который выталкивает верхний элемент из стека и возвращает его. Если стек пуст, выбрасывает исключение `IndexError` с сообщением "pop from empty stack".
- (d) Создайте метод `is_empty`, который возвращает `True`, если стек пуст, и `False` в противном случае.
- (e) Создайте метод `size`, который возвращает текущее количество элементов в стеке.
- (f) Создайте метод `peek`, который возвращает верхний элемент стека, если стек не пуст. Если стек пуст, выбрасывает исключение `IndexError` с сообщением "peek from empty stack".
- (g) Создайте экземпляр класса Stack, передав в конструктор список `[1, 2, 3]`.
- (h) Добавьте элементы 4, -5 (не добавится), 6.
- (i) Выведите размер стека и результат peek.
- (j) Вызовите pop трижды, каждый раз выводя результат.
- (k) После каждого pop проверяйте is_empty и выводите результат.

Пример использования:

```

import sys

stack = Stack([1, 2, 3])
stack.push(4)
stack.push(-5) # не добавится, выведет предупреждение
stack.push(6)

```

```

print("Размер стека:", stack.size())
print("Верхний элемент:", stack.peek())

for _ in range(3):
    popped = stack.pop()
    print("Вытолкнут:", popped)
    print("Стек пуст?", stack.is_empty())

```

4. Написать программу на Python, которая создает класс Stack для представления стека с инкапсуляцией. Класс должен содержать методы push, pop, is_empty, size и peek, которые реализуют операции вталкивания, выталкивания, проверки пустоты, получения размера и просмотра вершины стека соответственно. Программа также должна создавать экземпляр класса Stack, вталкивать в него элементы, выталкивать элементы и выводить информацию о стеке на экран.

Инструкции:

- (a) Создайте класс Stack с методом `__init__`, который инициализирует пустой стек. Принимает необязательный аргумент `allow_duplicates` (по умолчанию True). Если False, то дубликаты (элементы, уже присутствующие в стеке) не добавляются.
- (b) Создайте метод push, который принимает элемент и добавляет его в стек, только если `allow_duplicates=True` или если такого элемента еще нет в стеке. Возвращает True, если элемент добавлен, и False — если не добавлен.
- (c) Создайте метод pop, который выталкивает верхний элемент из стека и возвращает его. Если стек пуст, возвращает None.
- (d) Создайте метод is_empty, который возвращает True, если стек пуст, и False в противном случае.
- (e) Создайте метод size, который возвращает текущее количество элементов в стеке.
- (f) Создайте метод peek, который возвращает верхний элемент стека, если стек не пуст. Если стек пуст, возвращает None.
- (g) Создайте экземпляр класса Stack с `allow_duplicates=False`.
- (h) Добавьте элементы 10, 20, 10 (второй 10 не добавится), 30.
- (i) Выведите размер стека и верхний элемент.
- (j) Вызовите pop, выведите результат.
- (k) Повторите вывод размера и верхнего элемента.

Пример использования:

```

stack = Stack(allow_duplicates=False)
print(stack.push(10)) # True
print(stack.push(20)) # True
print(stack.push(10)) # False (дубликат)
print(stack.push(30)) # True

print("Размер стека:", stack.size())
print("Верхний элемент:", stack.peek())

popped = stack.pop()
print("Вытолкнут:", popped)
print("Размер после pop:", stack.size())
print("Верхний элемент:", stack.peek())

```

5. Написать программу на Python, которая создает класс Stack для представления стека с инкапсуляцией. Класс должен содержать методы push, pop, is_empty, size и peek, которые реализуют операции вталкивания, выталкивания, проверки пустоты, получения размера и просмотра вершины стека соответственно. Программа также должна создавать экземпляр класса Stack, вталкивать в него элементы, выталкивать элементы и выводить информацию о стеке на экран.

Инструкции:

- (a) Создайте класс Stack с методом `__init__`, который инициализирует пустой стек. Может принимать параметр `name` (строка) для именования стека (используется только для отладки, не влияет на логику).
- (b) Создайте метод `push`, который принимает элемент и добавляет его в стек. Если элемент не является числом (`int` или `float`), он не добавляется, а в `sys.stderr` выводится сообщение "Только числовые значения разрешены".
- (c) Создайте метод `pop`, который выталкивает верхний элемент из стека и возвращает его. Если стек пуст, возвращает `None`.
- (d) Создайте метод `is_empty`, который возвращает `True`, если стек пуст, и `False` в противном случае.
- (e) Создайте метод `size`, который возвращает текущее количество элементов в стеке.
- (f) Создайте метод `peek`, который возвращает верхний элемент стека, если стек не пуст. Если стек пуст, возвращает `None`.
- (g) Создайте экземпляр класса Stack с именем "NumericStack".
- (h) Добавьте элементы: 3.14, 42, "hello"(не добавится), 100, [1,2] (не добавится).
- (i) Выведите размер стека и верхний элемент.
- (j) Вызовите `pop` дважды, выводя каждый раз результат.
- (k) После каждого `pop` выводите размер стека.

Пример использования:

```
import sys

stack = Stack(name="NumericStack")
stack.push(3.14)
stack.push(42)
stack.push("hello")    # не добавится
stack.push(100)
stack.push([1,2])      # не добавится

print("Размер стека:", stack.size())
print("Верхний элемент:", stack.peek())

popped = stack.pop()
print("Вытолкнут:", popped)
print("Размер после pop:", stack.size())

popped = stack.pop()
print("Вытолкнут:", popped)
print("Размер после pop:", stack.size())
```

6. Написать программу на Python, которая создает класс Stack для представления стека с инкапсуляцией. Класс должен содержать методы push, pop, is_empty, size и peek, которые реализуют операции вталкивания, выталкивания, проверки пустоты, получения размера и просмотра вершины стека соответственно. Программа также должна создавать экземпляр класса Stack, вталкивать в него элементы, выталкивать элементы и выводить информацию о стеке на экран.

Инструкции:

- (a) Создайте класс Stack с методом `__init__`, который инициализирует пустой стек. Принимает необязательный параметр `auto_reverse=False`. Если `True`, то при добавлении элемента он вставляется не наверх, а вниз стека (реализуя поведение, обратное обычному стеку).
- (b) Создайте метод `push`, который принимает элемент и добавляет его: если `auto_reverse=False` — наверх (как обычно), если `True` — вниз (в начало внутреннего списка).
- (c) Создайте метод `pop`, который выталкивает верхний элемент из стека (последний добавленный, если `auto_reverse=False`, или первый добавленный, если `auto_reverse=True`) и возвращает его. Если стек пуст, возвращает "EMPTY".
- (d) Создайте метод `is_empty`, который возвращает `True`, если стек пуст, и `False` в противном случае.
- (e) Создайте метод `size`, который возвращает текущее количество элементов в стеке.
- (f) Создайте метод `peek`, который возвращает верхний элемент стека (последний в списке, если `auto_reverse=False`, или первый, если `auto_reverse=True`), если стек не пуст. Если стек пуст, возвращает "NO ELEMENT".
- (g) Создайте экземпляр класса Stack с `auto_reverse=True`.
- (h) Добавьте элементы: 1, 2, 3 (в стеке будет [3, 2, 1], где 3 — верх).
- (i) Выведите размер стека и результат `peek` (должен быть 3).
- (j) Вызовите `pop`, выведите результат (должен быть 3).
- (k) Повторите вывод размера и `peek` (теперь верх — 2).

Пример использования:

```
stack = Stack(auto_reverse=True)
stack.push(1)
stack.push(2)
stack.push(3)    # стек: [3,2,1], верх - 3

print("Размер стека:", stack.size())
print("Верхний элемент:", stack.peek())

popped = stack.pop()
print("Вытолкнут:", popped)    # 3
print("Размер после pop:", stack.size())
print("Верхний элемент:", stack.peek())    # 2
```

7. Написать программу на Python, которая создает класс Stack для представления стека с инкапсуляцией. Класс должен содержать методы push, pop, is_empty, size и peek, которые реализуют операции вталкивания, выталкивания, проверки пустоты, получения размера и просмотра вершины стека соответственно. Программа также должна создавать экземпляр класса Stack, вталкивать в него элементы, выталкивать элементы и выводить информацию о стеке на экран.

Инструкции:

- (a) Создайте класс `Stack` с методом `__init__`, который инициализирует пустой стек. Принимает параметр `case_sensitive=True`. Используется только если элементы — строки.
- (b) Создайте метод `push`, который принимает элемент. Если элемент — строка и `case_sensitive=False`, то перед добавлением преобразует её в нижний регистр. Добавляет элемент в стек.
- (c) Создайте метод `pop`, который выталкивает верхний элемент из стека и возвращает его. Если стек пуст, возвращает пустую строку.
- (d) Создайте метод `is_empty`, который возвращает `True`, если стек пуст, и `False` в противном случае.
- (e) Создайте метод `size`, который возвращает текущее количество элементов в стеке.
- (f) Создайте метод `peek`, который возвращает верхний элемент стека, если стек не пуст. Если стек пуст, возвращает пустую строку.
- (g) Создайте экземпляр класса `Stack` с `case_sensitive=False`.
- (h) Добавьте строки: "Hello "WORLD "Python".
- (i) Выведите размер стека и верхний элемент (должен быть "python").
- (j) Вызовите `pop`, выведите результат.
- (k) Повторите вывод размера и верхнего элемента.

Пример использования:

```
stack = Stack(case_sensitive=False)
stack.push("Hello")
stack.push("WORLD")
stack.push("Python")

print("Размер стека:", stack.size())
print("Верхний элемент:", stack.peek()) # "python"

popped = stack.pop()
print("Вытолкнут:", popped) # "python"
print("Размер после pop:", stack.size())
print("Верхний элемент:", stack.peek()) # "world"
```

8. Написать программу на Python, которая создает класс `Stack` для представления стека с инкапсуляцией. Класс должен содержать методы `push`, `pop`, `is_empty`, `size` и `peek`, которые реализуют операции вталкивания, выталкивания, проверки пустоты, получения размера и просмотра вершины стека соответственно. Программа также должна создавать экземпляр класса `Stack`, вталкивать в него элементы, выталкивать элементы и выводить информацию о стеке на экран.

Инструкции:

- (a) Создайте класс `Stack` с методом `__init__`, который инициализирует пустой стек. Принимает параметр `min_value=None`. Если задан, то при добавлении элемента проверяется, что он $\geq \text{min_value}$.
- (b) Создайте метод `push`, который принимает элемент. Если `min_value` задан и элемент $< \text{min_value}$, элемент не добавляется, а метод возвращает `False`. Иначе — добавляет и возвращает `True`.

- (c) Создайте метод `pop`, который выталкивает верхний элемент из стека и возвращает его. Если стек пуст, возвращает `None`.
- (d) Создайте метод `is_empty`, который возвращает `True`, если стек пуст, и `False` в противном случае.
- (e) Создайте метод `size`, который возвращает текущее количество элементов в стеке.
- (f) Создайте метод `peek`, который возвращает верхний элемент стека, если стек не пуст. Если стек пуст, возвращает `None`.
- (g) Создайте экземпляр класса `Stack` с `min_value=10`.
- (h) Добавьте элементы: 5 (не добавится), 15, 20, 8 (не добавится), 25.
- (i) Выведите размер стека и верхний элемент.
- (j) Вызовите `pop`, выведите результат.
- (k) Повторите вывод размера и верхнего элемента.

Пример использования:

```
stack = Stack(min_value=10)
print(stack.push(5))    # False
print(stack.push(15))   # True
print(stack.push(20))   # True
print(stack.push(8))    # False
print(stack.push(25))   # True

print("Размер стека:", stack.size())
print("Верхний элемент:", stack.peek())

popped = stack.pop()
print("Вытолкнут:", popped) # 25
print("Размер после pop:", stack.size())
print("Верхний элемент:", stack.peek()) # 20
```

9. Написать программу на Python, которая создает класс `Stack` для представления стека с инкапсуляцией. Класс должен содержать методы `push`, `pop`, `is_empty`, `size` и `peek`, которые реализуют операции вталкивания, выталкивания, проверки пустоты, получения размера и просмотра вершины стека соответственно. Программа также должна создавать экземпляр класса `Stack`, вталкивать в него элементы, выталкивать элементы и выводить информацию о стеке на экран.

Инструкции:

- (a) Создайте класс `Stack` с методом `__init__`, который инициализирует пустой стек. Принимает параметр `max_increments=0` — максимальное количество добавлений. Если 0 — без ограничений.
- (b) Создайте метод `push`, который принимает элемент. Если `max_increments > 0` и количество вызовов `push` превысило `max_increments`, элемент не добавляется, метод возвращает `False`. Иначе — добавляет и возвращает `True`.
- (c) Создайте метод `pop`, который выталкивает верхний элемент из стека и возвращает его. Если стек пуст, возвращает строку — "".
- (d) Создайте метод `is_empty`, который возвращает `True`, если стек пуст, и `False` в противном случае.
- (e) Создайте метод `size`, который возвращает текущее количество элементов в стеке.

- (f) Создайте метод `peek`, который возвращает верхний элемент стека, если стек не пуст. Если стек пуст, возвращает строку `—`.
- (g) Создайте экземпляр класса `Stack` с `max_increments=3`.
- (h) Добавьте элементы: 100, 200, 300, 400 (последний не добавится).
- (i) Выведите размер стека и верхний элемент.
- (j) Вызовите `pop`, выведите результат.
- (k) Повторите вывод размера и верхнего элемента.

Пример использования:

```
stack = Stack(max_increments=3)
print(stack.push(100)) # True
print(stack.push(200)) # True
print(stack.push(300)) # True
print(stack.push(400)) # False

print("Размер стека:", stack.size())
print("Верхний элемент:", stack.peek())

popped = stack.pop()
print("Вытолкнут:", popped) # 300
print("Размер после pop:", stack.size())
print("Верхний элемент:", stack.peek()) # 200
```

10. Написать программу на Python, которая создает класс `Stack` для представления стека с инкапсуляцией. Класс должен содержать методы `push`, `pop`, `is_empty`, `size` и `peek`, которые реализуют операции вталкивания, выталкивания, проверки пустоты, получения размера и просмотра вершины стека соответственно. Программа также должна создавать экземпляр класса `Stack`, вталкивать в него элементы, выталкивать элементы и выводить информацию о стеке на экран.

Инструкции:

- (a) Создайте класс `Stack` с методом `__init__`, который инициализирует пустой стек. Принимает параметр `validate_type=None`. Если задан (например, `int`), то при добавлении проверяется, что элемент является экземпляром этого типа.
- (b) Создайте метод `push`, который принимает элемент. Если `validate_type` задан и элемент не является его экземпляром, элемент не добавляется, метод возвращает `False`. Иначе — добавляет и возвращает `True`.
- (c) Создайте метод `pop`, который выталкивает верхний элемент из стека и возвращает его. Если стек пуст, возвращает `None`.
- (d) Создайте метод `is_empty`, который возвращает `True`, если стек пуст, и `False` в противном случае.
- (e) Создайте метод `size`, который возвращает текущее количество элементов в стеке.
- (f) Создайте метод `peek`, который возвращает верхний элемент стека, если стек не пуст. Если стек пуст, возвращает `None`.
- (g) Создайте экземпляр класса `Stack` с `validate_type=int`.
- (h) Добавьте элементы: 10, "20" (не добавится), 30, 40.5 (не добавится), 50.
- (i) Выведите размер стека и верхний элемент.

- (j) Вызовите pop, выведите результат.
- (k) Повторите вывод размера и верхнего элемента.

Пример использования:

```
stack = Stack(validate_type=int)
print(stack.push(10))      # True
print(stack.push("20"))   # False
print(stack.push(30))     # True
print(stack.push(40.5))   # False
print(stack.push(50))     # True

print("Размер стека:", stack.size())
print("Верхний элемент:", stack.peek())

popped = stack.pop()
print("Вытолкнут:", popped) # 50
print("Размер после pop:", stack.size())
print("Верхний элемент:", stack.peek()) # 30
```

11. Написать программу на Python, которая создает класс Stack для представления стека с инкапсуляцией. Класс должен содержать методы push, pop, is_empty, size и peek, которые реализуют операции вталкивания, выталкивания, проверки пустоты, получения размера и просмотра вершины стека соответственно. Программа также должна создавать экземпляры класса Stack, вталкивать в него элементы, выталкивать элементы и выводить информацию о стеке на экран.

Инструкции:

- (a) Создайте класс Stack с методом `__init__`, который инициализирует пустой стек. Принимает параметр `unique_per_session=False`. Если True, то не позволяет добавлять один и тот же элемент дважды за всё время жизни стека (даже если он был удален).
- (b) Создайте метод push, который принимает элемент. Если `unique_per_session=True` и элемент уже когда-либо был добавлен (даже если потом удален), он не добавляется, метод возвращает False. Иначе — добавляет и возвращает True.
- (c) Создайте метод pop, который выталкивает верхний элемент из стека и возвращает его. Если стек пуст, возвращает None.
- (d) Создайте метод is_empty, который возвращает True, если стек пуст, и False в противном случае.
- (e) Создайте метод size, который возвращает текущее количество элементов в стеке.
- (f) Создайте метод peek, который возвращает верхний элемент стека, если стек не пуст. Если стек пуст, возвращает None.
- (g) Создайте экземпляр класса Stack с `unique_per_session=True`.
- (h) Добавьте элементы: 7, 14, 7 (не добавится), 21, 14 (не добавится).
- (i) Выведите размер стека и верхний элемент.
- (j) Вызовите pop, выведите результат.
- (k) Попробуйте добавить 21 снова (не должно добавиться).
- (l) Выведите размер стека.

Пример использования:

```

stack = Stack(unique_per_session=True)
print(stack.push(7))    # True
print(stack.push(14))   # True
print(stack.push(7))    # False
print(stack.push(21))   # True
print(stack.push(14))   # False

print("Размер стека:", stack.size())
print("Верхний элемент:", stack.peek())

popped = stack.pop()
print("Вытолкнут:", popped) # 21

print(stack.push(21))    # False (уже был)
print("Размер стека:", stack.size()) # по-прежнему 2

```

12. Написать программу на Python, которая создает класс Stack для представления стека с инкапсуляцией. Класс должен содержать методы push, pop, is_empty, size и peek, которые реализуют операции вталкивания, выталкивания, проверки пустоты, получения размера и просмотра вершины стека соответственно. Программа также должна создавать экземпляр класса Stack, вталкивать в него элементы, выталкивать элементы и выводить информацию о стеке на экран.

Инструкции:

- (a) Создайте класс Stack с методом `__init__`, который инициализирует пустой стек. Принимает параметр `push_limit_per_call=1` (по умолчанию). Если >1 , то метод push может принимать несколько элементов (через `*args`) и добавлять их все за один вызов (но не более `push_limit_per_call` элементов за вызов).
- (b) Создайте метод push, который принимает один или несколько элементов (если `push_limit_per_call > 1`). Если передано больше элементов, чем `push_limit_per_call`, добавляются только первые `push_limit_per_call` элементов, остальные игнорируются. Возвращает количество реально добавленных элементов.
- (c) Создайте метод pop, который выталкивает верхний элемент из стека и возвращает его. Если стек пуст, возвращает None.
- (d) Создайте метод is_empty, который возвращает True, если стек пуст, и False в противном случае.
- (e) Создайте метод size, который возвращает текущее количество элементов в стеке.
- (f) Создайте метод peek, который возвращает верхний элемент стека, если стек не пуст. Если стек пуст, возвращает None.
- (g) Создайте экземпляр класса Stack с `push_limit_per_call=3`.
- (h) Вызовите push с элементами 1, 2, 3, 4, 5 — добавятся только 1,2,3.
- (i) Вызовите push с элементами 6, 7 — добавятся оба.
- (j) Выведите размер стека и верхний элемент.
- (k) Вызовите pop, выведите результат.
- (l) Повторите вывод размера и верхнего элемента.

Пример использования:

```

stack = Stack(push_limit_per_call=3)
added = stack.push(1, 2, 3, 4, 5) # добавим 1,2,3; вернет 3
print("Добавлено:", added)

added = stack.push(6, 7) # добавим 6,7; вернет 2
print("Добавлено:", added)

print("Размер стека:", stack.size())
print("Верхний элемент:", stack.peek())

popped = stack.pop()
print("Вытолкнут:", popped) # 7
print("Размер после pop:", stack.size())
print("Верхний элемент:", stack.peek()) # 6

```

13. Написать программу на Python, которая создает класс Stack для представления стека с инкапсуляцией. Класс должен содержать методы push, pop, is_empty, size и peek, которые реализуют операции вталкивания, выталкивания, проверки пустоты, получения размера и просмотра вершины стека соответственно. Программа также должна создавать экземпляр класса Stack, вталкивать в него элементы, выталкивать элементы и выводить информацию о стеке на экран.

Инструкции:

- (a) Создайте класс Stack с методом `__init__`, который инициализирует пустой стек. Принимает параметр `pop_multiple=False`. Если True, то метод pop может принимать необязательный аргумент count (по умолчанию 1) и возвращать список из count верхних элементов.
- (b) Создайте метод push, который принимает один элемент и добавляет его в стек. Возвращает None.
- (c) Создайте метод pop, который, если `pop_multiple=False`, выталкивает один верхний элемент и возвращает его. Если `pop_multiple=True`, принимает count (по умолчанию 1) и возвращает список из count верхних элементов (если запрошено больше, чем есть, возвращает все). Если стек пуст, возвращает пустой список [] (в режиме pop_multiple) или None (в обычном режиме).
- (d) Создайте метод is_empty, который возвращает True, если стек пуст, и False в противном случае.
- (e) Создайте метод size, который возвращает текущее количество элементов в стеке.
- (f) Создайте метод peek, который возвращает верхний элемент стека, если стек не пуст. Если стек пуст, возвращает None. Не поддерживает множественный просмотр.
- (g) Создайте экземпляр класса Stack с `pop_multiple=True`.
- (h) Добавьте элементы: 10, 20, 30, 40, 50.
- (i) Выведите размер стека и верхний элемент.
- (j) Вызовите pop с count=3, выведите результат (должен быть [50,40,30]).
- (k) Выведите размер стека и верхний элемент (теперь 20).

Пример использования:

```

stack = Stack(pop_multiple=True)
stack.push(10)
stack.push(20)
stack.push(30)
stack.push(40)
stack.push(50)

print("Размер стека:", stack.size())
print("Верхний элемент:", stack.peek())

popped = stack.pop(count=3)
print("Вытолкнуты:", popped) # [50, 40, 30]

print("Размер после pop:", stack.size())
print("Верхний элемент:", stack.peek()) # 20

```

14. Написать программу на Python, которая создает класс Stack для представления стека с инкапсуляцией. Класс должен содержать методы push, pop, is_empty, size и peek, которые реализуют операции вталкивания, выталкивания, проверки пустоты, получения размера и просмотра вершины стека соответственно. Программа также должна создавать экземпляр класса Stack, вталкивать в него элементы, выталкивать элементы и выводить информацию о стеке на экран.

Инструкции:

- (a) Создайте класс Stack с методом `__init__`, который инициализирует пустой стек. Принимает параметр `on_push_callback=None` — функция, которая будет вызываться после каждого успешного добавления элемента (с аргументом — добавленным элементом).
- (b) Создайте метод `push`, который принимает элемент и добавляет его в стек. Если `on_push_callback` не `None`, вызывает её с добавленным элементом. Возвращает добавленный элемент.
- (c) Создайте метод `pop`, который выталкивает верхний элемент из стека и возвращает его. Если стек пуст, возвращает `None`.
- (d) Создайте метод `is_empty`, который возвращает `True`, если стек пуст, и `False` в противном случае.
- (e) Создайте метод `size`, который возвращает текущее количество элементов в стеке.
- (f) Создайте метод `peek`, который возвращает верхний элемент стека, если стек не пуст. Если стек пуст, возвращает `None`.
- (g) Создайте функцию `logger(x)`: `print(f"[LOG] Добавлен: x")`
- (h) Создайте экземпляр класса Stack, передав `logger` в `on_push_callback`.
- (i) Добавьте элементы: 101, 202, 303 (при каждом добавлении должно выводиться сообщение).
- (j) Выведите размер стека и верхний элемент.
- (k) Вызовите `pop`, выведите результат.
- (l) Повторите вывод размера и верхнего элемента.

Пример использования:

```

def logger(x):
    print(f"[LOG] Добавлен: {x}")

stack = Stack(on_push_callback=logger)
stack.push(101) # выведет [LOG] Добавлен: 101
stack.push(202) # выведет [LOG] Добавлен: 202
stack.push(303) # выведет [LOG] Добавлен: 303

print("Размер стека:", stack.size())
print("Верхний элемент:", stack.peek())

popped = stack.pop()
print("Вытолкнут:", popped) # 303
print("Размер после pop:", stack.size())
print("Верхний элемент:", stack.peek()) # 202

```

15. Написать программу на Python, которая создает класс Stack для представления стека с инкапсуляцией. Класс должен содержать методы push, pop, is_empty, size и peek, которые реализуют операции вталкивания, выталкивания, проверки пустоты, получения размера и просмотра вершины стека соответственно. Программа также должна создавать экземпляр класса Stack, вталкивать в него элементы, выталкивать элементы и выводить информацию о стеке на экран.

Инструкции:

- (a) Создайте класс Stack с методом `__init__`, который инициализирует пустой стек. Принимает параметр `compress_on_push=False`. Если True, то при добавлении элемента, равного текущему верхнему, вместо добавления нового элемента увеличивается счетчик дубликатов у верхнего элемента (стек хранит пары (элемент, счетчик)).
- (b) Создайте метод push, который принимает элемент. Если `compress_on_push=True` и элемент равен текущему верхнему, увеличивает счетчик верхнего элемента. Иначе — добавляет новый элемент (со счетчиком 1, если режим сжатия включен).
- (c) Создайте метод pop, который выталкивает верхний элемент. Если режим сжатия включен и счетчик `>1`, уменьшает счетчик и возвращает элемент. Если счетчик=1, удаляет элемент. Если стек пуст, возвращает None.
- (d) Создайте метод is_empty, который возвращает True, если стек пуст, и False в противном случае.
- (e) Создайте метод size, который возвращает общее количество элементов (с учетом счетчиков, если режим сжатия включен).
- (f) Создайте метод peek, который возвращает верхний элемент (не счетчик, а само значение), если стек не пуст. Если стек пуст, возвращает None.
- (g) Создайте экземпляр класса Stack с `compress_on_push=True`.
- (h) Добавьте элементы: 5, 5, 5, 10, 10, 15.
- (i) Выведите размер стека (должен быть 6) и верхний элемент (15).
- (j) Вызовите pop, выведите результат (15).
- (k) Вызовите pop, выведите результат (10) — счетчик у 10 должен уменьшиться с 2 до 1.
- (l) Выведите размер стека (должен быть 4).

Пример использования:

```
stack = Stack(compress_on_push=True)
stack.push(5)
stack.push(5)
stack.push(5)
stack.push(10)
stack.push(10)
stack.push(15)

print("Размер стека:", stack.size())      # 6
print("Верхний элемент:", stack.peek())   # 15

popped = stack.pop()
print("Вытолкнут:", popped)              # 15

popped = stack.pop()
print("Вытолкнут:", popped)              # 10

print("Размер после двух pop:", stack.size()) # 4
```

16. Написать программу на Python, которая создает класс Stack для представления стека с инкапсуляцией. Класс должен содержать методы push, pop, is_empty, size и peek, которые реализуют операции вталкивания, выталкивания, проверки пустоты, получения размера и просмотра вершины стека соответственно. Программа также должна создавать экземпляр класса Stack, вталкивать в него элементы, выталкивать элементы и выводить информацию о стеке на экран.

Инструкции:

- (a) Создайте класс Stack с методом `__init__`, который инициализирует пустой стек. Принимает параметр `immutable_pop=False`. Если `True`, то метод `pop` не удаляет элемент из стека, а только возвращает его (поведение как `peek`, но называется `pop`).
- (b) Создайте метод `push`, который принимает элемент и добавляет его в стек.
- (c) Создайте метод `pop`, который, если `immutable_pop=False`, выталкивает верхний элемент и возвращает его. Если `immutable_pop=True`, возвращает верхний элемент, не удаляя его. Если стек пуст, возвращает `None`.
- (d) Создайте метод `is_empty`, который возвращает `True`, если стек пуст, и `False` в противном случае.
- (e) Создайте метод `size`, который возвращает текущее количество элементов в стеке.
- (f) Создайте метод `peek`, который возвращает верхний элемент стека, если стек не пуст. Если стек пуст, возвращает `None`. (Поведение не зависит от `immutable_pop`.)
- (g) Создайте экземпляр класса Stack с `immutable_pop=True`.
- (h) Добавьте элементы: 1, 3, 5, 7.
- (i) Выведите размер стека и результат `pop` (должен быть 7, но стек не изменится).
- (j) Снова вызовите `pop`, снова выведите результат (опять 7).
- (k) Выведите размер стека (по-прежнему 4).

Пример использования:

```

stack = Stack(immutable_pop=True)
stack.push(1)
stack.push(3)
stack.push(5)
stack.push(7)

print("Размер стека:", stack.size())
print("Первый pop:", stack.pop()) # 7
print("Второй pop:", stack.pop()) # 7 (стек не изменился)
print("Размер стека:", stack.size()) # 4

```

17. Написать программу на Python, которая создает класс Stack для представления стека с инкапсуляцией. Класс должен содержать методы push, pop, is_empty, size и peek, которые реализуют операции вталкивания, выталкивания, проверки пустоты, получения размера и просмотра вершины стека соответственно. Программа также должна создавать экземпляр класса Stack, вталкивать в него элементы, выталкивать элементы и выводить информацию о стеке на экран.

Инструкции:

- Создайте класс Stack с методом `__init__`, который инициализирует пустой стек. Принимает параметр `track_history=False`. Если True, то сохраняет историю всех когда-либо находившихся в стеке элементов (даже удаленных) в отдельном списке.
- Создайте метод `push`, который принимает элемент, добавляет его в стек, и если `track_history=True`, добавляет его и в историю.
- Создайте метод `pop`, который выталкивает верхний элемент из стека и возвращает его. Если стек пуст, возвращает None.
- Создайте метод `is_empty`, который возвращает True, если стек пуст, и False в противном случае.
- Создайте метод `size`, который возвращает текущее количество элементов в стеке.
- Создайте метод `peek`, который возвращает верхний элемент стека, если стек не пуст. Если стек пуст, возвращает None.
- Создайте метод `get_history` (только если `track_history=True`), который возвращает копию списка истории.
- Создайте экземпляр класса Stack с `track_history=True`.
 - Добавьте элементы: 2, 4, 6.
 - Вызовите `pop` (вернет 6).
 - Добавьте 8.
 - Выведите текущий стек (через `peek` и `size`) и историю (должна быть [2,4,6,8]).

Пример использования:

```

stack = Stack(track_history=True)
stack.push(2)
stack.push(4)
stack.push(6)
stack.pop() # 6
stack.push(8)

print("Текущий размер:", stack.size()) # 2
print("Верхний элемент:", stack.peek()) # 8
print("История:", stack.get_history()) # [2, 4, 6, 8]

```


18. Написать программу на Python, которая создает класс Stack для представления стека с инкапсуляцией. Класс должен содержать методы push, pop, is_empty, size и peek, которые реализуют операции вталкивания, выталкивания, проверки пустоты, получения размера и просмотра вершины стека соответственно. Программа также должна создавать экземпляр класса Stack, вталкивать в него элементы, выталкивать элементы и выводить информацию о стеке на экран.

Инструкции:

- (a) Создайте класс Stack с методом `__init__`, который инициализирует пустой стек. Принимает параметр `push_only_even=False`. Если `True`, то добавляются только четные числа (остальные игнорируются).
- (b) Создайте метод `push`, который принимает элемент. Если `push_only_even=True` и элемент не является четным целым числом, он не добавляется. Иначе — добавляется.
- (c) Создайте метод `pop`, который выталкивает верхний элемент из стека и возвращает его. Если стек пуст, возвращает `None`.
- (d) Создайте метод `is_empty`, который возвращает `True`, если стек пуст, и `False` в противном случае.
- (e) Создайте метод `size`, который возвращает текущее количество элементов в стеке.
- (f) Создайте метод `peek`, который возвращает верхний элемент стека, если стек не пуст. Если стек пуст, возвращает `None`.
- (g) Создайте экземпляр класса Stack с `push_only_even=True`.
- (h) Добавьте элементы: 1 (игнорируется), 2, 3 (игнорируется), 4, 5 (игнорируется), 6.
- (i) Выведите размер стека (должен быть 3) и верхний элемент (6).
- (j) Вызовите `pop`, выведите результат (6).
- (k) Повторите вывод размера и верхнего элемента (теперь 4).

Пример использования:

```
stack = Stack(push_only_even=True)
stack.push(1)  # игнорируется
stack.push(2)
stack.push(3)  # игнорируется
stack.push(4)
stack.push(5)  # игнорируется
stack.push(6)

print("Размер стека:", stack.size())      # 3
print("Верхний элемент:", stack.peek())   # 6

popped = stack.pop()
print("Вытолкнут:", popped)              # 6

print("Размер после pop:", stack.size())   # 2
print("Верхний элемент:", stack.peek())    # 4
```

19. Написать программу на Python, которая создает класс Stack для представления стека с инкапсуляцией. Класс должен содержать методы push, pop, is_empty, size и peek, которые реализуют операции вталкивания, выталкивания, проверки пустоты, получения размера и просмотра вершины стека соответственно. Программа также должна

создавать экземпляр класса `Stack`, вталкивать в него элементы, выталкивать элементы и выводить информацию о стеке на экран.

Инструкции:

- (a) Создайте класс `Stack` с методом `__init__`, который инициализирует пустой стек. Принимает параметр `reverse_pop=False`. Если `True`, то метод `pop` возвращает не верхний, а нижний элемент стека (и удаляет его).
- (b) Создайте метод `push`, который принимает элемент и добавляет его в стек (наверх).
- (c) Создайте метод `pop`, который, если `reverse_pop=False`, выталкивает верхний элемент и возвращает его. Если `reverse_pop=True`, выталкивает нижний элемент и возвращает его. Если стек пуст, возвращает `None`.
- (d) Создайте метод `is_empty`, который возвращает `True`, если стек пуст, и `False` в противном случае.
- (e) Создайте метод `size`, который возвращает текущее количество элементов в стеке.
- (f) Создайте метод `peek`, который возвращает верхний элемент стека, если стек не пуст. Если стек пуст, возвращает `None`. (Не зависит от `reverse_pop`.)
- (g) Создайте экземпляр класса `Stack` с `reverse_pop=True`.
- (h) Добавьте элементы: 10, 20, 30 (в стеке: [10,20,30], верх — 30).
- (i) Выведите результат `peek` (должен быть 30).
- (j) Вызовите `pop` — должен вернуться 10 (нижний), стек станет [20,30].
- (k) Выведите размер и снова `peek` (должен быть 30).

Пример использования:

```
stack = Stack(reverse_pop=True)
stack.push(10)
stack.push(20)
stack.push(30)

print("Верхний элемент (peek):", stack.peek()) # 30
popped = stack.pop()
print("Вытолкнут (нижний):", popped)          # 10
print("Размер после pop:", stack.size())        # 2
print("Верхний элемент (peek):", stack.peek()) # 30
```

20. Написать программу на Python, которая создает класс `Stack` для представления стека с инкапсуляцией. Класс должен содержать методы `push`, `pop`, `is_empty`, `size` и `peek`, которые реализуют операции вталкивания, выталкивания, проверки пустоты, получения размера и просмотра вершины стека соответственно. Программа также должна создавать экземпляр класса `Stack`, вталкивать в него элементы, выталкивать элементы и выводить информацию о стеке на экран.

Инструкции:

- (a) Создайте класс `Stack` с методом `__init__`, который инициализирует пустой стек. Принимает параметр `push_with_timestamp=False`. Если `True`, то при добавлении элемент сохраняется вместе с текущим временем (в формате Unix timestamp).
- (b) Создайте метод `push`, который принимает элемент. Если `push_with_timestamp=True`, сохраняет пару (элемент, `time.time()`). Иначе — только элемент.

- (c) Создайте метод `pop`, который выталкивает верхний элемент. Если режим с временем включен, возвращает пару (элемент, `timestamp`). Иначе — только элемент. Если стек пуст, возвращает `None`.
- (d) Создайте метод `is_empty`, который возвращает `True`, если стек пуст, и `False` в противном случае.
- (e) Создайте метод `size`, который возвращает текущее количество элементов в стеке.
- (f) Создайте метод `peek`, который возвращает верхний элемент (или пару, если включен режим времени), если стек не пуст. Если стек пуст, возвращает `None`.
- (g) Создайте экземпляр класса `Stack` с `push_with_timestamp=True`.
- (h) Добавьте элементы: "first" "second" "third".
- (i) Выведите размер стека и результат `peek` (должна быть пара ("third timestamp")).
- (j) Вызовите `pop`, выведите результат (тоже пара).
- (k) Повторите вывод размера и `peek`.

Пример использования:

```
import time

stack = Stack(push_with_timestamp=True)
stack.push("first")
stack.push("second")
stack.push("third")

print("Размер стека:", stack.size())
peek_result = stack.peek()
print("Верхний элемент и время:", peek_result)  # ('third',
1712345678.123456)

popped = stack.pop()
print("Вытолкнут:", popped)  # ('third', 1712345678.123456)

print("Размер после pop:", stack.size())
print("Верхний элемент:", stack.peek())  # ('second', ...)
```

21. Написать программу на Python, которая создает класс `Stack` для представления стека с инкапсуляцией. Класс должен содержать методы `push`, `pop`, `is_empty`, `size` и `peek`, которые реализуют операции вталкивания, выталкивания, проверки пустоты, получения размера и просмотра вершины стека соответственно. Программа также должна создавать экземпляр класса `Stack`, вталкивать в него элементы, выталкивать элементы и выводить информацию о стеке на экран.

Инструкции:

- (a) Создайте класс `Stack` с методом `__init__`, который инициализирует пустой стек. Принимает параметр `push_pairs=False`. Если `True`, то метод `push` ожидает два аргумента (`key`, `value`) и сохраняет их как кортеж. Если `False` — один аргумент.
- (b) Создайте метод `push`, который, если `push_pairs=False`, принимает один элемент. Если `push_pairs=True`, принимает два аргумента (`key`, `value`) и сохраняет (`key`, `value`). Возвращает сохраненный элемент (или кортеж).
- (c) Создайте метод `pop`, который выталкивает верхний элемент (или кортеж) и возвращает его. Если стек пуст, возвращает `None`.

- (d) Создайте метод `is_empty`, который возвращает `True`, если стек пуст, и `False` в противном случае.
- (e) Создайте метод `size`, который возвращает текущее количество элементов в стеке.
- (f) Создайте метод `peek`, который возвращает верхний элемент (или кортеж), если стек не пуст. Если стек пуст, возвращает `None`.
- (g) Создайте экземпляр класса `Stack` с `push_pairs=True`.
- (h) Добавьте пары: `("a 1")`, `("b 2")`, `("c 3")`.
- (i) Выведите размер стека и результат `peek` (должен быть `("c 3")`).
- (j) Вызовите `pop`, выведите результат.
- (k) Повторите вывод размера и `peek`.

Пример использования:

```
stack = Stack(push_pairs=True)
stack.push("a", 1)
stack.push("b", 2)
stack.push("c", 3)

print("Размер стека:", stack.size())
print("Верхний элемент:", stack.peek()) # ('c', 3)

popped = stack.pop()
print("Вытолкнут:", popped) # ('c', 3)

print("Размер после pop:", stack.size())
print("Верхний элемент:", stack.peek()) # ('b', 2)
```

22. Написать программу на Python, которая создает класс `Stack` для представления стека с инкапсуляцией. Класс должен содержать методы `push`, `pop`, `is_empty`, `size` и `peek`, которые реализуют операции вталкивания, выталкивания, проверки пустоты, получения размера и просмотра вершины стека соответственно. Программа также должна создавать экземпляр класса `Stack`, вталкивать в него элементы, выталкивать элементы и выводить информацию о стеке на экран.

Инструкции:

- (a) Создайте класс `Stack` с методом `__init__`, который инициализирует пустой стек. Принимает параметр `auto_dedup=False`. Если `True`, то при добавлении элемента, который уже есть в стеке (не обязательно на вершине), сначала удаляет все его предыдущие вхождения.
- (b) Создайте метод `push`, который принимает элемент. Если `auto_dedup=True` и такой элемент уже есть в стеке, удаляет все его вхождения, затем добавляет новый элемент. Иначе — просто добавляет.
- (c) Создайте метод `pop`, который выталкивает верхний элемент из стека и возвращает его. Если стек пуст, возвращает `None`.
- (d) Создайте метод `is_empty`, который возвращает `True`, если стек пуст, и `False` в противном случае.
- (e) Создайте метод `size`, который возвращает текущее количество элементов в стеке.
- (f) Создайте метод `peek`, который возвращает верхний элемент стека, если стек не пуст. Если стек пуст, возвращает `None`.

- (g) Создайте экземпляр класса Stack с `auto_dedup=True`.
- (h) Добавьте элементы: 1, 2, 1, 3, 2, 4.
- (i) После каждого добавления выводите содержимое стека (реализуйте вспомогательный метод `_debug_list`, возвращающий список элементов снизу вверх — только для отладки, не включайте в задание студентам; в решении можно использовать `stack._items`, если инкапсуляция не строгая).
- (j) Выведите итоговый размер и верхний элемент.

Пример использования (с отладочным выводом для ясности):

```
# (В решении студент не обязан реализовывать _debug_list, но для проверки мож
но временно добавить)
stack = Stack(auto_dedup=True)
stack.push(1)    # стек: [1]
stack.push(2)    # стек: [1, 2]
stack.push(1)    # удаляет старую 1, добавляет новую -> [2, 1]
stack.push(3)    # [2, 1, 3]
stack.push(2)    # удаляет 2, добавляет новую -> [1, 3, 2]
stack.push(4)    # [1, 3, 2, 4]

print("Размер стека:", stack.size())    # 4
print("Верхний элемент:", stack.peek()) # 4
```

23. Написать программу на Python, которая создает класс Stack для представления стека с инкапсуляцией. Класс должен содержать методы `push`, `pop`, `is_empty`, `size` и `peek`, которые реализуют операции вталкивания, выталкивания, проверки пустоты, получения размера и просмотра вершины стека соответственно. Программа также должна создавать экземпляр класса Stack, вталкивать в него элементы, выталкивать элементы и выводить информацию о стеке на экран.

Инструкции:

- (a) Создайте класс Stack с методом `__init__`, который инициализирует пустой стек. Принимает параметр `push_if_max=False`. Если True, то элемент добавляется только если он больше всех текущих элементов в стеке.
- (b) Создайте метод `push`, который принимает элемент. Если `push_if_max=True` и элемент не является строго больше всех элементов в стеке, он не добавляется. Иначе — добавляется.
- (c) Создайте метод `pop`, который выталкивает верхний элемент из стека и возвращает его. Если стек пуст, возвращает None.
- (d) Создайте метод `is_empty`, который возвращает True, если стек пуст, и False в противном случае.
- (e) Создайте метод `size`, который возвращает текущее количество элементов в стеке.
- (f) Создайте метод `peek`, который возвращает верхний элемент стека, если стек не пуст. Если стек пуст, возвращает None.
- (g) Создайте экземпляр класса Stack с `push_if_max=True`.
- (h) Добавьте элементы: 5, 3 (не добавится, т.к. $3 < 5$), 10, 7 (не добавится, т.к. $7 < 10$), 15.
- (i) Выведите размер стека (должен быть 3) и верхний элемент (15).
- (j) Вызовите `pop`, выведите результат (15).

(к) Повторите вывод размера и верхнего элемента (теперь 10).

Пример использования:

```
stack = Stack(push_if_max=True)
stack.push(5)
stack.push(3)    # не добавится
stack.push(10)
stack.push(7)    # не добавится
stack.push(15)

print("Размер стека:", stack.size())    # 3
print("Верхний элемент:", stack.peek()) # 15

popped = stack.pop()
print("Вытолкнут:", popped)    # 15

print("Размер после pop:", stack.size())    # 2
print("Верхний элемент:", stack.peek())    # 10
```

24. Написать программу на Python, которая создает класс Stack для представления стека с инкапсуляцией. Класс должен содержать методы push, pop, is_empty, size и peek, которые реализуют операции вталкивания, выталкивания, проверки пустоты, получения размера и просмотра вершины стека соответственно. Программа также должна создавать экземпляр класса Stack, вталкивать в него элементы, выталкивать элементы и выводить информацию о стеке на экран.

Инструкции:

- (a) Создайте класс Stack с методом `__init__`, который инициализирует пустой стек. Принимает параметр `cumulative=False`. Если `True`, то при добавлении элемента он суммируется с предыдущим верхним элементом (первый элемент добавляется как есть).
- (b) Создайте метод `push`, который принимает элемент. Если `cumulative=True` и стек не пуст, то добавляемый элемент становится `element + текущий_верх`. Затем этот результат добавляется в стек. Если стек пуст, добавляется `element` как есть.
- (c) Создайте метод `pop`, который выталкивает верхний элемент из стека и возвращает его. Если стек пуст, возвращает `None`.
- (d) Создайте метод `is_empty`, который возвращает `True`, если стек пуст, и `False` в противном случае.
- (e) Создайте метод `size`, который возвращает текущее количество элементов в стеке.
- (f) Создайте метод `peek`, который возвращает верхний элемент стека, если стек не пуст. Если стек пуст, возвращает `None`.
- (g) Создайте экземпляр класса Stack с `cumulative=True`.
- (h) Добавьте элементы: 1, 2, 3, 4.
- (i) Выведите содержимое стека после каждого добавления (для проверки: после 1 $\rightarrow [1]$; после 2 $\rightarrow [1,3]$; после 3 $\rightarrow [1,3,6]$; после 4 $\rightarrow [1,3,6,10]$).
- (j) Выведите итоговый размер и верхний элемент (10).
- (к) Вызовите `pop`, выведите результат (10).
- (l) Повторите вывод размера и верхнего элемента (теперь 6).

Пример использования:

```
stack = Stack(cumulative=True)
stack.push(1) # [1]
stack.push(2) # [1, 1+2=3]
stack.push(3) # [1, 3, 3+3=6]
stack.push(4) # [1, 3, 6, 6+4=10]

print("Размер стека:", stack.size()) # 4
print("Верхний элемент:", stack.peek()) # 10

popped = stack.pop()
print("Вытолкнут:", popped) # 10

print("Размер после pop:", stack.size()) # 3
print("Верхний элемент:", stack.peek()) # 6
```

25. Написать программу на Python, которая создает класс Stack для представления стека с инкапсуляцией. Класс должен содержать методы push, pop, is_empty, size и peek, которые реализуют операции вталкивания, выталкивания, проверки пустоты, получения размера и просмотра вершины стека соответственно. Программа также должна создавать экземпляр класса Stack, вталкивать в него элементы, выталкивать элементы и выводить информацию о стеке на экран.

Инструкции:

- (a) Создайте класс Stack с методом `__init__`, который инициализирует пустой стек. Принимает параметр `push_squared=False`. Если True, то при добавлении элемент возводится в квадрат перед добавлением.
- (b) Создайте метод push, который принимает элемент. Если `push_squared=True`, добавляет `element**2`. Иначе — `element`.
- (c) Создайте метод pop, который выталкивает верхний элемент из стека и возвращает его. Если стек пуст, возвращает None.
- (d) Создайте метод is_empty, который возвращает True, если стек пуст, и False в противном случае.
- (e) Создайте метод size, который возвращает текущее количество элементов в стеке.
- (f) Создайте метод peek, который возвращает верхний элемент стека, если стек не пуст. Если стек пуст, возвращает None.
- (g) Создайте экземпляр класса Stack с `push_squared=True`.
- (h) Добавьте элементы: 2, 3, 4, 5.
- (i) Выведите размер стека и верхний элемент (должен быть 25).
- (j) Вызовите pop, выведите результат (25).
- (k) Повторите вывод размера и верхнего элемента (теперь 16).

Пример использования:

```
stack = Stack(push_squared=True)
stack.push(2) # добавим 4
stack.push(3) # добавим 9
stack.push(4) # добавим 16
stack.push(5) # добавим 25

print("Размер стека:", stack.size()) # 4
```

```

print("Верхний элемент:", stack.peek())    # 25

popped = stack.pop()
print("Вытолкнут:", popped)    # 25

print("Размер после pop:", stack.size())    # 3
print("Верхний элемент:", stack.peek())    # 16

```

26. Написать программу на Python, которая создает класс Stack для представления стека с инкапсуляцией. Класс должен содержать методы push, pop, is_empty, size и peek, которые реализуют операции вталкивания, выталкивания, проверки пустоты, получения размера и просмотра вершины стека соответственно. Программа также должна создавать экземпляр класса Stack, вталкивать в него элементы, выталкивать элементы и выводить информацию о стеке на экран.

Инструкции:

- Создайте класс Stack с методом `__init__`, который инициализирует пустой стек. Принимает параметр `push_absolute=False`. Если `True`, то при добавлении сохраняется абсолютное значение элемента (`abs(element)`).
- Создайте метод `push`, который принимает элемент. Если `push_absolute=True`, добавляет `abs(element)`. Иначе — `element`.
- Создайте метод `pop`, который выталкивает верхний элемент из стека и возвращает его. Если стек пуст, возвращает `None`.
- Создайте метод `is_empty`, который возвращает `True`, если стек пуст, и `False` в противном случае.
- Создайте метод `size`, который возвращает текущее количество элементов в стеке.
- Создайте метод `peek`, который возвращает верхний элемент стека, если стек не пуст. Если стек пуст, возвращает `None`.
- Создайте экземпляр класса Stack с `push_absolute=True`.
- Добавьте элементы: -5, 3, -8, 2.
- Выведите размер стека и верхний элемент (должен быть 2).
- Вызовите `pop`, выведите результат (2).
- Повторите вывод размера и верхнего элемента (теперь 8).

Пример использования:

```

stack = Stack(push_absolute=True)
stack.push(-5)    # добавим 5
stack.push(3)     # добавим 3
stack.push(-8)    # добавим 8
stack.push(2)     # добавим 2

print("Размер стека:", stack.size())    # 4
print("Верхний элемент:", stack.peek())    # 2

popped = stack.pop()
print("Вытолкнут:", popped)    # 2

print("Размер после pop:", stack.size())    # 3
print("Верхний элемент:", stack.peek())    # 8

```


27. Написать программу на Python, которая создает класс `Stack` для представления стека с инкапсуляцией. Класс должен содержать методы `push`, `pop`, `is_empty`, `size` и `peek`, которые реализуют операции вталкивания, выталкивания, проверки пустоты, получения размера и просмотра вершины стека соответственно. Программа также должна создавать экземпляр класса `Stack`, вталкивать в него элементы, выталкивать элементы и выводить информацию о стеке на экран.

Инструкции:

- (a) Создайте класс `Stack` с методом `__init__`, который инициализирует пустой стек. Принимает параметр `push_rounded=False`. Если `True`, то при добавлении элемент округляется до целого числа (`round(element)`).
- (b) Создайте метод `push`, который принимает элемент. Если `push_rounded=True`, добавляет `round(element)`. Иначе — `element`.
- (c) Создайте метод `pop`, который выталкивает верхний элемент из стека и возвращает его. Если стек пуст, возвращает `None`.
- (d) Создайте метод `is_empty`, который возвращает `True`, если стек пуст, и `False` в противном случае.
- (e) Создайте метод `size`, который возвращает текущее количество элементов в стеке.
- (f) Создайте метод `peek`, который возвращает верхний элемент стека, если стек не пуст. Если стек пуст, возвращает `None`.
- (g) Создайте экземпляр класса `Stack` с `push_rounded=True`.
- (h) Добавьте элементы: 3.2, 4.7, 5.1, 6.9.
- (i) Выведите размер стека и верхний элемент (должен быть 7).
- (j) Вызовите `pop`, выведите результат (7).
- (k) Повторите вывод размера и верхнего элемента (теперь 5).

Пример использования:

```
stack = Stack(push_rounded=True)
stack.push(3.2)    # 3
stack.push(4.7)    # 5
stack.push(5.1)    # 5
stack.push(6.9)    # 7

print("Размер стека:", stack.size())    # 4
print("Верхний элемент:", stack.peek())  # 7

popped = stack.pop()
print("Вытолкнут:", popped)    # 7

print("Размер после pop:", stack.size())    # 3
print("Верхний элемент:", stack.peek())    # 5
```

28. Написать программу на Python, которая создает класс `Stack` для представления стека с инкапсуляцией. Класс должен содержать методы `push`, `pop`, `is_empty`, `size` и `peek`, которые реализуют операции вталкивания, выталкивания, проверки пустоты, получения размера и просмотра вершины стека соответственно. Программа также должна создавать экземпляр класса `Stack`, вталкивать в него элементы, выталкивать элементы и выводить информацию о стеке на экран.

Инструкции:

- (a) Создайте класс `Stack` с методом `__init__`, который инициализирует пустой стек. Принимает параметр `push_negated=False`. Если `True`, то при добавлении элемент сохраняется с обратным знаком (`-element`).
- (b) Создайте метод `push`, который принимает элемент. Если `push_negated=True`, добавляет `-element`. Иначе — `element`.
- (c) Создайте метод `pop`, который выталкивает верхний элемент из стека и возвращает его. Если стек пуст, возвращает `None`.
- (d) Создайте метод `is_empty`, который возвращает `True`, если стек пуст, и `False` в противном случае.
- (e) Создайте метод `size`, который возвращает текущее количество элементов в стеке.
- (f) Создайте метод `peek`, который возвращает верхний элемент стека, если стек не пуст. Если стек пуст, возвращает `None`.
- (g) Создайте экземпляр класса `Stack` с `push_negated=True`.
- (h) Добавьте элементы: 10, 20, 30, 40.
- (i) Выведите размер стека и верхний элемент (должен быть -40).
- (j) Вызовите `pop`, выведите результат (-40).
- (k) Повторите вывод размера и верхнего элемента (теперь -30).

Пример использования:

```
stack = Stack(push_negated=True)
stack.push(10) # -10
stack.push(20) # -20
stack.push(30) # -30
stack.push(40) # -40

print("Размер стека:", stack.size()) # 4
print("Верхний элемент:", stack.peek()) # -40

popped = stack.pop()
print("Вытолкнут:", popped) # -40

print("Размер после pop:", stack.size()) # 3
print("Верхний элемент:", stack.peek()) # -30
```

29. Написать программу на Python, которая создает класс `Stack` для представления стека с инкапсуляцией. Класс должен содержать методы `push`, `pop`, `is_empty`, `size` и `peek`, которые реализуют операции вталкивания, выталкивания, проверки пустоты, получения размера и просмотра вершины стека соответственно. Программа также должна создавать экземпляр класса `Stack`, вталкивать в него элементы, выталкивать элементы и выводить информацию о стеке на экран.

Инструкции:

- (a) Создайте класс `Stack` с методом `__init__`, который инициализирует пустой стек. Принимает параметр `push_doubled=False`. Если `True`, то при добавлении элемент умножается на 2.
- (b) Создайте метод `push`, который принимает элемент. Если `push_doubled=True`, добавляет `element * 2`. Иначе — `element`.
- (c) Создайте метод `pop`, который выталкивает верхний элемент из стека и возвращает его. Если стек пуст, возвращает `None`.

- (d) Создайте метод `is_empty`, который возвращает `True`, если стек пуст, и `False` в противном случае.
- (e) Создайте метод `size`, который возвращает текущее количество элементов в стеке.
- (f) Создайте метод `peek`, который возвращает верхний элемент стека, если стек не пуст. Если стек пуст, возвращает `None`.
- (g) Создайте экземпляр класса `Stack` с `push_doubled=True`.
- (h) Добавьте элементы: 1, 2, 3, 4.
- (i) Выведите размер стека и верхний элемент (должен быть 8).
- (j) Вызовите `pop`, выведите результат (8).
- (k) Повторите вывод размера и верхнего элемента (теперь 6).

Пример использования:

```
stack = Stack(push_doubled=True)
stack.push(1) # 2
stack.push(2) # 4
stack.push(3) # 6
stack.push(4) # 8

print("Размер стека:", stack.size()) # 4
print("Верхний элемент:", stack.peek()) # 8

popped = stack.pop()
print("Вытолкнут:", popped) # 8

print("Размер после pop:", stack.size()) # 3
print("Верхний элемент:", stack.peek()) # 6
```

30. Написать программу на Python, которая создает класс `Stack` для представления стека с инкапсуляцией. Класс должен содержать методы `push`, `pop`, `is_empty`, `size` и `peek`, которые реализуют операции вталкивания, выталкивания, проверки пустоты, получения размера и просмотра вершины стека соответственно. Программа также должна создавать экземпляр класса `Stack`, вталкивать в него элементы, выталкивать элементы и выводить информацию о стеке на экран.

Инструкции:

- (a) Создайте класс `Stack` с методом `__init__`, который инициализирует пустой стек. Принимает параметр `push_halved=False`. Если `True`, то при добавлении элемент делится на 2.0.
- (b) Создайте метод `push`, который принимает элемент. Если `push_halved=True`, добавляет `element / 2.0`. Иначе — `element`.
- (c) Создайте метод `pop`, который выталкивает верхний элемент из стека и возвращает его. Если стек пуст, возвращает `None`.
- (d) Создайте метод `is_empty`, который возвращает `True`, если стек пуст, и `False` в противном случае.
- (e) Создайте метод `size`, который возвращает текущее количество элементов в стеке.
- (f) Создайте метод `peek`, который возвращает верхний элемент стека, если стек не пуст. Если стек пуст, возвращает `None`.
- (g) Создайте экземпляр класса `Stack` с `push_halved=True`.

- (h) Добавьте элементы: 4, 8, 12, 16.
- (i) Выведите размер стека и верхний элемент (должен быть 8.0).
- (j) Вызовите pop, выведите результат (8.0).
- (k) Повторите вывод размера и верхнего элемента (теперь 6.0).

Пример использования:

```
stack = Stack(push_halved=True)
stack.push(4)    # 2.0
stack.push(8)    # 4.0
stack.push(12)   # 6.0
stack.push(16)   # 8.0

print("Размер стека:", stack.size())    # 4
print("Верхний элемент:", stack.peek()) # 8.0

popped = stack.pop()
print("Вытолкнут:", popped)    # 8.0

print("Размер после pop:", stack.size())    # 3
print("Верхний элемент:", stack.peek())    # 6.0
```

31. Написать программу на Python, которая создает класс Stack для представления стека с инкапсуляцией. Класс должен содержать методы push, pop, is_empty, size и peek, которые реализуют операции вталкивания, выталкивания, проверки пустоты, получения размера и просмотра вершины стека соответственно. Программа также должна создавать экземпляр класса Stack, вталкивать в него элементы, выталкивать элементы и выводить информацию о стеке на экран.

Инструкции:

- (a) Создайте класс Stack с методом `__init__`, который инициализирует пустой стек. Принимает параметр `push_as_string=False`. Если True, то при добавлении элемент преобразуется в строку `str(element)`.
- (b) Создайте метод `push`, который принимает элемент. Если `push_as_string=True`, добавляет `str(element)`. Иначе — `element`.
- (c) Создайте метод `pop`, который выталкивает верхний элемент из стека и возвращает его. Если стек пуст, возвращает None.
- (d) Создайте метод `is_empty`, который возвращает True, если стек пуст, и False в противном случае.
- (e) Создайте метод `size`, который возвращает текущее количество элементов в стеке.
- (f) Создайте метод `peek`, который возвращает верхний элемент стека, если стек не пуст. Если стек пуст, возвращает None.
- (g) Создайте экземпляр класса Stack с `push_as_string=True`.
- (h) Добавьте элементы: 100, 200, 300, 400.
- (i) Выведите размер стека и верхний элемент (должен быть "400").
- (j) Вызовите pop, выведите результат ("400").
- (k) Повторите вывод размера и верхнего элемента (теперь "300").

Пример использования:

```

stack = Stack(push_as_string=True)
stack.push(100) # "100"
stack.push(200) # "200"
stack.push(300) # "300"
stack.push(400) # "400"

print("Размер стека:", stack.size()) # 4
print("Верхний элемент:", stack.peek()) # "400"

popped = stack.pop()
print("Вытолкнут:", popped) # "400"

print("Размер после pop:", stack.size()) # 3
print("Верхний элемент:", stack.peek()) # "300"

```

32. Написать программу на Python, которая создает класс Stack для представления стека с инкапсуляцией. Класс должен содержать методы push, pop, is_empty, size и peek, которые реализуют операции вталкивания, выталкивания, проверки пустоты, получения размера и просмотра вершины стека соответственно. Программа также должна создавать экземпляр класса Stack, вталкивать в него элементы, выталкивать элементы и выводить информацию о стеке на экран.

Инструкции:

- Создайте класс Stack с методом `__init__`, который инициализирует пустой стек. Принимает параметр `push_with_index=False`. Если True, то при добавлении сохраняется кортеж (элемент, порядковый номер добавления).
- Создайте метод push, который принимает элемент. Если `push_with_index=True`, добавляет (element, self._counter), где `_counter` — внутренний счетчик, увеличивающийся при каждом добавлении. Иначе — element.
- Создайте метод pop, который выталкивает верхний элемент (или кортеж) и возвращает его. Если стек пуст, возвращает None.
- Создайте метод is_empty, который возвращает True, если стек пуст, и False в противном случае.
- Создайте метод size, который возвращает текущее количество элементов в стеке.
- Создайте метод peek, который возвращает верхний элемент (или кортеж), если стек не пуст. Если стек пуст, возвращает None.
- Создайте экземпляр класса Stack с `push_with_index=True`.
- Добавьте элементы: "alpha" "beta" "gamma".
- Выведите размер стека и результат peek (должен быть ("gamma 2) — если считать с 0).
- Вызовите pop, выведите результат.
- Повторите вывод размера и peek.

Пример использования:

```

stack = Stack(push_with_index=True)
stack.push("alpha") # ("alpha", 0)
stack.push("beta") # ("beta", 1)
stack.push("gamma") # ("gamma", 2)

print("Размер стека:", stack.size())

```

```

print("Верхний элемент:", stack.peek()) # ('gamma', 2)

popped = stack.pop()
print("Вытолкнут:", popped) # ('gamma', 2)

print("Размер после pop:", stack.size())
print("Верхний элемент:", stack.peek()) # ('beta', 1)

```

33. Написать программу на Python, которая создает класс Stack для представления стека с инкапсуляцией. Класс должен содержать методы push, pop, is_empty, size и peek, которые реализуют операции вталкивания, выталкивания, проверки пустоты, получения размера и просмотра вершины стека соответственно. Программа также должна создавать экземпляр класса Stack, вталкивать в него элементы, выталкивать элементы и выводить информацию о стеке на экран.

Инструкции:

- Создайте класс Stack с методом `__init__`, который инициализирует пустой стек. Принимает параметр `push_unique_top=False`. Если True, то при добавлении, если элемент равен текущему верхнему, он не добавляется.
- Создайте метод push, который принимает элемент. Если `push_unique_top=True` и стек не пуст и `element == текущий_верх`, то элемент не добавляется. Иначе — добавляется.
- Создайте метод pop, который выталкивает верхний элемент из стека и возвращает его. Если стек пуст, возвращает None.
- Создайте метод is_empty, который возвращает True, если стек пуст, и False в противном случае.
- Создайте метод size, который возвращает текущее количество элементов в стеке.
- Создайте метод peek, который возвращает верхний элемент стека, если стек не пуст. Если стек пуст, возвращает None.
- Создайте экземпляр класса Stack с `push_unique_top=True`.
- Добавьте элементы: 1, 2, 2, 3, 3, 3, 4.
- Выведите размер стека (должен быть 4) и верхний элемент (4).
- Вызовите pop, выведите результат (4).
- Повторите вывод размера и верхнего элемента (теперь 3).

Пример использования:

```

stack = Stack(push_unique_top=True)
stack.push(1)
stack.push(2)
stack.push(2) # не добавится
stack.push(3)
stack.push(3) # не добавится
stack.push(3) # не добавится
stack.push(4)

print("Размер стека:", stack.size()) # 4
print("Верхний элемент:", stack.peek()) # 4

popped = stack.pop()
print("Вытолкнут:", popped) # 4

```

```
print("Размер после pop:", stack.size())      # 3
print("Верхний элемент:", stack.peek())      # 3
```

34. Написать программу на Python, которая создает класс Stack для представления стека с инкапсуляцией. Класс должен содержать методы push, pop, is_empty, size и peek, которые реализуют операции вталкивания, выталкивания, проверки пустоты, получения размера и просмотра вершины стека соответственно. Программа также должна создавать экземпляр класса Stack, вталкивать в него элементы, выталкивать элементы и выводить информацию о стеке на экран.

Инструкции:

- Создайте класс Stack с методом `__init__`, который инициализирует пустой стек. Принимает параметр `push_even_only=False`. Если `True`, то добавляются только четные числа.
- Создайте метод `push`, который принимает элемент. Если `push_even_only=True` и `element % 2 != 0`, элемент не добавляется. Иначе — добавляется.
- Создайте метод `pop`, который выталкивает верхний элемент из стека и возвращает его. Если стек пуст, возвращает `None`.
- Создайте метод `is_empty`, который возвращает `True`, если стек пуст, и `False` в противном случае.
- Создайте метод `size`, который возвращает текущее количество элементов в стеке.
- Создайте метод `peek`, который возвращает верхний элемент стека, если стек не пуст. Если стек пуст, возвращает `None`.
- Создайте экземпляр класса Stack с `push_even_only=True`.
- Добавьте элементы: 1 (не добавится), 2, 3 (не добавится), 4, 5 (не добавится), 6.
- Выведите размер стека (должен быть 3) и верхний элемент (6).
- Вызовите `pop`, выведите результат (6).
- Повторите вывод размера и верхнего элемента (теперь 4).

Пример использования:

```
stack = Stack(push_even_only=True)
stack.push(1)  # нет
stack.push(2)
stack.push(3)  # нет
stack.push(4)
stack.push(5)  # нет
stack.push(6)

print("Размер стека:", stack.size())      # 3
print("Верхний элемент:", stack.peek())    # 6

popped = stack.pop()
print("Вытолкнут:", popped)              # 6

print("Размер после pop:", stack.size())    # 2
print("Верхний элемент:", stack.peek())     # 4
```

35. Написать программу на Python, которая создает класс Stack для представления стека с инкапсуляцией. Класс должен содержать методы push, pop, is_empty, size и peek, которые реализуют операции вталкивания, выталкивания, проверки пустоты, получения размера и просмотра вершины стека соответственно. Программа также должна создавать экземпляры класса Stack, вталкивать в него элементы, выталкивать элементы и выводить информацию о стеке на экран.

Инструкции:

- (a) Создайте класс Stack с методом `__init__`, который инициализирует пустой стек. Принимает параметр `push_odd_only=False`. Если True, то добавляются только нечетные числа.
- (b) Создайте метод push, который принимает элемент. Если `push_odd_only=True` и `element % 2 == 0`, элемент не добавляется. Иначе — добавляется.
- (c) Создайте метод pop, который выталкивает верхний элемент из стека и возвращает его. Если стек пуст, возвращает None.
- (d) Создайте метод is_empty, который возвращает True, если стек пуст, и False в противном случае.
- (e) Создайте метод size, который возвращает текущее количество элементов в стеке.
- (f) Создайте метод peek, который возвращает верхний элемент стека, если стек не пуст. Если стек пуст, возвращает None.
- (g) Создайте экземпляр класса Stack с `push_odd_only=True`.
- (h) Добавьте элементы: 2 (не добавится), 1, 4 (не добавится), 3, 6 (не добавится), 5.
- (i) Выведите размер стека (должен быть 3) и верхний элемент (5).
- (j) Вызовите pop, выведите результат (5).
- (k) Повторите вывод размера и верхнего элемента (теперь 3).

Пример использования:

```
stack = Stack(push_odd_only=True)
stack.push(2)    # нет
stack.push(1)
stack.push(4)    # нет
stack.push(3)
stack.push(6)    # нет
stack.push(5)

print("Размер стека:", stack.size())    # 3
print("Верхний элемент:", stack.peek()) # 5

popped = stack.pop()
print("Вытолкнут:", popped)    # 5

print("Размер после pop:", stack.size())    # 2
print("Верхний элемент:", stack.peek())    # 3
```

2.3.3 Задача 3 (двусвязный список)

1. Написать программу на Python, которая создает класс DoublyLinkedList, представляющий **двусвязный список** с инкапсуляцией внутренней структуры. Класс должен

содержать методы для отображения данных, вставки и удаления узлов. Программа также должна создавать экземпляр класса, вставлять узлы и удалять узлы.

Инструкции:

- (a) Создайте класс Node с методом `__init__`, который принимает данные data и сохраняет их в атрибуте `self._data`. Также инициализирует `self._next` и `self._prev` как None.
- (b) Создайте класс DoublyLinkedList с методом `__init__`, который инициализирует `self._head` и `self._tail` как None.
- (c) Создайте метод `display` в классе DoublyLinkedList, который выводит все элементы списка через пробел, двигаясь от головы к хвосту. Если список пуст — выводит "Список пуст".
- (d) Создайте метод `insert` в классе DoublyLinkedList, который принимает значение и вставляет новый узел **в конец списка**. Обновляет `self._tail` и ссылки `prev/next`.
- (e) Создайте метод `delete` в классе DoublyLinkedList, который принимает значение и удаляет **первое** вхождение узла с этим значением. Корректно обновляет соседние ссылки и `self._head/self._tail` при необходимости.
- (f) Создайте экземпляр класса DoublyLinkedList.
- (g) Вставьте узлы со значениями 10, 20, 30, 40.
- (h) Вызовите `display` и выведите результат.
- (i) Вставьте узел со значением 50.
- (j) Снова вызовите `display`.
- (k) Удалите узел со значением 20.
- (l) Снова вызовите `display`.

Пример использования:

```
dll = DoublyLinkedList()
dll.insert(10)
dll.insert(20)
dll.insert(30)
dll.insert(40)

print("Initial Doubly Linked List:")
dll.display()

dll.insert(50)
print("After inserting 50:")
dll.display()

dll.delete(20)
print("After deleting 20:")
dll.display()
```

2. Написать программу на Python, которая создает класс DoublyLinkedList, представляющий **двусвязный список** с инкапсуляцией. Класс должен содержать методы для отображения данных, вставки и удаления узлов. Программа также должна создавать экземпляр класса, вставлять узлы и удалять узлы.

Инструкции:

- (a) Создайте класс Node с методом `__init__`, который принимает `item` и сохраняет его в `self._value`. Инициализирует `self._next` и `self._previous` как `None`.
- (b) Создайте класс `DoublyLinkedList` с методом `__init__`, который инициализирует `self._first` и `self._last` как `None`.
- (c) Создайте метод `display` в классе `DoublyLinkedList`, который выводит элементы списка от первого к последнему, разделенные запятыми. Если список пуст — выводит "Нет элементов".
- (d) Создайте метод `insert` в классе `DoublyLinkedList`, который принимает элемент и вставляет его **в начало списка**. Обновляет `self._first` и ссылки.
- (e) Создайте метод `delete` в классе `DoublyLinkedList`, который принимает значение и удаляет **последнее** вхождение узла с этим значением. Корректно обновляет связи и границы списка.
- (f) Создайте экземпляр класса `DoublyLinkedList`.
- (g) Вставьте узлы: 5, 15, 25, 15.
- (h) Вызовите `display`.
- (i) Вставьте узел 35 в начало.
- (j) Снова вызовите `display`.
- (k) Удалите последнее вхождение 15.
- (l) Снова вызовите `display`.

Пример использования:

```
dll = DoublyLinkedList()
dll.insert(5)
dll.insert(15)
dll.insert(25)
dll.insert(15)

print("Initial Doubly Linked List:")
dll.display()

dll.insert(35)
print("After inserting 35 at start:")
dll.display()

dll.delete(15)
print("After deleting last occurrence of 15:")
dll.display()
```

3. Написать программу на Python, которая создает класс `DoublyLinkedList`, представляющий **двусвязный список** с инкапсуляцией. Класс должен содержать методы для отображения данных, вставки и удаления узлов. Программа также должна создавать экземпляр класса, вставлять узлы и удалять узлы.

Инструкции:

- (a) Создайте класс Node с методом `__init__`, который принимает `content` и сохраняет его в `self._payload`. Инициализирует `self._forward` и `self._backward` как `None`.
- (b) Создайте класс `DoublyLinkedList` с методом `__init__`, который инициализирует `self._root` и `self._end` как `None`.

- (c) Создайте метод `display` в классе `DoublyLinkedList`, который выводит элементы в формате "[элемент1] <-> [элемент2] <-> ...". Если пуст — "Пусто".
- (d) Создайте метод `insert` в классе `DoublyLinkedList`, который принимает значение и вставляет его **после первого узла** (если список не пуст; если пуст — вставляет как первый).
- (e) Создайте метод `delete` в классе `DoublyLinkedList`, который принимает значение и удаляет **все вхождения** этого значения. Обновляет ссылки и границы.
- (f) Создайте экземпляр класса `DoublyLinkedList`.
- (g) Вставьте узлы: 100, 200, 300.
- (h) Вызовите `display`.
- (i) Вставьте 150 после первого узла.
- (j) Снова вызовите `display`.
- (k) Удалите все вхождения 150.
- (l) Снова вызовите `display`.

Пример использования:

```
dll = DoublyLinkedList()
dll.insert(100)
dll.insert(200)
dll.insert(300)

print("Initial Doubly Linked List:")
dll.display()

dll.insert(150)
print("After inserting 150 after first:")
dll.display()

dll.delete(150)
print("After deleting all 150s:")
dll.display()
```

4. Написать программу на Python, которая создает класс `DoublyLinkedList`, представляющий **двусвязный список** с инкапсуляцией. Класс должен содержать методы для отображения данных, вставки и удаления узлов. Программа также должна создавать экземпляр класса, вставлять узлы и удалять узлы.

Инструкции:

- (a) Создайте класс `Node` с методом `__init__`, который принимает `entry` и сохраняет его в `self._item`. Инициализирует `self._succ` и `self._pred` как `None`.
- (b) Создайте класс `DoublyLinkedList` с методом `__init__`, который инициализирует `self._top` и `self._bottom` как `None`.
- (c) Создайте метод `display` в классе `DoublyLinkedList`, который выводит элементы в обратном порядке (от хвоста к голове), разделенные " ". Если пуст — "Обратный просмотр: пусто".
- (d) Создайте метод `insert` в классе `DoublyLinkedList`, который принимает значение и вставляет его **перед последним узлом** (если узлов > 1 ; если 0 или 1 — вставляет в конец).

- (e) Создайте метод `delete` в классе `DoublyLinkedList`, который принимает значение и удаляет первый найденный узел. Если узел — единственный, обнуляет `self._top` и `self._bottom`.
- (f) Создайте экземпляр класса `DoublyLinkedList`.
- (g) Вставьте узлы: 7, 14, 21.
- (h) Вызовите `display`.
- (i) Вставьте 18 перед последним узлом.
- (j) Снова вызовите `display`.
- (k) Удалите узел со значением 14.
- (l) Снова вызовите `display`.

Пример использования:

```
dll = DoublyLinkedList()
dll.insert(7)
dll.insert(14)
dll.insert(21)

print("Initial Doubly Linked List (reversed):")
dll.display()

dll.insert(18)
print("After inserting 18 before last:")
dll.display()

dll.delete(14)
print("After deleting 14:")
dll.display()
```

5. Написать программу на Python, которая создает класс `DoublyLinkedList`, представляющий **двусвязный список** с инкапсуляцией. Класс должен содержать методы для отображения данных, вставки и удаления узлов. Программа также должна создавать экземпляр класса, вставлять узлы и удалять узлы.

Инструкции:

- (a) Создайте класс `Node` с методом `__init__`, который принимает `value` и сохраняет его в `self._key`. Инициализирует `self._link_next` и `self._link_prev` как `None`.
- (b) Создайте класс `DoublyLinkedList` с методом `__init__`, который инициализирует `self._header` и `self._trailer` как `None`.
- (c) Создайте метод `display` в классе `DoublyLinkedList`, который выводит элементы в квадратных скобках через запятую: `[a, b, c]`. Если пуст — `[]`.
- (d) Создайте метод `insert` в классе `DoublyLinkedList`, который принимает значение и вставляет его **только если такого значения еще нет в списке**. Вставляет в конец.
- (e) Создайте метод `delete` в классе `DoublyLinkedList`, который принимает значение и удаляет узел, если он существует. Если не существует — ничего не делает.
- (f) Создайте экземпляр класса `DoublyLinkedList`.
- (g) Вставьте узлы: 3, 6, 9, 6 (второй 6 не вставится).
- (h) Вызовите `display`.

- (i) Вставьте 12.
- (j) Снова вызовите display.
- (k) Удалите 6.
- (l) Снова вызовите display.

Пример использования:

```
dll = DoublyLinkedList()
dll.insert(3)
dll.insert(6)
dll.insert(9)
dll.insert(6)  # игнорируется

print("Initial Doubly Linked List:")
dll.display()

dll.insert(12)
print("After inserting 12:")
dll.display()

dll.delete(6)
print("After deleting 6:")
dll.display()
```

6. Написать программу на Python, которая создает класс DoublyLinkedList, представляющий **двусвязный список** с инкапсуляцией. Класс должен содержать методы для отображения данных, вставки и удаления узлов. Программа также должна создавать экземпляр класса, вставлять узлы и удалять узлы.

Инструкции:

- (a) Создайте класс Node с методом `__init__`, который принимает `data_point` и сохраняет его в `self._datum`. Инициализирует `self._next_node` и `self._prev_node` как `None`.
- (b) Создайте класс DoublyLinkedList с методом `__init__`, который инициализирует `self._start` и `self._finish` как `None`.
- (c) Создайте метод `display` в классе DoublyLinkedList, который выводит элементы в формате "Элементы: val1 -> val2 -> val3 двигаясь от начала к концу. Если пуст — "Элементы: (нет)".
- (d) Создайте метод `insert` в классе DoublyLinkedList, который принимает значение и вставляет его **только если оно больше последнего элемента** (если список не пуст). Если пуст — вставляет. Иначе — игнорирует.
- (e) Создайте метод `delete` в классе DoublyLinkedList, который принимает значение и удаляет **первый узел**, если он равен значению. Не ищет дальше.
- (f) Создайте экземпляр класса DoublyLinkedList.
- (g) Вставьте узлы: 1, 5, 3 (игнорируется), 10.
- (h) Вызовите `display`.
- (i) Вставьте 7 (игнорируется, т.к. $7 < 10$).
- (j) Снова вызовите `display`.
- (k) Удалите 5.

(l) Снова вызовите display.

Пример использования:

```
dll = DoublyLinkedList()
dll.insert(1)
dll.insert(5)
dll.insert(3)  # игнорируется
dll.insert(10)

print("Initial Doubly Linked List:")
dll.display()

dll.insert(7)  # игнорируется
print("After attempting to insert 7:")
dll.display()

dll.delete(5)
print("After deleting 5:")
dll.display()
```

7. Написать программу на Python, которая создает класс DoublyLinkedList, представляющий **двусвязный список** с инкапсуляцией. Класс должен содержать методы для отображения данных, вставки и удаления узлов. Программа также должна создавать экземпляр класса, вставлять узлы и удалять узлы.

Инструкции:

- (a) Создайте класс Node с методом `__init__`, который принимает `item_value` и сохраняет его в `self._content`. Инициализирует `self._ptr_next` и `self._ptr_prev` как `None`.
- (b) Создайте класс DoublyLinkedList с методом `__init__`, который инициализирует `self._head_node` и `self._tail_node` как `None`.
- (c) Создайте метод `display` в классе DoublyLinkedList, который выводит элементы в виде строки, разделенной точками: "a.b.c". Если пуст — "пусто".
- (d) Создайте метод `insert` в классе DoublyLinkedList, который принимает значение и вставляет его **в середину списка** (если четное количество — после левой средней позиции; если нечетное — в центр). Если список пуст — вставляет как первый.
- (e) Создайте метод `delete` в классе DoublyLinkedList, который принимает значение и удаляет **все узлы с этим значением**.
- (f) Создайте экземпляр класса DoublyLinkedList.
- (g) Вставьте узлы: 10, 20, 30.
- (h) Вызовите `display`.
- (i) Вставьте 25 в середину (между 20 и 30).
- (j) Снова вызовите `display`.
- (k) Удалите все вхождения 25.
- (l) Снова вызовите `display`.

Пример использования:

```

dll = DoublyLinkedList()
dll.insert(10)
dll.insert(20)
dll.insert(30)

print("Initial Doubly Linked List:")
dll.display()

dll.insert(25)
print("After inserting 25 in middle:")
dll.display()

dll.delete(25)
print("After deleting 25:")
dll.display()

```

8. Написать программу на Python, которая создает класс DoublyLinkedList, представляющий **двусвязный список** с инкапсуляцией. Класс должен содержать методы для отображения данных, вставки и удаления узлов. Программа также должна создавать экземпляр класса, вставлять узлы и удалять узлы.

Инструкции:

- (a) Создайте класс Node с методом `__init__`, который принимает `node_data` и сохраняет его в `self._info`. Инициализирует `self._nxt` и `self._prv` как `None`.
- (b) Создайте класс DoublyLinkedList с методом `__init__`, который инициализирует `self._front` и `self._rear` как `None`.
- (c) Создайте метод `display` в классе DoublyLinkedList, который выводит элементы в формате "Front->Back: [значения]" и "Back->Front: [значения в обратном порядке]". Если пуст — "Список пуст в обоих направлениях".
- (d) Создайте метод `insert` в классе DoublyLinkedList, который принимает значение и вставляет его **в начало, только если значение четное**. Если нечетное — вставляет в конец.
- (e) Создайте метод `delete` в классе DoublyLinkedList, который принимает значение и удаляет **первое вхождение**.
- (f) Создайте экземпляр класса DoublyLinkedList.
- (g) Вставьте узлы: 4 (в начало), 7 (в конец), 6 (в начало), 9 (в конец).
- (h) Вызовите `display`.
- (i) Вставьте 8 (в начало).
- (j) Снова вызовите `display`.
- (k) Удалите 7.
- (l) Снова вызовите `display`.

Пример использования:

```

dll = DoublyLinkedList()
dll.insert(4)
dll.insert(7)
dll.insert(6)
dll.insert(9)

print("Initial Doubly Linked List:")

```

```

dll.display()

dll.insert(8)
print("After inserting 8:")
dll.display()

dll.delete(7)
print("After deleting 7:")
dll.display()

```

9. Написать программу на Python, которая создает класс DoublyLinkedList, представляющий **двусвязный список** с инкапсуляцией. Класс должен содержать методы для отображения данных, вставки и удаления узлов. Программа также должна создавать экземпляр класса, вставлять узлы и удалять узлы.

Инструкции:

- Создайте класс Node с методом `__init__`, который принимает value и сохраняет его в `self._element`. Инициализирует `self._next_elem` и `self._prev_elem` как None.
- Создайте класс DoublyLinkedList с методом `__init__`, который инициализирует `self._head_elem` и `self._tail_elem` как None.
- Создайте метод display в классе DoublyLinkedList, который выводит элементы в виде: "HEAD <-> val1 <-> val2 <-> ... <-> TAIL". Если пуст — "HEAD <-> TAIL (пусто)".
- Создайте метод insert в классе DoublyLinkedList, который принимает значение и вставляет его **после узла с наименьшим значением** (если несколько — после первого). Если список пуст — вставляет как единственный.
- Создайте метод delete в классе DoublyLinkedList, который принимает значение и удаляет **последнее вхождение**.
- Создайте экземпляр класса DoublyLinkedList.
- Вставьте узлы: 50, 30, 40.
- Вызовите display.
- Вставьте 35 (после 30 — минимального).
- Снова вызовите display.
- Удалите последнее вхождение 40.
- Снова вызовите display.

Пример использования:

```

dll = DoublyLinkedList()
dll.insert(50)
dll.insert(30)
dll.insert(40)

print("Initial Doubly Linked List:")
dll.display()

dll.insert(35)
print("After inserting 35 after min:")
dll.display()

dll.delete(40)
print("After deleting last occurrence of 40:")
dll.display()

```


10. Написать программу на Python, которая создает класс DoublyLinkedList, представляющий **двусвязный список** с инкапсуляцией. Класс должен содержать методы для отображения данных, вставки и удаления узлов. Программа также должна создавать экземпляр класса, вставлять узлы и удалять узлы.

Инструкции:

- (a) Создайте класс Node с методом `__init__`, который принимает data и сохраняет его в `self._val`. Инициализирует `self._link_f` и `self._link_b` как None.
- (b) Создайте класс DoublyLinkedList с методом `__init__`, который инициализирует `self._first_item` и `self._last_item` как None.
- (c) Создайте метод `display` в классе DoublyLinkedList, который выводит элементы в виде: "Элементы (прямой порядок): ... а затем "Элементы (обратный порядок): ...". Если пуст — "Нет данных".
- (d) Создайте метод `insert` в классе DoublyLinkedList, который принимает значение и вставляет его **перед узлом с наибольшим значением** (если несколько — перед первым). Если список пуст — вставляет как единственный.
- (e) Создайте метод `delete` в классе DoublyLinkedList, который принимает значение и удаляет **все вхождения**.
- (f) Создайте экземпляр класса DoublyLinkedList.
- (g) Вставьте узлы: 5, 15, 10.
- (h) Вызовите `display`.
- (i) Вставьте 12 (перед 15 — максимальным).
- (j) Снова вызовите `display`.
- (k) Удалите все вхождения 10.
- (l) Снова вызовите `display`.

Пример использования:

```
dll = DoublyLinkedList()
dll.insert(5)
dll.insert(15)
dll.insert(10)

print("Initial Doubly Linked List:")
dll.display()

dll.insert(12)
print("After inserting 12 before max:")
dll.display()

dll.delete(10)
print("After deleting all 10s:")
dll.display()
```

11. Написать программу на Python, которая создает класс DoublyLinkedList, представляющий **двусвязный список** с инкапсуляцией. Класс должен содержать методы для отображения данных, вставки и удаления узлов. Программа также должна создавать экземпляр класса, вставлять узлы и удалять узлы.

Инструкции:

- (a) Создайте класс Node с методом `__init__`, который принимает item и сохраняет его в `self._data_field`. Инициализирует `self._next_ref` и `self._prev_ref` как None.
- (b) Создайте класс DoublyLinkedList с методом `__init__`, который инициализирует `self._entry_point` и `self._exit_point` как None.
- (c) Создайте метод display в классе DoublyLinkedList, который выводит элементы в одну строку, разделенные `->` и в конце добавляет `-> None`. Если пуст — "None".
- (d) Создайте метод insert в классе DoublyLinkedList, который принимает значение и вставляет его **в позицию, равную значению по модулю длины списка** (если список не пуст; если пуст — вставляет как первый). Например, при длине 3 и значении 7: $7 \% 3 = 1 \rightarrow$ вставка на позицию 1 (второй элемент).
- (e) Создайте метод delete в классе DoublyLinkedList, который принимает значение и удаляет **первое вхождение**.
- (f) Создайте экземпляр класса DoublyLinkedList.
- (g) Вставьте узлы: 2, 4, 6.
- (h) Вызовите display.
- (i) Вставьте 5 ($5 \% 3 = 2 \rightarrow$ вставка на позицию 2, т.е. после 4, перед 6).
- (j) Снова вызовите display.
- (k) Удалите 4.
- (l) Снова вызовите display.

Пример использования:

```
dll = DoublyLinkedList()
dll.insert(2)
dll.insert(4)
dll.insert(6)

print("Initial Doubly Linked List:")
dll.display()

dll.insert(5)
print("After inserting 5 at position 5 % 3 = 2:")
dll.display()

dll.delete(4)
print("After deleting 4:")
dll.display()
```

12. Написать программу на Python, которая создает класс DoublyLinkedList, представляющий **двусвязный список** с инкапсуляцией. Класс должен содержать методы для отображения данных, вставки и удаления узлов. Программа также должна создавать экземпляр класса, вставлять узлы и удалять узлы.

Инструкции:

- (a) Создайте класс Node с методом `__init__`, который принимает content и сохраняет его в `self._stored_value`. Инициализирует `self._connection_next` и `self._connection_prev` как None.
- (b) Создайте класс DoublyLinkedList с методом `__init__`, который инициализирует `self._input` и `self._output` как None.

- (c) Создайте метод `display` в классе `DoublyLinkedList`, который выводит элементы в формате: "List: [значения через пробел] (размер: N)". Если пуст — "List: [] (размер: 0)".
- (d) Создайте метод `insert` в классе `DoublyLinkedList`, который принимает значение и вставляет его **только если оно не отрицательное**. Вставляет в конец.
- (e) Создайте метод `delete` в классе `DoublyLinkedList`, который принимает значение и удаляет **первое вхождение, только если значение положительное**. Если значение ≤ 0 — ничего не делает.
- (f) Создайте экземпляр класса `DoublyLinkedList`.
- (g) Вставьте узлы: -1 (игнорируется), 8, 0, 12, -5 (игнорируется).
- (h) Вызовите `display`.
- (i) Вставьте 10.
- (j) Снова вызовите `display`.
- (k) Удалите 0 (не удаляется, т.к. не положительное).
- (l) Снова вызовите `display`.

Пример использования:

```
dll = DoublyLinkedList()
dll.insert(-1)  # игнорируется
dll.insert(8)
dll.insert(0)
dll.insert(12)
dll.insert(-5)  # игнорируется

print("Initial Doubly Linked List:")
dll.display()

dll.insert(10)
print("After inserting 10:")
dll.display()

dll.delete(0)  # не удаляется
print("After attempting to delete 0:")
dll.display()
```

13. Написать программу на Python, которая создает класс `DoublyLinkedList`, представляющий **двусвязный список** с инкапсуляцией. Класс должен содержать методы для отображения данных, вставки и удаления узлов. Программа также должна создавать экземпляр класса, вставлять узлы и удалять узлы.

Инструкции:

- (a) Создайте класс `Node` с методом `__init__`, который принимает `data` и сохраняет его в `self._record`. Инициализирует `self._next_entry` и `self._prev_entry` как `None`.
- (b) Создайте класс `DoublyLinkedList` с методом `__init__`, который инициализирует `self._head_record` и `self._tail_record` как `None`.
- (c) Создайте метод `display` в классе `DoublyLinkedList`, который выводит элементы в виде: "Записи: val1, val2, ..., valN". Если пуст — "Записей нет".
- (d) Создайте метод `insert` в классе `DoublyLinkedList`, который принимает значение и вставляет его **в начало, если значение нечетное, и в конец, если четное**.

- (e) Создайте метод `delete` в классе `DoublyLinkedList`, который принимает значение и удаляет **все узлы с этим значением**.
- (f) Создайте экземпляр класса `DoublyLinkedList`.
- (g) Вставьте узлы: 3 (в начало), 4 (в конец), 5 (в начало), 6 (в конец).
- (h) Вызовите `display`.
- (i) Вставьте 7 (в начало).
- (j) Снова вызовите `display`.
- (k) Удалите все вхождения 4.
- (l) Снова вызовите `display`.

Пример использования:

```
dll = DoublyLinkedList()
dll.insert(3)
dll.insert(4)
dll.insert(5)
dll.insert(6)

print("Initial Doubly Linked List:")
dll.display()

dll.insert(7)
print("After inserting 7:")
dll.display()

dll.delete(4)
print("After deleting all 4s:")
dll.display()
```

14. Написать программу на Python, которая создает класс `DoublyLinkedList`, представляющий **двусвязный список** с инкапсуляцией. Класс должен содержать методы для отображения данных, вставки и удаления узлов. Программа также должна создавать экземпляр класса, вставлять узлы и удалять узлы.

Инструкции:

- (a) Создайте класс `Node` с методом `__init__`, который принимает `value` и сохраняет его в `self._cell`. Инициализирует `self._cell_next` и `self._cell_prev` как `None`.
- (b) Создайте класс `DoublyLinkedList` с методом `__init__`, который инициализирует `self._first_cell` и `self._last_cell` как `None`.
- (c) Создайте метод `display` в классе `DoublyLinkedList`, который выводит элементы в виде: "Ячейки: [значения]" и отдельно "Количество: N". Если пуст — "Список ячеек пуст".
- (d) Создайте метод `insert` в классе `DoublyLinkedList`, который принимает значение и вставляет его **после каждого узла, значение которого кратно 3** (если таких нет — вставляет в конец).
- (e) Создайте метод `delete` в классе `DoublyLinkedList`, который принимает значение и удаляет **первое вхождение**.
- (f) Создайте экземпляр класса `DoublyLinkedList`.
- (g) Вставьте узлы: 6, 9, 4.

- (h) Вызовите `display`.
- (i) Вставьте 12 (вставится после 6 и после 9 — но по условию вставляется только один узел; вставим после первого кратного 3, т.е. после 6).
- (j) Снова вызовите `display`.
- (k) Удалите 9.
- (l) Снова вызовите `display`.

Пример использования:

```
dll = DoublyLinkedList()
dll.insert(6)
dll.insert(9)
dll.insert(4)

print("Initial Doubly Linked List:")
dll.display()

dll.insert(12)
print("After inserting 12 after first multiple of 3:")
dll.display()

dll.delete(9)
print("After deleting 9:")
dll.display()
```

15. Написать программу на Python, которая создает класс `DoublyLinkedList`, представляющий **двусвязный список** с инкапсуляцией. Класс должен содержать методы для отображения данных, вставки и удаления узлов. Программа также должна создавать экземпляр класса, вставлять узлы и удалять узлы.

Инструкции:

- (a) Создайте класс `Node` с методом `__init__`, который принимает `item` и сохраняет его в `self._slot`. Инициализирует `self._slot_next` и `self._slot_prev` как `None`.
- (b) Создайте класс `DoublyLinkedList` с методом `__init__`, который инициализирует `self._start_slot` и `self._end_slot` как `None`.
- (c) Создайте метод `display` в классе `DoublyLinkedList`, который выводит элементы в виде: "Слоты: val1 | val2 | val3". Если пуст — "Слоты отсутствуют".
- (d) Создайте метод `insert` в классе `DoublyLinkedList`, который принимает значение и вставляет его **перед каждым узлом, значение которого кратно 5** (если таких нет — вставляет в начало).
- (e) Создайте метод `delete` в классе `DoublyLinkedList`, который принимает значение и удаляет **последнее вхождение**.
- (f) Создайте экземпляр класса `DoublyLinkedList`.
- (g) Вставьте узлы: 10, 15, 7.
- (h) Вызовите `display`.
- (i) Вставьте 5 (вставится перед 10 и перед 15 — но по условию вставляется только один узел; вставим перед первым кратным 5, т.е. перед 10).
- (j) Снова вызовите `display`.
- (k) Удалите последнее вхождение 15.

(l) Снова вызовите display.

Пример использования:

```
dll = DoublyLinkedList()
dll.insert(10)
dll.insert(15)
dll.insert(7)

print("Initial Doubly Linked List:")
dll.display()

dll.insert(5)
print("After inserting 5 before first multiple of 5:")
dll.display()

dll.delete(15)
print("After deleting last occurrence of 15:")
dll.display()
```

16. Написать программу на Python, которая создает класс DoublyLinkedList, представляющий **двусвязный список** с инкапсуляцией. Класс должен содержать методы для отображения данных, вставки и удаления узлов. Программа также должна создавать экземпляр класса, вставлять узлы и удалять узлы.

Инструкции:

- (a) Создайте класс Node с методом `__init__`, который принимает data и сохраняет его в `self._block`. Инициализирует `self._block_next` и `self._block_prev` как None.
- (b) Создайте класс DoublyLinkedList с методом `__init__`, который инициализирует `self._head_block` и `self._tail_block` как None.
- (c) Создайте метод display в классе DoublyLinkedList, который выводит элементы в виде: "Блоки: [значения]" и "Обратный порядок: [значения в обратном порядке]". Если пуст — "Нет блоков".
- (d) Создайте метод insert в классе DoublyLinkedList, который принимает значение и вставляет его **только если сумма цифр значения четная**. Вставляет в конец.
- (e) Создайте метод delete в классе DoublyLinkedList, который принимает значение и удаляет **все вхождения**.
- (f) Создайте экземпляр класса DoublyLinkedList.
- (g) Вставьте узлы: 23 ($2+3=5$ — нечет, не вставляется), 24 ($2+4=6$ — чет, вставляется), 35 ($3+5=8$ — чет, вставляется), 13 ($1+3=4$ — чет, вставляется).
- (h) Вызовите display.
- (i) Вставьте 46 ($4+6=10$ — чет, вставляется).
- (j) Снова вызовите display.
- (k) Удалите все вхождения 24.
- (l) Снова вызовите display.

Пример использования:

```
dll = DoublyLinkedList()
dll.insert(23)  # не вставляется
dll.insert(24)
```

```

dll.insert(35)
dll.insert(13)

print("Initial Doubly Linked List:")
dll.display()

dll.insert(46)
print("After inserting 46:")
dll.display()

dll.delete(24)
print("After deleting all 24s:")
dll.display()

```

17. Написать программу на Python, которая создает класс DoublyLinkedList, представляющий **двусвязный список** с инкапсуляцией. Класс должен содержать методы для отображения данных, вставки и удаления узлов. Программа также должна создавать экземпляр класса, вставлять узлы и удалять узлы.

Инструкции:

- (a) Создайте класс Node с методом `__init__`, который принимает value и сохраняет его в `self._unit`. Инициализирует `self._unit_next` и `self._unit_prev` как None.
- (b) Создайте класс DoublyLinkedList с методом `__init__`, который инициализирует `self._first_unit` и `self._last_unit` как None.
- (c) Создайте метод display в классе DoublyLinkedList, который выводит элементы в виде: "Единицы: val1 → val2 → val3 → null". Если пуст — "null".
- (d) Создайте метод insert в классе DoublyLinkedList, который принимает значение и вставляет его **только если оно простое число** (используйте вспомогательную функцию `is_prime`). Вставляет в начало.
- (e) Создайте метод delete в классе DoublyLinkedList, который принимает значение и удаляет **первое вхождение**.
- (f) Создайте вспомогательную функцию `is_prime(n)`.
- (g) Создайте экземпляр класса DoublyLinkedList.
- (h) Вставьте узлы: 4 (не простое), 5 (простое), 6 (не простое), 7 (простое), 8 (не простое), 11 (простое).
- (i) Вызовите display.
- (j) Вставьте 13 (простое).
- (k) Снова вызовите display.
- (l) Удалите 7.
- (m) Снова вызовите display.

Пример использования:

```

def is_prime(n):
    if n < 2:
        return False
    for i in range(2, int(n**0.5)+1):
        if n % i == 0:
            return False
    return True

```

```

dll = DoublyLinkedList()
dll.insert(4)      # не п
dll.insert(5)      # да
dll.insert(6)      # не п
dll.insert(7)      # да
dll.insert(8)      # не п
dll.insert(11)     # да

print("Initial Doubly Linked List:")
dll.display()

dll.insert(13)
print("After inserting 13:")
dll.display()

dll.delete(7)
print("After deleting 7:")
dll.display()

```

18. Написать программу на Python, которая создает класс DoublyLinkedList, представляющий **двусвязный список** с инкапсуляцией. Класс должен содержать методы для отображения данных, вставки и удаления узлов. Программа также должна создавать экземпляр класса, вставлять узлы и удалять узлы.

Инструкции:

- Создайте класс Node с методом `__init__`, который принимает item и сохраняет его в `self._segment`. Инициализирует `self._seg_next` и `self._seg_prev` как None.
- Создайте класс DoublyLinkedList с методом `__init__`, который инициализирует `self._head_seg` и `self._tail_seg` как None.
- Создайте метод `display` в классе DoublyLinkedList, который выводит элементы в виде: "Сегменты (вперед): ... "Сегменты (назад): ...". Если пуст — "Список сегментов пуст".
- Создайте метод `insert` в классе DoublyLinkedList, который принимает значение и вставляет его **только если оно палиндром** (например, 121, 33). Вставляет в конец.
- Создайте метод `delete` в классе DoublyLinkedList, который принимает значение и удаляет **последнее вхождение**.
- Создайте экземпляр класса DoublyLinkedList.
- Вставьте узлы: 12 (не палиндром), 22 (палиндром), 34 (не палиндром), 55 (палиндром), 121 (палиндром).
- Вызовите `display`.
- Вставьте 33 (палиндром).
- Снова вызовите `display`.
- Удалите последнее вхождение 55.
- Снова вызовите `display`.

Пример использования:


```

dll = DoublyLinkedList()
dll.insert(12) # нет
dll.insert(22) # да
dll.insert(34) # нет
dll.insert(55) # да
dll.insert(121) # да

print("Initial Doubly Linked List:")
dll.display()

dll.insert(33)
print("After inserting 33:")
dll.display()

dll.delete(55)
print("After deleting last occurrence of 55:")
dll.display()

```

19. Написать программу на Python, которая создает класс DoublyLinkedList, представляющий **двусвязный список** с инкапсуляцией. Класс должен содержать методы для отображения данных, вставки и удаления узлов. Программа также должна создавать экземпляр класса, вставлять узлы и удалять узлы.

Инструкции:

- Создайте класс Node с методом `__init__`, который принимает data и сохраняет его в `self._piece`. Инициализирует `self._piece_next` и `self._piece_prev` как None.
- Создайте класс DoublyLinkedList с методом `__init__`, который инициализирует `self._first_piece` и `self._last_piece` как None.
- Создайте метод `display` в классе DoublyLinkedList, который выводит элементы в виде: "Части: val1 - val2 - val3". Если пуст — "Нет частей".
- Создайте метод `insert` в классе DoublyLinkedList, который принимает значение и вставляет его **только если оно степень двойки** (1,2,4,8,16...). Вставляет в начало.
- Создайте метод `delete` в классе DoublyLinkedList, который принимает значение и удаляет **все вхождения**.
- Создайте экземпляр класса DoublyLinkedList.
- Вставьте узлы: 3 (нет), 4 (да), 5 (нет), 8 (да), 9 (нет), 16 (да).
- Вызовите `display`.
- Вставьте 32 (да).
- Снова вызовите `display`.
- Удалите все вхождения 8.
- Снова вызовите `display`.

Пример использования:

```

dll = DoublyLinkedList()
dll.insert(3) # нет
dll.insert(4) # да
dll.insert(5) # нет
dll.insert(8) # да
dll.insert(9) # нет

```

```

dll.insert(16)    # да

print("Initial Doubly Linked List:")
dll.display()

dll.insert(32)
print("After inserting 32:")
dll.display()

dll.delete(8)
print("After deleting all 8s:")
dll.display()

```

20. Написать программу на Python, которая создает класс DoublyLinkedList, представляющий **двусвязный список** с инкапсуляцией. Класс должен содержать методы для отображения данных, вставки и удаления узлов. Программа также должна создавать экземпляр класса, вставлять узлы и удалять узлы.

Инструкции:

- Создайте класс Node с методом `__init__`, который принимает value и сохраняет его в `self._fragment`. Инициализирует `self._frag_next` и `self._frag_prev` как None.
- Создайте класс DoublyLinkedList с методом `__init__`, который инициализирует `self._start_frag` и `self._end_frag` как None.
- Создайте метод `display` в классе DoublyLinkedList, который выводит элементы в виде: "Фрагменты → val1 → val2 → val3 → конец". Если пуст — "Фрагменты: конец".
- Создайте метод `insert` в классе DoublyLinkedList, который принимает значение и вставляет его **только если оно делится на 3 без остатка**. Вставляет в конец.
- Создайте метод `delete` в классе DoublyLinkedList, который принимает значение и удаляет **первое вхождение**.
- Создайте экземпляр класса DoublyLinkedList.
- Вставьте узлы: 1 (нет), 3 (да), 4 (нет), 6 (да), 7 (нет), 9 (да).
- Вызовите `display`.
- Вставьте 12 (да).
- Снова вызовите `display`.
- Удалите 6.
- Снова вызовите `display`.

Пример использования:

```

dll = DoublyLinkedList()
dll.insert(1)    # нет
dll.insert(3)    # да
dll.insert(4)    # нет
dll.insert(6)    # да
dll.insert(7)    # нет
dll.insert(9)    # да

print("Initial Doubly Linked List:")
dll.display()

```

```

dll.insert(12)
print("After inserting 12:")
dll.display()

dll.delete(6)
print("After deleting 6:")
dll.display()

```

21. Написать программу на Python, которая создает класс DoublyLinkedList, представляющий **двусвязный список** с инкапсуляцией. Класс должен содержать методы для отображения данных, вставки и удаления узлов. Программа также должна создавать экземпляр класса, вставлять узлы и удалять узлы.

Инструкции:

- (a) Создайте класс Node с методом `__init__`, который принимает item и сохраняет его в `self._chunk`. Инициализирует `self._chunk_next` и `self._chunk_prev` как None.
- (b) Создайте класс DoublyLinkedList с методом `__init__`, который инициализирует `self._head_chunk` и `self._tail_chunk` как None.
- (c) Создайте метод `display` в классе DoublyLinkedList, который выводит элементы в виде: "Чанки: [значения]" и "Размер: N". Если пуст — "Чанков нет".
- (d) Создайте метод `insert` в классе DoublyLinkedList, который принимает значение и вставляет его **только если оно не делится на 5**. Вставляет в начало.
- (e) Создайте метод `delete` в классе DoublyLinkedList, который принимает значение и удаляет **последнее вхождение**.
- (f) Создайте экземпляр класса DoublyLinkedList.
- (g) Вставьте узлы: 10 (делится на 5 — не вставляется), 11 (не делится — вставляется), 15 (делится — не вставляется), 16 (не делится — вставляется), 20 (делится — не вставляется), 21 (не делится — вставляется).
- (h) Вызовите `display`.
- (i) Вставьте 26 (не делится — вставляется).
- (j) Снова вызовите `display`.
- (k) Удалите последнее вхождение 16.
- (l) Снова вызовите `display`.

Пример использования:

```

dll = DoublyLinkedList()
dll.insert(10) # нет
dll.insert(11) # да
dll.insert(15) # нет
dll.insert(16) # да
dll.insert(20) # нет
dll.insert(21) # да

print("Initial Doubly Linked List:")
dll.display()

dll.insert(26)
print("After inserting 26:")
dll.display()

```

```

dll.delete(16)
print("After deleting last occurrence of 16:")
dll.display()

```

22. Написать программу на Python, которая создает класс DoublyLinkedList, представляющий **двусвязный список** с инкапсуляцией. Класс должен содержать методы для отображения данных, вставки и удаления узлов. Программа также должна создавать экземпляр класса, вставлять узлы и удалять узлы.

Инструкции:

- (a) Создайте класс Node с методом `__init__`, который принимает data и сохраняет его в `self._item_data`. Инициализирует `self._next_item` и `self._prev_item` как None.
- (b) Создайте класс DoublyLinkedList с методом `__init__`, который инициализирует `self._first_data` и `self._last_data` как None.
- (c) Создайте метод `display` в классе DoublyLinkedList, который выводит элементы в виде: "Данные ($\rightarrow$): val1, val2, val3" и "Данные ($\leftarrow$): val3, val2, val1". Если пуст — "Данные отсутствуют".
- (d) Создайте метод `insert` в классе DoublyLinkedList, который принимает значение и вставляет его **только если оно больше 10**. Вставляет в конец.
- (e) Создайте метод `delete` в классе DoublyLinkedList, который принимает значение и удаляет **все вхождения**.
- (f) Создайте экземпляр класса DoublyLinkedList.
- (g) Вставьте узлы: 5 (нет), 15 (да), 8 (нет), 20 (да), 12 (да).
- (h) Вызовите `display`.
- (i) Вставьте 25 (да).
- (j) Снова вызовите `display`.
- (k) Удалите все вхождения 20.
- (l) Снова вызовите `display`.

Пример использования:

```

dll = DoublyLinkedList()
dll.insert(5)      # нет
dll.insert(15)     # да
dll.insert(8)      # нет
dll.insert(20)     # да
dll.insert(12)     # да

print("Initial Doubly Linked List:")
dll.display()

dll.insert(25)
print("After inserting 25:")
dll.display()

dll.delete(20)
print("After deleting all 20s:")
dll.display()

```

23. Написать программу на Python, которая создает класс DoublyLinkedList, представляющий **двусвязный список** с инкапсуляцией. Класс должен содержать методы для отображения данных, вставки и удаления узлов. Программа также должна создавать экземпляр класса, вставлять узлы и удалять узлы.

Инструкции:

- (a) Создайте класс Node с методом `__init__`, который принимает value и сохраняет его в `self._node_value`. Инициализирует `self._node_next` и `self._node_prev` как None.
- (b) Создайте класс DoublyLinkedList с методом `__init__`, который инициализирует `self._start_node` и `self._end_node` как None.
- (c) Создайте метод display в классе DoublyLinkedList, который выводит элементы в виде: "Узлы: val1 <-> val2 <-> val3". Если пуст — "Нет узлов".
- (d) Создайте метод insert в классе DoublyLinkedList, который принимает значение и вставляет его **только если оно меньше 50**. Вставляет в начало.
- (e) Создайте метод delete в классе DoublyLinkedList, который принимает значение и удаляет **первое вхождение**.
- (f) Создайте экземпляр класса DoublyLinkedList.
- (g) Вставьте узлы: 60 (нет), 30 (да), 70 (нет), 40 (да), 45 (да).
- (h) Вызовите display.
- (i) Вставьте 25 (да).
- (j) Снова вызовите display.
- (k) Удалите 40.
- (l) Снова вызовите display.

Пример использования:

```
dll = DoublyLinkedList()
dll.insert(60) # нет
dll.insert(30) # да
dll.insert(70) # нет
dll.insert(40) # да
dll.insert(45) # да

print("Initial Doubly Linked List:")
dll.display()

dll.insert(25)
print("After inserting 25:")
dll.display()

dll.delete(40)
print("After deleting 40:")
dll.display()
```

24. Написать программу на Python, которая создает класс DoublyLinkedList, представляющий **двусвязный список** с инкапсуляцией. Класс должен содержать методы для отображения данных, вставки и удаления узлов. Программа также должна создавать экземпляр класса, вставлять узлы и удалять узлы.

Инструкции:

- (a) Создайте класс Node с методом `__init__`, который принимает item и сохраняет его в `self._data_item`. Инициализирует `self._item_next` и `self._item_prev` как None.
- (b) Создайте класс DoublyLinkedList с методом `__init__`, который инициализирует `self._head_item` и `self._tail_item` как None.
- (c) Создайте метод `display` в классе DoublyLinkedList, который выводит элементы в виде: "Элементы списка: val1 val2 val3 (всего N)". Если пуст — "Список пуст".
- (d) Создайте метод `insert` в классе DoublyLinkedList, который принимает значение и вставляет его **только если оно не равно 0**. Вставляет в конец.
- (e) Создайте метод `delete` в классе DoublyLinkedList, который принимает значение и удаляет **последнее вхождение**.
- (f) Создайте экземпляр класса DoublyLinkedList.
- (g) Вставьте узлы: 0 (нет), 10 (да), 0 (нет), 20 (да), 30 (да).
- (h) Вызовите `display`.
- (i) Вставьте 40 (да).
- (j) Снова вызовите `display`.
- (k) Удалите последнее вхождение 20.
- (l) Снова вызовите `display`.

Пример использования:

```
dll = DoublyLinkedList()
dll.insert(0)    # нет
dll.insert(10)   # да
dll.insert(0)    # нет
dll.insert(20)   # да
dll.insert(30)   # да

print("Initial Doubly Linked List:")
dll.display()

dll.insert(40)
print("After inserting 40:")
dll.display()

dll.delete(20)
print("After deleting last occurrence of 20:")
dll.display()
```

25. Написать программу на Python, которая создает класс DoublyLinkedList, представляющий **двусвязный список** с инкапсуляцией. Класс должен содержать методы для отображения данных, вставки и удаления узлов. Программа также должна создавать экземпляр класса, вставлять узлы и удалять узлы.

Инструкции:

- (a) Создайте класс Node с методом `__init__`, который принимает data и сохраняет его в `self._list_data`. Инициализирует `self._data_next` и `self._data_prev` как None.
- (b) Создайте класс DoublyLinkedList с методом `__init__`, который инициализирует `self._first_list` и `self._last_list` как None.

- (c) Создайте метод `display` в классе `DoublyLinkedList`, который выводит элементы в виде: "Список: val1 | val2 | val3 | ...". Если пуст — "Пустой список".
- (d) Создайте метод `insert` в классе `DoublyLinkedList`, который принимает значение и вставляет его **только если оно положительное**. Вставляет в начало.
- (e) Создайте метод `delete` в классе `DoublyLinkedList`, который принимает значение и удаляет **все вхождения**.
- (f) Создайте экземпляр класса `DoublyLinkedList`.
- (g) Вставьте узлы: -5 (нет), 15 (да), -3 (нет), 25 (да), 0 (нет, если считать 0 не положительным).
- (h) Вызовите `display`.
- (i) Вставьте 35 (да).
- (j) Снова вызовите `display`.
- (k) Удалите все вхождения 25.
- (l) Снова вызовите `display`.

Пример использования:

```
dll = DoublyLinkedList()
dll.insert(-5)    # нет
dll.insert(15)   # да
dll.insert(-3)   # нет
dll.insert(25)   # да
dll.insert(0)    # нет

print("Initial Doubly Linked List:")
dll.display()

dll.insert(35)
print("After inserting 35:")
dll.display()

dll.delete(25)
print("After deleting all 25s:")
dll.display()
```

26. Написать программу на Python, которая создает класс `DoublyLinkedList`, представляющий **двусвязный список** с инкапсуляцией. Класс должен содержать методы для отображения данных, вставки и удаления узлов. Программа также должна создавать экземпляр класса, вставлять узлы и удалять узлы.

Инструкции:

- (a) Создайте класс `Node` с методом `__init__`, который принимает `value` и сохраняет его в `self._entry_value`. Инициализирует `self._value_next` и `self._value_prev` как `None`.
- (b) Создайте класс `DoublyLinkedList` с методом `__init__`, который инициализирует `self._head_value` и `self._tail_value` как `None`.
- (c) Создайте метод `display` в классе `DoublyLinkedList`, который выводит элементы в виде: "Значения → val1 → val2 → val3 → конец". Если пуст — "→ конец".
- (d) Создайте метод `insert` в классе `DoublyLinkedList`, который принимает значение и вставляет его **только если оно нечетное**. Вставляет в конец.

- (e) Создайте метод delete в классе DoublyLinkedList, который принимает значение и удаляет **первое вхождение**.
- (f) Создайте экземпляр класса DoublyLinkedList.
- (g) Вставьте узлы: 2 (нет), 3 (да), 4 (нет), 5 (да), 6 (нет), 7 (да).
- (h) Вызовите display.
- (i) Вставьте 9 (да).
- (j) Снова вызовите display.
- (k) Удалите 5.
- (l) Снова вызовите display.

Пример использования:

```
dll = DoublyLinkedList()
dll.insert(2) # нет
dll.insert(3) # да
dll.insert(4) # нет
dll.insert(5) # да
dll.insert(6) # нет
dll.insert(7) # да

print("Initial Doubly Linked List:")
dll.display()

dll.insert(9)
print("After inserting 9:")
dll.display()

dll.delete(5)
print("After deleting 5:")
dll.display()
```

27. Написать программу на Python, которая создает класс DoublyLinkedList, представляющий **двусвязный список** с инкапсуляцией. Класс должен содержать методы для отображения данных, вставки и удаления узлов. Программа также должна создавать экземпляр класса, вставлять узлы и удалять узлы.

Инструкции:

- (a) Создайте класс Node с методом `__init__`, который принимает item и сохраняет его в `self._data_point`. Инициализирует `self._point_next` и `self._point_prev` как None.
- (b) Создайте класс DoublyLinkedList с методом `__init__`, который инициализирует `self._start_point` и `self._end_point` как None.
- (c) Создайте метод display в классе DoublyLinkedList, который выводит элементы в виде: "Точки: val1, val2, val3 (обратно: val3, val2, val1)". Если пуст — "Точек нет".
- (d) Создайте метод insert в классе DoublyLinkedList, который принимает значение и вставляет его **только если оно четное**. Вставляет в начало.
- (e) Создайте метод delete в классе DoublyLinkedList, который принимает значение и удаляет **последнее вхождение**.
- (f) Создайте экземпляр класса DoublyLinkedList.
- (g) Вставьте узлы: 1 (нет), 4 (да), 3 (нет), 6 (да), 5 (нет), 8 (да).

- (h) Вызовите display.
- (i) Вставьте 10 (да).
- (j) Снова вызовите display.
- (k) Удалите последнее вхождение 6.
- (l) Снова вызовите display.

Пример использования:

```
dll = DoublyLinkedList()
dll.insert(1)    # нет
dll.insert(4)    # да
dll.insert(3)    # нет
dll.insert(6)    # да
dll.insert(5)    # нет
dll.insert(8)    # да

print("Initial Doubly Linked List:")
dll.display()

dll.insert(10)
print("After inserting 10:")
dll.display()

dll.delete(6)
print("After deleting last occurrence of 6:")
dll.display()
```

28. Написать программу на Python, которая создает класс DoublyLinkedList, представляющий **двусвязный список** с инкапсуляцией. Класс должен содержать методы для отображения данных, вставки и удаления узлов. Программа также должна создавать экземпляр класса, вставлять узлы и удалять узлы.

Инструкции:

- (a) Создайте класс Node с методом `__init__`, который принимает data и сохраняет его в self._node_data. Инициализирует self._data_link_next и self._data_link_prev как None.
- (b) Создайте класс DoublyLinkedList с методом `__init__`, который инициализирует self._first_link и self._last_link как None.
- (c) Создайте метод display в классе DoublyLinkedList, который выводит элементы в виде: "Связи: val1 <-> val2 <-> val3". Если пуст — "Связи отсутствуют".
- (d) Создайте метод insert в классе DoublyLinkedList, который принимает значение и вставляет его **только если оно кратно 4**. Вставляет в конец.
- (e) Создайте метод delete в классе DoublyLinkedList, который принимает значение и удаляет **все вхождения**.
- (f) Создайте экземпляр класса DoublyLinkedList.
- (g) Вставьте узлы: 2 (нет), 4 (да), 6 (нет), 8 (да), 10 (нет), 12 (да).
- (h) Вызовите display.
- (i) Вставьте 16 (да).
- (j) Снова вызовите display.

- (k) Удалите все вхождения 8.
- (l) Снова вызовите display.

Пример использования:

```
dll = DoublyLinkedList()
dll.insert(2)    # нет
dll.insert(4)    # да
dll.insert(6)    # нет
dll.insert(8)    # да
dll.insert(10)   # нет
dll.insert(12)   # да

print("Initial Doubly Linked List:")
dll.display()

dll.insert(16)
print("After inserting 16:")
dll.display()

dll.delete(8)
print("After deleting all 8s:")
dll.display()
```

29. Написать программу на Python, которая создает класс DoublyLinkedList, представляющий **двусвязный список** с инкапсуляцией. Класс должен содержать методы для отображения данных, вставки и удаления узлов. Программа также должна создавать экземпляр класса, вставлять узлы и удалять узлы.

Инструкции:

- (a) Создайте класс Node с методом `__init__`, который принимает value и сохраняет его в `self._item_val`. Инициализирует `self._val_next` и `self._val_prev` как None.
- (b) Создайте класс DoublyLinkedList с методом `__init__`, который инициализирует `self._head_val` и `self._tail_val` как None.
- (c) Создайте метод display в классе DoublyLinkedList, который выводит элементы в виде: "Значения: val1 - val2 - val3 (размер N)". Если пуст — "Нет значений".
- (d) Создайте метод insert в классе DoublyLinkedList, который принимает значение и вставляет его **только если оно заканчивается на 5**. Вставляет в начало.
- (e) Создайте метод delete в классе DoublyLinkedList, который принимает значение и удаляет **первое вхождение**.
- (f) Создайте экземпляр класса DoublyLinkedList.
- (g) Вставьте узлы: 10 (нет), 15 (да), 20 (нет), 25 (да), 30 (нет), 35 (да).
- (h) Вызовите display.
- (i) Вставьте 45 (да).
- (j) Снова вызовите display.
- (k) Удалите 25.
- (l) Снова вызовите display.

Пример использования:

```

dll = DoublyLinkedList()
dll.insert(10) # нет
dll.insert(15) # да
dll.insert(20) # нет
dll.insert(25) # да
dll.insert(30) # нет
dll.insert(35) # да

print("Initial Doubly Linked List:")
dll.display()

dll.insert(45)
print("After inserting 45:")
dll.display()

dll.delete(25)
print("After deleting 25:")
dll.display()

```

30. Написать программу на Python, которая создает класс DoublyLinkedList, представляющий **двусвязный список** с инкапсуляцией. Класс должен содержать методы для отображения данных, вставки и удаления узлов. Программа также должна создавать экземпляр класса, вставлять узлы и удалять узлы.

Инструкции:

- Создайте класс Node с методом `__init__`, который принимает item и сохраняет его в `self._data_field`. Инициализирует `self._field_next` и `self._field_prev` как None.
- Создайте класс DoublyLinkedList с методом `__init__`, который инициализирует `self._first_field` и `self._last_field` как None.
- Создайте метод `display` в классе DoublyLinkedList, который выводит элементы в виде: "Поля: val1 → val2 → val3 → null". Если пуст — "null".
- Создайте метод `insert` в классе DoublyLinkedList, который принимает значение и вставляет его **только если первая цифра числа — 1**. Вставляет в конец.
- Создайте метод `delete` в классе DoublyLinkedList, который принимает значение и удаляет **последнее вхождение**.
- Создайте экземпляр класса DoublyLinkedList.
- Вставьте узлы: 5 (нет), 12 (да), 23 (нет), 18 (да), 31 (нет), 19 (да).
- Вызовите `display`.
- Вставьте 11 (да).
- Снова вызовите `display`.
- Удалите последнее вхождение 18.
- Снова вызовите `display`.

Пример использования:

```

dll = DoublyLinkedList()
dll.insert(5) # нет
dll.insert(12) # да
dll.insert(23) # нет
dll.insert(18) # да

```

```

dll.insert(31)    # нет
dll.insert(19)    # да

print("Initial Doubly Linked List:")
dll.display()

dll.insert(11)
print("After inserting 11:")
dll.display()

dll.delete(18)
print("After deleting last occurrence of 18:")
dll.display()

```

31. Написать программу на Python, которая создает класс DoublyLinkedList, представляющий **двусвязный список** с инкапсуляцией. Класс должен содержать методы для отображения данных, вставки и удаления узлов. Программа также должна создавать экземпляр класса, вставлять узлы и удалять узлы.

Инструкции:

- Создайте класс Node с методом `__init__`, который принимает data и сохраняет его в `self._record_data`. Инициализирует `self._data_record_next` и `self._data_record_prev` как None.
- Создайте класс DoublyLinkedList с методом `__init__`, который инициализирует `self._head_record` и `self._tail_record` как None.
- Создайте метод `display` в классе DoublyLinkedList, который выводит элементы в виде: "Записи: [val1, val2, val3]". Если пуст — "[]".
- Создайте метод `insert` в классе DoublyLinkedList, который принимает значение и вставляет его **только если оно начинается с цифры 2**. Вставляет в начало.
- Создайте метод `delete` в классе DoublyLinkedList, который принимает значение и удаляет **все вхождения**.
- Создайте экземпляр класса DoublyLinkedList.
- Вставьте узлы: 15 (нет), 25 (да), 35 (нет), 28 (да), 45 (нет), 22 (да).
- Вызовите `display`.
- Вставьте 20 (да).
- Снова вызовите `display`.
- Удалите все вхождения 28.
- Снова вызовите `display`.

Пример использования:

```

dll = DoublyLinkedList()
dll.insert(15)    # нет
dll.insert(25)    # да
dll.insert(35)    # нет
dll.insert(28)    # да
dll.insert(45)    # нет
dll.insert(22)    # да

print("Initial Doubly Linked List:")
dll.display()

```

```

dll.insert(20)
print("After inserting 20:")
dll.display()

dll.delete(28)
print("After deleting all 28s:")
dll.display()

```

32. Написать программу на Python, которая создает класс DoublyLinkedList, представляющий **двусвязный список** с инкапсуляцией. Класс должен содержать методы для отображения данных, вставки и удаления узлов. Программа также должна создавать экземпляр класса, вставлять узлы и удалять узлы.

Инструкции:

- Создайте класс Node с методом `__init__`, который принимает value и сохраняет его в `self._cell_value`. Инициализирует `self._value_cell_next` и `self._value_cell_prev` как None.
- Создайте класс DoublyLinkedList с методом `__init__`, который инициализирует `self._first_cell` и `self._last_cell` как None.
- Создайте метод display в классе DoublyLinkedList, который выводит элементы в виде: "Ячейки: val1 | val2 | val3 (всего N)". Если пуст — "Нет ячеек".
- Создайте метод insert в классе DoublyLinkedList, который принимает значение и вставляет его **только если сумма его цифр нечетная**. Вставляет в конец.
- Создайте метод delete в классе DoublyLinkedList, который принимает значение и удаляет **первое вхождение**.
- Создайте экземпляр класса DoublyLinkedList.
- Вставьте узлы: 12 ($1+2=3$ — нечет, да), 14 ($1+4=5$ — нечет, да), 16 ($1+6=7$ — нечет, да), 18 ($1+8=9$ — нечет, да), 20 ($2+0=2$ — чет, нет).
- Вызовите display.
- Вставьте 21 ($2+1=3$ — нечет, да).
- Снова вызовите display.
- Удалите 16.
- Снова вызовите display.

Пример использования:

```

dll = DoublyLinkedList()
dll.insert(12)    # да
dll.insert(14)    # да
dll.insert(16)    # да
dll.insert(18)    # да
dll.insert(20)    # нет

print("Initial Doubly Linked List:")
dll.display()

dll.insert(21)
print("After inserting 21:")
dll.display()

```

```
dll.delete(16)
print("After deleting 16:")
dll.display()
```

33. Написать программу на Python, которая создает класс DoublyLinkedList, представляющий **двусвязный список** с инкапсуляцией. Класс должен содержать методы для отображения данных, вставки и удаления узлов. Программа также должна создавать экземпляр класса, вставлять узлы и удалять узлы.

Инструкции:

- Создайте класс Node с методом `__init__`, который принимает item и сохраняет его в `self._slot_data`. Инициализирует `self._data_slot_next` и `self._data_slot_prev` как None.
- Создайте класс DoublyLinkedList с методом `__init__`, который инициализирует `self._head_slot` и `self._tail_slot` как None.
- Создайте метод `display` в классе DoublyLinkedList, который выводит элементы в виде: "Слоты → val1 → val2 → val3 → конец". Если пуст — "→ конец".
- Создайте метод `insert` в классе DoublyLinkedList, который принимает значение и вставляет его **только если оно заканчивается на 0**. Вставляет в начало.
- Создайте метод `delete` в классе DoublyLinkedList, который принимает значение и удаляет **последнее вхождение**.
- Создайте экземпляр класса DoublyLinkedList.
- Вставьте узлы: 5 (нет), 10 (да), 15 (нет), 20 (да), 25 (нет), 30 (да).
- Вызовите `display`.
- Вставьте 40 (да).
- Снова вызовите `display`.
- Удалите последнее вхождение 20.
- Снова вызовите `display`.

Пример использования:

```
dll = DoublyLinkedList()
dll.insert(5)      # нет
dll.insert(10)     # да
dll.insert(15)     # нет
dll.insert(20)     # да
dll.insert(25)     # нет
dll.insert(30)     # да

print("Initial Doubly Linked List:")
dll.display()

dll.insert(40)
print("After inserting 40:")
dll.display()

dll.delete(20)
print("After deleting last occurrence of 20:")
dll.display()
```

34. Написать программу на Python, которая создает класс DoublyLinkedList, представляющий **двусвязный список** с инкапсуляцией. Класс должен содержать методы для отображения данных, вставки и удаления узлов. Программа также должна создавать экземпляр класса, вставлять узлы и удалять узлы.

Инструкции:

- (a) Создайте класс Node с методом `__init__`, который принимает data и сохраняет его в `self._block_data`. Инициализирует `self._data_block_next` и `self._data_block_prev` как None.
- (b) Создайте класс DoublyLinkedList с методом `__init__`, который инициализирует `self._first_block` и `self._last_block` как None.
- (c) Создайте метод `display` в классе DoublyLinkedList, который выводит элементы в виде: "Блоки: val1, val2, val3 (обратный: val3, val2, val1)". Если пуст — "Пусто".
- (d) Создайте метод `insert` в классе DoublyLinkedList, который принимает значение и вставляет его **только если оно простое и больше 10**. Вставляет в конец.
- (e) Создайте метод `delete` в классе DoublyLinkedList, который принимает значение и удаляет **все вхождения**.
- (f) Создайте экземпляр класса DoublyLinkedList.
- (g) Вставьте узлы: 7 (простое, но ≤ 10 — нет), 11 (да), 13 (да), 15 (нет), 17 (да), 9 (нет).
- (h) Вызовите `display`.
- (i) Вставьте 19 (да).
- (j) Снова вызовите `display`.
- (k) Удалите все вхождения 13.
- (l) Снова вызовите `display`.

Пример использования:

```
def is_prime(n):
    if n < 2:
        return False
    for i in range(2, int(n**0.5)+1):
        if n % i == 0:
            return False
    return True

dll = DoublyLinkedList()
dll.insert(7)      # нет
dll.insert(11)     # да
dll.insert(13)     # да
dll.insert(15)     # нет
dll.insert(17)     # да
dll.insert(9)      # нет

print("Initial Doubly Linked List:")
dll.display()

dll.insert(19)
print("After inserting 19:")
dll.display()

dll.delete(13)
```

```
print("After deleting all 13s:")
dll.display()
```

35. Написать программу на Python, которая создает класс DoublyLinkedList, представляющий **двусвязный список** с инкапсуляцией. Класс должен содержать методы для отображения данных, вставки и удаления узлов. Программа также должна создавать экземпляр класса, вставлять узлы и удалять узлы.

Инструкции:

- Создайте класс Node с методом `__init__`, который принимает value и сохраняет его в `self._unit_value`. Инициализирует `self._value_unit_next` и `self._value_unit_prev` как None.
- Создайте класс DoublyLinkedList с методом `__init__`, который инициализирует `self._head_unit` и `self._tail_unit` как None.
- Создайте метод `display` в классе DoublyLinkedList, который выводит элементы в виде: "Единицы: val1 <-> val2 <-> val3". Если пуст — "Нет данных".
- Создайте метод `insert` в классе DoublyLinkedList, который принимает значение и вставляет его **только если оно палиндром и двузначное**. Вставляет в начало.
- Создайте метод `delete` в классе DoublyLinkedList, который принимает значение и удаляет **первое вхождение**.
- Создайте экземпляр класса DoublyLinkedList.
- Вставьте узлы: 121 (трехзначное — нет), 22 (да), 34 (нет), 55 (да), 5 (однозначное — нет), 66 (да).
- Вызовите `display`.
- Вставьте 77 (да).
- Снова вызовите `display`.
- Удалите 55.
- Снова вызовите `display`.

Пример использования:

```
dll = DoublyLinkedList()
dll.insert(121) # нет
dll.insert(22) # да
dll.insert(34) # нет
dll.insert(55) # да
dll.insert(5)  # нет
dll.insert(66) # да

print("Initial Doubly Linked List:")
dll.display()

dll.insert(77)
print("After inserting 77:")
dll.display()

dll.delete(55)
print("After deleting 55:")
dll.display()
```


2.3.4 Задача 4 (очередь)

1. Написать программу на Python, которая создает класс `Queue` для представления структуры данных очереди с инкапсуляцией. Класс должен содержать методы `enqueue`, `dequeue` и `is_empty`, которые реализуют операции добавления элементов в очередь, удаления элементов из очереди и проверки пустоты очереди соответственно. Программа также должна создавать экземпляр класса `Queue`, добавлять элементы в очередь, удалять элементы из очереди и выводить информацию о состоянии очереди на экран.

Инструкции:

- (a) Создайте класс `Queue` с методом `__init__`, который инициализирует пустую очередь (внутренний список `_elements`). Принимает необязательный параметр `max_size` (по умолчанию `None` — без ограничений).
- (b) Создайте метод `enqueue`, который принимает элемент и добавляет его в конец очереди, только если не превышает `max_size`. Если превышает — выбрасывает `ValueError("Очередь переполнена")`.
- (c) Создайте метод `dequeue`, который удаляет и возвращает элемент из начала очереди. Если очередь пуста — выбрасывает `IndexError("Очередь пуста")`.
- (d) Создайте метод `is_empty`, который возвращает `True`, если очередь пуста, и `False` в противном случае.
- (e) Создайте приватный метод `_debug_list` (только для отладки, не включайте в задание студентам; в решении можно использовать `queue._elements`) для вывода внутреннего состояния.
- (f) Создайте экземпляр класса `Queue` с `max_size=5`.
- (g) Добавьте элементы: 100, 200, 300, 400, 500.
- (h) Попытайтесь добавить 600 — должно вызвать исключение (перехватите его и выведите сообщение).
- (i) Выведите текущее состояние очереди.
- (j) Вызовите `dequeue` дважды, выводя каждый раз удаленный элемент.
- (k) Выведите обновленное состояние очереди.

Пример использования:

```
queue = Queue(max_size=5)
queue.enqueue(100)
queue.enqueue(200)
queue.enqueue(300)
queue.enqueue(400)
queue.enqueue(500)

try:
    queue.enqueue(600)
except ValueError as e:
    print("Ошибка:", e)

print("Current Queue:", queue._elements)  # только для проверки

dequeued_item = queue.dequeue()
print("Dequeued item:", dequeued_item)

dequeued_item = queue.dequeue()
```

```
print("Dequeued item:", dequeued_item)

print("Updated Queue:", queue._elements)
```

2. Написать программу на Python, которая создает класс Queue для представления структуры данных очереди с инкапсуляцией. Класс должен содержать методы enqueue, dequeue и is_empty, которые реализуют операции добавления элементов в очередь, удаления элементов из очереди и проверки пустоты очереди соответственно. Программа также должна создавать экземпляр класса Queue, добавлять элементы в очередь, удалять элементы из очереди и выводить информацию о состоянии очереди на экран.

Инструкции:

- (a) Создайте класс Queue с методом `__init__`, который инициализирует пустую очередь (список `_items`). Принимает параметр `allow_duplicates=True`. Если `False`, то не добавляет элемент, если он уже есть в очереди.
- (b) Создайте метод `enqueue`, который принимает элемент. Если `allow_duplicates=False` и элемент уже есть в очереди — не добавляет и возвращает `False`. Иначе — добавляет в конец и возвращает `True`.
- (c) Создайте метод `dequeue`, который удаляет и возвращает первый элемент. Если очередь пуста — возвращает `None` (не выбрасывает исключение).
- (d) Создайте метод `is_empty`, который возвращает `True`, если очередь пуста, и `False` в противном случае.
- (e) Создайте экземпляр класса Queue с `allow_duplicates=False`.
- (f) Добавьте элементы: 10, 20, 10 (не добавится), 30, 20 (не добавится), 40.
- (g) Выведите текущее состояние очереди.
- (h) Вызовите `dequeue` трижды, выводя каждый раз удаленный элемент.
- (i) Выведите обновленное состояние очереди.

Пример использования:

```
queue = Queue(allow_duplicates=False)
print(queue.enqueue(10)) # True
print(queue.enqueue(20)) # True
print(queue.enqueue(10)) # False
print(queue.enqueue(30)) # True
print(queue.enqueue(20)) # False
print(queue.enqueue(40)) # True

print("Current Queue:", queue._items)

for _ in range(3):
    dequeued_item = queue.dequeue()
    print("Dequeued item:", dequeued_item)

print("Updated Queue:", queue._items)
```

3. Написать программу на Python, которая создает класс Queue для представления структуры данных очереди с инкапсуляцией. Класс должен содержать методы enqueue, dequeue и is_empty, которые реализуют операции добавления элементов в очередь, удаления элементов из очереди и проверки пустоты очереди соответственно. Программа также должна создавать экземпляр класса Queue, добавлять элементы в очередь, удалять элементы из очереди и выводить информацию о состоянии очереди на экран.

Инструкции:

- (a) Создайте класс `Queue` с методом `__init__`, который инициализирует пустую очередь (список `_data`). Принимает параметр `auto_reverse=False`. Если `True`, то `enqueue` добавляет в начало, а `dequeue` удаляет с конца (поведение стека, но интерфейс очереди).
- (b) Создайте метод `enqueue`, который добавляет элемент: если `auto_reverse=False` — в конец, если `True` — в начало.
- (c) Создайте метод `dequeue`, который удаляет и возвращает элемент: если `auto_reverse=False` — из начала, если `True` — из конца. Если очередь пуста — выбрасывает `IndexError("Пусто")`.
- (d) Создайте метод `is_empty`, который возвращает `True`, если очередь пуста, и `False` в противном случае.
- (e) Создайте экземпляр класса `Queue` с `auto_reverse=True`.
- (f) Добавьте элементы: 1, 2, 3, 4, 5.
- (g) Выведите текущее состояние очереди.
- (h) Вызовите `dequeue` дважды, выводя каждый раз удаленный элемент.
- (i) Выведите обновленное состояние очереди.

Пример использования:

```
queue = Queue(auto_reverse=True)
queue.enqueue(1)
queue.enqueue(2)
queue.enqueue(3)
queue.enqueue(4)
queue.enqueue(5)

print("Current Queue:", queue._data) # [5,4,3,2,1]

dequeued_item = queue.dequeue() # удаляет 1
print("Dequeued item:", dequeued_item)

dequeued_item = queue.dequeue() # удаляет 2
print("Dequeued item:", dequeued_item)

print("Updated Queue:", queue._data) # [5,4,3]
```

4. Написать программу на Python, которая создает класс `Queue` для представления структуры данных очереди с инкапсуляцией. Класс должен содержать методы `enqueue`, `dequeue` и `is_empty`, которые реализуют операции добавления элементов в очередь, удаления элементов из очереди и проверки пустоты очереди соответственно. Программа также должна создавать экземпляр класса `Queue`, добавлять элементы в очередь, удалять элементы из очереди и выводить информацию о состоянии очереди на экран.

Инструкции:

- (a) Создайте класс `Queue` с методом `__init__`, который инициализирует пустую очередь (список `_buffer`). Принимает параметр `dequeue_all_at_once=False`. Если `True`, то `dequeue` возвращает список всех элементов и очищает очередь.
- (b) Создайте метод `enqueue`, который добавляет элемент в конец очереди.

- (c) Создайте метод `dequeue`, который, если `dequeue_all_at_once=False`, удаляет и возвращает первый элемент. Если `True` — возвращает список всех элементов и очищает очередь. Если очередь пуста — возвращает пустой список `[]`.
- (d) Создайте метод `is_empty`, который возвращает `True`, если очередь пуста, и `False` в противном случае.
- (e) Создайте экземпляр класса `Queue` с `dequeue_all_at_once=True`.
- (f) Добавьте элементы: 5, 15, 25, 35.
- (g) Выведите текущее состояние очереди.
- (h) Вызовите `dequeue` (вернет [5,15,25,35] и очистит очередь).
- (i) Выведите результат `dequeue` и состояние очереди после вызова.

Пример использования:

```
queue = Queue(dequeue_all_at_once=True)
queue.enqueue(5)
queue.enqueue(15)
queue.enqueue(25)
queue.enqueue(35)

print("Current Queue:", queue._buffer)

dequeued_items = queue.dequeue()
print("Dequeued items:", dequeued_items) # [5,15,25,35]
print("Updated Queue:", queue._buffer)  # []
```

5. Написать программу на Python, которая создает класс `Queue` для представления структуры данных очереди с инкапсуляцией. Класс должен содержать методы `enqueue`, `dequeue` и `is_empty`, которые реализуют операции добавления элементов в очередь, удаления элементов из очереди и проверки пустоты очереди соответственно. Программа также должна создавать экземпляр класса `Queue`, добавлять элементы в очередь, удалять элементы из очереди и выводить информацию о состоянии очереди на экран.

Инструкции:

- (a) Создайте класс `Queue` с методом `__init__`, который инициализирует пустую очередь (список `_store`). Принимает параметр `on_enqueue_callback=None` — функция, вызываемая при каждом добавлении (с аргументом — добавленным элементом).
- (b) Создайте метод `enqueue`, который добавляет элемент в конец и, если `on_enqueue_callback` не `None`, вызывает её с элементом.
- (c) Создайте метод `dequeue`, который удаляет и возвращает первый элемент. Если очередь пуста — выбрасывает `IndexError("Нельзя извлечь из пустой очереди")`.
- (d) Создайте метод `is_empty`, который возвращает `True`, если очередь пуста, и `False` в противном случае.
- (e) Создайте функцию `printer(x): print(f"[+] Добавлен: x")`
- (f) Создайте экземпляр класса `Queue`, передав `printer` в `on_enqueue_callback`.
- (g) Добавьте элементы: 101, 202, 303.
- (h) Выведите текущее состояние очереди.
- (i) Вызовите `dequeue`, выведите удаленный элемент.

- (j) Выведите обновленное состояние очереди.

Пример использования:

```
def printer(x):
    print(f"[+] Добавлен: {x}")

queue = Queue(on_enqueue_callback=printer)
queue.enqueue(101) # [+] Добавлен: 101
queue.enqueue(202) # [+] Добавлен: 202
queue.enqueue(303) # [+] Добавлен: 303

print("Current Queue:", queue._store)

dequeued_item = queue.dequeue()
print("Dequeued item:", dequeued_item)

print("Updated Queue:", queue._store)
```

6. Написать программу на Python, которая создает класс `Queue` для представления структуры данных очереди с инкапсуляцией. Класс должен содержать методы `enqueue`, `dequeue` и `is_empty`, которые реализуют операции добавления элементов в очередь, удаления элементов из очереди и проверки пустоты очереди соответственно. Программа также должна создавать экземпляры класса `Queue`, добавлять элементы в очередь, удалять элементы из очереди и выводить информацию о состоянии очереди на экран.

Инструкции:

- (a) Создайте класс `Queue` с методом `__init__`, который инициализирует пустую очередь (список `_pool`). Принимает параметр `compress_on_enqueue=False`. Если `True`, то при добавлении элемента, равного последнему в очереди, вместо добавления увеличивает счетчик дубликатов у последнего элемента (хранит пары (элемент, счетчик)).
- (b) Создайте метод `enqueue`, который, если `compress_on_enqueue=True` и очередь не пуста и элемент `==` последний элемент, увеличивает счетчик последнего элемента. Иначе — добавляет новый элемент (со счетчиком 1, если режим сжатия включен).
- (c) Создайте метод `dequeue`, который удаляет первый элемент. Если режим сжатия включен и счетчик `>1`, уменьшает счетчик и возвращает элемент. Если счетчик=1, удаляет элемент. Если очередь пуста — выбрасывает `IndexError("Очередь пуста")`.
- (d) Создайте метод `is_empty`, который возвращает `True`, если очередь пуста, и `False` в противном случае.
- (e) Создайте экземпляр класса `Queue` с `compress_on_enqueue=True`.
- (f) Добавьте элементы: 7, 7, 7, 14, 14, 21.
- (g) Выведите текущее состояние очереди (внутреннее представление).
- (h) Вызовите `dequeue` трижды, выводя каждый раз удаленный элемент.
- (i) Выведите обновленное состояние очереди.

Пример использования:

```

queue = Queue(compress_on_enqueue=True)
queue.enqueue(7)
queue.enqueue(7)
queue.enqueue(7)
queue.enqueue(14)
queue.enqueue(14)
queue.enqueue(21)

print("Current Queue:", queue._pool) # [(7,3), (14,2), (21,1)]

for _ in range(3):
    dequeued_item = queue.dequeue()
    print("Dequeued item:", dequeued_item) # 7, 7, 7

print("Updated Queue:", queue._pool) # [(14,2), (21,1)]

```

7. Написать программу на Python, которая создает класс Queue для представления структуры данных очереди с инкапсуляцией. Класс должен содержать методы enqueue, dequeue и is_empty, которые реализуют операции добавления элементов в очередь, удаления элементов из очереди и проверки пустоты очереди соответственно. Программа также должна создавать экземпляр класса Queue, добавлять элементы в очередь, удалять элементы из очереди и выводить информацию о состоянии очереди на экран.

Инструкции:

- Создайте класс Queue с методом `__init__`, который инициализирует пустую очередь (список `_line`). Принимает параметр `immutable_dequeue=False`. Если True, то dequeue возвращает первый элемент, но не удаляет его.
- Создайте метод enqueue, который добавляет элемент в конец очереди.
- Создайте метод dequeue, который, если `immutable_dequeue=False`, удаляет и возвращает первый элемент. Если True — возвращает первый элемент, не удаляя его. Если очередь пуста — возвращает None.
- Создайте метод is_empty, который возвращает True, если очередь пуста, и False в противном случае.
- Создайте экземпляр класса Queue с `immutable_dequeue=True`.
- Добавьте элементы: 1, 3, 5.
- Выведите текущее состояние очереди.
- Вызовите dequeue дважды, выводя каждый раз результат (должен быть 1 оба раза).
- Выведите состояние очереди (не должно измениться).

Пример использования:

```

queue = Queue(immutable_dequeue=True)
queue.enqueue(1)
queue.enqueue(3)
queue.enqueue(5)

print("Current Queue:", queue._line)

print("Dequeued item:", queue.dequeue()) # 1
print("Dequeued item:", queue.dequeue()) # 1 (не удалось)

print("Updated Queue:", queue._line) # [1,3,5]

```

8. Написать программу на Python, которая создает класс `Queue` для представления структуры данных очереди с инкапсуляцией. Класс должен содержать методы `enqueue`, `dequeue` и `is_empty`, которые реализуют операции добавления элементов в очередь, удаления элементов из очереди и проверки пустоты очереди соответственно. Программа также должна создавать экземпляр класса `Queue`, добавлять элементы в очередь, удалять элементы из очереди и выводить информацию о состоянии очереди на экран.

Инструкции:

- (a) Создайте класс `Queue` с методом `__init__`, который инициализирует пустую очередь (список `_stream`). Принимает параметр `track_history=False`. Если `True`, сохраняет историю всех когда-либо добавленных элементов (даже удаленных) в отдельном списке `_history`.
- (b) Создайте метод `enqueue`, который добавляет элемент в конец `_stream` и, если `track_history=True`, добавляет его в `_history`.
- (c) Создайте метод `dequeue`, который удаляет и возвращает первый элемент из `_stream`. Если очередь пуста — выбрасывает `IndexError("Пусто")`.
- (d) Создайте метод `is_empty`, который возвращает `True`, если `_stream` пуст, и `False` в противном случае.
- (e) Создайте метод `get_history` (только если `track_history=True`), возвращающий копию `_history`.
- (f) Создайте экземпляр класса `Queue` с `track_history=True`.
- (g) Добавьте элементы: 2, 4, 6.
- (h) Вызовите `dequeue` (вернет 2).
- (i) Добавьте 8.
- (j) Выведите текущую очередь и историю.

Пример использования:

```
queue = Queue(track_history=True)
queue.enqueue(2)
queue.enqueue(4)
queue.enqueue(6)
queue.dequeue() # 2
queue.enqueue(8)

print("Current Queue:", queue._stream) # [4, 6, 8]
print("History:", queue.get_history()) # [2, 4, 6, 8]
```

9. Написать программу на Python, которая создает класс `Queue` для представления структуры данных очереди с инкапсуляцией. Класс должен содержать методы `enqueue`, `dequeue` и `is_empty`, которые реализуют операции добавления элементов в очередь, удаления элементов из очереди и проверки пустоты очереди соответственно. Программа также должна создавать экземпляр класса `Queue`, добавлять элементы в очередь, удалять элементы из очереди и выводить информацию о состоянии очереди на экран.

Инструкции:

- (a) Создайте класс `Queue` с методом `__init__`, который инициализирует пустую очередь (список `_flow`). Принимает параметр `enqueue_only_even=False`. Если `True`, то добавляются только четные числа.

- (b) Создайте метод `enqueue`, который добавляет элемент в конец, только если `enqueue_only_even=False` или элемент четный.
- (c) Создайте метод `dequeue`, который удаляет и возвращает первый элемент. Если очередь пуста — выбрасывает `IndexError("Очередь пуста")`.
- (d) Создайте метод `is_empty`, который возвращает `True`, если очередь пуста, и `False` в противном случае.
- (e) Создайте экземпляр класса `Queue` с `enqueue_only_even=True`.
- (f) Добавьте элементы: 1 (игнорируется), 2, 3 (игнорируется), 4, 5 (игнорируется), 6.
- (g) Выведите текущее состояние очереди.
- (h) Вызовите `dequeue`, выведите удаленный элемент.
- (i) Выведите обновленное состояние очереди.

Пример использования:

```
queue = Queue(enqueue_only_even=True)
queue.enqueue(1) # игнорируется
queue.enqueue(2)
queue.enqueue(3) # игнорируется
queue.enqueue(4)
queue.enqueue(5) # игнорируется
queue.enqueue(6)

print("Current Queue:", queue._flow) # [2,4,6]

dequeued_item = queue.dequeue()
print("Dequeued item:", dequeued_item) # 2

print("Updated Queue:", queue._flow) # [4,6]
```

10. Написать программу на Python, которая создает класс `Queue` для представления структуры данных очереди с инкапсуляцией. Класс должен содержать методы `enqueue`, `dequeue` и `is_empty`, которые реализуют операции добавления элементов в очередь, удаления элементов из очереди и проверки пустоты очереди соответственно. Программа также должна создавать экземпляр класса `Queue`, добавлять элементы в очередь, удалять элементы из очереди и выводить информацию о состоянии очереди на экран.

Инструкции:

- (a) Создайте класс `Queue` с методом `__init__`, который инициализирует пустую очередь (список `_pipe`). Принимает параметр `reverse_dequeue=False`. Если `True`, то `dequeue` удаляет и возвращает не первый, а последний элемент.
- (b) Создайте метод `enqueue`, который добавляет элемент в конец очереди.
- (c) Создайте метод `dequeue`, который, если `reverse_dequeue=False`, удаляет и возвращает первый элемент. Если `True` — удаляет и возвращает последний элемент. Если очередь пуста — выбрасывает `IndexError("Пусто")`.
- (d) Создайте метод `is_empty`, который возвращает `True`, если очередь пуста, и `False` в противном случае.
- (e) Создайте экземпляр класса `Queue` с `reverse_dequeue=True`.
- (f) Добавьте элементы: 10, 20, 30.
- (g) Выведите текущее состояние очереди.

- (h) Вызовите `dequeue` — должен вернуться 30 (последний).
- (i) Выведите обновленное состояние очереди.

Пример использования:

```
queue = Queue(reverse_dequeue=True)
queue.enqueue(10)
queue.enqueue(20)
queue.enqueue(30)

print("Current Queue:", queue._pipe) # [10,20,30]

dequeued_item = queue.dequeue() # 30
print("Dequeued item:", dequeued_item)

print("Updated Queue:", queue._pipe) # [10,20]
```

11. Написать программу на Python, которая создает класс `Queue` для представления структуры данных очереди с инкапсуляцией. Класс должен содержать методы `enqueue`, `dequeue` и `is_empty`, которые реализуют операции добавления элементов в очередь, удаления элементов из очереди и проверки пустоты очереди соответственно. Программа также должна создавать экземпляр класса `Queue`, добавлять элементы в очередь, удалять элементы из очереди и выводить информацию о состоянии очереди на экран.

Инструкции:

- (a) Создайте класс `Queue` с методом `__init__`, который инициализирует пустую очередь (список `_channel`). Принимает параметр `enqueue_with_timestamp=False`. Если `True`, то при добавлении сохраняет пару (элемент, `time.time()`).
- (b) Создайте метод `enqueue`, который, если `enqueue_with_timestamp=True`, добавляет (элемент, `timestamp`). Иначе — элемент.
- (c) Создайте метод `dequeue`, который удаляет и возвращает первый элемент (или пару). Если очередь пуста — выбрасывает `IndexError("Очередь пуста")`.
- (d) Создайте метод `is_empty`, который возвращает `True`, если очередь пуста, и `False` в противном случае.
- (e) Создайте экземпляр класса `Queue` с `enqueue_with_timestamp=True`.
- (f) Добавьте элементы: "first" "second" "third".
- (g) Выведите текущее состояние очереди.
- (h) Вызовите `dequeue`, выведите результат (пару).
- (i) Выведите обновленное состояние очереди.

Пример использования:

```
import time

queue = Queue(enqueue_with_timestamp=True)
queue.enqueue("first")
queue.enqueue("second")
queue.enqueue("third")

print("Current Queue:", queue._channel)

dequeued_item = queue.dequeue()
print("Dequeued item:", dequeued_item) # ('first', timestamp)

print("Updated Queue:", queue._channel)
```

12. Написать программу на Python, которая создает класс `Queue` для представления структуры данных очереди с инкапсуляцией. Класс должен содержать методы `enqueue`, `dequeue` и `is_empty`, которые реализуют операции добавления элементов в очередь, удаления элементов из очереди и проверки пустоты очереди соответственно. Программа также должна создавать экземпляры класса `Queue`, добавлять элементы в очередь, удалять элементы из очереди и выводить информацию о состоянии очереди на экран.

Инструкции:

- (a) Создайте класс `Queue` с методом `__init__`, который инициализирует пустую очередь (список `_tube`). Принимает параметр `enqueue_pairs=False`. Если `True`, то `enqueue` принимает два аргумента (`key`, `value`) и сохраняет кортеж (`key`, `value`).
- (b) Создайте метод `enqueue`, который, если `enqueue_pairs=False`, принимает один элемент. Если `True` — два аргумента и сохраняет кортеж.
- (c) Создайте метод `dequeue`, который удаляет и возвращает первый элемент (или кортеж). Если очередь пуста — выбрасывает `IndexError("Пусто")`.
- (d) Создайте метод `is_empty`, который возвращает `True`, если очередь пуста, и `False` в противном случае.
- (e) Создайте экземпляр класса `Queue` с `enqueue_pairs=True`.
- (f) Добавьте пары: ("a 1"), ("b 2"), ("c 3").
- (g) Выведите текущее состояние очереди.
- (h) Вызовите `dequeue`, выведите результат.
- (i) Выведите обновленное состояние очереди.

Пример использования:

```
queue = Queue(enqueue_pairs=True)
queue.enqueue("a", 1)
queue.enqueue("b", 2)
queue.enqueue("c", 3)

print("Current Queue:", queue._tube)

dequeued_item = queue.dequeue()
print("Dequeued item:", dequeued_item) # ('a', 1)

print("Updated Queue:", queue._tube)
```

13. Написать программу на Python, которая создает класс `Queue` для представления структуры данных очереди с инкапсуляцией. Класс должен содержать методы `enqueue`, `dequeue` и `is_empty`, которые реализуют операции добавления элементов в очередь, удаления элементов из очереди и проверки пустоты очереди соответственно. Программа также должна создавать экземпляры класса `Queue`, добавлять элементы в очередь, удалять элементы из очереди и выводить информацию о состоянии очереди на экран.

Инструкции:

- (a) Создайте класс `Queue` с методом `__init__`, который инициализирует пустую очередь (список `_conduit`). Принимает параметр `auto_dedup=False`. Если `True`, то при добавлении, если элемент уже есть в очереди, сначала удаляет все его предыдущие вхождения.

- (b) Создайте метод `enqueue`, который, если `auto_dedup=True` и элемент уже есть, удаляет все его вхождения, затем добавляет в конец. Иначе — просто добавляет.
- (c) Создайте метод `dequeue`, который удаляет и возвращает первый элемент. Если очередь пуста — выбрасывает `IndexError("Очередь пуста")`.
- (d) Создайте метод `is_empty`, который возвращает `True`, если очередь пуста, и `False` в противном случае.
- (e) Создайте экземпляр класса `Queue` с `auto_dedup=True`.
- (f) Добавьте элементы: 1, 2, 1, 3, 2, 4.
- (g) Выведите текущее состояние очереди.
- (h) Вызовите `dequeue`, выведите удаленный элемент.
- (i) Выведите обновленное состояние очереди.

Пример использования:

```
queue = Queue(auto_dedup=True)
queue.enqueue(1) # [1]
queue.enqueue(2) # [1, 2]
queue.enqueue(1) # удаляет старую 1 -> [2, 1]
queue.enqueue(3) # [2, 1, 3]
queue.enqueue(2) # удаляет 2 -> [1, 3, 2]
queue.enqueue(4) # [1, 3, 2, 4]

print("Current Queue:", queue._conduit)

dequeued_item = queue.dequeue()
print("Dequeued item:", dequeued_item) # 1

print("Updated Queue:", queue._conduit) # [3, 2, 4]
```

14. Написать программу на Python, которая создает класс `Queue` для представления структуры данных очереди с инкапсуляцией. Класс должен содержать методы `enqueue`, `dequeue` и `is_empty`, которые реализуют операции добавления элементов в очередь, удаления элементов из очереди и проверки пустоты очереди соответственно. Программа также должна создавать экземпляр класса `Queue`, добавлять элементы в очередь, удалять элементы из очереди и выводить информацию о состоянии очереди на экран.

Инструкции:

- (a) Создайте класс `Queue` с методом `__init__`, который инициализирует пустую очередь (список `_duct`). Принимает параметр `enqueue_if_max=False`. Если `True`, то элемент добавляется только если он больше всех текущих элементов в очереди.
- (b) Создайте метод `enqueue`, который добавляет элемент, только если `enqueue_if_max=False` или элемент $>$ всех элементов в очереди.
- (c) Создайте метод `dequeue`, который удаляет и возвращает первый элемент. Если очередь пуста — выбрасывает `IndexError("Пусто")`.
- (d) Создайте метод `is_empty`, который возвращает `True`, если очередь пуста, и `False` в противном случае.
- (e) Создайте экземпляр класса `Queue` с `enqueue_if_max=True`.
- (f) Добавьте элементы: 5, 3 (не добавится), 10, 7 (не добавится), 15.
- (g) Выведите текущее состояние очереди.

- (h) Вызовите `dequeue`, выведите удаленный элемент.
- (i) Выведите обновленное состояние очереди.

Пример использования:

```
queue = Queue(enqueue_if_max=True)
queue.enqueue(5)
queue.enqueue(3)    # не добавится
queue.enqueue(10)
queue.enqueue(7)    # не добавится
queue.enqueue(15)

print("Current Queue:", queue._duet)  # [5,10,15]

dequeued_item = queue.dequeue()
print("Dequeued item:", dequeued_item)  # 5

print("Updated Queue:", queue._duet)  # [10,15]
```

15. Написать программу на Python, которая создает класс `Queue` для представления структуры данных очереди с инкапсуляцией. Класс должен содержать методы `enqueue`, `dequeue` и `is_empty`, которые реализуют операции добавления элементов в очередь, удаления элементов из очереди и проверки пустоты очереди соответственно. Программа также должна создавать экземпляр класса `Queue`, добавлять элементы в очередь, удалять элементы из очереди и выводить информацию о состоянии очереди на экран.

Инструкции:

- (a) Создайте класс `Queue` с методом `__init__`, который инициализирует пустую очередь (список `_pipe`). Принимает параметр `cumulative=False`. Если `True`, то при добавлении элемент становится `element + последний_элемент` (если очередь не пуста). Первый элемент добавляется как есть.
- (b) Создайте метод `enqueue`, который, если `cumulative=True` и очередь не пуста, добавляет `element + последний_элемент`. Иначе — `element`.
- (c) Создайте метод `dequeue`, который удаляет и возвращает первый элемент. Если очередь пуста — выбрасывает `IndexError("Очередь пуста")`.
- (d) Создайте метод `is_empty`, который возвращает `True`, если очередь пуста, и `False` в противном случае.
- (e) Создайте экземпляр класса `Queue` с `cumulative=True`.
- (f) Добавьте элементы: 1, 2, 3, 4.
- (g) Выведите текущее состояние очереди.
- (h) Вызовите `dequeue`, выведите удаленный элемент.
- (i) Выведите обновленное состояние очереди.

Пример использования:

```
queue = Queue(cumulative=True)
queue.enqueue(1)  # [1]
queue.enqueue(2)  # [1, 1+2=3]
queue.enqueue(3)  # [1, 3, 3+3=6]
queue.enqueue(4)  # [1, 3, 6, 6+4=10]

print("Current Queue:", queue._pipe)
```

```
dequeued_item = queue.dequeue()
print("Dequeued item:", dequeued_item) # 1

print("Updated Queue:", queue._pipe) # [3,6,10]
```

16. Написать программу на Python, которая создает класс Queue для представления структуры данных очереди с инкапсуляцией. Класс должен содержать методы enqueue, dequeue и is_empty, которые реализуют операции добавления элементов в очередь, удаления элементов из очереди и проверки пустоты очереди соответственно. Программа также должна создавать экземпляр класса Queue, добавлять элементы в очередь, удалять элементы из очереди и выводить информацию о состоянии очереди на экран.

Инструкции:

- Создайте класс Queue с методом `__init__`, который инициализирует пустую очередь (список `_line`). Принимает параметр `enqueue_squared=False`. Если True, то при добавлении сохраняется `element**2`.
- Создайте метод `enqueue`, который добавляет `element**2`, если `enqueue_squared=True`, иначе — `element`.
- Создайте метод `dequeue`, который удаляет и возвращает первый элемент. Если очередь пуста — выбрасывает `IndexError("Пусто")`.
- Создайте метод `is_empty`, который возвращает True, если очередь пуста, и False в противном случае.
- Создайте экземпляр класса Queue с `enqueue_squared=True`.
- Добавьте элементы: 2, 3, 4, 5.
- Выведите текущее состояние очереди.
- Вызовите `dequeue`, выведите удаленный элемент.
- Выведите обновленное состояние очереди.

Пример использования:

```
queue = Queue(enqueue_squared=True)
queue.enqueue(2) # 4
queue.enqueue(3) # 9
queue.enqueue(4) # 16
queue.enqueue(5) # 25

print("Current Queue:", queue._line)

dequeued_item = queue.dequeue()
print("Dequeued item:", dequeued_item) # 4

print("Updated Queue:", queue._line) # [9,16,25]
```

17. Написать программу на Python, которая создает класс Queue для представления структуры данных очереди с инкапсуляцией. Класс должен содержать методы enqueue, dequeue и is_empty, которые реализуют операции добавления элементов в очередь, удаления элементов из очереди и проверки пустоты очереди соответственно. Программа также должна создавать экземпляр класса Queue, добавлять элементы в очередь, удалять элементы из очереди и выводить информацию о состоянии очереди на экран.

Инструкции:

- (a) Создайте класс `Queue` с методом `__init__`, который инициализирует пустую очередь (список `_stream`). Принимает параметр `enqueue_absolute=False`. Если `True`, то при добавлении сохраняется `abs(element)`.
- (b) Создайте метод `enqueue`, который добавляет `abs(element)`, если `enqueue_absolute=True`, иначе — `element`.
- (c) Создайте метод `dequeue`, который удаляет и возвращает первый элемент. Если очередь пуста — выбрасывает `IndexError("Очередь пуста")`.
- (d) Создайте метод `is_empty`, который возвращает `True`, если очередь пуста, и `False` в противном случае.
- (e) Создайте экземпляр класса `Queue` с `enqueue_absolute=True`.
- (f) Добавьте элементы: -5, 3, -8, 2.
- (g) Выведите текущее состояние очереди.
- (h) Вызовите `dequeue`, выведите удаленный элемент.
- (i) Выведите обновленное состояние очереди.

Пример использования:

```
queue = Queue(enqueue_absolute=True)
queue.enqueue(-5) # 5
queue.enqueue(3) # 3
queue.enqueue(-8) # 8
queue.enqueue(2) # 2

print("Current Queue:", queue._stream)

dequeued_item = queue.dequeue()
print("Dequeued item:", dequeued_item) # 5

print("Updated Queue:", queue._stream) # [3, 8, 2]
```

18. Написать программу на Python, которая создает класс `Queue` для представления структуры данных очереди с инкапсуляцией. Класс должен содержать методы `enqueue`, `dequeue` и `is_empty`, которые реализуют операции добавления элементов в очередь, удаления элементов из очереди и проверки пустоты очереди соответственно. Программа также должна создавать экземпляр класса `Queue`, добавлять элементы в очередь, удалять элементы из очереди и выводить информацию о состоянии очереди на экран.

Инструкции:

- (a) Создайте класс `Queue` с методом `__init__`, который инициализирует пустую очередь (список `_buffer`). Принимает параметр `enqueue_rounded=False`. Если `True`, то при добавлении сохраняется `round(element)`.
- (b) Создайте метод `enqueue`, который добавляет `round(element)`, если `enqueue_rounded=True`, иначе — `element`.
- (c) Создайте метод `dequeue`, который удаляет и возвращает первый элемент. Если очередь пуста — выбрасывает `IndexError("Пусто")`.
- (d) Создайте метод `is_empty`, который возвращает `True`, если очередь пуста, и `False` в противном случае.
- (e) Создайте экземпляр класса `Queue` с `enqueue_rounded=True`.
- (f) Добавьте элементы: 3.2, 4.7, 5.1, 6.9.

- (g) Выведите текущее состояние очереди.
- (h) Вызовите `dequeue`, выведите удаленный элемент.
- (i) Выведите обновленное состояние очереди.

Пример использования:

```
queue = Queue(enqueue_rounded=True)
queue.enqueue(3.2) # 3
queue.enqueue(4.7) # 5
queue.enqueue(5.1) # 5
queue.enqueue(6.9) # 7

print("Current Queue:", queue._buffer)

dequeued_item = queue.dequeue()
print("Dequeued item:", dequeued_item) # 3

print("Updated Queue:", queue._buffer) # [5, 5, 7]
```

19. Написать программу на Python, которая создает класс `Queue` для представления структуры данных очереди с инкапсуляцией. Класс должен содержать методы `enqueue`, `dequeue` и `is_empty`, которые реализуют операции добавления элементов в очередь, удаления элементов из очереди и проверки пустоты очереди соответственно. Программа также должна создавать экземпляры класса `Queue`, добавлять элементы в очередь, удалять элементы из очереди и выводить информацию о состоянии очереди на экран.

Инструкции:

- (a) Создайте класс `Queue` с методом `__init__`, который инициализирует пустую очередь (список `_store`). Принимает параметр `enqueue_negated=False`. Если `True`, то при добавлении сохраняется `-element`.
- (b) Создайте метод `enqueue`, который добавляет `-element`, если `enqueue_negated=True`, иначе — `element`.
- (c) Создайте метод `dequeue`, который удаляет и возвращает первый элемент. Если очередь пуста — выбрасывает `IndexError("Очередь пуста")`.
- (d) Создайте метод `is_empty`, который возвращает `True`, если очередь пуста, и `False` в противном случае.
- (e) Создайте экземпляр класса `Queue` с `enqueue_negated=True`.
- (f) Добавьте элементы: 10, 20, 30, 40.
- (g) Выведите текущее состояние очереди.
- (h) Вызовите `dequeue`, выведите удаленный элемент.
- (i) Выведите обновленное состояние очереди.

Пример использования:

```
queue = Queue(enqueue_negated=True)
queue.enqueue(10) # -10
queue.enqueue(20) # -20
queue.enqueue(30) # -30
queue.enqueue(40) # -40

print("Current Queue:", queue._store)
```

```
dequeued_item = queue.dequeue()
print("Dequeued item:", dequeued_item) # -10

print("Updated Queue:", queue._store) # [-20, -30, -40]
```

20. Написать программу на Python, которая создает класс Queue для представления структуры данных очереди с инкапсуляцией. Класс должен содержать методы enqueue, dequeue и is_empty, которые реализуют операции добавления элементов в очередь, удаления элементов из очереди и проверки пустоты очереди соответственно. Программа также должна создавать экземпляр класса Queue, добавлять элементы в очередь, удалять элементы из очереди и выводить информацию о состоянии очереди на экран.

Инструкции:

- Создайте класс Queue с методом `__init__`, который инициализирует пустую очередь (список `_pool`). Принимает параметр `enqueue_doubled=False`. Если True, то при добавлении сохраняется `element * 2`.
- Создайте метод enqueue, который добавляет `element * 2`, если `enqueue_doubled=True`, иначе — `element`.
- Создайте метод dequeue, который удаляет и возвращает первый элемент. Если очередь пуста — выбрасывает `IndexError("Пусто")`.
- Создайте метод is_empty, который возвращает True, если очередь пуста, и False в противном случае.
- Создайте экземпляр класса Queue с `enqueue_doubled=True`.
- Добавьте элементы: 1, 2, 3, 4.
- Выведите текущее состояние очереди.
- Вызовите dequeue, выведите удаленный элемент.
- Выведите обновленное состояние очереди.

Пример использования:

```
queue = Queue(enqueue_doubled=True)
queue.enqueue(1) # 2
queue.enqueue(2) # 4
queue.enqueue(3) # 6
queue.enqueue(4) # 8

print("Current Queue:", queue._pool)

dequeued_item = queue.dequeue()
print("Dequeued item:", dequeued_item) # 2

print("Updated Queue:", queue._pool) # [4, 6, 8]
```

21. Написать программу на Python, которая создает класс Queue для представления структуры данных очереди с инкапсуляцией. Класс должен содержать методы enqueue, dequeue и is_empty, которые реализуют операции добавления элементов в очередь, удаления элементов из очереди и проверки пустоты очереди соответственно. Программа также должна создавать экземпляр класса Queue, добавлять элементы в очередь, удалять элементы из очереди и выводить информацию о состоянии очереди на экран.

Инструкции:

- (a) Создайте класс `Queue` с методом `__init__`, который инициализирует пустую очередь (список `_reservoir`). Принимает параметр `enqueue_halved=False`. Если `True`, то при добавлении сохраняется `element / 2.0`.
- (b) Создайте метод `enqueue`, который добавляет `element / 2.0`, если `enqueue_halved=True`, иначе — `element`.
- (c) Создайте метод `dequeue`, который удаляет и возвращает первый элемент. Если очередь пуста — выбрасывает `IndexError("Очередь пуста")`.
- (d) Создайте метод `is_empty`, который возвращает `True`, если очередь пуста, и `False` в противном случае.
- (e) Создайте экземпляр класса `Queue` с `enqueue_halved=True`.
- (f) Добавьте элементы: 4, 8, 12, 16.
- (g) Выведите текущее состояние очереди.
- (h) Вызовите `dequeue`, выведите удаленный элемент.
- (i) Выведите обновленное состояние очереди.

Пример использования:

```
queue = Queue(enqueue_halved=True)
queue.enqueue(4)      # 2.0
queue.enqueue(8)      # 4.0
queue.enqueue(12)     # 6.0
queue.enqueue(16)     # 8.0

print("Current Queue:", queue._reservoir)

dequeued_item = queue.dequeue()
print("Dequeued item:", dequeued_item) # 2.0

print("Updated Queue:", queue._reservoir) # [4.0, 6.0, 8.0]
```

22. Написать программу на Python, которая создает класс `Queue` для представления структуры данных очереди с инкапсуляцией. Класс должен содержать методы `enqueue`, `dequeue` и `is_empty`, которые реализуют операции добавления элементов в очередь, удаления элементов из очереди и проверки пустоты очереди соответственно. Программа также должна создавать экземпляр класса `Queue`, добавлять элементы в очередь, удалять элементы из очереди и выводить информацию о состоянии очереди на экран.

Инструкции:

- (a) Создайте класс `Queue` с методом `__init__`, который инициализирует пустую очередь (список `_tank`). Принимает параметр `enqueue_as_string=False`. Если `True`, то при добавлении сохраняется `str(element)`.
- (b) Создайте метод `enqueue`, который добавляет `str(element)`, если `enqueue_as_string=True`, иначе — `element`.
- (c) Создайте метод `dequeue`, который удаляет и возвращает первый элемент. Если очередь пуста — выбрасывает `IndexError("Пусто")`.
- (d) Создайте метод `is_empty`, который возвращает `True`, если очередь пуста, и `False` в противном случае.
- (e) Создайте экземпляр класса `Queue` с `enqueue_as_string=True`.
- (f) Добавьте элементы: 100, 200, 300, 400.

- (g) Выведите текущее состояние очереди.
- (h) Вызовите `dequeue`, выведите удаленный элемент.
- (i) Выведите обновленное состояние очереди.

Пример использования:

```
queue = Queue(enqueue_as_string=True)
queue.enqueue(100) # "100"
queue.enqueue(200) # "200"
queue.enqueue(300) # "300"
queue.enqueue(400) # "400"

print("Current Queue:", queue._tank)

dequeued_item = queue.dequeue()
print("Dequeued item:", dequeued_item) # "100"

print("Updated Queue:", queue._tank) # ["200", "300", "400"]
```

23. Написать программу на Python, которая создает класс `Queue` для представления структуры данных очереди с инкапсуляцией. Класс должен содержать методы `enqueue`, `dequeue` и `is_empty`, которые реализуют операции добавления элементов в очередь, удаления элементов из очереди и проверки пустоты очереди соответственно. Программа также должна создавать экземпляр класса `Queue`, добавлять элементы в очередь, удалять элементы из очереди и выводить информацию о состоянии очереди на экран.

Инструкции:

- (a) Создайте класс `Queue` с методом `__init__`, который инициализирует пустую очередь (список `_container`). Принимает параметр `enqueue_with_index=False`. Если `True`, то при добавлении сохраняется кортеж (`element`, порядковый_номер_добавления).
- (b) Создайте метод `enqueue`, который добавляет (`element`, `self._counter`), где `_counter` — внутренний счетчик, увеличивающийся при каждом добавлении. Иначе — `element`.
- (c) Создайте метод `dequeue`, который удаляет и возвращает первый элемент (или кортеж). Если очередь пуста — выбрасывает `IndexError("Очередь пуста")`.
- (d) Создайте метод `is_empty`, который возвращает `True`, если очередь пуста, и `False` в противном случае.
- (e) Создайте экземпляр класса `Queue` с `enqueue_with_index=True`.
- (f) Добавьте элементы: "alpha" "beta" "gamma".
- (g) Выведите текущее состояние очереди.
- (h) Вызовите `dequeue`, выведите удаленный элемент.
- (i) Выведите обновленное состояние очереди.

Пример использования:

```
queue = Queue(enqueue_with_index=True)
queue.enqueue("alpha") # ("alpha", 0)
queue.enqueue("beta")  # ("beta", 1)
queue.enqueue("gamma") # ("gamma", 2)

print("Current Queue:", queue._container)

dequeued_item = queue.dequeue()
```

```
print("Dequeued item:", dequeued_item) # ('alpha', 0)

print("Updated Queue:", queue._container) # [('beta',1), ('gamma',2)]
```

24. Написать программу на Python, которая создает класс Queue для представления структуры данных очереди с инкапсуляцией. Класс должен содержать методы enqueue, dequeue и is_empty, которые реализуют операции добавления элементов в очередь, удаления элементов из очереди и проверки пустоты очереди соответственно. Программа также должна создавать экземпляр класса Queue, добавлять элементы в очередь, удалять элементы из очереди и выводить информацию о состоянии очереди на экран.

Инструкции:

- Создайте класс Queue с методом `__init__`, который инициализирует пустую очередь (список `_vessel`). Принимает параметр `enqueue_unique_rear=False`. Если True, то при добавлении, если элемент равен текущему последнему, он не добавляется.
- Создайте метод `enqueue`, который добавляет элемент, только если `enqueue_unique_rear=False` или очередь пуста или `element != последний_элемент`.
- Создайте метод `dequeue`, который удаляет и возвращает первый элемент. Если очередь пуста — выбрасывает `IndexError("Пусто")`.
- Создайте метод `is_empty`, который возвращает True, если очередь пуста, и False в противном случае.
- Создайте экземпляр класса Queue с `enqueue_unique_rear=True`.
- Добавьте элементы: 1, 2, 2, 3, 3, 3, 4.
- Выведите текущее состояние очереди.
- Вызовите `dequeue`, выведите удаленный элемент.
- Выведите обновленное состояние очереди.

Пример использования:

```
queue = Queue(enqueue_unique_rear=True)
queue.enqueue(1)
queue.enqueue(2)
queue.enqueue(2) # не добавится
queue.enqueue(3)
queue.enqueue(3) # не добавится
queue.enqueue(3) # не добавится
queue.enqueue(4)

print("Current Queue:", queue._vessel) # [1,2,3,4]

dequeued_item = queue.dequeue()
print("Dequeued item:", dequeued_item) # 1

print("Updated Queue:", queue._vessel) # [2,3,4]
```

25. Написать программу на Python, которая создает класс Queue для представления структуры данных очереди с инкапсуляцией. Класс должен содержать методы enqueue, dequeue и is_empty, которые реализуют операции добавления элементов в очередь, удаления элементов из очереди и проверки пустоты очереди соответственно. Программа также должна создавать экземпляр класса Queue, добавлять элементы в очередь, удалять элементы из очереди и выводить информацию о состоянии очереди на экран.

Инструкции:

- (a) Создайте класс `Queue` с методом `__init__`, который инициализирует пустую очередь (список `_bin`). Принимает параметр `enqueue_even_only=False`. Если `True`, то добавляются только четные числа.
- (b) Создайте метод `enqueue`, который добавляет элемент, только если `enqueue_even_only=False` или `element % 2 == 0`.
- (c) Создайте метод `dequeue`, который удаляет и возвращает первый элемент. Если очередь пуста — выбрасывает `IndexError("Очередь пуста")`.
- (d) Создайте метод `is_empty`, который возвращает `True`, если очередь пуста, и `False` в противном случае.
- (e) Создайте экземпляр класса `Queue` с `enqueue_even_only=True`.
- (f) Добавьте элементы: 1 (не добавится), 2, 3 (не добавится), 4, 5 (не добавится), 6.
- (g) Выведите текущее состояние очереди.
- (h) Вызовите `dequeue`, выведите удаленный элемент.
- (i) Выведите обновленное состояние очереди.

Пример использования:

```
queue = Queue(enqueue_even_only=True)
queue.enqueue(1)    # нет
queue.enqueue(2)
queue.enqueue(3)    # нет
queue.enqueue(4)
queue.enqueue(5)    # нет
queue.enqueue(6)

print("Current Queue:", queue._bin)    # [2,4,6]

dequeued_item = queue.dequeue()
print("Dequeued item:", dequeued_item)  # 2

print("Updated Queue:", queue._bin)    # [4,6]
```

26. Написать программу на Python, которая создает класс `Queue` для представления структуры данных очереди с инкапсуляцией. Класс должен содержать методы `enqueue`, `dequeue` и `is_empty`, которые реализуют операции добавления элементов в очередь, удаления элементов из очереди и проверки пустоты очереди соответственно. Программа также должна создавать экземпляр класса `Queue`, добавлять элементы в очередь, удалять элементы из очереди и выводить информацию о состоянии очереди на экран.

Инструкции:

- (a) Создайте класс `Queue` с методом `__init__`, который инициализирует пустую очередь (список `_box`). Принимает параметр `enqueue_odd_only=False`. Если `True`, то добавляются только нечетные числа.
- (b) Создайте метод `enqueue`, который добавляет элемент, только если `enqueue_odd_only=False` или `element % 2 != 0`.
- (c) Создайте метод `dequeue`, который удаляет и возвращает первый элемент. Если очередь пуста — выбрасывает `IndexError("Пусто")`.

- (d) Создайте метод `is_empty`, который возвращает `True`, если очередь пуста, и `False` в противном случае.
- (e) Создайте экземпляр класса `Queue` с `enqueue_odd_only=True`.
- (f) Добавьте элементы: 2 (не добавится), 1, 4 (не добавится), 3, 6 (не добавится), 5.
- (g) Выведите текущее состояние очереди.
- (h) Вызовите `dequeue`, выведите удаленный элемент.
- (i) Выведите обновленное состояние очереди.

Пример использования:

```
queue = Queue(enqueue_odd_only=True)
queue.enqueue(2)    # нет
queue.enqueue(1)
queue.enqueue(4)    # нет
queue.enqueue(3)
queue.enqueue(6)    # нет
queue.enqueue(5)

print("Current Queue:", queue._box)    # [1,3,5]

dequeued_item = queue.dequeue()
print("Dequeued item:", dequeued_item)    # 1

print("Updated Queue:", queue._box)    # [3,5]
```

27. Написать программу на Python, которая создает класс `Queue` для представления структуры данных очереди с инкапсуляцией. Класс должен содержать методы `enqueue`, `dequeue` и `is_empty`, которые реализуют операции добавления элементов в очередь, удаления элементов из очереди и проверки пустоты очереди соответственно. Программа также должна создавать экземпляр класса `Queue`, добавлять элементы в очередь, удалять элементы из очереди и выводить информацию о состоянии очереди на экран.

Инструкции:

- (a) Создайте класс `Queue` с методом `__init__`, который инициализирует пустую очередь (список `_crate`). Принимает параметр `enqueue_positive_only=False`. Если `True`, то добавляются только положительные числа (>0).
- (b) Создайте метод `enqueue`, который добавляет элемент, только если `enqueue_positive_only=False` или `element > 0`.
- (c) Создайте метод `dequeue`, который удаляет и возвращает первый элемент. Если очередь пуста — выбрасывает `IndexError("Очередь пуста")`.
- (d) Создайте метод `is_empty`, который возвращает `True`, если очередь пуста, и `False` в противном случае.
- (e) Создайте экземпляр класса `Queue` с `enqueue_positive_only=True`.
- (f) Добавьте элементы: -1 (не добавится), 0 (не добавится), 1, 2, -5 (не добавится), 3.
- (g) Выведите текущее состояние очереди.
- (h) Вызовите `dequeue`, выведите удаленный элемент.
- (i) Выведите обновленное состояние очереди.

Пример использования:

```

queue = Queue(enqueue_positive_only=True)
queue.enqueue(-1) # нет
queue.enqueue(0) # нет
queue.enqueue(1)
queue.enqueue(2)
queue.enqueue(-5) # нет
queue.enqueue(3)

print("Current Queue:", queue._crate) # [1,2,3]

dequeued_item = queue.dequeue()
print("Dequeued item:", dequeued_item) # 1

print("Updated Queue:", queue._crate) # [2,3]

```

28. Написать программу на Python, которая создает класс Queue для представления структуры данных очереди с инкапсуляцией. Класс должен содержать методы enqueue, dequeue и is_empty, которые реализуют операции добавления элементов в очередь, удаления элементов из очереди и проверки пустоты очереди соответственно. Программа также должна создавать экземпляры класса Queue, добавлять элементы в очередь, удалять элементы из очереди и выводить информацию о состоянии очереди на экран.

Инструкции:

- Создайте класс Queue с методом `__init__`, который инициализирует пустую очередь (список `_carton`). Принимает параметр `enqueue_nonzero_only=False`. Если True, то добавляются только ненулевые числа.
- Создайте метод enqueue, который добавляет элемент, только если `enqueue_nonzero_only=False` или `element != 0`.
- Создайте метод dequeue, который удаляет и возвращает первый элемент. Если очередь пуста — выбрасывает `IndexError("Пусто")`.
- Создайте метод is_empty, который возвращает True, если очередь пуста, и False в противном случае.
- Создайте экземпляр класса Queue с `enqueue_nonzero_only=True`.
- Добавьте элементы: 0 (не добавится), 5, 0 (не добавится), 10, 15.
- Выведите текущее состояние очереди.
- Вызовите dequeue, выведите удаленный элемент.
- Выведите обновленное состояние очереди.

Пример использования:

```

queue = Queue(enqueue_nonzero_only=True)
queue.enqueue(0) # нет
queue.enqueue(5)
queue.enqueue(0) # нет
queue.enqueue(10)
queue.enqueue(15)

print("Current Queue:", queue._carton) # [5,10,15]

dequeued_item = queue.dequeue()
print("Dequeued item:", dequeued_item) # 5

print("Updated Queue:", queue._carton) # [10,15]

```

29. Написать программу на Python, которая создает класс `Queue` для представления структуры данных очереди с инкапсуляцией. Класс должен содержать методы `enqueue`, `dequeue` и `is_empty`, которые реализуют операции добавления элементов в очередь, удаления элементов из очереди и проверки пустоты очереди соответственно. Программа также должна создавать экземпляр класса `Queue`, добавлять элементы в очередь, удалять элементы из очереди и выводить информацию о состоянии очереди на экран.

Инструкции:

- (a) Создайте класс `Queue` с методом `__init__`, который инициализирует пустую очередь (список `_package`). Принимает параметр `enqueue_prime_only=False`. Если `True`, то добавляются только простые числа (реализуйте простую проверку).
- (b) Создайте метод `enqueue`, который добавляет элемент, только если `enqueue_prime_only=False` или `element` — простое число.
- (c) Создайте метод `dequeue`, который удаляет и возвращает первый элемент. Если очередь пуста — выбрасывает `IndexError("Очередь пуста")`.
- (d) Создайте метод `is_empty`, который возвращает `True`, если очередь пуста, и `False` в противном случае.
- (e) Создайте вспомогательную функцию `is_prime(n)` (вне класса).
- (f) Создайте экземпляр класса `Queue` с `enqueue_prime_only=True`.
- (g) Добавьте элементы: 4 (не простое), 5 (простое), 6 (не простое), 7 (простое), 8 (не простое), 11 (простое).
- (h) Выведите текущее состояние очереди.
- (i) Вызовите `dequeue`, выведите удаленный элемент.
- (j) Выведите обновленное состояние очереди.

Пример использования:

```
def is_prime(n):
    if n < 2:
        return False
    for i in range(2, int(n**0.5)+1):
        if n % i == 0:
            return False
    return True

queue = Queue(enqueue_prime_only=True)
queue.enqueue(4)    # нет
queue.enqueue(5)    # да
queue.enqueue(6)    # нет
queue.enqueue(7)    # да
queue.enqueue(8)    # нет
queue.enqueue(11)   # да

print("Current Queue:", queue._package) # [5, 7, 11]

dequeued_item = queue.dequeue()
print("Dequeued item:", dequeued_item)  # 5

print("Updated Queue:", queue._package) # [7, 11]
```

30. Написать программу на Python, которая создает класс `Queue` для представления структуры данных очереди с инкапсуляцией. Класс должен содержать методы `enqueue`,

`dequeue` и `is_empty`, которые реализуют операции добавления элементов в очередь, удаления элементов из очереди и проверки пустоты очереди соответственно. Программа также должна создавать экземпляр класса `Queue`, добавлять элементы в очередь, удалять элементы из очереди и выводить информацию о состоянии очереди на экран.

Инструкции:

- (a) Создайте класс `Queue` с методом `__init__`, который инициализирует пустую очередь (список `_parcel`). Принимает параметр `enqueue_fibonacci_only=False`. Если `True`, то добавляются только числа Фибоначчи (до 100: 0,1,1,2,3,5,8,13,21,34,55,89).
- (b) Создайте метод `enqueue`, который добавляет элемент, только если `enqueue_fibonacci_only=False` или `element` входит в `FIB_SET`.
- (c) Создайте метод `dequeue`, который удаляет и возвращает первый элемент. Если очередь пуста — выбрасывает `IndexError("Пусто")`.
- (d) Создайте метод `is_empty`, который возвращает `True`, если очередь пуста, и `False` в противном случае.
- (e) Создайте экземпляр класса `Queue` с `enqueue_fibonacci_only=True`.
- (f) Добавьте элементы: 4 (не Фибоначчи), 5 (Фибоначчи), 6 (не Фибоначчи), 8 (Фибоначчи), 7 (не Фибоначчи), 13 (Фибоначчи).
- (g) Выведите текущее состояние очереди.
- (h) Вызовите `dequeue`, выведите удаленный элемент.
- (i) Выведите обновленное состояние очереди.

Пример использования:

```
FIB_SET = {0, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89}

queue = Queue(enqueue_fibonacci_only=True)
queue.enqueue(4)    # нет
queue.enqueue(5)    # да
queue.enqueue(6)    # нет
queue.enqueue(8)    # да
queue.enqueue(7)    # нет
queue.enqueue(13)   # да

print("Current Queue:", queue._parcel)  # [5,8,13]

dequeued_item = queue.dequeue()
print("Dequeued item:", dequeued_item)  # 5

print("Updated Queue:", queue._parcel)  # [8,13]
```

31. Написать программу на Python, которая создает класс `Queue` для представления структуры данных очереди с инкапсуляцией. Класс должен содержать методы `enqueue`, `dequeue` и `is_empty`, которые реализуют операции добавления элементов в очередь, удаления элементов из очереди и проверки пустоты очереди соответственно. Программа также должна создавать экземпляр класса `Queue`, добавлять элементы в очередь, удалять элементы из очереди и выводить информацию о состоянии очереди на экран.

Инструкции:

- (a) Создайте класс `Queue` с методом `__init__`, который инициализирует пустую очередь (список `_sack`). Принимает параметр `enqueue_palindrome_only=False`. Если `True`, то добавляются только числа-палиндромы.

- (b) Создайте метод `enqueue`, который добавляет элемент, только если `enqueue_palindrome_only=False` или `element` — палиндром (`str(element) == str(element)[::-1]`).
- (c) Создайте метод `dequeue`, который удаляет и возвращает первый элемент. Если очередь пуста — выбрасывает `IndexError("Очередь пуста")`.
- (d) Создайте метод `is_empty`, который возвращает `True`, если очередь пуста, и `False` в противном случае.
- (e) Создайте экземпляр класса `Queue` с `enqueue_palindrome_only=True`.
- (f) Добавьте элементы: 12 (не палиндром), 22 (палиндром), 34 (не палиндром), 55 (палиндром), 123 (не палиндром), 121 (палиндром).
- (g) Выведите текущее состояние очереди.
- (h) Вызовите `dequeue`, выведите удаленный элемент.
- (i) Выведите обновленное состояние очереди.

Пример использования:

```
queue = Queue(enqueue_palindrome_only=True)
queue.enqueue(12)      # нет
queue.enqueue(22)      # да
queue.enqueue(34)      # нет
queue.enqueue(55)      # да
queue.enqueue(123)     # нет
queue.enqueue(121)     # да

print("Current Queue:", queue._sack)  # [22,55,121]

dequeued_item = queue.dequeue()
print("Dequeued item:", dequeued_item)  # 22

print("Updated Queue:", queue._sack)  # [55,121]
```

32. Написать программу на Python, которая создает класс `Queue` для представления структуры данных очереди с инкапсуляцией. Класс должен содержать методы `enqueue`, `dequeue` и `is_empty`, которые реализуют операции добавления элементов в очередь, удаления элементов из очереди и проверки пустоты очереди соответственно. Программа также должна создавать экземпляр класса `Queue`, добавлять элементы в очередь, удалять элементы из очереди и выводить информацию о состоянии очереди на экран.

Инструкции:

- (a) Создайте класс `Queue` с методом `__init__`, который инициализирует пустую очередь (список `_bag`). Принимает параметр `enqueue_power_of_two=False`. Если `True`, то добавляются только степени двойки.
- (b) Создайте метод `enqueue`, который добавляет элемент, только если `enqueue_power_of_two=False` или `element > 0` и `(element & (element-1)) == 0`.
- (c) Создайте метод `dequeue`, который удаляет и возвращает первый элемент. Если очередь пуста — выбрасывает `IndexError("Пусто")`.
- (d) Создайте метод `is_empty`, который возвращает `True`, если очередь пуста, и `False` в противном случае.
- (e) Создайте экземпляр класса `Queue` с `enqueue_power_of_two=True`.
- (f) Добавьте элементы: 3 (не степень), 4 (степень), 5 (не степень), 8 (степень), 9 (не степень), 16 (степень).

- (g) Выведите текущее состояние очереди.
- (h) Вызовите `dequeue`, выведите удаленный элемент.
- (i) Выведите обновленное состояние очереди.

Пример использования:

```
queue = Queue(enqueue_power_of_two=True)
queue.enqueue(3)    # нет
queue.enqueue(4)    # да
queue.enqueue(5)    # нет
queue.enqueue(8)    # да
queue.enqueue(9)    # нет
queue.enqueue(16)   # да

print("Current Queue:", queue._bag)    # [4,8,16]

dequeued_item = queue.dequeue()
print("Dequeued item:", dequeued_item) # 4

print("Updated Queue:", queue._bag)    # [8,16]
```

33. Написать программу на Python, которая создает класс `Queue` для представления структуры данных очереди с инкапсуляцией. Класс должен содержать методы `enqueue`, `dequeue` и `is_empty`, которые реализуют операции добавления элементов в очередь, удаления элементов из очереди и проверки пустоты очереди соответственно. Программа также должна создавать экземпляры класса `Queue`, добавлять элементы в очередь, удалять элементы из очереди и выводить информацию о состоянии очереди на экран.

Инструкции:

- (a) Создайте класс `Queue` с методом `__init__`, который инициализирует пустую очередь (список `_suitcase`). Принимает параметр `enqueue_divisible_by_three=False`. Если `True`, то добавляются только числа, делящиеся на 3.
- (b) Создайте метод `enqueue`, который добавляет элемент, только если `enqueue_divisible_by_three=False` или `element % 3 == 0`.
- (c) Создайте метод `dequeue`, который удаляет и возвращает первый элемент. Если очередь пуста — выбрасывает `IndexError("Очередь пуста")`.
- (d) Создайте метод `is_empty`, который возвращает `True`, если очередь пуста, и `False` в противном случае.
- (e) Создайте экземпляр класса `Queue` с `enqueue_divisible_by_three=True`.
- (f) Добавьте элементы: 1 (нет), 3 (да), 4 (нет), 6 (да), 7 (нет), 9 (да).
- (g) Выведите текущее состояние очереди.
- (h) Вызовите `dequeue`, выведите удаленный элемент.
- (i) Выведите обновленное состояние очереди.

Пример использования:

```
queue = Queue(enqueue_divisible_by_three=True)
queue.enqueue(1)    # нет
queue.enqueue(3)    # да
queue.enqueue(4)    # нет
queue.enqueue(6)    # да
queue.enqueue(7)    # нет
```

```

queue.enqueue(9)    # да

print("Current Queue:", queue._suitcase)    # [3,6,9]

dequeued_item = queue.dequeue()
print("Dequeued item:", dequeued_item)    # 3

print("Updated Queue:", queue._suitcase)    # [6,9]

```

34. Написать программу на Python, которая создает класс Queue для представления структуры данных очереди с инкапсуляцией. Класс должен содержать методы enqueue, dequeue и is_empty, которые реализуют операции добавления элементов в очередь, удаления элементов из очереди и проверки пустоты очереди соответственно. Программа также должна создавать экземпляр класса Queue, добавлять элементы в очередь, удалять элементы из очереди и выводить информацию о состоянии очереди на экран.

Инструкции:

- Создайте класс Queue с методом `__init__`, который инициализирует пустую очередь (список `_luggage`). Принимает параметр `enqueue_greater_than_prev=False`. Если True, то элемент добавляется только если он строго больше предыдущего добавленного элемента (первый — всегда).
- Создайте метод `enqueue`, который добавляет элемент, только если `enqueue_greater_than_prev=False` или очередь пуста или `element > последний_элемент`.
- Создайте метод `dequeue`, который удаляет и возвращает первый элемент. Если очередь пуста — выбрасывает `IndexError("Пусто")`.
- Создайте метод `is_empty`, который возвращает True, если очередь пуста, и False в противном случае.
- Создайте экземпляр класса Queue с `enqueue_greater_than_prev=True`.
- Добавьте элементы: 5, 3 (не добавится), 7, 6 (не добавится), 10, 8 (не добавится).
- Выведите текущее состояние очереди.
- Вызовите `dequeue`, выведите удаленный элемент.
- Выведите обновленное состояние очереди.

Пример использования:

```

queue = Queue(enqueue_greater_than_prev=True)
queue.enqueue(5)
queue.enqueue(3)    # нет
queue.enqueue(7)
queue.enqueue(6)    # нет
queue.enqueue(10)
queue.enqueue(8)    # нет

print("Current Queue:", queue._luggage)    # [5,7,10]

dequeued_item = queue.dequeue()
print("Dequeued item:", dequeued_item)    # 5

print("Updated Queue:", queue._luggage)    # [7,10]

```

35. Написать программу на Python, которая создает класс `Queue` для представления структуры данных очереди с инкапсуляцией. Класс должен содержать методы `enqueue`, `dequeue` и `is_empty`, которые реализуют операции добавления элементов в очередь, удаления элементов из очереди и проверки пустоты очереди соответственно. Программа также должна создавать экземпляр класса `Queue`, добавлять элементы в очередь, удалять элементы из очереди и выводить информацию о состоянии очереди на экран.

Инструкции:

- (a) Создайте класс `Queue` с методом `__init__`, который инициализирует пустую очередь (список `_trunk`). Принимает параметр `enqueue_less_than_prev=False`. Если `True`, то элемент добавляется только если он строго меньше предыдущего добавленного элемента (первый — всегда).
- (b) Создайте метод `enqueue`, который добавляет элемент, только если `enqueue_less_than_prev=False` или очередь пуста или `element < последний_элемент`.
- (c) Создайте метод `dequeue`, который удаляет и возвращает первый элемент. Если очередь пуста — выбрасывает `IndexError("Очередь пуста")`.
- (d) Создайте метод `is_empty`, который возвращает `True`, если очередь пуста, и `False` в противном случае.
- (e) Создайте экземпляр класса `Queue` с `enqueue_less_than_prev=True`.
- (f) Добавьте элементы: 10, 15 (не добавится), 8, 9 (не добавится), 5, 7 (не добавится).
- (g) Выведите текущее состояние очереди.
- (h) Вызовите `dequeue`, выведите удаленный элемент.
- (i) Выведите обновленное состояние очереди.

Пример использования:

```
queue = Queue(enqueue_less_than_prev=True)
queue.enqueue(10)
queue.enqueue(15)    # нет
queue.enqueue(8)
queue.enqueue(9)     # нет
queue.enqueue(5)
queue.enqueue(7)     # нет

print("Current Queue:", queue._trunk)    # [10, 8, 5]

dequeued_item = queue.dequeue()
print("Dequeued item:", dequeued_item)   # 10

print("Updated Queue:", queue._trunk)    # [8, 5]
```

2.4 Семинар «Структуры данных (закрепление) и __new__» (2 часа)

При создании подкласса неизменяемого встроенного типа данных (например, `float`, `str`, `int`) возникает проблема: значение объекта устанавливается *в момент его создания*, и метод `__init__` вызывается уже *после* этого, когда изменить базовое значение невозможно.

Кроме того, конструктор родительского неизменяемого типа (например, `float.__new__()`) часто не принимает дополнительные аргументы так же гибко, как `object.__new__()`, что приводит к ошибкам.

Решение: Использовать метод `__new__` для инициализации объекта *в момент его создания*.

Листинг 63: Пример: Класс `Distance` с использованием `__new__`

```
class Distance(float):
    def __new__(cls, value, unit):
        # 1. Создаем новый экземпляр float с заданным значением
        instance = super().__new__(cls, value)
        # 2. Настраиваем экземпляр, добавляя изменяемый атрибут
        instance.unit = unit
        # 3. Возвращаем настроенный экземпляр
        return instance

# Использование:
d = Distance(10.5, "km")
print(d)          # 10.5
print(d.unit)     # km
d.unit = "m"      # Атрибут unit изменяем!
print(d.unit)     # m
```

В этом примере `__new__` выполняет три шага:

1. Создает новый экземпляр текущего класса `cls`, вызывая `super().__new__(cls, value)`. Это обращение к `float.__new__()`, который создает и инициализирует новый экземпляр `float`.
2. Настраивает новый экземпляр, добавляя к нему изменяемый атрибут `unit`.
3. Возвращает новый, настроенный экземпляр.

Теперь класс `Distance` работает корректно, позволяя хранить единицы измерения в изменяемом атрибуте `unit`.

Замечание: для упрощения мы не применяли свойство ООП *инкапсуляция* в примере.

2.4.1 Задача 1 (Singleton)

Реализуйте задание согласно своему варианту. Обратите внимание, что мы не реализуем логику работы сложных вещей, а только её имитируем во всех вариантах.

Замечание: Singleton – это антипаттерн, в production его использовать не стоит, но для учебных целей он хорош и, кроме того, знание его сущности обязательно для разработчика.

1. Написать программу на Python, которая создает класс `'DataBase'` с использованием метода `'__new__'` для реализации паттерна Singleton (один экземпляр). Программа должна принимать параметры при создании и выводить сообщение при подключении.

Инструкции:

- (a) Создайте класс `'DataBase'`.

- (b) Добавьте приватный атрибут класса `‘_instance’` и инициализируйте его значением `‘None’`.
- (c) Переопределите метод `‘__new__’`, чтобы он проверял, существует ли уже экземпляр. Если нет — создает новый с помощью `‘super().__new__(cls)’` и присваивает его `‘_instance’`. Возвращает `‘_instance’`.
- (d) Переопределите метод `‘__init__’`, принимающий `‘user’`, `‘psw’`, `‘port’`. Устанавливает эти атрибуты экземпляра, но только если они еще не были установлены (чтобы не перезаписывать при повторном "создании").
- (e) Добавьте метод `‘connect’`, который выводит сообщение "Подключение к БД: {user}, {psw}, {port}".
- (f) Добавьте метод `‘__del__’`, который выводит "Закрытие соединения с БД".
- (g) Добавьте метод `‘get_data’`, который возвращает строку "Данные получены".
- (h) Добавьте метод `‘set_data’`, который принимает `‘data’` и выводит "Данные {data} записаны".
- (i) Создайте два экземпляра `‘db1’` и `‘db2’` с разными параметрами.
- (j) Вызовите `‘connect’` для `‘db1’`, затем для `‘db2’`.
- (k) Выведите `‘id(db1)’` и `‘id(db2)’` — они должны совпадать.

Пример использования:

```
db1 = DataBase("admin", "secret", 5432)
db2 = DataBase("user", "12345", 3306) # Это тот же объект, что и db1!

db1.connect()
db2.connect() # Выведет те же параметры, что и db1

print("ID db1:", id(db1))
print("ID db2:", id(db2)) # ID будут одинаковыми
```

2. Написать программу на Python, которая создает класс `‘ConnectionManager’` с использованием метода `‘__new__’` для реализации паттерна Singleton. Программа должна принимать параметры `‘host’`, `‘username’`, `‘timeout’` при создании экземпляра.

Инструкции:

- (a) Создайте класс `‘ConnectionManager’`.
- (b) Добавьте приватный атрибут класса `‘_shared_instance’` и инициализируйте его значением `‘None’`.
- (c) Переопределите метод `‘__new__’`, чтобы он возвращал существующий экземпляр, если он есть, или создавал новый.
- (d) Переопределите метод `‘__init__’`, принимающий `‘host’`, `‘username’`, `‘timeout’`. Устанавливает атрибуты, только если они еще не заданы.
- (e) Добавьте метод `‘establish’`, который выводит "Соединение установлено с {host} под пользователем {username} (таймаут: {timeout})".
- (f) Добавьте метод `‘__del__’`, который выводит "Соединение разорвано".
- (g) Добавьте метод `‘fetch’`, который возвращает "Запрос выполнен".
- (h) Добавьте метод `‘commit’`, который принимает `‘transaction’` и выводит "Транзакция {transaction} зафиксирована".

- (i) Создайте два экземпляра 'cm1' и 'cm2' с разными параметрами.
- (j) Вызовите 'establish' для 'cm1', затем для 'cm2'.
- (k) Выведите 'cm1 is cm2' — должно быть 'True'.

Пример использования:

```
cm1 = ConnectionManager("localhost", "root", 30)
cm2 = ConnectionManager("remote.server", "guest", 60)

cm1.establish()
cm2.establish()    # Параметры будут от cm1

print("cm1 is cm2:", cm1 is cm2)    # True
```

3. Написать программу на Python, которая создает класс 'ConfigLoader' с использованием метода '__new__' для реализации паттерна Singleton. Программа должна принимать параметры 'config_file', 'env', 'debug' при создании экземпляра.

Инструкции:

- (a) Создайте класс 'ConfigLoader'.
- (b) Добавьте приватный атрибут класса '_instance_ref' и инициализируйте его значением 'None'.
- (c) Переопределите метод '__new__', чтобы он обеспечивал единственный экземпляр.
- (d) Переопределите метод '__init__', принимающий 'config_file', 'env', 'debug'. Устанавливает атрибуты, только если они еще не заданы.
- (e) Добавьте метод 'load', который выводит "Конфигурация загружена из '{config_file}' для среды '{env}' (debug={debug})".
- (f) Добавьте метод '__del__', который выводит "Конфигурация выгружена".
- (g) Добавьте метод 'get_setting', который принимает 'key' и возвращает "Значение для {key}".
- (h) Добавьте метод 'set_setting', который принимает 'key', 'value' и выводит "Настройка {key} установлена в {value}".
- (i) Создайте два экземпляра 'cfg1' и 'cfg2' с разными параметрами.
- (j) Вызовите 'load' для 'cfg1', затем для 'cfg2'.
- (k) Проверьте, что 'cfg1.debug == cfg2.debug' (должно быть 'True', если первый был создан с 'debug=True').

Пример использования:

```
cfg1 = ConfigLoader("app.yaml", "prod", True)
cfg2 = ConfigLoader("dev.yaml", "dev", False)

cfg1.load()
cfg2.load()    # Параметры будут от cfg1

print("Debug mode (cfg1):", cfg1.debug)
print("Debug mode (cfg2):", cfg2.debug)    # Будет True, как у cfg1
```

4. Написать программу на Python, которая создает класс 'Logger' с использованием метода '__new__' для реализации паттерна Singleton. Программа должна принимать параметры 'log_level', 'output_file', 'rotate' при создании экземпляра.

Инструкции:

- (a) Создайте класс 'Logger'.
- (b) Добавьте приватный атрибут класса '_the_logger' и инициализируйте его значением 'None'.
- (c) Переопределите метод '__new__', чтобы он возвращал единственный экземпляр.
- (d) Переопределите метод '__init__', принимающий 'log_level', 'output_file', 'rotate'. Устанавливает атрибуты, только если они еще не заданы.
- (e) Добавьте метод 'log', который принимает 'message' и выводит "[{log_level}] {message} -> {output_file}".
- (f) Добавьте метод '__del__', который выводит "Логгер остановлен".
- (g) Добавьте метод 'set_level', который принимает 'level' и устанавливает 'self.log_level = level'.
- (h) Добавьте метод 'flush', который выводит "Буфер логов сброшен".
- (i) Создайте два экземпляра 'log1' и 'log2' с разными параметрами.
- (j) Вызовите 'log' для 'log1', затем 'set_level("ERROR")' для 'log2'.
- (k) Вызовите 'log' для 'log1' снова — уровень должен измениться.

Пример использования:

```
log1 = Logger("INFO", "app.log", True)
log2 = Logger("DEBUG", "debug.log", False)

log1.log("Старт приложения")
log2.set_level("ERROR") # Меняет уровень для log2 тоже!
log1.log("Ошибка!") # Выведет [ERROR] Ошибка! -> app.log
```

5. Написать программу на Python, которая создает класс 'Cache' с использованием метода '__new__' для реализации паттерна Singleton. Программа должна принимать параметры 'max_size', 'ttl', 'strategy' при создании экземпляра.

Инструкции:

- (a) Создайте класс 'Cache'.
- (b) Добавьте приватный атрибут класса '_cache_instance' и инициализируйте его значением 'None'.
- (c) Переопределите метод '__new__', чтобы он обеспечивал единственный экземпляр.
- (d) Переопределите метод '__init__', принимающий 'max_size', 'ttl', 'strategy'. Устанавливает атрибуты, только если они еще не заданы.
- (e) Добавьте метод 'put', который принимает 'key', 'value' и выводит "Ключ '{key}' закеширован (стратегия: {strategy})".
- (f) Добавьте метод '__del__', который выводит "Кеш очищен".

- (g) Добавьте метод 'get', который принимает 'key' и возвращает "Значение для {key}".
- (h) Добавьте метод 'clear', который выводит "Кеш принудительно очищен".
- (i) Создайте два экземпляра 'cache1' и 'cache2' с разными параметрами.
- (j) Вызовите 'put' для 'cache1', затем 'clear' для 'cache2'.
- (k) Проверьте, что 'cache1.max_size == cache2.max_size'.

Пример использования:

```
cache1 = Cache(1000, 3600, "LRU")
cache2 = Cache(500, 1800, "FIFO")

cache1.put("user\_123", \{"name": "Alice"\})
cache2.clear() # Очищает кеш cache1 тоже

print("Max size:", cache1.max\_size) # 1000 (от первого вызова)
```

6. Написать программу на Python, которая создает класс 'SessionHandler' с использованием метода '__new__' для реализации паттерна Singleton. Программа должна принимать параметры 'session_id', 'timeout', 'secure' при создании экземпляра.

Инструкции:

- (a) Создайте класс 'SessionHandler'.
- (b) Добавьте приватный атрибут класса '_handler' и инициализируйте его значением 'None'.
- (c) Переопределите метод '__new__', чтобы он возвращал единственный экземпляр.
- (d) Переопределите метод '__init__', принимающий 'session_id', 'timeout', 'secure'. Устанавливает атрибуты, только если они еще не заданы.
- (e) Добавьте метод 'start', который выводит "Сессия {session_id} начата (timeout={timeout}, secure={secure})".
- (f) Добавьте метод '__del__', который выводит "Сессия завершена".
- (g) Добавьте метод 'get_session_data', который возвращает "Данные сессии".
- (h) Добавьте метод 'invalidate', который выводит "Сессия аннулирована".
- (i) Создайте два экземпляра 'sh1' и 'sh2' с разными параметрами.
- (j) Вызовите 'start' для 'sh1', затем 'invalidate' для 'sh2'.
- (k) Выведите 'sh1.session_id' и 'sh2.session_id' — они должны быть одинаковыми.

Пример использования:

```
sh1 = SessionHandler("SID-001", 1800, True)
sh2 = SessionHandler("SID-999", 600, False)

sh1.start()
sh2.invalidate() # Аннулирует сессию sh1

print("Session ID sh1:", sh1.session\_id) # SID-001
print("Session ID sh2:", sh2.session\_id) # SID-001
```

7. Написать программу на Python, которая создает класс 'ResourceManager' с использованием метода '__new__' для реализации паттерна Singleton. Программа должна принимать параметры 'resource_type', 'capacity', 'priority' при создании экземпляра.

Инструкции:

- (a) Создайте класс 'ResourceManager'.
- (b) Добавьте приватный атрибут класса '_manager' и инициализируйте его значением 'None'.
- (c) Переопределите метод '__new__', чтобы он возвращал единственный экземпляр.
- (d) Переопределите метод '__init__', принимающий 'resource_type', 'capacity', 'priority'. Устанавливает атрибуты, только если они еще не заданы.
- (e) Добавьте метод 'allocate', который выводит "Выделено {capacity} ресурсов типа {resource_type} (приоритет: {priority})".
- (f) Добавьте метод '__del__', который выводит "Освобождение ресурсов".
- (g) Добавьте метод 'status', который возвращает "Ресурсы доступны".
- (h) Добавьте метод 'release', который выводит "Ресурсы освобождены".
- (i) Создайте два экземпляра 'rm1' и 'rm2' с разными параметрами.
- (j) Вызовите 'allocate' для 'rm1', затем 'release' для 'rm2'.
- (k) Проверьте, что 'rm1.capacity == rm2.capacity'.

Пример использования:

```
rm1 = ResourceManager("CPU", 4, 1)
rm2 = ResourceManager("GPU", 2, 2)

rm1.allocate()
rm2.release() # Освобождает ресурсы rm1

print("Capacity:", rm1.capacity) # 4
```

8. Написать программу на Python, которая создает класс 'PrinterPool' с использованием метода '__new__' для реализации паттерна Singleton. Программа должна принимать параметры 'printer_id', 'speed', 'color' при создании экземпляра.

Инструкции:

- (a) Создайте класс 'PrinterPool'.
- (b) Добавьте приватный атрибут класса '_pool' и инициализируйте его значением 'None'.
- (c) Переопределите метод '__new__', чтобы он возвращал единственный экземпляр.
- (d) Переопределите метод '__init__', принимающий 'printer_id', 'speed', 'color'. Устанавливает атрибуты, только если они еще не заданы.
- (e) Добавьте метод 'print', который выводит "Печать документа на принтере {printer_id} (скорость: {speed}, цвет: {color})".
- (f) Добавьте метод '__del__', который выводит "Принтер выключен".

- (g) Добавьте метод `get_status`, который возвращает "Готов к печати".
- (h) Добавьте метод `add_job`, который принимает `job` и выводит "Добавлено задание: {job}".
- (i) Создайте два экземпляра `pp1` и `pp2` с разными параметрами.
- (j) Вызовите `print` для `pp1`, затем `add_job` для `pp2`.
- (k) Проверьте, что `pp1.speed == pp2.speed`.

Пример использования:

```
pp1 = PrinterPool("P100", 10, "Yes")
pp2 = PrinterPool("P200", 8, "No")

pp1.print()
pp2.add_job("Report.pdf") # Добавляет задание для pp1

print("Speed:", pp1.speed) # 10
```

9. Написать программу на Python, которая создает класс `NetworkInterface` с использованием метода `__new__` для реализации паттерна Singleton. Программа должна принимать параметры `interface_name`, `ip`, `mac` при создании экземпляра.

Инструкции:

- (a) Создайте класс `NetworkInterface`.
- (b) Добавьте приватный атрибут класса `__interface` и инициализируйте его значением `None`.
- (c) Переопределите метод `__new__`, чтобы он возвращал единственный экземпляр.
- (d) Переопределите метод `__init__`, принимающий `interface_name`, `ip`, `mac`. Устанавливает атрибуты, только если они еще не заданы.
- (e) Добавьте метод `connect`, который выводит "Подключение к сети через {interface_name} ({ip}, {mac})".
- (f) Добавьте метод `__del__`, который выводит "Отключение от сети".
- (g) Добавьте метод `ping`, который возвращает "Пинг успешен".
- (h) Добавьте метод `configure`, который принимает `new_ip` и выводит "IP изменен на {new_ip}".
- (i) Создайте два экземпляра `ni1` и `ni2` с разными параметрами.
- (j) Вызовите `connect` для `ni1`, затем `configure` для `ni2`.
- (k) Проверьте, что `ni1.ip == ni2.ip`.

Пример использования:

```
ni1 = NetworkInterface("eth0", "192.168.1.100", "AA:BB:CC:DD:EE:FF")
ni2 = NetworkInterface("wlan0", "192.168.1.101", "11:22:33:44:55:66")

ni1.connect()
ni2.configure("192.168.1.102") # Изменяет IP для ni1

print("IP:", ni1.ip) # 192.168.1.102
```

10. Написать программу на Python, которая создает класс 'FileManager' с использованием метода '__new__' для реализации паттерна Singleton. Программа должна принимать параметры 'path', 'mode', 'buffered' при создании экземпляра.

Инструкции:

- (a) Создайте класс 'FileManager'.
- (b) Добавьте приватный атрибут класса '_manager' и инициализируйте его значением 'None'.
- (c) Переопределите метод '__new__', чтобы он возвращал единственный экземпляр.
- (d) Переопределите метод '__init__', принимающий 'path', 'mode', 'buffered'. Устанавливает атрибуты, только если они еще не заданы.
- (e) Добавьте метод 'open', который выводит "Открытие файла '{path}' в режиме '{mode}' (буферизация: {buffered})".
- (f) Добавьте метод '__del__', который выводит "Файл закрыт".
- (g) Добавьте метод 'read', который возвращает "Данные прочитаны".
- (h) Добавьте метод 'write', который принимает 'data' и выводит "Данные '{data}' записаны".
- (i) Создайте два экземпляра 'fm1' и 'fm2' с разными параметрами.
- (j) Вызовите 'open' для 'fm1', затем 'write' для 'fm2'.
- (k) Проверьте, что 'fm1.mode == fm2.mode'.

Пример использования:

```
fm1 = FileManager("data.txt", "r", True)
fm2 = FileManager("log.txt", "w", False)

fm1.open()
fm2.write("Hello") # Записывает в файл fm1

print("Mode:", fm1.mode) # r
```

11. Написать программу на Python, которая создает класс 'DatabaseConnector' с использованием метода '__new__' для реализации паттерна Singleton. Программа должна принимать параметры 'dbname', 'host', 'port' при создании экземпляра.

Инструкции:

- (a) Создайте класс 'DatabaseConnector'.
- (b) Добавьте приватный атрибут класса '_connector' и инициализируйте его значением 'None'.
- (c) Переопределите метод '__new__', чтобы он возвращал единственный экземпляр.
- (d) Переопределите метод '__init__', принимающий 'dbname', 'host', 'port'. Устанавливает атрибуты, только если они еще не заданы.
- (e) Добавьте метод 'connect', который выводит "Подключение к базе данных '{dbname}' на {host}:{port}".
- (f) Добавьте метод '__del__', который выводит "Отключение от базы данных".

- (g) Добавьте метод 'query', который возвращает "Запрос выполнен".
- (h) Добавьте метод 'disconnect', который выводит "Разрыв соединения".
- (i) Создайте два экземпляра 'dc1' и 'dc2' с разными параметрами.
- (j) Вызовите 'connect' для 'dc1', затем 'disconnect' для 'dc2'.
- (k) Проверьте, что 'dc1.port == dc2.port'.

Пример использования:

```
dc1 = DatabaseConnector("users", "localhost", 5432)
dc2 = DatabaseConnector("products", "db.example.com", 5432)

dc1.connect()
dc2.disconnect() # Разрывает соединение dc1

print("Port:", dc1.port) # 5432
```

12. Написать программу на Python, которая создает класс 'MessageQueue' с использованием метода '__new__' для реализации паттерна Singleton. Программа должна принимать параметры 'queue_name', 'max_messages', 'timeout' при создании экземпляра.

Инструкции:

- (a) Создайте класс 'MessageQueue'.
- (b) Добавьте приватный атрибут класса '_queue' и инициализируйте его значением 'None'.
- (c) Переопределите метод '__new__', чтобы он возвращал единственный экземпляр.
- (d) Переопределите метод '__init__', принимающий 'queue_name', 'max_messages', 'timeout'. Устанавливает атрибуты, только если они еще не заданы.
- (e) Добавьте метод 'send', который принимает 'message' и выводит "Отправка сообщения '{message}' в очередь {queue_name}".
- (f) Добавьте метод '__del__', который выводит "Очередь закрыта".
- (g) Добавьте метод 'receive', который возвращает "Сообщение получено".
- (h) Добавьте метод 'clear', который выводит "Очередь очищена".
- (i) Создайте два экземпляра 'mq1' и 'mq2' с разными параметрами.
- (j) Вызовите 'send' для 'mq1', затем 'clear' для 'mq2'.
- (k) Проверьте, что 'mq1.max_messages == mq2.max_messages'.

Пример использования:

```
mq1 = MessageQueue("orders", 100, 30)
mq2 = MessageQueue("notifications", 50, 60)

mq1.send("New order")
mq2.clear() # Очищает очередь mq1

print("Max messages:", mq1.max_messages) # 100
```

13. Написать программу на Python, которая создает класс 'StorageDevice' с использованием метода '__new__' для реализации паттерна Singleton. Программа должна принимать параметры 'device_id', 'capacity', 'type' при создании экземпляра.

Инструкции:

- (a) Создайте класс 'StorageDevice'.
- (b) Добавьте приватный атрибут класса '_device' и инициализируйте его значением 'None'.
- (c) Переопределите метод '__new__', чтобы он возвращал единственный экземпляр.
- (d) Переопределите метод '__init__', принимающий 'device_id', 'capacity', 'type'. Устанавливает атрибуты, только если они еще не заданы.
- (e) Добавьте метод 'mount', который выводит "Подключение устройства {device_id} (тип: {type}, емкость: {capacity})".
- (f) Добавьте метод '__del__', который выводит "Отключение устройства".
- (g) Добавьте метод 'read', который возвращает "Чтение данных".
- (h) Добавьте метод 'write', который принимает 'data' и выводит "Запись данных '{data}'".
- (i) Создайте два экземпляра 'sd1' и 'sd2' с разными параметрами.
- (j) Вызовите 'mount' для 'sd1', затем 'write' для 'sd2'.
- (k) Проверьте, что 'sd1.capacity == sd2.capacity'.

Пример использования:

```
sd1 = StorageDevice("SSD-001", 512, "SSD")
sd2 = StorageDevice("HDD-002", 1024, "HDD")

sd1.mount()
sd2.write("File.txt") # Записывает в устройство sd1

print("Capacity:", sd1.capacity) # 512
```

14. Написать программу на Python, которая создает класс 'APIGateway' с использованием метода '__new__' для реализации паттерна Singleton. Программа должна принимать параметры 'api_url', 'token', 'version' при создании экземпляра.

Инструкции:

- (a) Создайте класс 'APIGateway'.
- (b) Добавьте приватный атрибут класса '_gateway' и инициализируйте его значением 'None'.
- (c) Переопределите метод '__new__', чтобы он возвращал единственный экземпляр.
- (d) Переопределите метод '__init__', принимающий 'api_url', 'token', 'version'. Устанавливает атрибуты, только если они еще не заданы.
- (e) Добавьте метод 'call', который принимает 'endpoint' и выводит "Вызов API {endpoint} на {api_url} (версия: {version})".
- (f) Добавьте метод '__del__', который выводит "API шлюз отключен".

- (g) Добавьте метод 'get', который возвращает "Данные получены".
- (h) Добавьте метод 'post', который принимает 'data' и выводит "Отправлено: {data}".
- (i) Создайте два экземпляра 'ag1' и 'ag2' с разными параметрами.
- (j) Вызовите 'call' для 'ag1', затем 'post' для 'ag2'.
- (k) Проверьте, что 'ag1.version == ag2.version'.

Пример использования:

```
ag1 = APiGateway("https://api.example.com", "abc123", "v1")
ag2 = APiGateway("https://api.test.com", "def456", "v2")

ag1.call("/users")
ag2.post("Hello") # Отправляет данные через ag1

print("Version:", ag1.version) # v2
```

15. Написать программу на Python, которая создает класс 'TaskScheduler' с использованием метода '__new__' для реализации паттерна Singleton. Программа должна принимать параметры 'scheduler_id', 'interval', 'enabled' при создании экземпляра.

Инструкции:

- (a) Создайте класс 'TaskScheduler'.
- (b) Добавьте приватный атрибут класса '_scheduler' и инициализируйте его значением 'None'.
- (c) Переопределите метод '__new__', чтобы он возвращал единственный экземпляр.
- (d) Переопределите метод '__init__', принимающий 'scheduler_id', 'interval', 'enabled'. Устанавливает атрибуты, только если они еще не заданы.
- (e) Добавьте метод 'start', который выводит "Запуск планировщика {scheduler_id} (интервал: {interval}, включен: {enabled})".
- (f) Добавьте метод '__del__', который выводит "Планировщик остановлен".
- (g) Добавьте метод 'schedule', который принимает 'task' и выводит "Запланирована задача: {task}".
- (h) Добавьте метод 'stop', который выводит "Остановка планировщика".
- (i) Создайте два экземпляра 'ts1' и 'ts2' с разными параметрами.
- (j) Вызовите 'start' для 'ts1', затем 'schedule' для 'ts2'.
- (k) Проверьте, что 'ts1.interval == ts2.interval'.

Пример использования:

```
ts1 = TaskScheduler("daily", 3600, True)
ts2 = TaskScheduler("hourly", 300, False)

ts1.start()
ts2.schedule("Backup") # Запланирована задача для ts1

print("Interval:", ts1.interval) # 3600
```

16. Написать программу на Python, которая создает класс 'ServiceMonitor' с использованием метода '__new__' для реализации паттерна Singleton. Программа должна принимать параметры 'service_name', 'check_interval', 'threshold' при создании экземпляра.

Инструкции:

- (a) Создайте класс 'ServiceMonitor'.
- (b) Добавьте приватный атрибут класса '_monitor' и инициализируйте его значением 'None'.
- (c) Переопределите метод '__new__', чтобы он возвращал единственный экземпляр.
- (d) Переопределите метод '__init__', принимающий 'service_name', 'check_interval', 'threshold'. Устанавливает атрибуты, только если они еще не заданы.
- (e) Добавьте метод 'start', который выводит "Мониторинг службы {service_name} запущен (интервал: {check_interval}, порог: {threshold})".
- (f) Добавьте метод '__del__', который выводит "Мониторинг остановлен".
- (g) Добавьте метод 'check', который возвращает "Проверка завершена".
- (h) Добавьте метод 'alert', который принимает 'message' и выводит "Алерт: {message}".
- (i) Создайте два экземпляра 'sm1' и 'sm2' с разными параметрами.
- (j) Вызовите 'start' для 'sm1', затем 'alert' для 'sm2'.
- (k) Проверьте, что 'sm1.check_interval == sm2.check_interval'.

Пример использования:

```
sm1 = ServiceMonitor("web", 60, 0.9)
sm2 = ServiceMonitor("db", 30, 0.8)

sm1.start()
sm2.alert("High load") # Алерт для sm1

print("Check interval:", sm1.check_interval) # 60
```

17. Написать программу на Python, которая создает класс 'EventBus' с использованием метода '__new__' для реализации паттерна Singleton. Программа должна принимать параметры 'bus_id', 'topic', 'max_listeners' при создании экземпляра.

Инструкции:

- (a) Создайте класс 'EventBus'.
- (b) Добавьте приватный атрибут класса '_bus' и инициализируйте его значением 'None'.
- (c) Переопределите метод '__new__', чтобы он возвращал единственный экземпляр.
- (d) Переопределите метод '__init__', принимающий 'bus_id', 'topic', 'max_listeners'. Устанавливает атрибуты, только если они еще не заданы.
- (e) Добавьте метод 'publish', который принимает 'event' и выводит "Опубликовано событие '{event}' в топик {topic}".
- (f) Добавьте метод '__del__', который выводит "Шина событий закрыта".

- (g) Добавьте метод 'subscribe', который принимает 'listener' и выводит "Подписан слушатель: {listener}".
- (h) Добавьте метод 'unsubscribe', который принимает 'listener' и выводит "Отписка слушателя: {listener}".
- (i) Создайте два экземпляра 'eb1' и 'eb2' с разными параметрами.
- (j) Вызовите 'publish' для 'eb1', затем 'subscribe' для 'eb2'.
- (k) Проверьте, что 'eb1.max_listeners == eb2.max_listeners'.

Пример использования:

```
eb1 = EventBus("main", "system", 10)
eb2 = EventBus("backup", "alerts", 5)

eb1.publish("Start")
eb2.subscribe("User") # Подписка на eb1

print("Max listeners:", eb1.max_listeners) # 10
```

18. Написать программу на Python, которая создает класс 'SignalProcessor' с использованием метода '__new__' для реализации паттерна Singleton. Программа должна принимать параметры 'processor_id', 'sample_rate', 'filter_type' при создании экземпляра.

Инструкции:

- (a) Создайте класс 'SignalProcessor'.
- (b) Добавьте приватный атрибут класса '_processor' и инициализируйте его значением 'None'.
- (c) Переопределите метод '__new__', чтобы он возвращал единственный экземпляр.
- (d) Переопределите метод '__init__', принимающий 'processor_id', 'sample_rate', 'filter_type'. Устанавливает атрибуты, только если они еще не заданы.
- (e) Добавьте метод 'process', который принимает 'signal' и выводит "Обработка сигнала '{signal}' (частота: {sample_rate}, фильтр: {filter_type})".
- (f) Добавьте метод '__del__', который выводит "Процессор сигналов остановлен".
- (g) Добавьте метод 'analyze', который возвращает "Анализ завершен".
- (h) Добавьте метод 'apply_filter', который принимает 'filter_params' и выводит "Применён фильтр с параметрами: {filter_params}".
- (i) Создайте два экземпляра 'sp1' и 'sp2' с разными параметрами.
- (j) Вызовите 'process' для 'sp1', затем 'apply_filter' для 'sp2'.
- (k) Проверьте, что 'sp1.sample_rate == sp2.sample_rate'.

Пример использования:

```
sp1 = SignalProcessor("audio", 44100, "lowpass")
sp2 = SignalProcessor("video", 30000, "bandpass")

sp1.process("sound")
sp2.apply_filter({"cutoff": 1000}) # Применяет фильтр для sp1

print("Sample rate:", sp1.sample_rate) # 44100
```

19. Написать программу на Python, которая создает класс 'DataPipeline' с использованием метода '__new__' для реализации паттерна Singleton. Программа должна принимать параметры 'pipeline_id', 'source', 'destination' при создании экземпляра.

Инструкции:

- (a) Создайте класс 'DataPipeline'.
- (b) Добавьте приватный атрибут класса '_pipeline' и инициализируйте его значением 'None'.
- (c) Переопределите метод '__new__', чтобы он возвращал единственный экземпляр.
- (d) Переопределите метод '__init__', принимающий 'pipeline_id', 'source', 'destination'. Устанавливает атрибуты, только если они еще не заданы.
- (e) Добавьте метод 'start', который выводит "Запуск потока данных {pipeline_id} ({source} → {destination})".
- (f) Добавьте метод '__del__', который выводит "Поток данных остановлен".
- (g) Добавьте метод 'transform', который возвращает "Трансформация завершена".
- (h) Добавьте метод 'transfer', который принимает 'data' и выводит "Передача данных '{data}'".
- (i) Создайте два экземпляра 'dp1' и 'dp2' с разными параметрами.
- (j) Вызовите 'start' для 'dp1', затем 'transfer' для 'dp2'.
- (k) Проверьте, что 'dp1.destination == dp2.destination'.

Пример использования:

```
dp1 = DataPipeline("etl", "db", "cloud")
dp2 = DataPipeline("backup", "local", "cloud")

dp1.start()
dp2.transfer("records") # Передача данных через dp1

print("Destination:", dp1.destination) # cloud
```

20. Написать программу на Python, которая создает класс 'SecurityGuard' с использованием метода '__new__' для реализации паттерна Singleton. Программа должна принимать параметры 'guard_id', 'access_level', 'rules' при создании экземпляра.

Инструкции:

- (a) Создайте класс 'SecurityGuard'.
- (b) Добавьте приватный атрибут класса '_guard' и инициализируйте его значением 'None'.
- (c) Переопределите метод '__new__', чтобы он возвращал единственный экземпляр.
- (d) Переопределите метод '__init__', принимающий 'guard_id', 'access_level', 'rules'. Устанавливает атрибуты, только если они еще не заданы.
- (e) Добавьте метод 'authorize', который принимает 'request' и выводит "Авторизация запроса '{request}' (уровень: {access_level})".

- (f) Добавьте метод `'__del__'`, который выводит "Система безопасности отключена".
- (g) Добавьте метод `'audit'`, который возвращает "Аудит завершен".
- (h) Добавьте метод `'block'`, который принимает `'entity'` и выводит "Блокировка сущности: {entity}".
- (i) Создайте два экземпляра `'sg1'` и `'sg2'` с разными параметрами.
- (j) Вызовите `'authorize'` для `'sg1'`, затем `'block'` для `'sg2'`.
- (k) Проверьте, что `'sg1.access_level == sg2.access_level'`.

Пример использования:

```
sg1 = SecurityGuard("main", "admin", ["rule1"])
sg2 = SecurityGuard("backup", "user", ["rule2"])

sg1.authorize("login")
sg2.block("malware")    # Блокировка для sg1

print("Access level:", sg1.access_level)    # admin
```

21. Написать программу на Python, которая создает класс `'SystemTray'` с использованием метода `'__new__'` для реализации паттерна Singleton. Программа должна принимать параметры `'tray_id'`, `'icon'`, `'tooltip'` при создании экземпляра.

Инструкции:

- (a) Создайте класс `'SystemTray'`.
- (b) Добавьте приватный атрибут класса `'_tray'` и инициализируйте его значением `'None'`.
- (c) Переопределите метод `'__new__'`, чтобы он возвращал единственный экземпляр.
- (d) Переопределите метод `'__init__'`, принимающий `'tray_id'`, `'icon'`, `'tooltip'`. Устанавливает атрибуты, только если они еще не заданы.
- (e) Добавьте метод `'show'`, который выводит "Показать значок {icon} в трее (подсказка: {tooltip})".
- (f) Добавьте метод `'__del__'`, который выводит "Значок скрыт".
- (g) Добавьте метод `'hide'`, который выводит "Скрыть значок".
- (h) Добавьте метод `'notify'`, который принимает `'message'` и выводит "Уведомление: {message}".
- (i) Создайте два экземпляра `'st1'` и `'st2'` с разными параметрами.
- (j) Вызовите `'show'` для `'st1'`, затем `'notify'` для `'st2'`.
- (k) Проверьте, что `'st1.icon == st2.icon'`.

Пример использования:

```
st1 = SystemTray("app", "app.ico", "My App")
st2 = SystemTray("tool", "tool.ico", "My Tool")

st1.show()
st2.notify("Update available")    # Уведомление для st1

print("Icon:", st1.icon)    # app.ico
```

22. Написать программу на Python, которая создает класс 'ApplicationLauncher' с использованием метода '__new__' для реализации паттерна Singleton. Программа должна принимать параметры 'launcher_id', 'apps', 'auto_start' при создании экземпляра.

Инструкции:

- (a) Создайте класс 'ApplicationLauncher'.
- (b) Добавьте приватный атрибут класса '_launcher' и инициализируйте его значением 'None'.
- (c) Переопределите метод '__new__', чтобы он возвращал единственный экземпляр.
- (d) Переопределите метод '__init__', принимающий 'launcher_id', 'apps', 'auto_start'. Устанавливает атрибуты, только если они еще не заданы.
- (e) Добавьте метод 'launch', который принимает 'app' и выводит "Запуск приложения '{app}' (автозапуск: {auto_start})".
- (f) Добавьте метод '__del__', который выводит "Запускатор остановлен".
- (g) Добавьте метод 'list_apps', который возвращает "Список приложений: {apps}".
- (h) Добавьте метод 'add_app', который принимает 'app' и выводит "Добавлено приложение: {app}".
- (i) Создайте два экземпляра 'al1' и 'al2' с разными параметрами.
- (j) Вызовите 'launch' для 'al1', затем 'add_app' для 'al2'.
- (k) Проверьте, что 'al1.apps == al2.apps'.

Пример использования:

```
al1 = ApplicationLauncher("main", ["browser", "editor"], True)
al2 = ApplicationLauncher("backup", ["backup", "sync"], False)

al1.launch("browser")
al2.add_app("calc") # Добавляет приложение в al1

print("Apps:", al1.apps) # ['browser', 'editor', 'calc']
```

23. Написать программу на Python, которая создает класс 'NetworkScanner' с использованием метода '__new__' для реализации паттерна Singleton. Программа должна принимать параметры 'scanner_id', 'target', 'timeout' при создании экземпляра.

Инструкции:

- (a) Создайте класс 'NetworkScanner'.
- (b) Добавьте приватный атрибут класса '_scanner' и инициализируйте его значением 'None'.
- (c) Переопределите метод '__new__', чтобы он возвращал единственный экземпляр.
- (d) Переопределите метод '__init__', принимающий 'scanner_id', 'target', 'timeout'. Устанавливает атрибуты, только если они еще не заданы.
- (e) Добавьте метод 'scan', который выводит "Сканирование сети {target} (таймаут: {timeout})".
- (f) Добавьте метод '__del__', который выводит "Сканер остановлен".

- (g) Добавьте метод 'report', который возвращает "Отчет о сканировании".
- (h) Добавьте метод 'ping', который принимает 'host' и выводит "Проверка доступности {host}".
- (i) Создайте два экземпляра 'ns1' и 'ns2' с разными параметрами.
- (j) Вызовите 'scan' для 'ns1', затем 'ping' для 'ns2'.
- (k) Проверьте, что 'ns1.timeout == ns2.timeout'.

Пример использования:

```
ns1 = NetworkScanner("fast", "192.168.1.0/24", 1)
ns2 = NetworkScanner("slow", "10.0.0.0/8", 5)

ns1.scan()
ns2.ping("192.168.1.1") # Проверка для ns1

print("Timeout:", ns1.timeout) # 1
```

24. Написать программу на Python, которая создает класс 'HealthChecker' с использованием метода '__new__' для реализации паттерна Singleton. Программа должна принимать параметры 'checker_id', 'services', 'frequency' при создании экземпляра.

Инструкции:

- (a) Создайте класс 'HealthChecker'.
- (b) Добавьте приватный атрибут класса '_checker' и инициализируйте его значением 'None'.
- (c) Переопределите метод '__new__', чтобы он возвращал единственный экземпляр.
- (d) Переопределите метод '__init__', принимающий 'checker_id', 'services', 'frequency'. Устанавливает атрибуты, только если они еще не заданы.
- (e) Добавьте метод 'check', который выводит "Проверка состояния сервисов {services} (частота: {frequency})".
- (f) Добавьте метод '__del__', который выводит "Проверка здоровья остановлена".
- (g) Добавьте метод 'status', который возвращает "Состояние: ОК".
- (h) Добавьте метод 'alert', который принимает 'service' и выводит "Алерт: {service} не отвечает".
- (i) Создайте два экземпляра 'h1' и 'h2' с разными параметрами.
- (j) Вызовите 'check' для 'h1', затем 'alert' для 'h2'.
- (k) Проверьте, что 'h1.frequency == h2.frequency'.

Пример использования:

```
h1 = HealthChecker("main", ["web", "db"], 60)
h2 = HealthChecker("backup", ["cache", "redis"], 30)

h1.check()
h2.alert("redis") # Алерт для h1

print("Frequency:", h1.frequency) # 60
```

25. Написать программу на Python, которая создает класс 'PerformanceMonitor' с использованием метода '__new__' для реализации паттерна Singleton. Программа должна принимать параметры 'monitor_id', 'metrics', 'interval' при создании экземпляра.

Инструкции:

- (a) Создайте класс 'PerformanceMonitor'.
- (b) Добавьте приватный атрибут класса '_monitor' и инициализируйте его значением 'None'.
- (c) Переопределите метод '__new__', чтобы он возвращал единственный экземпляр.
- (d) Переопределите метод '__init__', принимающий 'monitor_id', 'metrics', 'interval'. Устанавливает атрибуты, только если они еще не заданы.
- (e) Добавьте метод 'start', который выводит "Мониторинг производительности {monitor_id} запущен (метрики: {metrics}, интервал: {interval})".
- (f) Добавьте метод '__del__', который выводит "Мониторинг остановлен".
- (g) Добавьте метод 'collect', который возвращает "Сбор метрик завершен".
- (h) Добавьте метод 'report', который принимает 'data' и выводит "Отчет: {data}".
- (i) Создайте два экземпляра 'pm1' и 'pm2' с разными параметрами.
- (j) Вызовите 'start' для 'pm1', затем 'report' для 'pm2'.
- (k) Проверьте, что 'pm1.interval == pm2.interval'.

Пример использования:

```
pm1 = PerformanceMonitor("cpu", ["usage", "temp"], 10)
pm2 = PerformanceMonitor("memory", ["ram", "swap"], 5)

pm1.start()
pm2.report("High CPU load") # Отчет для pm1

print("Interval:", pm1.interval) # 10
```

26. Написать программу на Python, которая создает класс 'LogAggregator' с использованием метода '__new__' для реализации паттерна Singleton. Программа должна принимать параметры 'aggregator_id', 'sources', 'format' при создании экземпляра.

Инструкции:

- (a) Создайте класс 'LogAggregator'.
- (b) Добавьте приватный атрибут класса '_aggregator' и инициализируйте его значением 'None'.
- (c) Переопределите метод '__new__', чтобы он возвращал единственный экземпляр.
- (d) Переопределите метод '__init__', принимающий 'aggregator_id', 'sources', 'format'. Устанавливает атрибуты, только если они еще не заданы.
- (e) Добавьте метод 'aggregate', который выводит "Агрегация логов из {sources} (формат: {format})".
- (f) Добавьте метод '__del__', который выводит "Агрегатор остановлен".

- (g) Добавьте метод 'forward', который принимает 'logs' и выводит "Передача логов: {logs}".
- (h) Добавьте метод 'filter', который принимает 'criteria' и выводит "Фильтрация по критерию: {criteria}".
- (i) Создайте два экземпляра 'la1' и 'la2' с разными параметрами.
- (j) Вызовите 'aggregate' для 'la1', затем 'forward' для 'la2'.
- (k) Проверьте, что 'la1.format == la2.format'.

Пример использования:

```
la1 = LogAggregator("main", ["app", "db"], "json")
la2 = LogAggregator("backup", ["web", "api"], "text")

la1.aggregate()
la2.forward("error logs") # Передача для la1

print("Format:", la1.format) # json
```

27. Написать программу на Python, которая создает класс 'ResourceTracker' с использованием метода '__new__' для реализации паттерна Singleton. Программа должна принимать параметры 'tracker_id', 'resources', 'threshold' при создании экземпляра.

Инструкции:

- (a) Создайте класс 'ResourceTracker'.
- (b) Добавьте приватный атрибут класса '_tracker' и инициализируйте его значением 'None'.
- (c) Переопределите метод '__new__', чтобы он возвращал единственный экземпляр.
- (d) Переопределите метод '__init__', принимающий 'tracker_id', 'resources', 'threshold'. Устанавливает атрибуты, только если они еще не заданы.
- (e) Добавьте метод 'track', который выводит "Отслеживание ресурсов {resources} (порог: {threshold})".
- (f) Добавьте метод '__del__', который выводит "Отслеживание остановлено".
- (g) Добавьте метод 'update', который принимает 'data' и выводит "Обновление данных: {data}".
- (h) Добавьте метод 'alarm', который принимает 'resource' и выводит "Авария: {resource} исчерпан".
- (i) Создайте два экземпляра 'rt1' и 'rt2' с разными параметрами.
- (j) Вызовите 'track' для 'rt1', затем 'alarm' для 'rt2'.
- (k) Проверьте, что 'rt1.threshold == rt2.threshold'.

Пример использования:

```
rt1 = ResourceTracker("cpu", ["cores", "freq"], 0.8)
rt2 = ResourceTracker("memory", ["ram", "swap"], 0.9)

rt1.track()
rt2.alarm("ram") # Авария для rt1

print("Threshold:", rt1.threshold) # 0.8
```

28. Написать программу на Python, которая создает класс 'NotificationCenter' с использованием метода '__new__' для реализации паттерна Singleton. Программа должна принимать параметры 'center_id', 'channels', 'priority' при создании экземпляра.

Инструкции:

- (a) Создайте класс 'NotificationCenter'.
- (b) Добавьте приватный атрибут класса '_center' и инициализируйте его значением 'None'.
- (c) Переопределите метод '__new__', чтобы он возвращал единственный экземпляр.
- (d) Переопределите метод '__init__', принимающий 'center_id', 'channels', 'priority'. Устанавливает атрибуты, только если они еще не заданы.
- (e) Добавьте метод 'notify', который принимает 'message' и выводит "Уведомление: {message} (каналы: {channels}, приоритет: {priority})".
- (f) Добавьте метод '__del__', который выводит "Центр уведомлений остановлен".
- (g) Добавьте метод 'subscribe', который принимает 'channel' и выводит "Подписан на канал: {channel}".
- (h) Добавьте метод 'unsubscribe', который принимает 'channel' и выводит "Отписка от канала: {channel}".
- (i) Создайте два экземпляра 'nc1' и 'nc2' с разными параметрами.
- (j) Вызовите 'notify' для 'nc1', затем 'subscribe' для 'nc2'.
- (k) Проверьте, что 'nc1.priority == nc2.priority'.

Пример использования:

```
nc1 = NotificationCenter("main", ["email", "push"], 1)
nc2 = NotificationCenter("backup", ["sms", "phone"], 2)

nc1.notify("Update ready")
nc2.subscribe("email") # Подписка на nc1

print("Priority:", nc1.priority) # 1
```

29. Написать программу на Python, которая создает класс 'ConfigurationManager' с использованием метода '__new__' для реализации паттерна Singleton. Программа должна принимать параметры 'manager_id', 'config_files', 'reload_on_change' при создании экземпляра.

Инструкции:

- (a) Создайте класс 'ConfigurationManager'.
- (b) Добавьте приватный атрибут класса '_manager' и инициализируйте его значением 'None'.
- (c) Переопределите метод '__new__', чтобы он возвращал единственный экземпляр.
- (d) Переопределите метод '__init__', принимающий 'manager_id', 'config_files', 'reload_on_change'. Устанавливает атрибуты, только если они еще не заданы.
- (e) Добавьте метод 'load', который выводит "Загрузка конфигураций из {config_files} (перезагрузка при изменении: {reload_on_change})".

- (f) Добавьте метод `__del__`, который выводит "Конфигурации выгружены".
- (g) Добавьте метод `get`, который принимает `key` и возвращает "Значение для {key}".
- (h) Добавьте метод `set`, который принимает `key`, `value` и выводит "Настройка {key} установлена в {value}".
- (i) Создайте два экземпляра `cm1` и `cm2` с разными параметрами.
- (j) Вызовите `load` для `cm1`, затем `set` для `cm2`.
- (k) Проверьте, что `cm1.reload_on_change == cm2.reload_on_change`.

Пример использования:

```
cm1 = ConfigurationManager("app", ["config.yaml"], True)
cm2 = ConfigurationManager("test", ["test.conf"], False)

cm1.load()
cm2.set("debug", True) # Установка для cm1

print("Reload on change:", cm1.reload_on_change) # True
```

30. Написать программу на Python, которая создает класс `JobScheduler` с использованием метода `__new__` для реализации паттерна Singleton. Программа должна принимать параметры `scheduler_id`, `jobs`, `time_zone` при создании экземпляра.

Инструкции:

- (a) Создайте класс `JobScheduler`.
- (b) Добавьте приватный атрибут класса `_scheduler` и инициализируйте его значением `None`.
- (c) Переопределите метод `__new__`, чтобы он возвращал единственный экземпляр.
- (d) Переопределите метод `__init__`, принимающий `scheduler_id`, `jobs`, `time_zone`. Устанавливает атрибуты, только если они еще не заданы.
- (e) Добавьте метод `schedule`, который выводит "Запланированы задания {jobs} (часовой пояс: {time_zone})".
- (f) Добавьте метод `__del__`, который выводит "Планировщик остановлен".
- (g) Добавьте метод `run`, который возвращает "Выполнение заданий".
- (h) Добавьте метод `cancel`, который принимает `job` и выводит "Отмена задания: {job}".
- (i) Создайте два экземпляра `js1` и `js2` с разными параметрами.
- (j) Вызовите `schedule` для `js1`, затем `cancel` для `js2`.
- (k) Проверьте, что `js1.time_zone == js2.time_zone`.

Пример использования:

```
js1 = JobScheduler("daily", ["backup", "cleanup"], "UTC")
js2 = JobScheduler("weekly", ["report", "archive"], "Europe/Moscow")

js1.schedule()
js2.cancel("report") # Отмена для js1

print("Time zone:", js1.time_zone) # UTC
```

31. Написать программу на Python, которая создает класс 'AnalyticsEngine' с использованием метода '__new__' для реализации паттерна Singleton. Программа должна принимать параметры 'engine_id', 'datasets', 'model' при создании экземпляра.

Инструкции:

- (a) Создайте класс 'AnalyticsEngine'.
- (b) Добавьте приватный атрибут класса '_engine' и инициализируйте его значением 'None'.
- (c) Переопределите метод '__new__', чтобы он возвращал единственный экземпляр.
- (d) Переопределите метод '__init__', принимающий 'engine_id', 'datasets', 'model'. Устанавливает атрибуты, только если они еще не заданы.
- (e) Добавьте метод 'analyze', который выводит "Анализ данных {datasets} с моделью {model}".
- (f) Добавьте метод '__del__', который выводит "Аналитический движок остановлен".
- (g) Добавьте метод 'train', который возвращает "Обучение модели завершено".
- (h) Добавьте метод 'predict', который принимает 'data' и выводит "Прогноз на основе данных: {data}".
- (i) Создайте два экземпляра 'ae1' и 'ae2' с разными параметрами.
- (j) Вызовите 'analyze' для 'ae1', затем 'predict' для 'ae2'.
- (k) Проверьте, что 'ae1.model == ae2.model'.

Пример использования:

```
ae1 = AnalyticsEngine("sales", ["orders", "customers"], "linear")
ae2 = AnalyticsEngine("marketing", ["ads", "clicks"], "neural")

ae1.analyze()
ae2.predict("next month") # Прогноз для ae1

print("Model:", ae1.model) # linear
```

32. Написать программу на Python, которая создает класс 'AuditTrail' с использованием метода '__new__' для реализации паттерна Singleton. Программа должна принимать параметры 'trail_id', 'events', 'retention' при создании экземпляра.

Инструкции:

- (a) Создайте класс 'AuditTrail'.
- (b) Добавьте приватный атрибут класса '_trail' и инициализируйте его значением 'None'.
- (c) Переопределите метод '__new__', чтобы он возвращал единственный экземпляр.
- (d) Переопределите метод '__init__', принимающий 'trail_id', 'events', 'retention'. Устанавливает атрибуты, только если они еще не заданы.
- (e) Добавьте метод 'log', который принимает 'action' и выводит "Запись действия '{action}' в журнал (события: {events}, срок хранения: {retention})".

- (f) Добавьте метод `__del__`, который выводит "Журнал аудита закрыт".
- (g) Добавьте метод `search`, который принимает `query` и возвращает "Результаты поиска: {query}".
- (h) Добавьте метод `purge`, который выводит "Очистка журнала".
- (i) Создайте два экземпляра `at1` и `at2` с разными параметрами.
- (j) Вызовите `log` для `at1`, затем `search` для `at2`.
- (k) Проверьте, что `at1.retention == at2.retention`.

Пример использования:

```
at1 = AuditTrail("security", ["login", "logout"], 365)
at2 = AuditTrail("operations", ["start", "stop"], 90)

at1.log("User login")
at2.search("logout") # Поиск в at1

print("Retention:", at1.retention) # 365
```

33. Написать программу на Python, которая создает класс `ContentFilter` с использованием метода `__new__` для реализации паттерна Singleton. Программа должна принимать параметры `filter_id`, `rules`, `strict` при создании экземпляра.

Инструкции:

- (a) Создайте класс `ContentFilter`.
- (b) Добавьте приватный атрибут класса `_filter` и инициализируйте его значением `None`.
- (c) Переопределите метод `__new__`, чтобы он возвращал единственный экземпляр.
- (d) Переопределите метод `__init__`, принимающий `filter_id`, `rules`, `strict`. Устанавливает атрибуты, только если они еще не заданы.
- (e) Добавьте метод `filter`, который принимает `content` и выводит "Фильтрация контента '{content}' (правила: {rules}, строгий режим: {strict})".
- (f) Добавьте метод `__del__`, который выводит "Фильтр отключен".
- (g) Добавьте метод `get_rules`, который возвращает "Правила: {rules}".
- (h) Добавьте метод `add_rule`, который принимает `rule` и выводит "Добавлено правило: {rule}".
- (i) Создайте два экземпляра `cf1` и `cf2` с разными параметрами.
- (j) Вызовите `filter` для `cf1`, затем `add_rule` для `cf2`.
- (k) Проверьте, что `cf1.strict == cf2.strict`.

Пример использования:

```
cf1 = ContentFilter("main", ["bad", "spam"], True)
cf2 = ContentFilter("backup", ["offensive", "inappropriate"], False)

cf1.filter("This is spam")
cf2.add_rule("hate") # Добавление правила для cf1

print("Strict:", cf1.strict) # True
```

34. Написать программу на Python, которая создает класс 'RateLimiter' с использованием метода '__new__' для реализации паттерна Singleton. Программа должна принимать параметры 'limiter_id', 'rate', 'burst' при создании экземпляра.

Инструкции:

- (a) Создайте класс 'RateLimiter'.
- (b) Добавьте приватный атрибут класса '_limiter' и инициализируйте его значением 'None'.
- (c) Переопределите метод '__new__', чтобы он возвращал единственный экземпляр.
- (d) Переопределите метод '__init__', принимающий 'limiter_id', 'rate', 'burst'. Устанавливает атрибуты, только если они еще не заданы.
- (e) Добавьте метод 'limit', который принимает 'request' и выводит "Ограничение запроса '{request}' (скорость: {rate}, burst: {burst})".
- (f) Добавьте метод '__del__', который выводит "Ограничитель отключен".
- (g) Добавьте метод 'allow', который возвращает "Запрос разрешен".
- (h) Добавьте метод 'deny', который принимает 'reason' и выводит "Запрос отклонен: {reason}".
- (i) Создайте два экземпляра 'rl1' и 'rl2' с разными параметрами.
- (j) Вызовите 'limit' для 'rl1', затем 'deny' для 'rl2'.
- (k) Проверьте, что 'rl1.rate == rl2.rate'.

Пример использования:

```
rl1 = RateLimiter("api", 10, 5)
rl2 = RateLimiter("web", 5, 3)

rl1.limit("GET /users")
rl2.deny("Too many requests") # Отклонение для rl1

print("Rate:", rl1.rate) # 10
```

35. Написать программу на Python, которая создает класс 'CacheManager' с использованием метода '__new__' для реализации паттерна Singleton. Программа должна принимать параметры 'manager_id', 'cache_size', 'eviction_policy' при создании экземпляра.

Инструкции:

- (a) Создайте класс 'CacheManager'.
- (b) Добавьте приватный атрибут класса '_manager' и инициализируйте его значением 'None'.
- (c) Переопределите метод '__new__', чтобы он возвращал единственный экземпляр.
- (d) Переопределите метод '__init__', принимающий 'manager_id', 'cache_size', 'eviction_policy'. Устанавливает атрибуты, только если они еще не заданы.
- (e) Добавьте метод 'put', который принимает 'key', 'value' и выводит "Кэширование ключа '{key}' (размер: {cache_size}, политика: {eviction_policy})".

- (f) Добавьте метод `__del__`, который выводит "Кэш очищен".
- (g) Добавьте метод `get`, который принимает `key` и возвращает "Значение для {key}".
- (h) Добавьте метод `evict`, который выводит "Освобождение места в кэше".
- (i) Создайте два экземпляра `cm1` и `cm2` с разными параметрами.
- (j) Вызовите `put` для `cm1`, затем `evict` для `cm2`.
- (k) Проверьте, что `cm1.cache_size == cm2.cache_size`.

Пример использования:

```
cm1 = CacheManager("main", 1000, "LRU")
cm2 = CacheManager("backup", 500, "FIFO")

cm1.put("user\_123", \{"name": "Alice"\})
cm2.evict() # Освобождение для cm1

print("Cache size:", cm1.cache\_size) # 1000
```

2.4.2 Задача 2 (ограничение количества экземпляров)

1. Написать программу на Python, которая создает класс `LimitedInstances` с использованием метода `__new__` для ограничения количества создаваемых экземпляров до 5.

Инструкции:

- (a) Создайте класс `LimitedInstances`.
- (b) Добавьте атрибут класса `_instances` и инициализируйте его пустым списком.
- (c) Добавьте атрибут класса `_limit` и инициализируйте его значением 5.
- (d) Переопределите метод `__new__`. Если `len(_instances) >= _limit`, выбросьте `RuntimeError("Превышен лимит объектов: 5")`. Иначе, создайте экземпляр с помощью `super().__new__(cls)`, добавьте его в `_instances` и верните.
- (e) Переопределите метод `__del__`, чтобы он удалял `self` из `_instances` при уничтожении объекта.
- (f) Переопределите метод `__init__`, который принимает `name` и устанавливает `self.name = name`.
- (g) Создайте 5 экземпляров класса.
- (h) Попробуйте создать 6-й экземпляр - должно возникнуть исключение `RuntimeError`.
- (i) Удалите один из первых 5 экземпляров (например, `del obj1`).
- (j) Создайте 6-й экземпляр - теперь это должно сработать.

Пример использования:

```
# Создаем 5 объектов
objs = [LimitedInstances(f"Obj{i}") for i in range(1, 6)]

# Попытка создать 6-й - вызовет ошибку
try:
    obj6 = LimitedInstances("Obj6")
except RuntimeError as e:
    print(e)
```

```
# Удаляем один объект
del objs[0]

# Теперь можно создать 6-й
obj6 = LimitedInstances("Obj6")
print("Успешно создан 6-й объект:", obj6.name)
```

2. Написать программу на Python, которая создает класс 'BoundedObjects' с использованием метода '__new__' для ограничения количества создаваемых экземпляров до 3.

Инструкции:

- (a) Создайте класс 'BoundedObjects'.
- (b) Добавьте атрибут класса '_pool' и инициализируйте его пустым списком.
- (c) Добавьте атрибут класса 'MAX_OBJECTS' и инициализируйте его значением 3.
- (d) Переопределите метод '__new__'. Если 'len(_pool) >= MAX_OBJECTS', выбросьте 'RuntimeError("Максимум 3 объекта!")'. Иначе, создайте экземпляр, добавьте в '_pool', верните.
- (e) Переопределите метод '__del__', чтобы он удалял 'self' из '_pool'.
- (f) Переопределите метод '__init__', который принимает 'id' и устанавливает 'self.object_id = id'.
- (g) Создайте 3 экземпляра.
- (h) Попробуйте создать 4-й - поймайте и выведите исключение.
- (i) Удалите один экземпляр.
- (j) Создайте 4-й экземпляр - должно сработать.

Пример использования:

```
# Создаем 3 объекта
obj1 = BoundedObjects(1)
obj2 = BoundedObjects(2)
obj3 = BoundedObjects(3)

# Попытка создать 4-й
try:
    obj4 = BoundedObjects(4)
except RuntimeError as e:
    print("Ошибка:", e)

# Удаляем один
del obj1

# Создаем 4-й - успешно
obj4 = BoundedObjects(4)
print("ID нового объекта:", obj4.object_id)
```

3. Написать программу на Python, которая создает класс 'ResourcePool' с использованием метода '__new__' для ограничения количества создаваемых экземпляров до 10.

Инструкции:

- (a) Создайте класс 'ResourcePool'.

- (b) Добавьте атрибут класса `‘_allocated’` и инициализируйте его пустым списком.
- (c) Добавьте атрибут класса `‘CAPACITY’` и инициализируйте его значением 10.
- (d) Переопределите метод `‘__new__’`. Если `‘len(_allocated) >= CAPACITY’`, выбросьте `‘RuntimeError("Ресурсы исчерпаны!")’`. Иначе, создайте экземпляр, добавьте в `‘_allocated’`, верните.
- (e) Переопределите метод `‘__del__’`, чтобы он удалял `‘self’` из `‘_allocated’`.
- (f) Переопределите метод `‘__init__’`, который принимает `‘resource_type’` и устанавливает `‘self.type = resource_type’`.
- (g) Создайте 10 экземпляров.
- (h) Попробуйте создать 11-й - поймайте и выведите исключение.
- (i) Удалите два экземпляра.
- (j) Создайте 11-й и 12-й экземпляры - должно сработать.

Пример использования:

```
# Создаем 10 объектов
resources = [ResourcePool(f"Type{i}") for i in range(10)]

# Попытка создать 11-й
try:
    r11 = ResourcePool("Type11")
except RuntimeError as e:
    print(e)

# Удаляем два
del resources[0], resources[1]

# Создаем 11-й и 12-й - успешно
r11 = ResourcePool("Type11")
r12 = ResourcePool("Type12")
print("Созданы:", r11.type, r12.type)
```

4. Написать программу на Python, которая создает класс `‘CarFleet’` с использованием метода `‘__new__’` для ограничения количества создаваемых экземпляров до 7.

Инструкции:

- (a) Создайте класс `‘CarFleet’`.
- (b) Добавьте атрибут класса `‘_cars’` и инициализируйте его пустым списком.
- (c) Добавьте атрибут класса `‘FLEET_SIZE’` и инициализируйте его значением 7.
- (d) Переопределите метод `‘__new__’`. Если `‘len(_cars) >= FLEET_SIZE’`, выбросьте `‘RuntimeError("Автопарк переполнен!")’`. Иначе, создайте экземпляр, добавьте в `‘_cars’`, верните.
- (e) Переопределите метод `‘__del__’`, чтобы он удалял `‘self’` из `‘_cars’`.
- (f) Переопределите метод `‘__init__’`, который принимает `‘model’` и устанавливает `‘self.model = model’`.
- (g) Создайте 7 экземпляров.
- (h) Попробуйте создать 8-й - поймайте и выведите исключение.
- (i) Удалите три экземпляра.

- (j) Создайте 8-й, 9-й и 10-й экземпляры - должно сработать.

Пример использования:

```
# Создаем 7 машин
fleet = [CarFleet(f"Model{i}") for i in range(7)]

# Попытка создать 8-ю
try:
    car8 = CarFleet("Model8")
except RuntimeError as e:
    print("Ошибка:", e)

# Удаляем три
del fleet[0], fleet[1], fleet[2]

# Создаем 8-ю, 9-ю, 10-ю - успешно
car8 = CarFleet("Model8")
car9 = CarFleet("Model9")
car10 = CarFleet("Model10")
print("Новые модели:", car8.model, car9.model, car10.model)
```

5. Написать программу на Python, которая создает класс 'StudentGroup' с использованием метода '__new__' для ограничения количества создаваемых экземпляров до 30.

Инструкции:

- Создайте класс 'StudentGroup'.
- Добавьте атрибут класса '_students' и инициализируйте его пустым списком.
- Добавьте атрибут класса 'GROUP_MAX' и инициализируйте его значением 30.
- Переопределите метод '__new__'. Если 'len(_students) >= GROUP_MAX', выбросьте 'RuntimeError("Группа заполнена!")'. Иначе, создайте экземпляр, добавьте в '_students', верните.
- Переопределите метод '__del__', чтобы он удалял 'self' из '_students'.
- Переопределите метод '__init__', который принимает 'student_name' и устанавливает 'self.name = student_name'.
- Создайте 30 экземпляров.
- Попытайтесь создать 31-й - поймите и выведите исключение.
- Удалите пять экземпляров.
- Создайте 31-й, 32-й, 33-й, 34-й, 35-й экземпляры - должно сработать.

Пример использования:

```
# Создаем 30 студентов
students = [StudentGroup(f"Student{i}") for i in range(30)]

# Попытка создать 31-го
try:
    s31 = StudentGroup("Alice")
except RuntimeError as e:
    print("Ошибка:", e)

# Удаляем пять
for i in range(5):
```



```
del students[0]

# Создаем 31-го, 32-го, 33-го, 34-го, 35-го - успешно
new_students = [StudentGroup(f"New{i}") for i in range(31, 36)]
for s in new_students:
    print("Добавлен:", s.name)
```

6. Написать программу на Python, которая создает класс 'TaskQueue' с использованием метода '__new__' для ограничения количества создаваемых экземпляров до 100.

Инструкции:

- Создайте класс 'TaskQueue'.
- Добавьте атрибут класса '_tasks' и инициализируйте его пустым списком.
- Добавьте атрибут класса 'QUEUE_LIMIT' и инициализируйте его значением 100.
- Переопределите метод '__new__'. Если 'len(_tasks) >= QUEUE_LIMIT', выбросьте 'RuntimeError("Очередь задач переполнена!")'. Иначе, создайте экземпляр, добавьте в '_tasks', верните.
- Переопределите метод '__del__', чтобы он удалял 'self' из '_tasks'.
- Переопределите метод '__init__', который принимает 'task_name' и устанавливает 'self.task = task_name'.
- Создайте 100 экземпляров.
- Попытайтесь создать 101-й - поймите и выведите исключение.
- Удалите десять экземпляров.
- Создайте 101-й, 102-й, ..., 110-й экземпляры - должно сработать.

Пример использования:

```
# Создаем 100 задач
tasks = [TaskQueue(f"Task{i}") for i in range(100)]

# Попытка создать 101-ю
try:
    t101 = TaskQueue("FinalTask")
except RuntimeError as e:
    print("Ошибка:", e)

# Удаляем 10 задач
for i in range(10):
    del tasks[0]

# Создаем 101-ю, 102-ю, ..., 110-ю - успешно
new_tasks = [TaskQueue(f"NewTask{i}") for i in range(101, 111)]
for t in new_tasks:
    print("Добавлена задача:", t.task)
```

7. Написать программу на Python, которая создает класс 'ConnectionPool' с использованием метода '__new__' для ограничения количества создаваемых экземпляров до 8.

Инструкции:

- Создайте класс 'ConnectionPool'.

- (b) Добавьте атрибут класса `'_connections'` и инициализируйте его пустым списком.
- (c) Добавьте атрибут класса `'POOL_SIZE'` и инициализируйте его значением 8.
- (d) Переопределите метод `'__new__'`. Если `'len(_connections) >= POOL_SIZE'`, выбросьте `'RuntimeError("Пул соединений полон!")'`. Иначе, создайте экземпляр, добавьте в `'_connections'`, верните.
- (e) Переопределите метод `'__del__'`, чтобы он удалял `'self'` из `'_connections'`.
- (f) Переопределите метод `'__init__'`, который принимает `'connection_id'` и устанавливает `'self.id = connection_id'`.
- (g) Создайте 8 экземпляров.
- (h) Попробуйте создать 9-й - поймайте и выведите исключение.
- (i) Удалите четыре экземпляра.
- (j) Создайте 9-й, 10-й, 11-й, 12-й экземпляры - должно сработать.

Пример использования:

```
# Создаем 8 соединений
pool = [ConnectionPool(f"Conn{i}") for i in range(8)]

# Попробуем создать 9-е
try:
    conn9 = ConnectionPool("Conn9")
except RuntimeError as e:
    print("Ошибка:", e)

# Удаляем 4
del pool[0], pool[1], pool[2], pool[3]

# Создаем 9-е, 10-е, 11-е, 12-е - успешно
new_conns = [ConnectionPool(f"Conn{i}") for i in range(9, 13)]
for c in new_conns:
    print("Создано соединение:", c.id)
```

8. Написать программу на Python, которая создает класс `'DeviceManager'` с использованием метода `'__new__'` для ограничения количества создаваемых экземпляров до 15.

Инструкции:

- (a) Создайте класс `'DeviceManager'`.
- (b) Добавьте атрибут класса `'_devices'` и инициализируйте его пустым списком.
- (c) Добавьте атрибут класса `'MANAGER_LIMIT'` и инициализируйте его значением 15.
- (d) Переопределите метод `'__new__'`. Если `'len(_devices) >= MANAGER_LIMIT'`, выбросьте `'RuntimeError("Менеджер устройств перегружен!")'`. Иначе, создайте экземпляр, добавьте в `'_devices'`, верните.
- (e) Переопределите метод `'__del__'`, чтобы он удалял `'self'` из `'_devices'`.
- (f) Переопределите метод `'__init__'`, который принимает `'device_name'` и устанавливает `'self.device = device_name'`.
- (g) Создайте 15 экземпляров.
- (h) Попробуйте создать 16-й - поймайте и выведите исключение.

- (i) Удалите семь экземпляров.
- (j) Создайте 16-й, 17-й, ..., 22-й экземпляры - должно сработать.

Пример использования:

```
# Создаем 15 устройств
devices = [DeviceManager(f"Device{i}") for i in range(15)]

# Попытка создать 16-е
try:
    d16 = DeviceManager("NewDevice")
except RuntimeError as e:
    print("Ошибка:", e)

# Удаляем 7
for i in range(7):
    del devices[0]

# Создаем 16-е, 17-е, ..., 22-е - успешно
new_devices = [DeviceManager(f"Device{i}") for i in range(16, 23)]
for d in new_devices:
    print("Добавлено устройство:", d.device)
```

9. Написать программу на Python, которая создает класс 'SessionPool' с использованием метода '__new__' для ограничения количества создаваемых экземпляров до 6.

Инструкции:

- (a) Создайте класс 'SessionPool'.
- (b) Добавьте атрибут класса '_sessions' и инициализируйте его пустым списком.
- (c) Добавьте атрибут класса 'SESSION_LIMIT' и инициализируйте его значением 6.
- (d) Переопределите метод '__new__'. Если 'len(_sessions) >= SESSION_LIMIT', выбросьте 'RuntimeError("Пул сессий исчерпан!")'. Иначе, создайте экземпляр, добавьте в '_sessions', верните.
- (e) Переопределите метод '__del__', чтобы он удалял 'self' из '_sessions'.
- (f) Переопределите метод '__init__', который принимает 'session_token' и устанавливает 'self.token = session_token'.
- (g) Создайте 6 экземпляров.
- (h) Попробуйте создать 7-й - поймайте и выведите исключение.
- (i) Удалите два экземпляра.
- (j) Создайте 7-й и 8-й экземпляры - должно сработать.

Пример использования:

```
# Создаем 6 сессий
sessions = [SessionPool(f"Token{i}") for i in range(6)]

# Попытка создать 7-ю
try:
    s7 = SessionPool("Token7")
except RuntimeError as e:
    print("Ошибка:", e)

# Удаляем 2
```

```
del sessions[0], sessions[1]

# Создаем 7-ю и 8-ю - успешно
s7 = SessionPool("Token7")
s8 = SessionPool("Token8")
print("Созданы токены:", s7.token, s8.token)
```

10. Написать программу на Python, которая создает класс 'ThreadPool' с использованием метода '__new__' для ограничения количества создаваемых экземпляров до 12.

Инструкции:

- Создайте класс 'ThreadPool'.
- Добавьте атрибут класса '_threads' и инициализируйте его пустым списком.
- Добавьте атрибут класса 'THREAD_MAX' и инициализируйте его значением 12.
- Переопределите метод '__new__'. Если 'len(_threads) >= THREAD_MAX', выбросьте 'RuntimeError("Достигнут лимит потоков!")'. Иначе, создайте экземпляр, добавьте в '_threads', верните.
- Переопределите метод '__del__', чтобы он удалял 'self' из '_threads'.
- Переопределите метод '__init__', который принимает 'thread_id' и устанавливает 'self.thread = thread_id'.
- Создайте 12 экземпляров.
- Попытайтесь создать 13-й - поймайте и выведите исключение.
- Удалите три экземпляра.
- Создайте 13-й, 14-й, 15-й экземпляры - должно сработать.

Пример использования:

```
# Создаем 12 потоков
threads = [ThreadPool(f"Thread{i}") for i in range(12)]

# Попытка создать 13-й
try:
    t13 = ThreadPool("Thread13")
except RuntimeError as e:
    print("Ошибка:", e)

# Удаляем 3
del threads[0], threads[1], threads[2]

# Создаем 13-й, 14-й, 15-й - успешно
t13 = ThreadPool("Thread13")
t14 = ThreadPool("Thread14")
t15 = ThreadPool("Thread15")
print("Созданы потоки:", t13.thread, t14.thread, t15.thread)
```

11. Написать программу на Python, которая создает класс 'CachePool' с использованием метода '__new__' для ограничения количества создаваемых экземпляров до 20.

Инструкции:

- Создайте класс 'CachePool'.
- Добавьте атрибут класса '_caches' и инициализируйте его пустым списком.

- (c) Добавьте атрибут класса 'CACHE_LIMIT' и инициализируйте его значением 20.
- (d) Переопределите метод '__new__'. Если 'len(_caches) >= CACHE_LIMIT', выбросьте 'RuntimeError("Кэш-пул переполнен!")'. Иначе, создайте экземпляр, добавьте в '_caches', верните.
- (e) Переопределите метод '__del__', чтобы он удалял 'self' из '_caches'.
- (f) Переопределите метод '__init__', который принимает 'cache_key' и устанавливает 'self.key = cache_key'.
- (g) Создайте 20 экземпляров.
- (h) Попробуйте создать 21-й - поймите и выведите исключение.
- (i) Удалите пять экземпляров.
- (j) Создайте 21-й, 22-й, ..., 25-й экземпляры - должно сработать.

Пример использования:

```
# Создаем 20 кэшей
caches = [CachePool(f"Key{i}") for i in range(20)]

# Попытка создать 21-й
try:
    c21 = CachePool("Key21")
except RuntimeError as e:
    print("Ошибка:", e)

# Удаляем 5
for i in range(5):
    del caches[0]

# Создаем 21-й, 22-й, ..., 25-й - успешно
new_caches = [CachePool(f"Key{i}") for i in range(21, 26)]
for c in new_caches:
    print("Создан ключ:", c.key)
```

12. Написать программу на Python, которая создает класс 'DatabasePool' с использованием метода '__new__' для ограничения количества создаваемых экземпляров до 4.

Инструкции:

- (a) Создайте класс 'DatabasePool'.
- (b) Добавьте атрибут класса '_databases' и инициализируйте его пустым списком.
- (c) Добавьте атрибут класса 'DB_LIMIT' и инициализируйте его значением 4.
- (d) Переопределите метод '__new__'. Если 'len(_databases) >= DB_LIMIT', выбросьте 'RuntimeError("Базы данных: лимит превышен!")'. Иначе, создайте экземпляр, добавьте в '_databases', верните.
- (e) Переопределите метод '__del__', чтобы он удалял 'self' из '_databases'.
- (f) Переопределите метод '__init__', который принимает 'db_name' и устанавливает 'self.name = db_name'.
- (g) Создайте 4 экземпляра.
- (h) Попробуйте создать 5-й - поймите и выведите исключение.
- (i) Удалите один экземпляр.

- (j) Создайте 5-й экземпляр - должно сработать.

Пример использования:

```
# Создаем 4 базы
dbs = [DatabasePool(f"DB{i}") for i in range(4)]

# Попытка создать 5-ю
try:
    db5 = DatabasePool("DB5")
except RuntimeError as e:
    print("Ошибка:", e)

# Удаляем одну
del dbs[0]

# Создаем 5-ю - успешно
db5 = DatabasePool("DB5")
print("Создана база:", db5.name)
```

13. Написать программу на Python, которая создает класс 'FileHandlerPool' с использованием метода '__new__' для ограничения количества создаваемых экземпляров до 9.

Инструкции:

- (a) Создайте класс 'FileHandlerPool'.
- (b) Добавьте атрибут класса '_handlers' и инициализируйте его пустым списком.
- (c) Добавьте атрибут класса 'HANDLER_MAX' и инициализируйте его значением 9.
- (d) Переопределите метод '__new__'. Если 'len(_handlers) >= HANDLER_MAX', выбросьте 'RuntimeError("Слишком много обработчиков файлов!")'. Иначе, создайте экземпляр, добавьте в '_handlers', верните.
- (e) Переопределите метод '__del__', чтобы он удалял 'self' из '_handlers'.
- (f) Переопределите метод '__init__', который принимает 'file_path' и устанавливает 'self.path = file_path'.
- (g) Создайте 9 экземпляров.
- (h) Попробуйте создать 10-й - поймайте и выведите исключение.
- (i) Удалите четыре экземпляра.
- (j) Создайте 10-й, 11-й, 12-й, 13-й экземпляры - должно сработать.

Пример использования:

```
# Создаем 9 обработчиков
handlers = [FileHandlerPool(f"/path/to/file{i}.txt") for i in range(9)]

# Попытка создать 10-й
try:
    h10 = FileHandlerPool("/path/to/newfile.txt")
except RuntimeError as e:
    print("Ошибка:", e)

# Удаляем 4
del handlers[0], handlers[1], handlers[2], handlers[3]
```

```
# Создаем 10-й, 11-й, 12-й, 13-й - успешно
new_handlers = [FileHandlerPool(f"/path/to/newfile{i}.txt") for i in range
(10, 14)]
for h in new_handlers:
    print("Обработчик для:", h.path)
```

14. Написать программу на Python, которая создает класс 'NetworkPool' с использованием метода '__new__' для ограничения количества создаваемых экземпляров до 11.

Инструкции:

- Создайте класс 'NetworkPool'.
- Добавьте атрибут класса '_networks' и инициализируйте его пустым списком.
- Добавьте атрибут класса 'NETWORK_CAP' и инициализируйте его значением 11.
- Переопределите метод '__new__'. Если 'len(_networks) >= NETWORK_CAP', выбросьте 'RuntimeError("Сеть: превышен лимит!")'. Иначе, создайте экземпляр, добавьте в '_networks', верните.
- Переопределите метод '__del__', чтобы он удалял 'self' из '_networks'.
- Переопределите метод '__init__', который принимает 'network_id' и устанавливает 'self.net_id = network_id'.
- Создайте 11 экземпляров.
- Попытайтесь создать 12-й - поймите и выведите исключение.
- Удалите шесть экземпляров.
- Создайте 12-й, 13-й, ..., 17-й экземпляры - должно сработать.

Пример использования:

```
# Создаем 11 сетей
networks = [NetworkPool(f"Net{i}") for i in range(11)]

# Попытка создать 12-ю
try:
    n12 = NetworkPool("Net12")
except RuntimeError as e:
    print("Ошибка:", e)

# Удаляем 6
for i in range(6):
    del networks[0]

# Создаем 12-ю, 13-ю, ..., 17-ю - успешно
new_networks = [NetworkPool(f"Net{i}") for i in range(12, 18)]
for n in new_networks:
    print("Создана сеть:", n.net_id)
```

15. Написать программу на Python, которая создает класс 'MemoryPool' с использованием метода '__new__' для ограничения количества создаваемых экземпляров до 25.

Инструкции:

- Создайте класс 'MemoryPool'.
- Добавьте атрибут класса '_blocks' и инициализируйте его пустым списком.

- (c) Добавьте атрибут класса 'MEMORY_LIMIT' и инициализируйте его значением 25.
- (d) Переопределите метод '__new__'. Если 'len(_blocks) >= MEMORY_LIMIT', выбросьте 'RuntimeError("Память: лимит блоков превышен!")'. Иначе, создайте экземпляр, добавьте в '_blocks', верните.
- (e) Переопределите метод '__del__', чтобы он удалял 'self' из '_blocks'.
- (f) Переопределите метод '__init__', который принимает 'block_size' и устанавливает 'self.size = block_size'.
- (g) Создайте 25 экземпляров.
- (h) Попробуйте создать 26-й - поймайте и выведите исключение.
- (i) Удалите десять экземпляров.
- (j) Создайте 26-й, 27-й, ..., 35-й экземпляры - должно сработать.

Пример использования:

```
# Создаем 25 блоков
blocks = [MemoryPool(f"Size{i}") for i in range(25)]

# Попытка создать 26-й
try:
    b26 = MemoryPool("Size26")
except RuntimeError as e:
    print("Ошибка:", e)

# Удаляем 10
for i in range(10):
    del blocks[0]

# Создаем 26-й, 27-й, ..., 35-й - успешно
new_blocks = [MemoryPool(f"Size{i}") for i in range(26, 36)]
for b in new_blocks:
    print("Создан блок размером:", b.size)
```

16. Написать программу на Python, которая создает класс 'ProcessPool' с использованием метода '__new__' для ограничения количества создаваемых экземпляров до 16.

Инструкции:

- (a) Создайте класс 'ProcessPool'.
- (b) Добавьте атрибут класса '_processes' и инициализируйте его пустым списком.
- (c) Добавьте атрибут класса 'PROCESS_MAX' и инициализируйте его значением 16.
- (d) Переопределите метод '__new__'. Если 'len(_processes) >= PROCESS_MAX', выбросьте 'RuntimeError("Процессы: лимит превышен!")'. Иначе, создайте экземпляр, добавьте в '_processes', верните.
- (e) Переопределите метод '__del__', чтобы он удалял 'self' из '_processes'.
- (f) Переопределите метод '__init__', который принимает 'process_name' и устанавливает 'self.name = process_name'.
- (g) Создайте 16 экземпляров.
- (h) Попробуйте создать 17-й - поймайте и выведите исключение.
- (i) Удалите восемь экземпляров.

- (j) Создайте 17-й, 18-й, ..., 24-й экземпляры - должно сработать.

Пример использования:

```
# Создаем 16 процессов
processes = [ProcessPool(f"Proc{i}") for i in range(16)]

# Попытка создать 17-й
try:
    p17 = ProcessPool("Proc17")
except RuntimeError as e:
    print("Ошибка:", e)

# Удаляем 8
for i in range(8):
    del processes[0]

# Создаем 17-й, 18-й, ..., 24-й - успешно
new_processes = [ProcessPool(f"Proc{i}") for i in range(17, 25)]
for p in new_processes:
    print("Запущен процесс:", p.name)
```

17. Написать программу на Python, которая создает класс 'BufferPool' с использованием метода '__new__' для ограничения количества создаваемых экземпляров до 18.

Инструкции:

- (a) Создайте класс 'BufferPool'.
- (b) Добавьте атрибут класса '_buffers' и инициализируйте его пустым списком.
- (c) Добавьте атрибут класса 'BUFFER_SIZE' и инициализируйте его значением 18.
- (d) Переопределите метод '__new__'. Если 'len(_buffers) >= BUFFER_SIZE', выбросьте 'RuntimeError("Буфер: переполнение!")'. Иначе, создайте экземпляр, добавьте в '_buffers', верните.
- (e) Переопределите метод '__del__', чтобы он удалял 'self' из '_buffers'.
- (f) Переопределите метод '__init__', который принимает 'buffer_id' и устанавливает 'self.id = buffer_id'.
- (g) Создайте 18 экземпляров.
- (h) Попытайтесь создать 19-й - поймайте и выведите исключение.
- (i) Удалите девять экземпляров.
- (j) Создайте 19-й, 20-й, ..., 27-й экземпляры - должно сработать.

Пример использования:

```
# Создаем 18 буферов
buffers = [BufferPool(f"Buf{i}") for i in range(18)]

# Попытка создать 19-й
try:
    b19 = BufferPool("Buf19")
except RuntimeError as e:
    print("Ошибка:", e)

# Удаляем 9
for i in range(9):
    del buffers[0]
```

```
# Создаем 19-й, 20-й, ..., 27-й - успешно
new_buffers = [BufferPool(f"Buf{i}") for i in range(19, 28)]
for b in new_buffers:
    print("Создан буфер:", b.id)
```

18. Написать программу на Python, которая создает класс 'ChannelPool' с использованием метода '__new__' для ограничения количества создаваемых экземпляров до 13.

Инструкции:

- Создайте класс 'ChannelPool'.
- Добавьте атрибут класса '_channels' и инициализируйте его пустым списком.
- Добавьте атрибут класса 'CHANNEL_LIMIT' и инициализируйте его значением 13.
- Переопределите метод '__new__'. Если 'len(_channels) >= CHANNEL_LIMIT', выбросьте 'RuntimeError("Каналы: лимит исчерпан!")'. Иначе, создайте экземпляр, добавьте в '_channels', верните.
- Переопределите метод '__del__', чтобы он удалял 'self' из '_channels'.
- Переопределите метод '__init__', который принимает 'channel_name' и устанавливает 'self.name = channel_name'.
- Создайте 13 экземпляров.
- Попытайтесь создать 14-й - поймите и выведите исключение.
- Удалите три экземпляра.
- Создайте 14-й, 15-й, 16-й экземпляры - должно сработать.

Пример использования:

```
# Создаем 13 каналов
channels = [ChannelPool(f"Channel{i}") for i in range(13)]

# Попытка создать 14-й
try:
    c14 = ChannelPool("Channel14")
except RuntimeError as e:
    print("Ошибка:", e)

# Удаляем 3
del channels[0], channels[1], channels[2]

# Создаем 14-й, 15-й, 16-й - успешно
c14 = ChannelPool("Channel14")
c15 = ChannelPool("Channel15")
c16 = ChannelPool("Channel16")
print("Созданы каналы:", c14.name, c15.name, c16.name)
```

19. Написать программу на Python, которая создает класс 'SocketPool' с использованием метода '__new__' для ограничения количества создаваемых экземпляров до 22.

Инструкции:

- Создайте класс 'SocketPool'.
- Добавьте атрибут класса '_sockets' и инициализируйте его пустым списком.

- (c) Добавьте атрибут класса 'SOCKET_MAX' и инициализируйте его значением 22.
- (d) Переопределите метод '__new__'. Если 'len(_sockets) >= SOCKET_MAX', выбросьте 'RuntimeError("Сокеты: лимит превышен!")'. Иначе, создайте экземпляр, добавьте в '_sockets', верните.
- (e) Переопределите метод '__del__', чтобы он удалял 'self' из '_sockets'.
- (f) Переопределите метод '__init__', который принимает 'socket_port' и устанавливает 'self.port = socket_port'.
- (g) Создайте 22 экземпляра.
- (h) Попробуйте создать 23-й - поймите и выведите исключение.
- (i) Удалите одиннадцать экземпляров.
- (j) Создайте 23-й, 24-й, ..., 33-й экземпляры - должно сработать.

Пример использования:

```
# Создаем 22 сокета
sockets = [SocketPool(8000 + i) for i in range(22)]

# Попытка создать 23-й
try:
    s23 = SocketPool(8022)
except RuntimeError as e:
    print("Ошибка:", e)

# Удаляем 11
for i in range(11):
    del sockets[0]

# Создаем 23-й, 24-й, ..., 33-й - успешно
new_sockets = [SocketPool(8022 + i) for i in range(11)]
for s in new_sockets:
    print("Создан сокет на порту:", s.port)
```

20. Написать программу на Python, которая создает класс 'LockPool' с использованием метода '__new__' для ограничения количества создаваемых экземпляров до 14.

Инструкции:

- (a) Создайте класс 'LockPool'.
- (b) Добавьте атрибут класса '_locks' и инициализируйте его пустым списком.
- (c) Добавьте атрибут класса 'LOCK_COUNT' и инициализируйте его значением 14.
- (d) Переопределите метод '__new__'. Если 'len(_locks) >= LOCK_COUNT', выбросьте 'RuntimeError("Замки: все заняты!")'. Иначе, создайте экземпляр, добавьте в '_locks', верните.
- (e) Переопределите метод '__del__', чтобы он удалял 'self' из '_locks'.
- (f) Переопределите метод '__init__', который принимает 'lock_name' и устанавливает 'self.name = lock_name'.
- (g) Создайте 14 экземпляров.
- (h) Попробуйте создать 15-й - поймите и выведите исключение.
- (i) Удалите семь экземпляров.

- (j) Создайте 15-й, 16-й, ..., 21-й экземпляры - должно сработать.

Пример использования:

```
# Создаем 14 замков
locks = [LockPool(f"Lock{i}") for i in range(14)]

# Попытка создать 15-й
try:
    l15 = LockPool("Lock15")
except RuntimeError as e:
    print("Ошибка:", e)

# Удаляем 7
for i in range(7):
    del locks[0]

# Создаем 15-й, 16-й, ..., 21-й - успешно
new_locks = [LockPool(f"Lock{i}") for i in range(15, 22)]
for l in new_locks:
    print("Создан замок:", l.name)
```

21. Написать программу на Python, которая создает класс 'QueuePool' с использованием метода '__new__' для ограничения количества создаваемых экземпляров до 19.

Инструкции:

- (a) Создайте класс 'QueuePool'.
- (b) Добавьте атрибут класса '_queues' и инициализируйте его пустым списком.
- (c) Добавьте атрибут класса 'QUEUE_COUNT' и инициализируйте его значением 19.
- (d) Переопределите метод '__new__'. Если 'len(_queues) >= QUEUE_COUNT', выбросьте 'RuntimeError("Очереди: лимит достигнут!")'. Иначе, создайте экземпляр, добавьте в '_queues', верните.
- (e) Переопределите метод '__del__', чтобы он удалял 'self' из '_queues'.
- (f) Переопределите метод '__init__', который принимает 'queue_name' и устанавливает 'self.name = queue_name'.
- (g) Создайте 19 экземпляров.
- (h) Попробуйте создать 20-й - поймите и выведите исключение.
- (i) Удалите десять экземпляров.
- (j) Создайте 20-й, 21-й, ..., 29-й экземпляры - должно сработать.

Пример использования:

```
# Создаем 19 очередей
queues = [QueuePool(f"Queue{i}") for i in range(19)]

# Попытка создать 20-ю
try:
    q20 = QueuePool("Queue20")
except RuntimeError as e:
    print("Ошибка:", e)

# Удаляем 10
for i in range(10):
```

```
del queues[0]

# Создаем 20-ю, 21-ю, ..., 29-ю - успешно
new_queues = [QueuePool(f"Queue{i}") for i in range(20, 30)]
for q in new_queues:
    print("Создана очередь:", q.name)
```

22. Написать программу на Python, которая создает класс ‘SemaphorePool’ с использованием метода ‘__new__’ для ограничения количества создаваемых экземпляров до 8.

Инструкции:

- Создайте класс ‘SemaphorePool’.
- Добавьте атрибут класса ‘_semaphores’ и инициализируйте его пустым списком.
- Добавьте атрибут класса ‘SEMA_LIMIT’ и инициализируйте его значением 8.
- Переопределите метод ‘__new__’. Если ‘len(_semaphores) >= SEMA_LIMIT’, выбросьте ‘RuntimeError("Семафоры: лимит превышен!")’. Иначе, создайте экземпляр, добавьте в ‘_semaphores’, верните.
- Переопределите метод ‘__del__’, чтобы он удалял ‘self’ из ‘_semaphores’.
- Переопределите метод ‘__init__’, который принимает ‘sema_id’ и устанавливает ‘self.id = sema_id’.
- Создайте 8 экземпляров.
- Попытайтесь создать 9-й - поймите и выведите исключение.
- Удалите четыре экземпляра.
- Создайте 9-й, 10-й, 11-й, 12-й экземпляры - должно сработать.

Пример использования:

```
# Создаем 8 семафоров
semas = [SemaphorePool(f"Sema{i}") for i in range(8)]

# Попытка создать 9-й
try:
    s9 = SemaphorePool("Sema9")
except RuntimeError as e:
    print("Ошибка:", e)

# Удаляем 4
del semas[0], semas[1], semas[2], semas[3]

# Создаем 9-й, 10-й, 11-й, 12-й - успешно
s9 = SemaphorePool("Sema9")
s10 = SemaphorePool("Sema10")
s11 = SemaphorePool("Sema11")
s12 = SemaphorePool("Sema12")
print("Созданы семафоры:", s9.id, s10.id, s11.id, s12.id)
```

23. Написать программу на Python, которая создает класс ‘TimerPool’ с использованием метода ‘__new__’ для ограничения количества создаваемых экземпляров до 21.

Инструкции:

- Создайте класс ‘TimerPool’.

- (b) Добавьте атрибут класса `'_timers'` и инициализируйте его пустым списком.
- (c) Добавьте атрибут класса `'TIMER_MAX'` и инициализируйте его значением 21.
- (d) Переопределите метод `'__new__'`. Если `'len(_timers) >= TIMER_MAX'`, выбросьте `'RuntimeError("Таймеры: лимит исчерпан!")'`. Иначе, создайте экземпляр, добавьте в `'_timers'`, верните.
- (e) Переопределите метод `'__del__'`, чтобы он удалял `'self'` из `'_timers'`.
- (f) Переопределите метод `'__init__'`, который принимает `'timer_duration'` и устанавливает `'self.duration = timer_duration'`.
- (g) Создайте 21 экземпляр.
- (h) Попробуйте создать 22-й - поймайте и выведите исключение.
- (i) Удалите одиннадцать экземпляров.
- (j) Создайте 22-й, 23-й, ..., 32-й экземпляры - должно сработать.

Пример использования:

```
# Создаем 21 таймер
timers = [TimerPool(i * 10) for i in range(21)]

# Попробуем создать 22-й
try:
    t22 = TimerPool(220)
except RuntimeError as e:
    print("Ошибка:", e)

# Удаляем 11
for i in range(11):
    del timers[0]

# Создаем 22-й, 23-й, ..., 32-й - успешно
new_timers = [TimerPool(i * 10) for i in range(22, 33)]
for t in new_timers:
    print("Создан таймер на:", t.duration, "сек")
```

24. Написать программу на Python, которая создает класс `'WorkerPool'` с использованием метода `'__new__'` для ограничения количества создаваемых экземпляров до 23.

Инструкции:

- (a) Создайте класс `'WorkerPool'`.
- (b) Добавьте атрибут класса `'_workers'` и инициализируйте его пустым списком.
- (c) Добавьте атрибут класса `'WORKER_LIMIT'` и инициализируйте его значением 23.
- (d) Переопределите метод `'__new__'`. Если `'len(_workers) >= WORKER_LIMIT'`, выбросьте `'RuntimeError("Рабочие: лимит превышен!")'`. Иначе, создайте экземпляр, добавьте в `'_workers'`, верните.
- (e) Переопределите метод `'__del__'`, чтобы он удалял `'self'` из `'_workers'`.
- (f) Переопределите метод `'__init__'`, который принимает `'worker_id'` и устанавливает `'self.id = worker_id'`.
- (g) Создайте 23 экземпляра.
- (h) Попробуйте создать 24-й - поймайте и выведите исключение.

- (i) Удалите двенадцать экземпляров.
- (j) Создайте 24-й, 25-й, ..., 35-й экземпляры - должно сработать.

Пример использования:

```
# Создаем 23 рабочих
workers = [WorkerPool(f"Worker{i}") for i in range(23)]

# Попытка создать 24-го
try:
    w24 = WorkerPool("Worker24")
except RuntimeError as e:
    print("Ошибка:", e)

# Удаляем 12
for i in range(12):
    del workers[0]

# Создаем 24-го, 25-го, ..., 35-го - успешно
new_workers = [WorkerPool(f"Worker{i}") for i in range(24, 36)]
for w in new_workers:
    print("Создан рабочий:", w.id)
```

25. Написать программу на Python, которая создает класс 'JobPool' с использованием метода '__new__' для ограничения количества создаваемых экземпляров до 26.

Инструкции:

- (a) Создайте класс 'JobPool'.
- (b) Добавьте атрибут класса '_jobs' и инициализируйте его пустым списком.
- (c) Добавьте атрибут класса 'JOB_CAP' и инициализируйте его значением 26.
- (d) Переопределите метод '__new__'. Если 'len(_jobs) >= JOB_CAP', выбросьте 'RuntimeError("Задания: лимит превышен!")'. Иначе, создайте экземпляр, добавьте в '_jobs', верните.
- (e) Переопределите метод '__del__', чтобы он удалял 'self' из '_jobs'.
- (f) Переопределите метод '__init__', который принимает 'job_name' и устанавливает 'self.name = job_name'.
- (g) Создайте 26 экземпляров.
- (h) Попытайтесь создать 27-й - поймайте и выведите исключение.
- (i) Удалите тринадцать экземпляров.
- (j) Создайте 27-й, 28-й, ..., 39-й экземпляры - должно сработать.

Пример использования:

```
# Создаем 26 заданий
jobs = [JobPool(f"Job{i}") for i in range(26)]

# Попытка создать 27-е
try:
    j27 = JobPool("Job27")
except RuntimeError as e:
    print("Ошибка:", e)

# Удаляем 13
```

```

for i in range(13):
    del jobs[0]

# Создаем 27-е, 28-е, ..., 39-е - успешно
new_jobs = [JobPool(f"Job{i}") for i in range(27, 40)]
for j in new_jobs:
    print("Создано задание:", j.name)

```

26. Написать программу на Python, которая создает класс 'RequestPool' с использованием метода '__new__' для ограничения количества создаваемых экземпляров до 27.

Инструкции:

- Создайте класс 'RequestPool'.
- Добавьте атрибут класса '_requests' и инициализируйте его пустым списком.
- Добавьте атрибут класса 'REQUEST_MAX' и инициализируйте его значением 27.
- Переопределите метод '__new__'. Если 'len(_requests) >= REQUEST_MAX', выбросьте 'RuntimeError("Запросы: лимит превышен!")'. Иначе, создайте экземпляр, добавьте в '_requests', верните.
- Переопределите метод '__del__', чтобы он удалял 'self' из '_requests'.
- Переопределите метод '__init__', который принимает 'request_url' и устанавливает 'self.url = request_url'.
- Создайте 27 экземпляров.
- Попытайтесь создать 28-й - поймайте и выведите исключение.
- Удалите четырнадцать экземпляров.
- Создайте 28-й, 29-й, ..., 41-й экземпляры - должно сработать.

Пример использования:

```

# Создаем 27 запросов
requests = [RequestPool(f"http://site{i}.com") for i in range(27)]

# Попытка создать 28-й
try:
    r28 = RequestPool("http://newsite.com")
except RuntimeError as e:
    print("Ошибка:", e)

# Удаляем 14
for i in range(14):
    del requests[0]

# Создаем 28-й, 29-й, ..., 41-й - успешно
new_requests = [RequestPool(f"http://newsite{i}.com") for i in range(28, 42)]
for r in new_requests:
    print("Создан запрос к:", r.url)

```

27. Написать программу на Python, которая создает класс 'EventPool' с использованием метода '__new__' для ограничения количества создаваемых экземпляров до 28.

Инструкции:

- Создайте класс 'EventPool'.

- (b) Добавьте атрибут класса `‘_events’` и инициализируйте его пустым списком.
- (c) Добавьте атрибут класса `‘EVENT_LIMIT’` и инициализируйте его значением 28.
- (d) Переопределите метод `‘__new__’`. Если `‘len(_events) >= EVENT_LIMIT’`, выбросьте `‘RuntimeError("События: лимит превышен!")’`. Иначе, создайте экземпляр, добавьте в `‘_events’`, верните.
- (e) Переопределите метод `‘__del__’`, чтобы он удалял `‘self’` из `‘_events’`.
- (f) Переопределите метод `‘__init__’`, который принимает `‘event_type’` и устанавливает `‘self.type = event_type’`.
- (g) Создайте 28 экземпляров.
- (h) Попробуйте создать 29-е - поймайте и выведите исключение.
- (i) Удалите пятнадцать экземпляров.
- (j) Создайте 29-е, 30-е, ..., 43-е экземпляры - должно сработать.

Пример использования:

```
# Создаем 28 событий
events = [EventPool(f"Event{i}") for i in range(28)]

# Попытка создать 29-е
try:
    e29 = EventPool("Event29")
except RuntimeError as e:
    print("Ошибка:", e)

# Удаляем 15
for i in range(15):
    del events[0]

# Создаем 29-е, 30-е, ..., 43-е - успешно
new_events = [EventPool(f"Event{i}") for i in range(29, 44)]
for e in new_events:
    print("Создано событие типа:", e.type)
```

28. Написать программу на Python, которая создает класс `‘MessagePool’` с использованием метода `‘__new__’` для ограничения количества создаваемых экземпляров до 29.

Инструкции:

- (a) Создайте класс `‘MessagePool’`.
- (b) Добавьте атрибут класса `‘_messages’` и инициализируйте его пустым списком.
- (c) Добавьте атрибут класса `‘MSG_MAX’` и инициализируйте его значением 29.
- (d) Переопределите метод `‘__new__’`. Если `‘len(_messages) >= MSG_MAX’`, выбросьте `‘RuntimeError("Сообщения: лимит превышен!")’`. Иначе, создайте экземпляр, добавьте в `‘_messages’`, верните.
- (e) Переопределите метод `‘__del__’`, чтобы он удалял `‘self’` из `‘_messages’`.
- (f) Переопределите метод `‘__init__’`, который принимает `‘message_text’` и устанавливает `‘self.text = message_text’`.
- (g) Создайте 29 экземпляров.
- (h) Попробуйте создать 30-й - поймайте и выведите исключение.

- (i) Удалите шестнадцать экземпляров.
- (j) Создайте 30-й, 31-й, ..., 45-й экземпляры - должно сработать.

Пример использования:

```
# Создаем 29 сообщений
messages = [MessagePool(f"Message{i}") for i in range(29)]

# Попытка создать 30-е
try:
    m30 = MessagePool("Message30")
except RuntimeError as e:
    print("Ошибка:", e)

# Удаляем 16
for i in range(16):
    del messages[0]

# Создаем 30-е, 31-е, ..., 45-е - успешно
new_messages = [MessagePool(f"Message{i}") for i in range(30, 46)]
for m in new_messages:
    print("Создано сообщение:", m.text)
```

29. Написать программу на Python, которая создает класс 'NotificationPool' с использованием метода '__new__' для ограничения количества создаваемых экземпляров до 31.

Инструкции:

- (a) Создайте класс 'NotificationPool'.
- (b) Добавьте атрибут класса '_notifications' и инициализируйте его пустым списком.
- (c) Добавьте атрибут класса 'NOTIF_LIMIT' и инициализируйте его значением 31.
- (d) Переопределите метод '__new__'. Если 'len(_notifications) >= NOTIF_LIMIT', выбросьте 'RuntimeError("Уведомления: лимит превышен!")'. Иначе, создайте экземпляр, добавьте в '_notifications', верните.
- (e) Переопределите метод '__del__', чтобы он удалял 'self' из '_notifications'.
- (f) Переопределите метод '__init__', который принимает 'notification_title' и устанавливает 'self.title = notification_title'.
- (g) Создайте 31 экземпляр.
- (h) Попробуйте создать 32-й - поймайте и выведите исключение.
- (i) Удалите семнадцать экземпляров.
- (j) Создайте 32-й, 33-й, ..., 48-й экземпляры - должно сработать.

Пример использования:

```
# Создаем 31 уведомление
notifications = [NotificationPool(f"Notif{i}") for i in range(31)]

# Попытка создать 32-е
try:
    n32 = NotificationPool("Notif32")
except RuntimeError as e:
    print("Ошибка:", e)
```

```

# Удаляем 17
for i in range(17):
    del notifications[0]

# Создаем 32-е, 33-е, ..., 48-е - успешно
new_notifications = [NotificationPool(f"Notif{i}") for i in range(32, 49)]
for n in new_notifications:
    print("Создано уведомление:", n.title)

```

30. Написать программу на Python, которая создает класс 'LoggerPool' с использованием метода '__new__' для ограничения количества создаваемых экземпляров до 5.

Инструкции:

- Создайте класс 'LoggerPool'.
- Добавьте атрибут класса '_loggers' и инициализируйте его пустым списком.
- Добавьте атрибут класса 'LOGGER_LIMIT' и инициализируйте его значением 5.
- Переопределите метод '__new__'. Если 'len(_loggers) >= LOGGER_LIMIT', выбросьте 'RuntimeError("Логгеры: лимит превышен!")'. Иначе, создайте экземпляр, добавьте в '_loggers', верните.
- Переопределите метод '__del__', чтобы он удалял 'self' из '_loggers'.
- Переопределите метод '__init__', который принимает 'logger_name' и устанавливает 'self.name = logger_name'.
- Создайте 5 экземпляров.
- Попытайтесь создать 6-й - поймайте и выведите исключение.
- Удалите два экземпляра.
- Создайте 6-й и 7-й экземпляры - должно сработать.

Пример использования:

```

# Создаем 5 логгеров
loggers = [LoggerPool(f"Logger{i}") for i in range(5)]

# Попытка создать 6-й
try:
    l6 = LoggerPool("Logger6")
except RuntimeError as e:
    print("Ошибка:", e)

# Удаляем 2
del loggers[0], loggers[1]

# Создаем 6-й и 7-й - успешно
l6 = LoggerPool("Logger6")
l7 = LoggerPool("Logger7")
print("Созданы логгеры:", l6.name, l7.name)

```

31. Написать программу на Python, которая создает класс 'ConfigPool' с использованием метода '__new__' для ограничения количества создаваемых экземпляров до 12.

Инструкции:

- Создайте класс 'ConfigPool'.
- Добавьте атрибут класса '_configs' и инициализируйте его пустым списком.

- (c) Добавьте атрибут класса 'CONFIG_MAX' и инициализируйте его значением 12.
- (d) Переопределите метод '__new__'. Если 'len(_configs) >= CONFIG_MAX', выбросьте 'RuntimeError("Конфигурации: лимит превышен!")'. Иначе, создайте экземпляр, добавьте в '_configs', верните.
- (e) Переопределите метод '__del__', чтобы он удалял 'self' из '_configs'.
- (f) Переопределите метод '__init__', который принимает 'config_name' и устанавливает 'self.name = config_name'.
- (g) Создайте 12 экземпляров.
- (h) Попробуйте создать 13-й - поймите и выведите исключение.
- (i) Удалите шесть экземпляров.
- (j) Создайте 13-й, 14-й, ..., 18-й экземпляры - должно сработать.

Пример использования:

```
# Создаем 12 конфигураций
configs = [ConfigPool(f"Config{i}") for i in range(12)]

# Попытка создать 13-ю
try:
    c13 = ConfigPool("Config13")
except RuntimeError as e:
    print("Ошибка:", e)

# Удаляем 6
for i in range(6):
    del configs[0]

# Создаем 13-ю, 14-ю, ..., 18-ю - успешно
new_configs = [ConfigPool(f"Config{i}") for i in range(13, 19)]
for c in new_configs:
    print("Создана конфигурация:", c.name)
```

32. Написать программу на Python, которая создает класс 'PluginPool' с использованием метода '__new__' для ограничения количества создаваемых экземпляров до 10.

Инструкции:

- (a) Создайте класс 'PluginPool'.
- (b) Добавьте атрибут класса '_plugins' и инициализируйте его пустым списком.
- (c) Добавьте атрибут класса 'PLUGIN_CAP' и инициализируйте его значением 10.
- (d) Переопределите метод '__new__'. Если 'len(_plugins) >= PLUGIN_CAP', выбросьте 'RuntimeError("Плагины: лимит исчерпан!")'. Иначе, создайте экземпляр, добавьте в '_plugins', верните.
- (e) Переопределите метод '__del__', чтобы он удалял 'self' из '_plugins'.
- (f) Переопределите метод '__init__', который принимает 'plugin_id' и устанавливает 'self.id = plugin_id'.
- (g) Создайте 10 экземпляров.
- (h) Попробуйте создать 11-й - поймите и выведите исключение.
- (i) Удалите пять экземпляров.

- (j) Создайте 11-й, 12-й, ..., 15-й экземпляры - должно сработать.

Пример использования:

```
# Создаем 10 плагинов
plugins = [PluginPool(f"Plugin{i}") for i in range(10)]

# Попытка создать 11-й
try:
    p11 = PluginPool("Plugin11")
except RuntimeError as e:
    print("Ошибка:", e)

# Удаляем 5
for i in range(5):
    del plugins[0]

# Создаем 11-й, 12-й, ..., 15-й - успешно
new_plugins = [PluginPool(f"Plugin{i}") for i in range(11, 16)]
for p in new_plugins:
    print("Создан плагин:", p.id)
```

33. Написать программу на Python, которая создает класс 'ServicePool' с использованием метода '__new__' для ограничения количества создаваемых экземпляров до 8.

Инструкции:

- (a) Создайте класс 'ServicePool'.
- (b) Добавьте атрибут класса '_services' и инициализируйте его пустым списком.
- (c) Добавьте атрибут класса 'SERVICE_LIMIT' и инициализируйте его значением 8.
- (d) Переопределите метод '__new__'. Если 'len(_services) >= SERVICE_LIMIT', выбросьте 'RuntimeError("Сервисы: лимит превышен!")'. Иначе, создайте экземпляр, добавьте в '_services', верните.
- (e) Переопределите метод '__del__', чтобы он удалял 'self' из '_services'.
- (f) Переопределите метод '__init__', который принимает 'service_name' и устанавливает 'self.name = service_name'.
- (g) Создайте 8 экземпляров.
- (h) Попробуйте создать 9-й - поймайте и выведите исключение.
- (i) Удалите четыре экземпляра.
- (j) Создайте 9-й, 10-й, 11-й, 12-й экземпляры - должно сработать.

Пример использования:

```
# Создаем 8 сервисов
services = [ServicePool(f"Service{i}") for i in range(8)]

# Попытка создать 9-й
try:
    s9 = ServicePool("Service9")
except RuntimeError as e:
    print("Ошибка:", e)

# Удаляем 4
del services[0], services[1], services[2], services[3]
```

```

# Создаем 9-й, 10-й, 11-й, 12-й - успешно
s9 = ServicePool("Service9")
s10 = ServicePool("Service10")
s11 = ServicePool("Service11")
s12 = ServicePool("Service12")
print("Созданы сервисы:", s9.name, s10.name, s11.name, s12.name)

```

34. Написать программу на Python, которая создает класс 'CacheEntryPool' с использованием метода '__new__' для ограничения количества создаваемых экземпляров до 15.

Инструкции:

- Создайте класс 'CacheEntryPool'.
- Добавьте атрибут класса '_entries' и инициализируйте его пустым списком.
- Добавьте атрибут класса 'ENTRY_MAX' и инициализируйте его значением 15.
- Переопределите метод '__new__'. Если 'len(_entries) >= ENTRY_MAX', выбросьте 'RuntimeError("Кэш-записи: лимит превышен!")'. Иначе, создайте экземпляр, добавьте в '_entries', верните.
- Переопределите метод '__del__', чтобы он удалял 'self' из '_entries'.
- Переопределите метод '__init__', который принимает 'entry_key' и устанавливает 'self.key = entry_key'.
- Создайте 15 экземпляров.
- Попытайтесь создать 16-й - поймайте и выведите исключение.
- Удалите семь экземпляров.
- Создайте 16-й, 17-й, ..., 22-й экземпляры - должно сработать.

Пример использования:

```

# Создаем 15 записей
entries = [CacheEntryPool(f"Key{i}") for i in range(15)]

# Попытка создать 16-ю
try:
    e16 = CacheEntryPool("Key16")
except RuntimeError as e:
    print("Ошибка:", e)

# Удаляем 7
for i in range(7):
    del entries[0]

# Создаем 16-ю, 17-ю, ..., 22-ю - успешно
new_entries = [CacheEntryPool(f"Key{i}") for i in range(16, 23)]
for e in new_entries:
    print("Создана запись с ключом:", e.key)

```

35. Написать программу на Python, которая создает класс 'ConnectionHandlerPool' с использованием метода '__new__' для ограничения количества создаваемых экземпляров до 20.

Инструкции:

- Создайте класс 'ConnectionHandlerPool'.

- (b) Добавьте атрибут класса `‘_handlers’` и инициализируйте его пустым списком.
- (c) Добавьте атрибут класса `‘HANDLER_LIMIT’` и инициализируйте его значением 20.
- (d) Переопределите метод `‘__new__’`. Если `‘len(_handlers) >= HANDLER_LIMIT’`, выбросьте `‘RuntimeError("Обработчики соединений: лимит превышен!")’`. Иначе, создайте экземпляр, добавьте в `‘_handlers’`, верните.
- (e) Переопределите метод `‘__del__’`, чтобы он удалял `‘self’` из `‘_handlers’`.
- (f) Переопределите метод `‘__init__’`, который принимает `‘handler_id’` и устанавливает `‘self.id = handler_id’`.
- (g) Создайте 20 экземпляров.
- (h) Попробуйте создать 21-й - поймайте и выведите исключение.
- (i) Удалите десять экземпляров.
- (j) Создайте 21-й, 22-й, ..., 30-й экземпляры - должно сработать.

Пример использования:

```
# Создаем 20 обработчиков
handlers = [ConnectionHandlerPool(f"H{i}") for i in range(20)]

# Попробуем создать 21-й
try:
    h21 = ConnectionHandlerPool("H21")
except RuntimeError as e:
    print("Ошибка:", e)

# Удаляем 10
for i in range(10):
    del handlers[0]

# Создаем 21-й, 22-й, ..., 30-й - успешно
new_handlers = [ConnectionHandlerPool(f"H{i}") for i in range(21, 31)]
for h in new_handlers:
    print("Создан обработчик:", h.id)
```

2.4.3 Задача 3 (именование)

1. Написать программу на Python, которая создает класс `‘Vagon’` с использованием метода `‘__new__’` для контроля именования. Имена должны начинаться с `"vagon_"`. Метод `‘__init__’` должен быть пустым.

Инструкции:

- (a) Создайте класс `‘Vagon’`.
- (b) Добавьте атрибут класса `‘numbers’` и инициализируйте его пустым словарем.
- (c) Переопределите метод `‘__new__’`, принимающий `‘cls’`, `‘name’`, `‘number’`.
- (d) В `‘__new__’`: если `‘name’` не начинается с `"vagon_"` выбросьте `‘ValueError("Имя должно начинаться с 'vagon_'")’`.
- (e) Извлеките номер вагона: `‘vagon_number = name[6:]’` (удаляем `"vagon_"`).
- (f) Создайте экземпляр: `‘instance = super().__new__(cls)’`.
- (g) Добавьте номер в словарь: `‘cls.numbers[vagon_number] = instance’`.

- (h) Установите атрибут экземпляра: `setattr(instance, f"vvagon_number number")`.
- (i) Верните `'instance'`.
- (j) Переопределите метод `'__init__'` как пустой: `def __init__(self, *args, **kwargs): pass`.
- (k) Создайте объект `'v1'` с именем `"vagon_1"` и номером 101.
- (l) Создайте объект `'v2'` с именем `"vagon_2"` и номером 102.
- (m) Попробуйте создать объект с именем `"car_3"`— должно возникнуть исключение `'ValueError'`.
- (n) Выведите `'v1.v1'` и `'v2.v2'`.
- (o) Выведите `'Vagon.numbers'`.

Пример использования:

```
v1 = Vagon("vagon_1", 101)
v2 = Vagon("vagon_2", 102)

try:
    v3 = Vagon("car_3", 103)
except ValueError as e:
    print("Ошибка:", e)

print("v1.v1:", v1.v1) # 101
print("v2.v2:", v2.v2) # 102
print("Vagon.numbers:", Vagon.numbers)
```

2. Написать программу на Python, которая создает класс `'Room'` с использованием метода `'__new__'` для контроля именования. Имена должны начинаться с `"room_"`. Метод `'__init__'` должен быть пустым.

Инструкции:

- (a) Создайте класс `'Room'`.
- (b) Добавьте атрибут класса `'registry'` и инициализируйте его пустым словарем.
- (c) Переопределите метод `'__new__'`, принимающий `'cls'`, `'name'`, `'capacity'`.
- (d) В `'__new__'`: если `'name'` не начинается с `"room_"` выбросьте `ValueError("Недопустимое имя комнаты")`.
- (e) Извлеките номер комнаты: `room_num = name[5:]`.
- (f) Создайте экземпляр: `instance = super().__new__(cls)`.
- (g) Добавьте номер в словарь: `cls.registry[room_num] = instance`.
- (h) Установите атрибут экземпляра: `setattr(instance, f"rroom_num capacity")`.
- (i) Верните `'instance'`.
- (j) Переопределите метод `'__init__'` как пустой.
- (k) Создайте объект `'r1'` с именем `"room_101"` и вместимостью 50.
- (l) Создайте объект `'r2'` с именем `"room_202"` и вместимостью 30.
- (m) Попробуйте создать объект с именем `"hall_A"`— поймайте исключение.
- (n) Выведите `'r1.r101'` и `'r2.r202'`.
- (o) Выведите `'Room.registry'`.

Пример использования:

```
r1 = Room("room_101", 50)
r2 = Room("room_202", 30)

try:
    r3 = Room("hall_A", 100)
except ValueError as e:
    print("Ошибка:", e)

print("r1.r101:", r1.r101) # 50
print("r2.r202:", r2.r202) # 30
print("Room.registry:", Room.registry)
```

3. Написать программу на Python, которая создает класс 'Device' с использованием метода '.__new__' для контроля именования. Имена должны начинаться с "dev_". Метод '.__init__' должен быть пустым.

Инструкции:

- (a) Создайте класс 'Device'.
- (b) Добавьте атрибут класса 'inventory' и инициализируйте его пустым словарем.
- (c) Переопределите метод '.__new__', принимающий 'cls', 'name', 'model'.
- (d) В '.__new__': если 'name' не начинается с "dev_" выбросьте 'ValueError("Неверный префикс устройства")'.
- (e) Извлеките ID устройства: 'dev_id = name[4:]'.
- (f) Создайте экземпляр: 'instance = super().__new__(cls)'.
- (g) Добавьте ID в словарь: 'cls.inventory[dev_id] = instance'.
- (h) Установите атрибут экземпляра: 'setattr(instance, f"ddev_id model)'.
- (i) Верните 'instance'.
- (j) Переопределите метод '.__init__' как пустой.
- (k) Создайте объект 'd1' с именем "dev_001" и моделью "X1".
- (l) Создайте объект 'd2' с именем "dev_002" и моделью "Y2".
- (m) Попытайтесь создать объект с именем "sensor_01"— поймайте исключение.
- (n) Выведите 'd1.d001' и 'd2.d002'.
- (o) Выведите 'Device.inventory'.

Пример использования:

```
d1 = Device("dev_001", "X1")
d2 = Device("dev_002", "Y2")

try:
    d3 = Device("sensor_01", "Z3")
except ValueError as e:
    print("Ошибка:", e)

print("d1.d001:", d1.d001) # X1
print("d2.d002:", d2.d002) # Y2
print("Device.inventory:", Device.inventory)
```

4. Написать программу на Python, которая создает класс 'Book' с использованием метода '__new__' для контроля именования. Имена должны начинаться с "book_". Метод '__init__' должен быть пустым.

Инструкции:

- (a) Создайте класс 'Book'.
- (b) Добавьте атрибут класса 'catalog' и инициализируйте его пустым словарем.
- (c) Переопределите метод '__new__', принимающий 'cls', 'name', 'author'.
- (d) В '__new__': если 'name' не начинается с "book_" выбросьте 'ValueError("Книга должна иметь префикс 'book_')'.
- (e) Извлеките ID книги: 'book_id = name[5:]'.
- (f) Создайте экземпляр: 'instance = super().__new__(cls)'.
- (g) Добавьте ID в словарь: 'cls.catalog[book_id] = instance'.
- (h) Установите атрибут экземпляра: 'setattr(instance, f"book_{book_id} author")'.
- (i) Верните 'instance'.
- (j) Переопределите метод '__init__' как пустой.
- (k) Создайте объект 'b1' с именем "book_001" и автором "Толстой".
- (l) Создайте объект 'b2' с именем "book_002" и автором "Достоевский".
- (m) Попробуйте создать объект с именем "magazine_01"— поймите исключение.
- (n) Выведите 'b1.b001' и 'b2.b002'.
- (o) Выведите 'Book.catalog'.

Пример использования:

```
b1 = Book("book_001", "Толстой")
b2 = Book("book_002", "Достоевский")

try:
    b3 = Book("magazine_01", "Пушкин")
except ValueError as e:
    print("Ошибка:", e)

print("b1.b001:", b1.b001) # Толстой
print("b2.b002:", b2.b002) # Достоевский
print("Book.catalog:", Book.catalog)
```

5. Написать программу на Python, которая создает класс 'File' с использованием метода '__new__' для контроля именования. Имена должны начинаться с "file_". Метод '__init__' должен быть пустым.

Инструкции:

- (a) Создайте класс 'File'.
- (b) Добавьте атрибут класса 'index' и инициализируйте его пустым словарем.
- (c) Переопределите метод '__new__', принимающий 'cls', 'name', 'size'.
- (d) В '__new__': если 'name' не начинается с "file_" выбросьте 'ValueError("Файл должен иметь префикс 'file_')'.
- (e) Извлеките ID файла: 'file_id = name[5:]'.

- (f) Создайте экземпляр: `instance = super().__new__(cls)`.
- (g) Добавьте ID в словарь: `cls.index[file_id] = instance`.
- (h) Установите атрибут экземпляра: `setattr(instance, f"file_id size")`.
- (i) Верните `instance`.
- (j) Переопределите метод `__init__` как пустой.
- (k) Создайте объект `'f1'` с именем `"file_config"` и размером 1024.
- (l) Создайте объект `'f2'` с именем `"file_data"` и размером 2048.
- (m) Попробуйте создать объект с именем `"document_1"` — поймите исключение.
- (n) Выведите `'f1.fconfig'` и `'f2.fdata'`.
- (o) Выведите `'File.index'`.

Пример использования:

```
f1 = File("file_config", 1024)
f2 = File("file_data", 2048)

try:
    f3 = File("document_1", 512)
except ValueError as e:
    print("Ошибка:", e)

print("f1.fconfig:", f1.fconfig) # 1024
print("f2.fdata:", f2.fdata)    # 2048
print("File.index:", File.index)
```

6. Написать программу на Python, которая создает класс `'User'` с использованием метода `__new__` для контроля именования. Имена должны начинаться с `"user_"`. Метод `__init__` должен быть пустым.

Инструкции:

- (a) Создайте класс `'User'`.
- (b) Добавьте атрибут класса `'directory'` и инициализируйте его пустым словарем.
- (c) Переопределите метод `__new__`, принимающий `'cls'`, `'name'`, `'email'`.
- (d) В `__new__`: если `'name'` не начинается с `"user_"` выбросьте `ValueError("Пользователь должен иметь префикс 'user_'")`.
- (e) Извлеките ID пользователя: `'user_id = name[5:]'`.
- (f) Создайте экземпляр: `instance = super().__new__(cls)`.
- (g) Добавьте ID в словарь: `cls.directory[user_id] = instance`.
- (h) Установите атрибут экземпляра: `setattr(instance, f"user_id email")`.
- (i) Верните `instance`.
- (j) Переопределите метод `__init__` как пустой.
- (k) Создайте объект `'u1'` с именем `"user_alice"` и email `"alice@example.com"`.
- (l) Создайте объект `'u2'` с именем `"user_bob"` и email `"bob@example.com"`.
- (m) Попробуйте создать объект с именем `"admin_john"` — поймите исключение.
- (n) Выведите `'u1.ualice'` и `'u2.ubob'`.

(o) Выведите 'User.directory'.

Пример использования:

```
u1 = User("user_alice", "alice@example.com")
u2 = User("user_bob", "bob@example.com")

try:
    u3 = User("admin_john", "john@example.com")
except ValueError as e:
    print("Ошибка:", e)

print("u1.ualice:", u1.ualice) # alice@example.com
print("u2.ubob:", u2.ubob)    # bob@example.com
print("User.directory:", User.directory)
```

7. Написать программу на Python, которая создает класс 'Product' с использованием метода '__new__' для контроля именования. Имена должны начинаться с "prod_". Метод '__init__' должен быть пустым.

Инструкции:

- (a) Создайте класс 'Product'.
- (b) Добавьте атрибут класса 'warehouse' и инициализируйте его пустым словарем.
- (c) Переопределите метод '__new__', принимающий 'cls', 'name', 'price'.
- (d) В '__new__': если 'name' не начинается с "prod_" выбросьте 'ValueError("Продукт должен иметь префикс 'prod_')'.
- (e) Извлеките ID продукта: 'prod_id = name[5:]'.
- (f) Создайте экземпляр: 'instance = super().__new__(cls)'.
- (g) Добавьте ID в словарь: 'cls.warehouse[prod_id] = instance'.
- (h) Установите атрибут экземпляра: 'setattr(instance, f"pprod_id price)'.
- (i) Верните 'instance'.
- (j) Переопределите метод '__init__' как пустой.
- (k) Создайте объект 'p1' с именем "prod_laptop" и ценой 999.
- (l) Создайте объект 'p2' с именем "prod_mouse" и ценой 25.
- (m) Попытайтесь создать объект с именем "item_keyboard"— поймайте исключение.
- (n) Выведите 'p1.plaptop' и 'p2.pmouse'.
- (o) Выведите 'Product.warehouse'.

Пример использования:

```
p1 = Product("prod_laptop", 999)
p2 = Product("prod_mouse", 25)

try:
    p3 = Product("item_keyboard", 50)
except ValueError as e:
    print("Ошибка:", e)

print("p1.plaptop:", p1.plaptop) # 999
print("p2.pmouse:", p2.pmouse)   # 25
print("Product.warehouse:", Product.warehouse)
```

8. Написать программу на Python, которая создает класс 'Employee' с использованием метода '__new__' для контроля именования. Имена должны начинаться с "emp_". Метод '__init__' должен быть пустым.

Инструкции:

- (a) Создайте класс 'Employee'.
- (b) Добавьте атрибут класса 'staff' и инициализируйте его пустым словарем.
- (c) Переопределите метод '__new__'; принимающий 'cls', 'name', 'department'.
- (d) В '__new__': если 'name' не начинается с "emp_" выбросьте 'ValueError("Сотрудник должен иметь префикс 'emp_')'.
- (e) Извлеките ID сотрудника: 'emp_id = name[4:]'.
- (f) Создайте экземпляр: 'instance = super().__new__(cls)'.
- (g) Добавьте ID в словарь: 'cls.staff[emp_id] = instance'.
- (h) Установите атрибут экземпляра: 'setattr(instance, f"emp_{emp_id} department")'.
- (i) Верните 'instance'.
- (j) Переопределите метод '__init__' как пустой.
- (k) Создайте объект 'e1' с именем "emp_001" и отделом "IT".
- (l) Создайте объект 'e2' с именем "emp_002" и отделом "HR".
- (m) Попробуйте создать объект с именем "worker_003"— поймите исключение.
- (n) Выведите 'e1.e001' и 'e2.e002'.
- (o) Выведите 'Employee.staff'.

Пример использования:

```
e1 = Employee("emp_001", "IT")
e2 = Employee("emp_002", "HR")

try:
    e3 = Employee("worker_003", "Sales")
except ValueError as e:
    print("Ошибка:", e)

print("e1.e001:", e1.e001) # IT
print("e2.e002:", e2.e002) # HR
print("Employee.staff:", Employee.staff)
```

9. Написать программу на Python, которая создает класс 'Order' с использованием метода '__new__' для контроля именования. Имена должны начинаться с "order_". Метод '__init__' должен быть пустым.

Инструкции:

- (a) Создайте класс 'Order'.
- (b) Добавьте атрибут класса 'ledger' и инициализируйте его пустым словарем.
- (c) Переопределите метод '__new__'; принимающий 'cls', 'name', 'total'.
- (d) В '__new__': если 'name' не начинается с "order_" выбросьте 'ValueError("Заказ должен иметь префикс 'order_')'.
- (e) Извлеките ID заказа: 'order_id = name[6:]'.

- (f) Создайте экземпляр: `instance = super().__new__(cls)`.
- (g) Добавьте ID в словарь: `cls.ledger[order_id] = instance`.
- (h) Установите атрибут экземпляра: `setattr(instance, f"o{order_id} total")`.
- (i) Верните `instance`.
- (j) Переопределите метод `__init__` как пустой.
- (k) Создайте объект `'o1'` с именем `"order_1001"` и суммой `150.0`.
- (l) Создайте объект `'o2'` с именем `"order_1002"` и суммой `89.99`.
- (m) Попробуйте создать объект с именем `"purchase_1003"` — поймите исключение.
- (n) Выведите `'o1.o1001'` и `'o2.o1002'`.
- (o) Выведите `'Order.ledger'`.

Пример использования:

```
o1 = Order("order_1001", 150.0)
o2 = Order("order_1002", 89.99)

try:
    o3 = Order("purchase_1003", 200.0)
except ValueError as e:
    print("Ошибка:", e)

print("o1.o1001:", o1.o1001) # 150.0
print("o2.o1002:", o2.o1002) # 89.99
print("Order.ledger:", Order.ledger)
```

10. Написать программу на Python, которая создает класс `'Ticket'` с использованием метода `__new__` для контроля именования. Имена должны начинаться с `"ticket_"`. Метод `__init__` должен быть пустым.

Инструкции:

- (a) Создайте класс `'Ticket'`.
- (b) Добавьте атрибут класса `'database'` и инициализируйте его пустым словарем.
- (c) Переопределите метод `__new__`, принимающий `'cls'`, `'name'`, `'priority'`.
- (d) В `__new__`: если `'name'` не начинается с `"ticket_"` выбросьте `ValueError("Тикет должен иметь префикс 'ticket_'")`.
- (e) Извлеките ID тикета: `ticket_id = name[7:]`.
- (f) Создайте экземпляр: `instance = super().__new__(cls)`.
- (g) Добавьте ID в словарь: `cls.database[ticket_id] = instance`.
- (h) Установите атрибут экземпляра: `setattr(instance, f"t{ticket_id} priority")`.
- (i) Верните `instance`.
- (j) Переопределите метод `__init__` как пустой.
- (k) Создайте объект `'t1'` с именем `"ticket_001"` и приоритетом `"High"`.
- (l) Создайте объект `'t2'` с именем `"ticket_002"` и приоритетом `"Low"`.
- (m) Попробуйте создать объект с именем `"issue_003"` — поймите исключение.
- (n) Выведите `'t1.t001'` и `'t2.t002'`.

(o) Выведите `Ticket.database`.

Пример использования:

```
t1 = Ticket("ticket_001", "High")
t2 = Ticket("ticket_002", "Low")

try:
    t3 = Ticket("issue_003", "Medium")
except ValueError as e:
    print("Ошибка:", e)

print("t1.t001:", t1.t001) # High
print("t2.t002:", t2.t002) # Low
print("Ticket.database:", Ticket.database)
```

11. Написать программу на Python, которая создает класс `Project` с использованием метода `__new__` для контроля именования. Имена должны начинаться с `"proj_"`. Метод `__init__` должен быть пустым.

Инструкции:

- (a) Создайте класс `Project`.
- (b) Добавьте атрибут класса `portfolio` и инициализируйте его пустым словарем.
- (c) Переопределите метод `__new__`, принимающий `cls`, `name`, `status`.
- (d) В `__new__`: если `name` не начинается с `"proj_"` выбросьте `ValueError("Проект должен иметь префикс 'proj_')"`.
- (e) Извлеките ID проекта: `proj_id = name[5:]`.
- (f) Создайте экземпляр: `instance = super().__new__(cls)`.
- (g) Добавьте ID в словарь: `cls.portfolio[proj_id] = instance`.
- (h) Установите атрибут экземпляра: `setattr(instance, f"prproj_id status")`.
- (i) Верните `instance`.
- (j) Переопределите метод `__init__` как пустой.
- (k) Создайте объект `pr1` с именем `"proj_alpha"` и статусом `"Active"`.
- (l) Создайте объект `pr2` с именем `"proj_beta"` и статусом `"Inactive"`.
- (m) Попытайтесь создать объект с именем `"task_gamma"`— поймайте исключение.
- (n) Выведите `pr1.pralpha` и `pr2.prbeta`.
- (o) Выведите `Project.portfolio`.

Пример использования:

```
pr1 = Project("proj_alpha", "Active")
pr2 = Project("proj_beta", "Inactive")

try:
    pr3 = Project("task_gamma", "Pending")
except ValueError as e:
    print("Ошибка:", e)

print("pr1.pralpha:", pr1.pralpha) # Active
print("pr2.prbeta:", pr2.prbeta)   # Inactive
print("Project.portfolio:", Project.portfolio)
```

12. Написать программу на Python, которая создает класс 'Sensor' с использованием метода '__new__' для контроля именования. Имена должны начинаться с "sensor_". Метод '__init__' должен быть пустым.

Инструкции:

- (a) Создайте класс 'Sensor'.
- (b) Добавьте атрибут класса 'registry' и инициализируйте его пустым словарем.
- (c) Переопределите метод '__new__'; принимающий 'cls', 'name', 'type'.
- (d) В '__new__': если 'name' не начинается с "sensor_" выбросьте 'ValueError("Сенсор должен иметь префикс 'sensor_')'.
- (e) Извлеките ID сенсора: 'sensor_id = name[7:]'.
- (f) Создайте экземпляр: 'instance = super().__new__(cls)'.
- (g) Добавьте ID в словарь: 'cls.registry[sensor_id] = instance'.
- (h) Установите атрибут экземпляра: 'setattr(instance, f"sensor_id type")'.
- (i) Верните 'instance'.
- (j) Переопределите метод '__init__' как пустой.
- (k) Создайте объект 's1' с именем "sensor_temp" и типом "Temperature".
- (l) Создайте объект 's2' с именем "sensor_humid" и типом "Humidity".
- (m) Попытайтесь создать объект с именем "device_press"— поймите исключение.
- (n) Выведите 's1.stemp' и 's2.shumid'.
- (o) Выведите 'Sensor.registry'.

Пример использования:

```
s1 = Sensor("sensor_temp", "Temperature")
s2 = Sensor("sensor_humid", "Humidity")

try:
    s3 = Sensor("device_press", "Pressure")
except ValueError as e:
    print("Ошибка:", e)

print("s1.stemp:", s1.stemp)      # Temperature
print("s2.shumid:", s2.shumid)   # Humidity
print("Sensor.registry:", Sensor.registry)
```

13. Написать программу на Python, которая создает класс 'Vehicle' с использованием метода '__new__' для контроля именования. Имена должны начинаться с "veh_". Метод '__init__' должен быть пустым.

Инструкции:

- (a) Создайте класс 'Vehicle'.
- (b) Добавьте атрибут класса 'fleet' и инициализируйте его пустым словарем.
- (c) Переопределите метод '__new__'; принимающий 'cls', 'name', 'model'.
- (d) В '__new__': если 'name' не начинается с "veh_" выбросьте 'ValueError("Транспортное средство должно иметь префикс 'veh_')'.
- (e) Извлеките ID транспорта: 'veh_id = name[4:]'.

- (f) Создайте экземпляр: `instance = super().__new__(cls)`.
- (g) Добавьте ID в словарь: `cls.fleet[veh_id] = instance`.
- (h) Установите атрибут экземпляра: `setattr(instance, f"vveh_id model")`.
- (i) Верните `instance`.
- (j) Переопределите метод `__init__` как пустой.
- (k) Создайте объект `v1` с именем `"veh_car1"` и моделью `"Sedan"`.
- (l) Создайте объект `v2` с именем `"veh_truck1"` и моделью `"Pickup"`.
- (m) Попробуйте создать объект с именем `"bike_01"` — поймите исключение.
- (n) Выведите `v1.vcar1` и `v2.vtruck1`.
- (o) Выведите `Vehicle.fleet`.

Пример использования:

```
v1 = Vehicle("veh_car1", "Sedan")
v2 = Vehicle("veh_truck1", "Pickup")

try:
    v3 = Vehicle("bike_01", "Mountain")
except ValueError as e:
    print("Ошибка:", e)

print("v1.vcar1:", v1.vcar1)      # Sedan
print("v2.vtruck1:", v2.vtruck1)  # Pickup
print("Vehicle.fleet:", Vehicle.fleet)
```

14. Написать программу на Python, которая создает класс `Animal` с использованием метода `__new__` для контроля именования. Имена должны начинаться с `"animal_"`. Метод `__init__` должен быть пустым.

Инструкции:

- (a) Создайте класс `Animal`.
- (b) Добавьте атрибут класса `zoo` и инициализируйте его пустым словарем.
- (c) Переопределите метод `__new__`, принимающий `cls`, `name`, `species`.
- (d) В `__new__`: если `name` не начинается с `"animal_"` выбросьте `ValueError("Животное должно иметь префикс 'animal_'")`.
- (e) Извлеките ID животного: `animal_id = name[7:]`.
- (f) Создайте экземпляр: `instance = super().__new__(cls)`.
- (g) Добавьте ID в словарь: `cls.zoo[animal_id] = instance`.
- (h) Установите атрибут экземпляра: `setattr(instance, f"animal_id species")`.
- (i) Верните `instance`.
- (j) Переопределите метод `__init__` как пустой.
- (k) Создайте объект `a1` с именем `"animal_lion"` и видом `"Panthera leo"`.
- (l) Создайте объект `a2` с именем `"animal_elephant"` и видом `"Loxodonta"`.
- (m) Попробуйте создать объект с именем `"creature_tiger"` — поймите исключение.
- (n) Выведите `a1.alion` и `a2.aelephant`.

(о) Выведите 'Animal.zoo'.

Пример использования:

```
a1 = Animal("animal_lion", "Panthera leo")
a2 = Animal("animal_elephant", "Loxodonta")

try:
    a3 = Animal("creature_tiger", "Panthera tigris")
except ValueError as e:
    print("Ошибка:", e)

print("a1.alion:", a1.alion)          # Panthera leo
print("a2.aelephant:", a2.aelephant) # Loxodonta
print("Animal.zoo:", Animal.zoo)
```

15. Написать программу на Python, которая создает класс 'Plant' с использованием метода '__new__' для контроля именования. Имена должны начинаться с "plant_". Метод '__init__' должен быть пустым.

Инструкции:

- (a) Создайте класс 'Plant'.
- (b) Добавьте атрибут класса 'greenhouse' и инициализируйте его пустым словарем.
- (c) Переопределите метод '__new__', принимающий 'cls', 'name', 'family'.
- (d) В '__new__': если 'name' не начинается с "plant_" выбросьте 'ValueError("Растение должно иметь префикс 'plant_')'.
- (e) Извлеките ID растения: 'plant_id = name[6:]'.
- (f) Создайте экземпляр: 'instance = super().__new__(cls)'.
- (g) Добавьте ID в словарь: 'cls.greenhouse[plant_id] = instance'.
- (h) Установите атрибут экземпляра: 'setattr(instance, f"pl{plant_id} family")'.
- (i) Верните 'instance'.
- (j) Переопределите метод '__init__' как пустой.
- (k) Создайте объект 'pl1' с именем "plant_rose" и семейством "Rosaceae".
- (l) Создайте объект 'pl2' с именем "plant_oak" и семейством "Fagaceae".
- (m) Попытайтесь создать объект с именем "tree_pine"— поймайте исключение.
- (n) Выведите 'pl1.plrose' и 'pl2.ploak'.
- (о) Выведите 'Plant.greenhouse'.

Пример использования:

```
pl1 = Plant("plant_rose", "Rosaceae")
pl2 = Plant("plant_oak", "Fagaceae")

try:
    pl3 = Plant("tree_pine", "Pinaceae")
except ValueError as e:
    print("Ошибка:", e)

print("pl1.plrose:", pl1.plrose) # Rosaceae
print("pl2.ploak:", pl2.ploak)   # Fagaceae
print("Plant.greenhouse:", Plant.greenhouse)
```

16. Написать программу на Python, которая создает класс 'Planet' с использованием метода '__new__' для контроля именования. Имена должны начинаться с "planet_". Метод '__init__' должен быть пустым.

Инструкции:

- (a) Создайте класс 'Planet'.
- (b) Добавьте атрибут класса 'solar_system' и инициализируйте его пустым словарем.
- (c) Переопределите метод '__new__', принимающий 'cls', 'name', 'type'.
- (d) В '__new__': если 'name' не начинается с "planet_" выбросьте 'ValueError("Планета должна иметь префикс 'planet_')'.
- (e) Извлеките ID планеты: 'planet_id = name[7:]'.
- (f) Создайте экземпляр: 'instance = super().__new__(cls)'.
- (g) Добавьте ID в словарь: 'cls.solar_system[planet_id] = instance'.
- (h) Установите атрибут экземпляра: 'setattr(instance, f"pnplanet_id type)'.
- (i) Верните 'instance'.
- (j) Переопределите метод '__init__' как пустой.
- (k) Создайте объект 'pn1' с именем "planet_earth" и типом "Terrestrial".
- (l) Создайте объект 'pn2' с именем "planet_jupiter" и типом "Gas Giant".
- (m) Попробуйте создать объект с именем "star_sun"— поймайте исключение.
- (n) Выведите 'pn1.pnearth' и 'pn2.pnjupiter'.
- (o) Выведите 'Planet.solar_system'.

Пример использования:

```
pn1 = Planet("planet_earth", "Terrestrial")
pn2 = Planet("planet_jupiter", "Gas Giant")

try:
    pn3 = Planet("star_sun", "Star")
except ValueError as e:
    print("Ошибка:", e)

print("pn1.pnearth:", pn1.pnearth)      # Terrestrial
print("pn2.pnjupiter:", pn2.pnjupiter) # Gas Giant
print("Planet.solar_system:", Planet.solar_system)
```

17. Написать программу на Python, которая создает класс 'Star' с использованием метода '__new__' для контроля именования. Имена должны начинаться с "star_". Метод '__init__' должен быть пустым.

Инструкции:

- (a) Создайте класс 'Star'.
- (b) Добавьте атрибут класса 'galaxy' и инициализируйте его пустым словарем.
- (c) Переопределите метод '__new__', принимающий 'cls', 'name', 'class_type'.
- (d) В '__new__': если 'name' не начинается с "star_" выбросьте 'ValueError("Звезда должна иметь префикс 'star_')'.
- (e) Извлеките ID звезды: 'star_id = name[5:]'.

- (f) Создайте экземпляр: `instance = super().__new__(cls)`.
- (g) Добавьте ID в словарь: `cls.galaxy[star_id] = instance`.
- (h) Установите атрибут экземпляра: `setattr(instance, f"star_id class_type")`.
- (i) Верните `instance`.
- (j) Переопределите метод `__init__` как пустой.
- (k) Создайте объект `st1` с именем `"star_sol"` и классом `"G2V"`.
- (l) Создайте объект `st2` с именем `"star_proxima"` и классом `"M5.5V"`.
- (m) Попробуйте создать объект с именем `"nova_1"` — поймите исключение.
- (n) Выведите `st1.stsol` и `st2.stproxima`.
- (o) Выведите `Star.galaxy`.

Пример использования:

```
st1 = Star("star_sol", "G2V")
st2 = Star("star_proxima", "M5.5V")

try:
    st3 = Star("nova_1", "Variable")
except ValueError as e:
    print("Ошибка:", e)

print("st1.stsol:", st1.stsol)          # G2V
print("st2.stproxima:", st2.stproxima) # M5.5V
print("Star.galaxy:", Star.galaxy)
```

18. Написать программу на Python, которая создает класс `'Galaxy'` с использованием метода `__new__` для контроля именования. Имена должны начинаться с `"galaxy_"`. Метод `__init__` должен быть пустым.

Инструкции:

- (a) Создайте класс `'Galaxy'`.
- (b) Добавьте атрибут класса `'universe'` и инициализируйте его пустым словарем.
- (c) Переопределите метод `__new__`, принимающий `'cls'`, `'name'`, `'type'`.
- (d) В `__new__`: если `'name'` не начинается с `"galaxy_"` выбросьте `ValueError("Галактика должна иметь префикс 'galaxy_'")`.
- (e) Извлеките ID галактики: `galaxy_id = name[7:]`.
- (f) Создайте экземпляр: `instance = super().__new__(cls)`.
- (g) Добавьте ID в словарь: `cls.universe[galaxy_id] = instance`.
- (h) Установите атрибут экземпляра: `setattr(instance, f"galaxy_id type")`.
- (i) Верните `instance`.
- (j) Переопределите метод `__init__` как пустой.
- (k) Создайте объект `'g1'` с именем `"galaxy_milkyway"` и типом `"Spiral"`.
- (l) Создайте объект `'g2'` с именем `"galaxy_andromeda"` и типом `"Spiral"`.
- (m) Попробуйте создать объект с именем `"cluster_virgo"` — поймите исключение.
- (n) Выведите `g1.gmilkyway` и `g2.gandromeda`.

(o) Выведите 'Galaxy.universe'.

Пример использования:

```
g1 = Galaxy("galaxy_milkyway", "Spiral")
g2 = Galaxy("galaxy_andromeda", "Spiral")

try:
    g3 = Galaxy("cluster_virgo", "Cluster")
except ValueError as e:
    print("Ошибка:", e)

print("g1.gmilkyway:", g1.gmilkyway)    # Spiral
print("g2.gandromeda:", g2.gandromeda)  # Spiral
print("Galaxy.universe:", Galaxy.universe)
```

19. Написать программу на Python, которая создает класс 'Constellation' с использованием метода '__new__' для контроля именования. Имена должны начинаться с "const_". Метод '__init__' должен быть пустым.

Инструкции:

- (a) Создайте класс 'Constellation'.
- (b) Добавьте атрибут класса 'sky_map' и инициализируйте его пустым словарем.
- (c) Переопределите метод '__new__', принимающий 'cls', 'name', 'stars'.
- (d) В '__new__': если 'name' не начинается с "const_" выбросьте 'ValueError("Созвездие должно иметь префикс 'const_')'.
- (e) Извлеките ID созвездия: 'const_id = name[6:]'.
- (f) Создайте экземпляр: 'instance = super().__new__(cls)'.
- (g) Добавьте ID в словарь: 'cls.sky_map[const_id] = instance'.
- (h) Установите атрибут экземпляра: 'setattr(instance, f"const_id stars)'.
- (i) Верните 'instance'.
- (j) Переопределите метод '__init__' как пустой.
- (k) Создайте объект 'c1' с именем "const_orion" и количеством звезд 81.
- (l) Создайте объект 'c2' с именем "const_ursa" и количеством звезд 20.
- (m) Попытайтесь создать объект с именем "asterism_bigdipper"— поймите исключение.
- (n) Выведите 'c1.corion' и 'c2.cursa'.
- (o) Выведите 'Constellation.sky_map'.

Пример использования:

```
c1 = Constellation("const_orion", 81)
c2 = Constellation("const_ursa", 20)

try:
    c3 = Constellation("asterism_bigdipper", 7)
except ValueError as e:
    print("Ошибка:", e)

print("c1.corion:", c1.corion)    # 81
print("c2.cursa:", c2.cursa)     # 20
print("Constellation.sky_map:", Constellation.sky_map)
```

20. Написать программу на Python, которая создает класс 'Asteroid' с использованием метода '__new__' для контроля именования. Имена должны начинаться с "ast_". Метод '__init__' должен быть пустым.

Инструкции:

- (a) Создайте класс 'Asteroid'.
- (b) Добавьте атрибут класса 'belt' и инициализируйте его пустым словарем.
- (c) Переопределите метод '__new__', принимающий 'cls', 'name', 'diameter'.
- (d) В '__new__': если 'name' не начинается с "ast_" выбросьте 'ValueError("Астероид должен иметь префикс 'ast_')'
- (e) Извлеките ID астероида: 'ast_id = name[4:]'.
- (f) Создайте экземпляр: 'instance = super().__new__(cls)'.
- (g) Добавьте ID в словарь: 'cls.belt[ast_id] = instance'.
- (h) Установите атрибут экземпляра: 'setattr(instance, f"aast_id diameter)'.
- (i) Верните 'instance'.
- (j) Переопределите метод '__init__' как пустой.
- (k) Создайте объект 'a1' с именем "ast_ceres" и диаметром 939.
- (l) Создайте объект 'a2' с именем "ast_vesta" и диаметром 525.
- (m) Попытайтесь создать объект с именем "meteor_id8"— поймите исключение.
- (n) Выведите 'a1.aceres' и 'a2.avesta'.
- (o) Выведите 'Asteroid.belt'.

Пример использования:

```
a1 = Asteroid("ast_ceres", 939)
a2 = Asteroid("ast_vesta", 525)

try:
    a3 = Asteroid("meteor_id8", 10)
except ValueError as e:
    print("Ошибка:", e)

print("a1.aceres:", a1.aceres) # 939
print("a2.avesta:", a2.avesta) # 525
print("Asteroid.belt:", Asteroid.belt)
```

21. Написать программу на Python, которая создает класс 'Comet' с использованием метода '__new__' для контроля именования. Имена должны начинаться с "comet_". Метод '__init__' должен быть пустым.

Инструкции:

- (a) Создайте класс 'Comet'.
- (b) Добавьте атрибут класса 'orbits' и инициализируйте его пустым словарем.
- (c) Переопределите метод '__new__', принимающий 'cls', 'name', 'period'.
- (d) В '__new__': если 'name' не начинается с "comet_" выбросьте 'ValueError("Комета должна иметь префикс 'comet_')'
- (e) Извлеките ID кометы: 'comet_id = name[6:]'.

- (f) Создайте экземпляр: `instance = super().__new__(cls)`.
- (g) Добавьте ID в словарь: `cls.orbits[comet_id] = instance`.
- (h) Установите атрибут экземпляра: `setattr(instance, f"cmcomet_id period")`.
- (i) Верните `instance`.
- (j) Переопределите метод `__init__` как пустой.
- (k) Создайте объект `cm1` с именем `"comet_halley"` и периодом 76.
- (l) Создайте объект `cm2` с именем `"comet_encke"` и периодом 3.3.
- (m) Попробуйте создать объект с именем `"meteor_shower"` — поймайте исключение.
- (n) Выведите `cm1.cmhalley` и `cm2.cmencke`.
- (o) Выведите `Comet.orbits`.

Пример использования:

```
cm1 = Comet("comet_halley", 76)
cm2 = Comet("comet_encke", 3.3)

try:
    cm3 = Comet("meteor_shower", 0.1)
except ValueError as e:
    print("Ошибка:", e)

print("cm1.cmhalley:", cm1.cmhalley) # 76
print("cm2.cmencke:", cm2.cmencke)   # 3.3
print("Comet.orbits:", Comet.orbits)
```

22. Написать программу на Python, которая создает класс `Satellite` с использованием метода `__new__` для контроля именования. Имена должны начинаться с `"sat_"`. Метод `__init__` должен быть пустым.

Инструкции:

- (a) Создайте класс `Satellite`.
- (b) Добавьте атрибут класса `orbiters` и инициализируйте его пустым словарем.
- (c) Переопределите метод `__new__`, принимающий `cls`, `name`, `planet`.
- (d) В `__new__`: если `name` не начинается с `"sat_"` выбросьте `ValueError("Спутник должен иметь префикс 'sat_'")`.
- (e) Извлеките ID спутника: `sat_id = name[4:]`.
- (f) Создайте экземпляр: `instance = super().__new__(cls)`.
- (g) Добавьте ID в словарь: `cls.orbiters[sat_id] = instance`.
- (h) Установите атрибут экземпляра: `setattr(instance, f"ssat_id planet")`.
- (i) Верните `instance`.
- (j) Переопределите метод `__init__` как пустой.
- (k) Создайте объект `s1` с именем `"sat_moon"` и планетой `"Earth"`.
- (l) Создайте объект `s2` с именем `"sat_phobos"` и планетой `"Mars"`.
- (m) Попробуйте создать объект с именем `"rover_curiosity"` — поймайте исключение.
- (n) Выведите `s1.smoon` и `s2.sphobos`.

(o) Выведите 'Satellite.orbiters'.

Пример использования:

```
s1 = Satellite("sat_moon", "Earth")
s2 = Satellite("sat_phobos", "Mars")

try:
    s3 = Satellite("rover_curiosity", "Mars")
except ValueError as e:
    print("Ошибка:", e)

print("s1.smoon:", s1.smoon)      # Earth
print("s2.sphobos:", s2.sphobos)  # Mars
print("Satellite.orbiters:", Satellite.orbiters)
```

23. Написать программу на Python, которая создает класс 'Rocket' с использованием метода '__new__' для контроля именования. Имена должны начинаться с "rocket_". Метод '__init__' должен быть пустым.

Инструкции:

- (a) Создайте класс 'Rocket'.
- (b) Добавьте атрибут класса 'launchpad' и инициализируйте его пустым словарем.
- (c) Переопределите метод '__new__', принимающий 'cls', 'name', 'payload'.
- (d) В '__new__': если 'name' не начинается с "rocket_" выбросьте 'ValueError("Ракета должна иметь префикс 'rocket_')'
- (e) Извлеките ID ракеты: 'rocket_id = name[7:]'.
- (f) Создайте экземпляр: 'instance = super().__new__(cls)'.
- (g) Добавьте ID в словарь: 'cls.launchpad[rocket_id] = instance'.
- (h) Установите атрибут экземпляра: 'setattr(instance, f"rocket_id payload')'.
- (i) Верните 'instance'.
- (j) Переопределите метод '__init__' как пустой.
- (k) Создайте объект 'r1' с именем "rocket_falcon9" и полезной нагрузкой "Starlink".
- (l) Создайте объект 'r2' с именем "rocket_atlas" и полезной нагрузкой "GPS".
- (m) Попытайтесь создать объект с именем "drone_delivery" — поймайте исключение.
- (n) Выведите 'r1.rfalcon9' и 'r2.ratlas'.
- (o) Выведите 'Rocket.launchpad'.

Пример использования:

```
r1 = Rocket("rocket_falcon9", "Starlink")
r2 = Rocket("rocket_atlas", "GPS")

try:
    r3 = Rocket("drone_delivery", "Package")
except ValueError as e:
    print("Ошибка:", e)

print("r1.rfalcon9:", r1.rfalcon9)  # Starlink
print("r2.ratlas:", r2.ratlas)      # GPS
print("Rocket.launchpad:", Rocket.launchpad)
```


24. Написать программу на Python, которая создает класс 'Drone' с использованием метода '__new__' для контроля именования. Имена должны начинаться с "drone_". Метод '__init__' должен быть пустым.

Инструкции:

- (a) Создайте класс 'Drone'.
- (b) Добавьте атрибут класса 'fleet' и инициализируйте его пустым словарем.
- (c) Переопределите метод '__new__', принимающий 'cls', 'name', 'range'.
- (d) В '__new__': если 'name' не начинается с "drone_" выбросьте 'ValueError("Дрон должен иметь префикс 'drone_')'.
- (e) Извлеките ID дрона: 'drone_id = name[6:]'.
- (f) Создайте экземпляр: 'instance = super().__new__(cls)'.
- (g) Добавьте ID в словарь: 'cls.fleet[drone_id] = instance'.
- (h) Установите атрибут экземпляра: 'setattr(instance, f"ddrone_id range")'.
- (i) Верните 'instance'.
- (j) Переопределите метод '__init__' как пустой.
- (k) Создайте объект 'd1' с именем "drone_x1" и дальностью 5.
- (l) Создайте объект 'd2' с именем "drone_x2" и дальностью 10.
- (m) Попытайтесь создать объект с именем "robot_r1"— поймите исключение.
- (n) Выведите 'd1.dx1' и 'd2.dx2'.
- (o) Выведите 'Drone.fleet'.

Пример использования:

```
d1 = Drone("drone_x1", 5)
d2 = Drone("drone_x2", 10)

try:
    d3 = Drone("robot_r1", 2)
except ValueError as e:
    print("Ошибка:", e)

print("d1.dx1:", d1.dx1) # 5
print("d2.dx2:", d2.dx2) # 10
print("Drone.fleet:", Drone.fleet)
```

25. Написать программу на Python, которая создает класс 'Robot' с использованием метода '__new__' для контроля именования. Имена должны начинаться с "robot_". Метод '__init__' должен быть пустым.

Инструкции:

- (a) Создайте класс 'Robot'.
- (b) Добавьте атрибут класса 'factory' и инициализируйте его пустым словарем.
- (c) Переопределите метод '__new__', принимающий 'cls', 'name', 'function'.
- (d) В '__new__': если 'name' не начинается с "robot_" выбросьте 'ValueError("Робот должен иметь префикс 'robot_')'.
- (e) Извлеките ID робота: 'robot_id = name[6:]'.

- (f) Создайте экземпляр: `instance = super().__new__(cls)`.
- (g) Добавьте ID в словарь: `cls.factory[robot_id] = instance`.
- (h) Установите атрибут экземпляра: `setattr(instance, f"r{robot_id}function")`.
- (i) Верните `instance`.
- (j) Переопределите метод `__init__` как пустой.
- (k) Создайте объект `rb1` с именем `"robot_arm"` и функцией `"Assembly"`.
- (l) Создайте объект `rb2` с именем `"robot_cleaner"` и функцией `"Cleaning"`.
- (m) Попробуйте создать объект с именем `"android_unit"` — поймите исключение.
- (n) Выведите `rb1.rbarm` и `rb2.rbcleaner`.
- (o) Выведите `Robot.factory`.

Пример использования:

```
rb1 = Robot("robot_arm", "Assembly")
rb2 = Robot("robot_cleaner", "Cleaning")

try:
    rb3 = Robot("android_unit", "General")
except ValueError as e:
    print("Ошибка:", e)

print("rb1.rbarm:", rb1.rbarm)          # Assembly
print("rb2.rbcleaner:", rb2.rbcleaner)  # Cleaning
print("Robot.factory:", Robot.factory)
```

26. Написать программу на Python, которая создает класс `AI` с использованием метода `__new__` для контроля именования. Имена должны начинаться с `"ai_"`. Метод `__init__` должен быть пустым.

Инструкции:

- (a) Создайте класс `AI`.
- (b) Добавьте атрибут класса `network` и инициализируйте его пустым словарем.
- (c) Переопределите метод `__new__`, принимающий `cls`, `name`, `capability`.
- (d) В `__new__`: если `name` не начинается с `"ai_"` выбросьте `ValueError("ИИ должен иметь префикс 'ai_'")`.
- (e) Извлеките ID ИИ: `ai_id = name[3:]`.
- (f) Создайте экземпляр: `instance = super().__new__(cls)`.
- (g) Добавьте ID в словарь: `cls.network[ai_id] = instance`.
- (h) Установите атрибут экземпляра: `setattr(instance, f"ai_{ai_id}capability")`.
- (i) Верните `instance`.
- (j) Переопределите метод `__init__` как пустой.
- (k) Создайте объект `a1` с именем `"ai_alpha"` и возможностью `"NLP"`.
- (l) Создайте объект `a2` с именем `"ai_beta"` и возможностью `"CV"`.
- (m) Попробуйте создать объект с именем `"ml_model"` — поймите исключение.
- (n) Выведите `a1.aalpha` и `a2.abeta`.

(o) Выведите 'AI.network'.

Пример использования:

```
a1 = AI("ai_alpha", "NLP")
a2 = AI("ai_beta", "CV")

try:
    a3 = AI("ml_model", "Regression")
except ValueError as e:
    print("Ошибка:", e)

print("a1.aalpha:", a1.aalpha) # NLP
print("a2.abeta:", a2.abeta)   # CV
print("AI.network:", AI.network)
```

27. Написать программу на Python, которая создает класс 'MLModel' с использованием метода '__new__' для контроля именования. Имена должны начинаться с "model_". Метод '__init__' должен быть пустым.

Инструкции:

- (a) Создайте класс 'MLModel'.
- (b) Добавьте атрибут класса 'repository' и инициализируйте его пустым словарем.
- (c) Переопределите метод '__new__', принимающий 'cls', 'name', 'algorithm'.
- (d) В '__new__': если 'name' не начинается с "model_" выбросьте 'ValueError("Модель должна иметь префикс 'model_')'.
- (e) Извлеките ID модели: 'model_id = name[6:]'.
- (f) Создайте экземпляр: 'instance = super().__new__(cls)'.
- (g) Добавьте ID в словарь: 'cls.repository[model_id] = instance'.
- (h) Установите атрибут экземпляра: 'setattr(instance, f"model_id_algorithm')'.
- (i) Верните 'instance'.
- (j) Переопределите метод '__init__' как пустой.
- (k) Создайте объект 'm1' с именем "model_logreg" и алгоритмом "Logistic Regression".
- (l) Создайте объект 'm2' с именем "model_svm" и алгоритмом "Support Vector Machine".
- (m) Попытайтесь создать объект с именем "algo_randomforest"— поймите исключение.
- (n) Выведите 'm1.mlogreg' и 'm2.msvm'.
- (o) Выведите 'MLModel.repository'.

Пример использования:

```
m1 = MLModel("model_logreg", "Logistic Regression")
m2 = MLModel("model_svm", "Support Vector Machine")

try:
    m3 = MLModel("algo_randomforest", "Random Forest")
except ValueError as e:
    print("Ошибка:", e)

print("m1.mlogreg:", m1.mlogreg) # Logistic Regression
print("m2.msvm:", m2.msvm)       # Support Vector Machine
print("MLModel.repository:", MLModel.repository)
```

28. Написать программу на Python, которая создает класс 'Dataset' с использованием метода '__new__' для контроля именования. Имена должны начинаться с "dataset_". Метод '__init__' должен быть пустым.

Инструкции:

- (a) Создайте класс 'Dataset'.
- (b) Добавьте атрибут класса 'catalog' и инициализируйте его пустым словарем.
- (c) Переопределите метод '__new__'; принимающий 'cls', 'name', 'size'.
- (d) В '__new__': если 'name' не начинается с "dataset_" выбросьте 'ValueError("Набор данных должен иметь префикс 'dataset_')'.
- (e) Извлеките ID набора данных: 'dataset_id = name[8:]'.
- (f) Создайте экземпляр: 'instance = super().__new__(cls)'.
- (g) Добавьте ID в словарь: 'cls.catalog[dataset_id] = instance'.
- (h) Установите атрибут экземпляра: 'setattr(instance, f"dsdataset_id size)'.
- (i) Верните 'instance'.
- (j) Переопределите метод '__init__' как пустой.
- (k) Создайте объект 'ds1' с именем "dataset_train" и размером 10000.
- (l) Создайте объект 'ds2' с именем "dataset_test" и размером 2000.
- (m) Попробуйте создать объект с именем "data_validation"— поймайте исключение.
- (n) Выведите 'ds1.dstrain' и 'ds2.dstest'.
- (o) Выведите 'Dataset.catalog'.

Пример использования:

```
ds1 = Dataset("dataset_train", 10000)
ds2 = Dataset("dataset_test", 2000)

try:
    ds3 = Dataset("data_validation", 2000)
except ValueError as e:
    print("Ошибка:", e)

print("ds1.dstrain:", ds1.dstrain) # 10000
print("ds2.dstest:", ds2.dstest) # 2000
print("Dataset.catalog:", Dataset.catalog)
```

29. Написать программу на Python, которая создает класс 'Feature' с использованием метода '__new__' для контроля именования. Имена должны начинаться с "feat_". Метод '__init__' должен быть пустым.

Инструкции:

- (a) Создайте класс 'Feature'.
- (b) Добавьте атрибут класса 'registry' и инициализируйте его пустым словарем.
- (c) Переопределите метод '__new__'; принимающий 'cls', 'name', 'type'.
- (d) В '__new__': если 'name' не начинается с "feat_" выбросьте 'ValueError("Признак должен иметь префикс 'feat_')'.
- (e) Извлеките ID признака: 'feat_id = name[5:]'.

- (f) Создайте экземпляр: `instance = super().__new__(cls)`.
- (g) Добавьте ID в словарь: `cls.registry[feat_id] = instance`.
- (h) Установите атрибут экземпляра: `setattr(instance, f"feat_{id} type")`.
- (i) Верните `instance`.
- (j) Переопределите метод `__init__` как пустой.
- (k) Создайте объект `f1` с именем `"feat_age"` и типом `"Numeric"`.
- (l) Создайте объект `f2` с именем `"feat_gender"` и типом `"Categorical"`.
- (m) Попробуйте создать объект с именем `"attr_income"` — поймите исключение.
- (n) Выведите `f1.fage` и `f2.fgender`.
- (o) Выведите `Feature.registry`.

Пример использования:

```
f1 = Feature("feat_age", "Numeric")
f2 = Feature("feat_gender", "Categorical")

try:
    f3 = Feature("attr_income", "Numeric")
except ValueError as e:
    print("Ошибка:", e)

print("f1.fage:", f1.fage)          # Numeric
print("f2.fgender:", f2.fgender)    # Categorical
print("Feature.registry:", Feature.registry)
```

30. Написать программу на Python, которая создает класс `Label` с использованием метода `__new__` для контроля именования. Имена должны начинаться с `"label_"`. Метод `__init__` должен быть пустым.

Инструкции:

- (a) Создайте класс `Label`.
- (b) Добавьте атрибут класса `index` и инициализируйте его пустым словарем.
- (c) Переопределите метод `__new__`, принимающий `cls`, `name`, `class_name`.
- (d) В `__new__`: если `name` не начинается с `"label_"` выбросьте `ValueError("Метка должна иметь префикс 'label_'")`.
- (e) Извлеките ID метки: `label_id = name[6:]`.
- (f) Создайте экземпляр: `instance = super().__new__(cls)`.
- (g) Добавьте ID в словарь: `cls.index[label_id] = instance`.
- (h) Установите атрибут экземпляра: `setattr(instance, f"label_{id} class_name")`.
- (i) Верните `instance`.
- (j) Переопределите метод `__init__` как пустой.
- (k) Создайте объект `l1` с именем `"label_cat"` и классом `"Animal"`.
- (l) Создайте объект `l2` с именем `"label_car"` и классом `"Vehicle"`.
- (m) Попробуйте создать объект с именем `"tag_dog"` — поймите исключение.
- (n) Выведите `l1.lcat` и `l2.lcar`.

(o) Выведите 'Label.index'.

Пример использования:

```
l1 = Label("label_cat", "Animal")
l2 = Label("label_car", "Vehicle")

try:
    l3 = Label("tag_dog", "Animal")
except ValueError as e:
    print("Ошибка:", e)

print("l1.lcat:", l1.lcat) # Animal
print("l2.lcar:", l2.lcar) # Vehicle
print("Label.index:", Label.index)
```

31. Написать программу на Python, которая создает класс 'Layer' с использованием метода '__new__' для контроля именования. Имена должны начинаться с "layer_". Метод '__init__' должен быть пустым.

Инструкции:

- (a) Создайте класс 'Layer'.
- (b) Добавьте атрибут класса 'stack' и инициализируйте его пустым словарем.
- (c) Переопределите метод '__new__', принимающий 'cls', 'name', 'neurons'.
- (d) В '__new__': если 'name' не начинается с "layer_" выбросьте 'ValueError("Слой должен иметь префикс 'layer_")'.
- (e) Извлеките ID слоя: 'layer_id = name[6:]'.
- (f) Создайте экземпляр: 'instance = super().__new__(cls)'.
- (g) Добавьте ID в словарь: 'cls.stack[layer_id] = instance'.
- (h) Установите атрибут экземпляра: 'setattr(instance, f"ly{layer_id}neurons")'.
- (i) Верните 'instance'.
- (j) Переопределите метод '__init__' как пустой.
- (k) Создайте объект 'ly1' с именем "layer_input" и нейронами 784.
- (l) Создайте объект 'ly2' с именем "layer_hidden" и нейронами 128.
- (m) Попробуйте создать объект с именем "unit_output"— поймайте исключение.
- (n) Выведите 'ly1.lyinput' и 'ly2.lyhidden'.
- (o) Выведите 'Layer.stack'.

Пример использования:

```
ly1 = Layer("layer_input", 784)
ly2 = Layer("layer_hidden", 128)

try:
    ly3 = Layer("unit_output", 10)
except ValueError as e:
    print("Ошибка:", e)

print("ly1.lyinput:", ly1.lyinput) # 784
print("ly2.lyhidden:", ly2.lyhidden) # 128
print("Layer.stack:", Layer.stack)
```

32. Написать программу на Python, которая создает класс 'Neuron' с использованием метода '__new__' для контроля именования. Имена должны начинаться с "neuron_". Метод '__init__' должен быть пустым.

Инструкции:

- (a) Создайте класс 'Neuron'.
- (b) Добавьте атрибут класса 'brain' и инициализируйте его пустым словарем.
- (c) Переопределите метод '__new__', принимающий 'cls', 'name', 'activation'.
- (d) В '__new__': если 'name' не начинается с "neuron_" выбросьте 'ValueError("Нейрон должен иметь префикс 'neuron_')'.
- (e) Извлеките ID нейрона: 'neuron_id = name[7:]'.
- (f) Создайте экземпляр: 'instance = super().__new__(cls)'.
- (g) Добавьте ID в словарь: 'cls.brain[neuron_id] = instance'.
- (h) Установите атрибут экземпляра: 'setattr(instance, f"n{neuron_id} activation")'.
- (i) Верните 'instance'.
- (j) Переопределите метод '__init__' как пустой.
- (k) Создайте объект 'n1' с именем "neuron_1" и активацией "ReLU".
- (l) Создайте объект 'n2' с именем "neuron_2" и активацией "Sigmoid".
- (m) Попытайтесь создать объект с именем "cell_3"— поймайте исключение.
- (n) Выведите 'n1.n1' и 'n2.n2'.
- (o) Выведите 'Neuron.brain'.

Пример использования:

```
n1 = Neuron("neuron_1", "ReLU")
n2 = Neuron("neuron_2", "Sigmoid")

try:
    n3 = Neuron("cell_3", "Tanh")
except ValueError as e:
    print("Ошибка:", e)

print("n1.n1:", n1.n1)    # ReLU
print("n2.n2:", n2.n2)    # Sigmoid
print("Neuron.brain:", Neuron.brain)
```

33. Написать программу на Python, которая создает класс 'Synapse' с использованием метода '__new__' для контроля именования. Имена должны начинаться с "syn_". Метод '__init__' должен быть пустым.

Инструкции:

- (a) Создайте класс 'Synapse'.
- (b) Добавьте атрибут класса 'connections' и инициализируйте его пустым словарем.
- (c) Переопределите метод '__new__', принимающий 'cls', 'name', 'weight'.
- (d) В '__new__': если 'name' не начинается с "syn_" выбросьте 'ValueError("Синапс должен иметь префикс 'syn_')'.
- (e) Извлеките ID синапса: 'syn_id = name[4:]'.

- (f) Создайте экземпляр: `instance = super().__new__(cls)`.
- (g) Добавьте ID в словарь: `cls.connections[syn_id] = instance`.
- (h) Установите атрибут экземпляра: `setattr(instance, f"syn_id weight")`.
- (i) Верните `instance`.
- (j) Переопределите метод `__init__` как пустой.
- (k) Создайте объект `'s1'` с именем `"syn_a1b1"` и весом 0.5.
- (l) Создайте объект `'s2'` с именем `"syn_a2b2"` и весом -0.3.
- (m) Попробуйте создать объект с именем `"link_x1y1"` — поймите исключение.
- (n) Выведите `'s1.sa1b1'` и `'s2.sa2b2'`.
- (o) Выведите `'Synapse.connections'`.

Пример использования:

```
s1 = Synapse("syn_a1b1", 0.5)
s2 = Synapse("syn_a2b2", -0.3)

try:
    s3 = Synapse("link_x1y1", 0.8)
except ValueError as e:
    print("Ошибка:", e)

print("s1.sa1b1:", s1.sa1b1) # 0.5
print("s2.sa2b2:", s2.sa2b2) # -0.3
print("Synapse.connections:", Synapse.connections)
```

34. Написать программу на Python, которая создает класс `'Container'` с использованием метода `__new__` для контроля именования. Имена должны начинаться с `"container_"`. Метод `__init__` должен быть пустым. Инструкции:

- (a) Создайте класс `'Container'`.
- (b) Добавьте атрибут класса `'depot'` и инициализируйте его пустым словарем.
- (c) Переопределите метод `__new__`, принимающий `'cls'`, `'name'`, `'volume'`.
- (d) В `__new__`: если `'name'` не начинается с `"container_"` выбросьте `ValueError("Контейнер должен иметь префикс 'container_')"`.
- (e) Извлеките ID контейнера: `container_id = name[9:]`.
- (f) Создайте экземпляр: `instance = super().__new__(cls)`.
- (g) Добавьте ID в словарь: `cls.depot[container_id] = instance`.
- (h) Установите атрибут экземпляра: `setattr(instance, f"cncontainer_id volume")`.
- (i) Верните `instance`.
- (j) Переопределите метод `__init__` как пустой.
- (k) Создайте объект `'cn1'` с именем `"container_20ft"` и объемом 33.
- (l) Создайте объект `'cn2'` с именем `"container_40ft"` и объемом 67.
- (m) Попробуйте создать объект с именем `"box_small"` — поймите исключение.
- (n) Выведите `'cn1.cn20ft'` и `'cn2.cn40ft'`.
- (o) Выведите `'Container.depot'`.

Пример использования:

```
cn1 = Container("container_20ft", 33)
cn2 = Container("container_40ft", 67)
try:
    cn3 = Container("box_small", 1)
except ValueError as e:
    print("Ошибка:", e)
print("cn1.cn20ft:", cn1.cn20ft) # 33
print("cn2.cn40ft:", cn2.cn40ft) # 67
print("Container.depot:", Container.depot)
```

35. Написать программу на Python, которая создает класс 'Module' с использованием метода '.__new__' для контроля именования. Имена должны начинаться с "mod_". Метод '.__init__' должен быть пустым. Инструкции:

- (a) Создайте класс 'Module'.
- (b) Добавьте атрибут класса 'system' и инициализируйте его пустым словарем.
- (c) Переопределите метод '.__new__', принимающий 'cls', 'name', 'version'.
- (d) В '.__new__': если 'name' не начинается с "mod_" выбросьте 'ValueError("Модуль должен иметь префикс 'mod_')'
- (e) Извлеките ID модуля: 'mod_id = name[4:]'.
- (f) Создайте экземпляр: 'instance = super().__new__(cls)'.
- (g) Добавьте ID в словарь: 'cls.system[mod_id] = instance'.
- (h) Установите атрибут экземпляра: 'setattr(instance, f"mdmod_id version)'.
- (i) Верните 'instance'.
- (j) Переопределите метод '.__init__' как пустой.
- (k) Создайте объект 'md1' с именем "mod_auth" и версией "1.2.0".
- (l) Создайте объект 'md2' с именем "mod_payment" и версией "3.0.1".
- (m) Попытайтесь создать объект с именем "lib_utils"— поймайте исключение.
- (n) Выведите 'md1.mdauth' и 'md2.mdpayment'.
- (o) Выведите 'Module.system'.

Пример использования:

```
md1 = Module("mod_auth", "1.2.0")
md2 = Module("mod_payment", "3.0.1")
try:
    md3 = Module("lib_utils", "0.9.5")
except ValueError as e:
    print("Ошибка:", e)
print("md1.mdauth:", md1.mdauth) # 1.2.0
print("md2.mdpayment:", md2.mdpayment) # 3.0.1
print("Module.system:", Module.system)
```

2.4.4 Задача 4

1 Написать программу на Python, которая создает класс **ShoppingCart** для представления корзины покупок. Класс должен содержать методы для добавления и удаления товаров, а также вычисления общего количества. Программа также должна создавать экземпляр класса **ShoppingCart**, добавлять товары в корзину, удалять товары из корзины и выводить информацию о корзине на экран.

- Создайте класс **ShoppingCart** с методом `__init__`, который создает пустой список товаров.
- Создайте метод `add_item`, который принимает название товара и количество в качестве аргументов и добавляет их в список товаров.
- Создайте метод `remove_item`, который удаляет товар из списка товаров по его названию.
- Создайте метод `calculate_total`, который вычисляет и возвращает общее количество всех товаров в корзине.
- Создайте экземпляр класса **ShoppingCart** и добавьте товары в корзину.
- Выведите информацию о текущих товарах в корзине на экран.
- Выведите общее количество всех товаров в корзине на экран.
- Удалите товар из корзины и выведите обновленную информацию о товарах в корзине на экран.
- Выведите общее количество всех товаров в корзине после удаления товара на экран.

Пример использования:

```
cart = ShoppingCart()
cart.add_item("Картофель", 100)
cart.add_item("Капуста", 200)
cart.add_item("Апельсин", 150)
print("Число товаров в корзине:")
for item in cart.items:
    print(item[0], "-", item[1])
total_qty = cart.calculate_total()
print("Общее количество:", total_qty)
cart.remove_item("Апельсин")
print("Обновление числа покупок в корзине после удаления апельсина:")
for item in cart.items:
    print(item[0], "-", item[1])
total_qty = cart.calculate_total()
print("Общее количество:", total_qty)
```

Вывод:

```
Число товаров в корзине:
Картофель - 100
Капуста - 200
```

Апельсин - 150
Общее количество: 450
Обновление числа покупок в корзине после удаления апельсина:
Картофель - 100
Капуста - 200
Общее количество: 300

2 Написать программу на Python, которая создает класс `BookCollection` для представления коллекции книг. Класс должен содержать методы для добавления и удаления книг, а также подсчета общего количества страниц. Программа также должна создавать экземпляр класса `BookCollection`, добавлять книги в коллекцию, удалять книги из коллекции и выводить информацию о коллекции на экран.

- Создайте класс `BookCollection` с методом `__init__`, который создает пустой список книг.
- Создайте метод `add_book`, который принимает название книги и количество страниц в качестве аргументов и добавляет их в список книг.
- Создайте метод `remove_book`, который удаляет книгу из списка по её названию.
- Создайте метод `total_pages`, который вычисляет и возвращает общее количество страниц всех книг в коллекции.
- Создайте экземпляр класса `BookCollection` и добавьте книги в коллекцию.
- Выведите информацию о текущих книгах в коллекции на экран.
- Выведите общее количество страниц всех книг на экран.
- Удалите книгу из коллекции и выведите обновленную информацию о книгах на экран.
- Выведите общее количество страниц после удаления книги на экран.

Пример использования:

```
collection = BookCollection()
collection.add_book("Война и мир", 1225)
collection.add_book("Преступление и наказание", 671)
collection.add_book("Мастер и Маргарита", 480)
print("Книги в коллекции:")
for book in collection.books:
    print(book[0], "-", book[1], "стр.")
total = collection.total_pages()
print("Общее количество страниц:", total)
collection.remove_book("Преступление и наказание")
print("Книги после удаления 'Преступления и наказания':")
for book in collection.books:
    print(book[0], "-", book[1], "стр.")
total = collection.total_pages()
print("Общее количество страниц:", total)
```

Вывод:

Книги в коллекции:

Война и мир - 1225 стр.

Преступление и наказание - 671 стр.

Мастер и Маргарита - 480 стр.

Общее количество страниц: 2376

Книги после удаления 'Преступления и наказания':

Война и мир - 1225 стр.

Мастер и Маргарита - 480 стр.

Общее количество страниц: 1705

3 Написать программу на Python, которая создает класс `Inventory` для представления складского запаса. Класс должен содержать методы для добавления и удаления предметов, а также вычисления общего количества единиц товара. Программа также должна создавать экземпляр класса `Inventory`, добавлять предметы на склад, удалять предметы со склада и выводить информацию о запасах на экран.

- Создайте класс `Inventory` с методом `__init__`, который создает пустой список предметов.
- Создайте метод `add_item`, который принимает название предмета и количество единиц в качестве аргументов и добавляет их в список.
- Создайте метод `remove_item`, который удаляет предмет из списка по его названию.
- Создайте метод `total_count`, который вычисляет и возвращает общее количество всех единиц товара на складе.
- Создайте экземпляр класса `Inventory` и добавьте предметы на склад.
- Выведите информацию о текущих предметах на складе на экран.
- Выведите общее количество единиц товара на экран.
- Удалите предмет со склада и выведите обновленную информацию о предметах на экран.
- Выведите общее количество единиц товара после удаления предмета на экран.

Пример использования:

```
inv = Inventory()
inv.add_item("Молотки", 50)
inv.add_item("Отвертки", 120)
inv.add_item("Гвозди", 1000)
print("Предметы на складе:")
for item in inv.items:
    print(item[0], "-", item[1])
total = inv.total_count()
print("Общее количество:", total)
inv.remove_item("Отвертки")
print("Предметы после удаления отверток:")
for item in inv.items:
    print(item[0], "-", item[1])
total = inv.total_count()
print("Общее количество:", total)
```

Вывод:

Предметы на складе:

Молотки - 50

Отвертки - 120

Гвозди - 1000

Общее количество: 1170

Предметы после удаления отверток:

Молотки - 50

Гвозди - 1000

Общее количество: 1050

- 4 Написать программу на Python, которая создает класс **Playlist** для представления музыкального плейлиста. Класс должен содержать методы для добавления и удаления треков, а также подсчета общего времени воспроизведения. Программа также должна создавать экземпляр класса **Playlist**, добавлять треки в плейлист, удалять треки из плейлиста и выводить информацию о плейлисте на экран.

- Создайте класс **Playlist** с методом `__init__`, который создает пустой список треков.
- Создайте метод `add_track`, который принимает название трека и его длительность (в секундах) в качестве аргументов и добавляет их в список.
- Создайте метод `remove_track`, который удаляет трек из списка по его названию.
- Создайте метод `total_duration`, который вычисляет и возвращает общую длительность всех треков в плейлисте (в секундах).
- Создайте экземпляр класса **Playlist** и добавьте треки в плейлист.
- Выведите информацию о текущих треках в плейлисте на экран.
- Выведите общую длительность всех треков на экран.
- Удалите трек из плейлиста и выведите обновленную информацию о треках на экран.
- Выведите общую длительность после удаления трека на экран.

Пример использования:

```
p1 = Playlist()
p1.add_track("Bohemian Rhapsody", 354)
p1.add_track("Imagine", 183)
p1.add_track("Smells Like Teen Spirit", 301)
print("Треки в плейлисте:")
for track in p1.tracks:
    print(track[0], "-", track[1], "сек.")
total = p1.total_duration()
print("Общая длительность:", total, "сек.")
p1.remove_track("Imagine")
print("Треки после удаления 'Imagine':")
for track in p1.tracks:
    print(track[0], "-", track[1], "сек.")
total = p1.total_duration()
print("Общая длительность:", total, "сек.")
```

Вывод:

Треки в плейлисте:

Bohemian Rhapsody - 354 сек.

Imagine - 183 сек.

Smells Like Teen Spirit - 301 сек.

Общая длительность: 838 сек.

Треки после удаления 'Imagine':

Bohemian Rhapsody - 354 сек.

Smells Like Teen Spirit - 301 сек.

Общая длительность: 655 сек.

- 5 Написать программу на Python, которая создает класс **StudentGrades** для представления оценок студента. Класс должен содержать методы для добавления и удаления оценок, а также вычисления среднего балла. Программа также должна создавать экземпляр класса **StudentGrades**, добавлять оценки, удалять оценки и выводить информацию об успеваемости на экран.

- Создайте класс **StudentGrades** с методом `__init__`, который создает пустой список оценок.
- Создайте метод `add_grade`, который принимает название предмета и оценку в качестве аргументов и добавляет их в список.
- Создайте метод `remove_grade`, который удаляет оценку по названию предмета.
- Создайте метод `average_grade`, который вычисляет и возвращает средний балл по всем предметам.
- Создайте экземпляр класса **StudentGrades** и добавьте оценки по разным предметам.
- Выведите информацию о текущих оценках на экран.
- Выведите средний балл на экран.
- Удалите оценку по одному из предметов и выведите обновленную информацию.
- Выведите средний балл после удаления оценки на экран.

Пример использования:

```
grades = StudentGrades()
grades.add_grade("Математика", 5)
grades.add_grade("Физика", 4)
grades.add_grade("Информатика", 5)
print("Оценки студента:")
for subject, grade in grades.grades:
    print(subject, "-", grade)
avg = grades.average_grade()
print("Средний балл:", round(avg, 2))
grades.remove_grade("Физика")
print("Оценки после удаления Физики:")
for subject, grade in grades.grades:
    print(subject, "-", grade)
avg = grades.average_grade()
print("Средний балл:", round(avg, 2))
```

Вывод:

Оценки студента:

Математика - 5

Физика - 4

Информатика - 5

Средний балл: 4.67

Оценки после удаления Физики:

Математика - 5

Информатика - 5

Средний балл: 5.0

6 Написать программу на Python, которая создает класс `TaskList` для представления списка задач. Класс должен содержать методы для добавления и удаления задач, а также подсчета общего количества задач. Программа также должна создавать экземпляр класса `TaskList`, добавлять задачи, удалять задачи и выводить информацию о списке задач на экран.

- Создайте класс `TaskList` с методом `__init__`, который создает пустой список задач.
- Создайте метод `add_task`, который принимает описание задачи и приоритет (целое число) в качестве аргументов и добавляет их в список.
- Создайте метод `remove_task`, который удаляет задачу из списка по её описанию.
- Создайте метод `task_count`, который возвращает общее количество задач в списке.
- Создайте экземпляр класса `TaskList` и добавьте несколько задач.
- Выведите информацию о текущих задачах на экран.
- Выведите общее количество задач на экран.
- Удалите одну из задач и выведите обновленный список задач.
- Выведите общее количество задач после удаления на экран.

Пример использования:

```
tasks = TaskList()
tasks.add_task("Написать отчет", 1)
tasks.add_task("Проверить почту", 3)
tasks.add_task("Подготовить презентацию", 2)
print("Список задач:")
for desc, priority in tasks.tasks:
    print(desc, "(приоритет", priority, ")")
count = tasks.task_count()
print("Всего задач:", count)
tasks.remove_task("Проверить почту")
print("Список задач после удаления 'Проверить почту':")
for desc, priority in tasks.tasks:
    print(desc, "(приоритет", priority, ")")
count = tasks.task_count()
print("Всего задач:", count)
```

Вывод:

Список задач:

Написать отчет (приоритет 1)

Проверить почту (приоритет 3)

Подготовить презентацию (приоритет 2)

Всего задач: 3

Список задач после удаления 'Проверить почту':

Написать отчет (приоритет 1)

Подготовить презентацию (приоритет 2)

Всего задач: 2

7 Написать программу на Python, которая создает класс **BankAccount** для представления банковского счета. Класс должен содержать методы для добавления и снятия средств, а также получения текущего баланса. Программа также должна создавать экземпляр класса **BankAccount**, выполнять операции пополнения и снятия, и выводить информацию о балансе на экран.

- Создайте класс **BankAccount** с методом `__init__`, который инициализирует баланс нулём.
- Создайте метод `deposit`, который принимает сумму и увеличивает баланс на неё.
- Создайте метод `withdraw`, который принимает сумму и уменьшает баланс на неё (если достаточно средств).
- Создайте метод `get_balance`, который возвращает текущий баланс.
- Создайте экземпляр класса **BankAccount**.
- Выполните несколько операций пополнения счета.
- Выведите текущий баланс на экран.
- Выполните операцию снятия средств и выведите обновленный баланс.
- Выведите окончательный баланс на экран.

Пример использования:

```
account = BankAccount()
account.deposit(1000)
account.deposit(500)
print("Баланс после пополнений:", account.get_balance())
account.withdraw(300)
print("Баланс после снятия 300:", account.get_balance())
account.withdraw(200)
print("Окончательный баланс:", account.get_balance())
```

Вывод:

Баланс после пополнений: 1500

Баланс после снятия 300: 1200

Окончательный баланс: 1000

8 Написать программу на Python, которая создает класс `Library` для представления библиотеки. Класс должен содержать методы для добавления и удаления книг, а также подсчета общего количества книг. Программа также должна создавать экземпляр класса `Library`, добавлять книги, удалять книги и выводить информацию о фонде на экран.

- Создайте класс `Library` с методом `__init__`, который создает пустой список книг.
- Создайте метод `add_book`, который принимает название книги и автора в качестве аргументов и добавляет их в список.
- Создайте метод `remove_book`, который удаляет книгу из списка по её названию.
- Создайте метод `book_count`, который возвращает общее количество книг в библиотеке.
- Создайте экземпляр класса `Library` и добавьте несколько книг.
- Выведите информацию о текущих книгах на экран.
- Выведите общее количество книг на экран.
- Удалите одну из книг и выведите обновленный список.
- Выведите общее количество книг после удаления на экран.

Пример использования:

```
lib = Library()
lib.add_book("1984", "Джордж Оруэлл")
lib.add_book("Гарри Поттер", "Дж.К. Роулинг")
lib.add_book("Гордость и предубеждение", "Джейн Остин")
print("Книги в библиотеке:")
for title, author in lib.books:
    print(title, "-", author)
count = lib.book_count()
print("Всего книг:", count)
lib.remove_book("Гарри Поттер")
print("Книги после удаления 'Гарри Поттера':")
for title, author in lib.books:
    print(title, "-", author)
count = lib.book_count()
print("Всего книг:", count)
```

Вывод:

```
Книги в библиотеке:
1984 - Джордж Оруэлл
Гарри Поттер - Дж.К. Роулинг
Гордость и предубеждение - Джейн Остин
Всего книг: 3
Книги после удаления 'Гарри Поттера':
1984 - Джордж Оруэлл
Гордость и предубеждение - Джейн Остин
Всего книг: 2
```

9 Написать программу на Python, которая создает класс **GroceryList** для представления списка покупок. Класс должен содержать методы для добавления и удаления продуктов, а также подсчета общего количества позиций. Программа также должна создавать экземпляр класса **GroceryList**, добавлять продукты, удалять продукты и выводить информацию о списке на экран.

- Создайте класс **GroceryList** с методом `__init__`, который создает пустой список продуктов.
- Создайте метод `add_product`, который принимает название продукта и количество в качестве аргументов и добавляет их в список.
- Создайте метод `remove_product`, который удаляет продукт из списка по его названию.
- Создайте метод `total_items`, который возвращает общее количество различных продуктов в списке.
- Создайте экземпляр класса **GroceryList** и добавьте несколько продуктов.
- Выведите информацию о текущих продуктах на экран.
- Выведите общее количество позиций на экран.
- Удалите один из продуктов и выведите обновленный список.
- Выведите общее количество позиций после удаления на экран.

Пример использования:

```
grocery = GroceryList()
grocery.add_product("Молоко", 2)
grocery.add_product("Хлеб", 1)
grocery.add_product("Яйца", 12)
print("Список покупок:")
for name, qty in grocery.products:
    print(name, "-", qty)
count = grocery.total_items()
print("Всего позиций:", count)
grocery.remove_product("Хлеб")
print("Список после удаления хлеба:")
for name, qty in grocery.products:
    print(name, "-", qty)
count = grocery.total_items()
print("Всего позиций:", count)
```

Вывод:

```
Список покупок:
Молоко - 2
Хлеб - 1
Яйца - 12
Всего позиций: 3
Список после удаления хлеба:
Молоко - 2
Яйца - 12
Всего позиций: 2
```

10 Написать программу на Python, которая создает класс `ContactList` для представления списка контактов. Класс должен содержать методы для добавления и удаления контактов, а также подсчета общего количества контактов. Программа также должна создавать экземпляр класса `ContactList`, добавлять контакты, удалять контакты и выводить информацию о списке на экран.

- Создайте класс `ContactList` с методом `__init__`, который создает пустой список контактов.
- Создайте метод `add_contact`, который принимает имя и номер телефона в качестве аргументов и добавляет их в список.
- Создайте метод `remove_contact`, который удаляет контакт из списка по имени.
- Создайте метод `contact_count`, который возвращает общее количество контактов в списке.
- Создайте экземпляр класса `ContactList` и добавьте несколько контактов.
- Выведите информацию о текущих контактах на экран.
- Выведите общее количество контактов на экран.
- Удалите один из контактов и выведите обновленный список.
- Выведите общее количество контактов после удаления на экран.

Пример использования:

```
contacts = ContactList()
contacts.add_contact("Анна", "+79001234567")
contacts.add_contact("Борис", "+79007654321")
contacts.add_contact("Виктория", "+79001112233")
print("Контакты:")
for name, phone in contacts.contacts:
    print(name, "-", phone)
count = contacts.contact_count()
print("Всего контактов:", count)
contacts.remove_contact("Борис")
print("Контакты после удаления Бориса:")
for name, phone in contacts.contacts:
    print(name, "-", phone)
count = contacts.contact_count()
print("Всего контактов:", count)
```

Вывод:

```
Контакты:
Анна - +79001234567
Борис - +79007654321
Виктория - +79001112233
Всего контактов: 3
Контакты после удаления Бориса:
Анна - +79001234567
Виктория - +79001112233
Всего контактов: 2
```

- 11 Написать программу на Python, которая создает класс `MovieCollection` для представления коллекции фильмов. Класс должен содержать методы для добавления и удаления фильмов, а также подсчета общего количества фильмов. Программа также должна создавать экземпляр класса `MovieCollection`, добавлять фильмы, удалять фильмы и выводить информацию о коллекции на экран.

- Создайте класс `MovieCollection` с методом `__init__`, который создает пустой список фильмов.
- Создайте метод `add_movie`, который принимает название фильма и год выпуска в качестве аргументов и добавляет их в список.
- Создайте метод `remove_movie`, который удаляет фильм из списка по его названию.
- Создайте метод `movie_count`, который возвращает общее количество фильмов в коллекции.
- Создайте экземпляр класса `MovieCollection` и добавьте несколько фильмов.
- Выведите информацию о текущих фильмах на экран.
- Выведите общее количество фильмов на экран.
- Удалите один из фильмов и выведите обновленный список.
- Выведите общее количество фильмов после удаления на экран.

Пример использования:

```
movies = MovieCollection()
movies.add_movie("Крёстный отец", 1972)
movies.add_movie("Побег из Шоушенка", 1994)
movies.add_movie("Тёмный рыцарь", 2008)
print("Фильмы в коллекции:")
for title, year in movies.movies:
    print(title, "(", year, ")")
count = movies.movie_count()
print("Всего фильмов:", count)
movies.remove_movie("Побег из Шоушенка")
print("Фильмы после удаления 'Побега из Шоушенка':")
for title, year in movies.movies:
    print(title, "(", year, ")")
count = movies.movie_count()
print("Всего фильмов:", count)
```

Вывод:

```
Фильмы в коллекции:
Крёстный отец ( 1972 )
Побег из Шоушенка ( 1994 )
Тёмный рыцарь ( 2008 )
Всего фильмов: 3
Фильмы после удаления 'Побега из Шоушенка':
Крёстный отец ( 1972 )
Тёмный рыцарь ( 2008 )
Всего фильмов: 2
```

12 Написать программу на Python, которая создает класс `RecipeBook` для представления кулинарной книги. Класс должен содержать методы для добавления и удаления рецептов, а также подсчета общего количества рецептов. Программа также должна создавать экземпляр класса `RecipeBook`, добавлять рецепты, удалять рецепты и выводить информацию о книге на экран.

- Создайте класс `RecipeBook` с методом `__init__`, который создает пустой список рецептов.
- Создайте метод `add_recipe`, который принимает название блюда и время приготовления (в минутах) в качестве аргументов и добавляет их в список.
- Создайте метод `remove_recipe`, который удаляет рецепт из списка по названию блюда.
- Создайте метод `recipe_count`, который возвращает общее количество рецептов в книге.
- Создайте экземпляр класса `RecipeBook` и добавьте несколько рецептов.
- Выведите информацию о текущих рецептах на экран.
- Выведите общее количество рецептов на экран.
- Удалите один из рецептов и выведите обновленный список.
- Выведите общее количество рецептов после удаления на экран.

Пример использования:

```
recipes = RecipeBook()
recipes.add_recipe("Борщ", 60)
recipes.add_recipe("Омлет", 10)
recipes.add_recipe("Паста", 20)
print("Рецепты в книге:")
for dish, time in recipes.recipes:
    print(dish, "-", time, "мин.")
count = recipes.recipe_count()
print("Всего рецептов:", count)
recipes.remove_recipe("Омлет")
print("Рецепты после удаления омлета:")
for dish, time in recipes.recipes:
    print(dish, "-", time, "мин.")
count = recipes.recipe_count()
print("Всего рецептов:", count)
```

Вывод:

```
Рецепты в книге:
Борщ - 60 мин.
Омлет - 10 мин.
Паста - 20 мин.
Всего рецептов: 3
Рецепты после удаления омлета:
Борщ - 60 мин.
Паста - 20 мин.
Всего рецептов: 2
```

13 Написать программу на Python, которая создает класс `CarGarage` для представления автосервиса. Класс должен содержать методы для добавления и удаления автомобилей, а также подсчета общего количества машин. Программа также должна создавать экземпляр класса `CarGarage`, добавлять автомобили, удалять автомобили и выводить информацию о гараже на экран.

- Создайте класс `CarGarage` с методом `__init__`, который создает пустой список автомобилей.
- Создайте метод `add_car`, который принимает марку и модель автомобиля в качестве аргументов и добавляет их в список.
- Создайте метод `remove_car`, который удаляет автомобиль из списка по марке.
- Создайте метод `car_count`, который возвращает общее количество автомобилей в гараже.
- Создайте экземпляр класса `CarGarage` и добавьте несколько автомобилей.
- Выведите информацию о текущих автомобилях на экран.
- Выведите общее количество машин на экран.
- Удалите один из автомобилей и выведите обновленный список.
- Выведите общее количество машин после удаления на экран.

Пример использования:

```
garage = CarGarage()
garage.add_car("Toyota", "Camry")
garage.add_car("BMW", "X5")
garage.add_car("Ford", "Focus")
print("Автомобили в гараже:")
for brand, model in garage.cars:
    print(brand, model)
count = garage.car_count()
print("Всего автомобилей:", count)
garage.remove_car("BMW")
print("Автомобили после удаления BMW:")
for brand, model in garage.cars:
    print(brand, model)
count = garage.car_count()
print("Всего автомобилей:", count)
```

Вывод:

```
Автомобили в гараже:
Toyota Camry
BMW X5
Ford Focus
Всего автомобилей: 3
Автомобили после удаления BMW:
Toyota Camry
Ford Focus
Всего автомобилей: 2
```

14 Написать программу на Python, которая создает класс **PetStore** для представления зоомагазина. Класс должен содержать методы для добавления и удаления животных, а также подсчета общего количества питомцев. Программа также должна создавать экземпляр класса **PetStore**, добавлять животных, удалять животных и выводить информацию о магазине на экран.

- Создайте класс **PetStore** с методом `__init__`, который создает пустой список животных.
- Создайте метод `add_pet`, который принимает вид животного и количество в качестве аргументов и добавляет их в список.
- Создайте метод `remove_pet`, который удаляет животное из списка по виду.
- Создайте метод `total_pets`, который возвращает общее количество всех питомцев в магазине.
- Создайте экземпляр класса **PetStore** и добавьте несколько видов животных.
- Выведите информацию о текущих животных на экран.
- Выведите общее количество питомцев на экран.
- Удалите один из видов животных и выведите обновленный список.
- Выведите общее количество питомцев после удаления на экран.

Пример использования:

```
store = PetStore()
store.add_pet("Кошки", 5)
store.add_pet("Собаки", 3)
store.add_pet("Попугаи", 10)
print("Животные в магазине:")
for species, count in store.pets:
    print(species, "-", count)
total = store.total_pets()
print("Всего питомцев:", total)
store.remove_pet("Собаки")
print("Животные после удаления собак:")
for species, count in store.pets:
    print(species, "-", count)
total = store.total_pets()
print("Всего питомцев:", total)
```

Вывод:

```
Животные в магазине:
Кошки - 5
Собаки - 3
Попугаи - 10
Всего питомцев: 18
Животные после удаления собак:
Кошки - 5
Попугаи - 10
Всего питомцев: 15
```

15 Написать программу на Python, которая создает класс `CourseRoster` для представления списка студентов на курсе. Класс должен содержать методы для добавления и удаления студентов, а также подсчета общего количества учащихся. Программа также должна создавать экземпляр класса `CourseRoster`, добавлять студентов, удалять студентов и выводить информацию о курсе на экран.

- Создайте класс `CourseRoster` с методом `__init__`, который создает пустой список студентов.
- Создайте метод `enroll_student`, который принимает имя студента и его ID в качестве аргументов и добавляет их в список.
- Создайте метод `drop_student`, который удаляет студента из списка по имени.
- Создайте метод `student_count`, который возвращает общее количество студентов на курсе.
- Создайте экземпляр класса `CourseRoster` и добавьте несколько студентов.
- Выведите информацию о текущих студентах на экран.
- Выведите общее количество студентов на экран.
- Удалите одного из студентов и выведите обновленный список.
- Выведите общее количество студентов после удаления на экран.

Пример использования:

```
roster = CourseRoster()
roster.enroll_student("Иван", 101)
roster.enroll_student("Мария", 102)
roster.enroll_student("Алексей", 103)
print("Студенты на курсе:")
for name, sid in roster.students:
    print(name, "(ID:", sid, ")")
count = roster.student_count()
print("Всего студентов:", count)
roster.drop_student("Мария")
print("Студенты после отчисления Марии:")
for name, sid in roster.students:
    print(name, "(ID:", sid, ")")
count = roster.student_count()
print("Всего студентов:", count)
```

Вывод:

```
Студенты на курсе:
Иван (ID: 101 )
Мария (ID: 102 )
Алексей (ID: 103 )
Всего студентов: 3
Студенты после отчисления Марии:
Иван (ID: 101 )
Алексей (ID: 103 )
Всего студентов: 2
```


16 Написать программу на Python, которая создает класс `TravelItinerary` для представления туристического маршрута. Класс должен содержать методы для добавления и удаления мест, а также подсчета общего количества пунктов назначения. Программа также должна создавать экземпляры класса `TravelItinerary`, добавлять места, удалять места и выводить информацию о маршруте на экран.

- Создайте класс `TravelItinerary` с методом `__init__`, который создает пустой список мест.
- Создайте метод `add_destination`, который принимает название города и количество дней пребывания в качестве аргументов и добавляет их в список.
- Создайте метод `remove_destination`, который удаляет место из списка по названию города.
- Создайте метод `destination_count`, который возвращает общее количество пунктов назначения в маршруте.
- Создайте экземпляр класса `TravelItinerary` и добавьте несколько городов.
- Выведите информацию о текущих местах на экран.
- Выведите общее количество пунктов назначения на экран.
- Удалите один из городов и выведите обновленный маршрут.
- Выведите общее количество пунктов назначения после удаления на экран.

Пример использования:

```
itinerary = TravelItinerary()
itinerary.add_destination("Париж", 4)
itinerary.add_destination("Рим", 3)
itinerary.add_destination("Барселона", 5)
print("Маршрут путешествия:")
for city, days in itinerary.destinations:
    print(city, "-", days, "дней")
count = itinerary.destination_count()
print("Всего пунктов:", count)
itinerary.remove_destination("Рим")
print("Маршрут после удаления Рима:")
for city, days in itinerary.destinations:
    print(city, "-", days, "дней")
count = itinerary.destination_count()
print("Всего пунктов:", count)
```

Вывод:

```
Маршрут путешествия:
Париж - 4 дней
Рим - 3 дней
Барселона - 5 дней
Всего пунктов: 3
Маршрут после удаления Рима:
Париж - 4 дней
Барселона - 5 дней
Всего пунктов: 2
```

17 Написать программу на Python, которая создает класс `FitnessTracker` для представления тренировочного плана. Класс должен содержать методы для добавления и удаления упражнений, а также подсчета общего количества подходов. Программа также должна создавать экземпляр класса `FitnessTracker`, добавлять упражнения, удалять упражнения и выводить информацию о плане на экран.

- Создайте класс `FitnessTracker` с методом `__init__`, который создает пустой список упражнений.
- Создайте метод `add_exercise`, который принимает название упражнения и количество подходов в качестве аргументов и добавляет их в список.
- Создайте метод `remove_exercise`, который удаляет упражнение из списка по его названию.
- Создайте метод `total_sets`, который возвращает общее количество подходов по всем упражнениям.
- Создайте экземпляр класса `FitnessTracker` и добавьте несколько упражнений.
- Выведите информацию о текущих упражнениях на экран.
- Выведите общее количество подходов на экран.
- Удалите одно из упражнений и выведите обновленный план.
- Выведите общее количество подходов после удаления на экран.

Пример использования:

```
tracker = FitnessTracker()
tracker.add_exercise("Приседания", 4)
tracker.add_exercise("Отжимания", 3)
tracker.add_exercise("Подтягивания", 5)
print("Тренировочный план:")
for ex, sets in tracker.exercises:
    print(ex, "-", sets, "подходов")
total = tracker.total_sets()
print("Всего подходов:", total)
tracker.remove_exercise("Отжимания")
print("План после удаления отжиманий:")
for ex, sets in tracker.exercises:
    print(ex, "-", sets, "подходов")
total = tracker.total_sets()
print("Всего подходов:", total)
```

Вывод:

```
Тренировочный план:
Приседания - 4 подходов
Отжимания - 3 подходов
Подтягивания - 5 подходов
Всего подходов: 12
План после удаления отжиманий:
Приседания - 4 подходов
Подтягивания - 5 подходов
Всего подходов: 9
```

18 Написать программу на Python, которая создает класс **ExpenseTracker** для представления расходов. Класс должен содержать методы для добавления и удаления трат, а также подсчета общей суммы расходов. Программа также должна создавать экземпляр класса **ExpenseTracker**, добавлять расходы, удалять расходы и выводить информацию о тратах на экран.

- Создайте класс **ExpenseTracker** с методом `__init__`, который создает пустой список расходов.
- Создайте метод `add_expense`, который принимает категорию и сумму в качестве аргументов и добавляет их в список.
- Создайте метод `remove_expense`, который удаляет расход из списка по категории.
- Создайте метод `total_expenses`, который возвращает общую сумму всех расходов.
- Создайте экземпляр класса **ExpenseTracker** и добавьте несколько расходов.
- Выведите информацию о текущих тратах на экран.
- Выведите общую сумму расходов на экран.
- Удалите один из расходов и выведите обновленный список.
- Выведите общую сумму расходов после удаления на экран.

Пример использования:

```
expenses = ExpenseTracker()
expenses.add_expense("Продукты", 2500)
expenses.add_expense("Транспорт", 800)
expenses.add_expense("Развлечения", 1200)
print("Расходы:")
for cat, amount in expenses.expenses:
    print(cat, "-", amount, "руб.")
total = expenses.total_expenses()
print("Общая сумма расходов:", total, "руб.")
expenses.remove_expense("Транспорт")
print("Расходы после удаления транспорта:")
for cat, amount in expenses.expenses:
    print(cat, "-", amount, "руб.")
total = expenses.total_expenses()
print("Общая сумма расходов:", total, "руб.")
```

Вывод:

```
Расходы:
Продукты - 2500 руб.
Транспорт - 800 руб.
Развлечения - 1200 руб.
Общая сумма расходов: 4500 руб.
Расходы после удаления транспорта:
Продукты - 2500 руб.
Развлечения - 1200 руб.
Общая сумма расходов: 3700 руб.
```

19 Написать программу на Python, которая создает класс `ProjectTasks` для представления задач проекта. Класс должен содержать методы для добавления и удаления задач, а также подсчета общего количества задач. Программа также должна создавать экземпляр класса `ProjectTasks`, добавлять задачи, удалять задачи и выводить информацию о проекте на экран.

- Создайте класс `ProjectTasks` с методом `__init__`, который создает пустой список задач.
- Создайте метод `add_task`, который принимает описание задачи и срок выполнения (в днях) в качестве аргументов и добавляет их в список.
- Создайте метод `remove_task`, который удаляет задачу из списка по её описанию.
- Создайте метод `task_count`, который возвращает общее количество задач в проекте.
- Создайте экземпляр класса `ProjectTasks` и добавьте несколько задач.
- Выведите информацию о текущих задачах на экран.
- Выведите общее количество задач на экран.
- Удалите одну из задач и выведите обновленный список.
- Выведите общее количество задач после удаления на экран.

Пример использования:

```
project = ProjectTasks()
project.add_task("Разработка интерфейса", 5)
project.add_task("Тестирование", 3)
project.add_task("Документация", 2)
print("Задачи проекта:")
for desc, days in project.tasks:
    print(desc, "-", days, "дней")
count = project.task_count()
print("Всего задач:", count)
project.remove_task("Тестирование")
print("Задачи после удаления тестирования:")
for desc, days in project.tasks:
    print(desc, "-", days, "дней")
count = project.task_count()
print("Всего задач:", count)
```

Вывод:

```
Задачи проекта:
Разработка интерфейса - 5 дней
Тестирование - 3 дней
Документация - 2 дней
Всего задач: 3
Задачи после удаления тестирования:
Разработка интерфейса - 5 дней
Документация - 2 дней
Всего задач: 2
```

20 Написать программу на Python, которая создает класс `EventSchedule` для представления расписания мероприятий. Класс должен содержать методы для добавления и удаления событий, а также подсчета общего количества мероприятий. Программа также должна создавать экземпляр класса `EventSchedule`, добавлять события, удалять события и выводить информацию о расписании на экран.

- Создайте класс `EventSchedule` с методом `__init__`, который создает пустой список мероприятий.
- Создайте метод `add_event`, который принимает название мероприятия и дату проведения в качестве аргументов и добавляет их в список.
- Создайте метод `remove_event`, который удаляет мероприятие из списка по его названию.
- Создайте метод `event_count`, который возвращает общее количество мероприятий в расписании.
- Создайте экземпляр класса `EventSchedule` и добавьте несколько мероприятий.
- Выведите информацию о текущих мероприятиях на экран.
- Выведите общее количество мероприятий на экран.
- Удалите одно из мероприятий и выведите обновленное расписание.
- Выведите общее количество мероприятий после удаления на экран.

Пример использования:

```
schedule = EventSchedule()
schedule.add_event("Конференция", "15.05.2024")
schedule.add_event("Воркшоп", "20.05.2024")
schedule.add_event("Выставка", "25.05.2024")
print("Расписание мероприятий:")
for name, date in schedule.events:
    print(name, "-", date)
count = schedule.event_count()
print("Всего мероприятий:", count)
schedule.remove_event("Воркшоп")
print("Расписание после удаления воркшопа:")
for name, date in schedule.events:
    print(name, "-", date)
count = schedule.event_count()
print("Всего мероприятий:", count)
```

Вывод:

```
Расписание мероприятий:
Конференция - 15.05.2024
Воркшоп - 20.05.2024
Выставка - 25.05.2024
Всего мероприятий: 3
Расписание после удаления воркшопа:
Конференция - 15.05.2024
Выставка - 25.05.2024
Всего мероприятий: 2
```

21 Написать программу на Python, которая создает класс **GardenPlanner** для представления садового участка. Класс должен содержать методы для добавления и удаления растений, а также подсчета общего количества видов растений. Программа также должна создавать экземпляр класса **GardenPlanner**, добавлять растения, удалять растения и выводить информацию о саде на экран.

- Создайте класс **GardenPlanner** с методом `__init__`, который создает пустой список растений.
- Создайте метод `add_plant`, который принимает название растения и количество экземпляров в качестве аргументов и добавляет их в список.
- Создайте метод `remove_plant`, который удаляет растение из списка по его названию.
- Создайте метод `plant_count`, который возвращает общее количество различных видов растений в саду.
- Создайте экземпляр класса **GardenPlanner** и добавьте несколько растений.
- Выведите информацию о текущих растениях на экран.
- Выведите общее количество видов растений на экран.
- Удалите одно из растений и выведите обновленный список.
- Выведите общее количество видов растений после удаления на экран.

Пример использования:

```
garden = GardenPlanner()
garden.add_plant("Розы", 10)
garden.add_plant("Тюльпаны", 20)
garden.add_plant("Лаванда", 5)
print("Растения в саду:")
for name, qty in garden.plants:
    print(name, "-", qty)
count = garden.plant_count()
print("Всего видов растений:", count)
garden.remove_plant("Тюльпаны")
print("Растения после удаления тюльпанов:")
for name, qty in garden.plants:
    print(name, "-", qty)
count = garden.plant_count()
print("Всего видов растений:", count)
```

Вывод:

```
Растения в саду:
Розы - 10
Тюльпаны - 20
Лаванда - 5
Всего видов растений: 3
Растения после удаления тюльпанов:
Розы - 10
Лаванда - 5
Всего видов растений: 2
```

22 Написать программу на Python, которая создает класс **Warehouse** для представления склада товаров. Класс должен содержать методы для добавления и удаления товаров, а также подсчета общего количества типов товаров. Программа также должна создавать экземпляр класса **Warehouse**, добавлять товары, удалять товары и выводить информацию о складе на экран.

- Создайте класс **Warehouse** с методом `__init__`, который создает пустой список товаров.
- Создайте метод `add_product`, который принимает название товара и количество единиц в качестве аргументов и добавляет их в список.
- Создайте метод `remove_product`, который удаляет товар из списка по его названию.
- Создайте метод `product_types`, который возвращает общее количество различных типов товаров на складе.
- Создайте экземпляр класса **Warehouse** и добавьте несколько товаров.
- Выведите информацию о текущих товарах на экран.
- Выведите общее количество типов товаров на экран.
- Удалите один из товаров и выведите обновленный список.
- Выведите общее количество типов товаров после удаления на экран.

Пример использования:

```
warehouse = Warehouse()
warehouse.add_product("Стулья", 50)
warehouse.add_product("Столы", 20)
warehouse.add_product("Лампы", 100)
print("Товары на складе:")
for name, qty in warehouse.products:
    print(name, "-", qty)
types = warehouse.product_types()
print("Всего типов товаров:", types)
warehouse.remove_product("Столы")
print("Товары после удаления столов:")
for name, qty in warehouse.products:
    print(name, "-", qty)
types = warehouse.product_types()
print("Всего типов товаров:", types)
```

Вывод:

```
Товары на складе:
Стулья - 50
Столы - 20
Лампы - 100
Всего типов товаров: 3
Товары после удаления столов:
Стулья - 50
Лампы - 100
Всего типов товаров: 2
```

23 Написать программу на Python, которая создает класс `GameInventory` для представления инвентаря игрока. Класс должен содержать методы для добавления и удаления предметов, а также подсчета общего количества типов предметов. Программа также должна создавать экземпляр класса `GameInventory`, добавлять предметы, удалять предметы и выводить информацию об инвентаре на экран.

- Создайте класс `GameInventory` с методом `__init__`, который создает пустой список предметов.
- Создайте метод `add_item`, который принимает название предмета и количество в качестве аргументов и добавляет их в список.
- Создайте метод `remove_item`, который удаляет предмет из списка по его названию.
- Создайте метод `item_types`, который возвращает общее количество различных типов предметов в инвентаре.
- Создайте экземпляр класса `GameInventory` и добавьте несколько предметов.
- Выведите информацию о текущих предметах на экран.
- Выведите общее количество типов предметов на экран.
- Удалите один из предметов и выведите обновленный инвентарь.
- Выведите общее количество типов предметов после удаления на экран.

Пример использования:

```
inventory = GameInventory()
inventory.add_item("Меч", 1)
inventory.add_item("Зелье", 5)
inventory.add_item("Щит", 1)
print("Инвентарь игрока:")
for name, qty in inventory.items:
    print(name, "-", qty)
types = inventory.item_types()
print("Всего типов предметов:", types)
inventory.remove_item("Зелье")
print("Инвентарь после удаления зелий:")
for name, qty in inventory.items:
    print(name, "-", qty)
types = inventory.item_types()
print("Всего типов предметов:", types)
```

Вывод:

```
Инвентарь игрока:
Меч - 1
Зелье - 5
Щит - 1
Всего типов предметов: 3
Инвентарь после удаления зелий:
Меч - 1
Щит - 1
Всего типов предметов: 2
```


24 Написать программу на Python, которая создает класс `MusicAlbum` для представления музыкального альбома. Класс должен содержать методы для добавления и удаления треков, а также подсчета общего количества треков. Программа также должна создавать экземпляр класса `MusicAlbum`, добавлять треки, удалять треки и выводить информацию об альбоме на экран.

- Создайте класс `MusicAlbum` с методом `__init__`, который создает пустой список треков.
- Создайте метод `add_track`, который принимает название трека и его длительность (в секундах) в качестве аргументов и добавляет их в список.
- Создайте метод `remove_track`, который удаляет трек из списка по его названию.
- Создайте метод `track_count`, который возвращает общее количество треков в альбоме.
- Создайте экземпляр класса `MusicAlbum` и добавьте несколько треков.
- Выведите информацию о текущих треках на экран.
- Выведите общее количество треков на экран.
- Удалите один из треков и выведите обновленный список.
- Выведите общее количество треков после удаления на экран.

Пример использования:

```
album = MusicAlbum()
album.add_track("Yesterday", 125)
album.add_track("Hey Jude", 431)
album.add_track("Let It Be", 243)
print("Треки в альбоме:")
for name, duration in album.tracks:
    print(name, "-", duration, "сек.")
count = album.track_count()
print("Всего треков:", count)
album.remove_track("Hey Jude")
print("Треки после удаления 'Hey Jude':")
for name, duration in album.tracks:
    print(name, "-", duration, "сек.")
count = album.track_count()
print("Всего треков:", count)
```

Вывод:

```
Треки в альбоме:
Yesterday - 125 сек.
Hey Jude - 431 сек.
Let It Be - 243 сек.
Всего треков: 3
Треки после удаления 'Hey Jude':
Yesterday - 125 сек.
Let It Be - 243 сек.
Всего треков: 2
```

25 Написать программу на Python, которая создает класс `EmployeeRoster` для представления списка сотрудников. Класс должен содержать методы для добавления и удаления сотрудников, а также подсчета общего количества работников. Программа также должна создавать экземпляр класса `EmployeeRoster`, добавлять сотрудников, удалять сотрудников и выводить информацию о персонале на экран.

- Создайте класс `EmployeeRoster` с методом `__init__`, который создает пустой список сотрудников.
- Создайте метод `hire_employee`, который принимает имя сотрудника и его должность в качестве аргументов и добавляет их в список.
- Создайте метод `fire_employee`, который удаляет сотрудника из списка по имени.
- Создайте метод `employee_count`, который возвращает общее количество сотрудников.
- Создайте экземпляр класса `EmployeeRoster` и добавьте несколько сотрудников.
- Выведите информацию о текущих сотрудниках на экран.
- Выведите общее количество работников на экран.
- Удалите одного из сотрудников и выведите обновленный список.
- Выведите общее количество работников после удаления на экран.

Пример использования:

```
roster = EmployeeRoster()
roster.hire_employee("Елена", "Менеджер")
roster.hire_employee("Дмитрий", "Разработчик")
roster.hire_employee("Ольга", "Дизайнер")
print("Сотрудники компании:")
for name, position in roster.employees:
    print(name, "-", position)
count = roster.employee_count()
print("Всего сотрудников:", count)
roster.fire_employee("Дмитрий")
print("Сотрудники после увольнения Дмитрия:")
for name, position in roster.employees:
    print(name, "-", position)
count = roster.employee_count()
print("Всего сотрудников:", count)
```

Вывод:

```
Сотрудники компании:
Елена - Менеджер
Дмитрий - Разработчик
Ольга - Дизайнер
Всего сотрудников: 3
Сотрудники после увольнения Дмитрия:
Елена - Менеджер
Ольга - Дизайнер
Всего сотрудников: 2
```

26 Написать программу на Python, которая создает класс `ShoppingWishlist` для представления списка желаний покупателя. Класс должен содержать методы для добавления и удаления товаров, а также подсчета общего количества позиций. Программа также должна создавать экземпляр класса `ShoppingWishlist`, добавлять товары, удалять товары и выводить информацию о списке желаний на экран.

- Создайте класс `ShoppingWishlist` с методом `__init__`, который создает пустой список товаров.
- Создайте метод `add_item`, который принимает название товара и его приоритет (от 1 до 5) в качестве аргументов и добавляет их в список.
- Создайте метод `remove_item`, который удаляет товар из списка по его названию.
- Создайте метод `item_count`, который возвращает общее количество товаров в списке желаний.
- Создайте экземпляр класса `ShoppingWishlist` и добавьте несколько товаров.
- Выведите информацию о текущих товарах на экран.
- Выведите общее количество позиций на экран.
- Удалите один из товаров и выведите обновленный список.
- Выведите общее количество позиций после удаления на экран.

Пример использования:

```
wishlist = ShoppingWishlist()
wishlist.add_item("Наушники", 5)
wishlist.add_item("Книга", 3)
wishlist.add_item("Флешка", 2)
print("Список желаний:")
for name, priority in wishlist.items:
    print(name, "(приоритет", priority, ")")
count = wishlist.item_count()
print("Всего позиций:", count)
wishlist.remove_item("Книга")
print("Список после удаления книги:")
for name, priority in wishlist.items:
    print(name, "(приоритет", priority, ")")
count = wishlist.item_count()
print("Всего позиций:", count)
```

Вывод:

```
Список желаний:
Наушники (приоритет 5 )
Книга (приоритет 3 )
Флешка (приоритет 2 )
Всего позиций: 3
Список после удаления книги:
Наушники (приоритет 5 )
Флешка (приоритет 2 )
Всего позиций: 2
```

27 Написать программу на Python, которая создает класс `DietPlan` для представления плана питания. Класс должен содержать методы для добавления и удаления блюд, а также подсчета общего количества приемов пищи. Программа также должна создавать экземпляр класса `DietPlan`, добавлять блюда, удалять блюда и выводить информацию о плане на экран.

- Создайте класс `DietPlan` с методом `__init__`, который создает пустой список блюд.
- Создайте метод `add_meal`, который принимает название блюда и количество калорий в качестве аргументов и добавляет их в список.
- Создайте метод `remove_meal`, который удаляет блюдо из списка по его названию.
- Создайте метод `meal_count`, который возвращает общее количество блюд в плане.
- Создайте экземпляр класса `DietPlan` и добавьте несколько блюд.
- Выведите информацию о текущих блюдах на экран.
- Выведите общее количество приемов пищи на экран.
- Удалите одно из блюд и выведите обновленный план.
- Выведите общее количество приемов пищи после удаления на экран.

Пример использования:

```
diet = DietPlan()
diet.add_meal("Овсянка", 300)
diet.add_meal("Салат", 150)
diet.add_meal("Курица", 400)
print("План питания:")
for dish, calories in diet.meals:
    print(dish, "-", calories, "ккал")
count = diet.meal_count()
print("Всего приемов пищи:", count)
diet.remove_meal("Салат")
print("План после удаления салата:")
for dish, calories in diet.meals:
    print(dish, "-", calories, "ккал")
count = diet.meal_count()
print("Всего приемов пищи:", count)
```

Вывод:

```
План питания:
Овсянка - 300 ккал
Салат - 150 ккал
Курица - 400 ккал
Всего приемов пищи: 3
План после удаления салата:
Овсянка - 300 ккал
Курица - 400 ккал
Всего приемов пищи: 2
```

28 Написать программу на Python, которая создает класс `PhotoAlbum` для представления фотоальбома. Класс должен содержать методы для добавления и удаления фотографий, а также подсчета общего количества снимков. Программа также должна создавать экземпляр класса `PhotoAlbum`, добавлять фотографии, удалять фотографии и выводить информацию об альбоме на экран.

- Создайте класс `PhotoAlbum` с методом `__init__`, который создает пустой список фотографий.
- Создайте метод `add_photo`, который принимает название фотографии и дату съемки в качестве аргументов и добавляет их в список.
- Создайте метод `remove_photo`, который удаляет фотографию из списка по её названию.
- Создайте метод `photo_count`, который возвращает общее количество фотографий в альбоме.
- Создайте экземпляр класса `PhotoAlbum` и добавьте несколько фотографий.
- Выведите информацию о текущих фотографиях на экран.
- Выведите общее количество снимков на экран.
- Удалите одну из фотографий и выведите обновленный альбом.
- Выведите общее количество снимков после удаления на экран.

Пример использования:

```
album = PhotoAlbum()
album.add_photo("Пляж", "2023-07-15")
album.add_photo("Горы", "2023-08-20")
album.add_photo("Семья", "2023-12-25")
print("Фотографии в альбоме:")
for name, date in album.photos:
    print(name, "-", date)
count = album.photo_count()
print("Всего фотографий:", count)
album.remove_photo("Горы")
print("Фотографии после удаления 'Горы':")
for name, date in album.photos:
    print(name, "-", date)
count = album.photo_count()
print("Всего фотографий:", count)
```

Вывод:

```
Фотографии в альбоме:
Пляж - 2023-07-15
Горы - 2023-08-20
Семья - 2023-12-25
Всего фотографий: 3
Фотографии после удаления 'Горы':
Пляж - 2023-07-15
Семья - 2023-12-25
Всего фотографий: 2
```

29 Написать программу на Python, которая создает класс **StudyMaterials** для представления учебных материалов. Класс должен содержать методы для добавления и удаления материалов, а также подсчета общего количества ресурсов. Программа также должна создавать экземпляр класса **StudyMaterials**, добавлять материалы, удалять материалы и выводить информацию о ресурсах на экран.

- Создайте класс **StudyMaterials** с методом `__init__`, который создает пустой список материалов.
- Создайте метод `add_material`, который принимает название материала и тип (например, "книга" "видео" "статья") в качестве аргументов и добавляет их в список.
- Создайте метод `remove_material`, который удаляет материал из списка по его названию.
- Создайте метод `material_count`, который возвращает общее количество учебных материалов.
- Создайте экземпляр класса **StudyMaterials** и добавьте несколько материалов.
- Выведите информацию о текущих материалах на экран.
- Выведите общее количество ресурсов на экран.
- Удалите один из материалов и выведите обновленный список.
- Выведите общее количество ресурсов после удаления на экран.

Пример использования:

```
materials = StudyMaterials()
materials.add_material("Алгоритмы", "книга")
materials.add_material("Python для начинающих", "видео")
materials.add_material("Структуры данных", "статья")
print("Учебные материалы:")
for name, mtype in materials.materials:
    print(name, "-", mtype)
count = materials.material_count()
print("Всего материалов:", count)
materials.remove_material("Python для начинающих")
print("Материалы после удаления видео:")
for name, mtype in materials.materials:
    print(name, "-", mtype)
count = materials.material_count()
print("Всего материалов:", count)
```

Вывод:

```
Учебные материалы:
Алгоритмы - книга
Python для начинающих - видео
Структуры данных - статья
Всего материалов: 3
Материалы после удаления видео:
Алгоритмы - книга
Структуры данных - статья
Всего материалов: 2
```

30 Написать программу на Python, которая создает класс `ArtCollection` для представления коллекции произведений искусства. Класс должен содержать методы для добавления и удаления работ, а также подсчета общего количества экспонатов. Программа также должна создавать экземпляр класса `ArtCollection`, добавлять работы, удалять работы и выводить информацию о коллекции на экран.

- Создайте класс `ArtCollection` с методом `__init__`, который создает пустой список произведений.
- Создайте метод `add_artwork`, который принимает название работы и имя художника в качестве аргументов и добавляет их в список.
- Создайте метод `remove_artwork`, который удаляет работу из списка по её названию.
- Создайте метод `artwork_count`, который возвращает общее количество произведений в коллекции.
- Создайте экземпляр класса `ArtCollection` и добавьте несколько работ.
- Выведите информацию о текущих произведениях на экран.
- Выведите общее количество экспонатов на экран.
- Удалите одну из работ и выведите обновленный список.
- Выведите общее количество экспонатов после удаления на экран.

Пример использования:

```
art = ArtCollection()
art.add_artwork("Звёздная ночь", "Ван Гог")
art.add_artwork("Мона Лиза", "Леонардо да Винчи")
art.add_artwork("Крик", "Мунк")
print("Произведения в коллекции:")
for title, artist in art.artworks:
    print(title, "-", artist)
count = art.artwork_count()
print("Всего экспонатов:", count)
art.remove_artwork("Мона Лиза")
print("Произведения после удаления 'Моны Лизы':")
for title, artist in art.artworks:
    print(title, "-", artist)
count = art.artwork_count()
print("Всего экспонатов:", count)
```

Вывод:

```
Произведения в коллекции:
Звёздная ночь - Ван Гог
Мона Лиза - Леонардо да Винчи
Крик - Мунк
Всего экспонатов: 3
Произведения после удаления 'Моны Лизы':
Звёздная ночь - Ван Гог
Крик - Мунк
Всего экспонатов: 2
```

31 Написать программу на Python, которая создает класс `FlightSchedule` для представления расписания рейсов. Класс должен содержать методы для добавления и удаления рейсов, а также подсчета общего количества перелетов. Программа также должна создавать экземпляр класса `FlightSchedule`, добавлять рейсы, удалять рейсы и выводить информацию о расписании на экран.

- Создайте класс `FlightSchedule` с методом `__init__`, который создает пустой список рейсов.
- Создайте метод `add_flight`, который принимает номер рейса и пункт назначения в качестве аргументов и добавляет их в список.
- Создайте метод `remove_flight`, который удаляет рейс из списка по его номеру.
- Создайте метод `flight_count`, который возвращает общее количество рейсов в расписании.
- Создайте экземпляр класса `FlightSchedule` и добавьте несколько рейсов.
- Выведите информацию о текущих рейсах на экран.
- Выведите общее количество перелетов на экран.
- Удалите один из рейсов и выведите обновленное расписание.
- Выведите общее количество перелетов после удаления на экран.

Пример использования:

```
flights = FlightSchedule()
flights.add_flight("SU123", "Париж")
flights.add_flight("SU456", "Лондон")
flights.add_flight("SU789", "Рим")
print("Рейсы:")
for num, dest in flights.flights:
    print(num, "-", dest)
count = flights.flight_count()
print("Всего рейсов:", count)
flights.remove_flight("SU456")
print("Рейсы после отмены SU456:")
for num, dest in flights.flights:
    print(num, "-", dest)
count = flights.flight_count()
print("Всего рейсов:", count)
```

Вывод:

```
Рейсы:
SU123 - Париж
SU456 - Лондон
SU789 - Рим
Всего рейсов: 3
Рейсы после отмены SU456:
SU123 - Париж
SU789 - Рим
Всего рейсов: 2
```


32 Написать программу на Python, которая создает класс `RecipeIngredients` для представления ингредиентов рецепта. Класс должен содержать методы для добавления и удаления ингредиентов, а также подсчета общего количества компонентов. Программа также должна создавать экземпляр класса `RecipeIngredients`, добавлять ингредиенты, удалять ингредиенты и выводить информацию о рецепте на экран.

- Создайте класс `RecipeIngredients` с методом `__init__`, который создает пустой список ингредиентов.
- Создайте метод `add_ingredient`, который принимает название ингредиента и количество (в граммах или штуках) в качестве аргументов и добавляет их в список.
- Создайте метод `remove_ingredient`, который удаляет ингредиент из списка по его названию.
- Создайте метод `ingredient_count`, который возвращает общее количество ингредиентов в рецепте.
- Создайте экземпляр класса `RecipeIngredients` и добавьте несколько ингредиентов.
- Выведите информацию о текущих ингредиентах на экран.
- Выведите общее количество компонентов на экран.
- Удалите один из ингредиентов и выведите обновленный список.
- Выведите общее количество компонентов после удаления на экран.

Пример использования:

```
recipe = RecipeIngredients()
recipe.add_ingredient("Мука", 200)
recipe.add_ingredient("Сахар", 100)
recipe.add_ingredient("Яйца", 2)
print("Ингредиенты рецепта:")
for name, qty in recipe.ingredients:
    print(name, "-", qty)
count = recipe.ingredient_count()
print("Всего ингредиентов:", count)
recipe.remove_ingredient("Сахар")
print("Ингредиенты после удаления сахара:")
for name, qty in recipe.ingredients:
    print(name, "-", qty)
count = recipe.ingredient_count()
print("Всего ингредиентов:", count)
```

Вывод:

```
Ингредиенты рецепта:
Мука - 200
Сахар - 100
Яйца - 2
Всего ингредиентов: 3
Ингредиенты после удаления сахара:
```

Мука - 200
Яйца - 2
Всего ингредиентов: 2

33 Написать программу на Python, которая создает класс `WorkoutPlan` для представления плана тренировок. Класс должен содержать методы для добавления и удаления упражнений, а также подсчета общего количества упражнений. Программа также должна создавать экземпляр класса `WorkoutPlan`, добавлять упражнения, удалять упражнения и выводить информацию о плане на экран.

- Создайте класс `WorkoutPlan` с методом `__init__`, который создает пустой список упражнений.
- Создайте метод `add_exercise`, который принимает название упражнения и количество повторений в качестве аргументов и добавляет их в список.
- Создайте метод `remove_exercise`, который удаляет упражнение из списка по его названию.
- Создайте метод `exercise_count`, который возвращает общее количество упражнений в плане.
- Создайте экземпляр класса `WorkoutPlan` и добавьте несколько упражнений.
- Выведите информацию о текущих упражнениях на экран.
- Выведите общее количество упражнений на экран.
- Удалите одно из упражнений и выведите обновленный план.
- Выведите общее количество упражнений после удаления на экран.

Пример использования:

```
workout = WorkoutPlan()
workout.add_exercise("Бег", 30)
workout.add_exercise("Планка", 3)
workout.add_exercise("Приседания", 20)
print("План тренировки:")
for name, reps in workout.exercises:
    print(name, "-", reps)
count = workout.exercise_count()
print("Всего упражнений:", count)
workout.remove_exercise("Планка")
print("План после удаления планки:")
for name, reps in workout.exercises:
    print(name, "-", reps)
count = workout.exercise_count()
print("Всего упражнений:", count)
```

Вывод:

План тренировки:
Бег - 30
Планка - 3

Приседания - 20
Всего упражнений: 3
План после удаления планки:
Бег - 30
Приседания - 20
Всего упражнений: 2

34 Написать программу на Python, которая создает класс `InventoryManager` для представления управления запасами. Класс должен содержать методы для добавления и удаления товаров, а также подсчета общего количества типов товаров. Программа также должна создавать экземпляр класса `InventoryManager`, добавлять товары, удалять товары и выводить информацию о запасах на экран.

- Создайте класс `InventoryManager` с методом `__init__`, который создает пустой список товаров.
- Создайте метод `add_product`, который принимает название товара и количество единиц в качестве аргументов и добавляет их в список.
- Создайте метод `remove_product`, который удаляет товар из списка по его названию.
- Создайте метод `product_types`, который возвращает общее количество различных типов товаров.
- Создайте экземпляр класса `InventoryManager` и добавьте несколько товаров.
- Выведите информацию о текущих товарах на экран.
- Выведите общее количество типов товаров на экран.
- Удалите один из товаров и выведите обновленный список.
- Выведите общее количество типов товаров после удаления на экран.

Пример использования:

```
inv = InventoryManager()
inv.add_product("Мыло", 100)
inv.add_product("Шампунь", 50)
inv.add_product("Зубная паста", 75)
print("Товары на складе:")
for name, qty in inv.products:
    print(name, "-", qty)
types = inv.product_types()
print("Всего типов товаров:", types)
inv.remove_product("Шампунь")
print("Товары после удаления шампуня:")
for name, qty in inv.products:
    print(name, "-", qty)
types = inv.product_types()
print("Всего типов товаров:", types)
```

Вывод:

Товары на складе:
Мыло - 100
Шампунь - 50
Зубная паста - 75
Всего типов товаров: 3
Товары после удаления шампуня:
Мыло - 100
Зубная паста - 75
Всего типов товаров: 2

35 Написать программу на Python, которая создает класс `EventGuestList` для представления списка гостей мероприятия. Класс должен содержать методы для добавления и удаления гостей, а также подсчета общего количества приглашенных. Программа также должна создавать экземпляр класса `EventGuestList`, добавлять гостей, удалять гостей и выводить информацию о списке на экран.

- Создайте класс `EventGuestList` с методом `__init__`, который создает пустой список гостей.
- Создайте метод `add_guest`, который принимает имя гостя и его статус (например, "подтвержден" "ожидает") в качестве аргументов и добавляет их в список.
- Создайте метод `remove_guest`, который удаляет гостя из списка по имени.
- Создайте метод `guest_count`, который возвращает общее количество гостей в списке.
- Создайте экземпляр класса `EventGuestList` и добавьте несколько гостей.
- Выведите информацию о текущих гостях на экран.
- Выведите общее количество приглашенных на экран.
- Удалите одного из гостей и выведите обновленный список.
- Выведите общее количество приглашенных после удаления на экран.

Пример использования:

```
guests = EventGuestList()
guests.add_guest("Андрей", "подтвержден")
guests.add_guest("Светлана", "ожидает")
guests.add_guest("Михаил", "подтвержден")
print("Список гостей:")
for name, status in guests.guests:
    print(name, "-", status)
count = guests.guest_count()
print("Всего гостей:", count)
guests.remove_guest("Светлана")
print("Список после отмены Светланы:")
for name, status in guests.guests:
    print(name, "-", status)
count = guests.guest_count()
print("Всего гостей:", count)
```

Вывод:

Список гостей:

Андрей - подтвержден

Светлана - ожидает

Михаил - подтвержден

Всего гостей: 3

Список после отмены Светланы:

Андрей - подтвержден

Михаил - подтвержден

Всего гостей: 2

2.4.5 Задача 5

- 1 Написать программу на Python, которая создает класс **Bank**, представляющий банк. Класс должен содержать методы для создания учетных записей клиентов, внесения депозитов, снятия средств и проверки баланса. Программа также должна создавать экземпляр класса **Bank**, создавать учетные записи клиентов, вносить депозиты, снимать средства и проверять баланс.

- Создайте класс **Bank** с методом `__init__`, который создает пустой словарь клиентов.
- Создайте метод `create_account`, который принимает номер счета и начальный баланс в качестве аргументов. Метод должен проверять, существует ли уже номер счета в словаре клиентов. Если это так, он должен выводить сообщение об ошибке. В противном случае, он должен добавить номер счета в словарь клиентов с начальным балансом в качестве значения.
- Создайте метод `make_deposit`, который принимает номер счета и сумму в качестве аргументов. Метод должен проверять, существует ли номер счета в словаре клиентов. Если это так, он должен добавить сумму к текущему балансу счета. Если номер счета не существует, он должен вывести сообщение об ошибке.
- Создайте метод `make_withdrawal`, который принимает номер счета и сумму в качестве аргументов. Метод должен проверять, существует ли номер счета в словаре клиентов. Если это так, он должен проверить, достаточно ли средств на счете для снятия. Если это так, он должен вычесть сумму из текущего баланса счета. В противном случае, он должен вывести сообщение об ошибке, указывающее на недостаточность средств. Если номер счета не существует, он должен вывести сообщение об ошибке.
- Создайте метод `check_balance`, который принимает номер счета в качестве аргумента. Метод должен проверять, существует ли номер счета в словаре клиентов. Если это так, он должен извлечь и вывести текущий баланс счета. Если номер счета не существует, он должен вывести сообщение об ошибке.
- Создайте экземпляр класса **Bank** и создайте учетные записи клиентов.
- Вносите депозиты на счета клиентов.
- Снимайте средства со счетов клиентов.
- Проверяйте баланс счетов клиентов.

Пример использования:

```

bank = Bank()
acno1 = "SB-123"
damt1 = 1000
print("Новый номер счета: ", acno1, " Внесенная сумма: ", damt1)
bank.create_account(acno1, damt1)
acno2 = "SB-124"
damt2 = 1500
print("Новый номер счета: ", acno2, " Внесенная сумма: ", damt2)
bank.create_account(acno2, damt2)
wamt1 = 600
print("\nДепозит средств: ", wamt1, " на счет № ", acno1)
bank.make_deposit(acno1, wamt1)
wamt2 = 350
print("Вывод средств: ", wamt2, " со счета № ", acno2)
bank.make_withdrawal(acno2, wamt2)
print("Номер расчетного счета: ", acno1)
bank.check_balance(acno1)
print("Номер расчетного счета: ", acno2)
bank.check_balance(acno2)
wamt3 = 1200
print("Вывод средств: ", wamt3, " со счета № ", acno2)
bank.make_withdrawal(acno2, wamt3)
acno3 = "SB-134"
print("Проверка баланса счета № ", acno3)
bank.check_balance(acno3) # Non-existent account number

```

2 Написать программу на Python, которая создает класс `CreditUnion`, представляющий кредитный союз. Класс должен содержать методы для открытия счетов участников, пополнения баланса, снятия денег и запроса текущего состояния счета. Программа также должна создавать экземпляр класса `CreditUnion`, открывать счета, выполнять операции и проверять балансы.

- Создайте класс `CreditUnion` с методом `__init__`, инициализирующим пустой словарь счетов.
- Создайте метод `open_account`, принимающий идентификатор счета и стартовый остаток. Если счет уже существует, выведите ошибку; иначе — добавьте запись.
- Создайте метод `deposit`, принимающий идентификатор счета и сумму. Если счет существует, увеличьте баланс; иначе — сообщите об ошибке.
- Создайте метод `withdraw`, принимающий идентификатор счета и сумму. Если счет существует и средств достаточно, уменьшите баланс; иначе — выведите соответствующую ошибку.
- Создайте метод `get_balance`, принимающий идентификатор счета. Если счет существует, выведите его баланс; иначе — сообщите об ошибке.
- Создайте экземпляр `CreditUnion`.
- Откройте несколько счетов.
- Выполните пополнения.
- Выполните снятия.

- Проверьте балансы.

Пример использования:

```
cu = CreditUnion()
cu.open_account("CU-001", 2000)
cu.open_account("CU-002", 500)
cu.deposit("CU-001", 300)
cu.withdraw("CU-002", 200)
cu.get_balance("CU-001")
cu.get_balance("CU-002")
cu.withdraw("CU-002", 400) # недостаточно средств
cu.get_balance("CU-999")  # несуществующий счет
```

3 Написать программу на Python, которая создает класс **SavingsBank**, моделирующий сберегательный банк. Класс должен поддерживать создание счетов, внесение вкладов, снятие средств и проверку баланса. Программа должна демонстрировать работу всех методов на примере нескольких счетов.

- Создайте класс **SavingsBank** с методом `__init__`, инициализирующим пустой словарь `accounts`.
- Метод `add_account` принимает номер счета и начальный депозит. Если счет уже есть — ошибка; иначе — добавление.
- Метод `credit` принимает номер счета и сумму. При наличии счета — пополнение; иначе — ошибка.
- Метод `debit` принимает номер счета и сумму. При наличии счета и достаточном балансе — снятие; иначе — ошибка.
- Метод `show_balance` принимает номер счета и выводит баланс или сообщение об ошибке.
- Создайте экземпляр **SavingsBank**.
- Добавьте два счета.
- Пополните один из них.
- Снимите средства с другого.
- Проверьте балансы обоих и несуществующего счета.

Пример использования:

```
sb = SavingsBank()
sb.add_account("SAV-101", 1000)
sb.add_account("SAV-102", 800)
sb.credit("SAV-101", 200)
sb.debit("SAV-102", 300)
sb.show_balance("SAV-101")
sb.show_balance("SAV-102")
sb.debit("SAV-102", 600) # недостаточно
sb.show_balance("SAV-999") # не существует
```

4 Написать программу на Python, которая создает класс `DigitalWallet`, представляющий цифровой кошелек. Класс должен поддерживать регистрацию кошельков, пополнение, списание и проверку баланса.

- Создайте класс `DigitalWallet` с методом `__init__`, создающим пустой словарь `wallets`.
- Метод `register_wallet` принимает ID кошелька и начальный баланс. Если ID занят — ошибка; иначе — регистрация.
- Метод `top_up` принимает ID и сумму. При существовании кошелька — пополнение; иначе — ошибка.
- Метод `spend` принимает ID и сумму. При наличии кошелька и достаточном балансе — списание; иначе — ошибка.
- Метод `get_wallet_balance` принимает ID и выводит баланс или сообщение об ошибке.
- Создайте экземпляр `DigitalWallet`.
- Зарегистрируйте два кошелька.
- Пополните один.
- Потратьте с другого.
- Проверьте балансы и попытайтесь проверить несуществующий.

Пример использования:

```
dw = DigitalWallet()
dw.register_wallet("WAL-01", 500)
dw.register_wallet("WAL-02", 300)
dw.top_up("WAL-01", 100)
dw.spend("WAL-02", 150)
dw.get_wallet_balance("WAL-01")
dw.get_wallet_balance("WAL-02")
dw.spend("WAL-02", 200) # недостаточно
dw.get_wallet_balance("WAL-99") # не существует
```

5 Написать программу на Python, которая создает класс `PaymentSystem`, моделирующий систему платежей. Класс должен поддерживать создание счетов, зачисление средств, списание и проверку баланса.

- Создайте класс `PaymentSystem` с методом `__init__`, инициализирующим пустой словарь `accounts`.
- Метод `create_user_account` принимает идентификатор и начальный баланс. Если уже есть — ошибка; иначе — создание.
- Метод `credit_account` принимает ID и сумму. При наличии счета — зачисление; иначе — ошибка.
- Метод `debit_account` принимает ID и сумму. При наличии счета и достаточном балансе — списание; иначе — ошибка.
- Метод `check_account_balance` принимает ID и выводит баланс или ошибку.

- Создайте экземпляр `PaymentSystem`.
- Создайте два счета.
- Зачислите средства на один.
- Спишите с другого.
- Проверьте балансы и несуществующий счет.

Пример использования:

```
ps = PaymentSystem()
ps.create_user_account("USR-1", 1200)
ps.create_user_account("USR-2", 700)
ps.credit_account("USR-1", 300)
ps.debit_account("USR-2", 200)
ps.check_account_balance("USR-1")
ps.check_account_balance("USR-2")
ps.debit_account("USR-2", 600) # недостаточно
ps.check_account_balance("USR-999") # не существует
```

6 Написать программу на Python, которая создает класс `MicroFinance`, представляющий микрофинансовую организацию. Класс должен поддерживать открытие счетов, пополнение, снятие и проверку баланса.

- Создайте класс `MicroFinance` с методом `__init__`, создающим пустой словарь `clients`.
- Метод `open_client_account` принимает номер счета и стартовый баланс. Если счет существует — ошибка; иначе — открытие.
- Метод `fund_account` принимает номер счета и сумму. При наличии счета — пополнение; иначе — ошибка.
- Метод `withdraw_funds` принимает номер счета и сумму. При наличии счета и достаточном балансе — снятие; иначе — ошибка.
- Метод `view_balance` принимает номер счета и выводит баланс или сообщение об ошибке.
- Создайте экземпляр `MicroFinance`.
- Откройте два счета.
- Пополните один.
- Снимите с другого.
- Проверьте балансы и несуществующий счет.

Пример использования:

```
mf = MicroFinance()
mf.open_client_account("MF-201", 900)
mf.open_client_account("MF-202", 400)
mf.fund_account("MF-201", 100)
mf.withdraw_funds("MF-202", 150)
```

```
mf.view_balance("MF-201")
mf.view_balance("MF-202")
mf.withdraw_funds("MF-202", 300) # недостаточно
mf.view_balance("MF-999") # не существует
```

7 Написать программу на Python, которая создает класс `OnlineBank`, моделирующий онлайн-банк. Класс должен поддерживать регистрацию счетов, депозиты, выводы и проверку баланса.

- Создайте класс `OnlineBank` с методом `__init__`, инициализирующим пустой словарь `accounts`.
- Метод `register_account` принимает ID и начальный баланс. Если ID занят — ошибка; иначе — регистрация.
- Метод `deposit_funds` принимает ID и сумму. При наличии счета — пополнение; иначе — ошибка.
- Метод `withdraw_funds` принимает ID и сумму. При наличии счета и достаточном балансе — снятие; иначе — ошибка.
- Метод `check_current_balance` принимает ID и выводит баланс или ошибку.
- Создайте экземпляр `OnlineBank`.
- Зарегистрируйте два счета.
- Пополните один.
- Снимите с другого.
- Проверьте балансы и несуществующий счет.

Пример использования:

```
ob = OnlineBank()
ob.register_account("ONB-501", 1500)
ob.register_account("ONB-502", 600)
ob.deposit_funds("ONB-501", 200)
ob.withdraw_funds("ONB-502", 250)
ob.check_current_balance("ONB-501")
ob.check_current_balance("ONB-502")
ob.withdraw_funds("ONB-502", 400) # недостаточно
ob.check_current_balance("ONB-999") # не существует
```

8 Написать программу на Python, которая создает класс `FinTechApp`, представляющий финтех-приложение. Класс должен поддерживать создание аккаунтов, пополнение, снятие и проверку баланса.

- Создайте класс `FinTechApp` с методом `__init__`, создающим пустой словарь `users`.
- Метод `create_user` принимает логин и начальный баланс. Если логин занят — ошибка; иначе — создание.
- Метод `add_money` принимает логин и сумму. При наличии аккаунта — пополнение; иначе — ошибка.

- Метод `remove_money` принимает логин и сумму. При наличии аккаунта и достаточном балансе — снятие; иначе — ошибка.
- Метод `get_user_balance` принимает логин и выводит баланс или ошибку.
- Создайте экземпляр `FinTechApp`.
- Создайте двух пользователей.
- Пополните одного.
- Снимите у другого.
- Проверьте балансы и несуществующего пользователя.

Пример использования:

```
ft = FinTechApp()
ft.create_user("alice", 2000)
ft.create_user("bob", 800)
ft.add_money("alice", 300)
ft.remove_money("bob", 200)
ft.get_user_balance("alice")
ft.get_user_balance("bob")
ft.remove_money("bob", 700) # недостаточно
ft.get_user_balance("charlie") # не существует
```

9 Написать программу на Python, которая создает класс `CryptoWallet`, моделирующий криптовалютный кошелек. Класс должен поддерживать создание кошельков, пополнение, перевод и проверку баланса.

- Создайте класс `CryptoWallet` с методом `__init__`, инициализирующим пустой словарь `wallets`.
- Метод `generate_wallet` принимает адрес и начальный баланс. Если адрес уже есть — ошибка; иначе — создание.
- Метод `receive_coins` принимает адрес и сумму. При наличии кошелька — пополнение; иначе — ошибка.
- Метод `send_coins` принимает адрес и сумму. При наличии кошелька и достаточном балансе — списание; иначе — ошибка.
- Метод `check_wallet_balance` принимает адрес и выводит баланс или ошибку.
- Создайте экземпляр `CryptoWallet`.
- Создайте два кошелька.
- Пополните один.
- Отправьте с другого.
- Проверьте балансы и несуществующий адрес.

Пример использования:

```

cw = CryptoWallet()
cw.generate_wallet("0x1a2b", 10.5)
cw.generate_wallet("0x3c4d", 5.0)
cw.receive_coins("0x1a2b", 2.0)
cw.send_coins("0x3c4d", 1.5)
cw.check_wallet_balance("0x1a2b")
cw.check_wallet_balance("0x3c4d")
cw.send_coins("0x3c4d", 4.0) # недостаточно
cw.check_wallet_balance("0x9999") # не существует

```

- 10 Написать программу на Python, которая создает класс **StudentFund**, представляющий студенческий фонд. Класс должен поддерживать создание счетов студентов, внесение средств, снятие и проверку баланса.

- Создайте класс **StudentFund** с методом `__init__`, создающим пустой словарь `students`.
- Метод `enroll_student` принимает ID студента и начальный грант. Если ID уже есть — ошибка; иначе — зачисление.
- Метод `add_grant` принимает ID и сумму. При наличии студента — пополнение; иначе — ошибка.
- Метод `use_funds` принимает ID и сумму. При наличии студента и достаточном балансе — списание; иначе — ошибка.
- Метод `view_student_balance` принимает ID и выводит баланс или ошибку.
- Создайте экземпляр **StudentFund**.
- Зачислите двух студентов.
- Пополните одного.
- Снимите у другого.
- Проверьте балансы и несуществующего студента.

Пример использования:

```

sf = StudentFund()
sf.enroll_student("STU-01", 5000)
sf.enroll_student("STU-02", 3000)
sf.add_grant("STU-01", 1000)
sf.use_funds("STU-02", 800)
sf.view_student_balance("STU-01")
sf.view_student_balance("STU-02")
sf.use_funds("STU-02", 2500) # недостаточно
sf.view_student_balance("STU-99") # не существует

```

- 11 Написать программу на Python, которая создает класс **GameCurrency**, моделирующий внутриигровую валюту. Класс должен поддерживать создание аккаунтов игроков, начисление монет, трату и проверку баланса.

- Создайте класс **GameCurrency** с методом `__init__`, инициализирующим пустой словарь `players`.

- Метод `create_player` принимает ник и начальный баланс. Если ник занят — ошибка; иначе — создание.
- Метод `award_coins` принимает ник и сумму. При наличии игрока — начисление; иначе — ошибка.
- Метод `spend_coins` принимает ник и сумму. При наличии игрока и достаточном балансе — списание; иначе — ошибка.
- Метод `get_player_balance` принимает ник и выводит баланс или ошибку.
- Создайте экземпляр `GameCurrency`.
- Создайте двух игроков.
- Начислите одному.
- Потратьте у другого.
- Проверьте балансы и несуществующего игрока.

Пример использования:

```
gc = GameCurrency()
gc.create_player("hero1", 100)
gc.create_player("hero2", 75)
gc.award_coins("hero1", 25)
gc.spend_coins("hero2", 30)
gc.get_player_balance("hero1")
gc.get_player_balance("hero2")
gc.spend_coins("hero2", 50) # недостаточно
gc.get_player_balance("hero99") # не существует
```

12 Написать программу на Python, которая создает класс `CharityFund`, представляющий благотворительный фонд. Класс должен поддерживать создание счетов доноров, получение пожертвований, выдачу средств и проверку баланса.

- Создайте класс `CharityFund` с методом `__init__`, создающим пустой словарь `donors`.
- Метод `register_donor` принимает ID и начальный взнос. Если ID есть — ошибка; иначе — регистрация.
- Метод `accept_donation` принимает ID и сумму. При наличии донора — пополнение; иначе — ошибка.
- Метод `distribute_funds` принимает ID и сумму. При наличии донора и достаточном балансе — списание; иначе — ошибка.
- Метод `check_donor_balance` принимает ID и выводит баланс или ошибку.
- Создайте экземпляр `CharityFund`.
- Зарегистрируйте двух доноров.
- Примите пожертвование от одного.
- Распределите средства от другого.
- Проверьте балансы и несуществующего донора.

Пример использования:

```

cf = CharityFund()
cf.register_donor("DON-1", 2000)
cf.register_donor("DON-2", 1500)
cf.accept_donation("DON-1", 500)
cf.distribute_funds("DON-2", 600)
cf.check_donor_balance("DON-1")
cf.check_donor_balance("DON-2")
cf.distribute_funds("DON-2", 1000) # недостаточно
cf.check_donor_balance("DON-99") # не существует

```

- 13 Написать программу на Python, которая создает класс **TravelWallet**, моделирующий кошелек для путешествий. Класс должен поддерживать создание профилей, пополнение, оплату и проверку баланса.

- Создайте класс **TravelWallet** с методом `__init__`, инициализирующим пустой словарь `profiles`.
- Метод `create_profile` принимает имя профиля и начальный бюджет. Если профиль существует — ошибка; иначе — создание.
- Метод `load_funds` принимает имя профиля и сумму. При наличии профиля — пополнение; иначе — ошибка.
- Метод `pay_expense` принимает имя профиля и сумму. При наличии профиля и достаточном балансе — списание; иначе — ошибка.
- Метод `check_budget` принимает имя профиля и выводит баланс или ошибку.
- Создайте экземпляр **TravelWallet**.
- Создайте два профиля.
- Пополните один.
- Оплатите по другому.
- Проверьте балансы и несуществующий профиль.

Пример использования:

```

tw = TravelWallet()
tw.create_profile("ParisTrip", 3000)
tw.create_profile("TokyoTrip", 2500)
tw.load_funds("ParisTrip", 500)
tw.pay_expense("TokyoTrip", 700)
tw.check_budget("ParisTrip")
tw.check_budget("TokyoTrip")
tw.pay_expense("TokyoTrip", 2000) # недостаточно
tw.check_budget("LondonTrip") # не существует

```

- 14 Написать программу на Python, которая создает класс **SchoolFund**, представляющий школьный фонд. Класс должен поддерживать создание счетов классов, внесение средств, расход и проверку баланса.

- Создайте класс **SchoolFund** с методом `__init__`, создающим пустой словарь `classes`.

- Метод `add_class` принимает номер класса и начальный бюджет. Если класс уже есть — ошибка; иначе — добавление.
- Метод `collect_money` принимает номер класса и сумму. При наличии класса — пополнение; иначе — ошибка.
- Метод `spend_money` принимает номер класса и сумму. При наличии класса и достаточном бюджете — списание; иначе — ошибка.
- Метод `get_class_balance` принимает номер класса и выводит баланс или ошибку.
- Создайте экземпляр `SchoolFund`.
- Добавьте два класса.
- Соберите средства у одного.
- Потратьте у другого.
- Проверьте балансы и несуществующий класс.

Пример использования:

```
sf = SchoolFund()
sf.add_class("10A", 1200)
sf.add_class("11B", 900)
sf.collect_money("10A", 300)
sf.spend_money("11B", 400)
sf.get_class_balance("10A")
sf.get_class_balance("11B")
sf.spend_money("11B", 600) # недостаточно
sf.get_class_balance("12C") # не существует
```

- 15 Написать программу на Python, которая создает класс `ClubAccount`, моделирующий счет клуба. Класс должен поддерживать создание счетов участников, пополнение взносами, снятие на мероприятия и проверку баланса.

- Создайте класс `ClubAccount` с методом `__init__`, инициализирующим пустой словарь `members`.
- Метод `join_club` принимает ID участника и вступительный взнос. Если ID есть — ошибка; иначе — добавление.
- Метод `pay_dues` принимает ID и сумму. При наличии участника — пополнение; иначе — ошибка.
- Метод `request_funds` принимает ID и сумму. При наличии участника и достаточном балансе — списание; иначе — ошибка.
- Метод `check_member_balance` принимает ID и выводит баланс или ошибку.
- Создайте экземпляр `ClubAccount`.
- Зарегистрируйте двух участников.
- Внесите взносы за одного.
- Запросите средства у другого.
- Проверьте балансы и несуществующего участника.

Пример использования:

```
ca = ClubAccount()
ca.join_club("MEM-01", 500)
ca.join_club("MEM-02", 400)
ca.pay_dues("MEM-01", 100)
ca.request_funds("MEM-02", 150)
ca.check_member_balance("MEM-01")
ca.check_member_balance("MEM-02")
ca.request_funds("MEM-02", 300) # недостаточно
ca.check_member_balance("MEM-99") # не существует
```

16 Написать программу на Python, которая создает класс `ProjectBudget`, представляющий бюджет проекта. Класс должен поддерживать создание проектов, выделение средств, расход и проверку остатка.

- Создайте класс `ProjectBudget` с методом `__init__`, создающим пустой словарь `projects`.
- Метод `initiate_project` принимает код проекта и начальный бюджет. Если проект существует — ошибка; иначе — инициализация.
- Метод `allocate_funds` принимает код проекта и сумму. При наличии проекта — пополнение; иначе — ошибка.
- Метод `expend_funds` принимает код проекта и сумму. При наличии проекта и достаточном бюджете — списание; иначе — ошибка.
- Метод `check_project_balance` принимает код проекта и выводит баланс или ошибку.
- Создайте экземпляр `ProjectBudget`.
- Иницилируйте два проекта.
- Выделите средства одному.
- Потратьте у другого.
- Проверьте балансы и несуществующий проект.

Пример использования:

```
pb = ProjectBudget()
pb.initiate_project("PRJ-Alpha", 10000)
pb.initiate_project("PRJ-Beta", 8000)
pb.allocate_funds("PRJ-Alpha", 2000)
pb.expend_funds("PRJ-Beta", 3000)
pb.check_project_balance("PRJ-Alpha")
pb.check_project_balance("PRJ-Beta")
pb.expend_funds("PRJ-Beta", 6000) # недостаточно
pb.check_project_balance("PRJ-Gamma") # не существует
```

17 Написать программу на Python, которая создает класс `EventFund`, моделирующий фонд мероприятия. Класс должен поддерживать создание событий, сбор средств, оплату расходов и проверку баланса.

- Создайте класс `EventFund` с методом `__init__`, инициализирующим пустой словарь `events`.
- Метод `create_event` принимает название события и стартовый бюджет. Если событие есть — ошибка; иначе — создание.
- Метод `collect_sponsorship` принимает название и сумму. При наличии события — пополнение; иначе — ошибка.
- Метод `pay_vendor` принимает название и сумму. При наличии события и достаточном бюджете — списание; иначе — ошибка.
- Метод `view_event_balance` принимает название и выводит баланс или ошибку.
- Создайте экземпляр `EventFund`.
- Создайте два события.
- Соберите спонсорские средства для одного.
- Оплатите поставщика для другого.
- Проверьте балансы и несуществующее событие.

Пример использования:

```
ef = EventFund()
ef.create_event("Conference", 5000)
ef.create_event("Workshop", 3000)
ef.collect_sponsorship("Conference", 1500)
ef.pay_vendor("Workshop", 1000)
ef.view_event_balance("Conference")
ef.view_event_balance("Workshop")
ef.pay_vendor("Workshop", 2500) # недостаточно
ef.view_event_balance("Seminar") # не существует
```

- 18 Написать программу на Python, которая создает класс `PersonalFinance`, представляющий личные финансы. Класс должен поддерживать создание категорий, пополнение доходами, списание расходами и проверку баланса.

- Создайте класс `PersonalFinance` с методом `__init__`, создающим пустой словарь `categories`.
- Метод `add_category` принимает название категории и начальный баланс. Если категория есть — ошибка; иначе — добавление.
- Метод `record_income` принимает название и сумму. При наличии категории — пополнение; иначе — ошибка.
- Метод `record_expense` принимает название и сумму. При наличии категории и достаточном балансе — списание; иначе — ошибка.
- Метод `check_category_balance` принимает название и выводит баланс или ошибку.
- Создайте экземпляр `PersonalFinance`.
- Добавьте две категории.
- Запишите доход в одну.

- Запишите расход в другую.
- Проверьте балансы и несуществующую категорию.

Пример использования:

```
pf = PersonalFinance()
pf.add_category("Salary", 25000)
pf.add_category("Entertainment", 2000)
pf.record_income("Salary", 5000)
pf.record_expense("Entertainment", 800)
pf.check_category_balance("Salary")
pf.check_category_balance("Entertainment")
pf.record_expense("Entertainment", 1500) # недостаточно
pf.check_category_balance("Travel") # не существует
```

19 Написать программу на Python, которая создает класс `InvestmentAccount`, моделирующий инвестиционный счет. Класс должен поддерживать создание счетов, внесение капитала, снятие прибыли и проверку баланса.

- Создайте класс `InvestmentAccount` с методом `__init__`, инициализирующим пустой словарь `accounts`.
- Метод `open_investment` принимает ID счета и начальный капитал. Если счет есть — ошибка; иначе — открытие.
- Метод `invest_more` принимает ID и сумму. При наличии счета — пополнение; иначе — ошибка.
- Метод `withdraw_profit` принимает ID и сумму. При наличии счета и достаточном балансе — списание; иначе — ошибка.
- Метод `check_investment_balance` принимает ID и выводит баланс или ошибку.
- Создайте экземпляр `InvestmentAccount`.
- Откройте два счета.
- Инвестируйте дополнительно в один.
- Снимите прибыль с другого.
- Проверьте балансы и несуществующий счет.

Пример использования:

```
ia = InvestmentAccount()
ia.open_investment("INV-01", 10000)
ia.open_investment("INV-02", 7000)
ia.invest_more("INV-01", 2000)
ia.withdraw_profit("INV-02", 1500)
ia.check_investment_balance("INV-01")
ia.check_investment_balance("INV-02")
ia.withdraw_profit("INV-02", 6000) # недостаточно
ia.check_investment_balance("INV-99") # не существует
```

20 Написать программу на Python, которая создает класс **FamilyBudget**, представляющий семейный бюджет. Класс должен поддерживать создание членов семьи, пополнение общими доходами, списание личными расходами и проверку баланса.

- Создайте класс **FamilyBudget** с методом `__init__`, создающим пустой словарь `members`.
- Метод `add_family_member` принимает имя и начальный вклад. Если имя есть — ошибка; иначе — добавление.
- Метод `add_income` принимает имя и сумму. При наличии члена — пополнение; иначе — ошибка.
- Метод `deduct_expense` принимает имя и сумму. При наличии члена и достаточном балансе — списание; иначе — ошибка.
- Метод `check_member_balance` принимает имя и выводит баланс или ошибку.
- Создайте экземпляр **FamilyBudget**.
- Добавьте двух членов семьи.
- Добавьте доход одному.
- Спишите расход у другого.
- Проверьте балансы и несуществующего члена.

Пример использования:

```
fb = FamilyBudget()
fb.add_family_member("Mother", 20000)
fb.add_family_member("Father", 25000)
fb.add_income("Mother", 5000)
fb.deduct_expense("Father", 3000)
fb.check_member_balance("Mother")
fb.check_member_balance("Father")
fb.deduct_expense("Father", 23000) # недостаточно
fb.check_member_balance("Child")  # не существует
```

21 Написать программу на Python, которая создает класс **StartupFund**, моделирующий фонд стартапа. Класс должен поддерживать создание стартапов, привлечение инвестиций, оплату расходов и проверку баланса.

- Создайте класс **StartupFund** с методом `__init__`, инициализирующим пустой словарь `startups`.
- Метод `launch_startup` принимает название и начальный капитал. Если стартап есть — ошибка; иначе — запуск.
- Метод `attract_investment` принимает название и сумму. При наличии стартапа — пополнение; иначе — ошибка.
- Метод `cover_costs` принимает название и сумму. При наличии стартапа и достаточном балансе — списание; иначе — ошибка.
- Метод `check_startup_balance` принимает название и выводит баланс или ошибку.

- Создайте экземпляр `StartupFund`.
- Запустите два стартапа.
- Привлеките инвестиции в один.
- Покройте расходы другого.
- Проверьте балансы и несуществующий стартап.

Пример использования:

```
sf = StartupFund()
sf.launch_startup("TechApp", 50000)
sf.launch_startup("EcoShop", 30000)
sf.attract_investment("TechApp", 20000)
sf.cover_costs("EcoShop", 10000)
sf.check_startup_balance("TechApp")
sf.check_startup_balance("EcoShop")
sf.cover_costs("EcoShop", 25000) # недостаточно
sf.check_startup_balance("FoodDelivery") # несуществует
```

22 Написать программу на Python, которая создает класс `NonProfitAccount`, представляющий счет некоммерческой организации. Класс должен поддерживать создание проектов, получение грантов, оплату деятельности и проверку баланса.

- Создайте класс `NonProfitAccount` с методом `__init__`, создающим пустой словарь `projects`.
- Метод `initiate_nonprofit_project` принимает ID и начальный грант. Если проект есть — ошибка; иначе — инициализация.
- Метод `receive_grant` принимает ID и сумму. При наличии проекта — пополнение; иначе — ошибка.
- Метод `pay_operational_costs` принимает ID и сумму. При наличии проекта и достаточном балансе — списание; иначе — ошибка.
- Метод `check_project_funds` принимает ID и выводит баланс или ошибку.
- Создайте экземпляр `NonProfitAccount`.
- Иницилируйте два проекта.
- Получите грант на один.
- Оплатите расходы другого.
- Проверьте балансы и несуществующий проект.

Пример использования:

```
np = NonProfitAccount()
np.initiate_nonprofit_project("EDU-01", 15000)
np.initiate_nonprofit_project("HEALTH-02", 12000)
np.receive_grant("EDU-01", 5000)
np.pay_operational_costs("HEALTH-02", 4000)
np.check_project_funds("EDU-01")
np.check_project_funds("HEALTH-02")
np.pay_operational_costs("HEALTH-02", 9000) # недостаточно
np.check_project_funds("ENV-99") # несуществует
```

23 Написать программу на Python, которая создает класс **FreelancerWallet**, моделирующий кошелек фрилансера. Класс должен поддерживать создание профилей, получение оплаты, оплату налогов и проверку баланса.

- Создайте класс **FreelancerWallet** с методом `__init__`, инициализирующим пустой словарь **freelancers**.
- Метод **register_freelancer** принимает ник и начальный баланс. Если ник есть — ошибка; иначе — регистрация.
- Метод **receive_payment** принимает ник и сумму. При наличии фрилансера — пополнение; иначе — ошибка.
- Метод **pay_taxes** принимает ник и сумму. При наличии фрилансера и достаточном балансе — списание; иначе — ошибка.
- Метод **check_freelancer_balance** принимает ник и выводит баланс или ошибку.
- Создайте экземпляр **FreelancerWallet**.
- Зарегистрируйте двух фрилансеров.
- Получите оплату для одного.
- Оплатите налоги для другого.
- Проверьте балансы и несуществующего фрилансера.

Пример использования:

```
fw = FreelancerWallet()
fw.register_freelancer("dev_alex", 0)
fw.register_freelancer("design_maria", 0)
fw.receive_payment("dev_alex", 10000)
fw.receive_payment("design_maria", 2500)
fw.pay_taxes("design_maria", 2000)
fw.check_freelancer_balance("dev_alex")
fw.check_freelancer_balance("design_maria")
fw.pay_taxes("design_maria", 1000) # недостаточно
fw.check_freelancer_balance("writer_john") # не существует
```

24 Написать программу на Python, которая создает класс **RentalIncome**, представляющий доход от аренды. Класс должен поддерживать создание объектов недвижимости, получение арендной платы, оплату расходов и проверку баланса.

- Создайте класс **RentalIncome** с методом `__init__`, создающим пустой словарь **properties**.
- Метод **add_property** принимает адрес и начальный баланс. Если адрес есть — ошибка; иначе — добавление.
- Метод **collect_rent** принимает адрес и сумму. При наличии объекта — пополнение; иначе — ошибка.
- Метод **pay_maintenance** принимает адрес и сумму. При наличии объекта и достаточном балансе — списание; иначе — ошибка.
- Метод **check_property_balance** принимает адрес и выводит баланс или ошибку.

- Создайте экземпляр `RentalIncome`.
- Добавьте два объекта.
- Соберите арендную плату с одного.
- Оплатите обслуживание другого.
- Проверьте балансы и несуществующий адрес.

Пример использования:

```
ri = RentalIncome()
ri.add_property("123 Main St", 0)
ri.add_property("456 Oak Ave", 0)
ri.collect_rent("123 Main St", 2000)
ri.collect_rent("456 Oak Ave", 700)
ri.pay_maintenance("456 Oak Ave", 300)
ri.check_property_balance("123 Main St")
ri.check_property_balance("456 Oak Ave")
ri.pay_maintenance("456 Oak Ave", 500) # недостаточно
ri.check_property_balance("789 Pine Rd") # не существует
```

25 Написать программу на Python, которая создает класс `ScholarshipFund`, моделирующий стипендиальный фонд. Класс должен поддерживать создание получателей, выдачу стипендий, возврат средств и проверку баланса.

- Создайте класс `ScholarshipFund` с методом `__init__`, инициализирующим пустой словарь `recipients`.
- Метод `enroll_recipient` принимает ID и начальную стипендию. Если ID есть — ошибка; иначе — зачисление.
- Метод `award_scholarship` принимает ID и сумму. При наличии получателя — пополнение; иначе — ошибка.
- Метод `return_funds` принимает ID и сумму. При наличии получателя и достаточном балансе — списание; иначе — ошибка.
- Метод `check_recipient_balance` принимает ID и выводит баланс или ошибку.
- Создайте экземпляр `ScholarshipFund`.
- Зачислите двух получателей.
- Выдайте стипендию одному.
- Примите возврат от другого.
- Проверьте балансы и несуществующего получателя.

Пример использования:

```
sf = ScholarshipFund()
sf.enroll_recipient("SCH-01", 5000)
sf.enroll_recipient("SCH-02", 4000)
sf.award_scholarship("SCH-01", 1000)
sf.return_funds("SCH-02", 500)
```

```

sf.check_recipient_balance("SCH-01")
sf.check_recipient_balance("SCH-02")
sf.return_funds("SCH-02", 4000) # недостаточно
sf.check_recipient_balance("SCH-99") # не существует

```

26 Написать программу на Python, которая создает класс **Crowdfunding**, представляющий краудфандинговую платформу. Класс должен поддерживать создание кампаний, сбор средств, возврат пожертвований и проверку баланса.

- Создайте класс **Crowdfunding** с методом `__init__`, создающим пустой словарь `campaigns`.
- Метод `start_campaign` принимает название и начальный баланс. Если кампания есть — ошибка; иначе — создание.
- Метод `donate` принимает название и сумму. При наличии кампании — пополнение; иначе — ошибка.
- Метод `refund` принимает название и сумму. При наличии кампании и достаточном балансе — списание; иначе — ошибка.
- Метод `check_campaign_balance` принимает название и выводит баланс или ошибку.
- Создайте экземпляр **Crowdfunding**.
- Запустите две кампании.
- Пожертвуйте в одну.
- Верните средства из другой.
- Проверьте балансы и несуществующую кампанию.

Пример использования:

```

cf = Crowdfunding()
cf.start_campaign("BookPublish", 10000)
cf.start_campaign("ArtExhibit", 8000)
cf.donate("BookPublish", 3000)
cf.refund("ArtExhibit", 500)
cf.check_campaign_balance("BookPublish")
cf.check_campaign_balance("ArtExhibit")
cf.refund("ArtExhibit", 8000) # недостаточно
cf.check_campaign_balance("FilmProject") # не существует

```

27 Написать программу на Python, которая создает класс **PiggyBank**, моделирующий копилку. Класс должен поддерживать создание копилки, добавление монет, извлечение средств и проверку баланса.

- Создайте класс **PiggyBank** с методом `__init__`, инициализирующим пустой словарь `banks`.
- Метод `create_piggy` принимает имя и начальную сумму. Если имя есть — ошибка; иначе — создание.

- Метод `add_coins` принимает имя и сумму. При наличии копилки — пополнение; иначе — ошибка.
- Метод `break_piggy` принимает имя и сумму. При наличии копилки и достаточном балансе — списание; иначе — ошибка.
- Метод `check_piggy_balance` принимает имя и выводит баланс или ошибку.
- Создайте экземпляр `PiggyBank`.
- Создайте две копилки.
- Добавьте монеты в одну.
- Разбейте другую частично.
- Проверьте балансы и несуществующую копилку.

Пример использования:

```
pb = PiggyBank()
pb.create_piggy("Vacation", 200)
pb.create_piggy("Gadget", 150)
pb.add_coins("Vacation", 100)
pb.break_piggy("Gadget", 50)
pb.check_piggy_balance("Vacation")
pb.check_piggy_balance("Gadget")
pb.break_piggy("Gadget", 120) # недостаточно
pb.check_piggy_balance("Car") # не существует
```

28 Написать программу на Python, которая создает класс `BusinessAccount`, представляющий бизнес-счет. Класс должен поддерживать создание компаний, зачисление выручки, оплату счетов и проверку баланса.

- Создайте класс `BusinessAccount` с методом `__init__`, создающим пустой словарь `companies`.
- Метод `register_business` принимает название и начальный капитал. Если компания есть — ошибка; иначе — регистрация.
- Метод `record_revenue` принимает название и сумму. При наличии компании — пополнение; иначе — ошибка.
- Метод `pay_bills` принимает название и сумму. При наличии компании и достаточном балансе — списание; иначе — ошибка.
- Метод `check_business_balance` принимает название и выводит баланс или ошибку.
- Создайте экземпляр `BusinessAccount`.
- Зарегистрируйте две компании.
- Запишите выручку одной.
- Оплатите счета другой.
- Проверьте балансы и несуществующую компанию.

Пример использования:


```

ba = BusinessAccount()
ba.register_business("TechCorp", 50000)
ba.register_business("CafeLtd", 20000)
ba.record_revenue("TechCorp", 15000)
ba.pay_bills("CafeLtd", 3000)
ba.check_business_balance("TechCorp")
ba.check_business_balance("CafeLtd")
ba.pay_bills("CafeLtd", 18000) # недостаточно
ba.check_business_balance("ShopInc") # не существует

```

29 Написать программу на Python, которая создает класс **GrantManager**, моделирующий управление грантами. Класс должен поддерживать создание грантов, выделение средств, отчетность и проверку баланса.

- Создайте класс **GrantManager** с методом `__init__`, инициализирующим пустой словарь `grants`.
- Метод `issue_grant` принимает код и сумму. Если код есть — ошибка; иначе — создание.
- Метод `disburse_funds` принимает код и сумму. При наличии гранта и достаточном балансе — списание (выдача средств); иначе — ошибка.
- Метод `submit_report` принимает код и сумму. При наличии гранта и достаточном балансе — списание; иначе — ошибка.
- Метод `check_grant_status` принимает код и выводит баланс или ошибку.
- Создайте экземпляр **GrantManager**.
- Выдайте два гранта.
- Распределите средства по одному.
- Подайте отчет по другому.
- Проверьте статусы и несуществующий грант.

Пример использования:

```

gm = GrantManager()
gm.issue_grant("GR-2024-01", 10000)
gm.issue_grant("GR-2024-02", 8000)
gm.disburse_funds("GR-2024-01", 4000)
gm.submit_report("GR-2024-02", 2000)
gm.check_grant_status("GR-2024-01")
gm.check_grant_status("GR-2024-02")
gm.submit_report("GR-2024-02", 7000) # недостаточно
gm.check_grant_status("GR-2024-99") # не существует

```

30 Написать программу на Python, которая создает класс **SubscriptionService**, представляющий сервис подписок. Класс должен поддерживать создание пользователей, оплату подписок, возврат средств и проверку баланса.

- Создайте класс **SubscriptionService** с методом `__init__`, создающим пустой словарь `subscribers`.

- Метод `subscribe_user` принимает email и начальный баланс. Если email есть — ошибка; иначе — подписка.
- Метод `charge_payment` принимает email и сумму. При наличии пользователя — пополнение; иначе — ошибка.
- Метод `refund_payment` принимает email и сумму. При наличии пользователя и достаточном балансе — списание; иначе — ошибка.
- Метод `check_subscription_balance` принимает email и выводит баланс или ошибку.
- Создайте экземпляр `SubscriptionService`.
- Подпишите двух пользователей.
- Спишите оплату с одного.
- Верните средства другому.
- Проверьте балансы и несуществующий email.

Пример использования:

```
ss = SubscriptionService()
ss.subscribe_user("user1@example.com", 100)
ss.subscribe_user("user2@example.com", 80)
ss.charge_payment("user1@example.com", 20)
ss.refund_payment("user2@example.com", 10)
ss.check_subscription_balance("user1@example.com")
ss.check_subscription_balance("user2@example.com")
ss.refund_payment("user2@example.com", 80) # недостаточно
ss.check_subscription_balance("user3@example.com") # не существует
```

31 Написать программу на Python, которая создает класс `LoyaltyProgram`, моделирующий программу лояльности. Класс должен поддерживать создание участников, начисление баллов, списание за вознаграждения и проверку баланса.

- Создайте класс `LoyaltyProgram` с методом `__init__`, инициализирующим пустой словарь `members`.
- Метод `enroll_member` принимает ID и начальные баллы. Если ID есть — ошибка; иначе — зачисление.
- Метод `earn_points` принимает ID и количество. При наличии участника — пополнение; иначе — ошибка.
- Метод `redeem_points` принимает ID и количество. При наличии участника и достаточном балансе — списание; иначе — ошибка.
- Метод `check_points_balance` принимает ID и выводит баланс или ошибку.
- Создайте экземпляр `LoyaltyProgram`.
- Зачислите двух участников.
- Начислите баллы одному.
- Спишите у другого.
- Проверьте балансы и несуществующего участника.

Пример использования:

```
lp = LoyaltyProgram()
lp.enroll_member("MEM-101", 500)
lp.enroll_member("MEM-102", 300)
lp.earn_points("MEM-101", 200)
lp.redeem_points("MEM-102", 100)
lp.check_points_balance("MEM-101")
lp.check_points_balance("MEM-102")
lp.redeem_points("MEM-102", 250) # недостаточно
lp.check_points_balance("MEM-999") # не существует
```

- 32 Написать программу на Python, которая создает класс `UtilityBill`, представляющий оплату коммунальных услуг. Класс должен поддерживать создание лицевых счетов, внесение платежей, списание задолженностей и проверку баланса.

- Создайте класс `UtilityBill` с методом `__init__`, создающим пустой словарь `accounts`.
- Метод `create_utility_account` принимает номер и начальный долг. Если номер есть — ошибка; иначе — создание.
- Метод `make_payment` принимает номер и сумму. При наличии счета — пополнение; иначе — ошибка.
- Метод `apply_charges` принимает номер и сумму. При наличии счета и достаточном балансе — списание; иначе — ошибка.
- Метод `check_account_status` принимает номер и выводит баланс или ошибку.
- Создайте экземпляр `UtilityBill`.
- Создайте два счета.
- Внесите платеж по одному.
- Начислите плату по другому.
- Проверьте статусы и несуществующий счет.

Пример использования:

```
ub = UtilityBill()
ub.create_utility_account("UTIL-01", 0)
ub.create_utility_account("UTIL-02", 0)
ub.make_payment("UTIL-01", 1500)
ub.make_payment("UTIL-02", 1200)
ub.apply_charges("UTIL-02", 800)
ub.check_account_status("UTIL-01")
ub.check_account_status("UTIL-02")
ub.apply_charges("UTIL-02", 1000) # недостаточно
ub.check_account_status("UTIL-99") # не существует
```

- 33 Написать программу на Python, которая создает класс `InsuranceFund`, моделирующий страховой фонд. Класс должен поддерживать создание полисов, уплату премий, выплату возмещений и проверку баланса.

- Создайте класс `InsuranceFund` с методом `__init__`, инициализирующим пустой словарь `policies`.
- Метод `issue_policy` принимает номер полиса и начальный взнос. Если полис есть — ошибка; иначе — выдача.
- Метод `pay_premium` принимает номер и сумму. При наличии полиса — пополнение; иначе — ошибка.
- Метод `process_claim` принимает номер и сумму. При наличии полиса и достаточном балансе — списание; иначе — ошибка.
- Метод `check_policy_balance` принимает номер и выводит баланс или ошибку.
- Создайте экземпляр `InsuranceFund`.
- Выдайте два полиса.
- Уплатите премию по одному.
- Обработайте заявку по другому.
- Проверьте балансы и несуществующий полис.

Пример использования:

```
ifund = InsuranceFund()
ifund.issue_policy("POL-501", 10000)
ifund.issue_policy("POL-502", 8000)
ifund.pay_premium("POL-501", 2000)
ifund.process_claim("POL-502", 3000)
ifund.check_policy_balance("POL-501")
ifund.check_policy_balance("POL-502")
ifund.process_claim("POL-502", 6000) # недостаточно
ifund.check_policy_balance("POL-999") # не существует
```

34 Написать программу на Python, которая создает класс `DonationBox`, представляющий ящик для пожертвований. Класс должен поддерживать создание ящиков, сбор средств, выдачу помощи и проверку баланса.

- Создайте класс `DonationBox` с методом `__init__`, создающим пустой словарь `boxes`.
- Метод `install_box` принимает локацию и начальный сбор. Если локация есть — ошибка; иначе — установка.
- Метод `collect_donations` принимает локацию и сумму. При наличии ящика — пополнение; иначе — ошибка.
- Метод `distribute_aid` принимает локацию и сумму. При наличии ящика и достаточном балансе — списание; иначе — ошибка.
- Метод `check_box_balance` принимает локацию и выводит баланс или ошибку.
- Создайте экземпляр `DonationBox`.
- Установите два ящика.
- Соберите пожертвования в один.
- Распределите помощь из другого.

- Проверьте балансы и несуществующую локацию.

Пример использования:

```
db = DonationBox()
db.install_box("Hospital", 0)
db.install_box("School", 0)
db.collect_donations("Hospital", 5000)
db.collect_donations("School", 1500)
db.distribute_aid("School", 1000)
db.check_box_balance("Hospital")
db.check_box_balance("School")
db.distribute_aid("School", 2000) # недостаточно
db.check_box_balance("Park") # не существует
```

35 Написать программу на Python, которая создает класс **RewardWallet**, моделирующий кошелек вознаграждений. Класс должен поддерживать создание кошельков, начисление бонусов, списание за покупки и проверку баланса.

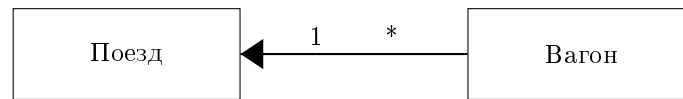
- Создайте класс **RewardWallet** с методом `__init__`, инициализирующим пустой словарь `wallets`.
- Метод `activate_wallet` принимает ID и начальные бонусы. Если ID есть — ошибка; иначе — активация.
- Метод `award_bonus` принимает ID и сумму. При наличии кошелька — пополнение; иначе — ошибка.
- Метод `redeem_reward` принимает ID и сумму. При наличии кошелька и достаточном балансе — списание; иначе — ошибка.
- Метод `check_reward_balance` принимает ID и выводит баланс или ошибку.
- Создайте экземпляр **RewardWallet**.
- Активируйте два кошелька.
- Начислите бонусы одному.
- Потратьте у другого.
- Проверьте балансы и несуществующий ID.

Пример использования:

```
rw = RewardWallet()
rw.activate_wallet("RW-001", 1000)
rw.activate_wallet("RW-002", 800)
rw.award_bonus("RW-001", 200)
rw.redeem_reward("RW-002", 300)
rw.check_reward_balance("RW-001")
rw.check_reward_balance("RW-002")
rw.redeem_reward("RW-002", 600) # недостаточно
rw.check_reward_balance("RW-999") # не существует
```

2.5 Семинар «Композиция» (2 часа)

Теория



Композиция — это концепция, позволяющая моделировать отношения между классами в программе. Она представляет собой один из способов организации взаимодействия классов. Композиция позволяет создавать сложные типы, комбинируя объекты других типов. Это означает, что один класс («целое») может содержать экземпляр другого класса («часть») как своё поле.

Классы, содержащие объекты других классов, обычно называются *композициями* (*composites*), а классы, используемые для создания более сложных типов, называются *компонентами* (*components*).

Композиция позволяет повторно использовать код путём включения объектов других классов, избегая при этом наследования.

Вопросы для подготовки к защите

- Что такое композиция?
- Сравните композицию и наследование.
- Опишите достоинства и недостатки указания типов аргументов и результатов у функций.

В ходе выполнения используйте свойство инкапсуляции (даже если в формулировке задания это не учтено).

2.5.1 Задача 1

1 Формирование состава груженных контейнеров

- (a) Создайте класс `Container`, который будет представлять собой контейнер с грузом. В конструкторе класса `Container` инициализируйте значения контейнера и груза из списков `NumList` и `MasList`, которые объявлены как общие атрибуты класса. `NumList: list[str]` — это список контейнеров (не менее 14):

```
["Контейнер_1", "Контейнер_2", ..., "Контейнер_14"]
```

`MasList: list[str]` — это список грузов для контейнеров (не менее 4):

```
["Электроника", "Мебель", "Одежда", "Продукты"]
```

Конструктор должен иметь сигнатуру: `__init__(self) -> None`.

- (b) Создайте класс `TrainOfContainers`, который будет представлять собой состав, состоящий из моделей контейнеров. В конструкторе класса `TrainOfContainers` инициализируйте список контейнеров `self.train: list[Container]` длиной 56.
- (c) Добавьте метод `shuffle(self) -> None` в класс `TrainOfContainers`, который будет перемешивать контейнеры в списке `self.train`.
- (d) Добавьте метод `get(self, i: int) -> Container`, который будет возвращать i -й контейнер и груз из списка `self.train`.

- (e) Создайте экземпляр класса `TrainOfContainers` и вызовите метод `shuffle` для перемешивания контейнеров.
- (f) Создайте цикл, который будет запрашивать у пользователя номер контейнера из состава и выводить информацию о выбранном контейнере.
- (g) Повторите шаги 5–6 до тех пор, пока пользователь не выберет все контейнеры или не завершит выбор.
- (h) В конце программы выводите сообщение о завершении выбора контейнеров.
- (i) Убедитесь, что пользователь вводит корректные номера контейнеров и что программа обрабатывает ошибки, связанные с вводом пользователя.
- (j) Проверьте работу программы, используя различные комбинации номеров контейнеров и грузов.

2 Формирование состава почтовых посылок

- (a) Создайте класс `Parcel`, который будет представлять собой посылку с содержимым. В конструкторе класса `Parcel` инициализируйте значения посылки и содержимого из списков `NumList` и `MasList`, которые объявлены как общие атрибуты класса. `NumList: list[str]` — это список посылок (не менее 14):

```
['Посылка_1', 'Посылка_2', ..., 'Посылка_14']
```

`MasList: list[str]` — это список содержимого посылок (не менее 4):

```
['Книги', 'Игрушки', 'Косметика', 'Спортивный инвентарь']
```

Конструктор должен иметь сигнатуру: `__init__(self) -> None`.

- (b) Создайте класс `TrainOfParcels`, который будет представлять собой состав, состоящий из моделей посылок. В конструкторе класса `TrainOfParcels` инициализируйте список посылок `self.train: list[Parcel]` длиной 56.
- (c) Добавьте метод `shuffle(self) -> None` в класс `TrainOfParcels`, который будет перемешивать посылки в списке `self.train`.
- (d) Добавьте метод `get(self, i: int) -> Parcel`, который будет возвращать i -ю посылку и её содержимое из списка `self.train`.
- (e) Создайте экземпляр класса `TrainOfParcels` и вызовите метод `shuffle` для перемешивания посылок.
- (f) Создайте цикл, который будет запрашивать у пользователя номер посылки из состава и выводить информацию о выбранной посылке.
- (g) Повторите шаги 5–6 до тех пор, пока пользователь не выберет все посылки или не завершит выбор.
- (h) В конце программы выводите сообщение о завершении выбора посылок.
- (i) Убедитесь, что пользователь вводит корректные номера посылок и что программа обрабатывает ошибки, связанные с вводом пользователя.
- (j) Проверьте работу программы, используя различные комбинации номеров посылок и содержимого.

3 Формирование автовоза с автомобилями

- (a) Создайте класс `Car`, который будет представлять собой автомобиль на автобусе. В конструкторе класса `Car` инициализируйте значения автомобиля и его марки из списков `NumList` и `MasList`, которые объявлены как общие атрибуты класса. `NumList: list[str]` — это список автомобилей (не менее 14):

```
["Автомобиль_1", "Автомобиль_2", ..., "Автомобиль_14"]
```

`MasList: list[str]` — это список марок автомобилей (не менее 4):

```
["Toyota", "BMW", "Lada", "Tesla"]
```

Конструктор должен иметь сигнатуру: `__init__(self) -> None`.

- (b) Создайте класс `CarCarrier`, который будет представлять собой автобус, состоящий из моделей автомобилей. В конструкторе класса `CarCarrier` инициализируйте список автомобилей `self.train: list[Car]` длиной 56.
- (c) Добавьте метод `shuffle(self) -> None` в класс `CarCarrier`, который будет перемешивать автомобили в списке `self.train`.
- (d) Добавьте метод `get(self, i: int) -> Car`, который будет возвращать i -й автомобиль и его марку из списка `self.train`.
- (e) Создайте экземпляр класса `CarCarrier` и вызовите метод `shuffle` для перемешивания автомобилей.
- (f) Создайте цикл, который будет запрашивать у пользователя номер автомобиля на автобусе и выводить информацию о выбранном автомобиле.
- (g) Повторите шаги 5–6 до тех пор, пока пользователь не выберет все автомобили или не завершит выбор.
- (h) В конце программы выводите сообщение о завершении выбора автомобилей.
- (i) Убедитесь, что пользователь вводит корректные номера автомобилей и что программа обрабатывает ошибки, связанные с вводом пользователя.
- (j) Проверьте работу программы, используя различные комбинации номеров автомобилей и марок.

4 Формирование багажного состава из чемоданов

- (a) Создайте класс `Suitcase`, который будет представлять собой чемодан с владельцем. В конструкторе класса `Suitcase` инициализируйте значения чемодана и владельца из списков `NumList` и `MasList`, которые объявлены как общие атрибуты класса. `NumList: list[str]` — это список чемоданов (не менее 14):

```
["Чемодан_1", "Чемодан_2", ..., "Чемодан_14"]
```

`MasList: list[str]` — это список владельцев (не менее 4):

```
["Иванов", "Петров", "Сидоров", "Кузнецов"]
```

Конструктор должен иметь сигнатуру: `__init__(self) -> None`.

- (b) Создайте класс `BaggageTrain`, который будет представлять собой багажный состав, состоящий из моделей чемоданов. В конструкторе класса `BaggageTrain` инициализируйте список чемоданов `self.train: list[Suitcase]` длиной 56.
- (c) Добавьте метод `shuffle(self) -> None` в класс `BaggageTrain`, который будет перемешивать чемоданы в списке `self.train`.

- (d) Добавьте метод `get(self, i: int) -> Suitcase`, который будет возвращать i -й чемодан и его владельца из списка `self.train`.
- (e) Создайте экземпляр класса `BaggageTrain` и вызовите метод `shuffle` для перемешивания чемоданов.
- (f) Создайте цикл, который будет запрашивать у пользователя номер чемодана из состава и выводить информацию о выбранном чемодане.
- (g) Повторите шаги 5–6 до тех пор, пока пользователь не выберет все чемоданы или не завершит выбор.
- (h) В конце программы выводите сообщение о завершении выбора чемоданов.
- (i) Убедитесь, что пользователь вводит корректные номера чемоданов и что программа обрабатывает ошибки, связанные с вводом пользователя.
- (j) Проверьте работу программы, используя различные комбинации номеров чемоданов и владельцев.

5 Формирование складского состава из ящиков

- (a) Создайте класс `Box`, который будет представлять собой ящик с содержимым. В конструкторе класса `Box` инициализируйте значения ящика и содержимого из списков `NumList` и `MasList`, которые объявлены как общие атрибуты класса. `NumList: list[str]` — это список ящиков (не менее 14):

```
["Ящик_1", "Ящик_2", ..., "Ящик_14"]
```

`MasList: list[str]` — это список типов содержимого (не менее 4):

```
["Инструменты", "Запчасти", "Химикаты", "Упаковка"]
```

Конструктор должен иметь сигнатуру: `__init__(self) -> None`.

- (b) Создайте класс `WarehouseTrain`, который будет представлять собой состав, состоящий из моделей ящиков. В конструкторе класса `WarehouseTrain` инициализируйте список ящиков `self.train: list[Box]` длиной 56.
- (c) Добавьте метод `shuffle(self) -> None` в класс `WarehouseTrain`, который будет перемешивать ящики в списке `self.train`.
- (d) Добавьте метод `get(self, i: int) -> Box`, который будет возвращать i -й ящик и его содержимое из списка `self.train`.
- (e) Создайте экземпляр класса `WarehouseTrain` и вызовите метод `shuffle` для перемешивания ящиков.
- (f) Создайте цикл, который будет запрашивать у пользователя номер ящика из состава и выводить информацию о выбранном ящике.
- (g) Повторите шаги 5–6 до тех пор, пока пользователь не выберет все ящики или не завершит выбор.
- (h) В конце программы выводите сообщение о завершении выбора ящиков.
- (i) Убедитесь, что пользователь вводит корректные номера ящиков и что программа обрабатывает ошибки, связанные с вводом пользователя.
- (j) Проверьте работу программы, используя различные комбинации номеров ящиков и содержимого.

6 Формирование состава морских судов с грузом

- (a) Создайте класс `Ship`, который будет представлять собой судно с грузом. В конструкторе класса `Ship` инициализируйте значения судна и груза из списков `NumList` и `MasList`, которые объявлены как общие атрибуты класса. `NumList: list[str]` — это список судов (не менее 14):

```
['Судно_1', 'Судно_2', ..., 'Судно_14']
```

`MasList: list[str]` — это список грузов (не менее 4):

```
['Нефть', 'Уголь', 'Зерно', 'Лес']
```

Конструктор должен иметь сигнатуру: `__init__(self) -> None`.

- (b) Создайте класс `Fleet`, который будет представлять собой флотилию, состоящую из моделей судов. В конструкторе класса `Fleet` инициализируйте список судов `self.train: list[Ship]` длиной 56.
- (c) Добавьте метод `shuffle(self) -> None` в класс `Fleet`, который будет перемешивать суда в списке `self.train`.
- (d) Добавьте метод `get(self, i: int) -> Ship`, который будет возвращать *i*-е судно и его груз из списка `self.train`.
- (e) Создайте экземпляр класса `Fleet` и вызовите метод `shuffle` для перемешивания судов.
- (f) Создайте цикл, который будет запрашивать у пользователя номер судна из флотилии и выводить информацию о выбранном судне.
- (g) Повторите шаги 5–6 до тех пор, пока пользователь не выберет все суда или не завершит выбор.
- (h) В конце программы выводите сообщение о завершении выбора судов.
- (i) Убедитесь, что пользователь вводит корректные номера судов и что программа обрабатывает ошибки, связанные с вводом пользователя.
- (j) Проверьте работу программы, используя различные комбинации номеров судов и грузов.

7 Формирование состава ракет-носителей

- (a) Создайте класс `Rocket`, который будет представлять собой ракету с полезной нагрузкой. В конструкторе класса `Rocket` инициализируйте значения ракеты и нагрузки из списков `NumList` и `MasList`, которые объявлены как общие атрибуты класса. `NumList: list[str]` — это список ракет (не менее 14):

```
['Ракета_1', 'Ракета_2', ..., 'Ракета_14']
```

`MasList: list[str]` — это список типов нагрузки (не менее 4):

```
['Спутник', 'Грузовой модуль', 'Экипаж', 'Научное оборудование']
```

Конструктор должен иметь сигнатуру: `__init__(self) -> None`.

- (b) Создайте класс `RocketTrain`, который будет представлять собой состав ракет. В конструкторе класса `RocketTrain` инициализируйте список ракет `self.train: list[Rocket]` длиной 56.
- (c) Добавьте метод `shuffle(self) -> None` в класс `RocketTrain`, который будет перемешивать ракеты в списке `self.train`.

- (d) Добавьте метод `get(self, i: int) -> Rocket`, который будет возвращать i -ю ракету и её нагрузку из списка `self.train`.
- (e) Создайте экземпляр класса `RocketTrain` и вызовите метод `shuffle` для перемешивания ракет.
- (f) Создайте цикл, который будет запрашивать у пользователя номер ракеты и выводить информацию о выбранной ракете.
- (g) Повторите шаги 5–6 до тех пор, пока пользователь не выберет все ракеты или не завершит выбор.
- (h) В конце программы выводите сообщение о завершении выбора ракет.
- (i) Убедитесь, что пользователь вводит корректные номера ракет и что программа обрабатывает ошибки, связанные с вводом пользователя.
- (j) Проверьте работу программы, используя различные комбинации номеров ракет и нагрузок.

8 Формирование состава дронов с грузом

- (a) Создайте класс `Drone`, который будет представлять собой дрон с миссией. В конструкторе класса `Drone` инициализируйте значения дрона и миссии из списков `NumList` и `MasList`, которые объявлены как общие атрибуты класса. `NumList: list[str]` — это список дронов (не менее 14):

```
["Дрон_1", "Дрон_2", ..., "Дрон_14"]
```

`MasList: list[str]` — это список миссий (не менее 4):

```
["Фотосъёмка", "Доставка", "Разведка", "Мониторинг"]
```

Конструктор должен иметь сигнатуру: `__init__(self) -> None`.

- (b) Создайте класс `DroneSquadron`, который будет представлять собой эскадрилью, состоящую из моделей дронов. В конструкторе класса `DroneSquadron` инициализируйте список дронов `self.train: list[Drone]` длиной 56.
- (c) Добавьте метод `shuffle(self) -> None` в класс `DroneSquadron`, который будет перемешивать дроны в списке `self.train`.
- (d) Добавьте метод `get(self, i: int) -> Drone`, который будет возвращать i -й дрон и его миссию из списка `self.train`.
- (e) Создайте экземпляр класса `DroneSquadron` и вызовите метод `shuffle` для перемешивания дронов.
- (f) Создайте цикл, который будет запрашивать у пользователя номер дрона и выводить информацию о выбранном дроне.
- (g) Повторите шаги 5–6 до тех пор, пока пользователь не выберет все дроны или не завершит выбор.
- (h) В конце программы выводите сообщение о завершении выбора дронов.
- (i) Убедитесь, что пользователь вводит корректные номера дронов и что программа обрабатывает ошибки, связанные с вводом пользователя.
- (j) Проверьте работу программы, используя различные комбинации номеров дронов и миссий.

9 Формирование состава тележек в супермаркете

- (a) Создайте класс `Trolley`, который будет представлять собой тележку с типом покупателя. В конструкторе класса `Trolley` инициализируйте значения тележки и типа покупателя из списков `NumList` и `MasList`, которые объявлены как общие атрибуты класса. `NumList: list[str]` — это список тележек (не менее 14):

```
["Тележка_1", "Тележка_2", ..., "Тележка_14"]
```

`MasList: list[str]` — это список типов покупателей (не менее 4):

```
["Семья", "Студент", "Пенсионер", "Турист"]
```

Конструктор должен иметь сигнатуру: `__init__(self) -> None`.

- (b) Создайте класс `TrolleyTrain`, который будет представлять собой состав тележек. В конструкторе класса `TrolleyTrain` инициализируйте список тележек `self.train: list[Trolley]` длиной 56.
- (c) Добавьте метод `shuffle(self) -> None` в класс `TrolleyTrain`, который будет перемешивать тележки в списке `self.train`.
- (d) Добавьте метод `get(self, i: int) -> Trolley`, который будет возвращать i -ю тележку и тип её покупателя из списка `self.train`.
- (e) Создайте экземпляр класса `TrolleyTrain` и вызовите метод `shuffle` для перемешивания тележек.
- (f) Создайте цикл, который будет запрашивать у пользователя номер тележки и выводить информацию о ней.
- (g) Повторите шаги 5–6 до тех пор, пока пользователь не выберет все тележки или не завершит выбор.
- (h) В конце программы выводите сообщение о завершении выбора тележек.
- (i) Убедитесь, что пользователь вводит корректные номера тележек и что программа обрабатывает ошибки, связанные с вводом пользователя.
- (j) Проверьте работу программы, используя различные комбинации номеров тележек и типов покупателей.

10 Формирование состава камер хранения

- (a) Создайте класс `Locker`, который будет представлять собой камеру хранения с содержимым. В конструкторе класса `Locker` инициализируйте значения камеры и содержимого из списков `NumList` и `MasList`, которые объявлены как общие атрибуты класса. `NumList: list[str]` — это список камер (не менее 14):

```
["Камера_1", "Камера_2", ..., "Камера_14"]
```

`MasList: list[str]` — это список типов содержимого (не менее 4):

```
["Велосипед", "Чемодан", "Инструменты", "Спортивный инвентарь"]
```

Конструктор должен иметь сигнатуру: `__init__(self) -> None`.

- (b) Создайте класс `StorageTrain`, который будет представлять собой состав камер хранения. В конструкторе класса `StorageTrain` инициализируйте список камер `self.train: list[Locker]` длиной 56.
- (c) Добавьте метод `shuffle(self) -> None` в класс `StorageTrain`, который будет перемешивать камеры в списке `self.train`.

- (d) Добавьте метод `get(self, i: int) -> Locker`, который будет возвращать i -ю камеру и её содержимое из списка `self.train`.
- (e) Создайте экземпляр класса `StorageTrain` и вызовите метод `shuffle` для перемешивания камер.
- (f) Создайте цикл, который будет запрашивать у пользователя номер камеры и выводить информацию о ней.
- (g) Повторите шаги 5–6 до тех пор, пока пользователь не выберет все камеры или не завершит выбор.
- (h) В конце программы выводите сообщение о завершении выбора камер.
- (i) Убедитесь, что пользователь вводит корректные номера камер и что программа обрабатывает ошибки, связанные с вводом пользователя.
- (j) Проверьте работу программы, используя различные комбинации номеров камер и содержимого.

11 Формирование состава самолётов с бортами

- (a) Создайте класс `Aircraft`, который будет представлять собой самолёт с типом рейса. В конструкторе класса `Aircraft` инициализируйте значения самолёта и типа рейса из списков `NumList` и `MasList`, которые объявлены как общие атрибуты класса. `NumList: list[str]` — это список самолётов (не менее 14):

`['Борт_1', 'Борт_2', ..., 'Борт_14']`

`MasList: list[str]` — это список типов рейсов (не менее 4):

`['Пассажирский', 'Грузовой', 'Военный', 'Санитарный']`

Конструктор должен иметь сигнатуру: `__init__(self) -> None`.

- (b) Создайте класс `AirFleet`, который будет представлять собой воздушный флот, состоящий из моделей самолётов. В конструкторе класса `AirFleet` инициализируйте список самолётов `self.train: list[Aircraft]` длиной 56.
- (c) Добавьте метод `shuffle(self) -> None` в класс `AirFleet`, который будет перемешивать самолёты в списке `self.train`.
- (d) Добавьте метод `get(self, i: int) -> Aircraft`, который будет возвращать i -й самолёт и его тип рейса из списка `self.train`.
- (e) Создайте экземпляр класса `AirFleet` и вызовите метод `shuffle` для перемешивания самолётов.
- (f) Создайте цикл, который будет запрашивать у пользователя номер самолёта и выводить информацию о нём.
- (g) Повторите шаги 5–6 до тех пор, пока пользователь не выберет все самолёты или не завершит выбор.
- (h) В конце программы выводите сообщение о завершении выбора самолётов.
- (i) Убедитесь, что пользователь вводит корректные номера самолётов и что программа обрабатывает ошибки, связанные с вводом пользователя.
- (j) Проверьте работу программы, используя различные комбинации номеров самолётов и типов рейсов.

12 Формирование состава танкеров с жидкостями

- (a) Создайте класс `Tanker`, который будет представлять собой танкер с жидкостью. В конструкторе класса `Tanker` инициализируйте значения танкера и жидкости из списков `NumList` и `MasList`, которые объявлены как общие атрибуты класса. `NumList: list[str]` — это список танкеров (не менее 14):

```
['Танкер_1', 'Танкер_2', ..., 'Танкер_14']
```

`MasList: list[str]` — это список жидкостей (не менее 4):

```
['Вода', 'Молоко', 'Топливо', 'Химикаты']
```

Конструктор должен иметь сигнатуру: `__init__(self) -> None`.

- (b) Создайте класс `TankerConvoy`, который будет представлять собой конвой танкеров. В конструкторе класса `TankerConvoy` инициализируйте список танкеров `self.train: list[Tanker]` длиной 56.
- (c) Добавьте метод `shuffle(self) -> None` в класс `TankerConvoy`, который будет перемешивать танкеры в списке `self.train`.
- (d) Добавьте метод `get(self, i: int) -> Tanker`, который будет возвращать i -й танкер и его жидкость из списка `self.train`.
- (e) Создайте экземпляр класса `TankerConvoy` и вызовите метод `shuffle` для перемешивания танкеров.
- (f) Создайте цикл, который будет запрашивать у пользователя номер танкера и выводить информацию о нём.
- (g) Повторите шаги 5–6 до тех пор, пока пользователь не выберет все танкеры или не завершит выбор.
- (h) В конце программы выводите сообщение о завершении выбора танкеров.
- (i) Убедитесь, что пользователь вводит корректные номера танкеров и что программа обрабатывает ошибки, связанные с вводом пользователя.
- (j) Проверьте работу программы, используя различные комбинации номеров танкеров и жидкостей.

13 Формирование состава паллет на складе

- (a) Создайте класс `Pallet`, который будет представлять собой паллету с товаром. В конструкторе класса `Pallet` инициализируйте значения паллеты и товара из списков `NumList` и `MasList`, которые объявлены как общие атрибуты класса. `NumList: list[str]` — это список паллет (не менее 14):

```
['Паллета_1', 'Паллета_2', ..., 'Паллета_14']
```

`MasList: list[str]` — это список типов товаров (не менее 4):

```
['Напитки', 'Консервы', 'Бытовая химия', 'Бумажная продукция']
```

Конструктор должен иметь сигнатуру: `__init__(self) -> None`.

- (b) Создайте класс `PalletTrain`, который будет представлять собой состав паллет. В конструкторе класса `PalletTrain` инициализируйте список паллет `self.train: list[Pallet]` длиной 56.
- (c) Добавьте метод `shuffle(self) -> None` в класс `PalletTrain`, который будет перемешивать паллеты в списке `self.train`.

- (d) Добавьте метод `get(self, i: int) -> Pallet`, который будет возвращать i -ю паллету и её товар из списка `self.train`.
- (e) Создайте экземпляр класса `PalletTrain` и вызовите метод `shuffle` для перемешивания паллет.
- (f) Создайте цикл, который будет запрашивать у пользователя номер паллеты и выводить информацию о ней.
- (g) Повторите шаги 5–6 до тех пор, пока пользователь не выберет все паллеты или не завершит выбор.
- (h) В конце программы выводите сообщение о завершении выбора паллет.
- (i) Убедитесь, что пользователь вводит корректные номера паллет и что программа обрабатывает ошибки, связанные с вводом пользователя.
- (j) Проверьте работу программы, используя различные комбинации номеров паллет и товаров.

14 Формирование состава вагонов-цистерн

- (a) Создайте класс `TankWagon`, который будет представлять собой цистерну с содержимым. В конструкторе класса `TankWagon` инициализируйте значения цистерны и содержимого из списков `NumList` и `MasList`, которые объявлены как общие атрибуты класса. `NumList: list[str]` — это список цистерн (не менее 14):

```
["Цистерна_1", "Цистерна_2", ..., "Цистерна_14"]
```

`MasList: list[str]` — это список типов содержимого (не менее 4):

```
["Бензин", "Дизель", "Газ", "Вода"]
```

Конструктор должен иметь сигнатуру: `__init__(self) -> None`.

- (b) Создайте класс `TankTrain`, который будет представлять собой состав цистерн. В конструкторе класса `TankTrain` инициализируйте список цистерн `self.train: list[TankWagon]` длиной 56.
- (c) Добавьте метод `shuffle(self) -> None` в класс `TankTrain`, который будет перемешивать цистерны в списке `self.train`.
- (d) Добавьте метод `get(self, i: int) -> TankWagon`, который будет возвращать i -ю цистерну и её содержимое из списка `self.train`.
- (e) Создайте экземпляр класса `TankTrain` и вызовите метод `shuffle` для перемешивания цистерн.
- (f) Создайте цикл, который будет запрашивать у пользователя номер цистерны и выводить информацию о ней.
- (g) Повторите шаги 5–6 до тех пор, пока пользователь не выберет все цистерны или не завершит выбор.
- (h) В конце программы выводите сообщение о завершении выбора цистерн.
- (i) Убедитесь, что пользователь вводит корректные номера цистерн и что программа обрабатывает ошибки, связанные с вводом пользователя.
- (j) Проверьте работу программы, используя различные комбинации номеров цистерн и содержимого.

15 Формирование состава промышленных роботов

- (a) Создайте класс `Robot`, который будет представлять собой робота с модулем. В конструкторе класса `Robot` инициализируйте значения робота и модуля из списков `NumList` и `MasList`, которые объявлены как общие атрибуты класса. `NumList: list[str]` — это список роботов (не менее 14):

```
['Робот_1', 'Робот_2', ..., 'Робот_14']
```

`MasList: list[str]` — это список модулей (не менее 4):

```
['Манипулятор', 'Камера', 'Сенсор', 'Батарея']
```

Конструктор должен иметь сигнатуру: `__init__(self) -> None`.

- (b) Создайте класс `RobotLine`, который будет представлять собой производственную линию роботов. В конструкторе класса `RobotLine` инициализируйте список роботов `self.train: list[Robot]` длиной 56.
- (c) Добавьте метод `shuffle(self) -> None` в класс `RobotLine`, который будет перемешивать роботов в списке `self.train`.
- (d) Добавьте метод `get(self, i: int) -> Robot`, который будет возвращать i -го робота и его модуль из списка `self.train`.
- (e) Создайте экземпляр класса `RobotLine` и вызовите метод `shuffle` для перемешивания роботов.
- (f) Создайте цикл, который будет запрашивать у пользователя номер робота и выводить информацию о нём.
- (g) Повторите шаги 5–6 до тех пор, пока пользователь не выберет всех роботов или не завершит выбор.
- (h) В конце программы выводите сообщение о завершении выбора роботов.
- (i) Убедитесь, что пользователь вводит корректные номера роботов и что программа обрабатывает ошибки, связанные с вводом пользователя.
- (j) Проверьте работу программы, используя различные комбинации номеров роботов и модулей.

16 Формирование состава клеток с животными

- (a) Создайте класс `Cage`, который будет представлять собой клетку с животным. В конструкторе класса `Cage` инициализируйте значения клетки и животного из списков `NumList` и `MasList`, которые объявлены как общие атрибуты класса. `NumList: list[str]` — это список клеток (не менее 14):

```
['Клетка_1', 'Клетка_2', ..., 'Клетка_14']
```

`MasList: list[str]` — это список животных (не менее 4):

```
['Собака', 'Кошка', 'Попугай', 'Кролик']
```

Конструктор должен иметь сигнатуру: `__init__(self) -> None`.

- (b) Создайте класс `ZooTrain`, который будет представлять собой состав клеток. В конструкторе класса `ZooTrain` инициализируйте список клеток `self.train: list[Cage]` длиной 56.
- (c) Добавьте метод `shuffle(self) -> None` в класс `ZooTrain`, который будет перемешивать клетки в списке `self.train`.

- (d) Добавьте метод `get(self, i: int) -> Cage`, который будет возвращать i -ю клетку и её животное из списка `self.train`.
- (e) Создайте экземпляр класса `ZooTrain` и вызовите метод `shuffle` для перемешивания клеток.
- (f) Создайте цикл, который будет запрашивать у пользователя номер клетки и выводить информацию о ней.
- (g) Повторите шаги 5–6 до тех пор, пока пользователь не выберет все клетки или не завершит выбор.
- (h) В конце программы выводите сообщение о завершении выбора клеток.
- (i) Убедитесь, что пользователь вводит корректные номера клеток и что программа обрабатывает ошибки, связанные с вводом пользователя.
- (j) Проверьте работу программы, используя различные комбинации номеров клеток и животных.

17 Формирование состава прицепов на автодороге

- (a) Создайте класс `Trailer`, который будет представлять собой прицеп с грузом. В конструкторе класса `Trailer` инициализируйте значения прицепа и груза из списков `NumList` и `MasList`, которые объявлены как общие атрибуты класса. `NumList: list[str]` — это список прицепов (не менее 14):

```
['Прицеп_1', 'Прицеп_2', ..., 'Прицеп_14']
```

`MasList: list[str]` — это список типов груза (не менее 4):

```
['Строительные материалы', 'Мебель', 'Техника', 'Сельхозпродукция']
```

Конструктор должен иметь сигнатуру: `__init__(self) -> None`.

- (b) Создайте класс `TrailerConvoy`, который будет представлять собой конвой прицепов. В конструкторе класса `TrailerConvoy` инициализируйте список прицепов `self.train: list[Trailer]` длиной 56.
- (c) Добавьте метод `shuffle(self) -> None` в класс `TrailerConvoy`, который будет перемешивать прицепы в списке `self.train`.
- (d) Добавьте метод `get(self, i: int) -> Trailer`, который будет возвращать i -й прицеп и его груз из списка `self.train`.
- (e) Создайте экземпляр класса `TrailerConvoy` и вызовите метод `shuffle` для перемешивания прицепов.
- (f) Создайте цикл, который будет запрашивать у пользователя номер прицепа и выводить информацию о нём.
- (g) Повторите шаги 5–6 до тех пор, пока пользователь не выберет все прицепы или не завершит выбор.
- (h) В конце программы выводите сообщение о завершении выбора прицепов.
- (i) Убедитесь, что пользователь вводит корректные номера прицепов и что программа обрабатывает ошибки, связанные с вводом пользователя.
- (j) Проверьте работу программы, используя различные комбинации номеров прицепов и грузов.

18 Формирование состава морских контейнеровозов

- (a) Создайте класс `Vessel`, который будет представлять собой контейнеровоз с типом контейнера. В конструкторе класса `Vessel` инициализируйте значения судна и типа контейнера из списков `NumList` и `MasList`, которые объявлены как общие атрибуты класса. `NumList: list[str]` — это список судов (не менее 14):

```
["Корабль_1", "Корабль_2", ..., "Корабль_14"]
```

`MasList: list[str]` — это список типов контейнеров (не менее 4):

```
["20ft", "40ft", "Рефрижератор", "Открытый"]
```

Конструктор должен иметь сигнатуру: `__init__(self) -> None`.

- (b) Создайте класс `VesselFleet`, который будет представлять собой флот контейнеровозов. В конструкторе класса `VesselFleet` инициализируйте список судов `self.train: list[Vessel]` длиной 56.
- (c) Добавьте метод `shuffle(self) -> None` в класс `VesselFleet`, который будет перемешивать суда в списке `self.train`.
- (d) Добавьте метод `get(self, i: int) -> Vessel`, который будет возвращать i -е судно и тип его контейнера из списка `self.train`.
- (e) Создайте экземпляр класса `VesselFleet` и вызовите метод `shuffle` для перемешивания судов.
- (f) Создайте цикл, который будет запрашивать у пользователя номер судна и выводить информацию о нём.
- (g) Повторите шаги 5–6 до тех пор, пока пользователь не выберет все суда или не завершит выбор.
- (h) В конце программы выводите сообщение о завершении выбора судов.
- (i) Убедитесь, что пользователь вводит корректные номера судов и что программа обрабатывает ошибки, связанные с вводом пользователя.
- (j) Проверьте работу программы, используя различные комбинации номеров судов и типов контейнеров.

19 Формирование состава банковских сейфов

- (a) Создайте класс `Safe`, который будет представлять собой сейф с содержимым. В конструкторе класса `Safe` инициализируйте значения сейфа и содержимого из списков `NumList` и `MasList`, которые объявлены как общие атрибуты класса. `NumList: list[str]` — это список сейфов (не менее 14):

```
["Сейф_1", "Сейф_2", ..., "Сейф_14"]
```

`MasList: list[str]` — это список типов содержимого (не менее 4):

```
["Документы", "Драгоценности", "Деньги", "Антиквариат"]
```

Конструктор должен иметь сигнатуру: `__init__(self) -> None`.

- (b) Создайте класс `VaultTrain`, который будет представлять собой состав сейфов. В конструкторе класса `VaultTrain` инициализируйте список сейфов `self.train: list[Safe]` длиной 56.
- (c) Добавьте метод `shuffle(self) -> None` в класс `VaultTrain`, который будет перемешивать сейфы в списке `self.train`.

- (d) Добавьте метод `get(self, i: int) -> Safe`, который будет возвращать i -й сейф и его содержимое из списка `self.train`.
- (e) Создайте экземпляр класса `VaultTrain` и вызовите метод `shuffle` для перемешивания сейфов.
- (f) Создайте цикл, который будет запрашивать у пользователя номер сейфа и выводить информацию о нём.
- (g) Повторите шаги 5–6 до тех пор, пока пользователь не выберет все сейфы или не завершит выбор.
- (h) В конце программы выводите сообщение о завершении выбора сейфов.
- (i) Убедитесь, что пользователь вводит корректные номера сейфов и что программа обрабатывает ошибки, связанные с вводом пользователя.
- (j) Проверьте работу программы, используя различные комбинации номеров сейфов и содержимого.

20 Формирование состава капсул экспресс-доставки

- (a) Создайте класс `Capsule`, который будет представлять собой капсулу с грузом. В конструкторе класса `Capsule` инициализируйте значения капсулы и груза из списков `NumList` и `MasList`, которые объявлены как общие атрибуты класса. `NumList: list[str]` — это список капсул (не менее 14):

```
["Капсула_1", "Капсула_2", ..., "Капсула_14"]
```

`MasList: list[str]` — это список типов груза (не менее 4):

```
["Медикаменты", "Еда", "Посылки", "Образцы"]
```

Конструктор должен иметь сигнатуру: `__init__(self) -> None`.

- (b) Создайте класс `CapsuleTrain`, который будет представлять собой состав капсул. В конструкторе класса `CapsuleTrain` инициализируйте список капсул `self.train: list[Capsule]` длиной 56.
- (c) Добавьте метод `shuffle(self) -> None` в класс `CapsuleTrain`, который будет перемешивать капсулы в списке `self.train`.
- (d) Добавьте метод `get(self, i: int) -> Capsule`, который будет возвращать i -ю капсулу и её груз из списка `self.train`.
- (e) Создайте экземпляр класса `CapsuleTrain` и вызовите метод `shuffle` для перемешивания капсул.
- (f) Создайте цикл, который будет запрашивать у пользователя номер капсулы и выводить информацию о ней.
- (g) Повторите шаги 5–6 до тех пор, пока пользователь не выберет все капсулы или не завершит выбор.
- (h) В конце программы выводите сообщение о завершении выбора капсул.
- (i) Убедитесь, что пользователь вводит корректные номера капсул и что программа обрабатывает ошибки, связанные с вводом пользователя.
- (j) Проверьте работу программы, используя различные комбинации номеров капсул и грузов.

21 Формирование состава тележек в аэропорту

- (a) Создайте класс `Trolley`, который будет представлять собой тележку с типом пассажира. В конструкторе класса `Trolley` инициализируйте значения тележки и типа пассажира из списков `NumList` и `MasList`, которые объявлены как общие атрибуты класса. `NumList: list[str]` — это список тележек (не менее 14):

```
["Тележка_1", "Тележка_2", ..., "Тележка_14"]
```

`MasList: list[str]` — это список типов пассажиров (не менее 4):

```
["Бизнес", "Эконом", "Первый класс", "Транзит"]
```

Конструктор должен иметь сигнатуру: `__init__(self) -> None`.

- (b) Создайте класс `AirportTrolleyTrain`, который будет представлять собой состав тележек. В конструкторе класса `AirportTrolleyTrain` инициализируйте список тележек `self.train: list[Trolley]` длиной 56.
- (c) Добавьте метод `shuffle(self) -> None` в класс `AirportTrolleyTrain`, который будет перемешивать тележки в списке `self.train`.
- (d) Добавьте метод `get(self, i: int) -> Trolley`, который будет возвращать i -ю тележку и тип её пассажира из списка `self.train`.
- (e) Создайте экземпляр класса `AirportTrolleyTrain` и вызовите метод `shuffle` для перемешивания тележек.
- (f) Создайте цикл, который будет запрашивать у пользователя номер тележки и выводить информацию о ней.
- (g) Повторите шаги 5–6 до тех пор, пока пользователь не выберет все тележки или не завершит выбор.
- (h) В конце программы выводите сообщение о завершении выбора тележек.
- (i) Убедитесь, что пользователь вводит корректные номера тележек и что программа обрабатывает ошибки, связанные с вводом пользователя.
- (j) Проверьте работу программы, используя различные комбинации номеров тележек и типов пассажиров.

22 Формирование состава мобильных платформ с оборудованием

- (a) Создайте класс `Platform`, который будет представлять собой платформу с оборудованием. В конструкторе класса `Platform` инициализируйте значения платформы и оборудования из списков `NumList` и `MasList`, которые объявлены как общие атрибуты класса. `NumList: list[str]` — это список платформ (не менее 14):

```
["Платформа_1", "Платформа_2", ..., "Платформа_14"]
```

`MasList: list[str]` — это список типов оборудования (не менее 4):

```
["Генератор", "Компрессор", "Насос", "Сварочный аппарат"]
```

Конструктор должен иметь сигнатуру: `__init__(self) -> None`.

- (b) Создайте класс `PlatformTrain`, который будет представлять собой состав платформ. В конструкторе класса `PlatformTrain` инициализируйте список платформ `self.train: list[Platform]` длиной 56.
- (c) Добавьте метод `shuffle(self) -> None` в класс `PlatformTrain`, который будет перемешивать платформы в списке `self.train`.

- (d) Добавьте метод `get(self, i: int) -> Platform`, который будет возвращать i -ю платформу и её оборудование из списка `self.train`.
- (e) Создайте экземпляр класса `PlatformTrain` и вызовите метод `shuffle` для перемешивания платформ.
- (f) Создайте цикл, который будет запрашивать у пользователя номер платформы и выводить информацию о ней.
- (g) Повторите шаги 5–6 до тех пор, пока пользователь не выберет все платформы или не завершит выбор.
- (h) В конце программы выводите сообщение о завершении выбора платформ.
- (i) Убедитесь, что пользователь вводит корректные номера платформ и что программа обрабатывает ошибки, связанные с вводом пользователя.
- (j) Проверьте работу программы, используя различные комбинации номеров платформ и оборудования.

23 Формирование состава ящиков с инструментами

- (a) Создайте класс `Toolbox`, который будет представлять собой ящик с набором инструментов. В конструкторе класса `Toolbox` инициализируйте значения ящика и набора из списков `NumList` и `MasList`, которые объявлены как общие атрибуты класса. `NumList: list[str]` — это список ящиков (не менее 14):

`["Ящик_1", "Ящик_2", ..., "Ящик_14"]`

`MasList: list[str]` — это список типов наборов (не менее 4):

`["Слесарный", "Электромонтажный", "Столярный", "Автомобильный"]`

Конструктор должен иметь сигнатуру: `__init__(self) -> None`.

- (b) Создайте класс `ToolTrain`, который будет представлять собой состав ящиков. В конструкторе класса `ToolTrain` инициализируйте список ящиков `self.train: list[Toolbox]` длиной 56.
- (c) Добавьте метод `shuffle(self) -> None` в класс `ToolTrain`, который будет перемешивать ящики в списке `self.train`.
- (d) Добавьте метод `get(self, i: int) -> Toolbox`, который будет возвращать i -й ящик и его набор инструментов из списка `self.train`.
- (e) Создайте экземпляр класса `ToolTrain` и вызовите метод `shuffle` для перемешивания ящиков.
- (f) Создайте цикл, который будет запрашивать у пользователя номер ящика и выводить информацию о нём.
- (g) Повторите шаги 5–6 до тех пор, пока пользователь не выберет все ящики или не завершит выбор.
- (h) В конце программы выводите сообщение о завершении выбора ящиков.
- (i) Убедитесь, что пользователь вводит корректные номера ящиков и что программа обрабатывает ошибки, связанные с вводом пользователя.
- (j) Проверьте работу программы, используя различные комбинации номеров ящиков и наборов инструментов.

24 Формирование состава подводных аппаратов

- (a) Создайте класс `Submersible`, который будет представлять собой аппарат с миссией. В конструкторе класса `Submersible` инициализируйте значения аппарата и миссии из списков `NumList` и `MasList`, которые объявлены как общие атрибуты класса. `NumList: list[str]` — это список аппаратов (не менее 14):

```
["Аппарат_1", "Аппарат_2", ..., "Аппарат_14"]
```

`MasList: list[str]` — это список миссий (не менее 4):

```
["Исследование", "Спасение", "Инспекция", "Добыча"]
```

Конструктор должен иметь сигнатуру: `__init__(self) -> None`.

- (b) Создайте класс `SubmersibleSquadron`, который будет представлять собой эскадрилью аппаратов. В конструкторе класса `SubmersibleSquadron` инициализируйте список аппаратов `self.train: list[Submersible]` длиной 56.
- (c) Добавьте метод `shuffle(self) -> None` в класс `SubmersibleSquadron`, который будет перемешивать аппараты в списке `self.train`.
- (d) Добавьте метод `get(self, i: int) -> Submersible`, который будет возвращать i -й аппарат и его миссию из списка `self.train`.
- (e) Создайте экземпляр класса `SubmersibleSquadron` и вызовите метод `shuffle` для перемешивания аппаратов.
- (f) Создайте цикл, который будет запрашивать у пользователя номер аппарата и выводить информацию о нём.
- (g) Повторите шаги 5–6 до тех пор, пока пользователь не выберет все аппараты или не завершит выбор.
- (h) В конце программы выводите сообщение о завершении выбора аппаратов.
- (i) Убедитесь, что пользователь вводит корректные номера аппаратов и что программа обрабатывает ошибки, связанные с вводом пользователя.
- (j) Проверьте работу программы, используя различные комбинации номеров аппаратов и миссий.

25 Формирование состава контейнеров с растениями

- (a) Создайте класс `Planter`, который будет представлять собой контейнер с растением. В конструкторе класса `Planter` инициализируйте значения контейнера и растения из списков `NumList` и `MasList`, которые объявлены как общие атрибуты класса. `NumList: list[str]` — это список контейнеров (не менее 14):

```
["Контейнер_1", "Контейнер_2", ..., "Контейнер_14"]
```

`MasList: list[str]` — это список растений (не менее 4):

```
["Орхидеи", "Кактусы", "Пальмы", "Бонсай"]
```

Конструктор должен иметь сигнатуру: `__init__(self) -> None`.

- (b) Создайте класс `GreenTrain`, который будет представлять собой состав контейнеров. В конструкторе класса `GreenTrain` инициализируйте список контейнеров `self.train: list[Planter]` длиной 56.
- (c) Добавьте метод `shuffle(self) -> None` в класс `GreenTrain`, который будет перемешивать контейнеры в списке `self.train`.

- (d) Добавьте метод `get(self, i: int) -> Planter`, который будет возвращать i -й контейнер и его растение из списка `self.train`.
- (e) Создайте экземпляр класса `GreenTrain` и вызовите метод `shuffle` для перемешивания контейнеров.
- (f) Создайте цикл, который будет запрашивать у пользователя номер контейнера и выводить информацию о нём.
- (g) Повторите шаги 5–6 до тех пор, пока пользователь не выберет все контейнеры или не завершит выбор.
- (h) В конце программы выводите сообщение о завершении выбора контейнеров.
- (i) Убедитесь, что пользователь вводит корректные номера контейнеров и что программа обрабатывает ошибки, связанные с вводом пользователя.
- (j) Проверьте работу программы, используя различные комбинации номеров контейнеров и растений.

26 Формирование состава машин скорой помощи

- (a) Создайте класс `Ambulance`, который будет представлять собой машину с типом бригады. В конструкторе класса `Ambulance` инициализируйте значения машины и типа бригады из списков `NumList` и `MasList`, которые объявлены как общие атрибуты класса. `NumList: list[str]` — это список машин (не менее 14):

```
['Машина_1', 'Машина_2', ..., 'Машина_14']
```

`MasList: list[str]` — это список типов бригад (не менее 4):

```
['Травматологи', 'Кардиологи', 'Психиатры', 'Реаниматологи']
```

Конструктор должен иметь сигнатуру: `__init__(self) -> None`.

- (b) Создайте класс `AmbulanceConvoy`, который будет представлять собой конвой машин. В конструкторе класса `AmbulanceConvoy` инициализируйте список машин `self.train: list[Ambulance]` длиной 56.
- (c) Добавьте метод `shuffle(self) -> None` в класс `AmbulanceConvoy`, который будет перемешивать машины в списке `self.train`.
- (d) Добавьте метод `get(self, i: int) -> Ambulance`, который будет возвращать i -ю машину и её бригаду из списка `self.train`.
- (e) Создайте экземпляр класса `AmbulanceConvoy` и вызовите метод `shuffle` для перемешивания машин.
- (f) Создайте цикл, который будет запрашивать у пользователя номер машины и выводить информацию о ней.
- (g) Повторите шаги 5–6 до тех пор, пока пользователь не выберет все машины или не завершит выбор.
- (h) В конце программы выводите сообщение о завершении выбора машин.
- (i) Убедитесь, что пользователь вводит корректные номера машин и что программа обрабатывает ошибки, связанные с вводом пользователя.
- (j) Проверьте работу программы, используя различные комбинации номеров машин и типов бригад.

27 Формирование состава пожарных машин

- (a) Создайте класс `FireTruck`, который будет представлять собой пожарную машину со специализацией. В конструкторе класса `FireTruck` инициализируйте значения машины и специализации из списков `NumList` и `MasList`, которые объявлены как общие атрибуты класса. `NumList: list[str]` — это список машин (не менее 14):

```
['Машина_1', 'Машина_2', ..., 'Машина_14']
```

`MasList: list[str]` — это список специализаций (не менее 4):

```
['Тушение', 'Спасение', 'Химзащита', 'Высотные работы']
```

Конструктор должен иметь сигнатуру: `__init__(self) -> None`.

- (b) Создайте класс `FireTrain`, который будет представлять собой состав пожарных машин. В конструкторе класса `FireTrain` инициализируйте список машин `self.train: list[FireTruck]` длиной 56.
- (c) Добавьте метод `shuffle(self) -> None` в класс `FireTrain`, который будет перемешивать машины в списке `self.train`.
- (d) Добавьте метод `get(self, i: int) -> FireTruck`, который будет возвращать i -ю машину и её специализацию из списка `self.train`.
- (e) Создайте экземпляр класса `FireTrain` и вызовите метод `shuffle` для перемешивания машин.
- (f) Создайте цикл, который будет запрашивать у пользователя номер машины и выводить информацию о ней.
- (g) Повторите шаги 5–6 до тех пор, пока пользователь не выберет все машины или не завершит выбор.
- (h) В конце программы выводите сообщение о завершении выбора машин.
- (i) Убедитесь, что пользователь вводит корректные номера машин и что программа обрабатывает ошибки, связанные с вводом пользователя.
- (j) Проверьте работу программы, используя различные комбинации номеров машин и специализаций.

28 Формирование состава эвакуаторов

- (a) Создайте класс `TowTruck`, который будет представлять собой эвакуатор с типом транспортного средства. В конструкторе класса `TowTruck` инициализируйте значения эвакуатора и типа ТС из списков `NumList` и `MasList`, которые объявлены как общие атрибуты класса. `NumList: list[str]` — это список эвакуаторов (не менее 14):

```
['Эвакуатор_1', 'Эвакуатор_2', ..., 'Эвакуатор_14']
```

`MasList: list[str]` — это список типов ТС (не менее 4):

```
['Легковой', 'Грузовик', 'Мотоцикл', 'Автобус']
```

Конструктор должен иметь сигнатуру: `__init__(self) -> None`.

- (b) Создайте класс `TowTrain`, который будет представлять собой состав эвакуаторов. В конструкторе класса `TowTrain` инициализируйте список эвакуаторов `self.train: list[TowTruck]` длиной 56.
- (c) Добавьте метод `shuffle(self) -> None` в класс `TowTrain`, который будет перемешивать эвакуаторы в списке `self.train`.

- (d) Добавьте метод `get(self, i: int) -> TowTruck`, который будет возвращать i -й эвакуатор и тип его ТС из списка `self.train`.
- (e) Создайте экземпляр класса `TowTrain` и вызовите метод `shuffle` для перемешивания эвакуаторов.
- (f) Создайте цикл, который будет запрашивать у пользователя номер эвакуатора и выводить информацию о нём.
- (g) Повторите шаги 5–6 до тех пор, пока пользователь не выберет все эвакуаторы или не завершит выбор.
- (h) В конце программы выводите сообщение о завершении выбора эвакуаторов.
- (i) Убедитесь, что пользователь вводит корректные номера эвакуаторов и что программа обрабатывает ошибки, связанные с вводом пользователя.
- (j) Проверьте работу программы, используя различные комбинации номеров эвакуаторов и типов ТС.

29 Формирование состава контейнеров с лекарствами

- (a) Создайте класс `MedBox`, который будет представлять собой контейнер с типом лекарств. В конструкторе класса `MedBox` инициализируйте значения контейнера и типа лекарств из списков `NumList` и `MasList`, которые объявлены как общие атрибуты класса. `NumList: list[str]` — это список контейнеров (не менее 14):

```
['Контейнер_1', 'Контейнер_2', ..., 'Контейнер_14']
```

`MasList: list[str]` — это список типов лекарств (не менее 4):

```
['Антибиотики', 'Вакцины', 'Обезболивающие', 'Витамины']
```

Конструктор должен иметь сигнатуру: `__init__(self) -> None`.

- (b) Создайте класс `MedTrain`, который будет представлять собой состав контейнеров. В конструкторе класса `MedTrain` инициализируйте список контейнеров `self.train: list[MedBox]` длиной 56.
- (c) Добавьте метод `shuffle(self) -> None` в класс `MedTrain`, который будет перемешивать контейнеры в списке `self.train`.
- (d) Добавьте метод `get(self, i: int) -> MedBox`, который будет возвращать i -й контейнер и его лекарства из списка `self.train`.
- (e) Создайте экземпляр класса `MedTrain` и вызовите метод `shuffle` для перемешивания контейнеров.
- (f) Создайте цикл, который будет запрашивать у пользователя номер контейнера и выводить информацию о нём.
- (g) Повторите шаги 5–6 до тех пор, пока пользователь не выберет все контейнеры или не завершит выбор.
- (h) В конце программы выводите сообщение о завершении выбора контейнеров.
- (i) Убедитесь, что пользователь вводит корректные номера контейнеров и что программа обрабатывает ошибки, связанные с вводом пользователя.
- (j) Проверьте работу программы, используя различные комбинации номеров контейнеров и типов лекарств.

30 Формирование состава транспорта с опасными грузами

- (a) Создайте класс `HazmatTruck`, который будет представлять собой грузовик с классом опасности. В конструкторе класса `HazmatTruck` инициализируйте значения грузовика и класса опасности из списков `NumList` и `MasList`, которые объявлены как общие атрибуты класса. `NumList: list[str]` — это список грузовиков (не менее 14):

```
["Грузовик_1", "Грузовик_2", ..., "Грузовик_14"]
```

`MasList: list[str]` — это список классов опасности (не менее 4):

```
["Взрывчатка", "Газы", "Легковоспламеняющиеся", "Токсичные"]
```

Конструктор должен иметь сигнатуру: `__init__(self) -> None`.

- (b) Создайте класс `HazmatConvoy`, который будет представлять собой конвой грузовиков. В конструкторе класса `HazmatConvoy` инициализируйте список грузовиков `self.train: list[HazmatTruck]` длиной 56.
- (c) Добавьте метод `shuffle(self) -> None` в класс `HazmatConvoy`, который будет перемешивать грузовики в списке `self.train`.
- (d) Добавьте метод `get(self, i: int) -> HazmatTruck`, который будет возвращать i -й грузовик и его класс опасности из списка `self.train`.
- (e) Создайте экземпляр класса `HazmatConvoy` и вызовите метод `shuffle` для перемешивания грузовиков.
- (f) Создайте цикл, который будет запрашивать у пользователя номер грузовика и выводить информацию о нём.
- (g) Повторите шаги 5–6 до тех пор, пока пользователь не выберет все грузовики или не завершит выбор.
- (h) В конце программы выводите сообщение о завершении выбора грузовиков.
- (i) Убедитесь, что пользователь вводит корректные номера грузовиков и что программа обрабатывает ошибки, связанные с вводом пользователя.
- (j) Проверьте работу программы, используя различные комбинации номеров грузовиков и классов опасности.

31 Формирование состава курьерских пакетов

- (a) Создайте класс `Package`, который будет представлять собой пакет с типом доставки. В конструкторе класса `Package` инициализируйте значения пакета и типа доставки из списков `NumList` и `MasList`, которые объявлены как общие атрибуты класса. `NumList: list[str]` — это список пакетов (не менее 14):

```
["Пакет_1", "Пакет_2", ..., "Пакет_14"]
```

`MasList: list[str]` — это список типов доставки (не менее 4):

```
["Экспресс", "Стандарт", "Международный", "Хрупкий"]
```

Конструктор должен иметь сигнатуру: `__init__(self) -> None`.

- (b) Создайте класс `PackageTrain`, который будет представлять собой состав пакетов. В конструкторе класса `PackageTrain` инициализируйте список пакетов `self.train: list[Package]` длиной 56.
- (c) Добавьте метод `shuffle(self) -> None` в класс `PackageTrain`, который будет перемешивать пакеты в списке `self.train`.

- (d) Добавьте метод `get(self, i: int) -> Package`, который будет возвращать i -й пакет и его тип доставки из списка `self.train`.
- (e) Создайте экземпляр класса `PackageTrain` и вызовите метод `shuffle` для перемешивания пакетов.
- (f) Создайте цикл, который будет запрашивать у пользователя номер пакета и выводить информацию о нём.
- (g) Повторите шаги 5–6 до тех пор, пока пользователь не выберет все пакеты или не завершит выбор.
- (h) В конце программы выводите сообщение о завершении выбора пакетов.
- (i) Убедитесь, что пользователь вводит корректные номера пакетов и что программа обрабатывает ошибки, связанные с вводом пользователя.
- (j) Проверьте работу программы, используя различные комбинации номеров пакетов и типов доставки.

32 Формирование состава мобильных медицинских лабораторий

- (a) Создайте класс `LabVan`, который будет представлять собой лабораторию с типом анализа. В конструкторе класса `LabVan` инициализируйте значения лаборатории и типа анализа из списков `NumList` и `MasList`, которые объявлены как общие атрибуты класса. `NumList: list[str]` — это список лабораторий (не менее 14):

```
["Лаборатория_1", "Лаборатория_2", ..., "Лаборатория_14"]
```

`MasList: list[str]` — это список типов анализов (не менее 4):

```
["PCR", "Анализ крови", "Токсикология", "Микробиология"]
```

Конструктор должен иметь сигнатуру: `__init__(self) -> None`.

- (b) Создайте класс `LabConvoy`, который будет представлять собой конвой лабораторий. В конструкторе класса `LabConvoy` инициализируйте список лабораторий `self.train: list[LabVan]` длиной 56.
- (c) Добавьте метод `shuffle(self) -> None` в класс `LabConvoy`, который будет перемешивать лаборатории в списке `self.train`.
- (d) Добавьте метод `get(self, i: int) -> LabVan`, который будет возвращать i -ю лабораторию и её тип анализа из списка `self.train`.
- (e) Создайте экземпляр класса `LabConvoy` и вызовите метод `shuffle` для перемешивания лабораторий.
- (f) Создайте цикл, который будет запрашивать у пользователя номер лаборатории и выводить информацию о ней.
- (g) Повторите шаги 5–6 до тех пор, пока пользователь не выберет все лаборатории или не завершит выбор.
- (h) В конце программы выводите сообщение о завершении выбора лабораторий.
- (i) Убедитесь, что пользователь вводит корректные номера лабораторий и что программа обрабатывает ошибки, связанные с вводом пользователя.
- (j) Проверьте работу программы, используя различные комбинации номеров лабораторий и типов анализов.

33 Формирование состава контейнеров с артефактами

- (a) Создайте класс `ArtifactCase`, который будет представлять собой кейс с происхождением артефакта. В конструкторе класса `ArtifactCase` инициализируйте значения кейса и происхождения из списков `NumList` и `MasList`, которые объявлены как общие атрибуты класса. `NumList: list[str]` — это список кейсов (не менее 14):

```
["Кейс_1", "Кейс_2", ..., "Кейс_14"]
```

`MasList: list[str]` — это список происхождений (не менее 4):

```
["Египет", "Греция", "Мезоамерика", "Древний Китай"]
```

Конструктор должен иметь сигнатуру: `__init__(self) -> None`.

- (b) Создайте класс `ArtifactTrain`, который будет представлять собой состав кейсов. В конструкторе класса `ArtifactTrain` инициализируйте список кейсов `self.train: list[ArtifactCase]` длиной 56.
- (c) Добавьте метод `shuffle(self) -> None` в класс `ArtifactTrain`, который будет перемешивать кейсы в списке `self.train`.
- (d) Добавьте метод `get(self, i: int) -> ArtifactCase`, который будет возвращать i -й кейс и происхождение его артефакта из списка `self.train`.
- (e) Создайте экземпляр класса `ArtifactTrain` и вызовите метод `shuffle` для перемешивания кейсов.
- (f) Создайте цикл, который будет запрашивать у пользователя номер кейса и выводить информацию о нём.
- (g) Повторите шаги 5–6 до тех пор, пока пользователь не выберет все кейсы или не завершит выбор.
- (h) В конце программы выводите сообщение о завершении выбора кейсов.
- (i) Убедитесь, что пользователь вводит корректные номера кейсов и что программа обрабатывает ошибки, связанные с вводом пользователя.
- (j) Проверьте работу программы, используя различные комбинации номеров кейсов и происхождений.

34 Формирование состава беспилотных грузовиков

- (a) Создайте класс `AutonomousTruck`, который будет представлять собой грузовик с типом маршрута. В конструкторе класса `AutonomousTruck` инициализируйте значения грузовика и маршрута из списков `NumList` и `MasList`, которые объявлены как общие атрибуты класса. `NumList: list[str]` — это список грузовиков (не менее 14):

```
["Грузовик_1", "Грузовик_2", ..., "Грузовик_14"]
```

`MasList: list[str]` — это список типов маршрутов (не менее 4):

```
["Город", "Шоссе", "Горы", "Пустыня"]
```

Конструктор должен иметь сигнатуру: `__init__(self) -> None`.

- (b) Создайте класс `TruckConvoy`, который будет представлять собой конвой грузовиков. В конструкторе класса `TruckConvoy` инициализируйте список грузовиков `self.train: list[AutonomousTruck]` длиной 56.

- (c) Добавьте метод `shuffle(self) -> None` в класс `TruckConvoy`, который будет перемешивать грузовики в списке `self.train`.
- (d) Добавьте метод `get(self, i: int) -> AutonomousTruck`, который будет возвращать i -й грузовик и его маршрут из списка `self.train`.
- (e) Создайте экземпляр класса `TruckConvoy` и вызовите метод `shuffle` для перемешивания грузовиков.
- (f) Создайте цикл, который будет запрашивать у пользователя номер грузовика и выводить информацию о нём.
- (g) Повторите шаги 5–6 до тех пор, пока пользователь не выберет все грузовики или не завершит выбор.
- (h) В конце программы выводите сообщение о завершении выбора грузовиков.
- (i) Убедитесь, что пользователь вводит корректные номера грузовиков и что программа обрабатывает ошибки, связанные с вводом пользователя.
- (j) Проверьте работу программы, используя различные комбинации номеров грузовиков и маршрутов.

35 Формирование состава грузовых вагонов (оригинальный вариант)

- (a) Создайте класс `Vagon`, который будет представлять собой вагон с грузом. В конструкторе класса `Vagon` инициализируйте значения вагона и груза из списков `NumList` и `MasList`, которые объявлены как общие атрибуты класса. `NumList: list[str]` — это список крытых вагонов (не менее 14):

`['Вагон_1', 'Вагон_2', ..., 'Вагон_14']`

`MasList: list[str]` — это список грузов для крытых вагонов (не менее 4):

`['Станки', 'Автозапчасти', 'Бумага', 'Керамическая плитка']`

Конструктор должен иметь сигнатуру: `__init__(self) -> None`.

- (b) Создайте класс `TrainOfVagons`, который будет представлять собой грузовой поезд, состоящий из моделей вагонов. В конструкторе класса `TrainOfVagons` инициализируйте список вагонов `self.train: list[Vagon]` длиной 56.
- (c) Добавьте метод `shuffle(self) -> None` в класс `TrainOfVagons`, который будет перемешивать вагоны в списке `self.train`.
- (d) Добавьте метод `get(self, i: int) -> Vagon`, который будет возвращать i -й вагон и груз из списка `self.train`.
- (e) Создайте экземпляр класса `TrainOfVagons` и вызовите метод `shuffle` для перемешивания вагонов.
- (f) Создайте цикл, который будет запрашивать у пользователя номер вагона из поезда и выводить информацию о выбранном вагоне.
- (g) Повторите шаги 5–6 до тех пор, пока пользователь не выберет все вагоны или не завершит выбор.
- (h) В конце программы выводите сообщение о завершении выбора вагонов.
- (i) Убедитесь, что пользователь вводит корректные номера вагонов и что программа обрабатывает ошибки, связанные с вводом пользователя.
- (j) Проверьте работу программы, используя различные комбинации номеров вагонов и грузов.

2.5.2 Задача 2

1 Расчёт площади стен в зависимости от наличия встроенных картин и зеркал

- (a) Создайте два класса: `PictureMirror` и `Room`. Класс `PictureMirror` представляет отдельный встроенный элемент (картину или зеркало). Его конструктор принимает два аргумента: `width` — ширина элемента в метрах (положительное дробное число), `height` — высота элемента в метрах (положительное дробное число). Объект этого класса хранит только эти два значения. Класс `Room` описывает прямоугольную комнату. Его конструктор принимает три аргумента: `width` — ширина комнаты в метрах, `length` — длина комнаты в метрах, `height` — высота стен в метрах. Все значения должны быть положительными. Объект `Room` хранит геометрические размеры комнаты и список объектов `PictureMirror`, изначально пустой.
- (b) В классе `Room` реализуйте следующие методы:
- `add_item(self, item: PictureMirror) -> None` — добавляет переданный объект `PictureMirror` в внутренний список встроенных элементов комнаты. Метод не проверяет, помещается ли элемент на стене; предполагается, что все элементы корректно размещены.
 - `get_area_to_cover(self) -> float` — вычисляет и возвращает площадь стен, подлежащую отделке. Общая площадь стен комнаты рассчитывается по формуле $2 \cdot \text{height} \cdot (\text{width} + \text{length})$. Из этой площади вычитается суммарная площадь всех встроенных элементов (каждый элемент вносит вклад $\text{width} \cdot \text{height}$). Результат не может быть отрицательным: если суммарная площадь элементов превышает площадь стен, метод возвращает 0.0.
 - `get_panels_count(self, panel_width: float, panel_height: float) -> int` — рассчитывает минимальное количество декоративных панелей, необходимых для отделки вычисленной ранее площади. Площадь одной панели равна $\text{panel_width} \cdot \text{panel_height}$. Количество панелей определяется как результат деления площади под отделку на площадь одной панели, округлённый вверх до ближайшего целого (поскольку панели продаются только целиком).
- (c) Создайте три различных экземпляра класса `Room` с разными размерами и разным набором встроенных элементов (например, комната без элементов, комната с одной большой картиной, комната с несколькими зеркалами). Для каждого экземпляра вызовите методы `add_item` (при необходимости), `get_area_to_cover` и `get_panels_count`, чтобы продемонстрировать корректность реализации.
- (d) Запросите у пользователя данные для одной комнаты: ширину, длину и высоту комнаты (все — дробные числа), а также ширину и высоту одной декоративной панели (дробные числа).
- (e) Выведите на экран два значения: площадь стен под отделку (в квадратных метрах, с дробной частью) и минимальное количество необходимых панелей (целое число, округлённое вверх).

2 Расчёт площади стен в зависимости от наличия встроенных настенных устройств

- (a) Создайте два класса: `WallDevice` и `Room`. Класс `WallDevice` представляет отдельное настенное устройство (например, панель управления). Его конструктор принимает два аргумента: `width` — ширина устройства в метрах (положительное

дробное число), `height` — высота устройства в метрах (положительное дробное число). Объект хранит только эти два значения. Класс `Room` описывает прямоугольную комнату. Его конструктор принимает три аргумента: `width` — ширина комнаты в метрах, `length` — длина комнаты в метрах, `height` — высота стен в метрах. Все значения должны быть положительными. Объект `Room` хранит размеры комнаты и список объектов `WallDevice`, изначально пустой.

- (b) В классе `Room` реализуйте следующие методы:
- `add_device(self, dev: WallDevice) -> None` — добавляет переданный объект `WallDevice` в список встроенных устройств комнаты.
 - `get_area_to_cover(self) -> float` — вычисляет площадь стен под отделку: из общей площади стен $2 \cdot \text{height} \cdot (\text{width} + \text{length})$ вычитается суммарная площадь всех устройств. Результат не может быть меньше нуля.
 - `get_tiles_count(self, tile_width: float, tile_height: float) -> int` — рассчитывает количество керамических плиток, необходимых для облицовки. Площадь одной плитки равна $\text{tile_width} \cdot \text{tile_height}$. Количество плиток — это результат деления площади под отделку на площадь плитки, округлённый вверх до целого числа.
- (c) Создайте три различных экземпляра класса `Room` с разными параметрами и разным числом устройств. Для каждого вызовите методы для получения площади и количества плиток.
- (d) Запросите у пользователя размеры комнаты (ширина, длина, высота) и размеры одной плитки (ширина и высота), все — дробные числа.
- (e) Выведите площадь стен под облицовку (м^2) и минимальное количество плиток (целое число, округлённое вверх).

3 Расчёт площади стен в зависимости от наличия встроенных светильников и бра

- (a) Создайте два класса: `Lamp` и `Room`. Класс `Lamp` представляет один настенный светильник или бра. Его конструктор принимает: `width` — ширина светильника в метрах, `height` — высота светильника в метрах. Оба значения — положительные дробные числа. Класс `Room` описывает комнату. Его конструктор принимает: `width`, `length`, `height` — размеры комнаты в метрах (все положительные). Объект `Room` хранит эти размеры и список объектов `Lamp`.
- (b) В классе `Room` реализуйте методы:
- `add_lamp(self, lamp: Lamp) -> None` — добавляет светильник в список встроенных элементов.
 - `get_area_to_cover(self) -> float` — возвращает площадь стен без учёта площадей всех светильников. Общая площадь стен: $2 \cdot \text{height} \cdot (\text{width} + \text{length})$. Из неё вычитается сумма площадей всех светильников. Результат ≥ 0 .
 - `get_wallpaper_rolls(self, roll_width: float, roll_length: float) -> int` — вычисляет количество рулонов обоев. Площадь одного рулона: $\text{roll_width} \cdot \text{roll_length}$. Количество рулонов — частное от деления площади под оклейку на площадь рулона, округлённое вверх до целого.
- (c) Создайте три разных экземпляра `Room` (с разным числом светильников) и протестируйте методы.

- (d) Запросите у пользователя размеры комнаты и размеры одного рулона обоев (все — дробные числа).
- (e) Выведите площадь под оклейку (м^2) и количество рулонов обоев (целое число, округлённое вверх).

4 Расчёт площади стен в зависимости от наличия встроенных полок и стеллажей

- (a) Создайте два класса: `Shelf` и `Room`. Класс `Shelf` описывает одну полку или стеллаж. Его конструктор принимает: `width` — ширина в метрах, `height` — высота в метрах (оба — положительные дробные числа). Класс `Room` описывает комнату с параметрами: `width`, `length`, `height` — размеры комнаты в метрах. Объект `Room` хранит список объектов `Shelf`.
- (b) В классе `Room` реализуйте методы:
 - `add_shelf(self, shelf: Shelf) -> None` — добавляет полку в комнату.
 - `get_area_to_cover(self) -> float` — площадь стен под покраску: общая площадь стен минус суммарная площадь всех полок (результат ≥ 0).
 - `get_paint_liters(self, coverage: float) -> float` — вычисляет необходимый объём краски в литрах. Аргумент `coverage` задаёт, сколько квадратных метров можно покрыть одним литром краски ($\text{м}^2/\text{л}$). Объём краски = площадь под покраску / `coverage`. Результат может быть дробным, так как краску можно купить нецелыми банками.
- (c) Создайте три различных комнаты с разным числом полок и проверьте работу методов.
- (d) Запросите у пользователя размеры комнаты и значение `coverage` (дробное число).
- (e) Выведите площадь под покраску (м^2) и необходимое количество литров краски (с дробной частью).

5 Расчёт площади стен в зависимости от наличия встроенных розеток и выключателей

- (a) Создайте два класса: `SocketSwitch` и `Room`. Класс `SocketSwitch` представляет одну розетку или выключатель. Его конструктор принимает: `width` — ширина в метрах, `height` — высота в метрах (положительные дробные числа). Класс `Room` описывает комнату с размерами `width`, `length`, `height` (все — положительные дробные числа) и хранит список объектов `SocketSwitch`.
- (b) В классе `Room` реализуйте методы:
 - `add_electrical(self, el: SocketSwitch) -> None` — добавляет электроаппаратуру в комнату.
 - `get_area_to_cover(self) -> float` — площадь стен под штукатурку: общая площадь стен минус сумма площадей всех розеток и выключателей (результат ≥ 0).
 - `get_plaster_bags(self, bag_coverage: float) -> int` — количество мешков штукатурки. Аргумент `bag_coverage` — сколько квадратных метров покрывает один мешок ($\text{м}^2/\text{мешок}$). Количество мешков = площадь под штукатурку / `bag_coverage`, округлённое вверх до целого.

- (c) Создайте три разных экземпляра `Room` с разным числом электроустройств и протестируйте методы.
- (d) Запросите у пользователя размеры комнаты и `bag_coverage` (дробное число).
- (e) Выведите площадь под штукатурку (м^2) и количество мешков (целое число, округлённое вверх).

6 Расчёт площади стен в зависимости от наличия встроенных вентиляционных решёток

- (a) Создайте два класса: `VentGrille` и `Room`. Класс `VentGrille` описывает одну вентиляционную решётку. Его конструктор принимает: `width` — ширина решётки в метрах, `height` — высота решётки в метрах (положительные дробные числа). Класс `Room` описывает комнату с размерами `width`, `length`, `height` и хранит список объектов `VentGrille`.
- (b) В классе `Room` реализуйте методы:
 - `add_vent(self, vent: VentGrille) -> None` — добавляет решётку в комнату.
 - `get_area_to_cover(self) -> float` — площадь стен под обшивку: общая площадь стен минус сумма площадей всех решёток (результат ≥ 0).
 - `get_panel_sheets(self, sheet_width: float, sheet_height: float) -> int` — количество листов панелей. Площадь одного листа = `sheet_width * sheet_height`. Количество листов — частное от деления площади под обшивку на площадь листа, округлённое вверх до целого.
- (c) Создайте три различных комнаты с разным числом решёток и проверьте методы.
- (d) Запросите у пользователя размеры комнаты и размеры одного листа панели (все — дробные числа).
- (e) Выведите площадь под обшивку (м^2) и количество листов (целое число, округлённое вверх).

7 Расчёт площади стен в зависимости от наличия встроенных настенных кондиционеров

- (a) Создайте два класса: `WallAC` и `Room`. Класс `WallAC` представляет один настенный кондиционер. Его конструктор принимает: `width` — ширина в метрах, `height` — высота в метрах (положительные дробные числа). Класс `Room` описывает комнату с размерами `width`, `length`, `height` и хранит список объектов `WallAC`.
- (b) В классе `Room` реализуйте методы:
 - `add_ac(self, ac: WallAC) -> None` — добавляет кондиционер в комнату.
 - `get_area_to_cover(self) -> float` — площадь стен под оклейку обоями: общая площадь стен минус сумма площадей всех кондиционеров (результат ≥ 0).
 - `get_wallpaper_rolls(self, roll_width: float, roll_length: float) -> int` — количество рулонов обоев. Площадь рулона = `roll_width * roll_length`. Количество рулонов — частное от деления площади под оклейку на площадь рулона, округлённое вверх до целого.
- (c) Создайте три разных экземпляра `Room` с разным числом кондиционеров и протестируйте методы.

- (d) Запросите у пользователя размеры комнаты и размеры рулона обоев (все — дробные числа).
- (e) Выведите площадь под оклейку (м^2) и количество рулонов (целое число, округлённое вверх).

8 Расчёт площади стен в зависимости от наличия встроенных настенных экранов

- (a) Создайте два класса: `WallScreen` и `Room`. Класс `WallScreen` описывает один настенный экран. Его конструктор принимает: `width` — ширина в метрах, `height` — высота в метрах (положительные дробные числа). Класс `Room` описывает комнату с размерами `width`, `length`, `height` и хранит список объектов `WallScreen`.
- (b) В классе `Room` реализуйте методы:
 - `add_screen(self, scr: WallScreen) -> None` — добавляет экран в комнату.
 - `get_area_to_cover(self) -> float` — площадь стен под покраску: общая площадь стен минус сумма площадей всех экранов (результат ≥ 0).
 - `get_paint_cans(self, can_coverage: float) -> int` — количество банок краски. Аргумент `can_coverage` — сколько квадратных метров покрывает одна банка ($\text{м}^2/\text{банка}$). Количество банок = площадь под покраску / `can_coverage`, округлённое вверх до целого.
- (c) Создайте три различных комнаты с разным числом экранов и проверьте работу методов.
- (d) Запросите у пользователя размеры комнаты и `can_coverage` (дробное число).
- (e) Выведите площадь под покраску (м^2) и количество банок (целое число, округлённое вверх).

9 Расчёт площади стен в зависимости от наличия встроенных настенных сейфов

- (a) Создайте два класса: `WallSafe` и `Room`. Класс `WallSafe` представляет один настенный сейф. Его конструктор принимает: `width` — ширина в метрах, `height` — высота в метрах (положительные дробные числа). Класс `Room` описывает комнату с размерами `width`, `length`, `height` и хранит список объектов `WallSafe`.
- (b) В классе `Room` реализуйте методы:
 - `add_safe(self, safe: WallSafe) -> None` — добавляет сейф в комнату.
 - `get_area_to_cover(self) -> float` — площадь стен под облицовку: общая площадь стен минус сумма площадей всех сейфов (результат ≥ 0).
 - `get_tiles_count(self, tile_width: float, tile_height: float) -> int` — количество плиток. Площадь одной плитки = `tile_width` · `tile_height`. Количество плиток — частное от деления площади под облицовку на площадь плитки, округлённое вверх до целого.
- (c) Создайте три разных экземпляра `Room` с разным числом сейфов и протестируйте методы.
- (d) Запросите у пользователя размеры комнаты и размеры плитки (все — дробные числа).
- (e) Выведите площадь под облицовку (м^2) и количество плиток (целое число, округлённое вверх).

10 Расчёт площади стен в зависимости от наличия встроенных настенных досок

- (a) Создайте два класса: `WallBoard` и `Room`. Класс `WallBoard` описывает одну настенную доску. Его конструктор принимает: `width` — ширина в метрах, `height` — высота в метрах (положительные дробные числа). Класс `Room` описывает комнату с размерами `width`, `length`, `height` и хранит список объектов `WallBoard`.
- (b) В классе `Room` реализуйте методы:
- `add_board(self, board: WallBoard) -> None` — добавляет доску в комнату.
 - `get_area_to_cover(self) -> float` — площадь стен под драпировку: общая площадь стен минус сумма площадей всех досок (результат ≥ 0).
 - `get_fabric_meters(self, fabric_width: float) -> float` — необходимая длина ткани в метрах. Аргумент `fabric_width` — ширина ткани в метрах. Длина ткани = площадь под драпировку / `fabric_width`. Результат может быть дробным, так как ткань продаётся погонными метрами.
- (c) Создайте три различных комнаты с разным числом досок и проверьте методы.
- (d) Запросите у пользователя размеры комнаты и ширину ткани (дробные числа).
- (e) Выведите площадь под драпировку (м^2) и количество метров ткани (с дробной частью).

11 Расчёт площади стен в зависимости от наличия встроенных настенных календарей

- (a) Создайте два класса: `WallCalendar` и `Room`. Класс `WallCalendar` представляет один настенный календарь. Его конструктор принимает: `width` — ширина в метрах, `height` — высота в метрах (положительные дробные числа). Класс `Room` описывает комнату с размерами `width`, `length`, `height` и хранит список объектов `WallCalendar`.
- (b) В классе `Room` реализуйте методы:
- `add_calendar(self, cal: WallCalendar) -> None` — добавляет календарь в комнату.
 - `get_area_to_cover(self) -> float` — площадь стен под оклейку: общая площадь стен минус сумма площадей всех календарей (результат ≥ 0).
 - `get_wallpaper_rolls(self, roll_width: float, roll_length: float) -> int` — количество рулонов обоев. Площадь рулона = `roll_width` · `roll_length`. Количество рулонов — частное от деления площади под оклейку на площадь рулона, округлённое вверх до целого.
- (c) Создайте три разных экземпляра `Room` с разным числом календарей и протестируйте методы.
- (d) Запросите у пользователя размеры комнаты и размеры рулона обоев (все — дробные числа).
- (e) Выведите площадь под оклейку (м^2) и количество рулонов (целое число, округлённое вверх).

12 Расчёт площади стен в зависимости от наличия встроенных настенных карт

- (a) Создайте два класса: `WallMap` и `Room`. Класс `WallMap` описывает одну настенную карту. Его конструктор принимает: `width` — ширина в метрах, `height` — высота в метрах (положительные дробные числа). Класс `Room` описывает комнату с размерами `width`, `length`, `height` и хранит список объектов `WallMap`.
- (b) В классе `Room` реализуйте методы:
 - `add_map(self, map: WallMap) -> None` — добавляет карту в комнату.
 - `get_area_to_cover(self) -> float` — площадь стен под покраску: общая площадь стен минус сумма площадей всех карт (результат ≥ 0).
 - `get_paint_liters(self, coverage: float) -> float` — объём краски в литрах. Аргумент `coverage` — расход краски ($\text{м}^2/\text{л}$). Объём = площадь под покраску / `coverage`. Результат может быть дробным.
- (c) Создайте три различных комнаты с разным числом карт и проверьте методы.
- (d) Запросите у пользователя размеры комнаты и `coverage` (дробное число).
- (e) Выведите площадь под покраску (м^2) и количество литров краски (с дробной частью).

13 Расчёт площади стен в зависимости от наличия встроенных настенных террариумов

- (a) Создайте два класса: `WallTerrarium` и `Room`. Класс `WallTerrarium` представляет один настенный террариум. Его конструктор принимает: `width` — ширина в метрах, `height` — высота в метрах (положительные дробные числа). Класс `Room` описывает комнату с размерами `width`, `length`, `height` и хранит список объектов `WallTerrarium`.
- (b) В классе `Room` реализуйте методы:
 - `add_terrarium(self, terr: WallTerrarium) -> None` — добавляет террариум в комнату.
 - `get_area_to_cover(self) -> float` — площадь стен под отделку: общая площадь стен минус сумма площадей всех террариумов (результат ≥ 0).
 - `get_panels_count(self, panel_width: float, panel_height: float) -> int` — количество декоративных панелей. Площадь одной панели = `panel_width` · `panel_height`. Количество панелей — частное от деления площади под отделку на площадь панели, округлённое вверх до целого.
- (c) Создайте три разных экземпляра `Room` с разным числом террариумов и протестируйте методы.
- (d) Запросите у пользователя размеры комнаты и размеры панели (все — дробные числа).
- (e) Выведите площадь под отделку (м^2) и количество панелей (целое число, округлённое вверх).

14 Расчёт площади стен в зависимости от наличия встроенных настенных аквариумов

- (a) Создайте два класса: `WallAquarium` и `Room`. Класс `WallAquarium` описывает один настенный аквариум. Его конструктор принимает: `width` — ширина в метрах, `height` — высота в метрах (положительные дробные числа). Класс `Room` описывает комнату с размерами `width`, `length`, `height` и хранит список объектов `WallAquarium`.

- (b) В классе `Room` реализуйте методы:
- `add_aquarium(self, aq: WallAquarium) -> None` — добавляет аквариум в комнату.
 - `get_area_to_cover(self) -> float` — площадь стен под облицовку: общая площадь стен минус сумма площадей всех аквариумов (результат ≥ 0).
 - `get_tiles_count(self, tile_width: float, tile_height: float) -> int` — количество плиток. Площадь одной плитки = `tile_width · tile_height`. Количество плиток — частное от деления площади под облицовку на площадь плитки, округлённое вверх до целого.
- (c) Создайте три различных комнаты с разным числом аквариумов и проверьте методы.
- (d) Запросите у пользователя размеры комнаты и размеры плитки (все — дробные числа).
- (e) Выведите площадь под облицовку (м^2) и количество плиток (целое число, округлённое вверх).

15 Расчёт площади стен в зависимости от наличия встроенных настенных динамиков

- (a) Создайте два класса: `WallSpeaker` и `Room`. Класс `WallSpeaker` представляет один настенный динамик. Его конструктор принимает: `width` — ширина в метрах, `height` — высота в метрах (положительные дробные числа). Класс `Room` описывает комнату с размерами `width`, `length`, `height` и хранит список объектов `WallSpeaker`.
- (b) В классе `Room` реализуйте методы:
- `add_speaker(self, sp: WallSpeaker) -> None` — добавляет динамик в комнату.
 - `get_area_to_cover(self) -> float` — площадь стен под оклейку: общая площадь стен минус сумма площадей всех динамиков (результат ≥ 0).
 - `get_wallpaper_rolls(self, roll_width: float, roll_length: float) -> int` — количество рулонов обоев. Площадь рулона = `roll_width · roll_length`. Количество рулонов — частное от деления площади под оклейку на площадь рулона, округлённое вверх до целого.
- (c) Создайте три разных экземпляра `Room` с разным числом динамиков и протестируйте методы.
- (d) Запросите у пользователя размеры комнаты и размеры рулона обоев (все — дробные числа).
- (e) Выведите площадь под оклейку (м^2) и количество рулонов (целое число, округлённое вверх).

16 Расчёт площади стен в зависимости от наличия встроенных настенных датчиков

- (a) Создайте два класса: `WallSensor` и `Room`. Класс `WallSensor` описывает один настенный датчик. Его конструктор принимает: `width` — ширина в метрах, `height` — высота в метрах (положительные дробные числа). Класс `Room` описывает комнату с размерами `width`, `length`, `height` и хранит список объектов `WallSensor`.

- (b) В классе `Room` реализуйте методы:
- `add_sensor(self, sens: WallSensor) -> None` — добавляет датчик в комнату.
 - `get_area_to_cover(self) -> float` — площадь стен под покраску: общая площадь стен минус сумма площадей всех датчиков (результат ≥ 0).
 - `get_paint_cans(self, can_coverage: float) -> int` — количество банок краски. Аргумент `can_coverage` — покрытие одной банки ($\text{м}^2/\text{банка}$). Количество банок = площадь под покраску / `can_coverage`, округлённое вверх до целого.
- (c) Создайте три различных комнаты с разным числом датчиков и проверьте методы.
- (d) Запросите у пользователя размеры комнаты и `can_coverage` (дробное число).
- (e) Выведите площадь под покраску (м^2) и количество банок (целое число, округлённое вверх).

17 Расчёт площади стен в зависимости от наличия встроенных настенных панно

- (a) Создайте два класса: `WallPanel` и `Room`. Класс `WallPanel` представляет одно настенное панно. Его конструктор принимает: `width` — ширина в метрах, `height` — высота в метрах (положительные дробные числа). Класс `Room` описывает комнату с размерами `width`, `length`, `height` и хранит список объектов `WallPanel`.
- (b) В классе `Room` реализуйте методы:
- `add_panel(self, p: WallPanel) -> None` — добавляет панно в комнату.
 - `get_area_to_cover(self) -> float` — площадь стен под драпировку: общая площадь стен минус сумма площадей всех панно (результат ≥ 0).
 - `get_fabric_meters(self, fabric_width: float) -> float` — длина ткани в метрах. Аргумент `fabric_width` — ширина ткани. Длина = площадь под драпировку / `fabric_width`. Результат может быть дробным.
- (c) Создайте три разных экземпляра `Room` с разным числом панно и протестируйте методы.
- (d) Запросите у пользователя размеры комнаты и ширину ткани (дробные числа).
- (e) Выведите площадь под драпировку (м^2) и количество метров ткани (с дробной частью).

18 Расчёт площади стен в зависимости от наличия встроенных настенных рамок

- (a) Создайте два класса: `WallFrame` и `Room`. Класс `WallFrame` описывает одну настенную рамку. Его конструктор принимает: `width` — ширина в метрах, `height` — высота в метрах (положительные дробные числа). Класс `Room` описывает комнату с размерами `width`, `length`, `height` и хранит список объектов `WallFrame`.
- (b) В классе `Room` реализуйте методы:
- `add_frame(self, f: WallFrame) -> None` — добавляет рамку в комнату.
 - `get_area_to_cover(self) -> float` — площадь стен под оклейку: общая площадь стен минус сумма площадей всех рамок (результат ≥ 0).

- `get_wallpaper_rolls(self, roll_width: float, roll_length: float) -> int` — количество рулонов обоев. Площадь рулона = `roll_width · roll_length`. Количество рулонов — частное от деления площади под оклейку на площадь рулона, округлённое вверх до целого.
- (c) Создайте три различных комнаты с разным числом рамок и проверьте методы.
- (d) Запросите у пользователя размеры комнаты и размеры рулона обоев (все — дробные числа).
- (e) Выведите площадь под оклейку (м^2) и количество рулонов (целое число, округлённое вверх).

19 Расчёт площади стен в зависимости от наличия встроенных настенных витрин

- (a) Создайте два класса: `WallShowcase` и `Room`. Класс `WallShowcase` представляет одну настенную витрину. Его конструктор принимает: `width` — ширина в метрах, `height` — высота в метрах (положительные дробные числа). Класс `Room` описывает комнату с размерами `width`, `length`, `height` и хранит список объектов `WallShowcase`.
- (b) В классе `Room` реализуйте методы:
 - `add_showcase(self, sc: WallShowcase) -> None` — добавляет витрину в комнату.
 - `get_area_to_cover(self) -> float` — площадь стен под облицовку: общая площадь стен минус сумма площадей всех витрин (результат ≥ 0).
 - `get_tiles_count(self, tile_width: float, tile_height: float) -> int` — количество плиток. Площадь одной плитки = `tile_width · tile_height`. Количество плиток — частное от деления площади под облицовку на площадь плитки, округлённое вверх до целого.
- (c) Создайте три разных экземпляра `Room` с разным числом витрин и протестируйте методы.
- (d) Запросите у пользователя размеры комнаты и размеры плитки (все — дробные числа).
- (e) Выведите площадь под облицовку (м^2) и количество плиток (целое число, округлённое вверх).

20 Расчёт площади стен в зависимости от наличия встроенных настенных кронштейнов

- (a) Создайте два класса: `WallBracket` и `Room`. Класс `WallBracket` описывает один настенный кронштейн. Его конструктор принимает: `width` — ширина в метрах, `height` — высота в метрах (положительные дробные числа). Класс `Room` описывает комнату с размерами `width`, `length`, `height` и хранит список объектов `WallBracket`.
- (b) В классе `Room` реализуйте методы:
 - `add_bracket(self, br: WallBracket) -> None` — добавляет кронштейн в комнату.
 - `get_area_to_cover(self) -> float` — площадь стен под покраску: общая площадь стен минус сумма площадей всех кронштейнов (результат ≥ 0).

- `get_paint_liters(self, coverage: float) -> float` — объём краски в литрах. Аргумент `coverage` — расход ($\text{м}^2/\text{л}$). Объём = площадь под покраску / `coverage`. Результат может быть дробным.
- (c) Создайте три различных комнаты с разным числом кронштейнов и проверьте методы.
- (d) Запросите у пользователя размеры комнаты и `coverage` (дробное число).
- (e) Выведите площадь под покраску (м^2) и количество литров краски (с дробной частью).

21 Расчёт площади стен в зависимости от наличия встроенных настенных жалюзи

- (a) Создайте два класса: `WallBlind` и `Room`. Класс `WallBlind` представляет одни настенные жалюзи. Его конструктор принимает: `width` — ширина в метрах, `height` — высота в метрах (положительные дробные числа). Класс `Room` описывает комнату с размерами `width`, `length`, `height` и хранит список объектов `WallBlind`.
- (b) В классе `Room` реализуйте методы:
 - `add_blind(self, bl: WallBlind) -> None` — добавляет жалюзи в комнату.
 - `get_area_to_cover(self) -> float` — площадь стен под драпировку: общая площадь стен минус сумма площадей всех жалюзи (результат ≥ 0).
 - `get_fabric_meters(self, fabric_width: float) -> float` — длина ткани в метрах. Аргумент `fabric_width` — ширина ткани. Длина = площадь под драпировку / `fabric_width`. Результат может быть дробным.
- (c) Создайте три разных экземпляра `Room` с разным числом жалюзи и протестируйте методы.
- (d) Запросите у пользователя размеры комнаты и ширину ткани (дробные числа).
- (e) Выведите площадь под драпировку (м^2) и количество метров ткани (с дробной частью).

22 Расчёт площади стен в зависимости от наличия встроенных настенных флагов

- (a) Создайте два класса: `WallFlag` и `Room`. Класс `WallFlag` описывает один настенный флаг. Его конструктор принимает: `width` — ширина в метрах, `height` — высота в метрах (положительные дробные числа). Класс `Room` описывает комнату с размерами `width`, `length`, `height` и хранит список объектов `WallFlag`.
- (b) В классе `Room` реализуйте методы:
 - `add_flag(self, fl: WallFlag) -> None` — добавляет флаг в комнату.
 - `get_area_to_cover(self) -> float` — площадь стен под оклейку: общая площадь стен минус сумма площадей всех флагов (результат ≥ 0).
 - `get_wallpaper_rolls(self, roll_width: float, roll_length: float) -> int` — количество рулонов обоев. Площадь рулона = `roll_width` · `roll_length`. Количество рулонов — частное от деления площади под оклейку на площадь рулона, округлённое вверх до целого.
- (c) Создайте три различных комнаты с разным числом флагов и проверьте методы.
- (d) Запросите у пользователя размеры комнаты и размеры рулона обоев (все — дробные числа).

- (e) Выведите площадь под оклейку (м^2) и количество рулонов (целое число, округлённое вверх).

23 Расчёт площади стен в зависимости от наличия встроенных грифельных досок

- (a) Создайте два класса: `Chalkboard` и `Room`. Класс `Chalkboard` представляет одну грифельную доску. Его конструктор принимает: `width` — ширина в метрах, `height` — высота в метрах (положительные дробные числа). Класс `Room` описывает комнату с размерами `width`, `length`, `height` и хранит список объектов `Chalkboard`.
- (b) В классе `Room` реализуйте методы:
- `add_board(self, cb: Chalkboard) -> None` — добавляет доску в комнату.
 - `get_area_to_cover(self) -> float` — площадь стен под покраску: общая площадь стен минус сумма площадей всех досок (результат ≥ 0).
 - `get_paint_cans(self, can_coverage: float) -> int` — количество банок краски. Аргумент `can_coverage` — покрытие одной банки ($\text{м}^2/\text{банка}$). Количество банок = площадь под покраску / `can_coverage`, округлённое вверх до целого.
- (c) Создайте три разных экземпляра `Room` с разным числом досок и протестируйте методы.
- (d) Запросите у пользователя размеры комнаты и `can_coverage` (дробное число).
- (e) Выведите площадь под покраску (м^2) и количество банок (целое число, округлённое вверх).

24 Расчёт площади стен в зависимости от наличия встроенных маркерных досок

- (a) Создайте два класса: `Whiteboard` и `Room`. Класс `Whiteboard` описывает одну маркерную доску. Его конструктор принимает: `width` — ширина в метрах, `height` — высота в метрах (положительные дробные числа). Класс `Room` описывает комнату с размерами `width`, `length`, `height` и хранит список объектов `Whiteboard`.
- (b) В классе `Room` реализуйте методы:
- `add_board(self, wb: Whiteboard) -> None` — добавляет доску в комнату.
 - `get_area_to_cover(self) -> float` — площадь стен под отделку: общая площадь стен минус сумма площадей всех досок (результат ≥ 0).
 - `get_panels_count(self, panel_width: float, panel_height: float) -> int` — количество декоративных панелей. Площадь одной панели = `panel_width` · `panel_height`. Количество панелей — частное от деления площади под отделку на площадь панели, округлённое вверх до целого.
- (c) Создайте три различных комнаты с разным числом досок и проверьте методы.
- (d) Запросите у пользователя размеры комнаты и размеры панели (все — дробные числа).
- (e) Выведите площадь под отделку (м^2) и количество панелей (целое число, округлённое вверх).

25 Расчёт площади стен в зависимости от наличия встроенных зеркал

- (a) Создайте два класса: `Mirror` и `Room`. Класс `Mirror` представляет одно зеркало. Его конструктор принимает: `width` — ширина в метрах, `height` — высота в метрах (положительные дробные числа). Класс `Room` описывает комнату с размерами `width`, `length`, `height` и хранит список объектов `Mirror`.
- (b) В классе `Room` реализуйте методы:
 - `add_mirror(self, m: Mirror) -> None` — добавляет зеркало в комнату.
 - `get_area_to_cover(self) -> float` — площадь стен под облицовку: общая площадь стен минус сумма площадей всех зеркал (результат ≥ 0).
 - `get_tiles_count(self, tile_width: float, tile_height: float) -> int` — количество плиток. Площадь одной плитки = `tile_width · tile_height`. Количество плиток — частное от деления площади под облицовку на площадь плитки, округлённое вверх до целого.
- (c) Создайте три разных экземпляра `Room` с разным числом зеркал и протестируйте методы.
- (d) Запросите у пользователя размеры комнаты и размеры плитки (все — дробные числа).
- (e) Выведите площадь под облицовку (м^2) и количество плиток (целое число, округлённое вверх).

26 Расчёт площади стен в зависимости от наличия встроенных часов

- (a) Создайте два класса: `WallClock` и `Room`. Класс `WallClock` описывает одни настенные часы. Его конструктор принимает: `width` — ширина в метрах, `height` — высота в метрах (положительные дробные числа). Класс `Room` описывает комнату с размерами `width`, `length`, `height` и хранит список объектов `WallClock`.
- (b) В классе `Room` реализуйте методы:
 - `add_clock(self, cl: WallClock) -> None` — добавляет часы в комнату.
 - `get_area_to_cover(self) -> float` — площадь стен под оклейку: общая площадь стен минус сумма площадей всех часов (результат ≥ 0).
 - `get_wallpaper_rolls(self, roll_width: float, roll_length: float) -> int` — количество рулонов обоев. Площадь рулона = `roll_width · roll_length`. Количество рулонов — частное от деления площади под оклейку на площадь рулона, округлённое вверх до целого.
- (c) Создайте три различных комнаты с разным числом часов и проверьте методы.
- (d) Запросите у пользователя размеры комнаты и размеры рулона обоев (все — дробные числа).
- (e) Выведите площадь под оклейку (м^2) и количество рулонов (целое число, округлённое вверх).

27 Расчёт площади стен в зависимости от наличия встроенных термометров

- (a) Создайте два класса: `Thermometer` и `Room`. Класс `Thermometer` представляет один настенный термометр. Его конструктор принимает: `width` — ширина в метрах, `height` — высота в метрах (положительные дробные числа). Класс `Room` описывает комнату с размерами `width`, `length`, `height` и хранит список объектов `Thermometer`.
- (b) В классе `Room` реализуйте методы:

- `add_thermometer(self, t: Thermometer) -> None` — добавляет термометр в комнату.
 - `get_area_to_cover(self) -> float` — площадь стен под покраску: общая площадь стен минус сумма площадей всех термометров (результат ≥ 0).
 - `get_paint_liters(self, coverage: float) -> float` — объём краски в литрах. Аргумент `coverage` — расход ($\text{м}^2/\text{л}$). Объём = площадь под покраску / `coverage`. Результат может быть дробным.
- (c) Создайте три разных экземпляра `Room` с разным числом термометров и протестируйте методы.
- (d) Запросите у пользователя размеры комнаты и `coverage` (дробное число).
- (e) Выведите площадь под покраску (м^2) и количество литров краски (с дробной частью).

28 Расчёт площади стен в зависимости от наличия встроенных барометров

- (a) Создайте два класса: `Barometer` и `Room`. Класс `Barometer` описывает один настенный барометр. Его конструктор принимает: `width` — ширина в метрах, `height` — высота в метрах (положительные дробные числа). Класс `Room` описывает комнату с размерами `width`, `length`, `height` и хранит список объектов `Barometer`.
- (b) В классе `Room` реализуйте методы:
- `add_barometer(self, b: Barometer) -> None` — добавляет барометр в комнату.
 - `get_area_to_cover(self) -> float` — площадь стен под драпировку: общая площадь стен минус сумма площадей всех барометров (результат ≥ 0).
 - `get_fabric_meters(self, fabric_width: float) -> float` — длина ткани в метрах. Аргумент `fabric_width` — ширина ткани. Длина = площадь под драпировку / `fabric_width`. Результат может быть дробным.
- (c) Создайте три различных комнаты с разным числом барометров и проверьте методы.
- (d) Запросите у пользователя размеры комнаты и ширину ткани (дробные числа).
- (e) Выведите площадь под драпировку (м^2) и количество метров ткани (с дробной частью).

29 Расчёт площади стен в зависимости от наличия встроенных гидрометров

- (a) Создайте два класса: `Hygrometer` и `Room`. Класс `Hygrometer` представляет один настенный гидрометр. Его конструктор принимает: `width` — ширина в метрах, `height` — высота в метрах (положительные дробные числа). Класс `Room` описывает комнату с размерами `width`, `length`, `height` и хранит список объектов `Hygrometer`.
- (b) В классе `Room` реализуйте методы:
- `add_hygrometer(self, h: Hygrometer) -> None` — добавляет гидрометр в комнату.
 - `get_area_to_cover(self) -> float` — площадь стен под оклейку: общая площадь стен минус сумма площадей всех гидрометров (результат ≥ 0).

- `get_wallpaper_rolls(self, roll_width: float, roll_length: float) -> int` — количество рулонов обоев. Площадь рулона = `roll_width · roll_length`. Количество рулонов — частное от деления площади под оклейку на площадь рулона, округлённое вверх до целого.
- (c) Создайте три разных экземпляра `Room` с разным числом гидрометров и протестируйте методы.
- (d) Запросите у пользователя размеры комнаты и размеры рулона обоев (все — дробные числа).
- (e) Выведите площадь под оклейку (м^2) и количество рулонов (целое число, округлённое вверх).

30 Расчёт площади стен в зависимости от наличия встроенных настенных растений

- (a) Создайте два класса: `WallPlant` и `Room`. Класс `WallPlant` описывает одно настенное растение (в кашпо или модуле). Его конструктор принимает: `width` — ширина в метрах, `height` — высота в метрах (положительные дробные числа). Класс `Room` описывает комнату с размерами `width`, `length`, `height` и хранит список объектов `WallPlant`.
- (b) В классе `Room` реализуйте методы:
 - `add_plant(self, p: WallPlant) -> None` — добавляет растение в комнату.
 - `get_area_to_cover(self) -> float` — площадь стен под покраску: общая площадь стен минус сумма площадей всех растений (результат ≥ 0).
 - `get_paint_cans(self, can_coverage: float) -> int` — количество банок краски. Аргумент `can_coverage` — покрытие одной банки ($\text{м}^2/\text{банка}$). Количество банок = площадь под покраску / `can_coverage`, округлённое вверх до целого.
- (c) Создайте три различных комнаты с разным числом растений и проверьте методы.
- (d) Запросите у пользователя размеры комнаты и `can_coverage` (дробное число).
- (e) Выведите площадь под покраску (м^2) и количество банок (целое число, округлённое вверх).

31 Расчёт площади стен в зависимости от наличия встроенных настенных фонарей

- (a) Создайте два класса: `WallLantern` и `Room`. Класс `WallLantern` представляет один настенный фонарь. Его конструктор принимает: `width` — ширина в метрах, `height` — высота в метрах (положительные дробные числа). Класс `Room` описывает комнату с размерами `width`, `length`, `height` и хранит список объектов `WallLantern`.
- (b) В классе `Room` реализуйте методы:
 - `add_lantern(self, l: WallLantern) -> None` — добавляет фонарь в комнату.
 - `get_area_to_cover(self) -> float` — площадь стен под отделку: общая площадь стен минус сумма площадей всех фонарей (результат ≥ 0).
 - `get_panels_count(self, panel_width: float, panel_height: float) -> int` — количество декоративных панелей. Площадь одной панели = `panel_width · panel_height`. Количество панелей — частное от деления площади под отделку на площадь панели, округлённое вверх до целого.

- (c) Создайте три разных экземпляра `Room` с разным числом фонарей и протестируйте методы.
- (d) Запросите у пользователя размеры комнаты и размеры панели (все — дробные числа).
- (e) Выведите площадь под отделку (м^2) и количество панелей (целое число, округлённое вверх).

32 Расчёт площади стен в зависимости от наличия встроенных настенных вентиляторов

- (a) Создайте два класса: `WallFan` и `Room`. Класс `WallFan` описывает один настенный вентилятор. Его конструктор принимает: `width` — ширина в метрах, `height` — высота в метрах (положительные дробные числа). Класс `Room` описывает комнату с размерами `width`, `length`, `height` и хранит список объектов `WallFan`.
- (b) В классе `Room` реализуйте методы:
 - `add_fan(self, f: WallFan) -> None` — добавляет вентилятор в комнату.
 - `get_area_to_cover(self) -> float` — площадь стен под облицовку: общая площадь стен минус сумма площадей всех вентиляторов (результат ≥ 0).
 - `get_tiles_count(self, tile_width: float, tile_height: float) -> int` — количество плиток. Площадь одной плитки = `tile_width` · `tile_height`. Количество плиток — частное от деления площади под облицовку на площадь плитки, округлённое вверх до целого.
- (c) Создайте три различных комнаты с разным числом вентиляторов и проверьте методы.
- (d) Запросите у пользователя размеры комнаты и размеры плитки (все — дробные числа).
- (e) Выведите площадь под облицовку (м^2) и количество плиток (целое число, округлённое вверх).

33 Расчёт площади стен в зависимости от наличия встроенных увлажнителей

- (a) Создайте два класса: `WallHumidifier` и `Room`. Класс `WallHumidifier` представляет один настенный увлажнитель. Его конструктор принимает: `width` — ширина в метрах, `height` — высота в метрах (положительные дробные числа). Класс `Room` описывает комнату с размерами `width`, `length`, `height` и хранит список объектов `WallHumidifier`.
- (b) В классе `Room` реализуйте методы:
 - `add_humidifier(self, h: WallHumidifier) -> None` — добавляет увлажнитель в комнату.
 - `get_area_to_cover(self) -> float` — площадь стен под оклейку: общая площадь стен минус сумма площадей всех увлажнителей (результат ≥ 0).
 - `get_wallpaper_rolls(self, roll_width: float, roll_length: float) -> int` — количество рулонов обоев. Площадь рулона = `roll_width` · `roll_length`. Количество рулонов — частное от деления площади под оклейку на площадь рулона, округлённое вверх до целого.
- (c) Создайте три разных экземпляра `Room` с разным числом увлажнителей и протестируйте методы.

- (d) Запросите у пользователя размеры комнаты и размеры рулона обоев (все — дробные числа).
- (e) Выведите площадь под оклейку (м^2) и количество рулонов (целое число, округлённое вверх).

34 Расчёт площади стен в зависимости от наличия встроенных обогревателей

- (a) Создайте два класса: `WallHeater` и `Room`. Класс `WallHeater` описывает один настенный обогреватель. Его конструктор принимает: `width` — ширина в метрах, `height` — высота в метрах (положительные дробные числа). Класс `Room` описывает комнату с размерами `width`, `length`, `height` и хранит список объектов `WallHeater`.
- (b) В классе `Room` реализуйте методы:
 - `add_heater(self, h: WallHeater) -> None` — добавляет обогреватель в комнату.
 - `get_area_to_cover(self) -> float` — площадь стен под покраску: общая площадь стен минус сумма площадей всех обогревателей (результат ≥ 0).
 - `get_paint_liters(self, coverage: float) -> float` — объём краски в литрах. Аргумент `coverage` — расход ($\text{м}^2/\text{л}$). Объём = площадь под покраску / `coverage`. Результат может быть дробным.
- (c) Создайте три различных комнаты с разным числом обогревателей и проверьте методы.
- (d) Запросите у пользователя размеры комнаты и `coverage` (дробное число).
- (e) Выведите площадь под покраску (м^2) и количество литров краски (с дробной частью).

35 Расчёт площади стен в зависимости от наличия окон и дверей

- (a) Создайте два класса: `WinDoor` и `Room`. Класс `WinDoor` представляет один проём (окно или дверь). Его конструктор принимает: `width` — ширина проёма в метрах, `height` — высота проёма в метрах (положительные дробные числа). Класс `Room` описывает комнату с размерами `width`, `length`, `height` и хранит список объектов `WinDoor`.
- (b) В классе `Room` реализуйте методы:
 - `add_windoor(self, wd: WinDoor) -> None` — добавляет проём в комнату.
 - `get_area_to_cover(self) -> float` — площадь стен под оклейку: общая площадь стен минус сумма площадей всех проёмов (результат ≥ 0).
 - `get_rolls_count(self, roll_width: float, roll_length: float) -> int` — количество рулонов обоев. Площадь одного рулона = `roll_width` · `roll_length`. Количество рулонов — частное от деления площади под оклейку на площадь рулона, округлённое вверх до целого.
- (c) Создайте три разных экземпляра `Room` с разным числом проёмов и протестируйте методы.
- (d) Запросите у пользователя размеры комнаты и размеры рулона обоев (все — дробные числа).
- (e) Выведите площадь под оклейку (м^2) и количество рулонов обоев (целое число, округлённое вверх).

2.5.3 Задача 3

1. Моделирование поединка между двумя дуэлянтами

- (a) Импортируйте функцию `randint` из модуля `random`.
- (b) Создайте класс `Duelist` («Дуэлянт»). В конструкторе класса должны задаваться имя дуэлянта и его начальное здоровье (по умолчанию — 100 единиц). Также реализуйте следующие методы:
 - `set_name` — позволяет изменить имя дуэлянта;
 - `attack` — моделирует атаку на другого дуэлянта: генерирует случайный урон в диапазоне от 10 до 30 и уменьшает здоровье противника на эту величину.
- (c) Создайте класс `Duel` («Дуэль»). Его конструктор принимает двух дуэлянтов и сохраняет их как внутренние атрибуты. Также в конструкторе инициализируется пустая строка для хранения результата поединка.
- (d) Реализуйте метод `fight`, который моделирует сам поединок:
 - Поединок продолжается, пока у обоих дуэлянтов здоровье больше нуля;
 - На каждом шаге случайным образом (с равной вероятностью) выбирается, кто из дуэлянтов наносит удар;
 - После каждой атаки, если здоровье любого из участников стало меньше или равно нулю, оно устанавливается в ноль.
- (e) После завершения поединка определите его исход:
 - Если у первого дуэлянта осталось здоровье, а у второго — нет, побеждает первый;
 - Если у второго осталось здоровье, а у первого — нет, побеждает второй;
 - Если здоровье обоих участников равно нулю, объявляется ничья.Результат сохраняется в виде понятной строки (например, «Алексей побеждает!» или «Ничья!»).
- (f) Добавьте метод `who_wins`, который выводит на экран сохранённый результат поединка.

2. Моделирование боя между двумя боксёрами

- (a) Импортируйте функцию `randint` из модуля `random`.
- (b) Создайте класс `Boxer` («Боксёр»). В конструкторе задаются имя и начальное здоровье (по умолчанию — 100). Реализуйте методы:
 - `set_name` — изменение имени боксёра;
 - `punch` — нанесение удара противнику с уроном от 10 до 30.
- (c) Создайте класс `BoxingMatch` («Боксёрский поединок»). Его конструктор принимает двух боксёров и инициализирует атрибут для хранения результата.
- (d) Реализуйте метод `match`, моделирующий бой по тем же правилам, что и в первом задании: случайный выбор атакующего, цикл до тех пор, пока у одного из участников не закончится здоровье, коррекция здоровья до нуля при необходимости.
- (e) После завершения боя определите победителя или ничью и сохраните результат в виде строки.
- (f) Добавьте метод `announce_winner`, выводящий результат на экран.

3. Моделирование поединка между двумя шахматистами

- (a) Импортируйте функцию `randint`.
- (b) Создайте класс `ChessPlayer` («Шахматист»). В конструкторе задаются имя и начальное здоровье (по умолчанию — 100). Реализуйте методы:
 - `set_name` — изменение имени;
 - `play_move` — «ход» в рамках метафорического интеллектуального поединка, наносящий урон от 10 до 30.
- (c) Создайте класс `ChessGame` («Шахматная партия»), принимающий двух шахматистов и хранящий результат.
- (d) Реализуйте метод `simulate`, моделирующий поединок: случайный выбор ходящего, цикл до обнуления здоровья одного или обоих участников.
- (e) Определите победителя или ничью по оставшемуся здоровью.
- (f) Добавьте метод `show_result`, выводящий итог партии.

4. Моделирование схватки между двумя борцами

- (a) Импортируйте функцию `randint`.
- (b) Создайте класс `Wrestler` («Борец») с именем и здоровьем (по умолчанию — 100). Методы:
 - `set_name` — изменение имени;
 - `grapple` — захват, наносящий урон от 10 до 30.
- (c) Создайте класс `WrestlingMatch` («Борцовский поединок»), принимающий двух борцов.
- (d) Реализуйте метод `compete`, моделирующий схватку по стандартной схеме: случайный выбор атакующего, цикл до поражения одного или обоих.
- (e) Определите исход схватки и сохраните его в виде строки.
- (f) Добавьте метод `declare_champion`, выводящий победителя.

5. Моделирование битвы между двумя магами

- (a) Импортируйте функцию `randint`.
- (b) Создайте класс `Mage` («Маг») с именем и здоровьем (по умолчанию — 100). Методы:
 - `set_name` — изменение имени;
 - `cast_spell` — заклинание, наносящее урон от 10 до 30.
- (c) Создайте класс `MagicDuel` («Магическая дуэль»), принимающий двух магов.
- (d) Реализуйте метод `duel`, моделирующий битву по стандартной логике.
- (e) Определите победителя или ничью.
- (f) Добавьте метод `reveal_winner`, выводящий результат.

6. Моделирование поединка между двумя киберспортсменами

- (a) Импортируйте функцию `randint`.

- (b) Создайте класс **Gamer** («Киберспортсмен») с именем и здоровьем (по умолчанию — 100). Методы:
 - **set_name** — изменение имени;
 - **make_move** — игровой ход в метафоре «кибер-боя», наносящий урон от 10 до 30.
- (c) Создайте класс **EsportsMatch** («Кибертурнир»), принимающий двух игроков.
- (d) Реализуйте метод **play**, моделирующий поединок.
- (e) Определите победителя или ничью.
- (f) Добавьте метод **show_champion**, выводящий чемпиона.

7. Моделирование соревнования между двумя пловцами

- (a) Импортируйте функцию **randint**.
- (b) Создайте класс **Swimmer** («Пловец») с именем и здоровьем (по умолчанию — 100). Методы:
 - **set_name** — изменение имени;
 - **swim_lap** — заплыв в игровой интерпретации как «атака», наносящая урон от 10 до 30.
- (c) Создайте класс **SwimRace** («Заплыв»), принимающий двух пловцов.
- (d) Реализуйте метод **race**, моделирующий соревнование.
- (e) Определите победителя или ничью.
- (f) Добавьте метод **announce_medal**, выводящий результат.

8. Моделирование боя между двумя роботами

- (a) Импортируйте функцию **randint**.
- (b) Создайте класс **RobotFighter** («Боевой робот») с именем и здоровьем (по умолчанию — 100). Методы:
 - **set_name** — изменение имени;
 - **strike** — удар, наносящий урон от 10 до 30.
- (c) Создайте класс **RobotBattle** («Робобой»), принимающий двух роботов.
- (d) Реализуйте метод **fight**, моделирующий сражение.
- (e) Определите победителя или ничью.
- (f) Добавьте метод **display_result**, выводящий результат.

9. Моделирование дуэли между двумя ковбоями

- (a) Импортируйте функцию **randint**.
- (b) Создайте класс **Cowboy** («Ковбой») с именем и здоровьем (по умолчанию — 100). Методы:
 - **set_name** — изменение имени;
 - **draw** — выстрел, наносящий урон от 10 до 30.
- (c) Создайте класс **Showdown** («Разборка»), принимающий двух ковбоев.

- (d) Реализуйте метод `shootout`, моделирующий перестрелку.
- (e) Определите победителя или ничью.
- (f) Добавьте метод `proclaim_winner`, выводящий результат.

10. Моделирование битвы между двумя ниндзя

- (a) Импортируйте функцию `randint`.
- (b) Создайте класс `Ninja` («Ниндзя») с именем и здоровьем (по умолчанию — 100). Методы:
 - `set_name` — изменение имени;
 - `throw_shuriken` — бросок сюрикена, наносящий урон от 10 до 30.
- (c) Создайте класс `NinjaClash` («Столкновение ниндзя»), принимающий двух бойцов.
- (d) Реализуйте метод `clash`, моделирующий битву.
- (e) Определите победителя или ничью.
- (f) Добавьте метод `declare_victor`, выводящий результат.

11. Моделирование поединка между двумя пиратами

- (a) Импортируйте функцию `randint`.
- (b) Создайте класс `Pirate` («Пират») с именем и здоровьем (по умолчанию — 100). Методы:
 - `set_name` — изменение имени;
 - `sword_fight` — удар мечом, наносящий урон от 10 до 30.
- (c) Создайте класс `PirateDuel` («Пиратская дуэль»), принимающий двух пиратов.
- (d) Реализуйте метод `battle`, моделирующий схватку.
- (e) Определите победителя или ничью.
- (f) Добавьте метод `shout_winner`, выводящий результат.

12. Моделирование схватки между двумя гладиаторами

- (a) Импортируйте функцию `randint`.
- (b) Создайте класс `Gladiator` («Гладиатор») с именем и здоровьем (по умолчанию — 100). Методы:
 - `set_name` — изменение имени;
 - `attack_with_sword` — удар мечом, наносящий урон от 10 до 30.
- (c) Создайте класс `ArenaFight` («Арена»), принимающий двух гладиаторов.
- (d) Реализуйте метод `fight_to_death`, моделирующий бой до конца.
- (e) Определите победителя или ничью.
- (f) Добавьте метод `crowd_cheers`, выводящий результат.

13. Моделирование поединка между двумя самураями

- (a) Импортируйте функцию `randint`.

- (b) Создайте класс `Samurai` («Самурай») с именем и здоровьем (по умолчанию — 100). Методы:
 - `set_name` — изменение имени;
 - `katana_strike` — удар катаной, наносящий урон от 10 до 30.
- (c) Создайте класс `SamuraiDuel` («Самурайская дуэль»), принимающий двух самураев.
- (d) Реализуйте метод `duel`, моделирующий поединок.
- (e) Определите победителя или ничью.
- (f) Добавьте метод `bow_to_winner`, выводящий результат.

14. Моделирование поединка между двумя драконами

- (a) Импортируйте функцию `randint`.
- (b) Создайте класс `Dragon` («Дракон») с именем и здоровьем (по умолчанию — 100). Методы:
 - `set_name` — изменение имени;
 - `breathe_fire` — огненное дыхание, наносящее урон от 10 до 30.
- (c) Создайте класс `DragonBattle` («Битва драконов»), принимающий двух драконов.
- (d) Реализуйте метод `clash`, моделирующий сражение.
- (e) Определите победителя или ничью.
- (f) Добавьте метод `roar_victory`, выводящий результат.

15. Моделирование битвы между двумя титанами

- (a) Импортируйте функцию `randint`.
- (b) Создайте класс `Titan` («Титан») с именем и здоровьем (по умолчанию — 100). Методы:
 - `set_name` — изменение имени;
 - `stomp` — топот, наносящий урон от 10 до 30.
- (c) Создайте класс `TitanClash` («Столкновение титанов»), принимающий двух титанов.
- (d) Реализуйте метод `battle`, моделирующий битву.
- (e) Определите победителя или ничью.
- (f) Добавьте метод `earth_shakes`, выводящий результат.

16. Моделирование поединка между двумя рыцарями

- (a) Импортируйте функцию `randint`.
- (b) Создайте класс `Knight` («Рыцарь») с именем и здоровьем (по умолчанию — 100). Методы:
 - `set_name` — изменение имени;
 - `lance_charge` — рывок с копьём, наносящий урон от 10 до 30.
- (c) Создайте класс `Joust` («Турнир»), принимающий двух рыцарей.

- (d) Реализуйте метод `tournament`, моделирующий поединок.
- (e) Определите победителя или ничью.
- (f) Добавьте метод `king_declares`, выводящий результат.

17. Моделирование поединка между двумя ведьмами

- (a) Импортируйте функцию `randint`.
- (b) Создайте класс `Witch` («Ведьма») с именем и здоровьем (по умолчанию — 100). Методы:
 - `set_name` — изменение имени;
 - `brew_curse` — наложение проклятия, наносящего урон от 10 до 30.
- (c) Создайте класс `WitchDuel` («Ведьмин поединок»), принимающий двух ведьм.
- (d) Реализуйте метод `hex_battle`, моделирующий битву.
- (e) Определите победителя или ничью.
- (f) Добавьте метод `cackle_in_triumph`, выводящий результат.

18. Моделирование боя между двумя зомби

- (a) Импортируйте функцию `randint`.
- (b) Создайте класс `Zombie` («Зомби») с именем и здоровьем (по умолчанию — 100). Методы:
 - `set_name` — изменение имени;
 - `bite` — укус, наносящий урон от 10 до 30.
- (c) Создайте класс `ZombieFight` («Зомби-битва»), принимающий двух зомби.
- (d) Реализуйте метод `apocalypse`, моделирующий сражение.
- (e) Определите победителя или ничью.
- (f) Добавьте метод `groan_winner`, выводящий результат.

19. Моделирование схватки между двумя вампирами

- (a) Импортируйте функцию `randint`.
- (b) Создайте класс `Vampire` («Вампир») с именем и здоровьем (по умолчанию — 100). Методы:
 - `set_name` — изменение имени;
 - `drain` — высасывание жизненных сил, наносящее урон от 10 до 30.
- (c) Создайте класс `VampireDuel` («Вампирская дуэль»), принимающий двух вампиров.
- (d) Реализуйте метод `night_fight`, моделирующий ночную битву.
- (e) Определите победителя или ничью.
- (f) Добавьте метод `howl_at_moon`, выводящий результат.

20. Моделирование битвы между двумя оборотнями

- (a) Импортируйте функцию `randint`.

- (b) Создайте класс `Werewolf` («Оборотень») с именем и здоровьем (по умолчанию — 100). Методы:
 - `set_name` — изменение имени;
 - `claw` — удар когтями, наносящий урон от 10 до 30.
- (c) Создайте класс `MoonBattle` («Лунная битва»), принимающий двух оборотней.
- (d) Реализуйте метод `howl_and_fight`, моделирующий сражение.
- (e) Определите победителя или ничью.
- (f) Добавьте метод `moon_witnesses`, выводящий результат.

21. Моделирование поединка между двумя призраками

- (a) Импортируйте функцию `randint`.
- (b) Создайте класс `Ghost` («Призрак») с именем и здоровьем (по умолчанию — 100). Методы:
 - `set_name` — изменение имени;
 - `terrify` — устрашение, наносящее урон от 10 до 30.
- (c) Создайте класс `HauntedDuel` («Призрачная дуэль»), принимающий двух призраков.
- (d) Реализуйте метод `scare_off`, моделирующий поединок.
- (e) Определите победителя или ничью.
- (f) Добавьте метод `echo_victory`, выводящий результат.

22. Моделирование боя между двумя гоблинами

- (a) Импортируйте функцию `randint`.
- (b) Создайте класс `Goblin` («Гоблин») с именем и здоровьем (по умолчанию — 100). Методы:
 - `set_name` — изменение имени;
 - `stab` — удар кинжалом, наносящий урон от 10 до 30.
- (c) Создайте класс `GoblinSkirmish` («Гоблинская стычка»), принимающий двух гоблинов.
- (d) Реализуйте метод `loot_fight`, моделирующий драку.
- (e) Определите победителя или ничью.
- (f) Добавьте метод `squeal_winner`, выводящий результат.

23. Моделирование схватки между двумя орками

- (a) Импортируйте функцию `randint`.
- (b) Создайте класс `Orc` («Орк») с именем и здоровьем (по умолчанию — 100). Методы:
 - `set_name` — изменение имени;
 - `bash` — мощный удар, наносящий урон от 10 до 30.
- (c) Создайте класс `OrcBattle` («Орковская битва»), принимающий двух орков.

- (d) Реализуйте метод `war_cry`, моделирующий сражение.
- (e) Определите победителя или ничью.
- (f) Добавьте метод `grunt_victory`, выводящий результат.

24. Моделирование битвы между двумя эльфами

- (a) Импортируйте функцию `randint`.
- (b) Создайте класс `Elf` («Эльф») с именем и здоровьем (по умолчанию — 100). Методы:
 - `set_name` — изменение имени;
 - `arrow_shot` — выстрел из лука, наносящий урон от 10 до 30.
- (c) Создайте класс `ElvenDuel` («Эльфийская дуэль»), принимающий двух эльфов.
- (d) Реализуйте метод `forest_clash`, моделирующий поединок.
- (e) Определите победителя или ничью.
- (f) Добавьте метод `whisper_winner`, выводящий результат.

25. Моделирование поединка между двумя гномами

- (a) Импортируйте функцию `randint`.
- (b) Создайте класс `Dwarf` («Гном») с именем и здоровьем (по умолчанию — 100). Методы:
 - `set_name` — изменение имени;
 - `swing_axe` — удар топором, наносящий урон от 10 до 30.
- (c) Создайте класс `DwarfFight` («Гномья драка»), принимающий двух гномов.
- (d) Реализуйте метод `mine_battle`, моделирующий сражение.
- (e) Определите победителя или ничью.
- (f) Добавьте метод `roar_ale`, выводящий результат.

26. Моделирование боя между двумя кентаврами

- (a) Импортируйте функцию `randint`.
- (b) Создайте класс `Centaur` («Кентавр») с именем и здоровьем (по умолчанию — 100). Методы:
 - `set_name` — изменение имени;
 - `gallop_attack` — атака в галопе, наносящая урон от 10 до 30.
- (c) Создайте класс `CentaurClash` («Столкновение кентавров»), принимающий двух кентавров.
- (d) Реализуйте метод `plain_duel`, моделирующий поединок.
- (e) Определите победителя или ничью.
- (f) Добавьте метод `neigh_victory`, выводящий результат.

27. Моделирование схватки между двумя минотаврами

- (a) Импортируйте функцию `randint`.

- (b) Создайте класс `Minotaur` («Минотавр») с именем и здоровьем (по умолчанию — 100). Методы:
 - `set_name` — изменение имени;
 - `gore` — удар рогами, наносящий урон от 10 до 30.
- (c) Создайте класс `LabyrinthFight` («Лабиринтная битва»), принимающий двух минотавров.
- (d) Реализуйте метод `maze_battle`, моделирующий сражение.
- (e) Определите победителя или ничью.
- (f) Добавьте метод `bellow_winner`, выводящий результат.

28. Моделирование битвы между двумя фениксами

- (a) Импортируйте функцию `randint`.
- (b) Создайте класс `Phoenix` («Феникс») с именем и здоровьем (по умолчанию — 100). Методы:
 - `set_name` — изменение имени;
 - `rebirth_strike` — удар, связанный с возрождением, наносящий урон от 10 до 30.
- (c) Создайте класс `PhoenixClash` («Столкновение фениксов»), принимающий двух фениксов.
- (d) Реализуйте метод `ash_duel`, моделирующий поединок.
- (e) Определите победителя или ничью.
- (f) Добавьте метод `soar_victorious`, выводящий результат.

29. Моделирование поединка между двумя единорогами

- (a) Импортируйте функцию `randint`.
- (b) Создайте класс `Unicorn` («Единорог») с именем и здоровьем (по умолчанию — 100). Методы:
 - `set_name` — изменение имени;
 - `horn_charge` — удар рогом, наносящий урон от 10 до 30.
- (c) Создайте класс `UnicornDuel` («Дуэль единорогов»), принимающий двух единорогов.
- (d) Реализуйте метод `meadow_clash`, моделирующий поединок.
- (e) Определите победителя или ничью.
- (f) Добавьте метод `gallop_in_glory`, выводящий результат.

30. Моделирование боя между двумя троллями

- (a) Импортируйте функцию `randint`.
- (b) Создайте класс `Troll` («Троль») с именем и здоровьем (по умолчанию — 100). Методы:
 - `set_name` — изменение имени;
 - `club_smash` — удар дубиной, наносящий урон от 10 до 30.

- (c) Создайте класс `TrollFight` («Тролля драка»), принимающий двух троллей.
- (d) Реализуйте метод `bridge_battle`, моделирующий сражение.
- (e) Определите победителя или ничью.
- (f) Добавьте метод `grunt_and_laugh`, выводящий результат.

31. Моделирование схватки между двумя грифонами

- (a) Импортируйте функцию `randint`.
- (b) Создайте класс `Griffin` («Грифон») с именем и здоровьем (по умолчанию — 100). Методы:
 - `set_name` — изменение имени;
 - `dive_attack` — пикирующая атака, наносящая урон от 10 до 30.
- (c) Создайте класс `GriffinClash` («Столкновение грифонов»), принимающий двух грифонов.
- (d) Реализуйте метод `aerial_duel`, моделирующий воздушный поединок.
- (e) Определите победителя или ничью.
- (f) Добавьте метод `screech_victory`, выводящий результат.

32. Моделирование битвы между двумя драконоборцами

- (a) Импортируйте функцию `randint`.
- (b) Создайте класс `Dragonslayer` («Драконоборец») с именем и здоровьем (по умолчанию — 100). Методы:
 - `set_name` — изменение имени;
 - `slay` — удар, направленный на убийство, наносящий урон от 10 до 30.
- (c) Создайте класс `SlayerDuel` («Дуэль драконоборцев»), принимающий двух героев.
- (d) Реализуйте метод `heroic_fight`, моделирующий битву.
- (e) Определите победителя или ничью.
- (f) Добавьте метод `bard_sings`, выводящий результат.

33. Моделирование поединка между двумя наёмниками

- (a) Импортируйте функцию `randint`.
- (b) Создайте класс `Mercenary` («Наёмник») с именем и здоровьем (по умолчанию — 100). Методы:
 - `set_name` — изменение имени;
 - `strike_for_hire` — удар за плату, наносящий урон от 10 до 30.
- (c) Создайте класс `MercenaryClash` («Стычка наёмников»), принимающий двух бойцов.
- (d) Реализуйте метод `contract_battle`, моделирующий сражение.
- (e) Определите победителя или ничью.
- (f) Добавьте метод `count_coins`, выводящий результат.

34. Моделирование боя между двумя ассасинами

- (a) Импортируйте функцию `randint`.
- (b) Создайте класс `Assassin` («Ассасин») с именем и здоровьем (по умолчанию — 100). Методы:
 - `set_name` — изменение имени;
 - `backstab` — удар в спину, наносящий урон от 10 до 30.
- (c) Создайте класс `ShadowDuel` («Теневая дуэль»), принимающий двух ассасинов.
- (d) Реализуйте метод `night_kill`, моделирующий ночной бой.
- (e) Определите победителя или ничью.
- (f) Добавьте метод `vanish_in_dark`, выводящий результат.

35. Моделирование сражения между двумя солдатами (оригинальный вариант)

- (a) Импортируйте функцию `randint` из модуля `random`.
- (b) Создайте класс `Soldier` («Солдат»). В конструкторе задаются имя и начальное здоровье (по умолчанию — 100). Реализуйте методы:
 - `set_name` — изменение имени;
 - `attack` — атака противника с уроном от 10 до 30.
- (c) Создайте класс `Battle` («Сражение»), принимающий двух солдат и хранящий результат.
- (d) Реализуйте метод `battle`, моделирующий бой:
 - Поединок продолжается, пока у обоих солдат здоровье больше нуля;
 - Атакующий выбирается случайно;
 - После каждой атаки здоровье, упавшее до нуля или ниже, устанавливается в ноль.
- (e) После завершения боя определите исход: победа одного из солдат или ничья — и сохраните результат в виде строки.
- (f) Добавьте метод `who_win`, выводящий результат на экран.

2.6 Семинар «Ограничения доступа и Unit-тестирование» (2 часа)

В ходе работы решите 2 задачи.

Первое задание предполагает просто описание способов доступа к свойствам и методам различными способами.

Второе задание – реализацию простого класса и unit-тестов для него.

2.6.1 Принципы unit-тестирования в Python

Unit-тестирование позволяет проверять отдельные части кода — функции, методы или классы. Основные принципы:

- Каждый тест проверяет **одну конкретную функциональность**.
- Тесты должны покрывать **все важные сценарии использования**, включая крайние и граничные значения.
- Тесты должны быть **повторяемыми и независимыми** друг от друга.
- Используются утверждения: `assertEqual`, `assertTrue`, `assertFalse`, `assertRaises`.

2.6.2 Как анализировать код для тестирования всех случаев

При разработке unit-тестов важно систематически анализировать код и выявлять все ветви и варианты поведения:

1. **Анализ условных операторов (if/else):** Для каждого условия нужно проверить как «истинный» путь, так и «ложный». Пример:

```
def divide(a, b):  
    if b == 0:  
        raise ValueError("Division by zero")  
    return a / b
```

Тесты должны проверять:

- деление на ненулевое число (if=False)
 - деление на ноль (if=True)
2. **Анализ циклов (for/while):** Циклы проверяются на:
 - пустой вход (0 итераций)
 - одну итерацию
 - несколько итераций
 - граничные случаи (максимально допустимое число элементов)
 3. **Граничные значения (boundary values):** Любой метод, работающий с числами или индексами, должен проверяться на:
 - минимальные допустимые значения
 - максимальные допустимые значения

- ноль и отрицательные значения (если применимо)
4. **Исключения и ошибки:** Нужно проверять, что код корректно реагирует на некорректные входные данные, выбрасывая ожидаемые исключения.
 5. **Комбинации входных данных:** Для методов с несколькими параметрами важно проверять сочетания «нормальных» и «краевых» значений.

2.6.3 Пример простого класса с unit-тестами

Рассмотрим класс `Calculator`, который выполняет сложение и деление чисел:

```
# calculator.py
class Calculator:
    def add(self, a, b):
        return a + b

    def divide(self, a, b):
        if b == 0:
            raise ValueError("Division by zero")
        return a / b

# test_calculator.py
import unittest
from calculator import Calculator

class TestCalculator(unittest.TestCase):

    def setUp(self):
        self.calc = Calculator()

    # Проверка всех важных случаев для сложения
    def test_add_positive_numbers(self):
        self.assertEqual(self.calc.add(2, 3), 5)

    def test_add_negative_numbers(self):
        self.assertEqual(self.calc.add(-2, -3), -5)

    def test_add_zero(self):
        self.assertEqual(self.calc.add(0, 5), 5)
        self.assertEqual(self.calc.add(5, 0), 5)

    # Проверка всех важных случаев для деления
    def test_divide_normal(self):
        self.assertEqual(self.calc.divide(10, 2), 5)

    def test_divide_fraction(self):
        self.assertEqual(self.calc.divide(1, 2), 0.5)

    def test_divide_by_zero(self):
        with self.assertRaises(ValueError):
```

```

        self.calc.divide(5, 0)

if __name__ == '__main__':
    unittest.main()

```

Объяснение:

- `setUp()` создает объект перед каждым тестом.
- Каждый метод, имя которого начинается с `test_`, проверяет отдельный сценарий.
- Мы покрыли:
 - положительные и отрицательные числа
 - ноль
 - дробные значения
 - исключения (деление на ноль)
- Для более сложного кода нужно аналогично анализировать все условия и ветвления.

Для сдачи работы будьте готовы пояснить или модифицировать любую часть кода, а также ответить на вопросы:

1. Как можно обеспечить инкапсуляцию в Python (перечислите все варианты)

Если вы нашли в задачнике ошибки, опечатки и другие недостатки, то вы можете сделать pull-request.

2.6.4 Задача 1

- 1 Разработать класс `Bus`, который будет описывать модель автобуса. В классе должны быть следующие поля с доступом уровня **private** (только внутри класса):

- `__speed`: скорость движения автобуса
- `__distance`: расстояние, которое автобус проехал
- `__max_speed`: максимальная разрешённая скорость движения автобуса
- `__passengers`: список пассажиров
- `__capacity`: максимальная вместимость пассажиров в автобусе
- `__empty_seats`: число свободных мест
- `__seats_occupied`: число занятых мест в автобусе
- `__fuel_tank`: объём топливного бака
- `__fuel`: количество топлива в литрах
- `__engine_oil_capacity`: объём картера масла двигателя (литры)
- `__engine_oil`: количество моторного масла в литрах
- `__luggage_spaces`: количество багажных мест
- `__luggage`: багаж автобуса

Уровень доступа к полям должен быть следующим:

- `__max_speed`, `__capacity`, `__fuel_tank`, `__engine_oil_capacity`, `__luggage_spaces`: **только чтение** (через геттеры)
- `__speed`, `__distance`, `__passengers`, `__empty_seats`, `__seats_occupied`, `__fuel`, `__engine_oil`, `__luggage`: **чтение и запись** (через геттеры и сеттеры)

Требования к сеттерам:

- Для полей `__empty_seats` и `__seats_occupied` в сеттерах необходимо проверять, что передаваемое значение не превышает `__capacity` и неотрицательно.
- Для поля `__passengers` в сеттере необходимо проверять, что количество пассажиров (длина списка) не превышает `__capacity`.
- Для поля `__speed` в сеттере необходимо проверять, что заданная скорость не превышает `__max_speed` и неотрицательна.
- Для поля `__luggage` в сеттере необходимо проверять, что количество единиц багажа не превышает `__luggage_spaces`.
- Для полей `__fuel` и `__engine_oil` значения не должны превышать соответствующие ёмкости (`__fuel_tank` и `__engine_oil_capacity`) и должны быть неотрицательными.

Реализовать метод вывода всех установленных через сеттеры значений закрытых полей экземпляра класса. На основе этого класса реализовать три подхода к управлению доступом:

- (a) **С использованием объекта `property`**: Для каждого поля определить отдельные методы-геттеры и сеттеры (например, `get_speed`, `set_speed`), а затем создать свойство:

```
speed = property(get_speed, set_speed)
```

Этот код должен располагаться после определения соответствующих методов. Первый аргумент — геттер, второй — сеттер. Продемонстрировать работу на трёх экземплярах класса: создать `mybus1`, `mybus2`, `mybus3`, установить значения через свойства и вывести их.

- (b) **С использованием декораторов `@property` и `@<имя>.setter`**: Создать новую версию класса, в которой геттеры оформляются с декоратором `@property`, а сеттеры — с декоратором вида `@speed.setter`. Имена методов должны совпадать и не содержать префиксов `get_/set_`. Пример:

```
@property
def speed(self):
    return self.__speed
@speed.setter
def speed(self, value):
    if 0 <= value <= self.__max_speed:
        self.__speed = value
    else:
        raise ValueError("Недопустимая скорость")
```

Продемонстрировать работу на трёх экземплярах и сделать выводы об оптимизации кода по сравнению с первым подходом.

- (с) **С использованием модуля accessify:** Установить модуль командой `pip install accessify` и импортировать:

```
from accessify import private, protected
```

Сделать поля `max_speed`, `capacity`, `fuel_tank`, `engine_oil_capacity`, `luggage_spaces` по-настоящему приватными с помощью функции `private` (например, как атрибуты класса до `__init__`). Удалить их из инициализатора. Проверки в сеттерах реализовать через вспомогательные методы, помеченные декоратором `@private`. Учитывать, что методы с `@private` нельзя вызывать из методов, использующих `@property`, поэтому для этой версии использовать только классические геттеры и сеттеры (`get...`, `set...`). Продемонстрировать, что попытка доступа извне (включая `mybus3._Bus__max_speed`) **не даёт результата**, а вызов приватного метода или чтение приватного поля вызывает ошибку доступа.

Для всех трёх подходов создать по три экземпляра автобуса, установить значения полей с учётом всех ограничений и вывести текущие значения всех полей каждого экземпляра.

- 2 Разработать класс `Train`, который будет описывать модель поезда. В классе должны быть следующие поля с доступом уровня **private** (только внутри класса):

- `__speed`: скорость движения поезда
- `__distance`: расстояние, которое поезд проехал
- `__max_speed`: максимальная разрешённая скорость движения поезда
- `__passengers`: список пассажиров
- `__capacity`: максимальная вместимость пассажиров в поезде
- `__empty_seats`: число свободных мест
- `__seats_occupied`: число занятых мест в поезде
- `__fuel_tank`: объём топливного бака (для дизельных поездов; для электрических — не применимо, но оставлено для единообразия)
- `__fuel`: количество топлива в литрах
- `__engine_oil_capacity`: объём картера масла двигателя (литры)
- `__engine_oil`: количество моторного масла в литрах
- `__luggage_spaces`: количество багажных мест
- `__luggage`: багаж поезда

Уровень доступа к полям должен быть следующим:

- `__max_speed`, `__capacity`, `__fuel_tank`, `__engine_oil_capacity`, `__luggage_spaces`: **только чтение** (через геттеры)
- `__speed`, `__distance`, `__passengers`, `__empty_seats`, `__seats_occupied`, `__fuel`, `__engine_oil`, `__luggage`: **чтение и запись** (через геттеры и сеттеры)

Требования к сеттерам:

- Для полей `__empty_seats` и `__seats_occupied` в сеттерах необходимо проверять, что передаваемое значение не превышает `__capacity` и неотрицательно.
- Для поля `__passengers` в сеттере необходимо проверять, что количество пассажиров (длина списка) не превышает `__capacity`.
- Для поля `__speed` в сеттере необходимо проверять, что заданная скорость не превышает `__max_speed` и неотрицательна.
- Для поля `__luggage` в сеттере необходимо проверять, что количество единиц багажа не превышает `__luggage_spaces`.
- Для полей `__fuel` и `__engine_oil` значения не должны превышать соответствующие ёмкости (`__fuel_tank` и `__engine_oil_capacity`) и должны быть неотрицательными.

Реализовать метод вывода всех установленных через сеттеры значений закрытых полей экземпляра класса. На основе этого класса реализовать три подхода к управлению доступом:

- (a) **С использованием объекта `property`:** Для каждого поля определить отдельные методы-геттеры и сеттеры (например, `get_speed`, `set_speed`), а затем создать свойство:

```
speed = property(get_speed, set_speed)
```

Этот код должен располагаться после определения соответствующих методов. Первый аргумент — геттер, второй — сеттер. Продемонстрировать работу на трёх экземплярах класса: создать `mytrain1`, `mytrain2`, `mytrain3`, установить значения через свойства и вывести их.

- (b) **С использованием декораторов `@property` и `@<имя>.setter`:** Создать новую версию класса, в которой геттеры оформляются с декоратором `@property`, а сеттеры — с декоратором вида `@speed.setter`. Имена методов должны совпадать и не содержать префиксов `get_/set_`. Пример:

```
@property
def speed(self):
    return self.__speed
@speed.setter
def speed(self, value):
    if 0 <= value <= self.__max_speed:
        self.__speed = value
    else:
        raise ValueError("Недопустимая скорость")
```

Продемонстрировать работу на трёх экземплярах и сделать выводы об оптимизации кода по сравнению с первым подходом.

- (c) **С использованием модуля `accessify`:** Установить модуль командой `pip install accessify` и импортировать:

```
from accessify import private, protected
```

Сделать поля `max_speed`, `capacity`, `fuel_tank`, `engine_oil_capacity`, `luggage_spaces` по-настоящему приватными с помощью функции `private` (например, как атрибуты класса до `__init__`). Удалить их из инициализатора. Проверки в сеттерах реализовать через вспомогательные методы, помеченные декоратором `@private`. Учитывать, что методы с `@private` нельзя вызывать из методов, использующих `@property`, поэтому для этой версии использовать только классические геттеры и сеттеры (`get_...`, `set_...`). Продемонстрировать, что попытка доступа извне (включая `mytrain3.Train__max_speed`) **не даёт результата**, а вызов приватного метода или чтение приватного поля вызывает ошибку доступа.

Для всех трёх подходов создать по три экземпляра поезда, установить значения полей с учётом всех ограничений и вывести текущие значения всех полей каждого экземпляра.

3 Разработать класс `Airplane`, который будет описывать модель самолёта. В классе должны быть следующие поля с доступом уровня **private** (только внутри класса):

- `__speed`: скорость движения самолёта
- `__distance`: расстояние, которое самолёт пролетел
- `__max_speed`: максимальная разрешённая скорость движения самолёта
- `__passengers`: список пассажиров
- `__capacity`: максимальная вместимость пассажиров в самолёте
- `__empty_seats`: число свободных мест
- `__seats_occupied`: число занятых мест в самолёте
- `__fuel_tank`: объём топливного бака
- `__fuel`: количество топлива в литрах
- `__engine_oil_capacity`: объём картера масла двигателя (литры)
- `__engine_oil`: количество моторного масла в литрах
- `__luggage_spaces`: количество багажных мест
- `__luggage`: багаж самолёта

Уровень доступа к полям должен быть следующим:

- `__max_speed`, `__capacity`, `__fuel_tank`, `__engine_oil_capacity`, `__luggage_spaces`: **только чтение** (через геттеры)
- `__speed`, `__distance`, `__passengers`, `__empty_seats`, `__seats_occupied`, `__fuel`, `__engine_oil`, `__luggage`: **чтение и запись** (через геттеры и сеттеры)

Требования к сеттерам:

- Для полей `__empty_seats` и `__seats_occupied` в сеттерах необходимо проверять, что передаваемое значение не превышает `__capacity` и неотрицательно.
- Для поля `__passengers` в сеттере необходимо проверять, что количество пассажиров (длина списка) не превышает `__capacity`.
- Для поля `__speed` в сеттере необходимо проверять, что заданная скорость не превышает `__max_speed` и неотрицательна.

- Для поля `__luggage` в сеттере необходимо проверять, что количество единиц багажа не превышает `__luggage_spaces`.
- Для полей `__fuel` и `__engine_oil` значения не должны превышать соответствующие ёмкости (`__fuel_tank` и `__engine_oil_capacity`) и должны быть неотрицательными.

Реализовать метод вывода всех установленных через сеттеры значений закрытых полей экземпляра класса. На основе этого класса реализовать три подхода к управлению доступом:

- (a) **С использованием объекта `property`:** Для каждого поля определить отдельные методы-геттеры и сеттеры (например, `get_speed`, `set_speed`), а затем создать свойство:

```
speed = property(get_speed, set_speed)
```

Этот код должен располагаться после определения соответствующих методов. Первый аргумент — геттер, второй — сеттер. Продемонстрировать работу на трёх экземплярах класса: создать `myplane1`, `myplane2`, `myplane3`, установить значения через свойства и вывести их.

- (b) **С использованием декораторов `@property` и `@<имя>.setter`:** Создать новую версию класса, в которой геттеры оформляются с декоратором `@property`, а сеттеры — с декоратором вида `@speed.setter`. Имена методов должны совпадать и не содержать префиксов `get_/set_`. Пример:

```
@property
def speed(self):
    return self.__speed
@speed.setter
def speed(self, value):
    if 0 <= value <= self.__max_speed:
        self.__speed = value
    else:
        raise ValueError("Недопустимая скорость")
```

Продемонстрировать работу на трёх экземплярах и сделать выводы об оптимизации кода по сравнению с первым подходом.

- (c) **С использованием модуля `accessify`:** Установить модуль командой `pip install accessify` и импортировать:

```
from accessify import private, protected
```

Сделать поля `max_speed`, `capacity`, `fuel_tank`, `engine_oil_capacity`, `luggage_spaces` по-настоящему приватными с помощью функции `private` (например, как атрибуты класса до `__init__`). Удалить их из инициализатора. Проверки в сеттерах реализовать через вспомогательные методы, помеченные декоратором `@private`. Учитывать, что методы с `@private` нельзя вызывать из методов, использующих `@property`, поэтому для этой версии использовать только классические геттеры и сеттеры (`get_...`, `set_...`). Продемонстрировать, что попытка доступа извне

(включая `myplane3._Airplane__max_speed`) **не даёт результата**, а вызов приватного метода или чтение приватного поля вызывает ошибку доступа.

Для всех трёх подходов создать по три экземпляра самолёта, установить значения полей с учётом всех ограничений и вывести текущие значения всех полей каждого экземпляра.

4 Разработать класс **Ship**, который будет описывать модель корабля. В классе должны быть следующие поля с доступом уровня **private** (только внутри класса):

- `__speed`: скорость движения корабля
- `__distance`: расстояние, которое корабль прошёл
- `__max_speed`: максимальная разрешённая скорость движения корабля
- `__passengers`: список пассажиров
- `__capacity`: максимальная вместимость пассажиров на корабле
- `__empty_seats`: число свободных мест
- `__seats_occupied`: число занятых мест на корабле
- `__fuel_tank`: объём топливного бака
- `__fuel`: количество топлива в литрах
- `__engine_oil_capacity`: объём картера масла двигателя (литры)
- `__engine_oil`: количество моторного масла в литрах
- `__luggage_spaces`: количество багажных мест
- `__luggage`: багаж корабля

Уровень доступа к полям должен быть следующим:

- `__max_speed`, `__capacity`, `__fuel_tank`, `__engine_oil_capacity`, `__luggage_spaces`: **только чтение** (через геттеры)
- `__speed`, `__distance`, `__passengers`, `__empty_seats`, `__seats_occupied`, `__fuel`, `__engine_oil`, `__luggage`: **чтение и запись** (через геттеры и сеттеры)

Требования к сеттерам:

- Для полей `__empty_seats` и `__seats_occupied` в сеттерах необходимо проверять, что передаваемое значение не превышает `__capacity` и неотрицательно.
- Для поля `__passengers` в сеттере необходимо проверять, что количество пассажиров (длина списка) не превышает `__capacity`.
- Для поля `__speed` в сеттере необходимо проверять, что заданная скорость не превышает `__max_speed` и неотрицательна.
- Для поля `__luggage` в сеттере необходимо проверять, что количество единиц багажа не превышает `__luggage_spaces`.
- Для полей `__fuel` и `__engine_oil` значения не должны превышать соответствующие ёмкости (`__fuel_tank` и `__engine_oil_capacity`) и должны быть неотрицательными.

Реализовать метод вывода всех установленных через сеттеры значений закрытых полей экземпляра класса. На основе этого класса реализовать три подхода к управлению доступом:

- (a) **С использованием объекта `property`:** Для каждого поля определить отдельные методы-геттеры и сеттеры (например, `get_speed`, `set_speed`), а затем создать свойство:

```
speed = property(get_speed, set_speed)
```

Этот код должен располагаться после определения соответствующих методов. Первый аргумент — геттер, второй — сеттер. Продемонстрировать работу на трёх экземплярах класса: создать `myship1`, `myship2`, `myship3`, установить значения через свойства и вывести их.

- (b) **С использованием декораторов `@property` и `@<имя>.setter`:** Создать новую версию класса, в которой геттеры оформляются с декоратором `@property`, а сеттеры — с декоратором вида `@speed.setter`. Имена методов должны совпадать и не содержать префиксов `get_/set_`. Пример:

```
@property
def speed(self):
    return self.__speed
@speed.setter
def speed(self, value):
    if 0 <= value <= self.__max_speed:
        self.__speed = value
    else:
        raise ValueError("Недопустимая скорость")
```

Продемонстрировать работу на трёх экземплярах и сделать выводы об оптимизации кода по сравнению с первым подходом.

- (c) **С использованием модуля `accessify`:** Установить модуль командой `pip install accessify` и импортировать:

```
from accessify import private, protected
```

Сделать поля `max_speed`, `capacity`, `fuel_tank`, `engine_oil_capacity`, `luggage_spaces` по-настоящему приватными с помощью функции `private` (например, как атрибуты класса до `__init__`). Удалить их из инициализатора. Проверки в сеттерах реализовать через вспомогательные методы, помеченные декоратором `@private`. Учитывать, что методы с `@private` нельзя вызывать из методов, использующих `@property`, поэтому для этой версии использовать только классические геттеры и сеттеры (`get_...`, `set_...`). Продемонстрировать, что попытка доступа извне (включая `myship3._Ship__max_speed`) **не даёт результата**, а вызов приватного метода или чтение приватного поля вызывает ошибку доступа.

Для всех трёх подходов создать по три экземпляра корабля, установить значения полей с учётом всех ограничений и вывести текущие значения всех полей каждого экземпляра.

- 5 Разработать класс `Truck`, который будет описывать модель грузовика. В классе должны быть следующие поля с доступом уровня **private** (только внутри класса):
- `__speed`: скорость движения грузовика

- `__distance`: расстояние, которое грузовик проехал
- `__max_speed`: максимальная разрешённая скорость движения грузовика
- `__passengers`: список пассажиров
- `__capacity`: максимальная вместимость пассажиров в грузовике
- `__empty_seats`: число свободных мест
- `__seats_occupied`: число занятых мест в грузовике
- `__fuel_tank`: объём топливного бака
- `__fuel`: количество топлива в литрах
- `__engine_oil_capacity`: объём картера масла двигателя (литры)
- `__engine_oil`: количество моторного масла в литрах
- `__luggage_spaces`: количество багажных мест
- `__luggage`: багаж грузовика

Уровень доступа к полям должен быть следующим:

- `__max_speed`, `__capacity`, `__fuel_tank`, `__engine_oil_capacity`, `__luggage_spaces`: **только чтение** (через геттеры)
- `__speed`, `__distance`, `__passengers`, `__empty_seats`, `__seats_occupied`, `__fuel`, `__engine_oil`, `__luggage`: **чтение и запись** (через геттеры и сеттеры)

Требования к сеттерам:

- Для полей `__empty_seats` и `__seats_occupied` в сеттерах необходимо проверять, что передаваемое значение не превышает `__capacity` и неотрицательно.
- Для поля `__passengers` в сеттере необходимо проверять, что количество пассажиров (длина списка) не превышает `__capacity`.
- Для поля `__speed` в сеттере необходимо проверять, что заданная скорость не превышает `__max_speed` и неотрицательна.
- Для поля `__luggage` в сеттере необходимо проверять, что количество единиц багажа не превышает `__luggage_spaces`.
- Для полей `__fuel` и `__engine_oil` значения не должны превышать соответствующие ёмкости (`__fuel_tank` и `__engine_oil_capacity`) и должны быть неотрицательными.

Реализовать метод вывода всех установленных через сеттеры значений закрытых полей экземпляра класса. На основе этого класса реализовать три подхода к управлению доступом:

- (а) **С использованием объекта `property`**: Для каждого поля определить отдельные методы-геттеры и сеттеры (например, `get_speed`, `set_speed`), а затем создать свойство:

```
speed = property(get_speed, set_speed)
```

Этот код должен располагаться после определения соответствующих методов. Первый аргумент — геттер, второй — сеттер. Продемонстрировать работу на трёх экземплярах класса: создать `mytruck1`, `mytruck2`, `mytruck3`, установить значения через свойства и вывести их.

- (b) **С использованием декораторов `@property` и `@<имя>.setter`:** Создать новую версию класса, в которой геттеры оформляются с декоратором `@property`, а сеттеры — с декоратором вида `@speed.setter`. Имена методов должны совпадать и не содержать префиксов `get_/set_`. Пример:

```
@property
def speed(self):
    return self.__speed
@speed.setter
def speed(self, value):
    if 0 <= value <= self.__max_speed:
        self.__speed = value
    else:
        raise ValueError("Недопустимая скорость")
```

Продемонстрировать работу на трёх экземплярах и сделать выводы об оптимизации кода по сравнению с первым подходом.

- (c) **С использованием модуля `accessify`:** Установить модуль командой `pip install accessify` и импортировать:

```
from accessify import private, protected
```

Сделать поля `max_speed`, `capacity`, `fuel_tank`, `engine_oil_capacity`, `luggage_spaces` по-настоящему приватными с помощью функции `private` (например, как атрибуты класса до `__init__`). Удалить их из инициализатора. Проверки в сеттерах реализовать через вспомогательные методы, помеченные декоратором `@private`. Учитывать, что методы с `@private` нельзя вызывать из методов, использующих `@property`, поэтому для этой версии использовать только классические геттеры и сеттеры (`get_...`, `set_...`). Продемонстрировать, что попытка доступа извне (включая `mytruck3.Truck__max_speed`) **не даёт результата**, а вызов приватного метода или чтение приватного поля вызывает ошибку доступа.

Для всех трёх подходов создать по три экземпляра грузовика, установить значения полей с учётом всех ограничений и вывести текущие значения всех полей каждого экземпляра.

- 6 Разработать класс `Motorcycle`, который будет описывать модель мотоцикла. В классе должны быть следующие поля с доступом уровня **private** (только внутри класса):

- `__speed`: скорость движения мотоцикла
- `__distance`: расстояние, которое мотоцикл проехал
- `__max_speed`: максимальная разрешённая скорость движения мотоцикла
- `__passengers`: список пассажиров
- `__capacity`: максимальная вместимость пассажиров на мотоцикле (обычно 1–2)
- `__empty_seats`: число свободных мест
- `__seats_occupied`: число занятых мест на мотоцикле
- `__fuel_tank`: объём топливного бака
- `__fuel`: количество топлива в литрах

- `__engine_oil_capacity`: объём картера масла двигателя (литры)
- `__engine_oil`: количество моторного масла в литрах
- `__luggage_spaces`: количество багажных мест (обычно сумки/кофры)
- `__luggage`: багаж мотоцикла

Уровень доступа к полям должен быть следующим:

- `__max_speed`, `__capacity`, `__fuel_tank`, `__engine_oil_capacity`, `__luggage_spaces`: **только чтение** (через геттеры)
- `__speed`, `__distance`, `__passengers`, `__empty_seats`, `__seats_occupied`, `__fuel`, `__engine_oil`, `__luggage`: **чтение и запись** (через геттеры и сеттеры)

Требования к сеттерам:

- Для полей `__empty_seats` и `__seats_occupied` в сеттерах необходимо проверять, что передаваемое значение не превышает `__capacity` и неотрицательно.
- Для поля `__passengers` в сеттере необходимо проверять, что количество пассажиров (длина списка) не превышает `__capacity`.
- Для поля `__speed` в сеттере необходимо проверять, что заданная скорость не превышает `__max_speed` и неотрицательна.
- Для поля `__luggage` в сеттере необходимо проверять, что количество единиц багажа не превышает `__luggage_spaces`.
- Для полей `__fuel` и `__engine_oil` значения не должны превышать соответствующие ёмкости (`__fuel_tank` и `__engine_oil_capacity`) и должны быть неотрицательными.

Реализовать метод вывода всех установленных через сеттеры значений закрытых полей экземпляра класса. На основе этого класса реализовать три подхода к управлению доступом:

- (a) **С использованием объекта `property`**: Для каждого поля определить отдельные методы-геттеры и сеттеры (например, `get_speed`, `set_speed`), а затем создать свойство:

```
speed = property(get_speed, set_speed)
```

Этот код должен располагаться после определения соответствующих методов. Первый аргумент — геттер, второй — сеттер. Продемонстрировать работу на трёх экземплярах класса: создать `mymoto1`, `mymoto2`, `mymoto3`, установить значения через свойства и вывести их.

- (b) **С использованием декораторов `@property` и `@<имя>.setter`**: Создать новую версию класса, в которой геттеры оформляются с декоратором `@property`, а сеттеры — с декоратором вида `@speed.setter`. Имена методов должны совпадать и не содержать префиксов `get_/set_`. Пример:

```
@property
def speed(self):
    return self.__speed
@speed.setter
```

```
def speed(self, value):
    if 0 <= value <= self.__max_speed:
        self.__speed = value
    else:
        raise ValueError("Недопустимая скорость")
```

Продемонстрировать работу на трёх экземплярах и сделать выводы об оптимизации кода по сравнению с первым подходом.

- (с) **С использованием модуля accessify:** Установить модуль командой `pip install accessify` и импортировать:

```
from accessify import private, protected
```

Сделать поля `max_speed`, `capacity`, `fuel_tank`, `engine_oil_capacity`, `luggage_spaces` по-настоящему приватными с помощью функции `private` (например, как атрибуты класса до `__init__`). Удалить их из инициализатора. Проверки в сеттерах реализовать через вспомогательные методы, помеченные декоратором `@private`. Учитывать, что методы с `@private` нельзя вызывать из методов, использующих `@property`, поэтому для этой версии использовать только классические геттеры и сеттеры (`get_...`, `set_...`). Продемонстрировать, что попытка доступа извне (включая `mymoto3._Motorcycle__max_speed`) **не даёт результата**, а вызов приватного метода или чтение приватного поля вызывает ошибку доступа.

Для всех трёх подходов создать по три экземпляра мотоцикла, установить значения полей с учётом всех ограничений и вывести текущие значения всех полей каждого экземпляра.

- 7 Разработать класс `Bicycle`, который будет описывать модель велосипеда. В классе должны быть следующие поля с доступом уровня **private** (только внутри класса):

- `__speed`: скорость движения велосипеда
- `__distance`: расстояние, которое велосипед проехал
- `__max_speed`: максимальная разрешённая скорость движения велосипеда
- `__passengers`: список пассажиров
- `__capacity`: максимальная вместимость пассажиров на велосипеде (обычно 1, иногда 2)
- `__empty_seats`: число свободных мест
- `__seats_occupied`: число занятых мест на велосипеде
- `__fuel_tank`: объём топливного бака (не применимо к обычному велосипеду; оставлено для единообразия)
- `__fuel`: количество топлива в литрах (обычно 0 для обычного велосипеда)
- `__engine_oil_capacity`: объём картера масла двигателя (не применимо; оставлено для единообразия)
- `__engine_oil`: количество моторного масла в литрах (обычно 0)
- `__luggage_spaces`: количество багажных мест
- `__luggage`: багаж велосипеда

Уровень доступа к полям должен быть следующим:

- `__max_speed`, `__capacity`, `__fuel_tank`, `__engine_oil_capacity`, `__luggage_spaces`: **только чтение** (через геттеры)
- `__speed`, `__distance`, `__passengers`, `__empty_seats`, `__seats_occupied`, `__fuel`, `__engine_oil`, `__luggage`: **чтение и запись** (через геттеры и сеттеры)

Требования к сеттерам:

- Для полей `__empty_seats` и `__seats_occupied` в сеттерах необходимо проверять, что передаваемое значение не превышает `__capacity` и неотрицательно.
- Для поля `__passengers` в сеттере необходимо проверять, что количество пассажиров (длина списка) не превышает `__capacity`.
- Для поля `__speed` в сеттере необходимо проверять, что заданная скорость не превышает `__max_speed` и неотрицательна.
- Для поля `__luggage` в сеттере необходимо проверять, что количество единиц багажа не превышает `__luggage_spaces`.
- Для полей `__fuel` и `__engine_oil` значения не должны превышать соответствующие ёмкости (`__fuel_tank` и `__engine_oil_capacity`) и должны быть неотрицательными (обычно 0).

Реализовать метод вывода всех установленных через сеттеры значений закрытых полей экземпляра класса. На основе этого класса реализовать три подхода к управлению доступом:

- (a) **С использованием объекта `property`**: Для каждого поля определить отдельные методы-геттеры и сеттеры (например, `get_speed`, `set_speed`), а затем создать свойство:

```
speed = property(get_speed, set_speed)
```

Этот код должен располагаться после определения соответствующих методов. Первый аргумент — геттер, второй — сеттер. Продемонстрировать работу на трёх экземплярах класса: создать `mybike1`, `mybike2`, `mybike3`, установить значения через свойства и вывести их.

- (b) **С использованием декораторов `@property` и `@<имя>.setter`**: Создать новую версию класса, в которой геттеры оформляются с декоратором `@property`, а сеттеры — с декоратором вида `@speed.setter`. Имена методов должны совпадать и не содержать префиксов `get_/set_`. Пример:

```
@property
def speed(self):
    return self.__speed
@speed.setter
def speed(self, value):
    if 0 <= value <= self.__max_speed:
        self.__speed = value
    else:
        raise ValueError("Недопустимая скорость")
```


Продемонстрировать работу на трёх экземплярах и сделать выводы об оптимизации кода по сравнению с первым подходом.

- (с) **С использованием модуля `accessify`:** Установить модуль командой `pip install accessify` и импортировать:

```
from accessify import private, protected
```

Сделать поля `max_speed`, `capacity`, `fuel_tank`, `engine_oil_capacity`, `luggage_spaces` по-настоящему приватными с помощью функции `private` (например, как атрибуты класса до `__init__`). Удалить их из инициализатора. Проверки в сеттерах реализовать через вспомогательные методы, помеченные декоратором `@private`. Учитывать, что методы с `@private` нельзя вызывать из методов, использующих `@property`, поэтому для этой версии использовать только классические геттеры и сеттеры (`get_...`, `set_...`). Продемонстрировать, что попытка доступа извне (включая `mybike3._Bicycle__max_speed`) **не даёт результата**, а вызов приватного метода или чтение приватного поля вызывает ошибку доступа.

Для всех трёх подходов создать по три экземпляра велосипеда, установить значения полей с учётом всех ограничений и вывести текущие значения всех полей каждого экземпляра.

- 8 Разработать класс `Helicopter`, который будет описывать модель вертолёт. В классе должны быть следующие поля с доступом уровня **private** (только внутри класса):

- `__speed`: скорость движения вертолёт
- `__distance`: расстояние, которое вертолёт пролетел
- `__max_speed`: максимальная разрешённая скорость движения вертолёт
- `__passengers`: список пассажиров
- `__capacity`: максимальная вместимость пассажиров в вертолёт
- `__empty_seats`: число свободных мест
- `__seats_occupied`: число занятых мест в вертолёт
- `__fuel_tank`: объём топливного бака
- `__fuel`: количество топлива в литрах
- `__engine_oil_capacity`: объём картера масла двигателя (литры)
- `__engine_oil`: количество моторного масла в литрах
- `__luggage_spaces`: количество багажных мест
- `__luggage`: багаж вертолёт

Уровень доступа к полям должен быть следующим:

- `__max_speed`, `__capacity`, `__fuel_tank`, `__engine_oil_capacity`, `__luggage_spaces`: **только чтение** (через геттеры)
- `__speed`, `__distance`, `__passengers`, `__empty_seats`, `__seats_occupied`, `__fuel`, `__engine_oil`, `__luggage`: **чтение и запись** (через геттеры и сеттеры)

Требования к сеттерам:

- Для полей `__empty_seats` и `__seats_occupied` в сеттерах необходимо проверять, что передаваемое значение не превышает `__capacity` и неотрицательно.
- Для поля `__passengers` в сеттере необходимо проверять, что количество пассажиров (длина списка) не превышает `__capacity`.
- Для поля `__speed` в сеттере необходимо проверять, что заданная скорость не превышает `__max_speed` и неотрицательна.
- Для поля `__luggage` в сеттере необходимо проверять, что количество единиц багажа не превышает `__luggage_spaces`.
- Для полей `__fuel` и `__engine_oil` значения не должны превышать соответствующие ёмкости (`__fuel_tank` и `__engine_oil_capacity`) и должны быть неотрицательными.

Реализовать метод вывода всех установленных через сеттеры значений закрытых полей экземпляра класса. На основе этого класса реализовать три подхода к управлению доступом:

- (a) **С использованием объекта `property`:** Для каждого поля определить отдельные методы-геттеры и сеттеры (например, `get_speed`, `set_speed`), а затем создать свойство:

```
speed = property(get_speed, set_speed)
```

Этот код должен располагаться после определения соответствующих методов. Первый аргумент — геттер, второй — сеттер. Продемонстрировать работу на трёх экземплярах класса: создать `myheli1`, `myheli2`, `myheli3`, установить значения через свойства и вывести их.

- (b) **С использованием декораторов `@property` и `@<имя>.setter`:** Создать новую версию класса, в которой геттеры оформляются с декоратором `@property`, а сеттеры — с декоратором вида `@speed.setter`. Имена методов должны совпадать и не содержать префиксов `get_/set_`. Пример:

```
@property
def speed(self):
    return self.__speed
@speed.setter
def speed(self, value):
    if 0 <= value <= self.__max_speed:
        self.__speed = value
    else:
        raise ValueError("Недопустимая скорость")
```

Продемонстрировать работу на трёх экземплярах и сделать выводы об оптимизации кода по сравнению с первым подходом.

- (c) **С использованием модуля `accessify`:** Установить модуль командой `pip install accessify` и импортировать:

```
from accessify import private, protected
```

Сделать поля `max_speed`, `capacity`, `fuel_tank`, `engine_oil_capacity`, `luggage_spaces` по-настоящему приватными с помощью функции `private` (например, как атрибуты класса до `__init__`). Удалить их из инициализатора. Проверки в сеттерах реализовать через вспомогательные методы, помеченные декоратором `@private`. Учитывать, что методы с `@private` нельзя вызывать из методов, использующих `@property`, поэтому для этой версии использовать только классические геттеры и сеттеры (`get_...`, `set_...`). Продемонстрировать, что попытка доступа извне (включая `myheli3._Helicopter__max_speed`) **не даёт результата**, а вызов приватного метода или чтение приватного поля вызывает ошибку доступа.

Для всех трёх подходов создать по три экземпляра вертолёта, установить значения полей с учётом всех ограничений и вывести текущие значения всех полей каждого экземпляра.

9 Разработать класс `Submarine`, который будет описывать модель подводной лодки. В классе должны быть следующие поля с доступом уровня **private** (только внутри класса):

- `__speed`: скорость движения подводной лодки
- `__distance`: расстояние, которое подводная лодка прошла
- `__max_speed`: максимальная разрешённая скорость движения подводной лодки
- `__passengers`: список пассажиров (обычно экипаж и, возможно, пассажиры)
- `__capacity`: максимальная вместимость пассажиров в подводной лодке
- `__empty_seats`: число свободных мест
- `__seats_occupied`: число занятых мест в подводной лодке
- `__fuel_tank`: объём топливного бака (для дизель-электрических; для атомных — не применимо, но оставлено для единообразия)
- `__fuel`: количество топлива в литрах
- `__engine_oil_capacity`: объём картера масла двигателя (литры)
- `__engine_oil`: количество моторного масла в литрах
- `__luggage_spaces`: количество багажных мест
- `__luggage`: багаж подводной лодки

Уровень доступа к полям должен быть следующим:

- `__max_speed`, `__capacity`, `__fuel_tank`, `__engine_oil_capacity`, `__luggage_spaces`: **только чтение** (через геттеры)
- `__speed`, `__distance`, `__passengers`, `__empty_seats`, `__seats_occupied`, `__fuel`, `__engine_oil`, `__luggage`: **чтение и запись** (через геттеры и сеттеры)

Требования к сеттерам:

- Для полей `__empty_seats` и `__seats_occupied` в сеттерах необходимо проверять, что передаваемое значение не превышает `__capacity` и неотрицательно.
- Для поля `__passengers` в сеттере необходимо проверять, что количество пассажиров (длина списка) не превышает `__capacity`.

- Для поля `__speed` в сеттере необходимо проверять, что заданная скорость не превышает `__max_speed` и неотрицательна.
- Для поля `__luggage` в сеттере необходимо проверять, что количество единиц багажа не превышает `__luggage_spaces`.
- Для полей `__fuel` и `__engine_oil` значения не должны превышать соответствующие ёмкости (`__fuel_tank` и `__engine_oil_capacity`) и должны быть неотрицательными.

Реализовать метод вывода всех установленных через сеттеры значений закрытых полей экземпляра класса. На основе этого класса реализовать три подхода к управлению доступом:

- (a) **С использованием объекта `property`:** Для каждого поля определить отдельные методы-геттеры и сеттеры (например, `get_speed`, `set_speed`), а затем создать свойство:

```
speed = property(get_speed, set_speed)
```

Этот код должен располагаться после определения соответствующих методов. Первый аргумент — геттер, второй — сеттер. Продемонстрировать работу на трёх экземплярах класса: создать `mysub1`, `mysub2`, `mysub3`, установить значения через свойства и вывести их.

- (b) **С использованием декораторов `@property` и `@<имя>.setter`:** Создать новую версию класса, в которой геттеры оформляются с декоратором `@property`, а сеттеры — с декоратором вида `@speed.setter`. Имена методов должны совпадать и не содержать префиксов `get_`/`set_`. Пример:

```
@property
def speed(self):
    return self.__speed
@speed.setter
def speed(self, value):
    if 0 <= value <= self.__max_speed:
        self.__speed = value
    else:
        raise ValueError("Недопустимая скорость")
```

Продемонстрировать работу на трёх экземплярах и сделать выводы об оптимизации кода по сравнению с первым подходом.

- (c) **С использованием модуля `accessify`:** Установить модуль командой `pip install accessify` и импортировать:

```
from accessify import private, protected
```

Сделать поля `max_speed`, `capacity`, `fuel_tank`, `engine_oil_capacity`, `luggage_spaces` по-настоящему приватными с помощью функции `private` (например, как атрибуты класса до `__init__`). Удалить их из инициализатора. Проверки в сеттерах реализовать через вспомогательные методы, помеченные декоратором `@private`. Учитывать, что методы с `@private` нельзя вызывать из методов, использующих

`@property`, поэтому для этой версии использовать только классические геттеры и сеттеры (`get_...`, `set_...`). Продемонстрировать, что попытка доступа извне (включая `mysub3._Submarine__max_speed`) **не даёт результата**, а вызов приватного метода или чтение приватного поля вызывает ошибку доступа.

Для всех трёх подходов создать по три экземпляра подводной лодки, установить значения полей с учётом всех ограничений и вывести текущие значения всех полей каждого экземпляра.

- 10 Разработать класс `Spaceship`, который будет описывать модель космического корабля. В классе должны быть следующие поля с доступом уровня **private** (только внутри класса):

- `__speed`: скорость движения космического корабля
- `__distance`: расстояние, которое космический корабль пролетел
- `__max_speed`: максимальная разрешённая скорость движения космического корабля
- `__passengers`: список пассажиров
- `__capacity`: максимальная вместимость пассажиров в космическом корабле
- `__empty_seats`: число свободных мест
- `__seats_occupied`: число занятых мест в космическом корабле
- `__fuel_tank`: объём топливного бака (для ракетного топлива)
- `__fuel`: количество топлива в литрах (или в условных единицах)
- `__engine_oil_capacity`: объём картера масла двигателя (не применимо к большинству космических двигателей; оставлено для единообразия)
- `__engine_oil`: количество моторного масла в литрах (обычно 0)
- `__luggage_spaces`: количество багажных мест
- `__luggage`: багаж космического корабля

Уровень доступа к полям должен быть следующим:

- `__max_speed`, `__capacity`, `__fuel_tank`, `__engine_oil_capacity`, `__luggage_spaces`: **только чтение** (через геттеры)
- `__speed`, `__distance`, `__passengers`, `__empty_seats`, `__seats_occupied`, `__fuel`, `__engine_oil`, `__luggage`: **чтение и запись** (через геттеры и сеттеры)

Требования к сеттерам:

- Для полей `__empty_seats` и `__seats_occupied` в сеттерах необходимо проверять, что передаваемое значение не превышает `__capacity` и неотрицательно.
- Для поля `__passengers` в сеттере необходимо проверять, что количество пассажиров (длина списка) не превышает `__capacity`.
- Для поля `__speed` в сеттере необходимо проверять, что заданная скорость не превышает `__max_speed` и неотрицательна.
- Для поля `__luggage` в сеттере необходимо проверять, что количество единиц багажа не превышает `__luggage_spaces`.

- Для полей `__fuel` и `__engine_oil` значения не должны превышать соответствующие ёмкости (`__fuel_tank` и `__engine_oil_capacity`) и должны быть неотрицательными.

Реализовать метод вывода всех установленных через сеттеры значений закрытых полей экземпляра класса. На основе этого класса реализовать три подхода к управлению доступом:

- (a) **С использованием объекта `property`**: Для каждого поля определить отдельные методы-геттеры и сеттеры (например, `get_speed`, `set_speed`), а затем создать свойство:

```
speed = property(get_speed, set_speed)
```

Этот код должен располагаться после определения соответствующих методов. Первый аргумент — геттер, второй — сеттер. Продемонстрировать работу на трёх экземплярах класса: создать `myspace1`, `myspace2`, `myspace3`, установить значения через свойства и вывести их.

- (b) **С использованием декораторов `@property` и `@<имя>.setter`**: Создать новую версию класса, в которой геттеры оформляются с декоратором `@property`, а сеттеры — с декоратором вида `@speed.setter`. Имена методов должны совпадать и не содержать префиксов `get_/set_`. Пример:

```
@property
def speed(self):
    return self.__speed
@speed.setter
def speed(self, value):
    if 0 <= value <= self.__max_speed:
        self.__speed = value
    else:
        raise ValueError("Недопустимая скорость")
```

Продемонстрировать работу на трёх экземплярах и сделать выводы об оптимизации кода по сравнению с первым подходом.

- (c) **С использованием модуля `accessify`**: Установить модуль командой `pip install accessify` и импортировать:

```
from accessify import private, protected
```

Сделать поля `max_speed`, `capacity`, `fuel_tank`, `engine_oil_capacity`, `luggage_spaces` по-настоящему приватными с помощью функции `private` (например, как атрибуты класса до `__init__`). Удалить их из инициализатора. Проверки в сеттерах реализовать через вспомогательные методы, помеченные декоратором `@private`. Учитывать, что методы с `@private` нельзя вызывать из методов, использующих `@property`, поэтому для этой версии использовать только классические геттеры и сеттеры (`get_...`, `set_...`). Продемонстрировать, что попытка доступа извне (включая `myspace3._Spaceship__max_speed`) **не даёт результата**, а вызов приватного метода или чтение приватного поля вызывает ошибку доступа.

Для всех трёх подходов создать по три экземпляра космического корабля, установить значения полей с учётом всех ограничений и вывести текущие значения всех полей каждого экземпляра.

- 11 Разработать класс **Drone**, который будет описывать модель дрона. В классе должны быть следующие поля с доступом уровня **private** (только внутри класса):

- **__speed**: скорость движения дрона
- **__distance**: расстояние, которое дрон пролетел
- **__max_speed**: максимальная разрешённая скорость движения дрона
- **__passengers**: список пассажиров (обычно пустой; оставлено для единообразия)
- **__capacity**: максимальная вместимость пассажиров на дроне (обычно 0)
- **__empty_seats**: число свободных мест (обычно 0)
- **__seats_occupied**: число занятых мест на дроне (обычно 0)
- **__fuel_tank**: объём топливного бака (для топливных дронов; для электрических — не применимо)
- **__fuel**: количество топлива в литрах
- **__engine_oil_capacity**: объём картера масла двигателя (обычно 0)
- **__engine_oil**: количество моторного масла в литрах (обычно 0)
- **__luggage_spaces**: количество багажных мест (для грузовых дронов)
- **__luggage**: багаж дрона

Уровень доступа к полям должен быть следующим:

- **__max_speed**, **__capacity**, **__fuel_tank**, **__engine_oil_capacity**, **__luggage_spaces**: **только чтение** (через геттеры)
- **__speed**, **__distance**, **__passengers**, **__empty_seats**, **__seats_occupied**, **__fuel**, **__engine_oil**, **__luggage**: **чтение и запись** (через геттеры и сеттеры)

Требования к сеттерам:

- Для полей **__empty_seats** и **__seats_occupied** в сеттерах необходимо проверять, что передаваемое значение не превышает **__capacity** и неотрицательно.
- Для поля **__passengers** в сеттере необходимо проверять, что количество пассажиров (длина списка) не превышает **__capacity**.
- Для поля **__speed** в сеттере необходимо проверять, что заданная скорость не превышает **__max_speed** и неотрицательна.
- Для поля **__luggage** в сеттере необходимо проверять, что количество единиц багажа не превышает **__luggage_spaces**.
- Для полей **__fuel** и **__engine_oil** значения не должны превышать соответствующие ёмкости (**__fuel_tank** и **__engine_oil_capacity**) и должны быть неотрицательными.

Реализовать метод вывода всех установленных через сеттеры значений закрытых полей экземпляра класса. На основе этого класса реализовать три подхода к управлению доступом:

- (a) **С использованием объекта `property`:** Для каждого поля определить отдельные методы-геттеры и сеттеры (например, `get_speed`, `set_speed`), а затем создать свойство:

```
speed = property(get_speed, set_speed)
```

Этот код должен располагаться после определения соответствующих методов. Первый аргумент — геттер, второй — сеттер. Продемонстрировать работу на трёх экземплярах класса: создать `mydrone1`, `mydrone2`, `mydrone3`, установить значения через свойства и вывести их.

- (b) **С использованием декораторов `@property` и `@<имя>.setter`:** Создать новую версию класса, в которой геттеры оформляются с декоратором `@property`, а сеттеры — с декоратором вида `@speed.setter`. Имена методов должны совпадать и не содержать префиксов `get_`/`set_`. Пример:

```
@property
def speed(self):
    return self.__speed
@speed.setter
def speed(self, value):
    if 0 <= value <= self.__max_speed:
        self.__speed = value
    else:
        raise ValueError("Недопустимая скорость")
```

Продемонстрировать работу на трёх экземплярах и сделать выводы об оптимизации кода по сравнению с первым подходом.

- (c) **С использованием модуля `accessify`:** Установить модуль командой `pip install accessify` и импортировать:

```
from accessify import private, protected
```

Сделать поля `max_speed`, `capacity`, `fuel_tank`, `engine_oil_capacity`, `luggage_spaces` по-настоящему приватными с помощью функции `private` (например, как атрибуты класса до `__init__`). Удалить их из инициализатора. Проверки в сеттерах реализовать через вспомогательные методы, помеченные декоратором `@private`. Учитывать, что методы с `@private` нельзя вызывать из методов, использующих `@property`, поэтому для этой версии использовать только классические геттеры и сеттеры (`get_...`, `set_...`). Продемонстрировать, что попытка доступа извне (включая `mydrone3._Drone__max_speed`) **не даёт результата**, а вызов приватного метода или чтение приватного поля вызывает ошибку доступа.

Для всех трёх подходов создать по три экземпляра дрона, установить значения полей с учётом всех ограничений и вывести текущие значения всех полей каждого экземпляра.

- 12 Разработать класс `Scooter`, который будет описывать модель скутера. В классе должны быть следующие поля с доступом уровня **private** (только внутри класса):

- `__speed`: скорость движения скутера
- `__distance`: расстояние, которое скутер проехал

- `__max_speed`: максимальная разрешённая скорость движения скутера
- `__passengers`: список пассажиров
- `__capacity`: максимальная вместимость пассажиров на скутере (обычно 1–2)
- `__empty_seats`: число свободных мест
- `__seats_occupied`: число занятых мест на скутере
- `__fuel_tank`: объём топливного бака
- `__fuel`: количество топлива в литрах
- `__engine_oil_capacity`: объём картера масла двигателя (литры)
- `__engine_oil`: количество моторного масла в литрах
- `__luggage_spaces`: количество багажных мест
- `__luggage`: багаж скутера

Уровень доступа к полям должен быть следующим:

- `__max_speed`, `__capacity`, `__fuel_tank`, `__engine_oil_capacity`, `__luggage_spaces`: **только чтение** (через геттеры)
- `__speed`, `__distance`, `__passengers`, `__empty_seats`, `__seats_occupied`, `__fuel`, `__engine_oil`, `__luggage`: **чтение и запись** (через геттеры и сеттеры)

Требования к сеттерам:

- Для полей `__empty_seats` и `__seats_occupied` в сеттерах необходимо проверять, что передаваемое значение не превышает `__capacity` и неотрицательно.
- Для поля `__passengers` в сеттере необходимо проверять, что количество пассажиров (длина списка) не превышает `__capacity`.
- Для поля `__speed` в сеттере необходимо проверять, что заданная скорость не превышает `__max_speed` и неотрицательна.
- Для поля `__luggage` в сеттере необходимо проверять, что количество единиц багажа не превышает `__luggage_spaces`.
- Для полей `__fuel` и `__engine_oil` значения не должны превышать соответствующие ёмкости (`__fuel_tank` и `__engine_oil_capacity`) и должны быть неотрицательными.

Реализовать метод вывода всех установленных через сеттеры значений закрытых полей экземпляра класса. На основе этого класса реализовать три подхода к управлению доступом:

- (а) **С использованием объекта `property`**: Для каждого поля определить отдельные методы-геттеры и сеттеры (например, `get_speed`, `set_speed`), а затем создать свойство:

```
speed = property(get_speed, set_speed)
```

Этот код должен располагаться после определения соответствующих методов. Первый аргумент — геттер, второй — сеттер. Продемонстрировать работу на трёх экземплярах класса: создать `myscoot1`, `myscoot2`, `myscoot3`, установить значения через свойства и вывести их.

- (b) **С использованием декораторов `@property` и `@<имя>.setter`:** Создать новую версию класса, в которой геттеры оформляются с декоратором `@property`, а сеттеры — с декоратором вида `@speed.setter`. Имена методов должны совпадать и не содержать префиксов `get_/set_`. Пример:

```
@property
def speed(self):
    return self.__speed
@speed.setter
def speed(self, value):
    if 0 <= value <= self.__max_speed:
        self.__speed = value
    else:
        raise ValueError("Недопустимая скорость")
```

Продемонстрировать работу на трёх экземплярах и сделать выводы об оптимизации кода по сравнению с первым подходом.

- (c) **С использованием модуля `accessify`:** Установить модуль командой `pip install accessify` и импортировать:

```
from accessify import private, protected
```

Сделать поля `max_speed`, `capacity`, `fuel_tank`, `engine_oil_capacity`, `luggage_spaces` по-настоящему приватными с помощью функции `private` (например, как атрибуты класса до `__init__`). Удалить их из инициализатора. Проверки в сеттерах реализовать через вспомогательные методы, помеченные декоратором `@private`. Учитывать, что методы с `@private` нельзя вызывать из методов, использующих `@property`, поэтому для этой версии использовать только классические геттеры и сеттеры (`get_...`, `set_...`). Продемонстрировать, что попытка доступа извне (включая `myscoot3.Scooter__max_speed`) **не даёт результата**, а вызов приватного метода или чтение приватного поля вызывает ошибку доступа.

Для всех трёх подходов создать по три экземпляра скутера, установить значения полей с учётом всех ограничений и вывести текущие значения всех полей каждого экземпляра.

- 13 Разработать класс `Taxi`, который будет описывать модель такси. В классе должны быть следующие поля с доступом уровня **private** (только внутри класса):

- `__speed`: скорость движения такси
- `__distance`: расстояние, которое такси проехало
- `__max_speed`: максимальная разрешённая скорость движения такси
- `__passengers`: список пассажиров
- `__capacity`: максимальная вместимость пассажиров в такси
- `__empty_seats`: число свободных мест
- `__seats_occupied`: число занятых мест в такси
- `__fuel_tank`: объём топливного бака
- `__fuel`: количество топлива в литрах

- `__engine_oil_capacity`: объём картера масла двигателя (литры)
- `__engine_oil`: количество моторного масла в литрах
- `__luggage_spaces`: количество багажных мест
- `__luggage`: багаж такси

Уровень доступа к полям должен быть следующим:

- `__max_speed`, `__capacity`, `__fuel_tank`, `__engine_oil_capacity`, `__luggage_spaces`: **только чтение** (через геттеры)
- `__speed`, `__distance`, `__passengers`, `__empty_seats`, `__seats_occupied`, `__fuel`, `__engine_oil`, `__luggage`: **чтение и запись** (через геттеры и сеттеры)

Требования к сеттерам:

- Для полей `__empty_seats` и `__seats_occupied` в сеттерах необходимо проверять, что передаваемое значение не превышает `__capacity` и неотрицательно.
- Для поля `__passengers` в сеттере необходимо проверять, что количество пассажиров (длина списка) не превышает `__capacity`.
- Для поля `__speed` в сеттере необходимо проверять, что заданная скорость не превышает `__max_speed` и неотрицательна.
- Для поля `__luggage` в сеттере необходимо проверять, что количество единиц багажа не превышает `__luggage_spaces`.
- Для полей `__fuel` и `__engine_oil` значения не должны превышать соответствующие ёмкости (`__fuel_tank` и `__engine_oil_capacity`) и должны быть неотрицательными.

Реализовать метод вывода всех установленных через сеттеры значений закрытых полей экземпляра класса. На основе этого класса реализовать три подхода к управлению доступом:

- (a) **С использованием объекта `property`**: Для каждого поля определить отдельные методы-геттеры и сеттеры (например, `get_speed`, `set_speed`), а затем создать свойство:

```
speed = property(get_speed, set_speed)
```

Этот код должен располагаться после определения соответствующих методов. Первый аргумент — геттер, второй — сеттер. Продемонстрировать работу на трёх экземплярах класса: создать `mytaxi1`, `mytaxi2`, `mytaxi3`, установить значения через свойства и вывести их.

- (b) **С использованием декораторов `@property` и `@<имя>.setter`**: Создать новую версию класса, в которой геттеры оформляются с декоратором `@property`, а сеттеры — с декоратором вида `@speed.setter`. Имена методов должны совпадать и не содержать префиксов `get_/set_`. Пример:

```
@property
def speed(self):
    return self.__speed
@speed.setter
```

```
def speed(self, value):
    if 0 <= value <= self.__max_speed:
        self.__speed = value
    else:
        raise ValueError("Недопустимая скорость")
```

Продемонстрировать работу на трёх экземплярах и сделать выводы об оптимизации кода по сравнению с первым подходом.

- (с) **С использованием модуля accessify:** Установить модуль командой `pip install accessify` и импортировать:

```
from accessify import private, protected
```

Сделать поля `max_speed`, `capacity`, `fuel_tank`, `engine_oil_capacity`, `luggage_spaces` по-настоящему приватными с помощью функции `private` (например, как атрибуты класса до `__init__`). Удалить их из инициализатора. Проверки в сеттерах реализовать через вспомогательные методы, помеченные декоратором `@private`. Учитывать, что методы с `@private` нельзя вызывать из методов, использующих `@property`, поэтому для этой версии использовать только классические геттеры и сеттеры (`get...`, `set...`). Продемонстрировать, что попытка доступа извне (включая `mytaxi3._Taxi__max_speed`) **не даёт результата**, а вызов приватного метода или чтение приватного поля вызывает ошибку доступа.

Для всех трёх подходов создать по три экземпляра такси, установить значения полей с учётом всех ограничений и вывести текущие значения всех полей каждого экземпляра.

- 14 Разработать класс `Ambulance`, который будет описывать модель скорой помощи. В классе должны быть следующие поля с доступом уровня **private** (только внутри класса):

- `__speed`: скорость движения скорой помощи
- `__distance`: расстояние, которое скорая помощь проехала
- `__max_speed`: максимальная разрешённая скорость движения скорой помощи
- `__passengers`: список пассажиров
- `__capacity`: максимальная вместимость пассажиров в скорой помощи
- `__empty_seats`: число свободных мест
- `__seats_occupied`: число занятых мест в скорой помощи
- `__fuel_tank`: объём топливного бака
- `__fuel`: количество топлива в литрах
- `__engine_oil_capacity`: объём картера масла двигателя (литры)
- `__engine_oil`: количество моторного масла в литрах
- `__luggage_spaces`: количество багажных мест
- `__luggage`: багаж скорой помощи

Уровень доступа к полям должен быть следующим:

- `__max_speed`, `__capacity`, `__fuel_tank`, `__engine_oil_capacity`, `__luggage_spaces`: **только чтение** (через геттеры)
- `__speed`, `__distance`, `__passengers`, `__empty_seats`, `__seats_occupied`, `__fuel`, `__engine_oil`, `__luggage`: **чтение и запись** (через геттеры и сеттеры)

Требования к сеттерам:

- Для полей `__empty_seats` и `__seats_occupied` в сеттерах необходимо проверять, что передаваемое значение не превышает `__capacity` и неотрицательно.
- Для поля `__passengers` в сеттере необходимо проверять, что количество пассажиров (длина списка) не превышает `__capacity`.
- Для поля `__speed` в сеттере необходимо проверять, что заданная скорость не превышает `__max_speed` и неотрицательна.
- Для поля `__luggage` в сеттере необходимо проверять, что количество единиц багажа не превышает `__luggage_spaces`.
- Для полей `__fuel` и `__engine_oil` значения не должны превышать соответствующие ёмкости (`__fuel_tank` и `__engine_oil_capacity`) и должны быть неотрицательными.

Реализовать метод вывода всех установленных через сеттеры значений закрытых полей экземпляра класса. На основе этого класса реализовать три подхода к управлению доступом:

- (a) **С использованием объекта `property`**: Для каждого поля определить отдельные методы-геттеры и сеттеры (например, `get_speed`, `set_speed`), а затем создать свойство:

```
speed = property(get_speed, set_speed)
```

Этот код должен располагаться после определения соответствующих методов. Первый аргумент — геттер, второй — сеттер. Продемонстрировать работу на трёх экземплярах класса: создать `myamb1`, `myamb2`, `myamb3`, установить значения через свойства и вывести их.

- (b) **С использованием декораторов `@property` и `@<имя>.setter`**: Создать новую версию класса, в которой геттеры оформляются с декоратором `@property`, а сеттеры — с декоратором вида `@speed.setter`. Имена методов должны совпадать и не содержать префиксов `get_/set_`. Пример:

```
@property
def speed(self):
    return self.__speed
@speed.setter
def speed(self, value):
    if 0 <= value <= self.__max_speed:
        self.__speed = value
    else:
        raise ValueError("Недопустимая скорость")
```

Продемонстрировать работу на трёх экземплярах и сделать выводы об оптимизации кода по сравнению с первым подходом.

- (с) **С использованием модуля accessify:** Установить модуль командой `pip install accessify` и импортировать:

```
from accessify import private, protected
```

Сделать поля `max_speed`, `capacity`, `fuel_tank`, `engine_oil_capacity`, `luggage_spaces` по-настоящему приватными с помощью функции `private` (например, как атрибуты класса до `__init__`). Удалить их из инициализатора. Проверки в сеттерах реализовать через вспомогательные методы, помеченные декоратором `@private`. Учитывать, что методы с `@private` нельзя вызывать из методов, использующих `@property`, поэтому для этой версии использовать только классические геттеры и сеттеры (`get_...`, `set_...`). Продемонстрировать, что попытка доступа извне (включая `myamb3.Ambulance.__max_speed`) **не даёт результата**, а вызов приватного метода или чтение приватного поля вызывает ошибку доступа.

Для всех трёх подходов создать по три экземпляра скорой помощи, установить значения полей с учётом всех ограничений и вывести текущие значения всех полей каждого экземпляра.

- 15 Разработать класс `FireTruck`, который будет описывать модель пожарной машины. В классе должны быть следующие поля с доступом уровня **private** (только внутри класса):

- `__speed`: скорость движения пожарной машины
- `__distance`: расстояние, которое пожарная машина проехала
- `__max_speed`: максимальная разрешённая скорость движения пожарной машины
- `__passengers`: список пассажиров
- `__capacity`: максимальная вместимость пассажиров в пожарной машине
- `__empty_seats`: число свободных мест
- `__seats_occupied`: число занятых мест в пожарной машине
- `__fuel_tank`: объём топливного бака
- `__fuel`: количество топлива в литрах
- `__engine_oil_capacity`: объём картера масла двигателя (литры)
- `__engine_oil`: количество моторного масла в литрах
- `__luggage_spaces`: количество багажных мест
- `__luggage`: багаж пожарной машины

Уровень доступа к полям должен быть следующим:

- `__max_speed`, `__capacity`, `__fuel_tank`, `__engine_oil_capacity`, `__luggage_spaces`: **только чтение** (через геттеры)
- `__speed`, `__distance`, `__passengers`, `__empty_seats`, `__seats_occupied`, `__fuel`, `__engine_oil`, `__luggage`: **чтение и запись** (через геттеры и сеттеры)

Требования к сеттерам:

- Для полей `__empty_seats` и `__seats_occupied` в сеттерах необходимо проверять, что передаваемое значение не превышает `__capacity` и неотрицательно.
- Для поля `__passengers` в сеттере необходимо проверять, что количество пассажиров (длина списка) не превышает `__capacity`.
- Для поля `__speed` в сеттере необходимо проверять, что заданная скорость не превышает `__max_speed` и неотрицательна.
- Для поля `__luggage` в сеттере необходимо проверять, что количество единиц багажа не превышает `__luggage_spaces`.
- Для полей `__fuel` и `__engine_oil` значения не должны превышать соответствующие ёмкости (`__fuel_tank` и `__engine_oil_capacity`) и должны быть неотрицательными.

Реализовать метод вывода всех установленных через сеттеры значений закрытых полей экземпляра класса. На основе этого класса реализовать три подхода к управлению доступом:

- (a) **С использованием объекта `property`:** Для каждого поля определить отдельные методы-геттеры и сеттеры (например, `get_speed`, `set_speed`), а затем создать свойство:

```
speed = property(get_speed, set_speed)
```

Этот код должен располагаться после определения соответствующих методов. Первый аргумент — геттер, второй — сеттер. Продемонстрировать работу на трёх экземплярах класса: создать `myfire1`, `myfire2`, `myfire3`, установить значения через свойства и вывести их.

- (b) **С использованием декораторов `@property` и `@<имя>.setter`:** Создать новую версию класса, в которой геттеры оформляются с декоратором `@property`, а сеттеры — с декоратором вида `@speed.setter`. Имена методов должны совпадать и не содержать префиксов `get_/set_`. Пример:

```
@property
def speed(self):
    return self.__speed
@speed.setter
def speed(self, value):
    if 0 <= value <= self.__max_speed:
        self.__speed = value
    else:
        raise ValueError("Недопустимая скорость")
```

Продемонстрировать работу на трёх экземплярах и сделать выводы об оптимизации кода по сравнению с первым подходом.

- (c) **С использованием модуля `accessify`:** Установить модуль командой `pip install accessify` и импортировать:

```
from accessify import private, protected
```

Сделать поля `max_speed`, `capacity`, `fuel_tank`, `engine_oil_capacity`, `luggage_spaces` по-настоящему приватными с помощью функции `private` (например, как атрибуты класса до `__init__`). Удалить их из инициализатора. Проверки в сеттерах реализовать через вспомогательные методы, помеченные декоратором `@private`. Учитывать, что методы с `@private` нельзя вызывать из методов, использующих `@property`, поэтому для этой версии использовать только классические геттеры и сеттеры (`get_...`, `set_...`). Продемонстрировать, что попытка доступа извне (включая `myfire3.FireTruck__max_speed`) **не даёт результата**, а вызов приватного метода или чтение приватного поля вызывает ошибку доступа.

Для всех трёх подходов создать по три экземпляра пожарной машины, установить значения полей с учётом всех ограничений и вывести текущие значения всех полей каждого экземпляра.

- 16 Разработать класс `PoliceCar`, который будет описывать модель полицейского автомобиля. В классе должны быть следующие поля с доступом уровня **private** (только внутри класса):

- `__speed`: скорость движения полицейского автомобиля
- `__distance`: расстояние, которое полицейский автомобиль проехал
- `__max_speed`: максимальная разрешённая скорость движения полицейского автомобиля
- `__passengers`: список пассажиров
- `__capacity`: максимальная вместимость пассажиров в полицейском автомобиле
- `__empty_seats`: число свободных мест
- `__seats_occupied`: число занятых мест в полицейском автомобиле
- `__fuel_tank`: объём топливного бака
- `__fuel`: количество топлива в литрах
- `__engine_oil_capacity`: объём картера масла двигателя (литры)
- `__engine_oil`: количество моторного масла в литрах
- `__luggage_spaces`: количество багажных мест
- `__luggage`: багаж полицейского автомобиля

Уровень доступа к полям должен быть следующим:

- `__max_speed`, `__capacity`, `__fuel_tank`, `__engine_oil_capacity`, `__luggage_spaces`: **только чтение** (через геттеры)
- `__speed`, `__distance`, `__passengers`, `__empty_seats`, `__seats_occupied`, `__fuel`, `__engine_oil`, `__luggage`: **чтение и запись** (через геттеры и сеттеры)

Требования к сеттерам:

- Для полей `__empty_seats` и `__seats_occupied` в сеттерах необходимо проверять, что передаваемое значение не превышает `__capacity` и неотрицательно.
- Для поля `__passengers` в сеттере необходимо проверять, что количество пассажиров (длина списка) не превышает `__capacity`.

- Для поля `__speed` в сеттере необходимо проверять, что заданная скорость не превышает `__max_speed` и неотрицательна.
- Для поля `__luggage` в сеттере необходимо проверять, что количество единиц багажа не превышает `__luggage_spaces`.
- Для полей `__fuel` и `__engine_oil` значения не должны превышать соответствующие ёмкости (`__fuel_tank` и `__engine_oil_capacity`) и должны быть неотрицательными.

Реализовать метод вывода всех установленных через сеттеры значений закрытых полей экземпляра класса. На основе этого класса реализовать три подхода к управлению доступом:

- (a) **С использованием объекта `property`:** Для каждого поля определить отдельные методы-геттеры и сеттеры (например, `get_speed`, `set_speed`), а затем создать свойство:

```
speed = property(get_speed, set_speed)
```

Этот код должен располагаться после определения соответствующих методов. Первый аргумент — геттер, второй — сеттер. Продемонстрировать работу на трёх экземплярах класса: создать `mypolice1`, `mypolice2`, `mypolice3`, установить значения через свойства и вывести их.

- (b) **С использованием декораторов `@property` и `@<имя>.setter`:** Создать новую версию класса, в которой геттеры оформляются с декоратором `@property`, а сеттеры — с декоратором вида `@speed.setter`. Имена методов должны совпадать и не содержать префиксов `get_/set_`. Пример:

```
@property
def speed(self):
    return self.__speed
@speed.setter
def speed(self, value):
    if 0 <= value <= self.__max_speed:
        self.__speed = value
    else:
        raise ValueError("Недопустимая скорость")
```

Продемонстрировать работу на трёх экземплярах и сделать выводы об оптимизации кода по сравнению с первым подходом.

- (c) **С использованием модуля `accessify`:** Установить модуль командой `pip install accessify` и импортировать:

```
from accessify import private, protected
```

Сделать поля `max_speed`, `capacity`, `fuel_tank`, `engine_oil_capacity`, `luggage_spaces` по-настоящему приватными с помощью функции `private` (например, как атрибуты класса до `__init__`). Удалить их из инициализатора. Проверки в сеттерах реализовать через вспомогательные методы, помеченные декоратором `@private`. Учитывать, что методы с `@private` нельзя вызывать из методов, использующих

`@property`, поэтому для этой версии использовать только классические геттеры и сеттеры (`get_...`, `set_...`). Продемонстрировать, что попытка доступа извне (включая `mypolice3._PoliceCar__max_speed`) **не даёт результата**, а вызов приватного метода или чтение приватного поля вызывает ошибку доступа.

Для всех трёх подходов создать по три экземпляра полицейского автомобиля, установить значения полей с учётом всех ограничений и вывести текущие значения всех полей каждого экземпляра.

17 Разработать класс `Crane`, который будет описывать модель подъёмного крана. В классе должны быть следующие поля с доступом уровня **private** (только внутри класса):

- `__speed`: скорость движения крана
- `__distance`: расстояние, которое кран проехал
- `__max_speed`: максимальная разрешённая скорость движения крана
- `__passengers`: список пассажиров
- `__capacity`: максимальная вместимость пассажиров в кране
- `__empty_seats`: число свободных мест
- `__seats_occupied`: число занятых мест в кране
- `__fuel_tank`: объём топливного бака
- `__fuel`: количество топлива в литрах
- `__engine_oil_capacity`: объём картера масла двигателя (литры)
- `__engine_oil`: количество моторного масла в литрах
- `__luggage_spaces`: количество багажных мест
- `__luggage`: багаж крана

Уровень доступа к полям должен быть следующим:

- `__max_speed`, `__capacity`, `__fuel_tank`, `__engine_oil_capacity`, `__luggage_spaces`: **только чтение** (через геттеры)
- `__speed`, `__distance`, `__passengers`, `__empty_seats`, `__seats_occupied`, `__fuel`, `__engine_oil`, `__luggage`: **чтение и запись** (через геттеры и сеттеры)

Требования к сеттерам:

- Для полей `__empty_seats` и `__seats_occupied` в сеттерах необходимо проверять, что передаваемое значение не превышает `__capacity` и неотрицательно.
- Для поля `__passengers` в сеттере необходимо проверять, что количество пассажиров (длина списка) не превышает `__capacity`.
- Для поля `__speed` в сеттере необходимо проверять, что заданная скорость не превышает `__max_speed` и неотрицательна.
- Для поля `__luggage` в сеттере необходимо проверять, что количество единиц багажа не превышает `__luggage_spaces`.
- Для полей `__fuel` и `__engine_oil` значения не должны превышать соответствующие ёмкости (`__fuel_tank` и `__engine_oil_capacity`) и должны быть неотрицательными.

Реализовать метод вывода всех установленных через сеттеры значений закрытых полей экземпляра класса. На основе этого класса реализовать три подхода к управлению доступом:

- (a) **С использованием объекта `property`:** Для каждого поля определить отдельные методы-геттеры и сеттеры (например, `get_speed`, `set_speed`), а затем создать свойство:

```
speed = property(get_speed, set_speed)
```

Этот код должен располагаться после определения соответствующих методов. Первый аргумент — геттер, второй — сеттер. Продемонстрировать работу на трёх экземплярах класса: создать `mycrane1`, `mycrane2`, `mycrane3`, установить значения через свойства и вывести их.

- (b) **С использованием декораторов `@property` и `@<имя>.setter`:** Создать новую версию класса, в которой геттеры оформляются с декоратором `@property`, а сеттеры — с декоратором вида `@speed.setter`. Имена методов должны совпадать и не содержать префиксов `get_`/`set_`. Пример:

```
@property
def speed(self):
    return self.__speed
@speed.setter
def speed(self, value):
    if 0 <= value <= self.__max_speed:
        self.__speed = value
    else:
        raise ValueError("Недопустимая скорость")
```

Продемонстрировать работу на трёх экземплярах и сделать выводы об оптимизации кода по сравнению с первым подходом.

- (c) **С использованием модуля `accessify`:** Установить модуль командой `pip install accessify` и импортировать:

```
from accessify import private, protected
```

Сделать поля `max_speed`, `capacity`, `fuel_tank`, `engine_oil_capacity`, `luggage_spaces` по-настоящему приватными с помощью функции `private` (например, как атрибуты класса до `__init__`). Удалить их из инициализатора. Проверки в сеттерах реализовать через вспомогательные методы, помеченные декоратором `@private`. Учитывать, что методы с `@private` нельзя вызывать из методов, использующих `@property`, поэтому для этой версии использовать только классические геттеры и сеттеры (`get_...`, `set_...`). Продемонстрировать, что попытка доступа извне (включая `mycrane3._Crane__max_speed`) **не даёт результата**, а вызов приватного метода или чтение приватного поля вызывает ошибку доступа.

Для всех трёх подходов создать по три экземпляра подъёмного крана, установить значения полей с учётом всех ограничений и вывести текущие значения всех полей каждого экземпляра.

18 Разработать класс `Excavator`, который будет описывать модель экскаватора. В классе должны быть следующие поля с доступом уровня **private** (только внутри класса):

- `__speed`: скорость движения экскаватора
- `__distance`: расстояние, которое экскаватор проехал
- `__max_speed`: максимальная разрешённая скорость движения экскаватора
- `__passengers`: список пассажиров
- `__capacity`: максимальная вместимость пассажиров в экскаваторе
- `__empty_seats`: число свободных мест
- `__seats_occupied`: число занятых мест в экскаваторе
- `__fuel_tank`: объём топливного бака
- `__fuel`: количество топлива в литрах
- `__engine_oil_capacity`: объём картера масла двигателя (литры)
- `__engine_oil`: количество моторного масла в литрах
- `__luggage_spaces`: количество багажных мест
- `__luggage`: багаж экскаватора

Уровень доступа к полям должен быть следующим:

- `__max_speed`, `__capacity`, `__fuel_tank`, `__engine_oil_capacity`, `__luggage_spaces`: **только чтение** (через геттеры)
- `__speed`, `__distance`, `__passengers`, `__empty_seats`, `__seats_occupied`, `__fuel`, `__engine_oil`, `__luggage`: **чтение и запись** (через геттеры и сеттеры)

Требования к сеттерам:

- Для полей `__empty_seats` и `__seats_occupied` в сеттерах необходимо проверять, что передаваемое значение не превышает `__capacity` и неотрицательно.
- Для поля `__passengers` в сеттере необходимо проверять, что количество пассажиров (длина списка) не превышает `__capacity`.
- Для поля `__speed` в сеттере необходимо проверять, что заданная скорость не превышает `__max_speed` и неотрицательна.
- Для поля `__luggage` в сеттере необходимо проверять, что количество единиц багажа не превышает `__luggage_spaces`.
- Для полей `__fuel` и `__engine_oil` значения не должны превышать соответствующие ёмкости (`__fuel_tank` и `__engine_oil_capacity`) и должны быть неотрицательными.

Реализовать метод вывода всех установленных через сеттеры значений закрытых полей экземпляра класса. На основе этого класса реализовать три подхода к управлению доступом:

- (a) **С использованием объекта `property`**: Для каждого поля определить отдельные методы-геттеры и сеттеры (например, `get_speed`, `set_speed`), а затем создать свойство:

```
speed = property(get_speed, set_speed)
```

Этот код должен располагаться после определения соответствующих методов. Первый аргумент — геттер, второй — сеттер. Продемонстрировать работу на трёх экземплярах класса: создать `myex1`, `myex2`, `myex3`, установить значения через свойства и вывести их.

- (b) **С использованием декораторов `@property` и `@<имя>.setter`:** Создать новую версию класса, в которой геттеры оформляются с декоратором `@property`, а сеттеры — с декоратором вида `@speed.setter`. Имена методов должны совпадать и не содержать префиксов `get_/set_`. Пример:

```
@property
def speed(self):
    return self.__speed
@speed.setter
def speed(self, value):
    if 0 <= value <= self.__max_speed:
        self.__speed = value
    else:
        raise ValueError("Недопустимая скорость")
```

Продемонстрировать работу на трёх экземплярах и сделать выводы об оптимизации кода по сравнению с первым подходом.

- (c) **С использованием модуля `accessify`:** Установить модуль командой `pip install accessify` и импортировать:

```
from accessify import private, protected
```

Сделать поля `max_speed`, `capacity`, `fuel_tank`, `engine_oil_capacity`, `luggage_spaces` по-настоящему приватными с помощью функции `private` (например, как атрибуты класса до `__init__`). Удалить их из инициализатора. Проверки в сеттерах реализовать через вспомогательные методы, помеченные декоратором `@private`. Учитывать, что методы с `@private` нельзя вызывать из методов, использующих `@property`, поэтому для этой версии использовать только классические геттеры и сеттеры (`get_...`, `set_...`). Продемонстрировать, что попытка доступа извне (включая `myex3._Excavator__max_speed`) **не даёт результата**, а вызов приватного метода или чтение приватного поля вызывает ошибку доступа.

Для всех трёх подходов создать по три экземпляра экскаватора, установить значения полей с учётом всех ограничений и вывести текущие значения всех полей каждого экземпляра.

- 19 Разработать класс `Tractor`, который будет описывать модель трактора. В классе должны быть следующие поля с доступом уровня **private** (только внутри класса):

- `__speed`: скорость движения трактора
- `__distance`: расстояние, которое трактор проехал
- `__max_speed`: максимальная разрешённая скорость движения трактора

- `__passengers`: список пассажиров
- `__capacity`: максимальная вместимость пассажиров в тракторе
- `__empty_seats`: число свободных мест
- `__seats_occupied`: число занятых мест в тракторе
- `__fuel_tank`: объём топливного бака
- `__fuel`: количество топлива в литрах
- `__engine_oil_capacity`: объём картера масла двигателя (литры)
- `__engine_oil`: количество моторного масла в литрах
- `__luggage_spaces`: количество багажных мест
- `__luggage`: багаж трактора

Уровень доступа к полям должен быть следующим:

- `__max_speed`, `__capacity`, `__fuel_tank`, `__engine_oil_capacity`, `__luggage_spaces`: **только чтение** (через геттеры)
- `__speed`, `__distance`, `__passengers`, `__empty_seats`, `__seats_occupied`, `__fuel`, `__engine_oil`, `__luggage`: **чтение и запись** (через геттеры и сеттеры)

Требования к сеттерам:

- Для полей `__empty_seats` и `__seats_occupied` в сеттерах необходимо проверять, что передаваемое значение не превышает `__capacity` и неотрицательно.
- Для поля `__passengers` в сеттере необходимо проверять, что количество пассажиров (длина списка) не превышает `__capacity`.
- Для поля `__speed` в сеттере необходимо проверять, что заданная скорость не превышает `__max_speed` и неотрицательна.
- Для поля `__luggage` в сеттере необходимо проверять, что количество единиц багажа не превышает `__luggage_spaces`.
- Для полей `__fuel` и `__engine_oil` значения не должны превышать соответствующие ёмкости (`__fuel_tank` и `__engine_oil_capacity`) и должны быть неотрицательными.

Реализовать метод вывода всех установленных через сеттеры значений закрытых полей экземпляра класса. На основе этого класса реализовать три подхода к управлению доступом:

- (а) **С использованием объекта `property`**: Для каждого поля определить отдельные методы-геттеры и сеттеры (например, `get_speed`, `set_speed`), а затем создать свойство:

```
speed = property(get_speed, set_speed)
```

Этот код должен располагаться после определения соответствующих методов. Первый аргумент — геттер, второй — сеттер. Продемонстрировать работу на трёх экземплярах класса: создать `mytractor1`, `mytractor2`, `mytractor3`, установить значения через свойства и вывести их.

- (b) **С использованием декораторов `@property` и `@<имя>.setter`**: Создать новую версию класса, в которой геттеры оформляются с декоратором `@property`, а сеттеры — с декоратором вида `@speed.setter`. Имена методов должны совпадать и не содержать префиксов `get_/set_`. Пример:

```
@property
def speed(self):
    return self.__speed
@speed.setter
def speed(self, value):
    if 0 <= value <= self.__max_speed:
        self.__speed = value
    else:
        raise ValueError("Недопустимая скорость")
```

Продемонстрировать работу на трёх экземплярах и сделать выводы об оптимизации кода по сравнению с первым подходом.

- (c) **С использованием модуля `accessify`**: Установить модуль командой `pip install accessify` и импортировать:

```
from accessify import private, protected
```

Сделать поля `max_speed`, `capacity`, `fuel_tank`, `engine_oil_capacity`, `luggage_spaces` по-настоящему приватными с помощью функции `private` (например, как атрибуты класса до `__init__`). Удалить их из инициализатора. Проверки в сеттерах реализовать через вспомогательные методы, помеченные декоратором `@private`. Учитывать, что методы с `@private` нельзя вызывать из методов, использующих `@property`, поэтому для этой версии использовать только классические геттеры и сеттеры (`get_...`, `set_...`). Продемонстрировать, что попытка доступа извне (включая `mytractor3._Tractor__max_speed`) **не даёт результата**, а вызов приватного метода или чтение приватного поля вызывает ошибку доступа.

Для всех трёх подходов создать по три экземпляра трактора, установить значения полей с учётом всех ограничений и вывести текущие значения всех полей каждого экземпляра.

- 20 Разработать класс `Snowmobile`, который будет описывать модель снегохода. В классе должны быть следующие поля с доступом уровня **private** (только внутри класса):

- `__speed`: скорость движения снегохода
- `__distance`: расстояние, которое снегоход проехал
- `__max_speed`: максимальная разрешённая скорость движения снегохода
- `__passengers`: список пассажиров
- `__capacity`: максимальная вместимость пассажиров на снегоходе
- `__empty_seats`: число свободных мест
- `__seats_occupied`: число занятых мест на снегоходе
- `__fuel_tank`: объём топливного бака
- `__fuel`: количество топлива в литрах

- `__engine_oil_capacity`: объём картера масла двигателя (литры)
- `__engine_oil`: количество моторного масла в литрах
- `__luggage_spaces`: количество багажных мест
- `__luggage`: багаж снегохода

Уровень доступа к полям должен быть следующим:

- `__max_speed`, `__capacity`, `__fuel_tank`, `__engine_oil_capacity`, `__luggage_spaces`: **только чтение** (через геттеры)
- `__speed`, `__distance`, `__passengers`, `__empty_seats`, `__seats_occupied`, `__fuel`, `__engine_oil`, `__luggage`: **чтение и запись** (через геттеры и сеттеры)

Требования к сеттерам:

- Для полей `__empty_seats` и `__seats_occupied` в сеттерах необходимо проверять, что передаваемое значение не превышает `__capacity` и неотрицательно.
- Для поля `__passengers` в сеттере необходимо проверять, что количество пассажиров (длина списка) не превышает `__capacity`.
- Для поля `__speed` в сеттере необходимо проверять, что заданная скорость не превышает `__max_speed` и неотрицательна.
- Для поля `__luggage` в сеттере необходимо проверять, что количество единиц багажа не превышает `__luggage_spaces`.
- Для полей `__fuel` и `__engine_oil` значения не должны превышать соответствующие ёмкости (`__fuel_tank` и `__engine_oil_capacity`) и должны быть неотрицательными.

Реализовать метод вывода всех установленных через сеттеры значений закрытых полей экземпляра класса. На основе этого класса реализовать три подхода к управлению доступом:

- (a) **С использованием объекта `property`**: Для каждого поля определить отдельные методы-геттеры и сеттеры (например, `get_speed`, `set_speed`), а затем создать свойство:

```
speed = property(get_speed, set_speed)
```

Этот код должен располагаться после определения соответствующих методов. Первый аргумент — геттер, второй — сеттер. Продемонстрировать работу на трёх экземплярах класса: создать `mysnow1`, `mysnow2`, `mysnow3`, установить значения через свойства и вывести их.

- (b) **С использованием декораторов `@property` и `@<имя>.setter`**: Создать новую версию класса, в которой геттеры оформляются с декоратором `@property`, а сеттеры — с декоратором вида `@speed.setter`. Имена методов должны совпадать и не содержать префиксов `get_/set_`. Пример:

```
@property
def speed(self):
    return self.__speed
@speed.setter
```



```
def speed(self, value):
    if 0 <= value <= self.__max_speed:
        self.__speed = value
    else:
        raise ValueError("Недопустимая скорость")
```

Продемонстрировать работу на трёх экземплярах и сделать выводы об оптимизации кода по сравнению с первым подходом.

- (с) **С использованием модуля accessify:** Установить модуль командой `pip install accessify` и импортировать:

```
from accessify import private, protected
```

Сделать поля `max_speed`, `capacity`, `fuel_tank`, `engine_oil_capacity`, `luggage_spaces` по-настоящему приватными с помощью функции `private` (например, как атрибуты класса до `__init__`). Удалить их из инициализатора. Проверки в сеттерах реализовать через вспомогательные методы, помеченные декоратором `@private`. Учитывать, что методы с `@private` нельзя вызывать из методов, использующих `@property`, поэтому для этой версии использовать только классические геттеры и сеттеры (`get...`, `set...`). Продемонстрировать, что попытка доступа извне (включая `mysnow3._Snowmobile__max_speed`) **не даёт результата**, а вызов приватного метода или чтение приватного поля вызывает ошибку доступа.

Для всех трёх подходов создать по три экземпляра снегохода, установить значения полей с учётом всех ограничений и вывести текущие значения всех полей каждого экземпляра.

- 21 Разработать класс ATV, который будет описывать модель вездехода (quad bike). В классе должны быть следующие поля с доступом уровня **private** (только внутри класса):

- `__speed`: скорость движения вездехода
- `__distance`: расстояние, которое вездеход проехал
- `__max_speed`: максимальная разрешённая скорость движения вездехода
- `__passengers`: список пассажиров
- `__capacity`: максимальная вместимость пассажиров на вездеходе
- `__empty_seats`: число свободных мест
- `__seats_occupied`: число занятых мест на вездеходе
- `__fuel_tank`: объём топливного бака
- `__fuel`: количество топлива в литрах
- `__engine_oil_capacity`: объём картера масла двигателя (литры)
- `__engine_oil`: количество моторного масла в литрах
- `__luggage_spaces`: количество багажных мест
- `__luggage`: багаж вездехода

Уровень доступа к полям должен быть следующим:

- `__max_speed`, `__capacity`, `__fuel_tank`, `__engine_oil_capacity`, `__luggage_spaces`: **только чтение** (через геттеры)
- `__speed`, `__distance`, `__passengers`, `__empty_seats`, `__seats_occupied`, `__fuel`, `__engine_oil`, `__luggage`: **чтение и запись** (через геттеры и сеттеры)

Требования к сеттерам:

- Для полей `__empty_seats` и `__seats_occupied` в сеттерах необходимо проверять, что передаваемое значение не превышает `__capacity` и неотрицательно.
- Для поля `__passengers` в сеттере необходимо проверять, что количество пассажиров (длина списка) не превышает `__capacity`.
- Для поля `__speed` в сеттере необходимо проверять, что заданная скорость не превышает `__max_speed` и неотрицательна.
- Для поля `__luggage` в сеттере необходимо проверять, что количество единиц багажа не превышает `__luggage_spaces`.
- Для полей `__fuel` и `__engine_oil` значения не должны превышать соответствующие ёмкости (`__fuel_tank` и `__engine_oil_capacity`) и должны быть неотрицательными.

Реализовать метод вывода всех установленных через сеттеры значений закрытых полей экземпляра класса. На основе этого класса реализовать три подхода к управлению доступом:

- (a) **С использованием объекта `property`**: Для каждого поля определить отдельные методы-геттеры и сеттеры (например, `get_speed`, `set_speed`), а затем создать свойство:

```
speed = property(get_speed, set_speed)
```

Этот код должен располагаться после определения соответствующих методов. Первый аргумент — геттер, второй — сеттер. Продемонстрировать работу на трёх экземплярах класса: создать `myatv1`, `myatv2`, `myatv3`, установить значения через свойства и вывести их.

- (b) **С использованием декораторов `@property` и `@<имя>.setter`**: Создать новую версию класса, в которой геттеры оформляются с декоратором `@property`, а сеттеры — с декоратором вида `@speed.setter`. Имена методов должны совпадать и не содержать префиксов `get_/set_`. Пример:

```
@property
def speed(self):
    return self.__speed
@speed.setter
def speed(self, value):
    if 0 <= value <= self.__max_speed:
        self.__speed = value
    else:
        raise ValueError("Недопустимая скорость")
```

Продемонстрировать работу на трёх экземплярах и сделать выводы об оптимизации кода по сравнению с первым подходом.

- (с) **С использованием модуля `accessify`:** Установить модуль командой `pip install accessify` и импортировать:

```
from accessify import private, protected
```

Сделать поля `max_speed`, `capacity`, `fuel_tank`, `engine_oil_capacity`, `luggage_spaces` по-настоящему приватными с помощью функции `private` (например, как атрибуты класса до `__init__`). Удалить их из инициализатора. Проверки в сеттерах реализовать через вспомогательные методы, помеченные декоратором `@private`. Учитывать, что методы с `@private` нельзя вызывать из методов, использующих `@property`, поэтому для этой версии использовать только классические геттеры и сеттеры (`get_...`, `set_...`). Продемонстрировать, что попытка доступа извне (включая `myatv3._ATV__max_speed`) **не даёт результата**, а вызов приватного метода или чтение приватного поля вызывает ошибку доступа.

Для всех трёх подходов создать по три экземпляра вездехода, установить значения полей с учётом всех ограничений и вывести текущие значения всех полей каждого экземпляра.

- 22 Разработать класс `Hovercraft`, который будет описывать модель судна на воздушной подушке. В классе должны быть следующие поля с доступом уровня **private** (только внутри класса):

- `__speed`: скорость движения судна на воздушной подушке
- `__distance`: расстояние, которое судно на воздушной подушке прошло
- `__max_speed`: максимальная разрешённая скорость движения судна на воздушной подушке
- `__passengers`: список пассажиров
- `__capacity`: максимальная вместимость пассажиров на судне на воздушной подушке
- `__empty_seats`: число свободных мест
- `__seats_occupied`: число занятых мест на судне на воздушной подушке
- `__fuel_tank`: объём топливного бака
- `__fuel`: количество топлива в литрах
- `__engine_oil_capacity`: объём картера масла двигателя (литры)
- `__engine_oil`: количество моторного масла в литрах
- `__luggage_spaces`: количество багажных мест
- `__luggage`: багаж судна на воздушной подушке

Уровень доступа к полям должен быть следующим:

- `__max_speed`, `__capacity`, `__fuel_tank`, `__engine_oil_capacity`, `__luggage_spaces`: **только чтение** (через геттеры)
- `__speed`, `__distance`, `__passengers`, `__empty_seats`, `__seats_occupied`, `__fuel`, `__engine_oil`, `__luggage`: **чтение и запись** (через геттеры и сеттеры)

Требования к сеттерам:

- Для полей `__empty_seats` и `__seats_occupied` в сеттерах необходимо проверять, что передаваемое значение не превышает `__capacity` и неотрицательно.
- Для поля `__passengers` в сеттере необходимо проверять, что количество пассажиров (длина списка) не превышает `__capacity`.
- Для поля `__speed` в сеттере необходимо проверять, что заданная скорость не превышает `__max_speed` и неотрицательна.
- Для поля `__luggage` в сеттере необходимо проверять, что количество единиц багажа не превышает `__luggage_spaces`.
- Для полей `__fuel` и `__engine_oil` значения не должны превышать соответствующие ёмкости (`__fuel_tank` и `__engine_oil_capacity`) и должны быть неотрицательными.

Реализовать метод вывода всех установленных через сеттеры значений закрытых полей экземпляра класса. На основе этого класса реализовать три подхода к управлению доступом:

- (a) **С использованием объекта `property`:** Для каждого поля определить отдельные методы-геттеры и сеттеры (например, `get_speed`, `set_speed`), а затем создать свойство:

```
speed = property(get_speed, set_speed)
```

Этот код должен располагаться после определения соответствующих методов. Первый аргумент — геттер, второй — сеттер. Продемонстрировать работу на трёх экземплярах класса: создать `myhover1`, `myhover2`, `myhover3`, установить значения через свойства и вывести их.

- (b) **С использованием декораторов `@property` и `@<имя>.setter`:** Создать новую версию класса, в которой геттеры оформляются с декоратором `@property`, а сеттеры — с декоратором вида `@speed.setter`. Имена методов должны совпадать и не содержать префиксов `get_/set_`. Пример:

```
@property
def speed(self):
    return self.__speed
@speed.setter
def speed(self, value):
    if 0 <= value <= self.__max_speed:
        self.__speed = value
    else:
        raise ValueError("Недопустимая скорость")
```

Продемонстрировать работу на трёх экземплярах и сделать выводы об оптимизации кода по сравнению с первым подходом.

- (c) **С использованием модуля `accessify`:** Установить модуль командой `pip install accessify` и импортировать:

```
from accessify import private, protected
```

Сделать поля `max_speed`, `capacity`, `fuel_tank`, `engine_oil_capacity`, `luggage_spaces` по-настоящему приватными с помощью функции `private` (например, как атрибуты класса до `__init__`). Удалить их из инициализатора. Проверки в сеттерах реализовать через вспомогательные методы, помеченные декоратором `@private`. Учитывать, что методы с `@private` нельзя вызывать из методов, использующих `@property`, поэтому для этой версии использовать только классические геттеры и сеттеры (`get_...`, `set_...`). Продемонстрировать, что попытка доступа извне (включая `myhover3._Hovercraft__max_speed`) **не даёт результата**, а вызов приватного метода или чтение приватного поля вызывает ошибку доступа.

Для всех трёх подходов создать по три экземпляра судна на воздушной подушке, установить значения полей с учётом всех ограничений и вывести текущие значения всех полей каждого экземпляра.

23 Разработать класс `Rocket`, который будет описывать модель ракеты. В классе должны быть следующие поля с доступом уровня **private** (только внутри класса):

- `__speed`: скорость движения ракеты
- `__distance`: расстояние, которое ракета пролетела
- `__max_speed`: максимальная разрешённая скорость движения ракеты
- `__passengers`: список пассажиров
- `__capacity`: максимальная вместимость пассажиров в ракете
- `__empty_seats`: число свободных мест
- `__seats_occupied`: число занятых мест в ракете
- `__fuel_tank`: объём топливного бака
- `__fuel`: количество топлива в литрах
- `__engine_oil_capacity`: объём картера масла двигателя (литры)
- `__engine_oil`: количество моторного масла в литрах
- `__luggage_spaces`: количество багажных мест
- `__luggage`: багаж ракеты

Уровень доступа к полям должен быть следующим:

- `__max_speed`, `__capacity`, `__fuel_tank`, `__engine_oil_capacity`, `__luggage_spaces`: **только чтение** (через геттеры)
- `__speed`, `__distance`, `__passengers`, `__empty_seats`, `__seats_occupied`, `__fuel`, `__engine_oil`, `__luggage`: **чтение и запись** (через геттеры и сеттеры)

Требования к сеттерам:

- Для полей `__empty_seats` и `__seats_occupied` в сеттерах необходимо проверять, что передаваемое значение не превышает `__capacity` и неотрицательно.
- Для поля `__passengers` в сеттере необходимо проверять, что количество пассажиров (длина списка) не превышает `__capacity`.
- Для поля `__speed` в сеттере необходимо проверять, что заданная скорость не превышает `__max_speed` и неотрицательна.

- Для поля `__luggage` в сеттере необходимо проверять, что количество единиц багажа не превышает `__luggage_spaces`.
- Для полей `__fuel` и `__engine_oil` значения не должны превышать соответствующие ёмкости (`__fuel_tank` и `__engine_oil_capacity`) и должны быть неотрицательными.

Реализовать метод вывода всех установленных через сеттеры значений закрытых полей экземпляра класса. На основе этого класса реализовать три подхода к управлению доступом:

- (a) **С использованием объекта `property`:** Для каждого поля определить отдельные методы-геттеры и сеттеры (например, `get_speed`, `set_speed`), а затем создать свойство:

```
speed = property(get_speed, set_speed)
```

Этот код должен располагаться после определения соответствующих методов. Первый аргумент — геттер, второй — сеттер. Продемонстрировать работу на трёх экземплярах класса: создать `myrocket1`, `myrocket2`, `myrocket3`, установить значения через свойства и вывести их.

- (b) **С использованием декораторов `@property` и `@<имя>.setter`:** Создать новую версию класса, в которой геттеры оформляются с декоратором `@property`, а сеттеры — с декоратором вида `@speed.setter`. Имена методов должны совпадать и не содержать префиксов `get_/set_`. Пример:

```
@property
def speed(self):
    return self.__speed
@speed.setter
def speed(self, value):
    if 0 <= value <= self.__max_speed:
        self.__speed = value
    else:
        raise ValueError("Недопустимая скорость")
```

Продемонстрировать работу на трёх экземплярах и сделать выводы об оптимизации кода по сравнению с первым подходом.

- (c) **С использованием модуля `accessify`:** Установить модуль командой `pip install accessify` и импортировать:

```
from accessify import private, protected
```

Сделать поля `max_speed`, `capacity`, `fuel_tank`, `engine_oil_capacity`, `luggage_spaces` по-настоящему приватными с помощью функции `private` (например, как атрибуты класса до `__init__`). Удалить их из инициализатора. Проверки в сеттерах реализовать через вспомогательные методы, помеченные декоратором `@private`. Учитывать, что методы с `@private` нельзя вызывать из методов, использующих `@property`, поэтому для этой версии использовать только классические геттеры и сеттеры (`get_...`, `set_...`). Продемонстрировать, что попытка доступа извне

(включая `myrocket3._Rocket__max_speed`) **не даёт результата**, а вызов приватного метода или чтение приватного поля вызывает ошибку доступа.

Для всех трёх подходов создать по три экземпляра ракеты, установить значения полей с учётом всех ограничений и вывести текущие значения всех полей каждого экземпляра.

24 Разработать класс `Glider`, который будет описывать модель планера. В классе должны быть следующие поля с доступом уровня **private** (только внутри класса):

- `__speed`: скорость движения планера
- `__distance`: расстояние, которое планер пролетел
- `__max_speed`: максимальная разрешённая скорость движения планера
- `__passengers`: список пассажиров
- `__capacity`: максимальная вместимость пассажиров в планере
- `__empty_seats`: число свободных мест
- `__seats_occupied`: число занятых мест в планере
- `__fuel_tank`: объём топливного бака
- `__fuel`: количество топлива в литрах
- `__engine_oil_capacity`: объём картера масла двигателя (литры)
- `__engine_oil`: количество моторного масла в литрах
- `__luggage_spaces`: количество багажных мест
- `__luggage`: багаж планера

Уровень доступа к полям должен быть следующим:

- `__max_speed`, `__capacity`, `__fuel_tank`, `__engine_oil_capacity`, `__luggage_spaces`: **только чтение** (через геттеры)
- `__speed`, `__distance`, `__passengers`, `__empty_seats`, `__seats_occupied`, `__fuel`, `__engine_oil`, `__luggage`: **чтение и запись** (через геттеры и сеттеры)

Требования к сеттерам:

- Для полей `__empty_seats` и `__seats_occupied` в сеттерах необходимо проверять, что передаваемое значение не превышает `__capacity` и неотрицательно.
- Для поля `__passengers` в сеттере необходимо проверять, что количество пассажиров (длина списка) не превышает `__capacity`.
- Для поля `__speed` в сеттере необходимо проверять, что заданная скорость не превышает `__max_speed` и неотрицательна.
- Для поля `__luggage` в сеттере необходимо проверять, что количество единиц багажа не превышает `__luggage_spaces`.
- Для полей `__fuel` и `__engine_oil` значения не должны превышать соответствующие ёмкости (`__fuel_tank` и `__engine_oil_capacity`) и должны быть неотрицательными.

Реализовать метод вывода всех установленных через сеттеры значений закрытых полей экземпляра класса. На основе этого класса реализовать три подхода к управлению доступом:

- (a) **С использованием объекта `property`:** Для каждого поля определить отдельные методы-геттеры и сеттеры (например, `get_speed`, `set_speed`), а затем создать свойство:

```
speed = property(get_speed, set_speed)
```

Этот код должен располагаться после определения соответствующих методов. Первый аргумент — геттер, второй — сеттер. Продемонстрировать работу на трёх экземплярах класса: создать `myglider1`, `myglider2`, `myglider3`, установить значения через свойства и вывести их.

- (b) **С использованием декораторов `@property` и `@<имя>.setter`:** Создать новую версию класса, в которой геттеры оформляются с декоратором `@property`, а сеттеры — с декоратором вида `@speed.setter`. Имена методов должны совпадать и не содержать префиксов `get_/set_`. Пример:

```
@property
def speed(self):
    return self.__speed
@speed.setter
def speed(self, value):
    if 0 <= value <= self.__max_speed:
        self.__speed = value
    else:
        raise ValueError("Недопустимая скорость")
```

Продемонстрировать работу на трёх экземплярах и сделать выводы об оптимизации кода по сравнению с первым подходом.

- (c) **С использованием модуля `accessify`:** Установить модуль командой `pip install accessify` и импортировать:

```
from accessify import private, protected
```

Сделать поля `max_speed`, `capacity`, `fuel_tank`, `engine_oil_capacity`, `luggage_spaces` по-настоящему приватными с помощью функции `private` (например, как атрибуты класса до `__init__`). Удалить их из инициализатора. Проверки в сеттерах реализовать через вспомогательные методы, помеченные декоратором `@private`. Учитывать, что методы с `@private` нельзя вызывать из методов, использующих `@property`, поэтому для этой версии использовать только классические геттеры и сеттеры (`get_...`, `set_...`). Продемонстрировать, что попытка доступа извне (включая `myglider3._Glider__max_speed`) **не даёт результата**, а вызов приватного метода или чтение приватного поля вызывает ошибку доступа.

Для всех трёх подходов создать по три экземпляра планера, установить значения полей с учётом всех ограничений и вывести текущие значения всех полей каждого экземпляра.

- 25 Разработать класс `Zeppelin`, который будет описывать модель дирижабля. В классе должны быть следующие поля с доступом уровня **private** (только внутри класса):
- `__speed`: скорость движения дирижабля

- `__distance`: расстояние, которое дирижабль пролетел
- `__max_speed`: максимальная разрешённая скорость движения дирижабля
- `__passengers`: список пассажиров
- `__capacity`: максимальная вместимость пассажиров в дирижабле
- `__empty_seats`: число свободных мест
- `__seats_occupied`: число занятых мест в дирижабле
- `__fuel_tank`: объём топливного бака
- `__fuel`: количество топлива в литрах
- `__engine_oil_capacity`: объём картера масла двигателя (литры)
- `__engine_oil`: количество моторного масла в литрах
- `__luggage_spaces`: количество багажных мест
- `__luggage`: багаж дирижабля

Уровень доступа к полям должен быть следующим:

- `__max_speed`, `__capacity`, `__fuel_tank`, `__engine_oil_capacity`, `__luggage_spaces`: **только чтение** (через геттеры)
- `__speed`, `__distance`, `__passengers`, `__empty_seats`, `__seats_occupied`, `__fuel`, `__engine_oil`, `__luggage`: **чтение и запись** (через геттеры и сеттеры)

Требования к сеттерам:

- Для полей `__empty_seats` и `__seats_occupied` в сеттерах необходимо проверять, что передаваемое значение не превышает `__capacity` и неотрицательно.
- Для поля `__passengers` в сеттере необходимо проверять, что количество пассажиров (длина списка) не превышает `__capacity`.
- Для поля `__speed` в сеттере необходимо проверять, что заданная скорость не превышает `__max_speed` и неотрицательна.
- Для поля `__luggage` в сеттере необходимо проверять, что количество единиц багажа не превышает `__luggage_spaces`.
- Для полей `__fuel` и `__engine_oil` значения не должны превышать соответствующие ёмкости (`__fuel_tank` и `__engine_oil_capacity`) и должны быть неотрицательными.

Реализовать метод вывода всех установленных через сеттеры значений закрытых полей экземпляра класса. На основе этого класса реализовать три подхода к управлению доступом:

- С использованием объекта `property`**: Для каждого поля определить отдельные методы-геттеры и сеттеры (например, `get_speed`, `set_speed`), а затем создать свойство:

```
speed = property(get_speed, set_speed)
```

Этот код должен располагаться после определения соответствующих методов. Первый аргумент — геттер, второй — сеттер. Продемонстрировать работу на трёх экземплярах класса: создать `myzer1`, `myzer2`, `myzer3`, установить значения через свойства и вывести их.

- (b) **С использованием декораторов `@property` и `@<имя>.setter`:** Создать новую версию класса, в которой геттеры оформляются с декоратором `@property`, а сеттеры — с декоратором вида `@speed.setter`. Имена методов должны совпадать и не содержать префиксов `get_/set_`. Пример:

```
@property
def speed(self):
    return self.__speed
@speed.setter
def speed(self, value):
    if 0 <= value <= self.__max_speed:
        self.__speed = value
    else:
        raise ValueError("Недопустимая скорость")
```

Продемонстрировать работу на трёх экземплярах и сделать выводы об оптимизации кода по сравнению с первым подходом.

- (c) **С использованием модуля `accessify`:** Установить модуль командой `pip install accessify` и импортировать:

```
from accessify import private, protected
```

Сделать поля `max_speed`, `capacity`, `fuel_tank`, `engine_oil_capacity`, `luggage_spaces` по-настоящему приватными с помощью функции `private` (например, как атрибуты класса до `__init__`). Удалить их из инициализатора. Проверки в сеттерах реализовать через вспомогательные методы, помеченные декоратором `@private`. Учитывать, что методы с `@private` нельзя вызывать из методов, использующих `@property`, поэтому для этой версии использовать только классические геттеры и сеттеры (`get_...`, `set_...`). Продемонстрировать, что попытка доступа извне (включая `myzer3.Zeppelin.__max_speed`) **не даёт результата**, а вызов приватного метода или чтение приватного поля вызывает ошибку доступа.

Для всех трёх подходов создать по три экземпляра дирижабля, установить значения полей с учётом всех ограничений и вывести текущие значения всех полей каждого экземпляра.

- 26 Разработать класс `Ferry`, который будет описывать модель парома. В классе должны быть следующие поля с доступом уровня **private** (только внутри класса):

- `__speed`: скорость движения парома
- `__distance`: расстояние, которое паром прошёл
- `__max_speed`: максимальная разрешённая скорость движения парома
- `__passengers`: список пассажиров
- `__capacity`: максимальная вместимость пассажиров на пароме
- `__empty_seats`: число свободных мест
- `__seats_occupied`: число занятых мест на пароме
- `__fuel_tank`: объём топливного бака
- `__fuel`: количество топлива в литрах

- `__engine_oil_capacity`: объём картера масла двигателя (литры)
- `__engine_oil`: количество моторного масла в литрах
- `__luggage_spaces`: количество багажных мест
- `__luggage`: багаж парома

Уровень доступа к полям должен быть следующим:

- `__max_speed`, `__capacity`, `__fuel_tank`, `__engine_oil_capacity`, `__luggage_spaces`: **только чтение** (через геттеры)
- `__speed`, `__distance`, `__passengers`, `__empty_seats`, `__seats_occupied`, `__fuel`, `__engine_oil`, `__luggage`: **чтение и запись** (через геттеры и сеттеры)

Требования к сеттерам:

- Для полей `__empty_seats` и `__seats_occupied` в сеттерах необходимо проверять, что передаваемое значение не превышает `__capacity` и неотрицательно.
- Для поля `__passengers` в сеттере необходимо проверять, что количество пассажиров (длина списка) не превышает `__capacity`.
- Для поля `__speed` в сеттере необходимо проверять, что заданная скорость не превышает `__max_speed` и неотрицательна.
- Для поля `__luggage` в сеттере необходимо проверять, что количество единиц багажа не превышает `__luggage_spaces`.
- Для полей `__fuel` и `__engine_oil` значения не должны превышать соответствующие ёмкости (`__fuel_tank` и `__engine_oil_capacity`) и должны быть неотрицательными.

Реализовать метод вывода всех установленных через сеттеры значений закрытых полей экземпляра класса. На основе этого класса реализовать три подхода к управлению доступом:

- (a) **С использованием объекта `property`**: Для каждого поля определить отдельные методы-геттеры и сеттеры (например, `get_speed`, `set_speed`), а затем создать свойство:

```
speed = property(get_speed, set_speed)
```

Этот код должен располагаться после определения соответствующих методов. Первый аргумент — геттер, второй — сеттер. Продемонстрировать работу на трёх экземплярах класса: создать `myferry1`, `myferry2`, `myferry3`, установить значения через свойства и вывести их.

- (b) **С использованием декораторов `@property` и `@<имя>.setter`**: Создать новую версию класса, в которой геттеры оформляются с декоратором `@property`, а сеттеры — с декоратором вида `@speed.setter`. Имена методов должны совпадать и не содержать префиксов `get_/set_`. Пример:

```
@property
def speed(self):
    return self.__speed
@speed.setter
```

```
def speed(self, value):
    if 0 <= value <= self.__max_speed:
        self.__speed = value
    else:
        raise ValueError("Недопустимая скорость")
```

Продemonстрировать работу на трёх экземплярах и сделать выводы об оптимизации кода по сравнению с первым подходом.

- (с) **С использованием модуля accessify:** Установить модуль командой `pip install accessify` и импортировать:

```
from accessify import private, protected
```

Сделать поля `max_speed`, `capacity`, `fuel_tank`, `engine_oil_capacity`, `luggage_spaces` по-настоящему приватными с помощью функции `private` (например, как атрибуты класса до `__init__`). Удалить их из инициализатора. Проверки в сеттерах реализовать через вспомогательные методы, помеченные декоратором `@private`. Учитывать, что методы с `@private` нельзя вызывать из методов, использующих `@property`, поэтому для этой версии использовать только классические геттеры и сеттеры (`get...`, `set...`). Продemonстрировать, что попытка доступа извне (включая `myferry3._Ferry__max_speed`) **не даёт результата**, а вызов приватного метода или чтение приватного поля вызывает ошибку доступа.

Для всех трёх подходов создать по три экземпляра парома, установить значения полей с учётом всех ограничений и вывести текущие значения всех полей каждого экземпляра.

- 27 Разработать класс `Yacht`, который будет описывать модель яхты. В классе должны быть следующие поля с доступом уровня **private** (только внутри класса):

- `__speed`: скорость движения яхты
- `__distance`: расстояние, которое яхта прошла
- `__max_speed`: максимальная разрешённая скорость движения яхты
- `__passengers`: список пассажиров
- `__capacity`: максимальная вместимость пассажиров на яхте
- `__empty_seats`: число свободных мест
- `__seats_occupied`: число занятых мест на яхте
- `__fuel_tank`: объём топливного бака
- `__fuel`: количество топлива в литрах
- `__engine_oil_capacity`: объём картера масла двигателя (литры)
- `__engine_oil`: количество моторного масла в литрах
- `__luggage_spaces`: количество багажных мест
- `__luggage`: багаж яхты

Уровень доступа к полям должен быть следующим:

- `__max_speed`, `__capacity`, `__fuel_tank`, `__engine_oil_capacity`, `__luggage_spaces`: **только чтение** (через геттеры)
- `__speed`, `__distance`, `__passengers`, `__empty_seats`, `__seats_occupied`, `__fuel`, `__engine_oil`, `__luggage`: **чтение и запись** (через геттеры и сеттеры)

Требования к сеттерам:

- Для полей `__empty_seats` и `__seats_occupied` в сеттерах необходимо проверять, что передаваемое значение не превышает `__capacity` и неотрицательно.
- Для поля `__passengers` в сеттере необходимо проверять, что количество пассажиров (длина списка) не превышает `__capacity`.
- Для поля `__speed` в сеттере необходимо проверять, что заданная скорость не превышает `__max_speed` и неотрицательна.
- Для поля `__luggage` в сеттере необходимо проверять, что количество единиц багажа не превышает `__luggage_spaces`.
- Для полей `__fuel` и `__engine_oil` значения не должны превышать соответствующие ёмкости (`__fuel_tank` и `__engine_oil_capacity`) и должны быть неотрицательными.

Реализовать метод вывода всех установленных через сеттеры значений закрытых полей экземпляра класса. На основе этого класса реализовать три подхода к управлению доступом:

- (a) **С использованием объекта `property`**: Для каждого поля определить отдельные методы-геттеры и сеттеры (например, `get_speed`, `set_speed`), а затем создать свойство:

```
speed = property(get_speed, set_speed)
```

Этот код должен располагаться после определения соответствующих методов. Первый аргумент — геттер, второй — сеттер. Продемонстрировать работу на трёх экземплярах класса: создать `myyacht1`, `myyacht2`, `myyacht3`, установить значения через свойства и вывести их.

- (b) **С использованием декораторов `@property` и `@<имя>.setter`**: Создать новую версию класса, в которой геттеры оформляются с декоратором `@property`, а сеттеры — с декоратором вида `@speed.setter`. Имена методов должны совпадать и не содержать префиксов `get_/set_`. Пример:

```
@property
def speed(self):
    return self.__speed
@speed.setter
def speed(self, value):
    if 0 <= value <= self.__max_speed:
        self.__speed = value
    else:
        raise ValueError("Недопустимая скорость")
```

Продемонстрировать работу на трёх экземплярах и сделать выводы об оптимизации кода по сравнению с первым подходом.

- (с) **С использованием модуля accessify:** Установить модуль командой `pip install accessify` и импортировать:

```
from accessify import private, protected
```

Сделать поля `max_speed`, `capacity`, `fuel_tank`, `engine_oil_capacity`, `luggage_spaces` по-настоящему приватными с помощью функции `private` (например, как атрибуты класса до `__init__`). Удалить их из инициализатора. Проверки в сеттерах реализовать через вспомогательные методы, помеченные декоратором `@private`. Учитывать, что методы с `@private` нельзя вызывать из методов, использующих `@property`, поэтому для этой версии использовать только классические геттеры и сеттеры (`get...`, `set...`). Продемонстрировать, что попытка доступа извне (включая `myyacht3.Yacht__max_speed`) **не даёт результата**, а вызов приватного метода или чтение приватного поля вызывает ошибку доступа.

Для всех трёх подходов создать по три экземпляра яхты, установить значения полей с учётом всех ограничений и вывести текущие значения всех полей каждого экземпляра.

- 28 Разработать класс `Speedboat`, который будет описывать модель быстроходной лодки. В классе должны быть следующие поля с доступом уровня **private** (только внутри класса):

- `__speed`: скорость движения быстроходной лодки
- `__distance`: расстояние, которое быстроходная лодка прошла
- `__max_speed`: максимальная разрешённая скорость движения быстроходной лодки
- `__passengers`: список пассажиров
- `__capacity`: максимальная вместимость пассажиров на быстроходной лодке
- `__empty_seats`: число свободных мест
- `__seats_occupied`: число занятых мест на быстроходной лодке
- `__fuel_tank`: объём топливного бака
- `__fuel`: количество топлива в литрах
- `__engine_oil_capacity`: объём картера масла двигателя (литры)
- `__engine_oil`: количество моторного масла в литрах
- `__luggage_spaces`: количество багажных мест
- `__luggage`: багаж быстроходной лодки

Уровень доступа к полям должен быть следующим:

- `__max_speed`, `__capacity`, `__fuel_tank`, `__engine_oil_capacity`, `__luggage_spaces`: **только чтение** (через геттеры)
- `__speed`, `__distance`, `__passengers`, `__empty_seats`, `__seats_occupied`, `__fuel`, `__engine_oil`, `__luggage`: **чтение и запись** (через геттеры и сеттеры)

Требования к сеттерам:

- Для полей `__empty_seats` и `__seats_occupied` в сеттерах необходимо проверять, что передаваемое значение не превышает `__capacity` и неотрицательно.
- Для поля `__passengers` в сеттере необходимо проверять, что количество пассажиров (длина списка) не превышает `__capacity`.
- Для поля `__speed` в сеттере необходимо проверять, что заданная скорость не превышает `__max_speed` и неотрицательна.
- Для поля `__luggage` в сеттере необходимо проверять, что количество единиц багажа не превышает `__luggage_spaces`.
- Для полей `__fuel` и `__engine_oil` значения не должны превышать соответствующие ёмкости (`__fuel_tank` и `__engine_oil_capacity`) и должны быть неотрицательными.

Реализовать метод вывода всех установленных через сеттеры значений закрытых полей экземпляра класса. На основе этого класса реализовать три подхода к управлению доступом:

- (a) **С использованием объекта `property`:** Для каждого поля определить отдельные методы-геттеры и сеттеры (например, `get_speed`, `set_speed`), а затем создать свойство:

```
speed = property(get_speed, set_speed)
```

Этот код должен располагаться после определения соответствующих методов. Первый аргумент — геттер, второй — сеттер. Продемонстрировать работу на трёх экземплярах класса: создать `myspeed1`, `myspeed2`, `myspeed3`, установить значения через свойства и вывести их.

- (b) **С использованием декораторов `@property` и `@<имя>.setter`:** Создать новую версию класса, в которой геттеры оформляются с декоратором `@property`, а сеттеры — с декоратором вида `@speed.setter`. Имена методов должны совпадать и не содержать префиксов `get_/set_`. Пример:

```
@property
def speed(self):
    return self.__speed
@speed.setter
def speed(self, value):
    if 0 <= value <= self.__max_speed:
        self.__speed = value
    else:
        raise ValueError("Недопустимая скорость")
```

Продемонстрировать работу на трёх экземплярах и сделать выводы об оптимизации кода по сравнению с первым подходом.

- (c) **С использованием модуля `accessify`:** Установить модуль командой `pip install accessify` и импортировать:

```
from accessify import private, protected
```

Сделать поля `max_speed`, `capacity`, `fuel_tank`, `engine_oil_capacity`, `luggage_spaces` по-настоящему приватными с помощью функции `private` (например, как атрибуты класса до `__init__`). Удалить их из инициализатора. Проверки в сеттерах реализовать через вспомогательные методы, помеченные декоратором `@private`. Учитывать, что методы с `@private` нельзя вызывать из методов, использующих `@property`, поэтому для этой версии использовать только классические геттеры и сеттеры (`get_...`, `set_...`). Продемонстрировать, что попытка доступа извне (включая `myspeed3._Speedboat__max_speed`) **не даёт результата**, а вызов приватного метода или чтение приватного поля вызывает ошибку доступа.

Для всех трёх подходов создать по три экземпляра быстроходной лодки, установить значения полей с учётом всех ограничений и вывести текущие значения всех полей каждого экземпляра.

- 29 Разработать класс `CargoPlane`, который будет описывать модель грузового самолёта. В классе должны быть следующие поля с доступом уровня **private** (только внутри класса):

- `__speed`: скорость движения грузового самолёта
- `__distance`: расстояние, которое грузовой самолёт пролетел
- `__max_speed`: максимальная разрешённая скорость движения грузового самолёта
- `__passengers`: список пассажиров
- `__capacity`: максимальная вместимость пассажиров в грузовом самолёте
- `__empty_seats`: число свободных мест
- `__seats_occupied`: число занятых мест в грузовом самолёте
- `__fuel_tank`: объём топливного бака
- `__fuel`: количество топлива в литрах
- `__engine_oil_capacity`: объём картера масла двигателя (литры)
- `__engine_oil`: количество моторного масла в литрах
- `__luggage_spaces`: количество багажных мест
- `__luggage`: багаж грузового самолёта

Уровень доступа к полям должен быть следующим:

- `__max_speed`, `__capacity`, `__fuel_tank`, `__engine_oil_capacity`, `__luggage_spaces`: **только чтение** (через геттеры)
- `__speed`, `__distance`, `__passengers`, `__empty_seats`, `__seats_occupied`, `__fuel`, `__engine_oil`, `__luggage`: **чтение и запись** (через геттеры и сеттеры)

Требования к сеттерам:

- Для полей `__empty_seats` и `__seats_occupied` в сеттерах необходимо проверять, что передаваемое значение не превышает `__capacity` и неотрицательно.
- Для поля `__passengers` в сеттере необходимо проверять, что количество пассажиров (длина списка) не превышает `__capacity`.
- Для поля `__speed` в сеттере необходимо проверять, что заданная скорость не превышает `__max_speed` и неотрицательна.

- Для поля `__luggage` в сеттере необходимо проверять, что количество единиц багажа не превышает `__luggage_spaces`.
- Для полей `__fuel` и `__engine_oil` значения не должны превышать соответствующие ёмкости (`__fuel_tank` и `__engine_oil_capacity`) и должны быть неотрицательными.

Реализовать метод вывода всех установленных через сеттеры значений закрытых полей экземпляра класса. На основе этого класса реализовать три подхода к управлению доступом:

- (a) **С использованием объекта `property`:** Для каждого поля определить отдельные методы-геттеры и сеттеры (например, `get_speed`, `set_speed`), а затем создать свойство:

```
speed = property(get_speed, set_speed)
```

Этот код должен располагаться после определения соответствующих методов. Первый аргумент — геттер, второй — сеттер. Продемонстрировать работу на трёх экземплярах класса: создать `mycargo1`, `mycargo2`, `mycargo3`, установить значения через свойства и вывести их.

- (b) **С использованием декораторов `@property` и `@<имя>.setter`:** Создать новую версию класса, в которой геттеры оформляются с декоратором `@property`, а сеттеры — с декоратором вида `@speed.setter`. Имена методов должны совпадать и не содержать префиксов `get_/set_`. Пример:

```
@property
def speed(self):
    return self.__speed
@speed.setter
def speed(self, value):
    if 0 <= value <= self.__max_speed:
        self.__speed = value
    else:
        raise ValueError("Недопустимая скорость")
```

Продемонстрировать работу на трёх экземплярах и сделать выводы об оптимизации кода по сравнению с первым подходом.

- (c) **С использованием модуля `accessify`:** Установить модуль командой `pip install accessify` и импортировать:

```
from accessify import private, protected
```

Сделать поля `max_speed`, `capacity`, `fuel_tank`, `engine_oil_capacity`, `luggage_spaces` по-настоящему приватными с помощью функции `private` (например, как атрибуты класса до `__init__`). Удалить их из инициализатора. Проверки в сеттерах реализовать через вспомогательные методы, помеченные декоратором `@private`. Учитывать, что методы с `@private` нельзя вызывать из методов, использующих `@property`, поэтому для этой версии использовать только классические геттеры и сеттеры (`get_...`, `set_...`). Продемонстрировать, что попытка доступа извне

(включая `mycargo3._CargoPlane__max_speed`) не даёт результата, а вызов приватного метода или чтение приватного поля вызывает ошибку доступа.

Для всех трёх подходов создать по три экземпляра грузового самолёта, установить значения полей с учётом всех ограничений и вывести текущие значения всех полей каждого экземпляра.

30 Разработать класс `PassengerPlane`, который будет описывать модель пассажирского самолёта. В классе должны быть следующие поля с доступом уровня **private** (только внутри класса):

- `__speed`: скорость движения пассажирского самолёта
- `__distance`: расстояние, которое пассажирский самолёт пролетел
- `__max_speed`: максимальная разрешённая скорость движения пассажирского самолёта
- `__passengers`: список пассажиров
- `__capacity`: максимальная вместимость пассажиров в пассажирском самолёте
- `__empty_seats`: число свободных мест
- `__seats_occupied`: число занятых мест в пассажирском самолёте
- `__fuel_tank`: объём топливного бака
- `__fuel`: количество топлива в литрах
- `__engine_oil_capacity`: объём картера масла двигателя (литры)
- `__engine_oil`: количество моторного масла в литрах
- `__luggage_spaces`: количество багажных мест
- `__luggage`: багаж пассажирского самолёта

Уровень доступа к полям должен быть следующим:

- `__max_speed`, `__capacity`, `__fuel_tank`, `__engine_oil_capacity`, `__luggage_spaces`: только чтение (через геттеры)
- `__speed`, `__distance`, `__passengers`, `__empty_seats`, `__seats_occupied`, `__fuel`, `__engine_oil`, `__luggage`: чтение и запись (через геттеры и сеттеры)

Требования к сеттерам:

- Для полей `__empty_seats` и `__seats_occupied` в сеттерах необходимо проверять, что передаваемое значение не превышает `__capacity` и неотрицательно.
- Для поля `__passengers` в сеттере необходимо проверять, что количество пассажиров (длина списка) не превышает `__capacity`.
- Для поля `__speed` в сеттере необходимо проверять, что заданная скорость не превышает `__max_speed` и неотрицательна.
- Для поля `__luggage` в сеттере необходимо проверять, что количество единиц багажа не превышает `__luggage_spaces`.
- Для полей `__fuel` и `__engine_oil` значения не должны превышать соответствующие ёмкости (`__fuel_tank` и `__engine_oil_capacity`) и должны быть неотрицательными.

Реализовать метод вывода всех установленных через сеттеры значений закрытых полей экземпляра класса. На основе этого класса реализовать три подхода к управлению доступом:

- (a) **С использованием объекта `property`:** Для каждого поля определить отдельные методы-геттеры и сеттеры (например, `get_speed`, `set_speed`), а затем создать свойство:

```
speed = property(get_speed, set_speed)
```

Этот код должен располагаться после определения соответствующих методов. Первый аргумент — геттер, второй — сеттер. Продемонстрировать работу на трёх экземплярах класса: создать `mypass1`, `mypass2`, `mypass3`, установить значения через свойства и вывести их.

- (b) **С использованием декораторов `@property` и `@<имя>.setter`:** Создать новую версию класса, в которой геттеры оформляются с декоратором `@property`, а сеттеры — с декоратором вида `@speed.setter`. Имена методов должны совпадать и не содержать префиксов `get_/set_`. Пример:

```
@property
def speed(self):
    return self.__speed
@speed.setter
def speed(self, value):
    if 0 <= value <= self.__max_speed:
        self.__speed = value
    else:
        raise ValueError("Недопустимая скорость")
```

Продемонстрировать работу на трёх экземплярах и сделать выводы об оптимизации кода по сравнению с первым подходом.

- (c) **С использованием модуля `accessify`:** Установить модуль командой `pip install accessify` и импортировать:

```
from accessify import private, protected
```

Сделать поля `max_speed`, `capacity`, `fuel_tank`, `engine_oil_capacity`, `luggage_spaces` по-настоящему приватными с помощью функции `private` (например, как атрибуты класса до `__init__`). Удалить их из инициализатора. Проверки в сеттерах реализовать через вспомогательные методы, помеченные декоратором `@private`. Учитывать, что методы с `@private` нельзя вызывать из методов, использующих `@property`, поэтому для этой версии использовать только классические геттеры и сеттеры (`get_...`, `set_...`). Продемонстрировать, что попытка доступа извне (включая `mypass3._PassengerPlane__max_speed`) **не даёт результата**, а вызов приватного метода или чтение приватного поля вызывает ошибку доступа.

Для всех трёх подходов создать по три экземпляра пассажирского самолёта, установить значения полей с учётом всех ограничений и вывести текущие значения всех полей каждого экземпляра.

31 Разработать класс `MetroCar`, который будет описывать модель вагона метро. В классе должны быть следующие поля с доступом уровня **private** (только внутри класса):

- `__speed`: скорость движения вагона метро
- `__distance`: расстояние, которое вагон метро проехал
- `__max_speed`: максимальная разрешённая скорость движения вагона метро
- `__passengers`: список пассажиров
- `__capacity`: максимальная вместимость пассажиров в вагоне метро
- `__empty_seats`: число свободных мест
- `__seats_occupied`: число занятых мест в вагоне метро
- `__fuel_tank`: объём топливного бака
- `__fuel`: количество топлива в литрах
- `__engine_oil_capacity`: объём картера масла двигателя (литры)
- `__engine_oil`: количество моторного масла в литрах
- `__luggage_spaces`: количество багажных мест
- `__luggage`: багаж вагона метро

Уровень доступа к полям должен быть следующим:

- `__max_speed`, `__capacity`, `__fuel_tank`, `__engine_oil_capacity`, `__luggage_spaces`: **только чтение** (через геттеры)
- `__speed`, `__distance`, `__passengers`, `__empty_seats`, `__seats_occupied`, `__fuel`, `__engine_oil`, `__luggage`: **чтение и запись** (через геттеры и сеттеры)

Требования к сеттерам:

- Для полей `__empty_seats` и `__seats_occupied` в сеттерах необходимо проверять, что передаваемое значение не превышает `__capacity` и неотрицательно.
- Для поля `__passengers` в сеттере необходимо проверять, что количество пассажиров (длина списка) не превышает `__capacity`.
- Для поля `__speed` в сеттере необходимо проверять, что заданная скорость не превышает `__max_speed` и неотрицательна.
- Для поля `__luggage` в сеттере необходимо проверять, что количество единиц багажа не превышает `__luggage_spaces`.
- Для полей `__fuel` и `__engine_oil` значения не должны превышать соответствующие ёмкости (`__fuel_tank` и `__engine_oil_capacity`) и должны быть неотрицательными.

Реализовать метод вывода всех установленных через сеттеры значений закрытых полей экземпляра класса. На основе этого класса реализовать три подхода к управлению доступом:

- (a) **С использованием объекта `property`**: Для каждого поля определить отдельные методы-геттеры и сеттеры (например, `get_speed`, `set_speed`), а затем создать свойство:

```
speed = property(get_speed, set_speed)
```

Этот код должен располагаться после определения соответствующих методов. Первый аргумент — геттер, второй — сеттер. Продемонстрировать работу на трёх экземплярах класса: создать `mymetro1`, `mymetro2`, `mymetro3`, установить значения через свойства и вывести их.

- (b) **С использованием декораторов `@property` и `@<имя>.setter`:** Создать новую версию класса, в которой геттеры оформляются с декоратором `@property`, а сеттеры — с декоратором вида `@speed.setter`. Имена методов должны совпадать и не содержать префиксов `get_`/`set_`. Пример:

```
@property
def speed(self):
    return self.__speed
@speed.setter
def speed(self, value):
    if 0 <= value <= self.__max_speed:
        self.__speed = value
    else:
        raise ValueError("Недопустимая скорость")
```

Продемонстрировать работу на трёх экземплярах и сделать выводы об оптимизации кода по сравнению с первым подходом.

- (c) **С использованием модуля `accessify`:** Установить модуль командой `pip install accessify` и импортировать:

```
from accessify import private, protected
```

Сделать поля `max_speed`, `capacity`, `fuel_tank`, `engine_oil_capacity`, `luggage_spaces` по-настоящему приватными с помощью функции `private` (например, как атрибуты класса до `__init__`). Удалить их из инициализатора. Проверки в сеттерах реализовать через вспомогательные методы, помеченные декоратором `@private`. Учитывать, что методы с `@private` нельзя вызывать из методов, использующих `@property`, поэтому для этой версии использовать только классические геттеры и сеттеры (`get_...`, `set_...`). Продемонстрировать, что попытка доступа извне (включая `mymetro3._MetroCar__max_speed`) **не даёт результата**, а вызов приватного метода или чтение приватного поля вызывает ошибку доступа.

Для всех трёх подходов создать по три экземпляра вагона метро, установить значения полей с учётом всех ограничений и вывести текущие значения всех полей каждого экземпляра.

- 32 Разработать класс `Trolleybus`, который будет описывать модель троллейбуса. В классе должны быть следующие поля с доступом уровня **private** (только внутри класса):

- `__speed`: скорость движения троллейбуса
- `__distance`: расстояние, которое троллейбус проехал
- `__max_speed`: максимальная разрешённая скорость движения троллейбуса

- `__passengers`: список пассажиров
- `__capacity`: максимальная вместимость пассажиров в троллейбусе
- `__empty_seats`: число свободных мест
- `__seats_occupied`: число занятых мест в троллейбусе
- `__fuel_tank`: объём топливного бака
- `__fuel`: количество топлива в литрах
- `__engine_oil_capacity`: объём картера масла двигателя (литры)
- `__engine_oil`: количество моторного масла в литрах
- `__luggage_spaces`: количество багажных мест
- `__luggage`: багаж троллейбуса

Уровень доступа к полям должен быть следующим:

- `__max_speed`, `__capacity`, `__fuel_tank`, `__engine_oil_capacity`, `__luggage_spaces`: **только чтение** (через геттеры)
- `__speed`, `__distance`, `__passengers`, `__empty_seats`, `__seats_occupied`, `__fuel`, `__engine_oil`, `__luggage`: **чтение и запись** (через геттеры и сеттеры)

Требования к сеттерам:

- Для полей `__empty_seats` и `__seats_occupied` в сеттерах необходимо проверять, что передаваемое значение не превышает `__capacity` и неотрицательно.
- Для поля `__passengers` в сеттере необходимо проверять, что количество пассажиров (длина списка) не превышает `__capacity`.
- Для поля `__speed` в сеттере необходимо проверять, что заданная скорость не превышает `__max_speed` и неотрицательна.
- Для поля `__luggage` в сеттере необходимо проверять, что количество единиц багажа не превышает `__luggage_spaces`.
- Для полей `__fuel` и `__engine_oil` значения не должны превышать соответствующие ёмкости (`__fuel_tank` и `__engine_oil_capacity`) и должны быть неотрицательными.

Реализовать метод вывода всех установленных через сеттеры значений закрытых полей экземпляра класса. На основе этого класса реализовать три подхода к управлению доступом:

- (а) **С использованием объекта `property`**: Для каждого поля определить отдельные методы-геттеры и сеттеры (например, `get_speed`, `set_speed`), а затем создать свойство:

```
speed = property(get_speed, set_speed)
```

Этот код должен располагаться после определения соответствующих методов. Первый аргумент — геттер, второй — сеттер. Продемонстрировать работу на трёх экземплярах класса: создать `mytrol1`, `mytrol2`, `mytrol3`, установить значения через свойства и вывести их.

- (b) **С использованием декораторов `@property` и `@<имя>.setter`:** Создать новую версию класса, в которой геттеры оформляются с декоратором `@property`, а сеттеры — с декоратором вида `@speed.setter`. Имена методов должны совпадать и не содержать префиксов `get_`/`set_`. Пример:

```
@property
def speed(self):
    return self.__speed
@speed.setter
def speed(self, value):
    if 0 <= value <= self.__max_speed:
        self.__speed = value
    else:
        raise ValueError("Недопустимая скорость")
```

Продемонстрировать работу на трёх экземплярах и сделать выводы об оптимизации кода по сравнению с первым подходом.

- (c) **С использованием модуля `accessify`:** Установить модуль командой `pip install accessify` и импортировать:

```
from accessify import private, protected
```

Сделать поля `max_speed`, `capacity`, `fuel_tank`, `engine_oil_capacity`, `luggage_spaces` по-настоящему приватными с помощью функции `private` (например, как атрибуты класса до `__init__`). Удалить их из инициализатора. Проверки в сеттерах реализовать через вспомогательные методы, помеченные декоратором `@private`. Учитывать, что методы с `@private` нельзя вызывать из методов, использующих `@property`, поэтому для этой версии использовать только классические геттеры и сеттеры (`get_...`, `set_...`). Продемонстрировать, что попытка доступа извне (включая `mytrol3._Trolleybus__max_speed`) **не даёт результата**, а вызов приватного метода или чтение приватного поля вызывает ошибку доступа.

Для всех трёх подходов создать по три экземпляра троллейбуса, установить значения полей с учётом всех ограничений и вывести текущие значения всех полей каждого экземпляра.

- 33 Разработать класс `ElectricCar`, который будет описывать модель электромобиля. В классе должны быть следующие поля с доступом уровня **private** (только внутри класса):

- `__speed`: скорость движения электромобиля
- `__distance`: расстояние, которое электромобиль проехал
- `__max_speed`: максимальная разрешённая скорость движения электромобиля
- `__passengers`: список пассажиров
- `__capacity`: максимальная вместимость пассажиров в электромобиле
- `__empty_seats`: число свободных мест
- `__seats_occupied`: число занятых мест в электромобиле
- `__fuel_tank`: объём топливного бака

- `__fuel`: количество топлива в литрах
- `__engine_oil_capacity`: объём картера масла двигателя (литры)
- `__engine_oil`: количество моторного масла в литрах
- `__luggage_spaces`: количество багажных мест
- `__luggage`: багаж электромобиля

Уровень доступа к полям должен быть следующим:

- `__max_speed`, `__capacity`, `__fuel_tank`, `__engine_oil_capacity`, `__luggage_spaces`: **только чтение** (через геттеры)
- `__speed`, `__distance`, `__passengers`, `__empty_seats`, `__seats_occupied`, `__fuel`, `__engine_oil`, `__luggage`: **чтение и запись** (через геттеры и сеттеры)

Требования к сеттерам:

- Для полей `__empty_seats` и `__seats_occupied` в сеттерах необходимо проверять, что передаваемое значение не превышает `__capacity` и неотрицательно.
- Для поля `__passengers` в сеттере необходимо проверять, что количество пассажиров (длина списка) не превышает `__capacity`.
- Для поля `__speed` в сеттере необходимо проверять, что заданная скорость не превышает `__max_speed` и неотрицательна.
- Для поля `__luggage` в сеттере необходимо проверять, что количество единиц багажа не превышает `__luggage_spaces`.
- Для полей `__fuel` и `__engine_oil` значения не должны превышать соответствующие ёмкости (`__fuel_tank` и `__engine_oil_capacity`) и должны быть неотрицательными.

Реализовать метод вывода всех установленных через сеттеры значений закрытых полей экземпляра класса. На основе этого класса реализовать три подхода к управлению доступом:

- (a) **С использованием объекта `property`**: Для каждого поля определить отдельные методы-геттеры и сеттеры (например, `get_speed`, `set_speed`), а затем создать свойство:

```
speed = property(get_speed, set_speed)
```

Этот код должен располагаться после определения соответствующих методов. Первый аргумент — геттер, второй — сеттер. Продемонстрировать работу на трёх экземплярах класса: создать `myev1`, `myev2`, `myev3`, установить значения через свойства и вывести их.

- (b) **С использованием декораторов `@property` и `@<имя>.setter`**: Создать новую версию класса, в которой геттеры оформляются с декоратором `@property`, а сеттеры — с декоратором вида `@speed.setter`. Имена методов должны совпадать и не содержать префиксов `get_/set_`. Пример:


```

@property
def speed(self):
    return self.__speed
@speed.setter
def speed(self, value):
    if 0 <= value <= self.__max_speed:
        self.__speed = value
    else:
        raise ValueError("Недопустимая скорость")

```

Продемонстрировать работу на трёх экземплярах и сделать выводы об оптимизации кода по сравнению с первым подходом.

- (с) **С использованием модуля accessify:** Установить модуль командой `pip install accessify` и импортировать:

```
from accessify import private, protected
```

Сделать поля `max_speed`, `capacity`, `fuel_tank`, `engine_oil_capacity`, `luggage_spaces` по-настоящему приватными с помощью функции `private` (например, как атрибуты класса до `__init__`). Удалить их из инициализатора. Проверки в сеттерах реализовать через вспомогательные методы, помеченные декоратором `@private`. Учитывать, что методы с `@private` нельзя вызывать из методов, использующих `@property`, поэтому для этой версии использовать только классические геттеры и сеттеры (`get_...`, `set_...`). Продемонстрировать, что попытка доступа извне (включая `myev3._ElectricCar.__max_speed`) **не даёт результата**, а вызов приватного метода или чтение приватного поля вызывает ошибку доступа.

Для всех трёх подходов создать по три экземпляра электромобиля, установить значения полей с учётом всех ограничений и вывести текущие значения всех полей каждого экземпляра.

- 34 Разработать класс `Hydrofoil`, который будет описывать модель гидроfoilа. В классе должны быть следующие поля с доступом уровня **private** (только внутри класса):

- `__speed`: скорость движения гидроfoilа
- `__distance`: расстояние, которое гидроfoil прошёл
- `__max_speed`: максимальная разрешённая скорость движения гидроfoilа
- `__passengers`: список пассажиров
- `__capacity`: максимальная вместимость пассажиров на гидроfoilе
- `__empty_seats`: число свободных мест
- `__seats_occupied`: число занятых мест на гидроfoilе
- `__fuel_tank`: объём топливного бака
- `__fuel`: количество топлива в литрах
- `__engine_oil_capacity`: объём картера масла двигателя (литры)
- `__engine_oil`: количество моторного масла в литрах
- `__luggage_spaces`: количество багажных мест

- `__luggage`: багаж гидрофойла

Уровень доступа к полям должен быть следующим:

- `__max_speed`, `__capacity`, `__fuel_tank`, `__engine_oil_capacity`, `__luggage_spaces`: **только чтение** (через геттеры)
- `__speed`, `__distance`, `__passengers`, `__empty_seats`, `__seats_occupied`, `__fuel`, `__engine_oil`, `__luggage`: **чтение и запись** (через геттеры и сеттеры)

Требования к сеттерам:

- Для полей `__empty_seats` и `__seats_occupied` в сеттерах необходимо проверять, что передаваемое значение не превышает `__capacity` и неотрицательно.
- Для поля `__passengers` в сеттере необходимо проверять, что количество пассажиров (длина списка) не превышает `__capacity`.
- Для поля `__speed` в сеттере необходимо проверять, что заданная скорость не превышает `__max_speed` и неотрицательна.
- Для поля `__luggage` в сеттере необходимо проверять, что количество единиц багажа не превышает `__luggage_spaces`.
- Для полей `__fuel` и `__engine_oil` значения не должны превышать соответствующие ёмкости (`__fuel_tank` и `__engine_oil_capacity`) и должны быть неотрицательными.

Реализовать метод вывода всех установленных через сеттеры значений закрытых полей экземпляра класса. На основе этого класса реализовать три подхода к управлению доступом:

- (a) **С использованием объекта `property`**: Для каждого поля определить отдельные методы-геттеры и сеттеры (например, `get_speed`, `set_speed`), а затем создать свойство:

```
speed = property(get_speed, set_speed)
```

Этот код должен располагаться после определения соответствующих методов. Первый аргумент — геттер, второй — сеттер. Продемонстрировать работу на трёх экземплярах класса: создать `myhydro1`, `myhydro2`, `myhydro3`, установить значения через свойства и вывести их.

- (b) **С использованием декораторов `@property` и `@<имя>.setter`**: Создать новую версию класса, в которой геттеры оформляются с декоратором `@property`, а сеттеры — с декоратором вида `@speed.setter`. Имена методов должны совпадать и не содержать префиксов `get_/set_`. Пример:

```
@property
def speed(self):
    return self.__speed
@speed.setter
def speed(self, value):
    if 0 <= value <= self.__max_speed:
        self.__speed = value
    else:
```

```
raise ValueError("Недопустимая скорость")
```

Продemonстрировать работу на трёх экземплярах и сделать выводы об оптимизации кода по сравнению с первым подходом.

- (с) **С использованием модуля accessify:** Установить модуль командой `pip install accessify` и импортировать:

```
from accessify import private, protected
```

Сделать поля `max_speed`, `capacity`, `fuel_tank`, `engine_oil_capacity`, `luggage_spaces` по-настоящему приватными с помощью функции `private` (например, как атрибуты класса до `__init__`). Удалить их из инициализатора. Проверки в сеттерах реализовать через вспомогательные методы, помеченные декоратором `@private`. Учитывать, что методы с `@private` нельзя вызывать из методов, использующих `@property`, поэтому для этой версии использовать только классические геттеры и сеттеры (`get_...`, `set_...`). Продemonстрировать, что попытка доступа извне (включая `myhydro3._Hydrofoil__max_speed`) **не даёт результата**, а вызов приватного метода или чтение приватного поля вызывает ошибку доступа.

Для всех трёх подходов создать по три экземпляра гидроfoilа, установить значения полей с учётом всех ограничений и вывести текущие значения всех полей каждого экземпляра.

- 35 Разработать класс `Segway`, который будет описывать модель сигвея. В классе должны быть следующие поля с доступом уровня **private** (только внутри класса):

- `__speed`: скорость движения сигвея
- `__distance`: расстояние, которое сигвей проехал
- `__max_speed`: максимальная разрешённая скорость движения сигвея
- `__passengers`: список пассажиров
- `__capacity`: максимальная вместимость пассажиров на сигвее
- `__empty_seats`: число свободных мест
- `__seats_occupied`: число занятых мест на сигвее
- `__fuel_tank`: объём топливного бака
- `__fuel`: количество топлива в литрах
- `__engine_oil_capacity`: объём картера масла двигателя (литры)
- `__engine_oil`: количество моторного масла в литрах
- `__luggage_spaces`: количество багажных мест
- `__luggage`: багаж сигвея

Уровень доступа к полям должен быть следующим:

- `__max_speed`, `__capacity`, `__fuel_tank`, `__engine_oil_capacity`, `__luggage_spaces`: **только чтение** (через геттеры)
- `__speed`, `__distance`, `__passengers`, `__empty_seats`, `__seats_occupied`, `__fuel`, `__engine_oil`, `__luggage`: **чтение и запись** (через геттеры и сеттеры)

Требования к сеттерам:

- Для полей `__empty_seats` и `__seats_occupied` в сеттерах необходимо проверять, что передаваемое значение не превышает `__capacity` и неотрицательно.
- Для поля `__passengers` в сеттере необходимо проверять, что количество пассажиров (длина списка) не превышает `__capacity`.
- Для поля `__speed` в сеттере необходимо проверять, что заданная скорость не превышает `__max_speed` и неотрицательна.
- Для поля `__luggage` в сеттере необходимо проверять, что количество единиц багажа не превышает `__luggage_spaces`.
- Для полей `__fuel` и `__engine_oil` значения не должны превышать соответствующие ёмкости (`__fuel_tank` и `__engine_oil_capacity`) и должны быть неотрицательными.

Реализовать метод вывода всех установленных через сеттеры значений закрытых полей экземпляра класса. На основе этого класса реализовать три подхода к управлению доступом:

- (a) **С использованием объекта `property`:** Для каждого поля определить отдельные методы-геттеры и сеттеры (например, `get_speed`, `set_speed`), а затем создать свойство:

```
speed = property(get_speed, set_speed)
```

Этот код должен располагаться после определения соответствующих методов. Первый аргумент — геттер, второй — сеттер. Продемонстрировать работу на трёх экземплярах класса: создать `myseg1`, `myseg2`, `myseg3`, установить значения через свойства и вывести их.

- (b) **С использованием декораторов `@property` и `@<имя>.setter`:** Создать новую версию класса, в которой геттеры оформляются с декоратором `@property`, а сеттеры — с декоратором вида `@speed.setter`. Имена методов должны совпадать и не содержать префиксов `get_/set_`. Пример:

```
@property
def speed(self):
    return self.__speed
@speed.setter
def speed(self, value):
    if 0 <= value <= self.__max_speed:
        self.__speed = value
    else:
        raise ValueError("Недопустимая скорость")
```

Продемонстрировать работу на трёх экземплярах и сделать выводы об оптимизации кода по сравнению с первым подходом.

- (c) **С использованием модуля `accessify`:** Установить модуль командой `pip install accessify` и импортировать:

```
from accessify import private, protected
```

Сделать поля `max_speed`, `capacity`, `fuel_tank`, `engine_oil_capacity`, `luggage_spaces` по-настоящему приватными с помощью функции `private` (например, как атрибуты класса до `__init__`). Удалить их из инициализатора. Проверки в сеттерах реализовать через вспомогательные методы, помеченные декоратором `@private`. Учитывать, что методы с `@private` нельзя вызывать из методов, использующих `@property`, поэтому для этой версии использовать только классические геттеры и сеттеры (`get_...`, `set_...`). Продемонстрировать, что попытка доступа извне (включая `myseg3._Segway__max_speed`) **не даёт результата**, а вызов приватного метода или чтение приватного поля вызывает ошибку доступа.

Для всех трёх подходов создать по три экземпляра сигвея, установить значения полей с учётом всех ограничений и вывести текущие значения всех полей каждого экземпляра.

2.6.5 Задача 2

Инструкция: Напишите функцию и соответствующие unit-тесты, покрывающие все важные случаи. Для заданий с ветвлениями (`if/elif/else`) обязательно проверяйте все ветви.

Пояснения:

- **Triangle types:**
 - `equilateral` — все стороны равны
 - `isosceles` — две стороны равны
 - `scalene` — все стороны разные
 - `invalid` — невозможно построить треугольник
 - **BMI (Body Mass Index):** индекс массы тела. Категории: `Underweight`, `Normal`, `Overweight`, `Obese`
 - **Palindrome:** строка или число, читающееся одинаково слева направо и справа налево
 - **Perfect number:** число, равное сумме своих делителей, исключая само число
 - **Triangle angles:**
 - `acute` — все углы $< 90^\circ$
 - `right` — один угол $= 90^\circ$
 - `obtuse` — один угол $> 90^\circ$
 - `invalid` — треугольник не существует
 - **Traffic fine:** штраф за превышение скорости. Функция должна учитывать разные зоны (`residential`, `city`, `highway`) и уровни превышения скорости.
1. `classify_triangle(a, b, c)` — возвращает тип треугольника.
 2. `classify_number(n)` — возвращает `"positive even,,", "positive odd,,", "negative even,,", "negative odd,,", "zero,,",`
 3. `middle_value(a, b, c)` — возвращает среднее (не арифметическое) число среди трёх, через сравнения.

4. `median_of_three(a, b, c)` — медиана трёх чисел через `if/elif/else`.
5. `is_leap_year(year)` — проверяет високосный год (делится на 4, но не на 100, или на 400).
6. `bmi_category(weight, height)` — возвращает категорию BMI.
7. `categorize_temperature(temp)` — диапазоны: "freezing, $\leq 0^{\circ}\text{C}$, "cold,, 1–10 $^{\circ}\text{C}$, "cool,, 11–20 $^{\circ}\text{C}$, "warm,, 21–30 $^{\circ}\text{C}$, "hot,, $>30^{\circ}\text{C}$.
8. `triangle_area_type(a, b, c)` — возвращает "acute,, "right,, "obtuse,, или "invalid,,
9. `quadrant(x, y)` — возвращает номер четверти (1–4) или "origin,,/"axis,,
10. `days_in_month(month, leap)` — возвращает число дней в месяце; `leap = True` для високосного года.
11. `traffic_fine(speed, zone)` — вычисляет штраф за превышение скорости.
 - **speed** — скорость автомобиля (км/ч)
 - **zone** — тип зоны: "residential,, "city,, "highway,,
 - **правила:**
 - "residential,,: превышение >20 км/ч $\rightarrow 200$, >10 км/ч $\rightarrow 100$, иначе 0
 - "city,,: превышение $>30 \rightarrow 150$, $>15 \rightarrow 75$, иначе 0
 - "highway,,: превышение $>40 \rightarrow 100$, $>20 \rightarrow 50$, иначе 0
 - некорректная зона $\rightarrow$ "invalid zone,,
 - **требования:** использовать ветвления `if/elif/else`, проверить все сценарии превышения и отсутствия превышения.
12. `compare_three_numbers(a, b, c)` — возвращает "all equal,, "all different,, или "two equal,,
13. `max_digit(n)` — наибольшая цифра числа.
14. `triangle_angle_category(a, b, c)` — вычисляет углы через теорему косинусов и возвращает "acute,, "right,, "obtuse,, "invalid,,
15. `next_day(day, month, leap)` — возвращает следующий день месяца, учитывая количество дней.
16. `is_inside_rectangle(x, y, x1, y1, x2, y2)` — проверка попадания точки в прямоугольник.
17. `discount(price)` — возврат цены со скидкой по условию ($>1000 \rightarrow 10$)
18. `closest_to_zero(lst)` — элемент списка, ближайший к нулю.
19. `season(month)` — "Winter,, "Spring,, "Summer,, "Autumn,,
20. `simple_calculator(a, b, op)` — +, -, *, /; деление на 0 $\rightarrow$ "error,,
21. `sum_positive(lst)` — сумма положительных чисел.
22. `sum_even(lst)` — сумма чётных чисел.

- 23. `sum_odd(lst)` — сумма нечётных чисел.
- 24. `reverse_signs(lst)` — меняет знак всех элементов.
- 25. `nearest_multiple(n, m)` — ближайшее к n кратное m .
- 26. `sort_three(a, b, c)` — тройка чисел в порядке возрастания через `if/elif/else`.
- 27. `validate_password(password)` — True, если длина ≥ 8 , есть цифра и заглавная буква.
- 28. `is_perfect(n)` — True, если число совершенное.
- 29. `sum_digits(n)` — сумма цифр числа.
- 30. `count_vowels(s)` — количество гласных.
- 31. `is_palindrome(s)` — True, если строка читается одинаково слева направо и справа налево.
- 32. `remove_duplicates(lst)` — возвращает список без повторов.
- 33. `factorial_iterative(n)` — факториал через цикл, без рекурсии.
- 34. `fizz_buzz(n)` — числа от 1 до n с заменой кратных 3 $\rightarrow$ "Fizz,,", кратных 5 $\rightarrow$ "Buzz,,", кратных 15 $\rightarrow$ "FizzBuzz,,

2.7 Семинар «Наследование #2» (2 часа)

2.7.1 Задача 1.

- 1 Паспорт. Класс `ForeignPassport` является производным от класса `Passport`. Метод `PrintInfo` существует в обоих классах. `PassportList` представляет собой список, содержащий объекты обоих классов.

Инструкция:

- (a) Создайте класс `Passport`, который будет базовым классом для класса `ForeignPassport`. В конструкторе класса `Passport` задайте параметры `first_name`, `last_name`, `country`, `date_of_birth` и `numb_of_pasport`.
- (b) В классе `Passport` создайте метод `PrintInfo`, который будет выводить информацию о паспорте.
- (c) Создайте класс `ForeignPassport`, который будет наследоваться от класса `Passport`. В конструкторе класса `ForeignPassport` добавьте параметр `visa`.
- (d) В классе `ForeignPassport` переопределите метод `PrintInfo`, чтобы он выводил информацию о паспорте и визу.
- (e) В основной части программы создайте список `PassportList` и добавьте в него объекты классов `Passport` и `ForeignPassport`.
- (f) Для каждого объекта в списке `PassportList` вызовите метод `PrintInfo`.

- 2 Автомобиль. Класс `ElectricCar` является производным от класса `Car`. Метод `DisplayDetails` существует в обоих классах. `CarFleet` представляет собой список, содержащий объекты обоих классов.

Инструкция:

- (a) Создайте класс `Car`, который будет базовым классом для класса `ElectricCar`. В конструкторе класса `Car` задайте параметры `brand`, `model`, `year` и `fuel_type`.
- (b) В классе `Car` создайте метод `DisplayDetails`, который будет выводить информацию об автомобиле.
- (c) Создайте класс `ElectricCar`, который будет наследоваться от класса `Car`. В конструкторе класса `ElectricCar` добавьте параметр `battery_capacity`.
- (d) В классе `ElectricCar` переопределите метод `DisplayDetails`, чтобы он выводил информацию об автомобиле и ёмкости батареи.
- (e) В основной части программы создайте список `CarFleet` и добавьте в него объекты классов `Car` и `ElectricCar`.
- (f) Для каждого объекта в списке `CarFleet` вызовите метод `DisplayDetails`.

- 3 Животное. Класс `Bird` является производным от класса `Animal`. Метод `MakeSound` существует в обоих классах. `Zoo` представляет собой список, содержащий объекты обоих классов.

Инструкция:

- (a) Создайте класс `Animal`, который будет базовым классом для класса `Bird`. В конструкторе класса `Animal` задайте параметры `name`, `species`, `age` и `habitat`.
- (b) В классе `Animal` создайте метод `MakeSound`, который будет выводить общий звук животного.

- (c) Создайте класс `Bird`, который будет наследоваться от класса `Animal`. В конструкторе класса `Bird` добавьте параметр `wing_span`.
- (d) В классе `Bird` переопределите метод `MakeSound`, чтобы он выводил характерный звук птицы.
- (e) В основной части программы создайте список `Zoo` и добавьте в него объекты классов `Animal` и `Bird`.
- (f) Для каждого объекта в списке `Zoo` вызовите метод `MakeSound`.

4 Сотрудник. Класс `Manager` является производным от класса `Employee`. Метод `ShowProfile` существует в обоих классах. `StaffList` представляет собой список, содержащий объекты обоих классов.

Инструкция:

- (a) Создайте класс `Employee`, который будет базовым классом для класса `Manager`. В конструкторе класса `Employee` задайте параметры `first_name`, `last_name`, `position` и `salary`.
- (b) В классе `Employee` создайте метод `ShowProfile`, который будет выводить информацию о сотруднике.
- (c) Создайте класс `Manager`, который будет наследоваться от класса `Employee`. В конструкторе класса `Manager` добавьте параметр `department`.
- (d) В классе `Manager` переопределите метод `ShowProfile`, чтобы он выводил информацию о сотруднике и управляемом отделе.
- (e) В основной части программы создайте список `StaffList` и добавьте в него объекты классов `Employee` и `Manager`.
- (f) Для каждого объекта в списке `StaffList` вызовите метод `ShowProfile`.

5 Фигура. Класс `Circle` является производным от класса `Shape`. Метод `CalculateArea` существует в обоих классах. `ShapesCollection` представляет собой список, содержащий объекты обоих классов.

Инструкция:

- (a) Создайте класс `Shape`, который будет базовым классом для класса `Circle`. В конструкторе класса `Shape` задайте параметр `name`.
- (b) В классе `Shape` создайте метод `CalculateArea`, который будет возвращать значение 0 (заглушка).
- (c) Создайте класс `Circle`, который будет наследоваться от класса `Shape`. В конструкторе класса `Circle` добавьте параметр `radius`.
- (d) В классе `Circle` переопределите метод `CalculateArea`, чтобы он вычислял и возвращал площадь круга.
- (e) В основной части программы создайте список `ShapesCollection` и добавьте в него объекты классов `Shape` и `Circle`.
- (f) Для каждого объекта в списке `ShapesCollection` вызовите метод `CalculateArea` и выведите результат.

6 Книга. Класс `Textbook` является производным от класса `Book`. Метод `GetInfo` существует в обоих классах. `Library` представляет собой список, содержащий объекты обоих классов.

Инструкция:

- (a) Создайте класс `Book`, который будет базовым классом для класса `Textbook`. В конструкторе класса `Book` задайте параметры `title`, `author`, `isbn` и `year`.
- (b) В классе `Book` создайте метод `GetInfo`, который будет выводить информацию о книге.
- (c) Создайте класс `Textbook`, который будет наследоваться от класса `Book`. В конструкторе класса `Textbook` добавьте параметр `subject`.
- (d) В классе `Textbook` переопределите метод `GetInfo`, чтобы он выводил информацию о книге и учебном предмете.
- (e) В основной части программы создайте список `Library` и добавьте в него объекты классов `Book` и `Textbook`.
- (f) Для каждого объекта в списке `Library` вызовите метод `GetInfo`.

7 Транспорт. Класс `Bicycle` является производным от класса `Vehicle`. Метод `Describe` существует в обоих классах. `Vehicles` представляет собой список, содержащий объекты обоих классов.

Инструкция:

- (a) Создайте класс `Vehicle`, который будет базовым классом для класса `Bicycle`. В конструкторе класса `Vehicle` задайте параметры `make`, `model`, `year` и `type`.
- (b) В классе `Vehicle` создайте метод `Describe`, который будет выводить общую информацию о транспорте.
- (c) Создайте класс `Bicycle`, который будет наследоваться от класса `Vehicle`. В конструкторе класса `Bicycle` добавьте параметр `frame_size`.
- (d) В классе `Bicycle` переопределите метод `Describe`, чтобы он выводил информацию о велосипеде, включая размер рамы.
- (e) В основной части программы создайте список `Vehicles` и добавьте в него объекты классов `Vehicle` и `Bicycle`.
- (f) Для каждого объекта в списке `Vehicles` вызовите метод `Describe`.

8 Продукт. Класс `PerishableProduct` является производным от класса `Product`. Метод `Display` существует в обоих классах. `Inventory` представляет собой список, содержащий объекты обоих классов.

Инструкция:

- (a) Создайте класс `Product`, который будет базовым классом для класса `PerishableProduct`. В конструкторе класса `Product` задайте параметры `name`, `price`, `category` и `barcode`.
- (b) В классе `Product` создайте метод `Display`, который будет выводить информацию о продукте.
- (c) Создайте класс `PerishableProduct`, который будет наследоваться от класса `Product`. В конструкторе класса `PerishableProduct` добавьте параметр `expiry_date`.
- (d) В классе `PerishableProduct` переопределите метод `Display`, чтобы он выводил информацию о продукте и сроке годности.
- (e) В основной части программы создайте список `Inventory` и добавьте в него объекты классов `Product` и `PerishableProduct`.
- (f) Для каждого объекта в списке `Inventory` вызовите метод `Display`.

- 9 Устройство. Класс **Smartphone** является производным от класса **Device**. Метод **DeviceInfo** существует в обоих классах. **Gadgets** представляет собой список, содержащий объекты обоих классов.

Инструкция:

- (a) Создайте класс **Device**, который будет базовым классом для класса **Smartphone**. В конструкторе класса **Device** задайте параметры **brand**, **model**, **serial_number** и **power_source**.
 - (b) В классе **Device** создайте метод **DeviceInfo**, который будет выводить информацию об устройстве.
 - (c) Создайте класс **Smartphone**, который будет наследоваться от класса **Device**. В конструкторе класса **Smartphone** добавьте параметр **os_version**.
 - (d) В классе **Smartphone** переопределите метод **DeviceInfo**, чтобы он выводил информацию об устройстве и версии ОС.
 - (e) В основной части программы создайте список **Gadgets** и добавьте в него объекты классов **Device** и **Smartphone**.
 - (f) Для каждого объекта в списке **Gadgets** вызовите метод **DeviceInfo**.
- 10 Студент. Класс **GraduateStudent** является производным от класса **Student**. Метод **ShowRecord** существует в обоих классах. **StudentsList** представляет собой список, содержащий объекты обоих классов.

Инструкция:

- (a) Создайте класс **Student**, который будет базовым классом для класса **GraduateStudent**. В конструкторе класса **Student** задайте параметры **first_name**, **last_name**, **student_id** и **major**.
 - (b) В классе **Student** создайте метод **ShowRecord**, который будет выводить информацию о студенте.
 - (c) Создайте класс **GraduateStudent**, который будет наследоваться от класса **Student**. В конструкторе класса **GraduateStudent** добавьте параметр **thesis_topic**.
 - (d) В классе **GraduateStudent** переопределите метод **ShowRecord**, чтобы он выводил информацию о студенте и теме диплома.
 - (e) В основной части программы создайте список **StudentsList** и добавьте в него объекты классов **Student** и **GraduateStudent**.
 - (f) Для каждого объекта в списке **StudentsList** вызовите метод **ShowRecord**.
- 11 Счёт. Класс **SavingsAccount** является производным от класса **BankAccount**. Метод **AccountSummary** существует в обоих классах. **Accounts** представляет собой список, содержащий объекты обоих классов.

Инструкция:

- (a) Создайте класс **BankAccount**, который будет базовым классом для класса **SavingsAccount**. В конструкторе класса **BankAccount** задайте параметры **account_holder**, **account_number** и **balance**.
- (b) В классе **BankAccount** создайте метод **AccountSummary**, который будет выводить информацию о счёте.

- (c) Создайте класс `SavingsAccount`, который будет наследоваться от класса `BankAccount`. В конструкторе класса `SavingsAccount` добавьте параметр `interest_rate`.
- (d) В классе `SavingsAccount` переопределите метод `AccountSummary`, чтобы он выводил информацию о счёте и процентной ставке.
- (e) В основной части программы создайте список `Accounts` и добавьте в него объекты классов `BankAccount` и `SavingsAccount`.
- (f) Для каждого объекта в списке `Accounts` вызовите метод `AccountSummary`.

12 Инструмент. Класс `PowerDrill` является производным от класса `Tool`. Метод `ToolInfo` существует в обоих классах. `Toolbox` представляет собой список, содержащий объекты обоих классов.

Инструкция:

- (a) Создайте класс `Tool`, который будет базовым классом для класса `PowerDrill`. В конструкторе класса `Tool` задайте параметры `name`, `brand`, `weight` и `material`.
- (b) В классе `Tool` создайте метод `ToolInfo`, который будет выводить информацию об инструменте.
- (c) Создайте класс `PowerDrill`, который будет наследоваться от класса `Tool`. В конструкторе класса `PowerDrill` добавьте параметр `voltage`.
- (d) В классе `PowerDrill` переопределите метод `ToolInfo`, чтобы он выводил информацию об инструменте и напряжении.
- (e) В основной части программы создайте список `Toolbox` и добавьте в него объекты классов `Tool` и `PowerDrill`.
- (f) Для каждого объекта в списке `Toolbox` вызовите метод `ToolInfo`.

13 Файл. Класс `AudioFile` является производным от класса `File`. Метод `FileInfo` существует в обоих классах. `FileList` представляет собой список, содержащий объекты обоих классов.

Инструкция:

- (a) Создайте класс `File`, который будет базовым классом для класса `AudioFile`. В конструкторе класса `File` задайте параметры `filename`, `size`, `extension` и `created_date`.
- (b) В классе `File` создайте метод `FileInfo`, который будет выводить информацию о файле.
- (c) Создайте класс `AudioFile`, который будет наследоваться от класса `File`. В конструкторе класса `AudioFile` добавьте параметр `duration`.
- (d) В классе `AudioFile` переопределите метод `FileInfo`, чтобы он выводил информацию о файле и длительности аудио.
- (e) В основной части программы создайте список `FileList` и добавьте в него объекты классов `File` и `AudioFile`.
- (f) Для каждого объекта в списке `FileList` вызовите метод `FileInfo`.

14 Пользователь. Класс `PremiumUser` является производным от класса `User`. Метод `UserProfile` существует в обоих классах. `UserDirectory` представляет собой список, содержащий объекты обоих классов.

Инструкция:

- (a) Создайте класс `User`, который будет базовым классом для класса `PremiumUser`. В конструкторе класса `User` задайте параметры `username`, `email`, `registration_date` и `status`.
- (b) В классе `User` создайте метод `UserProfile`, который будет выводить информацию о пользователе.
- (c) Создайте класс `PremiumUser`, который будет наследоваться от класса `User`. В конструкторе класса `PremiumUser` добавьте параметр `subscription_end`.
- (d) В классе `PremiumUser` переопределите метод `UserProfile`, чтобы он выводил информацию о пользователе и дате окончания подписки.
- (e) В основной части программы создайте список `UserDirectory` и добавьте в него объекты классов `User` и `PremiumUser`.
- (f) Для каждого объекта в списке `UserDirectory` вызовите метод `UserProfile`.

15 Игра. Класс `MultiplayerGame` является производным от класса `Game`. Метод `GameDetails` существует в обоих классах. `GameLibrary` представляет собой список, содержащий объекты обоих классов.

Инструкция:

- (a) Создайте класс `Game`, который будет базовым классом для класса `MultiplayerGame`. В конструкторе класса `Game` задайте параметры `title`, `genre`, `release_year` и `developer`.
- (b) В классе `Game` создайте метод `GameDetails`, который будет выводить информацию об игре.
- (c) Создайте класс `MultiplayerGame`, который будет наследоваться от класса `Game`. В конструкторе класса `MultiplayerGame` добавьте параметр `max_players`.
- (d) В классе `MultiplayerGame` переопределите метод `GameDetails`, чтобы он выводил информацию об игре и максимальном числе игроков.
- (e) В основной части программы создайте список `GameLibrary` и добавьте в него объекты классов `Game` и `MultiplayerGame`.
- (f) Для каждого объекта в списке `GameLibrary` вызовите метод `GameDetails`.

16 Документ. Класс `Invoice` является производным от класса `Document`. Метод `PrintDocument` существует в обоих классах. `DocumentArchive` представляет собой список, содержащий объекты обоих классов.

Инструкция:

- (a) Создайте класс `Document`, который будет базовым классом для класса `Invoice`. В конструкторе класса `Document` задайте параметры `doc_id`, `title`, `author` и `creation_date`.
- (b) В классе `Document` создайте метод `PrintDocument`, который будет выводить информацию о документе.
- (c) Создайте класс `Invoice`, который будет наследоваться от класса `Document`. В конструкторе класса `Invoice` добавьте параметр `amount`.
- (d) В классе `Invoice` переопределите метод `PrintDocument`, чтобы он выводил информацию о документе и сумму счёта.

- (e) В основной части программы создайте список `DocumentArchive` и добавьте в него объекты классов `Document` и `Invoice`.
- (f) Для каждого объекта в списке `DocumentArchive` вызовите метод `PrintDocument`.

17 Покупка. Класс `OnlineOrder` является производным от класса `Purchase`. Метод `OrderSummary` существует в обоих классах. `Orders` представляет собой список, содержащий объекты обоих классов.

Инструкция:

- (a) Создайте класс `Purchase`, который будет базовым классом для класса `OnlineOrder`. В конструкторе класса `Purchase` задайте параметры `customer_name`, `item`, `price` и `purchase_date`.
- (b) В классе `Purchase` создайте метод `OrderSummary`, который будет выводить информацию о покупке.
- (c) Создайте класс `OnlineOrder`, который будет наследоваться от класса `Purchase`. В конструкторе класса `OnlineOrder` добавьте параметр `delivery_address`.
- (d) В классе `OnlineOrder` переопределите метод `OrderSummary`, чтобы он выводил информацию о покупке и адресе доставки.
- (e) В основной части программы создайте список `Orders` и добавьте в него объекты классов `Purchase` и `OnlineOrder`.
- (f) Для каждого объекта в списке `Orders` вызовите метод `OrderSummary`.

18 Клиент. Класс `VipClient` является производным от класса `Client`. Метод `ClientCard` существует в обоих классах. `Clients` представляет собой список, содержащий объекты обоих классов.

Инструкция:

- (a) Создайте класс `Client`, который будет базовым классом для класса `VipClient`. В конструкторе класса `Client` задайте параметры `client_id`, `full_name`, `phone` и `email`.
- (b) В классе `Client` создайте метод `ClientCard`, который будет выводить информацию о клиенте.
- (c) Создайте класс `VipClient`, который будет наследоваться от класса `Client`. В конструкторе класса `VipClient` добавьте параметр `discount_rate`.
- (d) В классе `VipClient` переопределите метод `ClientCard`, чтобы он выводил информацию о клиенте и размере скидки.
- (e) В основной части программы создайте список `Clients` и добавьте в него объекты классов `Client` и `VipClient`.
- (f) Для каждого объекта в списке `Clients` вызовите метод `ClientCard`.

19 Событие. Класс `Conference` является производным от класса `Event`. Метод `EventInfo` существует в обоих классах. `EventsSchedule` представляет собой список, содержащий объекты обоих классов.

Инструкция:

- (a) Создайте класс `Event`, который будет базовым классом для класса `Conference`. В конструкторе класса `Event` задайте параметры `title`, `location`, `start_date` и `duration_days`.

- (b) В классе **Event** создайте метод **EventInfo**, который будет выводить информацию о событии.
- (c) Создайте класс **Conference**, который будет наследоваться от класса **Event**. В конструкторе класса **Conference** добавьте параметр **speakers_list**.
- (d) В классе **Conference** переопределите метод **EventInfo**, чтобы он выводил информацию о событии и списке спикеров.
- (e) В основной части программы создайте список **EventsSchedule** и добавьте в него объекты классов **Event** и **Conference**.
- (f) Для каждого объекта в списке **EventsSchedule** вызовите метод **EventInfo**.

20 Платёж. Класс **CreditCardPayment** является производным от класса **Payment**. Метод **PaymentDetails** существует в обоих классах. **PaymentsList** представляет собой список, содержащий объекты обоих классов.

Инструкция:

- (a) Создайте класс **Payment**, который будет базовым классом для класса **CreditCardPayment**. В конструкторе класса **Payment** задайте параметры **amount**, **currency**, **payment_date** и **status**.
- (b) В классе **Payment** создайте метод **PaymentDetails**, который будет выводить информацию о платеже.
- (c) Создайте класс **CreditCardPayment**, который будет наследоваться от класса **Payment**. В конструкторе класса **CreditCardPayment** добавьте параметр **card_last_digits**.
- (d) В классе **CreditCardPayment** переопределите метод **PaymentDetails**, чтобы он выводил информацию о платеже и последние цифры карты.
- (e) В основной части программы создайте список **PaymentsList** и добавьте в него объекты классов **Payment** и **CreditCardPayment**.
- (f) Для каждого объекта в списке **PaymentsList** вызовите метод **PaymentDetails**.

21 Рецепт. Класс **VeganRecipe** является производным от класса **Recipe**. Метод **ShowRecipe** существует в обоих классах. **Cookbook** представляет собой список, содержащий объекты обоих классов.

Инструкция:

- (a) Создайте класс **Recipe**, который будет базовым классом для класса **VeganRecipe**. В конструкторе класса **Recipe** задайте параметры **name**, **prep_time**, **servings** и **ingredients**.
- (b) В классе **Recipe** создайте метод **ShowRecipe**, который будет выводить информацию о рецепте.
- (c) Создайте класс **VeganRecipe**, который будет наследоваться от класса **Recipe**. В конструкторе класса **VeganRecipe** добавьте параметр **is_gluten_free**.
- (d) В классе **VeganRecipe** переопределите метод **ShowRecipe**, чтобы он выводил информацию о рецепте и отметку о безглютеновости.
- (e) В основной части программы создайте список **Cookbook** и добавьте в него объекты классов **Recipe** и **VeganRecipe**.
- (f) Для каждого объекта в списке **Cookbook** вызовите метод **ShowRecipe**.

- 22 Заказ. Класс `ExpressOrder` является производным от класса `Order`. Метод `OrderStatus` существует в обоих классах. `OrderQueue` представляет собой список, содержащий объекты обоих классов.

Инструкция:

- (a) Создайте класс `Order`, который будет базовым классом для класса `ExpressOrder`. В конструкторе класса `Order` задайте параметры `order_id`, `customer`, `items` и `order_date`.
- (b) В классе `Order` создайте метод `OrderStatus`, который будет выводить информацию о заказе.
- (c) Создайте класс `ExpressOrder`, который будет наследоваться от класса `Order`. В конструкторе класса `ExpressOrder` добавьте параметр `delivery_time_hours`.
- (d) В классе `ExpressOrder` переопределите метод `OrderStatus`, чтобы он выводил информацию о заказе и времени доставки.
- (e) В основной части программы создайте список `OrderQueue` и добавьте в него объекты классов `Order` и `ExpressOrder`.
- (f) Для каждого объекта в списке `OrderQueue` вызовите метод `OrderStatus`.

- 23 Пассажир. Класс `BusinessClassPassenger` является производным от класса `Passenger`. Метод `BoardingInfo` существует в обоих классах. `PassengerList` представляет собой список, содержащий объекты обоих классов.

Инструкция:

- (a) Создайте класс `Passenger`, который будет базовым классом для класса `BusinessClassPassenger`. В конструкторе класса `Passenger` задайте параметры `first_name`, `last_name`, `passport_number` и `seat_number`.
- (b) В классе `Passenger` создайте метод `BoardingInfo`, который будет выводить информацию о пассажире.
- (c) Создайте класс `BusinessClassPassenger`, который будет наследоваться от класса `Passenger`. В конструкторе класса `BusinessClassPassenger` добавьте параметр `lounge_access`.
- (d) В классе `BusinessClassPassenger` переопределите метод `BoardingInfo`, чтобы он выводил информацию о пассажире и доступе в лаунж.
- (e) В основной части программы создайте список `PassengerList` и добавьте в него объекты классов `Passenger` и `BusinessClassPassenger`.
- (f) Для каждого объекта в списке `PassengerList` вызовите метод `BoardingInfo`.

- 24 Товар. Класс `DigitalProduct` является производным от класса `ProductItem`. Метод `ProductInfo` существует в обоих классах. `ShoppingCart` представляет собой список, содержащий объекты обоих классов.

Инструкция:

- (a) Создайте класс `ProductItem`, который будет базовым классом для класса `DigitalProduct`. В конструкторе класса `ProductItem` задайте параметры `name`, `price`, `category` и `in_stock`.
- (b) В классе `ProductItem` создайте метод `ProductInfo`, который будет выводить информацию о товаре.

- (c) Создайте класс `DigitalProduct`, который будет наследоваться от класса `ProductItem`. В конструкторе класса `DigitalProduct` добавьте параметр `download_link`.
- (d) В классе `DigitalProduct` переопределите метод `ProductInfo`, чтобы он выводил информацию о товаре и ссылку на скачивание.
- (e) В основной части программы создайте список `ShoppingCart` и добавьте в него объекты классов `ProductItem` и `DigitalProduct`.
- (f) Для каждого объекта в списке `ShoppingCart` вызовите метод `ProductInfo`.

25 Курс. Класс `OnlineCourse` является производным от класса `Course`. Метод `CourseOutline` существует в обоих классах. `CourseCatalog` представляет собой список, содержащий объекты обоих классов.

Инструкция:

- (a) Создайте класс `Course`, который будет базовым классом для класса `OnlineCourse`. В конструкторе класса `Course` задайте параметры `title`, `instructor`, `duration_weeks` и `level`.
- (b) В классе `Course` создайте метод `CourseOutline`, который будет выводить информацию о курсе.
- (c) Создайте класс `OnlineCourse`, который будет наследоваться от класса `Course`. В конструкторе класса `OnlineCourse` добавьте параметр `platform`.
- (d) В классе `OnlineCourse` переопределите метод `CourseOutline`, чтобы он выводил информацию о курсе и платформе обучения.
- (e) В основной части программы создайте список `CourseCatalog` и добавьте в него объекты классов `Course` и `OnlineCourse`.
- (f) Для каждого объекта в списке `CourseCatalog` вызовите метод `CourseOutline`.

26 Поставка. Класс `InternationalShipment` является производным от класса `Shipment`. Метод `ShipmentInfo` существует в обоих классах. `ShipmentsLog` представляет собой список, содержащий объекты обоих классов.

Инструкция:

- (a) Создайте класс `Shipment`, который будет базовым классом для класса `InternationalShipment`. В конструкторе класса `Shipment` задайте параметры `tracking_number`, `origin`, `destination` и `weight_kg`.
- (b) В классе `Shipment` создайте метод `ShipmentInfo`, который будет выводить информацию о поставке.
- (c) Создайте класс `InternationalShipment`, который будет наследоваться от класса `Shipment`. В конструкторе класса `InternationalShipment` добавьте параметр `customs_cleared`.
- (d) В классе `InternationalShipment` переопределите метод `ShipmentInfo`, чтобы он выводил информацию о поставке и статусе таможенной очистки.
- (e) В основной части программы создайте список `ShipmentsLog` и добавьте в него объекты классов `Shipment` и `InternationalShipment`.
- (f) Для каждого объекта в списке `ShipmentsLog` вызовите метод `ShipmentInfo`.

- 27 Сотрудник службы поддержки. Класс `SeniorSupportAgent` является производным от класса `SupportAgent`. Метод `AgentProfile` существует в обоих классах. `SupportTeam` представляет собой список, содержащий объекты обоих классов.

Инструкция:

- (a) Создайте класс `SupportAgent`, который будет базовым классом для класса `SeniorSupportAgent`. В конструкторе класса `SupportAgent` задайте параметры `agent_id`, `name`, `department` и `hire_date`.
- (b) В классе `SupportAgent` создайте метод `AgentProfile`, который будет выводить информацию о сотруднике.
- (c) Создайте класс `SeniorSupportAgent`, который будет наследоваться от класса `SupportAgent`. В конструкторе класса `SeniorSupportAgent` добавьте параметр `certifications`.
- (d) В классе `SeniorSupportAgent` переопределите метод `AgentProfile`, чтобы он выводил информацию о сотруднике и его сертификатах.
- (e) В основной части программы создайте список `SupportTeam` и добавьте в него объекты классов `SupportAgent` и `SeniorSupportAgent`.
- (f) Для каждого объекта в списке `SupportTeam` вызовите метод `AgentProfile`.

- 28 Фотография. Класс `EditedPhoto` является производным от класса `Photo`. Метод `PhotoMetadata` существует в обоих классах. `PhotoAlbum` представляет собой список, содержащий объекты обоих классов.

Инструкция:

- (a) Создайте класс `Photo`, который будет базовым классом для класса `EditedPhoto`. В конструкторе класса `Photo` задайте параметры `filename`, `date_taken`, `location` и `camera_model`.
- (b) В классе `Photo` создайте метод `PhotoMetadata`, который будет выводить информацию о фотографии.
- (c) Создайте класс `EditedPhoto`, который будет наследоваться от класса `Photo`. В конструкторе класса `EditedPhoto` добавьте параметр `editing_software`.
- (d) В классе `EditedPhoto` переопределите метод `PhotoMetadata`, чтобы он выводил информацию о фотографии и программе редактирования.
- (e) В основной части программы создайте список `PhotoAlbum` и добавьте в него объекты классов `Photo` и `EditedPhoto`.
- (f) Для каждого объекта в списке `PhotoAlbum` вызовите метод `PhotoMetadata`.

- 29 Абонемент. Класс `FamilyMembership` является производным от класса `Membership`. Метод `MembershipCard` существует в обоих классах. `MembersList` представляет собой список, содержащий объекты обоих классов.

Инструкция:

- (a) Создайте класс `Membership`, который будет базовым классом для класса `FamilyMembership`. В конструкторе класса `Membership` задайте параметры `member_name`, `membership_id`, `start_date` и `type`.
- (b) В классе `Membership` создайте метод `MembershipCard`, который будет выводить информацию об абонементе.

- (c) Создайте класс `FamilyMembership`, который будет наследоваться от класса `Membership`. В конструкторе класса `FamilyMembership` добавьте параметр `family_size`.
- (d) В классе `FamilyMembership` переопределите метод `MembershipCard`, чтобы он выводил информацию об абонементе и количестве членов семьи.
- (e) В основной части программы создайте список `MembersList` и добавьте в него объекты классов `Membership` и `FamilyMembership`.
- (f) Для каждого объекта в списке `MembersList` вызовите метод `MembershipCard`.

30 Турист. Класс `Backpacker` является производным от класса `Traveler`. Метод `TravelProfile` существует в обоих классах. `TravelersGroup` представляет собой список, содержащий объекты обоих классов.

Инструкция:

- (a) Создайте класс `Traveler`, который будет базовым классом для класса `Backpacker`. В конструкторе класса `Traveler` задайте параметры `name`, `nationality`, `passport_num` и `current_location`.
- (b) В классе `Traveler` создайте метод `TravelProfile`, который будет выводить информацию о туристе.
- (c) Создайте класс `Backpacker`, который будет наследоваться от класса `Traveler`. В конструкторе класса `Backpacker` добавьте параметр `budget_per_day`.
- (d) В классе `Backpacker` переопределите метод `TravelProfile`, чтобы он выводил информацию о туристе и дневном бюджете.
- (e) В основной части программы создайте список `TravelersGroup` и добавьте в него объекты классов `Traveler` и `Backpacker`.
- (f) Для каждого объекта в списке `TravelersGroup` вызовите метод `TravelProfile`.

31 Климатическое устройство. Класс `AirConditioner` является производным от класса `ClimateDevice`. Метод `DeviceStatus` существует в обоих классах. `DevicesList` представляет собой список, содержащий объекты обоих классов.

Инструкция:

- (a) Создайте класс `ClimateDevice`, который будет базовым классом для класса `AirConditioner`. В конструкторе класса `ClimateDevice` задайте параметры `device_id`, `brand`, `power_watts` и `location`.
- (b) В классе `ClimateDevice` создайте метод `DeviceStatus`, который будет выводить информацию об устройстве.
- (c) Создайте класс `AirConditioner`, который будет наследоваться от класса `ClimateDevice`. В конструкторе класса `AirConditioner` добавьте параметр `cooling_capacity`.
- (d) В классе `AirConditioner` переопределите метод `DeviceStatus`, чтобы он выводил информацию об устройстве и мощности охлаждения.
- (e) В основной части программы создайте список `DevicesList` и добавьте в него объекты классов `ClimateDevice` и `AirConditioner`.
- (f) Для каждого объекта в списке `DevicesList` вызовите метод `DeviceStatus`.

32 Задание. Класс `GroupAssignment` является производным от класса `Assignment`. Метод `AssignmentInfo` существует в обоих классах. `AssignmentsList` представляет собой список, содержащий объекты обоих классов.

Инструкция:

- (a) Создайте класс `Assignment`, который будет базовым классом для класса `GroupAssignment`. В конструкторе класса `Assignment` задайте параметры `title`, `due_date`, `max_score` и `description`.
- (b) В классе `Assignment` создайте метод `AssignmentInfo`, который будет выводить информацию о задании.
- (c) Создайте класс `GroupAssignment`, который будет наследоваться от класса `Assignment`. В конструкторе класса `GroupAssignment` добавьте параметр `team_size`.
- (d) В классе `GroupAssignment` переопределите метод `AssignmentInfo`, чтобы он выводил информацию о задании и размере команды.
- (e) В основной части программы создайте список `AssignmentsList` и добавьте в него объекты классов `Assignment` и `GroupAssignment`.
- (f) Для каждого объекта в списке `AssignmentsList` вызовите метод `AssignmentInfo`.

33 Сертификат. Класс `ProfessionalCertification` является производным от класса `Certificate`. Метод `CertDetails` существует в обоих классах. `CertificatesPortfolio` представляет собой список, содержащий объекты обоих классов.

Инструкция:

- (a) Создайте класс `Certificate`, который будет базовым классом для класса `ProfessionalCertification`. В конструкторе класса `Certificate` задайте параметры `cert_id`, `title`, `issuer` и `issue_date`.
- (b) В классе `Certificate` создайте метод `CertDetails`, который будет выводить информацию о сертификате.
- (c) Создайте класс `ProfessionalCertification`, который будет наследоваться от класса `Certificate`. В конструкторе класса `ProfessionalCertification` добавьте параметр `valid_until`.
- (d) В классе `ProfessionalCertification` переопределите метод `CertDetails`, чтобы он выводил информацию о сертификате и дате окончания действия.
- (e) В основной части программы создайте список `CertificatesPortfolio` и добавьте в него объекты классов `Certificate` и `ProfessionalCertification`.
- (f) Для каждого объекта в списке `CertificatesPortfolio` вызовите метод `CertDetails`.

34 Мероприятие. Класс `Workshop` является производным от класса `Activity`. Метод `ActivitySummary` существует в обоих классах. `ActivitiesSchedule` представляет собой список, содержащий объекты обоих классов.

Инструкция:

- (a) Создайте класс `Activity`, который будет базовым классом для класса `Workshop`. В конструкторе класса `Activity` задайте параметры `name`, `location`, `start_time` и `duration_hours`.
- (b) В классе `Activity` создайте метод `ActivitySummary`, который будет выводить информацию о мероприятии.
- (c) Создайте класс `Workshop`, который будет наследоваться от класса `Activity`. В конструкторе класса `Workshop` добавьте параметр `materials_required`.
- (d) В классе `Workshop` переопределите метод `ActivitySummary`, чтобы он выводил информацию о мероприятии и необходимых материалах.

- (e) В основной части программы создайте список `ActivitiesSchedule` и добавьте в него объекты классов `Activity` и `Workshop`.
- (f) Для каждого объекта в списке `ActivitiesSchedule` вызовите метод `ActivitySummary`.

35 Транзакция. Класс `RefundTransaction` является производным от класса `Transaction`. Метод `TransactionReport` существует в обоих классах. `TransactionLog` представляет собой список, содержащий объекты обоих классов.

Инструкция:

- (a) Создайте класс `Transaction`, который будет базовым классом для класса `RefundTransaction`. В конструкторе класса `Transaction` задайте параметры `trans_id`, `amount`, `currency` и `timestamp`.
- (b) В классе `Transaction` создайте метод `TransactionReport`, который будет выводить информацию о транзакции.
- (c) Создайте класс `RefundTransaction`, который будет наследоваться от класса `Transaction`. В конструкторе класса `RefundTransaction` добавьте параметр `reason`.
- (d) В классе `RefundTransaction` переопределите метод `TransactionReport`, чтобы он выводил информацию о транзакции и причине возврата.
- (e) В основной части программы создайте список `TransactionLog` и добавьте в него объекты классов `Transaction` и `RefundTransaction`.
- (f) Для каждого объекта в списке `TransactionLog` вызовите метод `TransactionReport`.

2.7.2 Задача 2.

1 Модель склада оргтехники. Классы `Printer`, `Scanner` и `Xerox` являются производными от класса `Equipment`. Метод `__str__()` перегружен только в классе `Printer`, для остальных используется метод из базового класса. **Инструкция:**

- (a) Создайте класс `Equipment`, который будет базовым классом для классов `Printer`, `Scanner` и `Xerox`. В конструкторе класса `Equipment` задайте параметры `name`, `make` и `year`.
- (b) В классе `Equipment` создайте метод `action`, который будет возвращать строку 'Не определено'.
- (c) В классе `Equipment` создайте метод `__str__`, который будет возвращать строку с информацией о оборудовании.
- (d) Создайте класс `Printer`, который будет наследоваться от класса `Equipment`. В конструкторе класса `Printer` задайте параметры `series`, `name`, `make` и `year`. Используйте метод `super().__init__(...)`.
- (e) В классе `Printer` переопределите метод `action`, чтобы он возвращал строку 'Печатает'.
- (f) Создайте класс `Scanner`, который будет наследоваться от класса `Equipment`. В конструкторе класса `Scanner` задайте параметры `name`, `make` и `year`. Используйте метод `super().__init__(...)`.
- (g) В классе `Scanner` переопределите метод `action`, чтобы он возвращал строку 'Сканирует'.

- (h) Создайте класс `Xerox`, который будет наследоваться от класса `Equipment`. В конструкторе класса `Xerox` задайте параметры `name`, `make` и `year`. Используйте метод `super().__init__(...)`.
 - (i) В классе `Xerox` переопределите метод `action`, чтобы он возвращал строку 'Копирует'.
 - (j) В основной части программы создайте объекты классов `Printer`, `Scanner` и `Xerox` и добавьте их в список `warehouse`.
 - (k) Выведите содержимое списка `warehouse`, используя метод `action` каждого объекта.
 - (l) Удалите все объекты класса `Printer` из списка `warehouse`.
 - (m) Выведите оставшееся содержимое списка `warehouse`, используя метод `action` каждого объекта.
- 2 Модель зоопарка. Классы `Lion`, `Tiger` и `Bear` являются производными от класса `Animal`. Метод `__str__()` перегружен только в классе `Lion`, для остальных используется метод из базового класса. **Инструкция:**
- (a) Создайте класс `Animal`, который будет базовым классом для классов `Lion`, `Tiger` и `Bear`. В конструкторе класса `Animal` задайте параметры `name`, `species` и `age`.
 - (b) В классе `Animal` создайте метод `sound`, который будет возвращать строку 'Не определено'.
 - (c) В классе `Animal` создайте метод `__str__`, который будет возвращать строку с информацией о животном.
 - (d) Создайте класс `Lion`, который будет наследоваться от класса `Animal`. В конструкторе класса `Lion` задайте параметры `mane_color`, `name`, `species` и `age`. Используйте метод `super().__init__(...)`.
 - (e) В классе `Lion` переопределите метод `sound`, чтобы он возвращал строку 'Рычит'.
 - (f) Создайте класс `Tiger`, который будет наследоваться от класса `Animal`. В конструкторе класса `Tiger` задайте параметры `name`, `species` и `age`. Используйте метод `super().__init__(...)`.
 - (g) В классе `Tiger` переопределите метод `sound`, чтобы он возвращал строку 'Рычит громко'.
 - (h) Создайте класс `Bear`, который будет наследоваться от класса `Animal`. В конструкторе класса `Bear` задайте параметры `name`, `species` и `age`. Используйте метод `super().__init__(...)`.
 - (i) В классе `Bear` переопределите метод `sound`, чтобы он возвращал строку 'Ревёт'.
 - (j) В основной части программы создайте объекты классов `Lion`, `Tiger` и `Bear` и добавьте их в список `zoo`.
 - (k) Выведите содержимое списка `zoo`, используя метод `sound` каждого объекта.
 - (l) Удалите все объекты класса `Lion` из списка `zoo`.
 - (m) Выведите оставшееся содержимое списка `zoo`, используя метод `sound` каждого объекта.
- 3 Модель транспортного парка. Классы `Car`, `Truck` и `Motorcycle` являются производными от класса `Vehicle`. Метод `__str__()` перегружен только в классе `Car`, для остальных используется метод из базового класса. **Инструкция:**

- (a) Создайте класс `Vehicle`, который будет базовым классом для классов `Car`, `Truck` и `Motorcycle`. В конструкторе класса `Vehicle` задайте параметры `brand`, `model` и `year`.
 - (b) В классе `Vehicle` создайте метод `move`, который будет возвращать строку 'Двигается'.
 - (c) В классе `Vehicle` создайте метод `__str__`, который будет возвращать строку с информацией о транспортном средстве.
 - (d) Создайте класс `Car`, который будет наследоваться от класса `Vehicle`. В конструкторе класса `Car` задайте параметры `doors`, `brand`, `model` и `year`. Используйте метод `super().__init__()`.
 - (e) В классе `Car` переопределите метод `move`, чтобы он возвращал строку 'Едет по дороге'.
 - (f) Создайте класс `Truck`, который будет наследоваться от класса `Vehicle`. В конструкторе класса `Truck` задайте параметры `brand`, `model` и `year`. Используйте метод `super().__init__()`.
 - (g) В классе `Truck` переопределите метод `move`, чтобы он возвращал строку 'Перевозит груз'.
 - (h) Создайте класс `Motorcycle`, который будет наследоваться от класса `Vehicle`. В конструкторе класса `Motorcycle` задайте параметры `brand`, `model` и `year`. Используйте метод `super().__init__()`.
 - (i) В классе `Motorcycle` переопределите метод `move`, чтобы он возвращал строку 'Мчится'.
 - (j) В основной части программы создайте объекты классов `Car`, `Truck` и `Motorcycle` и добавьте их в список `fleet`.
 - (k) Выведите содержимое списка `fleet`, используя метод `move` каждого объекта.
 - (l) Удалите все объекты класса `Car` из списка `fleet`.
 - (m) Выведите оставшееся содержимое списка `fleet`, используя метод `move` каждого объекта.
- 4 Модель библиотеки. Классы `Book`, `Magazine` и `Newspaper` являются производными от класса `Publication`. Метод `__str__()` перегружен только в классе `Book`, для остальных используется метод из базового класса. **Инструкция:**
- (a) Создайте класс `Publication`, который будет базовым классом для классов `Book`, `Magazine` и `Newspaper`. В конструкторе класса `Publication` задайте параметры `title`, `publisher` и `year`.
 - (b) В классе `Publication` создайте метод `read`, который будет возвращать строку 'Читается'.
 - (c) В классе `Publication` создайте метод `__str__`, который будет возвращать строку с информацией об издании.
 - (d) Создайте класс `Book`, который будет наследоваться от класса `Publication`. В конструкторе класса `Book` задайте параметры `pages`, `title`, `publisher` и `year`. Используйте метод `super().__init__()`.
 - (e) В классе `Book` переопределите метод `read`, чтобы он возвращал строку 'Читается от корки до корки'.

- (f) Создайте класс `Magazine`, который будет наследоваться от класса `Publication`. В конструкторе класса `Magazine` задайте параметры `title`, `publisher` и `year`. Используйте метод `super().__init__(...)`.
 - (g) В классе `Magazine` переопределите метод `read`, чтобы он возвращал строку 'Листается'.
 - (h) Создайте класс `Newspaper`, который будет наследоваться от класса `Publication`. В конструкторе класса `Newspaper` задайте параметры `title`, `publisher` и `year`. Используйте метод `super().__init__(...)`.
 - (i) В классе `Newspaper` переопределите метод `read`, чтобы он возвращал строку 'Просматривается'.
 - (j) В основной части программы создайте объекты классов `Book`, `Magazine` и `Newspaper` и добавьте их в список `library`.
 - (k) Выведите содержимое списка `library`, используя метод `read` каждого объекта.
 - (l) Удалите все объекты класса `Book` из списка `library`.
 - (m) Выведите оставшееся содержимое списка `library`, используя метод `read` каждого объекта.
- 5 Модель музыкальных инструментов. Классы `Guitar`, `Piano` и `Drums` являются производными от класса `Instrument`. Метод `__str__()` перегружен только в классе `Guitar`, для остальных используется метод из базового класса. **Инструкция:**
- (a) Создайте класс `Instrument`, который будет базовым классом для классов `Guitar`, `Piano` и `Drums`. В конструкторе класса `Instrument` задайте параметры `name`, `maker` и `year`.
 - (b) В классе `Instrument` создайте метод `play`, который будет возвращать строку 'Играет'.
 - (c) В классе `Instrument` создайте метод `__str__`, который будет возвращать строку с информацией об инструменте.
 - (d) Создайте класс `Guitar`, который будет наследоваться от класса `Instrument`. В конструкторе класса `Guitar` задайте параметры `strings`, `name`, `maker` и `year`. Используйте метод `super().__init__(...)`.
 - (e) В классе `Guitar` переопределите метод `play`, чтобы он возвращал строку 'Звучит струнами'.
 - (f) Создайте класс `Piano`, который будет наследоваться от класса `Instrument`. В конструкторе класса `Piano` задайте параметры `name`, `maker` и `year`. Используйте метод `super().__init__(...)`.
 - (g) В классе `Piano` переопределите метод `play`, чтобы он возвращал строку 'Играет клавишами'.
 - (h) Создайте класс `Drums`, который будет наследоваться от класса `Instrument`. В конструкторе класса `Drums` задайте параметры `name`, `maker` и `year`. Используйте метод `super().__init__(...)`.
 - (i) В классе `Drums` переопределите метод `play`, чтобы он возвращал строку 'Бьёт в ритме'.
 - (j) В основной части программы создайте объекты классов `Guitar`, `Piano` и `Drums` и добавьте их в список `band`.
 - (k) Выведите содержимое списка `band`, используя метод `play` каждого объекта.

- (l) Удалите все объекты класса `Guitar` из списка `band`.
 - (m) Выведите оставшееся содержимое списка `band`, используя метод `play` каждого объекта.
- 6 Модель офисной мебели. Классы `Chair`, `Desk` и `Cabinet` являются производными от класса `Furniture`. Метод `__str__()` перегружен только в классе `Chair`, для остальных используется метод из базового класса. **Инструкция:**
- (a) Создайте класс `Furniture`, который будет базовым классом для классов `Chair`, `Desk` и `Cabinet`. В конструкторе класса `Furniture` задайте параметры `name`, `material` и `year`.
 - (b) В классе `Furniture` создайте метод `use`, который будет возвращать строку 'Используется'.
 - (c) В классе `Furniture` создайте метод `__str__`, который будет возвращать строку с информацией о мебели.
 - (d) Создайте класс `Chair`, который будет наследоваться от класса `Furniture`. В конструкторе класса `Chair` задайте параметры `wheels`, `name`, `material` и `year`. Используйте метод `super().__init__()`.
 - (e) В классе `Chair` переопределите метод `use`, чтобы он возвращал строку 'Сидят на нём'.
 - (f) Создайте класс `Desk`, который будет наследоваться от класса `Furniture`. В конструкторе класса `Desk` задайте параметры `name`, `material` и `year`. Используйте метод `super().__init__()`.
 - (g) В классе `Desk` переопределите метод `use`, чтобы он возвращал строку 'Работают за ним'.
 - (h) Создайте класс `Cabinet`, который будет наследоваться от класса `Furniture`. В конструкторе класса `Cabinet` задайте параметры `name`, `material` и `year`. Используйте метод `super().__init__()`.
 - (i) В классе `Cabinet` переопределите метод `use`, чтобы он возвращал строку 'Хранят в нём'.
 - (j) В основной части программы создайте объекты классов `Chair`, `Desk` и `Cabinet` и добавьте их в список `office`.
 - (k) Выведите содержимое списка `office`, используя метод `use` каждого объекта.
 - (l) Удалите все объекты класса `Chair` из списка `office`.
 - (m) Выведите оставшееся содержимое списка `office`, используя метод `use` каждого объекта.
- 7 Модель кухонной техники. Классы `Blender`, `Toaster` и `Kettle` являются производными от класса `Appliance`. Метод `__str__()` перегружен только в классе `Blender`, для остальных используется метод из базового класса. **Инструкция:**
- (a) Создайте класс `Appliance`, который будет базовым классом для классов `Blender`, `Toaster` и `Kettle`. В конструкторе класса `Appliance` задайте параметры `name`, `brand` и `year`.
 - (b) В классе `Appliance` создайте метод `operate`, который будет возвращать строку 'Работает'.

- (c) В классе `Appliance` создайте метод `__str__`, который будет возвращать строку с информацией о приборе.
 - (d) Создайте класс `Blender`, который будет наследоваться от класса `Appliance`. В конструкторе класса `Blender` задайте параметры `power`, `name`, `brand` и `year`. Используйте метод `super().__init__(...)`.
 - (e) В классе `Blender` переопределите метод `operate`, чтобы он возвращал строку 'Перемалывает'.
 - (f) Создайте класс `Toaster`, который будет наследоваться от класса `Appliance`. В конструкторе класса `Toaster` задайте параметры `name`, `brand` и `year`. Используйте метод `super().__init__(...)`.
 - (g) В классе `Toaster` переопределите метод `operate`, чтобы он возвращал строку 'Поджаривает'.
 - (h) Создайте класс `Kettle`, который будет наследоваться от класса `Appliance`. В конструкторе класса `Kettle` задайте параметры `name`, `brand` и `year`. Используйте метод `super().__init__(...)`.
 - (i) В классе `Kettle` переопределите метод `operate`, чтобы он возвращал строку 'Кипятит'.
 - (j) В основной части программы создайте объекты классов `Blender`, `Toaster` и `Kettle` и добавьте их в список `kitchen`.
 - (k) Выведите содержимое списка `kitchen`, используя метод `operate` каждого объекта.
 - (l) Удалите все объекты класса `Blender` из списка `kitchen`.
 - (m) Выведите оставшееся содержимое списка `kitchen`, используя метод `operate` каждого объекта.
- 8 Модель спортивного инвентаря. Классы `Ball`, `Racket` и `Skates` являются производными от класса `SportItem`. Метод `__str__()` перегружен только в классе `Ball`, для остальных используется метод из базового класса. **Инструкция:**
- (a) Создайте класс `SportItem`, который будет базовым классом для классов `Ball`, `Racket` и `Skates`. В конструкторе класса `SportItem` задайте параметры `name`, `sport` и `year`.
 - (b) В классе `SportItem` создайте метод `use`, который будет возвращать строку 'Используется в спорте'.
 - (c) В классе `SportItem` создайте метод `__str__`, который будет возвращать строку с информацией об инвентаре.
 - (d) Создайте класс `Ball`, который будет наследоваться от класса `SportItem`. В конструкторе класса `Ball` задайте параметры `material`, `name`, `sport` и `year`. Используйте метод `super().__init__(...)`.
 - (e) В классе `Ball` переопределите метод `use`, чтобы он возвращал строку 'Катится и отскакивает'.
 - (f) Создайте класс `Racket`, который будет наследоваться от класса `SportItem`. В конструкторе класса `Racket` задайте параметры `name`, `sport` и `year`. Используйте метод `super().__init__(...)`.

- (g) В классе **Racket** переопределите метод **use**, чтобы он возвращал строку 'Бьёт по мячу'.
 - (h) Создайте класс **Skates**, который будет наследоваться от класса **SportItem**. В конструкторе класса **Skates** задайте параметры **name**, **sport** и **year**. Используйте метод **super().__init__(...)**.
 - (i) В классе **Skates** переопределите метод **use**, чтобы он возвращал строку 'Скользит по льду'.
 - (j) В основной части программы создайте объекты классов **Ball**, **Racket** и **Skates** и добавьте их в список **inventory**.
 - (k) Выведите содержимое списка **inventory**, используя метод **use** каждого объекта.
 - (l) Удалите все объекты класса **Ball** из списка **inventory**.
 - (m) Выведите оставшееся содержимое списка **inventory**, используя метод **use** каждого объекта.
- 9 Модель электронных устройств. Классы **Phone**, **Tablet** и **Laptop** являются производными от класса **Device**. Метод **__str__()** перегружен только в классе **Phone**, для остальных используется метод из базового класса. **Инструкция:**
- (a) Создайте класс **Device**, который будет базовым классом для классов **Phone**, **Tablet** и **Laptop**. В конструкторе класса **Device** задайте параметры **name**, **brand** и **year**.
 - (b) В классе **Device** создайте метод **function**, который будет возвращать строку 'Функционирует'.
 - (c) В классе **Device** создайте метод **__str__**, который будет возвращать строку с информацией об устройстве.
 - (d) Создайте класс **Phone**, который будет наследоваться от класса **Device**. В конструкторе класса **Phone** задайте параметры **os**, **name**, **brand** и **year**. Используйте метод **super().__init__(...)**.
 - (e) В классе **Phone** переопределите метод **function**, чтобы он возвращал строку 'Звонит и пишет'.
 - (f) Создайте класс **Tablet**, который будет наследоваться от класса **Device**. В конструкторе класса **Tablet** задайте параметры **name**, **brand** и **year**. Используйте метод **super().__init__(...)**.
 - (g) В классе **Tablet** переопределите метод **function**, чтобы он возвращал строку 'Показывает контент'.
 - (h) Создайте класс **Laptop**, который будет наследоваться от класса **Device**. В конструкторе класса **Laptop** задайте параметры **name**, **brand** и **year**. Используйте метод **super().__init__(...)**.
 - (i) В классе **Laptop** переопределите метод **function**, чтобы он возвращал строку 'Работает с программами'.
 - (j) В основной части программы создайте объекты классов **Phone**, **Tablet** и **Laptop** и добавьте их в список **devices**.
 - (k) Выведите содержимое списка **devices**, используя метод **function** каждого объекта.
 - (l) Удалите все объекты класса **Phone** из списка **devices**.

- (m) Выведите оставшееся содержимое списка `devices`, используя метод `function` каждого объекта.
- 10 Модель школьных принадлежностей. Классы `Pen`, `Notebook` и `Ruler` являются производными от класса `Stationery`. Метод `__str__()` перегружен только в классе `Pen`, для остальных используется метод из базового класса. **Инструкция:**
- (a) Создайте класс `Stationery`, который будет базовым классом для классов `Pen`, `Notebook` и `Ruler`. В конструкторе класса `Stationery` задайте параметры `name`, `brand` и `color`.
 - (b) В классе `Stationery` создайте метод `apply`, который будет возвращать строку 'Применяется'.
 - (c) В классе `Stationery` создайте метод `__str__`, который будет возвращать строку с информацией о принадлежности.
 - (d) Создайте класс `Pen`, который будет наследоваться от класса `Stationery`. В конструкторе класса `Pen` задайте параметры `ink_type`, `name`, `brand` и `color`. Используйте метод `super().__init__(...)`.
 - (e) В классе `Pen` переопределите метод `apply`, чтобы он возвращал строку 'Пишет'.
 - (f) Создайте класс `Notebook`, который будет наследоваться от класса `Stationery`. В конструкторе класса `Notebook` задайте параметры `name`, `brand` и `color`. Используйте метод `super().__init__(...)`.
 - (g) В классе `Notebook` переопределите метод `apply`, чтобы он возвращал строку 'Заполняется'.
 - (h) Создайте класс `Ruler`, который будет наследоваться от класса `Stationery`. В конструкторе класса `Ruler` задайте параметры `name`, `brand` и `color`. Используйте метод `super().__init__(...)`.
 - (i) В классе `Ruler` переопределите метод `apply`, чтобы он возвращал строку 'Измеряет'.
 - (j) В основной части программы создайте объекты классов `Pen`, `Notebook` и `Ruler` и добавьте их в список `school`.
 - (k) Выведите содержимое списка `school`, используя метод `apply` каждого объекта.
 - (l) Удалите все объекты класса `Pen` из списка `school`.
 - (m) Выведите оставшееся содержимое списка `school`, используя метод `apply` каждого объекта.
- 11 Модель строительных материалов. Классы `Brick`, `Plank` и `Tile` являются производными от класса `Material`. Метод `__str__()` перегружен только в классе `Brick`, для остальных используется метод из базового класса. **Инструкция:**
- (a) Создайте класс `Material`, который будет базовым классом для классов `Brick`, `Plank` и `Tile`. В конструкторе класса `Material` задайте параметры `name`, `origin` и `year`.
 - (b) В классе `Material` создайте метод `build`, который будет возвращать строку 'Используется в строительстве'.
 - (c) В классе `Material` создайте метод `__str__`, который будет возвращать строку с информацией о материале.

- (d) Создайте класс `Brick`, который будет наследоваться от класса `Material`. В конструкторе класса `Brick` задайте параметры `strength`, `name`, `origin` и `year`. Используйте метод `super().__init__(...)`.
 - (e) В классе `Brick` переопределите метод `build`, чтобы он возвращал строку 'Кладётся в стену'.
 - (f) Создайте класс `Plank`, который будет наследоваться от класса `Material`. В конструкторе класса `Plank` задайте параметры `name`, `origin` и `year`. Используйте метод `super().__init__(...)`.
 - (g) В классе `Plank` переопределите метод `build`, чтобы он возвращал строку 'Укладывается на пол'.
 - (h) Создайте класс `Tile`, который будет наследоваться от класса `Material`. В конструкторе класса `Tile` задайте параметры `name`, `origin` и `year`. Используйте метод `super().__init__(...)`.
 - (i) В классе `Tile` переопределите метод `build`, чтобы он возвращал строку 'Облицовывает поверхность'.
 - (j) В основной части программы создайте объекты классов `Brick`, `Plank` и `Tile` и добавьте их в список `warehouse`.
 - (k) Выведите содержимое списка `warehouse`, используя метод `build` каждого объекта.
 - (l) Удалите все объекты класса `Brick` из списка `warehouse`.
 - (m) Выведите оставшееся содержимое списка `warehouse`, используя метод `build` каждого объекта.
- 12 Модель одежды. Классы `Jacket`, `Shirt` и `Pants` являются производными от класса `Clothing`. Метод `__str__()` перегружен только в классе `Jacket`, для остальных используется метод из базового класса. **Инструкция:**
- (a) Создайте класс `Clothing`, который будет базовым классом для классов `Jacket`, `Shirt` и `Pants`. В конструкторе класса `Clothing` задайте параметры `name`, `brand` и `season`.
 - (b) В классе `Clothing` создайте метод `wear`, который будет возвращать строку 'Носится'.
 - (c) В классе `Clothing` создайте метод `__str__`, который будет возвращать строку с информацией об одежде.
 - (d) Создайте класс `Jacket`, который будет наследоваться от класса `Clothing`. В конструкторе класса `Jacket` задайте параметры `insulation`, `name`, `brand` и `season`. Используйте метод `super().__init__(...)`.
 - (e) В классе `Jacket` переопределите метод `wear`, чтобы он возвращал строку 'Согревает'.
 - (f) Создайте класс `Shirt`, который будет наследоваться от класса `Clothing`. В конструкторе класса `Shirt` задайте параметры `name`, `brand` и `season`. Используйте метод `super().__init__(...)`.
 - (g) В классе `Shirt` переопределите метод `wear`, чтобы он возвращал строку 'Одевается'.
 - (h) Создайте класс `Pants`, который будет наследоваться от класса `Clothing`. В конструкторе класса `Pants` задайте параметры `name`, `brand` и `season`. Используйте метод `super().__init__(...)`.
 - (i) В классе `Pants` переопределите метод `wear`, чтобы он возвращал строку 'Надеваются'.

- (j) В основной части программы создайте объекты классов `Jacket`, `Shirt` и `Pants` и добавьте их в список `wardrobe`.
 - (k) Выведите содержимое списка `wardrobe`, используя метод `wear` каждого объекта.
 - (l) Удалите все объекты класса `Jacket` из списка `wardrobe`.
 - (m) Выведите оставшееся содержимое списка `wardrobe`, используя метод `wear` каждого объекта.
- 13 Модель бытовой химии. Классы `Detergent`, `Bleach` и `Softener` являются производными от класса `CleaningAgent`. Метод `__str__()` перегружен только в классе `Detergent`, для остальных используется метод из базового класса. **Инструкция:**
- (a) Создайте класс `CleaningAgent`, который будет базовым классом для классов `Detergent`, `Bleach` и `Softener`. В конструкторе класса `CleaningAgent` задайте параметры `name`, `brand` и `volume`.
 - (b) В классе `CleaningAgent` создайте метод `clean`, который будет возвращать строку 'Очищает'.
 - (c) В классе `CleaningAgent` создайте метод `__str__`, который будет возвращать строку с информацией о средстве.
 - (d) Создайте класс `Detergent`, который будет наследоваться от класса `CleaningAgent`. В конструкторе класса `Detergent` задайте параметры `scent`, `name`, `brand` и `volume`. Используйте метод `super().__init__()`.
 - (e) В классе `Detergent` переопределите метод `clean`, чтобы он возвращал строку 'Стирает грязь'.
 - (f) Создайте класс `Bleach`, который будет наследоваться от класса `CleaningAgent`. В конструкторе класса `Bleach` задайте параметры `name`, `brand` и `volume`. Используйте метод `super().__init__()`.
 - (g) В классе `Bleach` переопределите метод `clean`, чтобы он возвращал строку 'Отбеливает'.
 - (h) Создайте класс `Softener`, который будет наследоваться от класса `CleaningAgent`. В конструкторе класса `Softener` задайте параметры `name`, `brand` и `volume`. Используйте метод `super().__init__()`.
 - (i) В классе `Softener` переопределите метод `clean`, чтобы он возвращал строку 'Смягчает ткань'.
 - (j) В основной части программы создайте объекты классов `Detergent`, `Bleach` и `Softener` и добавьте их в список `shelf`.
 - (k) Выведите содержимое списка `shelf`, используя метод `clean` каждого объекта.
 - (l) Удалите все объекты класса `Detergent` из списка `shelf`.
 - (m) Выведите оставшееся содержимое списка `shelf`, используя метод `clean` каждого объекта.
- 14 Модель инструментов. Классы `Hammer`, `Screwdriver` и `Wrench` являются производными от класса `Tool`. Метод `__str__()` перегружен только в классе `Hammer`, для остальных используется метод из базового класса. **Инструкция:**
- (a) Создайте класс `Tool`, который будет базовым классом для классов `Hammer`, `Screwdriver` и `Wrench`. В конструкторе класса `Tool` задайте параметры `name`, `material` и `weight`.
 - (b) В классе `Tool` создайте метод `work`, который будет возвращать строку 'Работает'.

- (c) В классе `Tool` создайте метод `__str__`, который будет возвращать строку с информацией об инструменте.
 - (d) Создайте класс `Hammer`, который будет наследоваться от класса `Tool`. В конструкторе класса `Hammer` задайте параметры `head_type`, `name`, `material` и `weight`. Используйте метод `super().__init__(...)`.
 - (e) В классе `Hammer` переопределите метод `work`, чтобы он возвращал строку `'Забивает гвозди'`.
 - (f) Создайте класс `Screwdriver`, который будет наследоваться от класса `Tool`. В конструкторе класса `Screwdriver` задайте параметры `name`, `material` и `weight`. Используйте метод `super().__init__(...)`.
 - (g) В классе `Screwdriver` переопределите метод `work`, чтобы он возвращал строку `'Закручивает винты'`.
 - (h) Создайте класс `Wrench`, который будет наследоваться от класса `Tool`. В конструкторе класса `Wrench` задайте параметры `name`, `material` и `weight`. Используйте метод `super().__init__(...)`.
 - (i) В классе `Wrench` переопределите метод `work`, чтобы он возвращал строку `'Откручивает гайки'`.
 - (j) В основной части программы создайте объекты классов `Hammer`, `Screwdriver` и `Wrench` и добавьте их в список `toolbox`.
 - (k) Выведите содержимое списка `toolbox`, используя метод `work` каждого объекта.
 - (l) Удалите все объекты класса `Hammer` из списка `toolbox`.
 - (m) Выведите оставшееся содержимое списка `toolbox`, используя метод `work` каждого объекта.
- 15 Модель напитков. Классы `Coffee`, `Tea` и `Juice` являются производными от класса `Beverage`. Метод `__str__()` перегружен только в классе `Coffee`, для остальных используется метод из базового класса. **Инструкция:**
- (a) Создайте класс `Beverage`, который будет базовым классом для классов `Coffee`, `Tea` и `Juice`. В конструкторе класса `Beverage` задайте параметры `name`, `brand` и `temperature`.
 - (b) В классе `Beverage` создайте метод `consume`, который будет возвращать строку `'Пьётся'`.
 - (c) В классе `Beverage` создайте метод `__str__`, который будет возвращать строку с информацией о напитке.
 - (d) Создайте класс `Coffee`, который будет наследоваться от класса `Beverage`. В конструкторе класса `Coffee` задайте параметры `roast`, `name`, `brand` и `temperature`. Используйте метод `super().__init__(...)`.
 - (e) В классе `Coffee` переопределите метод `consume`, чтобы он возвращал строку `'Бодрит'`.
 - (f) Создайте класс `Tea`, который будет наследоваться от класса `Beverage`. В конструкторе класса `Tea` задайте параметры `name`, `brand` и `temperature`. Используйте метод `super().__init__(...)`.
 - (g) В классе `Tea` переопределите метод `consume`, чтобы он возвращал строку `'Успокаивает'`.

- (h) Создайте класс `Juice`, который будет наследоваться от класса `Beverage`. В конструкторе класса `Juice` задайте параметры `name`, `brand` и `temperature`. Используйте метод `super().__init__(...)`.
- (i) В классе `Juice` переопределите метод `consume`, чтобы он возвращал строку 'Освежает'.
- (j) В основной части программы создайте объекты классов `Coffee`, `Tea` и `Juice` и добавьте их в список `bar`.
- (k) Выведите содержимое списка `bar`, используя метод `consume` каждого объекта.
- (l) Удалите все объекты класса `Coffee` из списка `bar`.
- (m) Выведите оставшееся содержимое списка `bar`, используя метод `consume` каждого объекта.

16 Модель растений. Классы `Tree`, `Flower` и `Grass` являются производными от класса `Plant`. Метод `__str__()` перегружен только в классе `Tree`, для остальных используется метод из базового класса. **Инструкция:**

- (a) Создайте класс `Plant`, который будет базовым классом для классов `Tree`, `Flower` и `Grass`. В конструкторе класса `Plant` задайте параметры `name`, `family` и `height`.
- (b) В классе `Plant` создайте метод `grow`, который будет возвращать строку 'Растёт'.
- (c) В классе `Plant` создайте метод `__str__`, который будет возвращать строку с информацией о растении.
- (d) Создайте класс `Tree`, который будет наследоваться от класса `Plant`. В конструкторе класса `Tree` задайте параметры `wood_type`, `name`, `family` и `height`. Используйте метод `super().__init__(...)`.
- (e) В классе `Tree` переопределите метод `grow`, чтобы он возвращал строку 'Тянется к солнцу'.
- (f) Создайте класс `Flower`, который будет наследоваться от класса `Plant`. В конструкторе класса `Flower` задайте параметры `name`, `family` и `height`. Используйте метод `super().__init__(...)`.
- (g) В классе `Flower` переопределите метод `grow`, чтобы он возвращал строку 'Цветёт'.
- (h) Создайте класс `Grass`, который будет наследоваться от класса `Plant`. В конструкторе класса `Grass` задайте параметры `name`, `family` и `height`. Используйте метод `super().__init__(...)`.
- (i) В классе `Grass` переопределите метод `grow`, чтобы он возвращал строку 'Покрывает землю'.
- (j) В основной части программы создайте объекты классов `Tree`, `Flower` и `Grass` и добавьте их в список `garden`.
- (k) Выведите содержимое списка `garden`, используя метод `grow` каждого объекта.
- (l) Удалите все объекты класса `Tree` из списка `garden`.
- (m) Выведите оставшееся содержимое списка `garden`, используя метод `grow` каждого объекта.

17 Модель игрушек. Классы `Doll`, `CarToy` и `Puzzle` являются производными от класса `Toy`. Метод `__str__()` перегружен только в классе `Doll`, для остальных используется метод из базового класса. **Инструкция:**

- (a) Создайте класс `Toy`, который будет базовым классом для классов `Doll`, `CarToy` и `Puzzle`. В конструкторе класса `Toy` задайте параметры `name`, `brand` и `age_group`.
 - (b) В классе `Toy` создайте метод `play`, который будет возвращать строку 'Играют'.
 - (c) В классе `Toy` создайте метод `__str__`, который будет возвращать строку с информацией об игрушке.
 - (d) Создайте класс `Doll`, который будет наследоваться от класса `Toy`. В конструкторе класса `Doll` задайте параметры `hair_color`, `name`, `brand` и `age_group`. Используйте метод `super().__init__()`.
 - (e) В классе `Doll` переопределите метод `play`, чтобы он возвращал строку 'Одевают и укладывают'.
 - (f) Создайте класс `CarToy`, который будет наследоваться от класса `Toy`. В конструкторе класса `CarToy` задайте параметры `name`, `brand` и `age_group`. Используйте метод `super().__init__()`.
 - (g) В классе `CarToy` переопределите метод `play`, чтобы он возвращал строку 'Катают'.
 - (h) Создайте класс `Puzzle`, который будет наследоваться от класса `Toy`. В конструкторе класса `Puzzle` задайте параметры `name`, `brand` и `age_group`. Используйте метод `super().__init__()`.
 - (i) В классе `Puzzle` переопределите метод `play`, чтобы он возвращал строку 'Собирают'.
 - (j) В основной части программы создайте объекты классов `Doll`, `CarToy` и `Puzzle` и добавьте их в список `toys`.
 - (k) Выведите содержимое списка `toys`, используя метод `play` каждого объекта.
 - (l) Удалите все объекты класса `Doll` из списка `toys`.
 - (m) Выведите оставшееся содержимое списка `toys`, используя метод `play` каждого объекта.
- 18 Модель обуви. Классы `Sneakers`, `Boots` и `Sandals` являются производными от класса `Footwear`. Метод `__str__()` перегружен только в классе `Sneakers`, для остальных используется метод из базового класса. **Инструкция:**
- (a) Создайте класс `Footwear`, который будет базовым классом для классов `Sneakers`, `Boots` и `Sandals`. В конструкторе класса `Footwear` задайте параметры `name`, `brand` и `size`.
 - (b) В классе `Footwear` создайте метод `walk`, который будет возвращать строку 'Носится'.
 - (c) В классе `Footwear` создайте метод `__str__`, который будет возвращать строку с информацией об обуви.
 - (d) Создайте класс `Sneakers`, который будет наследоваться от класса `Footwear`. В конструкторе класса `Sneakers` задайте параметры `sole_type`, `name`, `brand` и `size`. Используйте метод `super().__init__()`.
 - (e) В классе `Sneakers` переопределите метод `walk`, чтобы он возвращал строку 'Бегает'.
 - (f) Создайте класс `Boots`, который будет наследоваться от класса `Footwear`. В конструкторе класса `Boots` задайте параметры `name`, `brand` и `size`. Используйте метод `super().__init__()`.
 - (g) В классе `Boots` переопределите метод `walk`, чтобы он возвращал строку 'Шагает по снегу'.

- (h) Создайте класс `Sandals`, который будет наследоваться от класса `Footwear`. В конструкторе класса `Sandals` задайте параметры `name`, `brand` и `size`. Используйте метод `super().__init__(...)`.
 - (i) В классе `Sandals` переопределите метод `walk`, чтобы он возвращал строку 'Гуляет по пляжу'.
 - (j) В основной части программы создайте объекты классов `Sneakers`, `Boots` и `Sandals` и добавьте их в список `shoes`.
 - (k) Выведите содержимое списка `shoes`, используя метод `walk` каждого объекта.
 - (l) Удалите все объекты класса `Sneakers` из списка `shoes`.
 - (m) Выведите оставшееся содержимое списка `shoes`, используя метод `walk` каждого объекта.
- 19 Модель посуды. Классы `Plate`, `Cup` и `Bowl` являются производными от класса `Dishware`. Метод `__str__()` перегружен только в классе `Plate`, для остальных используется метод из базового класса. **Инструкция:**
- (a) Создайте класс `Dishware`, который будет базовым классом для классов `Plate`, `Cup` и `Bowl`. В конструкторе класса `Dishware` задайте параметры `name`, `material` и `capacity`.
 - (b) В классе `Dishware` создайте метод `serve`, который будет возвращать строку 'Используется'.
 - (c) В классе `Dishware` создайте метод `__str__`, который будет возвращать строку с информацией о посуде.
 - (d) Создайте класс `Plate`, который будет наследоваться от класса `Dishware`. В конструкторе класса `Plate` задайте параметры `diameter`, `name`, `material` и `capacity`. Используйте метод `super().__init__(...)`.
 - (e) В классе `Plate` переопределите метод `serve`, чтобы он возвращал строку 'Подает еду'.
 - (f) Создайте класс `Cup`, который будет наследоваться от класса `Dishware`. В конструкторе класса `Cup` задайте параметры `name`, `material` и `capacity`. Используйте метод `super().__init__(...)`.
 - (g) В классе `Cup` переопределите метод `serve`, чтобы он возвращал строку 'Наливают напиток'.
 - (h) Создайте класс `Bowl`, который будет наследоваться от класса `Dishware`. В конструкторе класса `Bowl` задайте параметры `name`, `material` и `capacity`. Используйте метод `super().__init__(...)`.
 - (i) В классе `Bowl` переопределите метод `serve`, чтобы он возвращал строку 'Содержит суп'.
 - (j) В основной части программы создайте объекты классов `Plate`, `Cup` и `Bowl` и добавьте их в список `dishes`.
 - (k) Выведите содержимое списка `dishes`, используя метод `serve` каждого объекта.
 - (l) Удалите все объекты класса `Plate` из списка `dishes`.
 - (m) Выведите оставшееся содержимое списка `dishes`, используя метод `serve` каждого объекта.

20 Модель мебели для сада. Классы `Bench`, `Table` и `Umbrella` являются производными от класса `GardenFurniture`. Метод `__str__()` перегружен только в классе `Bench`, для остальных используется метод из базового класса. **Инструкция:**

- (a) Создайте класс `GardenFurniture`, который будет базовым классом для классов `Bench`, `Table` и `Umbrella`. В конструкторе класса `GardenFurniture` задайте параметры `name`, `material` и `weather_resistant`.
- (b) В классе `GardenFurniture` создайте метод `use`, который будет возвращать строку 'Используется на улице'.
- (c) В классе `GardenFurniture` создайте метод `__str__`, который будет возвращать строку с информацией о мебели.
- (d) Создайте класс `Bench`, который будет наследоваться от класса `GardenFurniture`. В конструкторе класса `Bench` задайте параметры `seats`, `name`, `material` и `weather_resistant`. Используйте метод `super().__init__()`.
- (e) В классе `Bench` переопределите метод `use`, чтобы он возвращал строку 'Сидят на ней'.
- (f) Создайте класс `Table`, который будет наследоваться от класса `GardenFurniture`. В конструкторе класса `Table` задайте параметры `name`, `material` и `weather_resistant`. Используйте метод `super().__init__()`.
- (g) В классе `Table` переопределите метод `use`, чтобы он возвращал строку 'Ставят на неё еду'.
- (h) Создайте класс `Umbrella`, который будет наследоваться от класса `GardenFurniture`. В конструкторе класса `Umbrella` задайте параметры `name`, `material` и `weather_resistant`. Используйте метод `super().__init__()`.
- (i) В классе `Umbrella` переопределите метод `use`, чтобы он возвращал строку 'Даёт тень'.
- (j) В основной части программы создайте объекты классов `Bench`, `Table` и `Umbrella` и добавьте их в список `garden`.
- (k) Выведите содержимое списка `garden`, используя метод `use` каждого объекта.
- (l) Удалите все объекты класса `Bench` из списка `garden`.
- (m) Выведите оставшееся содержимое списка `garden`, используя метод `use` каждого объекта.

21 Модель упаковки. Классы `Box`, `Bag` и `Envelope` являются производными от класса `Package`. Метод `__str__()` перегружен только в классе `Box`, для остальных используется метод из базового класса. **Инструкция:**

- (a) Создайте класс `Package`, который будет базовым классом для классов `Box`, `Bag` и `Envelope`. В конструкторе класса `Package` задайте параметры `name`, `material` и `capacity`.
- (b) В классе `Package` создайте метод `contain`, который будет возвращать строку 'Содержит'.
- (c) В классе `Package` создайте метод `__str__`, который будет возвращать строку с информацией об упаковке.
- (d) Создайте класс `Box`, который будет наследоваться от класса `Package`. В конструкторе класса `Box` задайте параметры `dimensions`, `name`, `material` и `capacity`. Используйте метод `super().__init__()`.

- (e) В классе `Box` переопределите метод `contain`, чтобы он возвращал строку 'Хранит предметы'.
- (f) Создайте класс `Bag`, который будет наследоваться от класса `Package`. В конструкторе класса `Bag` задайте параметры `name`, `material` и `capacity`. Используйте метод `super().__init__(...)`.
- (g) В классе `Bag` переопределите метод `contain`, чтобы он возвращал строку 'Носит вещи'.
- (h) Создайте класс `Envelope`, который будет наследоваться от класса `Package`. В конструкторе класса `Envelope` задайте параметры `name`, `material` и `capacity`. Используйте метод `super().__init__(...)`.
- (i) В классе `Envelope` переопределите метод `contain`, чтобы он возвращал строку 'Передаёт письма'.
- (j) В основной части программы создайте объекты классов `Box`, `Bag` и `Envelope` и добавьте их в список `storage`.
- (k) Выведите содержимое списка `storage`, используя метод `contain` каждого объекта.
- (l) Удалите все объекты класса `Box` из списка `storage`.
- (m) Выведите оставшееся содержимое списка `storage`, используя метод `contain` каждого объекта.

22 Модель косметики. Классы `Cream`, `Shampoo` и `Lipstick` являются производными от класса `Cosmetic`. Метод `__str__()` перегружен только в классе `Cream`, для остальных используется метод из базового класса. **Инструкция:**

- (a) Создайте класс `Cosmetic`, который будет базовым классом для классов `Cream`, `Shampoo` и `Lipstick`. В конструкторе класса `Cosmetic` задайте параметры `name`, `brand` и `volume`.
- (b) В классе `Cosmetic` создайте метод `apply`, который будет возвращать строку 'Наносится'.
- (c) В классе `Cosmetic` создайте метод `__str__`, который будет возвращать строку с информацией о косметике.
- (d) Создайте класс `Cream`, который будет наследоваться от класса `Cosmetic`. В конструкторе класса `Cream` задайте параметры `skin_type`, `name`, `brand` и `volume`. Используйте метод `super().__init__(...)`.
- (e) В классе `Cream` переопределите метод `apply`, чтобы он возвращал строку 'Увлажняет кожу'.
- (f) Создайте класс `Shampoo`, который будет наследоваться от класса `Cosmetic`. В конструкторе класса `Shampoo` задайте параметры `name`, `brand` и `volume`. Используйте метод `super().__init__(...)`.
- (g) В классе `Shampoo` переопределите метод `apply`, чтобы он возвращал строку 'Моет волосы'.
- (h) Создайте класс `Lipstick`, который будет наследоваться от класса `Cosmetic`. В конструкторе класса `Lipstick` задайте параметры `name`, `brand` и `volume`. Используйте метод `super().__init__(...)`.
- (i) В классе `Lipstick` переопределите метод `apply`, чтобы он возвращал строку 'Красит губы'.

- (j) В основной части программы создайте объекты классов `Cream`, `Shampoo` и `Lipstick` и добавьте их в список `beauty`.
 - (k) Выведите содержимое списка `beauty`, используя метод `apply` каждого объекта.
 - (l) Удалите все объекты класса `Cream` из списка `beauty`.
 - (m) Выведите оставшееся содержимое списка `beauty`, используя метод `apply` каждого объекта.
- 23 Модель канцелярских принадлежностей. Классы `Marker`, `Eraser` и `Stapler` являются производными от класса `OfficeSupply`. Метод `__str__()` перегружен только в классе `Marker`, для остальных используется метод из базового класса. **Инструкция:**
- (a) Создайте класс `OfficeSupply`, который будет базовым классом для классов `Marker`, `Eraser` и `Stapler`. В конструкторе класса `OfficeSupply` задайте параметры `name`, `brand` и `color`.
 - (b) В классе `OfficeSupply` создайте метод `use`, который будет возвращать строку 'Используется'.
 - (c) В классе `OfficeSupply` создайте метод `__str__`, который будет возвращать строку с информацией о принадлежности.
 - (d) Создайте класс `Marker`, который будет наследоваться от класса `OfficeSupply`. В конструкторе класса `Marker` задайте параметры `tip_type`, `name`, `brand` и `color`. Используйте метод `super().__init__(...)`.
 - (e) В классе `Marker` переопределите метод `use`, чтобы он возвращал строку 'Рисует'.
 - (f) Создайте класс `Eraser`, который будет наследоваться от класса `OfficeSupply`. В конструкторе класса `Eraser` задайте параметры `name`, `brand` и `color`. Используйте метод `super().__init__(...)`.
 - (g) В классе `Eraser` переопределите метод `use`, чтобы он возвращал строку 'Стирает'.
 - (h) Создайте класс `Stapler`, который будет наследоваться от класса `OfficeSupply`. В конструкторе класса `Stapler` задайте параметры `name`, `brand` и `color`. Используйте метод `super().__init__(...)`.
 - (i) В классе `Stapler` переопределите метод `use`, чтобы он возвращал строку 'Скрепляет'.
 - (j) В основной части программы создайте объекты классов `Marker`, `Eraser` и `Stapler` и добавьте их в список `supplies`.
 - (k) Выведите содержимое списка `supplies`, используя метод `use` каждого объекта.
 - (l) Удалите все объекты класса `Marker` из списка `supplies`.
 - (m) Выведите оставшееся содержимое списка `supplies`, используя метод `use` каждого объекта.
- 24 Модель транспортных средств для доставки. Классы `Bike`, `Van` и `Drone` являются производными от класса `DeliveryVehicle`. Метод `__str__()` перегружен только в классе `Bike`, для остальных используется метод из базового класса. **Инструкция:**
- (a) Создайте класс `DeliveryVehicle`, который будет базовым классом для классов `Bike`, `Van` и `Drone`. В конструкторе класса `DeliveryVehicle` задайте параметры `name`, `brand` и `max_load`.
 - (b) В классе `DeliveryVehicle` создайте метод `deliver`, который будет возвращать строку 'Доставляет'.

- (c) В классе `DeliveryVehicle` создайте метод `__str__`, который будет возвращать строку с информацией о транспорте.
 - (d) Создайте класс `Bike`, который будет наследоваться от класса `DeliveryVehicle`. В конструкторе класса `Bike` задайте параметры `electric`, `name`, `brand` и `max_load`. Используйте метод `super().__init__(...)`.
 - (e) В классе `Bike` переопределите метод `deliver`, чтобы он возвращал строку 'Едет по городу'.
 - (f) Создайте класс `Van`, который будет наследоваться от класса `DeliveryVehicle`. В конструкторе класса `Van` задайте параметры `name`, `brand` и `max_load`. Используйте метод `super().__init__(...)`.
 - (g) В классе `Van` переопределите метод `deliver`, чтобы он возвращал строку 'Везёт посылки'.
 - (h) Создайте класс `Drone`, который будет наследоваться от класса `DeliveryVehicle`. В конструкторе класса `Drone` задайте параметры `name`, `brand` и `max_load`. Используйте метод `super().__init__(...)`.
 - (i) В классе `Drone` переопределите метод `deliver`, чтобы он возвращал строку 'Летит с грузом'.
 - (j) В основной части программы создайте объекты классов `Bike`, `Van` и `Drone` и добавьте их в список `delivery`.
 - (k) Выведите содержимое списка `delivery`, используя метод `deliver` каждого объекта.
 - (l) Удалите все объекты класса `Bike` из списка `delivery`.
 - (m) Выведите оставшееся содержимое списка `delivery`, используя метод `deliver` каждого объекта.
- 25 Модель электронных компонентов. Классы `Resistor`, `Capacitor` и `Diode` являются производными от класса `Component`. Метод `__str__()` перегружен только в классе `Resistor`, для остальных используется метод из базового класса. **Инструкция:**
- (a) Создайте класс `Component`, который будет базовым классом для классов `Resistor`, `Capacitor` и `Diode`. В конструкторе класса `Component` задайте параметры `name`, `manufacturer` и `value`.
 - (b) В классе `Component` создайте метод `function`, который будет возвращать строку 'Работает в цепи'.
 - (c) В классе `Component` создайте метод `__str__`, который будет возвращать строку с информацией о компоненте.
 - (d) Создайте класс `Resistor`, который будет наследоваться от класса `Component`. В конструкторе класса `Resistor` задайте параметры `tolerance`, `name`, `manufacturer` и `value`. Используйте метод `super().__init__(...)`.
 - (e) В классе `Resistor` переопределите метод `function`, чтобы он возвращал строку 'Ограничивает ток'.
 - (f) Создайте класс `Capacitor`, который будет наследоваться от класса `Component`. В конструкторе класса `Capacitor` задайте параметры `name`, `manufacturer` и `value`. Используйте метод `super().__init__(...)`.

- (g) В классе `Capacitor` переопределите метод `function`, чтобы он возвращал строку 'Накапливает заряд'.
 - (h) Создайте класс `Diode`, который будет наследоваться от класса `Component`. В конструкторе класса `Diode` задайте параметры `name`, `manufacturer` и `value`. Используйте метод `super().__init__(...)`.
 - (i) В классе `Diode` переопределите метод `function`, чтобы он возвращал строку 'Пропускает ток в одну сторону'.
 - (j) В основной части программы создайте объекты классов `Resistor`, `Capacitor` и `Diode` и добавьте их в список `circuit`.
 - (k) Выведите содержимое списка `circuit`, используя метод `function` каждого объекта.
 - (l) Удалите все объекты класса `Resistor` из списка `circuit`.
 - (m) Выведите оставшееся содержимое списка `circuit`, используя метод `function` каждого объекта.
- 26 Модель продуктов питания. Классы `Bread`, `Cheese` и `Apple` являются производными от класса `Food`. Метод `__str__()` перегружен только в классе `Bread`, для остальных используется метод из базового класса. **Инструкция:**
- (a) Создайте класс `Food`, который будет базовым классом для классов `Bread`, `Cheese` и `Apple`. В конструкторе класса `Food` задайте параметры `name`, `brand` и `calories`.
 - (b) В классе `Food` создайте метод `eat`, который будет возвращать строку 'Едят'.
 - (c) В классе `Food` создайте метод `__str__`, который будет возвращать строку с информацией о продукте.
 - (d) Создайте класс `Bread`, который будет наследоваться от класса `Food`. В конструкторе класса `Bread` задайте параметры `type`, `name`, `brand` и `calories`. Используйте метод `super().__init__(...)`.
 - (e) В классе `Bread` переопределите метод `eat`, чтобы он возвращал строку 'Намазывают маслом'.
 - (f) Создайте класс `Cheese`, который будет наследоваться от класса `Food`. В конструкторе класса `Cheese` задайте параметры `name`, `brand` и `calories`. Используйте метод `super().__init__(...)`.
 - (g) В классе `Cheese` переопределите метод `eat`, чтобы он возвращал строку 'Кладут на бутерброд'.
 - (h) Создайте класс `Apple`, который будет наследоваться от класса `Food`. В конструкторе класса `Apple` задайте параметры `name`, `brand` и `calories`. Используйте метод `super().__init__(...)`.
 - (i) В классе `Apple` переопределите метод `eat`, чтобы он возвращал строку 'Грызут'.
 - (j) В основной части программы создайте объекты классов `Bread`, `Cheese` и `Apple` и добавьте их в список `fridge`.
 - (k) Выведите содержимое списка `fridge`, используя метод `eat` каждого объекта.
 - (l) Удалите все объекты класса `Bread` из списка `fridge`.
 - (m) Выведите оставшееся содержимое списка `fridge`, используя метод `eat` каждого объекта.

27 Модель аксессуаров для телефона. Классы `Case`, `Charger` и `Headphones` являются производными от класса `Accessory`. Метод `__str__()` перегружен только в классе `Case`, для остальных используется метод из базового класса. **Инструкция:**

- (a) Создайте класс `Accessory`, который будет базовым классом для классов `Case`, `Charger` и `Headphones`. В конструкторе класса `Accessory` задайте параметры `name`, `brand` и `compatibility`.
- (b) В классе `Accessory` создайте метод `use`, который будет возвращать строку 'Используется'.
- (c) В классе `Accessory` создайте метод `__str__`, который будет возвращать строку с информацией об аксессуаре.
- (d) Создайте класс `Case`, который будет наследоваться от класса `Accessory`. В конструкторе класса `Case` задайте параметры `material`, `name`, `brand` и `compatibility`. Используйте метод `super().__init__(...)`.
- (e) В классе `Case` переопределите метод `use`, чтобы он возвращал строку 'Защищает телефон'.
- (f) Создайте класс `Charger`, который будет наследоваться от класса `Accessory`. В конструкторе класса `Charger` задайте параметры `name`, `brand` и `compatibility`. Используйте метод `super().__init__(...)`.
- (g) В классе `Charger` переопределите метод `use`, чтобы он возвращал строку 'Заряжает'.
- (h) Создайте класс `Headphones`, который будет наследоваться от класса `Accessory`. В конструкторе класса `Headphones` задайте параметры `name`, `brand` и `compatibility`. Используйте метод `super().__init__(...)`.
- (i) В классе `Headphones` переопределите метод `use`, чтобы он возвращал строку 'Воспроизводит звук'.
- (j) В основной части программы создайте объекты классов `Case`, `Charger` и `Headphones` и добавьте их в список `gadgets`.
- (k) Выведите содержимое списка `gadgets`, используя метод `use` каждого объекта.
- (l) Удалите все объекты класса `Case` из списка `gadgets`.
- (m) Выведите оставшееся содержимое списка `gadgets`, используя метод `use` каждого объекта.

28 Модель музыкальных жанров. Классы `Rock`, `Jazz` и `Pop` являются производными от класса `Genre`. Метод `__str__()` перегружен только в классе `Rock`, для остальных используется метод из базового класса. **Инструкция:**

- (a) Создайте класс `Genre`, который будет базовым классом для классов `Rock`, `Jazz` и `Pop`. В конструкторе класса `Genre` задайте параметры `name`, `origin` и `decade`.
- (b) В классе `Genre` создайте метод `perform`, который будет возвращать строку 'Исполняется'.
- (c) В классе `Genre` создайте метод `__str__`, который будет возвращать строку с информацией о жанре.
- (d) Создайте класс `Rock`, который будет наследоваться от класса `Genre`. В конструкторе класса `Rock` задайте параметры `subgenre`, `name`, `origin` и `decade`. Используйте метод `super().__init__(...)`.
- (e) В классе `Rock` переопределите метод `perform`, чтобы он возвращал строку 'Гремит'.

- (f) Создайте класс `Jazz`, который будет наследоваться от класса `Genre`. В конструкторе класса `Jazz` задайте параметры `name`, `origin` и `decade`. Используйте метод `super().__init__(...)`.
 - (g) В классе `Jazz` переопределите метод `perform`, чтобы он возвращал строку 'Импровизирует'.
 - (h) Создайте класс `Pop`, который будет наследоваться от класса `Genre`. В конструкторе класса `Pop` задайте параметры `name`, `origin` и `decade`. Используйте метод `super().__init__(...)`.
 - (i) В классе `Pop` переопределите метод `perform`, чтобы он возвращал строку 'Звучит на радио'.
 - (j) В основной части программы создайте объекты классов `Rock`, `Jazz` и `Pop` и добавьте их в список `music`.
 - (k) Выведите содержимое списка `music`, используя метод `perform` каждого объекта.
 - (l) Удалите все объекты класса `Rock` из списка `music`.
 - (m) Выведите оставшееся содержимое списка `music`, используя метод `perform` каждого объекта.
- 29 Модель видов спорта. Классы `Football`, `Swimming` и `Chess` являются производными от класса `Sport`. Метод `__str__()` перегружен только в классе `Football`, для остальных используется метод из базового класса. **Инструкция:**
- (a) Создайте класс `Sport`, который будет базовым классом для классов `Football`, `Swimming` и `Chess`. В конструкторе класса `Sport` задайте параметры `name`, `type` и `players`.
 - (b) В классе `Sport` создайте метод `play`, который будет возвращать строку 'Практикуется'.
 - (c) В классе `Sport` создайте метод `__str__`, который будет возвращать строку с информацией о виде спорта.
 - (d) Создайте класс `Football`, который будет наследоваться от класса `Sport`. В конструкторе класса `Football` задайте параметры `field_type`, `name`, `type` и `players`. Используйте метод `super().__init__(...)`.
 - (e) В классе `Football` переопределите метод `play`, чтобы он возвращал строку 'Играют на поле'.
 - (f) Создайте класс `Swimming`, который будет наследоваться от класса `Sport`. В конструкторе класса `Swimming` задайте параметры `name`, `type` и `players`. Используйте метод `super().__init__(...)`.
 - (g) В классе `Swimming` переопределите метод `play`, чтобы он возвращал строку 'Плавают'.
 - (h) Создайте класс `Chess`, который будет наследоваться от класса `Sport`. В конструкторе класса `Chess` задайте параметры `name`, `type` и `players`. Используйте метод `super().__init__(...)`.
 - (i) В классе `Chess` переопределите метод `play`, чтобы он возвращал строку 'Думают'.
 - (j) В основной части программы создайте объекты классов `Football`, `Swimming` и `Chess` и добавьте их в список `sports`.
 - (k) Выведите содержимое списка `sports`, используя метод `play` каждого объекта.
 - (l) Удалите все объекты класса `Football` из списка `sports`.

- (m) Выведите оставшееся содержимое списка `sports`, используя метод `play` каждого объекта.
- 30 Модель видов транспорта. Классы `Airplane`, `Ship` и `Train` являются производными от класса `Transport`. Метод `__str__()` перегружен только в классе `Airplane`, для остальных используется метод из базового класса. **Инструкция:**
- (a) Создайте класс `Transport`, который будет базовым классом для классов `Airplane`, `Ship` и `Train`. В конструкторе класса `Transport` задайте параметры `name`, `capacity` и `speed`.
 - (b) В классе `Transport` создайте метод `move`, который будет возвращать строку 'Перемещается'.
 - (c) В классе `Transport` создайте метод `__str__`, который будет возвращать строку с информацией о транспорте.
 - (d) Создайте класс `Airplane`, который будет наследоваться от класса `Transport`. В конструкторе класса `Airplane` задайте параметры `range`, `name`, `capacity` и `speed`. Используйте метод `super().__init__()`.
 - (e) В классе `Airplane` переопределите метод `move`, чтобы он возвращал строку 'Летит'.
 - (f) Создайте класс `Ship`, который будет наследоваться от класса `Transport`. В конструкторе класса `Ship` задайте параметры `name`, `capacity` и `speed`. Используйте метод `super().__init__()`.
 - (g) В классе `Ship` переопределите метод `move`, чтобы он возвращал строку 'Плывёт'.
 - (h) Создайте класс `Train`, который будет наследоваться от класса `Transport`. В конструкторе класса `Train` задайте параметры `name`, `capacity` и `speed`. Используйте метод `super().__init__()`.
 - (i) В классе `Train` переопределите метод `move`, чтобы он возвращал строку 'Едет по рельсам'.
 - (j) В основной части программы создайте объекты классов `Airplane`, `Ship` и `Train` и добавьте их в список `transports`.
 - (k) Выведите содержимое списка `transports`, используя метод `move` каждого объекта.
 - (l) Удалите все объекты класса `Airplane` из списка `transports`.
 - (m) Выведите оставшееся содержимое списка `transports`, используя метод `move` каждого объекта.
- 31 Модель видов освещения. Классы `Lamp`, `Flashlight` и `Neon` являются производными от класса `Light`. Метод `__str__()` перегружен только в классе `Lamp`, для остальных используется метод из базового класса. **Инструкция:**
- (a) Создайте класс `Light`, который будет базовым классом для классов `Lamp`, `Flashlight` и `Neon`. В конструкторе класса `Light` задайте параметры `name`, `power` и `color`.
 - (b) В классе `Light` создайте метод `illuminate`, который будет возвращать строку 'Освещает'.
 - (c) В классе `Light` создайте метод `__str__`, который будет возвращать строку с информацией об источнике света.
 - (d) Создайте класс `Lamp`, который будет наследоваться от класса `Light`. В конструкторе класса `Lamp` задайте параметры `type`, `name`, `power` и `color`. Используйте метод `super().__init__()`.

- (e) В классе `Lamp` переопределите метод `illuminate`, чтобы он возвращал строку `'Светит в комнате'`.
- (f) Создайте класс `Flashlight`, который будет наследоваться от класса `Light`. В конструкторе класса `Flashlight` задайте параметры `name`, `power` и `color`. Используйте метод `super().__init__()`.
- (g) В классе `Flashlight` переопределите метод `illuminate`, чтобы он возвращал строку `'Светит в темноте'`.
- (h) Создайте класс `Neon`, который будет наследоваться от класса `Light`. В конструкторе класса `Neon` задайте параметры `name`, `power` и `color`. Используйте метод `super().__init__()`.
- (i) В классе `Neon` переопределите метод `illuminate`, чтобы он возвращал строку `'Мигает на вывеске'`.
- (j) В основной части программы создайте объекты классов `Lamp`, `Flashlight` и `Neon` и добавьте их в список `lights`.
- (k) Выведите содержимое списка `lights`, используя метод `illuminate` каждого объекта.
- (l) Удалите все объекты класса `Lamp` из списка `lights`.
- (m) Выведите оставшееся содержимое списка `lights`, используя метод `illuminate` каждого объекта.

32 Модель видов бумаги. Классы `PrinterPaper`, `NotebookPaper` и `WrappingPaper` являются производными от класса `Paper`. Метод `__str__()` перегружен только в классе `PrinterPaper`, для остальных используется метод из базового класса. **Инструкция:**

- (a) Создайте класс `Paper`, который будет базовым классом для классов `PrinterPaper`, `NotebookPaper` и `WrappingPaper`. В конструкторе класса `Paper` задайте параметры `name`, `brand` и `density`.
- (b) В классе `Paper` создайте метод `use`, который будет возвращать строку `'Используется'`.
- (c) В классе `Paper` создайте метод `__str__`, который будет возвращать строку с информацией о бумаге.
- (d) Создайте класс `PrinterPaper`, который будет наследоваться от класса `Paper`. В конструкторе класса `PrinterPaper` задайте параметры `format`, `name`, `brand` и `density`. Используйте метод `super().__init__()`.
- (e) В классе `PrinterPaper` переопределите метод `use`, чтобы он возвращал строку `'Печатается'`.
- (f) Создайте класс `NotebookPaper`, который будет наследоваться от класса `Paper`. В конструкторе класса `NotebookPaper` задайте параметры `name`, `brand` и `density`. Используйте метод `super().__init__()`.
- (g) В классе `NotebookPaper` переопределите метод `use`, чтобы он возвращал строку `'Пишут на нём'`.
- (h) Создайте класс `WrappingPaper`, который будет наследоваться от класса `Paper`. В конструкторе класса `WrappingPaper` задайте параметры `name`, `brand` и `density`. Используйте метод `super().__init__()`.
- (i) В классе `WrappingPaper` переопределите метод `use`, чтобы он возвращал строку `'Оборачивают подарки'`.

- (j) В основной части программы создайте объекты классов `PrinterPaper`, `NotebookPaper` и `WrappingPaper` и добавьте их в список `papers`.
 - (k) Выведите содержимое списка `papers`, используя метод `use` каждого объекта.
 - (l) Удалите все объекты класса `PrinterPaper` из списка `papers`.
 - (m) Выведите оставшееся содержимое списка `papers`, используя метод `use` каждого объекта.
- 33 Модель видов сумок. Классы `Backpack`, `Briefcase` и `Tote` являются производными от класса `Bag`. Метод `__str__()` перегружен только в классе `Backpack`, для остальных используется метод из базового класса. **Инструкция:**
- (a) Создайте класс `Bag`, который будет базовым классом для классов `Backpack`, `Briefcase` и `Tote`. В конструкторе класса `Bag` задайте параметры `name`, `material` и `capacity`.
 - (b) В классе `Bag` создайте метод `carry`, который будет возвращать строку 'Носится'.
 - (c) В классе `Bag` создайте метод `__str__`, который будет возвращать строку с информацией о сумке.
 - (d) Создайте класс `Backpack`, который будет наследоваться от класса `Bag`. В конструкторе класса `Backpack` задайте параметры `compartments`, `name`, `material` и `capacity`. Используйте метод `super().__init__()`.
 - (e) В классе `Backpack` переопределите метод `carry`, чтобы он возвращал строку 'Носится за спиной'.
 - (f) Создайте класс `Briefcase`, который будет наследоваться от класса `Bag`. В конструкторе класса `Briefcase` задайте параметры `name`, `material` и `capacity`. Используйте метод `super().__init__()`.
 - (g) В классе `Briefcase` переопределите метод `carry`, чтобы он возвращал строку 'Носится в руке'.
 - (h) Создайте класс `Tote`, который будет наследоваться от класса `Bag`. В конструкторе класса `Tote` задайте параметры `name`, `material` и `capacity`. Используйте метод `super().__init__()`.
 - (i) В классе `Tote` переопределите метод `carry`, чтобы он возвращал строку 'Носится на плече'.
 - (j) В основной части программы создайте объекты классов `Backpack`, `Briefcase` и `Tote` и добавьте их в список `bags`.
 - (k) Выведите содержимое списка `bags`, используя метод `carry` каждого объекта.
 - (l) Удалите все объекты класса `Backpack` из списка `bags`.
 - (m) Выведите оставшееся содержимое списка `bags`, используя метод `carry` каждого объекта.
- 34 Модель видов часов. Классы `Wristwatch`, `WallClock` и `AlarmClock` являются производными от класса `Clock`. Метод `__str__()` перегружен только в классе `Wristwatch`, для остальных используется метод из базового класса. **Инструкция:**
- (a) Создайте класс `Clock`, который будет базовым классом для классов `Wristwatch`, `WallClock` и `AlarmClock`. В конструкторе класса `Clock` задайте параметры `name`, `brand` и `power_source`.

- (b) В классе `Clock` создайте метод `show_time`, который будет возвращать строку 'Показывает время'.
 - (c) В классе `Clock` создайте метод `__str__`, который будет возвращать строку с информацией о часах.
 - (d) Создайте класс `Wristwatch`, который будет наследоваться от класса `Clock`. В конструкторе класса `Wristwatch` задайте параметры `strap_material`, `name`, `brand` и `power_source`. Используйте метод `super().__init__(...)`.
 - (e) В классе `Wristwatch` переопределите метод `show_time`, чтобы он возвращал строку 'На запястье'.
 - (f) Создайте класс `WallClock`, который будет наследоваться от класса `Clock`. В конструкторе класса `WallClock` задайте параметры `name`, `brand` и `power_source`. Используйте метод `super().__init__(...)`.
 - (g) В классе `WallClock` переопределите метод `show_time`, чтобы он возвращал строку 'На стене'.
 - (h) Создайте класс `AlarmClock`, который будет наследоваться от класса `Clock`. В конструкторе класса `AlarmClock` задайте параметры `name`, `brand` и `power_source`. Используйте метод `super().__init__(...)`.
 - (i) В классе `AlarmClock` переопределите метод `show_time`, чтобы он возвращал строку 'Будит утром'.
 - (j) В основной части программы создайте объекты классов `Wristwatch`, `WallClock` и `AlarmClock` и добавьте их в список `clocks`.
 - (k) Выведите содержимое списка `clocks`, используя метод `show_time` каждого объекта.
 - (l) Удалите все объекты класса `Wristwatch` из списка `clocks`.
 - (m) Выведите оставшееся содержимое списка `clocks`, используя метод `show_time` каждого объекта.
- 35 Модель видов контейнеров. Классы `Jar`, `Can` и `Bottle` являются производными от класса `Container`. Метод `__str__()` перегружен только в классе `Jar`, для остальных используется метод из базового класса. **Инструкция:**
- (a) Создайте класс `Container`, который будет базовым классом для классов `Jar`, `Can` и `Bottle`. В конструкторе класса `Container` задайте параметры `name`, `material` и `volume`.
 - (b) В классе `Container` создайте метод `hold`, который будет возвращать строку 'Содержит'.
 - (c) В классе `Container` создайте метод `__str__`, который будет возвращать строку с информацией о контейнере.
 - (d) Создайте класс `Jar`, который будет наследоваться от класса `Container`. В конструкторе класса `Jar` задайте параметры `lid_type`, `name`, `material` и `volume`. Используйте метод `super().__init__(...)`.
 - (e) В классе `Jar` переопределите метод `hold`, чтобы он возвращал строку 'Хранит варенье'.
 - (f) Создайте класс `Can`, который будет наследоваться от класса `Container`. В конструкторе класса `Can` задайте параметры `name`, `material` и `volume`. Используйте метод `super().__init__(...)`.

- (g) В классе `Can` переопределите метод `hold`, чтобы он возвращал строку 'Содержит газировку'.
- (h) Создайте класс `Bottle`, который будет наследоваться от класса `Container`. В конструкторе класса `Bottle` задайте параметры `name`, `material` и `volume`. Используйте метод `super().__init__(...)`.
- (i) В классе `Bottle` переопределите метод `hold`, чтобы он возвращал строку 'Наливают воду'.
- (j) В основной части программы создайте объекты классов `Jar`, `Can` и `Bottle` и добавьте их в список `containers`.
- (k) Выведите содержимое списка `containers`, используя метод `hold` каждого объекта.
- (l) Удалите все объекты класса `Jar` из списка `containers`.
- (m) Выведите оставшееся содержимое списка `containers`, используя метод `hold` каждого объекта.

2.7.3 Задача 3.

- 1 Создайте при помощи наследования ООП-модель команды ресторана по приготовлению, например, пиццы. Опишите наиболее универсальный класс `Employee`, который предоставляет общее поведение, такое как повышение зарплаты (`giveRaise`) и строковое представление объекта (`__repr__`). В команде два работника, поэтому должно быть два подкласса `Employee` — `Chef` (шеф-повар) и `Server` (официант). В обоих подклассах переопределяется метод `work`, чтобы выводить специфические сообщения. Также в команде должен быть робот по приготовлению пиццы, который моделируется более специфическим классом — `PizzaRobot` представляет собой разновидность класса `Chef`, который, в свою очередь, является видом `Employee`.

Инструкция:

- (a) Создайте класс `Employee`, который будет базовым классом для всех остальных классов. В конструкторе класса `Employee` задайте параметры `name` и `salary`.
- (b) В классе `Employee` создайте метод `giveRaise`, который будет увеличивать зарплату на заданный процент.
- (c) В классе `Employee` создайте метод `work`, который будет выводить сообщение о том, что сотрудник работает.
- (d) В классе `Employee` создайте метод `__repr__`, который будет возвращать строку с информацией о сотруднике.
- (e) Создайте класс `Chef`, который будет наследоваться от класса `Employee`. В конструкторе класса `Chef` задайте начальную зарплату.
- (f) В классе `Chef` переопределите метод `work`, чтобы он выводил сообщение о том, что повар готовит еду.
- (g) Создайте класс `Server`, который будет наследоваться от класса `Employee`. В конструкторе класса `Server` задайте начальную зарплату.
- (h) В классе `Server` переопределите метод `work`, чтобы он выводил сообщение о том, что официант обслуживает клиентов.
- (i) Создайте класс `PizzaRobot`, который будет наследоваться от класса `Chef`. В конструкторе класса `PizzaRobot` задайте начальную зарплату.

- (j) В классе `PizzaRobot` переопределите метод `work`, чтобы он выводил сообщение о том, что робот готовит пиццу.
 - (k) В основной части программы создайте объект `bob` класса `PizzaRobot` и вызовите его методы `work` и `giveRaise`.
 - (l) Для каждого класса `Employee`, `Chef`, `Server` и `PizzaRobot` создайте объект и вызовите его метод `work`.
- 2 Создайте иерархию классов для моделирования команды ветеринарной клиники. Базовый класс `Staff` должен содержать общие атрибуты и методы для всех сотрудников. От него наследуются два подкласса: `Veterinarian` (ветеринар) и `Receptionist` (администратор). Также в клинике используется робот-ассистент для базовой диагностики животных — `DiagRobot`, который является подклассом `Veterinarian`.

Инструкция:

- (a) Создайте класс `Staff` с параметрами `name` и `salary` в конструкторе.
 - (b) Реализуйте в `Staff` метод `giveRaise`, увеличивающий зарплату на заданный процент.
 - (c) Добавьте метод `work`, выводящий общее сообщение о работе.
 - (d) Реализуйте метод `__repr__`, возвращающий строку с именем и должностью.
 - (e) Создайте класс `Veterinarian`, наследующий `Staff`, с фиксированной начальной зарплатой.
 - (f) Переопределите `work` в `Veterinarian`, чтобы он выводил сообщение о лечении животных.
 - (g) Создайте класс `Receptionist`, наследующий `Staff`, с собственной начальной зарплатой.
 - (h) Переопределите `work` в `Receptionist`, чтобы он выводил сообщение о приёме звонков и записи клиентов.
 - (i) Создайте класс `DiagRobot`, наследующий `Veterinarian`, с начальной зарплатой.
 - (j) Переопределите `work` в `DiagRobot`, чтобы он выводил сообщение о проведении базовой диагностики.
 - (k) Создайте объект `robo` класса `DiagRobot` и вызовите его методы `work` и `giveRaise`.
 - (l) Создайте по одному экземпляру каждого класса и вызовите у каждого метод `work`.
- 3 Разработайте ООП-модель для команды автосервиса. Базовый класс `Worker` описывает общие свойства и поведение персонала. От него наследуются `Mechanic` (механик) и `Cashier` (кассир). Кроме того, в сервисе работает робот-мойщик — `WashRobot`, который является подклассом `Mechanic`.

Инструкция:

- (a) Создайте класс `Worker` с атрибутами `name` и `salary`.
- (b) Реализуйте метод `giveRaise` для увеличения зарплат на процент.
- (c) Добавьте метод `work`, выводящий общее сообщение о работе.
- (d) Реализуйте метод `__repr__` для строкового представления объекта.
- (e) Создайте класс `Mechanic`, наследующий `Worker`, с начальной зарплатой.

- (f) Переопределите `work` в `Mechanic`, чтобы он выводил сообщение о ремонте автомобилей.
 - (g) Создайте класс `Cashier`, наследующий `Worker`, с начальной зарплатой.
 - (h) Переопределите `work` в `Cashier`, чтобы он выводил сообщение о приёме платежей.
 - (i) Создайте класс `WashRobot`, наследующий `Mechanic`, с начальной зарплатой.
 - (j) Переопределите `work` в `WashRobot`, чтобы он выводил сообщение о мойке машин.
 - (k) Создайте объект `cleanbot` класса `WashRobot` и вызовите его методы `work` и `giveRaise`.
 - (l) Создайте по одному экземпляру каждого класса и вызовите у каждого метод `work`.
- 4 Создайте иерархию классов для моделирования команды библиотеки. Базовый класс `LibStaff` описывает общие черты сотрудников. От него наследуются `Librarian` (библиотекарь) и `Security` (охранник). Также в библиотеке используется робот-сортировщик книг — `SortRobot`, который является подклассом `Librarian`.

Инструкция:

- (a) Создайте класс `LibStaff` с параметрами `name` и `salary`.
 - (b) Реализуйте метод `giveRaise` для увеличения зарплаты.
 - (c) Добавьте метод `work`, выводящий общее сообщение.
 - (d) Реализуйте метод `__repr__` для строкового представления.
 - (e) Создайте класс `Librarian`, наследующий `LibStaff`, с начальной зарплатой.
 - (f) Переопределите `work` в `Librarian`, чтобы он выводил сообщение о выдаче книг.
 - (g) Создайте класс `Security`, наследующий `LibStaff`, с начальной зарплатой.
 - (h) Переопределите `work` в `Security`, чтобы он выводил сообщение о патрулировании.
 - (i) Создайте класс `SortRobot`, наследующий `Librarian`, с начальной зарплатой.
 - (j) Переопределите `work` в `SortRobot`, чтобы он выводил сообщение о сортировке книг.
 - (k) Создайте объект `bookbot` класса `SortRobot` и вызовите его методы `work` и `giveRaise`.
 - (l) Создайте по одному экземпляру каждого класса и вызовите у каждого метод `work`.
- 5 Создайте модель команды аптеки с использованием наследования. Базовый класс `PharmStaff` описывает общие свойства. От него наследуются `Pharmacist` (фармацевт) и `Cashier` (кассир). Также в аптеке есть робот-упаковщик — `PackRobot`, подкласс `Pharmacist`.

Инструкция:

- (a) Создайте класс `PharmStaff` с атрибутами `name` и `salary`.
- (b) Реализуйте метод `giveRaise`.
- (c) Добавьте метод `work` с общим сообщением.
- (d) Реализуйте метод `__repr__`.
- (e) Создайте класс `Pharmacist`, наследующий `PharmStaff`, с начальной зарплатой.

- (f) Переопределите `work` в `Pharmacist`, чтобы он выводил сообщение о выдаче лекарств.
- (g) Создайте класс `Cashier`, наследующий `PharmStaff`, с начальной зарплатой.
- (h) Переопределите `work` в `Cashier`, чтобы он выводил сообщение о приёме оплаты.
- (i) Создайте класс `PackRobot`, наследующий `Pharmacist`, с начальной зарплатой.
- (j) Переопределите `work` в `PackRobot`, чтобы он выводил сообщение об упаковке заказов.
- (k) Создайте объект `packy` класса `PackRobot` и вызовите его методы `work` и `giveRaise`.
- (l) Создайте по одному экземпляру каждого класса и вызовите у каждого метод `work`.

6 Разработайте иерархию классов для команды зоопарка. Базовый класс `ZooStaff` описывает общие черты. От него наследуются `Keeper` (смотритель) и `TicketSeller` (продавец билетов). Также в зоопарке работает робот-кормушка — `FeedRobot`, подкласс `Keeper`.

Инструкция:

- (a) Создайте класс `ZooStaff` с параметрами `name` и `salary`.
- (b) Реализуйте метод `giveRaise`.
- (c) Добавьте метод `work` с общим сообщением.
- (d) Реализуйте метод `__repr__`.
- (e) Создайте класс `Keeper`, наследующий `ZooStaff`, с начальной зарплатой.
- (f) Переопределите `work` в `Keeper`, чтобы он выводил сообщение о кормлении животных.
- (g) Создайте класс `TicketSeller`, наследующий `ZooStaff`, с начальной зарплатой.
- (h) Переопределите `work` в `TicketSeller`, чтобы он выводил сообщение о продаже билетов.
- (i) Создайте класс `FeedRobot`, наследующий `Keeper`, с начальной зарплатой.
- (j) Переопределите `work` в `FeedRobot`, чтобы он выводил сообщение о автоматической подаче корма.
- (k) Создайте объект `feedy` класса `FeedRobot` и вызовите его методы `work` и `giveRaise`.
- (l) Создайте по одному экземпляру каждого класса и вызовите у каждого метод `work`.

7 Создайте ООП-модель для команды кинотеатра. Базовый класс `CinemaStaff` описывает общие свойства. От него наследуются `Projectionist` (оператор) и `Usher` (расклейщик/проводник). Также в кинотеатре есть робот-уборщик — `CleanRobot`, подкласс `Usher`.

Инструкция:

- (a) Создайте класс `CinemaStaff` с атрибутами `name` и `salary`.
- (b) Реализуйте метод `giveRaise`.
- (c) Добавьте метод `work` с общим сообщением.
- (d) Реализуйте метод `__repr__`.

- (e) Создайте класс `Projectionist`, наследующий `CinemaStaff`, с начальной зарплатой.
- (f) Переопределите `work` в `Projectionist`, чтобы он выводил сообщение о запуске фильма.
- (g) Создайте класс `Usher`, наследующий `CinemaStaff`, с начальной зарплатой.
- (h) Переопределите `work` в `Usher`, чтобы он выводил сообщение о раздаче попкорна и указании мест.
- (i) Создайте класс `CleanRobot`, наследующий `Usher`, с начальной зарплатой.
- (j) Переопределите `work` в `CleanRobot`, чтобы он выводил сообщение об уборке зала после сеанса.
- (k) Создайте объект `cleany` класса `CleanRobot` и вызовите его методы `work` и `giveRaise`.
- (l) Создайте по одному экземпляру каждого класса и вызовите у каждого метод `work`.

8 Разработайте модель команды почтового отделения. Базовый класс `PostStaff` описывает общие черты. От него наследуются `Clerk` (почтальон) и `Manager` (менеджер). Также в отделении работает робот-сортировщик писем — `SortBot`, подкласс `Clerk`.

Инструкция:

- (a) Создайте класс `PostStaff` с параметрами `name` и `salary`.
- (b) Реализуйте метод `giveRaise`.
- (c) Добавьте метод `work` с общим сообщением.
- (d) Реализуйте метод `__repr__`.
- (e) Создайте класс `Clerk`, наследующий `PostStaff`, с начальной зарплатой.
- (f) Переопределите `work` в `Clerk`, чтобы он выводил сообщение о разное почты.
- (g) Создайте класс `Manager`, наследующий `PostStaff`, с начальной зарплатой.
- (h) Переопределите `work` в `Manager`, чтобы он выводил сообщение о координации работы.
- (i) Создайте класс `SortBot`, наследующий `Clerk`, с начальной зарплатой.
- (j) Переопределите `work` в `SortBot`, чтобы он выводил сообщение о сортировке корреспонденции.
- (k) Создайте объект `sorty` класса `SortBot` и вызовите его методы `work` и `giveRaise`.
- (l) Создайте по одному экземпляру каждого класса и вызовите у каждого метод `work`.

9 Создайте иерархию классов для команды кафе. Базовый класс `CafeStaff` описывает общие свойства. От него наследуются `Barista` (бариста) и `Host` (хост). Также в кафе работает робот-мытарь чашек — `WashBot`, подкласс `Barista`.

Инструкция:

- (a) Создайте класс `CafeStaff` с атрибутами `name` и `salary`.
- (b) Реализуйте метод `giveRaise`.
- (c) Добавьте метод `work` с общим сообщением.

- (d) Реализуйте метод `__repr__`.
- (e) Создайте класс `Barista`, наследующий `CafeStaff`, с начальной зарплатой.
- (f) Переопределите `work` в `Barista`, чтобы он выводил сообщение о приготовлении кофе.
- (g) Создайте класс `Host`, наследующий `CafeStaff`, с начальной зарплатой.
- (h) Переопределите `work` в `Host`, чтобы он выводил сообщение о встрече гостей.
- (i) Создайте класс `WashBot`, наследующий `Barista`, с начальной зарплатой.
- (j) Переопределите `work` в `WashBot`, чтобы он выводил сообщение о мытье посуды.
- (k) Создайте объект `washy` класса `WashBot` и вызовите его методы `work` и `giveRaise`.
- (l) Создайте по одному экземпляру каждого класса и вызовите у каждого метод `work`.

10 Разработайте модель команды автозаправки. Базовый класс `GasStaff` описывает общие черты. От него наследуются `Attendant` (оператор АЗС) и `ShopKeeper` (продавец в магазине при АЗС). Также работает робот-мойщик окон — `WindowBot`, подкласс `Attendant`.

Инструкция:

- (a) Создайте класс `GasStaff` с параметрами `name` и `salary`.
 - (b) Реализуйте метод `giveRaise`.
 - (c) Добавьте метод `work` с общим сообщением.
 - (d) Реализуйте метод `__repr__`.
 - (e) Создайте класс `Attendant`, наследующий `GasStaff`, с начальной зарплатой.
 - (f) Переопределите `work` в `Attendant`, чтобы он выводил сообщение о заправке автомобилей.
 - (g) Создайте класс `ShopKeeper`, наследующий `GasStaff`, с начальной зарплатой.
 - (h) Переопределите `work` в `ShopKeeper`, чтобы он выводил сообщение о продаже товаров.
 - (i) Создайте класс `WindowBot`, наследующий `Attendant`, с начальной зарплатой.
 - (j) Переопределите `work` в `WindowBot`, чтобы он выводил сообщение о мытье лобовых стёкол.
 - (k) Создайте объект `windowy` класса `WindowBot` и вызовите его методы `work` и `giveRaise`.
 - (l) Создайте по одному экземпляру каждого класса и вызовите у каждого метод `work`.
- 11 Создайте модель команды парикмахерской. Базовый класс `SalonStaff` описывает общие свойства. От него наследуются `Stylist` (стилист) и `Receptionist` (администратор). Также работает робот-массажист шеи — `NeckBot`, подкласс `Stylist`.

Инструкция:

- (a) Создайте класс `SalonStaff` с атрибутами `name` и `salary`.
- (b) Реализуйте метод `giveRaise`.
- (c) Добавьте метод `work` с общим сообщением.

- (d) Реализуйте метод `__repr__`.
- (e) Создайте класс `Stylist`, наследующий `SalonStaff`, с начальной зарплатой.
- (f) Переопределите `work` в `Stylist`, чтобы он выводил сообщение о стрижке клиентов.
- (g) Создайте класс `Receptionist`, наследующий `SalonStaff`, с начальной зарплатой.
- (h) Переопределите `work` в `Receptionist`, чтобы он выводил сообщение о записи клиентов.
- (i) Создайте класс `NeckBot`, наследующий `Stylist`, с начальной зарплатой.
- (j) Переопределите `work` в `NeckBot`, чтобы он выводил сообщение о массаже шеи.
- (k) Создайте объект `necko` класса `NeckBot` и вызовите его методы `work` и `giveRaise`.
- (l) Создайте по одному экземпляру каждого класса и вызовите у каждого метод `work`.

12 Разработайте модель команды фитнес-клуба. Базовый класс `GymStaff` описывает общие черты. От него наследуются `Trainer` (тренер) и `FrontDesk` (администратор стойки). Также работает робот-раздатчик полотенец — `TowelBot`, подкласс `FrontDesk`.

Инструкция:

- (a) Создайте класс `GymStaff` с параметрами `name` и `salary`.
- (b) Реализуйте метод `giveRaise`.
- (c) Добавьте метод `work` с общим сообщением.
- (d) Реализуйте метод `__repr__`.
- (e) Создайте класс `Trainer`, наследующий `GymStaff`, с начальной зарплатой.
- (f) Переопределите `work` в `Trainer`, чтобы он выводил сообщение о проведении тренировки.
- (g) Создайте класс `FrontDesk`, наследующий `GymStaff`, с начальной зарплатой.
- (h) Переопределите `work` в `FrontDesk`, чтобы он выводил сообщение о выдаче ключей и полотенец.
- (i) Создайте класс `TowelBot`, наследующий `FrontDesk`, с начальной зарплатой.
- (j) Переопределите `work` в `TowelBot`, чтобы он выводил сообщение о выдаче полотенца.
- (k) Создайте объект `towely` класса `TowelBot` и вызовите его методы `work` и `giveRaise`.
- (l) Создайте по одному экземпляру каждого класса и вызовите у каждого метод `work`.

13 Создайте модель команды музея. Базовый класс `MuseumStaff` описывает общие свойства. От него наследуются `Guide` (экскурсовод) и `TicketSeller` (кассир). Также работает робот-информатор — `InfoBot`, подкласс `Guide`.

Инструкция:

- (a) Создайте класс `MuseumStaff` с атрибутами `name` и `salary`.
- (b) Реализуйте метод `giveRaise`.
- (c) Добавьте метод `work` с общим сообщением.

- (d) Реализуйте метод `__repr__`.
 - (e) Создайте класс `Guide`, наследующий `MuseumStaff`, с начальной зарплатой.
 - (f) Переопределите `work` в `Guide`, чтобы он выводил сообщение о проведении экскурсии.
 - (g) Создайте класс `TicketSeller`, наследующий `MuseumStaff`, с начальной зарплатой.
 - (h) Переопределите `work` в `TicketSeller`, чтобы он выводил сообщение о продаже билетов.
 - (i) Создайте класс `InfoBot`, наследующий `Guide`, с начальной зарплатой.
 - (j) Переопределите `work` в `InfoBot`, чтобы он выводил сообщение о предоставлении информации.
 - (k) Создайте объект `infob` класса `InfoBot` и вызовите его методы `work` и `giveRaise`.
 - (l) Создайте по одному экземпляру каждого класса и вызовите у каждого метод `work`.
- 14 Разработайте модель команды отеля. Базовый класс `HotelStaff` описывает общие черты. От него наследуются `Housekeeper` (горничная) и `Concierge` (консьерж). Также работает робот-уборщик номеров — `RoomBot`, подкласс `Housekeeper`.

Инструкция:

- (a) Создайте класс `HotelStaff` с параметрами `name` и `salary`.
 - (b) Реализуйте метод `giveRaise`.
 - (c) Добавьте метод `work` с общим сообщением.
 - (d) Реализуйте метод `__repr__`.
 - (e) Создайте класс `Housekeeper`, наследующий `HotelStaff`, с начальной зарплатой.
 - (f) Переопределите `work` в `Housekeeper`, чтобы он выводил сообщение об уборке номеров.
 - (g) Создайте класс `Concierge`, наследующий `HotelStaff`, с начальной зарплатой.
 - (h) Переопределите `work` в `Concierge`, чтобы он выводил сообщение о помощи гостям.
 - (i) Создайте класс `RoomBot`, наследующий `Housekeeper`, с начальной зарплатой.
 - (j) Переопределите `work` в `RoomBot`, чтобы он выводил сообщение об автоматической уборке.
 - (k) Создайте объект `roomb` класса `RoomBot` и вызовите его методы `work` и `giveRaise`.
 - (l) Создайте по одному экземпляру каждого класса и вызовите у каждого метод `work`.
- 15 Создайте модель команды пекарни. Базовый класс `BakeryStaff` описывает общие свойства. От него наследуются `Baker` (пекарь) и `Salesperson` (продавец). Также работает робот-упаковщик булочек — `PackBot`, подкласс `Baker`.

Инструкция:

- (a) Создайте класс `BakeryStaff` с атрибутами `name` и `salary`.
- (b) Реализуйте метод `giveRaise`.

- (c) Добавьте метод `work` с общим сообщением.
- (d) Реализуйте метод `__repr__`.
- (e) Создайте класс `Baker`, наследующий `BakeryStaff`, с начальной зарплатой.
- (f) Переопределите `work` в `Baker`, чтобы он выводил сообщение о выпечке хлеба.
- (g) Создайте класс `Salesperson`, наследующий `BakeryStaff`, с начальной зарплатой.
- (h) Переопределите `work` в `Salesperson`, чтобы он выводил сообщение о продаже выпечки.
- (i) Создайте класс `PackBot`, наследующий `Baker`, с начальной зарплатой.
- (j) Переопределите `work` в `PackBot`, чтобы он выводил сообщение об упаковке изделий.
- (k) Создайте объект `packb` класса `PackBot` и вызовите его методы `work` и `giveRaise`.
- (l) Создайте по одному экземпляру каждого класса и вызовите у каждого метод `work`.

16 Разработайте модель команды автомойки. Базовый класс `WashStaff` описывает общие черты. От него наследуются `Operator` (оператор) и `Cashier` (кассир). Также работает робот-полировщик — `PolishBot`, подкласс `Operator`.

Инструкция:

- (a) Создайте класс `WashStaff` с параметрами `name` и `salary`.
- (b) Реализуйте метод `giveRaise`.
- (c) Добавьте метод `work` с общим сообщением.
- (d) Реализуйте метод `__repr__`.
- (e) Создайте класс `Operator`, наследующий `WashStaff`, с начальной зарплатой.
- (f) Переопределите `work` в `Operator`, чтобы он выводил сообщение об управлении мойкой.
- (g) Создайте класс `Cashier`, наследующий `WashStaff`, с начальной зарплатой.
- (h) Переопределите `work` в `Cashier`, чтобы он выводил сообщение о приёме оплаты.
- (i) Создайте класс `PolishBot`, наследующий `Operator`, с начальной зарплатой.
- (j) Переопределите `work` в `PolishBot`, чтобы он выводил сообщение о полировке кузова.
- (k) Создайте объект `polishb` класса `PolishBot` и вызовите его методы `work` и `giveRaise`.
- (l) Создайте по одному экземпляру каждого класса и вызовите у каждого метод `work`.

17 Создайте модель команды фермерского рынка. Базовый класс `MarketStaff` описывает общие свойства. От него наследуются `Farmer` (фермер) и `Cashier` (кассир). Также работает робот-сортировщик овощей — `SortVegBot`, подкласс `Farmer`.

Инструкция:

- (a) Создайте класс `MarketStaff` с атрибутами `name` и `salary`.
- (b) Реализуйте метод `giveRaise`.
- (c) Добавьте метод `work` с общим сообщением.

- (d) Реализуйте метод `__repr__`.
- (e) Создайте класс `Farmer`, наследующий `MarketStaff`, с начальной зарплатой.
- (f) Переопределите `work` в `Farmer`, чтобы он выводил сообщение о сборе урожая.
- (g) Создайте класс `Cashier`, наследующий `MarketStaff`, с начальной зарплатой.
- (h) Переопределите `work` в `Cashier`, чтобы он выводил сообщение о расчёте с покупателями.
- (i) Создайте класс `SortVegBot`, наследующий `Farmer`, с начальной зарплатой.
- (j) Переопределите `work` в `SortVegBot`, чтобы он выводил сообщение о сортировке овощей.
- (k) Создайте объект `sortveg` класса `SortVegBot` и вызовите его методы `work` и `giveRaise`.
- (l) Создайте по одному экземпляру каждого класса и вызовите у каждого метод `work`.

18 Разработайте модель команды цирка. Базовый класс `CircusStaff` описывает общие черты. От него наследуются `Performer` (артист) и `TicketSeller` (кассир). Также работает робот-дрессировщик — `TrainBot`, подкласс `Performer`.

Инструкция:

- (a) Создайте класс `CircusStaff` с параметрами `name` и `salary`.
- (b) Реализуйте метод `giveRaise`.
- (c) Добавьте метод `work` с общим сообщением.
- (d) Реализуйте метод `__repr__`.
- (e) Создайте класс `Performer`, наследующий `CircusStaff`, с начальной зарплатой.
- (f) Переопределите `work` в `Performer`, чтобы он выводил сообщение о выступлении.
- (g) Создайте класс `TicketSeller`, наследующий `CircusStaff`, с начальной зарплатой.
- (h) Переопределите `work` в `TicketSeller`, чтобы он выводил сообщение о продаже билетов.
- (i) Создайте класс `TrainBot`, наследующий `Performer`, с начальной зарплатой.
- (j) Переопределите `work` в `TrainBot`, чтобы он выводил сообщение о дрессировке животных.
- (k) Создайте объект `trainb` класса `TrainBot` и вызовите его методы `work` и `giveRaise`.
- (l) Создайте по одному экземпляру каждого класса и вызовите у каждого метод `work`.

19 Создайте модель команды театра. Базовый класс `TheaterStaff` описывает общие свойства. От него наследуются `Actor` (актёр) и `Stagehand` (рабочий сцены). Также работает робот-осветитель — `LightBot`, подкласс `Stagehand`.

Инструкция:

- (a) Создайте класс `TheaterStaff` с атрибутами `name` и `salary`.
- (b) Реализуйте метод `giveRaise`.
- (c) Добавьте метод `work` с общим сообщением.

- (d) Реализуйте метод `__repr__`.
- (e) Создайте класс `Actor`, наследующий `TheaterStaff`, с начальной зарплатой.
- (f) Переопределите `work` в `Actor`, чтобы он выводил сообщение об игре на сцене.
- (g) Создайте класс `Stagehand`, наследующий `TheaterStaff`, с начальной зарплатой.
- (h) Переопределите `work` в `Stagehand`, чтобы он выводил сообщение о подготовке реквизита.
- (i) Создайте класс `LightBot`, наследующий `Stagehand`, с начальной зарплатой.
- (j) Переопределите `work` в `LightBot`, чтобы он выводил сообщение об управлении освещением.
- (k) Создайте объект `lightb` класса `LightBot` и вызовите его методы `work` и `giveRaise`.
- (l) Создайте по одному экземпляру каждого класса и вызовите у каждого метод `work`.

20 Разработайте модель команды лаборатории. Базовый класс `LabStaff` описывает общие черты. От него наследуются `Scientist` (учёный) и `Admin` (администратор). Также работает робот-анализатор проб — `AnalyzeBot`, подкласс `Scientist`.

Инструкция:

- (a) Создайте класс `LabStaff` с параметрами `name` и `salary`.
- (b) Реализуйте метод `giveRaise`.
- (c) Добавьте метод `work` с общим сообщением.
- (d) Реализуйте метод `__repr__`.
- (e) Создайте класс `Scientist`, наследующий `LabStaff`, с начальной зарплатой.
- (f) Переопределите `work` в `Scientist`, чтобы он выводил сообщение о проведении экспериментов.
- (g) Создайте класс `Admin`, наследующий `LabStaff`, с начальной зарплатой.
- (h) Переопределите `work` в `Admin`, чтобы он выводил сообщение о ведении документации.
- (i) Создайте класс `AnalyzeBot`, наследующий `Scientist`, с начальной зарплатой.
- (j) Переопределите `work` в `AnalyzeBot`, чтобы он выводил сообщение об анализе проб.
- (k) Создайте объект `analyze` класса `AnalyzeBot` и вызовите его методы `work` и `giveRaise`.
- (l) Создайте по одному экземпляру каждого класса и вызовите у каждого метод `work`.

21 Создайте модель команды аэропорта. Базовый класс `AirportStaff` описывает общие свойства. От него наследуются `Pilot` (пилот) и `CheckInAgent` (агент регистрации). Также работает робот-грузчик багажа — `LoadBot`, подкласс `CheckInAgent`.

Инструкция:

- (a) Создайте класс `AirportStaff` с атрибутами `name` и `salary`.
- (b) Реализуйте метод `giveRaise`.
- (c) Добавьте метод `work` с общим сообщением.
- (d) Реализуйте метод `__repr__`.

- (e) Создайте класс `Pilot`, наследующий `AirportStaff`, с начальной зарплатой.
- (f) Переопределите `work` в `Pilot`, чтобы он выводил сообщение об управлении самолётом.
- (g) Создайте класс `CheckInAgent`, наследующий `AirportStaff`, с начальной зарплатой.
- (h) Переопределите `work` в `CheckInAgent`, чтобы он выводил сообщение о регистрации пассажиров.
- (i) Создайте класс `LoadBot`, наследующий `CheckInAgent`, с начальной зарплатой.
- (j) Переопределите `work` в `LoadBot`, чтобы он выводил сообщение о погрузке багажа.
- (k) Создайте объект `loadb` класса `LoadBot` и вызовите его методы `work` и `giveRaise`.
- (l) Создайте по одному экземпляру каждого класса и вызовите у каждого метод `work`.

22 Разработайте модель команды школы. Базовый класс `SchoolStaff` описывает общие черты. От него наследуются `Teacher` (учитель) и `Janitor` (дворник). Также работает робот-уборщик классов — `CleanClassBot`, подкласс `Janitor`.

Инструкция:

- (a) Создайте класс `SchoolStaff` с параметрами `name` и `salary`.
- (b) Реализуйте метод `giveRaise`.
- (c) Добавьте метод `work` с общим сообщением.
- (d) Реализуйте метод `__repr__`.
- (e) Создайте класс `Teacher`, наследующий `SchoolStaff`, с начальной зарплатой.
- (f) Переопределите `work` в `Teacher`, чтобы он выводил сообщение о проведении урока.
- (g) Создайте класс `Janitor`, наследующий `SchoolStaff`, с начальной зарплатой.
- (h) Переопределите `work` в `Janitor`, чтобы он выводил сообщение об уборке помещений.
- (i) Создайте класс `CleanClassBot`, наследующий `Janitor`, с начальной зарплатой.
- (j) Переопределите `work` в `CleanClassBot`, чтобы он выводил сообщение об автоматической уборке классов.
- (k) Создайте объект `cleanc` класса `CleanClassBot` и вызовите его методы `work` и `giveRaise`.
- (l) Создайте по одному экземпляру каждого класса и вызовите у каждого метод `work`.

23 Создайте модель команды больницы. Базовый класс `HospitalStaff` описывает общие свойства. От него наследуются `Doctor` (врач) и `Nurse` (медсестра). Также работает робот-разносчик лекарств — `MedBot`, подкласс `Nurse`.

Инструкция:

- (a) Создайте класс `HospitalStaff` с атрибутами `name` и `salary`.
- (b) Реализуйте метод `giveRaise`.
- (c) Добавьте метод `work` с общим сообщением.

- (d) Реализуйте метод `__repr__`.
- (e) Создайте класс `Doctor`, наследующий `HospitalStaff`, с начальной зарплатой.
- (f) Переопределите `work` в `Doctor`, чтобы он выводил сообщение о приёме пациентов.
- (g) Создайте класс `Nurse`, наследующий `HospitalStaff`, с начальной зарплатой.
- (h) Переопределите `work` в `Nurse`, чтобы он выводил сообщение об уходе за больными.
- (i) Создайте класс `MedBot`, наследующий `Nurse`, с начальной зарплатой.
- (j) Переопределите `work` в `MedBot`, чтобы он выводил сообщение о доставке лекарств.
- (k) Создайте объект `medb` класса `MedBot` и вызовите его методы `work` и `giveRaise`.
- (l) Создайте по одному экземпляру каждого класса и вызовите у каждого метод `work`.

24 Разработайте модель команды студии звукозаписи. Базовый класс `StudioStaff` описывает общие черты. От него наследуются `SoundEngineer` (звукорежиссёр) и `Receptionist` (администратор). Также работает робот-микшер — `MixBot`, подкласс `SoundEngineer`.

Инструкция:

- (a) Создайте класс `StudioStaff` с параметрами `name` и `salary`.
- (b) Реализуйте метод `giveRaise`.
- (c) Добавьте метод `work` с общим сообщением.
- (d) Реализуйте метод `__repr__`.
- (e) Создайте класс `SoundEngineer`, наследующий `StudioStaff`, с начальной зарплатой.
- (f) Переопределите `work` в `SoundEngineer`, чтобы он выводил сообщение о настройке звука.
- (g) Создайте класс `Receptionist`, наследующий `StudioStaff`, с начальной зарплатой.
- (h) Переопределите `work` в `Receptionist`, чтобы он выводил сообщение о приёме артистов.
- (i) Создайте класс `MixBot`, наследующий `SoundEngineer`, с начальной зарплатой.
- (j) Переопределите `work` в `MixBot`, чтобы он выводил сообщение о микшировании треков.
- (k) Создайте объект `mixb` класса `MixBot` и вызовите его методы `work` и `giveRaise`.
- (l) Создайте по одному экземпляру каждого класса и вызовите у каждого метод `work`.

25 Создайте модель команды кондитерской. Базовый класс `ConfectionStaff` описывает общие свойства. От него наследуются `PastryChef` (кондитер) и `Cashier` (кассир). Также работает робот-украшатель тортов — `DecorBot`, подкласс `PastryChef`.

Инструкция:

- (a) Создайте класс `ConfectionStaff` с атрибутами `name` и `salary`.
- (b) Реализуйте метод `giveRaise`.
- (c) Добавьте метод `work` с общим сообщением.

- (d) Реализуйте метод `__repr__`.
 - (e) Создайте класс `PastryChef`, наследующий `ConfectionStaff`, с начальной зарплатой.
 - (f) Переопределите `work` в `PastryChef`, чтобы он выводил сообщение о приготовлении тортов.
 - (g) Создайте класс `Cashier`, наследующий `ConfectionStaff`, с начальной зарплатой.
 - (h) Переопределите `work` в `Cashier`, чтобы он выводил сообщение о продаже изделий.
 - (i) Создайте класс `DecorBot`, наследующий `PastryChef`, с начальной зарплатой.
 - (j) Переопределите `work` в `DecorBot`, чтобы он выводил сообщение об украшении тортов.
 - (k) Создайте объект `decorb` класса `DecorBot` и вызовите его методы `work` и `giveRaise`.
 - (l) Создайте по одному экземпляру каждого класса и вызовите у каждого метод `work`.
- 26 Разработайте модель команды автобусного парка. Базовый класс `BusStaff` описывает общие черты. От него наследуются `Driver` (водитель) и `Dispatcher` (диспетчер). Также работает робот-контролёр билетов — `TicketBot`, подкласс `Dispatcher`.
- Инструкция:**
- (a) Создайте класс `BusStaff` с параметрами `name` и `salary`.
 - (b) Реализуйте метод `giveRaise`.
 - (c) Добавьте метод `work` с общим сообщением.
 - (d) Реализуйте метод `__repr__`.
 - (e) Создайте класс `Driver`, наследующий `BusStaff`, с начальной зарплатой.
 - (f) Переопределите `work` в `Driver`, чтобы он выводил сообщение о вождении автобуса.
 - (g) Создайте класс `Dispatcher`, наследующий `BusStaff`, с начальной зарплатой.
 - (h) Переопределите `work` в `Dispatcher`, чтобы он выводил сообщение о координации рейсов.
 - (i) Создайте класс `TicketBot`, наследующий `Dispatcher`, с начальной зарплатой.
 - (j) Переопределите `work` в `TicketBot`, чтобы он выводил сообщение о проверке билетов.
 - (k) Создайте объект `ticketb` класса `TicketBot` и вызовите его методы `work` и `giveRaise`.
 - (l) Создайте по одному экземпляру каждого класса и вызовите у каждого метод `work`.
- 27 Создайте модель команды спортивного магазина. Базовый класс `SportStaff` описывает общие свойства. От него наследуются `SalesRep` (продавец) и `RepairTech` (техник по ремонту). Также работает робот-упаковщик заказов — `PackOrderBot`, подкласс `SalesRep`.

Инструкция:

- (a) Создайте класс `SportStaff` с атрибутами `name` и `salary`.
- (b) Реализуйте метод `giveRaise`.

- (c) Добавьте метод `work` с общим сообщением.
- (d) Реализуйте метод `__repr__`.
- (e) Создайте класс `SalesRep`, наследующий `SportStaff`, с начальной зарплатой.
- (f) Переопределите `work` в `SalesRep`, чтобы он выводил сообщение о консультировании клиентов.
- (g) Создайте класс `RepairTech`, наследующий `SportStaff`, с начальной зарплатой.
- (h) Переопределите `work` в `RepairTech`, чтобы он выводил сообщение о ремонте инвентаря.
- (i) Создайте класс `PackOrderBot`, наследующий `SalesRep`, с начальной зарплатой.
- (j) Переопределите `work` в `PackOrderBot`, чтобы он выводил сообщение об упаковке онлайн-заказов.
- (k) Создайте объект `packord` класса `PackOrderBot` и вызовите его методы `work` и `giveRaise`.
- (l) Создайте по одному экземпляру каждого класса и вызовите у каждого метод `work`.

28 Разработайте модель команды кинопроката. Базовый класс `RentalStaff` описывает общие черты. От него наследуются `Clerk` (клерк) и `Cleaner` (уборщик). Также работает робот-сортировщик дисков — `DiscSortBot`, подкласс `Clerk`.

Инструкция:

- (a) Создайте класс `RentalStaff` с параметрами `name` и `salary`.
- (b) Реализуйте метод `giveRaise`.
- (c) Добавьте метод `work` с общим сообщением.
- (d) Реализуйте метод `__repr__`.
- (e) Создайте класс `Clerk`, наследующий `RentalStaff`, с начальной зарплатой.
- (f) Переопределите `work` в `Clerk`, чтобы он выводил сообщение о выдаче фильмов.
- (g) Создайте класс `Cleaner`, наследующий `RentalStaff`, с начальной зарплатой.
- (h) Переопределите `work` в `Cleaner`, чтобы он выводил сообщение об уборке помещения.
- (i) Создайте класс `DiscSortBot`, наследующий `Clerk`, с начальной зарплатой.
- (j) Переопределите `work` в `DiscSortBot`, чтобы он выводил сообщение о сортировке дисков.
- (k) Создайте объект `discsort` класса `DiscSortBot` и вызовите его методы `work` и `giveRaise`.
- (l) Создайте по одному экземпляру каждого класса и вызовите у каждого метод `work`.

29 Создайте модель команды прачечной. Базовый класс `LaundryStaff` описывает общие свойства. От него наследуются `Washer` (стиральщик) и `FoldingAgent` (упаковщик). Также работает робот-гладильщик — `IronBot`, подкласс `FoldingAgent`.

Инструкция:

- (a) Создайте класс `LaundryStaff` с атрибутами `name` и `salary`.

- (b) Реализуйте метод `giveRaise`.
- (c) Добавьте метод `work` с общим сообщением.
- (d) Реализуйте метод `__repr__`.
- (e) Создайте класс `Washer`, наследующий `LaundryStaff`, с начальной зарплатой.
- (f) Переопределите `work` в `Washer`, чтобы он выводил сообщение о стирке белья.
- (g) Создайте класс `FoldingAgent`, наследующий `LaundryStaff`, с начальной зарплатой.
- (h) Переопределите `work` в `FoldingAgent`, чтобы он выводил сообщение о складывании одежды.
- (i) Создайте класс `IronBot`, наследующий `FoldingAgent`, с начальной зарплатой.
- (j) Переопределите `work` в `IronBot`, чтобы он выводил сообщение о глажке рубашек.
- (k) Создайте объект `ironb` класса `IronBot` и вызовите его методы `work` и `giveRaise`.
- (l) Создайте по одному экземпляру каждого класса и вызовите у каждого метод `work`.

30 Разработайте модель команды фотостудии. Базовый класс `PhotoStaff` описывает общие черты. От него наследуются `Photographer` (фотограф) и `Retoucher` (ретушёр). Также работает робот-сортировщик фото — `SortPhotoBot`, подкласс `Retoucher`.

Инструкция:

- (a) Создайте класс `PhotoStaff` с параметрами `name` и `salary`.
- (b) Реализуйте метод `giveRaise`.
- (c) Добавьте метод `work` с общим сообщением.
- (d) Реализуйте метод `__repr__`.
- (e) Создайте класс `Photographer`, наследующий `PhotoStaff`, с начальной зарплатой.
- (f) Переопределите `work` в `Photographer`, чтобы он выводил сообщение о съёмке клиентов.
- (g) Создайте класс `Retoucher`, наследующий `PhotoStaff`, с начальной зарплатой.
- (h) Переопределите `work` в `Retoucher`, чтобы он выводил сообщение о ретуши снимков.
- (i) Создайте класс `SortPhotoBot`, наследующий `Retoucher`, с начальной зарплатой.
- (j) Переопределите `work` в `SortPhotoBot`, чтобы он выводил сообщение о сортировке фотографий.
- (k) Создайте объект `sortphoto` класса `SortPhotoBot` и вызовите его методы `work` и `giveRaise`.
- (l) Создайте по одному экземпляру каждого класса и вызовите у каждого метод `work`.

31 Создайте модель команды рыбного рынка. Базовый класс `FishMarketStaff` описывает общие свойства. От него наследуются `Fisherman` (рыбак) и `Seller` (продавец). Также работает робот-чистильщик рыбы — `CleanFishBot`, подкласс `Fisherman`.

Инструкция:

- (a) Создайте класс `FishMarketStaff` с атрибутами `name` и `salary`.
- (b) Реализуйте метод `giveRaise`.
- (c) Добавьте метод `work` с общим сообщением.
- (d) Реализуйте метод `__repr__`.
- (e) Создайте класс `Fisherman`, наследующий `FishMarketStaff`, с начальной зарплатой.
- (f) Переопределите `work` в `Fisherman`, чтобы он выводил сообщение о ловле рыбы.
- (g) Создайте класс `Seller`, наследующий `FishMarketStaff`, с начальной зарплатой.
- (h) Переопределите `work` в `Seller`, чтобы он выводил сообщение о продаже рыбы.
- (i) Создайте класс `CleanFishBot`, наследующий `Fisherman`, с начальной зарплатой.
- (j) Переопределите `work` в `CleanFishBot`, чтобы он выводил сообщение о чистке рыбы.
- (k) Создайте объект `cleanfish` класса `CleanFishBot` и вызовите его методы `work` и `giveRaise`.
- (l) Создайте по одному экземпляру каждого класса и вызовите у каждого метод `work`.

32 Разработайте модель команды цветочного магазина. Базовый класс `FlowerStaff` описывает общие черты. От него наследуются `Florist` (флорист) и `Cashier` (кассир). Также работает робот-поливальщик — `WaterBot`, подкласс `Florist`.

Инструкция:

- (a) Создайте класс `FlowerStaff` с параметрами `name` и `salary`.
- (b) Реализуйте метод `giveRaise`.
- (c) Добавьте метод `work` с общим сообщением.
- (d) Реализуйте метод `__repr__`.
- (e) Создайте класс `Florist`, наследующий `FlowerStaff`, с начальной зарплатой.
- (f) Переопределите `work` в `Florist`, чтобы он выводил сообщение о составлении букетов.
- (g) Создайте класс `Cashier`, наследующий `FlowerStaff`, с начальной зарплатой.
- (h) Переопределите `work` в `Cashier`, чтобы он выводил сообщение о расчёте с клиентами.
- (i) Создайте класс `WaterBot`, наследующий `Florist`, с начальной зарплатой.
- (j) Переопределите `work` в `WaterBot`, чтобы он выводил сообщение о поливе растений.
- (k) Создайте объект `waterb` класса `WaterBot` и вызовите его методы `work` и `giveRaise`.
- (l) Создайте по одному экземпляру каждого класса и вызовите у каждого метод `work`.

33 Создайте модель команды туристического агентства. Базовый класс `TravelStaff` описывает общие свойства. От него наследуются `Agent` (агент) и `Guide` (гид). Также работает робот-переводчик — `TransBot`, подкласс `Guide`.

Инструкция:

- (a) Создайте класс `TravelStaff` с атрибутами `name` и `salary`.
- (b) Реализуйте метод `giveRaise`.
- (c) Добавьте метод `work` с общим сообщением.
- (d) Реализуйте метод `__repr__`.
- (e) Создайте класс `Agent`, наследующий `TravelStaff`, с начальной зарплатой.
- (f) Переопределите `work` в `Agent`, чтобы он выводил сообщение о подборе туров.
- (g) Создайте класс `Guide`, наследующий `TravelStaff`, с начальной зарплатой.
- (h) Переопределите `work` в `Guide`, чтобы он выводил сообщение о сопровождении групп.
- (i) Создайте класс `TransBot`, наследующий `Guide`, с начальной зарплатой.
- (j) Переопределите `work` в `TransBot`, чтобы он выводил сообщение о переводе речи.
- (k) Создайте объект `transb` класса `TransBot` и вызовите его методы `work` и `giveRaise`.
- (l) Создайте по одному экземпляру каждого класса и вызовите у каждого метод `work`.

34 Разработайте модель команды компьютерного магазина. Базовый класс `CompStoreStaff` описывает общие черты. От него наследуются `Salesperson` (продавец) и `Technician` (техник). Также работает робот-тестировщик — `TestBot`, подкласс `Technician`.

Инструкция:

- (a) Создайте класс `CompStoreStaff` с параметрами `name` и `salary`.
- (b) Реализуйте метод `giveRaise`.
- (c) Добавьте метод `work` с общим сообщением.
- (d) Реализуйте метод `__repr__`.
- (e) Создайте класс `Salesperson`, наследующий `CompStoreStaff`, с начальной зарплатой.
- (f) Переопределите `work` в `Salesperson`, чтобы он выводил сообщение о консультировании покупателей.
- (g) Создайте класс `Technician`, наследующий `CompStoreStaff`, с начальной зарплатой.
- (h) Переопределите `work` в `Technician`, чтобы он выводил сообщение о ремонте техники.
- (i) Создайте класс `TestBot`, наследующий `Technician`, с начальной зарплатой.
- (j) Переопределите `work` в `TestBot`, чтобы он выводил сообщение о тестировании устройств.
- (k) Создайте объект `testb` класса `TestBot` и вызовите его методы `work` и `giveRaise`.
- (l) Создайте по одному экземпляру каждого класса и вызовите у каждого метод `work`.

35 Создайте модель команды книжного магазина. Базовый класс `BookStoreStaff` описывает общие свойства. От него наследуются `Bookseller` (продавец книг) и `StockClerk` (кладовщик). Также работает робот-рекомендатель — `RecBot`, подкласс `Bookseller`.

Инструкция:

- (a) Создайте класс `BookStoreStaff` с атрибутами `name` и `salary`.
- (b) Реализуйте метод `giveRaise`.
- (c) Добавьте метод `work` с общим сообщением.
- (d) Реализуйте метод `__repr__`.
- (e) Создайте класс `Bookseller`, наследующий `BookStoreStaff`, с начальной зарплатой.
- (f) Переопределите `work` в `Bookseller`, чтобы он выводил сообщение о рекомендации книг.
- (g) Создайте класс `StockClerk`, наследующий `BookStoreStaff`, с начальной зарплатой.
- (h) Переопределите `work` в `StockClerk`, чтобы он выводил сообщение о пополнении запасов.
- (i) Создайте класс `RecBot`, наследующий `Bookseller`, с начальной зарплатой.
- (j) Переопределите `work` в `RecBot`, чтобы он выводил сообщение о подборе книг по вкусу.
- (k) Создайте объект `recb` класса `RecBot` и вызовите его методы `work` и `giveRaise`.
- (l) Создайте по одному экземпляру каждого класса и вызовите у каждого метод `work`.

2.8 Семинар «Полиморфизм» (2 часа)

2.8.1 Полиморфизм с наследованием (частичная модификация метода родительского класса)

- 1
 - (a) Создайте класс `Person`, который будет базовым для класса `Manager`. В конструкторе класса `Person` задайте параметры `name`, `job` и `pay`.
 - (b) В классе `Person` создайте метод `last_name`, который будет возвращать фамилию человека (предполагается, что фамилия — это последнее слово в значении атрибута `name`).
 - (c) В классе `Person` создайте метод `give_rise`, который будет увеличивать значение атрибута `pay` на заданный процент (например, при вызове `give_rise(10)` зарплата должна увеличиться на 10 %).
 - (d) В классе `Person` создайте метод `__repr__`, который будет возвращать строковое представление объекта с информацией о человеке.
 - (e) Создайте класс `Manager`, наследующийся от класса `Person`. В конструкторе класса `Manager` задайте параметры `name`, `job` и `pay` (можно вызвать конструктор родительского класса).
 - (f) В классе `Manager` переопределите метод `give_rise` родительского класса с использованием функции `super()`, чтобы он увеличивал зарплату менеджера на заданный процент плюс дополнительный бонус (например, +10 % к указанному проценту).
 - (g) В основной части программы создайте объекты классов `Person` и `Manager` и вызовите их методы.
 - (h) Выведите информацию о каждом объекте (например, с помощью функции `print`, которая автоматически вызовет `__repr__`).
- 2
 - (a) Создайте класс `Employee`, который будет базовым для класса `TeamLead`. В конструкторе класса `Employee` задайте параметры `name`, `position` и `salary`.
 - (b) В классе `Employee` создайте метод `get_initials`, который будет возвращать инициалы сотрудника (первые буквы имени и фамилии, разделённые точкой).
 - (c) В классе `Employee` создайте метод `apply_bonus`, который будет увеличивать значение атрибута `salary` на заданный процент.
 - (d) В классе `Employee` создайте метод `__repr__`, который будет возвращать строковое представление объекта с информацией о сотруднике.
 - (e) Создайте класс `TeamLead`, наследующийся от класса `Employee`. В конструкторе класса `TeamLead` задайте параметры `name`, `position` и `salary`.
 - (f) В классе `TeamLead` переопределите метод `apply_bonus` с использованием `super()`, чтобы он увеличивал зарплату лидера на заданный процент плюс дополнительные 7 %.
 - (g) В основной части программы создайте объекты классов `Employee` и `TeamLead` и вызовите их методы.
 - (h) Выведите информацию о каждом объекте с помощью функции `print`.
- 3
 - (a) Создайте класс `Animal`, который будет базовым для класса `Dog`. В конструкторе класса `Animal` задайте параметры `name`, `species` и `age`.

- (b) В классе `Animal` создайте метод `get_species`, который будет возвращать вид животного.
 - (c) В классе `Animal` создайте метод `age_up`, который будет увеличивать возраст на заданное количество лет.
 - (d) В классе `Animal` создайте метод `__repr__`, который будет возвращать строковое представление объекта.
 - (e) Создайте класс `Dog`, наследующийся от класса `Animal`. В конструкторе класса `Dog` задайте параметры `name`, `species` и `age`.
 - (f) В классе `Dog` переопределите метод `age_up` с использованием `super()`, чтобы возраст увеличивался на указанное количество лет плюс дополнительный год (например, при вызове `age_up(2)` возраст увеличится на 3 года).
 - (g) В основной части программы создайте объекты классов `Animal` и `Dog` и вызовите их методы.
 - (h) Выведите информацию о каждом объекте с помощью функции `print`.
- 4
- (a) Создайте класс `Vehicle`, который будет базовым для класса `Truck`. В конструкторе класса `Vehicle` задайте параметры `brand`, `model` и `fuel_level`.
 - (b) В классе `Vehicle` создайте метод `refuel`, который будет увеличивать уровень топлива на заданное количество литров.
 - (c) В классе `Vehicle` создайте метод `get_model_year`, который будет возвращать год выпуска (предположим, что он закодирован в названии модели, например, последние 4 цифры).
 - (d) В классе `Vehicle` создайте метод `__repr__`, который будет возвращать строковое представление объекта.
 - (e) Создайте класс `Truck`, наследующийся от класса `Vehicle`. В конструкторе класса `Truck` задайте параметры `brand`, `model` и `fuel_level`.
 - (f) В классе `Truck` переопределите метод `refuel` с использованием `super()`, чтобы добавлялось указанное количество литров плюс бонус в 10 литров.
 - (g) В основной части программы создайте объекты классов `Vehicle` и `Truck` и вызовите их методы.
 - (h) Выведите информацию о каждом объекте с помощью функции `print`.
- 5
- (a) Создайте класс `Book`, который будет базовым для класса `Textbook`. В конструкторе класса `Book` задайте параметры `title`, `author` и `pages`.
 - (b) В классе `Book` создайте метод `get_author_last_name`, который будет возвращать фамилию автора (последнее слово в строке `author`).
 - (c) В классе `Book` создайте метод `add_pages`, который будет увеличивать количество страниц на заданное число.
 - (d) В классе `Book` создайте метод `__repr__`, который будет возвращать строковое представление объекта.
 - (e) Создайте класс `Textbook`, наследующийся от класса `Book`. В конструкторе класса `Textbook` задайте параметры `title`, `author` и `pages`.
 - (f) В классе `Textbook` переопределите метод `add_pages` с использованием `super()`, чтобы добавлялось указанное число страниц плюс дополнительные 20 страниц (например, для приложений).

- (g) В основной части программы создайте объекты классов `Book` и `Textbook` и вызовите их методы.
 - (h) Выведите информацию о каждом объекте с помощью функции `print`.
- 6
- (a) Создайте класс `Student`, который будет базовым для класса `GraduateStudent`. В конструкторе класса `Student` задайте параметры `name`, `major` и `gpa`.
 - (b) В классе `Student` создайте метод `get_initials`, который будет возвращать инициалы студента.
 - (c) В классе `Student` создайте метод `improve_gpa`, который будет увеличивать средний балл на заданное значение (например, `improve_gpa(0.2)`).
 - (d) В классе `Student` создайте метод `__repr__`, который будет возвращать строковое представление объекта.
 - (e) Создайте класс `GraduateStudent`, наследующийся от класса `Student`. В конструкторе класса `GraduateStudent` задайте параметры `name`, `major` и `gpa`.
 - (f) В классе `GraduateStudent` переопределите метод `improve_gpa` с использованием `super()`, чтобы повышение составляло указанное значение плюс дополнительные 0.1 балла.
 - (g) В основной части программы создайте объекты классов `Student` и `GraduateStudent` и вызовите их методы.
 - (h) Выведите информацию о каждом объекте с помощью функции `print`.
- 7
- (a) Создайте класс `Product`, который будет базовым для класса `PremiumProduct`. В конструкторе класса `Product` задайте параметры `name`, `category` и `price`.
 - (b) В классе `Product` создайте метод `apply_discount`, который будет уменьшать цену на заданный процент.
 - (c) В классе `Product` создайте метод `get_category_initial`, который будет возвращать первую букву категории.
 - (d) В классе `Product` создайте метод `__repr__`, который будет возвращать строковое представление объекта.
 - (e) Создайте класс `PremiumProduct`, наследующийся от класса `Product`. В конструкторе класса `PremiumProduct` задайте параметры `name`, `category` и `price`.
 - (f) В классе `PremiumProduct` переопределите метод `apply_discount` с использованием `super()`, чтобы скидка уменьшалась на 5 % от указанного значения (например, при запросе скидки 20 % применяется только 15 %).
 - (g) В основной части программы создайте объекты классов `Product` и `PremiumProduct` и вызовите их методы.
 - (h) Выведите информацию о каждом объекте с помощью функции `print`.
- 8
- (a) Создайте класс `Account`, который будет базовым для класса `SavingsAccount`. В конструкторе класса `Account` задайте параметры `owner`, `account_type` и `balance`.
 - (b) В классе `Account` создайте метод `deposit`, который будет увеличивать баланс на заданную сумму.
 - (c) В классе `Account` создайте метод `get_owner_last_name`, который будет возвращать фамилию владельца.

- (d) В классе **Account** создайте метод `__repr__`, который будет возвращать строковое представление объекта.
 - (e) Создайте класс **SavingsAccount**, наследующийся от класса **Account**. В конструкторе класса **SavingsAccount** задайте параметры `owner`, `account_type` и `balance`.
 - (f) В классе **SavingsAccount** переопределите метод `deposit` с использованием `super()`, чтобы при пополнении добавлялась указанная сумма плюс бонус в 1 % от неё.
 - (g) В основной части программы создайте объекты классов **Account** и **SavingsAccount** и вызовите их методы.
 - (h) Выведите информацию о каждом объекте с помощью функции `print`.
- 9
- (a) Создайте класс **GameCharacter**, который будет базовым для класса **Hero**. В конструкторе класса **GameCharacter** задайте параметры `name`, `class_type` и `health`.
 - (b) В классе **GameCharacter** создайте метод `heal`, который будет увеличивать здоровье на заданное количество единиц.
 - (c) В классе **GameCharacter** создайте метод `get_class_initial`, который будет возвращать первую букву класса персонажа.
 - (d) В классе **GameCharacter** создайте метод `__repr__`, который будет возвращать строковое представление объекта.
 - (e) Создайте класс **Hero**, наследующийся от класса **GameCharacter**. В конструкторе класса **Hero** задайте параметры `name`, `class_type` и `health`.
 - (f) В классе **Hero** переопределите метод `heal` с использованием `super()`, чтобы восстанавливалось указанное количество здоровья плюс дополнительные 10 единиц.
 - (g) В основной части программы создайте объекты классов **GameCharacter** и **Hero** и вызовите их методы.
 - (h) Выведите информацию о каждом объекте с помощью функции `print`.
- 10
- (a) Создайте класс **Device**, который будет базовым для класса **Smartphone**. В конструкторе класса **Device** задайте параметры `brand`, `model` и `battery_level`.
 - (b) В классе **Device** создайте метод `charge`, который будет увеличивать уровень заряда на заданное количество процентов.
 - (c) В классе **Device** создайте метод `get_brand_initial`, который будет возвращать первую букву бренда.
 - (d) В классе **Device** создайте метод `__repr__`, который будет возвращать строковое представление объекта.
 - (e) Создайте класс **Smartphone**, наследующийся от класса **Device**. В конструкторе класса **Smartphone** задайте параметры `brand`, `model` и `battery_level`.
 - (f) В классе **Smartphone** переопределите метод `charge` с использованием `super()`, чтобы заряд увеличивался на указанное значение плюс дополнительные 5 %.
 - (g) В основной части программы создайте объекты классов **Device** и **Smartphone** и вызовите их методы.
 - (h) Выведите информацию о каждом объекте с помощью функции `print`.
- 11
- (a) Создайте класс **Plant**, который будет базовым для класса **FloweringPlant**. В конструкторе класса **Plant** задайте параметры `name`, `species` и `height`.

- (b) В классе `Plant` создайте метод `grow`, который будет увеличивать высоту растения на заданное количество сантиметров.
 - (c) В классе `Plant` создайте метод `get_species_initial`, который будет возвращать первую букву вида.
 - (d) В классе `Plant` создайте метод `__repr__`, который будет возвращать строковое представление объекта.
 - (e) Создайте класс `FloweringPlant`, наследующийся от класса `Plant`. В конструкторе класса `FloweringPlant` задайте параметры `name`, `species` и `height`.
 - (f) В классе `FloweringPlant` переопределите метод `grow` с использованием `super()`, чтобы высота увеличивалась на указанное значение плюс дополнительные 2 см (для цветков).
 - (g) В основной части программы создайте объекты классов `Plant` и `FloweringPlant` и вызовите их методы.
 - (h) Выведите информацию о каждом объекте с помощью функции `print`.
- 12
- (a) Создайте класс `Musician`, который будет базовым для класса `Soloist`. В конструкторе класса `Musician` задайте параметры `name`, `instrument` и `experience`.
 - (b) В классе `Musician` создайте метод `practice`, который будет увеличивать опыт на заданное количество лет.
 - (c) В классе `Musician` создайте метод `get_instrument_initial`, который будет возвращать первую букву инструмента.
 - (d) В классе `Musician` создайте метод `__repr__`, который будет возвращать строковое представление объекта.
 - (e) Создайте класс `Soloist`, наследующийся от класса `Musician`. В конструкторе класса `Soloist` задайте параметры `name`, `instrument` и `experience`.
 - (f) В классе `Soloist` переопределите метод `practice` с использованием `super()`, чтобы опыт увеличивался на указанное значение плюс дополнительный год.
 - (g) В основной части программы создайте объекты классов `Musician` и `Soloist` и вызовите их методы.
 - (h) Выведите информацию о каждом объекте с помощью функции `print`.
- 13
- (a) Создайте класс `House`, который будет базовым для класса `Mansion`. В конструкторе класса `House` задайте параметры `address`, `rooms` и `area`.
 - (b) В классе `House` создайте метод `expand`, который будет увеличивать площадь на заданное количество квадратных метров.
 - (c) В классе `House` создайте метод `get_city_initial`, который будет возвращать первую букву города из адреса.
 - (d) В классе `House` создайте метод `__repr__`, который будет возвращать строковое представление объекта.
 - (e) Создайте класс `Mansion`, наследующийся от класса `House`. В конструкторе класса `Mansion` задайте параметры `address`, `rooms` и `area`.
 - (f) В классе `Mansion` переопределите метод `expand` с использованием `super()`, чтобы площадь увеличивалась на указанное значение плюс дополнительные 50 м².

- (g) В основной части программы создайте объекты классов `House` и `Mansion` и вызовите их методы.
 - (h) Выведите информацию о каждом объекте с помощью функции `print`.
- 14
- (a) Создайте класс `Writer`, который будет базовым для класса `Novelist`. В конструкторе класса `Writer` задайте параметры `name`, `genre` и `books_written`.
 - (b) В классе `Writer` создайте метод `publish_book`, который будет увеличивать количество написанных книг на 1.
 - (c) В классе `Writer` создайте метод `get_genre_initial`, который будет возвращать первую букву жанра.
 - (d) В классе `Writer` создайте метод `__repr__`, который будет возвращать строковое представление объекта.
 - (e) Создайте класс `Novelist`, наследующийся от класса `Writer`. В конструкторе класса `Novelist` задайте параметры `name`, `genre` и `books_written`.
 - (f) В классе `Novelist` переопределите метод `publish_book` с использованием `super()`, чтобы при публикации увеличивалось количество книг на 1 плюс бонус в 0.5 (для совместных работ).
 - (g) В основной части программы создайте объекты классов `Writer` и `Novelist` и вызовите их методы.
 - (h) Выведите информацию о каждом объекте с помощью функции `print`.
- 15
- (a) Создайте класс `BankClient`, который будет базовым для класса `VIPClient`. В конструкторе класса `BankClient` задайте параметры `name`, `client_type` и `balance`.
 - (b) В классе `BankClient` создайте метод `withdraw`, который будет уменьшать баланс на заданную сумму.
 - (c) В классе `BankClient` создайте метод `get_name_initial`, который будет возвращать первую букву имени.
 - (d) В классе `BankClient` создайте метод `__repr__`, который будет возвращать строковое представление объекта.
 - (e) Создайте класс `VIPClient`, наследующийся от класса `BankClient`. В конструкторе класса `VIPClient` задайте параметры `name`, `client_type` и `balance`.
 - (f) В классе `VIPClient` переопределите метод `withdraw` с использованием `super()`, чтобы при снятии уменьшалась сумма на 5 % меньше указанной (комиссия ниже).
 - (g) В основной части программы создайте объекты классов `BankClient` и `VIPClient` и вызовите их методы.
 - (h) Выведите информацию о каждом объекте с помощью функции `print`.
- 16
- (a) Создайте класс `Athlete`, который будет базовым для класса `Olympian`. В конструкторе класса `Athlete` задайте параметры `name`, `sport` и `medals`.
 - (b) В классе `Athlete` создайте метод `win_medal`, который будет увеличивать количество медалей на 1.
 - (c) В классе `Athlete` создайте метод `get_sport_initial`, который будет возвращать первую букву вида спорта.
 - (d) В классе `Athlete` создайте метод `__repr__`, который будет возвращать строковое представление объекта.

- (e) Создайте класс `Olympian`, наследующийся от класса `Athlete`. В конструкторе класса `Olympian` задайте параметры `name`, `sport` и `medals`.
 - (f) В классе `Olympian` переопределите метод `win_medal` с использованием `super()`, чтобы при победе увеличивалось количество медалей на 1 плюс бонус в 0.2 (для командных медалей).
 - (g) В основной части программы создайте объекты классов `Athlete` и `Olympian` и вызовите их методы.
 - (h) Выведите информацию о каждом объекте с помощью функции `print`.
- 17
- (a) Создайте класс `Computer`, который будет базовым для класса `GamingPC`. В конструкторе класса `Computer` задайте параметры `brand`, `model` и `ram_gb`.
 - (b) В классе `Computer` создайте метод `upgrade_ram`, который будет увеличивать объём ОЗУ на заданное количество гигабайт.
 - (c) В классе `Computer` создайте метод `get_brand_initial`, который будет возвращать первую букву бренда.
 - (d) В классе `Computer` создайте метод `__repr__`, который будет возвращать строковое представление объекта.
 - (e) Создайте класс `GamingPC`, наследующийся от класса `Computer`. В конструкторе класса `GamingPC` задайте параметры `brand`, `model` и `ram_gb`.
 - (f) В классе `GamingPC` переопределите метод `upgrade_ram` с использованием `super()`, чтобы объём ОЗУ увеличивался на указанное значение плюс дополнительные 2 ГБ.
 - (g) В основной части программы создайте объекты классов `Computer` и `GamingPC` и вызовите их методы.
 - (h) Выведите информацию о каждом объекте с помощью функции `print`.
- 18
- (a) Создайте класс `Teacher`, который будет базовым для класса `Professor`. В конструкторе класса `Teacher` задайте параметры `name`, `subject` и `experience`.
 - (b) В классе `Teacher` создайте метод `teach_more`, который будет увеличивать опыт на заданное количество лет.
 - (c) В классе `Teacher` создайте метод `get_subject_initial`, который будет возвращать первую букву предмета.
 - (d) В классе `Teacher` создайте метод `__repr__`, который будет возвращать строковое представление объекта.
 - (e) Создайте класс `Professor`, наследующийся от класса `Teacher`. В конструкторе класса `Professor` задайте параметры `name`, `subject` и `experience`.
 - (f) В классе `Professor` переопределите метод `teach_more` с использованием `super()`, чтобы опыт увеличивался на указанное значение плюс дополнительный год.
 - (g) В основной части программы создайте объекты классов `Teacher` и `Professor` и вызовите их методы.
 - (h) Выведите информацию о каждом объекте с помощью функции `print`.
- 19
- (a) Создайте класс `Robot`, который будет базовым для класса `ServiceRobot`. В конструкторе класса `Robot` задайте параметры `model`, `type` и `battery_life`.

- (b) В классе `Robot` создайте метод `recharge`, который будет увеличивать время работы от батареи на заданное количество часов.
 - (c) В классе `Robot` создайте метод `get_type_initial`, который будет возвращать первую букву типа.
 - (d) В классе `Robot` создайте метод `__repr__`, который будет возвращать строковое представление объекта.
 - (e) Создайте класс `ServiceRobot`, наследующийся от класса `Robot`. В конструкторе класса `ServiceRobot` задайте параметры `model`, `type` и `battery_life`.
 - (f) В классе `ServiceRobot` переопределите метод `recharge` с использованием `super()`, чтобы время работы увеличивалось на указанное значение плюс дополнительные 0.5 часа.
 - (g) В основной части программы создайте объекты классов `Robot` и `ServiceRobot` и вызовите их методы.
 - (h) Выведите информацию о каждом объекте с помощью функции `print`.
- 20
- (a) Создайте класс `Painter`, который будет базовым для класса `MasterPainter`. В конструкторе класса `Painter` задайте параметры `name`, `style` и `paintings`.
 - (b) В классе `Painter` создайте метод `create_painting`, который будет увеличивать количество картин на 1.
 - (c) В классе `Painter` создайте метод `get_style_initial`, который будет возвращать первую букву стиля.
 - (d) В классе `Painter` создайте метод `__repr__`, который будет возвращать строковое представление объекта.
 - (e) Создайте класс `MasterPainter`, наследующийся от класса `Painter`. В конструкторе класса `MasterPainter` задайте параметры `name`, `style` и `paintings`.
 - (f) В классе `MasterPainter` переопределите метод `create_painting` с использованием `super()`, чтобы количество картин увеличивалось на 1 плюс бонус в 0.3 (для серии работ).
 - (g) В основной части программы создайте объекты классов `Painter` и `MasterPainter` и вызовите их методы.
 - (h) Выведите информацию о каждом объекте с помощью функции `print`.
- 21
- (a) Создайте класс `Chef`, который будет базовым для класса `HeadChef`. В конструкторе класса `Chef` задайте параметры `name`, `cuisine` и `dishes_created`.
 - (b) В классе `Chef` создайте метод `invent_dish`, который будет увеличивать количество созданных блюд на 1.
 - (c) В классе `Chef` создайте метод `get_cuisine_initial`, который будет возвращать первую букву кухни.
 - (d) В классе `Chef` создайте метод `__repr__`, который будет возвращать строковое представление объекта.
 - (e) Создайте класс `HeadChef`, наследующийся от класса `Chef`. В конструкторе класса `HeadChef` задайте параметры `name`, `cuisine` и `dishes_created`.
 - (f) В классе `HeadChef` переопределите метод `invent_dish` с использованием `super()`, чтобы количество блюд увеличивалось на 1 плюс бонус в 0.4 (для командных разработок).

- (g) В основной части программы создайте объекты классов `Chef` и `HeadChef` и вызовите их методы.
 - (h) Выведите информацию о каждом объекте с помощью функции `print`.
- 22
- (a) Создайте класс `Photographer`, который будет базовым для класса `ProPhotographer`. В конструкторе класса `Photographer` задайте параметры `name`, `specialty` и `photos_taken`.
 - (b) В классе `Photographer` создайте метод `take_photo`, который будет увеличивать количество фотографий на 1.
 - (c) В классе `Photographer` создайте метод `get_specialty_initial`, который будет возвращать первую букву специализации.
 - (d) В классе `Photographer` создайте метод `__repr__`, который будет возвращать строковое представление объекта.
 - (e) Создайте класс `ProPhotographer`, наследующийся от класса `Photographer`. В конструкторе класса `ProPhotographer` задайте параметры `name`, `specialty` и `photos_taken`.
 - (f) В классе `ProPhotographer` переопределите метод `take_photo` с использованием `super()`, чтобы количество фотографий увеличивалось на 1 плюс бонус в 0.25 (для серийных съёмок).
 - (g) В основной части программы создайте объекты классов `Photographer` и `ProPhotographer` и вызовите их методы.
 - (h) Выведите информацию о каждом объекте с помощью функции `print`.
- 23
- (a) Создайте класс `Scientist`, который будет базовым для класса `LeadScientist`. В конструкторе класса `Scientist` задайте параметры `name`, `field` и `papers_published`.
 - (b) В классе `Scientist` создайте метод `publish_paper`, который будет увеличивать количество публикаций на 1.
 - (c) В классе `Scientist` создайте метод `get_field_initial`, который будет возвращать первую букву области науки.
 - (d) В классе `Scientist` создайте метод `__repr__`, который будет возвращать строковое представление объекта.
 - (e) Создайте класс `LeadScientist`, наследующийся от класса `Scientist`. В конструкторе класса `LeadScientist` задайте параметры `name`, `field` и `papers_published`.
 - (f) В классе `LeadScientist` переопределите метод `publish_paper` с использованием `super()`, чтобы количество публикаций увеличивалось на 1 плюс бонус в 0.6 (для руководства проектами).
 - (g) В основной части программы создайте объекты классов `Scientist` и `LeadScientist` и вызовите их методы.
 - (h) Выведите информацию о каждом объекте с помощью функции `print`.
- 24
- (a) Создайте класс `Car`, который будет базовым для класса `ElectricCar`. В конструкторе класса `Car` задайте параметры `make`, `model` и `mileage`.
 - (b) В классе `Car` создайте метод `drive`, который будет увеличивать пробег на заданное количество километров.
 - (c) В классе `Car` создайте метод `get_make_initial`, который будет возвращать первую букву марки.

- (d) В классе `Car` создайте метод `__repr__`, который будет возвращать строковое представление объекта.
 - (e) Создайте класс `ElectricCar`, наследующийся от класса `Car`. В конструкторе класса `ElectricCar` задайте параметры `make`, `model` и `mileage`.
 - (f) В классе `ElectricCar` переопределите метод `drive` с использованием `super()`, чтобы пробег увеличивался на указанное значение плюс дополнительные 5 км (для учёта регенеративного торможения).
 - (g) В основной части программы создайте объекты классов `Car` и `ElectricCar` и вызовите их методы.
 - (h) Выведите информацию о каждом объекте с помощью функции `print`.
- 25
- (a) Создайте класс `Artist`, который будет базовым для класса `DigitalArtist`. В конструкторе класса `Artist` задайте параметры `name`, `medium` и `artworks`.
 - (b) В классе `Artist` создайте метод `create_artwork`, который будет увеличивать количество работ на 1.
 - (c) В классе `Artist` создайте метод `get_medium_initial`, который будет возвращать первую букву носителя.
 - (d) В классе `Artist` создайте метод `__repr__`, который будет возвращать строковое представление объекта.
 - (e) Создайте класс `DigitalArtist`, наследующийся от класса `Artist`. В конструкторе класса `DigitalArtist` задайте параметры `name`, `medium` и `artworks`.
 - (f) В классе `DigitalArtist` переопределите метод `create_artwork` с использованием `super()`, чтобы количество работ увеличивалось на 1 плюс бонус в 0.35 (для цифровых серий).
 - (g) В основной части программы создайте объекты классов `Artist` и `DigitalArtist` и вызовите их методы.
 - (h) Выведите информацию о каждом объекте с помощью функции `print`.
- 26
- (a) Создайте класс `Farmer`, который будет базовым для класса `OrganicFarmer`. В конструкторе класса `Farmer` задайте параметры `name`, `crop` и `harvest_tons`.
 - (b) В классе `Farmer` создайте метод `harvest`, который будет увеличивать урожай на заданное количество тонн.
 - (c) В классе `Farmer` создайте метод `get_crop_initial`, который будет возвращать первую букву культуры.
 - (d) В классе `Farmer` создайте метод `__repr__`, который будет возвращать строковое представление объекта.
 - (e) Создайте класс `OrganicFarmer`, наследующийся от класса `Farmer`. В конструкторе класса `OrganicFarmer` задайте параметры `name`, `crop` и `harvest_tons`.
 - (f) В классе `OrganicFarmer` переопределите метод `harvest` с использованием `super()`, чтобы урожай увеличивался на указанное значение плюс дополнительные 0.8 тонны.
 - (g) В основной части программы создайте объекты классов `Farmer` и `OrganicFarmer` и вызовите их методы.
 - (h) Выведите информацию о каждом объекте с помощью функции `print`.

- 27 (a) Создайте класс `Singer`, который будет базовым для класса `LeadSinger`. В конструкторе класса `Singer` задайте параметры `name`, `genre` и `albums`.
- (b) В классе `Singer` создайте метод `release_album`, который будет увеличивать количество альбомов на 1.
- (c) В классе `Singer` создайте метод `get_genre_initial`, который будет возвращать первую букву жанра.
- (d) В классе `Singer` создайте метод `__repr__`, который будет возвращать строковое представление объекта.
- (e) Создайте класс `LeadSinger`, наследующийся от класса `Singer`. В конструкторе класса `LeadSinger` задайте параметры `name`, `genre` и `albums`.
- (f) В классе `LeadSinger` переопределите метод `release_album` с использованием `super()`, чтобы количество альбомов увеличивалось на 1 плюс бонус в 0.2 (для совместных релизов).
- (g) В основной части программы создайте объекты классов `Singer` и `LeadSinger` и вызовите их методы.
- (h) Выведите информацию о каждом объекте с помощью функции `print`.
- 28 (a) Создайте класс `Builder`, который будет базовым для класса `MasterBuilder`. В конструкторе класса `Builder` задайте параметры `name`, `specialty` и `buildings`.
- (b) В классе `Builder` создайте метод `build`, который будет увеличивать количество построенных зданий на 1.
- (c) В классе `Builder` создайте метод `get_specialty_initial`, который будет возвращать первую букву специализации.
- (d) В классе `Builder` создайте метод `__repr__`, который будет возвращать строковое представление объекта.
- (e) Создайте класс `MasterBuilder`, наследующийся от класса `Builder`. В конструкторе класса `MasterBuilder` задайте параметры `name`, `specialty` и `buildings`.
- (f) В классе `MasterBuilder` переопределите метод `build` с использованием `super()`, чтобы количество зданий увеличивалось на 1 плюс бонус в 0.5 (для комплексных проектов).
- (g) В основной части программы создайте объекты классов `Builder` и `MasterBuilder` и вызовите их методы.
- (h) Выведите информацию о каждом объекте с помощью функции `print`.
- 29 (a) Создайте класс `Pilot`, который будет базовым для класса `Captain`. В конструкторе класса `Pilot` задайте параметры `name`, `airline` и `flight_hours`.
- (b) В классе `Pilot` создайте метод `fly`, который будет увеличивать налёт на заданное количество часов.
- (c) В классе `Pilot` создайте метод `get_airline_initial`, который будет возвращать первую букву авиакомпании.
- (d) В классе `Pilot` создайте метод `__repr__`, который будет возвращать строковое представление объекта.
- (e) Создайте класс `Captain`, наследующийся от класса `Pilot`. В конструкторе класса `Captain` задайте параметры `name`, `airline` и `flight_hours`.

- (f) В классе `Captain` переопределите метод `fly` с использованием `super()`, чтобы налёт увеличивался на указанное значение плюс дополнительные 0.75 часа (для подготовки экипажа).
 - (g) В основной части программы создайте объекты классов `Pilot` и `Captain` и вызовите их методы.
 - (h) Выведите информацию о каждом объекте с помощью функции `print`.
- 30
- (a) Создайте класс `Designer`, который будет базовым для класса `CreativeDirector`. В конструкторе класса `Designer` задайте параметры `name`, `field` и `projects`.
 - (b) В классе `Designer` создайте метод `complete_project`, который будет увеличивать количество проектов на 1.
 - (c) В классе `Designer` создайте метод `get_field_initial`, который будет возвращать первую букву области дизайна.
 - (d) В классе `Designer` создайте метод `__repr__`, который будет возвращать строковое представление объекта.
 - (e) Создайте класс `CreativeDirector`, наследующийся от класса `Designer`. В конструкторе класса `CreativeDirector` задайте параметры `name`, `field` и `projects`.
 - (f) В классе `CreativeDirector` переопределите метод `complete_project` с использованием `super()`, чтобы количество проектов увеличивалось на 1 плюс бонус в 0.4 (для курируемых проектов).
 - (g) В основной части программы создайте объекты классов `Designer` и `CreativeDirector` и вызовите их методы.
 - (h) Выведите информацию о каждом объекте с помощью функции `print`.
- 31
- (a) Создайте класс `Engineer`, который будет базовым для класса `ChiefEngineer`. В конструкторе класса `Engineer` задайте параметры `name`, `discipline` и `projects_led`.
 - (b) В классе `Engineer` создайте метод `lead_project`, который будет увеличивать количество возглавленных проектов на 1.
 - (c) В классе `Engineer` создайте метод `get_discipline_initial`, который будет возвращать первую букву инженерной дисциплины.
 - (d) В классе `Engineer` создайте метод `__repr__`, который будет возвращать строковое представление объекта.
 - (e) Создайте класс `ChiefEngineer`, наследующийся от класса `Engineer`. В конструкторе класса `ChiefEngineer` задайте параметры `name`, `discipline` и `projects_led`.
 - (f) В классе `ChiefEngineer` переопределите метод `lead_project` с использованием `super()`, чтобы количество проектов увеличивалось на 1 плюс бонус в 0.6 (для стратегических инициатив).
 - (g) В основной части программы создайте объекты классов `Engineer` и `ChiefEngineer` и вызовите их методы.
 - (h) Выведите информацию о каждом объекте с помощью функции `print`.
- 32
- (a) Создайте класс `Actor`, который будет базовым для класса `LeadActor`. В конструкторе класса `Actor` задайте параметры `name`, `genre` и `roles`.
 - (b) В классе `Actor` создайте метод `take_role`, который будет увеличивать количество ролей на 1.

- (c) В классе `Actor` создайте метод `get_genre_initial`, который будет возвращать первую букву жанра.
 - (d) В классе `Actor` создайте метод `__repr__`, который будет возвращать строковое представление объекта.
 - (e) Создайте класс `LeadActor`, наследующийся от класса `Actor`. В конструкторе класса `LeadActor` задайте параметры `name`, `genre` и `roles`.
 - (f) В классе `LeadActor` переопределите метод `take_role` с использованием `super()`, чтобы количество ролей увеличивалось на 1 плюс бонус в 0.3 (для главных ролей в ансамблях).
 - (g) В основной части программы создайте объекты классов `Actor` и `LeadActor` и вызовите их методы.
 - (h) Выведите информацию о каждом объекте с помощью функции `print`.
- 33
- (a) Создайте класс `Dancer`, который будет базовым для класса `PrincipalDancer`. В конструкторе класса `Dancer` задайте параметры `name`, `style` и `performances`.
 - (b) В классе `Dancer` создайте метод `perform`, который будет увеличивать количество выступлений на 1.
 - (c) В классе `Dancer` создайте метод `get_style_initial`, который будет возвращать первую букву стиля танца.
 - (d) В классе `Dancer` создайте метод `__repr__`, который будет возвращать строковое представление объекта.
 - (e) Создайте класс `PrincipalDancer`, наследующийся от класса `Dancer`. В конструкторе класса `PrincipalDancer` задайте параметры `name`, `style` и `performances`.
 - (f) В классе `PrincipalDancer` переопределите метод `perform` с использованием `super()`, чтобы количество выступлений увеличивалось на 1 плюс бонус в 0.4 (для сольных номеров).
 - (g) В основной части программы создайте объекты классов `Dancer` и `PrincipalDancer` и вызовите их методы.
 - (h) Выведите информацию о каждом объекте с помощью функции `print`.
- 34
- (a) Создайте класс `Explorer`, который будет базовым для класса `LeadExplorer`. В конструкторе класса `Explorer` задайте параметры `name`, `region` и `expeditions`.
 - (b) В классе `Explorer` создайте метод `go_on_expedition`, который будет увеличивать количество экспедиций на 1.
 - (c) В классе `Explorer` создайте метод `get_region_initial`, который будет возвращать первую букву региона.
 - (d) В классе `Explorer` создайте метод `__repr__`, который будет возвращать строковое представление объекта.
 - (e) Создайте класс `LeadExplorer`, наследующийся от класса `Explorer`. В конструкторе класса `LeadExplorer` задайте параметры `name`, `region` и `expeditions`.
 - (f) В классе `LeadExplorer` переопределите метод `go_on_expedition` с использованием `super()`, чтобы количество экспедиций увеличивалось на 1 плюс бонус в 0.5 (для руководства группой).
 - (g) В основной части программы создайте объекты классов `Explorer` и `LeadExplorer` и вызовите их методы.

- (h) Выведите информацию о каждом объекте с помощью функции `print`.
- 35 (a) Создайте класс `Researcher`, который будет базовым для класса `SeniorResearcher`. В конструкторе класса `Researcher` задайте параметры `name`, `domain` и `studies_conducted`.
- (b) В классе `Researcher` создайте метод `conduct_study`, который будет увеличивать количество проведённых исследований на 1.
- (c) В классе `Researcher` создайте метод `get_domain_initial`, который будет возвращать первую букву области исследования.
- (d) В классе `Researcher` создайте метод `__repr__`, который будет возвращать строковое представление объекта.
- (e) Создайте класс `SeniorResearcher`, наследующийся от класса `Researcher`. В конструкторе класса `SeniorResearcher` задайте параметры `name`, `domain` и `studies_conducted`.
- (f) В классе `SeniorResearcher` переопределите метод `conduct_study` с использованием `super()`, чтобы количество исследований увеличивалось на 1 плюс бонус в 0.7 (для междисциплинарных проектов).
- (g) В основной части программы создайте объекты классов `Researcher` и `SeniorResearcher` и вызовите их методы.
- (h) Выведите информацию о каждом объекте с помощью функции `print`.

2.8.2 Полиморфизм с наследованием (полное переопределение методов родительского класса)

- 1 (a) Создайте класс `Vehicle`, который будет базовым для классов `Car` и `Bike`. В конструкторе класса `Vehicle` задайте параметры `name`, `color`, `price`, `travel_time`, `distance` и `speed`.
- (b) В классе `Vehicle` создайте метод `show`, который будет выводить информацию о транспортном средстве (например, с помощью функции `print`).
- (c) В классе `Vehicle` создайте метод `max_speed`, который будет выводить максимальную скорость транспортного средства.
- (d) В классе `Vehicle` создайте метод `change_gear`, который будет выводить количество передач транспортного средства.
- (e) В классе `Vehicle` создайте метод `calculate`, который будет рассчитывать время движения до пункта назначения по формуле $\text{time} = \frac{\text{distance}}{\text{speed}}$.
- (f) Создайте класс `Car`, наследующийся от класса `Vehicle`. В конструкторе класса `Car` задайте параметры `name`, `color` и `price` (остальные атрибуты, такие как `travel_time`, `distance`, `speed`, могут быть установлены по умолчанию или заданы при необходимости).
- (g) В классе `Car` полностью переопределите метод `max_speed`, чтобы он выводил максимальную скорость автомобиля (например, 180 км/ч).
- (h) В классе `Car` полностью переопределите метод `change_gear`, чтобы он выводил количество передач автомобиля (например, 6).
- (i) В классе `Car` полностью переопределите метод `calculate`, чтобы он рассчитывал пройденное расстояние по формуле $\text{distance} = \text{speed} \times \text{travel_time}$.
- (j) Создайте класс `Bike`, наследующийся от класса `Vehicle`. В конструкторе класса `Bike` задайте параметры `name`, `color` и `price` (остальные атрибуты могут быть заданы по умолчанию).

- (k) В классе **Bike** полностью переопределите метод **max_speed**, чтобы он выводил максимальную скорость мотоцикла (например, 120 км/ч).
 - (l) В классе **Bike** полностью переопределите метод **change_gear**, чтобы он выводил количество передач мотоцикла (например, 5).
 - (m) В классе **Bike** полностью переопределите метод **calculate**, чтобы он рассчитывал среднюю скорость по формуле $\text{speed} = \frac{\text{distance}}{\text{travel_time}}$.
 - (n) В основной части программы создайте объекты классов **Vehicle**, **Car** и **Bike** и вызовите их методы.
 - (o) В основной части программы создайте список, содержащий объекты разных классов (**Vehicle**, **Car**, **Bike**), и организуйте цикл по этой коллекции, в котором вызываются все общие методы (**show**, **max_speed**, **change_gear**, **calculate**) для каждого объекта — демонстрируя полиморфное поведение.
- 2 (a) Создайте класс **Animal**, который будет базовым для классов **Dog** и **Cat**. В конструкторе класса **Animal** задайте параметры **name**, **age**, **weight**, **food_per_day**, **activity_hours** и **calories_per_hour**.
- (b) В классе **Animal** создайте метод **info**, который будет выводить информацию о животном.
 - (c) В классе **Animal** создайте метод **max_speed**, который будет выводить максимальную скорость передвижения животного.
 - (d) В классе **Animal** создайте метод **sound**, который будет выводить звук, издаваемый животным.
 - (e) В классе **Animal** создайте метод **calculate**, который будет рассчитывать суточную калорийность по формуле $\text{calories} = \text{calories_per_hour} \times \text{activity_hours}$.
 - (f) Создайте класс **Dog**, наследующийся от **Animal**. В конструкторе задайте **name**, **age**, **weight**.
 - (g) В классе **Dog** полностью переопределите метод **max_speed** (например, 40 км/ч).
 - (h) В классе **Dog** полностью переопределите метод **sound** (например, «Гав!»).
 - (i) В классе **Dog** полностью переопределите метод **calculate**, чтобы он вычислял количество еды по формуле $\text{food_per_day} = \text{weight} \times 0.03$.
 - (j) Создайте класс **Cat**, наследующийся от **Animal**. В конструкторе задайте **name**, **age**, **weight**.
 - (k) В классе **Cat** полностью переопределите метод **max_speed** (например, 30 км/ч).
 - (l) В классе **Cat** полностью переопределите метод **sound** (например, «Мяу!»).
 - (m) В классе **Cat** полностью переопределите метод **calculate**, чтобы он вычислял активные часы по формуле $\text{activity_hours} = \frac{\text{food_per_day}}{0.02}$.
 - (n) Создайте объекты всех трёх классов и вызовите их методы.
 - (o) Создайте список из объектов разных классов и в цикле вызовите все общие методы, демонстрируя полиморфизм.
- 3 (a) Создайте класс **Employee**, который будет базовым для классов **Manager** и **Developer**. В конструкторе задайте параметры **name**, **department**, **base_salary**, **bonus_percent**, **hours_worked**, **hourly_rate**.

- (b) В классе `Employee` создайте метод `display`, который выводит информацию о сотруднике.
 - (c) В классе `Employee` создайте метод `get_role`, который выводит должность.
 - (d) В классе `Employee` создайте метод `get_tools`, который выводит основные инструменты работы.
 - (e) В классе `Employee` создайте метод `calculate`, который вычисляет итоговую зарплату по формуле $total = base_salary + base_salary \times \frac{bonus_percent}{100}$.
 - (f) Создайте класс `Manager`, наследующийся от `Employee`. В конструкторе задайте `name`, `department`, `base_salary`.
 - (g) В классе `Manager` полностью переопределите метод `get_role` (например, «Руководитель проекта»).
 - (h) В классе `Manager` полностью переопределите метод `get_tools` (например, «Диаграммы Ганта, Jira»).
 - (i) В классе `Manager` полностью переопределите метод `calculate`, чтобы он вычислял бонус как $bonus = hours_worked \times 500$.
 - (j) Создайте класс `Developer`, наследующийся от `Employee`. В конструкторе задайте `name`, `department`, `base_salary`.
 - (k) В классе `Developer` полностью переопределите метод `get_role` (например, «Программист»).
 - (l) В классе `Developer` полностью переопределите метод `get_tools` (например, «VS Code, Git»).
 - (m) В классе `Developer` полностью переопределите метод `calculate`, чтобы он вычислял зарплату по формуле $total = hours_worked \times hourly_rate$.
 - (n) Создайте объекты всех трёх классов и вызовите их методы.
 - (o) Создайте список из объектов разных классов и в цикле вызовите все общие методы, демонстрируя полиморфизм.
- 4 (a) Создайте класс `Appliance`, который будет базовым для классов `Fridge` и `Microwave`. В конструкторе задайте параметры `brand`, `model`, `price`, `power`, `usage_hours`, `efficiency`.
- (b) В классе `Appliance` создайте метод `details`, который выводит информацию об устройстве.
 - (c) В классе `Appliance` создайте метод `energy_class`, который выводит класс энергоэффективности.
 - (d) В классе `Appliance` создайте метод `functionality`, который выводит основную функцию устройства.
 - (e) В классе `Appliance` создайте метод `calculate`, который вычисляет месячное энергопотребление: $consumption = power \times usage_hours \times 30/1000$.
 - (f) Создайте класс `Fridge`, наследующийся от `Appliance`. В конструкторе задайте `brand`, `model`, `price`.
 - (g) В классе `Fridge` полностью переопределите метод `energy_class` (например, «A++»).
 - (h) В классе `Fridge` полностью переопределите метод `functionality` (например, «Охлаждение и хранение продуктов»).

- (i) В классе **Fridge** полностью переопределите метод **calculate**, чтобы он вычислял годовую стоимость: $\text{cost} = \text{consumption} \times 12 \times 6$.
 - (j) Создайте класс **Microwave**, наследующийся от **Appliance**. В конструкторе задайте **brand**, **model**, **price**.
 - (k) В классе **Microwave** полностью переопределите метод **energy_class** (например, «А»).
 - (l) В классе **Microwave** полностью переопределите метод **functionality** (например, «Разогрев и разморозка»).
 - (m) В классе **Microwave** полностью переопределите метод **calculate**, чтобы он вычислял эффективность: $\text{efficiency} = \frac{\text{power}}{\text{usage_hours} + 1}$.
 - (n) Создайте объекты всех трёх классов и вызовите их методы.
 - (o) Создайте список из объектов разных классов и в цикле вызовите все общие методы, демонстрируя полиморфизм.
- 5
- (a) Создайте класс **Shape**, который будет базовым для классов **Circle** и **Rectangle**. В конструкторе задайте параметры **color**, **border_width**, **area**, **perimeter**, **dimension1** (по умолчанию **None**), **dimension2** (по умолчанию **None**).
 - (b) В классе **Shape** создайте метод **draw**, который выводит информацию о фигуре.
 - (c) В классе **Shape** создайте метод **get_type**, который выводит тип фигуры.
 - (d) В классе **Shape** создайте метод **get_symmetry**, который выводит наличие симметрии.
 - (e) В классе **Shape** создайте метод **calculate**, который вычисляет площадь как $\text{area} = \text{dimension1} \times \text{dimension2}$ (если оба заданы).
 - (f) Создайте класс **Circle**, наследующийся от **Shape**. В конструкторе задайте **color**, **border_width**, **radius**. При вызове **super().__init__()** передайте **dimension1=radius**, **dimension2=radius**.
 - (g) В классе **Circle** полностью переопределите метод **get_type** (например, «Окружность»).
 - (h) В классе **Circle** полностью переопределите метод **get_symmetry** (например, «Осевая и центральная»).
 - (i) В классе **Circle** полностью переопределите метод **calculate**, чтобы он вычислял площадь как $\pi \times r^2$.
 - (j) Создайте класс **Rectangle**, наследующийся от **Shape**. В конструкторе задайте **color**, **border_width**, **width**, **height**. При вызове **super().__init__()** передайте **dimension1=width**, **dimension2=height**.
 - (k) В классе **Rectangle** полностью переопределите метод **get_type** (например, «Прямоугольник»).
 - (l) В классе **Rectangle** полностью переопределите метод **get_symmetry** (например, «Осевая по двум осям»).
 - (m) В классе **Rectangle** полностью переопределите метод **calculate**, чтобы он вычислял периметр: $2 \times (\text{width} + \text{height})$.
 - (n) Создайте объекты всех трёх классов и вызовите их методы.
 - (o) Создайте список из объектов разных классов и в цикле вызовите все общие методы, демонстрируя полиморфизм.

- 6
 - (a) Создайте класс `Book`, который будет базовым для классов `Fiction` и `Textbook`. В конструкторе задайте параметры `title`, `author`, `price`, `pages`, `reading_time`, `complexity`.
 - (b) В классе `Book` создайте метод `info`, который выводит информацию о книге.
 - (c) В классе `Book` создайте метод `genre`, который выводит жанр.
 - (d) В классе `Book` создайте метод `audience`, который выводит целевую аудиторию.
 - (e) В классе `Book` создайте метод `calculate`, который вычисляет среднюю скорость чтения: $\text{speed} = \frac{\text{pages}}{\text{reading_time}}$.
 - (f) Создайте класс `Fiction`, наследующийся от `Book`. В конструкторе задайте `title`, `author`, `price`.
 - (g) В классе `Fiction` полностью переопределите метод `genre` (например, «Художественная литература»).
 - (h) В классе `Fiction` полностью переопределите метод `audience` (например, «Все возрасты»).
 - (i) В классе `Fiction` полностью переопределите метод `calculate`, чтобы он вычислял стоимость за страницу: $\frac{\text{price}}{\text{pages}}$.
 - (j) Создайте класс `Textbook`, наследующийся от `Book`. В конструкторе задайте `title`, `author`, `price`.
 - (k) В классе `Textbook` полностью переопределите метод `genre` (например, «Учебная литература»).
 - (l) В классе `Textbook` полностью переопределите метод `audience` (например, «Студенты»).
 - (m) В классе `Textbook` полностью переопределите метод `calculate`, чтобы он вычислял сложность: $\text{complexity} = \frac{\text{pages}}{\text{reading_time} \times 2}$.
 - (n) Создайте объекты всех трёх классов и вызовите их методы.
 - (o) Создайте список из объектов разных классов и в цикле вызовите все общие методы, демонстрируя полиморфизм.
- 7
 - (a) Создайте класс `Instrument`, который будет базовым для классов `Guitar` и `Piano`. В конструкторе задайте параметры `name`, `brand`, `price`, `strings`, `keys`, `volume`.
 - (b) В классе `Instrument` создайте метод `describe`, который выводит описание инструмента.
 - (c) В классе `Instrument` создайте метод `play_style`, который выводит способ игры.
 - (d) В классе `Instrument` создайте метод `material`, который выводит основной материал.
 - (e) В классе `Instrument` создайте метод `calculate`, который вычисляет громкость: $\text{volume} = \text{strings} + \text{keys}$.
 - (f) Создайте класс `Guitar`, наследующийся от `Instrument`. В конструкторе задайте `name`, `brand`, `price`.
 - (g) В классе `Guitar` полностью переопределите метод `play_style` (например, «Перебор или боем»).
 - (h) В классе `Guitar` полностью переопределите метод `material` (например, «Дерево»).

- (i) В классе **Guitar** полностью переопределите метод **calculate**, чтобы он вычислял цену за струну: $\frac{\text{price}}{\text{strings}}$.
 - (j) Создайте класс **Piano**, наследующийся от **Instrument**. В конструкторе задайте **name**, **brand**, **price**.
 - (k) В классе **Piano** полностью переопределите метод **play_style** (например, «Клавиши»).
 - (l) В классе **Piano** полностью переопределите метод **material** (например, «Металл и дерево»).
 - (m) В классе **Piano** полностью переопределите метод **calculate**, чтобы он вычислял количество октав: $\frac{\text{keys}}{12}$.
 - (n) Создайте объекты всех трёх классов и вызовите их методы.
 - (o) Создайте список из объектов разных классов и в цикле вызовите все общие методы, демонстрируя полиморфизм.
- 8 (a) Создайте класс **Drink**, который будет базовым для классов **Coffee** и **Tea**. В конструкторе задайте параметры **name**, **temperature**, **volume**, **caffeine**, **brew_time**, **price**.
- (b) В классе **Drink** создайте метод **info**, который выводит информацию о напитке.
 - (c) В классе **Drink** создайте метод **origin**, который выводит страну происхождения.
 - (d) В классе **Drink** создайте метод **serving**, который выводит способ подачи.
 - (e) В классе **Drink** создайте метод **calculate**, который вычисляет концентрацию кофеина: $\frac{\text{caffeine}}{\text{volume}}$.
 - (f) Создайте класс **Coffee**, наследующийся от **Drink**. В конструкторе задайте **name**, **temperature**, **price**.
 - (g) В классе **Coffee** полностью переопределите метод **origin** (например, «Эфиопия»).
 - (h) В классе **Coffee** полностью переопределите метод **serving** (например, «В чашке с блюдцем»).
 - (i) В классе **Coffee** полностью переопределите метод **calculate**, чтобы он вычислял время заваривания: $\text{brew_time} = \frac{\text{volume}}{30}$.
 - (j) Создайте класс **Tea**, наследующийся от **Drink**. В конструкторе задайте **name**, **temperature**, **price**.
 - (k) В классе **Tea** полностью переопределите метод **origin** (например, «Китай»).
 - (l) В классе **Tea** полностью переопределите метод **serving** (например, «В пиале или стеклянном стакане»).
 - (m) В классе **Tea** полностью переопределите метод **calculate**, чтобы он вычислял объём: $\text{volume} = \text{brew_time} \times 25$.
 - (n) Создайте объекты всех трёх классов и вызовите их методы.
 - (o) Создайте список из объектов разных классов и в цикле вызовите все общие методы, демонстрируя полиморфизм.
- 9 (a) Создайте класс **Plant**, который будет базовым для классов **Flower** и **Tree**. В конструкторе задайте параметры **species**, **height**, **age**, **water_per_week**, **sunlight_hours**, **blooming_season**.

- (b) В классе `Plant` создайте метод `describe`, который выводит описание растения.
 - (c) В классе `Plant` создайте метод `type`, который выводит тип растения.
 - (d) В классе `Plant` создайте метод `care_level`, который выводит уровень ухода.
 - (e) В классе `Plant` создайте метод `calculate`, который вычисляет потребление воды в месяц: $\text{water_per_week} \times 4$.
 - (f) Создайте класс `Flower`, наследующийся от `Plant`. В конструкторе задайте `species`, `height`, `age`.
 - (g) В классе `Flower` полностью переопределите метод `type` (например, «Цветковое»).
 - (h) В классе `Flower` полностью переопределите метод `care_level` (например, «Высокий»).
 - (i) В классе `Flower` полностью переопределите метод `calculate`, чтобы он вычислял сезон цветения: $\text{blooming_season} = \text{age} \% 4 + 1$.
 - (j) Создайте класс `Tree`, наследующийся от `Plant`. В конструкторе задайте `species`, `height`, `age`.
 - (k) В классе `Tree` полностью переопределите метод `type` (например, «Древесное»).
 - (l) В классе `Tree` полностью переопределите метод `care_level` (например, «Низкий»).
 - (m) В классе `Tree` полностью переопределите метод `calculate`, чтобы он вычислял высоту: $\text{height} = \text{age} \times 0.5$.
 - (n) Создайте объекты всех трёх классов и вызовите их методы.
 - (o) Создайте список из объектов разных классов и в цикле вызовите все общие методы, демонстрируя полиморфизм.
- 10 (a) Создайте класс `Account`, который будет базовым для классов `Savings` и `Checking`. В конструкторе задайте параметры `owner`, `balance`, `interest_rate`, `monthly_fee`, `transactions`, `limit`.
- (b) В классе `Account` создайте метод `info`, который выводит информацию о счёте.
 - (c) В классе `Account` создайте метод `account_type`, который выводит тип счёта.
 - (d) В классе `Account` создайте метод `features`, который выводит особенности счёта.
 - (e) В классе `Account` создайте метод `calculate`, который вычисляет годовой доход от процентов: $\text{balance} \times \frac{\text{interest_rate}}{100}$.
 - (f) Создайте класс `Savings`, наследующийся от `Account`. В конструкторе задайте `owner`, `balance`, `interest_rate`.
 - (g) В классе `Savings` полностью переопределите метод `account_type` (например, «Сберегательный»).
 - (h) В классе `Savings` полностью переопределите метод `features` (например, «Высокая ставка, ограничение на снятие»).
 - (i) В классе `Savings` полностью переопределите метод `calculate`, чтобы он вычислял лимит снятия: $\text{limit} = \text{balance} \times 0.2$.
 - (j) Создайте класс `Checking`, наследующийся от `Account`. В конструкторе задайте `owner`, `balance`, `monthly_fee`.
 - (k) В классе `Checking` полностью переопределите метод `account_type` (например, «Текущий»).

- (l) В классе **Checking** полностью переопределите метод **features** (например, «Бесплатные переводы, дебетовая карта»).
 - (m) В классе **Checking** полностью переопределите метод **calculate**, чтобы он вычислял чистый баланс: $\text{balance} - \text{monthly_fee} \times 12$.
 - (n) Создайте объекты всех трёх классов и вызовите их методы.
 - (o) Создайте список из объектов разных классов и в цикле вызовите все общие методы, демонстрируя полиморфизм.
- 11
- (a) Создайте класс **Building**, который будет базовым для классов **House** и **Office**. В конструкторе задайте параметры **name**, **floors**, **area**, **rooms**, **occupants**, **construction_year**.
 - (b) В классе **Building** создайте метод **overview**, который выводит информацию о здании.
 - (c) В классе **Building** создайте метод **purpose**, который выводит назначение здания.
 - (d) В классе **Building** создайте метод **facilities**, который выводит доступные удобства.
 - (e) В классе **Building** создайте метод **calculate**, который вычисляет плотность заселения: $\frac{\text{occupants}}{\text{area}}$.
 - (f) Создайте класс **House**, наследующийся от **Building**. В конструкторе задайте **name**, **floors**, **area**.
 - (g) В классе **House** полностью переопределите метод **purpose** (например, «Жилое»).
 - (h) В классе **House** полностью переопределите метод **facilities** (например, «Сад, гараж»).
 - (i) В классе **House** полностью переопределите метод **calculate**, чтобы он вычислял количество комнат: $\text{rooms} = \text{floors} \times 3$.
 - (j) Создайте класс **Office**, наследующийся от **Building**. В конструкторе задайте **name**, **floors**, **area**.
 - (k) В классе **Office** полностью переопределите метод **purpose** (например, «Коммерческое»).
 - (l) В классе **Office** полностью переопределите метод **facilities** (например, «Конференц-зал, лифт»).
 - (m) В классе **Office** полностью переопределите метод **calculate**, чтобы он вычислял год постройки: $\text{construction_year} = 2025 - \text{floors} \times 2$.
 - (n) Создайте объекты всех трёх классов и вызовите их методы.
 - (o) Создайте список из объектов разных классов и в цикле вызовите все общие методы, демонстрируя полиморфизм.
- 12
- (a) Создайте класс **Game**, который будет базовым для классов **RPG** и **Puzzle**. В конструкторе задайте параметры **title**, **genre**, **price**, **playtime**, **difficulty**, **rating**.
 - (b) В классе **Game** создайте метод **summary**, который выводит краткое описание игры.
 - (c) В классе **Game** создайте метод **platform**, который выводит платформу.
 - (d) В классе **Game** создайте метод **audience**, который выводит целевую аудиторию.
 - (e) В классе **Game** создайте метод **calculate**, который вычисляет средний рейтинг: $\frac{\text{rating} + \text{difficulty}}{2}$.

- (f) Создайте класс **RPG**, наследующийся от **Game**. В конструкторе задайте **title**, **price**, **playtime**.
 - (g) В классе **RPG** полностью переопределите метод **platform** (например, «PC, консоли»).
 - (h) В классе **RPG** полностью переопределите метод **audience** (например, «Подростки и взрослые»).
 - (i) В классе **RPG** полностью переопределите метод **calculate**, чтобы он вычислял сложность: $\text{difficulty} = \frac{\text{playtime}}{10}$.
 - (j) Создайте класс **Puzzle**, наследующийся от **Game**. В конструкторе задайте **title**, **price**, **playtime**.
 - (k) В классе **Puzzle** полностью переопределите метод **platform** (например, «Мобильные устройства»).
 - (l) В классе **Puzzle** полностью переопределите метод **audience** (например, «Все возрасты»).
 - (m) В классе **Puzzle** полностью переопределите метод **calculate**, чтобы он вычислял рейтинг: $\text{rating} = 10 - \frac{\text{difficulty}}{2}$.
 - (n) Создайте объекты всех трёх классов и вызовите их методы.
 - (o) Создайте список из объектов разных классов и в цикле вызовите все общие методы, демонстрируя полиморфизм.
- 13
- (a) Создайте класс **Clothing**, который будет базовым для классов **TShirt** и **Jacket**. В конструкторе задайте параметры **brand**, **size**, **color**, **price**, **material**, **season**.
 - (b) В классе **Clothing** создайте метод **label**, который выводит информацию об одежде.
 - (c) В классе **Clothing** создайте метод **type**, который выводит тип изделия.
 - (d) В классе **Clothing** создайте метод **care_instructions**, который выводит рекомендации по уходу.
 - (e) В классе **Clothing** создайте метод **calculate**, который вычисляет соотношение цены к размеру: $\frac{\text{price}}{\text{size}}$.
 - (f) Создайте класс **TShirt**, наследующийся от **Clothing**. В конструкторе задайте **brand**, **size**, **color**.
 - (g) В классе **TShirt** полностью переопределите метод **type** (например, «Футболка»).
 - (h) В классе **TShirt** полностью переопределите метод **care_instructions** (например, «Стирка при 30°C»).
 - (i) В классе **TShirt** полностью переопределите метод **calculate**, чтобы он вычислял сезон: $\text{season} = \text{size} \% 4 + 1$.
 - (j) Создайте класс **Jacket**, наследующийся от **Clothing**. В конструкторе задайте **brand**, **size**, **color**.
 - (k) В классе **Jacket** полностью переопределите метод **type** (например, «Куртка»).
 - (l) В классе **Jacket** полностью переопределите метод **care_instructions** (например, «Химчистка»).
 - (m) В классе **Jacket** полностью переопределите метод **calculate**, чтобы он вычислял материал: **material** = «Нейлон» если **price** > 5000, иначе «Хлопок».

- (n) Создайте объекты всех трёх классов и вызовите их методы.
 - (o) Создайте список из объектов разных классов и в цикле вызовите все общие методы, демонстрируя полиморфизм.
- 14
- (a) Создайте класс **Electronics**, который будет базовым для классов **Phone** и **Laptop**. В конструкторе задайте параметры **model**, **brand**, **price**, **battery_life**, **weight**, **warranty**.
 - (b) В классе **Electronics** создайте метод **spec**, который выводит характеристики устройства.
 - (c) В классе **Electronics** создайте метод **category**, который выводит категорию.
 - (d) В классе **Electronics** создайте метод **connectivity**, который выводит типы подключения.
 - (e) В классе **Electronics** создайте метод **calculate**, который вычисляет стоимость гарантии: $\text{warranty} \times 500$.
 - (f) Создайте класс **Phone**, наследующийся от **Electronics**. В конструкторе задайте **model**, **brand**, **price**.
 - (g) В классе **Phone** полностью переопределите метод **category** (например, «Смартфон»).
 - (h) В классе **Phone** полностью переопределите метод **connectivity** (например, «5G, Wi-Fi, Bluetooth»).
 - (i) В классе **Phone** полностью переопределите метод **calculate**, чтобы он вычислял время работы: $\text{battery_life} = \text{price}/1000$.
 - (j) Создайте класс **Laptop**, наследующийся от **Electronics**. В конструкторе задайте **model**, **brand**, **price**.
 - (k) В классе **Laptop** полностью переопределите метод **category** (например, «Ноутбук»).
 - (l) В классе **Laptop** полностью переопределите метод **connectivity** (например, «Wi-Fi, Ethernet, HDMI»).
 - (m) В классе **Laptop** полностью переопределите метод **calculate**, чтобы он вычислял вес: $\text{weight} = \text{price}/20000 + 1$.
 - (n) Создайте объекты всех трёх классов и вызовите их методы.
 - (o) Создайте список из объектов разных классов и в цикле вызовите все общие методы, демонстрируя полиморфизм.
- 15
- (a) Создайте класс **Food**, который будет базовым для классов **Pizza** и **Salad**. В конструкторе задайте параметры **name**, **calories**, **price**, **weight**, **cooking_time**, **ingredients**.
 - (b) В классе **Food** создайте метод **nutrition**, который выводит пищевую ценность.
 - (c) В классе **Food** создайте метод **type**, который выводит тип блюда.
 - (d) В классе **Food** создайте метод **diet_suitable**, который выводит, подходит ли блюдо для диеты.
 - (e) В классе **Food** создайте метод **calculate**, который вычисляет калорийность на 100 г: $\frac{\text{calories}}{\text{weight}} \times 100$.

- (f) Создайте класс `Pizza`, наследующийся от `Food`. В конструкторе задайте `name`, `price`, `weight`.
 - (g) В классе `Pizza` полностью переопределите метод `type` (например, «Горячее»).
 - (h) В классе `Pizza` полностью переопределите метод `diet_suitable` (например, «Нет»).
 - (i) В классе `Pizza` полностью переопределите метод `calculate`, чтобы он вычислял время готовки: $\text{cooking_time} = \text{weight}/100 \times 3$.
 - (j) Создайте класс `Salad`, наследующийся от `Food`. В конструкторе задайте `name`, `price`, `weight`.
 - (k) В классе `Salad` полностью переопределите метод `type` (например, «Холодное»).
 - (l) В классе `Salad` полностью переопределите метод `diet_suitable` (например, «Да»).
 - (m) В классе `Salad` полностью переопределите метод `calculate`, чтобы он вычислял ингредиенты: $\text{ingredients} = \text{weight}/50$.
 - (n) Создайте объекты всех трёх классов и вызовите их методы.
 - (o) Создайте список из объектов разных классов и в цикле вызовите все общие методы, демонстрируя полиморфизм.
- 16
- (a) Создайте класс `Sport`, который будет базовым для классов `Football` и `Swimming`. В конструкторе задайте параметры `name`, `players`, `duration`, `equipment`, `intensity`, `calories_per_hour`.
 - (b) В классе `Sport` создайте метод `rules`, который выводит основные правила.
 - (c) В классе `Sport` создайте метод `venue`, который выводит место проведения.
 - (d) В классе `Sport` создайте метод `team_based`, который выводит, командный ли вид спорта.
 - (e) В классе `Sport` создайте метод `calculate`, который вычисляет общие калории: $\text{calories_per_hour} \times \frac{\text{duration}}{60}$.
 - (f) Создайте класс `Football`, наследующийся от `Sport`. В конструкторе задайте `name`, `duration`, `intensity`.
 - (g) В классе `Football` полностью переопределите метод `venue` (например, «Стадион»).
 - (h) В классе `Football` полностью переопределите метод `team_based` (например, «Да»).
 - (i) В классе `Football` полностью переопределите метод `calculate`, чтобы он вычислял количество игроков: `players = 11`.
 - (j) Создайте класс `Swimming`, наследующийся от `Sport`. В конструкторе задайте `name`, `duration`, `intensity`.
 - (k) В классе `Swimming` полностью переопределите метод `venue` (например, «Бассейн»).
 - (l) В классе `Swimming` полностью переопределите метод `team_based` (например, «Нет»).
 - (m) В классе `Swimming` полностью переопределите метод `calculate`, чтобы он вычислял оборудование: `equipment = «Купальник, очки»`.
 - (n) Создайте объекты всех трёх классов и вызовите их методы.
 - (o) Создайте список из объектов разных классов и в цикле вызовите все общие методы, демонстрируя полиморфизм.

- 17
 - (a) Создайте класс `Transport`, который будет базовым для классов `Train` и `Airplane`. В конструкторе задайте параметры `name`, `capacity`, `speed`, `fuel_consumption`, `range`, `ticket_price`.
 - (b) В классе `Transport` создайте метод `info`, который выводит информацию о транспорте.
 - (c) В классе `Transport` создайте метод `type`, который выводит тип транспорта.
 - (d) В классе `Transport` создайте метод `environmental_impact`, который выводит уровень воздействия на окружающую среду.
 - (e) В классе `Transport` создайте метод `calculate`, который вычисляет стоимость на пассажира: $\frac{\text{ticket_price} \times \text{capacity}}{\text{range}}$.
 - (f) Создайте класс `Train`, наследующийся от `Transport`. В конструкторе задайте `name`, `capacity`, `speed`.
 - (g) В классе `Train` полностью переопределите метод `type` (например, «Железнодорожный»).
 - (h) В классе `Train` полностью переопределите метод `environmental_impact` (например, «Низкий»).
 - (i) В классе `Train` полностью переопределите метод `calculate`, чтобы он вычислял дальность: $\text{range} = \text{speed} \times 10$.
 - (j) Создайте класс `Airplane`, наследующийся от `Transport`. В конструкторе задайте `name`, `capacity`, `speed`.
 - (k) В классе `Airplane` полностью переопределите метод `type` (например, «Воздушный»).
 - (l) В классе `Airplane` полностью переопределите метод `environmental_impact` (например, «Высокий»).
 - (m) В классе `Airplane` полностью переопределите метод `calculate`, чтобы он вычислял расход топлива: $\text{fuel_consumption} = \text{speed} \times 0.1$.
 - (n) Создайте объекты всех трёх классов и вызовите их методы.
 - (o) Создайте список из объектов разных классов и в цикле вызовите все общие методы, демонстрируя полиморфизм.
- 18
 - (a) Создайте класс `Pet`, который будет базовым для классов `Parrot` и `Hamster`. В конструкторе задайте параметры `name`, `age`, `weight`, `food_per_day`, `lifespan`, `noise_level`.
 - (b) В классе `Pet` создайте метод `profile`, который выводит профиль питомца.
 - (c) В классе `Pet` создайте метод `habitat`, который выводит среду обитания.
 - (d) В классе `Pet` создайте метод `trainable`, который выводит, обучаем ли питомец.
 - (e) В классе `Pet` создайте метод `calculate`, который вычисляет ожидаемую продолжительность жизни: $\text{lifespan} - \text{age}$.
 - (f) Создайте класс `Parrot`, наследующийся от `Pet`. В конструкторе задайте `name`, `age`, `weight`.
 - (g) В классе `Parrot` полностью переопределите метод `habitat` (например, «Клетка с игрушками»).
 - (h) В классе `Parrot` полностью переопределите метод `trainable` (например, «Да»).

- (i) В классе **Parrot** полностью переопределите метод **calculate**, чтобы он вычислял уровень шума: $\text{noise_level} = 10 - \text{age}$.
 - (j) Создайте класс **Hamster**, наследующийся от **Pet**. В конструкторе задайте **name**, **age**, **weight**.
 - (k) В классе **Hamster** полностью переопределите метод **habitat** (например, «Клетка с колесом»).
 - (l) В классе **Hamster** полностью переопределите метод **trainable** (например, «Нет»).
 - (m) В классе **Hamster** полностью переопределите метод **calculate**, чтобы он вычислял еду в день: $\text{food_per_day} = \text{weight} \times 0.05$.
 - (n) Создайте объекты всех трёх классов и вызовите их методы.
 - (o) Создайте список из объектов разных классов и в цикле вызовите все общие методы, демонстрируя полиморфизм.
- 19 (a) Создайте класс **Furniture**, который будет базовым для классов **Chair** и **Table**. В конструкторе задайте параметры **material**, **color**, **price**, **weight**, **height**, **assembly_time**.
- (b) В классе **Furniture** создайте метод **spec**, который выводит спецификацию мебели.
 - (c) В классе **Furniture** создайте метод **category**, который выводит категорию мебели.
 - (d) В классе **Furniture** создайте метод **indoor_use**, который выводит, предназначена ли мебель для использования внутри помещения.
 - (e) В классе **Furniture** создайте метод **calculate**, который вычисляет соотношение цены к весу: $\frac{\text{price}}{\text{weight}}$.
 - (f) Создайте класс **Chair**, наследующийся от **Furniture**. В конструкторе задайте **material**, **color**, **price**.
 - (g) В классе **Chair** полностью переопределите метод **category** (например, «Сидячая мебель»).
 - (h) В классе **Chair** полностью переопределите метод **indoor_use** (например, «Да»).
 - (i) В классе **Chair** полностью переопределите метод **calculate**, чтобы он вычислял высоту: $\text{height} = \text{price}/500$.
 - (j) Создайте класс **Table**, наследующийся от **Furniture**. В конструкторе задайте **material**, **color**, **price**.
 - (k) В классе **Table** полностью переопределите метод **category** (например, «Поверхность»).
 - (l) В классе **Table** полностью переопределите метод **indoor_use** (например, «Да»).
 - (m) В классе **Table** полностью переопределите метод **calculate**, чтобы он вычислял время сборки: $\text{assembly_time} = \text{weight}/2$.
 - (n) Создайте объекты всех трёх классов и вызовите их методы.
 - (o) Создайте список из объектов разных классов и в цикле вызовите все общие методы, демонстрируя полиморфизм.
- 20 (a) Создайте класс **Media**, который будет базовым для классов **Movie** и **Album**. В конструкторе задайте параметры **title**, **creator**, **year**, **rating**, **duration**, **genre**.

- (b) В классе **Media** создайте метод **details**, который выводит детали медиа.
 - (c) В классе **Media** создайте метод **format**, который выводит формат.
 - (d) В классе **Media** создайте метод **audience_rating**, который выводит возрастной рейтинг.
 - (e) В классе **Media** создайте метод **calculate**, который вычисляет среднюю оценку: $\frac{\text{rating} + \text{year} \% 10}{2}$.
 - (f) Создайте класс **Movie**, наследующийся от **Media**. В конструкторе задайте **title**, **creator**, **year**.
 - (g) В классе **Movie** полностью переопределите метод **format** (например, «Видео»).
 - (h) В классе **Movie** полностью переопределите метод **audience_rating** (например, «12+»).
 - (i) В классе **Movie** полностью переопределите метод **calculate**, чтобы он вычислял продолжительность: $\text{duration} = 90 + \text{rating} \times 5$.
 - (j) Создайте класс **Album**, наследующийся от **Media**. В конструкторе задайте **title**, **creator**, **year**.
 - (k) В классе **Album** полностью переопределите метод **format** (например, «Аудио»).
 - (l) В классе **Album** полностью переопределите метод **audience_rating** (например, «0+»).
 - (m) В классе **Album** полностью переопределите метод **calculate**, чтобы он вычислял жанр: **genre** = «Рок» если $\text{year} \% 5 == 0$, иначе «Поп».
 - (n) Создайте объекты всех трёх классов и вызовите их методы.
 - (o) Создайте список из объектов разных классов и в цикле вызовите все общие методы, демонстрируя полиморфизм.
- 21
- (a) Создайте класс **Tool**, который будет базовым для классов **Hammer** и **Screwdriver**. В конструкторе задайте параметры **name**, **material**, **price**, **weight**, **length**, **uses**.
 - (b) В классе **Tool** создайте метод **description**, который выводит описание инструмента.
 - (c) В классе **Tool** создайте метод **purpose**, который выводит назначение.
 - (d) В классе **Tool** создайте метод **durability**, который выводит долговечность.
 - (e) В классе **Tool** создайте метод **calculate**, который вычисляет соотношение длины к весу: $\frac{\text{length}}{\text{weight}}$.
 - (f) Создайте класс **Hammer**, наследующийся от **Tool**. В конструкторе задайте **name**, **material**, **price**.
 - (g) В классе **Hammer** полностью переопределите метод **purpose** (например, «Забивание гвоздей»).
 - (h) В классе **Hammer** полностью переопределите метод **durability** (например, «Высокая»).
 - (i) В классе **Hammer** полностью переопределите метод **calculate**, чтобы он вычислял длину: $\text{length} = \text{price} / 100$.
 - (j) Создайте класс **Screwdriver**, наследующийся от **Tool**. В конструкторе задайте **name**, **material**, **price**.

- (k) В классе **Screwdriver** полностью переопределите метод **purpose** (например, «Закручивание винтов»).
 - (l) В классе **Screwdriver** полностью переопределите метод **durability** (например, «Средняя»).
 - (m) В классе **Screwdriver** полностью переопределите метод **calculate**, чтобы он вычислял количество применений: $uses = price \times 2$.
 - (n) Создайте объекты всех трёх классов и вызовите их методы.
 - (o) Создайте список из объектов разных классов и в цикле вызовите все общие методы, демонстрируя полиморфизм.
- 22
- (a) Создайте класс **Course**, который будет базовым для классов **Programming** и **Design**. В конструкторе задайте параметры **title**, **instructor**, **duration_weeks**, **price**, **students**, **difficulty**.
 - (b) В классе **Course** создайте метод **outline**, который выводит программу курса.
 - (c) В классе **Course** создайте метод **category**, который выводит категорию.
 - (d) В классе **Course** создайте метод **certification**, который выводит наличие сертификата.
 - (e) В классе **Course** создайте метод **calculate**, который вычисляет стоимость за неделю: $\frac{price}{duration_weeks}$.
 - (f) Создайте класс **Programming**, наследующийся от **Course**. В конструкторе задайте **title**, **instructor**, **price**.
 - (g) В классе **Programming** полностью переопределите метод **category** (например, «ИТ»).
 - (h) В классе **Programming** полностью переопределите метод **certification** (например, «Да»).
 - (i) В классе **Programming** полностью переопределите метод **calculate**, чтобы он вычислял сложность: $difficulty = duration_weeks/2$.
 - (j) Создайте класс **Design**, наследующийся от **Course**. В конструкторе задайте **title**, **instructor**, **price**.
 - (k) В классе **Design** полностью переопределите метод **category** (например, «Творчество»).
 - (l) В классе **Design** полностью переопределите метод **certification** (например, «Нет»).
 - (m) В классе **Design** полностью переопределите метод **calculate**, чтобы он вычислял количество студентов: $students = price/1000$.
 - (n) Создайте объекты всех трёх классов и вызовите их методы.
 - (o) Создайте список из объектов разных классов и в цикле вызовите все общие методы, демонстрируя полиморфизм.
- 23
- (a) Создайте класс **Weather**, который будет базовым для классов **Rainy** и **Sunny**. В конструкторе задайте параметры **location**, **temperature**, **humidity**, **wind_speed**, **precipitation**, **uv_index**.
 - (b) В классе **Weather** создайте метод **forecast**, который выводит прогноз погоды.
 - (c) В классе **Weather** создайте метод **type**, который выводит тип погоды.
 - (d) В классе **Weather** создайте метод **advice**, который выводит рекомендации.

- (e) В классе **Weather** создайте метод **calculate**, который вычисляет индекс комфорта: $\text{temperature} - 0.55 \times (1 - \frac{\text{humidity}}{100}) \times (\text{temperature} - 14.5)$.
 - (f) Создайте класс **Rainy**, наследующийся от **Weather**. В конструкторе задайте **location**, **temperature**, **humidity**.
 - (g) В классе **Rainy** полностью переопределите метод **type** (например, «Дождливая»).
 - (h) В классе **Rainy** полностью переопределите метод **advice** (например, «Возьмите зонт»).
 - (i) В классе **Rainy** полностью переопределите метод **calculate**, чтобы он вычислял осадки: $\text{precipitation} = \text{humidity}/10$.
 - (j) Создайте класс **Sunny**, наследующийся от **Weather**. В конструкторе задайте **location**, **temperature**, **humidity**.
 - (k) В классе **Sunny** полностью переопределите метод **type** (например, «Солнечная»).
 - (l) В классе **Sunny** полностью переопределите метод **advice** (например, «Используйте солнцезащитный крем»).
 - (m) В классе **Sunny** полностью переопределите метод **calculate**, чтобы он вычислял УФ-индекс: $\text{uv_index} = \text{temperature}/5$.
 - (n) Создайте объекты всех трёх классов и вызовите их методы.
 - (o) Создайте список из объектов разных классов и в цикле вызовите все общие методы, демонстрируя полиморфизм.
- 24 (a) Создайте класс **Event**, который будет базовым для классов **Concert** и **Exhibition**. В конструкторе задайте параметры **name**, **date**, **location**, **price**, **duration_hours**, **capacity**.
- (b) В классе **Event** создайте метод **info**, который выводит информацию о событии.
 - (c) В классе **Event** создайте метод **category**, который выводит категорию события.
 - (d) В классе **Event** создайте метод **dress_code**, который выводит дресс-код.
 - (e) В классе **Event** создайте метод **calculate**, который вычисляет доход при полной загрузке: $\text{price} \times \text{capacity}$.
 - (f) Создайте класс **Concert**, наследующийся от **Event**. В конструкторе задайте **name**, **date**, **location**.
 - (g) В классе **Concert** полностью переопределите метод **category** (например, «Музыка»).
 - (h) В классе **Concert** полностью переопределите метод **dress_code** (например, «Casual»).
 - (i) В классе **Concert** полностью переопределите метод **calculate**, чтобы он вычислял продолжительность: $\text{duration_hours} = 2 + \text{price}/1000$.
 - (j) Создайте класс **Exhibition**, наследующийся от **Event**. В конструкторе задайте **name**, **date**, **location**.
 - (k) В классе **Exhibition** полностью переопределите метод **category** (например, «Искусство»).
 - (l) В классе **Exhibition** полностью переопределите метод **dress_code** (например, «Smart casual»).
 - (m) В классе **Exhibition** полностью переопределите метод **calculate**, чтобы он вычислял вместимость: $\text{capacity} = \text{price} \times 2$.

- (n) Создайте объекты всех трёх классов и вызовите их методы.
 - (o) Создайте список из объектов разных классов и в цикле вызовите все общие методы, демонстрируя полиморфизм.
- 25
- (a) Создайте класс `Currency`, который будет базовым для классов `Dollar` и `Euro`. В конструкторе задайте параметры `code`, `name`, `rate_to_rub`, `symbol`, `fractional_unit`, `issuing_country`.
 - (b) В классе `Currency` создайте метод `info`, который выводит информацию о валюте.
 - (c) В классе `Currency` создайте метод `region`, который выводит регион обращения.
 - (d) В классе `Currency` создайте метод `stability`, который выводит уровень стабильности.
 - (e) В классе `Currency` создайте метод `calculate`, который принимает параметр `amount` и вычисляет эквивалент в рублях: $\text{amount} \times \text{rate_to_rub}$.
 - (f) Создайте класс `Dollar`, наследующийся от `Currency`. В конструкторе задайте `code`, `name`, `rate_to_rub`.
 - (g) В классе `Dollar` полностью переопределите метод `region` (например, «США, Канада»).
 - (h) В классе `Dollar` полностью переопределите метод `stability` (например, «Высокая»).
 - (i) В классе `Dollar` полностью переопределите метод `calculate`, чтобы он вычислял дробную единицу: `fractional_unit = «Цент»`.
 - (j) Создайте класс `Euro`, наследующийся от `Currency`. В конструкторе задайте `code`, `name`, `rate_to_rub`.
 - (k) В классе `Euro` полностью переопределите метод `region` (например, «Еврозона»).
 - (l) В классе `Euro` полностью переопределите метод `stability` (например, «Высокая»).
 - (m) В классе `Euro` полностью переопределите метод `calculate`, чтобы он вычислял страну-эмитента: `issuing_country = «ЕЦБ»`.
 - (n) Создайте объекты всех трёх классов и вызовите их методы.
 - (o) Создайте список из объектов разных классов и в цикле вызовите все общие методы, демонстрируя полиморфизм.
- 26
- (a) Создайте класс `Language`, который будет базовым для классов `Python` и `JavaScript`. В конструкторе задайте параметры `name`, `year_created`, `paradigm`, `popularity_rank`, `typing`, `runtime`.
 - (b) В классе `Language` создайте метод `overview`, который выводит обзор языка.
 - (c) В классе `Language` создайте метод `domain`, который выводит основную сферу применения.
 - (d) В классе `Language` создайте метод `learning_curve`, который выводит сложность изучения.
 - (e) В классе `Language` создайте метод `calculate`, который вычисляет возраст языка: $2025 - \text{year_created}$.
 - (f) Создайте класс `Python`, наследующийся от `Language`. В конструкторе задайте `name`, `year_created`, `popularity_rank`.

- (g) В классе `Python` полностью переопределите метод `domain` (например, «Data Science, Backend»).
 - (h) В классе `Python` полностью переопределите метод `learning_curve` (например, «Низкая»).
 - (i) В классе `Python` полностью переопределите метод `calculate`, чтобы он вычислял типизацию: `typing = «Динамическая»`.
 - (j) Создайте класс `JavaScript`, наследующийся от `Language`. В конструкторе задайте `name`, `year_created`, `popularity_rank`.
 - (k) В классе `JavaScript` полностью переопределите метод `domain` (например, «Frontend, Web»).
 - (l) В классе `JavaScript` полностью переопределите метод `learning_curve` (например, «Средняя»).
 - (m) В классе `JavaScript` полностью переопределите метод `calculate`, чтобы он вычислял среду выполнения: `runtime = «V8, SpiderMonkey»`.
 - (n) Создайте объекты всех трёх классов и вызовите их методы.
 - (o) Создайте список из объектов разных классов и в цикле вызовите все общие методы, демонстрируя полиморфизм.
- 27 (a) Создайте класс `Vehicle2`, который будет базовым для классов `Truck` и `Motorcycle`. В конструкторе задайте параметры `brand`, `model`, `year`, `max_load`, `fuel_tank`, `engine_power`.
- (b) В классе `Vehicle2` создайте метод `spec`, который выводит спецификацию транспорта.
 - (c) В классе `Vehicle2` создайте метод `vehicle_class`, который выводит класс транспорта.
 - (d) В классе `Vehicle2` создайте метод `license_required`, который выводит тип водительских прав.
 - (e) В классе `Vehicle2` создайте метод `calculate`, который вычисляет пробег на полном баке: `fuel_tank × 10`.
 - (f) Создайте класс `Truck`, наследующийся от `Vehicle2`. В конструкторе задайте `brand`, `model`, `year`.
 - (g) В классе `Truck` полностью переопределите метод `vehicle_class` (например, «Грузовой»).
 - (h) В классе `Truck` полностью переопределите метод `license_required` (например, «Категория C»).
 - (i) В классе `Truck` полностью переопределите метод `calculate`, чтобы он вычислял грузоподъёмность: `max_load = engine_power × 10`.
 - (j) Создайте класс `Motorcycle`, наследующийся от `Vehicle2`. В конструкторе задайте `brand`, `model`, `year`.
 - (k) В классе `Motorcycle` полностью переопределите метод `vehicle_class` (например, «Мотоцикл»).
 - (l) В классе `Motorcycle` полностью переопределите метод `license_required` (например, «Категория A»).

- (m) В классе `Motorcycle` полностью переопределите метод `calculate`, чтобы он вычислял объём бака: `fuel_tank = engine_power/5`.
 - (n) Создайте объекты всех трёх классов и вызовите их методы.
 - (o) Создайте список из объектов разных классов и в цикле вызовите все общие методы, демонстрируя полиморфизм.
- 28
- (a) Создайте класс `Dish`, который будет базовым для классов `Soup` и `Dessert`. В конструкторе задайте параметры `name`, `temperature`, `calories`, `cooking_time`, `main_ingredient`, `cuisine`.
 - (b) В классе `Dish` создайте метод `recipe`, который выводит краткий рецепт.
 - (c) В классе `Dish` создайте метод `meal_time`, который выводит время подачи.
 - (d) В классе `Dish` создайте метод `allergens`, который выводит возможные аллергии.
 - (e) В классе `Dish` создайте метод `calculate`, который вычисляет калорийность в минуту готовки: $\frac{\text{calories}}{\text{cooking_time}}$.
 - (f) Создайте класс `Soup`, наследующийся от `Dish`. В конструкторе задайте `name`, `temperature`, `calories`. Инициализируйте `main_ingredient` и `cuisine` как `None`.
 - (g) В классе `Soup` полностью переопределите метод `meal_time` (например, «Обед»).
 - (h) В классе `Soup` полностью переопределите метод `allergens` (например, «Глютен»).
 - (i) В классе `Soup` полностью переопределите метод `calculate`, чтобы он вычислял основной ингредиент: `main_ingredient = «Овощи»`.
 - (j) Создайте класс `Dessert`, наследующийся от `Dish`. В конструкторе задайте `name`, `temperature`, `calories`. Инициализируйте `main_ingredient` и `cuisine` как `None`.
 - (k) В классе `Dessert` полностью переопределите метод `meal_time` (например, «После обеда»).
 - (l) В классе `Dessert` полностью переопределите метод `allergens` (например, «Молоко, орехи»).
 - (m) В классе `Dessert` полностью переопределите метод `calculate`, чтобы он вычислял кухню: `cuisine = «Французская»`.
 - (n) Создайте объекты всех трёх классов и вызовите их методы.
 - (o) Создайте список из объектов разных классов и в цикле вызовите все общие методы, демонстрируя полиморфизм.
- 29
- (a) Создайте класс `Device`, который будет базовым для классов `Smartwatch` и `Headphones`. В конструкторе задайте параметры `brand`, `model`, `price`, `battery_life`, `weight`, `connectivity`.
 - (b) В классе `Device` создайте метод `specs`, который выводит характеристики устройства.
 - (c) В классе `Device` создайте метод `category`, который выводит категорию устройства.
 - (d) В классе `Device` создайте метод `water_resistant`, который выводит уровень защиты от воды.
 - (e) В классе `Device` создайте метод `calculate`, который вычисляет соотношение цены к времени работы: $\frac{\text{price}}{\text{battery_life}}$.

- (f) Создайте класс **Smartwatch**, наследующийся от **Device**. В конструкторе задайте **brand**, **model**, **price**.
 - (g) В классе **Smartwatch** полностью переопределите метод **category** (например, «Носимая электроника»).
 - (h) В классе **Smartwatch** полностью переопределите метод **water_resistant** (например, «IP68»).
 - (i) В классе **Smartwatch** полностью переопределите метод **calculate**, чтобы он вычислял вес: $\text{weight} = \text{price}/1000$.
 - (j) Создайте класс **Headphones**, наследующийся от **Device**. В конструкторе задайте **brand**, **model**, **price**.
 - (k) В классе **Headphones** полностью переопределите метод **category** (например, «Аудиотехника»).
 - (l) В классе **Headphones** полностью переопределите метод **water_resistant** (например, «Нет»).
 - (m) В классе **Headphones** полностью переопределите метод **calculate**, чтобы он вычислял тип подключения: **connectivity** = «Bluetooth 5.0».
 - (n) Создайте объекты всех трёх классов и вызовите их методы.
 - (o) Создайте список из объектов разных классов и в цикле вызовите все общие методы, демонстрируя полиморфизм.
- 30 (a) Создайте класс **Document**, который будет базовым для классов **Invoice** и **Contract**. В конструкторе задайте параметры **doc_id**, **date**, **author**, **pages**, **status**, **total_amount**.
- (b) В классе **Document** создайте метод **summary**, который выводит краткое содержание.
 - (c) В классе **Document** создайте метод **type**, который выводит тип документа.
 - (d) В классе **Document** создайте метод **validity**, который выводит срок действия.
 - (e) В классе **Document** создайте метод **calculate**, который вычисляет среднюю стоимость страницы: $\frac{\text{total_amount}}{\text{pages}}$.
 - (f) Создайте класс **Invoice**, наследующийся от **Document**. В конструкторе задайте **doc_id**, **date**, **author**.
 - (g) В классе **Invoice** полностью переопределите метод **type** (например, «Счёт»).
 - (h) В классе **Invoice** полностью переопределите метод **validity** (например, «30 дней»).
 - (i) В классе **Invoice** полностью переопределите метод **calculate**, чтобы он вычислял итоговую сумму: $\text{total_amount} = \text{pages} \times 500$.
 - (j) Создайте класс **Contract**, наследующийся от **Document**. В конструкторе задайте **doc_id**, **date**, **author**.
 - (k) В классе **Contract** полностью переопределите метод **type** (например, «Договор»).
 - (l) В классе **Contract** полностью переопределите метод **validity** (например, «1 год»).
 - (m) В классе **Contract** полностью переопределите метод **calculate**, чтобы он вычислял статус: **status** = «Подписан».
 - (n) Создайте объекты всех трёх классов и вызовите их методы.
 - (o) Создайте список из объектов разных классов и в цикле вызовите все общие методы, демонстрируя полиморфизм.

- 31 (a) Создайте класс `Animal2`, который будет базовым для классов `Bird` и `Fish`. В конструкторе задайте параметры `species`, `habitat`, `lifespan`, `diet`, `speed`, `reproduction`.
- (b) В классе `Animal2` создайте метод `bio`, который выводит биологическое описание.
- (c) В классе `Animal2` создайте метод `locomotion`, который выводит способ передвижения.
- (d) В классе `Animal2` создайте метод `conservation_status`, который выводит статус сохранения.
- (e) В классе `Animal2` создайте метод `calculate`, который вычисляет продолжительность жизни в captivity: $\text{lifespan} \times 1.2$.
- (f) Создайте класс `Bird`, наследующийся от `Animal2`. В конструкторе задайте `species`, `habitat`, `diet`.
- (g) В классе `Bird` полностью переопределите метод `locomotion` (например, «Полёт»).
- (h) В классе `Bird` полностью переопределите метод `conservation_status` (например, «Зависит от вида»).
- (i) В классе `Bird` полностью переопределите метод `calculate`, чтобы он вычислял скорость: $\text{speed} = 50 + \text{lifespan}$.
- (j) Создайте класс `Fish`, наследующийся от `Animal2`. В конструкторе задайте `species`, `habitat`, `diet`.
- (k) В классе `Fish` полностью переопределите метод `locomotion` (например, «Плавание»).
- (l) В классе `Fish` полностью переопределите метод `conservation_status` (например, «Уязвимый»).
- (m) В классе `Fish` полностью переопределите метод `calculate`, чтобы он вычислял размножение: `reproduction = «Икрометание»`.
- (n) Создайте объекты всех трёх классов и вызовите их методы.
- (o) Создайте список из объектов разных классов и в цикле вызовите все общие методы, демонстрируя полиморфизм.
- 32 (a) Создайте класс `Subscription`, который будет базовым для классов `Streaming` и `Gym`. В конструкторе задайте параметры `name`, `monthly_fee`, `duration_months`, `features`, `auto_renew`, `discount`.
- (b) В классе `Subscription` создайте метод `details`, который выводит детали подписки.
- (c) В классе `Subscription` создайте метод `category`, который выводит категорию подписки.
- (d) В классе `Subscription` создайте метод `cancellation_policy`, который выводит политику отмены.
- (e) В классе `Subscription` создайте метод `calculate`, который вычисляет общую стоимость: $\text{monthly_fee} \times \text{duration_months} \times (1 - \frac{\text{discount}}{100})$.
- (f) Создайте класс `Streaming`, наследующийся от `Subscription`. В конструкторе задайте `name`, `monthly_fee`, `duration_months`.
- (g) В классе `Streaming` полностью переопределите метод `category` (например, «Развлечения»).

- (h) В классе **Streaming** полностью переопределите метод **cancellation_policy** (например, «В любое время»).
 - (i) В классе **Streaming** полностью переопределите метод **calculate**, чтобы он вычислял функции: `features = «4К, многопользовательский»`.
 - (j) Создайте класс **Gym**, наследующийся от **Subscription**. В конструкторе задайте `name, monthly_fee, duration_months`.
 - (k) В классе **Gym** полностью переопределите метод **category** (например, «Фитнес»).
 - (l) В классе **Gym** полностью переопределите метод **cancellation_policy** (например, «За 14 дней»).
 - (m) В классе **Gym** полностью переопределите метод **calculate**, чтобы он вычислял автопродление: `auto_renew = True`.
 - (n) Создайте объекты всех трёх классов и вызовите их методы.
 - (o) Создайте список из объектов разных классов и в цикле вызовите все общие методы, демонстрируя полиморфизм.
- 33 (a) Создайте класс **Ticket**, который будет базовым для классов **Flight** и **Concert2**. В конструкторе задайте параметры `event_name, price, date, seat, class_type, refundable`.
- (b) В классе **Ticket** создайте метод **info**, который выводит информацию о билете.
 - (c) В классе **Ticket** создайте метод **type**, который выводит тип билета.
 - (d) В классе **Ticket** создайте метод **restrictions**, который выводит ограничения.
 - (e) В классе **Ticket** создайте метод **calculate**, который вычисляет налог: `price × 0.13`.
 - (f) Создайте класс **Flight**, наследующийся от **Ticket**. В конструкторе задайте `event_name, price, date`.
 - (g) В классе **Flight** полностью переопределите метод **type** (например, «Авиабилет»).
 - (h) В классе **Flight** полностью переопределите метод **restrictions** (например, «Паспорт, регистрация»).
 - (i) В классе **Flight** полностью переопределите метод **calculate**, чтобы он вычислял класс: `class_type = «Эконом»`.
 - (j) Создайте класс **Concert2**, наследующийся от **Ticket**. В конструкторе задайте `event_name, price, date`.
 - (k) В классе **Concert2** полностью переопределите метод **type** (например, «Концерт»).
 - (l) В классе **Concert2** полностью переопределите метод **restrictions** (например, «18+»).
 - (m) В классе **Concert2** полностью переопределите метод **calculate**, чтобы он вычислял возврат: `refundable = False`.
 - (n) Создайте объекты всех трёх классов и вызовите их методы.
 - (o) Создайте список из объектов разных классов и в цикле вызовите все общие методы, демонстрируя полиморфизм.
- 34 (a) Создайте класс **Product**, который будет базовым для классов **Book2** и **Gadget**. В конструкторе задайте параметры `title, price, weight, manufacturer, warranty_months, in_stock`.

- (b) В классе **Product** создайте метод **description**, который выводит описание товара.
 - (c) В классе **Product** создайте метод **category**, который выводит категорию товара.
 - (d) В классе **Product** создайте метод **delivery_time**, который выводит срок доставки.
 - (e) В классе **Product** создайте метод **calculate**, который вычисляет стоимость доставки: $\text{weight} \times 50$.
 - (f) Создайте класс **Book2**, наследующийся от **Product**. В конструкторе задайте **title**, **price**, **weight**.
 - (g) В классе **Book2** полностью переопределите метод **category** (например, «Книги»).
 - (h) В классе **Book2** полностью переопределите метод **delivery_time** (например, «1–3 дня»).
 - (i) В классе **Book2** полностью переопределите метод **calculate**, чтобы он вычислял наличие: `in_stock = True`.
 - (j) Создайте класс **Gadget**, наследующийся от **Product**. В конструкторе задайте **title**, **price**, **weight**.
 - (k) В классе **Gadget** полностью переопределите метод **category** (например, «Электроника»).
 - (l) В классе **Gadget** полностью переопределите метод **delivery_time** (например, «3–7 дней»).
 - (m) В классе **Gadget** полностью переопределите метод **calculate**, чтобы он вычислял гарантию: `warranty_months = 12`.
 - (n) Создайте объекты всех трёх классов и вызовите их методы.
 - (o) Создайте список из объектов разных классов и в цикле вызовите все общие методы, демонстрируя полиморфизм.
- 35 (a) Создайте класс **Person**, который будет базовым для классов **Student** и **Teacher**. В конструкторе задайте параметры **name**, **age**, **email**, **schedule**, **workload**, **specialization**.
- (b) В классе **Person** создайте метод **profile**, который выводит профиль человека.
 - (c) В классе **Person** создайте метод **role**, который выводит роль.
 - (d) В классе **Person** создайте метод **availability**, который выводит доступность.
 - (e) В классе **Person** создайте метод **calculate**, который вычисляет нагрузку в часах: $\text{workload} \times 5$.
 - (f) Создайте класс **Student**, наследующийся от **Person**. В конструкторе задайте **name**, **age**, **email**.
 - (g) В классе **Student** полностью переопределите метод **role** (например, «Учащийся»).
 - (h) В классе **Student** полностью переопределите метод **availability** (например, «Пн–Пт, 9–18»).
 - (i) В классе **Student** полностью переопределите метод **calculate**, чтобы он вычислял расписание: `schedule = «Лекции, практики»`.
 - (j) Создайте класс **Teacher**, наследующийся от **Person**. В конструкторе задайте **name**, **age**, **email**.
 - (k) В классе **Teacher** полностью переопределите метод **role** (например, «Преподаватель»).

- (l) В классе **Teacher** полностью переопределите метод **availability** (например, «По расписанию»).
- (m) В классе **Teacher** полностью переопределите метод **calculate**, чтобы он вычислял специализацию: `specialization = «Программирование»`.
- (n) Создайте объекты всех трёх классов и вызовите их методы.
- (o) Создайте список из объектов разных классов и в цикле вызовите все общие методы, демонстрируя полиморфизм.

2.9 Семинар «Множественное наследование» (2 часа)

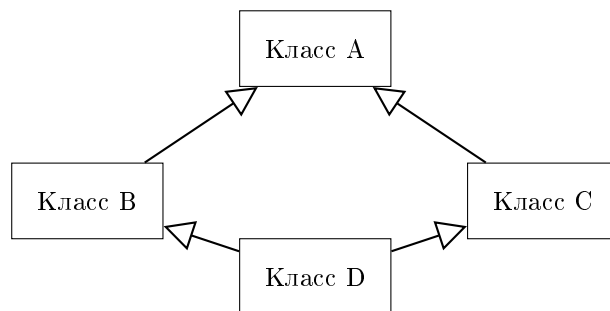


Рис. 36: Проблема ромбовидного наследования

Вопросы для защиты:

- Как решают проблему повторного вызова инициализатора при ромбовидном наследовании авторы языка Python?
- Опишите, что такое ромбовидное наследование
- Сравните способы вызова `__init__` родительских классов
- Что будет, если не вызывать методы `__init__` родительских классов и в каких случаях это полезно?

2.9.1 Задача 1

- 1 Создать классы, которые будут выполнять различные функции, такие как разделение текста на слова, подсчёт количества слов и нахождение уникальных символов, а также выводить результаты на печать методом `print`.
 - (a) Класс `SeparateText` должен разделять текст на слова. Класс `CountWords` — подсчитывать количество слов. Класс `Unique` — находить уникальные символы. Класс `Describer` объединяет функциональность всех трёх классов.
Создайте класс `SeparateText`, который содержит метод `__init__`, принимающий текст в качестве аргумента, разделяющий его на слова и сохраняющий список слов в атрибуте `splitword`.
 - (b) Создайте класс `CountWords`, который наследуется от `SeparateText`. Этот класс должен содержать метод `__init__`, вызывающий метод `__init__` базового класса с помощью `super()`, а затем подсчитывающий количество слов и сохраняющий это значение в атрибуте `word_count`.
 - (c) Создайте класс `Unique`, который также наследуется от `SeparateText`. Этот класс должен содержать метод `__init__`, вызывающий метод `__init__` базового класса с помощью `super()`, а затем находящий уникальные символы и сохраняющий их во множестве `unic`.
 - (d) Создайте класс `Describer`, который наследуется от `CountWords` и `Unique`. Класс `Describer` должен содержать метод `__init__`, корректно вызывающий инициализаторы всех родительских классов через `super()`, и после инициализации выводить информацию о количестве слов и уникальных символах.

- (e) Создайте экземпляр класса **Describer** и передайте ему текст для обработки.
- (f) Выведите результаты работы класса **Describer**, включая список слов (**splitword**), множество уникальных символов (**unic**) и количество слов (**word_count**).

В результате выполнения кода объект класса **Describer** должен содержать атрибуты **splitword**, **unic** и **word_count**, унаследованные от соответствующих базовых классов, которые будут содержать соответственно список слов, множество уникальных символов и количество слов в исходном тексте. Все дочерние классы должны использовать **super().__init__()** для делегирования инициализации родительских классов.

Также необходимо вывести в консоль сообщения о начале и завершении выполнения каждого метода **__init__** при создании одного объекта **Describer**. Пример вывода:

```
Start SeparateText.__init__
End SeparateText.__init__
Start CountWords.__init__
End CountWords.__init__
Start Unique.__init__
End Unique.__init__
Start Describer.__init__
End Describer.__init__
```

- 2 Создать классы, которые будут выполнять различные функции, такие как извлечение чисел из строки, подсчёт их количества и нахождение уникальных чисел, а также выводить результаты на печать методом **print**.
 - (a) Класс **ExtractNumbers** должен извлекать все целые числа из строки. Класс **CountNumbers** — подсчитывать количество извлечённых чисел. Класс **UniqueNumbers** — находить уникальные числа. Класс **Analyzer** объединяет функциональность всех трёх классов.
Создайте класс **ExtractNumbers**, который содержит метод **__init__**, принимающий строку в качестве аргумента, извлекающий из неё все целые числа и сохраняющий список чисел в атрибуте **numbers**.
 - (b) Создайте класс **CountNumbers**, который наследуется от **ExtractNumbers**. Этот класс должен содержать метод **__init__**, вызывающий метод **__init__** базового класса с помощью **super()**, а затем подсчитывающий количество чисел и сохраняющий это значение в атрибуте **num_count**.
 - (c) Создайте класс **UniqueNumbers**, который также наследуется от **ExtractNumbers**. Этот класс должен содержать метод **__init__**, вызывающий метод **__init__** базового класса с помощью **super()**, а затем находящий уникальные числа и сохраняющий их во множестве **unique_nums**.
 - (d) Создайте класс **Analyzer**, который наследуется от **CountNumbers** и **UniqueNumbers**. Класс **Analyzer** должен содержать метод **__init__**, корректно вызывающий инициализаторы всех родительских классов через **super()**, и после инициализации выводить информацию о количестве чисел и уникальных числах.
 - (e) Создайте экземпляр класса **Analyzer** и передайте ему строку для обработки.
 - (f) Выведите результаты работы класса **Analyzer**, включая список чисел (**numbers**), множество уникальных чисел (**unique_nums**) и количество чисел (**num_count**).

В результате выполнения кода объект класса **Analyzer** должен содержать атрибуты **numbers**, **unique_nums** и **num_count**, унаследованные от соответствующих базовых классов, которые будут содержать соответственно список чисел, множество уникальных чисел и количество чисел в исходной строке. Все дочерние классы должны использовать **super().__init__()** для делегирования инициализации родительских классов. Также необходимо вывести в консоль сообщения о начале и завершении выполнения каждого метода **__init__** при создании одного объекта **Analyzer**. Пример вывода:

```
Start ExtractNumbers.__init__
End ExtractNumbers.__init__
Start CountNumbers.__init__
End CountNumbers.__init__
Start UniqueNumbers.__init__
End UniqueNumbers.__init__
Start Analyzer.__init__
End Analyzer.__init__
```

- 3 Создать классы, которые будут выполнять различные функции, такие как извлечение гласных из текста, подсчёт их количества и нахождение уникальных гласных букв, а также выводить результаты на печать методом **print**.
 - (a) Класс **ExtractVowels** должен извлекать все гласные буквы из текста. Класс **CountVowels** — подсчитывать их количество. Класс **UniqueVowels** — находить уникальные гласные. Класс **VowelAnalyzer** объединяет функциональность всех трёх классов. Создайте класс **ExtractVowels**, который содержит метод **__init__**, принимающий текст в качестве аргумента, извлекающий из него все гласные буквы (в нижнем регистре) и сохраняющий список гласных в атрибуте **vowels**.
 - (b) Создайте класс **CountVowels**, который наследуется от **ExtractVowels**. Этот класс должен содержать метод **__init__**, вызывающий метод **__init__** базового класса с помощью **super()**, а затем подсчитывающий количество гласных и сохраняющий это значение в атрибуте **vowel_count**.
 - (c) Создайте класс **UniqueVowels**, который также наследуется от **ExtractVowels**. Этот класс должен содержать метод **__init__**, вызывающий метод **__init__** базового класса с помощью **super()**, а затем находящий уникальные гласные и сохраняющий их во множестве **unique_vowels**.
 - (d) Создайте класс **VowelAnalyzer**, который наследуется от **CountVowels** и **UniqueVowels**. Класс **VowelAnalyzer** должен содержать метод **__init__**, корректно вызывающий инициализаторы всех родительских классов через **super()**, и после инициализации выводить информацию о количестве гласных и уникальных гласных.
 - (e) Создайте экземпляр класса **VowelAnalyzer** и передайте ему текст для обработки.
 - (f) Выведите результаты работы класса **VowelAnalyzer**, включая список гласных (**vowels**), множество уникальных гласных (**unique_vowels**) и количество гласных (**vowel_count**).

В результате выполнения кода объект класса **VowelAnalyzer** должен содержать атрибуты **vowels**, **unique_vowels** и **vowel_count**, унаследованные от соответствующих базовых классов, которые будут содержать соответственно список гласных, множество

уникальных гласных и количество гласных в исходном тексте. Все дочерние классы должны использовать `super().__init__()` для делегирования инициализации родительских классов.

Также необходимо вывести в консоль сообщения о начале и завершении выполнения каждого метода `__init__` при создании одного объекта `VowelAnalyzer`. Пример вывода:

```
Start ExtractVowels.__init__
End ExtractVowels.__init__
Start CountVowels.__init__
End CountVowels.__init__
Start UniqueVowels.__init__
End UniqueVowels.__init__
Start VowelAnalyzer.__init__
End VowelAnalyzer.__init__
```

- 4 Создать классы, которые будут выполнять различные функции, такие как подсчёт частоты каждого символа в тексте, определение наиболее часто встречающегося символа и вывод этих данных, а также выводить результаты на печать методом `print`.
 - (a) Класс `CharFrequency` должен подсчитывать частоту каждого символа в тексте. Класс `MostFrequent` — находить наиболее часто встречающийся символ. Класс `FrequencyCounter` объединяет функциональность обоих классов.
Создайте класс `CharFrequency`, который содержит метод `__init__`, принимающий текст в качестве аргумента и сохраняющий частоты символов в виде словаря в атрибуте `freq_dict`.
 - (b) Создайте класс `MostFrequent`, который наследуется от `CharFrequency`. Этот класс должен содержать метод `__init__`, вызывающий метод `__init__` базового класса с помощью `super()`, а затем находящий наиболее часто встречающийся символ и сохраняющий его в атрибуте `most_common`.
 - (c) Создайте класс `FrequencyCounter`, который наследуется от `CharFrequency` и `MostFrequent` (в порядке, гарантирующем корректную инициализацию). Класс `FrequencyCounter` должен содержать метод `__init__`, корректно вызывающий инициализаторы всех родительских классов через `super()`, и после инициализации выводить информацию о частотах и наиболее частом символе.
 - (d) Создайте экземпляр класса `FrequencyCounter` и передайте ему текст для обработки.
 - (e) Выведите результаты работы класса `FrequencyCounter`, включая словарь частот (`freq_dict`) и наиболее частый символ (`most_common`).
 - (f) Убедитесь, что все родительские инициализаторы вызываются ровно один раз при создании объекта.

В результате выполнения кода объект класса `FrequencyCounter` должен содержать атрибуты `freq_dict` и `most_common`, унаследованные от соответствующих базовых классов. Все дочерние классы должны использовать `super().__init__()` для делегирования инициализации родительских классов.

Также необходимо вывести в консоль сообщения о начале и завершении выполнения каждого метода `__init__` при создании одного объекта `FrequencyCounter`. Пример вывода:

```

Start CharFrequency.__init__
End CharFrequency.__init__
Start MostFrequent.__init__
End MostFrequent.__init__
Start FrequencyCounter.__init__
End FrequencyCounter.__init__

```

5 Создать классы, которые будут выполнять различные функции, такие как извлечение слов длиной более трёх символов, подсчёт их количества и нахождение уникальных длин извлечённых слов, а также выводить результаты на печать методом `print`.

- (a) Класс `FilterLongWords` должен извлекать из текста только слова длиной более трёх символов. Класс `CountLongWords` — подсчитывать их количество. Класс `UniqueLengths` — находить уникальные длины этих слов. Класс `LongWordAnalyzer` объединяет функциональность всех трёх классов.
Создайте класс `FilterLongWords`, который содержит метод `__init__`, принимающий текст в качестве аргумента, фильтрующий слова по длине и сохраняющий список подходящих слов в атрибуте `long_words`.
- (b) Создайте класс `CountLongWords`, который наследуется от `FilterLongWords`. Этот класс должен содержать метод `__init__`, вызывающий метод `__init__` базового класса с помощью `super()`, а затем подсчитывающий количество слов и сохраняющий это значение в атрибуте `long_count`.
- (c) Создайте класс `UniqueLengths`, который также наследуется от `FilterLongWords`. Этот класс должен содержать метод `__init__`, вызывающий метод `__init__` базового класса с помощью `super()`, а затем находящий уникальные длины слов и сохраняющий их во множестве `lengths`.
- (d) Создайте класс `LongWordAnalyzer`, который наследуется от `CountLongWords` и `UniqueLengths`. Класс `LongWordAnalyzer` должен содержать метод `__init__`, корректно вызывающий инициализаторы всех родительских классов через `super()`, и после инициализации выводить информацию о количестве слов и уникальных длинах.
- (e) Создайте экземпляр класса `LongWordAnalyzer` и передайте ему текст для обработки.
- (f) Выведите результаты работы класса `LongWordAnalyzer`, включая список слов (`long_words`), множество уникальных длин (`lengths`) и количество слов (`long_count`).

В результате выполнения кода объект класса `LongWordAnalyzer` должен содержать атрибуты `long_words`, `lengths` и `long_count`, унаследованные от соответствующих базовых классов. Все дочерние классы должны использовать `super().__init__()` для делегирования инициализации родительских классов.

Также необходимо вывести в консоль сообщения о начале и завершении выполнения каждого метода `__init__` при создании одного объекта `LongWordAnalyzer`. Пример вывода:

```

Start FilterLongWords.__init__
End FilterLongWords.__init__
Start CountLongWords.__init__

```

```

End CountLongWords.__init__
Start UniqueLengths.__init__
End UniqueLengths.__init__
Start LongWordAnalyzer.__init__
End LongWordAnalyzer.__init__

```

- 6 Создать классы, которые будут выполнять различные функции, такие как преобразование всех букв текста в верхний регистр, подсчёт количества гласных и определение количества согласных, а также выводить результаты на печать методом `print`.

- (a) Класс `ToUpperCase` должен преобразовывать текст в верхний регистр. Класс `VowelCounter` — подсчитывать количество гласных. Класс `ConsonantCounter` — подсчитывать количество согласных. Класс `CaseAnalyzer` объединяет функциональность всех трёх классов.

Создайте класс `ToUpperCase`, который содержит метод `__init__`, принимающий текст в качестве аргумента, преобразующий его в верхний регистр и сохраняющий результат в атрибуте `upper_text`.

- (b) Создайте класс `VowelCounter`, который наследуется от `ToUpperCase`. Этот класс должен содержать метод `__init__`, вызывающий метод `__init__` базового класса с помощью `super()`, а затем подсчитывающий количество гласных и сохраняющий это значение в атрибуте `vowel_num`.
- (c) Создайте класс `ConsonantCounter`, который также наследуется от `ToUpperCase`. Этот класс должен содержать метод `__init__`, вызывающий метод `__init__` базового класса с помощью `super()`, а затем подсчитывающий количество согласных и сохраняющий это значение в атрибуте `consonant_num`.
- (d) Создайте класс `CaseAnalyzer`, который наследуется от `VowelCounter` и `ConsonantCounter`. Класс `CaseAnalyzer` должен содержать метод `__init__`, корректно вызывающий инициализаторы всех родительских классов через `super()`, и после инициализации выводить информацию о гласных, согласных и преобразованном тексте.
- (e) Создайте экземпляр класса `CaseAnalyzer` и передайте ему текст для обработки.
- (f) Выведите результаты работы класса `CaseAnalyzer`, включая текст в верхнем регистре (`upper_text`), количество гласных (`vowel_num`) и количество согласных (`consonant_num`).

В результате выполнения кода объект класса `CaseAnalyzer` должен содержать атрибуты `upper_text`, `vowel_num` и `consonant_num`, унаследованные от соответствующих базовых классов. Все дочерние классы должны использовать `super().__init__()` для делегирования инициализации родительских классов.

Также необходимо вывести в консоль сообщения о начале и завершении выполнения каждого метода `__init__` при создании одного объекта `CaseAnalyzer`. Пример вывода:

```

Start ToUpperCase.__init__
End ToUpperCase.__init__
Start VowelCounter.__init__
End VowelCounter.__init__
Start ConsonantCounter.__init__
End ConsonantCounter.__init__
Start CaseAnalyzer.__init__
End CaseAnalyzer.__init__

```

7 Создать классы, которые будут выполнять различные функции, такие как удаление всех знаков препинания из текста, подсчёт количества оставшихся слов и определение количества уникальных слов, а также выводить результаты на печать методом `print`.

- (a) Класс `RemovePunctuation` должен удалять все знаки препинания из текста. Класс `WordCounter` — подсчитывать количество слов. Класс `UniqueWords` — находить уникальные слова. Класс `TextCleaner` объединяет функциональность всех трёх классов.

Создайте класс `RemovePunctuation`, который содержит метод `__init__`, принимающий текст в качестве аргумента, удаляющий из него знаки препинания и сохраняющий очищенный текст в атрибуте `clean_text`.

- (b) Создайте класс `WordCounter`, который наследуется от `RemovePunctuation`. Этот класс должен содержать метод `__init__`, вызывающий метод `__init__` базового класса с помощью `super()`, а затем разделяющий текст на слова и подсчитывающий их количество, сохраняя результат в атрибуте `word_count`.

- (c) Создайте класс `UniqueWords`, который также наследуется от `RemovePunctuation`. Этот класс должен содержать метод `__init__`, вызывающий метод `__init__` базового класса с помощью `super()`, а затем находящий уникальные слова и сохраняющий их во множестве `unique_words`.

- (d) Создайте класс `TextCleaner`, который наследуется от `WordCounter` и `UniqueWords`. Класс `TextCleaner` должен содержать метод `__init__`, корректно вызывающий инициализаторы всех родительских классов через `super()`, и после инициализации выводить информацию о количестве слов и уникальных словах.

- (e) Создайте экземпляр класса `TextCleaner` и передайте ему текст для обработки.

- (f) Выведите результаты работы класса `TextCleaner`, включая очищенный текст (`clean_text`), количество слов (`word_count`) и множество уникальных слов (`unique_words`).

В результате выполнения кода объект класса `TextCleaner` должен содержать атрибуты `clean_text`, `word_count` и `unique_words`, унаследованные от соответствующих базовых классов. Все дочерние классы должны использовать `super().__init__()` для делегирования инициализации родительских классов.

Также необходимо вывести в консоль сообщения о начале и завершении выполнения каждого метода `__init__` при создании одного объекта `TextCleaner`. Пример вывода:

```
Start RemovePunctuation.__init__
End RemovePunctuation.__init__
Start WordCounter.__init__
End WordCounter.__init__
Start UniqueWords.__init__
End UniqueWords.__init__
Start TextCleaner.__init__
End TextCleaner.__init__
```

8 Создать классы, которые будут выполнять различные функции, такие как инвертирование порядка слов в тексте, подсчёт количества слов и сохранение исходного текста, а также выводить результаты на печать методом `print`.

- (a) Класс `OriginalText` должен сохранять исходный текст. Класс `InvertWords` — инвертировать порядок слов. Класс `InvertedWordCounter` — подсчитывать количество слов в инвертированном тексте. Класс `Inverter` объединяет функциональность всех трёх классов.
Создайте класс `OriginalText`, который содержит метод `__init__`, принимающий текст в качестве аргумента и сохраняющий его в атрибуте `original`.
- (b) Создайте класс `InvertWords`, который наследуется от `OriginalText`. Этот класс должен содержать метод `__init__`, вызывающий метод `__init__` базового класса с помощью `super()`, а затем инвертирующий порядок слов и сохраняющий результат в атрибуте `inverted`.
- (c) Создайте класс `InvertedWordCounter`, который также наследуется от `OriginalText`. Этот класс должен содержать метод `__init__`, вызывающий метод `__init__` базового класса с помощью `super()`, а затем подсчитывающий количество слов в инвертированном тексте и сохраняющий это значение в атрибуте `word_count`.
- (d) Создайте класс `Inverter`, который наследуется от `InvertedWordCounter`. Класс `Inverter` должен содержать метод `__init__`, корректно вызывающий инициализаторы всех родительских классов через `super()`, и после инициализации выводить информацию о исходном тексте, инвертированном тексте и количестве слов.
- (e) Создайте экземпляр класса `Inverter` и передайте ему текст для обработки.
- (f) Выведите результаты работы класса `Inverter`, включая исходный текст (`original`), инвертированный текст (`inverted`) и количество слов (`word_count`).

В результате выполнения кода объект класса `Inverter` должен содержать атрибуты `original`, `inverted` и `word_count`, унаследованные от соответствующих базовых классов. Все дочерние классы должны использовать `super().__init__()` для делегирования инициализации родительских классов.

Также необходимо вывести в консоль сообщения о начале и завершении выполнения каждого метода `__init__` при создании одного объекта `Inverter`. Пример вывода:

```
Start OriginalText.__init__
End OriginalText.__init__
Start InvertWords.__init__
End InvertWords.__init__
Start InvertedWordCounter.__init__
End InvertedWordCounter.__init__
Start Inverter.__init__
End Inverter.__init__
```

- 9 Создать классы, которые будут выполнять различные функции, такие как определение длины каждого слова в тексте, подсчёт количества слов и нахождение максимальной длины слова, а также выводить результаты на печать методом `print`.
 - (a) Класс `WordLengths` должен определять длину каждого слова в тексте. Класс `TotalWordCounter` — подсчитывать общее количество слов. Класс `MaxLengthFinder` — находить максимальную длину слова. Класс `LengthAnalyzer` объединяет функциональность всех трёх классов.
Создайте класс `WordLengths`, который содержит метод `__init__`, принимающий текст в качестве аргумента, разделяющий его на слова и сохраняющий список длин слов в атрибуте `lengths`.

- (b) Создайте класс `TotalWordCounter`, который наследуется от `WordLengths`. Этот класс должен содержать метод `__init__`, вызывающий метод `__init__` базового класса с помощью `super()`, а затем подсчитывающий количество слов и сохраняющий это значение в атрибуте `word_count`.
- (c) Создайте класс `MaxLengthFinder`, который также наследуется от `WordLengths`. Этот класс должен содержать метод `__init__`, вызывающий метод `__init__` базового класса с помощью `super()`, а затем находящий максимальную длину и сохраняющий её в атрибуте `max_len`.
- (d) Создайте класс `LengthAnalyzer`, который наследуется от `TotalWordCounter` и `MaxLengthFinder`. Класс `LengthAnalyzer` должен содержать метод `__init__`, корректно вызывающий инициализаторы всех родительских классов через `super()`, и после инициализации выводить информацию о количестве слов, длинах и максимальной длине.
- (e) Создайте экземпляр класса `LengthAnalyzer` и передайте ему текст для обработки.
- (f) Выведите результаты работы класса `LengthAnalyzer`, включая список длин (`lengths`), количество слов (`word_count`) и максимальную длину (`max_len`).

В результате выполнения кода объект класса `LengthAnalyzer` должен содержать атрибуты `lengths`, `word_count` и `max_len`, унаследованные от соответствующих базовых классов. Все дочерние классы должны использовать `super().__init__()` для делегирования инициализации родительских классов.

Также необходимо вывести в консоль сообщения о начале и завершении выполнения каждого метода `__init__` при создании одного объекта `LengthAnalyzer`. Пример вывода:

```
Start WordLengths.__init__
End WordLengths.__init__
Start TotalWordCounter.__init__
End TotalWordCounter.__init__
Start MaxLengthFinder.__init__
End MaxLengthFinder.__init__
Start LengthAnalyzer.__init__
End LengthAnalyzer.__init__
```

- 10 Создать классы, которые будут выполнять различные функции, такие как определение, является ли каждое слово палиндромом, подсчёт количества палиндромов и сохранение исходного текста, а также выводить результаты на печать методом `print`.

- (a) Класс `StoreText` должен сохранять исходный текст. Класс `PalindromeChecker` — определять, какие слова являются палиндромами. Класс `PalindromeCounter` — подсчитывать их количество. Класс `PalindromeAnalyzer` объединяет функциональность всех трёх классов.

Создайте класс `StoreText`, который содержит метод `__init__`, принимающий текст в качестве аргумента и сохраняющий его в атрибуте `text`.

- (b) Создайте класс `PalindromeChecker`, который наследуется от `StoreText`. Этот класс должен содержать метод `__init__`, вызывающий метод `__init__` базового класса с помощью `super()`, а затем определяющий список палиндромов и сохраняющий его в атрибуте `palindromes`.

- (c) Создайте класс `PalindromeCounter`, который также наследуется от `StoreText`. Этот класс должен содержать метод `__init__`, вызывающий метод `__init__` базового класса с помощью `super()`, а затем подсчитывающий количество палиндромов и сохраняющий это значение в атрибуте `count`.
- (d) Создайте класс `PalindromeAnalyzer`, который наследуется от `PalindromeCounter`. Класс `PalindromeAnalyzer` должен содержать метод `__init__`, корректно вызывающий инициализаторы всех родительских классов через `super()`, и после инициализации выводить информацию о палиндромах и их количестве.
- (e) Создайте экземпляр класса `PalindromeAnalyzer` и передайте ему текст для обработки.
- (f) Выведите результаты работы класса `PalindromeAnalyzer`, включая исходный текст (`text`), список палиндромов (`palindromes`) и их количество (`count`).

В результате выполнения кода объект класса `PalindromeAnalyzer` должен содержать атрибуты `text`, `palindromes` и `count`, унаследованные от соответствующих базовых классов. Все дочерние классы должны использовать `super().__init__()` для делегирования инициализации родительских классов.

Также необходимо вывести в консоль сообщения о начале и завершении выполнения каждого метода `__init__` при создании одного объекта `PalindromeAnalyzer`. Пример вывода:

```
Start StoreText.__init__
End StoreText.__init__
Start PalindromeChecker.__init__
End PalindromeChecker.__init__
Start PalindromeCounter.__init__
End PalindromeCounter.__init__
Start PalindromeAnalyzer.__init__
End PalindromeAnalyzer.__init__
```

- 11 Создать классы, которые будут выполнять различные функции, такие как преобразование текста в список символов, подсчёт количества символов и нахождение уникальных символов, а также выводить результаты на печать методом `print`.

- (a) Класс `CharLister` должен преобразовывать текст в список символов. Класс `CharCounter` — подсчитывать количество символов. Класс `UniqueCharFinder` — находить уникальные символы. Класс `CharacterAnalyzer` объединяет функциональность всех трёх классов.

Создайте класс `CharLister`, который содержит метод `__init__`, принимающий текст в качестве аргумента и сохраняющий список всех символов (включая пробелы и знаки препинания) в атрибуте `char_list`.

- (b) Создайте класс `CharCounter`, который наследуется от `CharLister`. Этот класс должен содержать метод `__init__`, вызывающий метод `__init__` базового класса с помощью `super()`, а затем подсчитывающий общее количество символов и сохраняющий это значение в атрибуте `total_chars`.
- (c) Создайте класс `UniqueCharFinder`, который также наследуется от `CharLister`. Этот класс должен содержать метод `__init__`, вызывающий метод `__init__` базового класса с помощью `super()`, а затем находящий уникальные символы и сохраняющий их во множестве `unique_chars`.

- (d) Создайте класс `CharacterAnalyzer`, который наследуется от `CharCounter` и `UniqueCharFinder`. Класс `CharacterAnalyzer` должен содержать метод `__init__`, корректно вызывающий инициализаторы всех родительских классов через `super()`, и после инициализации выводить информацию о символах.
- (e) Создайте экземпляр класса `CharacterAnalyzer` и передайте ему текст для обработки.
- (f) Выведите результаты работы класса `CharacterAnalyzer`, включая список символов (`char_list`), количество символов (`total_chars`) и множество уникальных символов (`unique_chars`).

В результате выполнения кода объект класса `CharacterAnalyzer` должен содержать атрибуты `char_list`, `total_chars` и `unique_chars`, унаследованные от соответствующих базовых классов. Все дочерние классы должны использовать `super().__init__()` для делегирования инициализации родительских классов.

Также необходимо вывести в консоль сообщения о начале и завершении выполнения каждого метода `__init__` при создании одного объекта `CharacterAnalyzer`. Пример вывода:

```
Start CharLister.__init__
End CharLister.__init__
Start CharCounter.__init__
End CharCounter.__init__
Start UniqueCharFinder.__init__
End UniqueCharFinder.__init__
Start CharacterAnalyzer.__init__
End CharacterAnalyzer.__init__
```

- 12 Создать классы, которые будут выполнять различные функции, такие как извлечение всех заглавных букв из текста, подсчёт их количества и нахождение уникальных заглавных букв, а также выводить результаты на печать методом `print`.

- (a) Класс `ExtractUppercase` должен извлекать все заглавные буквы из текста. Класс `UppercaseCounter` — подсчитывать их количество. Класс `UniqueUppercaseFinder` — находить уникальные заглавные буквы. Класс `UppercaseAnalyzer` объединяет функциональность всех трёх классов.

Создайте класс `ExtractUppercase`, который содержит метод `__init__`, принимающий текст в качестве аргумента, извлекающий из него все заглавные буквы и сохраняющий список в атрибуте `upper_letters`.

- (b) Создайте класс `UppercaseCounter`, который наследуется от `ExtractUppercase`. Этот класс должен содержать метод `__init__`, вызывающий метод `__init__` базового класса с помощью `super()`, а затем подсчитывающий количество заглавных букв и сохраняющий это значение в атрибуте `upper_count`.
- (c) Создайте класс `UniqueUppercaseFinder`, который также наследуется от `ExtractUppercase`. Этот класс должен содержать метод `__init__`, вызывающий метод `__init__` базового класса с помощью `super()`, а затем находящий уникальные заглавные буквы и сохраняющий их во множестве `unique_uppers`.

- (d) Создайте класс `UppercaseAnalyzer`, который наследуется от `UppercaseCounter` и `UniqueUppercaseFinder`. Класс `UppercaseAnalyzer` должен содержать метод `__init__`, корректно вызывающий инициализаторы всех родительских классов через `super()`, и после инициализации выводить информацию о заглавных буквах.
- (e) Создайте экземпляр класса `UppercaseAnalyzer` и передайте ему текст для обработки.
- (f) Выведите результаты работы класса `UppercaseAnalyzer`, включая список заглавных букв (`upper_letters`), количество (`upper_count`) и множество уникальных заглавных букв (`unique_uppers`).

В результате выполнения кода объект класса `UppercaseAnalyzer` должен содержать атрибуты `upper_letters`, `upper_count` и `unique_uppers`, унаследованные от соответствующих базовых классов. Все дочерние классы должны использовать `super().__init__()` для делегирования инициализации родительских классов.

Также необходимо вывести в консоль сообщения о начале и завершении выполнения каждого метода `__init__` при создании одного объекта `UppercaseAnalyzer`. Пример вывода:

```
Start ExtractUppercase.__init__
End ExtractUppercase.__init__
Start UppercaseCounter.__init__
End UppercaseCounter.__init__
Start UniqueUppercaseFinder.__init__
End UniqueUppercaseFinder.__init__
Start UppercaseAnalyzer.__init__
End UppercaseAnalyzer.__init__
```

- 13 Создать классы, которые будут выполнять различные функции, такие как замена всех цифр в тексте на символ «*», подсчёт количества заменённых цифр и сохранение преобразованного текста, а также выводить результаты на печать методом `print`.

- (a) Класс `OriginalString` должен сохранять исходную строку. Класс `DigitReplacer` — заменять все цифры на «*». Класс `DigitCounter` — подсчитывать количество заменённых цифр. Класс `DigitMasker` объединяет функциональность всех трёх классов.

Создайте класс `OriginalString`, который содержит метод `__init__`, принимающий строку в качестве аргумента и сохраняющий её в атрибуте `original`.

- (b) Создайте класс `DigitReplacer`, который наследуется от `OriginalString`. Этот класс должен содержать метод `__init__`, вызывающий метод `__init__` базового класса с помощью `super()`, а затем заменяющий все цифры на «*» и сохраняющий результат в атрибуте `masked`.
- (c) Создайте класс `DigitCounter`, который также наследуется от `OriginalString`. Этот класс должен содержать метод `__init__`, вызывающий метод `__init__` базового класса с помощью `super()`, а затем подсчитывающий количество цифр в исходной строке и сохраняющий это значение в атрибуте `digit_count`.

- (d) Создайте класс `DigitMasker`, который наследуется от `DigitCounter`. Класс `DigitMasker` должен содержать метод `__init__`, корректно вызывающий инициализаторы всех родительских классов через `super()`, и после инициализации выводить информацию о замаскированной строке и количестве цифр.
- (e) Создайте экземпляр класса `DigitMasker` и передайте ему строку для обработки.
- (f) Выведите результаты работы класса `DigitMasker`, включая исходную строку (`original`), замаскированную строку (`masked`) и количество цифр (`digit_count`).

В результате выполнения кода объект класса `DigitMasker` должен содержать атрибуты `original`, `masked` и `digit_count`, унаследованные от соответствующих базовых классов. Все дочерние классы должны использовать `super().__init__()` для делегирования инициализации родительских классов.

Также необходимо вывести в консоль сообщения о начале и завершении выполнения каждого метода `__init__` при создании одного объекта `DigitMasker`. Пример вывода:

```
Start OriginalString.__init__
End OriginalString.__init__
Start DigitReplacer.__init__
End DigitReplacer.__init__
Start DigitCounter.__init__
End DigitCounter.__init__
Start DigitMasker.__init__
End DigitMasker.__init__
```

- 14 Создать классы, которые будут выполнять различные функции, такие как определение количества предложений в тексте, подсчёт количества слов и сохранение исходного текста, а также выводить результаты на печать методом `print`.

- (a) Класс `TextSaver` должен сохранять исходный текст. Класс `SentenceCounter` — подсчитывать количество предложений. Класс `WordCounter` — подсчитывать количество слов. Класс `TextStats` объединяет функциональность всех трёх классов. Создайте класс `TextSaver`, который содержит метод `__init__`, принимающий текст в качестве аргумента и сохраняющий его в атрибуте `text`.
- (b) Создайте класс `SentenceCounter`, который наследуется от `TextSaver`. Этот класс должен содержать метод `__init__`, вызывающий метод `__init__` базового класса с помощью `super()`, а затем подсчитывающий количество предложений (разделённых '.', '!', '?') и сохраняющий это значение в атрибуте `sentences`.
- (c) Создайте класс `WordCounter`, который также наследуется от `TextSaver`. Этот класс должен содержать метод `__init__`, вызывающий метод `__init__` базового класса с помощью `super()`, а затем подсчитывающий количество слов и сохраняющий это значение в атрибуте `words`.
- (d) Создайте класс `TextStats`, который наследуется от `SentenceCounter` и `WordCounter`. Класс `TextStats` должен содержать метод `__init__`, корректно вызывающий инициализаторы всех родительских классов через `super()`, и после инициализации выводить информацию о тексте.
- (e) Создайте экземпляр класса `TextStats` и передайте ему текст для обработки.

- (f) Выведите результаты работы класса `TextStats`, включая исходный текст (`text`), количество предложений (`sentences`) и количество слов (`words`).

В результате выполнения кода объект класса `TextStats` должен содержать атрибуты `text`, `sentences` и `words`, унаследованные от соответствующих базовых классов. Все дочерние классы должны использовать `super().__init__()` для делегирования инициализации родительских классов.

Также необходимо вывести в консоль сообщения о начале и завершении выполнения каждого метода `__init__` при создании одного объекта `TextStats`. Пример вывода:

```
Start TextSaver.__init__
End TextSaver.__init__
Start SentenceCounter.__init__
End SentenceCounter.__init__
Start WordCounter.__init__
End WordCounter.__init__
Start TextStats.__init__
End TextStats.__init__
```

- 15 Создать классы, которые будут выполнять различные функции, такие как удаление всех цифр из текста, подсчёт количества удалённых цифр и сохранение очищенного текста, а также выводить результаты на печать методом `print`.

- (a) Класс `RawText` должен сохранять исходный текст. Класс `DigitRemover` — удалять все цифры. Класс `RemovedDigitCounter` — подсчитывать количество удалённых цифр. Класс `CleanTextProcessor` объединяет функциональность всех трёх классов.

Создайте класс `RawText`, который содержит метод `__init__`, принимающий текст в качестве аргумента и сохраняющий его в атрибуте `raw`.

- (b) Создайте класс `DigitRemover`, который наследуется от `RawText`. Этот класс должен содержать метод `__init__`, вызывающий метод `__init__` базового класса с помощью `super()`, а затем удаляющий все цифры и сохраняющий очищенный текст в атрибуте `clean`.

- (c) Создайте класс `RemovedDigitCounter`, который также наследуется от `RawText`. Этот класс должен содержать метод `__init__`, вызывающий метод `__init__` базового класса с помощью `super()`, а затем подсчитывающий количество удалённых цифр и сохраняющий это значение в атрибуте `removed_count`.

- (d) Создайте класс `CleanTextProcessor`, который наследуется от `RemovedDigitCounter`. Класс `CleanTextProcessor` должен содержать метод `__init__`, корректно вызывающий инициализаторы всех родительских классов через `super()`, и после инициализации выводить информацию об очистке текста.

- (e) Создайте экземпляр класса `CleanTextProcessor` и передайте ему текст для обработки.

- (f) Выведите результаты работы класса `CleanTextProcessor`, включая исходный текст (`raw`), очищенный текст (`clean`) и количество удалённых цифр (`removed_count`).

В результате выполнения кода объект класса `CleanTextProcessor` должен содержать атрибуты `raw`, `clean` и `removed_count`, унаследованные от соответствующих базовых

классов. Все дочерние классы должны использовать `super().__init__()` для делегирования инициализации родительских классов.

Также необходимо вывести в консоль сообщения о начале и завершении выполнения каждого метода `__init__` при создании одного объекта `CleanTextProcessor`. Пример вывода:

```
Start RawText.__init__
End RawText.__init__
Start DigitRemover.__init__
End DigitRemover.__init__
Start RemovedDigitCounter.__init__
End RemovedDigitCounter.__init__
Start CleanTextProcessor.__init__
End CleanTextProcessor.__init__
```

- 16 Создать классы, которые будут выполнять различные функции, такие как определение количества пробелов в тексте, подсчёт количества непробельных символов и сохранение исходного текста, а также выводить результаты на печать методом `print`.

- (a) Класс `InputText` должен сохранять исходный текст. Класс `SpaceCounter` — подсчитывать количество пробелов. Класс `NonSpaceCounter` — подсчитывать количество непробельных символов. Класс `WhitespaceAnalyzer` объединяет функциональность всех трёх классов.

Создайте класс `InputText`, который содержит метод `__init__`, принимающий текст в качестве аргумента и сохраняющий его в атрибуте `input_str`.

- (b) Создайте класс `SpaceCounter`, который наследуется от `InputText`. Этот класс должен содержать метод `__init__`, вызывающий метод `__init__` базового класса с помощью `super()`, а затем подсчитывающий количество пробелов и сохраняющий это значение в атрибуте `spaces`.

- (c) Создайте класс `NonSpaceCounter`, который также наследуется от `InputText`. Этот класс должен содержать метод `__init__`, вызывающий метод `__init__` базового класса с помощью `super()`, а затем подсчитывающий количество непробельных символов и сохраняющий это значение в атрибуте `non_spaces`.

- (d) Создайте класс `WhitespaceAnalyzer`, который наследуется от `SpaceCounter` и `NonSpaceCounter`. Класс `WhitespaceAnalyzer` должен содержать метод `__init__`, корректно вызывающий инициализаторы всех родительских классов через `super()`, и после инициализации выводить информацию о пробелах и непробельных символах.

- (e) Создайте экземпляр класса `WhitespaceAnalyzer` и передайте ему текст для обработки.

- (f) Выведите результаты работы класса `WhitespaceAnalyzer`, включая исходный текст (`input_str`), количество пробелов (`spaces`) и количество непробельных символов (`non_spaces`).

В результате выполнения кода объект класса `WhitespaceAnalyzer` должен содержать атрибуты `input_str`, `spaces` и `non_spaces`, унаследованные от соответствующих базовых классов. Все дочерние классы должны использовать `super().__init__()` для делегирования инициализации родительских классов.

Также необходимо вывести в консоль сообщения о начале и завершении выполнения каждого метода `__init__` при создании одного объекта `WhitespaceAnalyzer`. Пример вывода:

```
Start InputText.__init__
End InputText.__init__
Start SpaceCounter.__init__
End SpaceCounter.__init__
Start NonSpaceCounter.__init__
End NonSpaceCounter.__init__
Start WhitespaceAnalyzer.__init__
End WhitespaceAnalyzer.__init__
```

- 17 Создать классы, которые будут выполнять различные функции, такие как извлечение всех цифр из строки, подсчёт их суммы и нахождение максимальной цифры, а также выводить результаты на печать методом `print`.

- (a) Класс `DigitExtractor` должен извлекать все цифры из строки. Класс `DigitSumCalculator` — подсчитывать их сумму. Класс `MaxDigitFinder` — находить максимальную цифру. Класс `DigitStats` объединяет функциональность всех трёх классов.
Создайте класс `DigitExtractor`, который содержит метод `__init__`, принимающий строку в качестве аргумента, извлекающий из неё все цифры и сохраняющий список цифр (как целые числа) в атрибуте `digits`.
- (b) Создайте класс `DigitSumCalculator`, который наследуется от `DigitExtractor`. Этот класс должен содержать метод `__init__`, вызывающий метод `__init__` базового класса с помощью `super()`, а затем вычисляющий сумму цифр и сохраняющий это значение в атрибуте `digit_sum`.
- (c) Создайте класс `MaxDigitFinder`, который также наследуется от `DigitExtractor`. Этот класс должен содержать метод `__init__`, вызывающий метод `__init__` базового класса с помощью `super()`, а затем находящий максимальную цифру и сохраняющий её в атрибуте `max_digit`.
- (d) Создайте класс `DigitStats`, который наследуется от `DigitSumCalculator` и `MaxDigitFinder`. Класс `DigitStats` должен содержать метод `__init__`, корректно вызывающий инициализаторы всех родительских классов через `super()`, и после инициализации выводить информацию о цифрах.
- (e) Создайте экземпляр класса `DigitStats` и передайте ему строку для обработки.
- (f) Выведите результаты работы класса `DigitStats`, включая список цифр (`digits`), сумму (`digit_sum`) и максимальную цифру (`max_digit`).

В результате выполнения кода объект класса `DigitStats` должен содержать атрибуты `digits`, `digit_sum` и `max_digit`, унаследованные от соответствующих базовых классов. Все дочерние классы должны использовать `super().__init__()` для делегирования инициализации родительских классов.

Также необходимо вывести в консоль сообщения о начале и завершении выполнения каждого метода `__init__` при создании одного объекта `DigitStats`. Пример вывода:

```
Start DigitExtractor.__init__
```

```

End DigitExtractor.__init__
Start DigitSumCalculator.__init__
End DigitSumCalculator.__init__
Start MaxDigitFinder.__init__
End MaxDigitFinder.__init__
Start DigitStats.__init__
End DigitStats.__init__

```

18 Создать классы, которые будут выполнять различные функции, такие как определение количества слов, начинающихся с заглавной буквы, подсчёт общего количества слов и сохранение исходного текста, а также выводить результаты на печать методом `print`.

- (a) Класс `SourceText` должен сохранять исходный текст. Класс `CapitalizedWordCounter` — подсчитывать количество слов с заглавной буквы. Класс `TotalWordCounter` — подсчитывать общее количество слов. Класс `CapitalizationAnalyzer` объединяет функциональность всех трёх классов.
Создайте класс `SourceText`, который содержит метод `__init__`, принимающий текст в качестве аргумента и сохраняющий его в атрибуте `source`.
- (b) Создайте класс `CapitalizedWordCounter`, который наследуется от `SourceText`. Этот класс должен содержать метод `__init__`, вызывающий метод `__init__` базового класса с помощью `super()`, а затем подсчитывающий количество слов, начинающихся с заглавной буквы, и сохраняющий это значение в атрибуте `capitalized`.
- (c) Создайте класс `TotalWordCounter`, который также наследуется от `SourceText`. Этот класс должен содержать метод `__init__`, вызывающий метод `__init__` базового класса с помощью `super()`, а затем подсчитывающий общее количество слов и сохраняющий это значение в атрибуте `total`.
- (d) Создайте класс `CapitalizationAnalyzer`, который наследуется от `CapitalizedWordCounter` и `TotalWordCounter`. Класс `CapitalizationAnalyzer` должен содержать метод `__init__`, корректно вызывающий инициализаторы всех родительских классов через `super()`, и после инициализации выводить информацию о словах.
- (e) Создайте экземпляр класса `CapitalizationAnalyzer` и передайте ему текст для обработки.
- (f) Выведите результаты работы класса `CapitalizationAnalyzer`, включая исходный текст (`source`), количество слов с заглавной буквы (`capitalized`) и общее количество слов (`total`).

В результате выполнения кода объект класса `CapitalizationAnalyzer` должен содержать атрибуты `source`, `capitalized` и `total`, унаследованные от соответствующих базовых классов. Все дочерние классы должны использовать `super().__init__()` для делегирования инициализации родительских классов.

Также необходимо вывести в консоль сообщения о начале и завершении выполнения каждого метода `__init__` при создании одного объекта `CapitalizationAnalyzer`. Пример вывода:

```

Start SourceText.__init__
End SourceText.__init__

```

```

Start CapitalizedWordCounter.__init__
End CapitalizedWordCounter.__init__
Start TotalWordCounter.__init__
End TotalWordCounter.__init__
Start CapitalizationAnalyzer.__init__
End CapitalizationAnalyzer.__init__

```

- 19 Создать классы, которые будут выполнять различные функции, такие как удаление всех гласных из текста, подсчёт количества удалённых гласных и сохранение результата, а также выводить результаты на печать методом `print`.

- (a) Класс `InitialText` должен сохранять исходный текст. Класс `VowelRemover` — удалять все гласные. Класс `VowelDeletionCounter` — подсчитывать количество удалённых гласных. Класс `ConsonantOnlyText` объединяет функциональность всех трёх классов.
Создайте класс `InitialText`, который содержит метод `__init__`, принимающий текст в качестве аргумента и сохраняющий его в атрибуте `initial`.
- (b) Создайте класс `VowelRemover`, который наследуется от `InitialText`. Этот класс должен содержать метод `__init__`, вызывающий метод `__init__` базового класса с помощью `super()`, а затем удаляющий все гласные (в любом регистре) и сохраняющий результат в атрибуте `consonants_only`.
- (c) Создайте класс `VowelDeletionCounter`, который также наследуется от `InitialText`. Этот класс должен содержать метод `__init__`, вызывающий метод `__init__` базового класса с помощью `super()`, а затем подсчитывающий количество удалённых гласных и сохраняющий это значение в атрибуте `deleted_vowels`.
- (d) Создайте класс `ConsonantOnlyText`, который наследуется от `VowelDeletionCounter`. Класс `ConsonantOnlyText` должен содержать метод `__init__`, корректно вызывающий инициализаторы всех родительских классов через `super()`, и после инициализации выводить информацию об удалении гласных.
- (e) Создайте экземпляр класса `ConsonantOnlyText` и передайте ему текст для обработки.
- (f) Выведите результаты работы класса `ConsonantOnlyText`, включая исходный текст (`initial`), текст без гласных (`consonants_only`) и количество удалённых гласных (`deleted_vowels`).

В результате выполнения кода объект класса `ConsonantOnlyText` должен содержать атрибуты `initial`, `consonants_only` и `deleted_vowels`, унаследованные от соответствующих базовых классов. Все дочерние классы должны использовать `super().__init__()` для делегирования инициализации родительских классов.

Также необходимо вывести в консоль сообщения о начале и завершении выполнения каждого метода `__init__` при создании одного объекта `ConsonantOnlyText`. Пример вывода:

```

Start InitialText.__init__
End InitialText.__init__
Start VowelRemover.__init__
End VowelRemover.__init__

```

```

Start VowelDeletionCounter.__init__
End VowelDeletionCounter.__init__
Start ConsonantOnlyText.__init__
End ConsonantOnlyText.__init__

```

- 20 Создать классы, которые будут выполнять различные функции, такие как определение длины текста, подсчёт количества слов и сохранение исходного текста, а также выводить результаты на печать методом `print`.

- (a) Класс `BaseText` должен сохранять исходный текст. Класс `TextLengthCalculator` — определять длину текста. Класс `WordCountCalculator` — подсчитывать количество слов. Класс `TextMetrics` объединяет функциональность всех трёх классов. Создайте класс `BaseText`, который содержит метод `__init__`, принимающий текст в качестве аргумента и сохраняющий его в атрибуте `base`.
- (b) Создайте класс `TextLengthCalculator`, который наследуется от `BaseText`. Этот класс должен содержать метод `__init__`, вызывающий метод `__init__` базового класса с помощью `super()`, а затем определяющий длину текста и сохраняющий это значение в атрибуте `text_len`.
- (c) Создайте класс `WordCountCalculator`, который также наследуется от `BaseText`. Этот класс должен содержать метод `__init__`, вызывающий метод `__init__` базового класса с помощью `super()`, а затем подсчитывающий количество слов и сохраняющий это значение в атрибуте `word_num`.
- (d) Создайте класс `TextMetrics`, который наследуется от `TextLengthCalculator` и `WordCountCalculator`. Класс `TextMetrics` должен содержать метод `__init__`, корректно вызывающий инициализаторы всех родительских классов через `super()`, и после инициализации выводить информацию о тексте.
- (e) Создайте экземпляр класса `TextMetrics` и передайте ему текст для обработки.
- (f) Выведите результаты работы класса `TextMetrics`, включая исходный текст (`base`), длину текста (`text_len`) и количество слов (`word_num`).

В результате выполнения кода объект класса `TextMetrics` должен содержать атрибуты `base`, `text_len` и `word_num`, унаследованные от соответствующих базовых классов. Все дочерние классы должны использовать `super().__init__()` для делегирования инициализации родительских классов.

Также необходимо вывести в консоль сообщения о начале и завершении выполнения каждого метода `__init__` при создании одного объекта `TextMetrics`. Пример вывода:

```

Start BaseText.__init__
End BaseText.__init__
Start TextLengthCalculator.__init__
End TextLengthCalculator.__init__
Start WordCountCalculator.__init__
End WordCountCalculator.__init__
Start TextMetrics.__init__
End TextMetrics.__init__

```

- 21 Создать классы, которые будут выполнять различные функции, такие как определение количества уникальных слов в тексте, подсчёт общего количества слов и сохранение исходного текста, а также выводить результаты на печать методом `print`.

- (a) Класс `InputString` должен сохранять исходный текст. Класс `UniqueWordCounter` — подсчитывать количество уникальных слов. Класс `TotalWordCounter` — подсчитывать общее количество слов. Класс `VocabularyAnalyzer` объединяет функциональность всех трёх классов.
Создайте класс `InputString`, который содержит метод `__init__`, принимающий текст в качестве аргумента и сохраняющий его в атрибуте `input_str`.
- (b) Создайте класс `UniqueWordCounter`, который наследуется от `InputString`. Этот класс должен содержать метод `__init__`, вызывающий метод `__init__` базового класса с помощью `super()`, а затем определяющий количество уникальных слов и сохраняющий это значение в атрибуте `unique_count`.
- (c) Создайте класс `TotalWordCounter`, который также наследуется от `InputString`. Этот класс должен содержать метод `__init__`, вызывающий метод `__init__` базового класса с помощью `super()`, а затем подсчитывающий общее количество слов и сохраняющий это значение в атрибуте `total_count`.
- (d) Создайте класс `VocabularyAnalyzer`, который наследуется от `UniqueWordCounter` и `TotalWordCounter`. Класс `VocabularyAnalyzer` должен содержать метод `__init__`, корректно вызывающий инициализаторы всех родительских классов через `super()`, и после инициализации выводить информацию о словарном запасе текста.
- (e) Создайте экземпляр класса `VocabularyAnalyzer` и передайте ему текст для обработки.
- (f) Выведите результаты работы класса `VocabularyAnalyzer`, включая исходный текст (`input_str`), количество уникальных слов (`unique_count`) и общее количество слов (`total_count`).

В результате выполнения кода объект класса `VocabularyAnalyzer` должен содержать атрибуты `input_str`, `unique_count` и `total_count`, унаследованные от соответствующих базовых классов. Все дочерние классы должны использовать `super().__init__()` для делегирования инициализации родительских классов.

Также необходимо вывести в консоль сообщения о начале и завершении выполнения каждого метода `__init__` при создании одного объекта `VocabularyAnalyzer`. Пример вывода:

```
Start InputString.__init__
End InputString.__init__
Start UniqueWordCounter.__init__
End UniqueWordCounter.__init__
Start TotalWordCounter.__init__
End TotalWordCounter.__init__
Start VocabularyAnalyzer.__init__
End VocabularyAnalyzer.__init__
```

- 22 Создать классы, которые будут выполнять различные функции, такие как определение количества слов длиной ровно 5 символов, подсчёт общего количества слов и сохранение исходного текста, а также выводить результаты на печать методом `print`.

- (a) Класс `OriginalInput` должен сохранять исходный текст. Класс `FiveLetterWordCounter` — подсчитывать количество слов длиной 5. Класс `WordTotaler` — подсчитывать

общее количество слов. Класс `FiveLetterAnalyzer` объединяет функциональность всех трёх классов.

Создайте класс `OriginalInput`, который содержит метод `__init__`, принимающий текст в качестве аргумента и сохраняющий его в атрибуте `original`.

- (b) Создайте класс `FiveLetterWordCounter`, который наследуется от `OriginalInput`. Этот класс должен содержать метод `__init__`, вызывающий метод `__init__` базового класса с помощью `super()`, а затем подсчитывающий количество слов длиной ровно 5 символов и сохраняющий это значение в атрибуте `five_count`.
- (c) Создайте класс `WordTotaler`, который также наследуется от `OriginalInput`. Этот класс должен содержать метод `__init__`, вызывающий метод `__init__` базового класса с помощью `super()`, а затем подсчитывающий общее количество слов и сохраняющий это значение в атрибуте `total_words`.
- (d) Создайте класс `FiveLetterAnalyzer`, который наследуется от `FiveLetterWordCounter` и `WordTotaler`. Класс `FiveLetterAnalyzer` должен содержать метод `__init__`, корректно вызывающий инициализаторы всех родительских классов через `super()`, и после инициализации выводить информацию о пятибуквенных словах.
- (e) Создайте экземпляр класса `FiveLetterAnalyzer` и передайте ему текст для обработки.
- (f) Выведите результаты работы класса `FiveLetterAnalyzer`, включая исходный текст (`original`), количество пятибуквенных слов (`five_count`) и общее количество слов (`total_words`).

В результате выполнения кода объект класса `FiveLetterAnalyzer` должен содержать атрибуты `original`, `five_count` и `total_words`, унаследованные от соответствующих базовых классов. Все дочерние классы должны использовать `super().__init__()` для делегирования инициализации родительских классов.

Также необходимо вывести в консоль сообщения о начале и завершении выполнения каждого метода `__init__` при создании одного объекта `FiveLetterAnalyzer`. Пример вывода:

```
Start OriginalInput.__init__
End OriginalInput.__init__
Start FiveLetterWordCounter.__init__
End FiveLetterWordCounter.__init__
Start WordTotaler.__init__
End WordTotaler.__init__
Start FiveLetterAnalyzer.__init__
End FiveLetterAnalyzer.__init__
```

- 23 Создать классы, которые будут выполнять различные функции, такие как определение количества строчных букв в тексте, подсчёт количества заглавных букв и сохранение исходного текста, а также выводить результаты на печать методом `print`.

- (a) Класс `GivenText` должен сохранять исходный текст. Класс `LowercaseCounter` — подсчитывать количество строчных букв. Класс `UppercaseCounter` — подсчитывать количество заглавных букв. Класс `CaseAnalyzer` объединяет функциональность всех трёх классов.

Создайте класс `GivenText`, который содержит метод `__init__`, принимающий текст в качестве аргумента и сохраняющий его в атрибуте `given`.

- (b) Создайте класс `LowercaseCounter`, который наследуется от `GivenText`. Этот класс должен содержать метод `__init__`, вызывающий метод `__init__` базового класса с помощью `super()`, а затем подсчитывающий количество строчных букв и сохраняющий это значение в атрибуте `lower_count`.
- (c) Создайте класс `UppercaseCounter`, который также наследуется от `GivenText`. Этот класс должен содержать метод `__init__`, вызывающий метод `__init__` базового класса с помощью `super()`, а затем подсчитывающий количество заглавных букв и сохраняющий это значение в атрибуте `upper_count`.
- (d) Создайте класс `CaseAnalyzer`, который наследуется от `LowercaseCounter` и `UppercaseCounter`. Класс `CaseAnalyzer` должен содержать метод `__init__`, корректно вызывающий инициализаторы всех родительских классов через `super()`, и после инициализации выводить информацию о регистрах.
- (e) Создайте экземпляр класса `CaseAnalyzer` и передайте ему текст для обработки.
- (f) Выведите результаты работы класса `CaseAnalyzer`, включая исходный текст (`given`), количество строчных букв (`lower_count`) и количество заглавных букв (`upper_count`).

В результате выполнения кода объект класса `CaseAnalyzer` должен содержать атрибуты `given`, `lower_count` и `upper_count`, унаследованные от соответствующих базовых классов. Все дочерние классы должны использовать `super().__init__()` для делегирования инициализации родительских классов.

Также необходимо вывести в консоль сообщения о начале и завершении выполнения каждого метода `__init__` при создании одного объекта `CaseAnalyzer`. Пример вывода:

```
Start GivenText.__init__
End GivenText.__init__
Start LowercaseCounter.__init__
End LowercaseCounter.__init__
Start UppercaseCounter.__init__
End UppercaseCounter.__init__
Start CaseAnalyzer.__init__
End CaseAnalyzer.__init__
```

- 24 Создать классы, которые будут выполнять различные функции, такие как определение количества цифр в тексте, подсчёт количества букв и сохранение исходного текста, а также выводить результаты на печать методом `print`.

- (a) Класс `RawInput` должен сохранять исходный текст. Класс `DigitCounter` — подсчитывать количество цифр. Класс `LetterCounter` — подсчитывать количество букв. Класс `AlphaNumericAnalyzer` объединяет функциональность всех трёх классов.

Создайте класс `RawInput`, который содержит метод `__init__`, принимающий текст в качестве аргумента и сохраняющий его в атрибуте `raw`.

- (b) Создайте класс `DigitCounter`, который наследуется от `RawInput`. Этот класс должен содержать метод `__init__`, вызывающий метод `__init__` базового класса с помощью `super()`, а затем подсчитывающий количество цифр и сохраняющий это значение в атрибуте `digit_num`.

- (c) Создайте класс `LetterCounter`, который также наследуется от `RawInput`. Этот класс должен содержать метод `__init__`, вызывающий метод `__init__` базового класса с помощью `super()`, а затем подсчитывающий количество букв и сохраняющий это значение в атрибуте `letter_num`.
- (d) Создайте класс `AlphaNumericAnalyzer`, который наследуется от `DigitCounter` и `LetterCounter`. Класс `AlphaNumericAnalyzer` должен содержать метод `__init__`, корректно вызывающий инициализаторы всех родительских классов через `super()`, и после инициализации выводить информацию о цифрах и буквах.
- (e) Создайте экземпляр класса `AlphaNumericAnalyzer` и передайте ему текст для обработки.
- (f) Выведите результаты работы класса `AlphaNumericAnalyzer`, включая исходный текст (`raw`), количество цифр (`digit_num`) и количество букв (`letter_num`).

В результате выполнения кода объект класса `AlphaNumericAnalyzer` должен содержать атрибуты `raw`, `digit_num` и `letter_num`, унаследованные от соответствующих базовых классов. Все дочерние классы должны использовать `super().__init__()` для делегирования инициализации родительских классов.

Также необходимо вывести в консоль сообщения о начале и завершении выполнения каждого метода `__init__` при создании одного объекта `AlphaNumericAnalyzer`. Пример вывода:

```
Start RawInput.__init__
End RawInput.__init__
Start DigitCounter.__init__
End DigitCounter.__init__
Start LetterCounter.__init__
End LetterCounter.__init__
Start AlphaNumericAnalyzer.__init__
End AlphaNumericAnalyzer.__init__
```

- 25 Создать классы, которые будут выполнять различные функции, такие как удаление всех пробелов из текста, подсчёт количества удалённых пробелов и сохранение результата, а также выводить результаты на печать методом `print`.

- (a) Класс `InitialString` должен сохранять исходный текст. Класс `SpaceRemover` — удалять все пробелы. Класс `RemovedSpaceCounter` — подсчитывать количество удалённых пробелов. Класс `CompactText` объединяет функциональность всех трёх классов.

Создайте класс `InitialString`, который содержит метод `__init__`, принимающий текст в качестве аргумента и сохраняющий его в атрибуте `initial`.

- (b) Создайте класс `SpaceRemover`, который наследуется от `InitialString`. Этот класс должен содержать метод `__init__`, вызывающий метод `__init__` базового класса с помощью `super()`, а затем удаляющий все пробелы и сохраняющий результат в атрибуте `compact`.
- (c) Создайте класс `RemovedSpaceCounter`, который также наследуется от `InitialString`. Этот класс должен содержать метод `__init__`, вызывающий метод `__init__` базового класса с помощью `super()`, а затем подсчитывающий количество удалённых пробелов и сохраняющий это значение в атрибуте `removed_spaces`.

- (d) Создайте класс `CompactText`, который наследуется от `RemovedSpaceCounter`. Класс `CompactText` должен содержать метод `__init__`, корректно вызывающий инициализаторы всех родительских классов через `super()`, и после инициализации выводить информацию об удалении пробелов.
- (e) Создайте экземпляр класса `CompactText` и передайте ему текст для обработки.
- (f) Выведите результаты работы класса `CompactText`, включая исходный текст (`initial`), текст без пробелов (`compact`) и количество удалённых пробелов (`removed_spaces`).

В результате выполнения кода объект класса `CompactText` должен содержать атрибуты `initial`, `compact` и `removed_spaces`, унаследованные от соответствующих базовых классов. Все дочерние классы должны использовать `super().__init__()` для делегирования инициализации родительских классов.

Также необходимо вывести в консоль сообщения о начале и завершении выполнения каждого метода `__init__` при создании одного объекта `CompactText`. Пример вывода:

```
Start InitialString.__init__
End InitialString.__init__
Start SpaceRemover.__init__
End SpaceRemover.__init__
Start RemovedSpaceCounter.__init__
End RemovedSpaceCounter.__init__
Start CompactText.__init__
End CompactText.__init__
```

- 26 Создать классы, которые будут выполнять различные функции, такие как определение количества знаков препинания в тексте, подсчёт количества букв и сохранение исходного текста, а также выводить результаты на печать методом `print`.

- (a) Класс `StartText` должен сохранять исходный текст. Класс `PunctuationCounter` — подсчитывать количество знаков препинания. Класс `AlphabeticCounter` — подсчитывать количество букв. Класс `TextComposition` объединяет функциональность всех трёх классов.
Создайте класс `StartText`, который содержит метод `__init__`, принимающий текст в качестве аргумента и сохраняющий его в атрибуте `start`.
- (b) Создайте класс `PunctuationCounter`, который наследуется от `StartText`. Этот класс должен содержать метод `__init__`, вызывающий метод `__init__` базового класса с помощью `super()`, а затем подсчитывающий количество знаков препинания и сохраняющий это значение в атрибуте `punct_count`.
- (c) Создайте класс `AlphabeticCounter`, который также наследуется от `StartText`. Этот класс должен содержать метод `__init__`, вызывающий метод `__init__` базового класса с помощью `super()`, а затем подсчитывающий количество букв и сохраняющий это значение в атрибуте `alpha_count`.
- (d) Создайте класс `TextComposition`, который наследуется от `PunctuationCounter` и `AlphabeticCounter`. Класс `TextComposition` должен содержать метод `__init__`, корректно вызывающий инициализаторы всех родительских классов через `super()`, и после инициализации выводить информацию о составе текста.
- (e) Создайте экземпляр класса `TextComposition` и передайте ему текст для обработки.

- (f) Выведите результаты работы класса `TextComposition`, включая исходный текст (`start`), количество знаков препинания (`punct_count`) и количество букв (`alpha_count`).

В результате выполнения кода объект класса `TextComposition` должен содержать атрибуты `start`, `punct_count` и `alpha_count`, унаследованные от соответствующих базовых классов. Все дочерние классы должны использовать `super().__init__()` для делегирования инициализации родительских классов.

Также необходимо вывести в консоль сообщения о начале и завершении выполнения каждого метода `__init__` при создании одного объекта `TextComposition`. Пример вывода:

```
Start StartText.__init__
End StartText.__init__
Start PunctuationCounter.__init__
End PunctuationCounter.__init__
Start AlphabeticCounter.__init__
End AlphabeticCounter.__init__
Start TextComposition.__init__
End TextComposition.__init__
```

- 27 Создать классы, которые будут выполнять различные функции, такие как определение количества слов, состоящих только из цифр, подсчёт общего количества слов и сохранение исходного текста, а также выводить результаты на печать методом `print`.

- (a) Класс `OriginalStringInput` должен сохранять исходный текст. Класс `NumericWordCounter` — подсчитывать количество слов, состоящих только из цифр. Класс `TotalWordCounter` — подсчитывать общее количество слов. Класс `NumericWordAnalyzer` объединяет функциональность всех трёх классов.
Создайте класс `OriginalStringInput`, который содержит метод `__init__`, принимающий текст в качестве аргумента и сохраняющий его в атрибуте `original`.
- (b) Создайте класс `NumericWordCounter`, который наследуется от `OriginalStringInput`. Этот класс должен содержать метод `__init__`, вызывающий метод `__init__` базового класса с помощью `super()`, а затем подсчитывающий количество слов, состоящих только из цифр, и сохраняющий это значение в атрибуте `numeric_words`.
- (c) Создайте класс `TotalWordCounter`, который также наследуется от `OriginalStringInput`. Этот класс должен содержать метод `__init__`, вызывающий метод `__init__` базового класса с помощью `super()`, а затем подсчитывающий общее количество слов и сохраняющий это значение в атрибуте `total_words`.
- (d) Создайте класс `NumericWordAnalyzer`, который наследуется от `NumericWordCounter` и `TotalWordCounter`. Класс `NumericWordAnalyzer` должен содержать метод `__init__`, корректно вызывающий инициализаторы всех родительских классов через `super()`, и после инициализации выводить информацию о числовых словах.
- (e) Создайте экземпляр класса `NumericWordAnalyzer` и передайте ему текст для обработки.
- (f) Выведите результаты работы класса `NumericWordAnalyzer`, включая исходный текст (`original`), количество числовых слов (`numeric_words`) и общее количество слов (`total_words`).

В результате выполнения кода объект класса `NumericWordAnalyzer` должен содержать атрибуты `original`, `numeric_words` и `total_words`, унаследованные от соответствующих базовых классов. Все дочерние классы должны использовать `super().__init__()` для делегирования инициализации родительских классов.

Также необходимо вывести в консоль сообщения о начале и завершении выполнения каждого метода `__init__` при создании одного объекта `NumericWordAnalyzer`. Пример вывода:

```
Start OriginalStringInput.__init__
End OriginalStringInput.__init__
Start NumericWordCounter.__init__
End NumericWordCounter.__init__
Start TotalWordCounter.__init__
End TotalWordCounter.__init__
Start NumericWordAnalyzer.__init__
End NumericWordAnalyzer.__init__
```

- 28 Создать классы, которые будут выполнять различные функции, такие как определение количества слов, содержащих букву «а», подсчёт общего количества слов и сохранение исходного текста, а также выводить результаты на печать методом `print`.

- (a) Класс `BaseInput` должен сохранять исходный текст. Класс `WordsWithA` — подсчитывать количество слов, содержащих букву «а» (в любом регистре). Класс `TotalWords` — подсчитывать общее количество слов. Класс `LetterAAnalyzer` объединяет функциональность всех трёх классов.
Создайте класс `BaseInput`, который содержит метод `__init__`, принимающий текст в качестве аргумента и сохраняющий его в атрибуте `base`.
- (b) Создайте класс `WordsWithA`, который наследуется от `BaseInput`. Этот класс должен содержать метод `__init__`, вызывающий метод `__init__` базового класса с помощью `super()`, а затем подсчитывающий количество слов, содержащих букву «а», и сохраняющий это значение в атрибуте `words_with_a`.
- (c) Создайте класс `TotalWords`, который также наследуется от `BaseInput`. Этот класс должен содержать метод `__init__`, вызывающий метод `__init__` базового класса с помощью `super()`, а затем подсчитывающий общее количество слов и сохраняющий это значение в атрибуте `total`.
- (d) Создайте класс `LetterAAnalyzer`, который наследуется от `WordsWithA` и `TotalWords`. Класс `LetterAAnalyzer` должен содержать метод `__init__`, корректно вызывающий инициализаторы всех родительских классов через `super()`, и после инициализации выводить информацию о словах с буквой «а».
- (e) Создайте экземпляр класса `LetterAAnalyzer` и передайте ему текст для обработки.
- (f) Выведите результаты работы класса `LetterAAnalyzer`, включая исходный текст (`base`), количество слов с «а» (`words_with_a`) и общее количество слов (`total`).

В результате выполнения кода объект класса `LetterAAnalyzer` должен содержать атрибуты `base`, `words_with_a` и `total`, унаследованные от соответствующих базовых классов. Все дочерние классы должны использовать `super().__init__()` для делегирования инициализации родительских классов.

Также необходимо вывести в консоль сообщения о начале и завершении выполнения каждого метода `__init__` при создании одного объекта `LetterAnalyzer`. Пример вывода:

```
Start BaseInput.__init__
End BaseInput.__init__
Start WordsWithA.__init__
End WordsWithA.__init__
Start TotalWords.__init__
End TotalWords.__init__
Start LetterAnalyzer.__init__
End LetterAnalyzer.__init__
```

29 Создать классы, которые будут выполнять различные функции, такие как определение количества слов, длина которых кратна 3, подсчёт общего количества слов и сохранение исходного текста, а также выводить результаты на печать методом `print`.

- (a) Класс `InputData` должен сохранять исходный текст. Класс `WordsDivisibleByThree` — подсчитывать количество слов, длина которых кратна 3. Класс `SimpleWordCounter` — подсчитывать общее количество слов. Класс `LengthDivisibilityAnalyzer` объединяет функциональность всех трёх классов.
Создайте класс `InputData`, который содержит метод `__init__`, принимающий текст в качестве аргумента и сохраняющий его в атрибуте `input_data`.
- (b) Создайте класс `WordsDivisibleByThree`, который наследуется от `InputData`. Этот класс должен содержать метод `__init__`, вызывающий метод `__init__` базового класса с помощью `super()`, а затем подсчитывающий количество слов, длина которых делится на 3 без остатка, и сохраняющий это значение в атрибуте `div_by_3`.
- (c) Создайте класс `SimpleWordCounter`, который также наследуется от `InputData`. Этот класс должен содержать метод `__init__`, вызывающий метод `__init__` базового класса с помощью `super()`, а затем подсчитывающий общее количество слов и сохраняющий это значение в атрибуте `total`.
- (d) Создайте класс `LengthDivisibilityAnalyzer`, который наследуется от `WordsDivisibleByThree` и `SimpleWordCounter`. Класс `LengthDivisibilityAnalyzer` должен содержать метод `__init__`, корректно вызывающий инициализаторы всех родительских классов через `super()`, и после инициализации выводить информацию о делимости длин слов.
- (e) Создайте экземпляр класса `LengthDivisibilityAnalyzer` и передайте ему текст для обработки.
- (f) Выведите результаты работы класса `LengthDivisibilityAnalyzer`, включая исходный текст (`input_data`), количество слов с длиной, кратной 3 (`div_by_3`), и общее количество слов (`total`).

В результате выполнения кода объект класса `LengthDivisibilityAnalyzer` должен содержать атрибуты `input_data`, `div_by_3` и `total`, унаследованные от соответствующих базовых классов. Все дочерние классы должны использовать `super().__init__()` для делегирования инициализации родительских классов.

Также необходимо вывести в консоль сообщения о начале и завершении выполнения каждого метода `__init__` при создании одного объекта `LengthDivisibilityAnalyzer`. Пример вывода:

```
Start InputData.__init__
End InputData.__init__
Start WordsDivisibleByThree.__init__
End WordsDivisibleByThree.__init__
Start SimpleWordCounter.__init__
End SimpleWordCounter.__init__
Start LengthDivisibilityAnalyzer.__init__
End LengthDivisibilityAnalyzer.__init__
```

30 Создать классы, которые будут выполнять различные функции, такие как определение количества слов, начинающихся с гласной, подсчёт общего количества слов и сохранение исходного текста, а также выводить результаты на печать методом `print`.

- (a) Класс `TextContainer` должен сохранять исходный текст. Класс `VowelStartCounter` — подсчитывать количество слов, начинающихся с гласной. Класс `OverallWordCounter` — подсчитывать общее количество слов. Класс `VowelStartAnalyzer` объединяет функциональность всех трёх классов.

Создайте класс `TextContainer`, который содержит метод `__init__`, принимающий текст в качестве аргумента и сохраняющий его в атрибуте `text_container`.

- (b) Создайте класс `VowelStartCounter`, который наследуется от `TextContainer`. Этот класс должен содержать метод `__init__`, вызывающий метод `__init__` базового класса с помощью `super()`, а затем подсчитывающий количество слов, начинающихся с гласной (в любом регистре), и сохраняющий это значение в атрибуте `vowel_starts`.
- (c) Создайте класс `OverallWordCounter`, который также наследуется от `TextContainer`. Этот класс должен содержать метод `__init__`, вызывающий метод `__init__` базового класса с помощью `super()`, а затем подсчитывающий общее количество слов и сохраняющий это значение в атрибуте `overall`.
- (d) Создайте класс `VowelStartAnalyzer`, который наследуется от `VowelStartCounter` и `OverallWordCounter`. Класс `VowelStartAnalyzer` должен содержать метод `__init__`, корректно вызывающий инициализаторы всех родительских классов через `super()`, и после инициализации выводить информацию о словах, начинающихся с гласной.
- (e) Создайте экземпляр класса `VowelStartAnalyzer` и передайте ему текст для обработки.
- (f) Выведите результаты работы класса `VowelStartAnalyzer`, включая исходный текст (`text_container`), количество слов с гласной в начале (`vowel_starts`) и общее количество слов (`overall`).

В результате выполнения кода объект класса `VowelStartAnalyzer` должен содержать атрибуты `text_container`, `vowel_starts` и `overall`, унаследованные от соответствующих базовых классов. Все дочерние классы должны использовать `super().__init__()` для делегирования инициализации родительских классов.

Также необходимо вывести в консоль сообщения о начале и завершении выполнения каждого метода `__init__` при создании одного объекта `VowelStartAnalyzer`. Пример вывода:

```
Start TextContainer.__init__
End TextContainer.__init__
Start VowelStartCounter.__init__
End VowelStartCounter.__init__
Start OverallWordCounter.__init__
End OverallWordCounter.__init__
Start VowelStartAnalyzer.__init__
End VowelStartAnalyzer.__init__
```

- 31 Создать классы, которые будут выполнять различные функции, такие как определение количества слов, содержащих только латинские буквы, подсчёт общего количества слов и сохранение исходного текста, а также выводить результаты на печать методом `print`.
- (a) Класс `SourceData` должен сохранять исходный текст. Класс `LatinOnlyWordCounter` — подсчитывать количество слов, состоящих только из латинских букв. Класс `FullWordCounter` — подсчитывать общее количество слов. Класс `LatinWordAnalyzer` объединяет функциональность всех трёх классов.
Создайте класс `SourceData`, который содержит метод `__init__`, принимающий текст в качестве аргумента и сохраняющий его в атрибуте `source`.
 - (b) Создайте класс `LatinOnlyWordCounter`, который наследуется от `SourceData`. Этот класс должен содержать метод `__init__`, вызывающий метод `__init__` базового класса с помощью `super()`, а затем подсчитывающий количество слов, состоящих исключительно из латинских букв, и сохраняющий это значение в атрибуте `latin_only`.
 - (c) Создайте класс `FullWordCounter`, который также наследуется от `SourceData`. Этот класс должен содержать метод `__init__`, вызывающий метод `__init__` базового класса с помощью `super()`, а затем подсчитывающий общее количество слов и сохраняющий это значение в атрибуте `full`.
 - (d) Создайте класс `LatinWordAnalyzer`, который наследуется от `LatinOnlyWordCounter` и `FullWordCounter`. Класс `LatinWordAnalyzer` должен содержать метод `__init__`, корректно вызывающий инициализаторы всех родительских классов через `super()`, и после инициализации выводить информацию о латинских словах.
 - (e) Создайте экземпляр класса `LatinWordAnalyzer` и передайте ему текст для обработки.
 - (f) Выведите результаты работы класса `LatinWordAnalyzer`, включая исходный текст (`source`), количество слов только из латинских букв (`latin_only`) и общее количество слов (`full`).

В результате выполнения кода объект класса `LatinWordAnalyzer` должен содержать атрибуты `source`, `latin_only` и `full`, унаследованные от соответствующих базовых классов. Все дочерние классы должны использовать `super().__init__()` для делегирования инициализации родительских классов.

Также необходимо вывести в консоль сообщения о начале и завершении выполнения каждого метода `__init__` при создании одного объекта `LatinWordAnalyzer`. Пример вывода:

```
Start SourceData.__init__
End SourceData.__init__
Start LatinOnlyWordCounter.__init__
End LatinOnlyWordCounter.__init__
Start FullWordCounter.__init__
End FullWordCounter.__init__
Start LatinWordAnalyzer.__init__
End LatinWordAnalyzer.__init__
```

32 Создать классы, которые будут выполнять различные функции, такие как определение количества слов, оканчивающихся на «ing», подсчёт общего количества слов и сохранение исходного текста, а также выводить результаты на печать методом `print`.

- (a) Класс `TextBase` должен сохранять исходный текст. Класс `IngEndingCounter` — подсчитывать количество слов, оканчивающихся на «ing». Класс `AllWordCounter` — подсчитывать общее количество слов. Класс `IngAnalyzer` объединяет функциональность всех трёх классов.

Создайте класс `TextBase`, который содержит метод `__init__`, принимающий текст в качестве аргумента и сохраняющий его в атрибуте `text_base`.

- (b) Создайте класс `IngEndingCounter`, который наследуется от `TextBase`. Этот класс должен содержать метод `__init__`, вызывающий метод `__init__` базового класса с помощью `super()`, а затем подсчитывающий количество слов, оканчивающихся на «ing» (в любом регистре), и сохраняющий это значение в атрибуте `ing_count`.
- (c) Создайте класс `AllWordCounter`, который также наследуется от `TextBase`. Этот класс должен содержать метод `__init__`, вызывающий метод `__init__` базового класса с помощью `super()`, а затем подсчитывающий общее количество слов и сохраняющий это значение в атрибуте `all_words`.
- (d) Создайте класс `IngAnalyzer`, который наследуется от `IngEndingCounter` и `AllWordCounter`. Класс `IngAnalyzer` должен содержать метод `__init__`, корректно вызывающий инициализаторы всех родительских классов через `super()`, и после инициализации выводить информацию о словах с окончанием «ing».
- (e) Создайте экземпляр класса `IngAnalyzer` и передайте ему текст для обработки.
- (f) Выведите результаты работы класса `IngAnalyzer`, включая исходный текст (`text_base`), количество слов с окончанием «ing» (`ing_count`) и общее количество слов (`all_words`).

В результате выполнения кода объект класса `IngAnalyzer` должен содержать атрибуты `text_base`, `ing_count` и `all_words`, унаследованные от соответствующих базовых классов. Все дочерние классы должны использовать `super().__init__()` для делегирования инициализации родительских классов.

Также необходимо вывести в консоль сообщения о начале и завершении выполнения каждого метода `__init__` при создании одного объекта `IngAnalyzer`. Пример вывода:

```
Start TextBase.__init__
```

```

End TextBase.__init__
Start IngEndingCounter.__init__
End IngEndingCounter.__init__
Start AllWordCounter.__init__
End AllWordCounter.__init__
Start IngAnalyzer.__init__
End IngAnalyzer.__init__

```

33 Создать классы, которые будут выполнять различные функции, такие как определение количества слов, содержащих хотя бы одну цифру, подсчёт общего количества слов и сохранение исходного текста, а также выводить результаты на печать методом `print`.

- (a) Класс `InitialTextData` должен сохранять исходный текст. Класс `WordsWithDigits` — подсчитывать количество слов, содержащих хотя бы одну цифру. Класс `TotalWordCount` — подсчитывать общее количество слов. Класс `DigitInWordAnalyzer` объединяет функциональность всех трёх классов.
Создайте класс `InitialTextData`, который содержит метод `__init__`, принимающий текст в качестве аргумента и сохраняющий его в атрибуте `initial_text`.
- (b) Создайте класс `WordsWithDigits`, который наследуется от `InitialTextData`. Этот класс должен содержать метод `__init__`, вызывающий метод `__init__` базового класса с помощью `super()`, а затем подсчитывающий количество слов, содержащих хотя бы одну цифру, и сохраняющий это значение в атрибуте `words_with_digits`.
- (c) Создайте класс `TotalWordCount`, который также наследуется от `InitialTextData`. Этот класс должен содержать метод `__init__`, вызывающий метод `__init__` базового класса с помощью `super()`, а затем подсчитывающий общее количество слов и сохраняющий это значение в атрибуте `total_words`.
- (d) Создайте класс `DigitInWordAnalyzer`, который наследуется от `WordsWithDigits` и `TotalWordCount`. Класс `DigitInWordAnalyzer` должен содержать метод `__init__`, корректно вызывающий инициализаторы всех родительских классов через `super()`, и после инициализации выводить информацию о словах с цифрами.
- (e) Создайте экземпляр класса `DigitInWordAnalyzer` и передайте ему текст для обработки.
- (f) Выведите результаты работы класса `DigitInWordAnalyzer`, включая исходный текст (`initial_text`), количество слов с цифрами (`words_with_digits`) и общее количество слов (`total_words`).

В результате выполнения кода объект класса `DigitInWordAnalyzer` должен содержать атрибуты `initial_text`, `words_with_digits` и `total_words`, унаследованные от соответствующих базовых классов. Все дочерние классы должны использовать `super().__init__()` для делегирования инициализации родительских классов.

Также необходимо вывести в консоль сообщения о начале и завершении выполнения каждого метода `__init__` при создании одного объекта `DigitInWordAnalyzer`. Пример вывода:

```

Start InitialTextData.__init__
End InitialTextData.__init__

```

```

Start WordsWithDigits.__init__
End WordsWithDigits.__init__
Start TotalWordCount.__init__
End TotalWordCount.__init__
Start DigitInWordAnalyzer.__init__
End DigitInWordAnalyzer.__init__

```

34 Создать классы, которые будут выполнять различные функции, такие как определение количества слов, длина которых меньше 4 символов, подсчёт общего количества слов и сохранение исходного текста, а также выводить результаты на печать методом `print`.

- (a) Класс `BaseTextData` должен сохранять исходный текст. Класс `ShortWordCounter` — подсчитывать количество слов длиной менее 4 символов. Класс `OverallWordCounter` — подсчитывать общее количество слов. Класс `ShortWordAnalyzer` объединяет функциональность всех трёх классов.
Создайте класс `BaseTextData`, который содержит метод `__init__`, принимающий текст в качестве аргумента и сохраняющий его в атрибуте `base_text`.
- (b) Создайте класс `ShortWordCounter`, который наследуется от `BaseTextData`. Этот класс должен содержать метод `__init__`, вызывающий метод `__init__` базового класса с помощью `super()`, а затем подсчитывающий количество слов длиной менее 4 и сохраняющий это значение в атрибуте `short_count`.
- (c) Создайте класс `OverallWordCounter`, который также наследуется от `BaseTextData`. Этот класс должен содержать метод `__init__`, вызывающий метод `__init__` базового класса с помощью `super()`, а затем подсчитывающий общее количество слов и сохраняющий это значение в атрибуте `overall_count`.
- (d) Создайте класс `ShortWordAnalyzer`, который наследуется от `ShortWordCounter` и `OverallWordCounter`. Класс `ShortWordAnalyzer` должен содержать метод `__init__`, корректно вызывающий инициализаторы всех родительских классов через `super()`, и после инициализации выводить информацию о коротких словах.
- (e) Создайте экземпляр класса `ShortWordAnalyzer` и передайте ему текст для обработки.
- (f) Выведите результаты работы класса `ShortWordAnalyzer`, включая исходный текст (`base_text`), количество коротких слов (`short_count`) и общее количество слов (`overall_count`).

В результате выполнения кода объект класса `ShortWordAnalyzer` должен содержать атрибуты `base_text`, `short_count` и `overall_count`, унаследованные от соответствующих базовых классов. Все дочерние классы должны использовать `super().__init__()` для делегирования инициализации родительских классов.

Также необходимо вывести в консоль сообщения о начале и завершении выполнения каждого метода `__init__` при создании одного объекта `ShortWordAnalyzer`. Пример вывода:

```

Start BaseTextData.__init__
End BaseTextData.__init__
Start ShortWordCounter.__init__

```

```

End ShortWordCounter.__init__
Start OverallWordCounter.__init__
End OverallWordCounter.__init__
Start ShortWordAnalyzer.__init__
End ShortWordAnalyzer.__init__

```

35 Создать классы, которые будут выполнять различные функции, такие как определение количества слов, содержащих хотя бы один символ пунктуации, подсчёт общего количества слов и сохранение исходного текста, а также выводить результаты на печать методом `print`.

- (a) Класс `OriginalTextData` должен сохранять исходный текст. Класс `PunctuatedWordCounter` — подсчитывать количество слов, содержащих хотя бы один знак пунктуации. Класс `TotalWordTally` — подсчитывать общее количество слов. Класс `PunctuationInWordAnalyzer` объединяет функциональность всех трёх классов.
Создайте класс `OriginalTextData`, который содержит метод `__init__`, принимающий текст в качестве аргумента и сохраняющий его в атрибуте `original_text`.
- (b) Создайте класс `PunctuatedWordCounter`, который наследуется от `OriginalTextData`. Этот класс должен содержать метод `__init__`, вызывающий метод `__init__` базового класса с помощью `super()`, а затем подсчитывающий количество слов, содержащих хотя бы один знак пунктуации, и сохраняющий это значение в атрибуте `punct_words`.
- (c) Создайте класс `TotalWordTally`, который также наследуется от `OriginalTextData`. Этот класс должен содержать метод `__init__`, вызывающий метод `__init__` базового класса с помощью `super()`, а затем подсчитывающий общее количество слов и сохраняющий это значение в атрибуте `total_words`.
- (d) Создайте класс `PunctuationInWordAnalyzer`, который наследуется от `PunctuatedWordCounter` и `TotalWordTally`. Класс `PunctuationInWordAnalyzer` должен содержать метод `__init__`, корректно вызывающий инициализаторы всех родительских классов через `super()`, и после инициализации выводить информацию о словах со знаками пунктуации.
- (e) Создайте экземпляр класса `PunctuationInWordAnalyzer` и передайте ему текст для обработки.
- (f) Выведите результаты работы класса `PunctuationInWordAnalyzer`, включая исходный текст (`original_text`), количество слов со знаками пунктуации (`punct_words`) и общее количество слов (`total_words`).

В результате выполнения кода объект класса `PunctuationInWordAnalyzer` должен содержать атрибуты `original_text`, `punct_words` и `total_words`, унаследованные от соответствующих базовых классов. Все дочерние классы должны использовать `super().__init__()` для делегирования инициализации родительских классов.

Также необходимо вывести в консоль сообщения о начале и завершении выполнения каждого метода `__init__` при создании одного объекта `PunctuationInWordAnalyzer`. Пример вывода:

```

Start OriginalTextData.__init__
End OriginalTextData.__init__

```

```

Start PunctuatedWordCounter.__init__
End PunctuatedWordCounter.__init__
Start TotalWordTally.__init__
End TotalWordTally.__init__
Start PunctuationInWordAnalyzer.__init__
End PunctuationInWordAnalyzer.__init__

```

2.9.2 Задача 2

Реализовать задачу 1 без использования функции `super().__init__`. Сравнить результаты выполнения программ задач 1 и 2, описать, в чем преимущества использования функции `super().__init__` в решении данной задачи

2.9.3 Задача 3

- 1 (a) Создайте новый файл с расширением `.py`.
- (b) Создайте класс `Sedan` для автомобилей типа седан. Определите атрибуты `doors` и `engine_type`, а также методы-свойства (`@property`) для их получения и установки — геттеры и сеттеры. Также определите методы `start_engine()` и `accelerate()`.
- (c) Создайте класс `SUV` для автомобилей типа внедорожник. Определите атрибуты `doors`, `engine_type` и `cargo_space`, а также методы-свойства для их получения и установки. Также определите методы `start_engine()`, `accelerate()` и `brake()`.
- (d) Создайте класс `Hatchback` для автомобилей типа хэтчбэк. Определите атрибуты `doors`, `engine_type`, `cargo_space` и `transmission`, а также методы-свойства (`@property`) для их получения и установки. Также определите метод `start_transmission()`.
- (e) Создайте класс `Car` как общий класс автомобиля, который наследуется от всех трёх классов (`Sedan`, `SUV`, `Hatchback`). Определите метод `__init__()`, который корректно вызывает конструкторы родительских классов.
- (f) В классе `Car` определите метод `drive()`, который будет последовательно вызывать все методы родительских классов, связанные с управлением автомобилем: запуск двигателя, включение трансмиссии, ускорение и торможение.
- (g) В конце файла добавьте блок

```
if __name__ == "__main__":
```

чтобы протестировать ваш код. В этом блоке создайте экземпляр класса `Car` и вызовите его методы, которые выводят состояние автомобиля и значения его атрибутов.
- (h) Сохраните файл и запустите его в IDE, чтобы проверить его работу.

- 2 (a) Создайте новый файл с расширением `.py`.
- (b) Создайте класс `ElectricCar` для электромобилей. Определите атрибуты `battery_capacity` и `motor_type`, а также методы-свойства (`@property`) для их получения и установки. Также определите методы `charge_battery()` и `drive_electric()`.
- (c) Создайте класс `HybridCar` для гибридных автомобилей. Определите атрибуты `battery_capacity`, `engine_type` и `fuel_tank_size`, а также методы-свойства для их получения и установки. Также определите методы `switch_mode()`, `refuel()` и `drive_hybrid()`.

- (d) Создайте класс `FuelCellCar` для автомобилей с топливными элементами. Определите атрибуты `hydrogen_tank`, `motor_type`, `range_km` и `refuel_time`, а также методы-свойства (`@property`) для их получения и установки. Также определите метод `refill_hydrogen()`.
- (e) Создайте класс `GreenCar` как общий класс экологичного автомобиля, который наследуется от всех трёх классов (`ElectricCar`, `HybridCar`, `FuelCellCar`). Определите метод `__init__()`, который корректно вызывает конструкторы родительских классов.
- (f) В классе `GreenCar` определите метод `go_eco()`, который будет последовательно вызывать все методы родительских классов, связанные с использованием энергии: зарядка, заправка, переключение режимов и движение.
- (g) В конце файла добавьте блок

```
if __name__ == "__main__":
```

чтобы протестировать ваш код. В этом блоке создайте экземпляр класса `GreenCar` и вызовите его методы, которые выводят состояние автомобиля и значения его атрибутов.

- (h) Сохраните файл и запустите его в IDE, чтобы проверить его работу.

3 (a) Создайте новый файл с расширением `.py`.

- (b) Создайте класс `Van` для фургонов. Определите атрибуты `cargo_volume` и `engine_type`, а также методы-свойства (`@property`) для их получения и установки. Также определите методы `load_cargo()` и `start_engine()`.
- (c) Создайте класс `Pickup` для пикапов. Определите атрибуты `cargo_bed_size`, `engine_type` и `towing_capacity`, а также методы-свойства для их получения и установки. Также определите методы `hitch_trailer()`, `start_engine()` и `unload_cargo()`.
- (d) Создайте класс `Minivan` для минивэнов. Определите атрибуты `seats`, `engine_type`, `cargo_space` и `ac_system`, а также методы-свойства (`@property`) для их получения и установки. Также определите метод `activate_climate_control()`.
- (e) Создайте класс `UtilityVehicle` как общий класс коммерческого транспорта, который наследуется от всех трёх классов (`Van`, `Pickup`, `Minivan`). Определите метод `__init__()`, который корректно вызывает конструкторы родительских классов.
- (f) В классе `UtilityVehicle` определите метод `operate()`, который будет последовательно вызывать все методы родительских классов, связанные с эксплуатацией: загрузка, запуск двигателя, прицепка прицепа, активация климат-контроля и разгрузка.
- (g) В конце файла добавьте блок

```
if __name__ == "__main__":
```

чтобы протестировать ваш код. В этом блоке создайте экземпляр класса `UtilityVehicle` и вызовите его методы, которые выводят состояние автомобиля и значения его атрибутов.

- (h) Сохраните файл и запустите его в IDE, чтобы проверить его работу.

4 (a) Создайте новый файл с расширением `.py`.

- (b) Создайте класс `SportsCar` для спортивных автомобилей. Определите атрибуты `max_speed` и `engine_type`, а также методы-свойства (`@property`) для их получения и установки. Также определите методы `launch_control()` и `shift_gear()`.
- (c) Создайте класс `Coupe` для купе. Определите атрибуты `doors`, `engine_type` и `aerodynamics`, а также методы-свойства для их получения и установки. Также определите методы `activate_aero()`, `shift_gear()` и `cruise()`.
- (d) Создайте класс `Convertible` для кабриолетов. Определите атрибуты `roof_type`, `engine_type`, `wind_noise` и `top_operation_time`, а также методы-свойства (`@property`) для их получения и установки. Также определите метод `toggle_roof()`.
- (e) Создайте класс `PerformanceCar` как общий класс спортивного автомобиля, который наследуется от всех трёх классов (`SportsCar`, `Coupe`, `Convertible`). Определите метод `__init__()`, который корректно вызывает конструкторы родительских классов.
- (f) В классе `PerformanceCar` определите метод `race_mode()`, который будет последовательно вызывать все методы родительских классов, связанные с производительностью: запуск старта, активация аэродинамики, переключение передач и открытие/закрытие крыши.
- (g) В конце файла добавьте блок

```
if __name__ == "__main__":
```

чтобы протестировать ваш код. В этом блоке создайте экземпляр класса `PerformanceCar` и вызовите его методы, которые выводят состояние автомобиля и значения его атрибутов.

- (h) Сохраните файл и запустите его в IDE, чтобы проверить его работу.

5 (a) Создайте новый файл с расширением `.py`.

- (b) Создайте класс `Scooter` для электроскутеров. Определите атрибуты `battery_life` и `max_speed`, а также методы-свойства (`@property`) для их получения и установки. Также определите методы `power_on()` и `throttle()`.
- (c) Создайте класс `Motorcycle` для мотоциклов. Определите атрибуты `engine_displacement`, `fuel_type` и `seat_height`, а также методы-свойства для их получения и установки. Также определите методы `ignite()`, `throttle()` и `brake()`.
- (d) Создайте класс `Moped` для мопедов. Определите атрибуты `pedal_assist`, `engine_type`, `top_speed` и `fuel_efficiency`, а также методы-свойства (`@property`) для их получения и установки. Также определите метод `engage_pedals()`.
- (e) Создайте класс `TwoWheeler` как общий класс двухколёсного транспорта, который наследуется от всех трёх классов (`Scooter`, `Motorcycle`, `Moped`). Определите метод `__init__()`, который корректно вызывает конструкторы родительских классов.
- (f) В классе `TwoWheeler` определите метод `ride()`, который будет последовательно вызывать все методы родительских классов, связанные с управлением: включение питания, розжиг двигателя, нажатие педалей, дроссель и торможение.
- (g) В конце файла добавьте блок

```
if __name__ == "__main__":
```

чтобы протестировать ваш код. В этом блоке создайте экземпляр класса `TwoWheeler` и вызовите его методы, которые выводят состояние транспорта и значения его атрибутов.

(h) Сохраните файл и запустите его в IDE, чтобы проверить его работу.

6 (a) Создайте новый файл с расширением `.py`.

(b) Создайте класс `Bicycle` для обычных велосипедов. Определите атрибуты `frame_size` и `gear_count`, а также методы-свойства (`@property`) для их получения и установки. Также определите методы `pedal()` и `brake()`.

(c) Создайте класс `Ebike` для электровелосипедов. Определите атрибуты `battery_capacity`, `motor_power` и `range_km`, а также методы-свойства для их получения и установки. Также определите методы `turn_on_motor()`, `pedal()` и `recharge()`.

(d) Создайте класс `Tandem` для tandemных велосипедов. Определите атрибуты `rider_count`, `frame_material`, `gear_count` и `weight`, а также методы-свойства (`@property`) для их получения и установки. Также определите метод `coordinate_pedaling()`.

(e) Создайте класс `Cycle` как общий класс велосипедов, который наследуется от всех трёх классов (`Bicycle`, `Ebike`, `Tandem`). Определите метод `__init__()`, который корректно вызывает конструкторы родительских классов.

(f) В классе `Cycle` определите метод `ride_together()`, который будет последовательно вызывать все методы родительских классов, связанные с движением: педалирование, координация, включение мотора и торможение.

(g) В конце файла добавьте блок

```
if __name__ == "__main__":
```

чтобы протестировать ваш код. В этом блоке создайте экземпляр класса `Cycle` и вызовите его методы, которые выводят состояние велосипеда и значения его атрибутов.

(h) Сохраните файл и запустите его в IDE, чтобы проверить его работу.

7 (a) Создайте новый файл с расширением `.py`.

(b) Создайте класс `Drone` для дронов. Определите атрибуты `flight_time` и `camera_resolution`, а также методы-свойства (`@property`) для их получения и установки. Также определите методы `take_off()` и `hover()`.

(c) Создайте класс `Helicopter` для вертолётов. Определите атрибуты `rotor_diameter`, `fuel_capacity` и `max_altitude`, а также методы-свойства для их получения и установки. Также определите методы `start_rotors()`, `hover()` и `land()`.

(d) Создайте класс `Gyrocopter` для гирокоптеров. Определите атрибуты `engine_power`, `rotor_type`, `takeoff_distance` и `empty_weight`, а также методы-свойства (`@property`) для их получения и установки. Также определите метод `spin_rotor()`.

(e) Создайте класс `Rotorcraft` как общий класс летательного аппарата с несущим винтом, который наследуется от всех трёх классов (`Drone`, `Helicopter`, `Gyrocopter`). Определите метод `__init__()`, который корректно вызывает конструкторы родительских классов.

(f) В классе `Rotorcraft` определите метод `fly_vertical()`, который будет последовательно вызывать все методы родительских классов, связанные с полётом: запуск двигателей, вращение роторов, взлёт, зависание и посадка.

- (g) В конце файла добавьте блок

```
if __name__ == "__main__":
```

чтобы протестировать ваш код. В этом блоке создайте экземпляр класса `Rotorcraft` и вызовите его методы, которые выводят состояние аппарата и значения его атрибутов.

- (h) Сохраните файл и запустите его в IDE, чтобы проверить его работу.

- 8 (a) Создайте новый файл с расширением `.py`.

- (b) Создайте класс `Airplane` для самолётов. Определите атрибуты `wingspan` и `cruise_speed`, а также методы-свойства (`@property`) для их получения и установки. Также определите методы `taxi()` и `take_off()`.

- (c) Создайте класс `Jet` для реактивных самолётов. Определите атрибуты `engine_thrust`, `fuel_type` и `max_mach`, а также методы-свойства для их получения и установки. Также определите методы `ignite_afterburner()`, `take_off()` и `land()`.

- (d) Создайте класс `Glider` для планёров. Определите атрибуты `aspect_ratio`, `empty_weight`, `glide_ratio` и `tow_release_altitude`, а также методы-свойства (`@property`) для их получения и установки. Также определите метод `release_tow()`.

- (e) Создайте класс `FixedWing` как общий класс самолёта с неподвижным крылом, который наследуется от всех трёх классов (`Airplane`, `Jet`, `Glider`). Определите метод `__init__()`, который корректно вызывает конструкторы родительских классов.

- (f) В классе `FixedWing` определите метод `soar()`, который будет последовательно вызывать все методы родительских классов, связанные с полётом: руление, взлёт, включение форсажа, отцепка от буксировки и посадка.

- (g) В конце файла добавьте блок

```
if __name__ == "__main__":
```

чтобы протестировать ваш код. В этом блоке создайте экземпляр класса `FixedWing` и вызовите его методы, которые выводят состояние самолёта и значения его атрибутов.

- (h) Сохраните файл и запустите его в IDE, чтобы проверить его работу.

- 9 (a) Создайте новый файл с расширением `.py`.

- (b) Создайте класс `Cruiser` для круизных судов. Определите атрибуты `passenger_capacity` и `engine_power`, а также методы-свойства (`@property`) для их получения и установки. Также определите методы `depart()` и `sail()`.

- (c) Создайте класс `Ferry` для паромов. Определите атрибуты `vehicle_deck_size`, `engine_type` и `crossing_time`, а также методы-свойства для их получения и установки. Также определите методы `dock()`, `sail()` и `load_vehicles()`.

- (d) Создайте класс `Yacht` для яхт. Определите атрибуты `mast_height`, `engine_power`, `cabin_count` и `luxury_level`, а также методы-свойства (`@property`) для их получения и установки. Также определите метод `hoist_sail()`.

- (e) Создайте класс **Vessel** как общий класс водного транспорта, который наследуется от всех трёх классов (**Cruiser**, **Ferry**, **Yacht**). Определите метод `__init__()`, который корректно вызывает конструкторы родительских классов.
- (f) В классе **Vessel** определите метод `navigate()`, который будет последовательно вызывать все методы родительских классов, связанные с мореплаванием: отплытие, подъём парусов, перевозка транспорта, плавание и причаливание.
- (g) В конце файла добавьте блок


```
if __name__ == "__main__":
```

чтобы протестировать ваш код. В этом блоке создайте экземпляр класса **Vessel** и вызовите его методы, которые выводят состояние судна и значения его атрибутов.

- (h) Сохраните файл и запустите его в IDE, чтобы проверить его работу.

10 (a) Создайте новый файл с расширением `.py`.

- (b) Создайте класс **Submarine** для подводных лодок. Определите атрибуты `depth_rating` и `ballast_type`, а также методы-свойства (`@property`) для их получения и установки. Также определите методы `submerge()` и `navigate_underwater()`.
- (c) Создайте класс **Hovercraft** для судов на воздушной подушке. Определите атрибуты `skirt_material`, `engine_power` и `terrain_type`, а также методы-свойства для их получения и установки. Также определите методы `inflate_skirt()`, `navigate_underwater()` и `deflate()`.
- (d) Создайте класс **AmphibiousVehicle** для амфибий. Определите атрибуты `wheel_type`, `propeller_count`, `land_speed` и `water_speed`, а также методы-свойства (`@property`) для их получения и установки. Также определите метод `switch_mode()`.
- (e) Создайте класс **AmphibiousCraft** как общий класс транспорта, способного передвигаться по воде и суше, который наследуется от всех трёх классов (**Submarine**, **Hovercraft**, **AmphibiousVehicle**). Определите метод `__init__()`, который корректно вызывает конструкторы родительских классов.
- (f) В классе **AmphibiousCraft** определите метод `traverse()`, который будет последовательно вызывать все методы родительских классов, связанные с переходом между средами: погружение, надувание юбки, переключение режимов и подводная навигация.
- (g) В конце файла добавьте блок


```
if __name__ == "__main__":
```

чтобы протестировать ваш код. В этом блоке создайте экземпляр класса **AmphibiousCraft** и вызовите его методы, которые выводят состояние транспорта и значения его атрибутов.

- (h) Сохраните файл и запустите его в IDE, чтобы проверить его работу.

11 (a) Создайте новый файл с расширением `.py`.

- (b) Создайте класс **Tractor** для тракторов. Определите атрибуты `engine_hp` и `pto_speed`, а также методы-свойства (`@property`) для их получения и установки. Также определите методы `plow()` и `start_engine()`.

- (c) Создайте класс `Combine` для комбайнов. Определите атрибуты `grain_tank_size`, `engine_type` и `header_width`, а также методы-свойства для их получения и установки. Также определите методы `harvest()`, `start_engine()` и `unload_grain()`.
- (d) Создайте класс `Sprayer` для опрыскивателей. Определите атрибуты `tank_capacity`, `boom_width`, `pump_pressure` и `nozzle_type`, а также методы-свойства (`@property`) для их получения и установки. Также определите метод `activate_nozzles()`.
- (e) Создайте класс `AgriculturalMachine` как общий класс сельскохозяйственной техники, который наследуется от всех трёх классов (`Tractor`, `Combine`, `Sprayer`). Определите метод `__init__()`, который корректно вызывает конструкторы родительских классов.
- (f) В классе `AgriculturalMachine` определите метод `work_field()`, который будет последовательно вызывать все методы родительских классов, связанные с полевыми работами: запуск двигателя, вспашка, уборка урожая, активация форсунок и разгрузка.
- (g) В конце файла добавьте блок

```
if __name__ == "__main__":
```

чтобы протестировать ваш код. В этом блоке создайте экземпляр класса `AgriculturalMachine` и вызовите его методы, которые выводят состояние техники и значения её атрибутов.

- (h) Сохраните файл и запустите его в IDE, чтобы проверить его работу.

12 (a) Создайте новый файл с расширением `.py`.

- (b) Создайте класс `Excavator` для экскаваторов. Определите атрибуты `bucket_capacity` и `arm_reach`, а также методы-свойства (`@property`) для их получения и установки. Также определите методы `dig()` и `start_engine()`.
- (c) Создайте класс `Bulldozer` для бульдозеров. Определите атрибуты `blade_width`, `engine_power` и `ground_pressure`, а также методы-свойства для их получения и установки. Также определите методы `push_soil()`, `start_engine()` и `level_ground()`.
- (d) Создайте класс `Crane` для кранов. Определите атрибуты `max_load`, `boom_length`, `rotation_speed` и `counterweight`, а также методы-свойства (`@property`) для их получения и установки. Также определите метод `lift_load()`.
- (e) Создайте класс `ConstructionEquipment` как общий класс строительной техники, который наследуется от всех трёх классов (`Excavator`, `Bulldozer`, `Crane`). Определите метод `__init__()`, который корректно вызывает конструкторы родительских классов.
- (f) В классе `ConstructionEquipment` определите метод `build_site()`, который будет последовательно вызывать все методы родительских классов, связанные со строительством: запуск двигателя, копка, выравнивание, подъём груза и перемещение материалов.
- (g) В конце файла добавьте блок

```
if __name__ == "__main__":
```

чтобы протестировать ваш код. В этом блоке создайте экземпляр класса `ConstructionEquipment` и вызовите его методы, которые выводят состояние техники и значения её атрибутов.

(h) Сохраните файл и запустите его в IDE, чтобы проверить его работу.

13 (a) Создайте новый файл с расширением `.py`.

(b) Создайте класс `Ambulance` для машин скорой помощи. Определите атрибуты `medical_kit` и `siren_type`, а также методы-свойства (`@property`) для их получения и установки. Также определите методы `activate_siren()` и `load_patient()`.

(c) Создайте класс `FireTruck` для пожарных машин. Определите атрибуты `water_tank`, `pump_power` и `ladder_length`, а также методы-свойства для их получения и установки. Также определите методы `deploy_ladder()`, `activate_siren()` и `extinguish()`.

(d) Создайте класс `PoliceCar` для полицейских автомобилей. Определите атрибуты `radio_type`, `siren_type`, `cruiser_model` и `cage_installed`, а также методы-свойства (`@property`) для их получения и установки. Также определите метод `initiate_pursuit()`.

(e) Создайте класс `EmergencyVehicle` как общий класс служебного транспорта, который наследуется от всех трёх классов (`Ambulance`, `FireTruck`, `PoliceCar`). Определите метод `__init__()`, который корректно вызывает конструкторы родительских классов.

(f) В классе `EmergencyVehicle` определите метод `respond()`, который будет последовательно вызывать все методы родительских классов, связанные с экстренным реагированием: включение sireны, загрузка пациента, развёртывание лестницы, погоня и тушение.

(g) В конце файла добавьте блок

```
if __name__ == "__main__":
```

чтобы протестировать ваш код. В этом блоке создайте экземпляр класса `EmergencyVehicle` и вызовите его методы, которые выводят состояние транспорта и значения его атрибутов.

(h) Сохраните файл и запустите его в IDE, чтобы проверить его работу.

14 (a) Создайте новый файл с расширением `.py`.

(b) Создайте класс `Taxi` для такси. Определите атрибуты `passenger_seats` и `meter_type`, а также методы-свойства (`@property`) для их получения и установки. Также определите методы `pick_up()` и `drop_off()`.

(c) Создайте класс `RideShare` для каршеринговых автомобилей. Определите атрибуты `app_integration`, `fuel_level` и `unlock_method`, а также методы-свойства для их получения и установки. Также определите методы `unlock_car()`, `drop_off()` и `report_issue()`.

(d) Создайте класс `Limousine` для лимузинов. Определите атрибуты `interior_luxury`, `bar_installed`, `passenger_capacity` и `chauffeur_name`, а также методы-свойства (`@property`) для их получения и установки. Также определите метод `serve_champagne()`.

(e) Создайте класс `PassengerVehicle` как общий класс пассажирского транспорта, который наследуется от всех трёх классов (`Taxi`, `RideShare`, `Limousine`). Определите метод `__init__()`, который корректно вызывает конструкторы родительских классов.

- (f) В классе `PassengerVehicle` определите метод `transport_client()`, который будет последовательно вызывать все методы родительских классов, связанные с перевозкой: подбор клиента, открытие автомобиля, обслуживание и высадка.

- (g) В конце файла добавьте блок

```
if __name__ == "__main__":
```

чтобы протестировать ваш код. В этом блоке создайте экземпляр класса `PassengerVehicle` и вызовите его методы, которые выводят состояние транспорта и значения его атрибутов.

- (h) Сохраните файл и запустите его в IDE, чтобы проверить его работу.

- 15 (a) Создайте новый файл с расширением `.py`.

- (b) Создайте класс `SchoolBus` для школьных автобусов. Определите атрибуты `seat_belts` и `stop_sign`, а также методы-свойства (`@property`) для их получения и установки. Также определите методы `load_children()` и `activate_sign()`.

- (c) Создайте класс `Coach` для туристических автобусов. Определите атрибуты `toilet_installed`, `wifi_available` и `reclining_seats`, а также методы-свойства для их получения и установки. Также определите методы `depart_tour()`, `activate_sign()` и `play_audio()`.

- (d) Создайте класс `Minibus` для микроавтобусов. Определите атрибуты `door_type`, `ac_system`, `wheelchair_access` и `max_occupancy`, а также методы-свойства (`@property`) для их получения и установки. Также определите метод `open_door()`.

- (e) Создайте класс `Bus` как общий класс автобуса, который наследуется от всех трёх классов (`SchoolBus`, `Coach`, `Minibus`). Определите метод `__init__()`, который корректно вызывает конструкторы родительских классов.

- (f) В классе `Bus` определите метод `commute()`, который будет последовательно вызывать все методы родительских классов, связанные с перевозкой: загрузка пассажиров, активация знака, открытие дверей, аудиоинформирование и отправление в путь.

- (g) В конце файла добавьте блок

```
if __name__ == "__main__":
```

чтобы протестировать ваш код. В этом блоке создайте экземпляр класса `Bus` и вызовите его методы, которые выводят состояние автобуса и значения его атрибутов.

- (h) Сохраните файл и запустите его в IDE, чтобы проверить его работу.

- 16 (a) Создайте новый файл с расширением `.py`.

- (b) Создайте класс `Truck` для грузовиков. Определите атрибуты `payload_capacity` и `axle_count`, а также методы-свойства (`@property`) для их получения и установки. Также определите методы `load_freight()` и `start_engine()`.

- (c) Создайте класс `Semi` для седельных тягачей. Определите атрибуты `fifth_wheel`, `engine_torque` и `trailer_type`, а также методы-свойства для их получения и установки. Также определите методы `hook_trailer()`, `start_engine()` и `unhook_trailer()`.

- (d) Создайте класс `DumpTruck` для самосвалов. Определите атрибуты `bed_angle`, `hydraulic_pressure`, `dump_time` и `material_type`, а также методы-свойства (`@property`) для их получения и установки. Также определите метод `tip_bed()`.
- (e) Создайте класс `FreightVehicle` как общий класс грузового транспорта, который наследуется от всех трёх классов (`Truck`, `Semi`, `DumpTruck`). Определите метод `__init__()`, который корректно вызывает конструкторы родительских классов.
- (f) В классе `FreightVehicle` определите метод `haul()`, который будет последовательно вызывать все методы родительских классов, связанные с перевозкой груза: запуск двигателя, погрузка, прицепка прицепа, подъём кузова и разгрузка.
- (g) В конце файла добавьте блок

```
if __name__ == "__main__":
```

чтобы протестировать ваш код. В этом блоке создайте экземпляр класса `FreightVehicle` и вызовите его методы, которые выводят состояние транспорта и значения его атрибутов.

- (h) Сохраните файл и запустите его в IDE, чтобы проверить его работу.

17 (a) Создайте новый файл с расширением `.py`.

- (b) Создайте класс `Rocket` для ракет. Определите атрибуты `stage_count` и `fuel_type`, а также методы-свойства (`@property`) для их получения и установки. Также определите методы `ignite()` и `ascend()`.
- (c) Создайте класс `Spaceplane` для космопланов. Определите атрибуты `reentry_shield`, `orbital_speed` и `landing_gear`, а также методы-свойства для их получения и установки. Также определите методы `reenter_atmosphere()`, `ascend()` и `land()`.
- (d) Создайте класс `Lander` для посадочных модулей. Определите атрибуты `thruster_count`, `landing_legs`, `surface_type` и `cargo_mass`, а также методы-свойства (`@property`) для их получения и установки. Также определите метод `touch_down()`.
- (e) Создайте класс `Spacecraft` как общий класс космического аппарата, который наследуется от всех трёх классов (`Rocket`, `Spaceplane`, `Lander`). Определите метод `__init__()`, который корректно вызывает конструкторы родительских классов.
- (f) В классе `Spacecraft` определите метод `mission()`, который будет последовательно вызывать все методы родительских классов, связанные с космической миссией: запуск, выход на орбиту, вход в атмосферу, посадка и выгрузка груза.
- (g) В конце файла добавьте блок

```
if __name__ == "__main__":
```

чтобы протестировать ваш код. В этом блоке создайте экземпляр класса `Spacecraft` и вызовите его методы, которые выводят состояние аппарата и значения его атрибутов.

- (h) Сохраните файл и запустите его в IDE, чтобы проверить его работу.

18 (a) Создайте новый файл с расширением `.py`.

- (b) Создайте класс `Smartwatch` для умных часов. Определите атрибуты `battery_life` и `os_type`, а также методы-свойства (`@property`) для их получения и установки. Также определите методы `notify()` и `track_steps()`.

- (c) Создайте класс `FitnessTracker` для фитнес-трекеров. Определите атрибуты `heart_rate_sensor`, `water_resistance` и `sleep_analysis`, а также методы-свойства для их получения и установки. Также определите методы `monitor_hr()`, `track_steps()` и `sync_data()`.
- (d) Создайте класс `AR_Glasses` для очков дополненной реальности. Определите атрибуты `display_resolution`, `processor_type`, `fov_degrees` и `weight`, а также методы-свойства (`@property`) для их получения и установки. Также определите метод `render_overlay()`.
- (e) Создайте класс `WearableDevice` как общий класс носимой электроники, который наследуется от всех трёх классов (`Smartwatch`, `FitnessTracker`, `AR_Glasses`). Определите метод `__init__()`, который корректно вызывает конструкторы родительских классов.
- (f) В классе `WearableDevice` определите метод `operate_daily()`, который будет последовательно вызывать все методы родительских классов, связанные с использованием: уведомления, мониторинг ЧСС, синхронизация данных и отображение оверлея.
- (g) В конце файла добавьте блок

```
if __name__ == "__main__":
```

чтобы протестировать ваш код. В этом блоке создайте экземпляр класса `WearableDevice` и вызовите его методы, которые выводят состояние устройства и значения его атрибутов.

- (h) Сохраните файл и запустите его в IDE, чтобы проверить его работу.
- 19 (a) Создайте новый файл с расширением `.py`.
- (b) Создайте класс `Smartphone` для смартфонов. Определите атрибуты `screen_size` и `camera_mp`, а также методы-свойства (`@property`) для их получения и установки. Также определите методы `call()` и `take_photo()`.
 - (c) Создайте класс `Tablet` для планшетов. Определите атрибуты `stylus_support`, `battery_capacity` и `speaker_type`, а также методы-свойства для их получения и установки. Также определите методы `draw()`, `take_photo()` и `play_video()`.
 - (d) Создайте класс `Ereader` для электронных книг. Определите атрибуты `front_light`, `storage_gb`, `page_turn_time` и `glare_free`, а также методы-свойства (`@property`) для их получения и установки. Также определите метод `open_book()`.
 - (e) Создайте класс `MobileDevice` как общий класс портативного устройства, который наследуется от всех трёх классов (`Smartphone`, `Tablet`, `Ereader`). Определите метод `__init__()`, который корректно вызывает конструкторы родительских классов.
 - (f) В классе `MobileDevice` определите метод `use_device()`, который будет последовательно вызывать все методы родительских классов, связанные с использованием: звонок, съёмка фото, рисование, открытие книги и воспроизведение видео.
 - (g) В конце файла добавьте блок

```
if __name__ == "__main__":
```

чтобы протестировать ваш код. В этом блоке создайте экземпляр класса `MobileDevice` и вызовите его методы, которые выводят состояние устройства и значения его атрибутов.

(h) Сохраните файл и запустите его в IDE, чтобы проверить его работу.

20 (a) Создайте новый файл с расширением `.py`.

(b) Создайте класс `Laptop` для ноутбуков. Определите атрибуты `ram_gb` и `ssd_gb`, а также методы-свойства (`@property`) для их получения и установки. Также определите методы `boot_os()` и `run_app()`.

(c) Создайте класс `Desktop` для настольных ПК. Определите атрибуты `gpu_model`, `psu_wattage` и `case_type`, а также методы-свойства для их получения и установки. Также определите методы `power_on()`, `run_app()` и `shutdown()`.

(d) Создайте класс `Workstation` для рабочих станций. Определите атрибуты `cpu_cores`, `ecc_ram`, `raid_config` и `cooling_type`, а также методы-свойства (`@property`) для их получения и установки. Также определите метод `render_scene()`.

(e) Создайте класс `Computer` как общий класс вычислительного устройства, который наследуется от всех трёх классов (`Laptop`, `Desktop`, `Workstation`). Определите метод `__init__()`, который корректно вызывает конструкторы родительских классов.

(f) В классе `Computer` определите метод `compute()`, который будет последовательно вызывать все методы родительских классов, связанные с работой: запуск ОС, включение питания, выполнение приложения, рендеринг сцены и выключение.

(g) В конце файла добавьте блок

```
if __name__ == "__main__":
```

чтобы протестировать ваш код. В этом блоке создайте экземпляр класса `Computer` и вызовите его методы, которые выводят состояние устройства и значения его атрибутов.

(h) Сохраните файл и запустите его в IDE, чтобы проверить его работу.

21 (a) Создайте новый файл с расширением `.py`.

(b) Создайте класс `Router` для маршрутизаторов. Определите атрибуты `lan_ports` и `wifi_standard`, а также методы-свойства (`@property`) для их получения и установки. Также определите методы `connect_device()` и `broadcast_ssid()`.

(c) Создайте класс `Switch` для сетевых коммутаторов. Определите атрибуты `port_speed`, `vlan_support` и `managed`, а также методы-свойства для их получения и установки. Также определите методы `forward_packet()`, `broadcast_ssid()` и `configure_vlan()`.

(d) Создайте класс `Firewall` для сетевых экранов. Определите атрибуты `rules_count`, `inspection_type`, `throughput_mbps` и `log_traffic`, а также методы-свойства (`@property`) для их получения и установки. Также определите метод `block_intrusion()`.

(e) Создайте класс `NetworkDevice` как общий класс сетевого оборудования, который наследуется от всех трёх классов (`Router`, `Switch`, `Firewall`). Определите метод `__init__()`, который корректно вызывает конструкторы родительских классов.

- (f) В классе **NetworkDevice** определите метод **handle_traffic()**, который будет последовательно вызывать все методы родительских классов, связанные с обработкой трафика: подключение устройств, передача пакетов, трансляция SSID, настройка VLAN и блокировка угроз.

- (g) В конце файла добавьте блок

```
if __name__ == "__main__":
```

чтобы протестировать ваш код. В этом блоке создайте экземпляр класса **NetworkDevice** и вызовите его методы, которые выводят состояние устройства и значения его атрибутов.

- (h) Сохраните файл и запустите его в IDE, чтобы проверить его работу.

- 22 (a) Создайте новый файл с расширением **.py**.

- (b) Создайте класс **Printer** для принтеров. Определите атрибуты **ink_type** и **dpi_resolution**, а также методы-свойства (**@property**) для их получения и установки. Также определите методы **load_paper()** и **print_document()**.

- (c) Создайте класс **Scanner** для сканеров. Определите атрибуты **color_depth**, **scan_speed** и **adf_installed**, а также методы-свойства для их получения и установки. Также определите методы **scan_page()**, **print_document()** и **save_pdf()**.

- (d) Создайте класс **Copier** для копировальных аппаратов. Определите атрибуты **duplex**, **tray_capacity**, **warmup_time** и **copy_quality**, а также методы-свойства (**@property**) для их получения и установки. Также определите метод **make_copies()**.

- (e) Создайте класс **OfficeDevice** как общий класс офисной техники, который наследуется от всех трёх классов (**Printer**, **Scanner**, **Copier**). Определите метод **__init__()**, который корректно вызывает конструкторы родительских классов.

- (f) В классе **OfficeDevice** определите метод **process_document()**, который будет последовательно вызывать все методы родительских классов, связанные с обработкой документов: загрузка бумаги, сканирование, печать, сохранение PDF и копирование.

- (g) В конце файла добавьте блок

```
if __name__ == "__main__":
```

чтобы протестировать ваш код. В этом блоке создайте экземпляр класса **OfficeDevice** и вызовите его методы, которые выводят состояние устройства и значения его атрибутов.

- (h) Сохраните файл и запустите его в IDE, чтобы проверить его работу.

- 23 (a) Создайте новый файл с расширением **.py**.

- (b) Создайте класс **Camera** для цифровых фотоаппаратов. Определите атрибуты **sensor_size** и **lens_mount**, а также методы-свойства (**@property**) для их получения и установки. Также определите методы **focus()** и **capture()**.

- (c) Создайте класс **Camcorder** для видеокамер. Определите атрибуты **video_format**, **zoom_range** и **mic_input**, а также методы-свойства для их получения и установки. Также определите методы **record_video()**, **capture()** и **stop_recording()**.

- (d) Создайте класс `DroneCam` для камер дронов. Определите атрибуты `gimbal_axes`, `live_feed`, `bitrate_mbps` и `stabilization`, а также методы-свойства (`@property`) для их получения и установки. Также определите метод `stream_video()`.
- (e) Создайте класс `ImagingDevice` как общий класс устройства захвата изображения, который наследуется от всех трёх классов (`Camera`, `Camcorder`, `DroneCam`). Определите метод `__init__()`, который корректно вызывает конструкторы родительских классов.
- (f) В классе `ImagingDevice` определите метод `shoot()`, который будет последовательно вызывать все методы родительских классов, связанные с записью: фокусировка, съёмка фото, запись и остановка видео, трансляция потока.
- (g) В конце файла добавьте блок

```
if __name__ == "__main__":
```

чтобы протестировать ваш код. В этом блоке создайте экземпляр класса `ImagingDevice` и вызовите его методы, которые выводят состояние устройства и значения его атрибутов.

- (h) Сохраните файл и запустите его в IDE, чтобы проверить его работу.

24 (a) Создайте новый файл с расширением `.py`.

- (b) Создайте класс `Microwave` для микроволновых печей. Определите атрибуты `power_watt` и `turntable`, а также методы-свойства (`@property`) для их получения и установки. Также определите методы `set_timer()` и `start_heating()`.
- (c) Создайте класс `Oven` для духовых шкафов. Определите атрибуты `convection`, `max_temp` и `self_clean`, а также методы-свойства для их получения и установки. Также определите методы `preheat()`, `start_heating()` и `turn_off()`.
- (d) Создайте класс `AirFryer` для аэрогрилей. Определите атрибуты `basket_size`, `temp_range`, `digital_panel` и `auto_shutoff`, а также методы-свойства (`@property`) для их получения и установки. Также определите метод `cook_crispy()`.
- (e) Создайте класс `CookingAppliance` как общий класс кухонной техники, который наследуется от всех трёх классов (`Microwave`, `Oven`, `AirFryer`). Определите метод `__init__()`, который корректно вызывает конструкторы родительских классов.
- (f) В классе `CookingAppliance` определите метод `prepare_meal()`, который будет последовательно вызывать все методы родительских классов, связанные с приготовлением: установка таймера, предварительный нагрев, хрустящая готовка и отключение.
- (g) В конце файла добавьте блок

```
if __name__ == "__main__":
```

чтобы протестировать ваш код. В этом блоке создайте экземпляр класса `CookingAppliance` и вызовите его методы, которые выводят состояние прибора и значения его атрибутов.

- (h) Сохраните файл и запустите его в IDE, чтобы проверить его работу.

25 (a) Создайте новый файл с расширением `.py`.

- (b) Создайте класс `Refrigerator` для холодильников. Определите атрибуты `capacity_liters` и `freezer_type`, а также методы-свойства (`@property`) для их получения и установки. Также определите методы `cool_down()` и `store_food()`.
- (c) Создайте класс `Freezer` для морозильных камер. Определите атрибуты `temp_min`, `defrost_type` и `energy_class`, а также методы-свойства для их получения и установки. Также определите методы `freeze_items()`, `store_food()` и `alarm_temp()`.
- (d) Создайте класс `MiniFridge` для мини-холодильников. Определите атрибуты `door_type`, `noise_level`, `portable` и `can_holder`, а также методы-свойства (`@property`) для их получения и установки. Также определите метод `chill_beverage()`.
- (e) Создайте класс `CoolingAppliance` как общий класс охлаждающей техники, который наследуется от всех трёх классов (`Refrigerator`, `Freezer`, `MiniFridge`). Определите метод `__init__()`, который корректно вызывает конструкторы родительских классов.
- (f) В классе `CoolingAppliance` определите метод `preserve()`, который будет последовательно вызывать все методы родительских классов, связанные с хранением: охлаждение, заморозка, сигнализация, охлаждение напитков и размещение продуктов.
- (g) В конце файла добавьте блок

```
if __name__ == "__main__":
```

чтобы протестировать ваш код. В этом блоке создайте экземпляр класса `CoolingAppliance` и вызовите его методы, которые выводят состояние прибора и значения его атрибутов.

- (h) Сохраните файл и запустите его в IDE, чтобы проверить его работу.

- 26
- (a) Создайте новый файл с расширением `.py`.
 - (b) Создайте класс `WashingMachine` для стиральных машин. Определите атрибуты `drum_size` и `spin_rpm`, а также методы-свойства (`@property`) для их получения и установки. Также определите методы `load_laundry()` и `start_cycle()`.
 - (c) Создайте класс `Dryer` для сушилок. Определите атрибуты `heat_type`, `capacity_kg` и `sensor_dry`, а также методы-свойства для их получения и установки. Также определите методы `dry_clothes()`, `start_cycle()` и `cool_down()`.
 - (d) Создайте класс `Iron` для утюгов. Определите атрибуты `steam_output`, `soleplate_material`, `auto_off` и `vertical_steam`, а также методы-свойства (`@property`) для их получения и установки. Также определите метод `press_fabric()`.
 - (e) Создайте класс `LaundryAppliance` как общий класс техники для стирки, который наследуется от всех трёх классов (`WashingMachine`, `Dryer`, `Iron`). Определите метод `__init__()`, который корректно вызывает конструкторы родительских классов.
 - (f) В классе `LaundryAppliance` определите метод `clean_clothes()`, который будет последовательно вызывать все методы родительских классов, связанные с уходом за одеждой: загрузка белья, стирка, сушка, глажка и остывание.
 - (g) В конце файла добавьте блок

```
if __name__ == "__main__":
```

чтобы протестировать ваш код. В этом блоке создайте экземпляр класса `LaundryAppliance` и вызовите его методы, которые выводят состояние прибора и значения его атрибутов.

(h) Сохраните файл и запустите его в IDE, чтобы проверить его работу.

27 (a) Создайте новый файл с расширением `.py`.

(b) Создайте класс `Vacuum` для пылесосов. Определите атрибуты `suction_power` и `filter_type`, а также методы-свойства (`@property`) для их получения и установки. Также определите методы `start_suction()` и `empty_bin()`.

(c) Создайте класс `MopRobot` для роботов-мойщиков. Определите атрибуты `water_tank`, `navigation_type` и `pad_material`, а также методы-свойства для их получения и установки. Также определите методы `mop_floor()`, `empty_bin()` и `recharge()`.

(d) Создайте класс `SteamCleaner` для пароочистителей. Определите атрибуты `pressure_bar`, `steam_time`, `nozzle_types` и `tank_material`, а также методы-свойства (`@property`) для их получения и установки. Также определите метод `generate_steam()`.

(e) Создайте класс `CleaningDevice` как общий класс уборочной техники, который наследуется от всех трёх классов (`Vacuum`, `MopRobot`, `SteamCleaner`). Определите метод `__init__()`, который корректно вызывает конструкторы родительских классов.

(f) В классе `CleaningDevice` определите метод `sanitize()`, который будет последовательно вызывать все методы родительских классов, связанные с уборкой: всасывание, мойка пола, паровая очистка, опорожнение контейнера и подзарядка.

(g) В конце файла добавьте блок

```
if __name__ == "__main__":
```

чтобы протестировать ваш код. В этом блоке создайте экземпляр класса `CleaningDevice` и вызовите его методы, которые выводят состояние прибора и значения его атрибутов.

(h) Сохраните файл и запустите его в IDE, чтобы проверить его работу.

28 (a) Создайте новый файл с расширением `.py`.

(b) Создайте класс `Thermostat` для термостатов. Определите атрибуты `temp_range` и `wifi_enabled`, а также методы-свойства (`@property`) для их получения и установки. Также определите методы `set_target()` и `read_current()`.

(c) Создайте класс `Humidifier` для увлажнителей. Определите атрибуты `tank_liters`, `mist_type` и `auto_shutoff`, а также методы-свойства для их получения и установки. Также определите методы `emit_mist()`, `read_current()` и `refill_tank()`.

(d) Создайте класс `AirPurifier` для очистителей воздуха. Определите атрибуты `filter_hera`, `cadr_rating`, `noise_db` и `timer_hours`, а также методы-свойства (`@property`) для их получения и установки. Также определите метод `filter_air()`.

(e) Создайте класс `ClimateDevice` как общий класс климатической техники, который наследуется от всех трёх классов (`Thermostat`, `Humidifier`, `AirPurifier`). Определите метод `__init__()`, который корректно вызывает конструкторы родительских классов.

- (f) В классе `ClimateDevice` определите метод `adjust_indoor()`, который будет последовательно вызывать все методы родительских классов, связанные с микроклиматом: установка температуры, увлажнение, очистка воздуха, считывание параметров и пополнение резервуаров.

- (g) В конце файла добавьте блок

```
if __name__ == "__main__":
```

чтобы протестировать ваш код. В этом блоке создайте экземпляр класса `ClimateDevice` и вызовите его методы, которые выводят состояние прибора и значения его атрибутов.

- (h) Сохраните файл и запустите его в IDE, чтобы проверить его работу.

- 29 (a) Создайте новый файл с расширением `.py`.

- (b) Создайте класс `SmartLock` для умных замков. Определите атрибуты `unlock_method` и `battery_life`, а также методы-свойства (`@property`) для их получения и установки. Также определите методы `lock_door()` и `grant_access()`.

- (c) Создайте класс `SecurityCamera` для камер видеонаблюдения. Определите атрибуты `night_vision`, `motion_detect` и `cloud_storage`, а также методы-свойства для их получения и установки. Также определите методы `record_footage()`, `grant_access()` и `send_alert()`.

- (d) Создайте класс `AlarmSystem` для сигнализаций. Определите атрибуты `siren_db`, `zone_count`, `panic_button` и `armed_modes`, а также методы-свойства (`@property`) для их получения и установки. Также определите метод `trigger_alarm()`.

- (e) Создайте класс `SecurityDevice` как общий класс системы безопасности, который наследуется от всех трёх классов (`SmartLock`, `SecurityCamera`, `AlarmSystem`). Определите метод `__init__()`, который корректно вызывает конструкторы родительских классов.

- (f) В классе `SecurityDevice` определите метод `protect_home()`, который будет последовательно вызывать все методы родительских классов, связанные с безопасностью: блокировка двери, предоставление доступа, запись видео, отправка оповещений и активация sireны.

- (g) В конце файла добавьте блок

```
if __name__ == "__main__":
```

чтобы протестировать ваш код. В этом блоке создайте экземпляр класса `SecurityDevice` и вызовите его методы, которые выводят состояние устройства и значения его атрибутов.

- (h) Сохраните файл и запустите его в IDE, чтобы проверить его работу.

- 30 (a) Создайте новый файл с расширением `.py`.

- (b) Создайте класс `SmartBulb` для умных ламп. Определите атрибуты `color_temp` и `lumens`, а также методы-свойства (`@property`) для их получения и установки. Также определите методы `turn_on()` и `set_color()`.

- (c) Создайте класс `SmartPlug` для умных розеток. Определите атрибуты `max_wattage`, `energy_monitor` и `scheduling`, а также методы-свойства для их получения и установки. Также определите методы `power_device()`, `set_color()` и `log_usage()`.
- (d) Создайте класс `LightStrip` для светодиодных лент. Определите атрибуты `length_m`, `led_density`, `music_sync` и `waterproof`, а также методы-свойства (`@property`) для их получения и установки. Также определите метод `animate_pattern()`.
- (e) Создайте класс `LightingSystem` как общий класс осветительного оборудования, который наследуется от всех трёх классов (`SmartBulb`, `SmartPlug`, `LightStrip`). Определите метод `__init__()`, который корректно вызывает конструкторы родительских классов.
- (f) В классе `LightingSystem` определите метод `illuminate()`, который будет последовательно вызывать все методы родительских классов, связанные с освещением: включение, установка цвета, анимация, подача питания и учёт потребления.
- (g) В конце файла добавьте блок

```
if __name__ == "__main__":
```

чтобы протестировать ваш код. В этом блоке создайте экземпляр класса `LightingSystem` и вызовите его методы, которые выводят состояние системы и значения её атрибутов.

- (h) Сохраните файл и запустите его в IDE, чтобы проверить его работу.

31 (a) Создайте новый файл с расширением `.py`.

- (b) Создайте класс `GameConsole` для игровых приставок. Определите атрибуты `gpu_tflops` и `storage_tb`, а также методы-свойства (`@property`) для их получения и установки. Также определите методы `boot_system()` и `launch_game()`.
- (c) Создайте класс `Handheld` для портативных консолей. Определите атрибуты `screen_refresh`, `battery_hours` и `cartridge_slot`, а также методы-свойства для их получения и установки. Также определите методы `play_portable()`, `launch_game()` и `sleep_mode()`.
- (d) Создайте класс `VR_Headset` для VR-шлемов. Определите атрибуты `fov_degrees`, `resolution_per_eye`, `refresh_rate` и `inside_out_tracking`, а также методы-свойства (`@property`) для их получения и установки. Также определите метод `enter_vr()`.
- (e) Создайте класс `GamingDevice` как общий класс игрового оборудования, который наследуется от всех трёх классов (`GameConsole`, `Handheld`, `VR_Headset`). Определите метод `__init__()`, который корректно вызывает конструкторы родительских классов.
- (f) В классе `GamingDevice` определите метод `play()`, который будет последовательно вызывать все методы родительских классов, связанные с игрой: загрузка системы, запуск игры, вход в VR, портативная сессия и переход в спящий режим.
- (g) В конце файла добавьте блок

```
if __name__ == "__main__":
```

чтобы протестировать ваш код. В этом блоке создайте экземпляр класса `GamingDevice` и вызовите его методы, которые выводят состояние устройства и значения его атрибутов.

- (h) Сохраните файл и запустите его в IDE, чтобы проверить его работу.
- 32 (a) Создайте новый файл с расширением `.py`.
- (b) Создайте класс `ElectricGuitar` для электрогитар. Определите атрибуты `pickup_type` и `body_wood`, а также методы-свойства (`@property`) для их получения и установки. Также определите методы `tune_strings()` и `strum()`.
- (c) Создайте класс `Synthesizer` для синтезаторов. Определите атрибуты `polyphony`, `oscillator_count` и `patch_memory`, а также методы-свойства для их получения и установки. Также определите методы `select_preset()`, `strum()` и `modulate()`.
- (d) Создайте класс `DrumMachine` для драм-машин. Определите атрибуты `sample_quality`, `step_sequencer`, `pad_sensitivity` и `midi_out`, а также методы-свойства (`@property`) для их получения и установки. Также определите метод `program_beat()`.
- (e) Создайте класс `ElectronicInstrument` как общий класс электронных музыкальных инструментов, который наследуется от всех трёх классов (`ElectricGuitar`, `Synthesizer`, `DrumMachine`). Определите метод `__init__()`, который корректно вызывает конструкторы родительских классов.
- (f) В классе `ElectronicInstrument` определите метод `perform()`, который будет последовательно вызывать все методы родительских классов, связанные с исполнением: настройка струн, выбор пресета, программирование бита, модуляция и игра.
- (g) В конце файла добавьте блок
- ```
if __name__ == "__main__":
```
- чтобы протестировать ваш код. В этом блоке создайте экземпляр класса `ElectronicInstrument` и вызовите его методы, которые выводят состояние инструмента и значения его атрибутов.
- (h) Сохраните файл и запустите его в IDE, чтобы проверить его работу.
- 33 (a) Создайте новый файл с расширением `.py`.
- (b) Создайте класс `SmartThermostat` для умных термостатов. Определите атрибуты `learning_mode` и `geofencing`, а также методы-свойства (`@property`) для их получения и установки. Также определите методы `learn_schedule()` и `adjust_temp()`.
- (c) Создайте класс `SmartBlinds` для умных жалюзи. Определите атрибуты `motor_type`, `solar_powered` и `app_control`, а также методы-свойства для их получения и установки. Также определите методы `open_blinds()`, `adjust_temp()` и `sync_sun()`.
- (d) Создайте класс `SmartSprinkler` для умных поливальных систем. Определите атрибуты `zone_count`, `weather_sensor`, `water_usage` и `soil_moisture`, а также методы-свойства (`@property`) для их получения и установки. Также определите метод `water_lawn()`.
- (e) Создайте класс `SmartHomeSystem` как общий класс системы умного дома, который наследуется от всех трёх классов (`SmartThermostat`, `SmartBlinds`, `SmartSprinkler`). Определите метод `__init__()`, который корректно вызывает конструкторы родительских классов.
- (f) В классе `SmartHomeSystem` определите метод `automate_house()`, который будет последовательно вызывать все методы родительских классов, связанные с автоматизацией: обучение расписанию, открытие жалюзи, синхронизация с солнцем, полив газона и регулировка температуры.

- (g) В конце файла добавьте блок

```
if __name__ == "__main__":
```

чтобы протестировать ваш код. В этом блоке создайте экземпляр класса `SmartHomeSystem` и вызовите его методы, которые выводят состояние системы и значения её атрибутов.

- (h) Сохраните файл и запустите его в IDE, чтобы проверить его работу.

- 34 (a) Создайте новый файл с расширением `.py`.

- (b) Создайте класс `ElectricScooter` для электросамокатов. Определите атрибуты `max_speed` и `foldable`, а также методы-свойства (`@property`) для их получения и установки. Также определите методы `unlock_ride()` и `accelerate()`.

- (c) Создайте класс `Segway` для сигвеев. Определите атрибуты `balance_type`, `battery_life` и `led_lights`, а также методы-свойства для их получения и установки. Также определите методы `self_balance()`, `accelerate()` и `park_mode()`.

- (d) Создайте класс `Hoverboard` для хOVERбордов. Определите атрибуты `wheel_size`, `max_incline`, `bluetooth_speaker` и `auto_shutdown`, а также методы-свойства (`@property`) для их получения и установки. Также определите метод `activate_music()`.

- (e) Создайте класс `PersonalTransport` как общий класс персонального электротранспорта, который наследуется от всех трёх классов (`ElectricScooter`, `Segway`, `Hoverboard`). Определите метод `__init__()`, который корректно вызывает конструкторы родительских классов.

- (f) В классе `PersonalTransport` определите метод `commute_short()`, который будет последовательно вызывать все методы родительских классов, связанные с поездкой: разблокировка, балансировка, ускорение, включение музыки и парковка.

- (g) В конце файла добавьте блок

```
if __name__ == "__main__":
```

чтобы протестировать ваш код. В этом блоке создайте экземпляр класса `PersonalTransport` и вызовите его методы, которые выводят состояние транспорта и значения его атрибутов.

- (h) Сохраните файл и запустите его в IDE, чтобы проверить его работу.

- 35 (a) Создайте новый файл с расширением `.py`.

- (b) Создайте класс `CoffeeMachine` для кофемашин. Определите атрибуты `bean_type` и `milk_frother`, а также методы-свойства (`@property`) для их получения и установки. Также определите методы `grind_beans()` и `brew_coffee()`.

- (c) Создайте класс `Teapot` для электрочайников. Определите атрибуты `temp_control`, `keep_warm` и `material`, а также методы-свойства для их получения и установки. Также определите методы `boil_water()`, `brew_coffee()` и `auto_shutoff()`.

- (d) Создайте класс `Juicer` для соковыжималок. Определите атрибуты `rpm_speed`, `pulp_control`, `feed_chute_size` и `dual_blade`, а также методы-свойства (`@property`) для их получения и установки. Также определите метод `extract_juice()`.

- (e) Создайте класс `BeverageAppliance` как общий класс техники для напитков, который наследуется от всех трёх классов (`CoffeeMachine`, `Teapot`, `Juicer`). Определите метод `__init__()`, который корректно вызывает конструкторы родительских классов.
  - (f) В классе `BeverageAppliance` определите метод `prepare_drink()`, который будет последовательно вызывать все методы родительских классов, связанные с приготовлением напитков: помол зёрен, кипячение воды, экстракция сока, заваривание кофе и автоматическое отключение.
  - (g) В конце файла добавьте блок
- ```
if __name__ == "__main__":
```

чтобы протестировать ваш код. В этом блоке создайте экземпляр класса `BeverageAppliance` и вызовите его методы, которые выводят состояние прибора и значения его атрибутов.

- (h) Сохраните файл и запустите его в IDE, чтобы проверить его работу.

2.9.4 Задача 4

Измените код программы по задаче 3 так, чтобы в дочернем классе вызов инициализаторов всех трёх наследуемых классов производился через одну функцию `super().__init__`.

2.9.5 Задача 5

Измените код программы по задаче 4 так, чтобы программа реализовывала ромбовидную структуру множественного наследования (как в задаче 1) и в дочернем классе вызов инициализаторов всех трёх наследуемых классов производился через одну функцию `super().__init__`.

2.10 Семинар «Классы-миксины и множественное наследование» (2 часа)

Классы-миксины

Классы-миксины или метод подмешивания классов в основную иерархию классов во множественном наследовании ООП представляют собой простые классы, которые включают набор методов, предназначенных для добавления к другому классу, и позволяют расширять функциональность классов без глубокой иерархии наследования. Это устраняет проблемы, связанные с множественным наследованием, и делает миксины гибким средством для улучшения и модификации структуры кода. Миксины создаются для того, чтобы повторно использовать функции во множестве классов. Они не предполагают инициализацию объектов или метод `__init__` и не должны иметь своего состояния.

1 Задание 1

Написать класс-миксин на Python, который моделирует работу автомобильной климатической сплит-системы. Создать класс-миксин `SplitSystem`, который будет содержать методы для включения и выключения системы, установки температуры охлаждения, а также для переключения между режимами охлаждения и обогрева.

- (a) Класс `SplitSystem` определяет объект, который моделирует сплит-систему.
- (b) Метод `turn_on` включает сплит-систему, если она выключена, устанавливая начальные параметры: `is_on`, `mode` и `temperature`, и устанавливает режим в `"cooling"`.
- (c) Метод `turn_off` выключает сплит-систему, если она включена.
- (d) Метод `set_cooling_temperature` устанавливает температуру охлаждения, если сплит-система включена и находится в режиме `"cooling"`.
- (e) Метод `turn_on_cooling` включает режим охлаждения, если сплит-система выключена, или меняет режим на `"cooling"`, если система уже включена, но находится в другом режиме.
- (f) Метод `turn_off_cooling` выключает режим охлаждения, если сплит-система включена и находится в режиме `"cooling"`.
- (g) Метод `turn_on_heating` включает режим обогрева, если сплит-система выключена, или меняет режим на `"heating"`, если система уже включена, но находится в другом режиме.
- (h) Метод `turn_off_heating` выключает режим обогрева, если сплит-система включена и находится в режиме `"heating"`.
- (i) В примере использования класса создаётся объект `split_system`, после чего выполняются различные команды для управления сплит-системой.

Задание 2

Написать класс-миксин на Python, который моделирует работу автомобильного радио.

- (a) Создать класс-миксин `Radio`, который будет представлять радиоприёмник.

- (b) Определить метод `turn_on(self)`, который включает радиоприёмник. В методе установить переменную `self.is_on = False`, так как по умолчанию радио выключено. Также установить начальную станцию на 0. Если `self.is_on` было `False`, поменять его на `True` и вывести сообщение, что радио включено.
- (c) Определить метод `turn_off(self)`, который выключает радиоприёмник. Если радио включено, поменять `self.is_on` на `False` и вывести сообщение о выключении.
- (d) Определить метод `tune_to_station(self, station_number)`, который настраивает радио на нужную станцию. Если радио включено, установить `self.station = station_number` и вывести сообщение о настройке. Если радио выключено, вывести сообщение о необходимости сначала включить радио.
- (e) Создать экземпляр класса `Radio` и сохранить его в переменной `radio`.
- (f) Вызвать метод `turn_on()` для экземпляра `radio`.
- (g) Вызвать метод `tune_to_station(101.5)` для настройки на станцию 101.5.
- (h) Вызвать метод `turn_off()` для выключения радио.

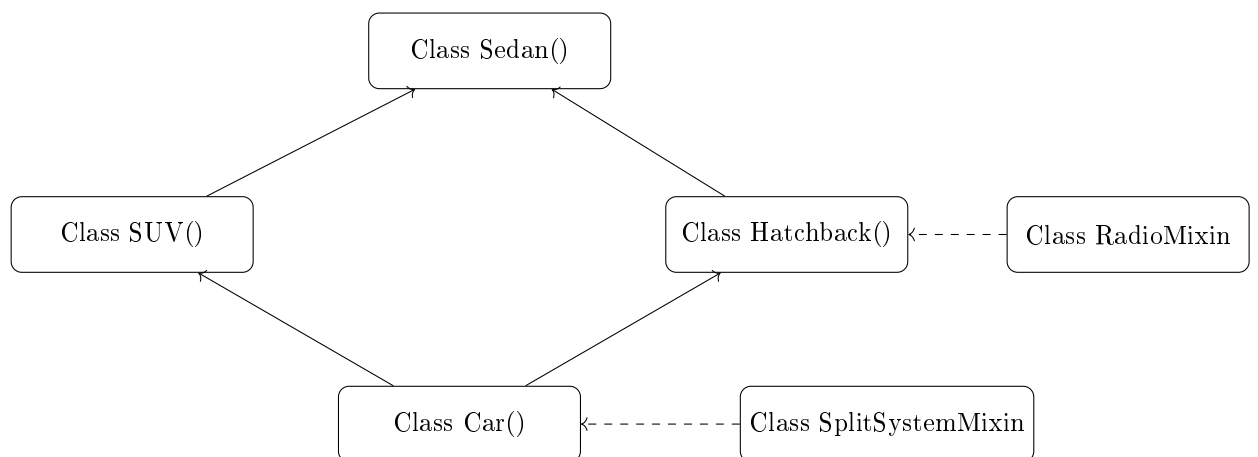
Задание 3

Используя программный код задания 5 из прошлого практикума текущего курса ООП, выполнить подмешивание класса-миксина `Radio` к классу `Hatchback(Sedan)` и подмешивание класса-миксина `SplitSystem` к классу `Car(SUV, Hatchback)`. При запуске класса `Car` должна быть отображена в консоли следующая последовательность действий.

Пример вывода программы

```
Запуск двигателя
Переключение трансмиссии
Ускорение
Сплит-система включена.
Температура охлаждения установлена на 22°C.
Режим охлаждения выключен.
Режим обогрева включен.
Радио включено.
Настроено на радиостанцию 101.5.
Режим обогрева выключен.
Радио выключено.
Торможение
Doors: 4, Engine type: 567-BAП-02, Cargo space: 5, Transmission: АВТОМАТ
```

Схема классов



2 Задание 1

Написать класс-миксин на Python, который моделирует работу системы рекуперативного торможения электромобиля. Создать класс-миксин **RegenerativeBraking**, который будет содержать методы для включения и выключения рекуперации, установки уровня рекуперации и отображения текущего уровня заряда аккумулятора.

- Класс **RegenerativeBraking** определяет объект, который моделирует систему рекуперативного торможения.
- Метод **enable_regen** включает рекуперацию, если она выключена, устанавливая начальные параметры: **is_enabled**, **regen_level** и **battery_level**, и устанавливает уровень рекуперации на 2.
- Метод **disable_regen** выключает рекуперацию, если она включена.
- Метод **set_regen_level** устанавливает уровень рекуперации от 1 до 3, если система включена.
- Метод **show_battery** выводит текущий уровень заряда аккумулятора в процентах.
- Метод **simulate_braking** имитирует торможение с рекуперацией, увеличивая уровень батареи на величину, зависящую от уровня рекуперации.
- В примере использования класса создаётся объект **regen**, после чего выполняются различные команды для управления системой.

Задание 2

Написать класс-миксин на Python, который моделирует работу режима «Эко» в гибридном автомобиле.

- Создать класс-миксин **EcoMode**, который будет представлять режим энергосбережения.
- Определить метод **activate_eco(self)**, который активирует режим «Эко». По умолчанию **self.eco_active = False**. Если режим неактивен, установить **True** и вывести сообщение об активации.

- (c) Определить метод `deactivate_eco(self)`, который деактивирует режим. Если активен, установить `False` и вывести сообщение.
- (d) Определить метод `optimize_consumption(self)`, который оптимизирует расход топлива и энергии, если режим активен, и выводит соответствующее сообщение.
- (e) Создать экземпляр класса `EcoMode` и сохранить его в переменной `eco`.
- (f) Вызвать метод `activate_eco()`.
- (g) Вызвать метод `optimize_consumption()`.
- (h) Вызвать метод `deactivate_eco()`.

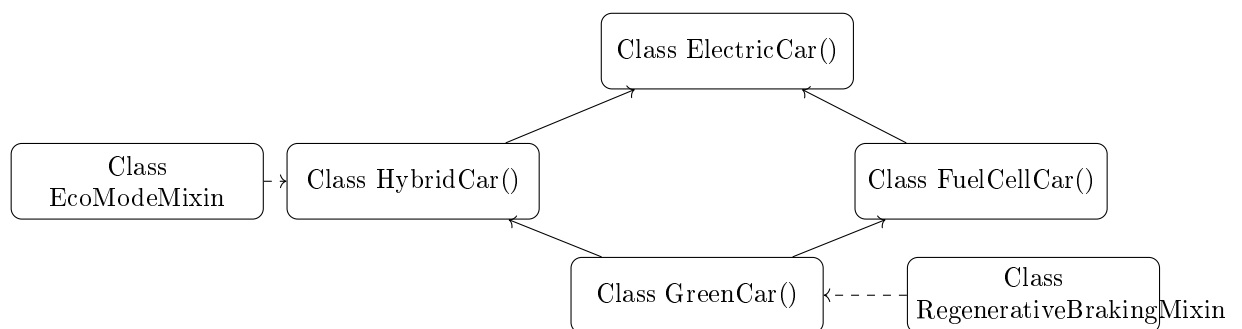
Задание 3

Используя программный код задания 2 из прошлого практикума текущего курса ООП, выполнить подмешивание класса-миксина `EcoMode` к классу `HybridCar` и подмешивание класса-миксина `RegenerativeBraking` к классу `GreenCar(ElectricCar, HybridCar, FuelCellCar)`. При запуске класса `GreenCar` должна быть отображена в консоли следующая последовательность действий.

Пример вывода программы

Режим «Эко» активирован.
 Оптимизация расхода топлива и энергии.
 Режим «Эко» деактивирован.
 Зарядка батареи
 Заправка водородом
 Переключение режимов
 Движение на электротяге
 Рекуперативное торможение включено.
 Уровень рекуперации: 3
 Уровень батареи: 85%
 Battery capacity: 75, Motor type: AC, Fuel tank size: 45, Hydrogen tank: 5 kg

Схема классов



3 Задание 1

Написать класс-миксин на Python, который моделирует работу системы контроля груза в коммерческом транспорте. Создать класс-миксин `CargoMonitor`, который будет содержать методы для проверки веса груза, блокировки дверей при превышении лимита и отправки уведомлений о состоянии груза.

- (a) Класс `CargoMonitor` определяет объект, который моделирует систему мониторинга груза.
- (b) Метод `load_check` проверяет, не превышен ли максимальный вес груза, и устанавливает флаг `overloaded`.
- (c) Метод `lock_doors` блокирует задние двери, если груз превышает лимит.
- (d) Метод `unlock_doors` разблокирует двери, если груз в пределах нормы.
- (e) Метод `send_alert` отправляет уведомление оператору при перегрузке.
- (f) Метод `reset_monitor` сбрасывает состояние системы после разгрузки.
- (g) В примере использования создаётся объект `monitor`, после чего вызываются методы для проверки и управления.

Задание 2

Написать класс-миксин на Python, который моделирует работу системы климат-контроля в минивэне.

- (a) Создать класс-миксин `ClimateZones`, который будет представлять многозонный климат-контроль.
- (b) Определить метод `activate_front(self)`, который включает климат в передней зоне.
- (c) Определить метод `activate_rear(self)`, который включает климат в задней зоне.
- (d) Определить метод `set_temperature(self, zone, temp)`, который устанавливает температуру для указанной зоны ("front" или "rear").
- (e) Создать экземпляр класса `ClimateZones` и сохранить его в переменной `climate`.
- (f) Вызвать метод `activate_front()`.
- (g) Вызвать метод `set_temperature("rear 23)`.
- (h) Вызвать метод `activate_rear()`.

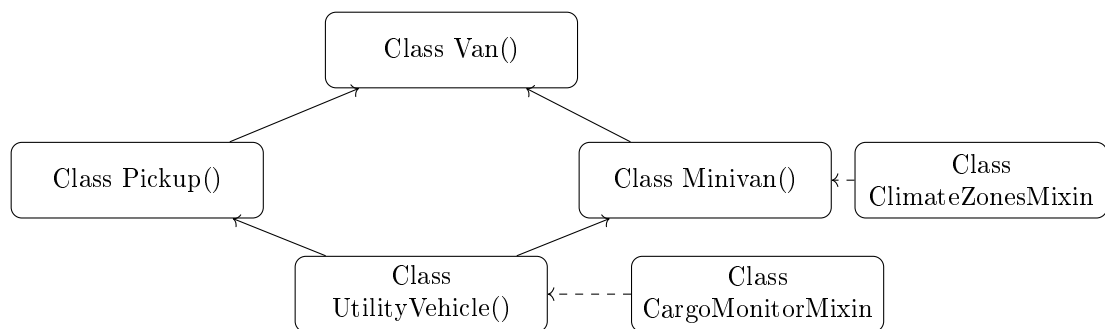
Задание 3

Используя программный код задания 3 из прошлого практикума текущего курса ООП, выполнить подмешивание класса-миксина `ClimateZones` к классу `Minivan` и подмешивание класса-миксина `CargoMonitor` к классу `UtilityVehicle` (`Van`, `Pickup`, `Minivan`). При запуске класса `UtilityVehicle` должна быть отображена в консоли следующая последовательность действий.

Пример вывода программы

Климат в передней зоне включён.
Температура в зоне rear установлена на 23°C.
Климат в задней зоне включён.
Загрузка груза
Проверка веса: перегрузка!
Двери заблокированы.
Уведомление отправлено.
Запуск двигателя
Прицепка прицепа
Разгрузка завершена. Состояние сброшено.
Cargo volume: 10 m³, Cargo bed size: 2.5 m, Seats: 7, AC system: Dual-zone

Схема классов



4 Задание 1

Написать класс-миксин на Python, который моделирует работу системы динамической стабилизации спортивного автомобиля. Создать класс-миксин `StabilityControl`, который будет содержать методы для включения и выключения системы, регулировки чувствительности и вывода статуса системы.

- (a) Класс `StabilityControl` определяет объект, который моделирует систему динамической стабилизации.
- (b) Метод `enable_dsc` включает систему стабилизации.
- (c) Метод `disable_dsc` отключает систему (например, для дрифта).
- (d) Метод `set_sensitivity` устанавливает уровень чувствительности от 1 до 5.
- (e) Метод `status_report` выводит текущее состояние системы.
- (f) В примере использования создаётся объект `dsc`, после чего вызываются методы для настройки.

Задание 2

Написать класс-миксин на Python, который моделирует работу системы управления открытым верхом кабриолета.

- (a) Создать класс-миксин `RoofManager`, который будет управлять крышей кабриолета.
- (b) Определить метод `open_roof(self)`, который открывает крышу, если скорость автомобиля ниже 50 км/ч.
- (c) Определить метод `close_roof(self)`, который закрывает крышу.
- (d) Определить метод `check_speed(self, speed)`, который проверяет допустимость открытия крыши.
- (e) Создать экземпляр класса `RoofManager` и сохранить его в переменной `roof`.
- (f) Вызвать метод `check_speed(40)`.
- (g) Вызвать метод `open_roof()`.
- (h) Вызвать метод `close_roof()`.

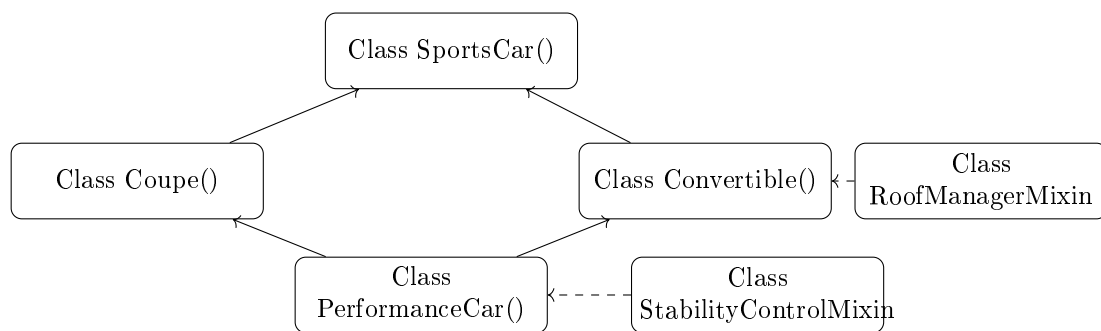
Задание 3

Используя программный код задания 4 из прошлого практикума текущего курса ООП, выполнить подмешивание класса-миксина `RoofManager` к классу `Convertible` и подмешивание класса-миксина `StabilityControl` к классу `PerformanceCar(SportsCar, Coupe, Convertible)`. При запуске класса `PerformanceCar` должна быть отображена в консоли следующая последовательность действий.

Пример вывода программы

```
Скорость 40 км/ч - крышу можно открыть.  
Крыша открыта.  
Крыша закрыта.  
Старт с контролем сцепления  
Аэродинамика активирована  
Переключение передач  
Система стабилизации включена.  
Чувствительность: 4  
Max speed: 320, Doors: 2, Roof type: Soft top, Aerodynamics: High
```

Схема классов



5 Задание 1

Написать класс-миксин на Python, который моделирует работу системы антиблокировки тормозов (ABS) для двухколёсного транспорта. Создать класс-миксин `ABS_System`, который будет содержать методы для активации ABS, имитации торможения и вывода состояния системы.

- Класс `ABS_System` определяет объект, который моделирует систему ABS.
- Метод `activate_abs` включает систему антиблокировки.
- Метод `simulate_braking` имитирует резкое торможение с активной ABS.
- Метод `abs_status` выводит текущее состояние системы.
- В примере использования создаётся объект `abs`, после чего вызываются методы.

Задание 2

Написать класс-миксин на Python, который моделирует работу подсветки приборной панели мотоцикла.

- Создать класс-миксин `DashboardLighting`, который управляет подсветкой.
- Определить метод `set_brightness(self, level)`, который устанавливает яркость от 1 до 10.
- Определить метод `enable_night_mode(self)`, который включает ночной режим (цвет подсветки — красный).
- Определить метод `disable_night_mode(self)`, который возвращает дневной режим.
- Создать экземпляр класса `DashboardLighting` и сохранить его в переменной `dash`.
- Вызвать метод `set_brightness(7)`.
- Вызвать метод `enable_night_mode()`.
- Вызвать метод `disable_night_mode()`.

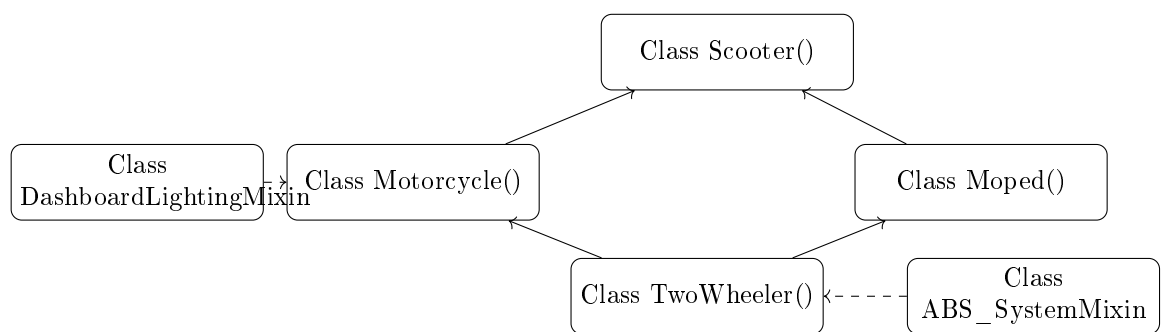
Задание 3

Используя программный код задания 5 из прошлого практикума текущего курса ООП, выполнить подмешивание класса-миксина `DashboardLighting` к классу `Motorcycle` и подмешивание класса-миксина `ABS_System` к классу `TwoWheeler(Scooter, Motorcycle, Moped)`. При запуске класса `TwoWheeler` должна быть отображена в консоли следующая последовательность действий.

Пример вывода программы

Яркость панели: 7.
Ночной режим включён.
Дневной режим восстановлен.
Включение питания
Розжиг двигателя
Нажатие педалей
Дроссель активирован
Антиблокировочная система включена.
Имитация торможения: ABS предотвратила блокировку колёс.
Battery life: 40 km, Engine displacement: 600 cc, Pedal assist: Yes

Схема классов



6 Задание 1

Написать класс-миксин на Python, который моделирует работу системы навигации для велосипедов. Создать класс-миксин **BikeNavigation**, который будет содержать методы для установки маршрута, отслеживания текущего положения и вывода указаний.

- (a) Класс **BikeNavigation** определяет объект, который моделирует велонавигатор.
- (b) Метод **set_route** устанавливает маршрут от точки А до точки В.
- (c) Метод **track_position** имитирует определение текущего положения.
- (d) Метод **give_direction** выводит следующее указание (например, «Поверните направо через 200 м»).
- (e) В примере использования создаётся объект **nav**, после чего вызываются методы.

Задание 2

Написать класс-миксин на Python, который моделирует работу антиугонной системы для электровелосипеда.

- (a) Создать класс-миксин **AntiTheft**, который будет управлять защитой от кражи.

- (b) Определить метод `lock_bike(self)`, который блокирует мотор и колёса.
- (c) Определить метод `unlock_bike(self, pin)`, который разблокирует велосипед при вводе правильного PIN.
- (d) Определить метод `trigger_alarm(self)`, который срабатывает при попытке перемещения заблокированного велосипеда.
- (e) Создать экземпляр класса `AntiTheft` и сохранить его в переменной `theft`.
- (f) Вызвать метод `lock_bike()`.
- (g) Вызвать метод `trigger_alarm()`.
- (h) Вызвать метод `unlock_bike(1234)`.

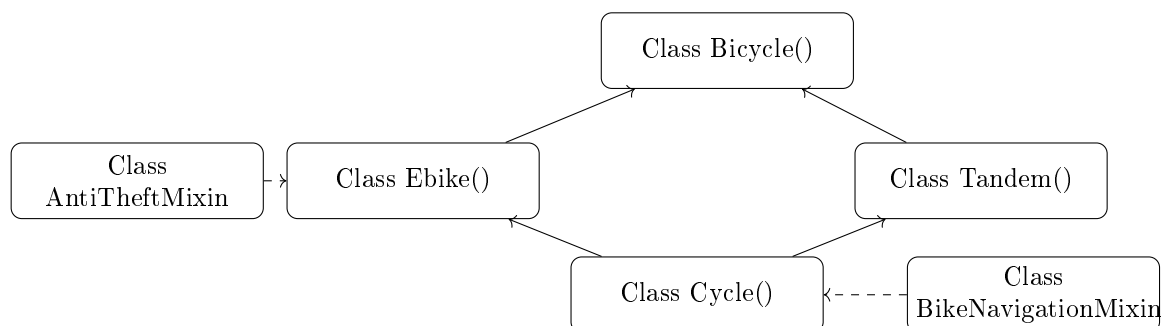
Задание 3

Используя программный код задания 6 из прошлого практикума текущего курса ООП, выполнить подмешивание класса-миксина `AntiTheft` к классу `Ebike` и подмешивание класса-миксина `BikeNavigation` к классу `Cycle(Bicycle, Ebike, Tandem)`. При запуске класса `Cycle` должна быть отображена в консоли следующая последовательность действий.

Пример вывода программы

```
Велосипед заблокирован.
Сработала сигнализация!
Велосипед разблокирован.
Маршрут установлен: от парка до станции.
Текущее положение отслеживается.
Указание: Поверните налево через 300 м.
Педалирование начато.
Координация на тандеме
Мотор включён.
Торможение выполнено.
Frame size: 56 cm, Battery capacity: 500 Wh, Rider count: 2
```

Схема классов



7 Задание 1

Написать класс-миксин на Python, который моделирует работу системы автоматического возврата дрона. Создать класс-миксин `ReturnToHome`, который будет содержать методы для установки домашней точки, активации возврата и отслеживания статуса.

- (a) Класс `ReturnToHome` определяет объект, который моделирует функцию «Вернуться домой».
- (b) Метод `set_home` устанавливает координаты домашней точки.
- (c) Метод `return_home` запускает автоматический возврат.
- (d) Метод `is_returning` выводит статус возврата.
- (e) В примере использования создаётся объект `rth`, после чего вызываются методы.

Задание 2

Написать класс-миксин на Python, который моделирует работу системы предотвращения столкновений для вертолёта.

- (a) Создать класс-миксин `CollisionAvoidance`, который будет анализировать окружение.
- (b) Определить метод `scan_obstacles(self)`, который сканирует пространство.
- (c) Определить метод `alert_pilot(self)`, который предупреждает пилота об опасности.
- (d) Определить метод `auto_maneuver(self)`, который автоматически уводит аппарат от препятствия.
- (e) Создать экземпляр класса `CollisionAvoidance` и сохранить его в переменной `avoid`.
- (f) Вызвать метод `scan_obstacles()`.
- (g) Вызвать метод `alert_pilot()`.
- (h) Вызвать метод `auto_maneuver()`.

Задание 3

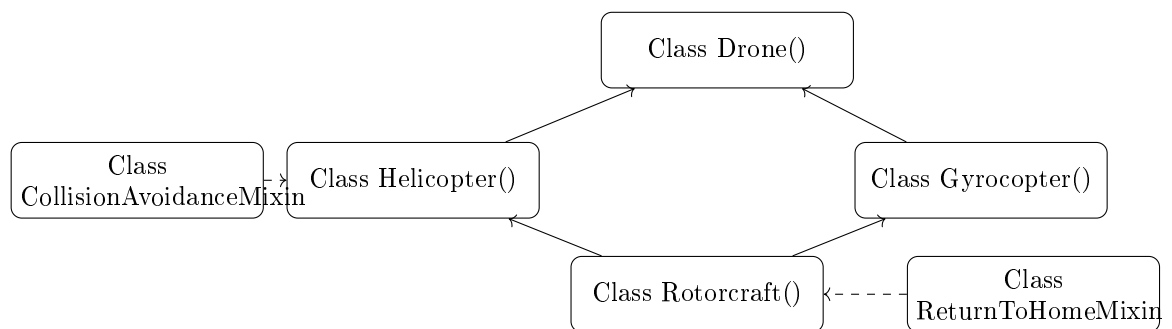
Используя программный код задания 7 из прошлого практикума текущего курса ООП, выполнить подмешивание класса-миксина `CollisionAvoidance` к классу `Helicopter` и подмешивание класса-миксина `ReturnToHome` к классу `Rotorcraft(Drone, Helicopter, Gyrocopter)`. При запуске класса `Rotorcraft` должна быть отображена в консоли следующая последовательность действий.

Пример вывода программы

```
Сканирование окружения...
Обнаружено препятствие!
Предупреждение пилота.
Автоманёвр выполнен.
```

Домашняя точка установлена: (0, 0, 0).
 Возврат домой инициирован.
 Запуск двигателей
 Вращение роторов
 Взлёт выполнен
 Зависание на высоте 50 м
 Посадка завершена.
 Flight time: 25 min, Rotor diameter: 12 m, Engine power: 150 hp

Схема классов



8 Задание 1

Написать класс-миксин на Python, который моделирует работу автопилота для самолёта. Создать класс-миксин **Autopilot**, который будет содержать методы для включения автопилота, установки высоты и курса, а также для деактивации.

- Класс **Autopilot** определяет объект, который моделирует автопилот.
- Метод **engage** включает автопилот.
- Метод **set_altitude** устанавливает целевую высоту.
- Метод **set_heading** устанавливает курс.
- Метод **disengage** отключает автопилот.
- В примере использования создаётся объект **ap**, после чего вызываются методы.

Задание 2

Написать класс-миксин на Python, который моделирует работу системы антиобледенения крыльев.

- Создать класс-миксин **DeicingSystem**, который предотвращает обледенение.
- Определить метод **activate_deicing(self)**, который включает систему.
- Определить метод **deactivate_deicing(self)**, который выключает систему.
- Определить метод **check_icing_risk(self)**, который оценивает риск обледенения.

- (e) Создать экземпляр класса `DeicingSystem` и сохранить его в переменной `deice`.
- (f) Вызвать метод `check_icing_risk()`.
- (g) Вызвать метод `activate_deicing()`.
- (h) Вызвать метод `deactivate_deicing()`.

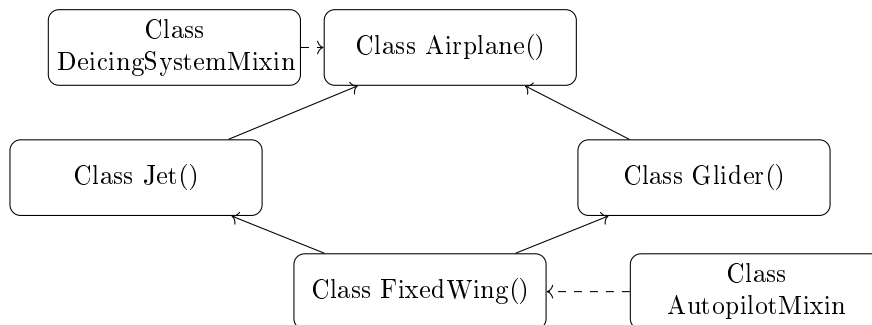
Задание 3

Используя программный код задания 8 из прошлого практикума текущего курса ООП, выполнить подмешивание класса-миксина `DeicingSystem` к классу `Airplane` и подмешивание класса-миксина `Autopilot` к классу `FixedWing(Airplane, Jet, Glider)`. При запуске класса `FixedWing` должна быть отображена в консоли следующая последовательность действий.

Пример вывода программы

```
Риск обледенения: высокий.
Система антиобледенения включена.
Система антиобледенения выключена.
Руление начато
Взлёт выполнен
Форсаж включён
Буксировка отцеплена
Автопилот включён.
Высота: 10000 м, курс: 270°
Wingspan: 35 м, Engine thrust: 100 kN, Aspect ratio: 20
```

Схема классов



9 Задание 1

Написать класс-миксин на Python, который моделирует работу системы оповещения пассажиров на круизном судне. Создать класс-миксин `PassengerAlert`, который будет содержать методы для объявления информации, экстренных оповещений и проверки систем связи.

- (a) Класс `PassengerAlert` определяет объект, который моделирует систему оповещения.
- (b) Метод `announce` делает общее объявление.
- (c) Метод `emergency_alert` запускает экстренное оповещение.
- (d) Метод `test_system` проверяет работоспособность динамиков.
- (e) В примере использования создаётся объект `alert`, после чего вызываются методы.

Задание 2

Написать класс-миксин на Python, который моделирует работу системы стабилизации качки на яхте.

- (a) Создать класс-миксин `Stabilizers`, который управляет гироскопами или плавниками.
- (b) Определить метод `activate_stabilizers(self)`, который включает систему.
- (c) Определить метод `deactivate_stabilizers(self)`, который выключает систему.
- (d) Определить метод `adjust_for_sea(self, wave_height)`, который адаптирует настройки под волнение.
- (e) Создать экземпляр класса `Stabilizers` и сохранить его в переменной `stab`.
- (f) Вызвать метод `activate_stabilizers()`.
- (g) Вызвать метод `adjust_for_sea(2.5)`.
- (h) Вызвать метод `deactivate_stabilizers()`.

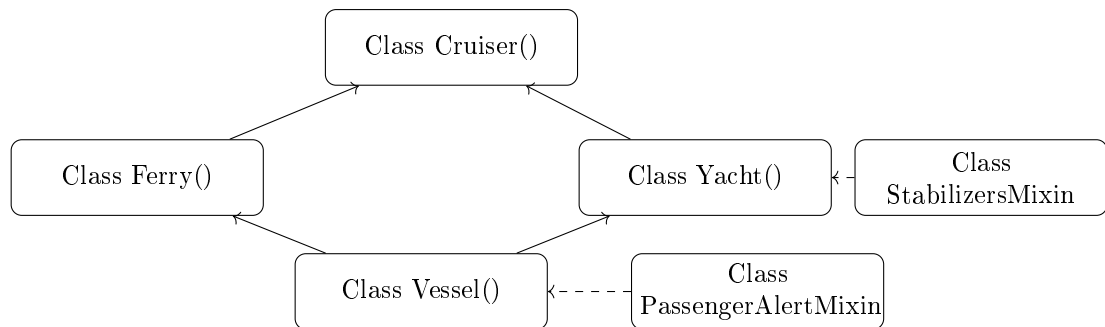
Задание 3

Используя программный код задания 9 из прошлого практикума текущего курса ООП, выполнить подмешивание класса-миксина `Stabilizers` к классу `Yacht` и подмешивание класса-миксина `PassengerAlert` к классу `Vessel(Cruiser, Ferry, Yacht)`. При запуске класса `Vessel` должна быть отображена в консоли следующая последовательность действий.

Пример вывода программы

```
Система стабилизации включена.
Настройка под волнение 2.5 м.
Система стабилизации выключена.
Объявление: Отплытие через 10 минут.
Экстренное оповещение: учения по безопасности.
Тест систем связи пройден.
Отплытие начато
Паруса подняты
Транспорт загружен
Плавание в режиме круиза
Причаливание завершено.
Passenger capacity: 3000, Vehicle deck size: 50 cars, Mast height: 30 m
```

Схема классов



10 Задание 1

Написать класс-миксин на Python, который моделирует работу системы обнаружения препятствий под водой. Создать класс-миксин `SonarSystem`, который будет содержать методы для сканирования, обнаружения объектов и вывода карты дна.

- (a) Класс `SonarSystem` определяет объект, который моделирует гидролокатор.
- (b) Метод `scan_area` запускает сканирование.
- (c) Метод `detect_object` определяет наличие препятствий.
- (d) Метод `map_seabed` генерирует упрощённую карту.
- (e) В примере использования создаётся объект `sonar`, после чего вызываются методы.

Задание 2

Написать класс-миксин на Python, который моделирует работу герметизации корпуса амфибии.

- (a) Создать класс-миксин `HullSeal`, который управляет герметичностью.
- (b) Определить метод `seal_hull(self)`, который закрывает все люки и клапаны.
- (c) Определить метод `unseal_hull(self)`, который открывает их.
- (d) Определить метод `check_pressure(self)`, который проверяет герметичность.
- (e) Создать экземпляр класса `HullSeal` и сохранить его в переменной `seal`.
- (f) Вызвать метод `seal_hull()`.
- (g) Вызвать метод `check_pressure()`.
- (h) Вызвать метод `unseal_hull()`.

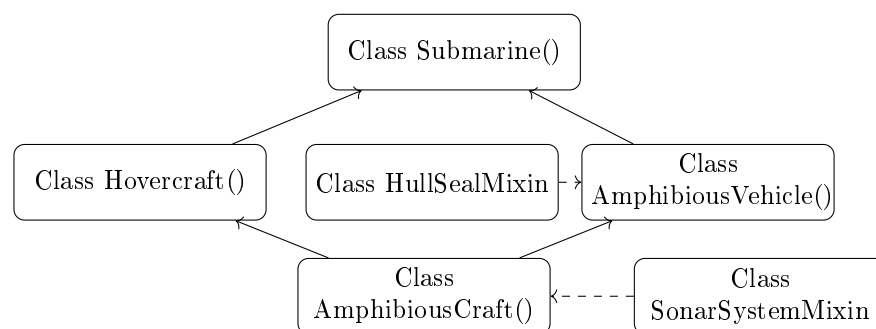
Задание 3

Используя программный код задания 10 из прошлого практикума текущего курса ООП, выполнить подмешивание класса-миксина `HullSeal` к классу `AmphibiousVehicle` и подмешивание класса-миксина `SonarSystem` к классу `AmphibiousCraft` (`Submarine`, `Hovercraft`, `AmphibiousVehicle`). При запуске класса `AmphibiousCraft` должна быть отображена в консоли следующая последовательность действий.

Пример вывода программы

Корпус герметизирован.
Давление в норме.
Корпус разгерметизирован.
Сканирование дна начато.
Объект обнаружен на глубине 15 м.
Карта дна сгенерирована.
Погружение начато
Юбка надута
Режим переключён: вода → суша
Подводная навигация активирована
Depth rating: 300 m, Skirt material: Neoprene, Wheel type: All-terrain

Схема классов



11 Задание 1

Написать класс-миксин на Python, который моделирует работу системы GPS-навигации для сельхозтехники. Создать класс-миксин `FieldGPS`, который будет содержать методы для загрузки поля, отслеживания пройденного пути и коррекции курса.

- (a) Класс `FieldGPS` определяет объект, который моделирует GPS-навигатор.
- (b) Метод `load_field_map` загружает карту поля.
- (c) Метод `track_path` записывает пройденный маршрут.
- (d) Метод `correct_course` корректирует направление движения.
- (e) В примере использования создаётся объект `gps`, после чего вызываются методы.

Задание 2

Написать класс-миксин на Python, который моделирует работу системы автоматической дозаправки комбайна.

- (a) Создать класс-миксин `AutoRefuel`, который управляет дозаправкой.
- (b) Определить метод `request_refuel(self)`, который отправляет запрос.

- (c) Определить метод `connect_hose(self)`, который имитирует подключение шланга.
- (d) Определить метод `complete_refuel(self)`, который завершает процесс.
- (e) Создать экземпляр класса `AutoRefuel` и сохранить его в переменной `refuel`.
- (f) Вызвать метод `request_refuel()`.
- (g) Вызвать метод `connect_hose()`.
- (h) Вызвать метод `complete_refuel()`.

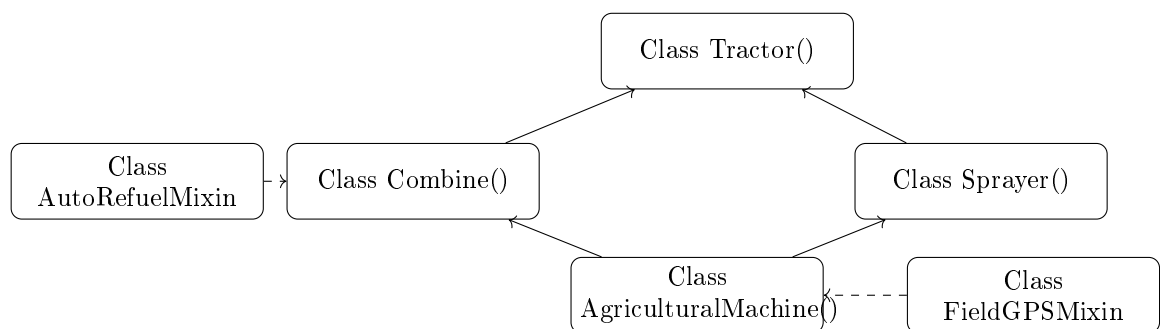
Задание 3

Используя программный код задания 11 из прошлого практикума текущего курса ООП, выполнить подмешивание класса-миксина `AutoRefuel` к классу `Combine` и подмешивание класса-миксина `FieldGPS` к классу `AgriculturalMachine(Tractor, Combine, Sprayer)`. При запуске класса `AgriculturalMachine` должна быть отображена в консоли следующая последовательность действий.

Пример вывода программы

Запрос на дозаправку отправлен.
 Шланг подключён.
 Дозаправка завершена.
 Карта поля загружена.
 Маршрут отслеживается.
 Курс скорректирован.
 Двигатель запущен
 Вспашка начата
 Уборка урожая в процессе
 Форсунки активированы
 Разгрузка завершена
 Engine HP: 120, Grain tank size: 8000 L, Tank capacity: 300 L

Схема классов



12 Задание 1

Написать класс-миксин на Python, который моделирует работу системы мониторинга нагрузки на кран. Создать класс-миксин `LoadSensor`, который будет содержать методы для проверки веса груза, предупреждения о перегрузке и блокировки подъёма.

- (a) Класс `LoadSensor` определяет объект, который моделирует датчик нагрузки.
- (b) Метод `measure_load` измеряет текущую нагрузку.
- (c) Метод `warn_overload` выводит предупреждение при превышении.
- (d) Метод `block_lift` блокирует подъём при критической перегрузке.
- (e) В примере использования создаётся объект `sensor`, после чего вызываются методы.

Задание 2

Написать класс-миксин на Python, который моделирует работу системы автоматического выравнивания экскаватора.

- (a) Создать класс-миксин `AutoLevel`, который управляет гидравликой для выравнивания.
- (b) Определить метод `check_slope(self)`, который определяет уклон.
- (c) Определить метод `adjust_stabilizers(self)`, который выравнивает платформу.
- (d) Определить метод `confirm_level(self)`, который подтверждает горизонталь.
- (e) Создать экземпляр класса `AutoLevel` и сохранить его в переменной `level`.
- (f) Вызвать метод `check_slope()`.
- (g) Вызвать метод `adjust_stabilizers()`.
- (h) Вызвать метод `confirm_level()`.

Задание 3

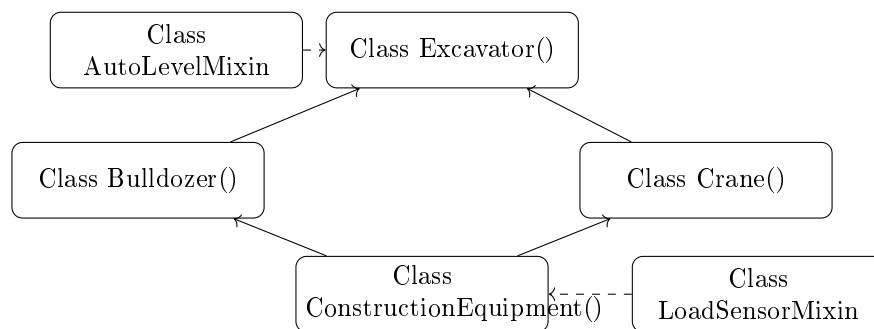
Используя программный код задания 12 из прошлого практикума текущего курса ООП, выполнить подмешивание класса-миксина `AutoLevel` к классу `Excavator` и подмешивание класса-миксина `LoadSensor` к классу `ConstructionEquipment(Excavator, Bulldozer, Crane)`. При запуске класса `ConstructionEquipment` должна быть отображена в консоли следующая последовательность действий.

Пример вывода программы

Уклон: 8 градусов.
Стабилизаторы отрегулированы.
Платформа выровнена.
Нагрузка: 4.8 тонн.
Предупреждение: близко к пределу!
Подъём разрешён.
Двигатель запущен

Копка выполнена
 Грунт выровнен
 Груз поднят
 Материалы перемещены
 Bucket capacity: 1.2 m³, Blade width: 3.5 m, Max load: 5 t

Схема классов



13 Задание 1

Написать класс-миксин на Python, который моделирует работу системы связи между экстренными службами. Создать класс-миксин **EmergencyComms**, который будет содержать методы для шифрования связи, передачи координат и экстренного вызова подкрепления.

- Класс **EmergencyComms** определяет объект, который моделирует защищённую связь.
- Метод **encrypt_channel** шифрует радиоканал.
- Метод **send_coordinates** передаёт GPS-координаты.
- Метод **request_backup** вызывает подкрепление.
- В примере использования создаётся объект **comms**, после чего вызываются методы.

Задание 2

Написать класс-миксин на Python, который моделирует работу медицинского холодильника в машине скорой помощи.

- Создать класс-миксин **MedRefrigerator**, который управляет температурой лекарств.
- Определить метод **cool_meds(self)**, который включает охлаждение.
- Определить метод **monitor_temp(self)**, который проверяет температуру.
- Определить метод **alert_temp_breach(self)**, который срабатывает при отклонении.
- Создать экземпляр класса **MedRefrigerator** и сохранить его в переменной **fridge**.

- (f) Вызвать метод `cool_meds()`.
- (g) Вызвать метод `monitor_temp()`.
- (h) Вызвать метод `alert_temp_breach()`.

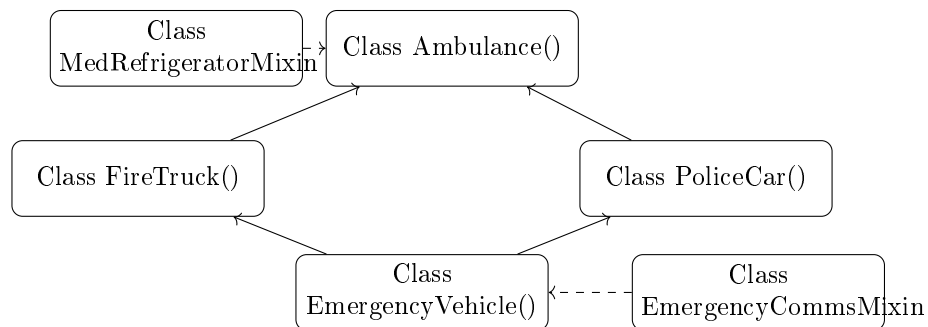
Задание 3

Используя программный код задания 13 из прошлого практикума текущего курса ООП, выполнить подмешивание класса-миксина `MedRefrigerator` к классу `Ambulance` и подмешивание класса-миксина `EmergencyComms` к классу `EmergencyVehicle(Ambulance, FireTruck, PoliceCar)`. При запуске класса `EmergencyVehicle` должна быть отображена в консоли следующая последовательность действий.

Пример вывода программы

```
Охлаждение лекарств включено.
Температура: 4°C.
Отклонение температуры: нет.
Канал связи зашифрован.
Координаты отправлены: 55.7558° N, 37.6176° E.
Подкрепление запрошено.
Сирена включена
Пациент загружен
Лестница развернута
Погоня начата
Пожар потушен
Medical kit: Advanced, Water tank: 2000 L, Radio type: Encrypted
```

Схема классов



14 Задание 1

Написать класс-миксин на Python, который моделирует работу системы оценки клиентов в такси. Создать класс-миксин `RatingSystem`, который будет содержать методы для отправки запроса на оценку, получения отзыва и расчёта рейтинга водителя.

- (a) Класс `RatingSystem` определяет объект, который моделирует систему рейтинга.

- (b) Метод `request_review` отправляет клиенту запрос.
- (c) Метод `submit_review` принимает оценку от 1 до 5.
- (d) Метод `calculate_rating` обновляет средний рейтинг.
- (e) В примере использования создаётся объект `rating`, после чего вызываются методы.

Задание 2

Написать класс-миксин на Python, который моделирует работу мини-бара в лимузине.

- (a) Создать класс-миксин `MiniBar`, который управляет напитками.
- (b) Определить метод `stock_bar(self)`, который заполняет бар.
- (c) Определить метод `serve_drink(self, drink)`, который подаёт напиток.
- (d) Определить метод `check_stock(self)`, который проверяет наличие.
- (e) Создать экземпляр класса `MiniBar` и сохранить его в переменной `bar`.
- (f) Вызвать метод `stock_bar()`.
- (g) Вызвать метод `serve_drink("Виски")`.
- (h) Вызвать метод `check_stock()`.

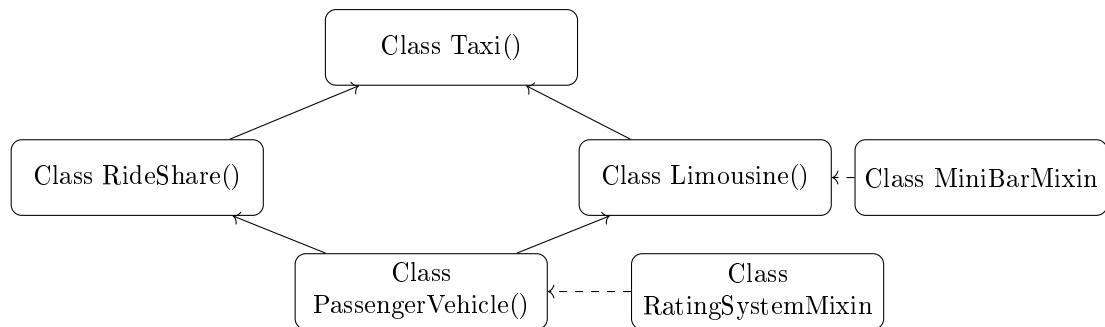
Задание 3

Используя программный код задания 14 из прошлого практикума текущего курса ООП, выполнить подмешивание класса-миксина `MiniBar` к классу `Limousine` и подмешивание класса-миксина `RatingSystem` к классу `PassengerVehicle(Taxi, RideShare, Limousine)`. При запуске класса `PassengerVehicle` должна быть отображена в консоли следующая последовательность действий.

Пример вывода программы

```
Мини-бар пополнен.  
Подан напиток: Виски.  
Остаток: 5 напитков.  
Запрос на оценку отправлен.  
Отзыв получен: 5 звёзд.  
Рейтинг обновлён: 4.8.  
Клиент подобран  
Автомобиль разблокирован  
Шампанское подано  
Клиент высаден  
Passenger seats: 4, App integration: Yes, Interior luxury: Premium
```


Схема классов



15 Задание 1

Написать класс-миксин на Python, который моделирует работу системы видеонаблюдения в школьном автобусе. Создать класс-миксин **BusCameras**, который будет содержать методы для включения записи, сохранения архива и обнаружения движения.

- (a) Класс **BusCameras** определяет объект, который моделирует систему камер.
- (b) Метод **start_recording** включает запись.
- (c) Метод **save_archive** сохраняет файлы.
- (d) Метод **detect_motion** определяет активность в салоне.
- (e) В примере использования создаётся объект **cams**, после чего вызываются методы.

Задание 2

Написать класс-миксин на Python, который моделирует работу системы развлечений в туристическом автобусе.

- (a) Создать класс-миксин **EntertainmentSystem**, который управляет мультимедиа.
- (b) Определить метод **play_movie(self)**, который запускает фильм.
- (c) Определить метод **connect_headphones(self)**, который подключает наушники.
- (d) Определить метод **volume_control(self, level)**, который регулирует громкость.
- (e) Создать экземпляр класса **EntertainmentSystem** и сохранить его в переменной **ent**.
- (f) Вызвать метод **play_movie()**.
- (g) Вызвать метод **connect_headphones()**.
- (h) Вызвать метод **volume_control(6)**.

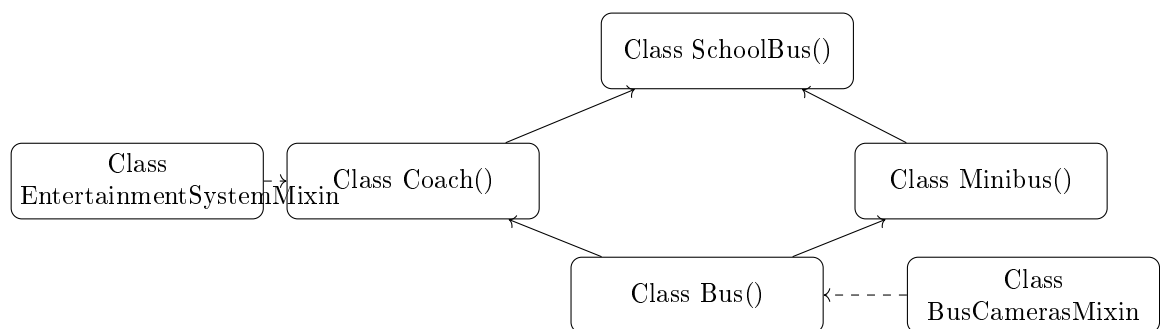
Задание 3

Используя программный код задания 15 из прошлого практикума текущего курса ООП, выполнить подмешивание класса-миксина **EntertainmentSystem** к классу **Coach** и подмешивание класса-миксина **BusCameras** к классу **Bus(SchoolBus, Coach, Minibus)**. При запуске класса **Bus** должна быть отображена в консоли следующая последовательность действий.

Пример вывода программы

Фильм запущен.
Наушники подключены.
Громкость установлена: 6.
Запись начата.
Движение обнаружено: дети зашли.
Архив сохранён.
Дети загружены
Знак активирован
Двери открыты
Аудиогид включён
Отправление начато
Seat belts: Yes, Toilet installed: Yes, Door type: Sliding

Схема классов



16 Задание 1

Написать класс-миксин на Python, который моделирует работу системы тахографа в грузовике. Создать класс-миксин **Tachograph**, который будет содержать методы для записи скорости, времени в пути и контроля режима труда водителя.

- (a) Класс **Tachograph** определяет объект, который моделирует тахограф.
- (b) Метод **start_log** начинает запись поездки.
- (c) Метод **record_speed** сохраняет текущую скорость.
- (d) Метод **check_driving_time** проверяет превышение лимита вождения.
- (e) В примере использования создаётся объект **tacho**, после чего вызываются методы.

Задание 2

Написать класс-миксин на Python, который моделирует работу системы автоматической разгрузки самосвала.

- (a) Создать класс-миксин `AutoUnload`, который управляет гидравликой кузова.
- (b) Определить метод `prepare_unload(self)`, который готовит к разгрузке.
- (c) Определить метод `execute_unload(self)`, который поднимает кузов.
- (d) Определить метод `confirm_empty(self)`, который проверяет пустоту кузова.
- (e) Создать экземпляр класса `AutoUnload` и сохранить его в переменной `unload`.
- (f) Вызвать метод `prepare_unload()`.
- (g) Вызвать метод `execute_unload()`.
- (h) Вызвать метод `confirm_empty()`.

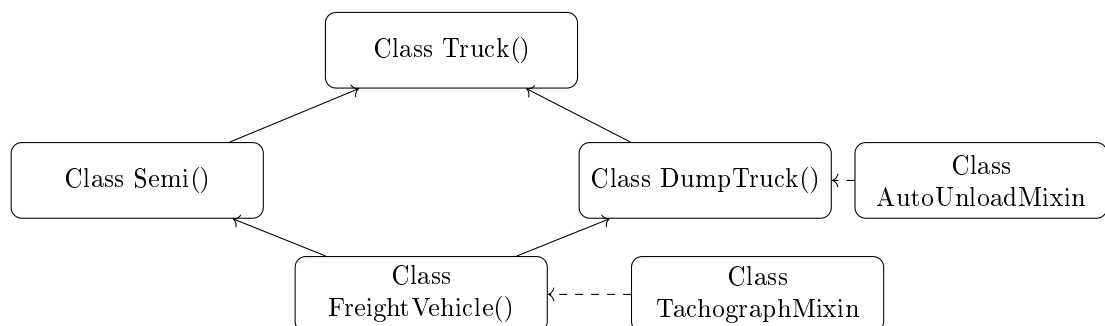
Задание 3

Используя программный код задания 16 из прошлого практикума текущего курса ООП, выполнить подмешивание класса-миксина `AutoUnload` к классу `DumpTruck` и подмешивание класса-миксина `Tachograph` к классу `FreightVehicle(Truck, Semi, DumpTruck)`. При запуске класса `FreightVehicle` должна быть отображена в консоли следующая последовательность действий.

Пример вывода программы

Подготовка к разгрузке.
 Кузов поднят.
 Кузов пуст.
 Запись поездки начата.
 Текущая скорость: 85 км/ч.
 Лимит вождения: ОК.
 Двигатель запущен
 Груз погружен
 Прицеп подцеплен
 Кузов поднят
 Разгрузка завершена
 Payload capacity: 20 t, Fifth wheel: Standard, Bed angle: 45°

Схема классов



17 Задание 1

Написать класс-миксин на Python, который моделирует работу системы термоконтроля в космическом аппарате. Создать класс-миксин `ThermalControl`, который будет содержать методы для охлаждения, обогрева и мониторинга температуры компонентов.

- (a) Класс `ThermalControl` определяет объект, который моделирует термосистему.
- (b) Метод `activate_cooling` включает радиаторы.
- (c) Метод `activate_heating` включает нагреватели.
- (d) Метод `monitor_temp` проверяет температуру модулей.
- (e) В примере использования создаётся объект `thermal`, после чего вызываются методы.

Задание 2

Написать класс-миксин на Python, который моделирует работу системы посадки по координатам.

- (a) Создать класс-миксин `PrecisionLanding`, который управляет посадкой.
- (b) Определить метод `acquire_target(self)`, который захватывает цель.
- (c) Определить метод `adjust_descent(self)`, который корректирует траекторию.
- (d) Определить метод `confirm_landing(self)`, который подтверждает контакт.
- (e) Создать экземпляр класса `PrecisionLanding` и сохранить его в переменной `landing`.
- (f) Вызвать метод `acquire_target()`.
- (g) Вызвать метод `adjust_descent()`.
- (h) Вызвать метод `confirm_landing()`.

Задание 3

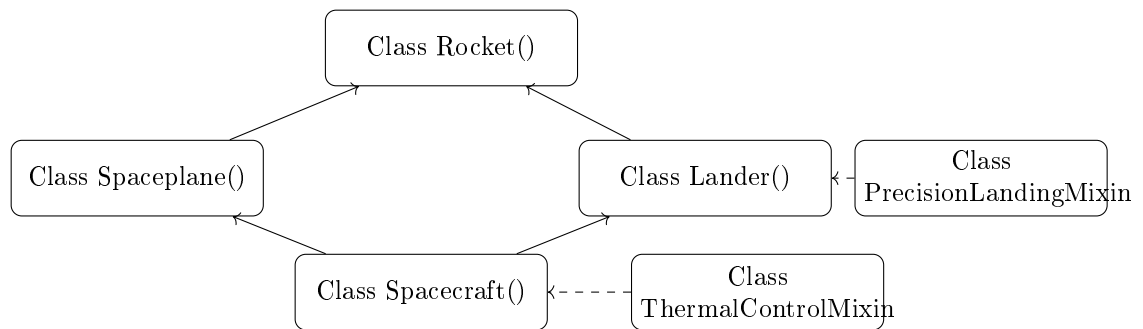
Используя программный код задания 17 из прошлого практикума текущего курса ООП, выполнить подмешивание класса-миксина `PrecisionLanding` к классу `Lander` и подмешивание класса-миксина `ThermalControl` к классу `Spacecraft(Rocket, Spaceplane, Lander)`. При запуске класса `Spacecraft` должна быть отображена в консоли следующая последовательность действий.

Пример вывода программы

Цель захвачена.
Траектория скорректирована.
Контакт с поверхностью подтверждён.
Охлаждение активировано.
Нагрев отключён.
Температура модулей: 22°C.
Запуск выполнен
Выход на орбиту

Вход в атмосферу
Посадка завершена
Груз выгружен
Stage count: 2, Reentry shield: Ceramic, Thruster count: 4

Схема классов



18 Задание 1

Написать класс-миксин на Python, который моделирует работу системы уведомлений в умных часах. Создать класс-миксин `NotificationHub`, который будет содержать методы для фильтрации уведомлений, вибрации и отображения иконок приложений.

- (a) Класс `NotificationHub` определяет объект, который моделирует центр уведомлений.
- (b) Метод `filter_apps` настраивает, от каких приложений приходят уведомления.
- (c) Метод `vibrate_alert` активирует вибрацию.
- (d) Метод `show_icon` отображает иконку приложения.
- (e) В примере использования создаётся объект `hub`, после чего вызываются методы.

Задание 2

Написать класс-миксин на Python, который моделирует работу системы отслеживания сна в фитнес-трекере.

- (a) Создать класс-миксин `SleepTracker`, который анализирует сон.
- (b) Определить метод `start_monitoring(self)`, который включает мониторинг.
- (c) Определить метод `analyze_sleep(self)`, который оценивает качество.
- (d) Определить метод `generate_report(self)`, который формирует отчёт.
- (e) Создать экземпляр класса `SleepTracker` и сохранить его в переменной `sleep`.
- (f) Вызвать метод `start_monitoring()`.
- (g) Вызвать метод `analyze_sleep()`.
- (h) Вызвать метод `generate_report()`.

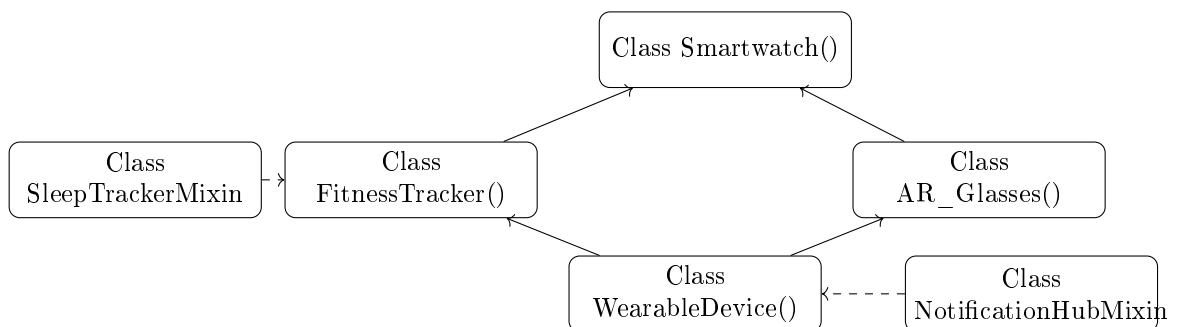
Задание 3

Используя программный код задания 18 из прошлого практикума текущего курса ООП, выполнить подмешивание класса-миксина `SleepTracker` к классу `FitnessTracker` и подмешивание класса-миксина `NotificationHub` к классу `WearableDevice(Smartwatch, FitnessTracker, AR_Glasses)`. При запуске класса `WearableDevice` должна быть отображена в консоли следующая последовательность действий.

Пример вывода программы

```
Мониторинг сна начат.  
Качество сна: 85%.  
Отчёт сформирован.  
Уведомления от WhatsApp разрешены.  
Вибрация активирована.  
Иконка отображена.  
Уведомления получены  
ЧСС в норме  
Данные синхронизированы  
Оверлей отображён  
Battery life: 2 days, Heart rate sensor: Optical, Display resolution: 1920x1080
```

Схема классов



19 Задание 1

Написать класс-миксин на Python, который моделирует работу системы распознавания лиц в смартфоне. Создать класс-миксин `FaceUnlock`, который будет содержать методы для сканирования лица, разблокировки и обработки ошибок.

- (a) Класс `FaceUnlock` определяет объект, который моделирует Face ID.
- (b) Метод `scan_face` запускает камеру для сканирования.
- (c) Метод `unlock_device` разблокирует устройство при совпадении.
- (d) Метод `handle_failure` обрабатывает неудачные попытки.
- (e) В примере использования создаётся объект `face`, после чего вызываются методы.

Задание 2

Написать класс-миксин на Python, который моделирует работу системы стилуса в планшете.

- (a) Создать класс-миксин `StylusManager`, который управляет стилусом.
- (b) Определить метод `detect_stylus(self)`, который проверяет подключение.
- (c) Определить метод `calibrate_pen(self)`, который калибрует чувствительность.
- (d) Определить метод `enable_palm_rejection(self)`, который отключает ладонь.
- (e) Создать экземпляр класса `StylusManager` и сохранить его в переменной `stylus`.
- (f) Вызвать метод `detect_stylus()`.
- (g) Вызвать метод `calibrate_pen()`.
- (h) Вызвать метод `enable_palm_rejection()`.

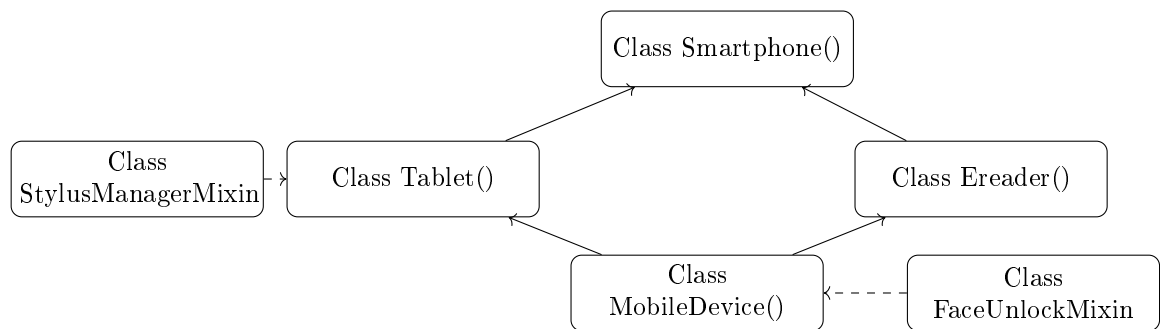
Задание 3

Используя программный код задания 19 из прошлого практикума текущего курса ООП, выполнить подмешивание класса-миксина `StylusManager` к классу `Tablet` и подмешивание класса-миксина `FaceUnlock` к классу `MobileDevice(Smartphone, Tablet, Ereader)`. При запуске класса `MobileDevice` должна быть отображена в консоли следующая последовательность действий.

Пример вывода программы

```
Стилус обнаружен.  
Калибровка завершена.  
Отклонение ладони включено.  
Лицо отсканировано.  
Устройство разблокировано.  
Звонок совершён  
Фото сделано  
Рисование начато  
Книга открыта  
Видео воспроизведено  
Screen size: 6.5", Stylus support: Yes, Front light: Yes
```

Схема классов



20 Задание 1

Написать класс-миксин на Python, который моделирует работу системы резервного копирования в компьютере. Создать класс-миксин **BackupSystem**, который будет содержать методы для создания резервной копии, проверки целостности и восстановления данных.

- (a) Класс **BackupSystem** определяет объект, который моделирует бэкап.
- (b) Метод **create_backup** сохраняет данные на внешний носитель.
- (c) Метод **verify_integrity** проверяет целостность архива.
- (d) Метод **restore_data** восстанавливает систему из резервной копии.
- (e) В примере использования создаётся объект **backup**, после чего вызываются методы.

Задание 2

Написать класс-миксин на Python, который моделирует работу системы мониторинга температуры в рабочей станции.

- (a) Создать класс-миксин **TempMonitor**, который отслеживает нагрев компонентов.
- (b) Определить метод **read_cpu_temp(self)**, который считывает температуру CPU.
- (c) Определить метод **read_gpu_temp(self)**, который считывает температуру GPU.
- (d) Определить метод **alert_overheat(self)**, который срабатывает при перегреве.
- (e) Создать экземпляр класса **TempMonitor** и сохранить его в переменной **temp**.
- (f) Вызвать метод **read_cpu_temp()**.
- (g) Вызвать метод **read_gpu_temp()**.
- (h) Вызвать метод **alert_overheat()**.

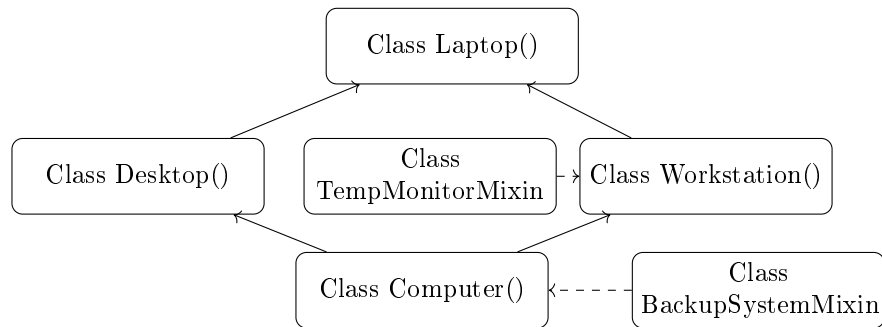
Задание 3

Используя программный код задания 20 из прошлого практикума текущего курса ООП, выполнить подмешивание класса-миксина **TempMonitor** к классу **Workstation** и подмешивание класса-миксина **BackupSystem** к классу **Computer(Laptop, Desktop, Workstation)**. При запуске класса **Computer** должна быть отображена в консоли следующая последовательность действий.

Пример вывода программы

Температура CPU: 68°C.
Температура GPU: 72°C.
Перегрев не обнаружен.
Резервная копия создана.
Целостность проверена.
Восстановление не требуется.
ОС загружена
Питание включено
Приложение запущено
Сцена отрендерена
Система выключена
RAM: 16 GB, GPU model: RTX 4090, CPU cores: 16

Схема классов



21 Задание 1

Написать класс-миксин на Python, который моделирует работу системы логирования трафика в сетевом устройстве. Создать класс-миксин **TrafficLogger**, который будет содержать методы для включения логирования, фильтрации по IP и экспорта логов.

- (a) Класс **TrafficLogger** определяет объект, который моделирует логгер.
- (b) Метод **enable_logging** включает запись трафика.
- (c) Метод **filter_by_ip** задаёт IP-адрес для фильтрации.
- (d) Метод **export_logs** сохраняет логи в файл.
- (e) В примере использования создаётся объект **logger**, после чего вызываются методы.

Задание 2

Написать класс-миксин на Python, который моделирует работу системы диагностики маршрутизатора.

- (a) Создать класс-миксин **RouterDiagnostics**, который проверяет работоспособность.

- (b) Определить метод `run_self_test(self)`, который запускает самодиагностику.
- (c) Определить метод `check_interfaces(self)`, который проверяет порты.
- (d) Определить метод `report_status(self)`, который выводит отчёт.
- (e) Создать экземпляр класса `RouterDiagnostics` и сохранить его в переменной `diag`.
- (f) Вызвать метод `run_self_test()`.
- (g) Вызвать метод `check_interfaces()`.
- (h) Вызвать метод `report_status()`.

Задание 3

Используя программный код задания 21 из прошлого практикума текущего курса ООП, выполнить подмешивание класса-миксина `RouterDiagnostics` к классу `Router` и подмешивание класса-миксина `TrafficLogger` к классу `NetworkDevice` (`Router`, `Switch`, `Firewall`). При запуске класса `NetworkDevice` должна быть отображена в консоли следующая последовательность действий.

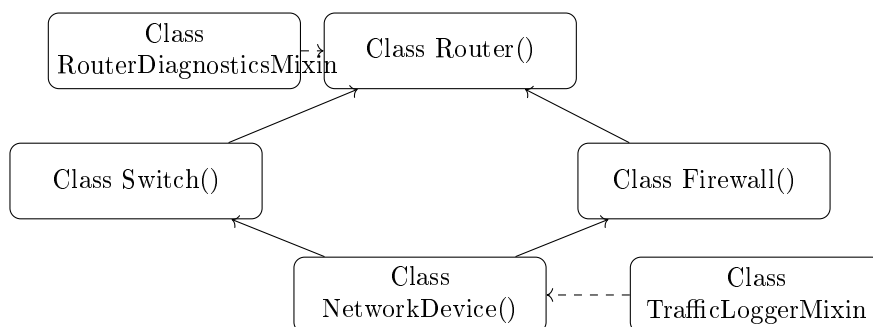
Пример вывода программы

```

Самодиагностика запущена.
Порты проверены: все активны.
Статус: норма.
Логирование включено.
Фильтрация по IP: 192.168.1.100.
Логи экспортированы.
Устройство подключено
Пакеты переданы
SSID транслируется
VLAN настроен
Угроза заблокирована
LAN ports: 4, Port speed: 1 Gbps, Rules count: 50

```

Схема классов



22 Задание 1

Написать класс-миксин на Python, который моделирует работу системы энергосбережения в офисной технике. Создать класс-миксин `EcoMode`, который будет содержать методы для перехода в спящий режим, отключения дисплея и учёта энергопотребления.

- (a) Класс `EcoMode` определяет объект, который моделирует режим энергосбережения.
- (b) Метод `enter_sleep` переводит устройство в сон.
- (c) Метод `turn_off_display` выключает экран.
- (d) Метод `log_energy` записывает потреблённую энергию.
- (e) В примере использования создаётся объект `eco`, после чего вызываются методы.

Задание 2

Написать класс-миксин на Python, который моделирует работу системы автоматической подачи бумаги в принтере.

- (a) Создать класс-миксин `AutoFeeder`, который управляет лотком.
- (b) Определить метод `load_tray(self)`, который заполняет лоток.
- (c) Определить метод `detect_jam(self)`, который определяет зажевывание.
- (d) Определить метод `resume_printing(self)`, который возобновляет печать.
- (e) Создать экземпляр класса `AutoFeeder` и сохранить его в переменной `feeder`.
- (f) Вызвать метод `load_tray()`.
- (g) Вызвать метод `detect_jam()`.
- (h) Вызвать метод `resume_printing()`.

Задание 3

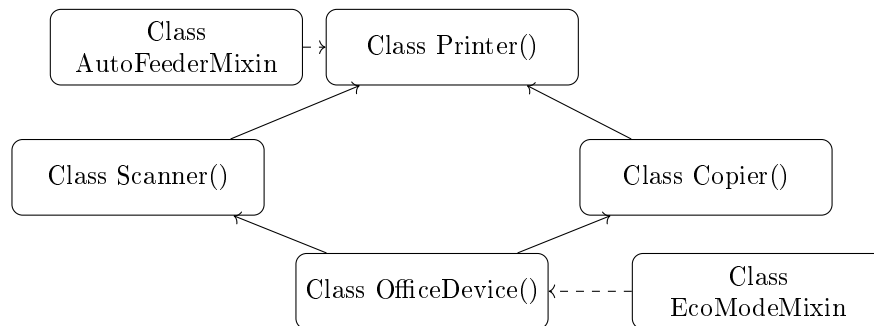
Используя программный код задания 22 из прошлого практикума текущего курса ООП, выполнить подмешивание класса-миксина `AutoFeeder` к классу `Printer` и подмешивание класса-миксина `EcoMode` к классу `OfficeDevice(Printer, Scanner, Copier)`. При запуске класса `OfficeDevice` должна быть отображена в консоли следующая последовательность действий.

Пример вывода программы

```
Лоток заполнен.  
Зажевывание не обнаружено.  
Печать продолжена.  
Спящий режим активирован.  
Дисплей выключен.  
Энергопотребление: 120 Вт·ч.  
Бумага загружена  
Страница отсканирована
```

Документ напечатан
PDF сохранён
Копирование завершено
Ink type: Laser, Color depth: 48-bit, Duplex: Yes

Схема классов



23 Задание 1

Написать класс-миксин на Python, который моделирует работу системы стабилизации изображения в камере. Создать класс-миксин `ImageStabilizer`, который будет содержать методы для включения оптической стабилизации, компенсации дрожания и вывода качества кадра.

- (a) Класс `ImageStabilizer` определяет объект, который моделирует стабилизацию.
- (b) Метод `enable_ois` включает оптическую стабилизацию.
- (c) Метод `compensate_shake` корректирует дрожание.
- (d) Метод `assess_quality` оценивает резкость кадра.
- (e) В примере использования создаётся объект `stab`, после чего вызываются методы.

Задание 2

Написать класс-миксин на Python, который моделирует работу системы распознавания объектов в видеокамере.

- (a) Создать класс-миксин `ObjectDetector`, который анализирует кадры.
- (b) Определить метод `scan_frame(self)`, который обрабатывает изображение.
- (c) Определить метод `identify_objects(self)`, который распознаёт объекты.
- (d) Определить метод `tag_scene(self)`, который добавляет метки.
- (e) Создать экземпляр класса `ObjectDetector` и сохранить его в переменной `detect`.
- (f) Вызвать метод `scan_frame()`.
- (g) Вызвать метод `identify_objects()`.
- (h) Вызвать метод `tag_scene()`.

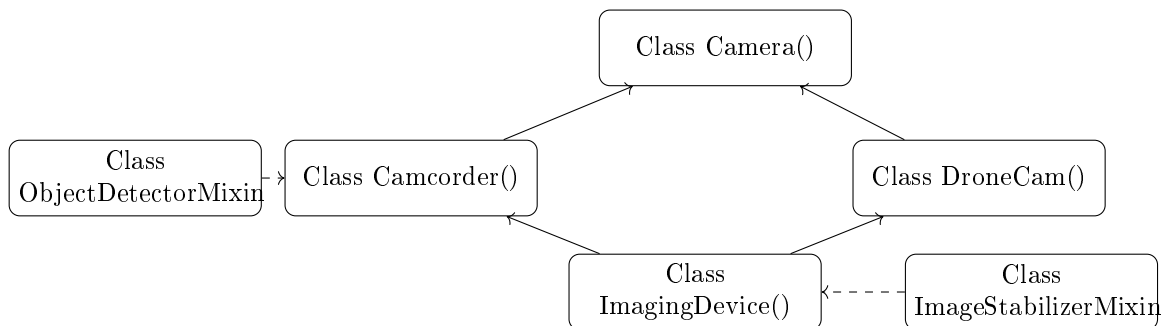
Задание 3

Используя программный код задания 23 из прошлого практикума текущего курса ООП, выполнить подмешивание класса-миксина `ObjectDetector` к классу `Camcorder` и подмешивание класса-миксина `ImageStabilizer` к классу `ImagingDevice(Camera, Camcorder, DroneCam)`. При запуске класса `ImagingDevice` должна быть отображена в консоли следующая последовательность действий.

Пример вывода программы

Кадр обработан.
Объекты распознаны: человек, автомобиль.
Метки добавлены.
Оптическая стабилизация включена.
Дрожание компенсировано.
Качество кадра: высокое.
Фокусировка выполнена
Фото сделано
Видео записано
Запись остановлена
Поток транслируется
Sensor size: Full-frame, Video format: 4K, Gimbal axes: 3

Схема классов



24 Задание 1

Написать класс-миксин на Python, который моделирует работу системы управления рецептами в кухонной технике. Создать класс-миксин `RecipeManager`, который будет содержать методы для загрузки рецепта, установки режимов приготовления и отслеживания прогресса.

- (a) Класс `RecipeManager` определяет объект, который моделирует систему рецептов.
- (b) Метод `load_recipe` загружает рецепт по названию.
- (c) Метод `set_program` устанавливает параметры приготовления.

- (d) Метод `track_progress` отслеживает этапы готовки.
- (e) В примере использования создаётся объект `recipe`, после чего вызываются методы.

Задание 2

Написать класс-миксин на Python, который моделирует работу системы самоочистки в духовке.

- (a) Создать класс-миксин `SelfCleanOven`, который управляет очисткой.
- (b) Определить метод `start_pyrolysis(self)`, который запускает пиролиз.
- (c) Определить метод `check_temperature(self)`, который контролирует нагрев.
- (d) Определить метод `complete_cycle(self)`, который завершает цикл.
- (e) Создать экземпляр класса `SelfCleanOven` и сохранить его в переменной `clean`.
- (f) Вызвать метод `start_pyrolysis()`.
- (g) Вызвать метод `check_temperature()`.
- (h) Вызвать метод `complete_cycle()`.

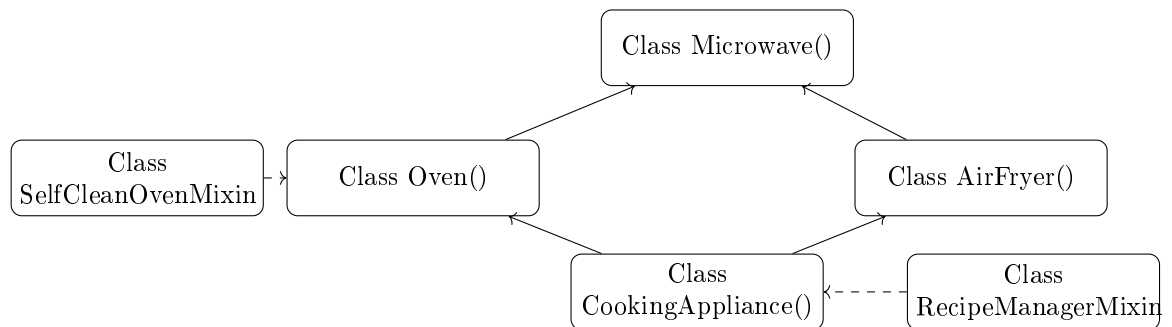
Задание 3

Используя программный код задания 24 из прошлого практикума текущего курса ООП, выполнить подмешивание класса-миксина `SelfCleanOven` к классу `Oven` и подмешивание класса-миксина `RecipeManager` к классу `CookingAppliance` (`Microwave`, `Oven`, `AirFryer`). При запуске класса `CookingAppliance` должна быть отображена в консоли следующая последовательность действий.

Пример вывода программы

```
Пиролиз запущен.  
Температура: 500°C.  
Цикл очистки завершён.  
Рецепт загружен: Курица гриль.  
Программа установлена.  
Этап: запекание.  
Таймер установлен  
Предварительный нагрев начат  
Хрустящая корочка готова  
Прибор выключен  
Power watt: 1000, Convection: Yes, Basket size: 5 L
```

Схема классов



25 Задание 1

Написать класс-миксин на Python, который моделирует работу системы энергомониторинга в охлаждающей технике. Создать класс-миксин **EnergyMonitor**, который будет содержать методы для измерения потребления, расчёта стоимости и рекомендаций по экономии.

- (a) Класс **EnergyMonitor** определяет объект, который моделирует мониторинг энергии.
- (b) Метод **measure_consumption** измеряет потребление за час.
- (c) Метод **calculate_cost** вычисляет стоимость.
- (d) Метод **suggest_savings** даёт советы по экономии.
- (e) В примере использования создаётся объект **energy**, после чего вызываются методы.

Задание 2

Написать класс-миксин на Python, который моделирует работу системы быстрой заморозки в морозильной камере.

- (a) Создать класс-миксин **QuickFreeze**, который управляет режимом заморозки.
- (b) Определить метод **activate_mode(self)**, который включает режим.
- (c) Определить метод **monitor_temp(self)**, который следит за температурой.
- (d) Определить метод **deactivate_mode(self)**, который выключает режим.
- (e) Создать экземпляр класса **QuickFreeze** и сохранить его в переменной **freeze**.
- (f) Вызвать метод **activate_mode()**.
- (g) Вызвать метод **monitor_temp()**.
- (h) Вызвать метод **deactivate_mode()**.

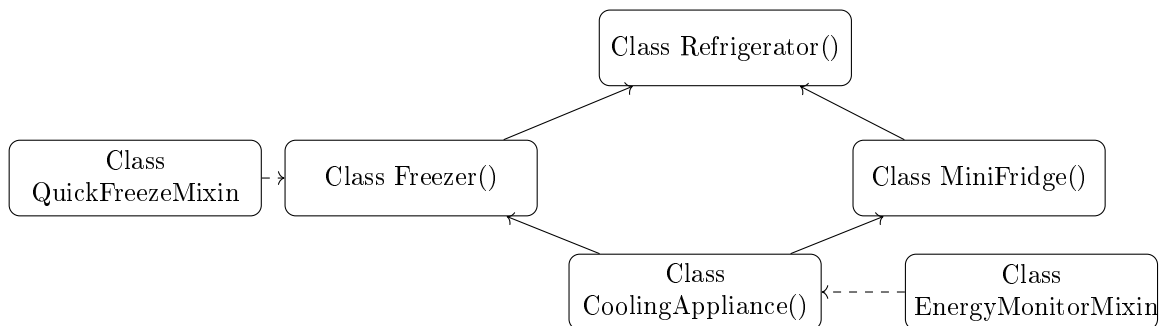
Задание 3

Используя программный код задания 25 из прошлого практикума текущего курса ООП, выполнить подмешивание класса-миксина `QuickFreeze` к классу `Freezer` и подмешивание класса-миксина `EnergyMonitor` к классу `CoolingAppliance(Refrigerator, Freezer, MiniFridge)`. При запуске класса `CoolingAppliance` должна быть отображена в консоли следующая последовательность действий.

Пример вывода программы

Режим быстрой заморозки включён.
Температура: -24°C.
Режим выключен.
Потребление: 1.2 кВт·ч/ч.
Стоимость: 6.5 руб/ч.
Совет: установите режим «Эко».
Охлаждение начато
Заморозка выполнена
Сигнализация активна
Напиток охлаждён
Продукты размещены
Capacity liters: 300, Temp min: -25°C, Door type: Glass

Схема классов



26 Задание 1

Написать класс-миксин на Python, который моделирует работу системы дозирования моющих средств в стиральной машине. Создать класс-миксин `DetergentDispenser`, который будет содержать методы для загрузки средства, автоматического дозирования и оповещения о замене.

- Класс `DetergentDispenser` определяет объект, который моделирует дозатор.
- Метод `load_detergent` загружает средство.
- Метод `auto_dose` дозирует средство в зависимости от загрузки.

- (d) Метод `alert_refill` оповещает о необходимости дозаправки.
- (e) В примере использования создаётся объект `disp`, после чего вызываются методы.

Задание 2

Написать класс-миксин на Python, который моделирует работу системы защиты от перегрева в утюге.

- (a) Создать класс-миксин `OverheatProtection`, который следит за температурой.
- (b) Определить метод `monitor_plate(self)`, который измеряет температуру подошвы.
- (c) Определить метод `cut_power(self)`, который отключает нагрев.
- (d) Определить метод `resume_heating(self)`, который возобновляет нагрев.
- (e) Создать экземпляр класса `OverheatProtection` и сохранить его в переменной `protect`.
- (f) Вызвать метод `monitor_plate()`.
- (g) Вызвать метод `cut_power()`.
- (h) Вызвать метод `resume_heating()`.

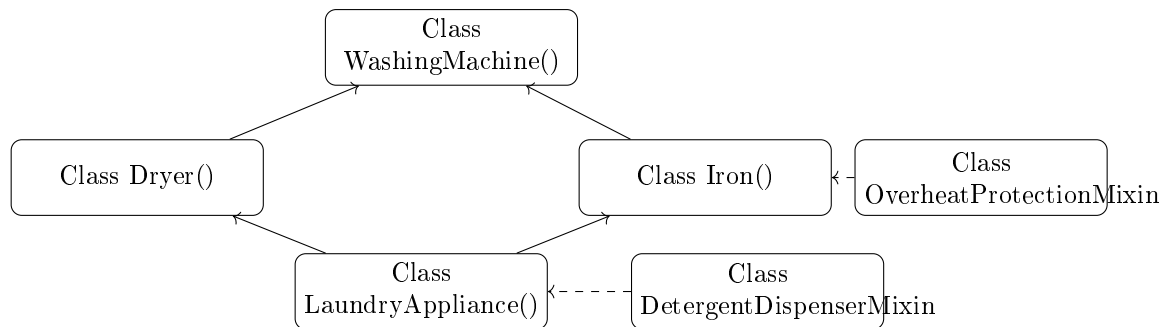
Задание 3

Используя программный код задания 26 из прошлого практикума текущего курса ООП, выполнить подмешивание класса-миксина `OverheatProtection` к классу `Iron` и подмешивание класса-миксина `DetergentDispenser` к классу `LaundryAppliance(WashingMachine, Dryer, Iron)`. При запуске класса `LaundryAppliance` должна быть отображена в консоли следующая последовательность действий.

Пример вывода программы

```
Температура подошвы: 210°C.  
Нагрев отключён.  
Нагрев возобновлён.  
Средство загружено.  
Дозирование выполнено.  
Дозаправка не требуется.  
Бельё загружено  
Стирка начата  
Сушка завершена  
Глажка выполнена  
Устройство остыло  
Drum size: 8 kg, Heat type: Condenser, Steam output: High
```

Схема классов



27 Задание 1

Написать класс-миксин на Python, который моделирует работу системы фильтрации воды в пароочистителе. Создать класс-миксин **WaterFilter**, который будет содержать методы для установки фильтра, проверки качества воды и замены картриджа.

- (a) Класс **WaterFilter** определяет объект, который моделирует фильтрацию.
- (b) Метод **install_filter** устанавливает фильтр.
- (c) Метод **check_purity** проверяет качество воды.
- (d) Метод **replace_cartridge** заменяет картридж.
- (e) В примере использования создаётся объект **filter**, после чего вызываются методы.

Задание 2

Написать класс-миксин на Python, который моделирует работу системы картографирования помещений в роботе-мойщике.

- (a) Создать класс-миксин **MappingSystem**, который строит карту.
- (b) Определить метод **scan_room(self)**, который сканирует помещение.
- (c) Определить метод **build_map(self)**, который создаёт карту.
- (d) Определить метод **plan_route(self)**, который строит маршрут уборки.
- (e) Создать экземпляр класса **MappingSystem** и сохранить его в переменной **mapsys**.
- (f) Вызвать метод **scan_room()**.
- (g) Вызвать метод **build_map()**.
- (h) Вызвать метод **plan_route()**.

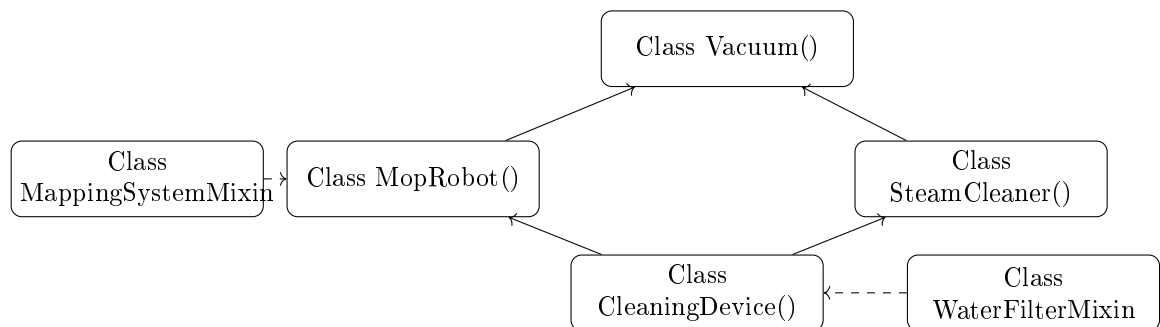
Задание 3

Используя программный код задания 27 из прошлого практикума текущего курса ООП, выполнить подмешивание класса-миксина **MappingSystem** к классу **MopRobot** и подмешивание класса-миксина **WaterFilter** к классу **CleaningDevice(Vacuum, MopRobot, SteamCleaner)**. При запуске класса **CleaningDevice** должна быть отображена в консоли следующая последовательность действий.

Пример вывода программы

Помещение отсканировано.
Карта построена.
Маршрут запланирован.
Фильтр установлен.
Чистота воды: высокая.
Картридж в порядке.
Всасывание начато
Пол вымыт
Паровая очистка выполнена
Контейнер опорожнён
Подзарядка завершена
Suction power: 200 W, Water tank: 0.8 L, Pressure: 4 bar

Схема классов



28 Задание 1

Написать класс-миксин на Python, который моделирует работу системы планирования графика работы климатической техники. Создать класс-миксин **ScheduleManager**, который будет содержать методы для установки времени включения, настройки дней недели и сохранения расписания.

- (a) Класс **ScheduleManager** определяет объект, который моделирует планировщик.
- (b) Метод **set_time** устанавливает время.
- (c) Метод **set_days** задаёт дни недели.
- (d) Метод **save_schedule** сохраняет расписание.
- (e) В примере использования создаётся объект **schedule**, после чего вызываются методы.

Задание 2

Написать класс-миксин на Python, который моделирует работу системы контроля качества воздуха в очистителе.

- (a) Создать класс-миксин `AirQualitySensor`, который измеряет параметры воздуха.
- (b) Определить метод `measure_pm25(self)`, который измеряет пыль.
- (c) Определить метод `measure_voc(self)`, который измеряет летучие соединения.
- (d) Определить метод `report_index(self)`, который выводит индекс качества.
- (e) Создать экземпляр класса `AirQualitySensor` и сохранить его в переменной `air`.
- (f) Вызвать метод `measure_pm25()`.
- (g) Вызвать метод `measure_voc()`.
- (h) Вызвать метод `report_index()`.

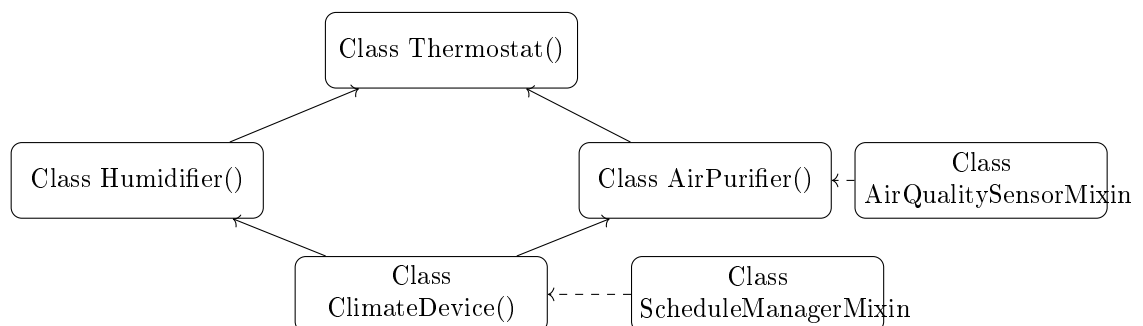
Задание 3

Используя программный код задания 28 из прошлого практикума текущего курса ООП, выполнить подмешивание класса-миксина `AirQualitySensor` к классу `AirPurifier` и подмешивание класса-миксина `ScheduleManager` к классу `ClimateDevice(Thermostat, Humidifier, AirPurifier)`. При запуске класса `ClimateDevice` должна быть отображена в консоли следующая последовательность действий.

Пример вывода программы

```
PM2.5: 12 мкг/м³.
ЛОС: 0.3 мг/м³.
Индекс качества: Отличный.
Время включения: 07:00.
Дни: Пн-Пт.
Расписание сохранено.
Температура установлена
Увлажнение начато
Воздух очищен
Параметры считаны
Резервуар пополнен
Temp range: 18-30°C, Tank liters: 4, Filter HEPA: True
```

Схема классов



29 Задание 1

Написать класс-миксин на Python, который моделирует работу системы распознавания лиц в умном замке. Создать класс-миксин `FaceRecognition`, который будет содержать методы для сканирования лица, верификации и разблокировки двери.

- (a) Класс `FaceRecognition` определяет объект, который моделирует Face ID для замка.
- (b) Метод `scan_visitor` сканирует лицо у двери.
- (c) Метод `verify_identity` проверяет соответствие базе.
- (d) Метод `unlock_if_authorized` разблокирует, если лицо разрешено.
- (e) В примере использования создаётся объект `face`, после чего вызываются методы.

Задание 2

Написать класс-миксин на Python, который моделирует работу системы хранения видео в камере наблюдения.

- (a) Создать класс-миксин `VideoStorage`, который управляет архивом.
- (b) Определить метод `start_recording(self)`, который записывает видео.
- (c) Определить метод `compress_files(self)`, который сжимает архив.
- (d) Определить метод `rotate_archive(self)`, который удаляет старые записи.
- (e) Создать экземпляр класса `VideoStorage` и сохранить его в переменной `storage`.
- (f) Вызвать метод `start_recording()`.
- (g) Вызвать метод `compress_files()`.
- (h) Вызвать метод `rotate_archive()`.

Задание 3

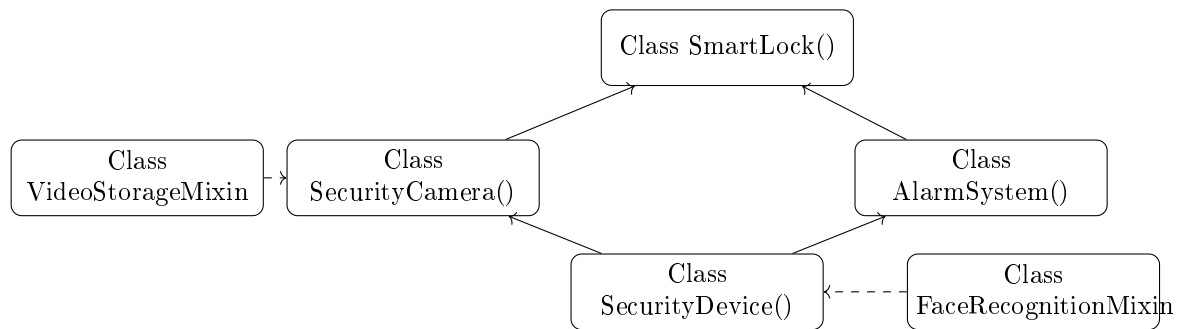
Используя программный код задания 29 из прошлого практикума текущего курса ООП, выполнить подмешивание класса-миксина `VideoStorage` к классу `SecurityCamera` и подмешивание класса-миксина `FaceRecognition` к классу `SecurityDevice(SmartLock, SecurityCamera, AlarmSystem)`. При запуске класса `SecurityDevice` должна быть отображена в консоли следующая последовательность действий.

Пример вывода программы

```
Запись начата.  
Архив сжат.  
Старые записи удалены.  
Лицо отсканировано.  
Идентичность подтверждена.  
Дверь разблокирована.  
Дверь заблокирована  
Доступ предоставлен
```

Видео записано
Оповещение отправлено
Сирена активирована
Unlock method: Biometric, Night vision: Yes, Siren dB: 120

Схема классов



30 Задание 1

Написать класс-миксин на Python, который моделирует работу системы музыкальной синхронизации в освещении. Создать класс-миксин **MusicSync**, который будет содержать методы для подключения к аудиисточнику, анализа ритма и синхронизации цвета.

- (a) Класс **MusicSync** определяет объект, который моделирует синхронизацию света с музыкой.
- (b) Метод **connect_audio** подключается к источнику.
- (c) Метод **analyze_beat** определяет ритм.
- (d) Метод **sync_light** меняет цвет в такт музыке.
- (e) В примере использования создаётся объект **sync**, после чего вызываются методы.

Задание 2

Написать класс-миксин на Python, который моделирует работу системы учёта энергопотребления в умной розетке.

- (a) Создать класс-миксин **PowerMeter**, который измеряет потребление.
- (b) Определить метод **measure_watts(self)**, который измеряет мощность.
- (c) Определить метод **log_daily_usage(self)**, который записывает дневное потребление.
- (d) Определить метод **alert_overload(self)**, который срабатывает при перегрузке.
- (e) Создать экземпляр класса **PowerMeter** и сохранить его в переменной **meter**.
- (f) Вызвать метод **measure_watts()**.
- (g) Вызвать метод **log_daily_usage()**.
- (h) Вызвать метод **alert_overload()**.

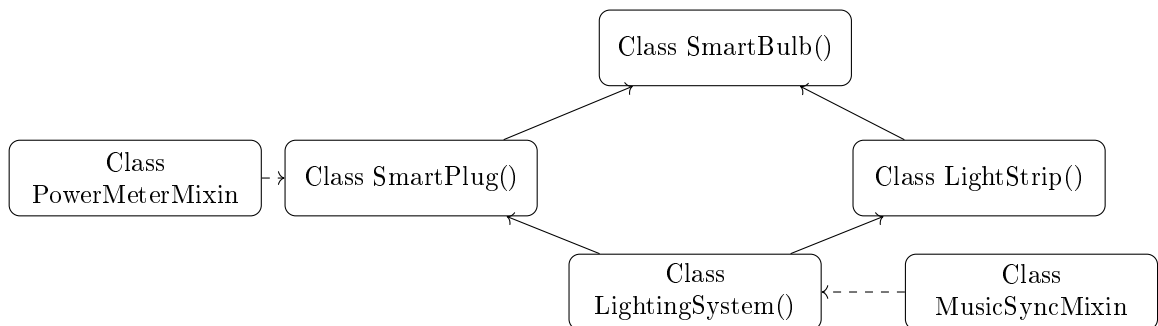
Задание 3

Используя программный код задания 30 из прошлого практикума текущего курса ООП, выполнить подмешивание класса-миксина `PowerMeter` к классу `SmartPlug` и подмешивание класса-миксина `MusicSync` к классу `LightingSystem(SmartBulb, SmartPlug, LightStrip)`. При запуске класса `LightingSystem` должна быть отображена в консоли следующая последовательность действий.

Пример вывода программы

Мощность: 85 Вт.
Потребление за день: 2.1 кВт·ч.
Перегрузка не обнаружена.
Аудиоисточник подключён.
Ритм распознан.
Свет синхронизирован.
Лампа включена
Цвет установлен
Анимация запущена
Питание подано
Потребление учтено
Color temp: 2700K, Max wattage: 2000 W, Length: 5 m

Схема классов



31 Задание 1

Написать класс-миксин на Python, который моделирует работу системы облачных сохранений в игровом устройстве. Создать класс-миксин `CloudSave`, который будет содержать методы для сохранения прогресса, синхронизации между устройствами и восстановления данных.

- (a) Класс `CloudSave` определяет объект, который моделирует облачное сохранение.
- (b) Метод `save_progress` сохраняет игру в облако.
- (c) Метод `sync_devices` синхронизирует данные.

- (d) Метод `restore_game` восстанавливает сохранение.
- (e) В примере использования создаётся объект `cloud`, после чего вызываются методы.

Задание 2

Написать класс-миксин на Python, который моделирует работу системы отслеживания времени игры.

- (a) Создать класс-миксин `PlayTimeTracker`, который контролирует сессии.
- (b) Определить метод `start_session(self)`, который запускает отсчёт.
- (c) Определить метод `pause_session(self)`, который приостанавливает.
- (d) Определить метод `report_hours(self)`, который выводит статистику.
- (e) Создать экземпляр класса `PlayTimeTracker` и сохранить его в переменной `tracker`.
- (f) Вызвать метод `start_session()`.
- (g) Вызвать метод `pause_session()`.
- (h) Вызвать метод `report_hours()`.

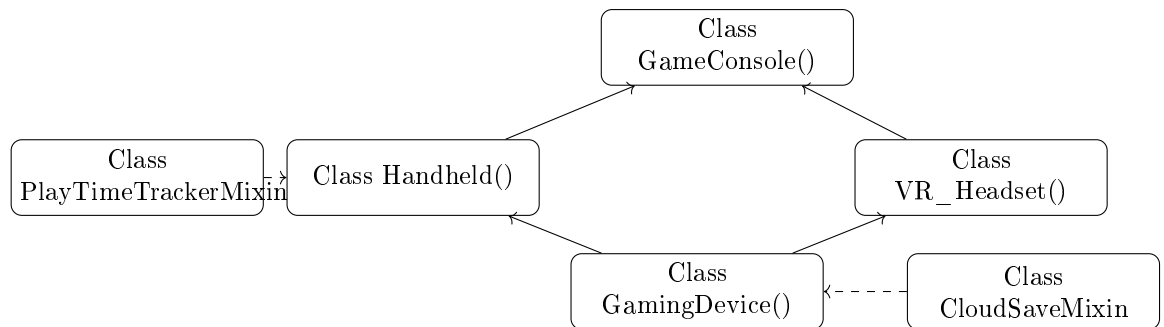
Задание 3

Используя программный код задания 31 из прошлого практикума текущего курса ООП, выполнить подмешивание класса-миксина `PlayTimeTracker` к классу `Handheld` и подмешивание класса-миксина `CloudSave` к классу `GamingDevice(GameConsole, Handheld, VR_Headset)`. При запуске класса `GamingDevice` должна быть отображена в консоли следующая последовательность действий.

Пример вывода программы

```
Сессия начата.  
Сессия приостановлена.  
Игровое время: 4.5 ч.  
Прогресс сохранён.  
Синхронизация выполнена.  
Сохранение восстановлено.  
Система загружена  
Игра запущена  
VR активирован  
Портативная сессия начата  
Режим сна включён  
GPU TFLOPS: 12, Screen refresh: 120 Hz, FOV: 110°
```


Схема классов



32 Задание 1

Написать класс-миксин на Python, который моделирует работу системы автоматического тюнинга в электрогитаре. Создать класс-миксин **AutoTuner**, который будет содержать методы для анализа звука, сравнения с эталоном и коррекции строя.

- Класс **AutoTuner** определяет объект, который моделирует автотюннер.
- Метод **detect_pitch** определяет текущую ноту.
- Метод **compare_to_standard** сравнивает с эталоном.
- Метод **adjust_string** корректирует натяжение струны.
- В примере использования создаётся объект **tuner**, после чего вызываются методы.

Задание 2

Написать класс-миксин на Python, который моделирует работу системы записи в драм-машине.

- Создать класс-миксин **PatternRecorder**, который записывает ритм-паттерны.
- Определить метод **start_recording(self)**, который начинает запись.
- Определить метод **save_pattern(self)**, который сохраняет паттерн.
- Определить метод **play_back(self)**, который воспроизводит запись.
- Создать экземпляр класса **PatternRecorder** и сохранить его в переменной **recorder**.
- Вызвать метод **start_recording()**.
- Вызвать метод **save_pattern()**.
- Вызвать метод **play_back()**.

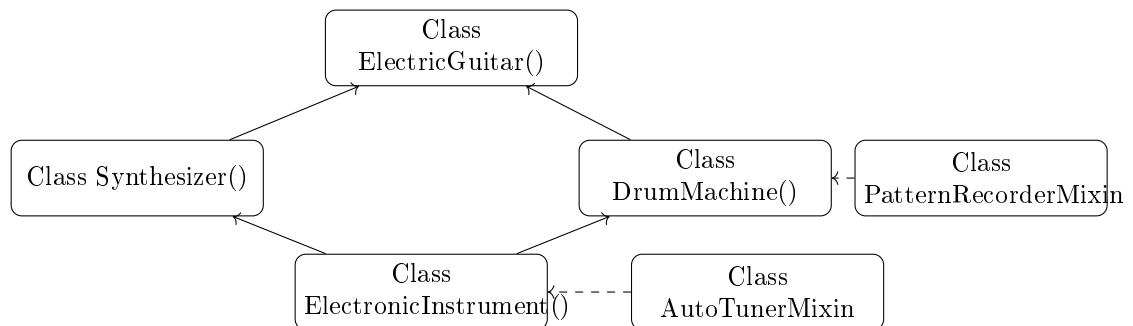
Задание 3

Используя программный код задания 32 из прошлого практикума текущего курса ООП, выполнить подмешивание класса-миксина **PatternRecorder** к классу **DrumMachine** и подмешивание класса-миксина **AutoTuner** к классу **ElectronicInstrument(ElectricGuitar, Synthesizer, DrumMachine)**. При запуске класса **ElectronicInstrument** должна быть отображена в консоли следующая последовательность действий.

Пример вывода программы

Запись начата.
Паттерн сохранён.
Воспроизведение запущено.
Нота: E2.
Отклонение: +3 цента.
Строй скорректирован.
Струны настроены
Пресет выбран
Бит запрограммирован
Модуляция активирована
Игра начата
Pickup type: Humbucker, Polyphony: 128, Sample quality: 24-bit

Схема классов



33 Задание 1

Написать класс-миксин на Python, который моделирует работу системы распознавания присутствия в умном доме. Создать класс-миксин **PresenceDetector**, который будет содержать методы для обнаружения движения, определения количества людей и настройки режимов.

- (a) Класс **PresenceDetector** определяет объект, который моделирует детектор присутствия.
- (b) Метод **detect_motion** обнаруживает перемещение.
- (c) Метод **count_occupants** оценивает число людей.
- (d) Метод **set_mode** устанавливает режим «Дома» или «Отсутствие».
- (e) В примере использования создаётся объект **presence**, после чего вызываются методы.

Задание 2

Написать класс-миксин на Python, который моделирует работу системы прогноза погоды в умных жалюзи.

- (a) Создать класс-миксин `WeatherForecast`, который получает данные о погоде.
- (b) Определить метод `get_forecast(self)`, который запрашивает прогноз.
- (c) Определить метод `adjust_for_sun(self)`, который настраивает жалюзи по солнцу.
- (d) Определить метод `protect_from_rain(self)`, который закрывает при дожде.
- (e) Создать экземпляр класса `WeatherForecast` и сохранить его в переменной `weather`.
- (f) Вызвать метод `get_forecast()`.
- (g) Вызвать метод `adjust_for_sun()`.
- (h) Вызвать метод `protect_from_rain()`.

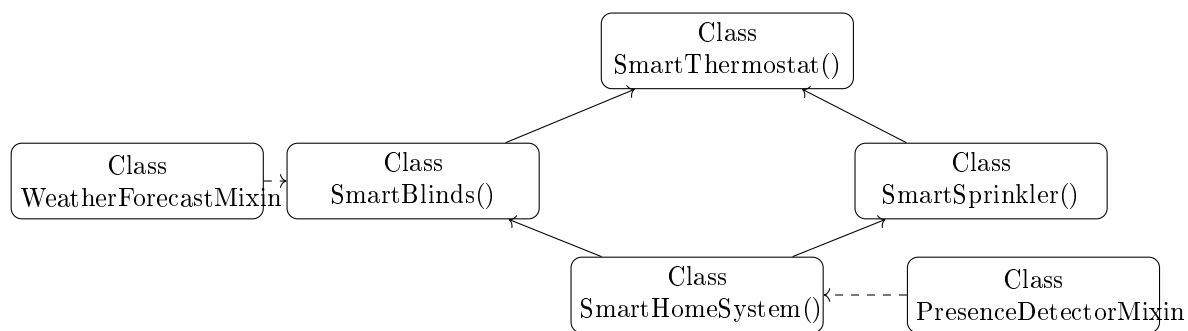
Задание 3

Используя программный код задания 33 из прошлого практикума текущего курса ООП, выполнить подмешивание класса-миксина `WeatherForecast` к классу `SmartBlinds` и подмешивание класса-миксина `PresenceDetector` к классу `SmartHomeSystem(SmartThermostat, SmartBlinds, SmartSprinkler)`. При запуске класса `SmartHomeSystem` должна быть отображена в консоли следующая последовательность действий.

Пример вывода программы

```
Прогноз получен: солнечно.  
Жалюзи настроены на солнце.  
Дождь не ожидается.  
Движение обнаружено.  
Людей: 2.  
Режим: Дома.  
Расписание изучено  
Жалюзи открыты  
Солнце синхронизировано  
Газон полит  
Температура скорректирована  
Learning mode: Adaptive, Motor type: Stepper, Zone count: 4
```

Схема классов



34 Задание 1

Написать класс-миксин на Python, который моделирует работу системы геолокации в персональном электротранспорте. Создать класс-миксин **GeoLocator**, который будет содержать методы для определения местоположения, отслеживания маршрута и поиска транспорта.

- (a) Класс **GeoLocator** определяет объект, который моделирует GPS-трекер.
- (b) Метод **get_coordinates** получает текущие координаты.
- (c) Метод **track_route** записывает пройденный путь.
- (d) Метод **locate_device** помогает найти транспорт при утере.
- (e) В примере использования создаётся объект **geo**, после чего вызываются методы.

Задание 2

Написать класс-миксин на Python, который моделирует работу системы балансировки в сиквее.

- (a) Создать класс-миксин **BalanceSystem**, который поддерживает равновесие.
- (b) Определить метод **calibrate_sensors(self)**, который калибрует гироскопы.
- (c) Определить метод **adjust_motors(self)**, который регулирует мощность моторов.
- (d) Определить метод **enter_standby(self)**, который переводит в режим ожидания.
- (e) Создать экземпляр класса **BalanceSystem** и сохранить его в переменной **balance**.
- (f) Вызвать метод **calibrate_sensors()**.
- (g) Вызвать метод **adjust_motors()**.
- (h) Вызвать метод **enter_standby()**.

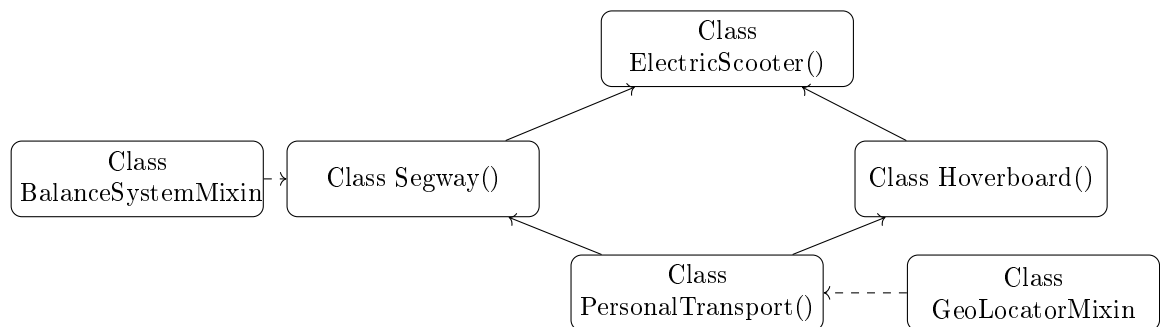
Задание 3

Используя программный код задания 34 из прошлого практикума текущего курса ООП, выполнить подмешивание класса-миксина **BalanceSystem** к классу **Segway** и подмешивание класса-миксина **GeoLocator** к классу **PersonalTransport(ElectricScooter, Segway, Hoverboard)**. При запуске класса **PersonalTransport** должна быть отображена в консоли следующая последовательность действий.

Пример вывода программы

Гироскопы откалиброваны.
Моторы сбалансированы.
Режим ожидания активирован.
Координаты: 55.7558° N, 37.6176° E.
Маршрут записан.
Транспорт найден.
Поездка разблокирована
Балансировка начата
Ускорение выполнено
Музыка включена
Парковка завершена
Max speed: 25 km/h, Balance type: Dynamic, Wheel size: 8.5"

Схема классов



35 Задание 1

Написать класс-миксин на Python, который моделирует работу системы управления вкусом в кофемашине. Создать класс-миксин **FlavorProfile**, который будет содержать методы для выбора профиля вкуса, настройки крепости и температуры подачи.

- (a) Класс **FlavorProfile** определяет объект, который моделирует настройку вкуса.
- (b) Метод **select_profile** выбирает профиль: «сбалансированный», «интенсивный» и т.д.
- (c) Метод **set_strength** устанавливает крепость от 1 до 10.
- (d) Метод **adjust_serving_temp** настраивает температуру подачи.
- (e) В примере использования создаётся объект **flavor**, после чего вызываются методы.

Задание 2

Написать класс-миксин на Python, который моделирует работу системы самоочистки в соковыжималке.

- (a) Создать класс-миксин `SelfCleanJuicer`, который управляет очисткой.
- (b) Определить метод `start_rinse(self)`, который запускает промывку.
- (c) Определить метод `flush_system(self)`, который промывает систему водой.
- (d) Определить метод `dry_components(self)`, который сушит детали.
- (e) Создать экземпляр класса `SelfCleanJuicer` и сохранить его в переменной `clean`.
- (f) Вызвать метод `start_rinse()`.
- (g) Вызвать метод `flush_system()`.
- (h) Вызвать метод `dry_components()`.

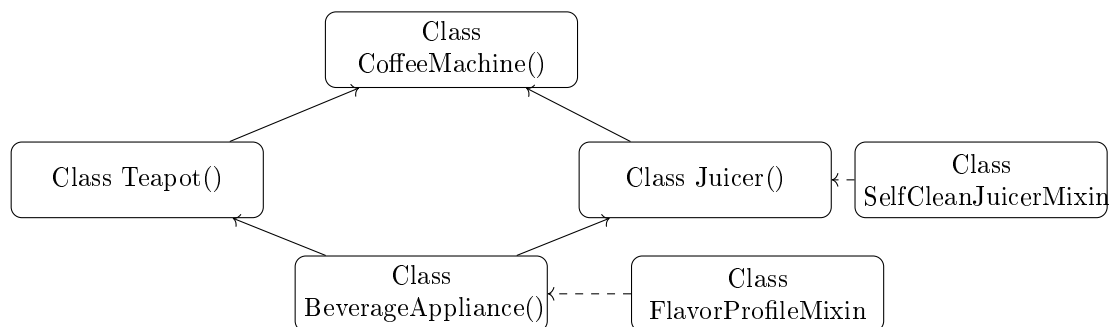
Задание 3

Используя программный код задания 35 из прошлого практикума текущего курса ООП, выполнить подмешивание класса-миксина `SelfCleanJuicer` к классу `Juicer` и подмешивание класса-миксина `FlavorProfile` к классу `BeverageAppliance` (`CoffeeMachine`, `Teapot`, `Juicer`). При запуске класса `BeverageAppliance` должна быть отображена в консоли следующая последовательность действий.

Пример вывода программы

```
Промывка начата.
Система промыта.
Детали высушены.
Профиль: Интенсивный.
Крепость: 8.
Температура подачи: 65°C.
Зёрна помолоты
Вода вскипячена
Сок выжат
Кофе заварен
Устройство выключено
Bean type: Arabica, Temp control: Yes, RPM speed: 12000
```

Схема классов



Вот полный исправленный текст ****Списка 3**** — задачи 1–35, в которых каждая задача соответствует тематике одноимённой задачи из Списка 1 (ООР-иерархия) и имеет корректную структуру (1/2/3 части, формулы, параметры, коэффициенты и т.д.).

2.11 Семинар «Классы-миксины и множественное наследование (продолжение)» (2 часа)

1 часть Написать класс Python `CarAcceleration`, выполняющий расчёт времени разгона автомобиля от 0 до 100 км/ч и от 100 до 200 км/ч. **Расчёт времени от 0 до 100 км/ч:**

$$t = \frac{m \cdot (\Delta V)^2}{2P_{\text{эфф}}}, \quad (1)$$

где:

- t — время разгона, с;
- m — масса автомобиля, кг;
- ΔV — изменение скорости, м/с;
- $P_{\text{эфф}}$ — эффективная мощность двигателя, Вт.

Формула расчёта эффективной мощности двигателя для ускорения от 0 до 100 км/ч:

$$P_{\text{эфф}} = P_{\text{макс}} \cdot k \cdot \eta, \quad (2)$$

где:

- $P_{\text{макс}}$ — максимальная мощность двигателя, л. с.;
- $k = 0,25$ — коэффициент использования мощности;
- $\eta = 0,8$ — КПД трансмиссии;
- 1 л. с. = 735 Вт.

Расчёт времени от 100 до 200 км/ч:

$$t = \frac{m \cdot (V_2^2 - V_1^2)}{2P_{\text{эфф}}}, \quad (3)$$

где:

- V_1 — начальная скорость, м/с;
- V_2 — конечная скорость, м/с;
- $P_{\text{эфф}}$ — эффективная мощность двигателя, Вт.

Формула расчёта эффективной мощности для ускорения от 100 до 200 км/ч:

$$P_{\text{эфф}} = P_{\text{макс}} \cdot k \cdot \eta, \quad (4)$$

где $k = 0,66$ — коэффициент использования мощности для высокоскоростного диапазона. **2 часть** На основе программного кода предыдущего практикума текущего курса ООП выполнить наследование класса `CarAcceleration` с использованием `super()` таким образом, чтобы через класс с множественным наследованием `Car` вывести на печать время ускорения до 100 км/ч и от 100 до 200 км/ч для трёх типов автомобилей при следующих параметрах:

- Sedan — масса 1100 кг, мощность двигателя 100 л. с.;
- Hatchback — масса 1200 кг, мощность двигателя 150 л. с.;
- SUV — масса 1700 кг, мощность двигателя 370 л. с.

3 часть На основе программного кода и параметров автомобилей из 2 части текущего практикума отобразить через класс с множественным наследованием `Car` расчёт времени разгона автомобилей от 100 до 200 км/ч через каждые 20 км/ч, то есть время разгона до 120, 140, 160, 180 и 200 км/ч, учитывая, что коэффициент мощности двигателя $k = 0,66$ увеличивается через каждые 20 км/ч на 0,08. **Коэффициенты мощности для различных диапазонов скоростей:**

- 100–120 км/ч: $k = 0,66$;
- 120–140 км/ч: $k = 0,74$;
- 140–160 км/ч: $k = 0,82$;
- 160–180 км/ч: $k = 0,90$;
- 180–200 км/ч: $k = 0,98$.

2 1 часть Написать класс Python `GreenCarAcceleration`, выполняющий расчёт времени разгона экологичного автомобиля от 0 до 100 км/ч и от 100 до 180 км/ч. **Расчёт времени от 0 до 100 км/ч:**

$$t = \frac{m \cdot (\Delta V)^2}{2P_{\text{эфф}}}, \quad (5)$$

где:

- t — время разгона, с;
- m — масса автомобиля, кг;
- ΔV — изменение скорости, м/с;
- $P_{\text{эфф}}$ — эффективная мощность, Вт.

Формула расчёта эффективной мощности для разгона от 0 до 100 км/ч:

$$P_{\text{эфф}} = P_{\text{макс}} \cdot k \cdot \eta, \quad (6)$$

где:

- $P_{\text{макс}}$ — максимальная мощность (электромотор + ДВС), л. с.;
- $k = 0,26$ — коэффициент использования мощности;
- $\eta = 0,82$ — КПД трансмиссии;
- 1 л. с. = 735 Вт.

Расчёт времени от 100 до 180 км/ч:

$$t = \frac{m \cdot (V_2^2 - V_1^2)}{2P_{\text{эфф}}}, \quad (7)$$

где $V_1 = 100$, $V_2 = 180$ км/ч (в м/с). **Коэффициент $k = 0,64$** для высокоскоростного режима. **2 часть** Через класс `GreenCar` вывести время разгона для:

- `ElectricCar` — масса 1600 кг, мощность 200 л. с.;

- HybridCar — масса 1800 кг, мощность 250 л. с.;
- FuelCellCar — масса 1700 кг, мощность 220 л. с.

3 часть Расчёт от 100 до 180 км/ч через каждые 20 км/ч. Коэффициент $k = 0,64$ растёт на 0,08:

- 100–120: $k = 0,64$
- 120–140: $k = 0,72$
- 140–160: $k = 0,80$
- 160–180: $k = 0,88$

3 1 часть Написать класс Python `UtilityAcceleration`, выполняющий расчёт времени разгона коммерческого транспорта от 0 до 60 км/ч и от 60 до 100 км/ч. **Для 0–60 км/ч:**

$$t = \frac{m \cdot (\Delta V)^2}{2P_{\text{эфф}}}, \quad k = 0,23, \eta = 0,80 \quad (8)$$

Для 60–100 км/ч:

$$t = \frac{m \cdot (V_2^2 - V_1^2)}{2P_{\text{эфф}}}, \quad k = 0,58 \quad (9)$$

2 часть Через `UtilityVehicle` для:

- Van — 2500 кг, 130 л. с.
- Pickup — 2200 кг, 180 л. с.
- Minivan — 2000 кг, 150 л. с.

3 часть Разгон от 60 до 100 км/ч через каждые 10 км/ч. $k = 0,58 \rightarrow +0,10$:

- 60–70: $k = 0,58$
- 70–80: $k = 0,68$
- 80–90: $k = 0,78$
- 90–100: $k = 0,88$

4 1 часть Написать класс Python `PerformanceAcceleration`, выполняющий расчёт времени разгона спортивного автомобиля от 0 до 100 км/ч и от 100 до 250 км/ч. **Для 0–100 км/ч:** $k = 0,30$, $\eta = 0,88$ **Для 100–250 км/ч:** $k = 0,70$ **2 часть** Через `PerformanceCar` для:

- SportsCar — 1400 кг, 450 л. с.
- Coupe — 1500 кг, 400 л. с.
- Convertible — 1600 кг, 380 л. с.

3 часть Разгон от 100 до 250 км/ч через каждые 30 км/ч. $k = 0,70 \rightarrow +0,10$

- 100–130: $k = 0,70$
- 130–160: $k = 0,80$
- 160–190: $k = 0,90$
- 190–220: $k = 1,00$

- 220–250: $k = 1,10$
- 5 **1 часть** Написать класс Python `TwoWheelerAcceleration`, выполняющий расчёт времени разгона двухколёсного транспорта от 0 до 60 км/ч и от 60 до 120 км/ч. **Для 0–60 км/ч:** $k = 0,28$, $\eta = 0,92$ **Для 60–120 км/ч:** $k = 0,55$ **2 часть** Через `TwoWheeler` для:
- Scooter — 100 кг, 500 Вт
 - Motorcycle — 220 кг, 80 л. с.
 - Moped — 120 кг, 5 л. с.
- 3 часть** Разгон от 60 до 120 км/ч через каждые 15 км/ч. $k = 0,55 \rightarrow +0,09$
- 60–75: $k = 0,55$
 - 75–90: $k = 0,64$
 - 90–105: $k = 0,73$
 - 105–120: $k = 0,82$
- 6 **1 часть** Написать класс Python `CycleAcceleration`, выполняющий расчёт времени разгона велосипеда от 0 до 20 км/ч и от 20 до 40 км/ч. **Для 0–20 км/ч:** $k = 0,30$, $\eta = 0,95$ **Для 20–40 км/ч:** $k = 0,50$ **2 часть** Через `Cycle` для:
- Bicycle — 80 кг, 200 Вт
 - Ebike — 25 кг, 250 Вт
 - Tandem — 130 кг, 350 Вт
- 3 часть** Разгон от 20 до 40 км/ч через каждые 5 км/ч. $k = 0,50 \rightarrow +0,04$
- 20–25: $k = 0,50$
 - 25–30: $k = 0,54$
 - 30–35: $k = 0,58$
 - 35–40: $k = 0,62$
- 7 **1 часть** Написать класс Python `RotorcraftAcceleration`, выполняющий расчёт времени набора высоты летательного аппарата с несущим винтом от 0 до 50 м и от 50 до 150 м. **Для 0–50 м:** $k = 0,25$, $\eta = 0,83$ **Для 50–150 м:** $k = 0,52$ **2 часть** Через `Rotorcraft` для:
- Drone — 1.2 кг, 1000 Вт
 - Helicopter — 2500 кг, 800 л. с.
 - Gyrocopter — 500 кг, 120 л. с.
- 3 часть** Набор высоты от 50 до 150 м через каждые 25 м. $k = 0,52 \rightarrow +0,10$
- 50–75: $k = 0,52$
 - 75–100: $k = 0,62$
 - 100–125: $k = 0,72$
 - 125–150: $k = 0,82$

8 **1 часть** Написать класс Python `FixedWingAcceleration`, выполняющий расчёт времени разгона самолёта с неподвижным крылом от 0 до 200 км/ч и от 200 до 800 км/ч. **Для 0–200 км/ч:** $k = 0,27$, $\eta = 0,85$ **Для 200–800 км/ч:** $k = 0,66$ **2 часть** Через `FixedWing` для:

- Airplane — 10000 кг, 3000 л. с.
- Jet — 15000 кг, 12000 л. с.
- Glider — 500 кг, буксировка 100 л. с.

3 часть Разгон от 200 до 800 км/ч через каждые 150 км/ч. $k = 0,66 \rightarrow +0,10$

- 200–350: $k = 0,66$
- 350–500: $k = 0,76$
- 500–650: $k = 0,86$
- 650–800: $k = 0,96$

9 **1 часть** Написать класс Python `VesselAcceleration`, выполняющий расчёт времени разгона водного транспорта от 0 до 30 узлов и от 30 до 50 узлов. **Для 0–30 узлов:** $k = 0,24$, $\eta = 0,78$ **Для 30–50 узлов:** $k = 0,58$ **2 часть** Через `Vessel` для:

- Cruiser — 20000 т, 50000 л. с.
- Ferry — 500 т, 10000 л. с.
- Yacht — 50 т, 2000 л. с.

3 часть Разгон от 30 до 50 узлов через каждые 5 узлов. $k = 0,58 \rightarrow +0,10$

- 30–35: $k = 0,58$
- 35–40: $k = 0,68$
- 40–45: $k = 0,78$
- 45–50: $k = 0,88$

10 **1 часть** Написать класс Python `AmphibiousAcceleration`, выполняющий расчёт времени разгона амфибии от 0 до 50 км/ч на суше и от 0 до 30 км/ч на воде. **На суше:** $k = 0,22$, $\eta = 0,80$ **На воде:** $k = 0,18$, $\eta = 0,70$ **2 часть** Через `AmphibiousCraft` для:

- Submarine — 1000 т, 5000 л. с.
- Hovercraft — 10 т, 1000 л. с.
- AmphibiousVehicle — 5 т, 300 л. с.

3 часть Разгон на суше от 0 до 50 км/ч через каждые 10 км/ч. $k = 0,22 \rightarrow +0,08$

- 0–10: $k = 0,22$
- 10–20: $k = 0,30$
- 20–30: $k = 0,38$
- 30–40: $k = 0,46$
- 40–50: $k = 0,54$

11 **1 часть** Написать класс Python `AgriculturalAcceleration`, выполняющий расчёт времени выхода сельскохозяйственной техники на рабочую скорость от 0 до 8 км/ч и от 8 до 16 км/ч. **Для 0–8 км/ч:** $k = 0,22$, $\eta = 0,82$ **Для 8–16 км/ч:** $k = 0,48$ **2 часть** Через `AgriculturalMachine` для:

- Tractor — 3500 кг, 120 л. с.
- Combine — 8500 кг, 350 л. с.
- Sprayer — 2200 кг, 90 л. с.

3 часть Разгон от 8 до 16 км/ч через каждые 2 км/ч. $k = 0,48 \rightarrow +0,06$

- 8–10: $k = 0,48$
- 10–12: $k = 0,54$
- 12–14: $k = 0,60$
- 14–16: $k = 0,66$

12 **1 часть** Написать класс Python `ConstructionAcceleration`, выполняющий расчёт времени разгона строительной техники от 0 до 10 км/ч и от 10 до 20 км/ч. **Для 0–10 км/ч:** $k = 0,20$, $\eta = 0,78$ **Для 10–20 км/ч:** $k = 0,52$ **2 часть** Через `ConstructionEquipment` для:

- Excavator — 12000 кг, 250 л. с.
- Bulldozer — 18000 кг, 400 л. с.
- Crane — 9000 кг, 200 л. с.

3 часть Разгон от 10 до 20 км/ч через каждые 2,5 км/ч. $k = 0,52 \rightarrow +0,08$

- 10–12,5: $k = 0,52$
- 12,5–15: $k = 0,60$
- 15–17,5: $k = 0,68$
- 17,5–20: $k = 0,76$

13 **1 часть** Написать класс Python `EmergencyAcceleration`, выполняющий расчёт времени разгона служебного транспорта от 0 до 60 км/ч и от 60 до 120 км/ч. **Для 0–60 км/ч:** $k = 0,28$, $\eta = 0,85$ **Для 60–120 км/ч:** $k = 0,64$ **2 часть** Через `EmergencyVehicle` для:

- Ambulance — 2800 кг, 180 л. с.
- FireTruck — 14000 кг, 450 л. с.
- PoliceCar — 2000 кг, 300 л. с.

3 часть Разгон от 60 до 120 км/ч через каждые 15 км/ч. $k = 0,64 \rightarrow +0,08$

- 60–75: $k = 0,64$
- 75–90: $k = 0,72$
- 90–105: $k = 0,80$
- 105–120: $k = 0,88$

14 **1 часть** Написать класс Python `PassengerAcceleration`, выполняющий расчёт времени разгона пассажирского транспорта от 0 до 30 км/ч и от 30 до 60 км/ч. **Для 0–30 км/ч:** $k = 0,26$, $\eta = 0,83$ **Для 30–60 км/ч:** $k = 0,60$ **2 часть** Через `PassengerVehicle` для:

- Taxi — 1600 кг, 120 л. с.
- RideShare — 1500 кг, 110 л. с.
- Limousine — 2500 кг, 200 л. с.

3 часть Разгон от 30 до 60 км/ч через каждые 7.5 км/ч. $k = 0,60 \rightarrow +0,08$

- 30–37.5: $k = 0,60$
- 37.5–45: $k = 0,68$
- 45–52.5: $k = 0,76$
- 52.5–60: $k = 0,84$

15 **1 часть** Написать класс Python `BusAcceleration`, выполняющий расчёт времени разгона автобуса от 0 до 25 км/ч и от 25 до 50 км/ч. **Для 0–25 км/ч:** $k = 0,24$, $\eta = 0,81$ **Для 25–50 км/ч:** $k = 0,58$ **2 часть** Через `Bus` для:

- SchoolBus — 7500 кг, 220 л. с.
- Coach — 12500 кг, 380 л. с.
- Minibus — 3200 кг, 130 л. с.

3 часть Разгон от 25 до 50 км/ч через каждые 5 км/ч. $k = 0,58 \rightarrow +0,06$

- 25–30: $k = 0,58$
- 30–35: $k = 0,64$
- 35–40: $k = 0,70$
- 40–45: $k = 0,76$
- 45–50: $k = 0,82$

16 **1 часть** Написать класс Python `FreightAcceleration`, выполняющий расчёт времени разгона грузового транспорта от 0 до 40 км/ч и от 40 до 80 км/ч. **Для 0–40 км/ч:** $k = 0,21$, $\eta = 0,79$ **Для 40–80 км/ч:** $k = 0,54$ **2 часть** Через `FreightVehicle` для:

- Truck — 12000 кг, 300 л. с.
- Semi — 36000 кг, 500 л. с.
- DumpTruck — 25000 кг, 400 л. с.

3 часть Разгон от 40 до 80 км/ч через каждые 10 км/ч. $k = 0,54 \rightarrow +0,08$

- 40–50: $k = 0,54$
- 50–60: $k = 0,62$
- 60–70: $k = 0,70$
- 70–80: $k = 0,78$

17 **1 часть** Написать класс Python `SpacecraftAcceleration`, выполняющий расчёт времени набора скорости космического аппарата от 0 до 1000 м/с и от 1000 до 3000 м/с. Для 0–1000 м/с: $k = 0,23$, $\eta = 0,92$ Для 1000–3000 м/с: $k = 0,62$ **2 часть** Через `Spacecraft` для:

- Rocket — 200000 кг, 25 МВт
- Spaceplane — 80000 кг, 18 МВт
- Lander — 15000 кг, 5 МВт

3 часть Набор скорости от 1000 до 3000 м/с через каждые 500 м/с. $k = 0,62 \rightarrow +0,08$

- 1000–1500: $k = 0,62$
- 1500–2000: $k = 0,70$
- 2000–2500: $k = 0,78$
- 2500–3000: $k = 0,86$

18 **1 часть** Написать класс Python `WearableAcceleration`, выполняющий расчёт времени перехода носимого устройства из спящего режима в активный и далее до полной функциональности.

Расчёт времени от спящего режима до активного:

$$t = \frac{C \cdot (\Delta V)^2}{2P_{\text{эфф}}}, \quad (10)$$

где:

- t — время пробуждения, с;
- C — эквивалентная ёмкость системы (моделирует инерцию старта), Дж/В²;
- ΔV — изменение уровня активности (условные единицы, от 0 до 1);
- $P_{\text{эфф}}$ — эффективная мощность, Вт.

Формула расчёта эффективной мощности для пробуждения:

$$P_{\text{эфф}} = P_{\text{макс}} \cdot k \cdot \eta, \quad (11)$$

где:

- $P_{\text{макс}}$ — пиковая мощность процессора, Вт;
- $k = 0,27$ — коэффициент использования при старте;
- $\eta = 0,95$ — КПД энергосистемы.

Расчёт времени от активного до полной функциональности:

$$t = \frac{C \cdot (V_2^2 - V_1^2)}{2P_{\text{эфф}}}, \quad (12)$$

где:

- $V_1 = 0,3$, $V_2 = 1,0$ — уровни активности;

- $P_{\text{эфф}}$ — эффективная мощность, Вт.

Формула расчёта эффективной мощности для полной активации:

$$P_{\text{эфф}} = P_{\text{макс}} \cdot k \cdot \eta, \quad (13)$$

где $k = 0,56$ — коэффициент при полной нагрузке.

2 часть

На основе программного кода предыдущего практикума текущего курса ООП выполнить наследование класса `WearableAcceleration` с использованием `super()` таким образом, чтобы через класс с множественным наследованием `WearableDevice` вывести на печать время перехода в активный режим и до полной функциональности для трёх типов устройств при следующих параметрах:

- Smartwatch — $C = 0,02$ Дж/В², $P = 2,5$ Вт;
- FitnessTracker — $C = 0,015$ Дж/В², $P = 1,8$ Вт;
- AR_Glasses — $C = 0,03$ Дж/В², $P = 4,0$ Вт.

3 часть

На основе программного кода и параметров устройств из 2 части текущего практикума отобразить через класс с множественным наследованием `WearableDevice` расчёт времени перехода от активного состояния до полной функциональности через каждые 0.15 условных единиц, то есть до 0.45, 0.60, 0.75, 0.90 и 1.00, учитывая, что коэффициент мощности $k = 0,56$ увеличивается через каждые 0.15 на 0,08.

Коэффициенты мощности для различных диапазонов активности:

- 0.30–0.45: $k = 0,56$;
- 0.45–0.60: $k = 0,64$;
- 0.60–0.75: $k = 0,72$;
- 0.75–0.90: $k = 0,80$;
- 0.90–1.00: $k = 0,88$.

19 **1 часть** Написать класс Python `MobileAcceleration`, выполняющий расчёт времени запуска мобильного устройства от нажатия кнопки до готовности к работе и до запуска сложного приложения.

Расчёт времени от нажатия кнопки до готовности:

$$t = \frac{C \cdot (\Delta V)^2}{2P_{\text{эфф}}}, \quad (14)$$

где:

- t — время загрузки, с;
- C — эквивалентная ёмкость системы (модель инициализации), Дж/В²;
- ΔV — изменение уровня готовности (от 0 до 1);
- $P_{\text{эфф}}$ — эффективная мощность, Вт.

Формула расчёта эффективной мощности для загрузки ОС:

$$P_{\text{эфф}} = P_{\text{макс}} \cdot k \cdot \eta, \quad (15)$$

где:

- $P_{\text{макс}}$ — пиковая мощность, Вт;
- $k = 0,29$ — коэффициент при старте;
- $\eta = 0,93$ — КПД энергосистемы.

Расчёт времени от готовности до запуска приложения:

$$t = \frac{C \cdot (V_2^2 - V_1^2)}{2P_{\text{эфф}}}, \quad (16)$$

где:

- $V_1 = 0,6$, $V_2 = 1,0$ — уровни загрузки;
- $P_{\text{эфф}}$ — эффективная мощность, Вт.

Формула расчёта эффективной мощности для запуска приложения:

$$P_{\text{эфф}} = P_{\text{макс}} \cdot k \cdot \eta, \quad (17)$$

где $k = 0,59$ — коэффициент при полной загрузке.

2 часть

На основе программного кода предыдущего практикума текущего курса ООП выполнить наследование класса `MobileAcceleration` с использованием `super()` таким образом, чтобы через класс с множественным наследованием `MobileDevice` вывести на печать время загрузки и запуска приложения для трёх типов устройств при следующих параметрах:

- Smartphone — $C = 0,018$ Дж/В², $P = 5,0$ Вт;
- Tablet — $C = 0,025$ Дж/В², $P = 8,0$ Вт;
- Ereader — $C = 0,008$ Дж/В², $P = 1,5$ Вт.

3 часть

На основе программного кода и параметров устройств из 2 части текущего практикума отобразить через класс с множественным наследованием `MobileDevice` расчёт времени запуска приложения от уровня 0.6 до 1.0 через каждые 0.1, учитывая, что коэффициент мощности $k = 0,59$ увеличивается через каждые 0.1 на 0,07.

Коэффициенты мощности для различных диапазонов загрузки:

- 0.6–0.7: $k = 0,59$;
- 0.7–0.8: $k = 0,66$;
- 0.8–0.9: $k = 0,73$;
- 0.9–1.0: $k = 0,80$.

20 **1 часть** Написать класс Python `ComputerAcceleration`, выполняющий расчёт времени запуска вычислительного устройства от включения питания до полной готовности и до выполнения ресурсоёмкой задачи.

Расчёт времени от включения до готовности:

$$t = \frac{C \cdot (\Delta V)^2}{2P_{\text{эфф}}}, \quad (18)$$

где:

- t — время запуска, с;
- C — эквивалентная ёмкость системы, Дж/В²;
- ΔV — изменение уровня активности (от 0 до 1);
- $P_{\text{эфф}}$ — эффективная мощность, Вт.

Формула расчёта эффективной мощности для загрузки:

$$P_{\text{эфф}} = P_{\text{макс}} \cdot k \cdot \eta, \quad (19)$$

где:

- $P_{\text{макс}}$ — пиковая мощность, Вт;
- $k = 0,25$ — коэффициент при старте;
- $\eta = 0,90$ — КПД блока питания.

Расчёт времени от готовности до выполнения задачи:

$$t = \frac{C \cdot (V_2^2 - V_1^2)}{2P_{\text{эфф}}}, \quad (20)$$

где:

- $V_1 = 0,5$, $V_2 = 1,0$ — уровни нагрузки;
- $P_{\text{эфф}}$ — эффективная мощность, Вт.

Формула расчёта эффективной мощности для выполнения задачи:

$$P_{\text{эфф}} = P_{\text{макс}} \cdot k \cdot \eta, \quad (21)$$

где $k = 0,61$ — коэффициент при высокой нагрузке.

2 часть

На основе программного кода предыдущего практикума текущего курса ООП выполнить наследование класса `ComputerAcceleration` с использованием `super()` таким образом, чтобы через класс с множественным наследованием `Computer` вывести на печать время запуска и выполнения задачи для трёх типов устройств при следующих параметрах:

- Laptop — $C = 0,03$ Дж/В², $P = 65$ Вт;
- Desktop — $C = 0,08$ Дж/В², $P = 650$ Вт;

- Workstation — $C = 0,12$ Дж/В², $P = 1200$ Вт.

3 часть

На основе программного кода и параметров устройств из 2 части текущего практикума отобразить через класс с множественным наследованием `Computer` расчёт времени выполнения задачи от уровня 0.5 до 1.0 через каждые 0.125, учитывая, что коэффициент мощности $k = 0,61$ увеличивается через каждые 0.125 на 0,07.

Коэффициенты мощности для различных диапазонов нагрузки:

- 0.50–0.625: $k = 0,61$;
- 0.625–0.75: $k = 0,68$;
- 0.75–0.875: $k = 0,75$;
- 0.875–1.00: $k = 0,82$.

- 21 **1 часть** Написать класс Python `NetworkAcceleration`, выполняющий расчёт времени подключения сетевого устройства к сети и времени полной активации всех сервисов.

Расчёт времени от выключения до подключения к сети:

$$t = \frac{C \cdot (\Delta V)^2}{2P_{\text{эфф}}}, \quad (22)$$

где:

- t — время подключения, с;
- C — эквивалентная ёмкость системы (модель инициализации), Дж/В²;
- ΔV — изменение уровня активности от 0 до 0.6;
- $P_{\text{эфф}}$ — эффективная мощность, Вт.

Формула расчёта эффективной мощности для подключения к сети:

$$P_{\text{эфф}} = P_{\text{макс}} \cdot k \cdot \eta, \quad (23)$$

где:

- $P_{\text{макс}}$ — пиковая мощность устройства, Вт;
- $k = 0,25$ — коэффициент использования при старте;
- $\eta = 0,94$ — КПД блока питания.

Расчёт времени от подключения до полной активации сервисов:

$$t = \frac{C \cdot (V_2^2 - V_1^2)}{2P_{\text{эфф}}}, \quad (24)$$

где:

- $V_1 = 0,6$, $V_2 = 1,0$ — уровни активации;
- $P_{\text{эфф}}$ — эффективная мощность, Вт.

Формула расчёта эффективной мощности для полной активации:

$$P_{\text{эфф}} = P_{\text{макс}} \cdot k \cdot \eta, \quad (25)$$

где $k = 0,60$ — коэффициент использования при полной нагрузке.

2 часть

На основе программного кода предыдущего практикума текущего курса ООП выполнить наследование класса `NetworkAcceleration` с использованием `super()` таким образом, чтобы через класс с множественным наследованием `NetworkDevice` вывести на печать время подключения к сети и время полной активации сервисов для трёх типов устройств при следующих параметрах:

- Router — $C = 0,008$ Дж/В², $P = 12$ Вт;
- Switch — $C = 0,012$ Дж/В², $P = 25$ Вт;
- Firewall — $C = 0,018$ Дж/В², $P = 40$ Вт.

3 часть

На основе программного кода и параметров устройств из 2 части текущего практикума отобразить через класс с множественным наследованием `NetworkDevice` расчёт времени активации сервисов от уровня 0.6 до 1.0 через каждые 0.1, учитывая, что коэффициент мощности $k = 0,60$ увеличивается через каждые 0.1 на 0,07.

Коэффициенты мощности для различных диапазонов активации:

- 0.6–0.7: $k = 0,60$;
- 0.7–0.8: $k = 0,67$;
- 0.8–0.9: $k = 0,74$;
- 0.9–1.0: $k = 0,81$.

22 1 часть Написать класс Python `OfficeAcceleration`, выполняющий расчёт времени готовности офисной техники к работе и до полной функциональности.

Расчёт времени от выключения до готовности:

$$t = \frac{C \cdot (\Delta V)^2}{2P_{\text{эфф}}}, \quad (26)$$

где:

- t — время готовности, с;
- C — эквивалентная ёмкость системы, Дж/В²;
- ΔV — изменение уровня активности от 0 до 0.5;
- $P_{\text{эфф}}$ — эффективная мощность, Вт.

Формула расчёта эффективной мощности для готовности:

$$P_{\text{эфф}} = P_{\text{макс}} \cdot k \cdot \eta, \quad (27)$$

где:

- $P_{\text{макс}}$ — пиковая мощность, Вт;
- $k = 0,24$ — коэффициент при старте;
- $\eta = 0,92$ — КПД блока питания.

Расчёт времени от готовности до полной функциональности:

$$t = \frac{C \cdot (V_2^2 - V_1^2)}{2P_{\text{эфф}}}, \quad (28)$$

где:

- $V_1 = 0,5$, $V_2 = 1,0$;
- $P_{\text{эфф}}$ — эффективная мощность, Вт.

Формула расчёта эффективной мощности для полной функциональности:

$$P_{\text{эфф}} = P_{\text{макс}} \cdot k \cdot \eta, \quad (29)$$

где $k = 0,58$ — коэффициент при полной загрузке.

2 часть

На основе программного кода предыдущего практикума текущего курса ООП выполнить наследование класса `OfficeAcceleration` с использованием `super()` таким образом, чтобы через класс с множественным наследованием `OfficeDevice` вывести на печать время готовности и полной функциональности для трёх типов устройств при следующих параметрах:

- Printer — $C = 0,015$ Дж/В², $P = 300$ Вт;
- Scanner — $C = 0,009$ Дж/В², $P = 20$ Вт;
- Copier — $C = 0,022$ Дж/В², $P = 500$ Вт.

3 часть

На основе программного кода и параметров устройств из 2 части текущего практикума отобразить через класс с множественным наследованием `OfficeDevice` расчёт времени полной функциональности от уровня 0.5 до 1.0 через каждые 0.125, учитывая, что коэффициент мощности $k = 0,58$ увеличивается через каждые 0.125 на 0,06.

Коэффициенты мощности для различных диапазонов активации:

- 0.50–0.625: $k = 0,58$;
- 0.625–0.75: $k = 0,64$;
- 0.75–0.875: $k = 0,70$;
- 0.875–1.00: $k = 0,76$.

- 23 **1 часть** Написать класс `Python ImagingAcceleration`, выполняющий расчёт времени готовности устройства захвата изображения к съёмке и до начала записи видео.

Расчёт времени от выключения до готовности к фото:

$$t = \frac{C \cdot (\Delta V)^2}{2P_{\text{эфф}}}, \quad (30)$$

где:

- t — время готовности, с;
- C — эквивалентная ёмкость системы, Дж/В²;
- $\Delta V = 0,5$ — изменение уровня активности;
- $P_{\text{эфф}}$ — эффективная мощность, Вт.

Формула расчёта эффективной мощности для фото:

$$P_{\text{эфф}} = P_{\text{макс}} \cdot k \cdot \eta, \quad (31)$$

где:

- $P_{\text{макс}}$ — пиковая мощность, Вт;
- $k = 0,26$ — коэффициент при старте;
- $\eta = 0,91$ — КПД энергосистемы.

Расчёт времени от фото до записи видео:

$$t = \frac{C \cdot (V_2^2 - V_1^2)}{2P_{\text{эфф}}}, \quad (32)$$

где:

- $V_1 = 0,5$, $V_2 = 1,0$;
- $P_{\text{эфф}}$ — эффективная мощность, Вт.

Формула расчёта эффективной мощности для видео:

$$P_{\text{эфф}} = P_{\text{макс}} \cdot k \cdot \eta, \quad (33)$$

где $k = 0,61$ — коэффициент при видеозаписи.

2 часть

На основе программного кода предыдущего практикума текущего курса ООП выполнить наследование класса `ImagingAcceleration` с использованием `super()` таким образом, чтобы через класс с множественным наследованием `ImagingDevice` вывести на печать время готовности и начала записи для трёх типов устройств при следующих параметрах:

- Camera — $C = 0,007$ Дж/В², $P = 10$ Вт;
- Camcorder — $C = 0,014$ Дж/В², $P = 25$ Вт;
- DroneCam — $C = 0,011$ Дж/В², $P = 18$ Вт.

3 часть

На основе программного кода и параметров устройств из 2 части текущего практикума отобразить через класс с множественным наследованием `ImagingDevice` расчёт времени записи видео от уровня 0.5 до 1.0 через каждые 0.125, учитывая, что коэффициент мощности $k = 0,61$ увеличивается через каждые 0.125 на 0,07.

Коэффициенты мощности для различных диапазонов активации:

- 0.50–0.625: $k = 0,61$;
- 0.625–0.75: $k = 0,68$;
- 0.75–0.875: $k = 0,75$;
- 0.875–1.00: $k = 0,82$.

24 1 часть Написать класс `Python CookingAcceleration`, выполняющий расчёт времени выхода кухонной техники на рабочую температуру и до полного цикла приготовления.

Расчёт времени от включения до рабочей температуры:

$$t = \frac{C \cdot (\Delta T)^2}{2P_{\text{эфф}}}, \quad (34)$$

где:

- t — время нагрева, с;
- C — теплоёмкость камеры, Дж/°C;
- ΔT — изменение температуры, °C;
- $P_{\text{эфф}}$ — эффективная мощность, Вт.

Формула расчёта эффективной мощности для нагрева:

$$P_{\text{эфф}} = P_{\text{макс}} \cdot k \cdot \eta, \quad (35)$$

где:

- $P_{\text{макс}}$ — номинальная мощность, Вт;
- $k = 0,27$ — коэффициент при старте;
- $\eta = 0,88$ — КПД нагревательного элемента.

Расчёт времени от рабочей температуры до завершения цикла:

$$t = \frac{C \cdot (T_2^2 - T_1^2)}{2P_{\text{эфф}}}, \quad (36)$$

где:

- $T_1 = 150$, $T_2 = 250$ °C;
- $P_{\text{эфф}}$ — эффективная мощность, Вт.

Формула расчёта эффективной мощности для приготовления:

$$P_{\text{эфф}} = P_{\text{макс}} \cdot k \cdot \eta, \quad (37)$$

где $k = 0,62$ — коэффициент при рабочем режиме.

2 часть

На основе программного кода предыдущего практикума текущего курса ООП выполнить наследование класса `CookingAcceleration` с использованием `super()` таким образом, чтобы через класс с множественным наследованием `CookingAppliance` вывести на печать время нагрева и приготовления для трёх типов приборов при следующих параметрах:

- Microwave — $C = 120$ Дж/°C, $P = 1000$ Вт;
- Oven — $C = 450$ Дж/°C, $P = 2500$ Вт;
- AirFryer — $C = 200$ Дж/°C, $P = 1500$ Вт.

3 часть

На основе программного кода и параметров приборов из 2 части текущего практикума отобразить через класс с множественным наследованием `CookingAppliance` расчёт времени приготовления от 150 °C до 250 °C через каждые 25 °C, учитывая, что коэффициент мощности $k = 0,62$ увеличивается через каждые 25 °C на 0,08.

Коэффициенты мощности для различных диапазонов температур:

- 150–175 °C: $k = 0,62$;
- 175–200 °C: $k = 0,70$;
- 200–225 °C: $k = 0,78$;
- 225–250 °C: $k = 0,86$.

25 1 часть Написать класс Python `CoolingAcceleration`, выполняющий расчёт времени охлаждения холодильника от 25 °C до 4 °C и от 4 °C до -18 °C.

Расчёт времени от 25 °C до 4 °C:

$$t = \frac{C \cdot (\Delta T)^2}{2P_{\text{эфф}}}, \quad (38)$$

где:

- t — время охлаждения, с;
- C — теплоёмкость содержимого, Дж/°C;
- $\Delta T = 21$ °C;
- $P_{\text{эфф}}$ — эффективная холодопроизводительность, Вт.

Формула расчёта эффективной холодопроизводительности для охлаждения до 4 °C:

$$P_{\text{эфф}} = P_{\text{макс}} \cdot k \cdot \eta, \quad (39)$$

где:

- $P_{\text{макс}}$ — номинальная мощность компрессора, Вт;
- $k = 0,35$ — коэффициент использования мощности;
- $\eta = 0,70$ — КПД холодильного цикла.

Расчёт времени от 4 °C до -18 °C:

$$t = \frac{C \cdot (T_2^2 - T_1^2)}{2P_{\text{эфф}}}, \quad (40)$$

где:

- $T_1 = 4$, $T_2 = -18$ (в °C, но используется квадрат разности);
- $P_{\text{эфф}}$ — эффективная холодопроизводительность, Вт.

Формула расчёта эффективной холодопроизводительности для заморозки:

$$P_{\text{эфф}} = P_{\text{макс}} \cdot k \cdot \eta, \quad (41)$$

где $k = 0,60$ — коэффициент использования мощности в режиме заморозки.

2 часть

На основе программного кода предыдущего практикума текущего курса ООП выполнить наследование класса `CoolingAcceleration` с использованием `super()` таким образом, чтобы через класс с множественным наследованием `CoolingAppliance` вывести на печать время охлаждения до 4 °C и от 4 °C до -18 °C для трёх типов устройств при следующих параметрах:

- Refrigerator — $C = 8000$ Дж/°C, $P = 120$ Вт;
- Freezer — $C = 6000$ Дж/°C, $P = 150$ Вт;
- MiniFridge — $C = 3000$ Дж/°C, $P = 60$ Вт.

3 часть

На основе программного кода и параметров устройств из 2 части текущего практикума отобразить через класс с множественным наследованием `CoolingAppliance` расчёт времени охлаждения от 4 °C до -18 °C через каждые 4.4 °C, то есть до -0.4, -4.8, -9.2, -13.6 и -18 °C, учитывая, что коэффициент мощности $k = 0,60$ увеличивается через каждые 4.4 °C на 0,08.

Коэффициенты мощности для различных диапазонов температур:

- $4 \rightarrow -0.4$ °C: $k = 0,60$;
- $-0.4 \rightarrow -4.8$ °C: $k = 0,68$;
- $-4.8 \rightarrow -9.2$ °C: $k = 0,76$;
- $-9.2 \rightarrow -13.6$ °C: $k = 0,84$;
- $-13.6 \rightarrow -18$ °C: $k = 0,92$.

26 1 часть Написать класс Python `LaundryAcceleration`, выполняющий расчёт времени запуска стиральной машины от включения до начала стирки и до завершения полного цикла.

Расчёт времени от включения до начала стирки:

$$t = \frac{C \cdot (\Delta V)^2}{2P_{\text{эфф}}}, \quad (42)$$

где:

- t — время запуска, с;
- C — эквивалентная ёмкость системы, Дж/В²;
- $\Delta V = 0,4$ — изменение уровня активности (от 0 до 0.4);

- $P_{\text{эфф}}$ — эффективная мощность, Вт.

Формула расчёта эффективной мощности для запуска:

$$P_{\text{эфф}} = P_{\text{макс}} \cdot k \cdot \eta, \quad (43)$$

где:

- $P_{\text{макс}}$ — номинальная мощность, Вт;
- $k = 0,26$ — коэффициент при старте;
- $\eta = 0,89$ — КПД электродвигателя.

Расчёт времени от начала стирки до завершения цикла:

$$t = \frac{C \cdot (V_2^2 - V_1^2)}{2P_{\text{эфф}}}, \quad (44)$$

где:

- $V_1 = 0,4$, $V_2 = 1,0$;
- $P_{\text{эфф}}$ — эффективная мощность, Вт.

Формула расчёта эффективной мощности для полного цикла:

$$P_{\text{эфф}} = P_{\text{макс}} \cdot k \cdot \eta, \quad (45)$$

где $k = 0,59$ — коэффициент при полной нагрузке.

2 часть

На основе программного кода предыдущего практикума текущего курса ООП выполнить наследование класса `LaundryAcceleration` с использованием `super()` таким образом, чтобы через класс с множественным наследованием `LaundryAppliance` вывести на печать время запуска и завершения цикла для трёх типов устройств при следующих параметрах:

- WashingMachine — $C = 0,018$ Дж/В², $P = 2000$ Вт;
- Dryer — $C = 0,022$ Дж/В², $P = 2500$ Вт;
- Iron — $C = 0,006$ Дж/В², $P = 1200$ Вт.

3 часть

На основе программного кода и параметров устройств из 2 части текущего практикума отобразить через класс с множественным наследованием `LaundryAppliance` расчёт времени завершения цикла от уровня 0.4 до 1.0 через каждые 0.15, учитывая, что коэффициент мощности $k = 0,59$ увеличивается через каждые 0.15 на 0,07.

Коэффициенты мощности для различных диапазонов активности:

- 0.40–0.55: $k = 0,59$;
- 0.55–0.70: $k = 0,66$;
- 0.70–0.85: $k = 0,73$;

- 0.85–1.00: $k = 0,80$.

27 1 часть Написать класс `Python CleaningAcceleration`, выполняющий расчёт времени готовности уборочной техники к работе и до завершения полного цикла уборки.

Расчёт времени от выключения до готовности:

$$t = \frac{C \cdot (\Delta V)^2}{2P_{\text{эфф}}}, \quad (46)$$

где:

- t — время готовности, с;
- C — эквивалентная ёмкость системы, Дж/В²;
- $\Delta V = 0,5$;
- $P_{\text{эфф}}$ — эффективная мощность, Вт.

Формула расчёта эффективной мощности для готовности:

$$P_{\text{эфф}} = P_{\text{макс}} \cdot k \cdot \eta, \quad (47)$$

где:

- $P_{\text{макс}}$ — пиковая мощность, Вт;
- $k = 0,25$;
- $\eta = 0,90$.

Расчёт времени от готовности до завершения уборки:

$$t = \frac{C \cdot (V_2^2 - V_1^2)}{2P_{\text{эфф}}}, \quad (48)$$

где:

- $V_1 = 0,5$, $V_2 = 1,0$;
- $P_{\text{эфф}}$ — эффективная мощность, Вт.

Формула расчёта эффективной мощности для полной уборки:

$$P_{\text{эфф}} = P_{\text{макс}} \cdot k \cdot \eta, \quad (49)$$

где $k = 0,60$ — коэффициент при полной нагрузке.

2 часть

На основе программного кода предыдущего практикума текущего курса ООП выполнить наследование класса `CleaningAcceleration` с использованием `super()` таким образом, чтобы через класс с множественным наследованием `CleaningDevice` вывести на печать время готовности и завершения уборки для трёх типов устройств при следующих параметрах:

- Vacuum — $C = 0,009$ Дж/В², $P = 200$ Вт;

- MopRobot — $C = 0,014 \text{ Дж/В}^2$, $P = 150 \text{ Вт}$;
- SteamCleaner — $C = 0,011 \text{ Дж/В}^2$, $P = 1800 \text{ Вт}$.

3 часть

На основе программного кода и параметров устройств из 2 части текущего практикума отобразить через класс с множественным наследованием `CleaningDevice` расчёт времени уборки от уровня 0.5 до 1.0 через каждые 0.125, учитывая, что коэффициент мощности $k = 0,60$ увеличивается через каждые 0.125 на 0,06.

Коэффициенты мощности для различных диапазонов активности:

- 0.50–0.625: $k = 0,60$;
- 0.625–0.75: $k = 0,66$;
- 0.75–0.875: $k = 0,72$;
- 0.875–1.00: $k = 0,78$.

28 **1 часть** Написать класс Python `ClimateAcceleration`, выполняющий расчёт времени выхода климатической техники на заданный режим и до полной стабилизации микроклимата.

Расчёт времени от включения до заданного режима:

$$t = \frac{C \cdot (\Delta V)^2}{2P_{\text{эфф}}}, \quad (50)$$

где:

- t — время установки режима, с;
- C — эквивалентная ёмкость системы, Дж/В²;
- $\Delta V = 0,5$;
- $P_{\text{эфф}}$ — эффективная мощность, Вт.

Формула расчёта эффективной мощности для установки режима:

$$P_{\text{эфф}} = P_{\text{макс}} \cdot k \cdot \eta, \quad (51)$$

где:

- $P_{\text{макс}}$ — номинальная мощность, Вт;
- $k = 0,24$;
- $\eta = 0,91$.

Расчёт времени от заданного режима до стабилизации:

$$t = \frac{C \cdot (V_2^2 - V_1^2)}{2P_{\text{эфф}}}, \quad (52)$$

где:

- $V_1 = 0,5$, $V_2 = 1,0$;

- $P_{\text{эфф}}$ — эффективная мощность, Вт.

Формула расчёта эффективной мощности для стабилизации:

$$P_{\text{эфф}} = P_{\text{макс}} \cdot k \cdot \eta, \quad (53)$$

где $k = 0,58$ — коэффициент при полной нагрузке.

2 часть

На основе программного кода предыдущего практикума текущего курса ООП выполнить наследование класса `ClimateAcceleration` с использованием `super()` таким образом, чтобы через класс с множественным наследованием `ClimateDevice` вывести на печать время установки режима и стабилизации для трёх типов устройств при следующих параметрах:

- Thermostat — $C = 0,005$ Дж/В², $P = 5$ Вт;
- Humidifier — $C = 0,008$ Дж/В², $P = 30$ Вт;
- AirPurifier — $C = 0,007$ Дж/В², $P = 45$ Вт.

3 часть

На основе программного кода и параметров устройств из 2 части текущего практикума отобразить через класс с множественным наследованием `ClimateDevice` расчёт времени стабилизации от уровня 0.5 до 1.0 через каждые 0.125, учитывая, что коэффициент мощности $k = 0,58$ увеличивается через каждые 0.125 на 0,06.

Коэффициенты мощности для различных диапазонов активности:

- 0.50–0.625: $k = 0,58$;
- 0.625–0.75: $k = 0,64$;
- 0.75–0.875: $k = 0,70$;
- 0.875–1.00: $k = 0,76$.

- 29 **1 часть** Написать класс `Python SecurityAcceleration`, выполняющий расчёт времени активации системы безопасности и до полной готовности всех модулей.

Расчёт времени от выключения до активации:

$$t = \frac{C \cdot (\Delta V)^2}{2P_{\text{эфф}}}, \quad (54)$$

где:

- t — время активации, с;
- C — эквивалентная ёмкость системы, Дж/В²;
- $\Delta V = 0,4$;
- $P_{\text{эфф}}$ — эффективная мощность, Вт.

Формула расчёта эффективной мощности для активации:

$$P_{\text{эфф}} = P_{\text{макс}} \cdot k \cdot \eta, \quad (55)$$

где:

- $P_{\text{макс}}$ — пиковая мощность, Вт;
- $k = 0,23$;
- $\eta = 0,93$.

Расчёт времени от активации до полной готовности:

$$t = \frac{C \cdot (V_2^2 - V_1^2)}{2P_{\text{эфф}}}, \quad (56)$$

где:

- $V_1 = 0,4$, $V_2 = 1,0$;
- $P_{\text{эфф}}$ — эффективная мощность, Вт.

Формула расчёта эффективной мощности для полной готовности:

$$P_{\text{эфф}} = P_{\text{макс}} \cdot k \cdot \eta, \quad (57)$$

где $k = 0,59$ — коэффициент при полной нагрузке.

2 часть

На основе программного кода предыдущего практикума текущего курса ООП выполнить наследование класса `SecurityAcceleration` с использованием `super()` таким образом, чтобы через класс с множественным наследованием `SecurityDevice` вывести на печать время активации и готовности для трёх типов устройств при следующих параметрах:

- SmartLock — $C = 0,004$ Дж/В², $P = 3$ Вт;
- SecurityCamera — $C = 0,009$ Дж/В², $P = 12$ Вт;
- AlarmSystem — $C = 0,007$ Дж/В², $P = 25$ Вт.

3 часть

На основе программного кода и параметров устройств из 2 части текущего практикума отобразить через класс с множественным наследованием `SecurityDevice` расчёт времени готовности от уровня 0.4 до 1.0 через каждые 0.15, учитывая, что коэффициент мощности $k = 0,59$ увеличивается через каждые 0.15 на 0,07.

Коэффициенты мощности для различных диапазонов активности:

- 0.40–0.55: $k = 0,59$;
- 0.55–0.70: $k = 0,66$;
- 0.70–0.85: $k = 0,73$;
- 0.85–1.00: $k = 0,80$.

- 30 **1 часть** Написать класс `Python LightingAcceleration`, выполняющий расчёт времени включения осветительной системы и до достижения полной яркости с анимацией.

Расчёт времени от выключения до включения:

$$t = \frac{C \cdot (\Delta V)^2}{2P_{\text{эфф}}}, \quad (58)$$

где:

- t — время включения, с;
- C — эквивалентная ёмкость системы, Дж/В²;
- $\Delta V = 0,3$;
- $P_{\text{эфф}}$ — эффективная мощность, Вт.

Формула расчёта эффективной мощности для включения:

$$P_{\text{эфф}} = P_{\text{макс}} \cdot k \cdot \eta, \quad (59)$$

где:

- $P_{\text{макс}}$ — номинальная мощность, Вт;
- $k = 0,22$;
- $\eta = 0,95$.

Расчёт времени от включения до полной яркости с анимацией:

$$t = \frac{C \cdot (V_2^2 - V_1^2)}{2P_{\text{эфф}}}, \quad (60)$$

где:

- $V_1 = 0,3$, $V_2 = 1,0$;
- $P_{\text{эфф}}$ — эффективная мощность, Вт.

Формула расчёта эффективной мощности для полной яркости:

$$P_{\text{эфф}} = P_{\text{макс}} \cdot k \cdot \eta, \quad (61)$$

где $k = 0,57$ — коэффициент при полной нагрузке.

2 часть

На основе программного кода предыдущего практикума текущего курса ООП выполнить наследование класса `LightingAcceleration` с использованием `super()` таким образом, чтобы через класс с множественным наследованием `LightingSystem` вывести на печать время включения и достижения полной яркости для трёх типов устройств при следующих параметрах:

- SmartBulb — $C = 0,002$ Дж/В², $P = 10$ Вт;
- SmartPlug — $C = 0,003$ Дж/В², $P = 2000$ Вт;
- LightStrip — $C = 0,006$ Дж/В², $P = 60$ Вт.

3 часть

На основе программного кода и параметров устройств из 2 части текущего практикума отобразить через класс с множественным наследованием `LightingSystem` расчёт времени достижения полной яркости от уровня 0.3 до 1.0 через каждые 0.175, учитывая, что коэффициент мощности $k = 0,57$ увеличивается через каждые 0.175 на 0,07.

Коэффициенты мощности для различных диапазонов активности:

- 0.30–0.475: $k = 0,57$;
- 0.475–0.65: $k = 0,64$;
- 0.65–0.825: $k = 0,71$;
- 0.825–1.00: $k = 0,78$.

31 **1 часть** Написать класс `Python GamingAcceleration`, выполняющий расчёт времени запуска игрового устройства от включения до готовности к игре и до полной загрузки игры.

Расчёт времени от включения до готовности:

$$t = \frac{C \cdot (\Delta V)^2}{2P_{\text{эфф}}}, \quad (62)$$

где:

- t — время запуска, с;
- C — эквивалентная ёмкость системы, Дж/В²;
- $\Delta V = 0,4$ — изменение уровня активности;
- $P_{\text{эфф}}$ — эффективная мощность, Вт.

Формула расчёта эффективной мощности для запуска:

$$P_{\text{эфф}} = P_{\text{макс}} \cdot k \cdot \eta, \quad (63)$$

где:

- $P_{\text{макс}}$ — пиковая мощность, Вт;
- $k = 0,25$ — коэффициент при старте;
- $\eta = 0,90$ — КПД блока питания.

Расчёт времени от готовности до полной загрузки игры:

$$t = \frac{C \cdot (V_2^2 - V_1^2)}{2P_{\text{эфф}}}, \quad (64)$$

где:

- $V_1 = 0,4$, $V_2 = 1,0$;
- $P_{\text{эфф}}$ — эффективная мощность, Вт.

Формула расчёта эффективной мощности для загрузки игры:

$$P_{\text{эфф}} = P_{\text{макс}} \cdot k \cdot \eta, \quad (65)$$

где $k = 0,60$ — коэффициент при полной нагрузке.

2 часть

На основе программного кода предыдущего практикума текущего курса ООП выполнить наследование класса `GamingAcceleration` с использованием `super()` таким образом, чтобы через класс с множественным наследованием `GamingDevice` вывести на печать время запуска и загрузки игры для трёх типов устройств при следующих параметрах:

- GameConsole — $C = 0,025 \text{ Дж/В}^2$, $P = 200 \text{ Вт}$;
- Handheld — $C = 0,008 \text{ Дж/В}^2$, $P = 15 \text{ Вт}$;
- VR_Headset — $C = 0,012 \text{ Дж/В}^2$, $P = 35 \text{ Вт}$.

3 часть

На основе программного кода и параметров устройств из 2 части текущего практикума отобразить через класс с множественным наследованием `GamingDevice` расчёт времени загрузки игры от уровня 0.4 до 1.0 через каждые 0.15, учитывая, что коэффициент мощности $k = 0,60$ увеличивается через каждые 0.15 на 0,06.

Коэффициенты мощности для различных диапазонов активности:

- 0.40–0.55: $k = 0,60$;
- 0.55–0.70: $k = 0,66$;
- 0.70–0.85: $k = 0,72$;
- 0.85–1.00: $k = 0,78$.

32 **1 часть** Написать класс Python `AudioAcceleration`, выполняющий расчёт времени запуска электронного музыкального инструмента от включения до готовности к исполнению и до полной настройки звука.

Расчёт времени от включения до готовности:

$$t = \frac{C \cdot (\Delta V)^2}{2P_{\text{эфф}}}, \quad (66)$$

где:

- t — время запуска, с;
- C — эквивалентная ёмкость системы, Дж/В²;
- $\Delta V = 0,3$;
- $P_{\text{эфф}}$ — эффективная мощность, Вт.

Формула расчёта эффективной мощности для запуска:

$$P_{\text{эфф}} = P_{\text{макс}} \cdot k \cdot \eta, \quad (67)$$

где:

- $P_{\text{макс}}$ — номинальная мощность, Вт;
- $k = 0,24$;
- $\eta = 0,88$.

Расчёт времени от готовности до полной настройки звука:

$$t = \frac{C \cdot (V_2^2 - V_1^2)}{2P_{\text{эфф}}}, \quad (68)$$

где:

- $V_1 = 0,3$, $V_2 = 1,0$;
- $P_{\text{эфф}}$ — эффективная мощность, Вт.

Формула расчёта эффективной мощности для настройки звука:

$$P_{\text{эфф}} = P_{\text{макс}} \cdot k \cdot \eta, \quad (69)$$

где $k = 0,59$ — коэффициент при полной настройке.

2 часть

На основе программного кода предыдущего практикума текущего курса ООП выполнить наследование класса `AudioAcceleration` с использованием `super()` таким образом, чтобы через класс с множественным наследованием `ElectronicInstrument` вывести на печать время запуска и настройки для трёх типов инструментов при следующих параметрах:

- ElectricGuitar — $C = 0,006$ Дж/В², $P = 50$ Вт;
- Synthesizer — $C = 0,014$ Дж/В², $P = 80$ Вт;
- DrumMachine — $C = 0,010$ Дж/В², $P = 60$ Вт.

3 часть

На основе программного кода и параметров инструментов из 2 части текущего практикума отобразить через класс с множественным наследованием `ElectronicInstrument` расчёт времени настройки от уровня 0.3 до 1.0 через каждые 0.175, учитывая, что коэффициент мощности $k = 0,59$ увеличивается через каждые 0.175 на 0,07.

Коэффициенты мощности для различных диапазонов активности:

- 0.30–0.475: $k = 0,59$;
- 0.475–0.65: $k = 0,66$;
- 0.65–0.825: $k = 0,73$;
- 0.825–1.00: $k = 0,80$.

33 1 часть Написать класс Python `SmartHomeAcceleration`, выполняющий расчёт времени активации системы умного дома от включения до готовности и до полной автоматизации.

Расчёт времени от включения до готовности:

$$t = \frac{C \cdot (\Delta V)^2}{2P_{\text{эфф}}}, \quad (70)$$

где:

- t — время активации, с;
- C — эквивалентная ёмкость системы, Дж/В²;
- $\Delta V = 0,4$;
- $P_{\text{эфф}}$ — эффективная мощность, Вт.

Формула расчёта эффективной мощности для активации:

$$P_{\text{эфф}} = P_{\text{макс}} \cdot k \cdot \eta, \quad (71)$$

где:

- $P_{\text{макс}}$ — пиковая мощность, Вт;
- $k = 0,23$;
- $\eta = 0,92$.

Расчёт времени от готовности до полной автоматизации:

$$t = \frac{C \cdot (V_2^2 - V_1^2)}{2P_{\text{эфф}}}, \quad (72)$$

где:

- $V_1 = 0,4$, $V_2 = 1,0$;
- $P_{\text{эфф}}$ — эффективная мощность, Вт.

Формула расчёта эффективной мощности для автоматизации:

$$P_{\text{эфф}} = P_{\text{макс}} \cdot k \cdot \eta, \quad (73)$$

где $k = 0,58$ — коэффициент при полной автоматизации.

2 часть

На основе программного кода предыдущего практикума текущего курса ООП выполнить наследование класса `SmartHomeAcceleration` с использованием `super()` таким образом, чтобы через класс с множественным наследованием `SmartHomeSystem` вывести на печать время активации и автоматизации для трёх типов устройств при следующих параметрах:

- `SmartThermostat` — $C = 0,005$ Дж/В², $P = 8$ Вт;
- `SmartBlinds` — $C = 0,007$ Дж/В², $P = 12$ Вт;
- `SmartSprinkler` — $C = 0,006$ Дж/В², $P = 10$ Вт.

3 часть

На основе программного кода и параметров устройств из 2 части текущего практикума отобразить через класс с множественным наследованием `SmartHomeSystem` расчёт времени автоматизации от уровня 0.4 до 1.0 через каждые 0.15, учитывая, что коэффициент мощности $k = 0,58$ увеличивается через каждые 0.15 на 0,06.

Коэффициенты мощности для различных диапазонов активности:

- 0.40–0.55: $k = 0,58$;
- 0.55–0.70: $k = 0,64$;
- 0.70–0.85: $k = 0,70$;
- 0.85–1.00: $k = 0,76$.

34 **1 часть** Написать класс Python `PersonalTransportAcceleration`, выполняющий расчёт времени готовности персонального электротранспорта к поездке и до достижения максимальной скорости.

Расчёт времени от выключения до готовности:

$$t = \frac{m \cdot (\Delta V)^2}{2P_{\text{эфф}}}, \quad (74)$$

где:

- t — время готовности, с;
- m — масса транспорта с водителем, кг;
- ΔV — изменение скорости от 0 до 5 км/ч (в м/с);
- $P_{\text{эфф}}$ — эффективная мощность, Вт.

Формула расчёта эффективной мощности для готовности:

$$P_{\text{эфф}} = P_{\text{макс}} \cdot k \cdot \eta, \quad (75)$$

где:

- $P_{\text{макс}}$ — номинальная мощность мотора, Вт;
- $k = 0,26$;
- $\eta = 0,89$.

Расчёт времени от готовности до максимальной скорости:

$$t = \frac{m \cdot (V_2^2 - V_1^2)}{2P_{\text{эфф}}}, \quad (76)$$

где:

- $V_1 = 5$ км/ч, $V_2 = 25$ км/ч (в м/с);
- $P_{\text{эфф}}$ — эффективная мощность, Вт.

Формула расчёта эффективной мощности для максимальной скорости:

$$P_{\text{эфф}} = P_{\text{макс}} \cdot k \cdot \eta, \quad (77)$$

где $k = 0,61$ — коэффициент при высокой скорости.

2 часть

На основе программного кода предыдущего практикума текущего курса ООП выполнить наследование класса `PersonalTransportAcceleration` с использованием `super()` таким образом, чтобы через класс с множественным наследованием `PersonalTransport` вывести на печать время готовности и достижения максимальной скорости для трёх типов транспорта при следующих параметрах:

- `ElectricScooter` — масса 80 кг, мощность 350 Вт;
- `Segway` — масса 90 кг, мощность 600 Вт;

- Hoverboard — масса 70 кг, мощность 700 Вт.

3 часть

На основе программного кода и параметров транспорта из 2 части текущего практикума отобразить через класс с множественным наследованием `PersonalTransport` расчёт времени разгона от 5 до 25 км/ч через каждые 5 км/ч, учитывая, что коэффициент мощности $k = 0,61$ увеличивается через каждые 5 км/ч на 0,08.

Коэффициенты мощности для различных диапазонов скоростей:

- 5–10 км/ч: $k = 0,61$;
- 10–15 км/ч: $k = 0,69$;
- 15–20 км/ч: $k = 0,77$;
- 20–25 км/ч: $k = 0,85$.

- 35 **1 часть** Написать класс Python `BeverageAcceleration`, выполняющий расчёт времени готовности техники для напитков от включения до подачи напитка и до завершения цикла.

Расчёт времени от включения до подачи напитка:

$$t = \frac{C \cdot (\Delta T)^2}{2P_{\text{эфф}}}, \quad (78)$$

где:

- t — время готовности, с;
- C — теплоёмкость системы, Дж/°C;
- $\Delta T = 60$ °C (от 20 °C до 80 °C);
- $P_{\text{эфф}}$ — эффективная мощность, Вт.

Формула расчёта эффективной мощности для подачи:

$$P_{\text{эфф}} = P_{\text{макс}} \cdot k \cdot \eta, \quad (79)$$

где:

- $P_{\text{макс}}$ — номинальная мощность, Вт;
- $k = 0,27$;
- $\eta = 0,87$.

Расчёт времени от подачи до завершения цикла:

$$t = \frac{C \cdot (T_2^2 - T_1^2)}{2P_{\text{эфф}}}, \quad (80)$$

где:

- $T_1 = 80$, $T_2 = 95$ °C;
- $P_{\text{эфф}}$ — эффективная мощность, Вт.

Формула расчёта эффективной мощности для завершения:

$$P_{\text{эфф}} = P_{\text{макс}} \cdot k \cdot \eta, \quad (81)$$

где $k = 0,62$ — коэффициент при полном цикле.

2 часть

На основе программного кода предыдущего практикума текущего курса ООП выполнить наследование класса **BeverageAcceleration** с использованием **super()** таким образом, чтобы через класс с множественным наследованием **BeverageAppliance** вывести на печать время подачи и завершения цикла для трёх типов приборов при следующих параметрах:

- CoffeeMachine — $C = 300$ Дж/°C, $P = 1500$ Вт;
- Teapot — $C = 250$ Дж/°C, $P = 2000$ Вт;
- Juicer — $C = 180$ Дж/°C, $P = 1200$ Вт.

3 часть

На основе программного кода и параметров приборов из 2 части текущего практикума отобразить через класс с множественным наследованием **BeverageAppliance** расчёт времени завершения цикла от 80 °C до 95 °C через каждые 3.75 °C, учитывая, что коэффициент мощности $k = 0,62$ увеличивается через каждые 3.75 °C на 0,08.

Коэффициенты мощности для различных диапазонов температур:

- 80–83.75 °C: $k = 0,62$;
- 83.75–87.5 °C: $k = 0,70$;
- 87.5–91.25 °C: $k = 0,78$;
- 91.25–95 °C: $k = 0,86$.

2.12 Семинар «Перегрузка операций в классах» (2 часа)

- 1 Создать класс `Person`, который будет представлять человека с именем, фамилией и возрастом. Класс должен поддерживать методы перегрузки операций `__setattr__`, `__getattr__`, `__getattribute__` и `__delattr__`. Метод `__setattr__` должен проверять, что присваиваемые атрибуты соответствуют определённым правилам (например, имя состоит только из буквенных символов, фамилия состоит только из буквенных и цифр, возраст является целым числом). Метод `__getattribute__` должен возвращать значение атрибута, если он существует, иначе должен возвращать сообщение об ошибке. Метод `__getattr__` должен возвращать `None`, если какой-либо атрибут не найден, а также создавать атрибут `City` с определённым именем города, например `“Moscow”`. Метод `__delattr__` должен удалять атрибут, если он существует.
 - (a) Класс `Person` определяет атрибуты `first_name`, `last_name` и `age`.
 - (b) Метод `__setattr__` используется для проверки ввода при присваивании атрибутов. Если введённое значение не соответствует ожидаемому формату, вызывается исключение `ValueError`.
 - (c) Метод `__getattribute__` используется для получения атрибутов объекта. Если атрибут не найден, возвращается сообщение об ошибке.
 - (d) Метод `__getattr__` используется для обработки запросов на получение несуществующих атрибутов, возвращает `None`, если не найден атрибут, кроме `City`, который создаётся на лету со значением имени города.
 - (e) Метод `__delattr__` используется для проверки возможности удаления атрибутов. Некоторые атрибуты (`first_name`, `last_name`, `age`) не могут быть удалены, и попытка их удаления вызывает исключение `AttributeError`.
 - (f) В примере использования создаётся экземпляр класса `Person` с именем `person`. Затем проверяется доступ к атрибутам и попытки их изменения.
- 2 Создать класс `Book`, который будет представлять книгу с названием, автором и годом издания. Класс должен поддерживать методы перегрузки операций `__setattr__`, `__getattr__`, `__getattribute__` и `__delattr__`. Метод `__setattr__` должен проверять корректность присваиваемых значений (название и автор — строковые значения, не пустые; год — целое число в диапазоне от 0 до текущего года). Метод `__getattribute__` должен возвращать значение атрибута, если он существует, иначе сообщение об ошибке. Метод `__getattr__` должен возвращать `None`, если атрибут не найден, за исключением атрибута `Genre`, который автоматически создаётся со значением `“Fiction”`. Метод `__delattr__` должен позволять удалять только необязательные атрибуты.
 - (a) Класс `Book` определяет атрибуты `title`, `author` и `year`.
 - (b) Метод `__setattr__` валидирует входные данные и вызывает `ValueError` при нарушении правил.
 - (c) Метод `__getattribute__` перехватывает доступ ко всем атрибутам и при неудаче возвращает строку с сообщением об ошибке.
 - (d) Метод `__getattr__` обрабатывает отсутствующие атрибуты, возвращая `None`, но для `Genre` создаёт его со значением `“Fiction”`.
 - (e) Метод `__delattr__` запрещает удаление обязательных атрибутов (`title`, `author`, `year`), вызывая `AttributeError`.

- (f) В примере создаётся объект `book` и тестируются операции присвоения, чтения и удаления атрибутов.
- 3 Создать класс `Car`, который будет представлять автомобиль с маркой, моделью и годом выпуска. Класс должен поддерживать методы перегрузки операций `__setattr__`, `__getattr__`, `__getattribute__` и `__delattr__`. Метод `__setattr__` должен проверять, что марка и модель — непустые строки без цифр, а год — целое число от 1886 до текущего года. Метод `__getattribute__` должен возвращать значение атрибута или сообщение об ошибке, если атрибут не существует. Метод `__getattr__` должен возвращать `None`, за исключением атрибута `Country`, который создаётся со значением `“Germany”`. Метод `__delattr__` должен запрещать удаление обязательных атрибутов.
- (a) Класс `Car` определяет атрибуты `make`, `model` и `year`.
 - (b) Метод `__setattr__` проводит валидацию значений и выбрасывает `ValueError` при нарушении формата.
 - (c) Метод `__getattribute__` перехватывает все попытки доступа к атрибутам; при отсутствии атрибута возвращает понятное сообщение об ошибке.
 - (d) Метод `__getattr__` создаёт атрибут `Country` со значением `“Germany”`, если запрашивается именно он; иначе возвращает `None`.
 - (e) Метод `__delattr__` не позволяет удалять `make`, `model` или `year`, вызывая `AttributeError`.
 - (f) Пример использования включает создание объекта `car` и демонстрацию работы всех четырёх методов.
- 4 Создать класс `Student`, который будет представлять студента с именем, номером зачётной книжки и средним баллом. Класс должен поддерживать методы перегрузки операций `__setattr__`, `__getattr__`, `__getattribute__` и `__delattr__`. Метод `__setattr__` должен проверять, что имя — строка из букв, номер зачётной книжки — строка из цифр длиной 6 символов, а средний балл — число от 0.0 до 5.0. Метод `__getattribute__` должен возвращать значение атрибута или сообщение об ошибке. Метод `__getattr__` должен возвращать `None`, за исключением атрибута `University`, который создаётся со значением `“State University”`. Метод `__delattr__` должен запрещать удаление обязательных атрибутов.
- (a) Класс `Student` определяет атрибуты `name`, `record_book` и `gpa`.
 - (b) Метод `__setattr__` валидирует данные и выбрасывает `ValueError` при несоответствии формату.
 - (c) Метод `__getattribute__` перехватывает все обращения к атрибутам; если атрибут отсутствует, возвращается ошибка в виде строки.
 - (d) Метод `__getattr__` возвращает `None` для несуществующих атрибутов, но создаёт `University` со значением `“State University”`.
 - (e) Метод `__delattr__` не позволяет удалить `name`, `record_book` или `gpa`, вызывая `AttributeError`.
 - (f) Пример: создаётся объект `student`, тестируются присвоение, чтение и удаление атрибутов.
- 5 Создать класс `Product`, который будет представлять товар с названием, ценой и категорией. Класс должен поддерживать методы перегрузки операций `__setattr__`,

`__getattribute__`, `__getattr__` и `__delattr__`. Метод `__setattr__` должен проверять, что название — непустая строка, цена — положительное число (целое или с плавающей точкой), категория — строка из букв. Метод `__getattribute__` должен возвращать значение атрибута или сообщение об ошибке. Метод `__getattr__` должен возвращать `None`, за исключением атрибута `Currency`, который создаётся со значением “USD”. Метод `__delattr__` должен запрещать удаление обязательных атрибутов.

- (a) Класс `Product` определяет атрибуты `name`, `price` и `category`.
- (b) Метод `__setattr__` проверяет корректность значений и вызывает `ValueError` при нарушении.
- (c) Метод `__getattribute__` возвращает атрибут или сообщение об ошибке, если его нет.
- (d) Метод `__getattr__` создаёт атрибут `Currency` со значением “USD”, иначе возвращает `None`.
- (e) Метод `__delattr__` не позволяет удалять `name`, `price` или `category`, вызывая `AttributeError`.
- (f) Пример использования включает создание объекта `product` и проверку всех операций.

6 Создать класс `Animal`, который будет представлять животное с видом, кличкой и возрастом. Класс должен поддерживать методы перегрузки операций `__setattr__`, `__getattribute__`, `__getattr__` и `__delattr__`. Метод `__setattr__` должен проверять, что вид и кличка — непустые строки из букв, возраст — целое число от 0 до 100. Метод `__getattribute__` должен возвращать значение атрибута или сообщение об ошибке. Метод `__getattr__` должен возвращать `None`, за исключением атрибута `Habitat`, который создаётся со значением “Wild”. Метод `__delattr__` должен запрещать удаление основных атрибутов.

- (a) Класс `Animal` определяет атрибуты `species`, `name` и `age`.
- (b) Метод `__setattr__` валидирует входные данные и вызывает `ValueError` при нарушении.
- (c) Метод `__getattribute__` возвращает атрибут или сообщение об ошибке.
- (d) Метод `__getattr__` создаёт `Habitat = “Wild”`, если запрашивается; иначе — `None`.
- (e) Метод `__delattr__` не позволяет удалять `species`, `name` или `age`, вызывая `AttributeError`.
- (f) Пример: создаётся объект `animal`, проверяются все операции с атрибутами.

7 Создать класс `Movie`, который будет представлять фильм с названием, режиссёром и рейтингом. Класс должен поддерживать методы перегрузки операций `__setattr__`, `__getattribute__`, `__getattr__` и `__delattr__`. Метод `__setattr__` должен проверять, что название и режиссёр — непустые строки, рейтинг — число от 0.0 до 10.0. Метод `__getattribute__` должен возвращать значение атрибута или сообщение об ошибке. Метод `__getattr__` должен возвращать `None`, за исключением атрибута `Studio`, который создаётся со значением “Universal”. Метод `__delattr__` должен запрещать удаление обязательных атрибутов.

- (a) Класс `Movie` определяет атрибуты `title`, `director` и `rating`.

- (b) Метод `__setattr__` проверяет формат значений и вызывает `ValueError` при ошибке.
- (c) Метод `__getattribute__` возвращает атрибут или понятное сообщение об ошибке.
- (d) Метод `__getattr__` создаёт `Studio = "Universal"`, если запрашивается; иначе — `None`.
- (e) Метод `__delattr__` не позволяет удалить основные атрибуты, вызывая `AttributeError`.
- (f) Пример: создаётся объект `movie`, проверяются все методы.

8 Создать класс `Employee`, который будет представлять сотрудника с именем, должностью и зарплатой. Класс должен поддерживать методы перегрузки операций `__setattr__`, `__getattribute__`, `__getattr__` и `__delattr__`. Метод `__setattr__` должен проверять, что имя — строка из букв, должность — непустая строка, зарплата — положительное число. Метод `__getattribute__` должен возвращать значение атрибута или сообщение об ошибке. Метод `__getattr__` должен возвращать `None`, за исключением атрибута `Department`, который создаётся со значением `"HR"`. Метод `__delattr__` должен запрещать удаление обязательных атрибутов.

- (a) Класс `Employee` определяет атрибуты `name`, `position` и `salary`.
- (b) Метод `__setattr__` валидирует данные и вызывает `ValueError` при нарушении.
- (c) Метод `__getattribute__` возвращает атрибут или сообщение об ошибке.
- (d) Метод `__getattr__` создаёт `Department = "HR"`, если запрашивается; иначе — `None`.
- (e) Метод `__delattr__` не позволяет удалять основные атрибуты, вызывая `AttributeError`.
- (f) Пример: создаётся объект `employee`, тестируются все операции.

9 Создать класс `Course`, который будет представлять учебный курс с названием, продолжительностью (в неделях) и уровнем сложности. Класс должен поддерживать методы перегрузки операций `__setattr__`, `__getattribute__`, `__getattr__` и `__delattr__`. Метод `__setattr__` должен проверять, что название — непустая строка, продолжительность — целое число от 1 до 52, уровень — строка из набора `{"Beginner", "Intermediate", "Advanced"}`. Метод `__getattribute__` должен возвращать значение атрибута или сообщение об ошибке. Метод `__getattr__` должен возвращать `None`, за исключением атрибута `Platform`, который создаётся со значением `"Online"`. Метод `__delattr__` должен запрещать удаление основных атрибутов.

- (a) Класс `Course` определяет атрибуты `title`, `duration` и `level`.
- (b) Метод `__setattr__` проверяет корректность и вызывает `ValueError` при ошибке.
- (c) Метод `__getattribute__` возвращает атрибут или сообщение об ошибке.
- (d) Метод `__getattr__` создаёт `Platform = "Online"`, если запрашивается; иначе — `None`.
- (e) Метод `__delattr__` не позволяет удалять основные атрибуты, вызывая `AttributeError`.
- (f) Пример: создаётся объект `course`, проверяются все методы.

- 10 Создать класс **BankAccount**, который будет представлять банковский счёт с номером счёта, владельцем и балансом. Класс должен поддерживать методы перегрузки операций `__setattr__`, `__getattr__`, `__getattribute__` и `__delattr__`. Метод `__setattr__` должен проверять, что номер счёта — строка из 10 цифр, владелец — непустая строка из букв, баланс — неотрицательное число. Метод `__getattribute__` должен возвращать значение атрибута или сообщение об ошибке. Метод `__getattr__` должен возвращать `None`, за исключением атрибута `Currency`, который создаётся со значением “EUR”. Метод `__delattr__` должен запрещать удаление основных атрибутов.
- (a) Класс **BankAccount** определяет атрибуты `account_number`, `owner` и `balance`.
 - (b) Метод `__setattr__` валидирует данные и вызывает `ValueError` при нарушении формата.
 - (c) Метод `__getattribute__` возвращает атрибут или сообщение об ошибке.
 - (d) Метод `__getattr__` создаёт `Currency = “EUR”`, если запрашивается; иначе — `None`.
 - (e) Метод `__delattr__` не позволяет удалять основные атрибуты, вызывая `AttributeError`.
 - (f) Пример: создаётся объект `account`, тестируются все операции.
- 11 Создать класс **Game**, который будет представлять видеоигру с названием, жанром и годом выпуска. Класс должен поддерживать методы перегрузки операций `__setattr__`, `__getattr__`, `__getattribute__` и `__delattr__`. Метод `__setattr__` должен проверять, что название и жанр — непустые строки, год — целое число от 1970 до текущего года. Метод `__getattribute__` должен возвращать значение атрибута или сообщение об ошибке. Метод `__getattr__` должен возвращать `None`, за исключением атрибута `Developer`, который создаётся со значением “Indie Studio”. Метод `__delattr__` должен запрещать удаление основных атрибутов.
- (a) Класс **Game** определяет атрибуты `name`, `genre` и `release_year`.
 - (b) Метод `__setattr__` проверяет корректность значений и вызывает `ValueError` при ошибке.
 - (c) Метод `__getattribute__` возвращает атрибут или сообщение об ошибке.
 - (d) Метод `__getattr__` создаёт `Developer = “Indie Studio”`, если запрашивается; иначе — `None`.
 - (e) Метод `__delattr__` не позволяет удалять основные атрибуты, вызывая `AttributeError`.
 - (f) Пример: создаётся объект `game`, проверяются все методы.
- 12 Создать класс **Flight**, который будет представлять авиарейс с номером рейса, аэропортом вылета и временем вылета (в формате HH:MM). Класс должен поддерживать методы перегрузки операций `__setattr__`, `__getattr__`, `__getattribute__` и `__delattr__`. Метод `__setattr__` должен проверять, что номер — строка из 4–6 символов (буквы и цифры), аэропорт — строка из 3 заглавных букв, время — строка в формате HH:MM с валидными часами и минутами. Метод `__getattribute__` должен возвращать значение атрибута или сообщение об ошибке. Метод `__getattr__` должен возвращать `None`, за исключением атрибута `Airline`, который создаётся со значением “SkyWings”. Метод `__delattr__` должен запрещать удаление основных атрибутов.
- (a) Класс **Flight** определяет атрибуты `flight_number`, `departure_airport` и `departure_time`.
 - (b) Метод `__setattr__` валидирует форматы и вызывает `ValueError` при ошибке.

- (c) Метод `__getattribute__` возвращает атрибут или сообщение об ошибке.
 - (d) Метод `__getattr__` создаёт `Airline = "SkyWings"`, если запрашивается; иначе — `None`.
 - (e) Метод `__delattr__` не позволяет удалять основные атрибуты, вызывая `AttributeError`.
 - (f) Пример: создаётся объект `flight`, проверяются все операции.
- 13 Создать класс `Restaurant`, который будет представлять ресторан с названием, кухней и рейтингом. Класс должен поддерживать методы перегрузки операций `__setattr__`, `__getattribute__`, `__getattr__` и `__delattr__`. Метод `__setattr__` должен проверять, что название и кухня — непустые строки, рейтинг — число от 0.0 до 5.0 с одним знаком после запятой. Метод `__getattribute__` должен возвращать значение атрибута или сообщение об ошибке. Метод `__getattr__` должен возвращать `None`, за исключением атрибута `Location`, который создаётся со значением `"Downtown"`. Метод `__delattr__` должен запрещать удаление основных атрибутов.
- (a) Класс `Restaurant` определяет атрибуты `name`, `cuisine` и `rating`.
 - (b) Метод `__setattr__` проверяет корректность и вызывает `ValueError` при ошибке.
 - (c) Метод `__getattribute__` возвращает атрибут или сообщение об ошибке.
 - (d) Метод `__getattr__` создаёт `Location = "Downtown"`, если запрашивается; иначе — `None`.
 - (e) Метод `__delattr__` не позволяет удалять основные атрибуты, вызывая `AttributeError`.
 - (f) Пример: создаётся объект `restaurant`, проверяются все методы.
- 14 Создать класс `Song`, который будет представлять музыкальную композицию с названием, исполнителем и длительностью (в секундах). Класс должен поддерживать методы перегрузки операций `__setattr__`, `__getattribute__`, `__getattr__` и `__delattr__`. Метод `__setattr__` должен проверять, что название и исполнитель — непустые строки, длительность — целое положительное число от 1 до 3600. Метод `__getattribute__` должен возвращать значение атрибута или сообщение об ошибке. Метод `__getattr__` должен возвращать `None`, за исключением атрибута `Album`, который создаётся со значением `"Greatest Hits"`. Метод `__delattr__` должен запрещать удаление основных атрибутов.
- (a) Класс `Song` определяет атрибуты `title`, `artist` и `duration`.
 - (b) Метод `__setattr__` валидирует значения и вызывает `ValueError` при ошибке.
 - (c) Метод `__getattribute__` возвращает атрибут или сообщение об ошибке.
 - (d) Метод `__getattr__` создаёт `Album = "Greatest Hits"`, если запрашивается; иначе — `None`.
 - (e) Метод `__delattr__` не позволяет удалять основные атрибуты, вызывая `AttributeError`.
 - (f) Пример: создаётся объект `song`, проверяются все операции.
- 15 Создать класс `Weather`, который будет представлять погодные условия с датой, температурой и описанием. Класс должен поддерживать методы перегрузки операций `__setattr__`, `__getattribute__`, `__getattr__` и `__delattr__`. Метод `__setattr__` должен проверять, что дата — строка в формате `YYYY-MM-DD`, температура — число от -100 до +60, описание — непустая строка. Метод `__getattribute__` должен возвращать

значение атрибута или сообщение об ошибке. Метод `__getattr__` должен возвращать `None`, за исключением атрибута `Unit`, который создаётся со значением `“Celsius”`. Метод `__delattr__` должен запрещать удаление основных атрибутов.

- (a) Класс `Weather` определяет атрибуты `date`, `temperature` и `description`.
- (b) Метод `__setattr__` проверяет корректность и вызывает `ValueError` при ошибке.
- (c) Метод `__getattribute__` возвращает атрибут или сообщение об ошибке.
- (d) Метод `__getattr__` создаёт `Unit = “Celsius”`, если запрашивается; иначе — `None`.
- (e) Метод `__delattr__` не позволяет удалять основные атрибуты, вызывая `AttributeError`.
- (f) Пример: создаётся объект `weather`, проверяются все методы.

16 Создать класс `Task`, который будет представлять задачу с описанием, статусом и дедлайном. Класс должен поддерживать методы перегрузки операций `__setattr__`, `__getattribute__`, `__getattr__` и `__delattr__`. Метод `__setattr__` должен проверять, что описание — непустая строка, статус — строка из набора `{“Pending”, “In Progress”, “Completed”}`, дедлайн — строка в формате `YYYY-MM-DD`. Метод `__getattribute__` должен возвращать значение атрибута или сообщение об ошибке. Метод `__getattr__` должен возвращать `None`, за исключением атрибута `Priority`, который создаётся со значением `“Medium”`. Метод `__delattr__` должен запрещать удаление основных атрибутов.

- (a) Класс `Task` определяет атрибуты `description`, `status` и `deadline`.
- (b) Метод `__setattr__` валидирует данные и вызывает `ValueError` при ошибке.
- (c) Метод `__getattribute__` возвращает атрибут или сообщение об ошибке.
- (d) Метод `__getattr__` создаёт `Priority = “Medium”`, если запрашивается; иначе — `None`.
- (e) Метод `__delattr__` не позволяет удалять основные атрибуты, вызывая `AttributeError`.
- (f) Пример: создаётся объект `task`, проверяются все операции.

17 Создать класс `House`, который будет представлять дом с адресом, количеством комнат и площадью. Класс должен поддерживать методы перегрузки операций `__setattr__`, `__getattribute__`, `__getattr__` и `__delattr__`. Метод `__setattr__` должен проверять, что адрес — непустая строка, количество комнат — целое число от 1 до 20, площадь — положительное число. Метод `__getattribute__` должен возвращать значение атрибута или сообщение об ошибке. Метод `__getattr__` должен возвращать `None`, за исключением атрибута `Type`, который создаётся со значением `“Residential”`. Метод `__delattr__` должен запрещать удаление основных атрибутов.

- (a) Класс `House` определяет атрибуты `address`, `rooms` и `area`.
- (b) Метод `__setattr__` проверяет корректность и вызывает `ValueError` при ошибке.
- (c) Метод `__getattribute__` возвращает атрибут или сообщение об ошибке.
- (d) Метод `__getattr__` создаёт `Type = “Residential”`, если запрашивается; иначе — `None`.
- (e) Метод `__delattr__` не позволяет удалять основные атрибуты, вызывая `AttributeError`.
- (f) Пример: создаётся объект `house`, проверяются все методы.

- 18 Создать класс `Planet`, который будет представлять планету с названием, диаметром и расстоянием до Солнца. Класс должен поддерживать методы перегрузки операций `__setattr__`, `__getattr__`, `__getattribute__` и `__delattr__`. Метод `__setattr__` должен проверять, что название — строка из букв, диаметр и расстояние — положительные числа. Метод `__getattribute__` должен возвращать значение атрибута или сообщение об ошибке. Метод `__getattr__` должен возвращать `None`, за исключением атрибута `System`, который создаётся со значением `"Solar"`. Метод `__delattr__` должен запрещать удаление основных атрибутов.
- (a) Класс `Planet` определяет атрибуты `name`, `diameter` и `distance_to_sun`.
 - (b) Метод `__setattr__` валидирует данные и вызывает `ValueError` при ошибке.
 - (c) Метод `__getattribute__` возвращает атрибут или сообщение об ошибке.
 - (d) Метод `__getattr__` создаёт `System = "Solar"`, если запрашивается; иначе — `None`.
 - (e) Метод `__delattr__` не позволяет удалять основные атрибуты, вызывая `AttributeError`.
 - (f) Пример: создаётся объект `planet`, проверяются все методы.
- 19 Создать класс `Event`, который будет представлять событие с названием, датой и местом проведения. Класс должен поддерживать методы перегрузки операций `__setattr__`, `__getattr__`, `__getattribute__` и `__delattr__`. Метод `__setattr__` должен проверять, что название и место — непустые строки, дата — строка в формате `YYYY-MM-DD`. Метод `__getattribute__` должен возвращать значение атрибута или сообщение об ошибке. Метод `__getattr__` должен возвращать `None`, за исключением атрибута `Organizer`, который создаётся со значением `"Community Group"`. Метод `__delattr__` должен запрещать удаление основных атрибутов.
- (a) Класс `Event` определяет атрибуты `name`, `date` и `location`.
 - (b) Метод `__setattr__` проверяет корректность и вызывает `ValueError` при ошибке.
 - (c) Метод `__getattribute__` возвращает атрибут или сообщение об ошибке.
 - (d) Метод `__getattr__` создаёт `Organizer = "Community Group"`, если запрашивается; иначе — `None`.
 - (e) Метод `__delattr__` не позволяет удалять основные атрибуты, вызывая `AttributeError`.
 - (f) Пример: создаётся объект `event`, проверяются все операции.
- 20 Создать класс `Device`, который будет представлять электронное устройство с моделью, производителем и годом выпуска. Класс должен поддерживать методы перегрузки операций `__setattr__`, `__getattr__`, `__getattribute__` и `__delattr__`. Метод `__setattr__` должен проверять, что модель и производитель — непустые строки, год — целое число от 1950 до текущего года. Метод `__getattribute__` должен возвращать значение атрибута или сообщение об ошибке. Метод `__getattr__` должен возвращать `None`, за исключением атрибута `OS`, который создаётся со значением `"Proprietary"`. Метод `__delattr__` должен запрещать удаление основных атрибутов.
- (a) Класс `Device` определяет атрибуты `model`, `manufacturer` и `year`.
 - (b) Метод `__setattr__` валидирует данные и вызывает `ValueError` при ошибке.
 - (c) Метод `__getattribute__` возвращает атрибут или сообщение об ошибке.
 - (d) Метод `__getattr__` создаёт `OS = "Proprietary"`, если запрашивается; иначе — `None`.

- (e) Метод `__delattr__` не позволяет удалять основные атрибуты, вызывая `AttributeError`.
- (f) Пример: создаётся объект `device`, проверяются все методы.
- 21 Создать класс `Recipe`, который будет представлять кулинарный рецепт с названием, списком ингредиентов и временем приготовления (в минутах). Класс должен поддерживать методы перегрузки операций `__setattr__`, `__getattr__`, `__delattr__` и `__getattribute__`. Метод `__setattr__` должен проверять, что название — непустая строка, ингредиенты — список непустых строк, время — целое число от 1 до 300. Метод `__getattribute__` должен возвращать значение атрибута или сообщение об ошибке. Метод `__getattr__` должен возвращать `None`, за исключением атрибута `Cuisine`, который создаётся со значением `“International”`. Метод `__delattr__` должен запрещать удаление основных атрибутов.
- (a) Класс `Recipe` определяет атрибуты `name`, `ingredients` и `cook_time`.
- (b) Метод `__setattr__` проверяет корректность и вызывает `ValueError` при ошибке.
- (c) Метод `__getattribute__` возвращает атрибут или сообщение об ошибке.
- (d) Метод `__getattr__` создаёт `Cuisine = “International”`, если запрашивается; иначе — `None`.
- (e) Метод `__delattr__` не позволяет удалять основные атрибуты, вызывая `AttributeError`.
- (f) Пример: создаётся объект `recipe`, проверяются все операции.
- 22 Создать класс `Project`, который будет представлять проект с названием, руководителем и бюджетом. Класс должен поддерживать методы перегрузки операций `__setattr__`, `__getattr__`, `__delattr__` и `__getattribute__`. Метод `__setattr__` должен проверять, что название и руководитель — непустые строки, бюджет — положительное число. Метод `__getattribute__` должен возвращать значение атрибута или сообщение об ошибке. Метод `__getattr__` должен возвращать `None`, за исключением атрибута `Status`, который создаётся со значением `“Active”`. Метод `__delattr__` должен запрещать удаление основных атрибутов.
- (a) Класс `Project` определяет атрибуты `name`, `manager` и `budget`.
- (b) Метод `__setattr__` валидирует данные и вызывает `ValueError` при ошибке.
- (c) Метод `__getattribute__` возвращает атрибут или сообщение об ошибке.
- (d) Метод `__getattr__` создаёт `Status = “Active”`, если запрашивается; иначе — `None`.
- (e) Метод `__delattr__` не позволяет удалять основные атрибуты, вызывая `AttributeError`.
- (f) Пример: создаётся объект `project`, проверяются все методы.
- 23 Создать класс `Ticket`, который будет представлять билет с идентификатором, типом и ценой. Класс должен поддерживать методы перегрузки операций `__setattr__`, `__getattr__`, `__delattr__` и `__getattribute__`. Метод `__setattr__` должен проверять, что идентификатор — строка из 8 символов (буквы и цифры), тип — строка из набора `{“Economy”, “Business”, “VIP”}`, цена — положительное число. Метод `__getattribute__` должен возвращать значение атрибута или сообщение об ошибке. Метод `__getattr__` должен возвращать `None`, за исключением атрибута `Validity`, который создаётся со значением `“1 year”`. Метод `__delattr__` должен запрещать удаление основных атрибутов.

- (a) Класс `Ticket` определяет атрибуты `id`, `type` и `price`.
 - (b) Метод `__setattr__` проверяет корректность и вызывает `ValueError` при ошибке.
 - (c) Метод `__getattr__` возвращает атрибут или сообщение об ошибке.
 - (d) Метод `__getattribute__` создаёт `Validity = "1 year"`, если запрашивается; иначе — `None`.
 - (e) Метод `__delattr__` не позволяет удалять основные атрибуты, вызывая `AttributeError`.
 - (f) Пример: создаётся объект `ticket`, проверяются все операции.
- 24 Создать класс `Document`, который будет представлять документ с названием, автором и датой создания. Класс должен поддерживать методы перегрузки операций `__setattr__`, `__getattribute__`, `__getattr__` и `__delattr__`. Метод `__setattr__` должен проверять, что название и автор — непустые строки, дата — строка в формате `YYYY-MM-DD`. Метод `__getattribute__` должен возвращать значение атрибута или сообщение об ошибке. Метод `__getattr__` должен возвращать `None`, за исключением атрибута `Format`, который создаётся со значением `"PDF"`. Метод `__delattr__` должен запрещать удаление основных атрибутов.
- (a) Класс `Document` определяет атрибуты `title`, `author` и `creation_date`.
 - (b) Метод `__setattr__` валидирует данные и вызывает `ValueError` при ошибке.
 - (c) Метод `__getattribute__` возвращает атрибут или сообщение об ошибке.
 - (d) Метод `__getattr__` создаёт `Format = "PDF"`, если запрашивается; иначе — `None`.
 - (e) Метод `__delattr__` не позволяет удалять основные атрибуты, вызывая `AttributeError`.
 - (f) Пример: создаётся объект `document`, проверяются все операции.
- 25 Создать класс `Pet`, который будет представлять домашнего питомца с кличкой, видом и весом. Класс должен поддерживать методы перегрузки операций `__setattr__`, `__getattribute__`, `__getattr__` и `__delattr__`. Метод `__setattr__` должен проверять, что кличка и вид — непустые строки из букв, вес — положительное число до 100. Метод `__getattribute__` должен возвращать значение атрибута или сообщение об ошибке. Метод `__getattr__` должен возвращать `None`, за исключением атрибута `Owner`, который создаётся со значением `"Anonymous"`. Метод `__delattr__` должен запрещать удаление основных атрибутов.
- (a) Класс `Pet` определяет атрибуты `name`, `species` и `weight`.
 - (b) Метод `__setattr__` проверяет корректность и вызывает `ValueError` при ошибке.
 - (c) Метод `__getattribute__` возвращает атрибут или сообщение об ошибке.
 - (d) Метод `__getattr__` создаёт `Owner = "Anonymous"`, если запрашивается; иначе — `None`.
 - (e) Метод `__delattr__` не позволяет удалять основные атрибуты, вызывая `AttributeError`.
 - (f) Пример: создаётся объект `pet`, проверяются все методы.
- 26 Создать класс `Lecture`, который будет представлять лекцию с темой, преподавателем и длительностью (в минутах). Класс должен поддерживать методы перегрузки операций `__setattr__`, `__getattribute__`, `__getattr__` и `__delattr__`. Метод `__setattr__`

должен проверять, что тема и преподаватель — непустые строки, длительность — целое число от 10 до 180. Метод `__getattribute__` должен возвращать значение атрибута или сообщение об ошибке. Метод `__getattr__` должен возвращать `None`, за исключением атрибута `Format`, который создаётся со значением `“In-person”`. Метод `__delattr__` должен запрещать удаление основных атрибутов.

- (a) Класс `Lecture` определяет атрибуты `topic`, `instructor` и `duration`.
- (b) Метод `__setattr__` валидирует данные и вызывает `ValueError` при ошибке.
- (c) Метод `__getattribute__` возвращает атрибут или сообщение об ошибке.
- (d) Метод `__getattr__` создаёт `Format = “In-person”`, если запрашивается; иначе — `None`.
- (e) Метод `__delattr__` не позволяет удалять основные атрибуты, вызывая `AttributeError`.
- (f) Пример: создаётся объект `lecture`, проверяются все операции.

27 Создать класс `Order`, который будет представлять заказ с номером, списком товаров и общей суммой. Класс должен поддерживать методы перегрузки операций `__setattr__`, `__getattribute__`, `__getattr__` и `__delattr__`. Метод `__setattr__` должен проверять, что номер — строка из 6 цифр, товары — список непустых строк, сумма — положительное число. Метод `__getattribute__` должен возвращать значение атрибута или сообщение об ошибке. Метод `__getattr__` должен возвращать `None`, за исключением атрибута `Status`, который создаётся со значением `“Processing”`. Метод `__delattr__` должен запрещать удаление основных атрибутов.

- (a) Класс `Order` определяет атрибуты `order_id`, `items` и `total`.
- (b) Метод `__setattr__` проверяет корректность и вызывает `ValueError` при ошибке.
- (c) Метод `__getattribute__` возвращает атрибут или сообщение об ошибке.
- (d) Метод `__getattr__` создаёт `Status = “Processing”`, если запрашивается; иначе — `None`.
- (e) Метод `__delattr__` не позволяет удалять основные атрибуты, вызывая `AttributeError`.
- (f) Пример: создаётся объект `order`, проверяются все методы.

28 Создать класс `Conference`, который будет представлять конференцию с названием, датой и местом проведения. Класс должен поддерживать методы перегрузки операций `__setattr__`, `__getattribute__`, `__getattr__` и `__delattr__`. Метод `__setattr__` должен проверять, что название и место — непустые строки, дата — строка в формате `YYYY-MM-DD`. Метод `__getattribute__` должен возвращать значение атрибута или сообщение об ошибке. Метод `__getattr__` должен возвращать `None`, за исключением атрибута `Theme`, который создаётся со значением `“Innovation”`. Метод `__delattr__` должен запрещать удаление основных атрибутов.

- (a) Класс `Conference` определяет атрибуты `name`, `date` и `location`.
- (b) Метод `__setattr__` валидирует данные и вызывает `ValueError` при ошибке.
- (c) Метод `__getattribute__` возвращает атрибут или сообщение об ошибке.
- (d) Метод `__getattr__` создаёт `Theme = “Innovation”`, если запрашивается; иначе — `None`.
- (e) Метод `__delattr__` не позволяет удалять основные атрибуты, вызывая `AttributeError`.

- (f) Пример: создаётся объект `conference`, проверяются все операции.
- 29 Создать класс `Album`, который будет представлять музыкальный альбом с названием, исполнителем и годом выпуска. Класс должен поддерживать методы перегрузки операций `__setattr__`, `__getattr__`, `__getattribute__` и `__delattr__`. Метод `__setattr__` должен проверять, что название и исполнитель — непустые строки, год — целое число от 1900 до текущего года. Метод `__getattribute__` должен возвращать значение атрибута или сообщение об ошибке. Метод `__getattr__` должен возвращать `None`, за исключением атрибута `Genre`, который создаётся со значением “Pop”. Метод `__delattr__` должен запрещать удаление основных атрибутов.
- (a) Класс `Album` определяет атрибуты `title`, `artist` и `year`.
 - (b) Метод `__setattr__` проверяет корректность и вызывает `ValueError` при ошибке.
 - (c) Метод `__getattribute__` возвращает атрибут или сообщение об ошибке.
 - (d) Метод `__getattr__` создаёт `Genre = “Pop”`, если запрашивается; иначе — `None`.
 - (e) Метод `__delattr__` не позволяет удалять основные атрибуты, вызывая `AttributeError`.
 - (f) Пример: создаётся объект `album`, проверяются все методы.
- 30 Создать класс `Building`, который будет представлять здание с названием, количеством этажей и годом постройки. Класс должен поддерживать методы перегрузки операций `__setattr__`, `__getattr__`, `__getattribute__` и `__delattr__`. Метод `__setattr__` должен проверять, что название — непустая строка, количество этажей — целое число от 1 до 200, год — целое число от 1000 до текущего года. Метод `__getattribute__` должен возвращать значение атрибута или сообщение об ошибке. Метод `__getattr__` должен возвращать `None`, за исключением атрибута `Use`, который создаётся со значением “Commercial”. Метод `__delattr__` должен запрещать удаление основных атрибутов.
- (a) Класс `Building` определяет атрибуты `name`, `floors` и `year_built`.
 - (b) Метод `__setattr__` проверяет корректность и вызывает `ValueError` при ошибке.
 - (c) Метод `__getattribute__` возвращает атрибут или сообщение об ошибке.
 - (d) Метод `__getattr__` создаёт `Use = “Commercial”`, если запрашивается; иначе — `None`.
 - (e) Метод `__delattr__` не позволяет удалять основные атрибуты, вызывая `AttributeError`.
 - (f) Пример: создаётся объект `building`, проверяются все операции.
- 31 Создать класс `Trip`, который будет представлять поездку с направлением, продолжительностью (в днях) и бюджетом. Класс должен поддерживать методы перегрузки операций `__setattr__`, `__getattr__`, `__getattribute__` и `__delattr__`. Метод `__setattr__` должен проверять, что направление — непустая строка, продолжительность — целое число от 1 до 365, бюджет — положительное число. Метод `__getattribute__` должен возвращать значение атрибута или сообщение об ошибке. Метод `__getattr__` должен возвращать `None`, за исключением атрибута `Type`, который создаётся со значением “Leisure”. Метод `__delattr__` должен запрещать удаление основных атрибутов.
- (a) Класс `Trip` определяет атрибуты `destination`, `duration` и `budget`.
 - (b) Метод `__setattr__` валидирует данные и вызывает `ValueError` при ошибке.
 - (c) Метод `__getattribute__` возвращает атрибут или сообщение об ошибке.

- (d) Метод `__getattr__` создаёт `Type = "Leisure"`, если запрашивается; иначе — `None`.
 - (e) Метод `__delattr__` не позволяет удалять основные атрибуты, вызывая `AttributeError`.
 - (f) Пример: создаётся объект `trip`, проверяются все методы.
- 32 Создать класс `Invoice`, который будет представлять счёт с номером, датой и суммой. Класс должен поддерживать методы перегрузки операций `__setattr__`, `__getattr__`, `__getattribute__` и `__delattr__`. Метод `__setattr__` должен проверять, что номер — строка из 8 символов (буквы и цифры), дата — строка в формате `YYYY-MM-DD`, сумма — положительное число. Метод `__getattribute__` должен возвращать значение атрибута или сообщение об ошибке. Метод `__getattr__` должен возвращать `None`, за исключением атрибута `Currency`, который создаётся со значением `"USD"`. Метод `__delattr__` должен запрещать удаление основных атрибутов.
- (a) Класс `Invoice` определяет атрибуты `invoice_number`, `date` и `amount`.
 - (b) Метод `__setattr__` проверяет корректность и вызывает `ValueError` при ошибке.
 - (c) Метод `__getattribute__` возвращает атрибут или сообщение об ошибке.
 - (d) Метод `__getattr__` создаёт `Currency = "USD"`, если запрашивается; иначе — `None`.
 - (e) Метод `__delattr__` не позволяет удалять основные атрибуты, вызывая `AttributeError`.
 - (f) Пример: создаётся объект `invoice`, проверяются все операции.
- 33 Создать класс `Library`, который будет представлять библиотеку с названием, адресом и годом основания. Класс должен поддерживать методы перегрузки операций `__setattr__`, `__getattr__`, `__getattribute__` и `__delattr__`. Метод `__setattr__` должен проверять, что название и адрес — непустые строки, год — целое число от 1000 до текущего года. Метод `__getattribute__` должен возвращать значение атрибута или сообщение об ошибке. Метод `__getattr__` должен возвращать `None`, за исключением атрибута `Type`, который создаётся со значением `"Public"`. Метод `__delattr__` должен запрещать удаление основных атрибутов.
- (a) Класс `Library` определяет атрибуты `name`, `address` и `founded`.
 - (b) Метод `__setattr__` проверяет корректность и вызывает `ValueError` при ошибке.
 - (c) Метод `__getattribute__` возвращает атрибут или сообщение об ошибке.
 - (d) Метод `__getattr__` создаёт `Type = "Public"`, если запрашивается; иначе — `None`.
 - (e) Метод `__delattr__` не позволяет удалять основные атрибуты, вызывая `AttributeError`.
 - (f) Пример: создаётся объект `library`, проверяются все методы.
- 34 Создать класс `Sensor`, который будет представлять датчик с идентификатором, типом и текущим значением. Класс должен поддерживать методы перегрузки операций `__setattr__`, `__getattr__`, `__getattribute__` и `__delattr__`. Метод `__setattr__` должен проверять, что идентификатор — строка из 6 цифр, тип — непустая строка, значение — число. Метод `__getattribute__` должен возвращать значение атрибута или сообщение об ошибке. Метод `__getattr__` должен возвращать `None`, за исключением атрибута `Unit`, который создаётся со значением `"V"`. Метод `__delattr__` должен запрещать удаление основных атрибутов.
- (a) Класс `Sensor` определяет атрибуты `id`, `type` и `value`.

- (b) Метод `__setattr__` проверяет корректность и вызывает `ValueError` при ошибке.
- (c) Метод `__getattribute__` возвращает атрибут или сообщение об ошибке.
- (d) Метод `__getattr__` создаёт `Unit = "V"`, если запрашивается; иначе — `None`.
- (e) Метод `__delattr__` не позволяет удалять основные атрибуты, вызывая `AttributeError`.
- (f) Пример: создаётся объект `sensor`, проверяются все операции.

35 Создать класс `Member`, который будет представлять участника клуба с именем, фамилией и датой регистрации. Класс должен поддерживать методы перегрузки операций `__setattr__`, `__getattribute__`, `__getattr__` и `__delattr__`. Метод `__setattr__` должен проверять, что имя и фамилия — строки только из букв, дата — строка в формате `YYYY-MM-DD`. Метод `__getattribute__` должен возвращать значение атрибута или сообщение об ошибке. Метод `__getattr__` должен возвращать `None`, за исключением атрибута `Status`, который создаётся со значением `"Active"`. Метод `__delattr__` должен запрещать удаление основных атрибутов.

- (a) Класс `Member` определяет атрибуты `first_name`, `last_name` и `registration_date`.
- (b) Метод `__setattr__` валидирует данные и вызывает `ValueError` при нарушении формата.
- (c) Метод `__getattribute__` возвращает атрибут или сообщение об ошибке.
- (d) Метод `__getattr__` создаёт `Status = "Active"`, если запрашивается; иначе — `None`.
- (e) Метод `__delattr__` не позволяет удалять `first_name`, `last_name` или `registration_date`, вызывая `AttributeError`.
- (f) Пример: создаётся объект `member`, проверяются все операции с атрибутами.

2.13 Семинар «Разработка метакласса и классов для создания таблиц в базах данных объектным способом»

Необходимо разработать программу на основе метакласса, которая позволяет создавать модели (таблицы) данных, используя технологию ORM. ORM (Object-Relational Mapping, объектно-реляционное отображение) — это технология ООП программирования, которая позволяет разработчикам взаимодействовать с реляционными базами данных, работая с ними как с обычными объектами ООП в памяти программы, а не напрямую с SQL-запросами. Основная цель ORM — абстракция доступа к данным, что делает работу с базами данных проще и безопаснее. Примером реализации такой технологии являются классы `models.Model` веб-фреймворка Django.

Важно: В рамках данного практического задания **не предполагается подключение к реальной базе данных**. Программа должна генерировать и **выводить SQL-запросы в консоль**, демонстрируя корректность формирования DDL (Data Definition Language) и DML (Data Manipulation Language) команд.

Для реализации этой задачи необходимо создать метакласс `ModelBase`, который будет служить основой для создания классов моделей данных. Этот метакласс должен обеспечивать следующие возможности:

1. **Сбор полей модели:** Метакласс должен уметь анализировать пространство имён создаваемого класса и собирать все поля, которые являются объектами класса `Field`. Собранные поля должны сохраняться в специальном атрибуте `_fields` нового класса, а сами поля должны удаляться из исходного пространства имён, чтобы избежать конфликтов.
2. Необходимо реализовать классы-типы данных, которые будут представлять различные типы полей в модели:
 - **CharField:** Представляет строку символов с возможностью указания максимальной длины и ограничений на допустимые значения.
 - **IntegerField:** Представляет целое число с возможностью задания ограничений на допустимые значения.
 - **DateField:** Представляет дату с возможностью задания ограничений на допустимые значения.

Эти классы должны наследоваться от общего класса `Field` и переопределять метод `to_sql()` для генерации соответствующей части SQL-запроса, учитывая особенности каждого типа данных.

3. **Создание класса:** Метакласс должен формировать новый класс на основе собранной информации и возвращать его. Новый класс должен содержать собранные поля в виде словаря `_fields`.
4. **Генерация SQL-запросов:** Класс модели должен предоставлять метод для генерации SQL-запроса, который «создаёт» таблицу в базе данных на основе собранных полей. Этот запрос должен учитывать типы данных и ограничения, заданные в соответствующих полях.

Алгоритм программирования задачи

Алгоритм программирования задачи, позволяющей создавать классы для описания таблиц баз данных с помощью метакласса `ModelBase` и других классов, таких как `Field`, `CharField`, `IntegerField`, `DateField`, `Model`, следующий:

1. **Определение класса `Field`:** Определяется базовый класс `Field`, который будет представлять различные типы данных в модели.
 - (a) Импортируйте модуль `datetime`.
 - (b) Определите класс `Field`.
 - (c) В конструкторе `__init__` примите аргументы: `field_type`, `max_length=None`, `null=False`, `default=None`.
 - (d) Сохраните их как атрибуты экземпляра.
 - (e) Добавьте метод `to_sql(self, name)`, который генерирует SQL-представление столбца:
 - Сопоставьте Python-типы с SQL-типами (например, `str` → `VARCHAR`, `int` → `INTEGER`, `datetime.date` → `DATE`).
 - Если `max_length` задано и тип — строковый, используйте `VARCHAR(max_length)`.
 - Добавьте `NOT NULL`, если `null=False`.
 - Укажите `DEFAULT` в соответствии со значением `default`.**Важно:** На данном этапе `default` используется только для генерации DDL. Для вызываемых значений (callable defaults, например, `datetime.date.today`) в DDL следует подставлять конкретное значение, полученное при создании модели (см. ниже).
2. **Реализация классов-типов данных:**
 - (a) `CharField`:
 - В `__init__` установите: `max_length=255` (по умолчанию), `null=False`, `default=""`.
 - Вызовите `super().__init__(str, max_length, null, default)`.
 - (b) `IntegerField`:
 - В `__init__` установите: `null=False`, `default=0`.
 - Вызовите `super().__init__(int, None, null, default)`.
 - (c) `DateField`:
 - В `__init__` установите: `null=False`, `default=datetime.date.today` (**без скобок!**).
 - Вызовите `super().__init__(datetime.date, None, null, default)`.
 - **Пояснение:** Передача `datetime.date.today` как callable позволяет вычислять «сегодняшнюю дату» на момент создания объекта модели, а не на момент определения класса.
3. **Создание метакласса `ModelBase`:**
 - (a) Определите `class ModelBase(type)`.
 - (b) Реализуйте `__new__(cls, name, bases, attrs)`:
 - Создайте пустой словарь `fields`.

- Пройдитесь по `attrs`, соберите все значения, являющиеся экземплярами `Field`, в `fields`, и удалите их из `attrs`.
- Вызовите `super().__new__(cls, name, bases, attrs)`.
- Присвойте `new_class._fields = fields`.
- Верните `new_class`.

4. Создание базового класса `Model`:

- Определите `class Model(metaclass=ModelBase)`.
- В `__init__(self, **kwargs)`:
 - Для каждого поля из `self._fields`:
 - Если имя поля есть в `kwargs`, используйте переданное значение.
 - Иначе, если у поля есть `default`:
 - * Если `default` — callable (например, `datetime.date.today`), вызовите его.
 - * Иначе — используйте значение напрямую.
 - Установите значение через `setattr(self, name, value)`.
- Реализуйте метод класса `@classmethod def create_table(cls, table_name)`:
 - Соберите SQL-представление каждого поля: `field.to_sql(name)`.
 - Сформируйте команду `CREATE TABLE IF NOT EXISTS {table_name} (...)`.
 - **Выведите её на печать.**
- Реализуйте метод `save(self)`:
 - Получите значения всех полей из `self._fields` через `getattr(self, name)`.
 - **Важно:** Для безопасной работы с данными в реальных ORM используют **параметризованные запросы**, чтобы предотвратить SQL-инъекции. В данном учебном задании, поскольку мы только выводим SQL, **значения должны быть корректно экранированы или преобразованы в SQL-литералы** (например, строки — в кавычки, числа — без кавычек, даты — в формате 'ГГГГ-ММ-ДД'; не забудьте про то, что внутри строки могут быть кавычки, переносы строк и другие особенности).
 - Сформируйте `INSERT INTO ... VALUES (...)`.
 - Выведите запрос на печать.

2.13.1 Тестирование созданной программы (по вариантам)

- Определите класс `Book`:


```
class Book(Model):
    title = CharField(max_length=255)
    author = CharField(max_length=100)
    published_date = DateField()
    year = IntegerField()
```
 - Вызовите: `Book.create_table('books')`
 - Создайте экземпляр:

```

book = Book(
    title='Python Cookbook',
    author='David Beazley',
    published_date=datetime.date(2013, 5, 10),
    year=2012,
)
book.save()

```

Программа должна вывести следующие SQL-запросы:

- (a) Запрос на создание таблицы (пример для даты выполнения 2025-04-05):

```

CREATE TABLE IF NOT EXISTS books (
    title VARCHAR(255) NOT NULL DEFAULT '',
    author VARCHAR(100) NOT NULL DEFAULT '',
    published_date DATE NOT NULL DEFAULT '2025-04-05',
    year INTEGER NOT NULL DEFAULT 0
);

```

- (b) Запрос на вставку данных:

```

INSERT INTO books (title, author, published_date, year) VALUES
('Python Cookbook', 'David Beazley', '2013-05-10', 2012);

```

- 2 (a) Определите класс Student:

```

class Student(Model):
    name = CharField(max_length=100)
    surname = CharField(max_length=100)
    birth_date = DateField()
    course = IntegerField()

```

- (b) Вызовите: Student.create_table('students')

- (c) Создайте экземпляр:

```

student = Student(
    name='Иван',
    surname='Петров',
    birth_date=datetime.date(2002, 8, 15),
    course=3,
)
student.save()

```

Программа должна вывести следующие SQL-запросы:

- (a) Запрос на создание таблицы (пример для даты выполнения 2025-04-05):

```
CREATE TABLE IF NOT EXISTS students (
    name VARCHAR(100) NOT NULL DEFAULT '',
    surname VARCHAR(100) NOT NULL DEFAULT '',
    birth_date DATE NOT NULL DEFAULT '2025-04-05',
    course INTEGER NOT NULL DEFAULT 0
);
```

- (b) Запрос на вставку данных:

```
INSERT INTO students (name, surname, birth_date, course) VALUES
('Иван', 'Петров', '2002-08-15', 3);
```

- 3 (a) Определите класс Product:

```
class Product(Model):
    name = CharField(max_length=200)
    brand = CharField(max_length=100)
    release_date = DateField()
    price_cents = IntegerField()
```

- (b) Вызовите: Product.create_table('products')

- (c) Создайте экземпляр:

```
product = Product(
    name='Wireless Headphones',
    brand='SoundMax',
    release_date=datetime.date(2023, 11, 1),
    price_cents=5999,
)
product.save()
```

Программа должна вывести следующие SQL-запросы:

- (a) Запрос на создание таблицы (пример для даты выполнения 2025-04-05):

```
CREATE TABLE IF NOT EXISTS products (
    name VARCHAR(200) NOT NULL DEFAULT '',
    brand VARCHAR(100) NOT NULL DEFAULT '',
    release_date DATE NOT NULL DEFAULT '2025-04-05',
    price_cents INTEGER NOT NULL DEFAULT 0
);
```

- (b) Запрос на вставку данных:

```
INSERT INTO products (name, brand, release_date, price_cents) VALUES
('Wireless Headphones', 'SoundMax', '2023-11-01', 5999);
```

- 4 (a) Определите класс Employee:


```
class Employee(Model):
    first_name = CharField(max_length=50)
    last_name = CharField(max_length=50)
    hire_date = DateField()
    department_id = IntegerField()
```

(b) Вызовите: `Employee.create_table('employees')`

(c) Создайте экземпляр:

```
employee = Employee(
    first_name='Анна',
    last_name='Смирнова',
    hire_date=datetime.date(2021, 3, 12),
    department_id=7,
)
employee.save()
```

Программа должна вывести следующие SQL-запросы:

(a) Запрос на создание таблицы (пример для даты выполнения 2025-04-05):

```
CREATE TABLE IF NOT EXISTS employees (
    first_name VARCHAR(50) NOT NULL DEFAULT '',
    last_name VARCHAR(50) NOT NULL DEFAULT '',
    hire_date DATE NOT NULL DEFAULT '2025-04-05',
    department_id INTEGER NOT NULL DEFAULT 0
);
```

(b) Запрос на вставку данных:

```
INSERT INTO employees (first_name, last_name, hire_date, department_id) VALUES
('Анна', 'Смирнова', '2021-03-12', 7);
```

5 (a) Определите класс `Movie`:

```
class Movie(Model):
    title = CharField(max_length=150)
    director = CharField(max_length=100)
    release_date = DateField()
    duration_minutes = IntegerField()
```

(b) Вызовите: `Movie.create_table('movies')`

(c) Создайте экземпляр:

```
movie = Movie(
    title='Inception',
    director='Christopher Nolan',
    release_date=datetime.date(2010, 7, 16),
```

```

        duration_minutes=148,
    )
    movie.save()

```

Программа должна вывести следующие SQL-запросы:

- (a) Запрос на создание таблицы (пример для даты выполнения 2025-04-05):

```

CREATE TABLE IF NOT EXISTS movies (
    title VARCHAR(150) NOT NULL DEFAULT '',
    director VARCHAR(100) NOT NULL DEFAULT '',
    release_date DATE NOT NULL DEFAULT '2025-04-05',
    duration_minutes INTEGER NOT NULL DEFAULT 0
);

```

- (b) Запрос на вставку данных:

```

INSERT INTO movies (title, director, release_date, duration_minutes) VALUES
('Inception', 'Christopher Nolan', '2010-07-16', 148);

```

- 6 (a) Определите класс Car:

```

class Car(Model):
    make = CharField(max_length=50)
    model = CharField(max_length=50)
    production_date = DateField()
    horsepower = IntegerField()

```

- (b) Вызовите: Car.create_table('cars')

- (c) Создайте экземпляр:

```

car = Car(
    make='Toyota',
    model='Corolla',
    production_date=datetime.date(2022, 9, 5),
    horsepower=139,
)
car.save()

```

Программа должна вывести следующие SQL-запросы:

- (a) Запрос на создание таблицы (пример для даты выполнения 2025-04-05):

```

CREATE TABLE IF NOT EXISTS cars (
    make VARCHAR(50) NOT NULL DEFAULT '',
    model VARCHAR(50) NOT NULL DEFAULT '',
    production_date DATE NOT NULL DEFAULT '2025-04-05',
    horsepower INTEGER NOT NULL DEFAULT 0
);

```

- (b) Запрос на вставку данных:

```
INSERT INTO cars (make, model, production_date, horsepower) VALUES
('Toyota', 'Corolla', '2022-09-05', 139);
```

- 7 (a) Определите класс Task:

```
class Task(Model):
    title = CharField(max_length=200)
    assignee = CharField(max_length=100)
    due_date = DateField()
    priority = IntegerField()
```

- (b) Вызовите: Task.create_table('tasks')

- (c) Создайте экземпляр:

```
task = Task(
    title='Refactor ORM module',
    assignee='developer@example.com',
    due_date=datetime.date(2025, 4, 20),
    priority=1,
)
task.save()
```

Программа должна вывести следующие SQL-запросы:

- (a) Запрос на создание таблицы (пример для даты выполнения 2025-04-05):

```
CREATE TABLE IF NOT EXISTS tasks (
    title VARCHAR(200) NOT NULL DEFAULT '',
    assignee VARCHAR(100) NOT NULL DEFAULT '',
    due_date DATE NOT NULL DEFAULT '2025-04-05',
    priority INTEGER NOT NULL DEFAULT 0
);
```

- (b) Запрос на вставку данных:

```
INSERT INTO tasks (title, assignee, due_date, priority) VALUES
('Refactor ORM module', 'developer@example.com', '2025-04-20', 1);
```

- 8 (a) Определите класс Course:

```
class Course(Model):
    name = CharField(max_length=255)
    instructor = CharField(max_length=100)
    start_date = DateField()
    credits = IntegerField()
```

- (b) Вызовите: Course.create_table('courses')

(c) Создайте экземпляр:

```
course = Course(  
    name='Advanced Python Programming',  
    instructor='Dr. Elena Volkova',  
    start_date=datetime.date(2025, 2, 10),  
    credits=6,  
)  
course.save()
```

Программа должна вывести следующие SQL-запросы:

(a) Запрос на создание таблицы (пример для даты выполнения 2025-04-05):

```
CREATE TABLE IF NOT EXISTS courses (  
    name VARCHAR(255) NOT NULL DEFAULT '',  
    instructor VARCHAR(100) NOT NULL DEFAULT '',  
    start_date DATE NOT NULL DEFAULT '2025-04-05',  
    credits INTEGER NOT NULL DEFAULT 0  
);
```

(b) Запрос на вставку данных:

```
INSERT INTO courses (name, instructor, start_date, credits) VALUES  
( 'Advanced Python Programming', 'Dr. Elena Volkova', '2025-02-10', 6);
```

9 (a) Определите класс `Order`:

```
class Order(Model):  
    customer_name = CharField(max_length=150)  
    product = CharField(max_length=200)  
    order_date = DateField()  
    quantity = IntegerField()
```

(b) Вызовите: `Order.create_table('orders')`

(c) Создайте экземпляр:

```
order = Order(  
    customer_name='John Doe',  
    product='Laptop Stand',  
    order_date=datetime.date(2025, 3, 22),  
    quantity=2,  
)  
order.save()
```

Программа должна вывести следующие SQL-запросы:

(a) Запрос на создание таблицы (пример для даты выполнения 2025-04-05):

```
CREATE TABLE IF NOT EXISTS orders (
    customer_name VARCHAR(150) NOT NULL DEFAULT '',
    product VARCHAR(200) NOT NULL DEFAULT '',
    order_date DATE NOT NULL DEFAULT '2025-04-05',
    quantity INTEGER NOT NULL DEFAULT 0
);
```

(b) Запрос на вставку данных:

```
INSERT INTO orders (customer_name, product, order_date, quantity) VALUES
('John Doe', 'Laptop Stand', '2025-03-22', 2);
```

10 (a) Определите класс Event:

```
class Event(Model):
    title = CharField(max_length=300)
    location = CharField(max_length=200)
    event_date = DateField()
    attendees = IntegerField()
```

(b) Вызовите: Event.create_table('events')

(c) Создайте экземпляр:

```
event = Event(
    title='Tech Conference 2025',
    location='Moscow Expo Center',
    event_date=datetime.date(2025, 6, 15),
    attendees=350,
)
event.save()
```

Программа должна вывести следующие SQL-запросы:

(a) Запрос на создание таблицы (пример для даты выполнения 2025-04-05):

```
CREATE TABLE IF NOT EXISTS events (
    title VARCHAR(300) NOT NULL DEFAULT '',
    location VARCHAR(200) NOT NULL DEFAULT '',
    event_date DATE NOT NULL DEFAULT '2025-04-05',
    attendees INTEGER NOT NULL DEFAULT 0
);
```

(b) Запрос на вставку данных:

```
INSERT INTO events (title, location, event_date, attendees) VALUES
('Tech Conference 2025', 'Moscow Expo Center', '2025-06-15', 350);
```

11 (a) Определите класс Animal:

```
class Animal(Model):
    species = CharField(max_length=100)
    name = CharField(max_length=50)
    birth_date = DateField()
    weight_kg = IntegerField()
```

(b) Вызовите: `Animal.create_table('animals')`

(c) Создайте экземпляр:

```
animal = Animal(
    species='Dog',
    name='Buddy',
    birth_date=datetime.date(2020, 11, 3),
    weight_kg=25,
)
animal.save()
```

Программа должна вывести следующие SQL-запросы:

(a) Запрос на создание таблицы (пример для даты выполнения 2025-04-05):

```
CREATE TABLE IF NOT EXISTS animals (
    species VARCHAR(100) NOT NULL DEFAULT '',
    name VARCHAR(50) NOT NULL DEFAULT '',
    birth_date DATE NOT NULL DEFAULT '2025-04-05',
    weight_kg INTEGER NOT NULL DEFAULT 0
);
```

(b) Запрос на вставку данных:

```
INSERT INTO animals (species, name, birth_date, weight_kg) VALUES
('Dog', 'Buddy', '2020-11-03', 25);
```

12 (a) Определите класс `Recipe`:

```
class Recipe(Model):
    dish_name = CharField(max_length=200)
    chef = CharField(max_length=100)
    created_date = DateField()
    difficulty = IntegerField()
```

(b) Вызовите: `Recipe.create_table('recipes')`

(c) Создайте экземпляр:

```
recipe = Recipe(
    dish_name='Beef Stroganoff',
    chef='Maria Ivanova',
    created_date=datetime.date(2023, 12, 1),
```

```

        difficulty=3,
    )
    recipe.save()

```

Программа должна вывести следующие SQL-запросы:

- (a) Запрос на создание таблицы (пример для даты выполнения 2025-04-05):

```

CREATE TABLE IF NOT EXISTS recipes (
    dish_name VARCHAR(200) NOT NULL DEFAULT '',
    chef VARCHAR(100) NOT NULL DEFAULT '',
    created_date DATE NOT NULL DEFAULT '2025-04-05',
    difficulty INTEGER NOT NULL DEFAULT 0
);

```

- (b) Запрос на вставку данных:

```

INSERT INTO recipes (dish_name, chef, created_date, difficulty) VALUES
('Beef Stroganoff', 'Maria Ivanova', '2023-12-01', 3);

```

- 13 (a) Определите класс Flight:

```

class Flight(Model):
    airline = CharField(max_length=100)
    flight_number = CharField(max_length=10)
    departure_date = DateField()
    seats = IntegerField()

```

- (b) Вызовите: Flight.create_table('flights')

- (c) Создайте экземпляр:

```

flight = Flight(
    airline='Aeroflot',
    flight_number='SU1234',
    departure_date=datetime.date(2025, 5, 8),
    seats=180,
)
flight.save()

```

Программа должна вывести следующие SQL-запросы:

- (a) Запрос на создание таблицы (пример для даты выполнения 2025-04-05):

```

CREATE TABLE IF NOT EXISTS flights (
    airline VARCHAR(100) NOT NULL DEFAULT '',
    flight_number VARCHAR(10) NOT NULL DEFAULT '',
    departure_date DATE NOT NULL DEFAULT '2025-04-05',
    seats INTEGER NOT NULL DEFAULT 0
);

```

- (b) Запрос на вставку данных:

```
INSERT INTO flights (airline, flight_number, departure_date, seats) VALUES
('Aeroflot', 'SU1234', '2025-05-08', 180);
```

- 14 (a) Определите класс User:

```
class User(Model):
    username = CharField(max_length=50)
    email = CharField(max_length=150)
    registration_date = DateField()
    score = IntegerField()
```

- (b) Вызовите: `User.create_table('users')`

- (c) Создайте экземпляр:

```
user = User(
    username='coder42',
    email='coder42@example.com',
    registration_date=datetime.date(2024, 1, 10),
    score=1250,
)
user.save()
```

Программа должна вывести следующие SQL-запросы:

- (a) Запрос на создание таблицы (пример для даты выполнения 2025-04-05):

```
CREATE TABLE IF NOT EXISTS users (
    username VARCHAR(50) NOT NULL DEFAULT '',
    email VARCHAR(150) NOT NULL DEFAULT '',
    registration_date DATE NOT NULL DEFAULT '2025-04-05',
    score INTEGER NOT NULL DEFAULT 0
);
```

- (b) Запрос на вставку данных:

```
INSERT INTO users (username, email, registration_date, score) VALUES
('coder42', 'coder42@example.com', '2024-01-10', 1250);
```

- 15 (a) Определите класс Project:

```
class Project(Model):
    name = CharField(max_length=200)
    manager = CharField(max_length=100)
    start_date = DateField()
    budget_thousands = IntegerField()
```

- (b) Вызовите: `Project.create_table('projects')`

(c) Создайте экземпляр:

```
project = Project(  
    name='ORM Framework Development',  
    manager='Alex Johnson',  
    start_date=datetime.date(2025, 1, 15),  
    budget_thousands=250,  
)  
project.save()
```

Программа должна вывести следующие SQL-запросы:

(a) Запрос на создание таблицы (пример для даты выполнения 2025-04-05):

```
CREATE TABLE IF NOT EXISTS projects (  
    name VARCHAR(200) NOT NULL DEFAULT '',  
    manager VARCHAR(100) NOT NULL DEFAULT '',  
    start_date DATE NOT NULL DEFAULT '2025-04-05',  
    budget_thousands INTEGER NOT NULL DEFAULT 0  
);
```

(b) Запрос на вставку данных:

```
INSERT INTO projects (name, manager, start_date, budget_thousands) VALUES  
( 'ORM Framework Development', 'Alex Johnson', '2025-01-15', 250);
```

16 (a) Определите класс Invoice:

```
class Invoice(Model):  
    client = CharField(max_length=150)  
    description = CharField(max_length=300)  
    issue_date = DateField()  
    amount = IntegerField()
```

(b) Вызовите: Invoice.create_table('invoices')

(c) Создайте экземпляр:

```
invoice = Invoice(  
    client='ABC Corp',  
    description='Software License',  
    issue_date=datetime.date(2025, 3, 1),  
    amount=4500,  
)  
invoice.save()
```

Программа должна вывести следующие SQL-запросы:

(a) Запрос на создание таблицы (пример для даты выполнения 2025-04-05):

```
CREATE TABLE IF NOT EXISTS invoices (
    client VARCHAR(150) NOT NULL DEFAULT '',
    description VARCHAR(300) NOT NULL DEFAULT '',
    issue_date DATE NOT NULL DEFAULT '2025-04-05',
    amount INTEGER NOT NULL DEFAULT 0
);
```

- (b) Запрос на вставку данных:

```
INSERT INTO invoices (client, description, issue_date, amount) VALUES
('ABC Corp', 'Software License', '2025-03-01', 4500);
```

- 17 (a) Определите класс License:

```
class License(Model):
    software = CharField(max_length=100)
    owner = CharField(max_length=150)
    valid_from = DateField()
    duration_days = IntegerField()
```

- (b) Вызовите: License.create_table('licenses')

- (c) Создайте экземпляр:

```
license = License(
    software='TextEditor Pro',
    owner='Jane Smith',
    valid_from=datetime.date(2025, 2, 20),
    duration_days=365,
)
license.save()
```

Программа должна вывести следующие SQL-запросы:

- (a) Запрос на создание таблицы (пример для даты выполнения 2025-04-05):

```
CREATE TABLE IF NOT EXISTS licenses (
    software VARCHAR(100) NOT NULL DEFAULT '',
    owner VARCHAR(150) NOT NULL DEFAULT '',
    valid_from DATE NOT NULL DEFAULT '2025-04-05',
    duration_days INTEGER NOT NULL DEFAULT 0
);
```

- (b) Запрос на вставку данных:

```
INSERT INTO licenses (software, owner, valid_from, duration_days) VALUES
('TextEditor Pro', 'Jane Smith', '2025-02-20', 365);
```

- 18 (a) Определите класс Sensor:

```
class Sensor(Model):
    model = CharField(max_length=50)
    location = CharField(max_length=200)
    install_date = DateField()
    reading_interval_sec = IntegerField()
```

(b) Вызовите: `Sensor.create_table('sensors')`

(c) Создайте экземпляр:

```
sensor = Sensor(
    model='TempX200',
    location='Warehouse A',
    install_date=datetime.date(2024, 10, 5),
    reading_interval_sec=30,
)
sensor.save()
```

Программа должна вывести следующие SQL-запросы:

(a) Запрос на создание таблицы (пример для даты выполнения 2025-04-05):

```
CREATE TABLE IF NOT EXISTS sensors (
    model VARCHAR(50) NOT NULL DEFAULT '',
    location VARCHAR(200) NOT NULL DEFAULT '',
    install_date DATE NOT NULL DEFAULT '2025-04-05',
    reading_interval_sec INTEGER NOT NULL DEFAULT 0
);
```

(b) Запрос на вставку данных:

```
INSERT INTO sensors (model, location, install_date, reading_interval_sec) VALUES
('TempX200', 'Warehouse A', '2024-10-05', 30);
```

19 (a) Определите класс `Membership`:

```
class Membership(Model):
    member_name = CharField(max_length=100)
    club = CharField(max_length=150)
    join_date = DateField()
    fee_paid = IntegerField()
```

(b) Вызовите: `Membership.create_table('memberships')`

(c) Создайте экземпляр:

```
membership = Membership(
    member_name='Olga Petrova',
    club='Photography Club',
    join_date=datetime.date(2023, 9, 12),
```

```

        fee_paid=1200,
    )
    membership.save()

```

Программа должна вывести следующие SQL-запросы:

- (a) Запрос на создание таблицы (пример для даты выполнения 2025-04-05):

```

CREATE TABLE IF NOT EXISTS memberships (
    member_name VARCHAR(100) NOT NULL DEFAULT '',
    club VARCHAR(150) NOT NULL DEFAULT '',
    join_date DATE NOT NULL DEFAULT '2025-04-05',
    fee_paid INTEGER NOT NULL DEFAULT 0
);

```

- (b) Запрос на вставку данных:

```

INSERT INTO memberships (member_name, club, join_date, fee_paid) VALUES
('Olga Petrova', 'Photography Club', '2023-09-12', 1200);

```

- 20 (a) Определите класс Document:

```

class Document(Model):
    title = CharField(max_length=255)
    author = CharField(max_length=100)
    created_date = DateField()
    pages = IntegerField()

```

- (b) Вызовите: Document.create_table('documents')

- (c) Создайте экземпляр:

```

doc = Document(
    title='Annual Report 2024',
    author='Finance Team',
    created_date=datetime.date(2025, 1, 25),
    pages=42,
)
doc.save()

```

Программа должна вывести следующие SQL-запросы:

- (a) Запрос на создание таблицы (пример для даты выполнения 2025-04-05):

```

CREATE TABLE IF NOT EXISTS documents (
    title VARCHAR(255) NOT NULL DEFAULT '',
    author VARCHAR(100) NOT NULL DEFAULT '',
    created_date DATE NOT NULL DEFAULT '2025-04-05',
    pages INTEGER NOT NULL DEFAULT 0
);

```

- (b) Запрос на вставку данных:

```
INSERT INTO documents (title, author, created_date, pages) VALUES
('Annual Report 2024', 'Finance Team', '2025-01-25', 42);
```

- 21 (a) Определите класс Exam:

```
class Exam(Model):
    subject = CharField(max_length=100)
    teacher = CharField(max_length=100)
    exam_date = DateField()
    max_score = IntegerField()
```

- (b) Вызовите: Exam.create_table('exams')

- (c) Создайте экземпляр:

```
exam = Exam(
    subject='Mathematics',
    teacher='Prof. Ivanov',
    exam_date=datetime.date(2025, 5, 20),
    max_score=100,
)
exam.save()
```

Программа должна вывести следующие SQL-запросы:

- (a) Запрос на создание таблицы (пример для даты выполнения 2025-04-05):

```
CREATE TABLE IF NOT EXISTS exams (
    subject VARCHAR(100) NOT NULL DEFAULT '',
    teacher VARCHAR(100) NOT NULL DEFAULT '',
    exam_date DATE NOT NULL DEFAULT '2025-04-05',
    max_score INTEGER NOT NULL DEFAULT 0
);
```

- (b) Запрос на вставку данных:

```
INSERT INTO exams (subject, teacher, exam_date, max_score) VALUES
('Mathematics', 'Prof. Ivanov', '2025-05-20', 100);
```

- 22 (a) Определите класс Game:

```
class Game(Model):
    name = CharField(max_length=200)
    developer = CharField(max_length=150)
    release_date = DateField()
    rating = IntegerField()
```

- (b) Вызовите: Game.create_table('games')

- (c) Создайте экземпляр:

```
game = Game(  
    name='Mystic Forest',  
    developer='Pixel Studios',  
    release_date=datetime.date(2024, 11, 30),  
    rating=92,  
)  
game.save()
```

Программа должна вывести следующие SQL-запросы:

- (a) Запрос на создание таблицы (пример для даты выполнения 2025-04-05):

```
CREATE TABLE IF NOT EXISTS games (  
    name VARCHAR(200) NOT NULL DEFAULT '',  
    developer VARCHAR(150) NOT NULL DEFAULT '',  
    release_date DATE NOT NULL DEFAULT '2025-04-05',  
    rating INTEGER NOT NULL DEFAULT 0  
);
```

- (b) Запрос на вставку данных:

```
INSERT INTO games (name, developer, release_date, rating) VALUES  
( 'Mystic Forest', 'Pixel Studios', '2024-11-30', 92);
```

- 23 (a) Определите класс Building:

```
class Building(Model):  
    name = CharField(max_length=150)  
    architect = CharField(max_length=100)  
    completion_date = DateField()  
    floors = IntegerField()
```

- (b) Вызовите: `Building.create_table('buildings')`

- (c) Создайте экземпляр:

```
building = Building(  
    name='Sky Tower',  
    architect='Mikhail Sokolov',  
    completion_date=datetime.date(2022, 7, 14),  
    floors=45,  
)  
building.save()
```

Программа должна вывести следующие SQL-запросы:

- (a) Запрос на создание таблицы (пример для даты выполнения 2025-04-05):

```
CREATE TABLE IF NOT EXISTS buildings (
    name VARCHAR(150) NOT NULL DEFAULT '',
    architect VARCHAR(100) NOT NULL DEFAULT '',
    completion_date DATE NOT NULL DEFAULT '2025-04-05',
    floors INTEGER NOT NULL DEFAULT 0
);
```

- (b) Запрос на вставку данных:

```
INSERT INTO buildings (name, architect, completion_date, floors) VALUES
('Sky Tower', 'Mikhail Sokolov', '2022-07-14', 45);
```

- 24 (a) Определите класс Subscription:

```
class Subscription(Model):
    service = CharField(max_length=100)
    subscriber = CharField(max_length=150)
    start_date = DateField()
    monthly_fee = IntegerField()
```

- (b) Вызовите: Subscription.create_table('subscriptions')

- (c) Создайте экземпляр:

```
sub = Subscription(
    service='Cloud Storage Plus',
    subscriber='user@domain.com',
    start_date=datetime.date(2025, 4, 1),
    monthly_fee=250,
)
sub.save()
```

Программа должна вывести следующие SQL-запросы:

- (a) Запрос на создание таблицы (пример для даты выполнения 2025-04-05):

```
CREATE TABLE IF NOT EXISTS subscriptions (
    service VARCHAR(100) NOT NULL DEFAULT '',
    subscriber VARCHAR(150) NOT NULL DEFAULT '',
    start_date DATE NOT NULL DEFAULT '2025-04-05',
    monthly_fee INTEGER NOT NULL DEFAULT 0
);
```

- (b) Запрос на вставку данных:

```
INSERT INTO subscriptions (service, subscriber, start_date, monthly_fee) VALUES
('Cloud Storage Plus', 'user@domain.com', '2025-04-01', 250);
```

- 25 (a) Определите класс Vehicle:

```
class Vehicle(Model):
    brand = CharField(max_length=50)
    model = CharField(max_length=50)
    manufacture_date = DateField()
    mileage = IntegerField()
```

(b) Вызовите: `Vehicle.create_table('vehicles')`

(c) Создайте экземпляр:

```
vehicle = Vehicle(
    brand='BMW',
    model='X5',
    manufacture_date=datetime.date(2021, 8, 22),
    mileage=45000,
)
vehicle.save()
```

Программа должна вывести следующие SQL-запросы:

(a) Запрос на создание таблицы (пример для даты выполнения 2025-04-05):

```
CREATE TABLE IF NOT EXISTS vehicles (
    brand VARCHAR(50) NOT NULL DEFAULT '',
    model VARCHAR(50) NOT NULL DEFAULT '',
    manufacture_date DATE NOT NULL DEFAULT '2025-04-05',
    mileage INTEGER NOT NULL DEFAULT 0
);
```

(b) Запрос на вставку данных:

```
INSERT INTO vehicles (brand, model, manufacture_date, mileage) VALUES
('BMW', 'X5', '2021-08-22', 45000);
```

26 (a) Определите класс `Conference`:

```
class Conference(Model):
    title = CharField(max_length=300)
    organizer = CharField(max_length=150)
    event_date = DateField()
    participants = IntegerField()
```

(b) Вызовите: `Conference.create_table('conferences')`

(c) Создайте экземпляр:

```
conf = Conference(
    title='AI Ethics Summit',
    organizer='Global Tech Ethics',
    event_date=datetime.date(2025, 9, 10),
```



```

        participants=200,
    )
    conf.save()

```

Программа должна вывести следующие SQL-запросы:

- (a) Запрос на создание таблицы (пример для даты выполнения 2025-04-05):

```

CREATE TABLE IF NOT EXISTS conferences (
    title VARCHAR(300) NOT NULL DEFAULT '',
    organizer VARCHAR(150) NOT NULL DEFAULT '',
    event_date DATE NOT NULL DEFAULT '2025-04-05',
    participants INTEGER NOT NULL DEFAULT 0
);

```

- (b) Запрос на вставку данных:

```

INSERT INTO conferences (title, organizer, event_date, participants) VALUES
('AI Ethics Summit', 'Global Tech Ethics', '2025-09-10', 200);

```

- 27 (a) Определите класс LogEntry:

```

class LogEntry(Model):
    level = CharField(max_length=20)
    message = CharField(max_length=500)
    timestamp_date = DateField()
    code = IntegerField()

```

- (b) Вызовите: LogEntry.create_table('log_entries')

- (c) Создайте экземпляр:

```

log = LogEntry(
    level='ERROR',
    message='Database connection failed',
    timestamp_date=datetime.date(2025, 4, 4),
    code=5001,
)
log.save()

```

Программа должна вывести следующие SQL-запросы:

- (a) Запрос на создание таблицы (пример для даты выполнения 2025-04-05):

```

CREATE TABLE IF NOT EXISTS log_entries (
    level VARCHAR(20) NOT NULL DEFAULT '',
    message VARCHAR(500) NOT NULL DEFAULT '',
    timestamp_date DATE NOT NULL DEFAULT '2025-04-05',
    code INTEGER NOT NULL DEFAULT 0
);

```

- (b) Запрос на вставку данных:

```
INSERT INTO log_entries (level, message, timestamp_date, code) VALUES
('ERROR', 'Database connection failed', '2025-04-04', 5001);
```

- 28 (a) Определите класс Certificate:

```
class Certificate(Model):
    course_name = CharField(max_length=200)
    student = CharField(max_length=150)
    issue_date = DateField()
    grade = IntegerField()
```

- (b) Вызовите: Certificate.create_table('certificates')

- (c) Создайте экземпляр:

```
cert = Certificate(
    course_name='Web Development Fundamentals',
    student='Nikolai Smirnov',
    issue_date=datetime.date(2025, 3, 28),
    grade=88,
)
cert.save()
```

Программа должна вывести следующие SQL-запросы:

- (a) Запрос на создание таблицы (пример для даты выполнения 2025-04-05):

```
CREATE TABLE IF NOT EXISTS certificates (
    course_name VARCHAR(200) NOT NULL DEFAULT '',
    student VARCHAR(150) NOT NULL DEFAULT '',
    issue_date DATE NOT NULL DEFAULT '2025-04-05',
    grade INTEGER NOT NULL DEFAULT 0
);
```

- (b) Запрос на вставку данных:

```
INSERT INTO certificates (course_name, student, issue_date, grade) VALUES
('Web Development Fundamentals', 'Nikolai Smirnov', '2025-03-28', 88);
```

- 29 (a) Определите класс Device:

```
class Device(Model):
    serial = CharField(max_length=100)
    manufacturer = CharField(max_length=100)
    purchase_date = DateField()
    warranty_months = IntegerField()
```

- (b) Вызовите: Device.create_table('devices')

- (c) Создайте экземпляр:

```
device = Device(
    serial='SN123456789',
    manufacturer='TechCorp',
    purchase_date=datetime.date(2024, 12, 15),
    warranty_months=24,
)
device.save()
```

Программа должна вывести следующие SQL-запросы:

- (a) Запрос на создание таблицы (пример для даты выполнения 2025-04-05):

```
CREATE TABLE IF NOT EXISTS devices (
    serial VARCHAR(100) NOT NULL DEFAULT '',
    manufacturer VARCHAR(100) NOT NULL DEFAULT '',
    purchase_date DATE NOT NULL DEFAULT '2025-04-05',
    warranty_months INTEGER NOT NULL DEFAULT 0
);
```

- (b) Запрос на вставку данных:

```
INSERT INTO devices (serial, manufacturer, purchase_date, warranty_months) VALUES
('SN123456789', 'TechCorp', '2024-12-15', 24);
```

- 30 (a) Определите класс Transaction:

```
class Transaction(Model):
    account = CharField(max_length=50)
    description = CharField(max_length=200)
    date = DateField()
    amount_cents = IntegerField()
```

- (b) Вызовите: Transaction.create_table('transactions')

- (c) Создайте экземпляр:

```
txn = Transaction(
    account='ACCT-8899',
    description='Online Purchase',
    date=datetime.date(2025, 4, 3),
    amount_cents=-12500,
)
txn.save()
```

Программа должна вывести следующие SQL-запросы:

- (a) Запрос на создание таблицы (пример для даты выполнения 2025-04-05):

```
CREATE TABLE IF NOT EXISTS transactions (
    account VARCHAR(50) NOT NULL DEFAULT '',
    description VARCHAR(200) NOT NULL DEFAULT '',
    date DATE NOT NULL DEFAULT '2025-04-05',
    amount_cents INTEGER NOT NULL DEFAULT 0
);
```

- (b) Запрос на вставку данных:

```
INSERT INTO transactions (account, description, date, amount_cents) VALUES
('ACCT-8899', 'Online Purchase', '2025-04-03', -12500);
```

- 31 (a) Определите класс Publication:

```
class Publication(Model):
    journal = CharField(max_length=200)
    author = CharField(max_length=150)
    publish_date = DateField()
    citations = IntegerField()
```

- (b) Вызовите: Publication.create_table('publications')

- (c) Создайте экземпляр:

```
pub = Publication(
    journal='Journal of AI Research',
    author='Dr. Anna Volkova',
    publish_date=datetime.date(2024, 6, 12),
    citations=34,
)
pub.save()
```

Программа должна вывести следующие SQL-запросы:

- (a) Запрос на создание таблицы (пример для даты выполнения 2025-04-05):

```
CREATE TABLE IF NOT EXISTS publications (
    journal VARCHAR(200) NOT NULL DEFAULT '',
    author VARCHAR(150) NOT NULL DEFAULT '',
    publish_date DATE NOT NULL DEFAULT '2025-04-05',
    citations INTEGER NOT NULL DEFAULT 0
);
```

- (b) Запрос на вставку данных:

```
INSERT INTO publications (journal, author, publish_date, citations) VALUES
('Journal of AI Research', 'Dr. Anna Volkova', '2024-06-12', 34);
```

- 32 (a) Определите класс Appointment:

```
class Appointment(Model):
    patient = CharField(max_length=100)
    doctor = CharField(max_length=100)
    appointment_date = DateField()
    duration_minutes = IntegerField()
```

(b) Вызовите: `Appointment.create_table('appointments')`

(c) Создайте экземпляр:

```
appt = Appointment(
    patient='Sergey Orlov',
    doctor='Dr. Elena Kuznetsova',
    appointment_date=datetime.date(2025, 4, 7),
    duration_minutes=30,
)
appt.save()
```

Программа должна вывести следующие SQL-запросы:

(a) Запрос на создание таблицы (пример для даты выполнения 2025-04-05):

```
CREATE TABLE IF NOT EXISTS appointments (
    patient VARCHAR(100) NOT NULL DEFAULT '',
    doctor VARCHAR(100) NOT NULL DEFAULT '',
    appointment_date DATE NOT NULL DEFAULT '2025-04-05',
    duration_minutes INTEGER NOT NULL DEFAULT 0
);
```

(b) Запрос на вставку данных:

```
INSERT INTO appointments (patient, doctor, appointment_date, duration_minutes) VALUES
('Sergey Orlov', 'Dr. Elena Kuznetsova', '2025-04-07', 30);
```

33 (a) Определите класс `Contract`:

```
class Contract(Model):
    party_a = CharField(max_length=150)
    party_b = CharField(max_length=150)
    signed_date = DateField()
    term_months = IntegerField()
```

(b) Вызовите: `Contract.create_table('contracts')`

(c) Создайте экземпляр:

```
contract = Contract(
    party_a='Company LLC',
    party_b='Freelancer Inc.',
    signed_date=datetime.date(2025, 2, 28),
```

```

        term_months=12,
    )
    contract.save()

```

Программа должна вывести следующие SQL-запросы:

- (a) Запрос на создание таблицы (пример для даты выполнения 2025-04-05):

```

CREATE TABLE IF NOT EXISTS contracts (
    party_a VARCHAR(150) NOT NULL DEFAULT '',
    party_b VARCHAR(150) NOT NULL DEFAULT '',
    signed_date DATE NOT NULL DEFAULT '2025-04-05',
    term_months INTEGER NOT NULL DEFAULT 0
);

```

- (b) Запрос на вставку данных:

```

INSERT INTO contracts (party_a, party_b, signed_date, term_months) VALUES
('Company LLC', 'Freelancer Inc.', '2025-02-28', 12);

```

- 34 (a) Определите класс Award:

```

class Award(Model):
    title = CharField(max_length=200)
    recipient = CharField(max_length=150)
    award_date = DateField()
    rank = IntegerField()

```

- (b) Вызовите: Award.create_table('awards')

- (c) Создайте экземпляр:

```

award = Award(
    title='Best Open Source Project',
    recipient='Team ORM',
    award_date=datetime.date(2024, 10, 15),
    rank=1,
)
award.save()

```

Программа должна вывести следующие SQL-запросы:

- (a) Запрос на создание таблицы (пример для даты выполнения 2025-04-05):

```

CREATE TABLE IF NOT EXISTS awards (
    title VARCHAR(200) NOT NULL DEFAULT '',
    recipient VARCHAR(150) NOT NULL DEFAULT '',
    award_date DATE NOT NULL DEFAULT '2025-04-05',
    rank INTEGER NOT NULL DEFAULT 0
);

```

- (b) Запрос на вставку данных:

```
INSERT INTO awards (title, recipient, award_date, rank) VALUES
('Best Open Source Project', 'Team ORM', '2024-10-15', 1);
```

- 35 (a) Определите класс Playlist:

```
class Playlist(Model):
    name = CharField(max_length=150)
    creator = CharField(max_length=100)
    created_date = DateField()
    track_count = IntegerField()
```

- (b) Вызовите: `Playlist.create_table('playlists')`

- (c) Создайте экземпляр:

```
playlist = Playlist(
    name='Coding Focus',
    creator='user123',
    created_date=datetime.date(2025, 3, 30),
    track_count=18,
)
playlist.save()
```

Программа должна вывести следующие SQL-запросы:

- (a) Запрос на создание таблицы (пример для даты выполнения 2025-04-05):

```
CREATE TABLE IF NOT EXISTS playlists (
    name VARCHAR(150) NOT NULL DEFAULT '',
    creator VARCHAR(100) NOT NULL DEFAULT '',
    created_date DATE NOT NULL DEFAULT '2025-04-05',
    track_count INTEGER NOT NULL DEFAULT 0
);
```

- (b) Запрос на вставку данных:

```
INSERT INTO playlists (name, creator, created_date, track_count) VALUES
('Coding Focus', 'user123', '2025-03-30', 18);
```